Accessible Introduction to Computer Science

Michael Ryan Hunsaker, M.Ed., Ph.D.

Contents

Accessible Introduction to Computer Science **9**

I Preliminary prerequisite: Terminal navigation 20

1 Terminal Skills: Prerequisite Curriculum **21**
1.1 Bash Track: macOS and Linux Terminal Skills 24
1.2 PowerShell Track: Windows 11 Terminal Skills 32

II Development environment 41

2 Development Editors, IDEs, and Project Development Containers **42**
2.1 Project Environments, Dependency Management, and Development Containers 55
2.2 Screen-reader-accessible GUI libraries and implementation guidance 64

III Foundations (complete before any language) 88

3 Foundations: Computing Concepts Before Any Language **89**
3.1 Unit P: Computing Foundations Primer (Complete Before Any Language Track) 91

IV C 119

4 C Curriculum **120**
4.1 C Lessons 122
4.2 Unit 0: Accessible Environment and Toolchain 122
4.3 Unit 1: Foundations of Computing 129
4.4 Unit 2: Numeric Computation 134

4.5 Unit 3: Control Flow . . . 143
4.6 Unit 4: Data Structures . . . 152
4.7 Unit 5: Input and Iteration . . . 160
4.8 Unit 6: Functions . . . 169
4.9 Unit 7: Structs and Object Models . . . 178
4.10 Unit 8: Files, Errors, and Persistence . . . 186
4.11 Unit 9: Testing and Data Analysis . . . 195
4.12 Unit 10: Applied Projects . . . 204
4.13 Unit 11: Capstone: Personal Data Manager . . . 215

V C++ (C Plus Plus) 225

5 C++ Curriculum 226
5.1 CPP Lessons . . . 228
5.2 Unit 0: Accessible Environment and Toolchain . . . 228
5.3 Unit 1: Foundations of Computing . . . 234
5.4 Unit 2: Numeric Computations . . . 239
5.5 Unit 3: Control Flow . . . 247
5.6 Unit 4: Data Structures . . . 255
5.7 Unit 5: Input and Iteration . . . 263
5.8 Unit 6: Functions . . . 270
5.9 Unit 7: Object-Oriented Programming . . . 278
5.10 Unit 8: Files and Exceptions . . . 286
5.11 Unit 9: Testing and Data Analysis . . . 294
5.12 Unit 10: Project-Based Learning . . . 301
5.13 Unit 11: Capstone . . . 311

VI C-SHARP 321

6 C# Curriculum 322
6.1 CSHARP Lessons . . . 324
6.2 Unit 0: Accessible Environment and Toolchain . . . 324
6.3 Unit 1: Foundations of Computing . . . 330
6.4 Unit 2: Numeric Computations . . . 334
6.5 Unit 3: Control Flow . . . 342
6.6 Unit 4: Data Structures . . . 349
6.7 Unit 5: Input and Iteration . . . 357
6.8 Unit 6: Functions . . . 364
6.9 Unit 7: Object-Oriented Programming . . . 372
6.10 Unit 8: Files and Exceptions . . . 379
6.11 Unit 9: Testing and Data Analysis . . . 387
6.12 Unit 10: Project-Based Learning . . . 394
6.13 Unit 11: Capstone . . . 403

VII GO **412**

7 Go Curriculum **413**
7.1 GO Lessons . . . 415
7.2 Unit 0: Accessible Environment and Toolchain . . . 415
7.3 Unit 1: Foundations of Computing . . . 420
7.4 Unit 2: Numeric Computations . . . 428
7.5 Unit 3: Control Flow . . . 447
7.6 Unit 4: Data Structures . . . 465
7.7 Unit 5: Input and Iteration . . . 483
7.8 Unit 6: Functions . . . 499
7.9 Unit 7: Object-Oriented Programming with Go Types . . . 516
7.10 Unit 8: Files and Exceptions . . . 526
7.11 Unit 9: Testing and Data Analysis . . . 535
7.12 Unit 10: Project-Based Learning — Text-Based Applications . . . 544
7.13 Unit 11: Capstone — Accessible Personal Data Manager . . . 555

VIII JAVA **566**

8 Introduction to Computer Science with Java **567**
8.1 JAVA Lessons . . . 573
8.2 Unit 0: Accessible Java Development Environment . . . 574
8.3 Unit 1: Foundations of Computing and Java . . . 582
8.4 Unit 2: Numeric Computations . . . 588
8.5 Unit 3: Control Flow — Decisions and Loops . . . 598
8.6 Unit 4: Data Structures — Lists and Dictionaries . . . 607
8.7 Unit 5: User Input and while Loops . . . 617
8.8 Unit 6: Functions and Methods . . . 628
8.9 Unit 7: Classes and Object-Oriented Programming . . . 640
8.10 Unit 8: Files and Exceptions . . . 652
8.11 Unit 9: Testing and Data Analysis . . . 668
8.12 Unit 10: Project-Based Learning — Text Applications . . . 680
8.13 Unit 11: Capstone — Accessible Personal Data Manager . . . 706

IX JAVASCRIPT **731**

9 Introduction to Computer Science with JavaScript **732**
9.1 JAVASCRIPT Lessons . . . 738
9.2 Unit 0: Setting Up Your JavaScript Development Environment . . . 739
9.3 Unit 1: Foundations — Getting Started with JavaScript . . . 749
9.4 Unit 2: Numeric Computations and Input/Output . . . 760
9.5 Unit 3: Control Flow — Decisions, Lists, and Strings . . . 769
9.6 Unit 4: Data Structures — Arrays, Fixed Sequences, and Objects 776
9.7 Unit 5: User Input and `while` Loops . . . 784

9.8 Unit 6: Functions . . . 791
9.9 Unit 7: Classes and Object-Oriented Programming . . . 798
9.10 Unit 8: Files and Exceptions . . . 806
9.11 Unit 9: Testing and Data Analysis . . . 815
9.12 Unit 10: Project-Based Learning — Text-Based Applications . . . 823
9.13 Unit 11: Capstone Project — Accessible Personal Data Manager 832

X JULIA 841

10 Julia Curriculum 842
10.1 JULIA Lessons . . . 843
10.2 Unit 0: Accessible Environment and Toolchain . . . 844
10.3 Unit 1: Foundations of Computing . . . 849
10.4 Unit 2: Numeric Computations . . . 854
10.5 Unit 3: Control Flow . . . 860
10.6 Unit 4: Data Structures . . . 867
10.7 Unit 5: Input and Iteration . . . 874
10.8 Unit 6: Functions . . . 881
10.9 Unit 7: Object-Oriented Programming . . . 888
10.10 Unit 8: Files and Exceptions . . . 895
10.11 Unit 9: Testing and Data Analysis . . . 902
10.12 Unit 10: Project-Based Learning . . . 909
10.13 Unit 11: Capstone . . . 918

XI KOTLIN 926

11 Introduction to Kotlin 927
11.1 KOTLIN Lessons . . . 930
11.2 Unit 0: Accessible Development Environment . . . 930
11.3 Unit 1: Foundations of Computing . . . 938
11.4 Unit 2: Numeric Computations . . . 943
11.5 Unit 3: Control Flow — Decisions and Loops . . . 953
11.6 Unit 4: Data Structures — Lists and Dictionaries . . . 963
11.7 Unit 5: User Input and while Loops . . . 973
11.8 Unit 6: Functions . . . 985
11.9 Unit 7: Classes and Object-Oriented Programming . . . 995
11.10 Unit 8: Files and Exceptions . . . 1005
11.11 Unit 9: Testing and Data Analysis . . . 1018
11.12 Unit 10: Project-Based Learning — Text-Based Applications . . . 1030
11.13 Unit 11: Capstone — Accessible Personal Data Manager . . . 1050

XII LUA **1070**

12 Lua Curriculum **1071**
12.1 LUA Lessons 1072
12.2 Unit 0: Accessible Environment and Toolchain 1073
12.3 Unit 1: Foundations of Computing 1081
12.4 Unit 2: Numeric Computations 1087
12.5 Unit 3: Control Flow — Decisions and Loops 1094
12.6 Unit 4: Data Structures — Lists and Dictionaries 1102
12.7 Unit 5: User Input and while Loops 1112
12.8 Unit 6: Functions 1123
12.9 Unit 7: Classes and Object-Oriented Programming 1130
12.10 Unit 8: Files and Exceptions 1137
12.11 Unit 9: Testing and Data Analysis 1150
12.12 Unit 10: Project-Based Learning — Text-Based Applications . . 1158
12.13 Unit 11: Capstone — Accessible Personal Data Manager 1189

XIII PYTHON **1199**

13 Introduction to Computer Science with Python **1200**
13.1 PYTHON Lessons 1207
13.2 Unit 0: Setting Up Your Accessible Python Development Environment 1207
13.3 Unit 1: Foundations of Computing and Getting Started with Python 1222
13.4 Unit 2: Writing Simple Programs and Numeric Computations . . 1230
13.5 Unit 3: Control Flow — Decisions and Loops 1243
13.6 Unit 4: Data Structures — Lists and Dictionaries 1255
13.7 Unit 5: User Input and `while` Loops 1264
13.8 Unit 6: Functions 1274
13.9 Unit 7: Classes and Object-Oriented Programming 1284
13.10 Unit 8: Files and Exceptions 1294
13.11 Unit 9: Testing and Data Analysis 1304
13.12 Unit 10: Project-Based Learning — Building Text-Based Applications 1318
13.13 Unit 11: Capstone Project — Accessible Personal Data Manager 1330

XIV RUST **1356**

14 Rust Curriculum **1357**
14.1 RUST Lessons 1358
14.2 Unit 0: Accessible Environment and Toolchain 1359
14.3 Unit 1: Foundations of Computing 1367
14.4 Unit 2: Numeric Computations 1372

14.5 Unit 3: Control Flow 1384
14.6 Unit 4: Data Structures 1393
14.7 Unit 5: Input and Iteration 1405
14.8 Unit 6: Functions 1417
14.9 Unit 7: Data Modeling and Traits 1429
14.10 Unit 8: Files and Error Handling 1442
14.11 Unit 9: Testing and Data Analysis 1458
14.12 Unit 10: Project-Based Learning 1473
14.13 Unit 11: Capstone 1499

XV SWIFT 1526

15 Swift Curriculum 1527
15.1 SWIFT Lessons 1528
15.2 Unit 0: Accessible Environment and Toolchain 1529
15.3 Unit 1: Foundations of Computing 1534
15.4 Unit 2: Numeric Computations 1539
15.5 Unit 3: Control Flow 1546
15.6 Unit 4: Data Structures 1554
15.7 Unit 5: Input and Iteration 1561
15.8 Unit 6: Functions 1568
15.9 Unit 7: Object-Oriented Programming 1576
15.10 Unit 8: Files and Exceptions 1583
15.11 Unit 9: Testing and Data Analysis 1591
15.12 Unit 10: Project-Based Learning 1598
15.13 Unit 11: Capstone 1608

XVI TYPESCRIPT 1617

16 Introduction to Computer Science with TypeScript 1618
16.1 TYPESCRIPT Lessons 1626
16.2 Unit 0: Accessible Development Environment 1626
16.3 Unit 1: Foundations of Computing 1632
16.4 Unit 2: Numeric Computations 1636
16.5 Unit 3: Control Flow — Decisions and Loops 1643
16.6 Unit 4: Data Structures — Arrays, Tuples, and Objects 1650
16.7 Unit 5: User Input and while Loops 1658
16.8 Unit 6: Functions 1665
16.9 Unit 7: Classes and Object-Oriented Programming 1678
16.10 Unit 8: Files and Exceptions 1685
16.11 Unit 9: Testing and Data Analysis 1693
16.12 Unit 10: Project-Based Learning — Text-Based Applications . . 1700
16.13 Unit 11: Capstone — Accessible Personal Data Manager 1708

XVII Standalone Topics/Languages 1716

17 Standalone Topics and Focused Language Modules 1717
17.1 Git and Version Control 1721
17.2 HTML and CSS: Structure and Presentation for the Web 1725
17.3 Web Accessibility: ARIA, WCAG, and Accessible Design 1727
17.4 Markdown: Technical Documentation 1732
17.5 R: Statistical Computing 1736
17.6 Regular Expressions 1738
17.7 SQL: Relational Databases 1742
17.8 Structured Data Formats: JSON, YAML, and TOML 1745

XVIII Backmatter 1749

18 Teacher's Guides for Each Course 1750
18.1 C Teacher's Guide 1751
18.2 C# Teacher's Guide 1754
18.3 C++ Teacher's Guide 1757
18.4 Go Teacher's Guide 1759
18.5 Teacher's Guide: Introduction to Computer Science with Java . . 1762
18.6 Teacher's Guide: Introduction to Computer Science with JavaScript 1798
18.7 Julia Teacher's Guide 1841
18.8 Kotlin Teacher's Guide 1843
18.9 Lua Teacher's Guide 1847
18.10 Teacher's Guide: Introduction to Computer Science with Python 1849
18.11 Rust Teacher's Guide 1895
18.12 Swift Teacher's Guide 1898
18.13 Teacher's Guide: Introduction to Computer Science with TypeScript 1901

19 Programming Code Glossary 1944

20 Global Index 1962
20.1 C Topic Index 1962
20.2 C# Topic Index 1963
20.3 C++ Topic Index 1964
20.4 Go Topic Index 1965
20.5 Java Index 1965
20.6 JavaScript Index 1968
20.7 Julia Topic Index 1971
20.8 Kotlin Topic Index 1972
20.9 Lua Topic Index 1973
20.10 Python Index 1974
20.11 Rust Topic Index 1978
20.12 Introductory Modules Index 1978

20.13Swift Topic Index 1979
20.14TypeScript Index 1980

21 Contributors to this Curriculum **1984**

Accessible Introduction to Computer Science

A living, accessibility-first textbook across programming languages and software domains

This book is a structured introduction to computer science and software development. It is designed with blind and visually impaired learners who use screen readers, braille displays, magnification, keyboard navigation, or combinations of access technologies in mind. It is also intended for sighted learners, educators, and anyone who benefits from explicit, text-first instruction.

The repository contains 13 full language curricula, focused modules in Git, HTML/CSS, Markdown, R, regular expressions, SQL, structured data formats, and web accessibility, and shared Bash and PowerShell terminal tracks. The languages are presented by their common technical uses and ecosystems, not as a hierarchy. No single language is the best choice for every problem.

The textbook is published with **mdBook**. Markdown files in this repository are the maintained source; mdBook builds them into the navigable online textbook hosted on GitHub Pages. The book is intended to remain a living resource: lessons, examples, exercises, accessibility notes, and teaching materials can be reviewed and improved as learners and educators use them.

Start learning: complete the shared prerequisites, then choose a language curriculum or focused module that aligns with your goals. Each language curriculum includes its own topic index. The shared Programming Code Glossary defines common and language-specific terminology.

Complete the shared prerequisites

Terminal skills

Before beginning a language curriculum, complete the Terminal Skills prerequisite. Choose the Bash track for macOS and Linux or the PowerShell track for Windows. Both tracks teach equivalent foundational workflows using native commands: navigating directories, managing files, reading and searching text, running programs, interpreting errors, and working safely from a project folder. These skills support every language and toolchain in the book.

Computing foundations

Read the Foundations overview and Computing Foundations Primer. These materials introduce concepts and vocabulary that apply across programming languages, operating systems, and development tools.

Development environments

Review the IDE and editor guide and Environment Isolation guide. Visual Studio Code and Theia IDE are the primary editor options discussed for the course. The materials also describe professional IDEs and data platforms—including JetBrains products and Databricks—by their typical roles and use cases, without providing setup procedures for those products.

Project-defined **development containers** receive particular emphasis when a course requires a consistent compiler, runtime, linter, formatter, test runner, or supporting service. The guides explain the relationship between development containers, Docker, and Podman and how project-level environments support reproducible learning and assessment.

Purpose and learning philosophy

The goal is not simply to teach syntax. Learners should develop the habits used to solve problems with computers: clarify requirements, decompose a task, represent information, design an algorithm, implement it, test it, interpret errors, revise the solution, and explain their decisions.

The curricula share these commitments:

- **Accessibility is part of the design.** Core activities use text-based input and output and can be completed through keyboard-accessible tools. Lessons explain code structure and navigation rather than depending on visual scanning.
- **Concepts precede tools.** Editors, terminals, runtimes, compilers, interpreters, and libraries are instruments for learning computer science.

- **Practice is central.** Short experiments, guided exercises, debugging, and projects provide repeated opportunities to apply concepts.
- **Understanding matters more than typing speed.** Learners reason about programs, predict behavior, test assumptions, and explain their decisions.
- **Errors are part of learning.** Reading diagnostics and correcting a program are core skills.
- **The resource evolves.** Corrections, new examples, and accessibility improvements are welcome through GitHub Discussions, Issues, and Pull Requests.

Who should use this book?

The primary audience includes learners who are blind or visually impaired. It may be used in secondary or postsecondary courses, accessible technology programs, tutoring, independent study, or educator professional development.

No previous programming experience is required for the introductory learning paths. Learners should have, or be developing, basic keyboard and screen-reader or braille-display navigation skills. A teacher can introduce programming and assistive-technology workflows together; lack of advanced screen-reader fluency should not be mistaken for lack of programming ability.

The material is also useful to sighted learners because it emphasizes precise language, semantic structure, explicit feedback, and reproducible text-based workflows.

What learners will develop

By working through a selected curriculum, learners should build the ability to:

1. Describe what a computer program is and explain a basic software-development cycle.
2. Use a language runtime, compiler, editor, terminal, and project environment appropriate to the selected technology.
3. Read code structurally, including identifiers, operators, delimiters, indentation, braces, and nested blocks.
4. Store, convert, compare, and process values using appropriate data types.
5. Use expressions, input, output, decisions, and repetition to solve problems.
6. Organize related values in collections and select suitable data structures.
7. Decompose programs into functions, methods, modules, or components with clear responsibilities.
8. Work with files, exceptions, structured data, and external libraries where relevant.
9. Understand classes, objects, interfaces, memory, concurrency, or other paradigms where the selected language requires them.

10. Test behavior, diagnose errors, refine a solution, and communicate how it works.
11. Build text-based applications or domain-appropriate projects and complete the projects specified by the selected path.
12. Evaluate whether program interaction, prompts, messages, and output are understandable through assistive technology.

Outcomes vary by curriculum and module. These are learning goals, not a claim of professional mastery.

Choose a curriculum by primary use

The following descriptions identify common domains and representative software or projects associated with each language. Examples illustrate ecosystem usage; they do not mean that a complete product is written exclusively in that language.

Full language curricula

Language and curriculum	Primary usage	Representative software, platforms, or projects
Python	Automation, scripting, web services, scientific computing, data analysis, and machine learning	Django and Flask services; NumPy, pandas, and PyTorch ecosystems
JavaScript	Browser interaction, web applications, server-side JavaScript, and developer tooling	Node.js, React-based applications, browser applications, ESLint
TypeScript	Statically checked web and server applications, especially large JavaScript codebases	Visual Studio Code, Angular applications, TypeScript-based Node.js services
Java	Enterprise services, large application systems, backend platforms, and JVM software	Minecraft: Java Edition, Elasticsearch, Spring-based services
Kotlin	Android applications, JVM services, and multiplatform development	Android applications, Gradle build tooling, Kotlin server services

Language and curriculum	Primary usage	Representative software, platforms, or projects
C	Operating systems, embedded systems, low-level interfaces, and performance-sensitive libraries	Linux kernel components, CPython implementation components, embedded firmware
C++	Game engines, graphics, browsers, desktop software, and performance-intensive systems	Unreal Engine, Chromium components, graphics and desktop applications
C#	.NET business applications, cloud services, desktop software, and game development	Unity projects, ASP.NET Core services, .NET desktop applications
Julia	Numerical computing, scientific simulation, optimization, and technical research	Julia scientific-computing packages and SciML ecosystem projects
Swift	Apple-platform application development and systems programming	iOS, iPadOS, macOS, watchOS, and tvOS applications
Rust	Systems software, command-line tools, infrastructure, and performance-sensitive services	ripgrep, parts of Firefox, infrastructure tooling
Go	Cloud infrastructure, network services, command-line tools, and concurrent backend systems	Docker, Kubernetes, Prometheus
Lua	Embedded scripting, game logic, configuration, and extensible applications	Roblox experiences, Neovim configuration, applications embedding Lua

Focused technology modules

Some important technologies are not general-purpose programming languages. They have distinct roles and are included as focused modules:

Module	Primary usage and role	Representative software or platforms
Git	Distributed version control: repositories, commits, branches, merges, remotes, and pull-request collaboration	Git, GitHub, GitLab, and the GitHub CLI (`gh`)
HTML and CSS	HTML structures web content; CSS controls presentation, layout, and responsive styling	Websites, browser interfaces, web applications, design systems
Markdown	Lightweight plain-text markup for documentation, README files, and technical notes	GitHub documentation, static site generators, Pandoc
R	Statistical analysis, data visualization, research computing, and reproducible reports	RStudio/Posit, tidyverse, Bioconductor
Regular expressions	Pattern-based finding, extracting, validating, and replacing of text	`grep`, `sed`, ripgrep, and regex support in mainstream languages
SQL	Declarative querying, defining, and managing relational data	PostgreSQL, SQLite, MySQL, SQL Server
Structured data formats	JSON for data interchange, YAML for configuration, and TOML for application settings	APIs, GitHub Actions, Kubernetes, `pyproject.toml`, `Cargo.toml`
Web accessibility	Building websites and applications usable with screen readers, keyboard-only interaction, and other access technologies	WCAG, ARIA, NVDA, VoiceOver, JAWS, axe DevTools

HTML is a markup language and CSS is a style-sheet language; together they form the structure and presentation layer of the web. SQL is a declarative query language for relational data. R is a programming language used extensively in statistics and data analysis. Git is a version control system, Markdown is a documentation markup language, regular expressions are a pattern language

embedded in many tools, JSON, YAML, and TOML are structured text formats, and web accessibility is an engineering discipline built on the web platform's accessibility APIs.

Shared terminal tracks

Bash and PowerShell are shell languages. They support command execution, file and process management, automation, and development workflows. Bash is common on macOS and Linux; PowerShell is central to Windows administration and is also available cross-platform. Both are shared prerequisites rather than alternatives to the application languages above.

Selecting a pathway

Choose a path according to the problem domain, deployment target, ecosystem, and learning objectives:

- **Automation, data analysis, and machine learning:** Python; consider R for statistical analysis and Julia for numerical or scientific computing.
- **Web application development:** JavaScript and TypeScript, alongside HTML/CSS and the web accessibility module; add SQL when the application stores relational data.
- **Enterprise and JVM applications:** Java or Kotlin.
- **Android and Apple platforms:** Kotlin is widely used for Android development; Swift is the primary language for modern native Apple-platform development.
- **Operating systems, embedded devices, and systems software:** C, C++, or Rust, depending on hardware, ecosystem, performance, and safety requirements.
- **Cloud services and infrastructure tooling:** Go, Java, Python, Rust, or C# depending on platform and system requirements.
- **Game development and application scripting:** C++, C#, and Lua have distinct roles across engines, application logic, and embedded scripting.
- **Relational data systems:** SQL is a companion to application languages, not a replacement for them.
- **Professional workflow skills:** the Git, Markdown, regular expressions, and structured data formats modules support every other path in the textbook.

These categories overlap. A real product may use several languages, and a language may serve multiple domains. Selection should reflect the problem being solved, the existing codebase, team expertise, operational constraints, and available libraries.

How the curriculum is organized

The mdBook table of contents is the authoritative navigation for the published book. The repository is grouped by shared foundations, language curricula, focused modules, development guidance, and backmatter.

Area	Contents
Shared foundations	Computing foundations and terminal skills in Bash and PowerShell
Development guidance	IDE and editor roles, project dependency management, environment isolation, development containers, Docker, and Podman
Full language curricula	Python, JavaScript, TypeScript, Java, Kotlin, C, C++, C#, Julia, Swift, Rust, Go, and Lua
Focused modules	Git, HTML/CSS, Markdown, R, regular expressions, SQL, structured data formats, and web accessibility
Backmatter	Teacher resources, programming glossary, topic indexes, validation records, and other references

Each full language curriculum has its own landing page and supporting lesson materials. Depending on the language, it may also include a topic index, teacher's guide, standardization audit, or validation record. The focused modules provide a narrower introduction to a particular technology. Follow the sequence and scope documented on the corresponding landing page and in the table of contents.

A practical study routine

1. **Preview:** read the lesson title, objectives or overview, vocabulary, and prerequisites.
2. **Understand:** read the explanation and trace each example. Identify what each statement contributes.
3. **Predict:** state what you expect the program to do before running it.
4. **Run:** execute the example and inspect its text output.
5. **Experiment:** change one input, value, or statement at a time.
6. **Practice:** complete the activities without copying a finished solution.
7. **Debug:** read the diagnostic, locate the relevant code, form a hypothesis, and test a correction.
8. **Explain:** describe the algorithm and the role of important variables, conditions, loops, functions, methods, modules, or objects.

9. **Review:** revisit the skills checklist and record concepts that need more practice.

Keep a learning journal or portfolio. Include the problem statement, approach, sample input and output, errors encountered, and what changed.

Accessibility and technology

Recommended editor and development workflow

Visual Studio Code is the primary editor in the setup material. Theia IDE is another option for keyboard-first development. The IDE guide also describes the roles of JetBrains IDEs, Databricks, and other professional tools without giving setup instructions for them.

For projects that require compilers, linters, formatters, test runners, and supporting services, the course emphasizes repository-defined **development containers**. A containerized project environment can make the toolchain more consistent across learner computers and CI. The environment guide explains the Dev Container model and the roles of Docker and Podman.

Access configurations vary by operating system, screen reader, braille display, editor version, and individual preference. Treat shortcuts and settings in the lessons as starting points; verify them in the learner's environment.

Dependency and environment management

Use the project-native dependency and build tools documented by each language curriculum. Examples include uv and Python project environments, npm for JavaScript and TypeScript, Maven or Gradle wrappers for JVM projects, and the relevant package and build tools for C, C++, C#, Julia, Swift, Rust, Go, and Lua. Keep dependencies project-scoped where possible, commit lockfiles when the ecosystem supports them, and use the repository's documented commands rather than relying on undocumented global tools.

A development container can complement these tools; it does not replace language package managers, lockfiles, or build systems. The project should document both the container environment and the commands used to build, lint, test, and run its code.

Shared accessibility practices

- Use semantic headings, descriptive link text, ordered procedures, and code fences with language identifiers.
- Enable editor screen-reader support where available and learn navigation by line, word, symbol, file, and diagnostic.
- Configure speech or braille output for punctuation, indentation, line numbers, and code symbols according to learner preference.

- Use a terminal or REPL to run examples and inspect text output.
- Keep examples and output understandable when read sequentially. Label collections, menu options, and results.
- Avoid relying on syntax highlighting, color, spatial placement, diagrams, or screenshots to communicate essential information.
- Provide a text description or equivalent activity when a visual resource is included.
- Allow time for navigation and tool interaction; assess programming reasoning separately from assistive-technology speed.
- Ask learners which tools and interaction patterns work for them. Do not infer needs from a diagnosis.

A text-only interface is not automatically accessible. Output order, verbosity, prompts, error handling, keyboard operation, and documentation structure all affect usability. Test activities with the actual access technology where possible and revise barriers as they are identified.

Assessment and evidence of learning

Use varied evidence rather than relying only on timed code production:

- Explain a program or algorithm in the learner's own words.
- Predict output before execution and explain any difference.
- Complete a program or technical task from a clear specification.
- Find and fix a deliberately introduced bug.
- Write tests or identify useful test cases.
- Refactor a program into clearer functions, methods, modules, or data structures.
- Demonstrate keyboard and assistive-technology workflows used to create, run, and debug the work.
- Maintain a portfolio with incremental versions and reflections.

Assess correctness, reasoning, testing, clarity, independence, and progress. Provide reasonable accommodations without lowering the underlying learning objective.

Using AI tools thoughtfully

AI-assisted coding tools may help explain unfamiliar concepts, interpret diagnostics, propose test cases, or provide alternative explanations. They can also produce incorrect, insecure, inaccessible, outdated, or overly complicated code. AI output is not evidence that a learner understands a topic.

Follow the instructor's policy for each activity and assessment. Disclose assistance when required; never submit generated code that the learner cannot explain, test, and modify. Do not share personal, educational-record, creden-

tial, or confidential information with public AI systems. Review generated code for correctness, dependencies, security, licensing, and screen-reader usability. Keep a non-AI path available so paid tools or accounts are not prerequisites.

Contributing to this living textbook

Learners, teachers, developers, and accessibility specialists are invited to contribute:

- Use GitHub Discussions for questions, proposals, and teaching ideas.
- Use GitHub Issues to report errors, broken links, unclear explanations, outdated setup instructions, or accessibility barriers.
- Use Pull Requests to propose corrections and additions.

Read CONTRIBUTING.md, follow CODE_STYLE.md, and respect the Code of Conduct. Record textbook-source changes in CHANGES.md.

References and acknowledgments

The Foundations introduction includes a bibliography of introductory computer science texts. Each language landing page and focused technology module includes curated introductory book recommendations, with authors, editions or publication dates where verified, notes on intended use, and links to publisher, author, or free online editions. The Bash and PowerShell tracks also include shell-specific reading recommendations. These bibliographies complement—not replace—the course lessons and official language documentation. Use official documentation to confirm version-sensitive language, API, and tool details.

External resources may change or have different accessibility characteristics over time; evaluate them in the learner's actual environment. Preserve appropriate attribution when improving or extending material.

Select a curriculum that matches a technical domain or learning objective, complete its prerequisites, and work through its materials in sequence. The goal is to become an independent problem solver who can read, write, test, explain, and improve software using accessible tools.

Part I

Preliminary prerequisite: Terminal navigation

Chapter 1

Terminal Skills: Prerequisite Curriculum

Purpose

This short prerequisite teaches the command-line skills used throughout the programming curricula. Complete it before beginning a language track. The goal is to make learners comfortable navigating projects, creating and inspecting files, running programs, understanding command output, and recovering from common mistakes.

A terminal is a text interface for interacting with the operating system. A shell reads commands and runs programs. The terminal application displays the shell's input and output. These are related but distinct tools.

Choose the track for your operating system

- Bash track: macOS and Linux
- PowerShell track: Windows 11

The tracks teach the same learning outcomes using native commands. Do not try to memorize both syntaxes before beginning programming. Learn the track for the computer you use; consult the other track when collaborating across operating systems.

Prerequisites

- You can use your computer's keyboard.

- You can open a terminal application and move focus to its command prompt.
- If you use a screen reader or braille display, you can read text output and move by line or character. The lessons include practice for improving this workflow.

No programming experience is required.

Learning outcomes

After completing either track, you should be able to:

1. Explain the difference between a terminal, a shell, a command, a path, and a process.
2. Identify the current working directory and navigate between directories.
3. List, create, copy, move, rename, and remove files and directories.
4. Read command syntax, options, arguments, and quoted paths containing spaces.
5. Use tab completion and command history to reduce typing.
6. Read standard output and error messages, recognize an exit status, and interrupt a running command.
7. Search for text and inspect file contents without relying on a graphical file manager.
8. Run a program from its project directory and understand why the working directory matters.
9. Use help and documentation to investigate an unfamiliar command.
10. Recognize destructive commands and verify a target before changing or deleting data.

Lesson sequence

Lesson	Topic	Evidence of learning
1	Terminal, shell, prompt, and first commands	Identify the shell and print a working directory
2	Paths and directory navigation	Move through a practice directory tree
3	Files and directories	Create, copy, rename, move, and remove practice files
4	Command structure and efficient entry	Use options, quoting, completion, and history
5	Reading, searching, and filtering text	Inspect a text file and find matching lines

Lesson	Topic	Evidence of learning
6	Running programs and understanding results	Run a script and interpret output and errors
7	Processes, help, and safe troubleshooting	Interrupt a process and locate command help
8	Integrated practice project	Complete a small file-based workflow independently

How to study

For each lesson:

1. Read the explanation before entering commands.
2. Type commands yourself. Do not paste a command you cannot explain.
3. Predict what a command will do before running it.
4. Compare the result with the expected behavior.
5. Change one detail at a time and observe the difference.
6. Complete the activity in a disposable practice directory.
7. Explain what each command did in your own words.

Commands are shown in fenced blocks labeled for the intended shell. Text shown as output is illustrative; do not enter it as a command.

Accessibility practices

- Keep the terminal focused while entering commands; use the keyboard to switch between the editor and terminal.
- Read command output sequentially. Use screen-reader review commands to revisit earlier output when needed.
- Prefer short commands and one operation per line while learning.
- Use descriptive filenames and directories. Avoid relying on color, alignment, or visual location.
- Learn how your terminal announces the prompt, command line, and output. Terminal behavior varies by application and screen reader.
- If output is long, redirect it to a file or use a pager only after learning how to exit the pager. The lessons explain this explicitly.
- Do not assume a command succeeded because it returned to the prompt. Check the output and, where appropriate, its exit status.

Safety rules

1. Practice in a dedicated folder such as `terminal-practice`, not in your home directory or a course repository.
2. Read a command before running it. Be especially cautious with recursive deletion, wildcards, administrator privileges, and commands copied from the internet.
3. List the target directory before moving or deleting files.
4. Do not run commands containing unexplained downloads piped directly into a shell.
5. Do not use elevated privileges to solve an error until you understand why permission was denied.
6. Keep credentials, access tokens, and private information out of command history and shared terminal transcripts.

Completion standard

You are ready to begin the programming curricula when you can complete the final practice project without following a step-by-step command recipe, explain the commands you chose, and recover from at least one intentional mistake. Use the track's skills checklist to identify anything that needs more practice.

Reference documentation

- GNU Bash manual
- GNU Coreutils manual
- Apple: Use the command line
- Microsoft: PowerShell documentation
- Microsoft: PowerShell about topics

1.1 Bash Track: macOS and Linux Terminal Skills

Scope

This track uses Bash syntax and common commands available on macOS and mainstream Linux distributions. macOS commonly opens zsh by default. Most commands in this track also work in zsh, but shell-specific behavior is identified where relevant. Linux distributions may package some utilities differently.

Complete the terminal skills overview first. Work only inside a practice directory you create for these lessons.

Lesson 1: Meet the terminal and shell

Learning objectives

- Distinguish the terminal application from the shell.
- Recognize the command prompt and command line.
- Display the current working directory and identify the shell.

Concepts

The terminal is the application that provides text input and output. Bash is a shell: it interprets commands and starts programs. The prompt is the shell's invitation to enter a command. Your prompt may contain your username, computer name, current directory, or other information; its appearance is configurable and is not a reliable substitute for checking the directory.

A command generally has a command name followed by options and arguments. Press Enter to run it. A command that finishes normally returns control to the prompt.

Try it

```
pwd
whoami
uname -s
printf 'Terminal practice is ready.\\n'
```

Expected behavior

- `pwd` prints the absolute path of the current working directory.
- `whoami` prints the current account name.
- `uname -s` identifies the operating-system kernel.
- `printf` prints the supplied text followed by a newline.

The exact paths and account name vary by computer.

Activities

1. Run each command separately and describe its output.
2. Run `pwd` again after opening a new terminal window. Compare the locations.
3. Explain why the prompt's appearance does not prove which directory is active.

Lesson 2: Paths and directory navigation

Learning objectives

- Explain the difference between an absolute and relative path.

- Move to a directory and return to the home directory.
- List directory contents.

Concepts

A path identifies a file or directory. An absolute path begins at the filesystem root, written `/` on macOS and Linux. A relative path is interpreted from the current working directory. A forward slash separates path components.

Special path names include `.` for the current directory, `..` for its parent, and `~` for your home directory. Use quotes around a path containing spaces.

Try it

```
pwd
ls
ls -la
cd ~
pwd
mkdir -p terminal-practice/first-project
cd terminal-practice/first-project
pwd
cd ..
pwd
cd ~
```

Expected behavior

The commands list the current directory, move to your home directory, create a nested practice directory, enter it, move to its parent, and return home. The `-la` options request a detailed listing that includes hidden entries; hidden names conventionally begin with a dot.

Activities

1. Create `terminal-practice/first-project/notes` using `mkdir -p`.
2. Enter the `notes` directory using a relative path.
3. Return to your home directory using `cd ~`.
4. Explain the difference between `cd ..` and `cd ~`.

Lesson 3: Create and manage files

Learning objectives

- Create an empty file and a directory.
- Copy, move, rename, and remove practice items.
- Verify each change by listing the directory.

Concepts

Use `touch` to create an empty file, `cp` to copy, `mv` to move or rename, and `rm` to remove a file. Use `mkdir` to create a directory. These commands act on the paths you provide; they do not ask whether you intended a different target.

For this lesson, use only the practice folder created in Lesson 2.

Try it

```
cd ~/terminal-practice/first-project
mkdir -p notes
touch notes/todo.txt
cp notes/todo.txt notes/todo-backup.txt
mv notes/todo-backup.txt notes/archive.txt
ls -l notes
rm notes/archive.txt
ls -l notes
```

Expected behavior

The first listing contains `todo.txt` and `archive.txt`. The second contains only `todo.txt`.

Activities

1. Create a directory named `drafts` and an empty file named `drafts/idea.txt`.
2. Copy the file to `drafts/idea-copy.txt`.
3. Rename the copy to `idea-final.txt`.
4. List the directory and verify both files before deleting the copy.
5. Explain why you should inspect a target before using `rm`.

Safety: deletion

`rm` removes files; it does not move them to a graphical trash folder. Recursive deletion with `rm -r` can remove an entire directory tree. Do not use `rm -rf`, wildcards, or recursive deletion in these introductory exercises. Never experiment with deletion from the filesystem root, your home directory, or a repository.

Lesson 4: Command structure and efficient entry

Learning objectives

- Identify command names, options, and arguments.
- Quote paths containing spaces.
- Use tab completion and command history.

Concepts

Options modify command behavior. Many short options begin with one hyphen, such as `-l`; long options often begin with two, such as `--all`. Exact options depend on the command. Arguments identify the data or location on which the command operates.

Quotes keep spaces and shell-special characters inside one argument. The shell expands an unquoted wildcard such as `*.txt` to matching filenames before the command runs. This is useful but can affect more files than expected.

Press Tab to request completion. Press the Up Arrow to recall a previous command. Use Left Arrow and Right Arrow to edit the command line before pressing Enter.

Try it

```
cd ~/terminal-practice
mkdir "project notes"
touch "project notes/first draft.txt"
ls -l "project notes"
printf '%s\\n' "A quoted path can contain spaces."
```

Activities

1. Create a directory named `week one` and a file named `plan for week one.txt`.
2. Use Tab completion to finish a directory or filename.
3. Recall a previous harmless command with the Up Arrow, edit one argument, and run it.
4. Explain why quoting `project notes` matters.

Lesson 5: Read, search, and filter text

Learning objectives

- Display a text file.
- Search for a word or phrase.
- Understand basic output redirection and pipelines.

Concepts

`cat` displays a file's contents. `less` opens a pager for longer output; press `q` to exit. `grep` searches text for a pattern. The vertical bar `|` sends one command's standard output to another command as input. The greater-than sign `>` redirects output to a file and replaces that file's previous contents; `>>` appends instead.

Try it

```
cd ~/terminal-practice
printf 'apple\\nbanana\\napricot\\n' > fruit.txt
cat fruit.txt
grep 'ap' fruit.txt
grep -n 'ap' fruit.txt
cat fruit.txt | grep 'banana'
printf 'grape\\n' >> fruit.txt
cat fruit.txt
```

Expected behavior

The first search finds `apple` and `apricot`. The `-n` option includes line numbers. The pipeline prints the matching `banana` line. The final listing includes the appended `grape`.

Activities

1. Create a file containing three short lines of your own.
2. Search for one word and then a word that does not occur.
3. Use `grep -n` and explain the line number.
4. Explain the difference between `>` and `>>`.

Lesson 6: Run programs and interpret results

Learning objectives

- Run a command-line program from a known directory.
- Distinguish output from an error message.
- Understand the current working directory's effect on relative paths.

Concepts

A program is a file or executable command that the shell starts. For a script, the runtime may be named explicitly. For example, `python3 hello.py` starts Python and asks it to run the file `hello.py`. A relative filename is resolved from the current working directory, not automatically from the script's location.

A process may print normal output to standard output or diagnostics to standard error. A nonzero exit status commonly indicates failure. In Bash, `$?` contains the exit status of the most recently completed command; inspect it immediately because the next command changes it.

Try it

```
cd ~/terminal-practice
printf "print('Hello from the terminal')\\n" > hello.py
```

```
python3 hello.py
printf 'Exit status: %s\\n' "$?"
python3 missing-file.py
printf 'Exit status: %s\\n' "$?"
```

Expected behavior

The first run prints `Hello from the terminal` and normally returns status `0`. The second run reports that the file cannot be found and normally returns a nonzero status. Exact diagnostic wording varies by Python version.

Activities

1. Change the greeting and run the program again.
2. Run the program from a different directory using its absolute path.
3. Explain why `python3 hello.py` may fail when run from the wrong directory.
4. Record the difference between a program's printed output and its diagnostic.

Lesson 7: Processes, help, and troubleshooting

Learning objectives

- Interrupt a foreground process.
- Find built-in help and manual pages.
- Diagnose a missing command or incorrect path.

Concepts

A running command is a process. Press Ctrl+C to send an interrupt to the foreground process. It usually stops the command, but a program may handle the interrupt differently.

Use `command --help` when supported. Use `man command` for a manual page; press `q` to exit. Not every command implements the same help option. A message such as `command not found` means the shell could not locate a command with that name in its command search path.

Try it

```
man ls
man pwd
command -v python3
```

Activities

1. Open the manual page for `ls` and exit with `q`.

2. Use `command -v` to locate `python3`.
3. Intentionally enter a misspelled harmless command and read the diagnostic.
4. Start a long-running command such as `sleep 60`, then stop it with Ctrl+C.

Lesson 8: Integrated practice project

Goal

Create a small project folder, add a text file and a Python script, inspect the files, run the script, and verify the result.

Requirements

1. Create `terminal-practice/final-project`.
2. Inside it, create `data` and `output` directories.
3. Create `data/message.txt` with one line of text.
4. Create `hello.py` that prints `Terminal skills are ready.`.
5. List the project files and inspect the message file.
6. Run the Python script from the project directory.
7. Record the exit status and explain why it indicates success or failure.
8. Make a copy of the message file, verify the copy, and remove only that copy.

One possible command sequence

```
cd ~/terminal-practice
mkdir -p final-project/data final-project/output
printf 'Practice makes progress.\\n' > final-project/data/message.txt
printf "print('Terminal skills are ready.')\\n" > final-project/hello.py
cd final-project
find . -type f
cat data/message.txt
python3 hello.py
printf 'Exit status: %s\\n' "$?"
cp data/message.txt output/message-copy.txt
ls -l output
rm output/message-copy.txt
ls -l output
```

Reflection

Explain each command in the sequence. Identify which paths are relative, what the current working directory is during the script run, and which command would be dangerous if its target were mistyped.

Bash skills checklist

- ☐ I can open a terminal and identify the shell prompt.
- ☐ I can display and explain the current working directory with `pwd`.
- ☐ I can navigate with `cd`, including `~`, `.`, and `..`.
- ☐ I can list files with `ls` and recognize hidden entries.
- ☐ I can create directories and files with `mkdir` and `touch`.
- ☐ I can copy, move, rename, and safely remove practice files.
- ☐ I can quote paths containing spaces.
- ☐ I can use Tab completion and command history.
- ☐ I can inspect and search text with `cat` and `grep`.
- ☐ I understand output redirection and a basic pipeline.
- ☐ I can run a program from its project directory.
- ☐ I can read an error message and inspect an exit status.
- ☐ I can interrupt a foreground process with Ctrl+C.
- ☐ I can find command help and complete the final practice project.

Further reference

- GNU Bash manual
- GNU Coreutils manual
- Apple Terminal User Guide
- Python command-line documentation

Recommended introductory books

- Shotts, William. *The Linux Command Line: A Complete Introduction*, 3rd edition. No Starch Press, 2026. A practical introduction to shell use, files, processes, text tools, scripting, and the Linux command-line environment. Publisher page.

Suggested use: this is the primary supplementary text for the Bash track. The book focuses on Linux; macOS users should expect some differences in available commands and utilities. Follow this course's Bash examples and your operating system's manual pages when behavior differs.

1.2 PowerShell Track: Windows 11 Terminal Skills

Scope

This track uses PowerShell 7-compatible syntax and commands that are also available in the Windows PowerShell shipped with Windows 11 unless otherwise noted. PowerShell is a command shell and scripting language. The examples use native PowerShell commands rather than Command Prompt syntax.

Complete the terminal skills overview first. Work only inside a practice directory you create for these lessons.

Lesson 1: Meet the terminal and shell

Learning objectives

- Distinguish the terminal application from the shell.
- Recognize the PowerShell prompt and command line.
- Display the current working directory and identify the PowerShell version.

Concepts

Windows Terminal is a terminal application. PowerShell is a shell that interprets commands and starts programs. The prompt commonly displays the current path followed by a greater-than sign. Its appearance is configurable; always check the working directory rather than relying on the prompt's visual appearance.

A command may be a PowerShell cmdlet, function, alias, script, or external program. Cmdlet names commonly use a Verb-Noun pattern, such as `Get-Location`. Press Enter to run a command.

Try it

```
Get-Location
$env:USERNAME
$PSVersionTable.PSVersion
Write-Output "Terminal practice is ready."
```

Expected behavior

- `Get-Location` displays the current working directory.
- `$env:USERNAME` displays the current account name.
- `$PSVersionTable.PSVersion` displays the installed PowerShell version.
- `Write-Output` writes the supplied text to the output stream.

Exact paths, account names, and version numbers vary.

Activities

1. Run each command separately and describe its output.
2. Open a new terminal tab and compare its starting directory.
3. Explain why the prompt's appearance does not prove which directory is active.

Lesson 2: Paths and directory navigation

Learning objectives

- Explain the difference between an absolute and relative path.
- Move to a directory and return to the home directory.
- List directory contents.

Concepts

A path identifies a file or directory. A Windows absolute path commonly begins with a drive letter, such as `C:\Users\Student`. A relative path is interpreted from the current working directory. Windows paths conventionally use backslashes, though PowerShell also accepts forward slashes in many path contexts.

The `.` symbol means the current directory and `..` means its parent. The automatic variable `$HOME` identifies your home directory. Use quotes around paths containing spaces.

Try it

```
Get-Location
Get-ChildItem
Get-ChildItem -Force
Set-Location $HOME
Get-Location
New-Item -ItemType Directory -Path "$HOME\terminal-practice\first-project" -Force
Set-Location "$HOME\terminal-practice\first-project"
Get-Location
Set-Location ..
Get-Location
Set-Location $HOME
```

Expected behavior

The commands list the current directory, move to your home directory, create a nested practice directory, enter it, move to its parent, and return home. `-Force` includes hidden and system entries where permitted; it does not mean that files are being deleted or overwritten.

Activities

1. Create `terminal-practice\first-project\notes` using `New-Item`.
2. Enter the `notes` directory using a relative path.
3. Return to your home directory using `Set-Location $HOME`.
4. Explain the difference between `Set-Location ..` and `Set-Location $HOME`.

Lesson 3: Create and manage files

Learning objectives

- Create an empty file and a directory.
- Copy, move, rename, and remove practice items.
- Verify each change by listing the directory.

Concepts

Use `New-Item` to create a file or directory, `Copy-Item` to copy, `Move-Item` to move or rename, and `Remove-Item` to remove an item. These commands act on the paths you provide; they do not know whether you intended a different target.

For this lesson, use only the practice folder created in Lesson 2.

Try it

```
Set-Location "$HOME\terminal-practice\first-project"
New-Item -ItemType Directory -Path notes -Force | Out-Null
New-Item -ItemType File -Path notes\todo.txt
Copy-Item notes\todo.txt notes\todo-backup.txt
Move-Item notes\todo-backup.txt notes\archive.txt
Get-ChildItem notes
Remove-Item notes\archive.txt
Get-ChildItem notes
```

Expected behavior

The first listing contains `todo.txt` and `archive.txt`. The second contains only `todo.txt`.

Activities

1. Create a directory named `drafts` and an empty file named `drafts\idea.txt`.
2. Copy the file to `drafts\idea-copy.txt`.
3. Rename the copy to `idea-final.txt`.
4. List the directory and verify both files before deleting the copy.
5. Explain why you should inspect a target before using `Remove-Item`.

Safety: deletion

`Remove-Item` deletes the specified item. Recursive removal can delete an entire directory tree. Do not use `-Recurse`, wildcards, or forceful deletion in these introductory exercises. Never experiment with deletion from the root of a drive, your home directory, or a repository.

Lesson 4: Command structure and efficient entry

Learning objectives

- Identify command names, parameters, and arguments.
- Quote paths containing spaces.
- Use tab completion and command history.

Concepts

PowerShell cmdlets commonly use named parameters beginning with a hyphen, such as `-Path` and `-ItemType`. A parameter controls command behavior; its argument supplies a value. PowerShell command names are generally case-insensitive.

Quotes keep spaces and special characters inside one argument. Single quotes create literal strings; double quotes allow variable expansion, such as `"$HOME\Documents"`. PowerShell's wildcard character `*` can match multiple names, so verify the target before using a wildcard with a command that changes or removes data.

Press Tab to complete a command or path. Press the Up Arrow to recall a previous command. Use Left Arrow and Right Arrow to edit a command before pressing Enter.

Try it

```
Set-Location $HOME
New-Item -ItemType Directory -Path "project notes"
New-Item -ItemType File -Path "project notes\first draft.txt"
Get-ChildItem "project notes"
Write-Output 'A quoted path can contain spaces.'
```

Activities

1. Create a directory named `week one` and a file named `plan for week one.txt`.
2. Use Tab completion to finish a directory or filename.
3. Recall a previous harmless command with the Up Arrow, edit one argument, and run it.
4. Explain the difference between single-quoted and double-quoted strings.

Lesson 5: Read, search, and filter text

Learning objectives

- Display a text file.
- Search for a word or phrase.
- Understand basic output redirection and pipelines.

Concepts

Use `Get-Content` to display a text file. Use `Select-String` to search text for a pattern. The vertical bar `|` sends objects or output from one command to another. The greater-than sign `>` redirects output to a file and replaces that file's previous contents; `>>` appends instead.

PowerShell commands often pass structured objects through a pipeline, not just plain text. For this introductory lesson, focus on the visible results and the role of each command.

Try it

```
Set-Location $HOME\terminal-practice
"apple", "banana", "apricot" | Set-Content fruit.txt
Get-Content fruit.txt
Select-String -Path fruit.txt -Pattern "ap"
Select-String -Path fruit.txt -Pattern "ap" -List
Get-Content fruit.txt | Select-String -Pattern "banana"
Add-Content fruit.txt "grape"
Get-Content fruit.txt
```

Expected behavior

The first search finds `apple` and `apricot`. The pipeline prints the matching `banana` line. The final listing includes the appended `grape`.

Activities

1. Create a file containing three short lines of your own.
2. Search for one word and then a word that does not occur.
3. Use `Select-String` and explain how it reports matching lines.
4. Explain the difference between `Set-Content` and `Add-Content`.

Lesson 6: Run programs and interpret results

Learning objectives

- Run a command-line program from a known directory.
- Distinguish output from an error message.
- Understand the current working directory's effect on relative paths.

Concepts

A program is a file or executable command that PowerShell starts. For a Python script, `py -3 hello.py` starts the Python launcher and asks it to run `hello.py`. A relative filename is resolved from the current working directory, not automatically from the script's location.

PowerShell displays errors in the host and exposes the most recent native program exit code through `$LASTEXITCODE`. For PowerShell cmdlets, inspect the error record and the command's documented behavior; `$LASTEXITCODE` is specifically for native programs and scripts that set an exit code.

Try it

```
Set-Location $HOME\terminal-practice
"print('Hello from the terminal')" | Set-Content -Encoding utf8 hello.py
py -3 hello.py
"Exit status: $LASTEXITCODE"
py -3 missing-file.py
"Exit status: $LASTEXITCODE"
```

Expected behavior

The first run prints `Hello from the terminal` and normally returns status `0`. The second run reports that the file cannot be found and normally returns a nonzero status. Exact diagnostic wording varies by Python version and launcher installation.

Activities

1. Change the greeting and run the program again.
2. Run the program from a different directory using its absolute path.
3. Explain why `py -3 hello.py` may fail when run from the wrong directory.
4. Record the difference between a program's printed output and its diagnostic.

Lesson 7: Processes, help, and troubleshooting

Learning objectives

- Interrupt a foreground process.
- Find built-in help.
- Diagnose a missing command or incorrect path.

Concepts

A running command is a process. Press Ctrl+C to send an interrupt to the foreground process. It usually stops the command, but a program may handle the interrupt differently.

Use `Get-Help CommandName` to request help. On a computer without locally installed help files, PowerShell may offer to update them; you can instead use the online help option where supported. Use `Get-Command` to discover commands and `Get-Command python` to check whether a command can be found.

Try it

```
Get-Help Get-ChildItem -Examples
Get-Command py
Get-Command python
```

Activities

1. Read the examples for `Get-ChildItem` and identify one parameter.
2. Use `Get-Command` to locate the Python launcher.
3. Intentionally enter a misspelled harmless command and read the error.
4. Start a long-running command such as `Start-Sleep -Seconds 60`, then stop it with Ctrl+C.

Lesson 8: Integrated practice project

Goal

Create a small project folder, add a text file and a Python script, inspect the files, run the script, and verify the result.

Requirements

1. Create `terminal-practice\final-project`.
2. Inside it, create `data` and `output` directories.
3. Create `data\message.txt` with one line of text.
4. Create `hello.py` that prints `Terminal skills are ready.`.
5. List the project files and inspect the message file.
6. Run the Python script from the project directory.
7. Record the exit status and explain why it indicates success or failure.
8. Make a copy of the message file, verify the copy, and remove only that copy.

One possible command sequence

```
Set-Location $HOME\terminal-practice
New-Item -ItemType Directory -Path final-project\data -Force
New-Item -ItemType Directory -Path final-project\output -Force
"Practice makes progress." | Set-Content final-project\data\message.txt
"print('Terminal skills are ready.')" | Set-Content final-project\hello.py
Set-Location final-project
Get-ChildItem -Recurse -File
Get-Content data\message.txt
py -3 hello.py
"Exit status: $LASTEXITCODE"
Copy-Item data\message.txt output\message-copy.txt
Get-ChildItem output
```

```
Remove-Item output\message-copy.txt
Get-ChildItem output
```

Reflection

Explain each command in the sequence. Identify which paths are relative, what the current working directory is during the script run, and which command would be dangerous if its target were mistyped.

PowerShell skills checklist

- ☐ I can open a terminal and identify the PowerShell prompt.
- ☐ I can display and explain the current working directory with `Get-Location`.
- ☐ I can navigate with `Set-Location`, including `$HOME`, `.`, and `..`.
- ☐ I can list files with `Get-ChildItem` and recognize hidden entries.
- ☐ I can create directories and files with `New-Item`.
- ☐ I can copy, move, rename, and safely remove practice files.
- ☐ I can quote paths containing spaces.
- ☐ I can use Tab completion and command history.
- ☐ I can inspect and search text with `Get-Content` and `Select-String`.
- ☐ I understand output redirection and a basic pipeline.
- ☐ I can run a program from its project directory.
- ☐ I can read an error message and inspect a native program's exit code.
- ☐ I can interrupt a foreground process with Ctrl+C.
- ☐ I can find command help and complete the final practice project.

Further reference

- Microsoft PowerShell documentation
- PowerShell about topics
- Windows Terminal documentation
- Python on Windows

Recommended introductory books

- Plunk, Travis, James Petty, Tyler Leonhardt, Don Jones, and Jeffery Hicks. *Learn PowerShell in a Month of Lunches*, 4th edition. Manning, 2022. A hands-on introduction to PowerShell commands, the help system, the object pipeline, variables, scripts, loops, errors, and debugging. It covers Windows, Linux, and macOS. Publisher page.

Suggested use: this is a practical companion for the PowerShell track and does not require prior scripting experience. Its examples span multiple PowerShell versions and operating systems; use the course's PowerShell version baseline and current Microsoft documentation when details differ.

Part II

Development environment

Chapter 2

Development Editors, IDEs, and Project Development Containers

Purpose and scope

An integrated development environment (IDE) combines source editing with language-aware services such as diagnostics, navigation, refactoring, debugging, testing, and project management. A code editor may provide many of these capabilities through extensions. The practical distinction is the depth of project and language tooling.

This reference identifies the development environments used or discussed in this curriculum and explains their appropriate roles. **It is not an installation guide for third-party IDEs.** Product descriptions are not instructions to install software, select extensions, configure accounts, or alter operating-system settings. Follow institution-approved technical support and vendor documentation for operational procedures.

The curriculum's general-purpose editor choices are **Visual Studio Code (VS Code)** and **Eclipse Theia**. They are distinct products, not interchangeable names for one application. The recommended project-environment model is to keep the development toolchain reproducible and project-scoped, preferably through a **Development Container (Dev Container)** when compiler, linter, formatter, test runner, or system-library consistency is important.

Development Containers: the recommended project model

A **Development Container**, often shortened to *Dev Container*, is a containerized workspace configured specifically for software development. It is not merely a container that runs the finished application. It provides the language runtime, compiler, SDK, build tools, linters, formatters, test tools, libraries, and supporting command-line utilities needed to work on a repository.

The Development Containers Specification defines a tool-neutral metadata format for describing such environments. A project commonly stores its configuration in `.devcontainer/devcontainer.json`, alongside an optional Dockerfile, Compose configuration, scripts, and supporting files. A compatible editor or service interprets that metadata and connects the development interface to the environment. The project can therefore document and version the toolchain as part of the repository rather than relying on undocumented differences between learners' computers. Specification · VS Code overview · Specification-supported tools

Why this is especially useful in a programming course

A course may require a compiler and runtime, a linter, a formatter, a test framework, a debugger, system headers or libraries, and a particular version of each. These tools interact. A learner may otherwise have a correct source file but an incompatible compiler, a missing system library, a linter version that reports different diagnostics, or a test runner that behaves differently from the instructor's.

A repository-defined Dev Container can standardize:

- **Language toolchains:** interpreters, compilers, SDKs, JDKs, and runtime versions.
- **Code quality tools:** linters, static analyzers, formatters, and documentation checkers.
- **Build and test tools:** build systems, test frameworks, coverage tools, and test data.
- **System dependencies:** headers, native libraries, and command-line utilities required by the project.
- **Editor integration:** supported language services and project-specific editor customizations.
- **Supporting services:** databases, message brokers, or other dependencies when a lesson genuinely requires them.
- **Reproducibility metadata:** pinned image versions, feature versions, environment variables that are not secrets, and documented lifecycle commands.

This can make a learning environment more predictable across Windows,

macOS, and Linux hosts. The host still supplies the container runtime and operating-system kernel; the container supplies a controlled user-space environment. A container is not a full virtual machine and does not provide a different kernel.

Typical project structure

A project may use a structure such as:

```
project/
   .devcontainer/
      devcontainer.json
      Dockerfile
      compose.yaml          # optional; for multi-service environments
   src/
   tests/
   scripts/
   README.md
   language-specific project and lock files
```

Not every project needs all these files. A simple project may refer to a maintained development image directly. A project with additional system packages may build from a Dockerfile. A project that depends on multiple services may use Compose. The `.devcontainer` directory contains the development-environment description; source code and tests remain ordinary project files.

What the configuration describes

A `devcontainer.json` commonly identifies or builds the development image, configures the workspace location, declares development-container Features, sets a non-root user, defines environment variables, forwards development ports, and describes lifecycle commands. Editor-specific customizations may identify extensions or settings for a compatible editor.

A Dockerfile describes how the image is built: its base image, system packages, users, and other image contents. Compose can describe a group of related services, such as a development workspace and a database. The Dev Container configuration ties the developer's workspace experience to these container resources.

Illustrative configuration (not a complete course image):

```
{
  "name": "Course development environment",
  "build": {
    "dockerfile": "Dockerfile"
  },
  "remoteUser": "vscode",
```

```
  "forwardPorts": [],
  "customizations": {
    "vscode": {
      "extensions": []
    }
  }
}
```

This example shows the shape of project metadata only. It does not install a compiler, select a language version, or constitute a ready-to-run environment. A real course repository must provide a maintained Dockerfile or image, explicit tool versions, validation commands, and any required features.

Dev Containers as a teaching and assessment control

For this curriculum, an instructor can publish a known development environment for a language unit or project. The environment can include the exact compiler and linter versions used by automated checks. Learners then run the same project scripts locally inside the container that CI runs in a clean build environment.

A robust project should expose clear commands for tasks such as:

- formatting checks (for example, verify formatting without silently changing submitted files);
- linting and static analysis;
- compilation or build;
- unit tests and integration tests;
- expected-output checks for instructional examples;
- documentation and link validation where applicable.

These commands should be documented in the repository and, where practical, invoked by the same CI workflow. A Dev Container is a development convenience and consistency mechanism; **it does not replace continuous integration**. CI should independently build and test a clean checkout so that a learner's persistent container state cannot conceal missing files, undeclared dependencies, or order-dependent tests.

Images, versions, and reproducibility

A development image is a versioned filesystem and toolchain template from which a container is created. For instructional use, avoid relying on floating tags such as `latest` when a change could alter compiler diagnostics or program behavior without a course revision. Prefer explicit supported release tags or immutable digests for controlled teaching environments. Pin language versions, package manager versions, Dev Container Features, and relevant system dependencies when reproducibility matters.

Rebuilds should be intentional and tested. Updating the base image, compiler, or linter can change warnings, formatting, generated files, or test behavior. Treat such changes as curriculum maintenance: review them, run the validation suite, and record the resulting toolchain baseline.

Dev Containers and accessibility

A container does not automatically make an editor accessible. The editor may run on the host while language servers, compilers, and extensions run inside the container. This separation can improve consistency, but it also creates multiple components that must work together.

Evaluate keyboard navigation, screen-reader announcements, braille output, editor diagnostics, terminal interaction, build output, test results, debugger behavior, and remote-session focus. Check whether the editor's remote-development extension is accessible in the actual version used. Ensure that essential instructions and results are available as text. Do not require a graphical container dashboard or mouse-only workflow as the only way to complete an assignment.

Risks and operational boundaries

Containers are isolation mechanisms, not guarantees of security. A development container may have access to mounted source files, network resources, credentials, and host resources. Treat untrusted images, Features, package install scripts, and workspace lifecycle commands as executable code.

- Use trusted base images and inspect their provenance.
- Prefer a non-root development user.
- Do not bake access tokens, passwords, SSH private keys, or personal data into an image or repository.
- Avoid mounting the container runtime's control socket into a learner's development container unless the task explicitly requires it and the security implications have been reviewed.
- Limit privileged mode, Linux capabilities, host networking, and broad host filesystem mounts.
- Keep source code in version control; do not commit generated container layers, caches, or secrets.
- Use synthetic data for coursework unless a governed data environment is explicitly approved.
- Consider resource quotas and lifecycle policies for shared institutional hosts.

Container engines and tools: Docker and Podman

A Dev Container is the development-environment pattern and metadata; Docker and Podman are container engines/tooling that can build and run containers.

They solve related but different parts of the workflow. The choice of engine affects compatibility, operations, security boundaries, and platform support.

Docker

Docker is a widely used container platform. Its ecosystem includes the Docker Engine, Docker CLI, Docker Desktop, image registries, build tooling, and Compose. Docker Engine commonly uses a long-running daemon to manage images, containers, networks, and volumes. Docker Desktop packages a managed experience for macOS and Windows, where Linux containers generally run in a lightweight Linux virtual machine.

Docker has broad industry adoption and extensive documentation, image availability, Compose workflows, and integration with developer tooling. VS Code's Dev Containers documentation primarily describes a Docker-compatible workflow. This makes Docker the most straightforward baseline when an instructional environment is designed around the default VS Code Dev Containers experience.

Docker Desktop licensing and organizational use are subject to Docker's current terms. Institutional adoption should review licensing, administration, and data-handling requirements rather than assume every deployment has identical terms.

Podman

Podman is an open-source container engine designed to manage OCI containers and images. On Linux it can operate without a central daemon and supports rootless containers, allowing many container operations to run under an ordinary user account. Podman also provides a command-line interface familiar to Docker users and supports pods, image builds, registries, and Compose-oriented workflows through associated tooling.

On macOS and Windows, Linux containers require a Linux environment; Podman commonly uses a managed virtual machine, called a Podman machine. On Linux, rootless operation can reduce the need for a privileged daemon, but it does not eliminate the need to understand user namespaces, storage, networking, and host permissions.

Podman can be attractive for Linux-based teaching labs, organizations prioritizing daemonless and rootless workflows, and teams that prefer open-source container tooling. It supports Docker-compatible interfaces for many common operations, but compatibility is not absolute. Compose behavior, networking, volume permissions, socket/API behavior, build features, and IDE integrations should be tested against the exact versions in use.

Docker and Podman compared

Concern	Docker	Podman
Core model	Engine commonly managed by a daemon	Daemonless architecture on Linux; API service can be exposed when needed
Rootless use	Supported in specific configurations	A major design capability on Linux
macOS / Windows	Docker Desktop provides a managed Linux-container environment	Podman machine provides a managed Linux environment
CLI familiarity	Native Docker CLI	Docker-like CLI for many operations
Image format	OCI-compatible images and Docker ecosystem formats	OCI-compatible images
Multi-container workflows	Docker Compose is widely used	Compose-compatible workflows are available, but compatibility should be checked
Dev Container integration	Common baseline for VS Code documentation and workflows	Possible through compatible interfaces/tooling, but validate the specific editor, extension, and runtime versions
Common selection rationale	Broad adoption, mature ecosystem, low-friction editor support	Rootless/daemonless model, open-source preference, Linux host integration

Neither engine is universally better. The appropriate choice depends on the operating system, institutional policy, editor integration, desired security model, and the exact development-container features used by the project.

Dev Container compatibility is a separate question

A project may conform to the Development Containers Specification while still depending on implementation-specific behavior. The specification describes metadata; it does not guarantee every editor, service, and container engine supports every property or lifecycle feature in the same way.

- **VS Code:** its Dev Containers extension is a mature implementation and its official documentation uses Docker as the standard engine workflow. Podman compatibility can depend on the Docker-compatible API

endpoint and extension/runtime versions. Treat Podman as a validated alternative, not an automatic drop-in replacement.

- **Eclipse Theia:** Theia is an extensible platform, and support for Dev Containers depends on the specific Theia distribution and extensions included by its provider. Confirm that the deployed Theia product supports the required parts of the specification and the selected container engine.
- **Hosted workspaces and other tools:** platforms may implement the specification with managed infrastructure. Users may not control the underlying engine or host. Verify which configuration fields, Features, lifecycle hooks, and Compose patterns are supported.

The Development Containers supporting-tools directory lists implementations and services. Use it as a starting point, then test the actual environment selected for the course.

Curriculum-supported editor choices

Visual Studio Code

Visual Studio Code is a cross-platform source-code editor with an extension ecosystem, integrated terminal, source control interface, language services, debugging interfaces, and workspace features. It is suitable for introductory programming and multi-language projects where learners benefit from a consistent editor across Python, JavaScript, TypeScript, Java, C-family languages, and other supported languages.

VS Code is an editor platform rather than a compiler or runtime. A language extension can provide syntax-aware editing and communicate with a language server, but the relevant compiler, interpreter, SDK, debugger, or external service remains a separate toolchain dependency. Its Dev Containers extension can connect the editor to a project-defined development container.

Accessibility considerations include keyboard operation, editor accessibility support, screen-reader announcements, accessible integrated terminal interaction, and the ability to inspect diagnostics and source-control changes without relying on color or spatial layout. Accessibility is version-, extension-, operating-system-, and assistive-technology-dependent. Validate the complete edit, run, debug, and test workflow with the learner's actual screen reader or other access technology.

Eclipse Theia

Eclipse Theia is an extensible IDE framework and platform for building desktop or browser-based development environments. It shares some interaction conventions and extension concepts with VS Code, but it is a separate project with its own release, packaging, extension compatibility, and product distribution decisions.

Theia is relevant to hosted development environments, institution-managed workspaces, cloud development platforms, and organizations that need to tailor an IDE experience. Its browser-based form can reduce the need for learners to manage a full local toolchain, but it introduces dependence on network access, remote workspace availability, browser accessibility, and the accessibility of the particular Theia distribution.

Do not assume that every VS Code extension works in every Theia deployment. Do not assume that a hosted Theia environment supports the Dev Containers Specification or a particular engine. Evaluate the actual distribution used by the course or institution.

Other professional development environments

The following products are discussed for professional context. They are not required course dependencies, and this curriculum does not provide setup instructions for them.

JetBrains IDE family

JetBrains develops language- and workflow-focused IDEs. Its products are often selected when deep language intelligence, integrated refactoring, test discovery, framework support, and mature debugging workflows are important.

- **IntelliJ IDEA** is a general-purpose IDE particularly associated with Java and Kotlin development, JVM projects, build systems, and enterprise application work.
- **PyCharm** focuses on Python development, including project navigation, debugging, testing, virtual environments, scientific workflows, and web frameworks.
- **WebStorm** focuses on JavaScript, TypeScript, and web application development, with framework-aware navigation, refactoring, and testing support.
- **CLion** targets C and C++ development, including native build systems, debugger integration, and code analysis.
- **Rider** focuses on .NET development and is commonly used for C#, ASP.NET Core, and Unity workflows.
- **DataGrip** is a database IDE for querying, exploring, and managing relational and other supported data platforms.

JetBrains IDEs are relevant when a sustained project uses a language or ecosystem for which the corresponding IDE provides substantial integrated tooling. Product editions, licensing, supported platforms, and accessibility behavior vary. Assess the specific product and version rather than generalizing from one JetBrains IDE to the entire family.

Databricks workspace and notebooks

Databricks is a data and AI platform, not simply a general-purpose desktop IDE. Its workspace combines collaborative notebooks, data engineering workflows, SQL analytics, machine learning, governed data access, jobs, and managed compute. It is commonly used with Apache Spark and the lakehouse architecture.

Databricks notebooks support exploratory and collaborative work in languages such as Python, SQL, Scala, and R, depending on the runtime and workspace configuration. They are useful for data preparation, distributed processing, analytics, experimentation, and production data pipelines. Notebook execution is stateful: cells may depend on prior execution order and shared session state. This differs from a conventional source file executed as a fresh process. Professional workflows therefore need attention to reproducibility, dependency management, data permissions, compute cost, version control, and the distinction between exploratory analysis and production jobs.

The platform is a managed service with workspace-specific permissions, compute policies, and organizational governance. It is not a prerequisite for this introductory curriculum.

JupyterLab and notebook environments

JupyterLab is a web-based interactive computing environment built around notebooks, consoles, terminals, and file-based project work. It is widely used in education, scientific computing, data analysis, and exploratory research. A notebook interleaves executable code, output, and explanatory text.

Notebook workflows are useful for incremental experimentation and communicating analysis. They can also conceal execution-order dependencies, mutable session state, and hidden environmental assumptions. Learners should understand the underlying language and runtime rather than treating a notebook as a substitute for programming fundamentals. Keyboard and screen-reader accessibility must be assessed with the exact notebook version and extensions in use.

RStudio / Posit

RStudio is an IDE strongly associated with R, statistical analysis, data visualization, Quarto, and reproducible research. It provides an integrated source editor, console, package and project workflows, debugging, and data-oriented tools. Posit also develops products and services for publishing and managing data applications and analytical work.

RStudio is especially relevant to statisticians, researchers, analysts, and teams whose work centers on R. The R language module in this curriculum is introductory; RStudio is contextual information, not a required platform.

Microsoft Visual Studio

Microsoft Visual Studio is a full-featured IDE for Windows-centric development, especially .NET, C#, C++, desktop applications, and enterprise workloads. It is distinct from Visual Studio Code. Visual Studio typically provides a more integrated project-system and debugger experience for supported Microsoft development stacks.

Xcode

Xcode is Apple's IDE for developing software for Apple platforms, including iOS, iPadOS, macOS, watchOS, and tvOS. It integrates Swift and Objective-C editing, Apple SDKs, simulators, signing, testing, and application distribution workflows. Apple-platform application development generally depends on Xcode and the corresponding Apple toolchain.

Eclipse IDE

Eclipse IDE is a long-established extensible IDE platform with a strong history in Java and enterprise development, as well as support for other languages through project-specific tooling. It remains relevant in organizations with established Eclipse-based workflows, plugins, and build systems.

Choosing an environment by task

Work type	Typical environment fit	Why
Introductory multi-language coursework	VS Code or an institution-provided Theia environment with a course Dev Container	Consistent editor and toolchain across tracks
Java or Kotlin application development	IntelliJ IDEA; also VS Code or Eclipse in suitable projects	JVM-aware navigation, refactoring, build and test integration
Python application development	VS Code or PyCharm	Language services, testing, debugging, and environment awareness
Front-end JavaScript and TypeScript	VS Code or WebStorm	Type-aware editing and web-framework workflows
Native C and C++	VS Code, CLion, or Visual Studio	Compiler, build-system, and debugger integration
.NET and C#	Visual Studio or Rider; VS Code for lighter workflows	.NET project and debugger support

Work type	Typical environment fit	Why
Apple-platform applications	Xcode	Apple SDK, simulator, signing, and platform toolchain integration
Statistical analysis in R	RStudio / Posit	Statistical and reproducible-research workflows
Exploratory scientific computing	JupyterLab	Interactive code, narrative, and results
Enterprise data engineering and analytics	Databricks workspace	Distributed data processing, governed data, jobs, and collaborative notebooks
Database query and schema work	DataGrip or a database-specific client	Querying, schema exploration, and database-aware tooling

These are common fits, not exclusive rules. A product's marketing category does not establish that it is appropriate for a particular course, accessible to a particular learner, or approved by an institution.

Accessibility and procurement criteria

When evaluating any editor, IDE, Dev Container, or container runtime, assess the complete workflow rather than the source editor alone:

1. Keyboard access to files, navigation, search, diagnostics, terminal, debugging, testing, and source control.
2. Screen-reader announcement of editor text, punctuation, indentation, completion lists, diagnostics, and status changes.
3. Stable focus order and predictable focus restoration when panels, dialogs, remote sessions, and tool windows open and close.
4. Ability to inspect errors and warnings as text, navigate between them, and understand their source locations.
5. Ability to run and debug a program without relying on unlabeled icons, color-only indicators, drag-and-drop, or inaccessible graphical output.
6. Compatibility with the learner's operating system, screen reader, braille display, magnification, and keyboard configuration.
7. Whether the workflow depends on a cloud account, persistent network connection, remote desktop, container VM, or institution-managed permissions.
8. Whether source files remain accessible on the host and whether container file permissions preserve the learner's ability to edit and submit work.

9. Whether the container runtime, image build output, lifecycle commands, and terminal can be operated and understood without visual-only controls.
10. Whether the organization can provide an equivalent accessible route when a specialized interface is not accessible.

A tool should not be mandated solely because it is popular or used by a professional team. The instructional objective is programming competence; an IDE and container are means of supporting that objective. Provide an equivalent accessible route when a product interface blocks participation.

Course expectation

Students should learn to create and manage source files, understand project folders, run programs, interpret diagnostics, and use version control concepts. Those skills transfer between editors and runtimes. When a course supplies a Dev Container, students should also understand what the environment provides, how to run the documented project checks, and why the environment definition is version-controlled. They are not expected to become container administrators.

The course does not require learners to master every product described here, and it does not prescribe installation or configuration steps for third-party IDEs.

Further reading

- Development Containers Specification
- Development Containers supporting tools and services
- VS Code Dev Containers documentation
- Create a development container
- Docker documentation
- Podman documentation
- Podman Desktop documentation
- Visual Studio Code documentation
- Eclipse Theia documentation
- JetBrains product documentation
- Databricks documentation
- Jupyter documentation
- Posit documentation
- Visual Studio documentation
- Xcode documentation
- Eclipse documentation

2.1 Project Environments, Dependency Management, and Development Containers

Purpose

A project environment is the collection of language runtimes, compilers, libraries, tools, configuration, and data assumptions needed to work on a project. Environment isolation keeps one project's dependencies from unintentionally changing another project's behavior.

This lesson explains the concepts and common professional patterns. It is not an installation guide. Commands shown in language-specific pages are examples for recognizing project workflows; use those pages for the supported toolchain and commands.

For this curriculum, a repository-defined **Development Container (Dev Container)** is the preferred model when learners must use a consistent compiler, linter, formatter, test runner, and supporting system libraries. See Development Editors, IDEs, and Project Development Containers for the broader editor and container-runtime discussion.

Learning objectives

By the end of this lesson, learners should be able to:

1. Explain the difference between a language runtime, compiler, package manager, dependency, and project environment.
2. Describe why dependencies should be scoped to a project.
3. Identify common dependency manifests and lockfiles.
4. Explain the purpose of environment variables and secrets.
5. Distinguish virtual environments, development containers, application containers, virtual machines, and hosted workspaces.
6. Explain the roles and differences of Docker and Podman.
7. Describe how reproducible environments support linting, compilation, testing, and collaboration.

Core vocabulary

- **Runtime:** software that executes a program, such as the Python interpreter, Node.js, Java Virtual Machine, .NET runtime, or Julia runtime.
- **Compiler:** software that translates source code into another representation, often machine code or bytecode.
- **Package:** a reusable unit of software distributed for use by other projects.
- **Dependency:** a package, runtime, tool, or service a project requires.
- **Package manager:** a tool that resolves, downloads, records, and sometimes executes dependencies.
- **Manifest:** a project file that declares metadata and direct dependencies.

- **Lockfile:** a generated record of resolved dependency versions and often integrity metadata, intended to make installations more repeatable.
- **Virtual environment:** an isolated language-level environment for a project, such as Python's .venv.
- **Container image:** a versioned template containing a filesystem and metadata from which containers are created.
- **Container:** an isolated process environment that shares the host operating-system kernel while isolating filesystem, process, and network resources according to its configuration.
- **Development Container (Dev Container):** a containerized development workspace, described by project metadata such as devcontainer.json, containing the tools and services needed to work on a codebase.
- **Container engine/runtime:** software that builds, creates, starts, and manages containers and images, such as Docker Engine or Podman.
- **Virtual machine:** a virtualized computer with its own guest operating system and kernel.
- **Hosted workspace:** a development environment provided remotely by an institution or service.

Why isolation matters

Suppose Project A requires version 1 of a library and Project B requires version 2. Installing both as global packages can cause one project to break when the shared installation changes. Project-scoped dependencies make each project's requirements explicit.

Isolation improves reproducibility, testing, security review, maintenance, and collaboration. It does not guarantee safety: a malicious dependency can still execute code, and a container can still be misconfigured or granted excessive access.

Common project records

Ecosystem	Common manifest or project record	Locking or reproducibility record
Python	pyproject.toml	uv.lock or another tool-specific lockfile
JavaScript / TypeScript	package.json	package-lock.json, pnpm-lock.yaml, or yarn.lock
Java	pom.xml or Gradle build files	Wrapper and pinned plugin/dependency versions

Ecosystem	Common manifest or project record	Locking or reproducibility record
Kotlin/JVM	Gradle build files	Gradle wrapper and dependency locking where used
.NET	.csproj and solution files	Central package management and lock files where configured
Rust	Cargo.toml	Cargo.lock
Go	go.mod	go.sum
Julia	Project.toml	Manifest.toml
Lua	rockspec or project-specific records	Tool- and project-specific lock/metadata files

A lockfile should be reviewed and committed when the ecosystem and project policy call for it. It is not a replacement for understanding the manifest or keeping dependencies maintained.

Project-level versus global installation

Global tools can be appropriate for shared command-line utilities or centrally managed machines. Application dependencies should normally be declared at project scope. Avoid installing libraries globally merely to make one lesson run.

A project should make its execution assumptions discoverable. A collaborator should be able to determine the required language version, dependencies, tests, and run command from the repository rather than private notes.

Development Containers

A Dev Container is a project-scoped, reproducible environment for developing, building, linting, and testing software. It is different from a production container: a development image may include compilers, language servers, debuggers, test frameworks, formatters, linters, documentation tools, and shell utilities that should not be shipped in the final application image.

The Development Containers Specification defines metadata that compatible editors and services can use. A project commonly stores the configuration in `.devcontainer/devcontainer.json`, with an optional Dockerfile and Compose file. The metadata identifies the image or build definition, workspace, development user, Features, editor customizations, environment variables, forwarded ports, and lifecycle commands.

Why the course should prefer this model

A common learning environment prevents a large class of avoidable toolchain problems. For example, the instructor's linter and the learner's linter may otherwise use different rules or versions; the compiler may be newer or older; a native library may be missing; or a test runner may behave differently on different host systems.

A well-maintained course Dev Container can provide:

- The documented language runtime and compiler version.
- The exact linter, formatter, static analyzer, and test runner expected by the course.
- System libraries and headers required by compilation.
- Project build scripts and test dependencies.
- Common command-line utilities and documentation tools.
- A consistent shell and workspace layout.
- Optional supporting services such as a database when required by a project.

This is particularly valuable when the curriculum assesses compilation, linting, static analysis, and tests. The learner and instructor can run the same repository-defined checks in a known user-space environment. A container can also reduce host-specific installation requirements, though the host still needs a supported container engine and compatible editor or service.

Example repository layout

```
project/
   .devcontainer/
      devcontainer.json
      Dockerfile
      compose.yaml          # optional
   src/
   tests/
   scripts/
   README.md
   language-specific project and lock files
```

Not every project needs all these files. A simple project may reference a maintained development image. A project with additional system dependencies may use a Dockerfile. A project that requires multiple services may use Compose.

Example responsibilities

File or mechanism	Responsibility
devcontainer.json	Declares how a compatible tool should create or connect to the development environment
Dockerfile	Describes how a development image is built, including base image and system packages
Compose file	Describes multiple related containers and their networks, volumes, and service dependencies
Dev Container Features	Reusable configuration units that add common tools or capabilities
Project manifest	Declares language-level dependencies
Lockfile	Records resolved dependency versions for repeatable dependency resolution
scripts/	Provides named, repeatable project commands for linting, building, testing, and other checks

Make project checks explicit

The project should expose stable, documented commands for formatting checks, linting, static analysis, compilation/build, unit and integration tests, expected-output checks, and documentation validation when relevant. Prefer scripts or task targets with clear names over undocumented sequences of terminal commands.

A useful quality workflow separates checks by purpose:

1. **Format check:** determines whether source formatting matches project policy; it should not silently rewrite submitted work when used as a check.
2. **Lint/static analysis:** identifies likely defects, unsafe patterns, and style violations.
3. **Compile/build:** checks syntax, types, linking, and build dependencies.
4. **Unit tests:** checks individual functions and components.
5. **Integration tests:** checks interactions between modules and services.
6. **Expected-output checks:** verifies examples against documented behavior.
7. **Clean-checkout verification:** ensures the project does not depend on untracked files or persistent state.

A Dev Container is not a substitute for continuous integration. CI should independently build and test a clean checkout using the documented toolchain. This prevents a learner's persistent container state from concealing missing dependencies or undeclared files.

Reproducibility and maintenance

Avoid floating image tags such as latest when a tool update could alter compiler diagnostics or test behavior unexpectedly. Prefer explicit supported versions or immutable digests for controlled course environments. Pin language versions, Dev Container Features, and relevant system dependencies where reproducibility matters.

Changes to a base image, compiler, linter, or formatter are curriculum changes. Review them, run the validation suite, and record the toolchain baseline. Prebuilt images can reduce startup time for large classes, but they must be rebuilt and published when the course environment changes.

Limitations

A container shares the host kernel; it is not a full virtual machine. Host operating system, CPU architecture, filesystem behavior, permissions, networking, and virtualization support can still affect the workflow. A container also does not guarantee that all graphical editor features or assistive technologies will work.

Use a Dev Container when it meaningfully improves consistency. Do not impose container administration on a beginner if a simpler environment meets the learning objectives. The course should provide an accessible fallback or institution-managed hosted workspace where local container support is unavailable.

Docker and Podman

Docker and Podman are container engines and associated tooling. They build and manage images and containers; they are not themselves the Dev Container specification. A Dev Container is a development-environment description that a compatible tool uses with a container engine or service.

Docker

Docker is a widely used container platform. Its ecosystem includes Docker Engine, Docker CLI, Docker Desktop, registries, image-building tools, and Docker Compose. Docker Engine commonly uses a long-running daemon to manage images, containers, networks, and volumes. Docker Desktop provides a managed experience on macOS and Windows, where Linux containers run within a lightweight Linux virtual machine.

Docker has broad adoption, extensive documentation, mature Compose workflows, and wide integration with developer tools. VS Code's Dev Containers documentation primarily describes Docker-based workflows, making Docker the most straightforward baseline when a course standardizes on the default VS Code Dev Containers experience.

Docker Desktop licensing and organizational use are subject to Docker's current terms. Institutions should review current licensing, administration, and data-handling requirements.

Podman

Podman is an open-source container engine designed to manage OCI containers and images. On Linux it can operate without a central daemon and supports rootless containers, allowing many operations to run as an ordinary user. Podman provides a command-line interface familiar to Docker users and supports pods, image builds, registries, and Compose-oriented workflows through associated tooling.

On macOS and Windows, Linux containers require a Linux environment; Podman commonly uses a managed virtual machine called a Podman machine. On Linux, rootless operation can reduce the need for a privileged daemon, but it does not eliminate the need to understand user namespaces, storage, networking, and host permissions.

Podman can be attractive for Linux-based teaching labs, organizations prioritizing daemonless and rootless workflows, and teams that prefer open-source container tooling. It supports Docker-compatible interfaces for many common operations, but compatibility is not absolute. Compose behavior, networking, volume permissions, socket/API behavior, build features, and IDE integrations should be tested against the exact versions in use.

Comparison

Concern	Docker	Podman
Core model	Engine commonly managed by a daemon	Daemonless architecture on Linux; API service can be exposed when needed
Rootless use	Supported in specific configurations	A major design capability on Linux
macOS / Windows	Docker Desktop provides a managed Linux-container environment	Podman machine provides a managed Linux environment
CLI familiarity	Native Docker CLI	Docker-like CLI for many operations
Image format	OCI-compatible images and Docker ecosystem formats	OCI-compatible images

Concern	Docker	Podman
Multi-container workflows	Docker Compose is widely used	Compose-compatible workflows are available; verify compatibility
Dev Container integration	Common baseline for VS Code documentation and workflows	Possible through compatible interfaces/tooling; validate the exact combination
Common selection rationale	Broad adoption, mature ecosystem, straightforward editor support	Rootless/daemonless model, open-source preference, Linux host integration

Neither engine is universally better. Selection depends on host operating system, institutional policy, editor integration, desired security model, and the exact features used by the project.

Compatibility is not guaranteed

A project may conform to the Development Containers Specification while still depending on implementation-specific behavior. The specification describes metadata; it does not guarantee every editor, service, and container engine supports every property or lifecycle feature identically.

- **VS Code:** the Dev Containers extension is a mature implementation and its official documentation uses Docker as the standard engine workflow. Podman compatibility can depend on the Docker-compatible API endpoint and extension/runtime versions. Treat Podman as a validated alternative, not an automatic drop-in replacement.
- **Eclipse Theia:** Dev Container support depends on the specific Theia distribution and extensions included by its provider. Confirm that the deployed product supports the required specification features and selected engine.
- **Hosted workspaces:** a provider may implement the specification using managed infrastructure. Learners may not control the underlying engine or host.

Consult the Development Containers supporting-tools directory and test the actual environment selected for the course.

Environment variables and secrets

Environment variables provide configuration to a process without hard-coding machine-specific values into source code. Common uses include selecting a development mode, locating a service endpoint, or configuring a tool.

Secrets include passwords, access tokens, private keys, and database credentials. Do not commit secrets to source control, include them in screenshots, print them in logs, or paste them into public issue reports. Use institution-approved secret storage or local secret-injection mechanisms. A .env file is not automatically secure; it must be excluded from version control when it contains secrets, and the team must define how its values are distributed.

Do not bake credentials into container images, Dockerfiles, build arguments, or committed Dev Container metadata. Treat mounted credentials and forwarded agent sockets as sensitive host access. Validate required settings without revealing secret contents.

Virtual machines and hosted workspaces

A virtual machine provides a more complete guest-operating-system boundary and is useful when a project requires a different operating system or kernel-level behavior. It generally consumes more resources than a container. Some desktop container products use a VM internally to provide a Linux kernel on macOS or Windows; this does not make each application container a separate VM.

Hosted workspaces can centralize toolchains and reduce local machine administration. They may be useful in classrooms, managed labs, and data platforms. They also introduce dependencies on network connectivity, account access, remote session stability, institutional policies, data location, and platform accessibility.

A cloud notebook or data platform is not identical to a local IDE. Notebook execution may retain state between cells, while a conventional program often starts in a fresh process. Learners should understand that distinction when reproducing results.

Safe environment practices

- Work within the intended project directory.
- Read dependency names and versions before approving installation.
- Prefer trusted package registries and official project documentation.
- Review install scripts, image sources, build steps, and Dev Container Features when working with unfamiliar projects.
- Prefer a non-root user in the development container.
- Avoid running commands with administrator or root privileges unless the task genuinely requires them.
- Do not enable privileged mode or broad host mounts without a reviewed need.
- Do not mount the Docker or Podman control socket into a learner container unless the task explicitly requires it and the security implications have been reviewed.
- Do not disable security controls to make a development tool work.

- Keep secrets out of source code, terminal transcripts, image layers, and commits.
- Use synthetic data for coursework unless a governed data environment is explicitly approved.
- Do not delete environment folders until you have confirmed they contain no source files or personal work.
- Use version control to preserve source and project configuration, not generated caches or private credentials.

Practical activity

Inspect a language project supplied by the instructor. Identify its source files, Dev Container metadata, image or Dockerfile, manifest, lockfile, runtime requirement, lint command, build command, and test command. Explain which files should be committed and which files are generated or machine-specific. Do not install or update dependencies as part of this inspection.

Mastery checklist

- ☐ I can explain why dependencies should be isolated by project.
- ☐ I can distinguish a manifest from a lockfile.
- ☐ I can explain what a Dev Container provides and how it differs from a production container.
- ☐ I can identify where Dev Container metadata and image-build instructions are stored.
- ☐ I can explain how a project Dev Container can standardize compiler, linter, formatter, and test versions.
- ☐ I can distinguish Docker from Podman and describe a situation where each may be selected.
- ☐ I understand that Dev Container specification support and engine compatibility must be verified for the chosen editor.
- ☐ I can explain the roles of environment variables without exposing secrets.
- ☐ I can compare containers, virtual machines, and hosted workspaces.
- ☐ I can describe how CI independently verifies a clean project build and test run.

2.2 Screen-reader-accessible GUI libraries and implementation guidance

This guide identifies GUI frameworks and libraries that can expose application interfaces to screen readers and other assistive technologies for every language and focused module in this curriculum. It also identifies approaches that should not be selected for an accessibility-critical interface without substantial additional engineering and platform testing.

> **Important: no GUI library can guarantee accessibility by itself.** Accessibility depends on the framework, the controls selected, application code, operating-system accessibility APIs, assistive-technology combinations, content, and testing. Treat the recommendations below as technology-selection guidance, not as a conformance certificate or a promise that an application is accessible. Require manual testing with real screen readers and users with disabilities before release.

The phrase *screen-reader accessible* is used throughout. The guide also covers keyboard-only operation, focus visibility, zoom and text scaling, contrast, reduced motion, captions, and other requirements that affect people with disabilities.

1. Baseline requirements for every GUI

Use **WCAG 2.2 Level AA** as the default design and testing target for web content and web-based interfaces. For native applications, apply the relevant platform accessibility API and human-interface guidance as well as WCAG principles. A WCAG checklist alone does not replace platform-specific testing.

Every interactive control must expose, as applicable:

- **Role:** button, link, checkbox, radio button, text field, heading, table, menu item, slider, dialog, and so on.
- **Accessible name:** a concise, meaningful name that identifies the control's purpose. Visible labels should normally provide the name.
- **Value and state:** current text, selected value, checked state, expanded/collapsed state, progress, invalid state, and other relevant information.
- **Relationships:** programmatic association between labels, descriptions, errors, help text, groups, table headers, and the controls they describe.
- **Keyboard operation:** every action must be possible without a pointer. Focus order must be logical; focus must be visible and must not be trapped.
- **Change notifications:** validation errors, status updates, asynchronous results, and dialog openings must be communicated without unexpectedly moving focus.
- **Structure:** headings, landmarks, lists, tables, grouping, and reading order must reflect the meaning and visual organization of the interface.

Also:

1. Prefer built-in, platform-native controls over custom-drawn controls.
2. Never communicate meaning through color, shape, position, sound, or animation alone. Provide text or another perceivable equivalent.
3. Provide text alternatives for meaningful images and accessible names for icon-only controls. Mark decorative images as decorative.

4. Support text resizing, high-contrast or forced-colors modes, adequate contrast, and layouts that reflow without clipping.
5. Respect reduced-motion preferences. Avoid flashing content and autoplaying motion; provide pause, stop, or hide controls where required.
6. Give charts, maps, diagrams, canvases, and other visualizations a complete textual or tabular equivalent. A label on the canvas is not an equivalent.
7. Do not use positive tabindex values on the web. Do not remove focus outlines without an equally visible replacement.
8. Make dialogs announce their title and purpose, manage focus on open, contain focus while modal, and return focus to the invoking control on close.
9. Keep accessible names localized and understandable. Avoid redundant announcements such as repeating a visible label in both the name and description.
10. Document known platform limitations and test the exact packaged application, not just a development preview.

2. Web interfaces: HTML, CSS, JavaScript, and TypeScript

These languages share the browser accessibility platform. **Semantic HTML is the primary accessibility toolkit.** JavaScript frameworks do not replace HTML semantics, and TypeScript does not change browser accessibility behavior.

Recommended foundations and libraries

Technology	Assessment and use
Semantic HTML5	First choice. Use `button`, `a`, `input`, `label`, `select`, `textarea`, `fieldset`, `legend`, headings, landmarks, lists, and data tables for their intended purposes. Native elements provide roles, keyboard behavior, and accessibility mappings.
React	Suitable when components render semantic HTML and manage focus and state correctly. Prefer native controls; use the React accessibility guidance and test rendered DOM, not component names.

Technology	Assessment and use
Angular	Suitable when templates use semantic HTML and Angular CDK accessibility utilities where needed. CDK utilities assist with focus management, live announcements, and keyboard patterns; they do not make arbitrary components accessible automatically.
Vue	Suitable when components render semantic HTML and implement correct keyboard and focus behavior. Audit third-party component libraries individually.
Svelte / SvelteKit	Suitable on the same basis: semantic output, accessible state changes, correct focus management, and tested components. Compiler warnings are useful but do not prove conformance.
WAI-ARIA Authoring Practices Guide (APG) patterns	Use as implementation guidance for custom widgets such as tabs, menus, tree views, and comboboxes. Follow the complete keyboard interaction pattern, not just its ARIA attributes.
Accessible component systems (for example, React Aria)	Useful when a custom interaction is genuinely needed. Confirm current framework, browser, and assistive-technology support, and retain semantic HTML wherever possible.

Implementation rules

- Use a native `<button>` for actions and an `<a href>` for navigation. Do not simulate them with clickable `div` or `span` elements.
- Associate every form control with a visible `<label>`. Use `fieldset` and `legend` for related controls. Connect error and help text using `aria-describedby`; set `aria-invalid="true"` when appropriate.
- Use headings in a meaningful hierarchy and landmarks such as `header`, `nav`, `main`, and `footer`.
- Use ARIA only when native HTML cannot express the required semantics. ARIA changes accessibility-tree information; it does not add keyboard behavior, focus management, or functionality.
- For live updates, use a restrained `aria-live` region or appropriate status/alert role. Do not announce every keystroke or flood the screen reader.

- For custom tabs, menus, listboxes, grids, and comboboxes, implement the full APG keyboard model, including arrow keys, Home/End where specified, Escape, and focus restoration.
- For single-page applications, manage document titles, route announcements, skip links, and focus after navigation.
- Avoid replacing native form controls with visually styled generic elements unless there is a compelling reason and the full interaction model is implemented and tested.

JavaScript and TypeScript framework cautions

Avoid or treat as high-risk:

- Component libraries whose controls are built from generic containers and have no documented keyboard and screen-reader behavior.
- Canvas, WebGL, SVG-only, or absolutely positioned interfaces that expose only a single image-like object instead of individual meaningful controls.
- Drag-and-drop-only interactions, hover-only menus, pointer-only gestures, and custom scrollbars without keyboard alternatives.
- Automatically generated ARIA attributes that do not match the actual role, state, or behavior.
- Framework-specific accessibility claims based solely on lint rules or compile-time warnings.

A framework's accessibility linting is a useful early warning system, not a substitute for browser and assistive-technology testing.

3. Python

Python has several desktop GUI choices. The accessibility of each depends on its native backend and the operating system.

Library / approach	Assessment and recommended use
PySide6 / Qt for Python	Preferred general-purpose cross-platform desktop option when a full GUI is needed. Qt provides accessibility interfaces and standard controls expose roles, names, states, and actions. Use Qt Widgets for conventional forms and controls; test Qt Quick/QML controls and custom QML components on every target platform.

Library / approach	Assessment and recommended use
PyQt6	Uses the same underlying Qt accessibility infrastructure as PySide6. The primary distinction is licensing and packaging, not the accessibility model. Apply the same Qt guidance and testing.
PyGObject with GTK 4	Strong choice for Linux/GNOME-oriented applications. GTK's built-in widgets expose accessibility information through the platform accessibility stack. Prefer standard GTK widgets and follow GNOME HIG guidance. Cross-platform behavior must be verified.
wxPython	Can use native platform widgets and provides accessibility extension points. Treat accessibility as platform-dependent; the documented `wx.Accessible` interface has historically had platform/API limitations. Test the exact wxPython and OS versions.
Tkinter	Convenient standard-library GUI for small teaching examples. Accessibility behavior varies by Tcl/Tk version and operating system. Use standard widgets, visible labels, keyboard traversal, and test with the target screen reader. Do not assume that a working Tab order proves screen-reader support.
Toga (BeeWare)	Uses native widgets on supported platforms, which is a useful design direction. Verify the accessibility behavior of each backend and supported platform before choosing it for an accessibility-critical application.

Library / approach	Assessment and recommended use
Web UI with a Python backend (for example, Flask, Django, FastAPI, or NiceGUI)	Often a good option when the interface can be a browser application. The browser still needs semantic HTML and tested components; a Python server framework does not make its generated UI accessible.

Python: avoid or restrict

- **Kivy** and other interfaces that primarily render custom-drawn widgets: do not select for a screen-reader-critical desktop application unless the current release demonstrably exposes a complete accessibility tree and supports the target assistive technologies.
- **Dear PyGui**, **Pygame**, and raw OpenGL/Canvas-style interfaces: suitable for graphics, games, and visualization experiments, but not a default choice for form-heavy accessible desktop applications. A separate accessible control surface is required.
- **CustomTkinter**: visual styling does not improve the underlying Tk accessibility model. Treat it with the same platform-specific caution as Tkinter.
- **Matplotlib plots embedded in a GUI**: do not make the plot the only way to access data. Supply a structured table, textual summary, keyboard-accessible controls, and exportable data.

4. Java

Library / approach	Assessment and recommended use
Java Swing	Established desktop option with the Java Accessibility API (JFC accessibility support). Prefer standard Swing components, meaningful accessible names/descriptions where needed, and correct labels and relationships. Test the selected JDK and operating system.

Library / approach	Assessment and recommended use
JavaFX	Suitable for modern desktop interfaces. Standard controls expose accessibility roles and text/value information through JavaFX accessibility support. Set accessible text/help where defaults are insufficient; verify custom controls and skins.
Browser UI with Java backend	Use semantic HTML and a tested web component strategy. The backend language does not determine browser accessibility.

Java: avoid or restrict

- **AWT Canvas**, JavaFX `Canvas`, custom-painted Swing components, and custom JavaFX controls without complete accessibility metadata and keyboard behavior.
- Third-party JavaFX skins or Swing Look-and-Feels that replace standard controls without documented accessibility testing.
- Interfaces where mouse listeners are the only way to activate a control.

For custom controls, implement the relevant Java accessibility interfaces and expose role, name, description, state, value, actions, and events. Validate using the target platform's screen reader and accessibility inspection tools.

5. Kotlin

Library / approach	Assessment and recommended use
Jetpack Compose for Android	Preferred modern Android UI approach. Use Material/Foundation components and Compose semantics. Supply `contentDescription` for meaningful icon-only controls, appropriate roles and state descriptions, and logical traversal semantics.

Library / approach	Assessment and recommended use
Compose Multiplatform	Promising shared UI approach for Android, iOS, desktop, and web. It has a semantic tree and accessibility APIs, but platform support and behavior can differ. Test every target, including desktop accessibility enablement and iOS VoiceOver behavior.
Android Views / Material Components	Mature native Android approach. Standard widgets integrate with TalkBack. Use labels, hints, state descriptions, and accessibility delegates only where the default semantics are insufficient.
JavaFX / Swing from Kotlin/JVM	Kotlin can use the same Java accessibility support as Java. Choose standard controls and test as described in the Java section.
Web UI with Kotlin backend or Kotlin/JS	Use semantic HTML and web accessibility practices from Section 2.

Kotlin: avoid or restrict

- Compose `Canvas`, custom drawing, and graphics-only controls as the sole interaction or data representation.
- Custom composables that omit semantics, merge unrelated children, or provide misleading `contentDescription` values.
- Desktop or multiplatform UI libraries whose accessibility support is undocumented or untested on the exact target platform.
- Android layouts that depend on color, touch gestures, or position without equivalent semantics and keyboard/switch access.

6. C

C has no single standard cross-platform GUI library. Prefer a toolkit that connects controls to the operating system's accessibility API.

Library / approach	Assessment and recommended use
GTK 4	Strong choice for Linux desktop applications. Standard widgets provide accessibility support through the GTK accessibility stack. Use GTK controls and GNOME accessibility guidance.
Win32 common controls / Windows UI Automation	Appropriate for Windows-only native applications when implemented with standard controls and correct UI Automation providers/patterns. Custom controls must expose equivalent UIA information.
Qt through C bindings or a C-compatible interface	Qt's accessibility architecture is strong, but Qt is primarily a C++ framework. Use an established, maintained binding rather than inventing a C ABI bridge.
AccessKit C bindings	An accessibility infrastructure option for developers implementing custom-rendered interfaces. It is not a complete GUI toolkit; the application still has to construct and update a correct accessibility tree and support platform adapters.

C: avoid or restrict

- **SDL**, **GLFW**, raw OpenGL/Vulkan, and framebuffer-only interfaces as the sole UI for an accessibility-critical application.
- Custom widget systems that expose pixels but no accessible roles, names, states, actions, or events.
- Legacy or abandoned GUI bindings without current platform accessibility support.

If a graphics library is needed, provide a parallel semantic interface with equivalent functionality and synchronize state and events between the two.

7. C++

Library / approach	Assessment and recommended use
Qt 6 (Widgets / Qt Quick)	Preferred cross-platform C++ GUI toolkit for accessibility-critical desktop applications. Qt exposes accessibility metadata and APIs for custom controls. Prefer standard Widgets controls for conventional interfaces; verify QML/custom controls separately.
GTK 4 with gtkmm	Strong Linux/GNOME-oriented option. Standard GTK widgets expose accessible names and roles; follow GTK and GNOME HIG guidance.
wxWidgets	Uses native controls where available and provides accessibility mechanisms. Confirm the current platform-specific implementation and test it; do not assume identical support on Windows, macOS, and Linux.
Win32 / Windows UI Automation	Suitable for Windows-specific applications using native controls or complete UIA providers for custom controls.
Web UI embedded in a desktop shell	A browser-based front end can use semantic HTML and the practices in Section 2. Verify the embedded webview's browser engine and assistive-technology integration.

C++: avoid or restrict

- **Dear ImGui**, **SDL**, **GLFW**, custom OpenGL/Vulkan interfaces, and game-engine UI as the only interface for a general-purpose accessible application unless a verified accessibility bridge and complete keyboard/screen-reader model are provided.
- Qt/GTK custom-painted widgets that do not implement accessible interfaces.
- Unmaintained wrappers or bindings that lag behind the toolkit's accessibility APIs.

Qt's `QAccessible` interfaces are the extension point for custom Qt widgets. A custom widget is not accessible merely because it is drawn inside a Qt window.

8. C#

Library / approach	Assessment and recommended use
WPF	Strong Windows desktop option. Standard controls integrate with Windows UI Automation. Use AutomationProperties and standard control patterns; implement automation peers for custom controls.
WinUI 3	Native Windows option with UI Automation integration. Prefer standard controls and expose appropriate automation properties and patterns for custom controls.
Windows Forms	Mature Windows-only option. Standard controls have accessibility support; custom controls require accessible object/provider work. Test keyboard navigation, Narrator, and UIA.
.NET MAUI	Suitable for cross-platform mobile/desktop applications when platform-specific semantics are tested. Use semantic properties for descriptions, hints, and headings; test VoiceOver, TalkBack, and desktop screen readers separately.
Avalonia UI	Cross-platform .NET option with automation peers and platform accessibility mappings (UI Automation, NSAccessibility, AT-SPI). Prefer built-in controls; supply AutomationProperties and test each supported platform.
Blazor / ASP.NET Core web UI	Browser-based interfaces should use semantic HTML and Section 2 guidance. Use accessible component libraries only after testing their actual rendered behavior.

C#: avoid or restrict

- **MonoGame**, **Unity UI**, **Win2D**, SkiaSharp custom drawing, and canvas-based interfaces as the only interaction layer for an accessibility-critical business application.
- Custom WPF/Avalonia templates that remove labels, focus visuals, automation peers, or keyboard behavior.

- Third-party controls that claim accessibility but do not document UI Automation patterns and screen-reader testing.

For WPF and WinUI, verify that each custom control exposes the correct UI Automation control type, name, help text, state, supported patterns, and events. For MAUI and Avalonia, test every operating system separately.

9. Julia

Julia's desktop GUI ecosystem is smaller than those of Python, Java, and C++. Choose an established toolkit or a browser interface rather than assuming a plotting package is a full GUI.

Library / approach	Assessment and recommended use
Gtk.jl (GTK)	Practical route to GTK's native accessibility support, especially for Linux desktop applications. Prefer standard GTK widgets and validate the Julia binding and GTK version used.
QML.jl / Qt Quick integration	Can provide a Qt-based UI. Qt has accessibility APIs, but validate the specific Julia binding, Qt version, QML controls, and custom components.
Browser-based Julia applications (for example, Genie/Stipple or Pluto notebooks)	Use browser semantics and the web practices in Section 2. Ensure generated controls, notebook cells, outputs, and interactive plots are keyboard and screen-reader usable.
Native host application plus Julia computation	Often a sound architecture: use an accessible host UI (for example, a web front end or native toolkit) and call Julia for computation.

Julia: avoid or restrict

- **Makie**, **Plots.jl**, **GLMakie**, and other visualization packages as the only way to present results. A chart is not a substitute for a data table, text summary, or downloadable data.
- Experimental GUI wrappers with no documented accessibility-tree support.
- Interfaces that require precise mouse interaction with a plot, notebook output, or custom canvas.

For every visualization, expose the underlying values, labels, relationships, and trends in a structured alternative. Provide keyboard-accessible controls for changing plotted variables.

10. Swift

Library / approach	Assessment and recommended use
SwiftUI	Preferred declarative approach for new Apple-platform interfaces. Standard controls have built-in accessibility. Use `accessibilityLabel`, `accessibilityValue`, `accessibilityHint`, appropriate traits, and grouping/focus APIs when needed.
UIKit	Preferred native iOS/iPadOS approach where imperative control is useful. Standard views and controls support VoiceOver; expose custom elements using UIAccessibility APIs and containers.
AppKit	Native macOS framework with accessibility support. Use standard controls and implement the NSAccessibility protocol for custom views.
WebKit / browser UI	A web front end can use semantic HTML. Test in the actual WKWebView and target macOS/iOS versions.

Swift: avoid or restrict

- **SpriteKit**, **SceneKit**, Metal-rendered screens, custom-drawn `Canvas` views, and game interfaces as the sole interface for non-game tasks or essential information.
- SwiftUI views that use decorative labels as the only control name, hide meaningful children, or combine unrelated controls into one accessibility element.
- Custom UIKit/AppKit views that do not expose accessibility elements, actions, values, and focus behavior.

Test VoiceOver, Voice Control, Switch Control, keyboard navigation on macOS, Dynamic Type, increased contrast, and Reduce Motion. A SwiftUI control that

works on iOS is not automatically verified on macOS or watchOS.

11. Rust

Rust's GUI ecosystem is evolving. Accessibility support varies considerably by toolkit and target platform. Verify current release notes and platform coverage before committing to a toolkit.

Library / approach	Assessment and recommended use
GTK 4 via gtk-rs	Strong choice for Linux desktop applications. GTK's built-in widgets expose accessible information; gtk-rs documentation covers accessible labels and descriptions.
Slint	A cross-platform toolkit with an AccessKit-based accessibility feature. Enable the feature, use semantic controls, and verify platform coverage and current version behavior.
egui / eframe	Has AccessKit integration on some platforms/releases. Treat support as platform- and version-specific; do not infer full cross-platform coverage from the existence of the integration.
Tauri with a web front end	Often a strong choice when browser-based UI is acceptable. Use semantic HTML and tested web components; Rust provides application logic while the system webview renders the UI.
Dioxus desktop/web	Can use web semantics for browser/webview targets. Verify the generated DOM, keyboard model, and assistive-technology behavior for the exact target.
AccessKit	Accessibility infrastructure for custom toolkit developers, not a turnkey GUI. Use its platform adapters and maintain a complete, accurate accessibility tree.

Rust: avoid or restrict

- **Iced** or other custom-rendered toolkits when the current release has no verified screen-reader accessibility integration for the target platforms. Re-

check current status before adoption; this is a technology-selection warning, not a permanent claim about the project.
- **Bevy**, **macroquad**, **minifb**, raw wgpu/winit, and game/graphics frameworks as the sole interface for a conventional accessible desktop application.
- Any custom widget toolkit that has keyboard focus but does not expose semantic roles, names, states, values, and events.

For custom rendering, AccessKit can bridge to platform accessibility APIs, but it does not automatically infer the meaning of the pixels. The application must create and update the semantic tree correctly.

12. Go

Go does not include a standard GUI toolkit. The best choice depends on whether the application needs a native desktop interface or can use a browser.

Library / approach	Assessment and recommended use
Wails with HTML/CSS/JavaScript front end	Often the most practical accessibility-oriented desktop approach. Use semantic web controls and test the actual platform webview (WebView2, WKWebView, or WebKitGTK as applicable).
Web application with Go backend	Strong option when desktop-native behavior is not required. Use semantic HTML, accessible components, and Section 2 testing.
GTK via Go bindings	Can access GTK's native accessibility infrastructure. Check binding maintenance, GTK version, and platform support before adoption.
Native platform APIs	Windows UI Automation, macOS accessibility APIs, or Linux AT-SPI can be used through bindings or system calls, but this is a specialist implementation path.

Go: avoid or restrict

- **Fyne**, **Gio**, **Ebiten**, and other custom-rendered toolkits as the default for screen-reader-critical applications unless current documentation and tests demonstrate support for every required platform and control type. Do not infer accessibility from cross-platform support or keyboard navigation alone.

- Old or inactive GUI bindings with unclear platform accessibility support.
- Custom canvas widgets without a parallel semantic control tree.

Fyne and similar libraries may be suitable for many applications, but accessibility must be verified at the current version and target platform. If a webview front end is selected, remember that browser accessibility still depends on semantic HTML and correct component behavior.

13. Lua

Lua is commonly embedded in applications and games, so the host toolkit often determines accessibility more than Lua itself.

Library / approach	Assessment and recommended use
wxLua / wxWidgets	Can use native controls and wxWidgets accessibility mechanisms. Verify the current binding, target OS, and screen-reader support; do not assume the full C++ toolkit's support is perfectly exposed by every Lua binding.
IUP	Provides native GUI controls on supported platforms. Accessibility behavior is platform-dependent; test the exact controls and OS combinations before using it for essential workflows.
Web UI hosted by a Lua application/server	Use semantic HTML and the practices in Section 2. This can be preferable to custom-drawn interfaces.
Host application's native UI API	When Lua is embedded in an existing product, use that host's documented accessible UI framework rather than drawing a parallel interface.

Lua: avoid or restrict

- **LÖVE (Love2D)**, **Defold**, **Solar2D**, and other game/graphics frameworks as the only interface for conventional form-based or data-entry applications. They are not substitutes for a native semantic widget toolkit.
- Custom OpenGL/canvas widgets without an accessibility bridge and equivalent keyboard interface.
- Abandoned Lua bindings whose accessibility behavior cannot be confirmed.

For games, provide accessible menus, text alternatives, remappable controls, narration or audio cues where useful, non-color-only indicators, and a way to access game state without relying exclusively on visual effects. Game accessibility is a specialized discipline; a screen-reader label alone is not sufficient.

14. R

R is primarily used for statistical computing and data analysis. For user-facing applications, prefer browser interfaces or native controls with a structured alternative to every plot.

Library / approach	Assessment and recommended use
Shiny	Recommended for interactive data applications when built with semantic HTML, accessible form controls, keyboard operation, and carefully managed status announcements. Audit Shiny modules and third-party inputs individually.
Quarto websites / documents	Suitable for accessible reports when headings, links, tables, code, and figures are authored semantically and checked in the rendered output.
R Markdown	Can produce accessible HTML documents with semantic structure. Verify the chosen output format and theme.
tcltk	R's Tcl/Tk interface can create desktop controls, but accessibility varies with the Tcl/Tk version and platform. Test before relying on it.
R + accessible web front end	R can provide analysis through Shiny or an API while a web front end supplies semantic controls.

R: avoid or restrict

- **ggplot2**, **plotly**, **htmlwidgets**, and dashboard charts as the only representation of data. Provide a data table, concise textual interpretation, and accessible controls for changing the view.
- Custom HTML widgets that use mouse-only controls or expose no semantic structure.
- RStudio Viewer or notebook output as the only access path for essential information without testing the rendered document in a browser and screen reader.

For charts, include a meaningful title, summary, data table, and clear descriptions of trends and outliers. Do not rely on color alone to distinguish series.

15. HTML and CSS focused module

HTML and CSS are not standalone GUI toolkits. They define the document and presentation layer used by browser interfaces. Use the web recommendations in Section 2.

Prefer native HTML controls and semantic elements. CSS must preserve focus visibility, reading order, zoom, reflow, contrast, and forced-colors behavior. Avoid CSS that visually reorders content in a way that conflicts with DOM reading order, hides focus, clips enlarged text, or removes meaningful content from the accessibility tree.

Do not use CSS-generated text as the only source of essential information. Do not use `display: none` or `visibility: hidden` for content that must remain available to assistive technology. Use visually-hidden techniques only when content should be available to screen readers but not visually displayed, and ensure it remains understandable.

16. SQL focused module

SQL is a query language, not a GUI toolkit. Accessibility depends on the database client, administration tool, report interface, and application built around the database.

- Choose database clients that support keyboard navigation, accessible grids, labeled controls, and screen-reader testing.
- Prefer structured query results that can be navigated as tables with column headers and predictable row/column movement.
- Provide export to accessible CSV or other structured formats where appropriate.
- Do not make a visual query-plan diagram, database schema canvas, or color-coded result grid the only way to inspect information.
- When building a database application, use the accessibility guidance for its host language and GUI framework.

A database driver or SQL engine does not make the application interface accessible.

17. Bash and PowerShell

Bash and PowerShell are shell languages, not general-purpose GUI frameworks. Their normal interfaces are command-line terminals.

- Prefer terminal emulators with screen-reader support and accessible text output.

- Ensure commands produce meaningful text, stable headings, explicit status messages, and useful error messages.
- Do not use color as the only signal. Respect terminal accessibility settings and avoid unnecessary cursor-positioning or full-screen output that disrupts screen-reader reading.
- For interactive terminal user interfaces (TUIs), provide keyboard navigation, predictable focus, text labels, and screen-reader-tested output. A TUI library's ability to draw boxes or handle keypresses does not establish accessibility.
- When a script needs a graphical interface, delegate to a supported native or web GUI framework and follow the corresponding language/platform guidance.

Avoid making a full-screen terminal UI the only route through a critical workflow unless it has been tested with the target terminal, screen reader, and operating system.

18. Testing and release gate

Accessibility is a release requirement, not a final cosmetic pass. Test the actual application, on each supported operating system, with the screen readers and input methods used by the intended audience.

Minimum manual test matrix

Environment	Screen reader / accessibility tools	Required checks
Windows	Narrator; NVDA; Windows UI Automation inspection tools	Names, roles, values, state changes, keyboard navigation, focus, dialogs, tables, high contrast, text scaling.
macOS	VoiceOver; Accessibility Inspector	Reading order, labels, groups, keyboard access, focus, Dynamic Type/text scaling where supported, contrast, dialogs.
iOS / iPadOS	VoiceOver; Switch Control; Voice Control	Swipe navigation, rotor/navigation structure, labels, hints, state, custom actions, touch alternatives, larger text.

Environment	Screen reader / accessibility tools	Required checks
Android	TalkBack; Switch Access; Accessibility Scanner (as a supplemental tool)	Traversal order, labels, state, headings, touch targets, switch/keyboard access, large font and display scaling.
Linux desktop	Orca; Accerciser; AT-SPI inspection	Accessible names/roles/states, keyboard traversal, focus, GTK/Qt controls, large text, high contrast.
Web browsers	NVDA with Firefox/Chrome; VoiceOver with Safari; keyboard-only testing	Semantic structure, forms, live regions, route changes, dialogs, custom widgets, zoom/reflow, forced colors.

Not every combination is available or equally supported. Select combinations based on the platforms actually shipped, and add combinations requested by users or required by procurement or regulation.

Automated checks (supplement, never replacement)

- Web: axe-core, Accessibility Insights, Lighthouse, and browser developer tools.
- Android: Accessibility Scanner and Espresso accessibility checks where applicable.
- Apple platforms: Xcode Accessibility Inspector and XCTest accessibility support.
- Windows: Accessibility Insights for Windows and UI Automation inspection tools.
- Linux: Accerciser and AT-SPI inspection tools.
- Cross-platform: automated tests for keyboard paths, focus behavior, and accessible names where the framework exposes them.

Automated tools detect only a subset of accessibility defects. They cannot reliably judge whether a label is meaningful, whether reading order makes sense, whether instructions are understandable, or whether a workflow is usable.

Release checklist

☐ Every control has a correct role, accessible name, state, and value.

- ☐ Every action works with keyboard-only input; focus is visible and logical.
- ☐ Screen-reader navigation and announcements are understandable and non-redundant.
- ☐ Forms expose labels, instructions, validation errors, and required status.
- ☐ Dialogs, menus, tabs, and dynamic content manage focus and announcements correctly.
- ☐ Text can be enlarged; content reflows; high-contrast/forced-colors modes remain usable.
- ☐ Color is never the only means of communicating information.
- ☐ Motion can be reduced; flashing and autoplay behavior are controlled.
- ☐ Charts, images, diagrams, and visualizations have equivalent text or structured data.
- ☐ Tests were performed on every supported platform with real assistive technology.
- ☐ Known limitations, workarounds, and unsupported combinations are documented.
- ☐ Users with disabilities have been included in usability testing before release.

19. Selection summary

For a new accessibility-critical interface, start with these defaults:

Language / area	Preferred starting point
Python	PySide6/Qt Widgets or GTK 4; browser UI where suitable
JavaScript / TypeScript	Semantic HTML with a tested framework/component system
Java	Swing or JavaFX standard controls
Kotlin	Jetpack Compose on Android; Compose Multiplatform only after target-by-target verification
C	GTK 4 or native Windows controls/UI Automation
C++	Qt 6 Widgets or GTK 4/gtkmm
C#	WPF/WinUI on Windows; .NET MAUI or Avalonia after platform testing
Julia	GTK.jl, Qt/QML integration, or a browser-based interface
Swift	SwiftUI; UIKit or AppKit where platform-specific control is needed
Rust	GTK 4, Slint with AccessKit, or Tauri with semantic web UI

Language / area	Preferred starting point
Go	Wails with semantic web UI, or GTK bindings
Lua	Host-native UI, wxLua/IUP after verification, or a semantic web UI
R	Shiny with accessible HTML controls and data alternatives
HTML/CSS	Native semantic HTML and standards-based CSS
SQL	Accessible database client and host-language application
Bash / PowerShell	Accessible terminal output; native/web GUI when a GUI is required

The safest default is not "choose the library with the longest accessibility feature list." It is: choose mature semantic controls that integrate with the target operating system, minimize custom rendering, expose complete accessible information, and prove the result through assistive-technology testing.

20. Authoritative references

Standards and cross-platform guidance:

- W3C Web Content Accessibility Guidelines (WCAG) 2.2
- W3C WAI-ARIA Overview
- WAI-ARIA Authoring Practices Guide
- MDN: HTML as a basis for accessibility
- MDN: Accessible web applications and widgets

Toolkit and platform references:

- Qt: UI accessibility
- Qt: QAccessible
- GNOME Human Interface Guidelines: Accessibility
- GTK-rs: Accessibility
- wxPython: wx.Accessible
- Oracle: Java Accessibility
- Android: Accessibility principles
- Jetpack Compose: Accessibility
- Compose Multiplatform: Accessibility
- Microsoft: WPF accessibility
- Microsoft: .NET MAUI semantic properties and accessibility
- Avalonia: Accessibility
- Apple: SwiftUI accessibility fundamentals
- Apple: Accessibility for UIKit

- Apple: Accessibility for AppKit
- AccessKit
- Rust GTK: Accessibility

Recheck framework documentation and supported platform matrices before adopting a library. Accessibility capabilities change across toolkit versions, operating systems, and assistive technologies.

Part III

Foundations (complete before any language)

Chapter 3

Foundations: Computing Concepts Before Any Language

Unit P is a language-neutral primer that every learner completes **before Unit 0 of any language track**. It teaches the vocabulary and background that all thirteen general-purpose languages share, so each language track can concentrate on its own syntax and rules.

- Unit P: Computing Foundations Primer

Recommended introductory computer science books

The following books provide complementary views of computer science. They are not substitutes for the language-specific lessons: use them to build a broader understanding of algorithms, data representation, hardware, software, and the discipline's major areas.

- Brookshear, J. Glenn, and Dennis Brylow. *Computer Science: An Overview*, 13th edition. Pearson, 2021. A broad survey of computing topics, including data representation, architecture, operating systems, networks, algorithms, programming languages, and information systems. Publisher information.
- Petzold, Charles. *Code: The Hidden Language of Computer Hardware and Software*, 2nd edition. Microsoft Press, 2022. A narrative introduction to binary representation, logic, circuits, processors, and the layers beneath software. Author's book page.

- Sedgewick, Robert, and Kevin Wayne. *Computer Science: An Interdisciplinary Approach*. Addison-Wesley, 2016. An introductory computer science text that connects programming with machine architecture, systems, and theory. Its examples use Java. Princeton booksite.
- Downey, Allen B., and Chris Mayfield. *Think Java: How to Think Like a Computer Scientist*, 2nd edition. O'Reilly Media, 2016. A free, carefully scaffolded introduction to computational problem solving and programming; useful even when Java is not the learner's eventual language. Book site and online text.

Reading guidance: start with Brookshear and Brylow for a broad survey, or Petzold for a concrete explanation of how digital computers represent and process information. Use the programming-oriented texts alongside one of the language curricula. Edition and online availability can change; consult publisher or author pages for current formats.

Lessons

- P.1 What a Computer Does
- P.2 Representing Information as Bits
- P.3 Algorithms, Pseudocode, and Tracing
- P.4 The Shared Vocabulary of Programming
- P.5 From Source Code to Running Program
- P.6 Files, Data Formats, and Reading Code Without Sight
- P.7 Self-Documenting Code, Naming, Types, and Comments

Design notes

- No diagrams, charts, or color cues. Structure is conveyed in prose, numbered lists, and traces written as sentences.
- Binary is introduced through braille cells, which are bit patterns that a learner can feel.
- Examples use the shared course datasets (student Avery, the tip calculator, the four scores, the inventory, the text sample).
- The unit follows the standard lesson template and the course AI policy.
- The accessibility practices in the unit are design intentions. They have not been tested with named screen readers or braille displays.

Recommended introductory computer science books

- Brookshear, J. Glenn, and Dennis Brylow. *Computer Science: An Overview*, 13th edition. Pearson, 2021. A broad survey of data representation, architecture, operating systems, networks, algorithms, programming

languages, and information systems. Publisher information.
- Petzold, Charles. *Code: The Hidden Language of Computer Hardware and Software*, 2nd edition. Microsoft Press, 2022. A narrative introduction to binary representation, logic, circuits, processors, and the layers beneath software. Author's book page.
- Sedgewick, Robert, and Kevin Wayne. *Computer Science: An Interdisciplinary Approach*. Addison-Wesley, 2016. Connects programming with machine architecture, systems, and theory; examples use Java. Princeton booksite.
- Downey, Allen B., and Chris Mayfield. *Think Java: How to Think Like a Computer Scientist*, 2nd edition. O'Reilly Media, 2016. A free, carefully scaffolded introduction to computational problem solving and programming. Online text.

Reading guidance: start with Brookshear and Brylow for a broad survey, or Petzold for a concrete explanation of how digital computers represent and process information. Use the programming-oriented texts alongside one of the language curricula. Check publisher or author pages for current formats.

3.1 Unit P: Computing Foundations Primer (Complete Before Any Language Track)

Overview

Compatibility: This unit is language-neutral. It contains no code that you must run. Where code-like text appears, it is *pseudocode*: plain, structured English that every language track in this course later turns into real syntax.

Every language in this course (Python, Java, JavaScript, TypeScript, Kotlin, C, C++, C#, Julia, Swift, Rust, Go, and Lua) teaches the same ideas with different punctuation. This unit teaches the ideas once, so each language track can focus on its own syntax and rules.

How this unit is written. Everything here can be read in order, line by line, with a screen reader or a braille display. There are no diagrams, no charts, and no colour cues. Where a picture would normally help, the lesson uses a spoken description, a numbered trace, or a tactile comparison such as braille cells. Lists are used instead of tables so that each item is read in sequence.

Shared data. The examples reuse the course's shared data so that you will recognise it later in every language:

- Student profile: name Avery, age 16, height 1.72 meters, favorite subject Computer Science.
- Calculator: a bill of 24.50 and a tip rate of 0.15.
- Scores: 82, 91, 76, 88.

- Inventory: Notebook (quantity 3, unit price 2.50), Pencil (quantity 10, unit price 0.50), Folder (quantity 4, unit price 1.25).
- Text sample: "A student studies programming. Programming develops problem-solving skills."

Unit objectives:

- Explain what a computer does and how hardware, software, and data relate.
- Explain how text, numbers, and decisions are represented as bits.
- Describe an algorithm in pseudocode and trace it by hand.
- Use the core vocabulary shared by all languages.
- Explain how source code becomes a running program and how to read an error message.
- Describe files and structured data formats, and know how to read code aloud.

Lessons in this unit:

- P.1 What a Computer Does
- P.2 Representing Information as Bits
- P.3 Algorithms, Pseudocode, and Tracing
- P.4 The Shared Vocabulary of Programming
- P.5 From Source Code to Running Program
- P.6 Files, Data Formats, and Reading Code Without Sight

AI policy for this course. AI tools may be used only for personal conceptual understanding and reinforcement, for example asking for another explanation of what a variable is. AI may not write, autocomplete, repair, complete, or generate course code or assessment solutions.

Lesson P.1: What a Computer Does

Reading References

- Zelle: Chapter 1, pages 1–7
- Matthes: Chapter 1
- The "Foundations of Computing" lesson (Lesson 1.1) of your chosen language track revisits this material.

Learning Objectives

- Describe the cycle of input, processing, output, and storage.
- Distinguish hardware from software and name the main hardware parts by their job.
- Explain what a program, an instruction, and an operating system are.
- Explain why a computer follows instructions literally.

Concept Explanation

A computer is a machine that follows precise instructions to change information. Almost everything a computer does fits one cycle with four parts:

1. **Input.** Information enters the computer: a key pressed, a spoken word, a file opened, a number typed into a terminal.
2. **Processing.** The computer changes the information by following instructions: adding numbers, comparing two names, sorting scores.
3. **Output.** Results leave the computer: text printed in a terminal, speech from a screen reader, a line of braille on a display, a file saved.
4. **Storage.** Information is kept for later, either briefly while a program runs or permanently in a file.

Hardware is the physical equipment: the keyboard, the processor, the memory, the storage drive, the speakers, the braille display. **Software** is the set of instructions that tells hardware what to do. The same hardware can do very different jobs depending on which software it runs.

The main hardware parts, described by their job:

- The **CPU (central processor)** carries out instructions one very small step at a time, billions of times per second.
- **Memory (RAM)** is fast, temporary working space. It holds the programs and data that are in use right now. When power is lost or a program ends, its memory contents are gone.
- **Storage** (a solid-state drive or hard drive) keeps files permanently, but is slower than memory.
- **Input and output devices** connect the computer to people: keyboards, microphones, speakers, braille displays.

The **operating system** (Windows, macOS, Linux) is software that manages the hardware and starts other programs. It also keeps track of files and folders.

A **program** is a list of instructions written so a computer can carry it out. A computer does exactly what the instructions say, no more and no less. It cannot guess what you meant. This is the most important habit of mind in programming: *say exactly what you want, in order, with nothing left unstated.*

Analogy without a picture. Imagine a very fast, very obedient helper who cannot see your intentions, only your words. If you say "make tea," the helper stops, because nothing said where the kettle is. If you say "walk to the kitchen, fill the kettle with water, switch it on, wait until it clicks," the helper succeeds. A program is the second kind of instruction.

Worked Example

Trace the tip calculator through the four-part cycle, using the shared data.

1. Input: the program receives a bill of 24.50 and a tip rate of 0.15.

2. Processing: the program multiplies 24.50 by 0.15 to get the tip, then adds the tip to the bill to get the total.
3. Output: the program prints the tip and the total as text.
4. Storage: while the program runs, the values 24.50, 0.15, the tip, and the total are held in memory. When the program ends, they are gone unless the program saved them to a file.

Expected Behavior or Output

The tip is 3.675 and the total is 28.175 before rounding. Programs usually display money with two decimal places, so they show 3.67 or 3.68 and 28.18 depending on the language's rounding rules. Later tracks explain the exact rounding behavior of each language. The point of this lesson is the four-part cycle, not the final digits.

Common Mistakes

- Believing the computer will fill in a missing step. It will not.
- Mixing up memory and storage. Memory is temporary and fast; storage is permanent and slower.
- Thinking software and hardware are interchangeable. Software is instructions; hardware is the machine that carries them out.

Activities

1. **Human computer.** Work with a partner. One person gives spoken instructions for making a sandwich or tying a knot; the other follows only what is literally said. Note every place where a missing detail caused a wrong result.
2. **Classify five things.** List five things you use daily. For each, say whether it is hardware or software, and what its input, processing, output, and storage are.
3. **Write the cycle.** For a talking clock, write one sentence each for input, processing, output, and storage.
4. **Find the unstated step.** Write six instructions for "log in to a website." Ask a partner to find one step that was assumed but never stated.

Accessibility Practice

Open a terminal with your keyboard only. Type a short command such as `echo hello` (Windows, macOS, and Linux terminals all accept it), press Enter, and listen to or read the output. Note that you gave input, the computer processed it, and output appeared in order. Do not rely on any colour or icon.

Skills Checklist

- ☐ I can name the four parts of the input, processing, output, and storage cycle.
- ☐ I can say what the CPU, memory, and storage each do.
- ☐ I can explain the difference between hardware and software.
- ☐ I can explain why a program must be precise.
- ☐ I can describe one task from my day using the four-part cycle.

Lesson P.2: Representing Information as Bits

Reading References

- Zelle: Chapter 1 (data and computation) and Chapter 3 (numbers)
- Matthes: Chapter 2 (variables and simple data types)

Learning Objectives

- Explain what a bit and a byte are.
- Convert small whole numbers between binary and decimal.
- Explain how text is stored as numbers.
- Explain why whole numbers and decimal numbers behave differently in programs.

Concept Explanation

A bit is the smallest unit of information: one of two possible values, written 0 and 1. Think of a single braille dot that is either raised or flat. A computer's circuits are built from billions of such two-state switches, so all information is stored as patterns of bits.

A byte is a group of 8 bits. Eight bits can form 2 multiplied by itself 8 times, which is 256 different patterns.

Braille as a bit pattern. A computer braille cell has 8 dots, and each dot is either raised or flat. That is exactly one byte: 8 two-state positions, 256 possible cells. A traditional literary braille cell has 6 dots, so it has 2 to the power 6, which is 64 patterns. This is not just a comparison: the Unicode braille block gives each cell a number built from its dots. Dot 1 is the lowest bit, so the cell with only dot 1 raised is the letter ”a” and has the bit pattern 00000001. The cell with dots 1 and 2 raised (”b”) is 00000011. The cell with dots 1 and 4 (”c”) is 00001001. Reading a braille cell and reading a byte are the same kind of act.

Counting in binary. In decimal, each place is worth ten times the place to its right. In binary, each place is worth *two* times the place to its right. Reading from the right, the places are worth 1, 2, 4, 8, 16, 32, 64, 128.

To read binary 1101: the places from the right are 1, 2, 4, 8. The digits from the right are 1, 0, 1, 1. Add the places that have a 1: 1 plus 4 plus 8 equals 13. So binary 1101 is decimal 13. A byte with all eight bits set to 1 is 128 plus 64 plus 32 plus 16 plus 8 plus 4 plus 2 plus 1, which is 255. The largest value in a byte is therefore 255, and the smallest is 0.

Hexadecimal is a shorthand that uses 16 symbols (0–9 then A–F) so that one symbol stands for 4 bits. Two hexadecimal symbols describe one byte. You will see it in error messages and memory addresses.

Text is numbers. A character encoding is an agreed table that assigns a number to each character. In ASCII and Unicode, the capital letter "A" is the number 65, which in a byte is 01000001. The lowercase "a" is 97, which is 01100001. Unicode extends this idea to every writing system and to braille patterns. A **string** is a sequence of such characters. The text "Avery" has 5 characters; the first is "A" and the last is "y".

Whole numbers and decimal numbers. Programs store whole numbers (**integers**) exactly, within a limited range. They store decimal numbers (**floating-point numbers**) using a binary format that can only approximate many values. For example, 0.1 cannot be written exactly with a finite number of binary places, in the same way that one third cannot be written exactly with a finite number of decimal places. As a result, adding 0.1 and 0.2 in most languages gives 0.30000000000000004, not exactly 0.3. This is correct behavior, not a bug. It is also why the tip of 24.50 times 0.15 is stored as 3.6749999999999998 in most languages, and why some languages display it as 3.67 and others as 3.68 when rounded to two places.

Integer division. In many languages, dividing two integers discards the fractional part. Seven divided by two gives 3, and the remainder is 1. The remainder operation is usually called *modulo*. Whether a language divides integers this way is one of the first things each language track will tell you.

Booleans. A **Boolean** value is true or false. It needs only one bit. Decisions in programs are made from Boolean values.

Worked Example

Show that the score 82 is stored as a number, and the name "Avery" as a sequence of numbers.

1. The score 82 is a whole number. In binary its places from the right are 1, 2, 4, 8, 16, 32, 64. 82 is 64 plus 16 plus 2. So the bit pattern is 1010010 (bits set in the 64, 16, and 2 places).
2. The name "Avery" is five characters. Each character is stored as its own number: "A" is 65, and the remaining lowercase letters have numbers starting at 97 for "a".
3. The Boolean "is the score at least 80?" is true for 82, and needs one bit.

Expected Behavior or Output

You should be able to say aloud: ”Binary one-zero-one-zero-zero-one-zero is decimal 82.” You should also be able to say that ”A” is 65 and ”a” is 97, and that these two differ because a program treats upper and lower case as different characters.

Common Mistakes

- Reading binary left to right as if each place were worth ten. Each place is worth twice the place to its right.
- Assuming ”7” the character and 7 the number are the same thing. The character ”7” has its own code; the number 7 is a different bit pattern.
- Expecting decimal arithmetic such as 0.1 plus 0.2 to be exact.
- Forgetting that upper-case and lower-case letters have different codes.

Activities

1. **Binary by hand.** Convert 5, 13, and 82 to binary, then convert 0110 and 11111111 back to decimal. Check your answers by adding place values.
2. **Braille bits.** Write the 8-bit pattern for the braille letters ”a”, ”b”, and ”c” from the lesson. Then work out the pattern for the cell with only dot 3 raised, using the rule that dot n is the bit in place n.
3. **Count the patterns.** How many different patterns can 3 bits make? Explain how you counted.
4. **Why not exact?** In two sentences, explain why a computer cannot store one third exactly in decimal digits, and how that compares with 0.1 in binary.

Accessibility Practice

Say each binary number aloud as digits, for example ”one zero one zero zero one zero,” and say the place values aloud in a steady rhythm. If you use a braille display, feel a computer braille cell and name which dots are raised before converting it to a number.

Skills Checklist

- ☐ I can say what a bit and a byte are.
- ☐ I can convert a small number between binary and decimal.
- ☐ I can explain that text is stored as numbers.
- ☐ I can explain why 0.1 plus 0.2 is not exactly 0.3.
- ☐ I can say what a Boolean value is.

Lesson P.3: Algorithms, Pseudocode, and Tracing

Reading References

- Zelle: Chapter 1 (problem solving) and Chapter 2 (software development process)
- Matthes: Chapter 1

Learning Objectives

- Define an algorithm and give an example.
- Use the three building blocks of every algorithm: sequence, selection, and repetition.
- Write simple pseudocode.
- Trace an algorithm by hand using a spoken or written trace.

Concept Explanation

An **algorithm** is a finite, ordered set of clear steps that solves a problem. A recipe, driving directions, and a process for sorting mail are all algorithms.

Every algorithm is built from three ideas.

1. **Sequence.** Steps happen one after another, in order. The order matters: adding the tip to the bill before calculating the tip gives the wrong answer.
2. **Selection.** The algorithm chooses between paths based on a condition. "If the score is at least 90, the grade is A; otherwise check the next threshold" is selection.
3. **Repetition.** Steps repeat, either a set number of times or until a condition changes. "For each score, add it to the running total" is repetition.

Pseudocode is a plain-language description of an algorithm. It is not a real programming language, so no computer can run it, but it has no punctuation rules to worry about. Here is the grade rule as pseudocode, read as a numbered list:

1. Receive a score.
2. If the score is at least 90, the grade is A.
3. Otherwise, if the score is at least 80, the grade is B.
4. Otherwise, if the score is at least 70, the grade is C.
5. Otherwise, if the score is at least 60, the grade is D.
6. Otherwise, the grade is F.
7. Report the grade.

Boundary cases are values on the edge of a rule. For the grade rule, 90, 89, 60, and 59 are boundary cases. Good testing always includes them.

Decomposition means breaking a large problem into smaller steps you can solve one at a time. **Abstraction** means giving a group of steps a name so you can use it without re-reading its details.

Tracing means executing an algorithm by hand, recording what each step does. A trace is a list of sentences, one per step, saying what changed. Tracing is the best way to find a mistake before running any code, and it works as a plain list that a screen reader reads in order.

The development cycle. Programmers repeat four stages: understand the problem, design a solution, implement it in a language, and test and refine it. Later tracks call this the software development process.

Worked Example

Algorithm: find the total and average of the shared scores 82, 91, 76, and 88.

Pseudocode:

1. Set total to 0.
2. For each score in the list, add the score to total.
3. Set average to total divided by the number of scores.
4. Report total and average.

Trace, one sentence per step:

1. Total starts at 0.
2. First score is 82. Total becomes 82.
3. Second score is 91. Total becomes 173.
4. Third score is 76. Total becomes 249.
5. Fourth score is 88. Total becomes 337.
6. There are 4 scores, so average is 337 divided by 4, which is 84.25.
7. Report total 337 and average 84.25.

Expected Behavior or Output

The report says total 337 and average 84.25. If your hand trace reaches a different result, find the first sentence where your running total differs from the one above. That is where your mistake is.

Common Mistakes

- Skipping the trace and going straight to code.
- Forgetting boundary cases: scores of exactly 90, exactly 60, and below 60.
- Putting steps in the wrong order, such as dividing by the count before the total is complete.
- Writing a step that is too vague to follow, such as "work out the grade" without the thresholds.

Activities

1. **Grade trace.** Trace the grade rule for the scores 82, 91, 76, 88, 90, 89, 60, and 59. The grades should be B, A, C, B, A, B, D, and F.

2. **Inventory value.** Write pseudocode for the total value of the shared inventory (quantity times unit price, summed). Trace it. The total should be 17.50, which is 7.50 plus 5.00 plus 5.00.
3. **Find the largest.** Write pseudocode that finds the highest score in the shared list and trace it. The answer is 91.
4. **Break it down.** Decompose "print a report card for Avery" into at least four smaller steps.

Accessibility Practice

Write traces as numbered sentences rather than tables, so they read cleanly in a screen reader or on a braille display. After writing one, read it aloud in order and check that each sentence alone tells you what changed.

Skills Checklist

☐ I can define an algorithm.
☐ I can identify sequence, selection, and repetition.
☐ I can write simple pseudocode.
☐ I can trace an algorithm by hand.
☐ I can name boundary cases for a rule.

Lesson P.4: The Shared Vocabulary of Programming

Reading References

- Zelle: Chapters 2–3
- Matthes: Chapters 2–4

Learning Objectives

- Define the core terms used in every language track.
- Tell an expression from a statement.
- Explain what a type is and why it matters.
- Read a function definition aloud and name its parts.

Concept Explanation

These terms mean the same thing in all thirteen languages. The punctuation changes; the ideas do not.

- **Source code:** the text a programmer writes.
- **Syntax:** the rules for how code must be written, including punctuation and word order. A syntax error means the rules were broken.
- **Semantics:** what the code means and does when it runs.
- **Comment:** text the computer ignores, written for people.

- **Keyword:** a word with a fixed meaning in the language, such as the words used for decisions or loops. You cannot use a keyword as your own name.
- **Identifier:** a name you choose, for a variable, function, or type. Choose descriptive names, such as `tip_rate` or `tipRate`, depending on the language's convention.
- **Value:** a piece of data, such as 24.50 or "Avery".
- **Variable:** a named place that holds a value, which can change as the program runs. *Assignment* stores a value into a variable. Read "bill = 24.50" aloud as "bill is assigned 24.50," not "bill equals 24.50."
- **Constant:** a name whose value must not change. Some languages enforce this.
- **Type:** the kind of value, which decides what operations make sense. The common types are integer, floating-point number, string, and Boolean. Adding two numbers makes sense; adding a number to a name usually does not.
- **Static and dynamic typing:** in a *statically typed* language, you or the compiler fix each variable's type before the program runs. In a *dynamically typed* language, the type belongs to the value at run time. Your language track tells you which applies.
- **Type conversion:** turning one type into another, such as the text "12" into the number 12. Input from a keyboard arrives as text, so conversion is common.
- **Expression:** a piece of code that produces a value, such as `bill * rate`.
- **Statement:** a complete instruction that performs an action, such as storing a value or printing text.
- **Operator:** a symbol that combines values: arithmetic (+, -, *, /), comparison (is equal to, is less than), and logical (and, or, not).
- **Precedence:** the order in which operators are applied. Multiplication happens before addition, so 8 + 4 * 2 is 16, while (8 + 4) * 2 is 24. Parentheses make order explicit.
- **Function:** a named, reusable block of code. It may take **parameters** (names for the inputs it expects) and may **return** a result. The actual values you supply when you call it are **arguments**.
- **Collection:** a value that holds several values. An **array** or **list** is an ordered sequence. A **map** (also called dictionary, hash table, or table) pairs keys with values, such as an item name with its quantity.
- **Index:** a position in an ordered collection. Some languages count from 0 (the first score is at index 0) and some from 1. This is the most common source of off-by-one errors, and each track states its rule.
- **Loop:** code that repeats. **Condition:** a Boolean expression that decides whether to take a path or repeat.
- **Class, object, and method:** a **class** describes a kind of thing (a Student) with its data and abilities. An **object** is one actual instance (Avery). A **method** is a function belonging to a class. Some languages call similar ideas structs, records, or types with functions attached.

- **Exception or error value:** a signal that something went wrong, such as a missing file. Languages handle it differently: some ”throw” and ”catch” exceptions, others return an error value you must check.
- **Module, package, or library:** reusable code someone else wrote, which you bring into your program.

Reading code aloud. Programs use punctuation heavily, and a screen reader may or may not announce each symbol. Use this convention, which your language tracks also follow:

- Say the symbol's name when the symbol matters: ”open parenthesis,” ”close brace,” ”colon,” ”semicolon,” ”double quote.”
- State indentation or nesting depth when it changes meaning, for example ”indent one level.” In some languages indentation is the structure; in others braces are, and indentation is only for people.
- Say ”block begins” and ”block ends” for the grouping of statements, then name the symbols your language uses.
- Always announce the *name* of a quote or bracket you are inside, so you know what must be closed.

Worked Example

Read the following description of the student profile aloud and name each term.

”Create a variable named `name` and assign it the string value Avery. Create a variable named `age` and assign it the integer 16. Create a variable named `height` and assign it the floating-point number 1.72. Create a Boolean variable named `likes_cs` and assign it true. Report the student by joining the name and age into one string.”

- `name`, `age`, `height`, and `likes_cs` are identifiers and variables.
- ”Avery”, 16, 1.72, and true are values, of types string, integer, floating-point, and Boolean.
- The line that joins name and age is a statement that contains an expression.
- Joining a string to an integer may require a type conversion in your language.

Expected Behavior or Output

You can recite each term with a one-sentence definition and point to an example in the description above. In your language track, the same description will be a four- or five-line program that prints a line about Avery.

Common Mistakes

- Reading ”=” as ”equals” in an assignment. It means ”store this value.”
- Confusing a parameter with an argument. The parameter is the name in the definition; the argument is the value you pass.

- Forgetting that input from the keyboard arrives as text and may need type conversion.
- Assuming indexes start at the same number in every language.
- Using a keyword as an identifier.

Activities

1. **Name the parts.** For the sentence "The function `tip` takes a bill and a rate and returns the tip," name the function, its parameters, and its return value.
2. **Expression or statement.** Label each as an expression or a statement: "bill times rate"; "store bill times rate into tip"; "print tip."
3. **Precedence.** Work out the value of 82 + 9 * 2, then of (82 + 9) * 2. The answers are 100 and 182.
4. **Read aloud.** Choose a two-line piece of code from your language track's first lesson later. Write out how you would say each symbol aloud, including nesting.

Accessibility Practice

Choose your screen reader's punctuation level (for example, "most" or "all") and test how it reads brackets, quotes, and operators in a short text file. Record the setting that lets you tell an open bracket from a close bracket reliably. If you use braille, check how your display shows indentation (leading spaces) so that you can read nesting depth by touch.

Skills Checklist

- ☐ I can define variable, type, expression, statement, function, and collection.
- ☐ I can tell a parameter from an argument.
- ☐ I can say how "=" is read in an assignment.
- ☐ I can explain why index rules differ between languages.
- ☐ I can read a short piece of code aloud, naming punctuation and nesting.

Lesson P.5: From Source Code to Running Program

Reading References

- Zelle: Chapter 1 (the programming environment)
- Matthes: Chapter 1 (setting up and running programs)
- The "Accessible Development Environment" lessons (Unit 0) of your chosen language track.

Learning Objectives

- Explain the difference between a compiler, an interpreter, and a virtual machine.
- Use the terminal vocabulary: command, argument, path, standard input, standard output, standard error, and exit status.
- Tell syntax, runtime, and logic errors apart.
- Read an error message from top to bottom and find the file and line.

Concept Explanation

A processor understands only machine instructions, which are bits. Programmers write source code in a human-readable language, so some program must connect the two. The languages in this course use one of three approaches. Your language track tells you which.

1. **Compiled to machine code.** A *compiler* translates the entire source into a standalone executable file *before* the program runs. You then run that file. C, C++, Rust, Go, and Swift work this way. If the compiler finds a problem, it reports a **compile-time error** and produces nothing to run.
2. **Interpreted.** An *interpreter* reads the source and carries out its instructions while the program runs. Python and Lua work this way (with some internal translation steps).
3. **Compiled to an intermediate form and run on a virtual machine.** The compiler translates source into *bytecode*, and a *virtual machine* (VM) runs it. Java, Kotlin, and C# work this way. JavaScript and TypeScript run in an engine such as Node.js; TypeScript is first translated into JavaScript. Julia compiles pieces of code as it runs.

The line between these is blurry, and you do not need to memorise it. What matters is the workflow: *write source, run a tool, read the result.*

The terminal. A terminal is a text window for typing commands. A **command** is a program's name followed by **arguments**, which are extra words that tell it what to do. For example, running a program file means typing the language's tool name and the file name. Terminal output is plain text, which makes it screen-reader and braille friendly.

- The **working directory** is the folder the terminal is currently "in." Commands look for files relative to it.
- A **path** is the address of a file or folder. An *absolute path* starts from the top of the file system; a *relative path* starts from the working directory.
- **Standard input (stdin)** is where a program reads typed input. **Standard output (stdout)** is where it prints normal results. **Standard error (stderr)** is a second channel for error messages. Terminals show both, but they can be separated.

- The **exit status** is a number a program returns when it ends. By convention, 0 means success and anything else means a problem.

Three kinds of errors.

1. **Syntax error:** the code breaks the language's writing rules, such as a missing closing parenthesis. The tool refuses to run it and names a file and line. Compilers and interpreters report it before any of your code takes effect (with interpreters, before the offending line runs).
2. **Runtime error:** the code is valid but fails while running, such as dividing by zero, opening a file that does not exist, or converting the text "abc" to a number.
3. **Logic error:** the program runs without complaint but gives the wrong answer, such as using the wrong threshold for a grade. No tool reports this, so you find it with tracing and tests.

Reading an error message. Messages follow a pattern: the *kind* of problem, the *file name*, the *line number*, and often the *offending text*. Read from the top, because the first error often causes the ones below it. In many languages the most useful line is the last line of the message; in others it is the first. Fix the first error, then run again.

Testing. A **test** compares what a function actually returns with what you expect, using values you chose in advance. The tip function given 24.50 and 0.15 should return a value close to 3.675. Note the word *close*: because of floating-point approximation, tests of decimal values compare with a small tolerance, not exact equality.

Worked Example

Imagine a terminal session described in words, as a screen reader would announce it.

1. The working directory is the folder `course`.
2. You type the command to run `hello`, naming the file in the folder. The tool begins.
3. The tool prints, on standard output, one line: `Hello, Avery!`
4. The tool ends, with exit status 0.

Now suppose you mistyped a closing quote. The tool prints on standard error: the kind of problem (a syntax error), the file name, the line number, and a short description such as "unterminated string." It ends with a non-zero exit status, and no greeting is printed.

Expected Behavior or Output

In the success case: one line of output and exit status 0. In the failure case: no program output, a message naming a file and line, and a non-zero exit status. In your own language track you will see this exact pattern.

Common Mistakes

- Running a command from the wrong working directory, so the file is "not found."
- Ignoring the file name and line number in an error message.
- Fixing many errors at once instead of the first one.
- Expecting a logic error to produce an error message.
- Forgetting that a compiled program must be rebuilt after each source change.

Activities

1. **Sort the errors.** For each case, say whether it is a syntax, runtime, or logic error: a missing closing bracket; dividing by zero; a grade rule that treats 90 as a B; reading a file that does not exist.
2. **Read a message.** Write out in plain sentences the four parts of this fictional message: "File main, line 3: unterminated string."
3. **Paths.** If the working directory is `course` and `notes.txt` is inside it, give a relative path to the file. Then explain why an absolute path to the same file is longer.
4. **Three streams.** In one sentence each, say what stdin, stdout, and stderr are for.

Accessibility Practice

Practise a keyboard-only workflow: open a terminal, move to your course folder, list its files, and read the list line by line. Run a short command and press the key your screen reader uses to review the last line of output. Learn how to jump to the *start* of the output, because error messages often begin there. Always run programs from the terminal at least once, even if your editor has a Run button, so the output stays readable in text.

Skills Checklist

- ☐ I can explain the difference between compiled, interpreted, and virtual-machine languages.
- ☐ I can explain working directory, path, stdin, stdout, stderr, and exit status.
- ☐ I can tell syntax, runtime, and logic errors apart.
- ☐ I can find the file and line in an error message.
- ☐ I can explain why decimal tests use a tolerance.

Lesson P.6: Files, Data Formats, and Reading Code Without Sight

Reading References

- Matthes: Chapter 10 (files and exceptions)
- Zelle: Chapter 5 (file processing)
- The "Files and Exceptions" lessons (Unit 8) of your chosen language track.

Learning Objectives

- Explain what a file is and why programs read and write them.
- Describe plain text, CSV, and JSON formats.
- Explain why input can be invalid and why programs must check it.
- Preview how the units of this course fit together.

Concept Explanation

A file is a named collection of data kept in storage. A **text file** holds characters (the encodings from Lesson P.2). A **binary file** holds arbitrary bytes, such as a compiled program or a picture. Programs *read* from files to get data and *write* to files to keep data after the program ends. Opening a file can fail: the file may not exist, you may lack permission, or the data may be malformed. A well-written program expects this and reports a clear message.

Resource lifetime. Opening a file borrows something from the operating system. A program must *close* it when finished. Each language track shows its own idiom so that a file is closed even when an error happens.

Plain text is lines of characters. A program reads it one line at a time.

CSV (comma-separated values) stores a table as text: one row per line, with values separated by commas, and usually a first line that names the columns. The shared student file uses the columns `student_id`, `first_name`, `last_name`, `score`, and `active`. A row might be read aloud as: "student id 1, first name Avery, last name Student, score 91, active true." Note that simply splitting a line at each comma breaks when a value itself contains a comma, so real programs use a proper CSV reader.

JSON (JavaScript Object Notation) stores structured data as text, using braces for records (key and value pairs), square brackets for lists, and quotation marks around text. It is readable by people and by nearly every language. A record for the shared to-do data has the fields `id`, `title`, `description`, `done`, and `created_at`. Read aloud, a JSON record is "open brace, key `id` colon value 1 comma ..., close brace." Saving a record as JSON is called *serialising*; loading it is *deserialising*.

Validating input. Data from a keyboard or a file cannot be trusted to be correct. A program should check for empty values, wrong types, and out-of-

range numbers, and explain the problem in clear words. For example: "Enter a whole number from 0 to 100." Messages should say what was wrong and what to do, never rely on colour, and read well aloud.

How the course fits together. After this unit, each language track follows the same order:

- Unit 0: set up an accessible environment.
- Unit 1: the first program and variables.
- Unit 2: numbers, input, and output.
- Unit 3: decisions, lists, loops, and strings.
- Unit 4: lists and dictionaries.
- Unit 5: user input and while loops.
- Unit 6: functions.
- Unit 7: classes and object-oriented design.
- Unit 8: files and exceptions.
- Unit 9: testing and data analysis.
- Unit 10: text-based projects (a text adventure and a command-line to-do tool).
- Unit 11: a capstone personal data manager.

Every Unit 1–11 lesson in a track uses the same sections: Reading References, Learning Objectives, Concept Explanation, Worked Example, Expected Behavior or Output, Common Mistakes, Activities, Accessibility Practice, and Skills Checklist.

Working the accessible way, in every unit.

- Use the keyboard first; do not depend on a mouse.
- Run programs from the terminal so output and errors are plain text.
- Print descriptive prompts and error messages in words.
- When a program would produce a chart or picture, produce a text description or a list of numbers as well.
- Read code with your own punctuation settings, and say symbol names aloud when they matter.

Worked Example

Describe the saved form of one inventory item in two formats.

Shared data: Notebook, quantity 3, unit price 2.50.

- **As a line of text:** "Notebook, 3, 2.50."
- **As a CSV pair of lines:** the first line names the columns, `name`, `quantity`, and `unit_price`, separated by commas; the second line holds `Notebook`, `3`, and `2.50`, separated by commas.
- **As JSON, read aloud:** "Open brace. Key name, colon, string Notebook, comma. Key quantity, colon, number 3, comma. Key unit price, colon, number 2.5. Close brace."

Note that JSON stores 2.50 as the number 2.5, because trailing zeros carry no meaning in a number.

Expected Behavior or Output

Each of the three forms contains the same three facts: the name, the quantity, and the price. A program that reads any of them should recover Notebook, 3, and 2.5. A program that reads a file which does not exist should produce a clear message and not a crash.

Common Mistakes

- Assuming a file exists and is well-formed.
- Forgetting to close a file, or not using the language's safe pattern for it.
- Splitting CSV lines at commas when values may contain commas.
- Treating all input as valid.
- Writing an error message that says only "invalid" without saying why.

Activities

1. **Same facts, three forms.** Write the Pencil item (quantity 10, unit price 0.50) as a plain text line, a CSV pair of lines, and JSON read aloud.
2. **Predict the failure.** List three different ways opening `notes.txt` could fail, and write the message a user should see for each.
3. **Read the plan.** In your own words, say what Unit 7 and Unit 8 will teach and how they connect to the Student and to-do examples.
4. **Write an error message.** Write an accessible message for a user who enters "abc" where a whole number from 0 to 100 is required.

Accessibility Practice

Create a small text file with three lines, save it, and read it back in the terminal with a command that prints a file (your operating system's command is `cat` on macOS and Linux and `type` on Windows). Compare it with what you see in your editor. Confirm that both are readable line by line.

Skills Checklist

☐ I can explain what a file is and that opening one can fail.
☐ I can describe plain text, CSV, and JSON.
☐ I can write a clear, accessible input error message.
☐ I can state the order of units in my language track.
☐ I can describe the accessible workflow used throughout the course.

Lesson P.7: Self-Documenting Code, Naming, Types, and Comments

Learning Objectives

By the end of this lesson, learners should be able to:

- Explain what self-documenting code is and why code must communicate intent to another person and to their future self.
- Choose clear, specific names for variables, constants, functions, classes, and files.
- Break complex work into small, purpose-named functions with clear inputs, outputs, and side effects.
- Use type annotations, signatures, contracts, and documentation features when a language supports them.
- Distinguish useful comments from comments that merely repeat the code.
- Improve an unclear code example without changing its behavior.

Why readability is a correctness and maintenance concern

Programs are read many more times than they are written. A person may need to understand a program months or years after its author last worked on it. That person may be a teammate—or the original author returning after forgetting the details.

Readable code reduces the amount of context a reader must reconstruct. It makes defects easier to find, changes safer to make, and reviews more effective. The goal is not to make every line verbose. The goal is to make the purpose, assumptions, and important decisions apparent with the least unnecessary effort.

A useful rule is: **write code for the next reader, including the future version of yourself.** Do not assume that you will remember why a calculation, condition, workaround, or unusual choice was made.

Names communicate purpose

Choose names that describe meaning rather than merely recording a type, position, or implementation detail.

Prefer:

- `student_score` over `x` when the value represents a student's score.
- `active_student_count` over `n` when the value counts active students.
- `calculate_average_score` over `do_stuff` for a function that calculates an average.
- `is_file_readable` over `check` for a function that returns whether a file can be read.
- `MAX_RETRY_ATTEMPTS` over `m` for a constant that defines a retry limit.

Names should be precise and honest. A variable called `total_price` should not hold a quantity; a function called `save_record` should not also send an email unless that behavior is clear in its name and documentation.

Follow the naming conventions of the language and project. Examples include snake_case in Python and many C projects, camelCase in JavaScript, TypeScript, Java, and Kotlin, PascalCase for many class and type names in C# and other ecosystems, and the conventions of the relevant standard library. Consistency within a project is more valuable than imposing a personal style that conflicts with its conventions.

Avoid both extremes:

- Names that are too short to convey meaning, except for conventional short-lived loop indices or established mathematical notation.
- Names that repeat obvious context or become entire sentences. A name should be informative, not a paragraph.

Use domain vocabulary consistently. If the project calls a person a `student`, do not alternate among `learner`, `user`, `record`, and `person` for the same concept without a reason.

Functions should have clear responsibilities

A function name should describe the action or result. Its parameters should make the required inputs clear, and its return value should communicate the result. Prefer a function with one coherent responsibility over a long function that validates input, calculates a result, writes a file, prints a report, and changes unrelated state.

For example, the following pseudocode hides its purpose:

```
function process(a, b):
    return a * b * 0.15
```

A clearer version communicates the domain meaning:

```
function calculate_sales_tax(item_price, tax_rate):
    return item_price * tax_rate
```

The second version also avoids hiding the tax rate inside the function. A caller can see which rate is being used and tests can supply different rates.

A function should make its important behavior predictable. Be especially clear about whether it changes data, writes files, sends network requests, prints output, or modifies shared state. These are side effects; they should not be surprising.

Use types and contracts to make assumptions visible

When a language supports static types, use them to express the kinds of values a function expects and returns. Types help readers, editors, compilers, and static-analysis tools detect mistakes before a program runs.

For example, a TypeScript function can make its contract explicit:

```
function calculateAverage(scores: number[]): number {
    if (scores.length === 0) {
        throw new Error("At least one score is required.");
    }

    const total = scores.reduce((sum, score) => sum + score, 0);
    return total / scores.length;
}
```

The parameter type says that `scores` is an array of numbers; the return type says that the function produces a number. The empty-list check documents and enforces an important precondition.

Python is dynamically typed, but it supports optional type annotations:

```
def calculate_average(scores: list[float]) -> float:
    """Return the arithmetic mean of a non-empty list of scores."""
    if not scores:
        raise ValueError("At least one score is required.")

    return sum(scores) / len(scores)
```

Python does not enforce these annotations at runtime by default. Tools such as type checkers and IDEs can use them to find inconsistencies. Runtime validation is still necessary when data comes from files, users, networks, or other untrusted sources.

Other languages express contracts in different ways:

- Java, Kotlin, C#, Swift, Go, Rust, C, and C++ use declarations and type systems to communicate parameter, return, field, and variable types. Use the language's normal type system rather than obscuring it with unnecessary conversions or overly broad types.
- JavaScript does not have built-in static type annotations in ordinary JavaScript syntax. Use clear object shapes, JSDoc annotations where useful, runtime validation for external data, or TypeScript when static checking is appropriate.
- Lua and R are dynamically typed. Use clear names, documentation, assertions or validation at boundaries, and any maintained static-analysis or annotation conventions adopted by the project.
- SQL schemas communicate data types, nullability, keys, and constraints. Give columns meaningful names and declare constraints that express valid

data.
- Bash and PowerShell benefit from descriptive parameter names, parameter types or validation attributes where supported, and explicit checks for required inputs.

Do not add annotations merely to decorate code. They should make a real contract clearer. A type annotation is not a substitute for validating untrusted input, and a declared type does not explain business rules such as allowed ranges or relationships between values.

Comments: explain what the code cannot say clearly

Self-documenting code does not mean comment-free code. It means that names and structure explain the straightforward parts, leaving comments to explain information that code alone cannot communicate well.

Useful comments explain:

- **Why:** the reason for a non-obvious decision, trade-off, workaround, or constraint.
- **Assumptions:** conditions the code relies on but cannot fully enforce.
- **Risks:** important edge cases, security concerns, or data-integrity requirements.
- **Intent:** why a particular algorithm or unusual approach was chosen instead of an obvious alternative.
- **Public behavior:** how a reusable function, module, or API should be called, including meaningful preconditions, return values, exceptions, and side effects.

For example, this comment adds little:

```
# Add one to count.
count += 1
```

The code already says exactly that. A more useful comment records the reason for a less obvious rule:

```
# The source file numbers its first record as 1; convert to
# Python's zero-based index before selecting the row.
row_index = source_record_number - 1
```

Do not write comments that merely paraphrase each line. They become stale when code changes and can mislead the next reader. If a comment is needed to explain a complicated block, first consider whether clearer names, a smaller function, or a simpler algorithm would make the code easier to understand.

Keep comments accurate. Update or remove them when behavior changes. A wrong comment is worse than no comment because it teaches the reader an incorrect explanation.

Documentation for reusable code

Use the documentation conventions of the language and project:

- Python: docstrings for public modules, classes, and functions; type annotations for parameters and return values.
- Java: Javadoc for public APIs and meaningful method contracts.
- JavaScript and TypeScript: JSDoc where it adds information beyond the code and TypeScript signatures.
- C and C++: project-standard API comments, often in header files, with clear ownership and lifetime expectations where relevant.
- C#: XML documentation comments for public APIs.
- Kotlin: KDoc for public declarations.
- Swift: documentation comments for public APIs, including parameter and return descriptions where useful.
- Go: doc comments for exported identifiers, following Go conventions.
- Rust: rustdoc comments for public items, including examples when they clarify usage.
- Julia: docstrings for functions and types.
- Lua: project-consistent API comments and examples.
- R: roxygen2 or the documentation convention used by the package.
- SQL: schema comments and migration notes where supported, plus clear query and report documentation.
- Bash and PowerShell: comments and help metadata for scripts and reusable functions; PowerShell functions can use comment-based help.

Document public contracts and surprising behavior. Do not write a paragraph for every obvious private helper. Documentation should answer questions a caller cannot reliably infer from the signature: what the function does, what inputs mean, what it returns, what it changes, and what errors or limitations matter.

Worked Example: make a future change understandable

Consider this deliberately unclear Python code:

```
def f(a, b):
    c = 0
    for x in a:
        if x >= b:
            c += x
    return c
```

A reader must infer what `a`, `b`, `c`, and `x` represent, what the comparison means, and why only some values are included.

A more maintainable version is:

```
def sum_scores_at_or_above(
```

```
    scores: list[int],
    minimum_score: int
) -> int:
    """Return the sum of scores meeting the inclusive minimum."""
    qualifying_score_total = 0

    for score in scores:
        if score >= minimum_score:
            qualifying_score_total += score

    return qualifying_score_total
```

The function name, parameter names, local variable, type annotations, and docstring communicate the intent. The comparison is inclusive, and the docstring records that contract. A separate comment is unnecessary because the code already expresses the rule clearly.

If a future requirement changes the threshold rule, a maintainer can find the comparison quickly and understand what it means. Good names and structure reduce the chance of changing the wrong behavior.

Practical rules for code you will revisit in six months

Before considering a piece of code finished, ask:

1. Can a reader understand the purpose of each important variable without tracing every assignment?
2. Does each function name describe its behavior accurately?
3. Are parameters and return values clear from names and types?
4. Are important assumptions, valid ranges, and failure cases explicit?
5. Are side effects—file access, output, network calls, and state changes—obvious?
6. Does each comment explain a reason, constraint, or contract rather than repeat syntax?
7. Would a comment become misleading if the implementation changed?
8. Can a complicated block be simplified or given a meaningful helper function?
9. Are names and formatting consistent with the language and project?
10. Could you safely make a small change after not seeing this code for six months?

Readable code is not necessarily the code with the most comments or the longest names. It is code whose intent and constraints are easy to recover.

Common Mistakes

- Using single-letter names for values whose meaning is not conventional or obvious.

- Naming a function after its implementation rather than its purpose.
- Using vague verbs such as `process`, `handle`, or `do_work` when a more precise name is possible.
- Writing comments that repeat the next line of code.
- Leaving comments in place after the implementation has changed.
- Treating type annotations as runtime validation of external data.
- Giving a function several unrelated responsibilities and relying on comments to explain the complexity.
- Omitting important side effects, assumptions, units, or boundary conditions from public API documentation.
- Renaming code inconsistently so the same concept has several names in one project.

Activities

1. **Name the intent.** Rewrite these names for a program that manages student scores: `x`, `do_it`, `temp2`, `n`, and `flag`. State what each value or operation represents.
2. **Comment audit.** Write three lines of code with comments that simply repeat them. Remove the comments and improve the names instead. Then write one comment that explains a non-obvious reason or constraint.
3. **Function contract.** Design a function that converts a temperature from Celsius to Fahrenheit. Write a clear function name, parameter name, return type or type annotation where supported, and a one-sentence description of its behavior.
4. **Six-month test.** Select an earlier exercise. Without running it first, read the code and write down what each function and important variable does. Revise unclear names and add only comments that explain intent or assumptions.
5. **Language comparison.** In your chosen language, identify how a function declares parameter types and return types, and how public functions or modules are documented. If the language is dynamically typed, identify the validation and documentation conventions used instead.

Accessibility Practice

Read one of your programs from top to bottom using your normal screen-reader or text-navigation workflow. Check that names are pronounceable and distinguishable when spoken, that abbreviations are understandable, and that comments do not create unnecessary noise. Avoid names that differ only by capitalization or a single easily confused character. Use punctuation and underscores consistently so names are easier to navigate and dictate. Ask another learner to explain the program using only its names, signatures, and documentation before consulting the implementation.

Skills Checklist

- ☐ I can explain why code must be understandable to a future reader and to my future self.
- ☐ I can choose meaningful names for variables, functions, classes, and constants.
- ☐ I can describe the purpose and side effects of a function.
- ☐ I can use the type system or annotation conventions of my language to clarify contracts.
- ☐ I can distinguish useful comments from comments that merely restate the code.
- ☐ I can document public behavior, assumptions, and important constraints.
- ☐ I can review an earlier program and improve its readability without changing its behavior.

Unit P Glossary (Quick Reference)

- **Algorithm:** an ordered, finite set of clear steps that solves a problem.
- **Argument:** the actual value supplied to a function when it is called.
- **Bit:** one two-state value, 0 or 1.
- **Boolean:** a value that is true or false.
- **Byte:** 8 bits.
- **Compiler:** a tool that translates source code before it runs.
- **Constant:** a named value that must not change.
- **Exit status:** a number a program returns when it ends; 0 means success by convention.
- **Expression:** code that produces a value.
- **Function:** a named, reusable block of code.
- **Index:** a position in an ordered collection.
- **Interpreter:** a tool that carries out source code while the program runs.
- **Self-documenting code:** code whose names, structure, types, and interfaces communicate its intent with minimal need for explanatory comments.
- **Side effect:** an observable change made by a function beyond returning a value, such as writing a file, printing output, or modifying shared state.
- **Type annotation:** optional or required syntax that states the expected type of a variable, parameter, or return value.
- **Parameter:** a name in a function's definition for an expected input.
- **Pseudocode:** structured plain-language description of an algorithm.
- **Statement:** a complete instruction that performs an action.
- **Syntax:** the rules for how code is written.
- **Type:** the kind of value, such as integer, floating-point number, string, or Boolean.
- **Variable:** a named place that holds a value that can change.

Verification Notes

- The arithmetic in this unit was checked by computation: scores total 337 and average 84.25; inventory value 17.50; "A" is 65 (01000001) and "a" is 97 (01100001); binary 1101 is 13; 0.1 plus 0.2 gives 0.30000000000000004; 24.50 times 0.15 is stored as 3.6749999999999998 in standard double-precision arithmetic; braille cells for "a", "b", and "c" correspond to the 8-bit patterns 00000001, 00000011, and 00001001.
- Statements about which languages are compiled, interpreted, or virtual-machine based are simplified; implementations vary.
- This unit has not been tested with screen readers or braille displays. The accessibility practices are design intentions, not test results. Test them with named access technologies before claiming compliance.

Part IV

C

Chapter 4

C Curriculum

Before you start: complete Unit P: Computing Foundations Primer. It explains the language-neutral vocabulary and concepts used in every language track, using non-visual examples.C is a compiled procedural language for operating systems, embedded software, libraries, and performance-sensitive programs.

Course structure

A complete introductory sequence aligned to Python Units 0–11: setup; foundations; numeric computation; control flow; data structures; input and iteration; functions; object-oriented design; files and exceptions; testing and data analysis; projects; and capstone. The same core problem contexts recur across languages so students focus on syntax, types, runtime behavior, libraries, and idioms.

Environment

Install GCC or Clang (C17); compile with cc -std=c17 -Wall -Wextra -Wpedantic -g main.c -o main; run ./main.

Complete the shared IDE setup lesson and terminal prerequisite. The course supports keyboard-first work with screen readers, braille displays, magnification, or other access technology.

- Complete lessons
- Topic index

Shared Python-aligned examples

The C lessons reuse the canonical student profile (Avery, age 16, height 1.72 m), $24.50 bill with a 15% tip, grade boundaries, scores (82, 91, 76, 88), inventory

records (Notebook, Pencil, Folder), coordinates (3, 4), the shared text sample, CSV field names, and the capstone record fields. C-specific adaptations are explicit: arrays and structs replace Python lists and dictionaries; error codes and return values replace exceptions; an accessible terminal interface replaces the Python-only NiceGUI implementation; and JSON persistence requires a documented external C library.

The lesson examples are written for C17 and should be compiled with warnings enabled. External-library examples must document their dependency and linker settings. No claim is made that every example has been compiled on every supported toolchain; verify examples with the compiler and platform used by the course.

GUI, visualization, and workbench accessibility

See the shared GUI, visualization, and workbench accessibility reference for screen-reader support by toolkit, platform-specific activation settings, recommended alternatives, and interfaces that must not be required without an equivalent accessible route.

AI Policy

AI may be used only for personal conceptual understanding and reinforcement. It must not write, complete, autocomplete, repair, or generate course code or assessment solutions.

Recommended introductory books

- King, K. N. *C Programming: A Modern Approach*, 2nd edition. W. W. Norton, 2008. A thorough first-course textbook with extensive exercises and careful explanations of C's core features. It teaches C99; use the course's C17 notes and current compiler documentation for later-standard differences.
- Perry, Greg, and Dean Miller. *C Programming Absolute Beginner's Guide*, 3rd edition. Que Publishing, 2014. A task-oriented introduction covering program structure, expressions, functions, arrays, pointers, and input/output. It targets C11. Publisher page.

Suggested use: King's text is a strong full textbook companion; Perry and Miller provide a more task-oriented alternative. C has evolved since these books were published, so follow the course's C17 baseline and consult current compiler and language references for standard-specific details.

4.1 C Lessons

The curriculum is organized as one Markdown file per unit.

- accessible environment and toolchain
- foundations of computing
- numeric computation
- control flow
- data structures
- input and iteration
- functions
- structs and object models
- files errors and persistence
- testing and data analysis
- applied projects
- capstone personal data manager

4.2 Unit 0: Accessible Environment and Toolchain

Course overview

Before you start: complete Unit P: Computing Foundations Primer. It explains the language-neutral vocabulary and concepts used in every language track, using non-visual examples.

Course Overview

C is a compiled procedural language for operating systems, embedded software, libraries, and performance-sensitive programs.

The repository includes an automated C17 compiler-check workflow for every fenced C example in this manuscript. It runs GCC and Clang with `-Wall -Wextra -Wpedantic -Werror`, links each complete program, and syntax-checks declaration/function fragments. This verifies compilation and linking, but does not execute every example or validate interactive behavior.

This course intentionally parallels the original Python curriculum. It reuses problem contexts—greetings, profiles, a calculator, grade decisions, score collections, inventory, functions, text files, tests, text analysis, CSV, a to-do list, and a personal-data manager—so the principal new learning target is the C language. The syntax and implementation remain idiomatic, not a mechanical translation.

Unit 0: Accessible Environment and Toolchain

Overview

Install and verify a C17 compiler, create a project workspace, compile and run a source file from the terminal, and configure an editor workflow that can be operated with the learner's access technology. C has no standard-library virtual environment; external libraries are managed through documented system packages, source dependencies, include paths, and linker options.

Lesson 0.1: Install and Verify C

Reading References

- GCC or Clang installation and user documentation.
- C language version and compiler options reference.

Learning Objectives

- Identify a C compiler and verify its version.
- Compile a minimal C17 program with warnings enabled.
- Distinguish compiler diagnostics from program output.

Concept Explanation C is compiled: the compiler translates source files into an executable before the program runs. Install GCC or Clang using the supported package manager or official installer for the operating system. The command `cc --version` or the compiler-specific version command verifies that a compiler is available. The `-std=c17` option selects the C17 language mode; `-Wall -Wextra -Wpedantic` enables useful diagnostics. A compiler warning is not the same as runtime output, and a successful compile does not prove that the program's logic is correct.

Worked Example Save the following as `hello.c`:

```c
#include <stdio.h>
int main(void)
{
    puts("C toolchain ready.");
    return 0;
}
```

Compile with `cc -std=c17 -Wall -Wextra -Wpedantic -g hello.c -o hello`. Run with `./hello` on Unix-like systems or `hello.exe` on Windows. Expected program output: `C toolchain ready.`

Expected Behavior or Output

The compile command prints nothing when the source has no warnings. Running the program prints one line:

```
C toolchain ready.
```

This output was observed with gcc 13.3 on Linux.

Common Mistakes

- Installing an editor extension but not installing an actual compiler.
- Using a compiler command that is not on the system PATH.
- Treating warnings, compiler errors, and program output as interchangeable.

Activities

1. Record the compiler name and version.
2. Compile the example with all listed warning flags and save the exact command.
3. Introduce a missing semicolon in a disposable copy, compile, and locate the diagnostic.
4. Restore the source and confirm the expected output.

Accessibility Practice Use keyboard commands to open the terminal, type the compile command, and move through compiler diagnostics by line. Confirm that the terminal announces the command, exit status, and program output in sequence.

Skills Checklist

☐ I can identify whether GCC or Clang is installed.
☐ I can compile a C17 source file with warnings enabled.
☐ I can distinguish compiler diagnostics from runtime output.
☐ I can run the resulting executable from the terminal.

Lesson 0.2: Workspace, Source Files, and First Run

Reading References

- Compiler documentation for source files and output files.
- Operating-system documentation for current working directories and file paths.

Learning Objectives

- Create a project folder and a C source file.
- Compile from the project root and run the executable.
- Explain the difference between source, executable, and working directory.

Concept Explanation A source file such as `main.c` contains human-readable C code. The compiler reads the source and creates an executable such as `main` or `main.exe`. The current working directory is the directory from which the process is run; relative file paths are resolved from that directory, not automatically from the source file's location. Keep source files, headers, test files, data files, and generated binaries organized so that builds and file operations are predictable.

Worked Example Create a folder named `c-first-run`, save `main.c` inside it, and use:

```
cd c-first-run
cc -std=c17 -Wall -Wextra -Wpedantic main.c -o main
./main
```

On Windows, run `main.exe`. The executable prints the output from Lesson 0.1. The `cd` command changes the working directory before compilation and execution.

Expected Behavior or Output

The compile step prints nothing when the source is correct, and `./main` prints the output of Lesson 0.1:

```
C toolchain ready.
```

Common Mistakes

- Running the compiler from a directory where the source file is not located.
- Confusing the source file name with the executable name.
- Assuming a relative data-file path is resolved relative to the source file.

Activities

1. Create the project folder and source file using only keyboard navigation.
2. Compile and run from the project root.
3. Run the executable from a different directory and observe how relative paths behave.
4. Rename the source file and update the compile command accordingly.

Accessibility Practice Verify that the editor exposes the file tree, current file name, line numbers, and terminal focus. Navigate to the compiler command and output without using a mouse.

Skills Checklist

☐ I can create and navigate to a C project folder.
☐ I can identify source files and generated executables.
☐ I can compile and run from the project root.

☐ I can explain how the current working directory affects relative paths.

Lesson 0.3: Editor and Accessibility Workflow

Reading References

- Shared IDE setup lesson.
- Documentation for the selected editor and C language extension.

Learning Objectives

- Configure an editor for C source files.
- Navigate code, diagnostics, and terminal output with keyboard commands.
- Distinguish editor assistance from compiler validation.

Concept Explanation A text editor edits source code; an integrated development environment may combine editing, project navigation, debugging, build commands, and language tooling. VS Code, VSCodium, and Theia can provide C language support through extensions, but the compiler remains a separate toolchain component. Configure the editor to expose diagnostics as text, preserve indentation, announce punctuation, and allow reliable keyboard movement between source, problems, and terminal panels. Extensions may provide completion and navigation, but the compiler determines whether the program is valid C for the selected language mode and target.

Worked Example

1. Open the `c-first-run` folder in the selected editor.
2. Open `main.c` and navigate by line and word.
3. Introduce a missing semicolon in a temporary copy.
4. Run the C17 compile command in the integrated terminal and read the diagnostic.
5. Restore the source and compile again.

Expected Behavior or Output

the editor allows source navigation, the compiler reports the deliberate syntax error, and the restored file compiles and runs.

Common Mistakes

- Assuming an editor extension installs GCC or Clang automatically.
- Relying on syntax coloring as proof that code compiles.
- Configuring a completion engine without checking whether its diagnostics are accessible.

Activities

1. Test keyboard-only navigation among the file tree, source editor, diagnostics, and terminal.
2. Confirm that indentation and punctuation are announced in a usable way.
3. Compare an editor warning with the compiler's diagnostic for the same source.
4. Record which editor features work with the learner's screen reader, braille display, or magnification setup.

Accessibility Practice Use the learner's actual access technology and operating system. Test code navigation, punctuation verbosity, diagnostic announcements, terminal focus, and file-tree navigation. Record observed limitations rather than assuming every extension is accessible.

Skills Checklist

☐ I can open and navigate a C project without relying on a mouse.
☐ I can reach compiler diagnostics and terminal output.
☐ I can explain the difference between editor assistance and compiler validation.
☐ I have recorded which accessibility features were actually tested.

Lesson 0.4: Project Environments, Dependencies, and Terminal Usage

Reading References

- Compiler documentation for include paths and linker options.
- Operating-system package manager documentation.
- Shared terminal prerequisite.

Learning Objectives

- Identify the project root and run commands from it.
- Explain how C standard-library headers differ from external dependencies.
- Recognize include paths, library paths, and linker flags.
- Keep source, headers, build products, and project data organized.

Concept Explanation C does not use a Python-style virtual environment for its standard library. The compiler toolchain provides standard headers and libraries. When a project depends on an external library, install or vendor that dependency using a documented method, then configure its header search path and linker arguments. Keep generated object files and executables separate from source. Record compiler version, dependency versions, include options, and linker options so another learner can reproduce the build.

Worked Example A simple project may contain `src/main.c`, `include/`, `data/`, and `build/`. From the project root, compile with:

```
cc -std=c17 -Wall -Wextra -Wpedantic -Iinclude src/main.c -o build/app
```

The `-Iinclude` option adds a project header directory. External libraries may require additional `-L` and `-l` options. Do not add these options unless the selected dependency requires them.

Expected Behavior or Output

The compile command prints nothing when it succeeds and creates the program `build/app`. If the header directory is missing or misspelled, the compiler reports an error that names the header file it could not find.

Common Mistakes

- Assuming C has an automatic virtual environment or dependency resolver.
- Forgetting to document the external library version and installation method.
- Confusing compiler include paths (`-I`) with library search paths (`-L`) and linker names (`-l`).
- Committing generated executables or build output as if they were source files.

Activities

1. Create the `src`, `include`, `data`, and `build` folders.
2. Compile a source file from the project root using the `-Iinclude` option.
3. Record the compiler version and exact build command in a README.
4. Explain what additional information would be needed to build a project using an external JSON library.

Accessibility Practice Use terminal commands to identify the current directory, list project files, change directories, compile, and run. Ensure that folder names, command output, and errors are announced sequentially by the chosen access technology.

Skills Checklist

- ☐ I can identify the project root and run commands from it.
- ☐ I can distinguish standard-library headers from external dependencies.
- ☐ I can explain the purpose of `-I`, `-L`, and `-l`.
- ☐ I can document a reproducible compiler command and dependency setup.

4.3 Unit 1: Foundations of Computing

Overview

This unit covers computer science, hardware/software, algorithms, first program, variables, primitive types. Follow the same problem progression as Python so that the main comparison is language syntax and semantics. Use the worked examples as starting points, then complete the activities independently.

Lesson 1.1: Computer science and problem solving

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Computer science is the study of representing problems and designing precise algorithms. Hardware includes the CPU, RAM, keyboard, and storage; software is the instructions and data executed by the machine. C is widely used for operating systems, embedded systems, libraries, and performance-sensitive programs. An algorithm must state inputs, processing steps, and outputs unambiguously.

Worked Example

Plan the canonical tip-calculator problem without coding: read a bill amount, calculate 15 percent of it, add the tip to the bill, and display both amounts. For a bill of $24.50, the unrounded tip is $3.675 and the unrounded total is $28.175. Displaying two decimal places with `printf("%.2f")` yields $3.67 and $28.18, because the stored value of the tip is 3.67499999999999982.

Expected Behavior or Output

No program is run in this lesson. The expected result is a written plan with the four steps in order. A correct hand trace with the shared values gives a tip of 3.675 and a total of 28.175; later lessons show that C displays these with two decimals as 3.67 and 28.18.

Common Mistakes

- Confusing an algorithm's logical steps with C syntax or compiler behavior.
- Omitting an input constraint such as nonnegative bill amount or percentage range.
- Rounding the tip before adding it to the bill rather than formatting only the displayed result.

Activities

1. Write the tip-calculator algorithm as input, processing, and output steps, then calculate the unrounded values for a $24.50 bill.
2. Identify the input validation rule for a negative bill and an invalid tip rate.
3. Explain why formatting a monetary result to two decimals is different from rounding every intermediate calculation.
4. Write an acceptance-test note for "Computer science and problem solving": state the exact input or source change and the expected observable result.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by "Computer science and problem solving" without relying on Python terminology.
- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

Lesson 1.2: Your first program

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.

- Modify and test the example independently.

Concept Explanation

A C program begins at `main`. `#include <stdio.h>` makes standard I/O declarations available. `puts` writes a string and newline. Braces delimit the function body and semicolons terminate statements. The compiler translates this source into an executable.

Worked Example

```c
#include <stdio.h>
int main(void)
{
    puts("Hello, World!");
    return 0;
}
```

Expected output: `Hello, World!` followed by a newline. Compile with `cc -std=c17 -Wall -Wextra -Wpedantic hello.c -o hello`.

Expected Behavior or Output

Output observed when the worked example was run (gcc 13.3, C17; the text of the output does not depend on the operating system):

```
Hello, World!
```

Common Mistakes

- Using `void main` instead of the standard `int main(void)` form.
- Omitting `#include <stdio.h>` or a statement terminator.
- Assuming a successful compile guarantees the executable is being run from the intended directory.

Activities

1. Compile the hello program with C17 and all requested warnings; record every diagnostic.
2. Change the greeting and add a second output line, predicting the exact line order.
3. Remove a semicolon in a disposable copy, compile, and use the diagnostic to locate the syntax error.
4. Write an acceptance-test note for "Your first program": state the exact input or source change and the expected observable result.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by "Your first program" without relying on Python terminology.
- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

Lesson 1.3: Variables and data types

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

C is statically typed. Use `int` for age, `double` for fractional measurements, `char` for one character, and a null-terminated character array or `const char *` for text. A C string buffer has a finite capacity and must include room for its terminating null character. Format specifiers in `printf` must match argument types.

Worked Example

```c
#include <stdio.h>
int main(void)
{
    const char *name = "Avery";
    int age = 16;
    double height_m = 1.72;
```

```
    const char *favorite = "Computer Science";
    printf("Name: %s\nAge: %d\nHeight: %.2f m\nFavorite subject: %s\n",
           name, age, height_m, favorite);
    return 0;
}
```

Expected output labels Avery, 16, 1.72 m, and Computer Science on separate lines. `%s`, `%d`, and `%.2f` correspond to string, integer, and floating-point arguments.

Expected Behavior or Output

Output observed when the worked example was run (gcc 13.3, C17; the text of the output does not depend on the operating system):

```
Name: Avery
Age: 16
Height: 1.72 m
Favorite subject: Computer Science
```

Common Mistakes

- Using a format string that does not match the declared type.
- Forgetting that C strings need a null terminator and sufficient buffer capacity.
- Using an uninitialized local variable or assuming a variable has Python-like dynamic typing.

Activities

1. Add student ID and a middle initial to Avery's profile, selecting appropriate C types.
2. Print the height using `%.1f`, `%.2f`, and `%.3f` and compare the displayed precision.
3. Explain why the string buffer size must include the terminating null byte.
4. Write an acceptance-test note for "Variables and data types": state the exact input or source change and the expected observable result.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by "Variables and data types" without relying on Python terminology.

- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

4.4 Unit 2: Numeric Computation

Overview

Use C expressions, numeric types, input/output, conversions, and the math library to solve the shared calculator and measurement problems. Focus on operator precedence, integer versus floating-point arithmetic, validated input, and predictable formatting.

Prerequisite: Unit 1. Learners should be able to compile a C program and declare basic variables.

Lesson 2.1: Design and tracing

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Use the design–implement–test cycle. Design identifies inputs, outputs, constraints, and edge cases; implementation translates the algorithm into C; testing compares actual behavior with expected behavior. Compile incrementally with warnings enabled and test both ordinary and boundary inputs.

Worked Example

For the canonical calculator, input is bill = 24.50 and rate = 0.15. Calculate tip = bill * rate and total = bill + tip. Expected unrounded values are 3.675 and 28.175; formatted to cents with `printf("%.2f")` they appear as 3.67 and 28.18 (the stored tip is 3.67499999999999982). Test zero, typical, and large bills.

Expected Behavior or Output

No program is run in this lesson. The expected result is a trace table written as sentences: bill 24.50, rate 0.15, tip 3.675, total 28.175, and test cases for zero, typical, and large bills with the result predicted before any code is written.

Common Mistakes

- Coding before writing down the bill, rate, tip, and total as separate inputs/intermediate/output values.
- Testing only $24.50 and missing zero, negative, or unusually large bill values.
- Rounding intermediate arithmetic and then obtaining a different displayed total.

Activities

1. Write pseudocode for the $24.50 bill and 15% tip, naming each input and intermediate result.
2. Create acceptance cases for bills 0.00, 24.50, and -1.00.
3. Trace the values 24.50, 0.15, 3.675, and 28.175 in order.
4. Record the exact expected numeric result before running the modified program; include 3.675 tip and 28.175 total before two-decimal display where the calculator context applies.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by "Design and tracing" without relying on Python terminology.
- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

Lesson 2.2: Expressions and assignment

Reading References

- Official C language guide and standard-library documentation.

- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Expressions combine operands and operators. C follows operator precedence; parentheses make intent explicit. Assignment (`=`) stores a value, while equality (`==`) compares values. Integer division discards the fractional part; at least one operand must be floating point for a fractional quotient.

Worked Example

```c
#include <stdio.h>
int main(void)
{
    int score = 50;
    printf("Initial score: %d\n", score);
    score = score + 20;
    printf("New score: %d\n", score);
    printf("Integer expression: %d\n", 10 / 3 + 3);
    printf("Floating expression: %.2f\n", 10.0 / 3 + 3);
    return 0;
}
```

Expected score values are 50 and 70. The integer expression is 6; the floating expression is approximately 6.33.

Expected Behavior or Output

Output observed when the worked example was run (gcc 13.3, C17; the text of the output does not depend on the operating system):

```
Initial score: 50
New score: 70
Integer expression: 6
Floating expression: 6.33
```

Common Mistakes

- Using integer division in a calculation that requires cents or a fractional average.
- Using `=` in a condition where `==` is intended.

- Depending on implicit precedence rather than making the intended grouping explicit.

Activities

1. Predict the results of `10 / 3`, `10.0 / 3`, and `10 / 3.0` before compiling.
2. Rewrite a compound expression with parentheses to show the intended precedence.
3. Change the score assignment from 50 to 70 and trace the printed values.
4. Record the exact expected numeric result before running the modified program; include 3.675 tip and 28.175 total before two-decimal display where the calculator context applies.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by "Expressions and assignment" without relying on Python terminology.
- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

Lesson 2.3: Input and output

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Use `printf` for formatted output. For line input, use `fgets` with a fixed-size buffer and check its return value. Convert numeric text with `strtol` or `strtod`; do not assume input is valid. Unlike Python's `input`, C input requires explicit buffer management and conversion.

Worked Example

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int main(void)
{
    char name[80], line[80], *end;
    long first, second;
    printf("Name: ");
    if (!fgets(name, sizeof name, stdin)) return 1;
    name[strcspn(name, "\n")] = '\0';
    printf("First whole number: ");
    if (!fgets(line, sizeof line, stdin)) return 1;
    first = strtol(line, &end, 10);
    if (end == line) { puts("Invalid number."); return 1; }
    printf("Second whole number: ");
    if (!fgets(line, sizeof line, stdin)) return 1;
    second = strtol(line, &end, 10);
    if (end == line) { puts("Invalid number."); return 1; }
    printf("Hello, %s!\nSum: %ld\n", name, first + second);
    return 0;
}
```

For inputs Avery, 12, and 8, output includes `Hello, Avery!` and `Sum: 20`. A production parser should also validate trailing characters and range errors.

Expected Behavior or Output

With these lines typed as standard input:

```
Avery
12
8
```

Output observed when the worked example was run (gcc 13.3, C17; the text of the output does not depend on the operating system):

```
Name: First whole number: Second whole number: Hello, Avery!
Sum: 20
```

Common Mistakes

- Reading a full name with `%s`, which stops at whitespace.
- Ignoring the return from `fgets` or accepting a numeric prefix followed by junk.
- Failing to detect a number outside the destination type's range.

Activities

1. Enter Avery, 12, and 8; verify the greeting and sum.
2. Enter a name containing spaces and confirm that the entire line is retained.
3. Extend the parser to reject `12abc`, blank input, and an out-of-range integer.
4. Record the exact expected numeric result before running the modified program; include 3.675 tip and 28.175 total before two-decimal display where the calculator context applies.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by "Input and output" without relying on Python terminology.
- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

Lesson 2.4: Numeric conversions and mathematics

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

C integer types represent whole numbers; `float` and `double` represent approximate floating-point values. Converting a floating value to an integer truncates toward zero. Convert text with `strtol` or `strtod`, validate the end pointer, and check range errors. Mixed arithmetic follows the usual arithmetic conversions.

Worked Example

```c
#include <stdio.h>
#include <stdlib.h>
int main(void)
{
    char age_text[] = "16", weight_text[] = "52.5", *end;
    long age = strtol(age_text, &end, 10);
    double weight = strtod(weight_text, &end);
    printf("Age doubled: %ld\nWeight plus age: %.1f\n", age * 2, weight + age);
    printf("Integer division: %d\nFloating division: %.1f\n", 5 / 2, 5.0 / 2);
    return 0;
}
```

Expected values: 32, 68.5, 2, and 2.5. Extend the example to reject trailing characters and check `errno` for overflow.

Expected Behavior or Output

Output observed when the worked example was run (gcc 13.3, C17; the text of the output does not depend on the operating system):

```
Age doubled: 32
Weight plus age: 68.5
Integer division: 2
Floating division: 2.5
```

Common Mistakes

- Assuming a cast rounds rather than truncates toward zero.
- Passing the wrong type to `printf` or using an incompatible format specifier.
- Failing to check `strtol`/`strtod` conversion end pointers and range errors.

Activities

1. Predict the results for age 16 and weight 52.5 before running the conversion example.
2. Convert -2.9 to an integer and explain truncation toward zero.
3. Add end-pointer and `errno` checks for malformed and out-of-range text.

4. Record the exact expected numeric result before running the modified program; include 3.675 tip and 28.175 total before two-decimal display where the calculator context applies.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by "Numeric conversions and mathematics" without relying on Python terminology.
- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

Lesson 2.5: Using the C Math Library

Reading References

- C17 standard library reference: `math.h`.
- Python curriculum Lesson 2.5: Using the Math Library.

Learning Objectives

- Include and use the C math library.
- Calculate circle area and circumference.
- Calculate distance from the origin using the Pythagorean theorem.
- Explain radians and the platform-specific math-library linker option.

Concept Explanation

The C math library declares functions such as `sqrt`, `pow`, `ceil`, `floor`, and trigonometric functions in `math.h`. Some GCC/Clang environments require linking with `-lm`. Trigonometric functions use radians. The standard C17 interface does not require the non-standard `M_PI` macro; `acos(-1.0)` is a portable way to obtain pi. Check function domains before calling functions such as `sqrt`.

Worked Example

```c
#include <stdio.h>
#include <math.h>
```

```c
int main(void)
{
    const double pi = acos(-1.0);
    const double radius = 5.0;
    const double x = 3.0;
    const double y = 4.0;
    double area = pi * radius * radius;
    double circumference = 2.0 * pi * radius;
    double distance = sqrt(x * x + y * y);

    printf("Area: %.2f\n", area);
    printf("Circumference: %.2f\n", circumference);
    printf("Distance from origin: %.2f\n", distance);
    return 0;
}
```

Expected output: area 78.54, circumference 31.42, and distance 5.00. Compile with `cc -std=c17 -Wall -Wextra -Wpedantic math_example.c -o math_example -lm` on systems that require explicit math-library linking.

Expected Behavior or Output

Output observed when the worked example was run (gcc 13.3, C17; the text of the output does not depend on the operating system):

```
Area: 78.54
Circumference: 31.42
Distance from origin: 5.00
```

Common Mistakes

- Forgetting `math.h` or the `-lm` linker option where needed.
- Passing degrees to trig functions that expect radians.
- Assuming `M_PI` is guaranteed by C17 or passing a negative value to `sqrt`.

Activities

1. Change the radius to 2.5 and predict area and circumference.
2. Calculate `sqrt(81.0)`, `ceil(4.2)`, and `floor(4.8)` by hand and verify.
3. Add validated radius input and reject negative values.
4. Record the exact expected numeric result before running the modified program; include 3.675 tip and 28.175 total before two-decimal display where the calculator context applies.

Accessibility Practice

Read the complete source line by line and inspect the punctuation in the function calls and arithmetic expressions. Run the compile command in the terminal and read compiler or linker diagnostics sequentially. Confirm that each output label identifies its value without relying on visual formatting.

Skills Checklist

- ☐ I can explain the C rule demonstrated by "Using the C Math Library" without relying on Python terminology.
- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

4.5 Unit 3: Control Flow

Overview

This unit covers Boolean expressions, conditionals, lists or arrays, iteration, string operations. Follow the same problem progression as Python so that the main comparison is language syntax and semantics. Use the worked examples as starting points, then complete the activities independently.

Lesson 3.1: Boolean expressions

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Relational operators (`==`, `!=`, `<`, `>`, `<=`, `>=`) produce an integer truth value: zero is false and nonzero is true. Logical operators are `&&`, `||`, and `!`; `&&` and `||`

short-circuit. Use braces for conditional blocks.

Worked Example

```c
#include <stdio.h>
int main(void)
{
    int score = 82, attendance = 95;
    if (score >= 70 && attendance >= 80) puts("Meets the course criteria.");
    if (score == 100 || attendance == 100) puts("At least one perfect result.");
    if (!(score < 0)) puts("Score is not negative.");
    return 0;
}
```

Expected output: `Meets the course criteria.` and `Score is not negative.`

Expected Behavior or Output

Output observed when the worked example was run (gcc 13.3, C17; the text of the output does not depend on the operating system):

```
Meets the course criteria.
Score is not negative.
```

Common Mistakes

- Using bitwise `&`/`|` when short-circuit logical `&&`/`||` is required.
- Reversing the score and attendance conditions.
- Assuming true is represented only by integer 1; any nonzero value is true.

Activities

1. Evaluate the score/attendance condition for (82,95), (69,95), and (82,70).
2. Create an expression where `&&` short-circuits and explain which operand is skipped.
3. Add a test for score zero and explain C's truth-value convention.
4. Write expected results for the shared scores 82, 91, 76, and 88, and include at least one boundary condition for the concept.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by ”Boolean expressions” without relying on Python terminology.
- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

Lesson 3.2: Chained conditionals

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Use `if`, `else if`, and `else` for mutually exclusive choices. Apply the shared grade thresholds from highest to lowest and reject scores outside 0–100 before classification.

Worked Example

```c
#include <stdio.h>
char grade_for(int score)
{
    if (score < 0 || score > 100) return '?';
    if (score >= 90) return 'A';
    else if (score >= 80) return 'B';
    else if (score >= 70) return 'C';
    else if (score >= 60) return 'D';
    else return 'F';
}
int main(void)
{
    int tests[] = {91, 90, 82, 80, 76, 70, 60, 0, -1, 101};
    for (size_t i = 0; i < sizeof tests / sizeof tests[0]; ++i)
```

```
        printf("%d -> %c\n", tests[i], grade_for(tests[i]));
    return 0;
}
```

Expected mapping: 91/A, 90/A, 82/B, 80/B, 76/C, 70/C, 60/D, 0/F; -1 and 101 are invalid (`?`).

Expected Behavior or Output

Output observed when the worked example was run (gcc 13.3, C17; the text of the output does not depend on the operating system):

```
91 -> A
90 -> A
82 -> B
80 -> B
76 -> C
70 -> C
60 -> D
0 -> F
-1 -> ?
101 -> ?
```

Common Mistakes

- Checking a lower grade boundary before a higher one and making a branch unreachable.
- Omitting the 0–100 validity check before assigning a grade.
- Using independent `if` statements when exactly one grade branch should run.

Activities

1. Run the grade function for 91, 90, 82, 80, 76, 70, 60, and 0.
2. Add tests for 89, 79, 69, and 59 and predict the grade at each boundary.
3. Verify that -1 and 101 are rejected rather than assigned a grade.
4. Write expected results for the shared scores 82, 91, 76, and 88, and include at least one boundary condition for the concept.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by ”Chained conditionals” without relying on Python terminology.
- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

Lesson 3.3: Sequences and indexing

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

C arrays store a fixed number of elements of one type. Indexing begins at zero and C does not automatically check bounds. A four-element array has valid indices 0–3. When an array is passed to a function, pass its length separately.

Worked Example

```c
#include <stdio.h>
int main(void)
{
    int scores[] = {82, 91, 76, 88};
    size_t count = sizeof scores / sizeof scores[0];
    printf("First: %d\nLast: %d\nCount: %zu\n", scores[0], scores[count - 1], count);
    scores[2] = 80;
    printf("Updated third score: %d\n", scores[2]);
    return 0;
}
```

Expected output: first 82, last 88, count 4, updated third score 80.

Expected Behavior or Output

Output observed when the worked example was run (gcc 13.3, C17; the text of the output does not depend on the operating system):

```
First: 82
Last: 88
Count: 4
Updated third score: 80
```

Common Mistakes

- Accessing an array at index `count` instead of `count - 1` for its last element.
- Assuming C performs bounds checks automatically.
- Calculating array length after it has decayed to a pointer in a function parameter.

Activities

1. Print each score with both its zero-based index and one-based position.
2. Change the third score from 76 to 80 and verify the updated array.
3. Explain why the valid indices for four elements are 0 through 3.
4. Write expected results for the shared scores 82, 91, 76, and 88, and include at least one boundary condition for the concept.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by "Sequences and indexing" without relying on Python terminology.
- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

Lesson 3.4: For loops and iteration

Reading References

- Official C language guide and standard-library documentation.

- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

A `for` loop has initialization, continuation condition, and update expression. Traverse the shared score array with a known count. Use `size_t` for indices and convert the total to `double` before calculating the average.

Worked Example

```c
#include <stdio.h>
int main(void)
{
    int scores[] = {82, 91, 76, 88};
    size_t count = sizeof scores / sizeof scores[0];
    int total = 0;
    for (size_t i = 0; i < count; ++i) { printf("Score %zu: %d\n", i + 1, scores[i]); total
    printf("Total: %d\nAverage: %.2f\n", total, (double) total / count);
    return 0;
}
```

Expected total: 337; average: 84.25.

Expected Behavior or Output

Output observed when the worked example was run (gcc 13.3, C17; the text of the output does not depend on the operating system):

```
Score 1: 82
Score 2: 91
Score 3: 76
Score 4: 88
Total: 337
Average: 84.25
```

Common Mistakes

- Using `i <= count` and reading beyond the final score.
- Forgetting to update the index or updating it twice.
- Calculating the average with integer division.

Activities

1. Trace the index, running total, and output for each of the four scores.
2. Count scores greater than or equal to 80 and report the result.
3. Calculate the average using integer division and floating-point division; compare.
4. Write expected results for the shared scores 82, 91, 76, and 88, and include at least one boundary condition for the concept.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by "For loops and iteration" without relying on Python terminology.
- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

Lesson 3.5: String operations

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

C strings are null-terminated character sequences. Use `strlen`, `strcmp`, and `strstr` from `string.h`; ensure every destination buffer has enough capacity. This example processes ASCII bytes. `strlen` counts bytes, not user-perceived Unicode characters.

Worked Example

```c
#include <stdio.h>
#include <string.h>
#include <ctype.h>
int main(void)
{
    const char sample[] = "A student studies programming. Programming develops problem-solvi
    char normalized[sizeof sample];
    size_t length = strlen(sample);
    for (size_t i = 0; i <= length; ++i) normalized[i] = (char)tolower((unsigned char)sample
    printf("Byte length: %zu\n", length);
    printf("Lowercase: %s\n", normalized);
    printf("Contains programming: %s\n", strstr(normalized, "programming") ? "yes" : "no");
    return 0;
}
```

Expected byte length: 75; substring result: yes. The cast to `unsigned char` is required for safe use of `tolower`.

Expected Behavior or Output

Output observed when the worked example was run (gcc 13.3, C17; the text of the output does not depend on the operating system):

```
Byte length: 75
Lowercase: a student studies programming. programming develops problem-solving skills.
Contains programming: yes
```

Common Mistakes

- Writing a lowercase result into a buffer without space for the null terminator.
- Passing a negative plain `char` to `tolower` rather than an `unsigned char` value.
- Assuming byte length from `strlen` equals Unicode character count.

Activities

1. Print the byte length and lowercase form of the canonical text sample.
2. Count occurrences of `programming` and test a missing substring.
3. Add a non-ASCII UTF-8 character and explain why byte length is not character count.
4. Write expected results for the shared scores 82, 91, 76, and 88, and include at least one boundary condition for the concept.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by ”String operations” without relying on Python terminology.
- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

4.6 Unit 4: Data Structures

Overview

Represent collections and records using C arrays, structs, pointers, and standard-library algorithms. Compare these explicit data structures with Python lists, tuples, and dictionaries, and document capacity and ownership.

Prerequisite: Unit 3. Learners should understand arrays, indexing, loops, and string basics.

Lesson 4.1: Mutable sequences

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

C arrays have fixed capacity. For a resizable sequence, manage a dynamically allocated buffer with a pointer, element count, and capacity. Every allocation

must be checked and every successful allocation must eventually be released with `free`.

Worked Example

```c
#include <stdio.h>
int main(void)
{
    int scores[8] = {82, 91, 76, 88};
    size_t count = 4;
    scores[count++] = 95;
    scores[count++] = 84;
    printf("Count: %zu; last: %d\n", count, scores[count - 1]);
    return 0;
}
```

Expected output: `Count: 6; last: 84`. The array has capacity 8 and currently contains 6 values; do not confuse capacity with count.

Expected Behavior or Output

Output observed when the worked example was run (gcc 13.3, C17; the text of the output does not depend on the operating system):

```
Count: 6; last: 84
```

Common Mistakes

- Treating array capacity as the number of currently valid elements.
- Appending after capacity is exhausted.
- Overwriting the original pointer with a failed `realloc` result.

Activities

1. Append 95 and 84 while tracking count separately from capacity.
2. Attempt to append beyond capacity and design a safe rejection or growth policy.
3. Sketch a `realloc` helper that checks failure without losing the original pointer.
4. Write a small test case that proves the collection count and capacity remain distinct and that all inventory fields stay associated.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by "Mutable sequences" without relying on Python terminology.
- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

Lesson 4.2: Sorting and organizing data

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Use `qsort` from `stdlib.h` to sort an array. The comparator must return a negative value, zero, or a positive value and must avoid subtraction that could overflow. Sorting changes the order of the original array.

Worked Example

```c
#include <stdio.h>
#include <stdlib.h>
int compare_ints(const void *a, const void *b)
{
    int left = *(const int *)a, right = *(const int *)b;
    return (left > right) - (left < right);
}
int main(void)
{
    int scores[] = {82, 91, 76, 88};
    size_t n = sizeof scores / sizeof scores[0];
    qsort(scores, n, sizeof scores[0], compare_ints);
    for (size_t i = 0; i < n; ++i) printf("%d ", scores[i]);
    puts("");
```

```
    return 0;
}
```

Expected ascending order: 76 82 88 91. Reverse traversal prints descending order.

Expected Behavior or Output

Output observed when the worked example was run (gcc 13.3, C17; the text of the output does not depend on the operating system):

```
76 82 88 91
```

Common Mistakes

- Returning `a - b` from a comparator, which can overflow.
- Returning inconsistent results for equal values.
- Sorting only keys and accidentally separating them from their inventory record fields.

Activities

1. Sort scores and verify the order 76, 82, 88, 91.
2. Change the comparator to descending order and test equal values.
3. Sort inventory records by quantity without separating fields from their item.
4. Write a small test case that proves the collection count and capacity remain distinct and that all inventory fields stay associated.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by ”Sorting and organizing data” without relying on Python terminology.
- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

Lesson 4.3: Immutable records

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Python tuples are immutable sequences. C has no direct tuple type; use a `struct` for a named record and make the object `const` when it must not be modified. A struct groups fields that may have different types.

Worked Example

```c
#include <stdio.h>
typedef struct { int x; int y; } Point;
int main(void)
{
    const Point origin_offset = {3, 4};
    printf("x: %d\ny: %d\n", origin_offset.x, origin_offset.y);
    return 0;
}
```

Expected output: x: 3 and y: 4. `const` prevents modifying the fields through this object.

Expected Behavior or Output

Output observed when the worked example was run (gcc 13.3, C17; the text of the output does not depend on the operating system):

```
x: 3
y: 4
```

Common Mistakes

- Assuming C structs are immutable without `const`.
- Assuming copying a struct containing pointers also copies the pointed-to data.
- Relying on positional initializers when field order is unclear.

Activities

1. Create a `Point` record for coordinates (3,4) and calculate its distance.
2. Compare designated and positional initialization for the same record.
3. Add a pointer member and explain why a struct copy is not a deep copy.
4. Write a small test case that proves the collection count and capacity remain distinct and that all inventory fields stay associated.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by "Immutable records" without relying on Python terminology.
- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

Lesson 4.4: Maps and key-value data

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

C does not include a built-in dictionary or hash map. For a small fixed inventory, use an array of structs and search by key; for larger data, introduce a hash-table implementation or a library. The key and value fields must be represented explicitly.

Worked Example

```c
#include <stdio.h>
typedef struct { const char *name; int quantity; double unit_price; } Item;
int main(void)
{
    Item inventory[] = {{"Notebook",3,2.50},{"Pencil",10,0.50},{"Folder",4,1.25}};
    for (size_t i = 0; i < sizeof inventory / sizeof inventory[0]; ++i)
        printf("%s: %d at $%.2f each\n", inventory[i].name, inventory[i].quantity, inventory
    return 0;
}
```

Expected records: Notebook 3 at $2.50; Pencil 10 at $0.50; Folder 4 at $1.25.

Expected Behavior or Output

Output observed when the worked example was run (gcc 13.3, C17; the text of the output does not depend on the operating system):

```
Notebook: 3 at $2.50 each
Pencil: 10 at $0.50 each
Folder: 4 at $1.25 each
```

Common Mistakes

- Expecting a built-in C dictionary or hash map.
- Comparing string keys with `==` instead of `strcmp`.
- Using a search result without checking for the not-found case.

Activities

1. Search the inventory for `Pencil` and display quantity and unit price.
2. Search for a missing key and report not found without using an invalid index.
3. Explain when an array scan becomes inefficient and a hash table is justified.
4. Write a small test case that proves the collection count and capacity remain distinct and that all inventory fields stay associated.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by "Maps and key-value data" without relying on Python terminology.

- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

Lesson 4.5: Collection traversal

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Traverse an array of records with an index and explicit count. A struct member is accessed with the dot operator. When passing records to helper functions, use pointers to avoid unnecessary copies and `const` when the function should not modify the record.

Worked Example

```c
#include <stdio.h>
typedef struct { const char *name; int quantity; double unit_price; } Item;
void show_item(const Item *item)
{
    printf("%s: quantity %d; value $%.2f\n", item->name, item->quantity, item->quantity * it
}
int main(void)
{
    Item items[] = {{"Notebook",3,2.50},{"Pencil",10,0.50},{"Folder",4,1.25}};
    for (size_t i = 0; i < sizeof items / sizeof items[0]; ++i) show_item(&items[i]);
    return 0;
}
```

Expected inventory values: $7.50, $5.00, and $5.00.

Expected Behavior or Output

Output observed when the worked example was run (gcc 13.3, C17; the text of the output does not depend on the operating system):

```
Notebook: quantity 3; value $7.50
Pencil: quantity 10; value $5.00
Folder: quantity 4; value $5.00
```

Common Mistakes

- Passing an array without its count to a function.
- Using `.` on a pointer or `->` on a non-pointer object.
- Returning a pointer to a local record that no longer exists.

Activities

1. Print all inventory items and calculate each quantity-times-price value.
2. Write a helper that receives a `const Item *` and explain the `->` operator.
3. Test the function with zero items and with all three inventory records.
4. Write a small test case that proves the collection count and capacity remain distinct and that all inventory fields stay associated.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by ”Collection traversal” without relying on Python terminology.
- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

4.7 Unit 5: Input and Iteration

Overview

Build robust input loops, validate parsed values, and control repetition with while, break, continue, and explicit state. Practice the same score and menu contexts while accounting for C's input and conversion APIs.

Prerequisite: Unit 4. Learners should be able to traverse arrays and write simple conditions.

Lesson 5.1: Input and validation

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Read user input as text with `fgets`, then parse and validate it. A robust parser checks whether any digits were consumed, whether trailing characters remain, and whether the value is in range. Keep prompting until valid input arrives or end-of-file is reached.

Worked Example

```c
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
int main(void)
{
    char line[80], *end;
    long value;
    for (;;) {
        printf("Enter a score from 0 to 100: ");
        if (!fgets(line, sizeof line, stdin)) return 1;
        value = strtol(line, &end, 10);
        while (isspace((unsigned char)*end)) ++end;
        if (end != line && *end == '\0' && value >= 0 && value <= 100) break;
        puts("Invalid score. Enter a whole number from 0 to 100.");
    }
    printf("Accepted score: %ld\n", value);
    return 0;
}
```

For input 82, output is `Accepted score: 82`; invalid values cause another prompt.

Expected Behavior or Output

With these lines typed as standard input:

```
abc
105
88
```

Output observed when the worked example was run (gcc 13.3, C17; the text of the output does not depend on the operating system):

```
Enter a score from 0 to 100: Invalid score. Enter a whole number from 0 to 100.
Enter a score from 0 to 100: Invalid score. Enter a whole number from 0 to 100.
Enter a score from 0 to 100: Accepted score: 88
```

Common Mistakes

- Accepting `82abc` as a valid score because `strtol` parsed the numeric prefix.
- Failing to reject values outside 0–100 or detect overflow.
- Treating blank input or end-of-file as a valid score.

Activities

1. Verify that 0, 82, and 100 are accepted as scores.
2. Verify that -1, 101, blank input, and `82abc` are rejected.
3. Test a value outside the range of `long` and check overflow handling.
4. Write a trace table for one valid input, one rejected input, and the exact state at loop termination.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by ”Input and validation” without relying on Python terminology.
- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

Lesson 5.2: While loops

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

A `while` loop repeats while its condition is nonzero. Update the condition's state inside the loop so it eventually becomes false. A sentinel-controlled loop is useful for repeated menu input; document the sentinel and handle end-of-file.

Worked Example

```c
#include <stdio.h>
int main(void)
{
    int value, total = 0;
    puts("Enter scores; enter -1 to finish.");
    while (scanf("%d", &value) == 1 && value != -1) {
        total += value;
        printf("Running total: %d\n", total);
    }
    printf("Final total: %d\n", total);
    return 0;
}
```

For input 82, 91, -1, the running totals are 82 and 173; final total is 173. This compact example assumes integer tokens; the previous lesson's `fgets` parser is safer for robust input.

Expected Behavior or Output

Output observed when the worked example was run (gcc 13.3, C17; the text of the output does not depend on the operating system):

```
Enter scores; enter -1 to finish.
Final total: 0
```

Common Mistakes

- Failing to update the sentinel-controlled loop state.
- Ignoring `scanf`'s return value and repeatedly processing an invalid token.
- Choosing a sentinel that could also be a legitimate score without documenting the rule.

Activities

1. Enter 82, 91, and -1 and trace the running total after each accepted score.
2. Enter -1 first and verify that the final total is zero.
3. Enter a nonnumeric token and explain why the unchecked `scanf` loop is not robust.
4. Write a trace table for one valid input, one rejected input, and the exact state at loop termination.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by ”While loops” without relying on Python terminology.
- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

Lesson 5.3: Flags and break

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

A Boolean-like flag records whether a condition has occurred. Use `break` to leave the nearest loop once a search succeeds. Initialize the flag before the loop and update it at the point where the condition is met.

Worked Example

```c
#include <stdio.h>
int main(void)
{
    int scores[] = {82, 91, 76, 88};
    int found = 0;
    for (size_t i = 0; i < sizeof scores / sizeof scores[0]; ++i) {
        if (scores[i] == 76) { found = 1; break; }
    }
    puts(found ? "Score found." : "Score not found.");
    return 0;
}
```

Expected output: `Score found.` Change the search value to 100 to exercise the not-found path.

Expected Behavior or Output

Output observed when the worked example was run (gcc 13.3, C17; the text of the output does not depend on the operating system):

```
Score found.
```

Common Mistakes

- Setting the found flag before the equality test.
- Forgetting to set the flag when the target is found.
- Breaking out of the wrong loop in a nested search.

Activities

1. Search for 76 and trace when the flag changes and when the loop breaks.
2. Search for 100 and verify the not-found branch.
3. Rewrite the search as a function returning the matching index or -1.
4. Write a trace table for one valid input, one rejected input, and the exact state at loop termination.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by ”Flags and break” without relying on Python terminology.
- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

Lesson 5.4: Continue and filtering

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Use `continue` to skip the remainder of the current loop iteration. It is useful for filtering records, but the condition must be explicit and understandable. Avoid using it to conceal complex control flow.

Worked Example

```c
#include <stdio.h>
int main(void)
{
    int scores[] = {82, 91, 76, 88};
    for (size_t i = 0; i < sizeof scores / sizeof scores[0]; ++i) {
        if (scores[i] < 80) continue;
        printf("Passing score: %d\n", scores[i]);
    }
    return 0;
}
```

Expected output: Passing score: 82, 91, and 88, each on its own line.

Expected Behavior or Output

Output observed when the worked example was run (gcc 13.3, C17; the text of the output does not depend on the operating system):

```
Passing score: 82
Passing score: 91
Passing score: 88
```

Common Mistakes

- Printing a score before applying the filter condition.
- Skipping a required state update with `continue` in a `while` loop.
- Using `< 80` when the intended filter is `>= 80`.

Activities

1. Filter the scores and print only values greater than or equal to 80.
2. Change the threshold to 90 and predict the remaining values.
3. Rewrite the loop without `continue` and compare the trace.
4. Write a trace table for one valid input, one rejected input, and the exact state at loop termination.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by "Continue and filtering" without relying on Python terminology.
- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

Lesson 5.5: Infinite-loop prevention

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

An infinite loop occurs when the termination condition never becomes false or the state used by the condition is not updated. Check loop invariants, increment/decrement operations, sentinel handling, and all paths that can exit. Prefer a clear loop condition over an unconditional loop unless an explicit `break` is guaranteed.

Worked Example

```c
#include <stdio.h>
int main(void)
{
    int remaining = 3;
    while (remaining > 0) {
        printf("%d\n", remaining);
        --remaining;
    }
    puts("Finished.");
    return 0;
}
```

Expected output: 3, 2, 1, then Finished. Removing `--remaining` creates a non-terminating loop; diagnose it by tracing the condition and state.

Expected Behavior or Output

Output observed when the worked example was run (gcc 13.3, C17; the text of the output does not depend on the operating system):

```
3
2
1
Finished.
```

Common Mistakes

- Leaving the variable in the loop condition unchanged.
- Moving the counter in the wrong direction or using a condition that cannot become false.
- Failing to trace early exits, EOF, and break conditions.

Activities

1. Trace `remaining` from 3 to 0 and identify the terminating condition.
2. Remove the decrement in a disposable copy and explain why it loops indefinitely.
3. Add a maximum-iteration safeguard to a loop that processes untrusted input.
4. Write a trace table for one valid input, one rejected input, and the exact state at loop termination.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by "Infinite-loop prevention" without relying on Python terminology.
- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

4.8 Unit 6: Functions

Overview

Decompose programs into functions with clear parameter, return, array-count, and pointer contracts. Understand pass-by-value, scope, and the additional responsibilities of variadic interfaces.

Prerequisite: Unit 5. Learners should be comfortable with loops, arrays, and validated input.

Lesson 6.1: Defining functions

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

A function gives a name to a reusable operation. Declare its return type, name, and parameter types. Put a prototype before the first call when the definition appears later. Keep one clear responsibility per function.

Worked Example

```c
#include <stdio.h>
double calculate_tip(double bill, double rate)
{
    return bill * rate;
}
int main(void)
{
    double tip = calculate_tip(24.50, 0.15);
    printf("Tip: $%.2f\n", tip);
    return 0;
}
```

Expected tip display: $3.67 (the exact product 3.675 is stored as 3.67499999999999982, so `%.2f` rounds down). The function returns a value and does not print it itself.

Expected Behavior or Output

Output observed when the worked example was run (gcc 13.3, C17; the text of the output does not depend on the operating system):

```
Tip: $3.67
```

Common Mistakes

- Calling a function before its prototype is visible.
- Declaring a non-void function but failing to return a value on every path.
- Combining calculation and printing so the function cannot be reused by another caller.

Activities

1. Write `calculate_tip` and call it for a $24.50 bill at 15%.
2. Move the function below `main` and add a correct prototype.
3. Separate calculation from printing so a second caller can reuse the result.

4. Document the function contract: parameter types, return type, array count or pointer ownership where relevant, and the expected result for one call.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by "Defining functions" without relying on Python terminology.
- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

Lesson 6.2: Parameters and arguments

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Arguments are the values supplied at a call site; parameters are the local variables receiving them. C passes arguments by value. To modify a caller's object, pass its address using a pointer. C has no default or named arguments; use explicit wrapper functions or configuration structs when needed.

Worked Example

```c
#include <stdio.h>
double total_with_tip(double bill, double rate)
{
```

```c
    return bill + bill * rate;
}
int main(void)
{
    printf("Total: $%.2f\n", total_with_tip(24.50, 0.15));
    return 0;
}
```

Expected total display: $28.18. Swap the two arguments and explain why the result changes; C uses positional arguments.

Expected Behavior or Output

Output observed when the worked example was run (gcc 13.3, C17; the text of the output does not depend on the operating system):

```
Total: $28.18
```

Common Mistakes

- Assuming C passes ordinary arguments by reference.
- Swapping positional arguments with the same type and expecting the compiler to detect intent.
- Passing a pointer where the function expects a value or vice versa.

Activities

1. Call `total_with_tip(24.50, 0.15)` and predict the output.
2. Swap the arguments and explain why C cannot infer their semantic roles when types match.
3. Write a second function that modifies a caller's integer through a pointer.
4. Document the function contract: parameter types, return type, array count or pointer ownership where relevant, and the expected result for one call.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by ”Parameters and arguments” without relying on Python terminology.
- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.

- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

Lesson 6.3: Return values

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

A non-void function returns a value using `return`. The caller can store it, pass it to another function, or print it. Use a `void` return type only when no value is returned. Every control-flow path in a non-void function must return an appropriate value.

Worked Example

```c
#include <stdio.h>
int sum_scores(const int scores[], size_t count)
{
    int total = 0;
    for (size_t i = 0; i < count; ++i) total += scores[i];
    return total;
}
int main(void)
{
    int scores[] = {82, 91, 76, 88};
    printf("Total: %d\n", sum_scores(scores, 4));
    return 0;
}
```

Expected total: 337. The function receives the array pointer and its length separately.

Expected Behavior or Output

Output observed when the worked example was run (gcc 13.3, C17; the text of the output does not depend on the operating system):

```
Total: 337
```

Common Mistakes

- Omitting a return on one control-flow path.
- Returning a pointer to a local automatic variable.
- Returning a value that is silently converted to an unintended type.

Activities

1. Call the score-summing function with the shared four values and verify 337.
2. Test a one-element array and an empty collection.
3. Add an average function and define behavior for count zero.
4. Document the function contract: parameter types, return type, array count or pointer ownership where relevant, and the expected result for one call.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by "Return values" without relying on Python terminology.
- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

Lesson 6.4: Functions over collections

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Pass arrays to functions with an explicit element count. Use `const` when the function only reads the collection. A pointer parameter does not contain the original array length; the caller and callee must agree on the count.

Worked Example

```c
#include <stdio.h>
double average(const int values[], size_t count)
{
    int total = 0;
    for (size_t i = 0; i < count; ++i) total += values[i];
    return count ? (double) total / count : 0.0;
}
int main(void)
{
    int scores[] = {82, 91, 76, 88};
    printf("Average: %.2f\n", average(scores, 4));
    return 0;
}
```

Expected average: 84.25. Add a separate function that searches the inventory array by item name.

Expected Behavior or Output

Output observed when the worked example was run (gcc 13.3, C17; the text of the output does not depend on the operating system):

```
Average: 84.25
```

Common Mistakes

- Assuming an array parameter carries its length.
- Passing a count larger than the actual array.
- Dividing by zero for an empty collection without defining a result.

Activities

1. Calculate the average of the shared scores using the array-plus-count interface.
2. Test count zero and document the function's defined result.

3. Add a maximum function and test one-element and repeated-maximum arrays.
4. Document the function contract: parameter types, return type, array count or pointer ownership where relevant, and the expected result for one call.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by "Functions over collections" without relying on Python terminology.
- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

Lesson 6.5: Scope and variadic arguments

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Local variables have block scope. A variable declared inside a function is not directly accessible elsewhere. C supports variadic functions through `<stdarg.h>`, but a format string or another explicit count is needed to describe the extra arguments. Prefer a typed array-plus-count interface for beginner exercises because it is easier to validate.

Worked Example

```c
#include <stdio.h>
#include <stdarg.h>
int sum_values(size_t count, ...)
{
    int total = 0;
    va_list args;
    va_start(args, count);
    for (size_t i = 0; i < count; ++i) total += va_arg(args, int);
    va_end(args);
    return total;
}
int main(void)
{
    printf("Sum: %d\n", sum_values(4, 82, 91, 76, 88));
    return 0;
}
```

Expected sum: 337. The caller must pass the correct count and types; variadic arguments are not type-checked by the function signature.

Expected Behavior or Output

Output observed when the worked example was run (gcc 13.3, C17; the text of the output does not depend on the operating system):

```
Sum: 337
```

Common Mistakes

- Using a block-local variable outside its scope.
- Passing the wrong promoted type to `va_arg`.
- Forgetting `va_end` or passing a variadic count inconsistent with the arguments.

Activities

1. Move a variable into a nested block and test its scope.
2. Call the variadic sum with the four shared scores and verify 337.
3. Replace the variadic interface with an array-plus-count function and compare type safety.
4. Document the function contract: parameter types, return type, array count or pointer ownership where relevant, and the expected result for one call.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by "Scope and variadic arguments" without relying on Python terminology.
- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

4.9 Unit 7: Structs and Object Models

Overview

Use structs and functions to organize related data and behavior. C does not provide class methods or built-in inheritance; use composition and explicit interfaces, and document pointer ownership and object lifetime.

Prerequisite: Unit 6. Learners should understand functions, pointers, and record fields.

Lesson 7.1: Types and object models

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

C does not have classes in the C++/Java sense. Use `struct` to group related data and functions that operate on the data to model an object. Encapsulation is

a design discipline in C: expose an interface in a header and keep implementation details in a source file. The shared domain is a student or inventory item.

Worked Example

```c
typedef struct {
    char name[40];
    int quantity;
    double unit_price;
} Item;
```

This record models one inventory item with the same fields used throughout the course: name, quantity, and unit price. A C struct stores fields; it does not automatically provide constructors, methods, or memory management.

Expected Behavior or Output

The struct definition prints nothing by itself, and it is not a complete program. Use it in a program that creates an `Item` (see Lesson 7.2) to see output. A correct answer to the activities lists the three fields and says that C leaves construction, validation, and cleanup to your own functions.

Common Mistakes

- Describing a C struct as a class with built-in methods and constructors.
- Failing to separate public declarations from implementation details in headers and source files.
- Using pointer fields without documenting ownership and lifetime.

Activities

1. Define an `Item` struct with name, quantity, and unit price.
2. Move the declaration into a header and include it from a source file.
3. Explain why C structs group fields but do not supply methods or automatic constructors.
4. Document the struct fields, initialization values, ownership rules, and the expected state after one operation.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by "Types and object models" without relying on Python terminology.

- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

Lesson 7.2: Creating instances

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Declare a struct variable to create an instance. Initialize each field deliberately; aggregate initialization follows field order unless designated initializers are used. Copying a struct copies its fields, including pointer values, not the memory referenced by those pointers.

Worked Example

```c
#include <stdio.h>
typedef struct { const char *name; int quantity; double unit_price; } Item;
int main(void)
{
    Item notebook = {.name = "Notebook", .quantity = 3, .unit_price = 2.50};
    Item pencil = {.name = "Pencil", .quantity = 10, .unit_price = 0.50};
    printf("%s: %d; %s: %d\n", notebook.name, notebook.quantity, pencil.name, pencil.quantit
    return 0;
}
```

Expected output: Notebook: 3; Pencil: 10. The two variables are separate struct instances.

Expected Behavior or Output

Output observed when the worked example was run (gcc 13.3, C17; the text of the output does not depend on the operating system):

```
Notebook: 3; Pencil: 10
```

Common Mistakes

- Using field-order initialization incorrectly.
- Leaving a field unintentionally uninitialized in an automatic object.
- Assuming struct assignment performs a deep copy of pointer members.

Activities

1. Initialize Notebook, Pencil, and Folder with designated initializers.
2. Copy one record and explain which data is copied when a member is a pointer.
3. Change one record's quantity and verify the other instance is unchanged.
4. Document the struct fields, initialization values, ownership rules, and the expected state after one operation.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by ”Creating instances” without relying on Python terminology.
- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

Lesson 7.3: Fields and methods

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Access struct fields with `.` when you have an object and `->` when you have a pointer to an object. Functions that operate on a struct can receive a pointer; use `const` for read-only access. Keep data and operations conceptually together even though C does not have member methods.

Worked Example

```c
#include <stdio.h>
typedef struct { const char *name; int quantity; double unit_price; } Item;
double item_value(const Item *item)
{
    return item->quantity * item->unit_price;
}
int main(void)
{
    Item notebook = {"Notebook", 3, 2.50};
    printf("%s inventory value: $%.2f\n", notebook.name, item_value(&notebook));
    return 0;
}
```

Expected output: Notebook inventory value: $7.50.

Expected Behavior or Output

Output observed when the worked example was run (gcc 13.3, C17; the text of the output does not depend on the operating system):

```
Notebook inventory value: $7.50
```

Common Mistakes

- Confusing `.` and `->` member access.
- Dereferencing a null pointer.
- Omitting `const` when a function should only inspect an item.

Activities

1. Calculate the value of three notebooks through `item_value`.
2. Compare `.` member access on an object with `->` on a pointer.
3. Add a null-pointer check and a `const` parameter.
4. Document the struct fields, initialization values, ownership rules, and the expected state after one operation.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal,

and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by ”Fields and methods” without relying on Python terminology.
- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

Lesson 7.4: Inheritance and interfaces

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

C has no built-in class inheritance. Model shared behavior with composition, function pointers, or tagged unions when appropriate. Do not force an inheritance hierarchy into C. For a simple specialization, use a base record as a field in a larger record and document the relationship explicitly.

Worked Example

```c
#include <stdio.h>
typedef struct { const char *name; int quantity; double unit_price; } Item;
typedef struct { Item item; int reorder_level; } StockedItem;
int needs_reorder(const StockedItem *stock)
{
    return stock->item.quantity <= stock->reorder_level;
}
int main(void)
{
    StockedItem notebook = {{"Notebook", 3, 2.50}, 3};
```

```
    printf("%s reorder required: %s\n", notebook.item.name,
           needs_reorder(&notebook) ? "yes" : "no");
    return 0;
}
```

Expected output: Notebook reorder required: yes. This is composition, not language-level inheritance.

Expected Behavior or Output

Output observed when the worked example was run (gcc 13.3, C17; the text of the output does not depend on the operating system):

```
Notebook reorder required: yes
```

Common Mistakes

- Claiming C has native class inheritance or virtual dispatch.
- Treating an embedded struct as automatically substitutable for its containing type.
- Using function pointers without a matching declared signature.

Activities

1. Create a `StockedItem` by embedding an `Item` and a reorder threshold.
2. Test quantities below, equal to, and above the threshold.
3. Explain why this composition is not C++-style inheritance.
4. Document the struct fields, initialization values, ownership rules, and the expected state after one operation.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by "Inheritance and interfaces" without relying on Python terminology.
- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

Lesson 7.5: Composition

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Composition builds a larger object from smaller related records. For the course's personal data manager, a manager can contain an array of items plus a count and capacity. Define ownership clearly: decide which function allocates and frees any dynamically allocated memory.

Worked Example

```c
#include <stdio.h>
typedef struct { const char *title; int done; } Task;
typedef struct { Task tasks[10]; size_t count; } TaskManager;
int main(void)
{
    TaskManager manager = {.tasks = {{"Review C arrays", 0}, {"Practice functions", 1}}, .co
    for (size_t i = 0; i < manager.count; ++i)
        printf("%s: %s\n", manager.tasks[i].title, manager.tasks[i].done ? "done" : "not don
    return 0;
}
```

Expected output lists the two tasks and their completion states.

Expected Behavior or Output

Output observed when the worked example was run (gcc 13.3, C17; the text of the output does not depend on the operating system):

```
Review C arrays: not done
Practice functions: done
```

Common Mistakes

- Calling composition inheritance.
- Allowing the task count to exceed the fixed array capacity.
- Failing to define whether nested strings are owned or borrowed.

Activities

1. Create a task manager with two tasks and print completion states.
2. Attempt to add an eleventh task to a ten-slot manager and handle capacity safely.
3. Document whether the manager owns task strings or borrows pointers.
4. Document the struct fields, initialization values, ownership rules, and the expected state after one operation.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by "Composition" without relying on Python terminology.
- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

4.10 Unit 8: Files, Errors, and Persistence

Overview

Read and write files with explicit error checks and resource cleanup. Distinguish standard C error reporting from exceptions, and use a maintained external library for JSON rather than ad hoc parsing.

Prerequisite: Unit 7. Learners should understand structs, pointers, and function return contracts.

Lesson 8.1: Reading text files

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Open a text file with `fopen` and check for `NULL` before reading. Read lines with `fgets` into a bounded buffer; process each line before reading the next. Always close a successfully opened file. Use a relative path resolved from the program's working directory.

Worked Example

```c
#include <stdio.h>
int main(void)
{
    FILE *file = fopen("notes.txt", "r");
    char line[256];
    if (file == NULL) { perror("notes.txt"); return 1; }
    while (fgets(line, sizeof line, file) != NULL) fputs(line, stdout);
    if (ferror(file)) { perror("Reading notes.txt"); fclose(file); return 1; }
    fclose(file);
    return 0;
}
```

Create `notes.txt` with two lines before running. The program reproduces those lines. A missing file produces a diagnostic and a nonzero exit status.

Expected Behavior or Output

With `notes.txt` in the working directory, containing:

```
Practice C.
Review arrays and functions.
```

Output observed when the worked example was run (gcc 13.3, C17; the text of the output does not depend on the operating system):

```
Practice C.
Review arrays and functions.
```

Common Mistakes

- Using a `FILE *` after `fopen` returned `NULL`.
- Treating every failed `fgets` as normal EOF without checking `ferror`.
- Assuming a relative path is based on the source file location rather than the current working directory.

Activities

1. Create a two-line notes file and verify that both lines are read.
2. Run with the file missing and inspect the diagnostic and exit status.
3. Create an overlong line and explain how truncation can be detected.
4. Record the file or data operation's success result, failure result, and the resource cleanup expected on each path.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by "Reading text files" without relying on Python terminology.
- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

Lesson 8.2: Writing and appending files

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Open a file with mode `w` to create or truncate it, and mode `a` to append. Check both the `fopen` result and the return value of write operations. Close the file and check for errors when durability matters. Do not overwrite a user's file without explaining the effect.

Worked Example

```c
#include <stdio.h>
int main(void)
{
    FILE *file = fopen("notes.txt", "a");
    if (file == NULL) { perror("notes.txt"); return 1; }
    if (fprintf(file, "Practice the C file API.\n") < 0) {
        perror("Writing notes.txt"); fclose(file); return 1;
    }
    if (fclose(file) == EOF) { perror("Closing notes.txt"); return 1; }
    puts("Note appended.");
    return 0;
}
```

Expected output: `Note appended.` The new line is added to the end of `notes.txt`.

Expected Behavior or Output

Output observed when the worked example was run (gcc 13.3, C17; the text of the output does not depend on the operating system):

```
Note appended.
```

Common Mistakes

- Opening in `w` mode and truncating a file that should have been appended.
- Ignoring `fprintf` and `fclose` failure results.
- Assuming buffered output has necessarily reached durable storage before close.

Activities

1. Append a second note and verify that the original contents remain.
2. Run the append program twice and count the resulting lines.
3. Change append mode to write mode in a disposable copy and observe truncation.
4. Record the file or data operation's success result, failure result, and the resource cleanup expected on each path.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by "Writing and appending files" without relying on Python terminology.
- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

Lesson 8.3: Errors and exceptions

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

C does not have language-level exceptions. Standard library functions report errors through return values, error indicators, or `errno`. Check each function's documented failure convention and use `perror` or `strerror` to report system errors. Return status codes from functions so callers can respond.

Worked Example

```c
#include <stdio.h>
int main(void)
{
    FILE *file = fopen("missing_notes.txt", "r");
    if (file == NULL) {
        perror("Unable to open missing_notes.txt");
        return 1;
    }
    fclose(file);
    return 0;
}
```

If the file is absent, the program prints an operating-system diagnostic and exits with status 1. This is explicit error handling, not a thrown exception.

Expected Behavior or Output

Output observed when the worked example was run (gcc 13.3, C17; the text of the output does not depend on the operating system):

```
Unable to open missing_notes.txt: No such file or directory
```

Common Mistakes

- Expecting C to provide language-level exceptions.
- Ignoring a function's documented error return convention or `errno`.
- Continuing with invalid state after reporting an error.

Activities

1. Run with a missing file and identify the error return and diagnostic.
2. Run with an existing file and verify the success path.
3. Write a helper that returns an error code to its caller rather than printing inside the helper.
4. Record the file or data operation's success result, failure result, and the resource cleanup expected on each path.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by ”Errors and exceptions” without relying on Python terminology.
- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

Lesson 8.4: Resource cleanup

Reading References

- Official C language guide and standard-library documentation.

- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

C resource lifetime is manual for resources such as `FILE *` handles and dynamically allocated memory. Every successful acquisition must have a corresponding release on all control-flow paths. Use a single cleanup section when multiple operations can fail; do not close an uninitialized or already-closed resource.

Worked Example

```c
#include <stdio.h>
int main(void)
{
    int status = 1;
    FILE *file = fopen("notes.txt", "r");
    if (file == NULL) { perror("notes.txt"); goto cleanup; }
    puts("File opened successfully.");
    status = 0;
cleanup:
    if (file != NULL && fclose(file) == EOF) { perror("fclose"); status = 1; }
    return status;
}
```

The cleanup path closes the file exactly once if opening succeeded. In larger programs, structure functions so cleanup remains easy to audit.

Expected Behavior or Output

With `notes.txt` in the working directory, containing:

```
Practice C.
Review arrays and functions.
```

Output observed when the worked example was run (gcc 13.3, C17; the text of the output does not depend on the operating system):

```
File opened successfully.
```

Common Mistakes

- Returning from a function after acquiring a resource without releasing it.

- Closing a file twice or using it after close.
- Running cleanup on a resource that was never successfully acquired.

Activities

1. Trace both the successful and failed `fopen` paths and locate the `fclose`.
2. Add a second resource and determine the correct cleanup order.
3. Introduce an early error after opening the file and verify that cleanup still occurs.
4. Record the file or data operation's success result, failure result, and the resource cleanup expected on each path.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by ”Resource cleanup” without relying on Python terminology.
- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

Lesson 8.5: JSON persistence

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

The C standard library does not include a JSON parser or serializer. Choose and document a maintained JSON library for projects that need JSON, and

do not hand-parse arbitrary JSON with string operations. Teach the shared record schema independently of the library: `id`, `title`, `description`, `category`, `created_at`, and `updated_at`. Explain how JSON strings, numbers, booleans, arrays, and objects map to C data.

Worked Example

```c
typedef struct {
    int id;
    char title[80];
    char description[240];
    char category[40];
    char created_at[32];
    char updated_at[32];
} DataItem;
```

This C struct is the in-memory shape of one capstone record. The JSON library layer must serialize every field and validate required fields when loading. Record the selected library and its build/link instructions in the project README.

Expected Behavior or Output

The struct definition prints nothing by itself, and it is not a complete program. The expected result of this lesson is a written plan that names the JSON library you chose, lists every field that must be saved and loaded, and says what happens when a field is missing.

Common Mistakes

- Trying to parse arbitrary JSON with comma splitting or substring search.
- Failing to validate required fields and JSON value types.
- Leaking parser-owned memory or retaining pointers after the JSON document is freed.

Activities

1. Map each canonical capstone field to a C member and a JSON value type.
2. Select a maintained C JSON library and document its version and link command.
3. Test missing required fields, malformed JSON, and a valid round-trip record.
4. Record the file or data operation's success result, failure result, and the resource cleanup expected on each path.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal,

and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by "JSON persistence" without relying on Python terminology.
- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

4.11 Unit 9: Testing and Data Analysis

Overview

Test functions and state changes, use pseudo-random values responsibly, and analyze text and CSV records while recognizing the limits of byte-oriented C examples and simple parsers.

Prerequisite: Unit 8. Learners should understand functions, structs, files, and error handling.

Lesson 9.1: Testing functions

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Test deterministic functions with known inputs and expected results. C has no standard unit-test framework; begin with assertions from `assert.h`, then introduce a test framework if a project grows. Compile tests separately from the application and ensure tests cover ordinary values, boundaries, and invalid inputs.

Worked Example

```c
#include <assert.h>
int add_two_numbers(int a, int b) { return a + b; }
int main(void)
{
    assert(add_two_numbers(5, 3) == 8);
    assert(add_two_numbers(-5, -3) == -8);
    assert(add_two_numbers(10, -4) == 6);
    return 0;
}
```

No output and exit status 0 indicate that all assertions passed. An assertion failure prints a diagnostic and terminates the test executable.

Expected Behavior or Output

Output observed when the worked example was run (gcc 13.3, C17; the text of the output does not depend on the operating system):

The program prints nothing and exits normally.

Common Mistakes

- Testing only one ordinary input and omitting boundaries and invalid cases.
- Writing assertions that duplicate implementation details instead of checking expected behavior.
- Compiling with `NDEBUG` and assuming assertions still execute.

Activities

1. Test addition with positive, negative, and zero values.
2. Create one failing assertion and inspect the diagnostic.
3. Compile with and without `NDEBUG` and explain the effect on assertions.
4. Write the expected test result and explain what a failing assertion or diagnostic would indicate.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by ”Testing functions” without relying on Python terminology.
- ☐ I can predict and verify the exact behavior of this lesson's worked example.

- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

Lesson 9.2: Testing stateful objects

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Test stateful records by creating a fresh instance for each test, changing its state through the public operations, and checking the resulting fields. In C, write explicit setup code in each test or a helper function; there is no built-in `setUp` lifecycle.

Worked Example

```c
#include <assert.h>
typedef struct { int quantity; } Item;
void add_stock(Item *item, int amount) { if (amount > 0) item->quantity += amount; }
int main(void)
{
    Item notebook = {.quantity = 3};
    add_stock(&notebook, 2);
    assert(notebook.quantity == 5);
    add_stock(&notebook, -1);
    assert(notebook.quantity == 5);
    return 0;
}
```

The assertions verify that a positive restock changes quantity and a negative restock is rejected.

Expected Behavior or Output

Output observed when the worked example was run (gcc 13.3, C17; the text of the output does not depend on the operating system):

The program prints nothing and exits normally.

Common Mistakes

- Allowing one test's mutation to leak into another test.
- Checking only the function result rather than the changed item state.
- Failing to verify that a rejected restock leaves quantity unchanged.

Activities

1. Restock by a positive amount and verify quantity changes.
2. Try zero and negative restock amounts and verify state remains unchanged.
3. Create fresh records for each test to prevent test-order dependence.
4. Write the expected test result and explain what a failing assertion or diagnostic would indicate.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by ”Testing stateful objects” without relying on Python terminology.
- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

Lesson 9.3: Random values

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Use the pseudo-random number generator in `stdlib.h`. Seed once with `srand`, then obtain values with `rand`. The sequence is deterministic for a given implementation and seed; use a fixed seed in tests. Avoid modulo bias for applications requiring statistically sound sampling or security; `rand` is not cryptographically secure.

Worked Example

```c
#include <stdio.h>
#include <stdlib.h>
int main(void)
{
    srand(42);
    int die = rand() % 6 + 1;
    printf("Die roll: %d\n", die);
    return 0;
}
```

The output is reproducible for a given C library implementation and seed, but the exact value may differ across implementations. Do not use `rand` for passwords or security tokens.

Expected Behavior or Output

Output observed when the worked example was run (gcc 13.3, C17; the text of the output does not depend on the operating system):

```
Die roll: 1
```

Common Mistakes

- Treating `rand()` as cryptographically secure.
- Assuming `% 6` produces perfectly unbiased die rolls.
- Expecting the exact sequence to be portable across different C library implementations.

Activities

1. Run twice with the same seed on the same C library and compare results.
2. Generate 1000 die values and verify each is between 1 and 6.

3. Explain why modulo reduction may be biased and why `rand` is not for secrets.
4. Write the expected test result and explain what a failing assertion or diagnostic would indicate.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by "Random values" without relying on Python terminology.
- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

Lesson 9.4: Text analysis

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Analyze the same shared text sample by splitting it into words and counting frequencies. C requires explicit tokenization and storage. For a beginner exercise, normalize ASCII letters and count a small fixed set of words; explain that a production tokenizer must handle punctuation, UTF-8, and dynamic word storage correctly.

Worked Example

```c
#include <stdio.h>
#include <string.h>
#include <ctype.h>
int main(void)
{
    char text[] = "A student studies programming. Programming develops problem-solving skill
    for (size_t i = 0; text[i]; ++i) text[i] = (char)tolower((unsigned char)text[i]);
    const char *word = "programming";
    int count = 0;
    char *p = text;
    while ((p = strstr(p, word)) != NULL) { ++count; p += strlen(word); }
    printf("Occurrences of programming: %d\n", count);
    return 0;
}
```

Expected output: 2 occurrences. This counts substring matches, not general word tokens; extend it to tokenize and count all words.

Expected Behavior or Output

Output observed when the worked example was run (gcc 13.3, C17; the text of the output does not depend on the operating system):

```
Occurrences of programming: 2
```

Common Mistakes

- Counting substring matches as whole words.
- Passing a negative plain `char` to `tolower`.
- Treating ASCII byte operations as Unicode-aware tokenization.

Activities

1. Count the two `programming` occurrences in the canonical sample.
2. Change case and punctuation and compare substring count with whole-word count.
3. Test a UTF-8 input and document the limits of byte-wise lowercase conversion.
4. Write the expected test result and explain what a failing assertion or diagnostic would indicate.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by "Text analysis" without relying on Python terminology.
- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

Lesson 9.5: CSV records

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

CSV is a delimited text format with quoting rules; splitting every line on commas is not a correct general CSV parser because quoted fields may contain commas, quotes, and newlines. Use a documented CSV library for production data. For the first exercise, read a simple known CSV file and state that the format excludes quoted commas.

Worked Example

```c
#include <stdio.h>
#include <string.h>
int main(void)
{
    FILE *file = fopen("students.csv", "r");
    char line[256];
    if (!file) { perror("students.csv"); return 1; }
    if (fgets(line, sizeof line, file)) printf("Columns: %s", line);
    while (fgets(line, sizeof line, file)) printf("Record: %s", line);
    fclose(file);
    return 0;
}
```

Create a header with `student_id,first_name,last_name,score,active` and at least two rows. The program prints the header and each record line. A production parser must handle CSV quoting correctly.

Expected Behavior or Output

With `students.csv` in the working directory, containing:

```
student_id,first_name,last_name,score,active
1,Avery,Student,91,true
2,Jordan,Lee,82,true
3,Casey,Smith,76,false
```

Output observed when the worked example was run (gcc 13.3, C17; the text of the output does not depend on the operating system):

```
Columns: student_id,first_name,last_name,score,active
Record: 1,Avery,Student,91,true
Record: 2,Jordan,Lee,82,true
Record: 3,Casey,Smith,76,false
```

Common Mistakes

- Splitting every CSV line on commas despite quoted commas and escaped quotes.
- Ignoring overlong lines or malformed rows.
- Assuming every row contains the same number of fields without validating it.

Activities

1. Create the canonical students.csv header and two data rows.
2. Test missing file, empty file, and an overlong row.
3. Add a quoted comma and explain why naive comma splitting is invalid CSV parsing.
4. Write the expected test result and explain what a failing assertion or diagnostic would indicate.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by ”CSV records” without relying on Python terminology.
- ☐ I can predict and verify the exact behavior of this lesson's worked example.

- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

4.12 Unit 10: Applied Projects

Overview

Apply C data structures, control flow, input validation, and state transitions to a text adventure and a command-line to-do manager. Prioritize explicit feedback, keyboard navigation, and safe handling of fixed-capacity data.

Prerequisite: Units 1–9. Learners should be able to build and test a small multi-function program.

Lesson 10.1: Requirements and design

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Use top-down design to break an application into input, processing, storage, and output functions. Preserve the same requirements as the Python project: clear prompts, labeled output, confirmation messages, validation, and text-only operation. Write pseudocode and acceptance tests before coding.

Worked Example

Design the to-do manager around the operations add task, view tasks, mark task complete, save tasks, load tasks, and quit. Define the task fields as `id`, `title`, `description`, `done`, and `created_at`. Specify the behavior for an empty list, an invalid task number, and a missing data file.

Expected Behavior or Output

No program is run in this lesson. The expected result is a written design: the six operations, the five task fields, and the exact messages for an empty list, an invalid task number, and a missing data file.

Common Mistakes

- Starting implementation without defining acceptance criteria and failure cases.
- Omitting empty-list, duplicate-ID, invalid-input, and unavailable-storage behavior.
- Selecting a persistence format without documenting its limitations.

Activities

1. Write acceptance criteria for add, view, edit, delete, and save/load operations.
2. Define behavior for empty data, duplicate IDs, unknown IDs, and unavailable storage.
3. Draw a module plan separating input, validation, data operations, persistence, and output.
4. Write an acceptance test with the initial project state, user action, expected response, and unchanged state after a rejected action.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by "Requirements and design" without relying on Python terminology.
- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

Lesson 10.2: Text adventure: five rooms and movement

Reading References

- C language standard library: stdio.h, string.h, and ctype.h.

- Canonical shared project specification: Python Unit 10, Lesson 10.3.

Learning Objectives

- Represent all five canonical rooms with C records and integer room indexes.
- Validate movement before changing the current room.
- Explain why the treasure room is not reachable until the key puzzle is implemented.

Concept Explanation

Use five fixed room indexes: start_room is 0, tunnel is 1, clearing is 2, locked_door_room is 3, and treasure_room is 4. Each exit stores a destination index or -1 when no exit exists. The graph is start north to tunnel; tunnel south to start and east to clearing; clearing west to tunnel and north to locked_door_room; and locked_door_room south to clearing. Part 1 deliberately has no north exit from locked_door_room.

The command loop reads a line, normalizes it to lowercase, checks the selected exit, and updates the current room only when the destination is valid. End-of-file acts like quit.

Worked Example

```c
#include <ctype.h>
#include <stdio.h>
#include <string.h>

typedef struct {
    const char *id;
    const char *description;
    int north, east, south, west;
} Room;

int main(void) {
    Room rooms[5] = {
        {"start_room", "A dimly lit cave. A narrow passage leads north.", 1, -1, -1, -1},
        {"tunnel", "A damp tunnel. Passages lead south and east.", -1, 2, 0, -1},
        {"clearing", "A quiet clearing. An antechamber lies to the north.", 3, -1, -1, 1},
        {"locked_door_room", "A chamber with a heavy door to the north.", -1, -1, 2, -1},
        {"treasure_room", "You found the treasure. You win!", -1, -1, -1, -1}
    };
    int current = 0;
    char command[80];

    for (;;) {
```

```
        printf("---\n%s\nWhat do you want to do?\n", rooms[current].description);
        if (!fgets(command, sizeof command, stdin)) strcpy(command, "quit");
        command[strcspn(command, "\r\n")] = '\0';
        for (size_t i = 0; command[i] != '\0'; ++i)
            command[i] = (char)tolower((unsigned char)command[i]);

        if (strcmp(command, "quit") == 0) {
            puts("Exiting game. Goodbye!");
            break;
        }

        int destination = -1;
        if (strcmp(command, "north") == 0) destination = rooms[current].north;
        else if (strcmp(command, "east") == 0) destination = rooms[current].east;
        else if (strcmp(command, "south") == 0) destination = rooms[current].south;
        else if (strcmp(command, "west") == 0) destination = rooms[current].west;

        if (destination >= 0 && destination < 5) current = destination;
        else puts("You cannot go that way. Enter a listed direction or quit.");
    }
    return 0;
}
```

Expected Behavior or Output

Enter north, east, north, north, quit. The player reaches tunnel, clearing, and locked_door_room. The second north is rejected and the player remains in locked_door_room. The treasure room is present in the room table but has no incoming exit in Part 1.

Common Mistakes

- Using -1 as an array index instead of checking it first.
- Updating the current room before validating the destination.
- Confusing a room index with the room's display name.
- Adding the treasure exit before the key puzzle is implemented.

Activities

1. Trace north, east, north and record the room index after each command.
2. Try west from start_room and verify the current room does not change.
3. Verify every nonnegative exit index is between 0 and 4.
4. Explain why treasure_room is unreachable in Part 1.

Accessibility Practice

Read the room description and prompt in order. The invalid-movement response explains what happened and how to continue. Compile and run from the terminal, then inspect output sequentially with the learner's access technology.

Skills Checklist

- ☐ I can represent all five rooms with C records.
- ☐ I can represent missing exits with a sentinel value.
- ☐ I validate an exit before changing rooms.
- ☐ I can reject an invalid direction without changing state.
- ☐ I can explain why the treasure room is not yet reachable.

Lesson 10.3: Text adventure: shiny key and locked door

Reading References

- C language standard library: stdio.h, string.h, and ctype.h.
- Canonical shared project specification: Python Unit 10, Lesson 10.3.

Learning Objectives

- Keep the shiny key in start_room until it is collected.
- Transfer the key to inventory exactly once.
- Require the player to use the key in locked_door_room before moving north to treasure_room.
- End the game after the player reaches treasure_room.

Concept Explanation

Keep the five-room graph from Lesson 10.2. The shiny key begins in start_room. The take shiny key command changes key_in_room to false and has_key to true. The use shiny key command succeeds only when the player is in locked_door_room and has_key is true. North from locked_door_room is blocked while door_locked is true. Reaching treasure_room ends the game.

Worked Example

```c
#include <ctype.h>
#include <stdio.h>
#include <string.h>

typedef struct {
    const char *id;
    const char *description;
```

```c
    int north, east, south, west;
} Room;

int main(void) {
    Room rooms[5] = {
        {"start_room", "A dimly lit cave.", 1, -1, -1, -1},
        {"tunnel", "A damp tunnel. Passages lead south and east.", -1, 2, 0, -1},
        {"clearing", "A quiet clearing. An antechamber lies to the north.", 3, -1, -1, 1},
        {"locked_door_room", "A chamber with a heavy door to the north.", 4, -1, 2, -1},
        {"treasure_room", "You found the treasure. You win!", -1, -1, -1, -1}
    };
    int current = 0, key_in_room = 1, has_key = 0, door_locked = 1;
    char command[80];

    for (;;) {
        printf("\n%s\n", rooms[current].description);
        if (current == 4) break;
        if (current == 0 && key_in_room) puts("You see: shiny key");
        puts(has_key ? "Your inventory: shiny key" : "Your inventory: empty");
        puts("What do you want to do?");
        if (!fgets(command, sizeof command, stdin)) strcpy(command, "quit");
        command[strcspn(command, "\r\n")] = '\0';
        for (size_t i = 0; command[i] != '\0'; ++i)
            command[i] = (char)tolower((unsigned char)command[i]);

        if (strcmp(command, "quit") == 0) {
            puts("You quit the adventure.");
            break;
        } else if (strcmp(command, "take shiny key") == 0) {
            if (current == 0 && key_in_room) {
                key_in_room = 0;
                has_key = 1;
                puts("You picked up the shiny key.");
            } else puts("There is no shiny key here to take.");
        } else if (strcmp(command, "use shiny key") == 0) {
            if (current != 3) puts("You must be at the locked door to use the key.");
            else if (!has_key) puts("You do not have the shiny key.");
            else {
                door_locked = 0;
                puts("You unlocked the door!");
            }
        } else if (current == 3 && strcmp(command, "north") == 0 && door_locked) {
            puts("The door is locked! You need to use the shiny key here.");
        } else {
            int destination = -1;
            if (strcmp(command, "north") == 0) destination = rooms[current].north;
```

```
            else if (strcmp(command, "east") == 0) destination = rooms[current].east;
            else if (strcmp(command, "south") == 0) destination = rooms[current].south;
            else if (strcmp(command, "west") == 0) destination = rooms[current].west;
            if (destination >= 0 && destination < 5) current = destination;
            else puts("Invalid command. Enter north, south, east, west, take shiny key, use
        }
    }
    return 0;
}
```

Expected Behavior or Output

Enter north, east, north, north, south, west, south, take shiny key, north, east, north, use shiny key, north. The first north from locked_door_room is rejected. The key is removed from the room and added to inventory. Using it outside locked_door_room is rejected. Using it there unlocks the door; the final north reaches treasure_room and prints the win description. End-of-file acts like quit.

Common Mistakes

- Allowing the key to be collected more than once.
- Unlocking the door without checking both room location and inventory.
- Allowing north movement before door_locked becomes false.
- Leaving the key listed in start_room after pickup.
- Treating a failed command as a state change.

Activities

1. Run the full command sequence and record room, key location, inventory, and door state.
2. Try taking the key twice; the second attempt must be rejected.
3. Try using the key in start_room and clearing; both attempts must be rejected.
4. Try north from locked_door_room before and after unlocking.
5. Add a help command that lists every supported command.

Accessibility Practice

Announce the room, visible item, inventory, and prompt as separate text lines. Every failed action explains why it failed. Test the full sequence with a screen reader and verify that item and door state are unambiguous.

Skills Checklist

- ☐ I can implement the five-room graph.
- ☐ I can transfer the shiny key to inventory exactly once.
- ☐ I can require key use in the locked-door room.

☐ I can block and unlock the north exit.
☐ I can end the game in the treasure room.

Lesson 10.4: Command-line to-do manager

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Build a command-line to-do tool with a menu loop and separate functions for adding, viewing, completing, saving, and loading tasks. Use the shared task fields and provide confirmations. C has no standard JSON support; for this project either choose a maintained JSON library and document its build instructions, or use a clearly documented simple text format while teaching JSON serialization separately in Lesson 8.5. Do not label a custom text format as JSON.

Worked Example

```c
#include <stdio.h>
#include <string.h>
typedef struct { char title[80]; int done; } Task;
void show_tasks(const Task tasks[], size_t count)
{
    if (count == 0) { puts("No tasks in your list."); return; }
    for (size_t i = 0; i < count; ++i)
        printf("%zu. [%c] %s\n", i + 1, tasks[i].done ? 'X' : ' ', tasks[i].title);
}
int main(void)
{
    Task tasks[50] = {0}; size_t count = 0; char line[100];
    for (;;) {
        puts("1 Add task  2 View tasks  3 Quit");
        printf("Choice: ");
        if (!fgets(line, sizeof line, stdin)) break;
        if (line[0] == '1' && count < 50) {
```

```
            printf("Task title: ");
            if (!fgets(tasks[count].title, sizeof tasks[count].title, stdin)) break;
            tasks[count].title[strcspn(tasks[count].title, "\n")] = '\0';
            tasks[count++].done = 0; puts("Task added.");
        } else if (line[0] == '2') show_tasks(tasks, count);
        else if (line[0] == '3') break;
        else puts("Invalid choice.");
    }
    return 0;
}
```

This starter implements add, view, and quit with a fixed capacity of 50 tasks. Extend it with completion and persistence, checking every input and file operation.

Expected Behavior or Output

Output observed when the worked example was run (gcc 13.3, C17; the text of the output does not depend on the operating system):

```
1 Add task  2 View tasks  3 Quit
Choice:
```

Common Mistakes

- Writing beyond the fixed task array's capacity.
- Treating the first character of a menu line as fully validated input.
- Completing or deleting a task by index without checking the selected range.

Activities

1. Add two tasks and view them in insertion order.
2. Complete a task by ID and reject an unknown ID.
3. Implement save/load and verify data remains after restarting.
4. Write an acceptance test with the initial project state, user action, expected response, and unchanged state after a rejected action.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by ”Command-line to-do manager” without relying on Python terminology.

- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

Lesson 10.5: Accessible UX and validation

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Accessible text interfaces use explicit prompts, consistent labels, predictable menu order, and clear confirmations. Validate all input before changing state. On errors, state what was wrong and how to correct it. Do not rely on color, position, or symbols alone to convey task status. For a screen-reader workflow, ensure output is announced in a useful sequence.

Worked Example

```c
#include <stdio.h>
#include <stdlib.h>
int main(void)
{
    char line[80], *end;
    long age;
    for (;;) {
        printf("Enter your age as a positive whole number: ");
        if (!fgets(line, sizeof line, stdin)) return 1;
        age = strtol(line, &end, 10);
        if (end != line && age > 0) break;
        puts("Invalid age. Enter a whole number greater than zero.");
    }
    printf("You entered age: %ld\n", age);
```

```
    return 0;
}
```

The program repeats until it receives a positive integer. Improve it by rejecting trailing non-whitespace characters and checking `errno` for overflow.

Expected Behavior or Output

With these lines typed as standard input:

```
abc
-4
16
```

Output observed when the worked example was run (gcc 13.3, C17; the text of the output does not depend on the operating system):

```
Enter your age as a positive whole number: Invalid age. Enter a whole number greater than ze
Enter your age as a positive whole number: Invalid age. Enter a whole number greater than ze
Enter your age as a positive whole number: You entered age: 16
```

Common Mistakes

- Using color or a symbol as the only indication of completion.
- Displaying an error without telling the user how to correct it.
- Silently truncating an overlong title or failing to announce a successful operation.

Activities

1. Complete the menu workflow using keyboard only and read prompts sequentially.
2. Submit blank, overlong, and invalid menu input and verify corrective instructions.
3. Test with a named screen reader and OS; record observed results rather than assuming compatibility.
4. Write an acceptance test with the initial project state, user action, expected response, and unchanged state after a rejected action.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by "Accessible UX and validation" without relying on Python terminology.

- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

4.13 Unit 11: Capstone: Personal Data Manager

Overview

Plan, implement, test, document, and present a C personal-data manager using the shared record fields and required CRUD/persistence behavior. Preserve the Python capstone's data and functional requirements while using a C-appropriate terminal-first interface.

Prerequisite: Units 1–10. Learners should be able to organize structs, functions, files, validation, and tests.

Lesson 11.1: Capstone requirements

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

The Python capstone uses NiceGUI, but that framework is Python-specific. The C course must preserve the same user-facing data-manager requirements without pretending that NiceGUI has a C equivalent. Implement a keyboard-operable terminal interface first; a graphical interface is an optional extension using a separately selected C-compatible toolkit. The core record fields, operations, validation, persistence, tests, and accessibility acceptance criteria remain the same.

Worked Example

Choose one collection: contacts, books/movies, to-do tasks, or inventory. Define fields, required values, validation rules, and the five required operations: add, view, edit, delete, save/load. For the to-do option, use `id`, `title`, `description`, `done`, and `created_at`. Document that the reference Python GUI is replaced by an accessible terminal interface in this C implementation.

Expected Behavior or Output

No program is run in this lesson. The expected result is a written specification that names the chosen collection, lists the shared record fields, states the validation rules, and describes the five required operations and the terminal interface.

Common Mistakes

- Omitting one of add, view, edit, delete, save, or load from the requirements.
- Designing a GUI-specific solution despite selecting a terminal interface.
- Failing to define required fields, validation, and acceptance tests.

Activities

1. Choose a capstone domain and list fields, validation rules, and user operations.
2. Write acceptance tests for add, view, edit, delete, save, and load before implementation.
3. Document why the C implementation uses a terminal interface rather than the Python-specific NiceGUI framework.
4. Add one capstone acceptance-test case to the project test plan, including test data, expected persisted fields, and failure behavior.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by "Capstone requirements" without relying on Python terminology.
- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.

- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

Lesson 11.2: Data model and storage

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Define a C struct for the shared capstone record and choose a clear ownership strategy for strings. Fixed-size arrays are acceptable for a small beginner project if maximum lengths are documented and input is bounded. For persistent data, use a maintained JSON library and document its version, dependency setup, and error handling. The C standard library does not provide JSON serialization.

Worked Example

```c
#include <stddef.h>
typedef struct {
    int id;
    char title[80];
    char description[240];
    char category[40];
    char created_at[32];
    char updated_at[32];
} DataItem;
```

The record contains the exact canonical capstone fields. Define which fields are required, how empty strings are handled, and how IDs and timestamps are generated. The serialization layer must preserve all fields when saving and loading.

Expected Behavior or Output

The struct definition prints nothing by itself, and it is not a complete program. The expected result is a written storage design: which fields are required, how empty strings are treated, how IDs and timestamps are produced, and how a save followed by a load preserves every field.

Common Mistakes

- Using fixed-size fields without documenting maximum lengths and truncation policy.
- Generating duplicate IDs or inconsistent timestamp formats.
- Serializing only part of the canonical record or trusting loaded fields without validation.

Activities

1. Define every C record field and its maximum length or allocation strategy.
2. Specify ID generation, timestamp format, and string ownership.
3. Round-trip a record through the chosen JSON library and compare every field.
4. Add one capstone acceptance-test case to the project test plan, including test data, expected persisted fields, and failure behavior.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by ”Data model and storage” without relying on Python terminology.
- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

Lesson 11.3: Create/read/update/delete

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Implement create, read, update, and delete operations as separate functions that operate on the same collection and report success or failure. Validate an identifier before modifying a record. After every operation, refresh the displayed list and persist successful changes. Avoid returning pointers to local stack variables.

Worked Example

```c
#include <stdio.h>
#include <string.h>
typedef struct { int id; char title[80]; int done; } Task;
int find_task(const Task tasks[], size_t count, int id)
{
    for (size_t i = 0; i < count; ++i) if (tasks[i].id == id) return (int)i;
    return -1;
}
int main(void)
{
    Task tasks[] = {{1, "Read C lesson", 0}, {2, "Practice arrays", 1}};
    size_t count = sizeof tasks / sizeof tasks[0];
    int index = find_task(tasks, count, 1);
    if (index >= 0) { strcpy(tasks[index].title, "Review C arrays"); tasks[index].done = 1;
    for (size_t i = 0; i < count; ++i)
        printf("%d: %s (%s)\n", tasks[i].id, tasks[i].title, tasks[i].done ? "done" : "open"
    return 0;
}
```

Expected state: task 1 is renamed to Review C arrays and marked done; task 2 is unchanged. Add bounds-safe input and implement add/delete operations using the same ID lookup.

Expected Behavior or Output

Output observed when the worked example was run (gcc 13.3, C17; the text of the output does not depend on the operating system):

```
1: Review C arrays (done)
2: Practice arrays (done)
```

Common Mistakes

- Using an unchecked lookup result as an array index.
- Copying a title into a fixed-size buffer without checking length.
- Changing in-memory state before persistence succeeds without a recovery strategy.

Activities

1. Add two records and verify their stable display order.
2. Edit one record by ID and confirm unrelated fields remain unchanged.
3. Delete an existing record, then attempt to delete the same ID again.
4. Add one capstone acceptance-test case to the project test plan, including test data, expected persisted fields, and failure behavior.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by "Create/read/update/delete" without relying on Python terminology.
- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

Lesson 11.4: Validation and testing

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Validate required fields before creating or updating a record. Test duplicate IDs, empty titles, overlong input, missing files, malformed JSON, and operations on unknown IDs. Keep deterministic logic separate from terminal input so it can be tested. In C, use assertions or a test framework and run with compiler warnings; consider AddressSanitizer and UndefinedBehaviorSanitizer during development.

Worked Example

```c
#include <assert.h>
#include <string.h>
int valid_title(const char *title)
{
    return title != NULL && title[0] != '\0';
}
int main(void)
{
    assert(valid_title("Read C lesson"));
    assert(!valid_title(""));
    assert(!valid_title(NULL));
    return 0;
}
```

All assertions pass with exit status 0. Add tests for the full CRUD functions and compile a debug build with `-fsanitize=address,undefined -fno-omit-frame-pointer` where supported.

Expected Behavior or Output

Output observed when the worked example was run (gcc 13.3, C17; the text of the output does not depend on the operating system):

The program prints nothing and exits normally.

Common Mistakes

- Testing only successful operations and ignoring malformed input or missing files.
- Ignoring compiler warnings or sanitizer diagnostics.
- Using assertions for recoverable user input errors in the release application.

Activities

1. Create a test matrix for valid, boundary, malformed, missing-file, and unknown-ID cases.
2. Compile with warnings and address every diagnostic.
3. Run AddressSanitizer and UndefinedBehaviorSanitizer where available and investigate reports.
4. Add one capstone acceptance-test case to the project test plan, including test data, expected persisted fields, and failure behavior.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal,

and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the C rule demonstrated by "Validation and testing" without relying on Python terminology.
- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

Lesson 11.5: Documentation and presentation

Reading References

- Official C language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

The deliverable includes source code, sample data, a README, build/run instructions, a description of the data format, and a reflection on accessibility. Document the compiler version, exact build command, external dependencies, file locations, and keyboard commands. Ensure prompts and status messages are meaningful when read linearly. Do not claim screen-reader testing unless it has actually been performed.

Worked Example

README acceptance checklist: explain the selected data-manager domain; list the exact fields; document add/view/edit/delete/save/load; provide compile and run commands; describe how to navigate the menu by keyboard; explain how to recover from invalid input; include sample data and known limitations; record the screen reader and operating-system tests actually performed.

Expected Behavior or Output

No program is run in this lesson. The expected result is a README that satisfies the acceptance checklist, with compile and run commands that another person can follow from an empty folder, and a record of the screen reader and operating-system tests you actually performed.

Common Mistakes

- Omitting compiler version, build command, or dependency setup from the README.
- Claiming assistive-technology compatibility without testing a named screen reader and OS.
- Leaving sample data, menu commands, or known limitations undocumented.

Activities

1. Build and run the project from a clean checkout using only the README instructions.
2. Document compiler, dependency versions, build command, file paths, and menu commands.
3. Test the documented keyboard workflow with a named access technology and record limitations.
4. Add one capstone acceptance-test case to the project test plan, including test data, expected persisted fields, and failure behavior.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Runnable Reference Implementation

The repository includes a tested C domain and JSON-storage reference core in capstone/README.md. Install the JSON development dependency described there, then run the acceptance command:

```
gcc -D_GNU_SOURCE -std=c17 -Wall -Wextra -Werror C/capstone/records.c
C/capstone/test_records.c $(pkg-config --cflags --libs json-c) -o
/tmp/c-capstone-tests && /tmp/c-capstone-tests
```

The reference core does not replace the requirement to build and evaluate an accessible user interface.

Skills Checklist

- ☐ I can explain the C rule demonstrated by ”Documentation and presentation” without relying on Python terminology.
- ☐ I can predict and verify the exact behavior of this lesson's worked example.
- ☐ I can complete the lesson-specific boundary or extension tests above and explain the result.
- ☐ I can recognize the three C-specific failure modes listed in this lesson and apply a correction.
- ☐ I can navigate, compile, run, and inspect the source and output using my chosen access technology.

AI Policy

AI may be used only for personal conceptual understanding and reinforcement. It must not write, complete, autocomplete, repair, or generate course code or assessment solutions.

Part V

C++ (C Plus Plus)

Chapter 5

C++ Curriculum

Before you start: complete Unit P: Computing Foundations Primer. It explains the language-neutral vocabulary and concepts used in every language track, using non-visual examples.C++ is used in game engines, simulation, robotics, finance, desktop software, and performance-critical systems.

Course structure

A complete introductory sequence aligned to Python Units 0–11: setup; foundations; numeric computation; control flow; data structures; input and iteration; functions; object-oriented design; files and exceptions; testing and data analysis; projects; and capstone. The same core problem contexts recur across languages so students focus on syntax, types, runtime behavior, libraries, and idioms.

Environment

Install GCC, Clang, or MSVC with C++20 support. On GCC/Clang, compile with `c++ -std=c++20 -Wall -Wextra -Wpedantic -Werror main.cpp -o main`; run with `./main`.

Complete the shared IDE setup lesson and terminal prerequisite. The course supports keyboard-first work with screen readers, braille displays, magnification, or other access technology.

- Complete lessons
- Topic index

Lesson example verification

The course includes 53 numbered lessons across Units 1–11 and four structured setup lessons in Unit 0. The numbered lessons have distinct worked examples, lesson-specific mistakes, four practice activities, and five mastery checks.

- C++ example validation script
- GCC/Clang GitHub Actions workflow
- Validation run 41318426947

GCC 13.3.0 and Clang 18.1.3 each compiled and linked all 54 fenced C++ examples in C++20 mode with `-Wall -Wextra -Wpedantic -Werror`. The checker wraps lesson fragments in a minimal program and supplies common standard-library headers where needed.

This verifies compilation and linking only. The examples have not all been executed with representative input or file fixtures. A fresh mdBook build, complete link crawl, and hands-on screen-reader/braille testing remain outstanding.

GUI, visualization, and workbench accessibility

See the shared GUI, visualization, and workbench accessibility reference for screen-reader support by toolkit, platform-specific activation settings, recommended alternatives, and interfaces that must not be required without an equivalent accessible route.

AI Policy

AI may be used only for personal conceptual understanding and reinforcement. It must not write, complete, autocomplete, repair, or generate course code or assessment solutions.

Recommended introductory books

- Stroustrup, Bjarne. *Programming: Principles and Practice Using C++*, 3rd edition. Addison-Wesley, 2024. An introduction to programming principles through C++, covering procedural, object-oriented, and generic programming. Author's book page.
- Gaddis, Tony. *Starting Out with C++: From Control Structures Through Objects*, 10th edition. Pearson, 2020. A structured first-course text introducing control flow, functions, arrays, pointers, and object-oriented programming. Publisher catalog.
- Downey, Allen B. *Think C++: How to Think Like a Computer Scientist*. Green Tea Press. A free introductory text emphasizing problem solving and computational thinking. Book page.

Suggested use: Stroustrup's third edition is the most current full introduction in this list. Gaddis offers a conventional course sequence. *Think C++* is freely available but uses an older C++ teaching context; use it for transferable problem-solving concepts rather than as the authority for modern C++20 syntax and libraries.

5.1 CPP Lessons

The curriculum is organized as one Markdown file per unit.

- accessible environment and toolchain
- foundations of computing
- numeric computations
- control flow
- data structures
- input and iteration
- functions
- object oriented programming
- files and exceptions
- testing and data analysis
- project based learning
- capstone

5.2 Unit 0: Accessible Environment and Toolchain

Course overview

Before you start: complete Unit P: Computing Foundations Primer. It explains the language-neutral vocabulary and concepts used in every language track, using non-visual examples.

Course Overview

C++ is used in game engines, simulation, robotics, finance, desktop software, and performance-critical systems.

This course intentionally parallels the original Python curriculum. It reuses problem contexts—greetings, profiles, a calculator, grade decisions, score collections, inventory, functions, text files, tests, text analysis, CSV, a to-do list, and a personal-data manager—so the principal new learning target is the C++ language. The syntax and implementation remain idiomatic, not a mechanical translation.

Unit 0: Accessible Environment and Toolchain

Overview

Install a C++20 compiler, configure an editor, create a reproducible project workspace, and verify keyboard and assistive-technology access to source code, diagnostics, and terminal output.

Lesson 0.1: Install and Verify C++

Reading References

- GCC, Clang, or Microsoft C++ compiler installation documentation.
- C++ language support and compiler-version documentation.

Learning Objectives

- Identify the compiler and its version.
- Compile, link, and run a minimal C++20 program.
- Explain the difference between a compiler diagnostic and program output.

Concept Explanation C++ is a compiled language. The compiler translates each translation unit; the linker combines object code and required libraries into an executable. The course uses C++20. Keep the compiler, editor language service, and command-line build on the same standard.

Worked Example

```cpp
#include <iostream>
int main() {
    std::cout << "Hello, C++20!\n";
    return 0;
}
```

Compile with `c++ -std=c++20 -Wall -Wextra -Wpedantic -Werror -g main.cpp -o main`; run with `./main` (or the platform executable form).

Expected Behavior or Output

The compile command prints nothing when there are no warnings. Running the program prints one line:

```
Hello, C++20!
```

This output was observed with g++ 13.3 on Linux.

Common Mistakes

- Installing only editor syntax highlighting and assuming it includes a compiler.
- Omitting the C++20 standard flag and unknowingly using a different language baseline.
- Treating a compiler error as output produced by the running program.

Activities

1. Install GCC, Clang, or MSVC and record its exact version.
2. Compile and run the example from the terminal.
3. Remove the semicolon after the output statement, compile again, and locate the diagnostic.
4. Restore the semicolon and confirm that the build succeeds.

Accessibility Practice Navigate the source by line, word, and character. Confirm that punctuation, braces, indentation, compiler diagnostics, and terminal output can be read without relying on color or mouse-only controls.

Skills Checklist

☐ I can identify the installed compiler and version.
☐ I can compile and link a C++20 program from the terminal.
☐ I can run the executable and distinguish its output from diagnostics.
☐ I can navigate the source and diagnostic text with my access technology.

Lesson 0.2: Workspace and First Run

Reading References

- Compiler command-line documentation.
- The shared terminal prerequisite.

Learning Objectives

- Create a course workspace and source file.
- Identify the current working directory and executable path.
- Rebuild and run a program after changing its source.

Concept Explanation A source file such as `main.cpp` is input to the compiler; it is not itself the executable. Relative paths are interpreted from the process's current working directory. Keep source files separate from generated executables and temporary build output.

Worked Example

```
mkdir -p cpp-course/hello
cd cpp-course/hello
# Save the C++ example as main.cpp
c++ -std=c++20 -Wall -Wextra -Wpedantic main.cpp -o hello
./hello
```

Expected Behavior or Output

Compiling prints nothing when the source is correct, and `./hello` prints:

```
Hello, C++20!
```

Common Mistakes

- Running the compile command from a directory that does not contain the source file.
- Editing one copy of a file while compiling a different copy.
- Confusing the source filename with the executable filename.

Activities

1. Create a course folder and a `hello` subfolder.
2. Save the first program as `main.cpp` and compile it to `hello`.
3. Change the output text, rebuild, and verify the new output.
4. List the folder contents and identify source versus generated files.

Accessibility Practice Use terminal commands to confirm the working directory and list files. Navigate the folder tree with keyboard commands and verify the executable name from text output.

Skills Checklist

☐ I can create and navigate a project directory.
☐ I can identify the current working directory.
☐ I can compile a named source file into a separate executable.
☐ I can rebuild after an edit and verify the changed output.

Lesson 0.3: Editor and Accessibility Workflow

Reading References

- IDE setup lesson.
- Documentation for the chosen editor's C++ language extension and screen-reader support.

Learning Objectives

- Open the project root rather than an individual source file only.
- Configure C++ language services and compiler selection.
- Navigate source, diagnostics, terminal, and project files by keyboard.

Concept Explanation

An editor provides text editing; an IDE adds integrated language services, build, debugging, and project navigation. VS Code, VSCodium, and Theia can provide C++ support through extensions, but their extension marketplaces differ. A language server is not a substitute for an installed compiler.

Worked Example

Open the folder containing `main.cpp`, install the appropriate C++ language extension, select the same compiler used in the terminal, and confirm that diagnostics identify a deliberately introduced syntax error.

Expected Behavior or Output

The editor shows a diagnostic at the syntax error that you introduce, with a line number and message that match what the compiler prints in the terminal. After you fix the error, the editor and the compiler are both clean.

Common Mistakes

- Opening only a file instead of the project root, preventing project-level configuration from loading.
- Installing an extension from a marketplace unavailable to the selected editor distribution.
- Assuming code completion or syntax highlighting proves that the compiler is configured.

Activities

1. Open the project folder in VS Code, VSCodium, or Theia.
2. Verify language-server diagnostics and compiler selection.
3. Navigate to a line, inspect punctuation, and read a diagnostic using the screen reader.
4. Build and run from the integrated terminal, then repeat from the system terminal.

Accessibility Practice Test keyboard-only access to the editor, file tree, diagnostics panel, integrated terminal, and output. Adjust punctuation verbosity and indentation announcements to a usable level.

Skills Checklist

- ☐ I can open the project root in my chosen editor.
- ☐ I can identify the C++ extension and compiler configuration.
- ☐ I can navigate diagnostics and terminal output by keyboard.
- ☐ I can complete the build/run workflow without mouse-only actions.

Lesson 0.4: Project Environments, Dependencies, and Terminal Usage

Reading References

- CMake getting-started documentation.
- Compiler and linker command-line documentation.

Learning Objectives

- Explain the working directory and the roles of source, header, object, and executable files.
- Compile and link a multi-file project.
- Record external dependencies and reproducible build instructions.

Concept Explanation C++ dependencies may be supplied by the standard library or external libraries. Header lookup happens during preprocessing/compilation; unresolved external symbols are generally linker errors. Larger projects should record build configuration and use an out-of-source build directory such as a CMake `build/` folder.

Worked Example

```
c++ -std=c++20 -Wall -Wextra -Wpedantic -Werror -g main.cpp helper.cpp -o app
./app
```

Expected Behavior or Output

The compile command builds one program from both source files and prints nothing when it succeeds; `./app` then runs it. If `helper.cpp` is left off the command, the linker reports an undefined reference error.

Common Mistakes

- Running a relative path from the wrong working directory.
- Treating a missing header as a linker error.
- Omitting a required implementation file or library from the link command.

Activities

1. Compile a one-file program from the project root and record the exact command.
2. Split a program into a header, implementation file, and `main.cpp`; compile and link all translation units.
3. Remove one implementation file from the command and interpret the linker diagnostic.
4. Record compiler version, standard flag, dependencies, and build steps in a README.

Accessibility Practice Navigate each project file and terminal diagnostic by keyboard. Confirm that build errors can be read sequentially and are not communicated by color alone.

Skills Checklist

☐ I can identify the project root and run commands from it.
☐ I can explain the difference between a header error and a linker error.
☐ I can compile and link a multi-file project.
☐ I can document dependencies and reproducible build steps.

5.3 Unit 1: Foundations of Computing

Overview

This unit covers computer science, hardware/software, algorithms, first program, variables, primitive types. Follow the same problem progression as Python so that the main comparison is language syntax and semantics. Use the worked examples as starting points, then complete the activities independently.

Lesson 1.1: Computer science and problem solving

Reading References

- Official C++ language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Computer science and problem solving is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: distinguish an algorithm from its C++ implementation; a clean build does not prove algorithmic correctness.

Worked Example

```
std::cout << "Computer science turns problems into precise, testable steps.\n";
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
Computer science turns problems into precise, testable steps.
```

Common Mistakes

- A successful compile does not establish that the algorithm satisfies the requirement.
- Distinguish an algorithm from its C++ implementation.
- A clean build does not prove algorithmic correctness.

Activities

1. Trace the computer science and problem solving example and predict its output before compiling.
2. Adapt the example to Avery's profile and greeting and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.
4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal,

and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the concept in C++ terminology.
- ☐ I can trace and predict the example.
- ☐ I can modify and run the code independently.
- ☐ I can identify and correct a common error.
- ☐ I can read source and output accessibly.

Lesson 1.2: Your first program

Reading References

- Official C++ language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Your first program is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: provide main with the correct signature; include the declaration header and use std::cout.

Worked Example

```
const std::string greeting_name = "Avery"; std::cout << "Hello, " << greeting_name << "!\n";
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
Hello, Avery!
```

Common Mistakes

- Forgetting to rebuild after editing can leave an older executable on disk.
- Provide main with the correct signature.
- Include the declaration header and use std::cout.

Activities

1. Trace the your first program example and predict its output before compiling.
2. Adapt the example to Avery's profile and greeting and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.
4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

☐ I can explain the concept in C++ terminology.
☐ I can trace and predict the example.
☐ I can modify and run the code independently.
☐ I can identify and correct a common error.
☐ I can read source and output accessibly.

Lesson 1.3: Variables and data types

Reading References

- Official C++ language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Variables and data types is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: initialize every built-in variable; select types that preserve fractional values; distinguish assignment from equality.

Worked Example

```
const std::string name = "Avery"; const int age = 16; const double height_m = 1.72; const st
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
Avery, 16, 1.72 m, Computer Science
```

Common Mistakes

- Initialize every built-in variable.
- Select types that preserve fractional values.
- Distinguish assignment from equality.

Activities

1. Trace the variables and data types example and predict its output before compiling.
2. Adapt the example to Avery's profile and greeting and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.
4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain variables and data types using correct C++ terminology and distinguish it from the closest Python equivalent.
- ☐ I can trace the example and justify its output using C++ rules.
- ☐ I can implement the same requirement for Avery's profile and greeting without copying Python syntax.
- ☐ I can test a normal case, a boundary case, and a relevant invalid or failure case.
- ☐ I can compile the program, interpret diagnostics, and navigate source and terminal output using my access technology.

5.4 Unit 2: Numeric Computations

Overview

This unit covers development process, expressions, assignment, input/output, conversions, arithmetic, math library. Follow the same problem progression as Python so that the main comparison is language syntax and semantics. Use the worked examples as starting points, then complete the activities independently.

Lesson 2.1: Design and tracing

Reading References

- Official C++ language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Design and tracing is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: calculate the tip from the bill and keep the rate as a fraction; separate intermediate values to make tracing possible.

Worked Example

```
const double bill = 24.50; const double tip = bill * 0.15; std::cout << "Tip: $" << tip << '
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
Tip: $3.675; total: $28.175
```

Common Mistakes

- Printing currency without a defined rounding/formatting policy can misrepresent fractional cents.
- Calculate the tip from the bill and keep the rate as a fraction.
- Separate intermediate values to make tracing possible.

Activities

1. Trace the design and tracing example and predict its output before compiling.
2. Adapt the example to the $24.50 bill and 15% tip and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.
4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

☐ I can explain the concept in C++ terminology.
☐ I can trace and predict the example.
☐ I can modify and run the code independently.
☐ I can identify and correct a common error.
☐ I can read source and output accessibly.

Lesson 2.2: Expressions and assignment

Reading References

- Official C++ language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Expressions and assignment is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: use compound assignment without overwriting prior values; distinguish percentage rates from percentage points; avoid narrowing conversions.

Worked Example

```
double score = 70.0; score += 12.5; score *= 1.1; std::cout << "Adjusted score: " << score <
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
Adjusted score: 90.75
```

Common Mistakes

- Use compound assignment without overwriting prior values.
- Distinguish percentage rates from percentage points.
- Avoid narrowing conversions.

Activities

1. Trace the expressions and assignment example and predict its output before compiling.
2. Adapt the example to the $24.50 bill and 15% tip and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.
4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

☐ I can explain the concept in C++ terminology.
☐ I can trace and predict the example.
☐ I can modify and run the code independently.
☐ I can identify and correct a common error.
☐ I can read source and output accessibly.

Lesson 2.3: Input and output

Reading References

- Official C++ language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Input and output is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: check stream extraction before using input; handle multiword text with getline; account for the pending newline after formatted input.

Worked Example

```
std::istringstream input("Avery 6 7"); std::string name; int first = 0, second = 0; if (inpu
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
Hello, Avery; sum: 13
```

Common Mistakes

- Check stream extraction before using input.
- Handle multiword text with getline.
- Account for the pending newline after formatted input.

Activities

1. Trace the input and output example and predict its output before compiling.
2. Adapt the example to the $24.50 bill and 15% tip and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.
4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

☐ I can explain the concept in C++ terminology.
☐ I can trace and predict the example.
☐ I can modify and run the code independently.
☐ I can identify and correct a common error.
☐ I can read source and output accessibly.

Lesson 2.4: Numeric conversions and mathematics

Reading References

- Official C++ language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Numeric conversions and mathematics is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: cast before division to avoid integer truncation; define behavior for invalid square-root inputs; distinguish rounding from truncation.

Worked Example

```
const int completed = 7, total = 10; const double ratio = static_cast<double>(completed) / t
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
Percent: 70; rounded: 70
```

Common Mistakes

- Cast before division to avoid integer truncation.
- Define behavior for invalid square-root inputs.
- Distinguish rounding from truncation.

Activities

1. Trace the numeric conversions and mathematics example and predict its output before compiling.
2. Adapt the example to the $24.50 bill and 15% tip and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.
4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain numeric conversions and mathematics using correct C++ terminology and distinguish it from the closest Python equivalent.
- ☐ I can trace the example and justify its output using C++ rules.
- ☐ I can implement the same requirement for the $24.50 bill and 15% tip without copying Python syntax.
- ☐ I can test a normal case, a boundary case, and a relevant invalid or failure case.
- ☐ I can compile the program, interpret diagnostics, and navigate source and terminal output using my access technology.

Lesson 2.5: Using the Math Library

Reading References

- C++ standard library documentation for `<cmath>`.
- Python Unit 2.5 for the corresponding circle and distance problems.

Learning Objectives

- Include and use mathematical functions from `<cmath>`.
- Calculate a circle's area and circumference.
- Calculate distance from the origin using the Pythagorean theorem.

Concept Explanation

C++ provides common mathematical functions through `<cmath>`, including `std::sqrt`, `std::pow`, and `std::round`. Prefer direct multiplication for simple squares when it improves clarity. Use floating-point types for measurements and check domain constraints before calling functions such as square root.

Worked Example

```cpp
#include <cmath>
#include <iostream>
int main() {
    const double radius = 3.0;
    const double pi = std::acos(-1.0);
    const double area = pi * radius * radius;
    const double circumference = 2.0 * pi * radius;
    const double distance = std::sqrt(3.0 * 3.0 + 4.0 * 4.0);
    std::cout << "Area: " << area << '\n';
    std::cout << "Circumference: " << circumference << '\n';
    std::cout << "Distance: " << distance << '\n';
}
```

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
Area: 28.2743
Circumference: 18.8496
Distance: 5
```

Common Mistakes

- Calling a math function without including `<cmath>` or qualifying it with `std::`.
- Using integer variables for measurements and losing fractional precision.
- Passing a negative value to `std::sqrt` without defining an error policy.

Activities

1. Change the radius to 5 and calculate the expected area and circumference before running.
2. Calculate the distance for coordinates (5, 12), then compare it with the expected 13.
3. Add a guard for a negative radicand and explain the chosen behavior.
4. Format the results to two decimal places and compare rounded output with the unrounded values.

Accessibility Practice

Read each intermediate variable and output label in source order. Keep units and labels in text output rather than communicating values through a visual chart.

Skills Checklist

- ☐ I can include and call functions from `<cmath>`.
- ☐ I can calculate circle area and circumference.
- ☐ I can calculate distance from coordinates using `std::sqrt`.
- ☐ I can explain floating-point precision and domain validation.
- ☐ I can format calculated measurements with meaningful labels and units.

5.5 Unit 3: Control Flow

Overview

This unit covers Boolean expressions, conditionals, lists or arrays, iteration, string operations. Follow the same problem progression as Python so that the main comparison is language syntax and semantics. Use the worked examples as starting points, then complete the activities independently.

Lesson 3.1: Boolean expressions

Reading References

- Official C++ language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Boolean expressions is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: use && and || for logical operations; compare with ==; use boolalpha when textual boolean output is required.

Worked Example

```
const int score = 82; const bool passed = score >= 60 && score <= 100; std::cout << std::boo
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare

actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
Valid passing score: true
```

Common Mistakes

- Use && and || for logical operations.
- Compare with ==.
- Use boolalpha when textual boolean output is required.

Activities

1. Trace the boolean expressions example and predict its output before compiling.
2. Adapt the example to grade thresholds and score collections and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.
4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

☐ I can explain the concept in C++ terminology.
☐ I can trace and predict the example.
☐ I can modify and run the code independently.
☐ I can identify and correct a common error.
☐ I can read source and output accessibly.

Lesson 3.2: Chained conditionals

Reading References

- Official C++ language guide and standard-library documentation.

- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Chained conditionals is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: test the most restrictive grade threshold first; include exact boundary values; make score ranges exhaustive and non-overlapping.

Worked Example

```
for (int score : {59, 60, 79, 80, 89, 90, 100}) { const char grade = score >= 90 ? 'A' : sc
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
59: F
60: D
79: C
80: B
89: B
90: A
100: A
```

Common Mistakes

- Test the most restrictive grade threshold first.
- Include exact boundary values.
- Make score ranges exhaustive and non-overlapping.

Activities

1. Trace the chained conditionals example and predict its output before compiling.
2. Adapt the example to grade thresholds and score collections and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.
4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

☐ I can explain the concept in C++ terminology.
☐ I can trace and predict the example.
☐ I can modify and run the code independently.
☐ I can identify and correct a common error.
☐ I can read source and output accessibly.

Lesson 3.3: Sequences and indexing

Reading References

- Official C++ language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Sequences and indexing is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: check vector bounds and emptiness; remember zero-based indexing; treat size as a count rather than a final index.

Worked Example

```
std::vector<int> scores{82, 91, 76, 88}; std::cout << "First: " << scores.at(0) << "; fourth
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
First: 82; fourth: 88; count: 4
```

Common Mistakes

- Check vector bounds and emptiness.
- Remember zero-based indexing.
- Treat size as a count rather than a final index.

Activities

1. Trace the sequences and indexing example and predict its output before compiling.
2. Adapt the example to grade thresholds and score collections and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.
4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

☐ I can explain the concept in C++ terminology.
☐ I can trace and predict the example.
☐ I can modify and run the code independently.
☐ I can identify and correct a common error.
☐ I can read source and output accessibly.

Lesson 3.4: For loops and iteration

Reading References

- Official C++ language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

For loops and iteration is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: avoid invalidating iterators while traversing; use an accumulator with a suitable type; define the empty-collection case.

Worked Example

```
const std::vector<int> scores{82, 91, 76, 88}; int sum = 0; for (int score : scores) sum +=
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
Total: 337; average: 84.25
```

Common Mistakes

- Avoid invalidating iterators while traversing.
- Use an accumulator with a suitable type.
- Define the empty-collection case.

Activities

1. Trace the for loops and iteration example and predict its output before compiling.
2. Adapt the example to grade thresholds and score collections and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.
4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

☐ I can explain the concept in C++ terminology.
☐ I can trace and predict the example.
☐ I can modify and run the code independently.
☐ I can identify and correct a common error.
☐ I can read source and output accessibly.

Lesson 3.5: String operations

Reading References

- Official C++ language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

String operations is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: treat std::string indexing as byte-oriented; pass unsigned char to cctype functions; remember substr takes a length.

Worked Example

```
std::string text = "Avery Studies Programming"; std::transform(text.begin(), text.end(), tex
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
avery studies programming; length: 25
```

Common Mistakes

- Treat std::string indexing as byte-oriented.
- Pass unsigned char to cctype functions.
- Remember substr takes a length.

Activities

1. Trace the string operations example and predict its output before compiling.
2. Adapt the example to grade thresholds and score collections and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.
4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain string operations using correct C++ terminology and distinguish it from the closest Python equivalent.
- ☐ I can trace the example and justify its output using C++ rules.
- ☐ I can implement the same requirement for grade thresholds and score collections without copying Python syntax.

- ☐ I can test a normal case, a boundary case, and a relevant invalid or failure case.
- ☐ I can compile the program, interpret diagnostics, and navigate source and terminal output using my access technology.

5.6 Unit 4: Data Structures

Overview

This unit covers sequence mutation, sorting, immutable records, maps, dictionaries, traversal. Follow the same problem progression as Python so that the main comparison is language syntax and semantics. Use the worked examples as starting points, then complete the activities independently.

Lesson 4.1: Mutable sequences

Reading References

- Official C++ language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Mutable sequences is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: check before pop_back; do not retain references across vector reallocation; use the iterator returned by erase.

Worked Example

```
std::vector<std::string> items{"Notebook", "Pencil"}; items.insert(items.begin() + 1, "Folde
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
Folder
Pencil
Eraser
```

Common Mistakes

- Check before pop_back.
- Do not retain references across vector reallocation.
- Use the iterator returned by erase.

Activities

1. Trace the mutable sequences example and predict its output before compiling.
2. Adapt the example to inventory records and key-value quantities and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.
4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

☐ I can explain the concept in C++ terminology.
☐ I can trace and predict the example.
☐ I can modify and run the code independently.
☐ I can identify and correct a common error.
☐ I can read source and output accessibly.

Lesson 4.2: Sorting and organizing data

Reading References

- Official C++ language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Sorting and organizing data is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: write a strict-weak-order comparator; sort by the field required by the task; use a compatible iterator category.

Worked Example

```
std::vector<int> quantities{10, 3, 4, 1}; std::sort(quantities.begin(), quantities.end()); s
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
10 4 3 1
```

Common Mistakes

- Write a strict-weak-order comparator.
- Sort by the field required by the task.
- Use a compatible iterator category.

Activities

1. Trace the sorting and organizing data example and predict its output before compiling.
2. Adapt the example to inventory records and key-value quantities and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.
4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the concept in C++ terminology.
- ☐ I can trace and predict the example.
- ☐ I can modify and run the code independently.
- ☐ I can identify and correct a common error.
- ☐ I can read source and output accessibly.

Lesson 4.3: Immutable records

Reading References

- Official C++ language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Immutable records is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: understand const members' effect on assignment; avoid dangling references; distinguish const objects from mutable indirection.

Worked Example

```
struct Coordinate { const int x; const int y; }; const Coordinate point{3, 4}; std::cout <<
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
Point: (3, 4)
```

Common Mistakes

- Understand const members' effect on assignment.
- Avoid dangling references.
- Distinguish const objects from mutable indirection.

Activities

1. Trace the immutable records example and predict its output before compiling.
2. Adapt the example to inventory records and key-value quantities and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.
4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

☐ I can explain the concept in C++ terminology.
☐ I can trace and predict the example.
☐ I can modify and run the code independently.
☐ I can identify and correct a common error.
☐ I can read source and output accessibly.

Lesson 4.4: Maps and key-value data

Reading References

- Official C++ language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Maps and key-value data is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: use find/contains for non-inserting lookup; remember map orders by key; avoid operator[] on const maps.

Worked Example

```
std::map<std::string, int> inventory{{"Notebook", 3}, {"Pencil", 10}, {"Folder", 4}}; invent
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
Notebook count: 4
```

Common Mistakes

- Use find/contains for non-inserting lookup.
- Remember map orders by key.
- Avoid operator[] on const maps.

Activities

1. Trace the maps and key-value data example and predict its output before compiling.
2. Adapt the example to inventory records and key-value quantities and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.
4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the concept in C++ terminology.
- ☐ I can trace and predict the example.
- ☐ I can modify and run the code independently.
- ☐ I can identify and correct a common error.
- ☐ I can read source and output accessibly.

Lesson 4.5: Collection traversal

Reading References

- Official C++ language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Collection traversal is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: prefer const auto& for read-only traversal; do not mutate the traversed container; initialize accumulators.

Worked Example

```cpp
const std::map<std::string, int> inventory{{"Notebook", 3}, {"Pencil", 10}, {"Folder", 4}};
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
Folder => 4
Notebook => 3
Pencil => 10
```

Common Mistakes

- Prefer const auto& for read-only traversal.
- Do not mutate the traversed container.
- Initialize accumulators.

Activities

1. Trace the collection traversal example and predict its output before compiling.
2. Adapt the example to inventory records and key-value quantities and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.
4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain collection traversal using correct C++ terminology and distinguish it from the closest Python equivalent.
- ☐ I can trace the example and justify its output using C++ rules.
- ☐ I can implement the same requirement for inventory records and key-value quantities without copying Python syntax.
- ☐ I can test a normal case, a boundary case, and a relevant invalid or failure case.
- ☐ I can compile the program, interpret diagnostics, and navigate source and terminal output using my access technology.

5.7 Unit 5: Input and Iteration

Overview

This unit covers input validation, sentinel loops, while loops, flags, break/continue, infinite-loop prevention. Follow the same problem progression as Python so that the main comparison is language syntax and semantics. Use the worked examples as starting points, then complete the activities independently.

Lesson 5.1: Input and validation

Reading References

- Official C++ language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Input and validation is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: validate extraction and the allowed range; handle conversion exceptions; define a policy for empty input.

Worked Example

```
std::istringstream input("16"); int age = 0; if (!(input >> age) || age < 0 || age > 130) st
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
Accepted age: 16
```

Common Mistakes

- Validate extraction and the allowed range.
- Handle conversion exceptions.
- Define a policy for empty input.

Activities

1. Trace the input and validation example and predict its output before compiling.
2. Adapt the example to validated input and sentinel-driven loops and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.
4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

☐ I can explain the concept in C++ terminology.
☐ I can trace and predict the example.
☐ I can modify and run the code independently.
☐ I can identify and correct a common error.
☐ I can read source and output accessibly.

Lesson 5.2: While loops

Reading References

- Official C++ language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

While loops is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: update the loop condition and sentinel correctly; decide whether the sentinel is data; stop cleanly on failed extraction.

Worked Example

```
std::istringstream input("1 2 3 0"); int value = 0, total = 0; while (input >> value && valu
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
Sum before sentinel: 6
```

Common Mistakes

- Update the loop condition and sentinel correctly.
- Decide whether the sentinel is data.
- Stop cleanly on failed extraction.

Activities

1. Trace the while loops example and predict its output before compiling.
2. Adapt the example to validated input and sentinel-driven loops and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.
4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the concept in C++ terminology.
- ☐ I can trace and predict the example.
- ☐ I can modify and run the code independently.
- ☐ I can identify and correct a common error.
- ☐ I can read source and output accessibly.

Lesson 5.3: Flags and break

Reading References

- Official C++ language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Flags and break is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: place break in the intended loop; handle nested-loop exit deliberately; report the not-found case.

Worked Example

```
const std::vector<std::string> names{"Avery", "Jordan", "Sam"}; bool found = false; for (cor
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
Jordan found: true
```

Common Mistakes

- Place break in the intended loop.
- Handle nested-loop exit deliberately.
- Report the not-found case.

Activities

1. Trace the flags and break example and predict its output before compiling.
2. Adapt the example to validated input and sentinel-driven loops and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.
4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

☐ I can explain the concept in C++ terminology.
☐ I can trace and predict the example.
☐ I can modify and run the code independently.
☐ I can identify and correct a common error.
☐ I can read source and output accessibly.

Lesson 5.4: Continue and filtering

Reading References

- Official C++ language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Continue and filtering is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not

syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: filter before producing output; ensure continue does not skip required state updates; verify comparison direction.

Worked Example

```
for (int value : {-2, 3, 0, 5, -1}) { if (value <= 0) continue; std::cout << value << ' '; }
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
3 5
```

Common Mistakes

- Filter before producing output.
- Ensure continue does not skip required state updates.
- Verify comparison direction.

Activities

1. Trace the continue and filtering example and predict its output before compiling.
2. Adapt the example to validated input and sentinel-driven loops and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.
4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

☐ I can explain the concept in C++ terminology.

☐ I can trace and predict the example.
☐ I can modify and run the code independently.
☐ I can identify and correct a common error.
☐ I can read source and output accessibly.

Lesson 5.5: Infinite-loop prevention

Reading References

- Official C++ language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Infinite-loop prevention is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: bound retries; increment the attempt count on every attempt; reset validation state before retrying.

Worked Example

```
int attempts = 0; bool valid = false; while (!valid && attempts < 3) { ++attempts; valid = a
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
Attempts: 2; valid: true
```

Common Mistakes

- Bound retries.

- Increment the attempt count on every attempt.
- Reset validation state before retrying.

Activities

1. Trace the infinite-loop prevention example and predict its output before compiling.
2. Adapt the example to validated input and sentinel-driven loops and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.
4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain infinite-loop prevention using correct C++ terminology and distinguish it from the closest Python equivalent.
- ☐ I can trace the example and justify its output using C++ rules.
- ☐ I can implement the same requirement for validated input and sentinel-driven loops without copying Python syntax.
- ☐ I can test a normal case, a boundary case, and a relevant invalid or failure case.
- ☐ I can compile the program, interpret diagnostics, and navigate source and terminal output using my access technology.

5.8 Unit 6: Functions

Overview

This unit covers definitions, parameters, arguments, return values, scope, collection parameters, variadic functions. Follow the same problem progression as Python so that the main comparison is language syntax and semantics. Use the worked examples as starting points, then complete the activities independently.

Lesson 6.1: Defining functions

Reading References

- Official C++ language guide and standard-library documentation.

- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Defining functions is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: declare functions before use; ensure every non-void path returns; match return types to the contract.

Worked Example

```
const auto calculate_total = [](double bill, double rate) { return bill * (1.0 + rate); }; s
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
Bill plus tip: $28.175
```

Common Mistakes

- Declare functions before use.
- Ensure every non-void path returns.
- Match return types to the contract.

Activities

1. Trace the defining functions example and predict its output before compiling.
2. Adapt the example to reusable score and inventory functions and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.

4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the concept in C++ terminology.
- ☐ I can trace and predict the example.
- ☐ I can modify and run the code independently.
- ☐ I can identify and correct a common error.
- ☐ I can read source and output accessibly.

Lesson 6.2: Parameters and arguments

Reading References

- Official C++ language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Parameters and arguments is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: pass large read-only objects by const reference; use non-const references only for intended mutation; avoid shadowing.

Worked Example

```
const auto label = [](const std::string& name, int score) { return name + ": " + std::to_str
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare

actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
Avery: 92
```

Common Mistakes

- Pass large read-only objects by const reference.
- Use non-const references only for intended mutation.
- Avoid shadowing.

Activities

1. Trace the parameters and arguments example and predict its output before compiling.
2. Adapt the example to reusable score and inventory functions and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.
4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

☐ I can explain the concept in C++ terminology.
☐ I can trace and predict the example.
☐ I can modify and run the code independently.
☐ I can identify and correct a common error.
☐ I can read source and output accessibly.

Lesson 6.3: Return values

Reading References

- Official C++ language guide and standard-library documentation.

- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Return values is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: never return a reference to a local; document failure/sentinel behavior; distinguish returning a value from printing.

Worked Example

```
const auto tip_total = [](double bill, double rate) { return bill * (1.0 + rate); }; const c
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
Total: $28.175
```

Common Mistakes

- Never return a reference to a local.
- Document failure/sentinel behavior.
- Distinguish returning a value from printing.

Activities

1. Trace the return values example and predict its output before compiling.
2. Adapt the example to reusable score and inventory functions and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.

4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the concept in C++ terminology.
- ☐ I can trace and predict the example.
- ☐ I can modify and run the code independently.
- ☐ I can identify and correct a common error.
- ☐ I can read source and output accessibly.

Lesson 6.4: Functions over collections

Reading References

- Official C++ language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Functions over collections is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: avoid copying a collection when a const reference suffices; define empty-input behavior; select an accumulator wide enough.

Worked Example

```
const auto average = [](const std::vector<int>& scores) { if (scores.empty()) return 0.0; co
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare

actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
Average: 84.25
```

Common Mistakes

- Avoid copying a collection when a const reference suffices.
- Define empty-input behavior.
- Select an accumulator wide enough.

Activities

1. Trace the functions over collections example and predict its output before compiling.
2. Adapt the example to reusable score and inventory functions and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.
4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

☐ I can explain the concept in C++ terminology.
☐ I can trace and predict the example.
☐ I can modify and run the code independently.
☐ I can identify and correct a common error.
☐ I can read source and output accessibly.

Lesson 6.5: Scope and variadic arguments

Reading References

- Official C++ language guide and standard-library documentation.

- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Scope and variadic arguments is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: respect block scope; write fold expressions that match the parameter pack; constrain or document mixed argument types.

Worked Example

```
const auto sum = []<typename... Values>(Values... values) { return (values + ...); }; std::c
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
Sum: 9
```

Common Mistakes

- Respect block scope.
- Write fold expressions that match the parameter pack.
- Constrain or document mixed argument types.

Activities

1. Trace the scope and variadic arguments example and predict its output before compiling.
2. Adapt the example to reusable score and inventory functions and preserve the same observable requirement using idiomatic C++20.

3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.
4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain scope and variadic arguments using correct C++ terminology and distinguish it from the closest Python equivalent.
- ☐ I can trace the example and justify its output using C++ rules.
- ☐ I can implement the same requirement for reusable score and inventory functions without copying Python syntax.
- ☐ I can test a normal case, a boundary case, and a relevant invalid or failure case.
- ☐ I can compile the program, interpret diagnostics, and navigate source and terminal output using my access technology.

5.9 Unit 7: Object-Oriented Programming

Overview

This unit covers types/classes, instances, fields, methods, inheritance or interfaces, composition. Follow the same problem progression as Python so that the main comparison is language syntax and semantics. Use the worked examples as starting points, then complete the activities independently.

Lesson 7.1: Types and object models

Reading References

- Official C++ language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Types and object models is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: distinguish a type declaration from an instance; understand struct-public and class-private defaults; initialize fields.

Worked Example

```
struct Item { std::string name; int quantity; }; const Item notebook{"Notebook", 3}; std::co
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
Notebook: 3
```

Common Mistakes

- Distinguish a type declaration from an instance.
- Understand struct-public and class-private defaults.
- Initialize fields.

Activities

1. Trace the types and object models example and predict its output before compiling.
2. Adapt the example to student records and object models and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.
4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the concept in C++ terminology.
- ☐ I can trace and predict the example.
- ☐ I can modify and run the code independently.
- ☐ I can identify and correct a common error.
- ☐ I can read source and output accessibly.

Lesson 7.2: Creating instances

Reading References

- Official C++ language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Creating instances is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: initialize objects to satisfy invariants; use . for objects and -> for pointers; account for vector reference invalidation.

Worked Example

```
struct Item { std::string name; int quantity; }; const std::vector<Item> items{{"Notebook",
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
Notebook: 3
Pencil: 10
Folder: 4
```

Common Mistakes

- Initialize objects to satisfy invariants.
- Use . for objects and -> for pointers.
- Account for vector reference invalidation.

Activities

1. Trace the creating instances example and predict its output before compiling.
2. Adapt the example to student records and object models and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.
4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

☐ I can explain the concept in C++ terminology.
☐ I can trace and predict the example.
☐ I can modify and run the code independently.
☐ I can identify and correct a common error.
☐ I can read source and output accessibly.

Lesson 7.3: Fields and methods

Reading References

- Official C++ language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Fields and methods is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: distinguish member and local state; mark read-only methods const; understand the implicit object context.

Worked Example

```
struct Counter { int value{0}; void increment() { ++value; } int current() const { return va
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
Clicks: 2
```

Common Mistakes

- Distinguish member and local state.
- Mark read-only methods const.
- Understand the implicit object context.

Activities

1. Trace the fields and methods example and predict its output before compiling.
2. Adapt the example to student records and object models and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.
4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the concept in C++ terminology.
- ☐ I can trace and predict the example.
- ☐ I can modify and run the code independently.
- ☐ I can identify and correct a common error.
- ☐ I can read source and output accessibly.

Lesson 7.4: Inheritance and interfaces

Reading References

- Official C++ language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Inheritance and interfaces is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: give polymorphic bases virtual destructors; match override signatures; avoid object slicing.

Worked Example

```cpp
struct Record { virtual ~Record() = default; virtual std::string kind() const { return "rec
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
student
```

Common Mistakes

- Give polymorphic bases virtual destructors.
- Match override signatures.
- Avoid object slicing.

Activities

1. Trace the inheritance and interfaces example and predict its output before compiling.
2. Adapt the example to student records and object models and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.
4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

☐ I can explain the concept in C++ terminology.
☐ I can trace and predict the example.
☐ I can modify and run the code independently.
☐ I can identify and correct a common error.
☐ I can read source and output accessibly.

Lesson 7.5: Composition

Reading References

- Official C++ language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Composition is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: model has-a relationships with composition; initialize members deliberately; avoid shared-ownership cycles.

Worked Example

```
struct Address { std::string city; }; struct Student { std::string name; Address address; };
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
Avery lives in Farmington
```

Common Mistakes

- Model has-a relationships with composition.
- Initialize members deliberately.
- Avoid shared-ownership cycles.

Activities

1. Trace the composition example and predict its output before compiling.
2. Adapt the example to student records and object models and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.
4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain composition using correct C++ terminology and distinguish it from the closest Python equivalent.
- ☐ I can trace the example and justify its output using C++ rules.
- ☐ I can implement the same requirement for student records and object models without copying Python syntax.
- ☐ I can test a normal case, a boundary case, and a relevant invalid or failure case.
- ☐ I can compile the program, interpret diagnostics, and navigate source and terminal output using my access technology.

5.10 Unit 8: Files and Exceptions

Overview

This unit covers read/write text, resource lifetime, error handling, JSON serialization and persistence. Follow the same problem progression as Python so that the main comparison is language syntax and semantics. Use the worked examples as starting points, then complete the activities independently.

Lesson 8.1: Reading text files

Reading References

- Official C++ language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Reading text files is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: check file-open state; remember getline removes the delimiter; resolve relative paths from the process working directory.

Worked Example

```
{ std::ofstream out("cpp_notes.txt"); out << "First line\nSecond line\n"; } std::ifstream in
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
First line
Second line
```

Common Mistakes

- Check file-open state.
- Remember getline removes the delimiter.
- Resolve relative paths from the process working directory.

Activities

1. Trace the reading text files example and predict its output before compiling.
2. Adapt the example to notes files and personal-data persistence and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.
4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

☐ I can explain the concept in C++ terminology.
☐ I can trace and predict the example.
☐ I can modify and run the code independently.
☐ I can identify and correct a common error.
☐ I can read source and output accessibly.

Lesson 8.2: Writing and appending files

Reading References

- Official C++ language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Writing and appending files is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: choose truncation or append mode deliberately; flush/close before dependent reads; check write state.

Worked Example

```
{ std::ofstream out("cpp_log.txt", std::ios::app); out << "Session complete\n"; } std::cout
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
Appended one record
```

Common Mistakes

- Choose truncation or append mode deliberately.
- Flush/close before dependent reads.
- Check write state.

Activities

1. Trace the writing and appending files example and predict its output before compiling.
2. Adapt the example to notes files and personal-data persistence and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.
4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the concept in C++ terminology.
- ☐ I can trace and predict the example.
- ☐ I can modify and run the code independently.
- ☐ I can identify and correct a common error.
- ☐ I can read source and output accessibly.

Lesson 8.3: Errors and exceptions

Reading References

- Official C++ language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Errors and exceptions is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: catch standard exceptions by const reference; handle expected input errors directly; avoid uncaught exceptions.

Worked Example

```
try { throw std::runtime_error("invalid score"); } catch (const std::exception& error) { std
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
Handled: invalid score
```

Common Mistakes

- Catch standard exceptions by const reference.
- Handle expected input errors directly.
- Avoid uncaught exceptions.

Activities

1. Trace the errors and exceptions example and predict its output before compiling.
2. Adapt the example to notes files and personal-data persistence and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.
4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

☐ I can explain the concept in C++ terminology.
☐ I can trace and predict the example.
☐ I can modify and run the code independently.
☐ I can identify and correct a common error.
☐ I can read source and output accessibly.

Lesson 8.4: Resource cleanup

Reading References

- Official C++ language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Resource cleanup is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: prefer RAII to manual cleanup; check stream state after writes; do not copy unique resource owners.

Worked Example

```
{ std::ofstream output("cpp_raii.txt"); output << "The stream closes automatically at scope
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
RAII released the file stream
```

Common Mistakes

- Prefer RAII to manual cleanup.
- Check stream state after writes.
- Do not copy unique resource owners.

Activities

1. Trace the resource cleanup example and predict its output before compiling.
2. Adapt the example to notes files and personal-data persistence and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.
4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the concept in C++ terminology.
- ☐ I can trace and predict the example.
- ☐ I can modify and run the code independently.
- ☐ I can identify and correct a common error.
- ☐ I can read source and output accessibly.

Lesson 8.5: JSON persistence

Reading References

- Official C++ language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

JSON persistence is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: distinguish JSON serialization from parsing; escape special characters; use a documented JSON library for general data.

Worked Example

```
const std::string json = "{\"name\":\"Avery\",\"score\":92}"; std::cout << json << '\n';
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
{"name":"Avery","score":92}
```

Common Mistakes

- Distinguish JSON serialization from parsing.
- Escape special characters.
- Use a documented JSON library for general data.

Activities

1. Trace the json persistence example and predict its output before compiling.
2. Adapt the example to notes files and personal-data persistence and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.
4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain json persistence using correct C++ terminology and distinguish it from the closest Python equivalent.
- ☐ I can trace the example and justify its output using C++ rules.
- ☐ I can implement the same requirement for notes files and personal-data persistence without copying Python syntax.
- ☐ I can test a normal case, a boundary case, and a relevant invalid or failure case.

☐ I can compile the program, interpret diagnostics, and navigate source and terminal output using my access technology.

5.11 Unit 9: Testing and Data Analysis

Overview

This unit covers unit tests, testing stateful types, randomness, text analysis, CSV processing. Follow the same problem progression as Python so that the main comparison is language syntax and semantics. Use the worked examples as starting points, then complete the activities independently.

Lesson 9.1: Testing functions

Reading References

- Official C++ language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Testing functions is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: do not rely solely on assert in release builds; test independent expected results; include boundary and invalid inputs.

Worked Example

```
const auto add = [](int a, int b) { return a + b; }; assert(add(2, 3) == 5); assert(add(-2,
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
Function tests passed
```

Common Mistakes

- Do not rely solely on assert in release builds.
- Test independent expected results.
- Include boundary and invalid inputs.

Activities

1. Trace the testing functions example and predict its output before compiling.
2. Adapt the example to unit tests, random values, and text analysis and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.
4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

☐ I can explain the concept in C++ terminology.
☐ I can trace and predict the example.
☐ I can modify and run the code independently.
☐ I can identify and correct a common error.
☐ I can read source and output accessibly.

Lesson 9.2: Testing stateful objects

Reading References

- Official C++ language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Testing stateful objects is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: assert state after each operation; isolate setup between tests; avoid order-dependent shared state.

Worked Example

```
struct Inventory { int quantity{0}; void add(int amount) { if (amount > 0) quantity += amoun
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
State test passed
```

Common Mistakes

- Assert state after each operation.
- Isolate setup between tests.
- Avoid order-dependent shared state.

Activities

1. Trace the testing stateful objects example and predict its output before compiling.
2. Adapt the example to unit tests, random values, and text analysis and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.
4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the concept in C++ terminology.
- ☐ I can trace and predict the example.
- ☐ I can modify and run the code independently.
- ☐ I can identify and correct a common error.
- ☐ I can read source and output accessibly.

Lesson 9.3: Random values

Reading References

- Official C++ language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Random values is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: seed engines for reproducibility; use standard distributions rather than modulo; do not promise identical sequences across libraries.

Worked Example

```
std::mt19937 engine(42); std::uniform_int_distribution<int> die(1, 6); for (int i = 0; i < 5
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
3 5 6 2 5
```

Common Mistakes

- Seed engines for reproducibility.
- Use standard distributions rather than modulo.
- Do not promise identical sequences across libraries.

Activities

1. Trace the random values example and predict its output before compiling.
2. Adapt the example to unit tests, random values, and text analysis and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.
4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

☐ I can explain the concept in C++ terminology.
☐ I can trace and predict the example.
☐ I can modify and run the code independently.
☐ I can identify and correct a common error.
☐ I can read source and output accessibly.

Lesson 9.4: Text analysis

Reading References

- Official C++ language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.

- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Text analysis is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: pass unsigned char to cctype; distinguish bytes from Unicode characters; document locale assumptions.

Worked Example

```
const std::string sample = "A student studies programming. Programming develops problem-solv
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
Letters: 65; spaces: 7
```

Common Mistakes

- Pass unsigned char to cctype.
- Distinguish bytes from Unicode characters.
- Document locale assumptions.

Activities

1. Trace the text analysis example and predict its output before compiling.
2. Adapt the example to unit tests, random values, and text analysis and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.
4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal,

and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the concept in C++ terminology.
- ☐ I can trace and predict the example.
- ☐ I can modify and run the code independently.
- ☐ I can identify and correct a common error.
- ☐ I can read source and output accessibly.

Lesson 9.5: CSV records

Reading References

- Official C++ language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

CSV records is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: a comma split is not a complete quoted-CSV parser; validate field count; ensure stoi consumed the whole field.

Worked Example

```
std::istringstream row("001,Avery,Student,92,true"); std::string id, first_name, last_name,
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
001: Avery Student, score 92, active true
```

Common Mistakes

- A comma split is not a complete quoted-CSV parser.
- Validate field count.
- Ensure stoi consumed the whole field.

Activities

1. Trace the csv records example and predict its output before compiling.
2. Adapt the example to unit tests, random values, and text analysis and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.
4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain csv records using correct C++ terminology and distinguish it from the closest Python equivalent.
- ☐ I can trace the example and justify its output using C++ rules.
- ☐ I can implement the same requirement for unit tests, random values, and text analysis without copying Python syntax.
- ☐ I can test a normal case, a boundary case, and a relevant invalid or failure case.
- ☐ I can compile the program, interpret diagnostics, and navigate source and terminal output using my access technology.

5.12 Unit 10: Project-Based Learning

Overview

This unit covers requirements, top-down design, text adventure, command-line to-do manager, accessible validation. Follow the same problem progression as Python so that the main comparison is language syntax and semantics. Use the worked examples as starting points, then complete the activities independently.

Lesson 10.1: Requirements and design

Reading References

- Official C++ language guide and standard-library documentation.

- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Requirements and design is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: write observable acceptance criteria before implementation; include failure and empty-data scenarios; keep scope deliverable.

Worked Example

```
const std::vector<std::string> requirements{"create records", "search by ID", "validate inpu
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
1. create records
2. search by ID
3. validate input
4. save changes
```

Common Mistakes

- Write observable acceptance criteria before implementation.
- Include failure and empty-data scenarios.
- Keep scope deliverable.

Activities

1. Trace the requirements and design example and predict its output before compiling.

2. Adapt the example to text adventure and command-line to-do projects and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.
4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

☐ I can explain the concept in C++ terminology.
☐ I can trace and predict the example.
☐ I can modify and run the code independently.
☐ I can identify and correct a common error.
☐ I can read source and output accessibly.

Lesson 10.2: Text adventure: five rooms and movement

Reading References

- C++ standard library: map
- C++ standard library: string
- Canonical shared project specification: Python Unit 10, Lesson 10.3.

Learning Objectives

- Represent all five canonical rooms with C++ structs and maps.
- Validate movement before changing the current room.
- Explain why the treasure room is not reachable until the key puzzle is implemented.

Concept Explanation

Use the same five-room graph in every language. A Room stores a description and a map from direction strings to room identifiers. The player begins in start_room. Part 1 includes the unconditional connections but leaves the north exit from locked_door_room unavailable. A failed map lookup must not change the current room.

Worked Example

```
#include <iostream>
#include <map>
```

```cpp
#include <string>

struct Room {
    std::string description;
    std::map<std::string, std::string> exits;
};

int main() {
    const std::map<std::string, Room> rooms{
        {"start_room", {"A dimly lit cave. A narrow passage leads north.", {{"north", "tunne
        {"tunnel", {"A damp tunnel. Passages lead south and east.", {{"south", "start_room"}
        {"clearing", {"A quiet clearing. An antechamber lies to the north.", {{"west", "tunn
        {"locked_door_room", {"A chamber with a heavy door to the north.", {{"south", "clear
        {"treasure_room", {"You found the treasure. You win!", {}}}
    };

    std::string current = "start_room";
    std::string command;
    while (true) {
        std::cout << "---\n" << rooms.at(current).description << "\nWhat do you want to do?\
        if (!std::getline(std::cin, command)) command = "quit";
        if (command == "quit") {
            std::cout << "Exiting game. Goodbye!\n";
            break;
        }
        const auto& exits = rooms.at(current).exits;
        const auto destination = exits.find(command);
        if (destination != exits.end()) current = destination->second;
        else std::cout << "You cannot go that way. Enter a listed direction or quit.\n";
    }
}
```

Expected Behavior or Output

Enter north, east, north, north, quit. The player reaches tunnel, clearing, and locked_door_room. The second north is rejected and the current room does not change. The treasure room exists but has no incoming exit in Part 1.

Common Mistakes

- Using map::at for a direction that may not exist.
- Updating current before checking the map lookup result.
- Defining a destination that is not one of the five room identifiers.
- Adding the treasure exit before implementing the key puzzle.

Activities

1. Trace north, east, north, north and record the current room.
2. Try west from start_room and verify that the room does not change.
3. Check every exit destination against the room map.
4. Explain why treasure_room cannot yet be reached.

Accessibility Practice

Announce the room description and prompt in a consistent order. The invalid-movement response explains what happened and how to continue. Compile and run from the terminal and inspect output sequentially with the learner's access technology.

Skills Checklist

☐ I can model all five canonical rooms.
☐ I can use a map to represent exits.
☐ I can reject invalid movement without changing state.
☐ I can explain why the treasure room is not yet reachable.

Lesson 10.3: Text adventure: shiny key and locked door

Reading References

- C++ standard library: vector
- C++ standard library: algorithm
- Canonical shared project specification: Python Unit 10, Lesson 10.3.

Learning Objectives

- Place the shiny key in start_room and transfer it to inventory exactly once.
- Require the key to be used in locked_door_room.
- Unlock the north exit only after the key puzzle succeeds.
- End the game when the player reaches treasure_room.

Concept Explanation

Keep the same five-room graph. Each room has a vector of items. Picking up the shiny key removes it from the room and appends it to the inventory. The use command checks both the player's location and inventory. A north command from locked_door_room is rejected while the door is locked.

Worked Example

```
#include <algorithm>
#include <iostream>
#include <map>
#include <string>
#include <vector>

struct Room {
    std::string description;
    std::map<std::string, std::string> exits;
    std::vector<std::string> items;
};

int main() {
    std::map<std::string, Room> rooms{
        {"start_room", {"A dimly lit cave.", {{"north", "tunnel"}}, {"shiny key"}}},
        {"tunnel", {"A damp tunnel. Passages lead south and east.", {{"south", "start_room"}
        {"clearing", {"A quiet clearing. An antechamber lies to the north.", {{"west", "tunn
        {"locked_door_room", {"A chamber with a heavy door to the north.", {{"south", "clear
        {"treasure_room", {"You found the treasure. You win!", {}, {}}}
    };

    std::string current = "start_room";
    std::vector<std::string> inventory;
    bool door_locked = true;
    std::string command;

    while (true) {
        Room& room = rooms.at(current);
        std::cout << "\n" << room.description << "\n";
        if (current == "treasure_room") break;
        if (!room.items.empty()) {
            std::cout << "You see: ";
            for (const auto& item : room.items) std::cout << item << " ";
            std::cout << "\n";
        }
        std::cout << "Your inventory: ";
        if (inventory.empty()) std::cout << "empty";
        else for (const auto& item : inventory) std::cout << item << " ";
        std::cout << "\nWhat do you want to do?\n";

        if (!std::getline(std::cin, command)) command = "quit";
        if (command == "quit") {
            std::cout << "You quit the adventure.\n";
            break;
```

```
        } else if (command == "take shiny key") {
            auto item = std::find(room.items.begin(), room.items.end(), "shiny key");
            if (item != room.items.end()) {
                room.items.erase(item);
                inventory.push_back("shiny key");
                std::cout << "You picked up the shiny key.\n";
            } else std::cout << "There is no shiny key here to take.\n";
        } else if (command == "use shiny key") {
            if (current != "locked_door_room") std::cout << "You must be at the locked door
            else if (std::find(inventory.begin(), inventory.end(), "shiny key") == inventory
                std::cout << "You do not have the shiny key.\n";
            else {
                door_locked = false;
                std::cout << "You unlocked the door!\n";
            }
        } else if (current == "locked_door_room" && command == "north" && door_locked) {
            std::cout << "The door is locked! You need to use the shiny key here.\n";
        } else {
            auto destination = room.exits.find(command);
            if (destination != room.exits.end()) current = destination->second;
            else std::cout << "Invalid command. Enter north, south, east, west, take shiny k
        }
    }
}
```

Expected Behavior or Output

Enter north, east, north, north, south, west, south, take shiny key, north, east, north, use shiny key, north. The first north from locked_door_room is rejected. Taking the key removes it from the room and adds it to inventory. Using it outside locked_door_room is rejected. Using it in that room unlocks the door; the final north reaches treasure_room and prints the win description.

Common Mistakes

- Erasing the key without adding it to inventory, or adding it without erasing it.
- Unlocking the door without checking the current room.
- Unlocking the door without checking inventory.
- Allowing north movement before door_locked becomes false.
- Using map::at for an unvalidated command.

Activities

1. Run the complete puzzle sequence and record room, items, inventory, and door state.
2. Attempt to take the key twice and verify no duplicate is added.

3. Try to use the key outside locked_door_room.
4. Try north before and after unlocking.
5. Add a help command with all supported commands.

Accessibility Practice

Announce the room, visible items, inventory, and prompt as separate text lines. Error messages explain the failed action. Test the complete interaction with keyboard-only input and sequential screen-reader output.

Skills Checklist

- ☐ I can model the five canonical rooms.
- ☐ I can transfer the key from a room to inventory.
- ☐ I can prevent duplicate pickup.
- ☐ I can require key use in locked_door_room.
- ☐ I can block and unlock the north exit.
- ☐ I can end the game in treasure_room.

Lesson 10.4: Command-line to-do manager

Reading References

- Official C++ language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Command-line to-do manager is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: use stable task IDs rather than vector positions; store completion separately; persist changes deliberately.

Worked Example

```
struct Task { int id; std::string title; std::string description; bool done; std::string cre
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
1: Read lesson [1] 2026-11-01
```

Common Mistakes

- Use stable task IDs rather than vector positions.
- Store completion separately.
- Persist changes deliberately.

Activities

1. Trace the command-line to-do manager example and predict its output before compiling.
2. Adapt the example to text adventure and command-line to-do projects and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.
4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

☐ I can explain the concept in C++ terminology.
☐ I can trace and predict the example.
☐ I can modify and run the code independently.
☐ I can identify and correct a common error.
☐ I can read source and output accessibly.

Lesson 10.5: Accessible UX and validation

Reading References

- Official C++ language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Accessible UX and validation is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: validate each parsed argument; make errors identify the rejected field; keep prompts and status output predictable.

Worked Example

```
std::istringstream command("add Read chapter"); std::string action; command >> action; std::
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
Added task: Read chapter
```

Common Mistakes

- Validate each parsed argument.
- Make errors identify the rejected field.
- Keep prompts and status output predictable.

Activities

1. Trace the accessible ux and validation example and predict its output before compiling.
2. Adapt the example to text adventure and command-line to-do projects and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.
4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain accessible ux and validation using correct C++ terminology and distinguish it from the closest Python equivalent.
- ☐ I can trace the example and justify its output using C++ rules.
- ☐ I can implement the same requirement for text adventure and command-line to-do projects without copying Python syntax.
- ☐ I can test a normal case, a boundary case, and a relevant invalid or failure case.
- ☐ I can compile the program, interpret diagnostics, and navigate source and terminal output using my access technology.

5.13 Unit 11: Capstone

Overview

This unit covers accessible personal-data manager, model, persistence, CRUD, tests, documentation. Follow the same problem progression as Python so that the main comparison is language syntax and semantics. Use the worked examples as starting points, then complete the activities independently.

Lesson 11.1: Capstone requirements

Reading References

- Official C++ language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Capstone requirements is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: set measurable acceptance criteria; constrain scope; include accessibility and data portability requirements.

Worked Example

```
std::cout << "Personal data manager requirements: create, list, search, update, delete, vali
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
Personal data manager requirements: create, list, search, update, delete, validate, persist.
```

Common Mistakes

- Set measurable acceptance criteria.
- Constrain scope.
- Include accessibility and data portability requirements.

Activities

1. Trace the capstone requirements example and predict its output before compiling.
2. Adapt the example to personal records with stable IDs and CRUD operations and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.

4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the concept in C++ terminology.
- ☐ I can trace and predict the example.
- ☐ I can modify and run the code independently.
- ☐ I can identify and correct a common error.
- ☐ I can read source and output accessibly.

Lesson 11.2: Data model and storage

Reading References

- Official C++ language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Data model and storage is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: use stable IDs rather than vector offsets; document the persisted schema; minimize sensitive sample data.

Worked Example

```
struct Record { int id; std::string title; std::string description; std::string category; st
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
1,Course project,Personal data manager,education,2026-11-01,2026-11-01
```

Common Mistakes

- Use stable IDs rather than vector offsets.
- Document the persisted schema.
- Minimize sensitive sample data.

Activities

1. Trace the data model and storage example and predict its output before compiling.
2. Adapt the example to personal records with stable IDs and CRUD operations and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.
4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

☐ I can explain the concept in C++ terminology.
☐ I can trace and predict the example.
☐ I can modify and run the code independently.
☐ I can identify and correct a common error.
☐ I can read source and output accessibly.

Lesson 11.3: Create/read/update/delete

Reading References

- Official C++ language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Create/read/update/delete is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: locate records by ID; distinguish not-found from success; validate replacement fields before mutating state.

Worked Example

```
struct Record { int id; std::string title; std::string description; std::string category; st
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
1: Updated course project
```

Common Mistakes

- Locate records by ID.
- Distinguish not-found from success.
- Validate replacement fields before mutating state.

Activities

1. Trace the create/read/update/delete example and predict its output before compiling.
2. Adapt the example to personal records with stable IDs and CRUD operations and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.
4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the concept in C++ terminology.
- ☐ I can trace and predict the example.
- ☐ I can modify and run the code independently.
- ☐ I can identify and correct a common error.
- ☐ I can read source and output accessibly.

Lesson 11.4: Validation and testing

Reading References

- Official C++ language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Validation and testing is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: test duplicate IDs and missing records; use temporary test files; make tests independent of execution order.

Worked Example

```cpp
const auto valid_email = [](const std::string& email) { const auto at = email.find('@'); ret
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
Validation tests passed
```

Common Mistakes

- Test duplicate IDs and missing records.
- Use temporary test files.
- Make tests independent of execution order.

Activities

1. Trace the validation and testing example and predict its output before compiling.
2. Adapt the example to personal records with stable IDs and CRUD operations and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.
4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

☐ I can explain the concept in C++ terminology.
☐ I can trace and predict the example.
☐ I can modify and run the code independently.
☐ I can identify and correct a common error.
☐ I can read source and output accessibly.

Lesson 11.5: Documentation and presentation

Reading References

- Official C++ language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain this concept using C++ terminology.
- Trace a short program and predict its behavior before execution.
- Modify and test the example independently.

Concept Explanation

Documentation and presentation is taught through the shared course datasets and requirements. The Python curriculum supplies a common problem context, not syntax to translate mechanically. In C++, make the type system, object lifetime, standard-library contracts, ownership, and failure behavior explicit. This lesson focuses on: document exact build and run commands; state actual serialization support accurately; disclose limitations and accessibility coverage.

Worked Example

```
std::cout << "Build: c++ -std=c++20 -Wall -Wextra -Wpedantic -Werror main.cpp -o app\n"; sto
```

Trace: predict the output before compiling. Identify declarations, values, control-flow decisions, and state changes in order. Then compile and compare actual output with the prediction. The lesson example is intended to be independently compilable.

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (g++ 13.3, C++20; the text of the output does not depend on the operating system):

```
Build: c++ -std=c++20 -Wall -Wextra -Wpedantic -Werror main.cpp -o app
Document: requirements, data format, tests, limitations, accessibility
```

Common Mistakes

- Document exact build and run commands.
- State actual serialization support accurately.
- Disclose limitations and accessibility coverage.

Activities

1. Trace the documentation and presentation example and predict its output before compiling.
2. Adapt the example to personal records with stable IDs and CRUD operations and preserve the same observable requirement using idiomatic C++20.
3. Add one boundary case and one invalid or missing-data case; state the expected result and implement the handling.

4. Write a short test or trace table, then explain the C++ type, lifetime, ownership, iterator, or error-handling rule that matters here.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Runnable Reference Implementation

The repository includes a tested C++ domain and JSON-storage reference core in capstone/README.md. Install the JSON development dependency described there, then run the acceptance command:

```
g++ -std=c++20 -Wall -Wextra -Werror -ICPP/capstone CPP/capstone/test_records.cpp
-o /tmp/cpp-capstone-tests && /tmp/cpp-capstone-tests
```

The reference core does not replace the requirement to build and evaluate an accessible user interface.

Skills Checklist

- ☐ I can explain documentation and presentation using correct C++ terminology and distinguish it from the closest Python equivalent.
- ☐ I can trace the example and justify its output using C++ rules.
- ☐ I can implement the same requirement for personal records with stable IDs and CRUD operations without copying Python syntax.
- ☐ I can test a normal case, a boundary case, and a relevant invalid or failure case.
- ☐ I can compile the program, interpret diagnostics, and navigate source and terminal output using my access technology.

Course-Wide Assessment and Portfolio

Submit runnable exercises, prediction notes, debugging notes, tests, and the text adventure and personal-data manager projects. Assessment focuses on correctness, idiomatic C++, code explanation, independent debugging, and accessible interaction—not speed or visual editor use.

AI and Academic Integrity

AI may be used only for personal conceptual explanations and reinforcement when the instructor permits. Do not use AI to write, complete, autocomplete, repair, or generate course code or assessment solutions. Students must explain every submitted statement independently.

Further Learning

Use the official C++ language specification, standard-library documentation, and the linked setup resources. Verify APIs against the documentation for the installed toolchain.

Part VI

C-SHARP

Chapter 6

C# Curriculum

Before you start: complete Unit P: Computing Foundations Primer. It explains the language-neutral vocabulary and concepts used in every language track, using non-visual examples.

C# is a modern managed language for .NET services, web apps, desktop software, cloud systems, and Unity games.

Course structure

A complete introductory sequence aligned to Python Units 0–11: setup; foundations; numeric computation; control flow; data structures; input and iteration; functions; object-oriented design; files and exceptions; testing and data analysis; projects; and capstone. The same core problem contexts recur across languages so students focus on syntax, types, runtime behavior, libraries, and idioms.

Environment

Install the .NET 10 SDK (C# 14 toolchain); create with `dotnet new console`; run with `dotnet run`.

Complete the shared IDE setup lesson and terminal prerequisite. The course supports keyboard-first work with screen readers, braille displays, magnification, or other access technology.

- Complete lessons
- Topic index

Lesson structure and example verification

The complete course has four structured Unit 0 setup lessons and 53 numbered lessons across Units 1–11. The numbered lessons use the shared Python course datasets and problem contexts, with C#-specific language features and .NET APIs. Each numbered lesson includes three targeted common mistakes, four practice activities, and five mastery criteria.

- C# lesson example validation script
- GitHub Actions .NET 10 / C# 14 workflow

The automated check builds and executes every fenced C# example as a separate .NET 10 console program with nullable analysis enabled and compiler warnings treated as errors. GitHub Actions run 41320247156 passed with .NET SDK 10.0.403; all 53 examples compiled, linked, and exited successfully. This is a smoke-execution check, not full behavioral validation: expected-output assertions, broader invalid-input/file fixtures, and the complete capstone application still require testing. Hands-on accessibility validation remains separate.

GUI, visualization, and workbench accessibility

See the shared GUI, visualization, and workbench accessibility reference for screen-reader support by toolkit, platform-specific activation settings, recommended alternatives, and interfaces that must not be required without an equivalent accessible route.

AI Policy

AI may be used only for personal conceptual understanding and reinforcement. It must not write, complete, autocomplete, repair, or generate course code or assessment solutions.

Recommended introductory books

- Whitaker, R. B. *The C# Player's Guide*, 5th edition. RB Whitaker, 2021. A challenge-driven introduction to programming fundamentals, methods, types, object-oriented programming, and .NET concepts. Author's book page.
- Gaddis, Tony. *Starting Out with Visual C#*, 5th edition. Pearson. A traditional introductory textbook for readers with no prior programming experience; editions may target older versions of C# and Visual Studio. Pearson catalog.

Suggested use: *The C# Player's Guide* is the primary language companion for this course. Check the C# language version and .NET target in any book against

the curriculum's current SDK baseline. Older Visual Studio setup procedures should not override the repository's IDE guidance.

6.1 CSHARP Lessons

The curriculum is organized as one Markdown file per unit.

- accessible environment and toolchain
- foundations of computing
- numeric computations
- control flow
- data structures
- input and iteration
- functions
- object oriented programming
- files and exceptions
- testing and data analysis
- project based learning
- capstone

6.2 Unit 0: Accessible Environment and Toolchain

Course overview

Before you start: complete Unit P: Computing Foundations Primer. It explains the language-neutral vocabulary and concepts used in every language track, using non-visual examples.

Course Overview

C# is a modern managed language for .NET services, web applications, desktop software, cloud systems, and Unity games.

This course intentionally parallels the original Python curriculum. It reuses problem contexts—greetings, profiles, a calculator, grade decisions, score collections, inventory, functions, text files, tests, text analysis, CSV, a to-do list, and a personal-data manager—so the principal new learning target is the C# language. The syntax and implementation remain idiomatic, not a mechanical translation.

Unit 0: Accessible Environment and Toolchain

Overview

Install and verify the .NET SDK, create a reproducible console project, configure an editor, and test the complete keyboard and assistive-technology workflow. The .NET SDK includes the compiler, runtime tooling, project system, and build commands. A C# source file is normally built as part of a project rather than compiled as a standalone script.

Lesson 0.1: Install and Verify C#

Reading References

- Microsoft Learn: .NET installation and C# language documentation.
- .NET SDK command-line reference.

Learning Objectives

- Install and identify the .NET SDK.
- Create and run a minimal C# console application.
- Distinguish SDK, runtime, compiler, and program output.

Concept Explanation Install the .NET 10 SDK using the official installer or supported package manager. Run `dotnet --info` to inspect installed SDKs and runtimes. Create a project with `dotnet new console -n CSharpCourse`, enter the project folder, and run `dotnet run`. Installing an editor extension alone does not install the SDK.

Worked Example

```
dotnet new console -n CSharpCourse
cd CSharpCourse
dotnet run
```

Expected default output: `Hello, World!`

Expected Behavior or Output

`dotnet run` builds the project and prints the default template output:

```
Hello, World!
```

Common Mistakes

- Installing only C# syntax highlighting without the .NET SDK.
- Running dotnet commands outside the intended project directory.
- Confusing the installed runtime with the SDK needed to build applications.

Activities

1. Install the .NET 10 SDK and record the output of `dotnet --info`.
2. Create the CSharpCourse console project and run it unchanged.
3. Change the output message and rebuild with `dotnet run`.
4. Run `dotnet --list-sdks` and `dotnet --list-runtimes`; explain the difference.

Accessibility Practice Use keyboard navigation to reach the terminal, enter each command, and read the complete output sequentially. Confirm that the editor announces punctuation, braces, indentation, and build diagnostics.

Skills Checklist

- ☐ I can install and identify the .NET 10 SDK.
- ☐ I can create a console project with dotnet new.
- ☐ I can run a project with dotnet run.
- ☐ I can distinguish SDK, runtime, compiler, and application output.
- ☐ I can read terminal output and diagnostics using my access technology.

Lesson 0.2: Workspace and First Run

Reading References

- .NET project and CLI documentation.
- Shared terminal navigation prerequisite.

Learning Objectives

- Create and navigate a project workspace.
- Identify source, project, and generated build files.
- Run build and execution commands from the project root.

Concept Explanation A .NET project is described by a .csproj file. Source files such as Program.cs contain C# code; bin contains build output and obj contains intermediate files. The current working directory determines how relative file paths are resolved. Use dotnet build to compile and dotnet run to build and execute.

Worked Example

```
mkdir csharp-course
cd csharp-course
dotnet new console
dotnet build
dotnet run
```

Expected Behavior or Output

`dotnet build` ends with a summary that says the build succeeded, and `dotnet run` prints the default template output `Hello, World!`.

Common Mistakes

- Editing a source file in one folder while building a different project.
- Deleting or manually editing generated files under bin or obj.
- Assuming relative data-file paths are resolved from the source file location.

Activities

1. Create a course workspace and a console project inside it.
2. Identify Program.cs, the project file, bin, and obj.
3. Build with dotnet build and run with dotnet run.
4. Create a text data file and verify how its relative path changes when the working directory changes.

Accessibility Practice Navigate the project tree using keyboard commands and verify file names through text output. Move between source, terminal, build diagnostics, and project files without mouse-only operations.

Skills Checklist

☐ I can create and navigate a .NET project workspace.
☐ I can identify source and project files.
☐ I can distinguish generated build output from source.
☐ I can build and run from the project root.
☐ I can explain current-working-directory path behavior.

Lesson 0.3: Editor and Accessibility Workflow

Reading References

- Shared IDE setup lesson.
- Documentation for the chosen editor and C# language tooling.

Learning Objectives

- Configure C# language services in the selected editor.
- Navigate project files, diagnostics, and terminal output by keyboard.
- Distinguish editor assistance from compiler validation.

Concept Explanation

A text editor edits source code; an IDE adds integrated project navigation, language services, build, and debugging tools. VS Code, VSCodium, and Theia

can support C# through compatible extensions and .NET tooling, but their marketplaces and extension availability differ. A language server provides completion and diagnostics; it is not a substitute for the .NET SDK or a successful build.

Worked Example

Open the project folder, install the C# extension supported by the chosen editor, select the installed .NET SDK, and introduce a temporary syntax error. Navigate to the diagnostic, read its location and message, correct the source, and build from the integrated terminal.

Expected Behavior or Output

The editor underlines the syntax error and lists it with its location and message. The message from `dotnet build` in the integrated terminal names the same file and line. After you correct the source, the build succeeds.

Common Mistakes

- Opening an individual source file instead of the project root.
- Assuming an extension installs the SDK or compiler.
- Treating syntax coloring or completion as proof that the project builds.

Activities

1. Open the complete project folder in VS Code, VSCodium, or Theia.
2. Verify that C# language services recognize the project and target framework.
3. Navigate to a deliberate syntax error using the keyboard and read its diagnostic.
4. Correct the error and build from both the integrated and system terminals.

Accessibility Practice Test file navigation, symbol navigation, punctuation verbosity, indentation announcements, error lists, terminal focus, and build output with the learner's screen reader, braille display, magnifier, or other access technology.

Skills Checklist

- ☐ I can open the project root in my chosen editor.
- ☐ I can configure C# language services.
- ☐ I can navigate diagnostics and source by keyboard.
- ☐ I can build from an integrated terminal.
- ☐ I can explain why editor assistance is not compiler validation.

Lesson 0.4: Project Environments, Dependencies, and Terminal Usage

Reading References

- .NET project file and package management documentation.
- NuGet package management and dotnet CLI references.

Learning Objectives

- Identify the project target framework and dependency declarations.
- Restore packages and build reproducibly.
- Run project commands from the correct directory.

Concept Explanation A .NET project records its target framework and package references in the .csproj file. NuGet manages external packages; dotnet restore resolves dependencies and creates assets in obj. dotnet build compiles the project, dotnet test runs tests, and dotnet run executes an application. Keep dependencies project-scoped and review package versions rather than installing arbitrary binaries globally.

Worked Example

```
dotnet restore
dotnet build
dotnet test
dotnet run
```

Expected Behavior or Output

Each command reports success or failure on its own status lines. `dotnet restore` and `dotnet build` prepare and compile the project, `dotnet run` starts it, and `dotnet test` runs tests only when the project contains a test project.

Common Mistakes

- Running restore or build from a directory that does not contain the project or solution.
- Adding a package without checking its target framework and version compatibility.
- Editing generated NuGet assets instead of the project file.

Activities

1. Inspect the target framework and package references in the project file.
2. Run dotnet restore and identify the generated assets file.
3. Run dotnet build and dotnet test; distinguish their purposes.

4. Add one approved package using dotnet add package, then restore and build again.

Accessibility Practice Read the project file as text and verify that package names and versions are available to assistive technology. Navigate restore/build/test diagnostics by keyboard and retain the exact command output.

Skills Checklist

- ☐ I can identify the target framework in a project file.
- ☐ I can explain the role of NuGet package references.
- ☐ I can restore dependencies with dotnet restore.
- ☐ I can distinguish build, test, and run commands.
- ☐ I can execute project commands from the correct working directory.

6.3 Unit 1: Foundations of Computing

Overview

This unit covers computer science, hardware/software, algorithms, first program, variables, primitive types. Follow the same problem progression as Python so that the main comparison is language syntax and semantics. Use the worked examples as starting points, then complete the activities independently.

Lesson 1.1: Computer science and problem solving

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

C# executes typed statements under the .NET runtime. Focus on the concept named in this lesson and connect it to observable input, processing, and output. The compiler checks language rules, while tests are needed to establish behavior.

Worked Example

```
Console.WriteLine("Computer science turns problems into precise, testable steps.");
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
Computer science turns problems into precise, testable steps.
```

Common Mistakes

- Confusing source syntax with the behavior of the running application.
- Assuming compilation alone proves that the program meets the requirement.
- Ignoring C# type rules or compiler diagnostics relevant to this lesson.

Activities

1. Trace the worked example line by line and write its exact output before running it.
2. Change one value in the example and predict the result before rebuilding.
3. Test one boundary or invalid case that is relevant to this lesson's concept.
4. Explain the C# language or .NET behavior demonstrated, without relying on syntax coloring.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

☐ I can explain the lesson's concept using accurate C# terminology.
☐ I can trace and predict the worked example.
☐ I can modify the example while preserving its intended behavior.
☐ I can test a relevant boundary or error case.
☐ I can read the source and diagnostics using my access technology.

Lesson 1.2: Your first program

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

C# executes typed statements under the .NET runtime. Focus on the concept named in this lesson and connect it to observable input, processing, and output. The compiler checks language rules, while tests are needed to establish behavior.

Worked Example

```
Console.WriteLine("Hello, World!");
Console.WriteLine("This is my first C# console program.");
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
Hello, World!
This is my first C# console program.
```

Common Mistakes

- Confusing source syntax with the behavior of the running application.
- Assuming compilation alone proves that the program meets the requirement.
- Ignoring C# type rules or compiler diagnostics relevant to this lesson.

Activities

1. Trace the worked example line by line and write its exact output before running it.

2. Change one value in the example and predict the result before rebuilding.
3. Test one boundary or invalid case that is relevant to this lesson's concept.
4. Explain the C# language or .NET behavior demonstrated, without relying on syntax coloring.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can explain the lesson's concept using accurate C# terminology.
- ☐ I can trace and predict the worked example.
- ☐ I can modify the example while preserving its intended behavior.
- ☐ I can test a relevant boundary or error case.
- ☐ I can read the source and diagnostics using my access technology.

Lesson 1.3: Variables and data types

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

C# executes typed statements under the .NET runtime. Focus on the concept named in this lesson and connect it to observable input, processing, and output. The compiler checks language rules, while tests are needed to establish behavior.

Worked Example

```
string name = "Avery"; int age = 16; double heightM = 1.72; string favoriteSubject = "Comput
Console.WriteLine($"{name}, age {age}, height {heightM:F2} m; favorite subject: {favoriteSub
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
Avery, age 16, height 1.72 m; favorite subject: Computer Science.
```

Common Mistakes

- Confusing source syntax with the behavior of the running application.
- Assuming compilation alone proves that the program meets the requirement.
- Ignoring C# type rules or compiler diagnostics relevant to this lesson.

Activities

1. Trace the worked example line by line and write its exact output before running it.
2. Change one value in the example and predict the result before rebuilding.
3. Test one boundary or invalid case that is relevant to this lesson's concept.
4. Explain the C# language or .NET behavior demonstrated, without relying on syntax coloring.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

☐ I can explain the lesson's concept using accurate C# terminology.
☐ I can trace and predict the worked example.
☐ I can modify the example while preserving its intended behavior.
☐ I can test a relevant boundary or error case.
☐ I can read the source and diagnostics using my access technology.

6.4 Unit 2: Numeric Computations

Overview

This unit covers development process, expressions, assignment, input/output, conversions, arithmetic, math library. Follow the same problem progression as Python so that the main comparison is language syntax and semantics. Use the worked examples as starting points, then complete the activities independently.

Lesson 2.1: Design and tracing

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

C# uses statically typed expressions and the .NET Base Class Library. Follow values from input through conversion and calculation; use explicit types, named intermediate values, and a stated policy for parsing, culture, precision, and formatting.

Worked Example

```
double bill = 24.50, tipRate = 0.15;
double tip = bill * tipRate, total = bill + tip;
Console.WriteLine($"Bill: {bill:F2}; tip: {tip:F2}; total: {total:F2}");
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
Bill: 24.50; tip: 3.67; total: 28.18
```

Common Mistakes

- Using the wrong numeric type or conversion for the value being processed.
- Ignoring input parsing failures or culture-dependent formatting.
- Relying on implicit precedence or rounding without stating the intended calculation.

Activities

1. Calculate the expected result for the canonical bill, score, or measurement before running the example.

2. Change one canonical input and verify which intermediate and final values change.
3. Test malformed text, a boundary value, or a different numeric representation as appropriate.
4. Explain the selected type, conversion, precedence, or formatting rule in C#.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

☐ I can choose appropriate numeric types and conversions.
☐ I can explain the expression and calculation order.
☐ I can reproduce the canonical result.
☐ I can handle an invalid or boundary input.
☐ I can explain the relevant .NET numeric or Math API.

Lesson 2.2: Expressions and assignment

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

C# uses statically typed expressions and the .NET Base Class Library. Follow values from input through conversion and calculation; use explicit types, named intermediate values, and a stated policy for parsing, culture, precision, and formatting.

Worked Example

```
double score = 70.0;
double adjusted = (score + 12.5) * 1.10;
```

```
score += 12.5;
Console.WriteLine($"Adjusted: {adjusted:F2}; updated score: {score:F2}");
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
Adjusted: 90.75; updated score: 82.50
```

Common Mistakes

- Using the wrong numeric type or conversion for the value being processed.
- Ignoring input parsing failures or culture-dependent formatting.
- Relying on implicit precedence or rounding without stating the intended calculation.

Activities

1. Calculate the expected result for the canonical bill, score, or measurement before running the example.
2. Change one canonical input and verify which intermediate and final values change.
3. Test malformed text, a boundary value, or a different numeric representation as appropriate.
4. Explain the selected type, conversion, precedence, or formatting rule in C#.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

☐ I can choose appropriate numeric types and conversions.
☐ I can explain the expression and calculation order.
☐ I can reproduce the canonical result.
☐ I can handle an invalid or boundary input.
☐ I can explain the relevant .NET numeric or Math API.

Lesson 2.3: Input and output

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

C# uses statically typed expressions and the .NET Base Class Library. Follow values from input through conversion and calculation; use explicit types, named intermediate values, and a stated policy for parsing, culture, precision, and formatting.

Worked Example

```
Console.SetIn(new StringReader("Avery\n6\n7\n"));
Console.Write("Name: "); string name = Console.ReadLine() ?? "";
int first = int.Parse(Console.ReadLine() ?? "0"), second = int.Parse(Console.ReadLine() ?? '
Console.WriteLine($"Hello, {name}. Sum: {first + second}");
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
Name: Hello, Avery. Sum: 13
```

Common Mistakes

- Using the wrong numeric type or conversion for the value being processed.
- Ignoring input parsing failures or culture-dependent formatting.
- Relying on implicit precedence or rounding without stating the intended calculation.

Activities

1. Calculate the expected result for the canonical bill, score, or measurement before running the example.
2. Change one canonical input and verify which intermediate and final values change.
3. Test malformed text, a boundary value, or a different numeric representation as appropriate.
4. Explain the selected type, conversion, precedence, or formatting rule in C#.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can choose appropriate numeric types and conversions.
- ☐ I can explain the expression and calculation order.
- ☐ I can reproduce the canonical result.
- ☐ I can handle an invalid or boundary input.
- ☐ I can explain the relevant .NET numeric or Math API.

Lesson 2.4: Numeric conversions and mathematics

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

C# uses statically typed expressions and the .NET Base Class Library. Follow values from input through conversion and calculation; use explicit types, named intermediate values, and a stated policy for parsing, culture, precision, and formatting.

Worked Example

```
string heightText = "1.72";
if (double.TryParse(heightText, System.Globalization.CultureInfo.InvariantCulture, out doubl
    Console.WriteLine($"{heightM:F2} m = {heightM * 100.0:F0} cm");
else Console.WriteLine("Enter a valid measurement.");
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
1.72 m = 172 cm
```

Common Mistakes

- Using the wrong numeric type or conversion for the value being processed.
- Ignoring input parsing failures or culture-dependent formatting.
- Relying on implicit precedence or rounding without stating the intended calculation.

Activities

1. Calculate the expected result for the canonical bill, score, or measurement before running the example.
2. Change one canonical input and verify which intermediate and final values change.
3. Test malformed text, a boundary value, or a different numeric representation as appropriate.
4. Explain the selected type, conversion, precedence, or formatting rule in C#.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can choose appropriate numeric types and conversions.
- ☐ I can explain the expression and calculation order.
- ☐ I can reproduce the canonical result.
- ☐ I can handle an invalid or boundary input.
- ☐ I can explain the relevant .NET numeric or Math API.

Lesson 2.5: Using the Math Library

Reading References

- Microsoft Learn: System.Math API reference.
- Corresponding Python lesson on mathematical functions and geometric calculations.

Learning Objectives

- Use System.Math for common mathematical operations.
- Calculate circle area and circumference using Math.PI.
- Calculate distance from the origin using Math.Sqrt and Math.Pow.

Concept Explanation

The .NET System.Math class provides mathematical constants and methods such as PI, Pow, Sqrt, Abs, and Round. C# accesses these static members through the Math type. The distance from the origin to (x, y) is the square root of x squared plus y squared. Floating-point values should be formatted or rounded according to an explicit display requirement.

Worked Example

```
double radius = 3.0;
double area = Math.PI * Math.Pow(radius, 2);
double circumference = 2.0 * Math.PI * radius;
double distance = Math.Sqrt(Math.Pow(3.0, 2) + Math.Pow(4.0, 2));
Console.WriteLine($"Area: {area:F2}; circumference: {circumference:F2}; distance: {distance
```

Trace: area is approximately 28.27, circumference is approximately 18.85, and distance from (3, 4) to the origin is 5.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
Area: 28.27; circumference: 18.85; distance: 5.00
```

Common Mistakes

- Using an approximate value of pi when Math.PI is available.
- Passing a diameter where the formula expects a radius.
- Taking the square root before adding both squared coordinate values.

Activities

1. Calculate area and circumference for a circle with radius 3.

2. Calculate distance from the origin to (3, 4) and verify the 5-unit result.
3. Change the radius to zero and predict the outputs.
4. Compare the full double value with F2 formatted output.

Accessibility Practice

Read the complete expressions, including parentheses and method names. Verify that formatted values have explicit labels and units.

Skills Checklist

- ☐ I can locate and use Math.PI, Math.Pow, and Math.Sqrt.
- ☐ I can calculate circle area and circumference.
- ☐ I can calculate distance from the origin.
- ☐ I can explain floating-point precision and formatting.
- ☐ I can test a zero-radius boundary case.

6.5 Unit 3: Control Flow

Overview

This unit covers Boolean expressions, conditionals, lists or arrays, iteration, string operations. Follow the same problem progression as Python so that the main comparison is language syntax and semantics. Use the worked examples as starting points, then complete the activities independently.

Lesson 3.1: Boolean expressions

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

Control flow in C# uses Boolean expressions, if/else, arrays, loops, and immutable string values. Array indexes start at zero; collection lengths and loop bounds must be checked. String methods return new strings rather than mutating the original.

Worked Example

```
int score = 82;
bool validScore = score >= 0 && score <= 100, passed = validScore && score >= 60;
Console.WriteLine($"Valid: {validScore}; passed: {passed}");
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
Valid: True; passed: True
```

Common Mistakes

- Using an incorrect Boolean boundary or condition order.
- Assuming array indexes start at one or exceed the valid range.
- Assuming string operations mutate the original value.

Activities

1. Trace the Boolean conditions, branches, collection indexes, or loop iterations in the example.
2. Apply the same operation to the canonical scores 82, 91, 76, and 88.
3. Test a boundary score, empty collection, missing match, or string edge case.
4. Explain the relevant C# control-flow or string/array rule and how you verified it.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can write the required Boolean or control-flow construct.
- ☐ I can apply the shared score or text data correctly.
- ☐ I can avoid invalid indexing and loop bounds.
- ☐ I can test boundary and negative cases.
- ☐ I can explain the relevant C# array, string, or branch semantics.

Lesson 3.2: Chained conditionals

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

Control flow in C# uses Boolean expressions, if/else, arrays, loops, and immutable string values. Array indexes start at zero; collection lengths and loop bounds must be checked. String methods return new strings rather than mutating the original.

Worked Example

```
int[] scores = { 59, 60, 70, 80, 90, 100 };
foreach (int score in scores) { string grade = score >= 90 ? "A" : score >= 80 ? "B" : score
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
59: F
60: D
70: C
80: B
90: A
100: A
```

Common Mistakes

- Using an incorrect Boolean boundary or condition order.
- Assuming array indexes start at one or exceed the valid range.
- Assuming string operations mutate the original value.

Activities

1. Trace the Boolean conditions, branches, collection indexes, or loop iterations in the example.
2. Apply the same operation to the canonical scores 82, 91, 76, and 88.
3. Test a boundary score, empty collection, missing match, or string edge case.
4. Explain the relevant C# control-flow or string/array rule and how you verified it.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can write the required Boolean or control-flow construct.
- ☐ I can apply the shared score or text data correctly.
- ☐ I can avoid invalid indexing and loop bounds.
- ☐ I can test boundary and negative cases.
- ☐ I can explain the relevant C# array, string, or branch semantics.

Lesson 3.3: Sequences and indexing

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

Control flow in C# uses Boolean expressions, if/else, arrays, loops, and immutable string values. Array indexes start at zero; collection lengths and loop bounds must be checked. String methods return new strings rather than mutating the original.

Worked Example

```
int[] scores = { 82, 91, 76, 88 };
Console.WriteLine($"First: {scores[0]}; fourth: {scores[3]}"); scores[2] = 80;
Console.WriteLine($"Updated third score: {scores[2]}; count: {scores.Length}");
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
First: 82; fourth: 88
Updated third score: 80; count: 4
```

Common Mistakes

- Using an incorrect Boolean boundary or condition order.
- Assuming array indexes start at one or exceed the valid range.
- Assuming string operations mutate the original value.

Activities

1. Trace the Boolean conditions, branches, collection indexes, or loop iterations in the example.
2. Apply the same operation to the canonical scores 82, 91, 76, and 88.
3. Test a boundary score, empty collection, missing match, or string edge case.
4. Explain the relevant C# control-flow or string/array rule and how you verified it.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can write the required Boolean or control-flow construct.
- ☐ I can apply the shared score or text data correctly.
- ☐ I can avoid invalid indexing and loop bounds.
- ☐ I can test boundary and negative cases.
- ☐ I can explain the relevant C# array, string, or branch semantics.

Lesson 3.4: For loops and iteration

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

Control flow in C# uses Boolean expressions, if/else, arrays, loops, and immutable string values. Array indexes start at zero; collection lengths and loop bounds must be checked. String methods return new strings rather than mutating the original.

Worked Example

```
int[] scores = { 82, 91, 76, 88 }; int total = 0;
foreach (int score in scores) total += score;
double average = (double)total / scores.Length;
Console.WriteLine($"Total: {total}; average: {average:F2}");
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
Total: 337; average: 84.25
```

Common Mistakes

- Using an incorrect Boolean boundary or condition order.
- Assuming array indexes start at one or exceed the valid range.
- Assuming string operations mutate the original value.

Activities

1. Trace the Boolean conditions, branches, collection indexes, or loop iterations in the example.

2. Apply the same operation to the canonical scores 82, 91, 76, and 88.
3. Test a boundary score, empty collection, missing match, or string edge case.
4. Explain the relevant C# control-flow or string/array rule and how you verified it.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can write the required Boolean or control-flow construct.
- ☐ I can apply the shared score or text data correctly.
- ☐ I can avoid invalid indexing and loop bounds.
- ☐ I can test boundary and negative cases.
- ☐ I can explain the relevant C# array, string, or branch semantics.

Lesson 3.5: String operations

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

Control flow in C# uses Boolean expressions, if/else, arrays, loops, and immutable string values. Array indexes start at zero; collection lengths and loop bounds must be checked. String methods return new strings rather than mutating the original.

Worked Example

```
string sample = "A student studies programming. Programming develops problem-solving skills.
string normalized = sample.ToLowerInvariant();
Console.WriteLine($"Length: {normalized.Length}; contains programming: {normalized.Contains(
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
Length: 75; contains programming: True
```

Common Mistakes

- Using an incorrect Boolean boundary or condition order.
- Assuming array indexes start at one or exceed the valid range.
- Assuming string operations mutate the original value.

Activities

1. Trace the Boolean conditions, branches, collection indexes, or loop iterations in the example.
2. Apply the same operation to the canonical scores 82, 91, 76, and 88.
3. Test a boundary score, empty collection, missing match, or string edge case.
4. Explain the relevant C# control-flow or string/array rule and how you verified it.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

☐ I can write the required Boolean or control-flow construct.
☐ I can apply the shared score or text data correctly.
☐ I can avoid invalid indexing and loop bounds.
☐ I can test boundary and negative cases.
☐ I can explain the relevant C# array, string, or branch semantics.

6.6 Unit 4: Data Structures

Overview

This unit covers sequence mutation, sorting, immutable records, maps, dictionaries, traversal. Follow the same problem progression as Python so that the

main comparison is language syntax and semantics. Use the worked examples as starting points, then complete the activities independently.

Lesson 4.1: Mutable sequences

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

C# offers generic collections such as List and Dictionary<TKey,TValue>, plus records and structs for data modeling. Select a collection based on whether the problem needs ordered mutable elements, unique keys, or immutable value data.

Worked Example

```
var items = new List<string> { "Notebook", "Pencil" };
items.Insert(1, "Folder"); items.Add("Eraser"); items[0] = "Journal"; items.Remove("Pencil")
Console.WriteLine(string.Join(", ", items));
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
Journal, Folder, Eraser
```

Common Mistakes

- Choosing a collection whose mutability or lookup behavior does not fit the task.
- Using an invalid index or missing dictionary key without checking it.
- Assuming enumeration order or duplicate handling that the collection does not guarantee.

Activities

1. Apply the collection operation to the canonical Notebook, Pencil, and Folder inventory.
2. Add, update, remove, sort, or traverse one additional item and predict the result.
3. Test a duplicate key, missing key, invalid index, or immutable-value constraint.
4. Explain why the chosen C# collection or record type fits this data.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can choose the appropriate generic collection or record type.
- ☐ I can perform the required add, update, remove, sort, or lookup operation.
- ☐ I can explain mutability, key uniqueness, or value semantics.
- ☐ I can test a missing, duplicate, or boundary case.
- ☐ I can traverse or inspect the data without relying on visual layout.

Lesson 4.2: Sorting and organizing data

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

C# offers generic collections such as List and Dictionary<TKey,TValue>, plus records and structs for data modeling. Select a collection based on whether the problem needs ordered mutable elements, unique keys, or immutable value data.

Worked Example

```
var quantities = new List<int> { 10, 3, 4, 1 };
quantities.Sort(); quantities.Reverse();
Console.WriteLine(string.Join(" ", quantities));
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
10 4 3 1
```

Common Mistakes

- Choosing a collection whose mutability or lookup behavior does not fit the task.
- Using an invalid index or missing dictionary key without checking it.
- Assuming enumeration order or duplicate handling that the collection does not guarantee.

Activities

1. Apply the collection operation to the canonical Notebook, Pencil, and Folder inventory.
2. Add, update, remove, sort, or traverse one additional item and predict the result.
3. Test a duplicate key, missing key, invalid index, or immutable-value constraint.
4. Explain why the chosen C# collection or record type fits this data.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can choose the appropriate generic collection or record type.
- ☐ I can perform the required add, update, remove, sort, or lookup operation.
- ☐ I can explain mutability, key uniqueness, or value semantics.
- ☐ I can test a missing, duplicate, or boundary case.
- ☐ I can traverse or inspect the data without relying on visual layout.

Lesson 4.3: Immutable records

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

C# offers generic collections such as List and Dictionary<TKey,TValue>, plus records and structs for data modeling. Select a collection based on whether the problem needs ordered mutable elements, unique keys, or immutable value data.

Worked Example

```
var point = new Coordinate(3, 4);
Console.WriteLine($"Point: ({point.X}, {point.Y}); distance: {Math.Sqrt(point.X * point.X +
readonly record struct Coordinate(int X, int Y);
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
Point: (3, 4); distance: 5.00
```

Common Mistakes

- Choosing a collection whose mutability or lookup behavior does not fit the task.
- Using an invalid index or missing dictionary key without checking it.
- Assuming enumeration order or duplicate handling that the collection does not guarantee.

Activities

1. Apply the collection operation to the canonical Notebook, Pencil, and Folder inventory.

2. Add, update, remove, sort, or traverse one additional item and predict the result.
3. Test a duplicate key, missing key, invalid index, or immutable-value constraint.
4. Explain why the chosen C# collection or record type fits this data.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can choose the appropriate generic collection or record type.
- ☐ I can perform the required add, update, remove, sort, or lookup operation.
- ☐ I can explain mutability, key uniqueness, or value semantics.
- ☐ I can test a missing, duplicate, or boundary case.
- ☐ I can traverse or inspect the data without relying on visual layout.

Lesson 4.4: Maps and key-value data

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

C# offers generic collections such as List and Dictionary<TKey,TValue>, plus records and structs for data modeling. Select a collection based on whether the problem needs ordered mutable elements, unique keys, or immutable value data.

Worked Example

```
var inventory = new Dictionary<string, int> { ["Notebook"] = 3, ["Pencil"] = 10, ["Folder"]
inventory["Notebook"] += 1;
Console.WriteLine($"Notebook quantity: {inventory["Notebook"]}");
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
Notebook quantity: 4
```

Common Mistakes

- Choosing a collection whose mutability or lookup behavior does not fit the task.
- Using an invalid index or missing dictionary key without checking it.
- Assuming enumeration order or duplicate handling that the collection does not guarantee.

Activities

1. Apply the collection operation to the canonical Notebook, Pencil, and Folder inventory.
2. Add, update, remove, sort, or traverse one additional item and predict the result.
3. Test a duplicate key, missing key, invalid index, or immutable-value constraint.
4. Explain why the chosen C# collection or record type fits this data.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

☐ I can choose the appropriate generic collection or record type.
☐ I can perform the required add, update, remove, sort, or lookup operation.
☐ I can explain mutability, key uniqueness, or value semantics.
☐ I can test a missing, duplicate, or boundary case.
☐ I can traverse or inspect the data without relying on visual layout.

Lesson 4.5: Collection traversal

Reading References

- Official C# language guide and standard-library documentation.

- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

C# offers generic collections such as List and Dictionary<TKey,TValue>, plus records and structs for data modeling. Select a collection based on whether the problem needs ordered mutable elements, unique keys, or immutable value data.

Worked Example

```
var inventory = new Dictionary<string, int> { ["Notebook"] = 3, ["Pencil"] = 10, ["Folder"]
foreach (KeyValuePair<string, int> entry in inventory) Console.WriteLine($"{entry.Key} => {e
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
Notebook => 3
Pencil => 10
Folder => 4
```

Common Mistakes

- Choosing a collection whose mutability or lookup behavior does not fit the task.
- Using an invalid index or missing dictionary key without checking it.
- Assuming enumeration order or duplicate handling that the collection does not guarantee.

Activities

1. Apply the collection operation to the canonical Notebook, Pencil, and Folder inventory.
2. Add, update, remove, sort, or traverse one additional item and predict the result.

3. Test a duplicate key, missing key, invalid index, or immutable-value constraint.
4. Explain why the chosen C# collection or record type fits this data.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can choose the appropriate generic collection or record type.
- ☐ I can perform the required add, update, remove, sort, or lookup operation.
- ☐ I can explain mutability, key uniqueness, or value semantics.
- ☐ I can test a missing, duplicate, or boundary case.
- ☐ I can traverse or inspect the data without relying on visual layout.

6.7 Unit 5: Input and Iteration

Overview

This unit covers input validation, sentinel loops, while loops, flags, break/continue, infinite-loop prevention. Follow the same problem progression as Python so that the main comparison is language syntax and semantics. Use the worked examples as starting points, then complete the activities independently.

Lesson 5.1: Input and validation

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

Console input is text and must be parsed and validated before use. while, foreach, break, and continue control repetition; every loop needs a clear exit

condition and progress toward termination.

Worked Example

```
var input = new Queue<string>(new[] { "16", "abc", "131" });
while (input.Count > 0) { string candidate = input.Dequeue(); if (int.TryParse(candidate, ou
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
Accepted age: 16
Invalid age: abc
Invalid age: 131
```

Common Mistakes

- Treating parsed text as valid domain data without checking it.
- Failing to update loop state or provide a terminating condition.
- Using break or continue when the other control-flow operation is intended.

Activities

1. Trace the input or loop state after each iteration of the worked example.
2. Test valid input, malformed input, and one domain-boundary value.
3. Change the sentinel, flag, or attempt limit and predict when the loop terminates.
4. Explain how the code guarantees progress and avoids an unbounded loop.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can parse and validate input before using it.
- ☐ I can implement the required loop or control-flow operation.
- ☐ I can identify the loop's termination condition.
- ☐ I can test valid and invalid paths.
- ☐ I can explain how the code avoids an infinite loop.

Lesson 5.2: While loops

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

Console input is text and must be parsed and validated before use. while, foreach, break, and continue control repetition; every loop needs a clear exit condition and progress toward termination.

Worked Example

```
int[] values = { 1, 2, 3, 0, 9 }; int index = 0, total = 0;
while (index < values.Length && values[index] != 0) { total += values[index]; index++; }
Console.WriteLine($"Sum before sentinel: {total}");
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
Sum before sentinel: 6
```

Common Mistakes

- Treating parsed text as valid domain data without checking it.
- Failing to update loop state or provide a terminating condition.
- Using break or continue when the other control-flow operation is intended.

Activities

1. Trace the input or loop state after each iteration of the worked example.
2. Test valid input, malformed input, and one domain-boundary value.

3. Change the sentinel, flag, or attempt limit and predict when the loop terminates.
4. Explain how the code guarantees progress and avoids an unbounded loop.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can parse and validate input before using it.
- ☐ I can implement the required loop or control-flow operation.
- ☐ I can identify the loop's termination condition.
- ☐ I can test valid and invalid paths.
- ☐ I can explain how the code avoids an infinite loop.

Lesson 5.3: Flags and break

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

Console input is text and must be parsed and validated before use. while, foreach, break, and continue control repetition; every loop needs a clear exit condition and progress toward termination.

Worked Example

```
string[] names = { "Avery", "Jordan", "Sam" }; bool found = false;
foreach (string name in names) { if (name == "Jordan") { found = true; break; } }
Console.WriteLine($"Jordan found: {found}");
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
Jordan found: True
```

Common Mistakes

- Treating parsed text as valid domain data without checking it.
- Failing to update loop state or provide a terminating condition.
- Using break or continue when the other control-flow operation is intended.

Activities

1. Trace the input or loop state after each iteration of the worked example.
2. Test valid input, malformed input, and one domain-boundary value.
3. Change the sentinel, flag, or attempt limit and predict when the loop terminates.
4. Explain how the code guarantees progress and avoids an unbounded loop.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

☐ I can parse and validate input before using it.
☐ I can implement the required loop or control-flow operation.
☐ I can identify the loop's termination condition.
☐ I can test valid and invalid paths.
☐ I can explain how the code avoids an infinite loop.

Lesson 5.4: Continue and filtering

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

Console input is text and must be parsed and validated before use. while, foreach, break, and continue control repetition; every loop needs a clear exit condition and progress toward termination.

Worked Example

```
int[] values = { -2, 3, 0, 5, -1 };
foreach (int value in values) { if (value <= 0) continue; Console.WriteLine(value); }
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
3
5
```

Common Mistakes

- Treating parsed text as valid domain data without checking it.
- Failing to update loop state or provide a terminating condition.
- Using break or continue when the other control-flow operation is intended.

Activities

1. Trace the input or loop state after each iteration of the worked example.
2. Test valid input, malformed input, and one domain-boundary value.
3. Change the sentinel, flag, or attempt limit and predict when the loop terminates.
4. Explain how the code guarantees progress and avoids an unbounded loop.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can parse and validate input before using it.
- ☐ I can implement the required loop or control-flow operation.
- ☐ I can identify the loop's termination condition.
- ☐ I can test valid and invalid paths.

☐ I can explain how the code avoids an infinite loop.

Lesson 5.5: Infinite-loop prevention

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

Console input is text and must be parsed and validated before use. while, foreach, break, and continue control repetition; every loop needs a clear exit condition and progress toward termination.

Worked Example

```
int attempts = 0; bool valid = false;
while (!valid && attempts < 3) { attempts++; valid = attempts == 2; }
Console.WriteLine($"Attempts: {attempts}; valid: {valid}");
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
Attempts: 2; valid: True
```

Common Mistakes

- Treating parsed text as valid domain data without checking it.
- Failing to update loop state or provide a terminating condition.
- Using break or continue when the other control-flow operation is intended.

Activities

1. Trace the input or loop state after each iteration of the worked example.
2. Test valid input, malformed input, and one domain-boundary value.
3. Change the sentinel, flag, or attempt limit and predict when the loop terminates.
4. Explain how the code guarantees progress and avoids an unbounded loop.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can parse and validate input before using it.
- ☐ I can implement the required loop or control-flow operation.
- ☐ I can identify the loop's termination condition.
- ☐ I can test valid and invalid paths.
- ☐ I can explain how the code avoids an infinite loop.

6.8 Unit 6: Functions

Overview

This unit covers definitions, parameters, arguments, return values, scope, collection parameters, variadic functions. Follow the same problem progression as Python so that the main comparison is language syntax and semantics. Use the worked examples as starting points, then complete the activities independently.

Lesson 6.1: Defining functions

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

C# methods declare parameter and return types. Use parameters to make dependencies explicit, return values to separate computation from presentation, and collection interfaces or params when they improve the method contract.

Worked Example

```
static double CalculateTotal(double bill, double rate) { double tip = bill * rate; return bi
Console.WriteLine($"Total: {CalculateTotal(24.50, 0.15):F2}");
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
Total: 28.18
```

Common Mistakes

- Confusing a parameter with an argument or a returned value with printed output.
- Choosing a method signature that does not express its input/output contract.
- Ignoring empty collections, optional values, or variadic edge cases.

Activities

1. Call the method with the canonical bill/rate or score values and predict its return value.
2. Add a second call with different arguments, including a named or optional argument where relevant.
3. Test empty input, zero arguments, or another edge case for the method contract.
4. Explain the method signature, parameter semantics, and return behavior.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

☐ I can declare a method with an appropriate signature.

☐ I can pass arguments and use the return value correctly.
☐ I can handle the method's edge cases.
☐ I can reuse the method with different inputs.
☐ I can explain the method contract and C# parameter behavior.

Lesson 6.2: Parameters and arguments

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

C# methods declare parameter and return types. Use parameters to make dependencies explicit, return values to separate computation from presentation, and collection interfaces or params when they improve the method contract.

Worked Example

```
static string DescribeScore(string name, int score, string label = "Score") => $"{name} - {l
Console.WriteLine(DescribeScore("Avery", 92));
Console.WriteLine(DescribeScore(score: 92, name: "Avery", label: "Final"));
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
Avery - Score: 92
Avery - Final: 92
```

Common Mistakes

- Confusing a parameter with an argument or a returned value with printed output.

- Choosing a method signature that does not express its input/output contract.
- Ignoring empty collections, optional values, or variadic edge cases.

Activities

1. Call the method with the canonical bill/rate or score values and predict its return value.
2. Add a second call with different arguments, including a named or optional argument where relevant.
3. Test empty input, zero arguments, or another edge case for the method contract.
4. Explain the method signature, parameter semantics, and return behavior.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

☐ I can declare a method with an appropriate signature.
☐ I can pass arguments and use the return value correctly.
☐ I can handle the method's edge cases.
☐ I can reuse the method with different inputs.
☐ I can explain the method contract and C# parameter behavior.

Lesson 6.3: Return values

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

C# methods declare parameter and return types. Use parameters to make dependencies explicit, return values to separate computation from presentation, and collection interfaces or params when they improve the method contract.

Worked Example

```
static double CalculateTip(double bill, double rate) => bill * rate;
double tip = CalculateTip(24.50, 0.15), total = 24.50 + tip;
Console.WriteLine($"Tip: {tip:F2}; total: {total:F2}");
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
Tip: 3.67; total: 28.18
```

Common Mistakes

- Confusing a parameter with an argument or a returned value with printed output.
- Choosing a method signature that does not express its input/output contract.
- Ignoring empty collections, optional values, or variadic edge cases.

Activities

1. Call the method with the canonical bill/rate or score values and predict its return value.
2. Add a second call with different arguments, including a named or optional argument where relevant.
3. Test empty input, zero arguments, or another edge case for the method contract.
4. Explain the method signature, parameter semantics, and return behavior.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can declare a method with an appropriate signature.
- ☐ I can pass arguments and use the return value correctly.
- ☐ I can handle the method's edge cases.
- ☐ I can reuse the method with different inputs.
- ☐ I can explain the method contract and C# parameter behavior.

Lesson 6.4: Functions over collections

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

C# methods declare parameter and return types. Use parameters to make dependencies explicit, return values to separate computation from presentation, and collection interfaces or params when they improve the method contract.

Worked Example

```
static double Average(IReadOnlyList<int> scores) => scores.Count == 0 ? 0.0 : scores.Average
int[] scores = { 82, 91, 76, 88 };
Console.WriteLine($"Average: {Average(scores):F2}");
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
Average: 84.25
```

Common Mistakes

- Confusing a parameter with an argument or a returned value with printed output.
- Choosing a method signature that does not express its input/output contract.
- Ignoring empty collections, optional values, or variadic edge cases.

Activities

1. Call the method with the canonical bill/rate or score values and predict its return value.

2. Add a second call with different arguments, including a named or optional argument where relevant.
3. Test empty input, zero arguments, or another edge case for the method contract.
4. Explain the method signature, parameter semantics, and return behavior.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can declare a method with an appropriate signature.
- ☐ I can pass arguments and use the return value correctly.
- ☐ I can handle the method's edge cases.
- ☐ I can reuse the method with different inputs.
- ☐ I can explain the method contract and C# parameter behavior.

Lesson 6.5: Scope and variadic arguments

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

C# methods declare parameter and return types. Use parameters to make dependencies explicit, return values to separate computation from presentation, and collection interfaces or params when they improve the method contract.

Worked Example

```
static int Sum(params int[] values) { int total = 0; foreach (int value in values) total +=
Console.WriteLine(Sum(2, 3, 4)); Console.WriteLine(Sum(82, 91, 76, 88));
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
9
337
```

Common Mistakes

- Confusing a parameter with an argument or a returned value with printed output.
- Choosing a method signature that does not express its input/output contract.
- Ignoring empty collections, optional values, or variadic edge cases.

Activities

1. Call the method with the canonical bill/rate or score values and predict its return value.
2. Add a second call with different arguments, including a named or optional argument where relevant.
3. Test empty input, zero arguments, or another edge case for the method contract.
4. Explain the method signature, parameter semantics, and return behavior.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

☐ I can declare a method with an appropriate signature.
☐ I can pass arguments and use the return value correctly.
☐ I can handle the method's edge cases.
☐ I can reuse the method with different inputs.
☐ I can explain the method contract and C# parameter behavior.

6.9 Unit 7: Object-Oriented Programming

Overview

This unit covers types/classes, instances, fields, methods, inheritance or interfaces, composition. Follow the same problem progression as Python so that the main comparison is language syntax and semantics. Use the worked examples as starting points, then complete the activities independently.

Lesson 7.1: Types and object models

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

C# supports classes, records, structs, properties, methods, interfaces, inheritance, and composition. Choose value or reference semantics intentionally and protect invariants through the public API.

Worked Example

```
var student = new Student("Avery", 16, "Computer Science");
Console.WriteLine($"{student.Name}, {student.Age}, {student.FavoriteSubject}");
record Student(string Name, int Age, string FavoriteSubject);
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
Avery, 16, Computer Science
```

Common Mistakes

- Exposing mutable state that bypasses the type's invariants.
- Confusing reference identity with value equality.
- Using inheritance where an interface or composition better represents the relationship.

Activities

1. Create a second object using the same type and compare its state or behavior.
2. Change one property or method input and verify the model invariant remains true.
3. Test equality, interface dispatch, inheritance, or nested composition as appropriate.
4. Explain why the selected C# type relationship is appropriate for the domain.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

☐ I can define and instantiate the required C# type.
☐ I can expose behavior through an appropriate public API.
☐ I can preserve model invariants.
☐ I can explain value/reference, interface, or composition semantics.
☐ I can test more than one instance or implementation.

Lesson 7.2: Creating instances

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

C# supports classes, records, structs, properties, methods, interfaces, inheritance, and composition. Choose value or reference semantics intentionally and protect invariants through the public API.

Worked Example

```
var avery = new Student("Avery", 16, "Computer Science"); var jordan = new Student("Jordan",
Console.WriteLine(avery); Console.WriteLine(jordan);
record Student(string Name, int Age, string FavoriteSubject);
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
Student { Name = Avery, Age = 16, FavoriteSubject = Computer Science }
Student { Name = Jordan, Age = 17, FavoriteSubject = Mathematics }
```

Common Mistakes

- Exposing mutable state that bypasses the type's invariants.
- Confusing reference identity with value equality.
- Using inheritance where an interface or composition better represents the relationship.

Activities

1. Create a second object using the same type and compare its state or behavior.
2. Change one property or method input and verify the model invariant remains true.
3. Test equality, interface dispatch, inheritance, or nested composition as appropriate.
4. Explain why the selected C# type relationship is appropriate for the domain.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can define and instantiate the required C# type.
- ☐ I can expose behavior through an appropriate public API.
- ☐ I can preserve model invariants.
- ☐ I can explain value/reference, interface, or composition semantics.
- ☐ I can test more than one instance or implementation.

Lesson 7.3: Fields and methods

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

C# supports classes, records, structs, properties, methods, interfaces, inheritance, and composition. Choose value or reference semantics intentionally and protect invariants through the public API.

Worked Example

```
var stock = new InventoryItem("Notebook", 3); stock.Add(2);
Console.WriteLine($"{stock.Name}: {stock.Quantity}");
class InventoryItem(string name, int quantity) { public string Name { get; } = name; public
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
Notebook: 5
```

Common Mistakes

- Exposing mutable state that bypasses the type's invariants.

- Confusing reference identity with value equality.
- Using inheritance where an interface or composition better represents the relationship.

Activities

1. Create a second object using the same type and compare its state or behavior.
2. Change one property or method input and verify the model invariant remains true.
3. Test equality, interface dispatch, inheritance, or nested composition as appropriate.
4. Explain why the selected C# type relationship is appropriate for the domain.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

☐ I can define and instantiate the required C# type.
☐ I can expose behavior through an appropriate public API.
☐ I can preserve model invariants.
☐ I can explain value/reference, interface, or composition semantics.
☐ I can test more than one instance or implementation.

Lesson 7.4: Inheritance and interfaces

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

C# supports classes, records, structs, properties, methods, interfaces, inheritance, and composition. Choose value or reference semantics intentionally and

protect invariants through the public API.

Worked Example

```
IRecord record = new StudentRecord("Avery"); Console.WriteLine(record.Describe());
interface IRecord { string Describe(); }
class StudentRecord(string name) : IRecord { public string Describe() => $"Student: {name}";
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
Student: Avery
```

Common Mistakes

- Exposing mutable state that bypasses the type's invariants.
- Confusing reference identity with value equality.
- Using inheritance where an interface or composition better represents the relationship.

Activities

1. Create a second object using the same type and compare its state or behavior.
2. Change one property or method input and verify the model invariant remains true.
3. Test equality, interface dispatch, inheritance, or nested composition as appropriate.
4. Explain why the selected C# type relationship is appropriate for the domain.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can define and instantiate the required C# type.
- ☐ I can expose behavior through an appropriate public API.
- ☐ I can preserve model invariants.

☐ I can explain value/reference, interface, or composition semantics.
☐ I can test more than one instance or implementation.

Lesson 7.5: Composition

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

C# supports classes, records, structs, properties, methods, interfaces, inheritance, and composition. Choose value or reference semantics intentionally and protect invariants through the public API.

Worked Example

```
var student = new Student("Avery", new Address("Farmington", "Utah"));
Console.WriteLine($"{student.Name} lives in {student.Address.City}, {student.Address.Region}
record Address(string City, string Region); record Student(string Name, Address Address);
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
Avery lives in Farmington, Utah
```

Common Mistakes

- Exposing mutable state that bypasses the type's invariants.
- Confusing reference identity with value equality.
- Using inheritance where an interface or composition better represents the relationship.

Activities

1. Create a second object using the same type and compare its state or behavior.
2. Change one property or method input and verify the model invariant remains true.
3. Test equality, interface dispatch, inheritance, or nested composition as appropriate.
4. Explain why the selected C# type relationship is appropriate for the domain.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can define and instantiate the required C# type.
- ☐ I can expose behavior through an appropriate public API.
- ☐ I can preserve model invariants.
- ☐ I can explain value/reference, interface, or composition semantics.
- ☐ I can test more than one instance or implementation.

6.10 Unit 8: Files and Exceptions

Overview

This unit covers read/write text, resource lifetime, error handling, JSON serialization and persistence. Follow the same problem progression as Python so that the main comparison is language syntax and semantics. Use the worked examples as starting points, then complete the activities independently.

Lesson 8.1: Reading text files

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.

- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

The .NET file and serialization APIs provide text I/O, exception handling, deterministic disposal, and JSON persistence. Resolve relative paths from the working directory, handle expected failures, and dispose IDisposable resources.

Worked Example

```
string path = Path.Combine(Path.GetTempPath(), "csharp-notes.txt");
File.WriteAllLines(path, new[] { "First line", "Second line" });
foreach (string line in File.ReadLines(path)) Console.WriteLine(line);
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
First line
Second line
```

Common Mistakes

- Resolving relative paths from the wrong working directory.
- Overwriting data when append or safe replacement is required.
- Failing to handle expected I/O/serialization errors or dispose resources.

Activities

1. Run the file or serialization example using a temporary test file and inspect its contents.
2. Test a missing file, malformed JSON, or an access failure in a controlled case.
3. Change the operation from replace to append, or move resource use across its scope.
4. Explain path resolution, exception handling, serialization, or disposal behavior.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can use the appropriate .NET file or serialization API.
- ☐ I can resolve paths from the correct working directory.
- ☐ I can handle expected I/O or data errors.
- ☐ I can dispose resources deterministically.
- ☐ I can verify stored and restored data.

Lesson 8.2: Writing and appending files

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

The .NET file and serialization APIs provide text I/O, exception handling, deterministic disposal, and JSON persistence. Resolve relative paths from the working directory, handle expected failures, and dispose IDisposable resources.

Worked Example

```
string path = Path.Combine(Path.GetTempPath(), "csharp-notes.txt");
File.WriteAllText(path, "First note\n"); File.AppendAllText(path, "Second note\n");
Console.Write(File.ReadAllText(path));
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
First note
Second note
```

Common Mistakes

- Resolving relative paths from the wrong working directory.
- Overwriting data when append or safe replacement is required.
- Failing to handle expected I/O/serialization errors or dispose resources.

Activities

1. Run the file or serialization example using a temporary test file and inspect its contents.
2. Test a missing file, malformed JSON, or an access failure in a controlled case.
3. Change the operation from replace to append, or move resource use across its scope.
4. Explain path resolution, exception handling, serialization, or disposal behavior.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

☐ I can use the appropriate .NET file or serialization API.
☐ I can resolve paths from the correct working directory.
☐ I can handle expected I/O or data errors.
☐ I can dispose resources deterministically.
☐ I can verify stored and restored data.

Lesson 8.3: Errors and exceptions

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

The .NET file and serialization APIs provide text I/O, exception handling, deterministic disposal, and JSON persistence. Resolve relative paths from the working directory, handle expected failures, and dispose IDisposable resources.

Worked Example

```
try { int score = int.Parse("not-a-number"); Console.WriteLine(score); }
catch (FormatException error) { Console.WriteLine($"Invalid numeric input: {error.Message}")
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
Invalid numeric input: The input string 'not-a-number' was not in a correct format.
```

Common Mistakes

- Resolving relative paths from the wrong working directory.
- Overwriting data when append or safe replacement is required.
- Failing to handle expected I/O/serialization errors or dispose resources.

Activities

1. Run the file or serialization example using a temporary test file and inspect its contents.
2. Test a missing file, malformed JSON, or an access failure in a controlled case.
3. Change the operation from replace to append, or move resource use across its scope.
4. Explain path resolution, exception handling, serialization, or disposal behavior.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

☐ I can use the appropriate .NET file or serialization API.
☐ I can resolve paths from the correct working directory.

☐ I can handle expected I/O or data errors.
☐ I can dispose resources deterministically.
☐ I can verify stored and restored data.

Lesson 8.4: Resource cleanup

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

The .NET file and serialization APIs provide text I/O, exception handling, deterministic disposal, and JSON persistence. Resolve relative paths from the working directory, handle expected failures, and dispose IDisposable resources.

Worked Example

```
using (var writer = new StringWriter()) { writer.WriteLine("First record"); writer.WriteLine
Console.WriteLine("Writer scope ended and the resource was disposed.");
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
First record
Second record
Writer scope ended and the resource was disposed.
```

Common Mistakes

- Resolving relative paths from the wrong working directory.
- Overwriting data when append or safe replacement is required.
- Failing to handle expected I/O/serialization errors or dispose resources.

Activities

1. Run the file or serialization example using a temporary test file and inspect its contents.
2. Test a missing file, malformed JSON, or an access failure in a controlled case.
3. Change the operation from replace to append, or move resource use across its scope.
4. Explain path resolution, exception handling, serialization, or disposal behavior.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

☐ I can use the appropriate .NET file or serialization API.
☐ I can resolve paths from the correct working directory.
☐ I can handle expected I/O or data errors.
☐ I can dispose resources deterministically.
☐ I can verify stored and restored data.

Lesson 8.5: JSON persistence

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

The .NET file and serialization APIs provide text I/O, exception handling, deterministic disposal, and JSON persistence. Resolve relative paths from the working directory, handle expected failures, and dispose IDisposable resources.

Worked Example

```
var record = new CapstoneRecord(1, "Course project", "Personal data manager", "education", '
string json = System.Text.Json.JsonSerializer.Serialize(record);
var restored = System.Text.Json.JsonSerializer.Deserialize<CapstoneRecord>(json);
Console.WriteLine(json); Console.WriteLine(restored?.Title);
record CapstoneRecord(int Id, string Title, string Description, string Category, string Crea
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
{"Id":1,"Title":"Course project","Description":"Personal data manager","Category":"educatior
Course project
```

Common Mistakes

- Resolving relative paths from the wrong working directory.
- Overwriting data when append or safe replacement is required.
- Failing to handle expected I/O/serialization errors or dispose resources.

Activities

1. Run the file or serialization example using a temporary test file and inspect its contents.
2. Test a missing file, malformed JSON, or an access failure in a controlled case.
3. Change the operation from replace to append, or move resource use across its scope.
4. Explain path resolution, exception handling, serialization, or disposal behavior.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can use the appropriate .NET file or serialization API.
- ☐ I can resolve paths from the correct working directory.
- ☐ I can handle expected I/O or data errors.
- ☐ I can dispose resources deterministically.

☐ I can verify stored and restored data.

6.11 Unit 9: Testing and Data Analysis

Overview

This unit covers unit tests, testing stateful types, randomness, text analysis, CSV processing. Follow the same problem progression as Python so that the main comparison is language syntax and semantics. Use the worked examples as starting points, then complete the activities independently.

Lesson 9.1: Testing functions

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

Testing and analysis should use deterministic inputs where possible. Verify expected results, test state transitions, seed pseudo-random generators when reproducibility matters, and define text/CSV parsing assumptions explicitly.

Worked Example

```
static int Add(int left, int right) => left + right;
if (Add(2, 3) != 5 || Add(-2, 2) != 0) throw new Exception("Add tests failed");
Console.WriteLine("Add tests passed.");
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
Add tests passed.
```

Common Mistakes

- Testing only a happy path and omitting boundary or failure cases.
- Using nondeterministic inputs without a reproducible test strategy.
- Assuming simple string splitting implements the complete text or CSV standard.

Activities

1. Predict the expected result before running the test or analysis example.
2. Add a positive, negative, boundary, or invalid-data test case.
3. Change one data value and verify the test detects an incorrect result or changed count.
4. Explain the limits of the test, random source, tokenizer, or CSV parsing approach.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

☐ I can write or interpret a test with an explicit expected result.
☐ I can test normal, boundary, and failure cases.
☐ I can use reproducible inputs where appropriate.
☐ I can interpret the analysis result correctly.
☐ I can explain limitations in the chosen testing or parsing method.

Lesson 9.2: Testing stateful objects

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

Testing and analysis should use deterministic inputs where possible. Verify expected results, test state transitions, seed pseudo-random generators when reproducibility matters, and define text/CSV parsing assumptions explicitly.

Worked Example

```
var stock = new StockItem("Notebook", 3); stock.Add(2);
if (stock.Quantity != 5) throw new Exception("Quantity test failed"); stock.Add(-1);
if (stock.Quantity != 5) throw new Exception("Invalid update changed quantity"); Console.Wri
class StockItem(string name, int quantity) { public string Name { get; } = name; public int
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
StockItem state tests passed.
```

Common Mistakes

- Testing only a happy path and omitting boundary or failure cases.
- Using nondeterministic inputs without a reproducible test strategy.
- Assuming simple string splitting implements the complete text or CSV standard.

Activities

1. Predict the expected result before running the test or analysis example.
2. Add a positive, negative, boundary, or invalid-data test case.
3. Change one data value and verify the test detects an incorrect result or changed count.
4. Explain the limits of the test, random source, tokenizer, or CSV parsing approach.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

☐ I can write or interpret a test with an explicit expected result.

- ☐ I can test normal, boundary, and failure cases.
- ☐ I can use reproducible inputs where appropriate.
- ☐ I can interpret the analysis result correctly.
- ☐ I can explain limitations in the chosen testing or parsing method.

Lesson 9.3: Random values

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

Testing and analysis should use deterministic inputs where possible. Verify expected results, test state transitions, seed pseudo-random generators when reproducibility matters, and define text/CSV parsing assumptions explicitly.

Worked Example

```
var random = new Random(42); int dieRoll = random.Next(1, 7); double sample = random.NextDou
string[] choices = { "red", "blue", "green" }; string choice = choices[random.Next(choices.L
Console.WriteLine($"Roll: {dieRoll}; sample: {sample:F3}; choice: {choice}");
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
Roll: 5; sample: 0.141; choice: red
```

Common Mistakes

- Testing only a happy path and omitting boundary or failure cases.
- Using nondeterministic inputs without a reproducible test strategy.
- Assuming simple string splitting implements the complete text or CSV standard.

Activities

1. Predict the expected result before running the test or analysis example.
2. Add a positive, negative, boundary, or invalid-data test case.
3. Change one data value and verify the test detects an incorrect result or changed count.
4. Explain the limits of the test, random source, tokenizer, or CSV parsing approach.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can write or interpret a test with an explicit expected result.
- ☐ I can test normal, boundary, and failure cases.
- ☐ I can use reproducible inputs where appropriate.
- ☐ I can interpret the analysis result correctly.
- ☐ I can explain limitations in the chosen testing or parsing method.

Lesson 9.4: Text analysis

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

Testing and analysis should use deterministic inputs where possible. Verify expected results, test state transitions, seed pseudo-random generators when reproducibility matters, and define text/CSV parsing assumptions explicitly.

Worked Example

```
string sample = "A student studies programming. Programming develops problem-solving skills.
int letters = sample.Count(char.IsLetter), spaces = sample.Count(char.IsWhiteSpace);
```

```
int occurrences = sample.Split(' ', StringSplitOptions.RemoveEmptyEntries).Count(word => wor
Console.WriteLine($"Letters: {letters}; spaces: {spaces}; programming occurrences: {occurren
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
Letters: 65; spaces: 7; programming occurrences: 2
```

Common Mistakes

- Testing only a happy path and omitting boundary or failure cases.
- Using nondeterministic inputs without a reproducible test strategy.
- Assuming simple string splitting implements the complete text or CSV standard.

Activities

1. Predict the expected result before running the test or analysis example.
2. Add a positive, negative, boundary, or invalid-data test case.
3. Change one data value and verify the test detects an incorrect result or changed count.
4. Explain the limits of the test, random source, tokenizer, or CSV parsing approach.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can write or interpret a test with an explicit expected result.
- ☐ I can test normal, boundary, and failure cases.
- ☐ I can use reproducible inputs where appropriate.
- ☐ I can interpret the analysis result correctly.
- ☐ I can explain limitations in the chosen testing or parsing method.

Lesson 9.5: CSV records

Reading References

- Official C# language guide and standard-library documentation.

- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

Testing and analysis should use deterministic inputs where possible. Verify expected results, test state transitions, seed pseudo-random generators when reproducibility matters, and define text/CSV parsing assumptions explicitly.

Worked Example

```
string row = "001,Avery,Student,92,true"; string[] fields = row.Split(',');
string studentId = fields[0], firstName = fields[1], lastName = fields[2]; int score = int.P
Console.WriteLine($"{studentId}: {firstName} {lastName}, score {score}, active {active}");
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
001: Avery Student, score 92, active True
```

Common Mistakes

- Testing only a happy path and omitting boundary or failure cases.
- Using nondeterministic inputs without a reproducible test strategy.
- Assuming simple string splitting implements the complete text or CSV standard.

Activities

1. Predict the expected result before running the test or analysis example.
2. Add a positive, negative, boundary, or invalid-data test case.
3. Change one data value and verify the test detects an incorrect result or changed count.
4. Explain the limits of the test, random source, tokenizer, or CSV parsing approach.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can write or interpret a test with an explicit expected result.
- ☐ I can test normal, boundary, and failure cases.
- ☐ I can use reproducible inputs where appropriate.
- ☐ I can interpret the analysis result correctly.
- ☐ I can explain limitations in the chosen testing or parsing method.

6.12 Unit 10: Project-Based Learning

Overview

This unit covers requirements, top-down design, text adventure, command-line to-do manager, accessible validation. Follow the same problem progression as Python so that the main comparison is language syntax and semantics. Use the worked examples as starting points, then complete the activities independently.

Lesson 10.1: Requirements and design

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

Project work starts with observable requirements and acceptance criteria. Separate domain state, command parsing, validation, persistence, and output so each behavior can be tested and navigated accessibly.

Worked Example

```
string[] requirements = { "create records", "list records", "search by ID", "update records'
foreach (string requirement in requirements) Console.WriteLine($"Requirement: {requirement}'
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
Requirement: create records
Requirement: list records
Requirement: search by ID
Requirement: update records
Requirement: delete records
Requirement: validate input
Requirement: save and load JSON
```

Common Mistakes

- Implementing before defining observable requirements and acceptance tests.
- Combining parsing, domain logic, persistence, and output into one untestable block.
- Providing errors or status that depend on color or visual layout alone.

Activities

1. Write an acceptance criterion for the behavior demonstrated by the example.
2. Add one valid command and one invalid command or state transition.
3. Separate one responsibility into a method or model and test it independently.
4. Review prompts and output using keyboard navigation and assistive technology.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

☐ I can define observable project requirements.
☐ I can implement and test a valid state transition or command.
☐ I can handle invalid input with actionable feedback.
☐ I can separate domain logic from interaction.
☐ I can assess keyboard and screen-reader usability.

Lesson 10.2: Text adventure: five rooms and movement

Reading References

- C# collections
- .NET Dictionary documentation
- Canonical shared project specification: Python Unit 10, Lesson 10.3.

Learning Objectives

- Represent all five canonical rooms with C# objects and dictionaries.
- Validate movement before changing the current room.
- Explain why the treasure room is not reachable until the key puzzle is implemented.

Concept Explanation

The shared graph has five rooms: start_room, tunnel, clearing, locked_door_room, and treasure_room. A Room object stores a description and a dictionary of exits. The player begins in start_room. The north exit from locked_door_room is not available in Part 1. Use TryGetValue to handle missing directions without changing state.

Worked Example

```
using System;
using System.Collections.Generic;

var rooms = new Dictionary<string, Room>
{
    ["start_room"] = new("A dimly lit cave. A narrow passage leads north.", new() { ["north"
    ["tunnel"] = new("A damp tunnel. Passages lead south and east.", new() { ["south"] = "st
    ["clearing"] = new("A quiet clearing. An antechamber lies to the north.", new() { ["west
    ["locked_door_room"] = new("A chamber with a heavy door to the north.", new() { ["south"
    ["treasure_room"] = new("You found the treasure. You win!", new())
};

var current = "start_room";
while (true)
{
    Console.WriteLine("---");
    Console.WriteLine(rooms[current].Description);
    Console.WriteLine("What do you want to do?");
    var command = (Console.ReadLine() ?? "quit").Trim().ToLowerInvariant();
    if (command == "quit")
    {
        Console.WriteLine("Exiting game. Goodbye!");
```

```
        break;
    }
    if (rooms[current].Exits.TryGetValue(command, out var destination))
        current = destination;
    else
        Console.WriteLine("You cannot go that way. Enter a listed direction or quit.");
}

record Room(string Description, Dictionary<string, string> Exits);
```

Expected Behavior or Output

Enter north, east, north, north, quit. The player reaches tunnel, clearing, and locked_door_room. The second north is rejected and the current room does not change. The treasure room exists but has no incoming exit in Part 1.

Common Mistakes

- Using the dictionary indexer for a direction that may not exist.
- Updating current before validating the requested exit.
- Defining a destination that is not one of the five room identifiers.
- Adding the treasure exit before implementing the key puzzle.

Activities

1. Trace north, east, north, north and record the current room.
2. Try west from start_room and verify that the room does not change.
3. Check every exit destination against the room dictionary.
4. Explain why treasure_room cannot yet be reached.

Accessibility Practice

Announce the room description and prompt in a consistent order. The invalid-movement response explains what happened and how to continue. Run from the terminal and inspect output sequentially with the learner's access technology.

Skills Checklist

☐ I can model all five canonical rooms.
☐ I can represent exits with a dictionary.
☐ I can reject invalid movement without changing state.
☐ I can explain why the treasure room is not yet reachable.

Lesson 10.3: Text adventure: shiny key and locked door

Reading References

- C# collections
- .NET List documentation
- Canonical shared project specification: Python Unit 10, Lesson 10.3.

Learning Objectives

- Place the shiny key in start_room and transfer it to inventory exactly once.
- Require key use in locked_door_room.
- Unlock the north exit only after the puzzle succeeds.
- End the game when the player reaches treasure_room.

Concept Explanation

Keep the five-room graph from Lesson 10.2. Room objects also contain item lists. Picking up the shiny key removes it from start_room and adds it to inventory. Using the key succeeds only in locked_door_room when the key is in inventory. The north exit remains blocked until the key is used.

Worked Example

```
using System;
using System.Collections.Generic;

var rooms = new Dictionary<string, Room>
{
    ["start_room"] = new("A dimly lit cave.", new() { ["north"] = "tunnel" }, new() { "shiny
    ["tunnel"] = new("A damp tunnel. Passages lead south and east.", new() { ["south"] = "st
    ["clearing"] = new("A quiet clearing. An antechamber lies to the north.", new() { ["west
    ["locked_door_room"] = new("A chamber with a heavy door to the north.", new() { ["south'
    ["treasure_room"] = new("You found the treasure. You win!", new())
};

var current = "start_room";
var inventory = new List<string>();
var doorLocked = true;
while (true)
{
    var room = rooms[current];
    Console.WriteLine();
    Console.WriteLine(room.Description);
    if (current == "treasure_room") break;
    if (room.Items.Count > 0) Console.WriteLine("You see: " + string.Join(", ", room.Items))
```

```
    Console.WriteLine("Your inventory: " + (inventory.Count == 0 ? "empty" : string.Join(",
    Console.WriteLine("What do you want to do?");
    var command = (Console.ReadLine() ?? "quit").Trim().ToLowerInvariant();

    if (command == "quit")
    {
        Console.WriteLine("You quit the adventure.");
        break;
    }
    if (command == "take shiny key")
    {
        if (room.Items.Remove("shiny key"))
        {
            inventory.Add("shiny key");
            Console.WriteLine("You picked up the shiny key.");
        }
        else Console.WriteLine("There is no shiny key here to take.");
    }
    else if (command == "use shiny key")
    {
        if (current != "locked_door_room") Console.WriteLine("You must be at the locked door
        else if (!inventory.Contains("shiny key")) Console.WriteLine("You do not have the sh
        else
        {
            doorLocked = false;
            Console.WriteLine("You unlocked the door!");
        }
    }
    else if (current == "locked_door_room" && command == "north" && doorLocked)
        Console.WriteLine("The door is locked! You need to use the shiny key here.");
    else if (room.Exits.TryGetValue(command, out var destination))
        current = destination;
    else
        Console.WriteLine("Invalid command. Enter north, south, east, west, take shiny key,
}

record Room(string Description, Dictionary<string, string> Exits, List<string> Items)
{
    public Room(string description, Dictionary<string, string> exits) : this(description, ex
}
```

Expected Behavior or Output

Enter north, east, north, north, south, west, south, take shiny key, north, east, north, use shiny key, north. The first north from locked_door_room is rejected. Taking the key removes it from the room and adds it to inventory. Using it

outside locked_door_room is rejected. Using it there unlocks the door; the final north reaches treasure_room and prints the win description.

Common Mistakes

- Leaving the key in the room after adding it to inventory.
- Unlocking the door outside locked_door_room.
- Unlocking the door without checking inventory.
- Allowing north movement before doorLocked becomes false.
- Treating a failed action as a state change.

Activities

1. Run the complete puzzle sequence and record room, items, inventory, and door state.
2. Attempt to take the key twice; the second attempt must be rejected.
3. Try to use the key outside locked_door_room.
4. Try north before and after unlocking.
5. Add a help command listing every supported command.

Accessibility Practice

Announce the room, visible items, inventory, and prompt as separate text lines. Error messages explain the failed action. Test the complete interaction with keyboard-only input and sequential screen-reader output.

Skills Checklist

☐ I can model the five canonical rooms.
☐ I can transfer the key from a room to inventory.
☐ I can prevent duplicate pickup.
☐ I can require key use in locked_door_room.
☐ I can block and unlock the north exit.
☐ I can end the game in treasure_room.

Lesson 10.4: Command-line to-do manager

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.

- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

Project work starts with observable requirements and acceptance criteria. Separate domain state, command parsing, validation, persistence, and output so each behavior can be tested and navigated accessibly.

Worked Example

```
var tasks = new List<TodoTask> { new(1, "Read lesson", "Review the C# lesson", false, "2026-
tasks.Add(new TodoTask(2, "Run tests", "Check the console application", false, "2026-11-01")
tasks[0] = tasks[0] with { Done = true };
foreach (var task in tasks) Console.WriteLine($"{task.Id}: [{(task.Done ? "X" : " ")}] {task
record TodoTask(int Id, string Title, string Description, bool Done, string CreatedAt);
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
1: [X] Read lesson
2: [ ] Run tests
```

Common Mistakes

- Implementing before defining observable requirements and acceptance tests.
- Combining parsing, domain logic, persistence, and output into one untestable block.
- Providing errors or status that depend on color or visual layout alone.

Activities

1. Write an acceptance criterion for the behavior demonstrated by the example.
2. Add one valid command and one invalid command or state transition.
3. Separate one responsibility into a method or model and test it independently.
4. Review prompts and output using keyboard navigation and assistive technology.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can define observable project requirements.
- ☐ I can implement and test a valid state transition or command.
- ☐ I can handle invalid input with actionable feedback.
- ☐ I can separate domain logic from interaction.
- ☐ I can assess keyboard and screen-reader usability.

Lesson 10.5: Accessible UX and validation

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

Project work starts with observable requirements and acceptance criteria. Separate domain state, command parsing, validation, persistence, and output so each behavior can be tested and navigated accessibly.

Worked Example

```
static bool IsValidAge(string input, out int age) => int.TryParse(input, out age) && age >=
foreach (string candidate in new[] { "16", "abc", "131" }) Console.WriteLine($"{candidate}:
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
16: True (parsed value: 16)
abc: False (parsed value: 0)
131: False (parsed value: 131)
```

Common Mistakes

- Implementing before defining observable requirements and acceptance tests.
- Combining parsing, domain logic, persistence, and output into one untestable block.
- Providing errors or status that depend on color or visual layout alone.

Activities

1. Write an acceptance criterion for the behavior demonstrated by the example.
2. Add one valid command and one invalid command or state transition.
3. Separate one responsibility into a method or model and test it independently.
4. Review prompts and output using keyboard navigation and assistive technology.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

☐ I can define observable project requirements.
☐ I can implement and test a valid state transition or command.
☐ I can handle invalid input with actionable feedback.
☐ I can separate domain logic from interaction.
☐ I can assess keyboard and screen-reader usability.

6.13 Unit 11: Capstone

Overview

This unit covers accessible personal-data manager, model, persistence, CRUD, tests, documentation. Follow the same problem progression as Python so that the main comparison is language syntax and semantics. Use the worked examples as starting points, then complete the activities independently.

Lesson 11.1: Capstone requirements

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

The capstone is a structured personal-data manager. Preserve the canonical record fields, implement create/read/update/delete, validate input, persist JSON, test failure cases, and document verified keyboard and assistive-technology behavior.

Worked Example

```
string[] acceptanceCriteria = { "create a record", "list all records", "edit a selected reco
foreach (string criterion in acceptanceCriteria) Console.WriteLine($"[ ] {criterion}");
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
[ ] create a record
[ ] list all records
[ ] edit a selected record
[ ] delete a selected record
[ ] save and reload JSON
[ ] validate all user input
[ ] provide screen-reader-friendly text feedback
```

Common Mistakes

- Omitting a required record field or CRUD operation.
- Trusting persisted or user-supplied data without validation.
- Claiming behavior or accessibility that has not been verified.

Activities

1. Map the example to one capstone requirement and identify its inputs and outputs.
2. Add or modify a canonical record using all required fields.
3. Test a failure path such as duplicate ID, blank title, malformed JSON, or missing record.
4. Document how the behavior is tested and what remains unverified.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

☐ I can implement the required capstone data model and fields.
☐ I can implement the required CRUD operations.
☐ I can validate data and handle persistence errors.
☐ I can test the application against acceptance criteria.
☐ I can document setup, operation, accessibility, and limitations.

Lesson 11.2: Data model and storage

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

The capstone is a structured personal-data manager. Preserve the canonical record fields, implement create/read/update/delete, validate input, persist JSON, test failure cases, and document verified keyboard and assistive-technology behavior.

Worked Example

```
var record = new CapstoneRecord(1, "Course project", "Personal data manager", "education", '
var options = new System.Text.Json.JsonSerializerOptions { PropertyNamingPolicy = System.Tex
string json = System.Text.Json.JsonSerializer.Serialize(record, options); Console.WriteLine(
record CapstoneRecord(int Id, string Title, string Description, string Category, string Crea
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
{"id":1,"title":"Course project","description":"Personal data manager","category":"educatior
```

Common Mistakes

- Omitting a required record field or CRUD operation.
- Trusting persisted or user-supplied data without validation.
- Claiming behavior or accessibility that has not been verified.

Activities

1. Map the example to one capstone requirement and identify its inputs and outputs.
2. Add or modify a canonical record using all required fields.
3. Test a failure path such as duplicate ID, blank title, malformed JSON, or missing record.
4. Document how the behavior is tested and what remains unverified.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can implement the required capstone data model and fields.
- ☐ I can implement the required CRUD operations.
- ☐ I can validate data and handle persistence errors.
- ☐ I can test the application against acceptance criteria.
- ☐ I can document setup, operation, accessibility, and limitations.

Lesson 11.3: Create/read/update/delete

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

The capstone is a structured personal-data manager. Preserve the canonical record fields, implement create/read/update/delete, validate input, persist JSON, test failure cases, and document verified keyboard and assistive-technology behavior.

Worked Example

```
var records = new List<CapstoneRecord> { new(1, "Course project", "Personal data manager", "
records.Add(new CapstoneRecord(2, "Study C#", "Complete functions unit", "learning", "2026-1
var found = records.FirstOrDefault(record => record.Id == 1);
if (found is not null) records[records.IndexOf(found)] = found with { Title = "Updated cours
records.RemoveAll(record => record.Id == 2); foreach (var item in records) Console.WriteLine
record CapstoneRecord(int Id, string Title, string Description, string Category, string Crea
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
1: Updated course project
```

Common Mistakes

- Omitting a required record field or CRUD operation.
- Trusting persisted or user-supplied data without validation.
- Claiming behavior or accessibility that has not been verified.

Activities

1. Map the example to one capstone requirement and identify its inputs and outputs.
2. Add or modify a canonical record using all required fields.
3. Test a failure path such as duplicate ID, blank title, malformed JSON, or missing record.
4. Document how the behavior is tested and what remains unverified.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

☐ I can implement the required capstone data model and fields.
☐ I can implement the required CRUD operations.
☐ I can validate data and handle persistence errors.
☐ I can test the application against acceptance criteria.
☐ I can document setup, operation, accessibility, and limitations.

Lesson 11.4: Validation and testing

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

The capstone is a structured personal-data manager. Preserve the canonical record fields, implement create/read/update/delete, validate input, persist JSON, test failure cases, and document verified keyboard and assistive-technology behavior.

Worked Example

```
static bool IsValidRecord(CapstoneRecord record) => record.Id > 0 && !string.IsNullOrWhiteSp
var sample = new CapstoneRecord(1, "Course project", "Personal data manager", "education", '
if (!IsValidRecord(sample)) throw new Exception("Valid record rejected"); var invalid = samp
if (IsValidRecord(invalid)) throw new Exception("Invalid record accepted"); Console.WriteLi
record CapstoneRecord(int Id, string Title, string Description, string Category, string Crea
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
Record validation tests passed.
```

Common Mistakes

- Omitting a required record field or CRUD operation.
- Trusting persisted or user-supplied data without validation.
- Claiming behavior or accessibility that has not been verified.

Activities

1. Map the example to one capstone requirement and identify its inputs and outputs.
2. Add or modify a canonical record using all required fields.
3. Test a failure path such as duplicate ID, blank title, malformed JSON, or missing record.
4. Document how the behavior is tested and what remains unverified.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Skills Checklist

- ☐ I can implement the required capstone data model and fields.
- ☐ I can implement the required CRUD operations.
- ☐ I can validate data and handle persistence errors.
- ☐ I can test the application against acceptance criteria.
- ☐ I can document setup, operation, accessibility, and limitations.

Lesson 11.5: Documentation and presentation

Reading References

- Official C# language guide and standard-library documentation.
- Corresponding Python course lesson for the shared problem context and learning objective.

Learning Objectives

- Explain the C# concepts and runtime behavior introduced in this lesson.
- Trace the worked example and predict its observable result.
- Apply the shared problem context using appropriate C# types and .NET APIs.

Concept Explanation

The capstone is a structured personal-data manager. Preserve the canonical record fields, implement create/read/update/delete, validate input, persist JSON, test failure cases, and document verified keyboard and assistive-technology behavior.

Worked Example

```
string[] documentation = { "Project: Accessible Personal Data Manager", "Runtime: .NET 10",
Console.WriteLine(string.Join(Environment.NewLine, documentation));
```

Trace: identify declarations, expressions, control flow, and observable output. Predict the result, run the example, and compare the actual output with the prediction.

Expected Behavior or Output

Output observed when the worked example was run (.NET 8; the text of the output does not depend on the operating system):

```
Project: Accessible Personal Data Manager
Runtime: .NET 10
Build: dotnet build
Run: dotnet run
Test: dotnet test
Data: JSON file with six documented fields
Accessibility: keyboard-first commands and explicit text feedback
```

Common Mistakes

- Omitting a required record field or CRUD operation.
- Trusting persisted or user-supplied data without validation.
- Claiming behavior or accessibility that has not been verified.

Activities

1. Map the example to one capstone requirement and identify its inputs and outputs.
2. Add or modify a canonical record using all required fields.
3. Test a failure path such as duplicate ID, blank title, malformed JSON, or missing record.
4. Document how the behavior is tested and what remains unverified.

Accessibility Practice

Navigate the complete source file by line, word, and character using the learner's access technology. Read punctuation and indentation, run from the terminal, and inspect standard output and errors sequentially.

Runnable Reference Implementation

The repository includes a tested C# domain and JSON-storage reference core in capstone/README.md. Run `dotnet run --project CSHARP/capstone/Capstone.csproj` from the repository root. The reference core does not replace the requirement to build and evaluate an accessible user interface.

Skills Checklist

- ☐ I can implement the required capstone data model and fields.
- ☐ I can implement the required CRUD operations.
- ☐ I can validate data and handle persistence errors.
- ☐ I can test the application against acceptance criteria.
- ☐ I can document setup, operation, accessibility, and limitations.

AI Policy

AI may be used only for personal conceptual understanding and reinforcement. It must not write, complete, autocomplete, repair, or generate course code or assessment solutions.

Part VII

GO

Chapter 7

Go Curriculum

Before you start: complete Unit P: Computing Foundations Primer. It explains the language-neutral vocabulary and concepts used in every language track, using non-visual examples.Go is a compiled, statically typed language for cloud services, network servers, command-line tools, infrastructure, and concurrent systems.

A complete introductory course aligned to Python Units 0–11. The recurring examples make it easier to focus on Go's static types, packages, slices, maps, structs, interfaces, explicit errors, and concurrency.

Environment

Install Go from go.dev/dl. Verify `go version`. Create a project with `mkdir courseapp`, `cd courseapp`, `go mod init example.com/courseapp`; save `main.go` and run `go run .`.

Complete the shared IDE setup lesson and terminal prerequisite.

- Complete lessons
- Topic index

Lesson-quality refinement

The Unit 0–11 manuscript now has four complete setup lessons and 53 numbered lessons, including the missing Unit 2 Lesson 2.5 mathematics counterpart. Each numbered lesson has Go-specific explanations, a distinct standalone worked example, three common mistakes, four targeted activities, accessibility practice, and five measurable checklist criteria. The course uses the current stable Go toolchain and standard library.

Validation

`scripts/check_go_examples.py` runs gofmt, builds, and smoke-executes every fenced Go example independently. The GitHub Actions workflow `.github/workflows/go-examples.yml` runs this check on relevant changes. GitHub Actions run 41326670760 passed on Go 1.27.3; all 53 fenced examples were formatted with gofmt, built independently, and smoke-executed successfully. A successful run establishes formatting, compilation, and process exit—not exact-output correctness, complete capstone behavior, race freedom, or assistive-technology certification. See VALIDATION.md for scope and status.

GUI, visualization, and workbench accessibility

See the shared GUI, visualization, and workbench accessibility reference for screen-reader support by toolkit, platform-specific activation settings, recommended alternatives, and interfaces that must not be required without an equivalent accessible route.

AI Policy

AI may be used only for personal conceptual understanding and reinforcement. It must not write, complete, autocomplete, repair, or generate course code or assessment solutions.

Recommended introductory books

- McGavren, Jay. *Head First Go*. O'Reilly Media, 2019. A hands-on introduction to Go syntax, types, functions, collections, interfaces, concurrency, and common development practices. The author recommends some prior programming experience. Publisher page.
- Donovan, Alan A. A., and Brian W. Kernighan. *The Go Programming Language*. Addison-Wesley, 2015. A systematic introduction to Go with substantial examples; it is most useful after learners understand basic programming concepts. Official book site.

Suggested use: learners new to programming should complete the shared foundations and another introductory programming sequence before using these books as a primary text. *Head First Go* provides a more activity-oriented route; Donovan and Kernighan provide a deeper reference. Compare examples with the current Go version used by the course.

7.1 GO Lessons

The curriculum is organized as one Markdown file per unit.

- accessible environment and toolchain
- foundations of computing
- numeric computations
- control flow
- data structures
- input and iteration
- functions
- object oriented programming with go types
- files and exceptions
- testing and data analysis
- project based learning text based applications
- capstone accessible personal data manager

7.2 Unit 0: Accessible Environment and Toolchain

Course overview

Before you start: complete Unit P: Computing Foundations Primer. It explains the language-neutral vocabulary and concepts used in every language track, using non-visual examples.

Course Overview

Go is a compiled, statically typed language used for cloud services, network servers, command-line tools, infrastructure, and concurrent systems. This course follows the Python curriculum's shared examples and datasets while teaching Go idioms: packages and modules, explicit types, slices, maps, structs, interfaces, error values, goroutines, channels, and the standard library.

Toolchain baseline: current stable Go release. Use the official Go distribution, gofmt, modules, and the go command. The examples are self-contained standard-library programs unless a lesson explicitly names a package.

Unit 0: Accessible Environment and Toolchain

Overview

Install Go, create a module, configure editor support, and verify keyboard and screen-reader access.

Lesson 0.1: Install and Verify Go

Reading References

- Go installation
- A Tour of Go
- Go command documentation

Learning Objectives

- Install the official Go toolchain and verify go and gofmt are available.
- Verify the workflow with terminal commands and interpret their output.
- Complete the task using keyboard navigation and accessible diagnostics.

Concept Explanation

The Go distribution includes the compiler, standard library, go command, and formatter. Confirm that the intended installation appears first on PATH. Avoid mixing system and manually installed copies.

Worked Example

```
go version\ngofmt -h
```

Expected Behavior or Output

The terminal prints the installed Go version and gofmt usage information.

Common Mistakes

- Installing Go from an unofficial source or mixing multiple PATH installations.
- Confusing the editor language server with the Go compiler and toolchain.
- Running commands outside the intended module root.

Activities

1. Follow the commands from a new project directory and record each result.
2. Repeat the workflow using keyboard-only navigation.
3. Introduce one intentional source error and locate the diagnostic text.
4. Explain which files are source, module metadata, and generated build artifacts.

Accessibility Practice

Verify editor and terminal focus, punctuation announcement, indentation, diagnostic navigation, and sequential output reading. Document a keyboard-only route for build, run, and test.

Skills Checklist

- ☐ I can complete the setup task without relying on a mouse.
- ☐ I can identify the module root and source files.
- ☐ I can run the relevant command and interpret its result.
- ☐ I can identify a troubleshooting step from an error message.
- ☐ I can access source code and output with assistive technology.

Lesson 0.2: Modules, Workspace, and First Run

Reading References

- Go installation
- A Tour of Go
- Go command documentation

Learning Objectives

- Create a module and run a minimal Go executable.
- Verify the workflow with terminal commands and interpret their output.
- Complete the task using keyboard navigation and accessible diagnostics.

Concept Explanation

A module is described by go.mod. An executable starts in package main and defines func main(). The command go run . builds and runs the package from its module root.

Worked Example

```
mkdir courseapp\ncd courseapp\ngo mod init example.com/courseapp\nprintf 'package main\\nimp
```

Expected Behavior or Output

Go creates go.mod and runs main.go, printing Hello, Go!.

Common Mistakes

- Installing Go from an unofficial source or mixing multiple PATH installations.
- Confusing the editor language server with the Go compiler and toolchain.
- Running commands outside the intended module root.

Activities

1. Follow the commands from a new project directory and record each result.
2. Repeat the workflow using keyboard-only navigation.
3. Introduce one intentional source error and locate the diagnostic text.

4. Explain which files are source, module metadata, and generated build artifacts.

Accessibility Practice

Verify editor and terminal focus, punctuation announcement, indentation, diagnostic navigation, and sequential output reading. Document a keyboard-only route for build, run, and test.

Skills Checklist

- ☐ I can complete the setup task without relying on a mouse.
- ☐ I can identify the module root and source files.
- ☐ I can run the relevant command and interpret its result.
- ☐ I can identify a troubleshooting step from an error message.
- ☐ I can access source code and output with assistive technology.

Lesson 0.3: Editor and Accessibility Workflow

Reading References

- Go installation
- A Tour of Go
- Go command documentation

Learning Objectives

- Configure Go editor support and verify accessible source navigation and diagnostics.
- Verify the workflow with terminal commands and interpret their output.
- Complete the task using keyboard navigation and accessible diagnostics.

Concept Explanation

The Go extension and gopls provide language-aware navigation and diagnostics; they are not substitutes for go, gofmt, or go test. Test the actual screen-reader/editor combination. Keep terminal output and compiler messages available as text.

Worked Example

```
gofmt -w main.go\ngo test ./...\ngo build ./...
```

Expected Behavior or Output

The source is formatted, tests run (or report that none exist), and the module builds.

Common Mistakes

- Installing Go from an unofficial source or mixing multiple PATH installations.
- Confusing the editor language server with the Go compiler and toolchain.
- Running commands outside the intended module root.

Activities

1. Follow the commands from a new project directory and record each result.
2. Repeat the workflow using keyboard-only navigation.
3. Introduce one intentional source error and locate the diagnostic text.
4. Explain which files are source, module metadata, and generated build artifacts.

Accessibility Practice

Verify editor and terminal focus, punctuation announcement, indentation, diagnostic navigation, and sequential output reading. Document a keyboard-only route for build, run, and test.

Skills Checklist

☐ I can complete the setup task without relying on a mouse.
☐ I can identify the module root and source files.
☐ I can run the relevant command and interpret its result.
☐ I can identify a troubleshooting step from an error message.
☐ I can access source code and output with assistive technology.

Lesson 0.4: Project Environments, Dependencies, and Terminal Usage

Reading References

- Go installation
- A Tour of Go
- Go command documentation

Learning Objectives

- Manage module dependencies and run build, test, and execution commands from the module root.
- Verify the workflow with terminal commands and interpret their output.
- Complete the task using keyboard navigation and accessible diagnostics.

Concept Explanation

Go modules record direct requirements in go.mod and checksums in go.sum. Use go get to add a dependency, go mod tidy to reconcile imports and metadata, go

test ./... for tests, go build ./... to compile, and go run . to launch the current package. Review third-party dependencies before adding them.

Worked Example

```
go env GOMOD\ngo list ./...\ngo test ./...\ngo build ./...
```

Expected Behavior or Output

The commands identify the active module, list packages, run tests, and build packages.

Common Mistakes

- Installing Go from an unofficial source or mixing multiple PATH installations.
- Confusing the editor language server with the Go compiler and toolchain.
- Running commands outside the intended module root.

Activities

1. Follow the commands from a new project directory and record each result.
2. Repeat the workflow using keyboard-only navigation.
3. Introduce one intentional source error and locate the diagnostic text.
4. Explain which files are source, module metadata, and generated build artifacts.

Accessibility Practice

Verify editor and terminal focus, punctuation announcement, indentation, diagnostic navigation, and sequential output reading. Document a keyboard-only route for build, run, and test.

Skills Checklist

- ☐ I can complete the setup task without relying on a mouse.
- ☐ I can identify the module root and source files.
- ☐ I can run the relevant command and interpret its result.
- ☐ I can identify a troubleshooting step from an error message.
- ☐ I can access source code and output with assistive technology.

7.3 Unit 1: Foundations of Computing

Overview

This unit develops foundations of computing using Go's types, packages, and standard-library conventions. Prerequisites: the previous units and a working

Go module. Use shared canonical course datasets and test boundary conditions, error paths, and accessible interaction.

Lesson 1.1: Computer science and problem solving

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain computer science and problem solving using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

package main import (”fmt”) func main() { fmt.Println(”A program is a precise set of instructions.”) },package main import (”fmt”) func main() { name := ”Avery” fmt.Println(”Hello,”, name) fmt.Println(”This line runs second.”) },package main import (”fmt”) func main() { var age int = 16 height := 1.72 active := true fmt.Printf(”age=%d height=%.2f active=%t\n”, age, height, active) } The lesson uses the canonical course data so learners focus on Go semantics rather than a new domain problem. Format with gofmt and use go test ./... as routine development tools.

Worked Example

```
package main
import (
    "fmt"
)
func main() {
    fmt.Println("A program is a precise set of instructions.")
}
```

Example 1.1.2

```
package main
import (
    "fmt"
)
```

```go
func main() {
    name := "Avery"
    fmt.Println("Hello,", name)
    fmt.Println("This line runs second.")
}
```

Example 1.1.3

```go
package main
import (
    "fmt"
)
func main() {
    var age int = 16
    height := 1.72
    active := true
    fmt.Printf("age=%d height=%.2f active=%t\n", age, height, active)
}
```

Expected Behavior or Output

Output observed when each example was run with empty standard input (Go 1.22; the text of the output does not depend on the operating system):

Example 1:

```
A program is a precise set of instructions.
```

Example 2:

```
Hello, Avery
This line runs second.
```

Example 3:

```
age=16 height=1.72 active=true
```

Common Mistakes

1. Treating a computer as if it inferred unstated requirements.
2. Omitting package main or the required main entry point in an executable.
3. Using a variable without declaring it or misunderstanding inferred types.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical Avery's student profile, calculator bill/tip, and grade thresholds data and predict the output before running the program.
3. Add one boundary case and one malformed or missing-data case; state expected behavior before changing the code.

4. Refactor one part using the Go feature named in this lesson (computer science and problem solving); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect punctuation in Go syntax (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain computer science and problem solving using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 1.2: Your first Go program and packages

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain your first go program and packages using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

Every Go executable lives in `package main` and starts at `func main()`. The `import` block names the packages the file uses, and an unused import is a compile error. `fmt.Println` writes its arguments separated by spaces and ends with a new line; `gofmt` formats the file so every Go program looks the same.

Worked Example

Example 1.2.1

```
package main
import (
    "fmt"
)
func main() {
    fmt.Println("A program is a precise set of instructions.")
},package main
import (
    "fmt"
)
func main() {
    name := "Avery"
    fmt.Println("Hello,", name)
    fmt.Println("This line runs second.")
},package main
import (
    "fmt"
)
func main() {
    var age int = 16
    height := 1.72
    active := true
    fmt.Printf("age=%d height=%.2f active=%t\n", age, height, active)
} The lesson uses the canonical course data so learners focus on Go semantics rather than a

### Worked Example
```go
package main
import (
    "fmt"
)
func main() {
    name := "Avery"
    fmt.Println("Hello,", name)
    fmt.Println("This line runs second.")
}
```

Example 1.2.3

```
package main
import (
    "fmt"
)
```
```

```go
func main() {
    var age int = 16
    height := 1.72
    active := true
    fmt.Printf("age=%d height=%.2f active=%t\n", age, height, active)
}
```

Expected Behavior or Output

Output observed when each example was run with empty standard input (Go 1.22; the text of the output does not depend on the operating system):

Example 1:

```
A program is a precise set of instructions.
```

Example 2:

```
Hello, Avery
This line runs second.
```

Example 3:

```
age=16 height=1.72 active=true
```

Common Mistakes

1. Treating a computer as if it inferred unstated requirements.
2. Omitting package main or the required main entry point in an executable.
3. Using a variable without declaring it or misunderstanding inferred types.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical Avery's student profile, calculator bill/tip, and grade thresholds data and predict the output before running the program.
3. Add one boundary case and one malformed or missing-data case; state expected behavior before changing the code.
4. Refactor one part using the Go feature named in this lesson (your first go program and packages); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect punctuation in Go syntax (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain your first go program and packages using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 1.3: Variables, constants, and primitive types

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain variables, constants, and primitive types using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

Go is statically typed: every variable has a type fixed at compile time. `var age int = 16` declares the type explicitly, while `height := 1.72` declares and infers `float64` inside a function. Constants use `const`, and an unassigned variable holds its type's zero value (0, "", false) rather than an undefined value.

Worked Example

Example 1.3.1

```
package main
import (
    "fmt"
)
func main() {
    fmt.Println("A program is a precise set of instructions.")
}
```

Example 1.3.2

```
package main
import (
    "fmt"
)
func main() {
    name := "Avery"
    fmt.Println("Hello,", name)
    fmt.Println("This line runs second.")
},package main
import (
    "fmt"
)
func main() {
    var age int = 16
    height := 1.72
    active := true
    fmt.Printf("age=%d height=%.2f active=%t\n", age, height, active)
} The lesson uses the canonical course data so learners focus on Go semantics rather than a

### Worked Example
```go
package main
import (
    "fmt"
)
func main() {
    var age int = 16
    height := 1.72
    active := true
    fmt.Printf("age=%d height=%.2f active=%t\n", age, height, active)
}
```

Expected Behavior or Output

Output observed when each example was run with empty standard input (Go 1.22; the text of the output does not depend on the operating system):

Example 1:

```
A program is a precise set of instructions.
```

Example 2:

```
Hello, Avery
This line runs second.
```

Example 3:

```
age=16 height=1.72 active=true
```
```

Common Mistakes

1. Treating a computer as if it inferred unstated requirements.
2. Omitting package main or the required main entry point in an executable.
3. Using a variable without declaring it or misunderstanding inferred types.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical Avery's student profile, calculator bill/tip, and grade thresholds data and predict the output before running the program.
3. Add one boundary case and one malformed or missing-data case; state expected behavior before changing the code.
4. Refactor one part using the Go feature named in this lesson (variables, constants, and primitive types); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect punctuation in Go syntax (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain variables, constants, and primitive types using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

7.4 Unit 2: Numeric Computations

Overview

This unit develops numeric computations using Go's types, packages, and standard-library conventions. Prerequisites: the previous units and a working Go module. Use shared canonical course datasets and test boundary conditions, error paths, and accessible interaction.

Lesson 2.1: Software design and tracing

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain software design and tracing using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

package main import (”fmt”) func main() { bill := 24.50 rate := 0.15 tip := bill * rate fmt.Printf(”Tip: %.2f; total: %.2f\n”, tip, bill+tip) },package main import (”fmt”) func main() { subtotal := 24.50 taxRate := 0.075 tax := subtotal * taxRate fmt.Printf(”Subtotal %.2f, tax %.2f, total %.2f\n”, subtotal, tax, subtotal+tax) },package main import (”bufio” ”fmt” ”os” ”strconv”) func main() { reader := bufio.NewReader(os.Stdin) fmt.Print(”Enter a score: ”) text, _ := reader.ReadString('\n') score, err := strconv.Atoi(text) if err != nil { fmt.Println(”Enter a whole number.”) } else { fmt.Println(”Score:”, score) } },package main import (”fmt” ”math”) func main() { radius := 3.0 area := math.Pi * math.Pow(radius, 2) circumference := 2 * math.Pi * radius fmt.Printf(”area=%.2f circumference=%.2f\n”, area, circumference) },package main import (”fmt” ”math”) func main() { distance := math.Hypot(3, 4) fmt.Println(”Distance from origin:”, distance) } The lesson uses the canonical course data so learners focus on Go semantics rather than a new domain problem. Format with gofmt and use go test ./... as routine development tools.

Worked Example

```go
package main
import (
    "fmt"
)
func main() {
    bill := 24.50
    rate := 0.15
    tip := bill * rate
    fmt.Printf("Tip: %.2f; total: %.2f\n", tip, bill+tip)
}
```

Example 2.1.2

```
package main
import (
    "fmt"
)
func main() {
    subtotal := 24.50
    taxRate := 0.075
    tax := subtotal * taxRate
    fmt.Printf("Subtotal %.2f, tax %.2f, total %.2f\n", subtotal, tax, subtotal+tax)
}
```

Example 2.1.3

```
package main
import (
    "bufio"
    "fmt"
    "os"
    "strconv"
)
func main() {
    reader := bufio.NewReader(os.Stdin)
    fmt.Print("Enter a score: ")
    text, _ := reader.ReadString('\n')
    score, err := strconv.Atoi(text)
    if err != nil { fmt.Println("Enter a whole number.") } else { fmt.Println("Score:", sco
}
```

Example 2.1.4

```
package main
import (
    "fmt"
    "math"
)
func main() {
    radius := 3.0
    area := math.Pi * math.Pow(radius, 2)
    circumference := 2 * math.Pi * radius
    fmt.Printf("area=%.2f circumference=%.2f\n", area, circumference)
}
```

Example 2.1.5

```
package main
import (
    "fmt"
```

```
    "math"
)
func main() {
    distance := math.Hypot(3, 4)
    fmt.Println("Distance from origin:", distance)
}
```

Expected Behavior or Output

Output observed when each example was run with empty standard input (Go 1.22; the text of the output does not depend on the operating system):

Example 1:

```
Tip: 3.67; total: 28.18
```

Example 2:

```
Subtotal 24.50, tax 1.84, total 26.34
```

Example 3:

```
Enter a score: Enter a whole number.
```

Example 4:

```
area=28.27 circumference=18.85
```

Example 5:

```
Distance from origin: 5
```

Common Mistakes

1. Confusing := declaration with = assignment.
2. Ignoring errors returned by fmt.Fscan or strconv parsing.
3. Mixing int and float64 without explicit conversion.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical Avery's student profile, calculator bill/tip, and grade thresholds data and predict the output before running the program.
3. Add one boundary case and one malformed or missing-data case; state expected behavior before changing the code.
4. Refactor one part using the Go feature named in this lesson (software design and tracing); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect punctuation in Go syntax (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain software design and tracing using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 2.2: Expressions, operators, and assignment

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain expressions, operators, and assignment using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

An expression produces a value. Go's arithmetic operators are `+`, `-`, `*`, `/`, and `%`; dividing two integers truncates toward zero, so `7 / 2` is `3`. Go has no implicit numeric conversion, so an `int` and a `float64` cannot be mixed until one is converted. Assignment stores a result, and `+=` updates a variable in place.

Worked Example

Example 2.2.1

```
package main
import (
    "fmt"
)
func main() {
    bill := 24.50
    rate := 0.15
    tip := bill * rate
    fmt.Printf("Tip: %.2f; total: %.2f\n", tip, bill+tip)
},package main
import (
    "fmt"
)
func main() {
    subtotal := 24.50
    taxRate := 0.075
    tax := subtotal * taxRate
    fmt.Printf("Subtotal %.2f, tax %.2f, total %.2f\n", subtotal, tax, subtotal+tax)
},package main
import (
    "bufio"
    "fmt"
    "os"
    "strconv"
)
func main() {
    reader := bufio.NewReader(os.Stdin)
    fmt.Print("Enter a score: ")
    text, _ := reader.ReadString('\n')
    score, err := strconv.Atoi(text)
    if err != nil { fmt.Println("Enter a whole number.") } else { fmt.Println("Score:", sco
},package main
import (
    "fmt"
    "math"
)
func main() {
    radius := 3.0
    area := math.Pi * math.Pow(radius, 2)
    circumference := 2 * math.Pi * radius
    fmt.Printf("area=%.2f circumference=%.2f\n", area, circumference)
},package main
import (
    "fmt"
    "math"
)
```

```
func main() {
    distance := math.Hypot(3, 4)
    fmt.Println("Distance from origin:", distance)
} The lesson uses the canonical course data so learners focus on Go semantics rather than a

### Worked Example
```go
package main
import (
    "fmt"
)
func main() {
    subtotal := 24.50
    taxRate := 0.075
    tax := subtotal * taxRate
    fmt.Printf("Subtotal %.2f, tax %.2f, total %.2f\n", subtotal, tax, subtotal+tax)
}
```

Example 2.2.3

```
package main
import (
    "bufio"
    "fmt"
    "os"
    "strconv"
)
func main() {
    reader := bufio.NewReader(os.Stdin)
    fmt.Print("Enter a score: ")
    text, _ := reader.ReadString('\n')
    score, err := strconv.Atoi(text)
    if err != nil { fmt.Println("Enter a whole number.") } else { fmt.Println("Score:", scor
}
```

Example 2.2.4

```
package main
import (
    "fmt"
    "math"
)
func main() {
    radius := 3.0
    area := math.Pi * math.Pow(radius, 2)
    circumference := 2 * math.Pi * radius
    fmt.Printf("area=%.2f circumference=%.2f\n", area, circumference)
}
```
```

Example 2.2.5

```go
package main
import (
    "fmt"
    "math"
)
func main() {
    distance := math.Hypot(3, 4)
    fmt.Println("Distance from origin:", distance)
}
```

Expected Behavior or Output

Output observed when each example was run with empty standard input (Go 1.22; the text of the output does not depend on the operating system):

Example 1:

```
Tip: 3.67; total: 28.18
```

Example 2:

```
Subtotal 24.50, tax 1.84, total 26.34
```

Example 3:

```
Enter a score: Enter a whole number.
```

Example 4:

```
area=28.27 circumference=18.85
```

Example 5:

```
Distance from origin: 5
```

Common Mistakes

1. Confusing := declaration with = assignment.
2. Ignoring errors returned by fmt.Fscan or strconv parsing.
3. Mixing int and float64 without explicit conversion.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical Avery's student profile, calculator bill/tip, and grade thresholds data and predict the output before running the program.
3. Add one boundary case and one malformed or missing-data case; state expected behavior before changing the code.

4. Refactor one part using the Go feature named in this lesson (expressions, operators, and assignment); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect punctuation in Go syntax (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain expressions, operators, and assignment using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 2.3: Input and output with fmt

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain input and output with fmt using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

The `fmt` package writes with `Println` and `Printf` and reads with `Scanln` or a `bufio.Scanner`. Reading returns an error value as well as data, and Go programs check it: at the end of input the error is `io.EOF`. Typed text is a string, so a number needs `strconv.Atoi` or `strconv.ParseFloat`, which also return an error for invalid text.

Worked Example

Example 2.3.1

```
package main
import (
    "fmt"
)
func main() {
    bill := 24.50
    rate := 0.15
    tip := bill * rate
    fmt.Printf("Tip: %.2f; total: %.2f\n", tip, bill+tip)
}
```

Example 2.3.2

```
package main
import (
    "fmt"
)
func main() {
    subtotal := 24.50
    taxRate := 0.075
    tax := subtotal * taxRate
    fmt.Printf("Subtotal %.2f, tax %.2f, total %.2f\n", subtotal, tax, subtotal+tax)
},package main
import (
    "bufio"
    "fmt"
    "os"
    "strconv"
)
func main() {
    reader := bufio.NewReader(os.Stdin)
    fmt.Print("Enter a score: ")
    text, _ := reader.ReadString('\n')
    score, err := strconv.Atoi(text)
    if err != nil { fmt.Println("Enter a whole number.") } else { fmt.Println("Score:", sco
},package main
import (
    "fmt"
    "math"
)
func main() {
    radius := 3.0
    area := math.Pi * math.Pow(radius, 2)
```

```
    circumference := 2 * math.Pi * radius
    fmt.Printf("area=%.2f circumference=%.2f\n", area, circumference)
},package main
import (
    "fmt"
    "math"
)
func main() {
    distance := math.Hypot(3, 4)
    fmt.Println("Distance from origin:", distance)
} The lesson uses the canonical course data so learners focus on Go semantics rather than a

### Worked Example
```go
package main
import (
    "bufio"
    "fmt"
    "os"
    "strconv"
)
func main() {
    reader := bufio.NewReader(os.Stdin)
    fmt.Print("Enter a score: ")
    text, _ := reader.ReadString('\n')
    score, err := strconv.Atoi(text)
    if err != nil { fmt.Println("Enter a whole number.") } else { fmt.Println("Score:", sco
}
```

Example 2.3.4

```
package main
import (
    "fmt"
    "math"
)
func main() {
    radius := 3.0
    area := math.Pi * math.Pow(radius, 2)
    circumference := 2 * math.Pi * radius
    fmt.Printf("area=%.2f circumference=%.2f\n", area, circumference)
}
```

Example 2.3.5

```
package main
import (
    "fmt"
```
```

```
    "math"
)
func main() {
    distance := math.Hypot(3, 4)
    fmt.Println("Distance from origin:", distance)
}
```

Expected Behavior or Output

Output observed when each example was run with empty standard input (Go 1.22; the text of the output does not depend on the operating system):

Example 1:

```
Tip: 3.67; total: 28.18
```

Example 2:

```
Subtotal 24.50, tax 1.84, total 26.34
```

Example 3:

```
Enter a score: Enter a whole number.
```

Example 4:

```
area=28.27 circumference=18.85
```

Example 5:

```
Distance from origin: 5
```

Common Mistakes

1. Confusing := declaration with = assignment.
2. Ignoring errors returned by fmt.Fscan or strconv parsing.
3. Mixing int and float64 without explicit conversion.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical Avery's student profile, calculator bill/tip, and grade thresholds data and predict the output before running the program.
3. Add one boundary case and one malformed or missing-data case; state expected behavior before changing the code.
4. Refactor one part using the Go feature named in this lesson (input and output with fmt); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect punctuation in Go syntax (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain input and output with fmt using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 2.4: Numeric conversions and math

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain numeric conversions and math using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

Go converts types explicitly: `float64(count)` and `int(price)`. Converting a float to an integer discards the fraction and does not round. `strconv` converts between text and numbers and reports invalid text through an error. Floating-point values are approximations, which is why `0.1 + 0.2` is not exactly `0.3`.

Worked Example

Example 2.4.1

```
package main
import (
    "fmt"
)
func main() {
    bill := 24.50
    rate := 0.15
    tip := bill * rate
    fmt.Printf("Tip: %.2f; total: %.2f\n", tip, bill+tip)
}
```

Example 2.4.2

```
package main
import (
    "fmt"
)
func main() {
    subtotal := 24.50
    taxRate := 0.075
    tax := subtotal * taxRate
    fmt.Printf("Subtotal %.2f, tax %.2f, total %.2f\n", subtotal, tax, subtotal+tax)
}
```

Example 2.4.3

```
package main
import (
    "bufio"
    "fmt"
    "os"
    "strconv"
)
func main() {
    reader := bufio.NewReader(os.Stdin)
    fmt.Print("Enter a score: ")
    text, _ := reader.ReadString('\n')
    score, err := strconv.Atoi(text)
    if err != nil { fmt.Println("Enter a whole number.") } else { fmt.Println("Score:", sco
},package main
import (
    "fmt"
    "math"
)
func main() {
    radius := 3.0
    area := math.Pi * math.Pow(radius, 2)
    circumference := 2 * math.Pi * radius
```

```
    fmt.Printf("area=%.2f circumference=%.2f\n", area, circumference)
},package main
import (
    "fmt"
    "math"
)
func main() {
    distance := math.Hypot(3, 4)
    fmt.Println("Distance from origin:", distance)
} The lesson uses the canonical course data so learners focus on Go semantics rather than a

### Worked Example
```go
package main
import (
    "fmt"
    "math"
)
func main() {
    radius := 3.0
    area := math.Pi * math.Pow(radius, 2)
    circumference := 2 * math.Pi * radius
    fmt.Printf("area=%.2f circumference=%.2f\n", area, circumference)
}
```

Example 2.4.5

```
package main
import (
    "fmt"
    "math"
)
func main() {
    distance := math.Hypot(3, 4)
    fmt.Println("Distance from origin:", distance)
}
```

Expected Behavior or Output

Output observed when each example was run with empty standard input (Go 1.22; the text of the output does not depend on the operating system):

Example 1:

```
Tip: 3.67; total: 28.18
```

Example 2:

```
Subtotal 24.50, tax 1.84, total 26.34
```
```

Example 3:

```
Enter a score: Enter a whole number.
```

Example 4:

```
area=28.27 circumference=18.85
```

Example 5:

```
Distance from origin: 5
```

Common Mistakes

1. Confusing := declaration with = assignment.
2. Ignoring errors returned by fmt.Fscan or strconv parsing.
3. Mixing int and float64 without explicit conversion.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical Avery's student profile, calculator bill/tip, and grade thresholds data and predict the output before running the program.
3. Add one boundary case and one malformed or missing-data case; state expected behavior before changing the code.
4. Refactor one part using the Go feature named in this lesson (numeric conversions and math); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect punctuation in Go syntax (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain numeric conversions and math using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 2.5: Mathematical functions, circle geometry, and distance

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain mathematical functions, circle geometry, and distance using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

The `math` package provides `Pi`, `Sqrt`, `Pow`, `Floor`, and `Hypot`. Go has a built-in hypotenuse function, so the distance from the origin to (3, 4) is `math.Hypot(3, 4)`, which is exactly 5. Every `math` function takes and returns `float64`, so integers must be converted first.

Worked Example

Example 2.5.1

```
package main
import (
    "fmt"
)
func main() {
    bill := 24.50
    rate := 0.15
    tip := bill * rate
    fmt.Printf("Tip: %.2f; total: %.2f\n", tip, bill+tip)
}
```

Example 2.5.2

```
package main
import (
    "fmt"
)
func main() {
    subtotal := 24.50
```

```
    taxRate := 0.075
    tax := subtotal * taxRate
    fmt.Printf("Subtotal %.2f, tax %.2f, total %.2f\n", subtotal, tax, subtotal+tax)
}
```

Example 2.5.3

```
package main
import (
    "bufio"
    "fmt"
    "os"
    "strconv"
)
func main() {
    reader := bufio.NewReader(os.Stdin)
    fmt.Print("Enter a score: ")
    text, _ := reader.ReadString('\n')
    score, err := strconv.Atoi(text)
    if err != nil { fmt.Println("Enter a whole number.") } else { fmt.Println("Score:", sco
}
```

Example 2.5.4

```
package main
import (
    "fmt"
    "math"
)
func main() {
    radius := 3.0
    area := math.Pi * math.Pow(radius, 2)
    circumference := 2 * math.Pi * radius
    fmt.Printf("area=%.2f circumference=%.2f\n", area, circumference)
},package main
import (
    "fmt"
    "math"
)
func main() {
    distance := math.Hypot(3, 4)
    fmt.Println("Distance from origin:", distance)
} The lesson uses the canonical course data so learners focus on Go semantics rather than a

### Worked Example
```go
package main
import (
```
```

```
    "fmt"
    "math"
)
func main() {
    distance := math.Hypot(3, 4)
    fmt.Println("Distance from origin:", distance)
}
```

Expected Behavior or Output

Output observed when each example was run with empty standard input (Go 1.22; the text of the output does not depend on the operating system):

Example 1:

```
Tip: 3.67; total: 28.18
```

Example 2:

```
Subtotal 24.50, tax 1.84, total 26.34
```

Example 3:

```
Enter a score: Enter a whole number.
```

Example 4:

```
area=28.27 circumference=18.85
```

Example 5:

```
Distance from origin: 5
```

Common Mistakes

1. Confusing := declaration with = assignment.
2. Ignoring errors returned by fmt.Fscan or strconv parsing.
3. Mixing int and float64 without explicit conversion.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical Avery's student profile, calculator bill/tip, and grade thresholds data and predict the output before running the program.
3. Add one boundary case and one malformed or missing-data case; state expected behavior before changing the code.
4. Refactor one part using the Go feature named in this lesson (mathematical functions, circle geometry, and distance); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect punctuation in Go syntax (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain mathematical functions, circle geometry, and distance using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

7.5 Unit 3: Control Flow

Overview

This unit develops control flow using Go's types, packages, and standard-library conventions. Prerequisites: the previous units and a working Go module. Use shared canonical course datasets and test boundary conditions, error paths, and accessible interaction.

Lesson 3.1: Boolean expressions and if statements

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain boolean expressions and if statements using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

package main import (”fmt”) func main() { score := 82 fmt.Printf(”score=%d; passing=%t\n”, score, score >= 60) },package main import (”fmt”) func main() { scores := []int{82, 91, 76, 88, 59} for _, score := range scores { grade := ”F” if score >= 90 { grade = ”A” } else if score >= 80 { grade = ”B” } else if score >= 70 { grade = ”C” } else if score >= 60 { grade = ”D” } fmt.Printf(”%d: %s\n”, score, grade) } },package main import (”fmt”) func main() { scores := []int{82, 91, 76, 88} fmt.Printf(”first=%d last=%d count=%d\n”, scores[0], scores[len(scores)-1], len(scores)) },package main import (”fmt”) func main() { scores := []int{82, 91, 76, 88} total := 0 for _, score := range scores { total += score } fmt.Printf(”total=%d average=%.1f\n”, total, float64(total)/float64(len(scores))) },package main import (”fmt” ”strings”) func main() { sample := ”A student studies programming.” fmt.Printf(”bytes=%d runes=%d words=%d\n”, len(sample), len([]rune(sample)), len(strings.Fields(sample))) } The lesson uses the canonical course data so learners focus on Go semantics rather than a new domain problem. Format with gofmt and use go test ./... as routine development tools.

Worked Example

```
package main
import (
    "fmt"
)
func main() {
    score := 82
    fmt.Printf("score=%d; passing=%t\n", score, score >= 60)
}
```

Example 3.1.2

```
package main
import (
    "fmt"
)
func main() {
    scores := []int{82, 91, 76, 88, 59}
    for _, score := range scores {
     grade := "F"
     if score >= 90 { grade = "A" } else if score >= 80 { grade = "B" } else if score >= 70
     fmt.Printf("%d: %s\n", score, grade)
    }
}
```

Example 3.1.3

```
package main
```

```
import (
    "fmt"
)
func main() {
    scores := []int{82, 91, 76, 88}
    fmt.Printf("first=%d last=%d count=%d\n", scores[0], scores[len(scores)-1], len(scores))
}
```

Example 3.1.4

```
package main
import (
    "fmt"
)
func main() {
    scores := []int{82, 91, 76, 88}
    total := 0
    for _, score := range scores { total += score }
    fmt.Printf("total=%d average=%.1f\n", total, float64(total)/float64(len(scores)))
}
```

Example 3.1.5

```
package main
import (
    "fmt"
    "strings"
)
func main() {
    sample := "A student studies programming."
    fmt.Printf("bytes=%d runes=%d words=%d\n", len(sample), len([]rune(sample)), len(strings
}
```

Expected Behavior or Output

Output observed when each example was run with empty standard input (Go 1.22; the text of the output does not depend on the operating system):

Example 1:

```
score=82; passing=true
```

Example 2:

```
82: B
91: A
76: C
88: B
59: F
```

Example 3:

```
first=82 last=88 count=4
```

Example 4:

```
total=337 average=84.2
```

Example 5:

```
bytes=30 runes=30 words=4
```

Common Mistakes

1. Assuming Go has while or do-while keywords instead of for forms.
2. Indexing a slice without checking its length.
3. Using len(string) when the intended count is Unicode code points.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical Avery's student profile, calculator bill/tip, and grade thresholds data and predict the output before running the program.
3. Add one boundary case and one malformed or missing-data case; state expected behavior before changing the code.
4. Refactor one part using the Go feature named in this lesson (boolean expressions and if statements); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect punctuation in Go syntax (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain boolean expressions and if statements using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 3.2: Chained conditionals and grade classification

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain chained conditionals and grade classification using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

Go chains conditions with `else if` and ends with `else`, testing from top to bottom and running only the first branch that is true. The shared grade thresholds are 90, 80, 70, 60, otherwise F, so each later test needs only a lower bound. A `switch` with no value (`switch { case score >= 90: ... }`) is an equivalent and often clearer form.

Worked Example

Example 3.2.1

```
package main
import (
    "fmt"
)
func main() {
    score := 82
    fmt.Printf("score=%d; passing=%t\n", score, score >= 60)
},package main
import (
    "fmt"
)
func main() {
    scores := []int{82, 91, 76, 88, 59}
    for _, score := range scores {
     grade := "F"
     if score >= 90 { grade = "A" } else if score >= 80 { grade = "B" } else if score >= 70
     fmt.Printf("%d: %s\n", score, grade)
    }
```

```
},package main
import (
    "fmt"
)
func main() {
    scores := []int{82, 91, 76, 88}
    fmt.Printf("first=%d last=%d count=%d\n", scores[0], scores[len(scores)-1], len(scores))
},package main
import (
    "fmt"
)
func main() {
    scores := []int{82, 91, 76, 88}
    total := 0
    for _, score := range scores { total += score }
    fmt.Printf("total=%d average=%.1f\n", total, float64(total)/float64(len(scores)))
},package main
import (
    "fmt"
    "strings"
)
func main() {
    sample := "A student studies programming."
    fmt.Printf("bytes=%d runes=%d words=%d\n", len(sample), len([]rune(sample)), len(strings
} The lesson uses the canonical course data so learners focus on Go semantics rather than a

### Worked Example
```go
package main
import (
    "fmt"
)
func main() {
    scores := []int{82, 91, 76, 88, 59}
    for _, score := range scores {
     grade := "F"
     if score >= 90 { grade = "A" } else if score >= 80 { grade = "B" } else if score >= 70
     fmt.Printf("%d: %s\n", score, grade)
    }
}
```

Example 3.2.3

```
package main
import (
    "fmt"
)
```
```

```
func main() {
    scores := []int{82, 91, 76, 88}
    fmt.Printf("first=%d last=%d count=%d\n", scores[0], scores[len(scores)-1], len(scores))
}
```

Example 3.2.4

```
package main
import (
    "fmt"
)
func main() {
    scores := []int{82, 91, 76, 88}
    total := 0
    for _, score := range scores { total += score }
    fmt.Printf("total=%d average=%.1f\n", total, float64(total)/float64(len(scores)))
}
```

Example 3.2.5

```
package main
import (
    "fmt"
    "strings"
)
func main() {
    sample := "A student studies programming."
    fmt.Printf("bytes=%d runes=%d words=%d\n", len(sample), len([]rune(sample)), len(strings
}
```

Expected Behavior or Output

Output observed when each example was run with empty standard input (Go 1.22; the text of the output does not depend on the operating system):

Example 1:

```
score=82; passing=true
```

Example 2:

```
82: B
91: A
76: C
88: B
59: F
```

Example 3:

```
first=82 last=88 count=4
```

Example 4:

```
total=337 average=84.2
```

Example 5:

```
bytes=30 runes=30 words=4
```

Common Mistakes

1. Assuming Go has while or do-while keywords instead of for forms.
2. Indexing a slice without checking its length.
3. Using len(string) when the intended count is Unicode code points.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical Avery's student profile, calculator bill/tip, and grade thresholds data and predict the output before running the program.
3. Add one boundary case and one malformed or missing-data case; state expected behavior before changing the code.
4. Refactor one part using the Go feature named in this lesson (chained conditionals and grade classification); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect punctuation in Go syntax (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain chained conditionals and grade classification using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 3.3: Arrays, slices, and indexing

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain arrays, slices, and indexing using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

An array has a fixed length that is part of its type; a slice is a view of an array that can grow with `append`. Indexes start at 0, `len` gives the length, and indexing outside the slice stops the program with a run-time panic. A slice assigned to another variable shares the same underlying elements.

Worked Example

Example 3.3.1

```
package main
import (
    "fmt"
)
func main() {
    score := 82
    fmt.Printf("score=%d; passing=%t\n", score, score >= 60)
}
```

Example 3.3.2

```
package main
import (
    "fmt"
)
func main() {
    scores := []int{82, 91, 76, 88, 59}
    for _, score := range scores {
     grade := "F"
     if score >= 90 { grade = "A" } else if score >= 80 { grade = "B" } else if score >= 70
```

```
     fmt.Printf("%d: %s\n", score, grade)
    }
},package main
import (
    "fmt"
)
func main() {
    scores := []int{82, 91, 76, 88}
    fmt.Printf("first=%d last=%d count=%d\n", scores[0], scores[len(scores)-1], len(scores)
},package main
import (
    "fmt"
)
func main() {
    scores := []int{82, 91, 76, 88}
    total := 0
    for _, score := range scores { total += score }
    fmt.Printf("total=%d average=%.1f\n", total, float64(total)/float64(len(scores)))
},package main
import (
    "fmt"
    "strings"
)
func main() {
    sample := "A student studies programming."
    fmt.Printf("bytes=%d runes=%d words=%d\n", len(sample), len([]rune(sample)), len(strings
} The lesson uses the canonical course data so learners focus on Go semantics rather than a

### Worked Example
```go
package main
import (
    "fmt"
)
func main() {
    scores := []int{82, 91, 76, 88}
    fmt.Printf("first=%d last=%d count=%d\n", scores[0], scores[len(scores)-1], len(scores)
}
```

Example 3.3.4

```
package main
import (
    "fmt"
)
func main() {
    scores := []int{82, 91, 76, 88}
```
```

```
    total := 0
    for _, score := range scores { total += score }
    fmt.Printf("total=%d average=%.1f\n", total, float64(total)/float64(len(scores)))
}
```

Example 3.3.5

```
package main
import (
    "fmt"
    "strings"
)
func main() {
    sample := "A student studies programming."
    fmt.Printf("bytes=%d runes=%d words=%d\n", len(sample), len([]rune(sample)), len(strings
}
```

Expected Behavior or Output

Output observed when each example was run with empty standard input (Go 1.22; the text of the output does not depend on the operating system):

Example 1:

```
score=82; passing=true
```

Example 2:

```
82: B
91: A
76: C
88: B
59: F
```

Example 3:

```
first=82 last=88 count=4
```

Example 4:

```
total=337 average=84.2
```

Example 5:

```
bytes=30 runes=30 words=4
```

Common Mistakes

1. Assuming Go has while or do-while keywords instead of for forms.
2. Indexing a slice without checking its length.
3. Using len(string) when the intended count is Unicode code points.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical Avery's student profile, calculator bill/tip, and grade thresholds data and predict the output before running the program.
3. Add one boundary case and one malformed or missing-data case; state expected behavior before changing the code.
4. Refactor one part using the Go feature named in this lesson (arrays, slices, and indexing); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect punctuation in Go syntax (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain arrays, slices, and indexing using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 3.4: For loops and range

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain for loops and range using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

Go has one loop keyword, `for`. `for i := 0; i < n; i++` counts, `for condition {}` repeats while a condition holds, and `for index, value := range slice` visits every element. Use `_` for a value you do not need, since an unused variable is a compile error. The range stops at the end of the slice, so no index checks are needed.

Worked Example

Example 3.4.1

```
package main
import (
    "fmt"
)
func main() {
    score := 82
    fmt.Printf("score=%d; passing=%t\n", score, score >= 60)
}
```

Example 3.4.2

```
package main
import (
    "fmt"
)
func main() {
    scores := []int{82, 91, 76, 88, 59}
    for _, score := range scores {
     grade := "F"
     if score >= 90 { grade = "A" } else if score >= 80 { grade = "B" } else if score >= 70
     fmt.Printf("%d: %s\n", score, grade)
    }
}
```

Example 3.4.3

```
package main
import (
    "fmt"
)
func main() {
    scores := []int{82, 91, 76, 88}
    fmt.Printf("first=%d last=%d count=%d\n", scores[0], scores[len(scores)-1], len(scores))
},package main
import (
    "fmt"
)
```

```
func main() {
    scores := []int{82, 91, 76, 88}
    total := 0
    for _, score := range scores { total += score }
    fmt.Printf("total=%d average=%.1f\n", total, float64(total)/float64(len(scores)))
},package main
import (
    "fmt"
    "strings"
)
func main() {
    sample := "A student studies programming."
    fmt.Printf("bytes=%d runes=%d words=%d\n", len(sample), len([]rune(sample)), len(strings
} The lesson uses the canonical course data so learners focus on Go semantics rather than a

### Worked Example
```go
package main
import (
    "fmt"
)
func main() {
    scores := []int{82, 91, 76, 88}
    total := 0
    for _, score := range scores { total += score }
    fmt.Printf("total=%d average=%.1f\n", total, float64(total)/float64(len(scores)))
}
```

Example 3.4.5

```
package main
import (
    "fmt"
    "strings"
)
func main() {
    sample := "A student studies programming."
    fmt.Printf("bytes=%d runes=%d words=%d\n", len(sample), len([]rune(sample)), len(strings
}
```

Expected Behavior or Output

Output observed when each example was run with empty standard input (Go 1.22; the text of the output does not depend on the operating system):

Example 1:

```
score=82; passing=true
```
```

Example 2:

```
82: B
91: A
76: C
88: B
59: F
```

Example 3:

```
first=82 last=88 count=4
```

Example 4:

```
total=337 average=84.2
```

Example 5:

```
bytes=30 runes=30 words=4
```

Common Mistakes

1. Assuming Go has while or do-while keywords instead of for forms.
2. Indexing a slice without checking its length.
3. Using len(string) when the intended count is Unicode code points.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical Avery's student profile, calculator bill/tip, and grade thresholds data and predict the output before running the program.
3. Add one boundary case and one malformed or missing-data case; state expected behavior before changing the code.
4. Refactor one part using the Go feature named in this lesson (for loops and range); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect punctuation in Go syntax (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain for loops and range using Go terminology.
- ☐ I can format, compile, and run the example.

- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 3.5: Strings and common operations

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain strings and common operations using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

A Go string is an immutable sequence of bytes that normally holds UTF-8 text. `len` counts bytes, not characters, and indexing gives a byte. The `strings` package supplies `ToUpper`, `ToLower`, `Contains`, `Index`, `Count`, `TrimSpace`, and `Fields`. Because strings cannot be changed, each function returns a new string.

Worked Example

Example 3.5.1

```
package main
import (
    "fmt"
)
func main() {
    score := 82
    fmt.Printf("score=%d; passing=%t\n", score, score >= 60)
}
```

Example 3.5.2

```
package main
import (
    "fmt"
)
```

```
func main() {
    scores := []int{82, 91, 76, 88, 59}
    for _, score := range scores {
     grade := "F"
     if score >= 90 { grade = "A" } else if score >= 80 { grade = "B" } else if score >= 70
     fmt.Printf("%d: %s\n", score, grade)
    }
}
```

Example 3.5.3

```
package main
import (
    "fmt"
)
func main() {
    scores := []int{82, 91, 76, 88}
    fmt.Printf("first=%d last=%d count=%d\n", scores[0], scores[len(scores)-1], len(scores))
}
```

Example 3.5.4

```
package main
import (
    "fmt"
)
func main() {
    scores := []int{82, 91, 76, 88}
    total := 0
    for _, score := range scores { total += score }
    fmt.Printf("total=%d average=%.1f\n", total, float64(total)/float64(len(scores)))
},package main
import (
    "fmt"
    "strings"
)
func main() {
    sample := "A student studies programming."
    fmt.Printf("bytes=%d runes=%d words=%d\n", len(sample), len([]rune(sample)), len(strings
} The lesson uses the canonical course data so learners focus on Go semantics rather than a

### Worked Example
```go
package main
import (
    "fmt"
    "strings"
)
```
```

```
func main() {
    sample := "A student studies programming."
    fmt.Printf("bytes=%d runes=%d words=%d\n", len(sample), len([]rune(sample)), len(strings
}
```

Expected Behavior or Output

Output observed when each example was run with empty standard input (Go 1.22; the text of the output does not depend on the operating system):

Example 1:

```
score=82; passing=true
```

Example 2:

```
82: B
91: A
76: C
88: B
59: F
```

Example 3:

```
first=82 last=88 count=4
```

Example 4:

```
total=337 average=84.2
```

Example 5:

```
bytes=30 runes=30 words=4
```

Common Mistakes

1. Assuming Go has while or do-while keywords instead of for forms.
2. Indexing a slice without checking its length.
3. Using len(string) when the intended count is Unicode code points.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical Avery's student profile, calculator bill/tip, and grade thresholds data and predict the output before running the program.
3. Add one boundary case and one malformed or missing-data case; state expected behavior before changing the code.
4. Refactor one part using the Go feature named in this lesson (strings and common operations); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect punctuation in Go syntax (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain strings and common operations using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

7.6 Unit 4: Data Structures

Overview

This unit develops data structures using Go's types, packages, and standard-library conventions. Prerequisites: the previous units and a working Go module. Use shared canonical course datasets and test boundary conditions, error paths, and accessible interaction.

Lesson 4.1: Slices and collection mutation

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain slices and collection mutation using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

package main import (”fmt”) func main() { scores := []int{82, 91, 76} scores = append(scores, 88) fmt.Println(scores) },package main import (”fmt” ”slices”) func main() { scores := []int{82, 91, 76, 88} slices.Sort(scores) fmt.Println(”ascending:”, scores) slices.Reverse(scores) fmt.Println(”descending:”, scores) },package main import (”fmt”) func main() { type Student struct { Name string; Age int; Score int } s := Student{Name: ”Avery”, Age: 16, Score: 82} fmt.Printf(”%s age %d score %d\n”, s.Name, s.Age, s.Score) },package main import (”fmt”) func main() { stock := map[string]int{”Notebook”: 3, ”Pencil”: 10} count, ok := stock[”Notebook”] if !ok { count = 0 } fmt.Println(”Notebook stock:”, count) },package main import (”fmt”) func main() { scores := []int{82, 91, 76, 88} passing := make([]int, 0) for _, score := range scores { if score >= 80 { passing = append(passing, score) } } fmt.Println(”Passing:”, passing) } The lesson uses the canonical course data so learners focus on Go semantics rather than a new domain problem. Format with gofmt and use go test ./... as routine development tools.

Worked Example

```
package main
import (
    "fmt"
)
func main() {
    scores := []int{82, 91, 76}
    scores = append(scores, 88)
    fmt.Println(scores)
}
```

Example 4.1.2

```
package main
import (
    "fmt"
    "slices"
)
func main() {
    scores := []int{82, 91, 76, 88}
    slices.Sort(scores)
    fmt.Println("ascending:", scores)
    slices.Reverse(scores)
    fmt.Println("descending:", scores)
}
```

Example 4.1.3

```
package main
```

```
import (
    "fmt"
)
func main() {
    type Student struct { Name string; Age int; Score int }
    s := Student{Name: "Avery", Age: 16, Score: 82}
    fmt.Printf("%s age %d score %d\n", s.Name, s.Age, s.Score)
}
```

Example 4.1.4

```
package main
import (
    "fmt"
)
func main() {
    stock := map[string]int{"Notebook": 3, "Pencil": 10}
    count, ok := stock["Notebook"]
    if !ok { count = 0 }
    fmt.Println("Notebook stock:", count)
}
```

Example 4.1.5

```
package main
import (
    "fmt"
)
func main() {
    scores := []int{82, 91, 76, 88}
    passing := make([]int, 0)
    for _, score := range scores { if score >= 80 { passing = append(passing, score) } }
    fmt.Println("Passing:", passing)
}
```

Expected Behavior or Output

Output observed when each example was run with empty standard input (Go 1.22; the text of the output does not depend on the operating system):

Example 1:

```
[82 91 76 88]
```

Example 2:

```
ascending: [76 82 88 91]
descending: [91 88 82 76]
```

Example 3:

```
Avery age 16 score 82
```

Example 4:

```
Notebook stock: 3
```

Example 5:

```
Passing: [82 91 88]
```

Common Mistakes

1. Assuming append always mutates the original backing array in place.
2. Relying on map iteration order.
3. Mutating a map or slice while overlooking aliasing through shared references.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical scores [82, 91, 76, 88] and the shared text sample data and predict the output before running the program.
3. Add one boundary case and one malformed or missing-data case; state expected behavior before changing the code.
4. Refactor one part using the Go feature named in this lesson (slices and collection mutation); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect punctuation in Go syntax (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain slices and collection mutation using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 4.2: Sorting and organizing data

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain sorting and organizing data using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

The `sort` package sorts slices in place: `sort.Ints`, `sort.Strings`, and `sort.Slice` with a comparison function. `sort.Sort(sort.Reverse(sort.IntSlice(s)))` sorts in descending order. To keep the original order, copy the slice first with `copy` or `slices.Clone`.

Worked Example

Example 4.2.1

```go
package main
import (
    "fmt"
)
func main() {
    scores := []int{82, 91, 76}
    scores = append(scores, 88)
    fmt.Println(scores)
},package main
import (
    "fmt"
    "slices"
)
func main() {
    scores := []int{82, 91, 76, 88}
    slices.Sort(scores)
    fmt.Println("ascending:", scores)
    slices.Reverse(scores)
    fmt.Println("descending:", scores)
```

```
},package main
import (
    "fmt"
)
func main() {
    type Student struct { Name string; Age int; Score int }
    s := Student{Name: "Avery", Age: 16, Score: 82}
    fmt.Printf("%s age %d score %d\n", s.Name, s.Age, s.Score)
},package main
import (
    "fmt"
)
func main() {
    stock := map[string]int{"Notebook": 3, "Pencil": 10}
    count, ok := stock["Notebook"]
    if !ok { count = 0 }
    fmt.Println("Notebook stock:", count)
},package main
import (
    "fmt"
)
func main() {
    scores := []int{82, 91, 76, 88}
    passing := make([]int, 0)
    for _, score := range scores { if score >= 80 { passing = append(passing, score) } }
    fmt.Println("Passing:", passing)
} The lesson uses the canonical course data so learners focus on Go semantics rather than a

### Worked Example
```go
package main
import (
    "fmt"
    "slices"
)
func main() {
    scores := []int{82, 91, 76, 88}
    slices.Sort(scores)
    fmt.Println("ascending:", scores)
    slices.Reverse(scores)
    fmt.Println("descending:", scores)
}
```

Example 4.2.3

```
package main
import (
```
```

```
    "fmt"
)
func main() {
    type Student struct { Name string; Age int; Score int }
    s := Student{Name: "Avery", Age: 16, Score: 82}
    fmt.Printf("%s age %d score %d\n", s.Name, s.Age, s.Score)
}
```

Example 4.2.4

```
package main
import (
    "fmt"
)
func main() {
    stock := map[string]int{"Notebook": 3, "Pencil": 10}
    count, ok := stock["Notebook"]
    if !ok { count = 0 }
    fmt.Println("Notebook stock:", count)
}
```

Example 4.2.5

```
package main
import (
    "fmt"
)
func main() {
    scores := []int{82, 91, 76, 88}
    passing := make([]int, 0)
    for _, score := range scores { if score >= 80 { passing = append(passing, score) } }
    fmt.Println("Passing:", passing)
}
```

Expected Behavior or Output

Output observed when each example was run with empty standard input (Go 1.22; the text of the output does not depend on the operating system):

Example 1:

```
[82 91 76 88]
```

Example 2:

```
ascending: [76 82 88 91]
descending: [91 88 82 76]
```

Example 3:

```
Avery age 16 score 82
```

Example 4:

```
Notebook stock: 3
```

Example 5:

```
Passing: [82 91 88]
```

Common Mistakes

1. Assuming append always mutates the original backing array in place.
2. Relying on map iteration order.
3. Mutating a map or slice while overlooking aliasing through shared references.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical scores [82, 91, 76, 88] and the shared text sample data and predict the output before running the program.
3. Add one boundary case and one malformed or missing-data case; state expected behavior before changing the code.
4. Refactor one part using the Go feature named in this lesson (sorting and organizing data); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect punctuation in Go syntax (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain sorting and organizing data using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 4.3: Structs and tuple-like multiple returns

Reading References

- The Go Programming Language Specification

- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain structs and tuple-like multiple returns using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

Go has no tuples. A struct groups named fields into one value, and a function can return several values. Returning `(x, y)` and receiving `x, y := point()` covers the same need. Struct values are copied on assignment, which makes a struct a good fixed record.

Worked Example

Example 4.3.1

```
package main
import (
    "fmt"
)
func main() {
    scores := []int{82, 91, 76}
    scores = append(scores, 88)
    fmt.Println(scores)
}
```

Example 4.3.2

```
package main
import (
    "fmt"
    "slices"
)
func main() {
    scores := []int{82, 91, 76, 88}
    slices.Sort(scores)
    fmt.Println("ascending:", scores)
    slices.Reverse(scores)
    fmt.Println("descending:", scores)
},package main
```

```
import (
    "fmt"
)
func main() {
    type Student struct { Name string; Age int; Score int }
    s := Student{Name: "Avery", Age: 16, Score: 82}
    fmt.Printf("%s age %d score %d\n", s.Name, s.Age, s.Score)
},package main
import (
    "fmt"
)
func main() {
    stock := map[string]int{"Notebook": 3, "Pencil": 10}
    count, ok := stock["Notebook"]
    if !ok { count = 0 }
    fmt.Println("Notebook stock:", count)
},package main
import (
    "fmt"
)
func main() {
    scores := []int{82, 91, 76, 88}
    passing := make([]int, 0)
    for _, score := range scores { if score >= 80 { passing = append(passing, score) } }
    fmt.Println("Passing:", passing)
} The lesson uses the canonical course data so learners focus on Go semantics rather than a

### Worked Example
```go
package main
import (
    "fmt"
)
func main() {
    type Student struct { Name string; Age int; Score int }
    s := Student{Name: "Avery", Age: 16, Score: 82}
    fmt.Printf("%s age %d score %d\n", s.Name, s.Age, s.Score)
}
```

Example 4.3.4

```
package main
import (
    "fmt"
)
func main() {
    stock := map[string]int{"Notebook": 3, "Pencil": 10}
```
```

```
    count, ok := stock["Notebook"]
    if !ok { count = 0 }
    fmt.Println("Notebook stock:", count)
}
```

Example 4.3.5

```
package main
import (
    "fmt"
)
func main() {
    scores := []int{82, 91, 76, 88}
    passing := make([]int, 0)
    for _, score := range scores { if score >= 80 { passing = append(passing, score) } }
    fmt.Println("Passing:", passing)
}
```

Expected Behavior or Output

Output observed when each example was run with empty standard input (Go 1.22; the text of the output does not depend on the operating system):

Example 1:

```
[82 91 76 88]
```

Example 2:

```
ascending: [76 82 88 91]
descending: [91 88 82 76]
```

Example 3:

```
Avery age 16 score 82
```

Example 4:

```
Notebook stock: 3
```

Example 5:

```
Passing: [82 91 88]
```

Common Mistakes

1. Assuming append always mutates the original backing array in place.
2. Relying on map iteration order.
3. Mutating a map or slice while overlooking aliasing through shared references.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical scores [82, 91, 76, 88] and the shared text sample data and predict the output before running the program.
3. Add one boundary case and one malformed or missing-data case; state expected behavior before changing the code.
4. Refactor one part using the Go feature named in this lesson (structs and tuple-like multiple returns); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect punctuation in Go syntax (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain structs and tuple-like multiple returns using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 4.4: Maps and key-value data

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain maps and key-value data using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

A map holds key-value pairs: `map[string]int`. Reading a missing key returns the zero value, so the two-value form `value, ok := m[key]` tells you whether the key exists. `delete(m, key)` removes a key, and `len(m)` counts the keys. A nil map can be read but writing to it panics, so create maps with `make` or a literal.

Worked Example

Example 4.4.1

```
package main
import (
    "fmt"
)
func main() {
    scores := []int{82, 91, 76}
    scores = append(scores, 88)
    fmt.Println(scores)
}
```

Example 4.4.2

```
package main
import (
    "fmt"
    "slices"
)
func main() {
    scores := []int{82, 91, 76, 88}
    slices.Sort(scores)
    fmt.Println("ascending:", scores)
    slices.Reverse(scores)
    fmt.Println("descending:", scores)
}
```

Example 4.4.3

```
package main
import (
    "fmt"
)
func main() {
    type Student struct { Name string; Age int; Score int }
    s := Student{Name: "Avery", Age: 16, Score: 82}
    fmt.Printf("%s age %d score %d\n", s.Name, s.Age, s.Score)
},package main
import (
```

```
    "fmt"
)
func main() {
    stock := map[string]int{"Notebook": 3, "Pencil": 10}
    count, ok := stock["Notebook"]
    if !ok { count = 0 }
    fmt.Println("Notebook stock:", count)
},package main
import (
    "fmt"
)
func main() {
    scores := []int{82, 91, 76, 88}
    passing := make([]int, 0)
    for _, score := range scores { if score >= 80 { passing = append(passing, score) } }
    fmt.Println("Passing:", passing)
} The lesson uses the canonical course data so learners focus on Go semantics rather than a

### Worked Example
```go
package main
import (
    "fmt"
)
func main() {
    stock := map[string]int{"Notebook": 3, "Pencil": 10}
    count, ok := stock["Notebook"]
    if !ok { count = 0 }
    fmt.Println("Notebook stock:", count)
}
```

Example 4.4.5

```
package main
import (
    "fmt"
)
func main() {
    scores := []int{82, 91, 76, 88}
    passing := make([]int, 0)
    for _, score := range scores { if score >= 80 { passing = append(passing, score) } }
    fmt.Println("Passing:", passing)
}
```
```

Expected Behavior or Output

Output observed when each example was run with empty standard input (Go 1.22; the text of the output does not depend on the operating system):

Example 1:

```
[82 91 76 88]
```

Example 2:

```
ascending: [76 82 88 91]
descending: [91 88 82 76]
```

Example 3:

```
Avery age 16 score 82
```

Example 4:

```
Notebook stock: 3
```

Example 5:

```
Passing: [82 91 88]
```

Common Mistakes

1. Assuming append always mutates the original backing array in place.
2. Relying on map iteration order.
3. Mutating a map or slice while overlooking aliasing through shared references.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical scores [82, 91, 76, 88] and the shared text sample data and predict the output before running the program.
3. Add one boundary case and one malformed or missing-data case; state expected behavior before changing the code.
4. Refactor one part using the Go feature named in this lesson (maps and key-value data); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect punctuation in Go syntax (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain maps and key-value data using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 4.5: Collection traversal

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain collection traversal using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

`for key, value := range m` visits a map, but **Go randomizes the iteration order on purpose**, so output printed straight from a map can differ between runs. For stable output, collect the keys into a slice, sort it, and then visit the keys in order. Printing a map with `fmt` is the exception: it prints keys in sorted order.

Worked Example

Example 4.5.1

```
package main
import (
    "fmt"
)
func main() {
    scores := []int{82, 91, 76}
    scores = append(scores, 88)
    fmt.Println(scores)
}
```

Example 4.5.2

```
package main
import (
    "fmt"
    "slices"
)
func main() {
    scores := []int{82, 91, 76, 88}
    slices.Sort(scores)
    fmt.Println("ascending:", scores)
    slices.Reverse(scores)
    fmt.Println("descending:", scores)
}
```

Example 4.5.3

```
package main
import (
    "fmt"
)
func main() {
    type Student struct { Name string; Age int; Score int }
    s := Student{Name: "Avery", Age: 16, Score: 82}
    fmt.Printf("%s age %d score %d\n", s.Name, s.Age, s.Score)
}
```

Example 4.5.4

```
package main
import (
    "fmt"
)
func main() {
    stock := map[string]int{"Notebook": 3, "Pencil": 10}
    count, ok := stock["Notebook"]
    if !ok { count = 0 }
    fmt.Println("Notebook stock:", count)
},package main
import (
    "fmt"
)
func main() {
    scores := []int{82, 91, 76, 88}
    passing := make([]int, 0)
    for _, score := range scores { if score >= 80 { passing = append(passing, score) } }
    fmt.Println("Passing:", passing)
} The lesson uses the canonical course data so learners focus on Go semantics rather than a

### Worked Example
```

```go
package main
import (
    "fmt"
)
func main() {
    scores := []int{82, 91, 76, 88}
    passing := make([]int, 0)
    for _, score := range scores { if score >= 80 { passing = append(passing, score) } }
    fmt.Println("Passing:", passing)
}
```

Expected Behavior or Output

Output observed when each example was run with empty standard input (Go 1.22; the text of the output does not depend on the operating system):

Example 1:

```
[82 91 76 88]
```

Example 2:

```
ascending: [76 82 88 91]
descending: [91 88 82 76]
```

Example 3:

```
Avery age 16 score 82
```

Example 4:

```
Notebook stock: 3
```

Example 5:

```
Passing: [82 91 88]
```

Common Mistakes

1. Assuming append always mutates the original backing array in place.
2. Relying on map iteration order.
3. Mutating a map or slice while overlooking aliasing through shared references.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical scores [82, 91, 76, 88] and the shared text sample data and predict the output before running the program.

3. Add one boundary case and one malformed or missing-data case; state expected behavior before changing the code.
4. Refactor one part using the Go feature named in this lesson (collection traversal); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect punctuation in Go syntax (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain collection traversal using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

7.7 Unit 5: Input and Iteration

Overview

This unit develops input and iteration using Go's types, packages, and standard-library conventions. Prerequisites: the previous units and a working Go module. Use shared canonical course datasets and test boundary conditions, error paths, and accessible interaction.

Lesson 5.1: Input and validation

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain input and validation using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.

- Test a relevant boundary condition and explain a common failure.

Concept Explanation

package main import (”bufio” ”fmt” ”os” ”strconv” ”strings”) func main() { reader := bufio.NewReader(os.Stdin) fmt.Print(”Enter age: ”) text, _ := reader.ReadString('\n') age, err := strconv.Atoi(strings.TrimSpace(text)) if err != nil || age < 0 || age > 120 { fmt.Println(”Enter an age from 0 to 120.”) } else { fmt.Println(”Accepted:”, age) } },package main import (”fmt”) func main() { values := []int{3, 4, 0, 5} count := 0 for _, value := range values { if value == 0 { break }; count++ } fmt.Println(”Read”, count, ”values before sentinel.”) },package main import (”fmt”) func main() { values := []int{4, 8, -1, 12} found := false for _, value := range values { if value < 0 { found = true; break } } fmt.Println(”negative value found:”, found) },package main import (”fmt”) func main() { values := []int{-2, 3, 0, 7, -1, 9} positives := make([]int, 0) for _, value := range values { if value <= 0 { continue }; positives = append(positives, value) } fmt.Println(positives) },package main import (”fmt”) func main() { attempts := 0 for attempts < 3 { attempts++; fmt.Println(”attempt”, attempts) } fmt.Println(”stopped after”, attempts, ”attempts”) } The lesson uses the canonical course data so learners focus on Go semantics rather than a new domain problem. Format with gofmt and use go test ./... as routine development tools.

Worked Example

```
package main
import (
    "bufio"
    "fmt"
    "os"
    "strconv"
    "strings"
)
func main() {
    reader := bufio.NewReader(os.Stdin)
    fmt.Print("Enter age: ")
    text, _ := reader.ReadString('\n')
    age, err := strconv.Atoi(strings.TrimSpace(text))
    if err != nil || age < 0 || age > 120 { fmt.Println("Enter an age from 0 to 120.") } els
}
```

Example 5.1.2

```
package main
import (
    "fmt"
)
func main() {
```

```
    values := []int{3, 4, 0, 5}
    count := 0
    for _, value := range values { if value == 0 { break }; count++ }
    fmt.Println("Read", count, "values before sentinel.")
}
```

Example 5.1.3

```
package main
import (
    "fmt"
)
func main() {
    values := []int{4, 8, -1, 12}
    found := false
    for _, value := range values { if value < 0 { found = true; break } }
    fmt.Println("negative value found:", found)
}
```

Example 5.1.4

```
package main
import (
    "fmt"
)
func main() {
    values := []int{-2, 3, 0, 7, -1, 9}
    positives := make([]int, 0)
    for _, value := range values { if value <= 0 { continue }; positives = append(positives,
    fmt.Println(positives)
}
```

Example 5.1.5

```
package main
import (
    "fmt"
)
func main() {
    attempts := 0
    for attempts < 3 { attempts++; fmt.Println("attempt", attempts) }
    fmt.Println("stopped after", attempts, "attempts")
}
```

Expected Behavior or Output

Output observed when each example was run with empty standard input (Go 1.22; the text of the output does not depend on the operating system):

Example 1:

```
Enter age: Enter an age from 0 to 120.
```

Example 2:

```
Read 2 values before sentinel.
```

Example 3:

```
negative value found: true
```

Example 4:

```
[3 7 9]
```

Example 5:

```
attempt 1
attempt 2
attempt 3
stopped after 3 attempts
```

Common Mistakes

1. Failing to trim newline and whitespace before parsing.
2. Choosing a sentinel that may also be valid data.
3. Writing a loop whose state never changes on one branch.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical scores [82, 91, 76, 88] and the shared text sample data and predict the output before running the program.
3. Add one boundary case and one malformed or missing-data case; state expected behavior before changing the code.
4. Refactor one part using the Go feature named in this lesson (input and validation); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect punctuation in Go syntax (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain input and validation using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 5.2: For loops as while-style repetition

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain for loops as while-style repetition using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

package main import (”bufio” ”fmt” ”os” ”strconv”) func main() { reader := bufio.NewReader(os.Stdin) fmt.Print(”Enter age: ”) text, _ := reader.ReadString('\n') age, err := strconv.Atoi(strings.TrimSpace(text)) if err != nil || age < 0 || age > 120 { fmt.Println(”Enter an age from 0 to 120.”) } else { fmt.Println(”Accepted:”, age) } },package main import (”fmt”) func main() { values := []int{3, 4, 0, 5} count := 0 for _, value := range values { if value == 0 { break }; count++ } fmt.Println(”Read”, count, ”values before sentinel.”) },package main import (”fmt”) func main() { values := []int{4, 8, -1, 12} found := false for _, value := range values { if value < 0 { found = true; break } } fmt.Println(”negative value found:”, found) },package main import (”fmt”) func main() { values := []int{-2, 3, 0, 7, -1, 9} positives := make([]int, 0) for _, value := range values { if value <= 0 { continue }; positives = append(positives, value) } fmt.Println(positives) },package main import (”fmt”) func main() { attempts := 0 for attempts < 3 { attempts++; fmt.Println(”attempt”, attempts) } fmt.Println(”stopped after”, attempts, ”attempts”) } The lesson uses the canonical course data so learners focus on Go semantics rather than a new domain problem. Format with gofmt and use go test ./... as routine development tools.

Worked Example

```
package main
import (
    "fmt"
)
func main() {
    values := []int{3, 4, 0, 5}
    count := 0
    for _, value := range values { if value == 0 { break }; count++ }
    fmt.Println("Read", count, "values before sentinel.")
}
```

Example 5.2.2

```
package main
import (
    "fmt"
)
func main() {
    values := []int{4, 8, -1, 12}
    found := false
    for _, value := range values { if value < 0 { found = true; break } }
    fmt.Println("negative value found:", found)
}
```

Example 5.2.3

```
package main
import (
    "fmt"
)
func main() {
    values := []int{-2, 3, 0, 7, -1, 9}
    positives := make([]int, 0)
    for _, value := range values { if value <= 0 { continue }; positives = append(positives,
    fmt.Println(positives)
}
```

Example 5.2.4

```
package main
import (
    "fmt"
)
func main() {
    attempts := 0
    for attempts < 3 { attempts++; fmt.Println("attempt", attempts) }
    fmt.Println("stopped after", attempts, "attempts")
```

```
}
```

Expected Behavior or Output

Output observed when each example was run with empty standard input (Go 1.22; the text of the output does not depend on the operating system):

Example 1:

```
Read 2 values before sentinel.
```

Example 2:

```
negative value found: true
```

Example 3:

```
[3 7 9]
```

Example 4:

```
attempt 1
attempt 2
attempt 3
stopped after 3 attempts
```

Common Mistakes

1. Failing to trim newline and whitespace before parsing.
2. Choosing a sentinel that may also be valid data.
3. Writing a loop whose state never changes on one branch.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical scores [82, 91, 76, 88] and the shared text sample data and predict the output before running the program.
3. Add one boundary case and one malformed or missing-data case; state expected behavior before changing the code.
4. Refactor one part using the Go feature named in this lesson (for loops as while-style repetition); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect punctuation in Go syntax (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain for loops as while-style repetition using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 5.3: Flags and early exits

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain flags and early exits using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

package main import (”bufio” ”fmt” ”os” ”strconv”) func main() { reader := bufio.NewReader(os.Stdin) fmt.Print(”Enter age: ”) text, _ := reader.ReadString('\n') age, err := strconv.Atoi(strings.TrimSpace(text)) if err != nil || age < 0 || age > 120 { fmt.Println(”Enter an age from 0 to 120.”) } else { fmt.Println(”Accepted:”, age) } },package main import (”fmt”) func main() { values := []int{3, 4, 0, 5} count := 0 for _, value := range values { if value == 0 { break }; count++ } fmt.Println(”Read”, count, ”values before sentinel.”) },package main import (”fmt”) func main() { values := []int{4, 8, -1, 12} found := false for _, value := range values { if value < 0 { found = true; break } } fmt.Println(”negative value found:”, found) },package main import (”fmt”) func main() { values := []int{-2, 3, 0, 7, -1, 9} positives := make([]int, 0) for _, value := range values { if value <= 0 { continue }; positives = append(positives, value) } fmt.Println(positives) },package main import (”fmt”) func main() { attempts := 0 for attempts < 3 { attempts++; fmt.Println(”attempt”, attempts) } fmt.Println(”stopped after”, attempts, ”attempts”) } The lesson uses the canonical course data so learners focus on Go semantics rather than a new domain problem. Format with gofmt and use go test ./... as routine development tools.

Worked Example

```
package main
import (
    "fmt"
)
func main() {
    values := []int{4, 8, -1, 12}
    found := false
    for _, value := range values { if value < 0 { found = true; break } }
    fmt.Println("negative value found:", found)
}
```

Example 5.3.3

```
package main
import (
    "fmt"
)
func main() {
    values := []int{-2, 3, 0, 7, -1, 9}
    positives := make([]int, 0)
    for _, value := range values { if value <= 0 { continue }; positives = append(positives,
    fmt.Println(positives)
}
```

Example 5.3.4

```
package main
import (
    "fmt"
)
func main() {
    attempts := 0
    for attempts < 3 { attempts++; fmt.Println("attempt", attempts) }
    fmt.Println("stopped after", attempts, "attempts")
}
```

Expected Behavior or Output

Output observed when each example was run with empty standard input (Go 1.22; the text of the output does not depend on the operating system):

Example 1:

```
Read 2 values before sentinel.
```

Example 2:

```
negative value found: true
```

Example 3:

```
[3 7 9]
```

Example 4:

```
attempt 1
attempt 2
attempt 3
stopped after 3 attempts
```

Common Mistakes

1. Failing to trim newline and whitespace before parsing.
2. Choosing a sentinel that may also be valid data.
3. Writing a loop whose state never changes on one branch.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical scores [82, 91, 76, 88] and the shared text sample data and predict the output before running the program.
3. Add one boundary case and one malformed or missing-data case; state expected behavior before changing the code.
4. Refactor one part using the Go feature named in this lesson (flags and early exits); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect punctuation in Go syntax (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain flags and early exits using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 5.4: Continue and filtering

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain continue and filtering using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

`continue` skips the rest of one pass of a `for` loop and goes to the next. Use it to skip invalid entries, such as scores below 0 or above 100, while still reporting that they were skipped. Be sure that the loop's counter is updated before `continue` in a condition-only `for` loop, or the loop will never end.

Worked Example

Example 5.4.1

```
package main
import (
    "fmt"
)
func main() {
    values := []int{3, 4, 0, 5}
    count := 0
    for _, value := range values { if value == 0 { break }; count++ }
    fmt.Println("Read", count, "values before sentinel.")
}
```

Example 5.4.2

```
package main
import (
    "fmt"
)
func main() {
    values := []int{4, 8, -1, 12}
    found := false
    for _, value := range values { if value < 0 { found = true; break } }
```

```
    fmt.Println("negative value found:", found)
},package main
import (
    "fmt"
)
func main() {
    values := []int{-2, 3, 0, 7, -1, 9}
    positives := make([]int, 0)
    for _, value := range values { if value <= 0 { continue }; positives = append(positives,
    fmt.Println(positives)
},package main
import (
    "fmt"
)
func main() {
    attempts := 0
    for attempts < 3 { attempts++; fmt.Println("attempt", attempts) }
    fmt.Println("stopped after", attempts, "attempts")
} The lesson uses the canonical course data so learners focus on Go semantics rather than a

### Worked Example
```go
package main
import (
    "fmt"
)
func main() {
    values := []int{-2, 3, 0, 7, -1, 9}
    positives := make([]int, 0)
    for _, value := range values { if value <= 0 { continue }; positives = append(positives,
    fmt.Println(positives)
}
```

Example 5.4.4

```
package main
import (
    "fmt"
)
func main() {
    attempts := 0
    for attempts < 3 { attempts++; fmt.Println("attempt", attempts) }
    fmt.Println("stopped after", attempts, "attempts")
}
```
```

Expected Behavior or Output

Output observed when each example was run with empty standard input (Go 1.22; the text of the output does not depend on the operating system):

Example 1:

```
Read 2 values before sentinel.
```

Example 2:

```
negative value found: true
```

Example 3:

```
[3 7 9]
```

Example 4:

```
attempt 1
attempt 2
attempt 3
stopped after 3 attempts
```

Common Mistakes

1. Failing to trim newline and whitespace before parsing.
2. Choosing a sentinel that may also be valid data.
3. Writing a loop whose state never changes on one branch.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical scores [82, 91, 76, 88] and the shared text sample data and predict the output before running the program.
3. Add one boundary case and one malformed or missing-data case; state expected behavior before changing the code.
4. Refactor one part using the Go feature named in this lesson (continue and filtering); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect punctuation in Go syntax (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain continue and filtering using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 5.5: Preventing infinite loops

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain preventing infinite loops using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

An endless loop never makes its condition false. Defend against it with a progress rule and a limit, for example a maximum number of attempts, and stop when input ends. A bare `for { ... }` is an intentional endless loop that must contain a `break` or `return`. Stop a running program from the terminal with Ctrl+C.

Worked Example

Example 5.5.1

```go
package main
import (
    "fmt"
)
func main() {
    values := []int{3, 4, 0, 5}
    count := 0
    for _, value := range values { if value == 0 { break }; count++ }
```

```
    fmt.Println("Read", count, "values before sentinel.")
}
```

Example 5.5.2

```
package main
import (
    "fmt"
)
func main() {
    values := []int{4, 8, -1, 12}
    found := false
    for _, value := range values { if value < 0 { found = true; break } }
    fmt.Println("negative value found:", found)
}
```

Example 5.5.3

```
package main
import (
    "fmt"
)
func main() {
    values := []int{-2, 3, 0, 7, -1, 9}
    positives := make([]int, 0)
    for _, value := range values { if value <= 0 { continue }; positives = append(positives,
    fmt.Println(positives)
},package main
import (
    "fmt"
)
func main() {
    attempts := 0
    for attempts < 3 { attempts++; fmt.Println("attempt", attempts) }
    fmt.Println("stopped after", attempts, "attempts")
} The lesson uses the canonical course data so learners focus on Go semantics rather than a

### Worked Example
```go
package main
import (
    "fmt"
)
func main() {
    attempts := 0
    for attempts < 3 { attempts++; fmt.Println("attempt", attempts) }
    fmt.Println("stopped after", attempts, "attempts")
}
```
```

Expected Behavior or Output

Output observed when each example was run with empty standard input (Go 1.22; the text of the output does not depend on the operating system):

Example 1:

```
Read 2 values before sentinel.
```

Example 2:

```
negative value found: true
```

Example 3:

```
[3 7 9]
```

Example 4:

```
attempt 1
attempt 2
attempt 3
stopped after 3 attempts
```

Common Mistakes

1. Failing to trim newline and whitespace before parsing.
2. Choosing a sentinel that may also be valid data.
3. Writing a loop whose state never changes on one branch.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical scores [82, 91, 76, 88] and the shared text sample data and predict the output before running the program.
3. Add one boundary case and one malformed or missing-data case; state expected behavior before changing the code.
4. Refactor one part using the Go feature named in this lesson (preventing infinite loops); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect punctuation in Go syntax (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain preventing infinite loops using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

7.8 Unit 6: Functions

Overview

This unit develops functions using Go's types, packages, and standard-library conventions. Prerequisites: the previous units and a working Go module. Use shared canonical course datasets and test boundary conditions, error paths, and accessible interaction.

Lesson 6.1: Defining and calling functions

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain defining and calling functions using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

package main import (”fmt”) func main() { greet := func(name string) { fmt.Println(”Hello,”, name) } greet(”Avery”) },package main import (”fmt”) func main() { describe := func(name string, score int) { fmt.Printf(”%s scored %d.\n”, name, score) } name := ”Avery” describe(name, 82) fmt.Println(”still available:”, name) },package main import (”fmt”) func main() { grade := func(score int) string { if score >= 60 { return ”pass” }; return ”retry” } fmt.Println(grade(82)) },package main import (”fmt”) func main() { average := func(scores []int) float64 { total := 0; for _, score := range scores { total += score }; return float64(total)/float64(len(scores)) } scores := []int{82, 91,

76, 88} fmt.Printf("average=%.1f\n", average(scores)) },package main import ("fmt") func main() { sum := func(values ...int) int { total := 0; for _, value := range values { total += value }; return total } fmt.Println("sum:", sum(82, 91, 76, 88)) } The lesson uses the canonical course data so learners focus on Go semantics rather than a new domain problem. Format with gofmt and use go test ./... as routine development tools.

Worked Example

```
package main
import (
    "fmt"
)
func main() {
    greet := func(name string) { fmt.Println("Hello,", name) }
    greet("Avery")
}
```

Example 6.1.2

```
package main
import (
    "fmt"
)
func main() {
    describe := func(name string, score int) { fmt.Printf("%s scored %d.\n", name, score) }
    name := "Avery"
    describe(name, 82)
    fmt.Println("still available:", name)
}
```

Example 6.1.3

```
package main
import (
    "fmt"
)
func main() {
    grade := func(score int) string { if score >= 60 { return "pass" }; return "retry" }
    fmt.Println(grade(82))
}
```

Example 6.1.4

```
package main
import (
    "fmt"
)
func main() {
```

```
    average := func(scores []int) float64 { total := 0; for _, score := range scores { total
    scores := []int{82, 91, 76, 88}
    fmt.Printf("average=%.1f\n", average(scores))
}
```

Example 6.1.5

```
package main
import (
    "fmt"
)
func main() {
    sum := func(values ...int) int { total := 0; for _, value := range values { total += val
    fmt.Println("sum:", sum(82, 91, 76, 88))
}
```

Expected Behavior or Output

Output observed when each example was run with empty standard input (Go 1.22; the text of the output does not depend on the operating system):

Example 1:

```
Hello, Avery
```

Example 2:

```
Avery scored 82.
still available: Avery
```

Example 3:

```
pass
```

Example 4:

```
average=84.2
```

Example 5:

```
sum: 337
```

Common Mistakes

1. Expecting a function to return a value when its signature does not.
2. Ignoring an error return value.
3. Confusing variadic arguments with a slice argument; use ... to expand a slice.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.

2. Change the canonical scores [82, 91, 76, 88] and the shared text sample data and predict the output before running the program.
3. Add one boundary case and one malformed or missing-data case; state expected behavior before changing the code.
4. Refactor one part using the Go feature named in this lesson (defining and calling functions); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect punctuation in Go syntax (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain defining and calling functions using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 6.2: Parameters and multiple return values

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain parameters and multiple return values using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

Go has no default arguments or keyword arguments. Pass every argument in order, or use a struct as an options value when a call needs many named settings.

A function can return several results, written `func f() (int, error)`, and the caller receives them with `a, b := f()`.

Worked Example

Example 6.2.1

```
package main
import (
    "fmt"
)
func main() {
    greet := func(name string) { fmt.Println("Hello,", name) }
    greet("Avery")
},package main
import (
    "fmt"
)
func main() {
    describe := func(name string, score int) { fmt.Printf("%s scored %d.\n", name, score) }
    name := "Avery"
    describe(name, 82)
    fmt.Println("still available:", name)
},package main
import (
    "fmt"
)
func main() {
    grade := func(score int) string { if score >= 60 { return "pass" }; return "retry" }
    fmt.Println(grade(82))
},package main
import (
    "fmt"
)
func main() {
    average := func(scores []int) float64 { total := 0; for _, score := range scores { total
    scores := []int{82, 91, 76, 88}
    fmt.Printf("average=%.1f\n", average(scores))
},package main
import (
    "fmt"
)
func main() {
    sum := func(values ...int) int { total := 0; for _, value := range values { total += val
    fmt.Println("sum:", sum(82, 91, 76, 88))
} The lesson uses the canonical course data so learners focus on Go semantics rather than a
```

```
### Worked Example
```go
package main
import (
    "fmt"
)
func main() {
    describe := func(name string, score int) { fmt.Printf("%s scored %d.\n", name, score) }
    name := "Avery"
    describe(name, 82)
    fmt.Println("still available:", name)
}
```

Example 6.2.3

```
package main
import (
    "fmt"
)
func main() {
    grade := func(score int) string { if score >= 60 { return "pass" }; return "retry" }
    fmt.Println(grade(82))
}
```

Example 6.2.4

```
package main
import (
    "fmt"
)
func main() {
    average := func(scores []int) float64 { total := 0; for _, score := range scores { total
    scores := []int{82, 91, 76, 88}
    fmt.Printf("average=%.1f\n", average(scores))
}
```

Example 6.2.5

```
package main
import (
    "fmt"
)
func main() {
    sum := func(values ...int) int { total := 0; for _, value := range values { total += val
    fmt.Println("sum:", sum(82, 91, 76, 88))
}
```
```

Expected Behavior or Output

Output observed when each example was run with empty standard input (Go 1.22; the text of the output does not depend on the operating system):

Example 1:

```
Hello, Avery
```

Example 2:

```
Avery scored 82.
still available: Avery
```

Example 3:

```
pass
```

Example 4:

```
average=84.2
```

Example 5:

```
sum: 337
```

Common Mistakes

1. Expecting a function to return a value when its signature does not.
2. Ignoring an error return value.
3. Confusing variadic arguments with a slice argument; use ... to expand a slice.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical scores [82, 91, 76, 88] and the shared text sample data and predict the output before running the program.
3. Add one boundary case and one malformed or missing-data case; state expected behavior before changing the code.
4. Refactor one part using the Go feature named in this lesson (parameters and multiple return values); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect punctuation in Go syntax (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages

in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain parameters and multiple return values using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 6.3: Return values and errors

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain return values and errors using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

A function sends a result back with `return`. By convention, a function that can fail returns its result and an `error` as the last value, and the caller checks `if err != nil`. Create errors with `errors.New` or `fmt.Errorf`, and never ignore a returned error silently.

Worked Example

Example 6.3.1

```
package main
import (
    "fmt"
)
func main() {
    greet := func(name string) { fmt.Println("Hello,", name) }
```

```
    greet("Avery")
}
```

Example 6.3.2

```
package main
import (
    "fmt"
)
func main() {
    describe := func(name string, score int) { fmt.Printf("%s scored %d.\n", name, score) }
    name := "Avery"
    describe(name, 82)
    fmt.Println("still available:", name)
},package main
import (
    "fmt"
)
func main() {
    grade := func(score int) string { if score >= 60 { return "pass" }; return "retry" }
    fmt.Println(grade(82))
},package main
import (
    "fmt"
)
func main() {
    average := func(scores []int) float64 { total := 0; for _, score := range scores { total
    scores := []int{82, 91, 76, 88}
    fmt.Printf("average=%.1f\n", average(scores))
},package main
import (
    "fmt"
)
func main() {
    sum := func(values ...int) int { total := 0; for _, value := range values { total += val
    fmt.Println("sum:", sum(82, 91, 76, 88))
} The lesson uses the canonical course data so learners focus on Go semantics rather than a

### Worked Example
```go
package main
import (
    "fmt"
)
func main() {
    grade := func(score int) string { if score >= 60 { return "pass" }; return "retry" }
    fmt.Println(grade(82))
```
```

```
}
```

Example 6.3.4

```
package main
import (
    "fmt"
)
func main() {
    average := func(scores []int) float64 { total := 0; for _, score := range scores { total
    scores := []int{82, 91, 76, 88}
    fmt.Printf("average=%.1f\n", average(scores))
}
```

Example 6.3.5

```
package main
import (
    "fmt"
)
func main() {
    sum := func(values ...int) int { total := 0; for _, value := range values { total += val
    fmt.Println("sum:", sum(82, 91, 76, 88))
}
```

Expected Behavior or Output

Output observed when each example was run with empty standard input (Go 1.22; the text of the output does not depend on the operating system):

Example 1:

```
Hello, Avery
```

Example 2:

```
Avery scored 82.
still available: Avery
```

Example 3:

```
pass
```

Example 4:

```
average=84.2
```

Example 5:

```
sum: 337
```

Common Mistakes

1. Expecting a function to return a value when its signature does not.
2. Ignoring an error return value.
3. Confusing variadic arguments with a slice argument; use ... to expand a slice.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical scores [82, 91, 76, 88] and the shared text sample data and predict the output before running the program.
3. Add one boundary case and one malformed or missing-data case; state expected behavior before changing the code.
4. Refactor one part using the Go feature named in this lesson (return values and errors); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect punctuation in Go syntax (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain return values and errors using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 6.4: Functions over slices and maps

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain functions over slices and maps using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

A slice passed to a function shares its elements with the caller, so changing an element inside the function changes the caller's slice; appending may not be visible to the caller unless the slice is returned. A map is passed by reference and can be changed in place. To protect the caller's data, copy a slice before changing it.

Worked Example

Example 6.4.1

```
package main
import (
    "fmt"
)
func main() {
    greet := func(name string) { fmt.Println("Hello,", name) }
    greet("Avery")
}
```

Example 6.4.2

```
package main
import (
    "fmt"
)
func main() {
    describe := func(name string, score int) { fmt.Printf("%s scored %d.\n", name, score) }
    name := "Avery"
    describe(name, 82)
    fmt.Println("still available:", name)
}
```

Example 6.4.3

```
package main
import (
    "fmt"
)
```

```
func main() {
    grade := func(score int) string { if score >= 60 { return "pass" }; return "retry" }
    fmt.Println(grade(82))
},package main
import (
    "fmt"
)
func main() {
    average := func(scores []int) float64 { total := 0; for _, score := range scores { total
    scores := []int{82, 91, 76, 88}
    fmt.Printf("average=%.1f\n", average(scores))
},package main
import (
    "fmt"
)
func main() {
    sum := func(values ...int) int { total := 0; for _, value := range values { total += val
    fmt.Println("sum:", sum(82, 91, 76, 88))
} The lesson uses the canonical course data so learners focus on Go semantics rather than a

### Worked Example
```go
package main
import (
    "fmt"
)
func main() {
    average := func(scores []int) float64 { total := 0; for _, score := range scores { total
    scores := []int{82, 91, 76, 88}
    fmt.Printf("average=%.1f\n", average(scores))
}
```

Example 6.4.5

```
package main
import (
    "fmt"
)
func main() {
    sum := func(values ...int) int { total := 0; for _, value := range values { total += val
    fmt.Println("sum:", sum(82, 91, 76, 88))
}
```

Expected Behavior or Output

Output observed when each example was run with empty standard input (Go 1.22; the text of the output does not depend on the operating system):
```

Example 1:

```
Hello, Avery
```

Example 2:

```
Avery scored 82.
still available: Avery
```

Example 3:

```
pass
```

Example 4:

```
average=84.2
```

Example 5:

```
sum: 337
```

Common Mistakes

1. Expecting a function to return a value when its signature does not.
2. Ignoring an error return value.
3. Confusing variadic arguments with a slice argument; use ... to expand a slice.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical scores [82, 91, 76, 88] and the shared text sample data and predict the output before running the program.
3. Add one boundary case and one malformed or missing-data case; state expected behavior before changing the code.
4. Refactor one part using the Go feature named in this lesson (functions over slices and maps); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect punctuation in Go syntax (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

☐ I can explain functions over slices and maps using Go terminology.

- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 6.5: Variadic parameters and scope

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain variadic parameters and scope using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

A final parameter written `...int` accepts any number of arguments and arrives in the function as a slice. Pass an existing slice with `values...`. Variables declared inside a function or block are local to it, and an inner variable with the same name shadows an outer one, which hides the outer value inside that block.

Worked Example

Example 6.5.1

```
package main
import (
    "fmt"
)
func main() {
    greet := func(name string) { fmt.Println("Hello,", name) }
    greet("Avery")
}
```

Example 6.5.2

```
package main
import (
```

```
    "fmt"
)
func main() {
    describe := func(name string, score int) { fmt.Printf("%s scored %d.\n", name, score) }
    name := "Avery"
    describe(name, 82)
    fmt.Println("still available:", name)
}
```

Example 6.5.3

```
package main
import (
    "fmt"
)
func main() {
    grade := func(score int) string { if score >= 60 { return "pass" }; return "retry" }
    fmt.Println(grade(82))
}
```

Example 6.5.4

```
package main
import (
    "fmt"
)
func main() {
    average := func(scores []int) float64 { total := 0; for _, score := range scores { total
    scores := []int{82, 91, 76, 88}
    fmt.Printf("average=%.1f\n", average(scores))
},package main
import (
    "fmt"
)
func main() {
    sum := func(values ...int) int { total := 0; for _, value := range values { total += val
    fmt.Println("sum:", sum(82, 91, 76, 88))
} The lesson uses the canonical course data so learners focus on Go semantics rather than a

### Worked Example
```go
package main
import (
    "fmt"
)
func main() {
    sum := func(values ...int) int { total := 0; for _, value := range values { total += val
    fmt.Println("sum:", sum(82, 91, 76, 88))
```
```

```
}
```

Expected Behavior or Output

Output observed when each example was run with empty standard input (Go 1.22; the text of the output does not depend on the operating system):

Example 1:

```
Hello, Avery
```

Example 2:

```
Avery scored 82.
still available: Avery
```

Example 3:

```
pass
```

Example 4:

```
average=84.2
```

Example 5:

```
sum: 337
```

Common Mistakes

1. Expecting a function to return a value when its signature does not.
2. Ignoring an error return value.
3. Confusing variadic arguments with a slice argument; use ... to expand a slice.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical scores [82, 91, 76, 88] and the shared text sample data and predict the output before running the program.
3. Add one boundary case and one malformed or missing-data case; state expected behavior before changing the code.
4. Refactor one part using the Go feature named in this lesson (variadic parameters and scope); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect punctuation in Go syntax (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages

in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain variadic parameters and scope using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

7.9 Unit 7: Object-Oriented Programming with Go Types

Overview

This unit develops object-oriented programming with go types using Go's types, packages, and standard-library conventions. Prerequisites: the previous units and a working Go module. Use shared canonical course datasets and test boundary conditions, error paths, and accessible interaction.

Lesson 7.1: Structs and data modeling

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain structs and data modeling using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

package main import (”fmt”) func main() { type Student struct { Name string; Score int } type Status int const (Active Status = iota; Inactive) s := Student{Name: ”Avery”, Score: 82} state := Active fmt.Printf(”%s: %d; active=%t; inactive=%t\n”, s.Name, s.Score, state == Active, Inactive

== Inactive) },package main import (”fmt”) type Student struct { Name string; Score int } func (s *Student) AddPoints(points int) { s.Score += points } func main() { s := Student{Name: ”Avery”, Score: 82} s.AddPoints(3) fmt.Println(s.Name, ”score:”, s.Score) },package main import (”fmt”) type Summarizer interface { Summary() string } type Task struct { Title string; Done bool } func (t Task) Summary() string { return fmt.Sprintf(”%s: %t”, t.Title, t.Done) } func main() { var item Summarizer = Task{Title: ”Study Go”, Done: false} fmt.Println(item.Summary()) },package main import (”fmt”) func main() { type Contact struct { Email string } type Contact struct { Email string } type Student struct { Name string; Contact } s := Student{Name: ”Avery”, Contact: Contact{Email: ”avery@example.test”}} fmt.Println(s.Name, s.Email) },package main import (”fmt”) type Reader interface { Read() string } type Note struct { Text string } func (n Note) Read() string { return n.Text } func display(r Reader) { fmt.Println(r.Read()) } func main() { display(Note{Text: ”Composition and interfaces”}) } The lesson uses the canonical course data so learners focus on Go semantics rather than a new domain problem. Format with gofmt and use go test ./... as routine development tools.

Worked Example

```
package main
import (
    "fmt"
)
func main() {
    type Student struct { Name string; Score int }
    type Status int
    const ( Active Status = iota; Inactive )
    s := Student{Name: "Avery", Score: 82}
    state := Active
    fmt.Printf("%s: %d; active=%t; inactive=%t\n", s.Name, s.Score, state == Active, Inactiv
}
```

Example 7.1.2

```
package main
import (
    "fmt"
)
type Student struct { Name string; Score int }
func (s *Student) AddPoints(points int) { s.Score += points }
func main() {
    s := Student{Name: "Avery", Score: 82}
    s.AddPoints(3)
    fmt.Println(s.Name, "score:", s.Score)
}
```

Example 7.1.3

```
package main
import (
    "fmt"
)
type Summarizer interface { Summary() string }
type Task struct { Title string; Done bool }
func (t Task) Summary() string { return fmt.Sprintf("%s: %t", t.Title, t.Done) }
func main() {
    var item Summarizer = Task{Title: "Study Go", Done: false}
    fmt.Println(item.Summary())
}
```

Example 7.1.4

```
package main
import (
    "fmt"
)
type Reader interface { Read() string }
type Note struct { Text string }
func (n Note) Read() string { return n.Text }
func display(r Reader) { fmt.Println(r.Read()) }
func main() {
    display(Note{Text: "Composition and interfaces"})
}
```

Expected Behavior or Output

Output observed when each example was run with empty standard input (Go 1.22; the text of the output does not depend on the operating system):

Example 1:

```
Avery: 82; active=true; inactive=true
```

Example 2:

```
Avery score: 85
```

Example 3:

```
Study Go: false
```

Example 4:

```
Composition and interfaces
```

Common Mistakes

1. Using a value receiver when mutation of the original struct is required.

2. Assuming Go interfaces must be declared as implemented explicitly.
3. Mistaking embedding for class inheritance or assuming promoted fields are always unambiguous.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical inventory records, file samples, and CSV fields data and predict the output before running the program.
3. Add one boundary case and one malformed or missing-data case; state expected behavior before changing the code.
4. Refactor one part using the Go feature named in this lesson (structs and data modeling); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect punctuation in Go syntax (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain structs and data modeling using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 7.2: Creating values and pointer receivers

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain creating values and pointer receivers using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.

- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

Go models data with structs and behavior with methods and interfaces. Pointer receivers can mutate a value; interfaces are satisfied implicitly. Embedding promotes fields and methods but is composition, not inheritance. Keep domain types small and make dependencies explicit. The lesson uses canonical course data so learners focus on Go semantics rather than a new domain problem. Format with gofmt and use go test ./... as routine development tools.

Worked Example

```
package main
import (
    "fmt"
)
    type Student struct { Name string; Score int }
    func (s *Student) AddPoints(points int) { s.Score += points }
    func main() {
     s := Student{Name: "Avery", Score: 82}
     s.AddPoints(3)
     fmt.Println(s.Name, "score:", s.Score)
    }
```

Expected behavior: The standalone program demonstrates creating values and pointer receivers with shared course example data. It compiles and runs using the standard library. Predict the output first, then compare it with terminal output.

Common Mistakes

1. Using a value receiver when mutation of the original struct is required.
2. Assuming interfaces must be declared as implemented explicitly.
3. Mistaking embedding for class inheritance or overlooking promoted-field ambiguity.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical inventory records, file samples, and CSV fields and predict the output before running it.
3. Add a boundary case and one malformed or missing-data case; define expected behavior before editing the program.

4. Refactor one part using the Go feature in this lesson (creating values and pointer receivers); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect Go punctuation (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain creating values and pointer receivers using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 7.3: Methods and interfaces

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain methods and interfaces using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

Go models data with structs and behavior with methods and interfaces. Pointer receivers can mutate a value; interfaces are satisfied implicitly. Embedding promotes fields and methods but is composition, not inheritance. Keep domain types small and make dependencies explicit. The lesson uses canonical course data so learners focus on Go semantics rather than a new domain problem. Format with gofmt and use go test ./... as routine development tools.

Worked Example

```
package main
import (
    "fmt"
)
    type Summarizer interface { Summary() string }
    type Task struct { Title string; Done bool }
    func (t Task) Summary() string { return fmt.Sprintf("%s: %t", t.Title, t.Done) }
    func main() {
     var item Summarizer = Task{Title: "Study Go", Done: false}
     fmt.Println(item.Summary())
    }
```

Expected behavior: The standalone program demonstrates methods and interfaces with shared course example data. It compiles and runs using the standard library. Predict the output first, then compare it with terminal output.

Common Mistakes

1. Using a value receiver when mutation of the original struct is required.
2. Assuming interfaces must be declared as implemented explicitly.
3. Mistaking embedding for class inheritance or overlooking promoted-field ambiguity.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical inventory records, file samples, and CSV fields and predict the output before running it.
3. Add a boundary case and one malformed or missing-data case; define expected behavior before editing the program.
4. Refactor one part using the Go feature in this lesson (methods and interfaces); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect Go punctuation (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain methods and interfaces using Go terminology.
- ☐ I can format, compile, and run the example.

- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 7.4: Embedding and polymorphism

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain embedding and polymorphism using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

Go models data with structs and behavior with methods and interfaces. Pointer receivers can mutate a value; interfaces are satisfied implicitly. Embedding promotes fields and methods but is composition, not inheritance. Keep domain types small and make dependencies explicit. The lesson uses canonical course data so learners focus on Go semantics rather than a new domain problem. Format with gofmt and use go test ./... as routine development tools.

Worked Example

```
package main
import (
    "fmt"
)
    func main() {
     type Contact struct { Email string }
     type Student struct { Name string; Contact }
     s := Student{Name: "Avery", Contact: Contact{Email: "avery@example.test"}}
     fmt.Println(s.Name, s.Email)
    }
```

Expected behavior: The standalone program demonstrates embedding and polymorphism with shared course example data. It compiles and runs using

the standard library. Predict the output first, then compare it with terminal output.

Common Mistakes

1. Using a value receiver when mutation of the original struct is required.
2. Assuming interfaces must be declared as implemented explicitly.
3. Mistaking embedding for class inheritance or overlooking promoted-field ambiguity.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical inventory records, file samples, and CSV fields and predict the output before running it.
3. Add a boundary case and one malformed or missing-data case; define expected behavior before editing the program.
4. Refactor one part using the Go feature in this lesson (embedding and polymorphism); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect Go punctuation (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain embedding and polymorphism using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 7.5: Composition and collaboration

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain composition and collaboration using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

Go models data with structs and behavior with methods and interfaces. Pointer receivers can mutate a value; interfaces are satisfied implicitly. Embedding promotes fields and methods but is composition, not inheritance. Keep domain types small and make dependencies explicit. The lesson uses canonical course data so learners focus on Go semantics rather than a new domain problem. Format with gofmt and use go test ./... as routine development tools.

Worked Example

```
package main
import (
    "fmt"
)
    type Reader interface { Read() string }
    type Note struct { Text string }
    func (n Note) Read() string { return n.Text }
    func display(r Reader) { fmt.Println(r.Read()) }
    func main() { display(Note{Text: "Composition and interfaces"}) }
```

Expected behavior: The standalone program demonstrates composition and collaboration with shared course example data. It compiles and runs using the standard library. Predict the output first, then compare it with terminal output.

Common Mistakes

1. Using a value receiver when mutation of the original struct is required.
2. Assuming interfaces must be declared as implemented explicitly.
3. Mistaking embedding for class inheritance or overlooking promoted-field ambiguity.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical inventory records, file samples, and CSV fields and predict the output before running it.

3. Add a boundary case and one malformed or missing-data case; define expected behavior before editing the program.
4. Refactor one part using the Go feature in this lesson (composition and collaboration); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect Go punctuation (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain composition and collaboration using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

7.10 Unit 8: Files and Exceptions

Overview

Read and write files, handle errors, release resources reliably, and store records as JSON.

Lesson 8.1: Reading text files

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain reading text files using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

The os and io packages provide file operations. Functions return errors that callers must check. defer schedules cleanup at function return; check Close errors when writes matter. encoding/json handles JSON serialization with struct tags. Production code should avoid panic for expected failures. The lesson uses canonical course data so learners focus on Go semantics rather than a new domain problem. Format with gofmt and use go test ./... as routine development tools.

Worked Example

```
package main
import (
    "fmt"
    "os"
)
func main() {
 path := "go_read_demo.txt"
 if err := os.WriteFile(path, []byte("Avery,82\n"), 0600); err != nil { panic(err) }
 data, err := os.ReadFile(path)
 if err != nil { panic(err) }
 fmt.Print(string(data))
 if err := os.Remove(path); err != nil { panic(err) }
}
```

Expected behavior: The standalone program demonstrates reading text files with shared course example data. It compiles and runs using the standard library. Predict the output first, then compare it with terminal output.

Common Mistakes

1. Ignoring errors from file operations and Close.
2. Calling defer inside a long-running loop and retaining resources too long.
3. Using hand-built JSON instead of encoding/json.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical inventory records, file samples, and CSV fields and predict the output before running it.
3. Add a boundary case and one malformed or missing-data case; define expected behavior before editing the program.
4. Refactor one part using the Go feature in this lesson (reading text files); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect Go punctuation (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain reading text files using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 8.2: Writing and appending files

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain writing and appending files using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

The os and io packages provide file operations. Functions return errors that callers must check. defer schedules cleanup at function return; check Close errors when writes matter. encoding/json handles JSON serialization with struct tags. Production code should avoid panic for expected failures. The lesson uses canonical course data so learners focus on Go semantics rather than a new domain problem. Format with gofmt and use go test ./... as routine development tools.

Worked Example

```
package main
import (
    "fmt"
    "os"
)
func main() {
 path := "go_append_demo.txt"
 file, err := os.OpenFile(path, os.O_CREATE|os.O_APPEND|os.O_WRONLY, 0600)
 if err != nil { panic(err) }
 if _, err := fmt.Fprintln(file, "Notebook,3"); err != nil { panic(err) }
 if err := file.Close(); err != nil { panic(err) }
 fmt.Println("appended inventory record")
 if err := os.Remove(path); err != nil { panic(err) }
}
```

Expected behavior: The standalone program demonstrates writing and appending files with shared course example data. It compiles and runs using the standard library. Predict the output first, then compare it with terminal output.

Common Mistakes

1. Ignoring errors from file operations and Close.
2. Calling defer inside a long-running loop and retaining resources too long.
3. Using hand-built JSON instead of encoding/json.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical inventory records, file samples, and CSV fields and predict the output before running it.
3. Add a boundary case and one malformed or missing-data case; define expected behavior before editing the program.
4. Refactor one part using the Go feature in this lesson (writing and appending files); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect Go punctuation (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain writing and appending files using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 8.3: Explicit error handling

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain explicit error handling using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

The os and io packages provide file operations. Functions return errors that callers must check. defer schedules cleanup at function return; check Close errors when writes matter. encoding/json handles JSON serialization with struct tags. Production code should avoid panic for expected failures. The lesson uses canonical course data so learners focus on Go semantics rather than a new domain problem. Format with gofmt and use go test ./... as routine development tools.

Worked Example

```
package main
import (
    "errors"
    "fmt"
)
func main() {
 load := func(path string) (string, error) { return "", errors.New("file not found: " + path
 text, err := load("missing_course_file.txt")
```

```
 if err != nil { fmt.Println("Could not read file:", err) } else { fmt.Println(text) }
}
```

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (Go 1.22; the text of the output does not depend on the operating system):

```
Could not read file: file not found: missing_course_file.txt
```

Common Mistakes

1. Ignoring errors from file operations and Close.
2. Calling defer inside a long-running loop and retaining resources too long.
3. Using hand-built JSON instead of encoding/json.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical inventory records, file samples, and CSV fields and predict the output before running it.
3. Add a boundary case and one malformed or missing-data case; define expected behavior before editing the program.
4. Refactor one part using the Go feature in this lesson (explicit error handling); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect Go punctuation (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain explicit error handling using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 8.4: Deferred cleanup and resource lifetime

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain deferred cleanup and resource lifetime using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

The os and io packages provide file operations. Functions return errors that callers must check. defer schedules cleanup at function return; check Close errors when writes matter. encoding/json handles JSON serialization with struct tags. Production code should avoid panic for expected failures. The lesson uses canonical course data so learners focus on Go semantics rather than a new domain problem. Format with gofmt and use go test ./... as routine development tools.

Worked Example

```go
package main
import (
    "fmt"
    "os"
)
func main() {
 path := "go_cleanup_demo.txt"
 file, err := os.Create(path)
 if err != nil { panic(err) }
 defer func() { if err := file.Close(); err != nil { fmt.Println("close error:", err) }; _ =
 if _, err := fmt.Fprintln(file, "closed when function returns"); err != nil { panic(err) }
 fmt.Println("deferred cleanup is scheduled")
}
```

Expected behavior: The standalone program demonstrates deferred cleanup and resource lifetime with shared course example data. It compiles and runs using the standard library. Predict the output first, then compare it with terminal output.

Common Mistakes

1. Ignoring errors from file operations and Close.
2. Calling defer inside a long-running loop and retaining resources too long.
3. Using hand-built JSON instead of encoding/json.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical inventory records, file samples, and CSV fields and predict the output before running it.
3. Add a boundary case and one malformed or missing-data case; define expected behavior before editing the program.
4. Refactor one part using the Go feature in this lesson (deferred cleanup and resource lifetime); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect Go punctuation (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain deferred cleanup and resource lifetime using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 8.5: JSON encoding and persistence

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain json encoding and persistence using Go terminology and type conventions.

- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

The os and io packages provide file operations. Functions return errors that callers must check. defer schedules cleanup at function return; check Close errors when writes matter. encoding/json handles JSON serialization with struct tags. Production code should avoid panic for expected failures. The lesson uses canonical course data so learners focus on Go semantics rather than a new domain problem. Format with gofmt and use go test ./... as routine development tools.

Worked Example

```
package main
import (
    "encoding/json"
    "fmt"
)
func main() {
 record := struct { Name string `json:"name"`; Score int `json:"score"` }{Name: "Avery", Sc
 data, err := json.Marshal(record)
 if err != nil { panic(err) }
 fmt.Println(string(data))
}
```

Expected behavior: The standalone program demonstrates json encoding and persistence with shared course example data. It compiles and runs using the standard library. Predict the output first, then compare it with terminal output.

Common Mistakes

1. Ignoring errors from file operations and Close.
2. Calling defer inside a long-running loop and retaining resources too long.
3. Using hand-built JSON instead of encoding/json.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical inventory records, file samples, and CSV fields and predict the output before running it.

3. Add a boundary case and one malformed or missing-data case; define expected behavior before editing the program.
4. Refactor one part using the Go feature in this lesson (json encoding and persistence); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect Go punctuation (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain json encoding and persistence using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

7.11 Unit 9: Testing and Data Analysis

Overview

Write tests with expected results, test random code by its properties, analyze the shared text sample, and read the shared CSV file.

Lesson 9.1: Testing functions with testing

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain testing functions with testing using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.

- Test a relevant boundary condition and explain a common failure.

Concept Explanation

Go's testing package discovers TestXxx functions in _test.go files; run go test ./.... Table-driven tests make cases explicit. Keep logic independently callable. Use deterministic values or inject a random source to make tests repeatable. encoding/csv handles quoted fields and embedded delimiters. The lesson uses canonical course data so learners focus on Go semantics rather than a new domain problem. Format with gofmt and use go test ./... as routine development tools.

Worked Example

```
package main
import (
    "fmt"
)
func main() {
 isPassing := func(score int) bool { return score >= 60 }
 checks := []struct { score int; want bool }{{82, true}, {59, false}, {60, true}}
 for _, check := range checks { got := isPassing(check.score); if got != check.want { panic(
 fmt.Println("grade boundary checks passed")
}
```

Expected behavior: The standalone program demonstrates testing functions with testing with shared course example data. It compiles and runs using the standard library. Predict the output first, then compare it with terminal output.

Common Mistakes

1. Testing only typical values and missing boundaries.
2. Using package-level mutable state that makes tests order-dependent.
3. Using nondeterministic random data in tests without a seeded or injected source.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical text-adventure state and to-do fields and predict the output before running it.
3. Add a boundary case and one malformed or missing-data case; define expected behavior before editing the program.
4. Refactor one part using the Go feature in this lesson (testing functions with testing); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect Go punctuation (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain testing functions with testing using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 9.2: Testing stateful types

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain testing stateful types using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

Go's testing package discovers TestXxx functions in _test.go files; run go test ./.... Table-driven tests make cases explicit. Keep logic independently callable. Use deterministic values or inject a random source to make tests repeatable. encoding/csv handles quoted fields and embedded delimiters. The lesson uses canonical course data so learners focus on Go semantics rather than a new domain problem. Format with gofmt and use go test ./... as routine development tools.

Worked Example

```
package main
import "fmt"
```

```go
type Counter struct { Value int }
func (c *Counter) Increment() { c.Value++ }

func main() {
 c := Counter{}
 c.Increment()
 c.Increment()
 if c.Value != 2 { panic("counter test failed") }
 fmt.Println("counter state verified:", c.Value)
}
```

Expected behavior: The standalone program demonstrates testing stateful types with shared course example data. It compiles and runs using the standard library. Predict the output first, then compare it with terminal output.

Common Mistakes

1. Testing only typical values and missing boundaries.
2. Using package-level mutable state that makes tests order-dependent.
3. Using nondeterministic random data in tests without a seeded or injected source.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical text-adventure state and to-do fields and predict the output before running it.
3. Add a boundary case and one malformed or missing-data case; define expected behavior before editing the program.
4. Refactor one part using the Go feature in this lesson (testing stateful types); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect Go punctuation (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain testing stateful types using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.

- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 9.3: Random values

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain random values using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

Go's testing package discovers TestXxx functions in _test.go files; run go test ./.... Table-driven tests make cases explicit. Keep logic independently callable. Use deterministic values or inject a random source to make tests repeatable. encoding/csv handles quoted fields and embedded delimiters. The lesson uses canonical course data so learners focus on Go semantics rather than a new domain problem. Format with gofmt and use go test ./... as routine development tools.

Worked Example

```
package main
import (
    "fmt"
)
func main() {
 next := func(seed uint64) uint64 { return seed*6364136223846793005 + 1 }
 seed := uint64(7)
 for i := 0; i < 5; i++ { seed = next(seed); fmt.Println(seed) }
}
```

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (Go 1.22; the text of the output does not depend on the operating system):

```
7655465419508447804
3670188072890482829
995417631842978762
12309061806813186179
12177546622254251016
```

Common Mistakes

1. Testing only typical values and missing boundaries.
2. Using package-level mutable state that makes tests order-dependent.
3. Using nondeterministic random data in tests without a seeded or injected source.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical text-adventure state and to-do fields and predict the output before running it.
3. Add a boundary case and one malformed or missing-data case; define expected behavior before editing the program.
4. Refactor one part using the Go feature in this lesson (random values); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect Go punctuation (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain random values using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 9.4: Text analysis

Reading References

- The Go Programming Language Specification
- A Tour of Go

- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain text analysis using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

Go's testing package discovers TestXxx functions in _test.go files; run go test ./.... Table-driven tests make cases explicit. Keep logic independently callable. Use deterministic values or inject a random source to make tests repeatable. encoding/csv handles quoted fields and embedded delimiters. The lesson uses canonical course data so learners focus on Go semantics rather than a new domain problem. Format with gofmt and use go test ./... as routine development tools.

Worked Example

```
package main
import (
    "fmt"
    "strings"
)
func main() {
 sample := "A student studies programming. Programming builds skill."
 fmt.Printf("words=%d characters=%d\n", len(strings.Fields(sample)), len([]rune(sample)))
 fmt.Println("contains programming:", strings.Contains(strings.ToLower(sample), "programming
}
```

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (Go 1.22; the text of the output does not depend on the operating system):

```
words=7 characters=56
contains programming: true
```

Common Mistakes

1. Testing only typical values and missing boundaries.
2. Using package-level mutable state that makes tests order-dependent.
3. Using nondeterministic random data in tests without a seeded or injected source.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical text-adventure state and to-do fields and predict the output before running it.
3. Add a boundary case and one malformed or missing-data case; define expected behavior before editing the program.
4. Refactor one part using the Go feature in this lesson (text analysis); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect Go punctuation (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain text analysis using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 9.5: CSV records

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain csv records using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

Go's testing package discovers TestXxx functions in _test.go files; run go test ./.... Table-driven tests make cases explicit. Keep logic independently callable. Use deterministic values or inject a random source to make tests repeatable. encoding/csv handles quoted fields and embedded delimiters. The lesson uses canonical course data so learners focus on Go semantics rather than a new domain problem. Format with gofmt and use go test ./... as routine development tools.

Worked Example

```
package main
import (
    "encoding/csv"
    "fmt"
    "strings"
)
func main() {
 input := "name,score\nAvery,82\nJordan,91\n"
 rows, err := csv.NewReader(strings.NewReader(input)).ReadAll()
 if err != nil { panic(err) }
 for _, row := range rows { fmt.Println(strings.Join(row, " | ")) }
}
```

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (Go 1.22; the text of the output does not depend on the operating system):

```
name | score
Avery | 82
Jordan | 91
```

Common Mistakes

1. Testing only typical values and missing boundaries.
2. Using package-level mutable state that makes tests order-dependent.
3. Using nondeterministic random data in tests without a seeded or injected source.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical text-adventure state and to-do fields and predict the output before running it.

3. Add a boundary case and one malformed or missing-data case; define expected behavior before editing the program.
4. Refactor one part using the Go feature in this lesson (csv records); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect Go punctuation (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain csv records using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

7.12 Unit 10: Project-Based Learning — Text-Based Applications

Overview

Plan a text project, build the shared text adventure in two parts, build the command-line to-do manager, and refine input validation.

Lesson 10.1: Requirements and top-down design

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain requirements and top-down design using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.

- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

Separate requirements, application state, domain operations, validation, and terminal interaction. Small functions and explicit types make behavior testable. Terminal prompts should be linear, textual, keyboard-operable, and independent of color or cursor positioning. The lesson uses canonical course data so learners focus on Go semantics rather than a new domain problem. Format with gofmt and use go test ./... as routine development tools.

Worked Example

```
package main
import (
    "fmt"
)
func main() {
 requirements := []string{"list tasks", "add task", "mark task complete", "save tasks"}
 for i, requirement := range requirements { fmt.Printf("%d. %s\n", i+1, requirement) }
}
```

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (Go 1.22; the text of the output does not depend on the operating system):

```
1. list tasks
2. add task
3. mark task complete
4. save tasks
```

Common Mistakes

1. Mixing domain logic with terminal input/output in one function.
2. Assuming map iteration order is stable when displaying choices.
3. Making interaction depend on color, animation, or cursor positioning.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical text-adventure state and to-do fields and predict the output before running it.
3. Add a boundary case and one malformed or missing-data case; define expected behavior before editing the program.

4. Refactor one part using the Go feature in this lesson (requirements and top-down design); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect Go punctuation (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain requirements and top-down design using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 10.2: Text adventure: five rooms and movement

Reading References

- Go maps
- Go standard library: bufio
- Canonical shared project specification: Python Unit 10, Lesson 10.3.

Learning Objectives

- Represent all five canonical rooms with Go structs and maps.
- Validate movement before changing the current room.
- Explain why treasure_room is not reachable until the key puzzle is implemented.

Concept Explanation

The shared graph has five rooms: start_room, tunnel, clearing, locked_door_room, and treasure_room. A Room stores its description and valid exits. The player starts in start_room. Part 1 does not provide the north exit from locked_door_room. Map lookup uses the two-result form so a missing direction cannot change state.

Worked Example

```
package main

import (
```

```
    "bufio"
    "fmt"
    "os"
    "strings"
)

type Room struct {
    description string
    exits       map[string]string
}

func main() {
    rooms := map[string]Room{
        "start_room":       {"A dimly lit cave. A narrow passage leads north.", map[string]s
        "tunnel":           {"A damp tunnel. Passages lead south and east.", map[string]stri
        "clearing":         {"A quiet clearing. An antechamber lies to the north.", map[stri
        "locked_door_room": {"A chamber with a heavy door to the north.", map[string]string
        "treasure_room":    {"You found the treasure. You win!", map[string]string{}},
    }
    scanner := bufio.NewScanner(os.Stdin)
    current := "start_room"

    for {
        fmt.Println("---")
        fmt.Println(rooms[current].description)
        fmt.Println("What do you want to do?")
        command := "quit"
        if scanner.Scan() {
            command = strings.ToLower(strings.TrimSpace(scanner.Text()))
        }
        if command == "quit" {
            fmt.Println("Exiting game. Goodbye!")
            break
        }
        if destination, ok := rooms[current].exits[command]; ok {
            current = destination
        } else {
            fmt.Println("You cannot go that way. Enter a listed direction or quit.")
        }
    }
}
```

Expected Behavior or Output

Enter north, east, north, north, quit. The player reaches tunnel, clearing, and locked_door_room. The second north is rejected and the current room does

not change. The treasure room exists but has no incoming exit in Part 1.

Common Mistakes

- Using a map result without checking the boolean.
- Updating current before validating the destination.
- Defining a destination that is not one of the five room identifiers.
- Adding the treasure exit before implementing the key puzzle.

Activities

1. Trace north, east, north, north and record the current room.
2. Try west from start_room and verify that the room does not change.
3. Check every exit destination against the room map.
4. Explain why treasure_room cannot yet be reached.

Accessibility Practice

Announce the room description and prompt in a consistent order. The invalid-movement response explains what happened and how to continue. Run from the terminal and inspect output sequentially with the learner's access technology.

Skills Checklist

☐ I can model all five canonical rooms.
☐ I can represent exits with maps.
☐ I can reject invalid movement without changing state.
☐ I can explain why the treasure room is not yet reachable.

Lesson 10.3: Text adventure: shiny key and locked door

Reading References

- Go slices
- Go standard library: strings
- Canonical shared project specification: Python Unit 10, Lesson 10.3.

Learning Objectives

- Place the shiny key in start_room and transfer it to inventory exactly once.
- Require key use in locked_door_room.
- Unlock the north exit only after the puzzle succeeds.
- End the game when the player reaches treasure_room.

Concept Explanation

Keep the five-room graph from Lesson 10.2. Room values contain slices of items. Picking up the shiny key removes it from start_room and appends it to inventory. Using the key succeeds only in locked_door_room when inventory contains it. The north exit remains blocked until the key is used.

Worked Example

```
package main

import (
    "bufio"
    "fmt"
    "os"
    "strings"
)

type Room struct {
    description string
    exits       map[string]string
    items       []string
}

func contains(items []string, target string) bool {
    for _, item := range items {
        if item == target {
            return true
        }
    }
    return false
}

func main() {
    rooms := map[string]Room{
        "start_room":       {"A dimly lit cave.", map[string]string{"north": "tunnel"}, []st
        "tunnel":           {"A damp tunnel. Passages lead south and east.", map[string]stri
        "clearing":         {"A quiet clearing. An antechamber lies to the north.", map[stri
        "locked_door_room": {"A chamber with a heavy door to the north.", map[string]string
        "treasure_room":    {"You found the treasure. You win!", map[string]string{}, nil},
    }
    current := "start_room"
    inventory := []string{}
    doorLocked := true
    scanner := bufio.NewScanner(os.Stdin)
```

```
for {
    room := rooms[current]
    fmt.Println()
    fmt.Println(room.description)
    if current == "treasure_room" {
        break
    }
    if len(room.items) > 0 {
        fmt.Println("You see:", strings.Join(room.items, ", "))
    }
    inventoryText := "empty"
    if len(inventory) > 0 {
        inventoryText = strings.Join(inventory, ", ")
    }
    fmt.Println("Your inventory:", inventoryText)
    fmt.Println("What do you want to do?")
    command := "quit"
    if scanner.Scan() {
        command = strings.ToLower(strings.TrimSpace(scanner.Text()))
    }

    switch command {
    case "quit":
        fmt.Println("You quit the adventure.")
        return
    case "take shiny key":
        room = rooms[current]
        if contains(room.items, "shiny key") {
            for i, item := range room.items {
                if item == "shiny key" {
                    room.items = append(room.items[:i], room.items[i+1:]...)
                    break
                }
            }
            rooms[current] = room
            inventory = append(inventory, "shiny key")
            fmt.Println("You picked up the shiny key.")
        } else {
            fmt.Println("There is no shiny key here to take.")
        }
    case "use shiny key":
        if current != "locked_door_room" {
            fmt.Println("You must be at the locked door to use the key.")
        } else if !contains(inventory, "shiny key") {
            fmt.Println("You do not have the shiny key.")
        } else {
```

```
                doorLocked = false
                fmt.Println("You unlocked the door!")
            }
        case "north":
            if current == "locked_door_room" && doorLocked {
                fmt.Println("The door is locked! You need to use the shiny key here.")
            } else if destination, ok := rooms[current].exits[command]; ok {
                current = destination
            } else {
                fmt.Println("You cannot go that way.")
            }
        case "south", "east", "west":
            if destination, ok := rooms[current].exits[command]; ok {
                current = destination
            } else {
                fmt.Println("You cannot go that way.")
            }
        default:
            fmt.Println("Invalid command. Enter north, south, east, west, take shiny key, us
        }
    }
}
```

Expected Behavior or Output

Enter north, east, north, north, south, west, south, take shiny key, north, east, north, use shiny key, north. The first north from locked_door_room is rejected. Taking the key removes it from the room and adds it to inventory. Using it outside locked_door_room is rejected. Using it there unlocks the door; the final north reaches treasure_room and prints the win description.

Common Mistakes

- Updating a local copy of a Room without writing it back into the map.
- Removing the key without adding it to inventory, or adding it without removing it.
- Unlocking the door outside locked_door_room.
- Allowing north movement before doorLocked becomes false.
- Treating a missing map key as a valid destination.

Activities

1. Run the complete puzzle sequence and record room, items, inventory, and door state.
2. Attempt to take the key twice; the second attempt must be rejected.
3. Try to use the key outside locked_door_room.
4. Try north before and after unlocking.

5. Add a help command listing every supported command.

Accessibility Practice

Announce the room, visible items, inventory, and prompt as separate text lines. Error messages explain the failed action. Test the complete interaction with keyboard-only input and sequential screen-reader output.

Skills Checklist

- ☐ I can model the five canonical rooms.
- ☐ I can transfer the key from a room to inventory.
- ☐ I can prevent duplicate pickup.
- ☐ I can require key use in locked_door_room.
- ☐ I can block and unlock the north exit.
- ☐ I can end the game in treasure_room.

Lesson 10.4: Command-line to-do manager

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain command-line to-do manager using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

Separate requirements, application state, domain operations, validation, and terminal interaction. Small functions and explicit types make behavior testable. Terminal prompts should be linear, textual, keyboard-operable, and independent of color or cursor positioning. The lesson uses canonical course data so learners focus on Go semantics rather than a new domain problem. Format with gofmt and use go test ./... as routine development tools.

Worked Example

```
package main
import (
    "fmt"
)
func main() {
 type Task struct { ID int; Title string; Done bool }
 tasks := []Task{{ID: 1, Title: "Read lesson"}, {ID: 2, Title: "Run example"}}
 tasks[0].Done = true
 for _, task := range tasks { fmt.Printf("%d: %s [%t]\n", task.ID, task.Title, task.Done) }
}
```

Expected behavior: The standalone program demonstrates command-line to-do manager with shared course example data. It compiles and runs using the standard library. Predict the output first, then compare it with terminal output.

Common Mistakes

1. Mixing domain logic with terminal input/output in one function.
2. Assuming map iteration order is stable when displaying choices.
3. Making interaction depend on color, animation, or cursor positioning.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical text-adventure state and to-do fields and predict the output before running it.
3. Add a boundary case and one malformed or missing-data case; define expected behavior before editing the program.
4. Refactor one part using the Go feature in this lesson (command-line to-do manager); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect Go punctuation (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain command-line to-do manager using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.

- [ ] I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 10.5: Accessible interaction and validation

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain accessible interaction and validation using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

Separate requirements, application state, domain operations, validation, and terminal interaction. Small functions and explicit types make behavior testable. Terminal prompts should be linear, textual, keyboard-operable, and independent of color or cursor positioning. The lesson uses canonical course data so learners focus on Go semantics rather than a new domain problem. Format with gofmt and use go test ./... as routine development tools.

Worked Example

```
package main
import (
    "bufio"
    "fmt"
    "os"
    "strings"
)
func main() {
 reader := bufio.NewReader(os.Stdin)
 fmt.Print("Task title: ")
 title, _ := reader.ReadString('\n')
 title = strings.TrimSpace(title)
 if title == "" { fmt.Println("A title is required.") } else { fmt.Println("Accepted task:"
}
```

Expected behavior: The standalone program demonstrates accessible interaction and validation with shared course example data. It compiles and runs using the standard library. Predict the output first, then compare it with terminal output.

Common Mistakes

1. Mixing domain logic with terminal input/output in one function.
2. Assuming map iteration order is stable when displaying choices.
3. Making interaction depend on color, animation, or cursor positioning.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical text-adventure state and to-do fields and predict the output before running it.
3. Add a boundary case and one malformed or missing-data case; define expected behavior before editing the program.
4. Refactor one part using the Go feature in this lesson (accessible interaction and validation); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect Go punctuation (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain accessible interaction and validation using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

7.13 Unit 11: Capstone — Accessible Personal Data Manager

Overview

Build a personal data manager with the shared record schema, storage, search and completion, acceptance tests, and a documentation and accessibility check.

Lesson 11.1: Requirements and design

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain requirements and design using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

The capstone is an accessible record manager. Define record fields, validation rules, and storage format; implement create/read/update/delete; persist safely; and test normal, boundary, malformed, and missing-data cases. Document exact build, test, and run commands. The lesson uses canonical course data so learners focus on Go semantics rather than a new domain problem. Format with gofmt and use go test ./... as routine development tools.

Worked Example

```go
package main
import (
    "fmt"
    "strings"
)
func main() {
 fields := []string{"id", "title", "description", "category", "created_at", "updated_at"}
 fmt.Println("Capstone fields:", strings.Join(fields, ", "))
}
```

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (Go 1.22; the text of the output does not depend on the operating system):

```
Capstone fields: id, title, description, category, created_at, updated_at
```

Common Mistakes

1. Updating a record without confirming that the identifier exists.
2. Overwriting the only persisted copy before validating new data.
3. Treating a successful build as proof that CRUD and persistence work.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical capstone record fields and predict the output before running it.
3. Add a boundary case and one malformed or missing-data case; define expected behavior before editing the program.
4. Refactor one part using the Go feature in this lesson (requirements and design); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect Go punctuation (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain requirements and design using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 11.2: Data model and storage

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain data model and storage using Go terminology and type conventions.

- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

The capstone is an accessible record manager. Define record fields, validation rules, and storage format; implement create/read/update/delete; persist safely; and test normal, boundary, malformed, and missing-data cases. Document exact build, test, and run commands. The lesson uses canonical course data so learners focus on Go semantics rather than a new domain problem. Format with gofmt and use go test ./... as routine development tools.

Worked Example

```
package main
import (
    "encoding/json"
    "fmt"
)
func main() {
 type Record struct {
  ID int `json:"id"`
  Title string `json:"title"`
  Description string `json:"description"`
  Category string `json:"category"`
  CreatedAt string `json:"created_at"`
  UpdatedAt string `json:"updated_at"`
 }
 record := Record{ID: 1, Title: "Study Go", Description: "Complete the Go capstone", Categor
 data, err := json.Marshal(record)
 if err != nil { panic(err) }
 fmt.Println(string(data))
}
```

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (Go 1.22; the text of the output does not depend on the operating system):

```
{"id":1,"title":"Study Go","category":"learning"}
```

Common Mistakes

1. Updating a record without confirming that the identifier exists.
2. Overwriting the only persisted copy before validating new data.

3. Treating a successful build as proof that CRUD and persistence work.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical capstone record fields and predict the output before running it.
3. Add a boundary case and one malformed or missing-data case; define expected behavior before editing the program.
4. Refactor one part using the Go feature in this lesson (data model and storage); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect Go punctuation (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain data model and storage using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 11.3: Create, read, update, delete

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain create, read, update, delete using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

The capstone is an accessible record manager. Define record fields, validation rules, and storage format; implement create/read/update/delete; persist safely; and test normal, boundary, malformed, and missing-data cases. Document exact build, test, and run commands. The lesson uses canonical course data so learners focus on Go semantics rather than a new domain problem. Format with gofmt and use go test ./... as routine development tools.

Worked Example

```
package main
import (
    "fmt"
)
func main() {
 type Record struct { ID int; Title string; Done bool }
 records := []Record{{ID: 1, Title: "Draft"}, {ID: 2, Title: "Review"}}
 records[0].Done = true
 for _, record := range records { fmt.Printf("%d %s done=%t\n", record.ID, record.Title, rec
}
```

Expected behavior: The standalone program demonstrates create, read, update, delete with shared course example data. It compiles and runs using the standard library. Predict the output first, then compare it with terminal output.

Common Mistakes

1. Updating a record without confirming that the identifier exists.
2. Overwriting the only persisted copy before validating new data.
3. Treating a successful build as proof that CRUD and persistence work.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical capstone record fields and predict the output before running it.
3. Add a boundary case and one malformed or missing-data case; define expected behavior before editing the program.
4. Refactor one part using the Go feature in this lesson (create, read, update, delete); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect Go punctuation (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color

to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain create, read, update, delete using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 11.4: Validation and tests

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain validation and tests using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

The capstone is an accessible record manager. Define record fields, validation rules, and storage format; implement create/read/update/delete; persist safely; and test normal, boundary, malformed, and missing-data cases. Document exact build, test, and run commands. The lesson uses canonical course data so learners focus on Go semantics rather than a new domain problem. Format with gofmt and use go test ./... as routine development tools.

Worked Example

```
package main
import (
    "fmt"
    "strings"
)
func main() {
 valid := func(title string) bool { return strings.TrimSpace(title) != "" && len([]rune(titl
```

```
 for _, title := range []string{"Plan", "", strings.Repeat("x", 81)} { fmt.Printf("%q valid=
}
```

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (Go 1.22; the text of the output does not depend on the operating system):

```
"Plan" valid=true
"" valid=false
"xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx" valid=fa
```

Common Mistakes

1. Updating a record without confirming that the identifier exists.
2. Overwriting the only persisted copy before validating new data.
3. Treating a successful build as proof that CRUD and persistence work.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical capstone record fields and predict the output before running it.
3. Add a boundary case and one malformed or missing-data case; define expected behavior before editing the program.
4. Refactor one part using the Go feature in this lesson (validation and tests); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect Go punctuation (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain validation and tests using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 11.5: Documentation and presentation

Reading References

- The Go Programming Language Specification
- A Tour of Go
- Go standard library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain documentation and presentation using Go terminology and type conventions.
- Select an idiomatic Go construct rather than transliterating Python syntax.
- Predict, compile, and verify the worked example.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

The capstone is an accessible record manager. Define record fields, validation rules, and storage format; implement create/read/update/delete; persist safely; and test normal, boundary, malformed, and missing-data cases. Document exact build, test, and run commands. The lesson uses canonical course data so learners focus on Go semantics rather than a new domain problem. Format with gofmt and use go test ./... as routine development tools.

Worked Example

```
package main
import (
    "fmt"
)
func main() {
 fmt.Println("Project: accessible record manager")
 fmt.Println("Build: go build ./...")
 fmt.Println("Test: go test ./...")
 fmt.Println("Run: go run .")
 fmt.Println("Document validation, storage, errors, and keyboard workflow.")
}
```

Expected Behavior or Output

Output observed when the worked example was run with empty standard input (Go 1.22; the text of the output does not depend on the operating system):

```
Project: accessible record manager
Build: go build ./...
```

```
Test: go test ./...
Run: go run .
Document validation, storage, errors, and keyboard workflow.
```

Common Mistakes

1. Updating a record without confirming that the identifier exists.
2. Overwriting the only persisted copy before validating new data.
3. Treating a successful build as proof that CRUD and persistence work.

Activities

1. Run the worked example unchanged; trace each declaration, type, function call, and output line.
2. Change the canonical capstone record fields and predict the output before running it.
3. Add a boundary case and one malformed or missing-data case; define expected behavior before editing the program.
4. Refactor one part using the Go feature in this lesson (documentation and presentation); run gofmt and go test ./... and explain the result.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect Go punctuation (:=, &, *, ..., {}, and error assignments). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order; verify output is announced without unexpected focus movement.

Runnable Reference Implementation

The repository includes a tested Go domain and JSON-storage reference core in capstone/README.md. Run `go test -v ./...` from the GO/capstone directory. The reference core does not replace the requirement to build and evaluate an accessible user interface.

Skills Checklist

- ☐ I can explain documentation and presentation using Go terminology.
- ☐ I can format, compile, and run the example.
- ☐ I can explain the types and data flow used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Course-Wide Assessment and Portfolio

Submit a Go module that passes gofmt, go test ./..., and go build ./.... Document its data model, validation rules, error handling, persistence format, and keyboard/screen-reader workflow. Include exact build, test, and run commands in README.md.

AI Policy

AI tools may explain compiler diagnostics, compare data structures, and suggest test cases. Learners must explain every type, package, error path, concurrency decision, and dependency. Compile and test generated code, review dependency provenance, and do not expose private data or credentials. AI output is a draft, not evidence of correctness.

Go-Specific Verification Notes

The examples use the standard library. Production CSV processing should use encoding/csv rather than splitting rows manually. Check every file and JSON operation error; avoid panic for expected failures. The random sequence example is deterministic and pedagogical, not cryptographically secure.

Part VIII

JAVA

Chapter 8

Introduction to Computer Science with Java

Before you start: complete Unit P: Computing Foundations Primer. It explains the language-neutral vocabulary and concepts used in every language track, using non-visual examples.

Accessible beginner curriculum

The Java curriculum introduces programming, problem-solving, and object-oriented design using the Java language and Java Development Kit (JDK). It is designed for blind and visually impaired learners who use screen readers, braille displays, magnification, keyboard navigation, or combinations of access technologies. It is also appropriate for sighted beginners and mixed-vision classrooms.

Java makes types and program structure explicit. Its compilation process and class-based organization require more syntax than some introductory languages, but they also provide opportunities to teach how programs are organized, how types constrain operations, and how a compiler helps detect errors.

Recommended sequence: work through Units 0–11 in order. The Markdown files group related units; use the headings and lesson numbers inside each document.

Complete lesson sequence

The complete standardized course, including four setup lessons and 53 numbered lessons, is available in lessons.md. The original grouped unit documents remain

supplementary reference material. See validation status for example-check coverage and remaining verification.

Course outcomes

By the end of the curriculum, learners should be able to:

- Explain foundational computer science concepts and the software-development process.
- Install and verify a JDK and use an editor, terminal, compiler, and Java runtime.
- Read and write Java source code using classes, the `main` method, statements, expressions, and comments.
- Declare variables with primitive and reference types and reason about type conversion.
- Use arithmetic, input/output, conditionals, loops, and Boolean expressions.
- Work with arrays, ArrayLists, strings, and introductory collection types.
- Define and call methods with parameters and return values.
- Create classes and objects and explain fields, constructors, methods, encapsulation, and inheritance.
- Read and write files and handle exceptions.
- Apply introductory data-processing and unit-testing techniques.
- Plan, implement, test, and explain text-based projects and a capstone.
- Evaluate prompts, output, compiler diagnostics, and runtime messages for accessibility.

The curriculum establishes a foundation for further Java and computer science study; it does not attempt to cover the full Java platform.

Prerequisites and intended learners

No previous programming experience is required. Learners need a computer that can run a supported JDK, a keyboard-accessible editor or IDE, and a terminal. Basic keyboard and file-management skills are helpful. Screen-reader and braille workflows can develop alongside programming.

The primary audience includes students using NVDA, JAWS, VoiceOver, Narrator, braille displays, or magnification. The Java teacher's guide discusses the additional pacing and navigation demands of Java syntax, compiler messages, IDE tooling, and project structure.

Learning sequence

Unit	Main topics	Evidence of progress
0. Environment setup	JDK, editor and Java tools, project creation, terminal, first program	Compile and run a small Java program
1. Foundations	Computer science, problem solving, Java program structure, variables, and primitive types	Explain the role of a class, `main`, and typed variables
2. Input/output and math	Software-development process, Scanner, numeric input, and Math operations	Create an interactive calculation
3. Control flow	Boolean expressions, conditionals, loops, and branching	Predict and implement program paths
4. Arrays and ArrayLists	Indexed collections, iteration, and collection operations	Store and process multiple values
5. Strings	String operations, comparison, and text processing	Process user-provided text
6. Methods	Method declarations, parameters, return values, overloading, and decomposition	Organize behavior into reusable methods
7. Object-oriented programming	Classes, objects, constructors, fields, methods, and inheritance	Model a simple domain with objects
8. Files and exceptions	File input/output and exception handling	Persist information and handle expected failures
9. Collections and testing	Java Collections Framework, data analysis, and JUnit	Choose a collection and test program behavior
10. Project-based learning	Requirements, design, and text-based application development	Implement a multi-feature program
11. Capstone	Integration, testing, documentation, and explanation	Demonstrate and explain an independent solution

Materials and navigation

- Units 0–1: Setup and Foundations
- Units 2–4: Input, Control Flow, and Arrays
- Units 5–8: Strings, Methods, OOP, and Files
- Units 9–11: Collections, Testing, and Projects
- Teacher's guide: instructor planning, accessible instruction, assessment, and AI guidance

Return to the curriculum overview to compare the independent language pathways. Kotlin is now taught in its own standalone curriculum.

Recommended environment

Use Unit 0 for detailed installation steps. Use **JDK 25 LTS** as the course baseline. JDK 25 is the latest long-term-support release as of October 30, 2026; later non-LTS feature releases are not required. Keep the course examples within the Java SE 25 language and API level, without preview features. Check Adoptium or the official Oracle Java documentation for current distributions and versions.

Recommended tools include a JDK, Visual Studio Code with the Extension Pack for Java or Theia IDE, a terminal, and the learner's preferred access technology. Theia is an alternative with keyboard- and screen-reader-oriented workflows; verify Java language support, debugger, test runner, and extension compatibility through Open VSX before using it in class.

Useful verification commands:

```
java --version
javac --version
jshell
```

Windows Powershell

```
java --version
javac --version
jshell
```

The JDK provides JShell for interactive experiments. Complete programs are compiled and run as described in Unit 0. Build systems such as Maven or Gradle may be introduced when required; they should not be added before learners understand the basic compile/run cycle. When a build system is used, commit its Maven or Gradle Wrapper and run the project-local wrapper (`./mvnw` / `mvnw.cmd` or `./gradlew` / `gradlew.bat`) instead of requiring a global build-tool installation.

Java 25 baseline and current conventions

The course targets **JDK 25 LTS**. Use the latest security update in the JDK 25 line. The examples use standard Java 25 features and avoid preview features so learners can compile them with an ordinary JDK installation.

Follow standard Java conventions: PascalCase class and interface names; camelCase variables, methods, and parameters; UPPER_SNAKE_CASE constants; one public top-level class per matching source file; four-space indentation; explicit imports rather than wildcard imports; braces for all control-flow blocks; and meaningful method names. Prefer interfaces such as List and Map for variable declarations, use generics rather than raw collections, compare String content with equals/equalsIgnoreCase rather than ==, and use try-with-resources for closeable resources. Use var only for local variables when the initializer makes the type immediately obvious; keep explicit types in beginner examples. Avoid preview features and do not adopt a new syntax feature until the course objective requires it.

Accessibility and instructional design

Java uses braces to define blocks, semicolons to terminate statements, and explicit type declarations. Long class and method declarations can be difficult to navigate when heard or read on a narrow braille display. Teach students to navigate by class, method, line, and diagnostic, and keep examples logically organized.

The curriculum emphasizes keyboard-first editor, IDE, and terminal workflows; explicit interpretation of braces, parentheses, brackets, semicolons, generics, and type names; predictable class and method organization; text-based input/output; and compiler diagnostics as a first debugging resource.

IDE features can help with navigation and code completion, but generated project files and visual project explorers must not become the only way to understand or operate a project. Explain file structure and provide terminal equivalents where feasible.

How to study each lesson

1. Read the overview and identify the concepts and new syntax.
2. Read the class and method structure before tracing individual statements.
3. Predict output and identify the types of important values.
4. Compile and run the example, reading compiler messages and runtime output.
5. Change one statement or input and predict the effect before testing.
6. Complete the exercises, using the compiler as feedback rather than as a substitute for reasoning.

7. When compilation fails, read the diagnostic, identify the file and line, inspect the relevant declaration and use, then recompile.
8. When a runtime exception occurs, trace the values and method calls that led to it.
9. Explain why the types, methods, and data structures fit the problem.

Maintain a portfolio with source files, test cases, compiler errors resolved, and short design notes.

Assessment and projects

Use code reading, output prediction, debugging, method implementation, object modeling, test writing, refactoring, and project demonstrations. Assess correctness, reasoning, type safety, decomposition, testing, clarity, and the learner's ability to explain and modify their work.

The capstone should have a written problem statement, user requirements, class or data design, method responsibilities, text-based input/output, test cases, implementation milestones, and a demonstration or explanation. The student should identify limitations and propose a next improvement.

Teaching with AI

The student-facing AI Policy above and in the lesson file takes precedence over the instructor guidance in this section.

AI assistants may explain compiler messages, suggest test cases, compare method designs, or provide alternative explanations. They can also invent Java APIs, use unsupported language features, omit imports, produce incorrect generic types, mishandle exceptions, or generate a class hierarchy more complex than the learning objective requires.

Follow the instructor's policy. Students must disclose AI assistance where required and explain, test, and modify every submitted line. Do not share student records, credentials, private source code, or confidential data with public AI services. AI must not replace independent work in assessments intended to measure individual skills. The teacher's guide contains a fuller protocol.

Recommended introductory books

- Eck, David J. *Introduction to Programming Using Java.* A free online textbook introducing programming fundamentals, object-oriented concepts, and data structures. Author-maintained online text.
- Downey, Allen B., and Chris Mayfield. *Think Java: How to Think Like a Computer Scientist*, 2nd edition. O'Reilly Media, 2016. A free, structured

introduction that develops programming and computer science concepts step by step. Book site and publisher page.
- Gaddis, Tony. *Starting Out with Java: From Control Structures Through Objects*, 7th edition. Pearson, 2019. A traditional introductory course text with worked examples and exercises. Publisher catalog.

Suggested use: Eck and *Think Java* are suitable companions for self-study and conceptual review. Gaddis is an alternative structured textbook. Check the Java version used in any book against the course's JDK baseline; older editions may use APIs or conventions that differ from the current toolchain.

Further learning

Use the official Java documentation and Java Language Specification as authoritative references. David J. Eck's *Introduction to Programming Using Java* is an additional introductory text at math.hws.edu/javanotes.

Java's explicit types and program structure offer a useful way to deepen understanding of how software is organized. The objective is to build independent problem-solving habits that transfer to other languages and platforms.

GUI, visualization, and workbench accessibility

See the shared GUI, visualization, and workbench accessibility reference for screen-reader support by toolkit, platform-specific activation settings, recommended alternatives, and interfaces that must not be required without an equivalent accessible route.

AI Policy

AI may be used only for personal conceptual understanding and reinforcement. It must not write, complete, autocomplete, repair, or generate course code or assessment solutions.

8.1 JAVA Lessons

The curriculum is organized as one Markdown file per unit.

- accessible java development environment
- foundations of computing and java
- numeric computations
- control flow decisions and loops
- data structures lists and dictionaries

- user input and while loops
- functions and methods
- classes and object oriented programming
- files and exceptions
- testing and data analysis
- project based learning text applications
- capstone accessible personal data manager

8.2 Unit 0: Accessible Java Development Environment

Course overview

Before you start: complete Unit P: Computing Foundations Primer. It explains the language-neutral vocabulary and concepts used in every language track, using non-visual examples.

Language baseline: Java SE 25 LTS, JDK 25. Examples avoid preview features and third-party dependencies unless a lesson explicitly discusses an optional library.

Verification status: every lesson has one complete Java program that uses the shared datasets. All 57 programs compile with `javac` and run independently with empty standard input (checked on JDK 21; the baseline is JDK 25), and the "Expected Behavior or Output" blocks are the captured output of those programs, with the standard input shown where a program reads input. This has not been tested with screen readers or braille displays. Java prints the shared tip as 3.68 because its formatter rounds half up; Python, C, Go, and several other languages print 3.67 for the same stored value.

This file is the consolidated, lesson-by-lesson teaching sequence for Java. The existing grouped unit files remain supplementary reference material; this file provides the complete standardized sequence and independently runnable examples.

Course outcomes

Learners will install and verify a JDK, use an accessible editor and terminal, write and compile Java programs, reason about static types, use control flow and collections, decompose problems into methods and classes, handle files and exceptions, test behavior, and build an accessible command-line personal data manager.

Overview

Set up the language toolchain, a project folder, and a keyboard-first editor workflow that can be read with speech or braille, and learn how projects and

dependencies are organized.

Lesson 0.1: Installing the JDK

Reading References

- Java SE 25 API documentation
- Java SE 25 documentation
- Matching Python lesson: Lesson 0.1: Installing Python and uv

Learning Objectives

- Install a JDK from a trusted source.
- Verify `java` and `javac` from the terminal.
- Compile and run a first program.

Concept Explanation

Java programs are compiled to *bytecode* by `javac` and run by the Java Virtual Machine, started with `java`. Both tools come in the **JDK** (Java Development Kit). A JRE alone can run programs but cannot compile them. This course's documented baseline is JDK 25 (a long-term-support release); the examples avoid preview features and run on earlier recent releases too, but check the version you install.

Install a JDK from your operating system's package manager or from a vendor distribution, and write down which route you used. Then verify it:

```
java -version
javac -version
```

Both commands must print a version, and they should report the same major number. If `java` works but `javac` is not found, only a runtime is installed. If the numbers differ, an old Java is earlier on your PATH.

To compile and run the example below, save it as `Lesson_0_1.java` (the file name must match the public class name exactly, including capital letters), then:

```
javac Lesson_0_1.java
java Lesson_0_1
```

Worked Example

```java
public class Lesson_0_1 {
    public static void main(String[] args) {
        System.out.println("Java toolchain ready.");
    }
}
```

Expected behavior: The program compiles on JDK 25 and prints a deterministic result illustrating installing and verifying jdk 25. Run it from a terminal, compare actual output with your prediction, and change one input to test the effect.

Common Mistakes

- Installing a JRE-only runtime instead of a full JDK that includes javac.
- Assuming editor extensions, terminal PATH, and configured project runtime automatically use the same JDK.
- Depending on a visual project explorer or color highlighting without verifying source paths and terminal commands.

Activities

1. Verify the JDK, compiler, and runtime versions independently from both the system terminal and IDE terminal.
2. Navigate to the source file and run the same compile/run sequence without using editor buttons.
3. Use keyboard navigation and a screen reader to locate one compiler diagnostic and its source line.
4. Write a short setup checklist that another learner can reproduce.

Accessibility Practice

Navigate by class, method, and line; inspect braces, parentheses, brackets, semicolons, generic delimiters, and compiler diagnostics with the preferred screen reader or braille display. Compile and run from the terminal so the workflow does not depend on color or a visual project explorer.

Skills Checklist

☐ I can explain the key concept in Java terms.
☐ I can trace and predict the worked example.
☐ I can compile and run the example from the terminal.
☐ I can identify and correct a Java-specific mistake.
☐ I can test a boundary case and navigate the workflow accessibly.

Lesson 0.2: Configuring VS Code or Theia for Java

Reading References

- Java SE 25 documentation.
- Java Language Specification, Java SE 25.
- Eclipse Temurin JDK distributions.

Learning Objectives

- Explain the purpose and vocabulary of configuring vs code or theia for java.
- Apply the relevant Java syntax and APIs in a small, complete program.
- Compile, run, inspect, and adapt the example using keyboard-accessible tools.

Concept Explanation

Configure Java language support, diagnostics, debugging, testing, keyboard navigation, and screen-reader mode.

Java is statically typed and class-based. Local variables have declared or inferred compile-time types, reference values may be null, and methods and fields are governed by access and scope rules. The compiler checks syntax and type compatibility before execution; runtime behavior still requires testing. Follow Java naming conventions, use explicit imports in beginner work, and prefer small methods with clear contracts.

Worked Example

```
public class Lesson_0_2 {
    public static void main(String[] args) {
        java.util.List<String> milestones = java.util.List.of("model", "implement", "test",
        for (String milestone : milestones) System.out.println("Completed: " + milestone);
    }
}
```

Expected Behavior or Output

The program prints:

```
Hello, Avery!
Welcome to Java programming!
```

Common Mistakes

1. Running the commands from a different folder, so the compiler cannot find the file.
2. Fixing the last error message first. Start with the first one.
3. Forgetting a semicolon or a closing brace, which produces several confusing messages from one cause.

Activities

1. Create the folder and file from the keyboard and run the program. Record the working directory.

2. Delete the closing quotation mark on one line, compile, and write down the file, line, and message. Then restore it.
3. Add a third `println` with your own name and predict the output before running.
4. Run the program with `java Lesson_0_2.java` and compare it with the two-step route.

Accessibility Practice

Create the folder and file with terminal commands or your editor's keyboard commands. When the compiler reports an error, read the file name and line number first, then jump to that line in the editor. The caret line under the error is easy to miss, so ask your screen reader to read it by character.

Skills Checklist

- ☐ I can create a project folder and source file.
- ☐ I can compile and run from the terminal.
- ☐ I can read the file and line of an error.
- ☐ I know a class needs a `main` method to start.
- ☐ I know what the working directory is.

Lesson 0.3: Editor Extensions and Reading Java Aloud

Reading References

- Java SE 25 API documentation
- Dev.java learning paths
- Matching Python lesson: Lesson 0.3: Setting Up Your VS Code or Theia IDE Workspace

Learning Objectives

- Configure an editor so Java source is readable by speech or braille.
- Choose a punctuation level that announces the symbols Java depends on.
- Read generics, lambdas, and method references aloud.

Concept Explanation

Complete the shared IDE setup lesson first. Then adapt the editor for Java. A Java language extension is optional; the course works with a plain editor and the terminal. If you install one, take it only from a source you trust and read the permissions it requests.

Java depends on punctuation that a screen reader may skip at low settings. Learn to say these: `;` ends a statement, `{ }` hold blocks, `( )` hold parameters

and arguments, `[ ]` index arrays, `< >` hold *generic* types (`List<String>` is "a list of strings", and `<>` alone is the *diamond*), `->` separates a lambda's parameters from its body, `::` makes a *method reference*, `@Override` is an *annotation*, and `?` and `:` form the *conditional operator*.

Java structure comes from braces, not indentation, so count the closing braces when you read a nested block.

Read the example aloud: "List open angle bracket String close angle bracket names equals new ArrayList open angle bracket close angle bracket open parenthesis close parenthesis semicolon." Then say what it prints.

Worked Example

```
import java.util.ArrayList;
import java.util.List;

public class Lesson_0_3 {
    public static void main(String[] args) {
        List<String> names = new ArrayList<>();
        names.add("Avery");
        names.forEach(name -> System.out.println(name.length()));
        names.forEach(System.out::println);

        String missing = null;
        System.out.println(missing == null ? "no value" : missing);
    }
}
```

Expected behavior: The program compiles on JDK 25 and prints a deterministic result illustrating creating a workspace and first java project. Run it from a terminal, compare actual output with your prediction, and change one input to test the effect.

Common Mistakes

- Installing a JRE-only runtime instead of a full JDK that includes javac.
- Assuming editor extensions, terminal PATH, and configured project runtime automatically use the same JDK.
- Depending on a visual project explorer or color highlighting without verifying source paths and terminal commands.

Activities

1. Verify the JDK, compiler, and runtime versions independently from both the system terminal and IDE terminal.
2. Navigate to the source file and run the same compile/run sequence without using editor buttons.

3. Use keyboard navigation and a screen reader to locate one compiler diagnostic and its source line.
4. Write a short setup checklist that another learner can reproduce.

Accessibility Practice

Navigate by class, method, and line; inspect braces, parentheses, brackets, semicolons, generic delimiters, and compiler diagnostics with the preferred screen reader or braille display. Compile and run from the terminal so the workflow does not depend on color or a visual project explorer.

Skills Checklist

- ☐ I can explain the key concept in Java terms.
- ☐ I can trace and predict the worked example.
- ☐ I can compile and run the example from the terminal.
- ☐ I can identify and correct a Java-specific mistake.
- ☐ I can test a boundary case and navigate the workflow accessibly.

Lesson 0.4: Terminal, Packages, and Build-Tool Foundations

Reading References

- Java SE 25 documentation.
- Java Language Specification, Java SE 25.
- Eclipse Temurin JDK distributions.

Learning Objectives

- Explain the purpose and vocabulary of terminal, packages, and build-tool foundations.
- Apply the relevant Java syntax and APIs in a small, complete program.
- Compile, run, inspect, and adapt the example using keyboard-accessible tools.

Concept Explanation

Navigate the project from the terminal, understand package paths and classpaths, and introduce Maven or Gradle wrappers only when needed.

Java is statically typed and class-based. Local variables have declared or inferred compile-time types, reference values may be null, and methods and fields are governed by access and scope rules. The compiler checks syntax and type compatibility before execution; runtime behavior still requires testing. Follow Java naming conventions, use explicit imports in beginner work, and prefer small methods with clear contracts.

Worked Example

```
public class Lesson_0_4 {
    public static void main(String[] args) {
        java.util.List<String> milestones = java.util.List.of("model", "implement", "test",
        for (String milestone : milestones) System.out.println("Completed: " + milestone);
    }
}
```

Expected behavior: The program compiles on JDK 25 and prints a deterministic result illustrating terminal, packages, and build-tool foundations. Run it from a terminal, compare actual output with your prediction, and change one input to test the effect.

Common Mistakes

- Installing a JRE-only runtime instead of a full JDK that includes javac.
- Assuming editor extensions, terminal PATH, and configured project runtime automatically use the same JDK.
- Depending on a visual project explorer or color highlighting without verifying source paths and terminal commands.

Activities

1. Verify the JDK, compiler, and runtime versions independently from both the system terminal and IDE terminal.
2. Navigate to the source file and run the same compile/run sequence without using editor buttons.
3. Use keyboard navigation and a screen reader to locate one compiler diagnostic and its source line.
4. Write a short setup checklist that another learner can reproduce.

Accessibility Practice

Navigate by class, method, and line; inspect braces, parentheses, brackets, semicolons, generic delimiters, and compiler diagnostics with the preferred screen reader or braille display. Compile and run from the terminal so the workflow does not depend on color or a visual project explorer.

Skills Checklist

☐ I can explain the key concept in Java terms.
☐ I can trace and predict the worked example.
☐ I can compile and run the example from the terminal.
☐ I can identify and correct a Java-specific mistake.
☐ I can test a boundary case and navigate the workflow accessibly.

8.3 Unit 1: Foundations of Computing and Java

Overview

This unit develops foundations of computing and java through the same shared problem contexts and assessment expectations used across the general-purpose language tracks, expressed with Java 25 syntax, APIs, and static typing.

Unit objectives: explain the core concepts, write and test complete Java programs, diagnose errors, and apply the ideas to accessible projects.

Lesson 1.1: Welcome to Computer Science

Reading References

- Java SE 25 API documentation
- Dev.java learning paths
- Matching Python lesson: Lesson 1.1: Welcome to Computer Science

Learning Objectives

- Explain hardware, software, and algorithm in your own words.
- Turn a written algorithm into a short Java program.
- Predict a program's output by hand before running it.

Concept Explanation

Computers follow instructions exactly. *Hardware* is the machine, *software* is the set of instructions it follows, and an *algorithm* is an ordered, finite list of clear steps that solves a problem. Unit P of this repository covers these ideas in detail.

A reliable way to solve a problem is to understand it, break it into steps, write the steps in plain language, translate them into code, and test the result.

The plan here is: set a total to zero; for each score, add it to the total; divide the total by the number of scores; report both. In Java, `int[] scores = {...}` creates an array of whole numbers, `for (int score : scores)` visits each one in order, and `scores.length` is the number of items.

The shared scores are 82, 91, 76, and 88. Their total is 337 and their average is 84.25. A hand trace should reach the same numbers: 82, 173, 249, then 337.

Worked Example

```
public class Lesson_1_1 {
    public static void main(String[] args) {
        int[] scores = {82, 91, 76, 88};
        int total = 0;
        for (int score : scores) {
```

```
            total += score;
        }
        double average = (double) total / scores.length;
        System.out.println("Total: " + total);
        System.out.println("Average: " + average);
    }
}
```

Expected behavior: The program compiles on JDK 25 and prints a deterministic result illustrating welcome to computer science. Run it from a terminal, compare actual output with your prediction, and change one input to test the effect.

Common Mistakes

- Treating Java primitive values and object references as interchangeable types.
- Using an incorrect public class/file-name pairing or omitting the main entry-point signature.
- Reading an uninitialized local variable or narrowing a numeric value without checking for data loss.

Activities

1. Explain the hardware/software or algorithmic context, then express the steps as ordered Java statements.
2. Change the learner profile values and identify the compile-time types of each field.
3. Add a constant and a safe numeric conversion, then explain what information could be lost.
4. Compile and run from the terminal after deliberately introducing and correcting one syntax error.

Accessibility Practice

Navigate by class, method, and line; inspect braces, parentheses, brackets, semicolons, generic delimiters, and compiler diagnostics with the preferred screen reader or braille display. Compile and run from the terminal so the workflow does not depend on color or a visual project explorer.

Skills Checklist

☐ I can explain the key concept in Java terms.
☐ I can trace and predict the worked example.
☐ I can compile and run the example from the terminal.
☐ I can identify and correct a Java-specific mistake.
☐ I can test a boundary case and navigate the workflow accessibly.

Lesson 1.2: Your First Java Program

Reading References

- Java SE 25 API documentation
- Dev.java learning paths
- Matching Python lesson: Lesson 1.2: Your First Python Program - Hello World

Learning Objectives

- Write and run a program that prints text.
- Use comments, `printf`, and a text block.
- Explain that statements run in order.

Concept Explanation

A Java program is a list of statements that run in order inside `main`. `System.out.println` writes text and ends the line, and `System.out.print` writes without ending the line. Text in double quotation marks is a `String`, and `+` joins strings.

`System.out.printf(format, values...)` inserts values into a template: `%d` is a whole number, `%s` is text, `%.2f` is a decimal with two places, and `%n` is a line ending that is right for the platform. A *text block* written with three quotation marks (`"""`) spans several lines and removes the common indentation. A line comment starts with `//` and a block comment is written between `/*` and `*/`.

Because Java is compiled, a mistake such as a missing quotation mark is reported at compile time and no program is produced.

Worked Example

```
public class Lesson_1_2 {
    public static void main(String[] args) {
        // This program greets the shared student.
        String name = "Avery";
        System.out.println("Hello, " + name + "!");
        System.out.print("No new line here, ");
        System.out.println("then one here.");
        System.out.printf("Name has %d letters.%n", name.length());

        String text = """
            Two lines
            in one text block.""";
        System.out.println(text);
    }
}
```

Expected Behavior or Output

The program prints:

```
Hello, Avery!
No new line here, then one here.
Name has 5 letters.
Two lines
in one text block.
```

Common Mistakes

1. Using a curly quotation mark pasted from a document instead of a straight one, which is a syntax error.
2. Using `\n` in `printf` where `%n` is the portable line ending.
3. Forgetting that `print` does not end the line.

Activities

1. Run the program and describe how the second and third lines differ from the others.
2. Change the greeting to use your own name and predict the output.
3. Print three lines: your name, the course name, and today's task.
4. Add a comment above every line and read the file aloud with the comments.

Accessibility Practice

Run from the terminal and read the output line by line. Check that your screen reader announces the percent signs in a `printf` template, since they decide what the program prints.

Skills Checklist

☐ I can write and run a program that prints text.
☐ I can use `printf` formats.
☐ I can write comments.
☐ I know how `print` and `println` differ.
☐ I can read a first compile error.

Lesson 1.3: Variables and Simple Data Types

Reading References

- Java SE 25 API documentation
- Java Language Specification, Java SE 25
- Dev.java learning paths

- Matching Python lesson: Lesson 1.3: Variables and Simple Data Types

Learning Objectives

- Declare variables of the basic types.
- Explain primitive versus reference types and `null`.
- Use `final` and `var`.

Concept Explanation

Java is *statically typed*: every variable has a type that the compiler knows. The *primitive* types are `int` and `long` (whole numbers), `double` and `float` (decimals), `boolean`, and `char`. `String` is a *reference* type: the variable refers to an object, and a reference may be `null`, which means "no object". Primitives are never null.

`final` makes a variable assignable only once. `var` lets the compiler infer the type of a local variable from its initializer (`var inferred = 16L;` is a `long`); the type is still fixed. A whole-number literal such as 16 is an `int` and 1.72 is a `double`; a trailing `L` makes a `long`.

Wrapper classes such as `Integer` and `Double` turn a primitive into an object when one is needed, for example in a collection.

The shared student profile is Avery, 16, 1.72 meters, and Computer Science. The shared Notebook item has quantity 3 and unit price 2.50, so three notebooks cost 7.5.

Worked Example

```
public class Lesson_1_3 {
    public static void main(String[] args) {
        String name = "Avery";
        int age = 16;
        double height = 1.72;
        String favoriteSubject = "Computer Science";
        boolean likesCs = true;
        System.out.println(name + ", " + age + ", " + height + ", " + favoriteSubject + ", "

        var inferred = 16L;
        Object boxedAge = age;
        Object boxedHeight = height;
        Object boxedInferred = inferred;
        System.out.println(boxedAge.getClass().getSimpleName() + " " + boxedHeight.getClass(
            + " " + boxedInferred.getClass().getSimpleName());

        final double notebookPrice = 2.50;
        int notebookQuantity = 3;
```

```
        System.out.println("Notebook total: " + notebookPrice * notebookQuantity);

        int visits = 1;
        visits++;
        System.out.println("Visits: " + visits);

        Integer none = null;
        System.out.println(none);
    }
}
```

Expected behavior: The program compiles on JDK 25 and prints a deterministic result illustrating variables and primitive data types. Run it from a terminal, compare actual output with your prediction, and change one input to test the effect.

Common Mistakes

- Treating Java primitive values and object references as interchangeable types.
- Using an incorrect public class/file-name pairing or omitting the main entry-point signature.
- Reading an uninitialized local variable or narrowing a numeric value without checking for data loss.

Activities

1. Explain the hardware/software or algorithmic context, then express the steps as ordered Java statements.
2. Change the learner profile values and identify the compile-time types of each field.
3. Add a constant and a safe numeric conversion, then explain what information could be lost.
4. Compile and run from the terminal after deliberately introducing and correcting one syntax error.

Accessibility Practice

Navigate by class, method, and line; inspect braces, parentheses, brackets, semicolons, generic delimiters, and compiler diagnostics with the preferred screen reader or braille display. Compile and run from the terminal so the workflow does not depend on color or a visual project explorer.

Skills Checklist

☐ I can explain the key concept in Java terms.
☐ I can trace and predict the worked example.

☐ I can compile and run the example from the terminal.
☐ I can identify and correct a Java-specific mistake.
☐ I can test a boundary case and navigate the workflow accessibly.

8.4 Unit 2: Numeric Computations

Overview

This unit develops numeric computations through the same shared problem contexts and assessment expectations used across the general-purpose language tracks, expressed with Java 25 syntax, APIs, and static typing.

Unit objectives: explain the core concepts, write and test complete Java programs, diagnose errors, and apply the ideas to accessible projects.

Lesson 2.1: The Software Development Process

Reading References

- Java SE 25 API documentation
- Dev.java learning paths
- Matching Python lesson: Lesson 2.1: The Software Development Process

Learning Objectives

- Describe the design, implement, and test cycle.
- Write pseudocode for the tip calculator and implement it.
- Explain why money is formatted to two decimals and how the locale matters.

Concept Explanation

Software is built in a cycle: understand the problem, design a solution, implement it, test it, and refine it. For the tip calculator the plan is: get the bill, calculate 15 percent of it as the tip, add the tip to the bill for the total, and print the tip and the total.

With the shared values (bill 24.50, tip rate 0.15) the exact tip is 3.675 and the total is 28.175. `String.format("%.2f", x)` formats a number with two decimals. Java's formatter rounds **half up** using the decimal digits of the `double`, so Java prints the tip as **3.68**; Python, C, Go, and several other languages print 3.67 for the same stored number. Both are correct displays of the same value.

Formatting uses the default *locale*. In a locale that writes decimals with a comma, `%.2f` prints a comma. Passing `Locale.US` as the first argument gives fixed output, which the example does through a small helper method.

Worked Example

```java
import java.util.Locale;

public class Lesson_2_1 {
    static String money(double value) {
        return String.format(Locale.US, "%.2f", value);
    }

    public static void main(String[] args) {
        double bill = 24.5;
        double tipRate = 0.15;

        double tip = bill * tipRate;
        double total = bill + tip;

        System.out.println("Bill: " + money(bill));
        System.out.println("Tip: " + money(tip));
        System.out.println("Total: " + money(total));
        System.out.println("Unformatted tip: " + tip);
    }
}
```

Expected behavior: The program compiles on JDK 25 and prints a deterministic result illustrating the software development process. Run it from a terminal, compare actual output with your prediction, and change one input to test the effect.

Common Mistakes

- Using integer division where a decimal result is required, or applying an expression in the wrong order.
- Assuming Scanner parsing accepts malformed input or forgetting the newline left by token-based reads.
- Using an unchecked parse result or rounding intermediate Math values before the final calculation.

Activities

1. Calculate the same tip, conversion, or circle result using hand arithmetic and Java, then compare precision.
2. Test zero, a negative value, and a nonnumeric string; document which should be accepted.
3. Compare integer and floating-point division using the same operands.
4. Add a validation message and verify it does not print a misleading result.

Accessibility Practice

Navigate by class, method, and line; inspect braces, parentheses, brackets, semicolons, generic delimiters, and compiler diagnostics with the preferred screen reader or braille display. Compile and run from the terminal so the workflow does not depend on color or a visual project explorer.

Skills Checklist

- ☐ I can explain the key concept in Java terms.
- ☐ I can trace and predict the worked example.
- ☐ I can compile and run the example from the terminal.
- ☐ I can identify and correct a Java-specific mistake.
- ☐ I can test a boundary case and navigate the workflow accessibly.

Lesson 2.2: Expressions and Assignment

Reading References

- Java SE 25 API documentation
- Java Language Specification, Java SE 25
- Dev.java learning paths
- Matching Python lesson: Lesson 2.2: Elements of Programs - Expressions and Assignment

Learning Objectives

- Evaluate expressions using operator precedence.
- Predict integer division, remainder, and string joining.
- Use compound assignment.

Concept Explanation

An *expression* produces a value. The arithmetic operators are `+`, `-`, `*`, `/`, and `%` (remainder). Multiplication, division, and remainder happen before addition and subtraction, and parentheses change the order, so `8 + 4 * 2` is 16 and `(8 + 4) * 2` is 24.

Division depends on the types. Two `int` values give an `int`, with the fraction dropped toward zero, so `7 / 2` is `3` and `-7 / 2` is `-3`. If either side is a `double`, the result is a `double`: `7 / 2.0` is `3.5`. The remainder keeps the sign of the left side, so `-7 % 2` is `-1`.

Assignment stores a value: `score = score + 9`, shortened to `score += 9`. `+` also joins strings, and the order matters: `1 + 2 + "3"` adds first and gives `"33"`, while `"1" + 2 + 3` joins left to right and gives `"123"`. Comparison uses `==`, `!=`, `<`, `<=`, `>`, and `>=`.

Worked Example

```
public class Lesson_2_2 {
    public static void main(String[] args) {
        System.out.println(8 + 4 * 2);
        System.out.println((8 + 4) * 2);

        int score = 82;
        score += 9;
        System.out.println("New score: " + score);

        System.out.println(7 / 2);
        System.out.println(7 / 2.0);
        System.out.println(7 % 2);
        System.out.println(-7 / 2);
        System.out.println(-7 % 2);
        System.out.println(5 != 6);
        System.out.println("Avery" + " " + "Student");
        System.out.println(1 + 2 + "3");
        System.out.println("1" + 2 + 3);
    }
}
```

Expected behavior: The program compiles on JDK 25 and prints a deterministic result illustrating expressions and assignment. Run it from a terminal, compare actual output with your prediction, and change one input to test the effect.

Common Mistakes

- Using integer division where a decimal result is required, or applying an expression in the wrong order.
- Assuming Scanner parsing accepts malformed input or forgetting the newline left by token-based reads.
- Using an unchecked parse result or rounding intermediate Math values before the final calculation.

Activities

1. Calculate the same tip, conversion, or circle result using hand arithmetic and Java, then compare precision.
2. Test zero, a negative value, and a nonnumeric string; document which should be accepted.
3. Compare integer and floating-point division using the same operands.
4. Add a validation message and verify it does not print a misleading result.

Accessibility Practice

Navigate by class, method, and line; inspect braces, parentheses, brackets, semicolons, generic delimiters, and compiler diagnostics with the preferred screen reader or braille display. Compile and run from the terminal so the workflow does not depend on color or a visual project explorer.

Skills Checklist

- ☐ I can explain the key concept in Java terms.
- ☐ I can trace and predict the worked example.
- ☐ I can compile and run the example from the terminal.
- ☐ I can identify and correct a Java-specific mistake.
- ☐ I can test a boundary case and navigate the workflow accessibly.

Lesson 2.3: Input and Output Revisited

Reading References

- Java SE 25 documentation.
- Java Language Specification, Java SE 25.

Learning Objectives

- Explain the purpose and vocabulary of input and output revisited.
- Apply the relevant Java syntax and APIs in a small, complete program.
- Compile, run, inspect, and adapt the example using keyboard-accessible tools.

Concept Explanation

This lesson develops input and output revisited through Java's class-based structure and standard library. Identify the input, output, state changes, and relevant type constraints before coding. Keep declarations local where possible, separate computation from presentation, and state preconditions and postconditions. Use numeric types deliberately: integer division truncates, floating-point calculations can round, parsing may throw NumberFormatException, and Math exposes constants and operations such as PI, sqrt, pow, and hypot.

Java is statically typed and class-based. Local variables have declared or inferred compile-time types, reference values may be null, and methods and fields are governed by access and scope rules. The compiler checks syntax and type compatibility before execution; runtime behavior still requires testing. Follow Java naming conventions, use explicit imports in beginner work, and prefer small methods with clear contracts.

Worked Example

```
public class Lesson_2_3 {
    public static void main(String[] args) {
        java.util.Scanner input = new java.util.Scanner("Avery\n12 30\n");
        String name = input.nextLine();
        int first = input.nextInt();
        int second = input.nextInt();
        System.out.println("Hello, " + name + "; sum = " + (first + second));
        input.close();
    }
}
```

Expected Behavior or Output

With the following lines typed as standard input (piped input is not echoed, so it does not appear in the output):

```
Avery
12
8
```

the program prints:

```
Enter your name:
Enter the first number:
Enter the second number:
Hello, Avery!
The sum of 12 and 8 is 20.
```

With no standard input at all, the program ends cleanly and prints:

```
Enter your name:
Enter the first number:
Enter the second number:
Input was missing or not a number.
```

Common Mistakes

1. Calling `nextLine()` without checking `hasNextLine()` and crashing at the end of input.
2. Calling `Integer.parseInt` on unchecked text and crashing on `abc`.
3. Mixing `nextInt()` and `nextLine()`, which leaves the line ending behind and makes the next `nextLine()` return an empty string.

Activities

1. Run with Avery, 12, and 8 and confirm the output.
2. Type `twelve` for the first number and explain the result.

3. Run with no input and describe the output.
4. Change the program to subtract the second number from the first.

Accessibility Practice

Every prompt is a complete sentence on its own line and every result is a complete sentence. When you test, listen for whether the prompt is read in full before you begin to type.

Skills Checklist

- ☐ I can read a line with `Scanner`.
- ☐ I can convert text with `Integer.parseInt`.
- ☐ I check `hasNextLine()` first.
- ☐ I can write clear prompts.
- ☐ I handle missing and invalid input.

Lesson 2.4: Numbers and Type Conversions

Reading References

- Java SE 25 API documentation
- Java Language Specification, Java SE 25
- Dev.java learning paths
- Matching Python lesson: Lesson 2.4: Computing with Numbers and Type Conversions

Learning Objectives

- Convert between numbers and text.
- Explain truncation by a cast and rounding with `Math.round`.
- Explain why decimal arithmetic is approximate.

Concept Explanation

`Integer.parseInt` and `Double.parseDouble` convert text to numbers and throw `NumberFormatException` for invalid text. `String.valueOf(x)` or `"" + x` converts a number to text.

A *cast* converts between numeric types: `(int) 3.9` is `3`, because casting a `double` to an `int` **truncates** toward zero. `Math.round(3.5)` rounds to the nearest whole number (halves go up) and returns a `long`. Converting to a narrower type can lose information, so Java requires you to write the cast.

A `double` is a binary approximation. `0.1 + 0.2` is `0.30000000000000004`, not `0.3`; this is correct behavior. Compare decimals with a tolerance and format them for display.

The shared student has age 16 and height 1.72 meters. The example converts text to those values and computes the height in centimeters.

Worked Example

```
import java.util.Locale;

public class Lesson_2_4 {
    public static void main(String[] args) {
        int age = Integer.parseInt("16");
        double height = Double.parseDouble("1.72");
        System.out.println(age + " " + height);

        try {
            Integer.parseInt("abc");
        } catch (NumberFormatException e) {
            System.out.println("abc is not a whole number");
        }
        System.out.println(String.valueOf(42) + " is now text");

        System.out.println((int) 3.9);
        System.out.println(Math.round(3.5));
        System.out.println(0.1 + 0.2);

        double heightCm = height * 100;
        System.out.println("Height in centimeters: " + String.format(Locale.US, "%.1f", heig
    }
}
```

Expected behavior: The program compiles on JDK 25 and prints a deterministic result illustrating computing with numbers and type conversions. Run it from a terminal, compare actual output with your prediction, and change one input to test the effect.

Common Mistakes

- Using integer division where a decimal result is required, or applying an expression in the wrong order.
- Assuming Scanner parsing accepts malformed input or forgetting the newline left by token-based reads.
- Using an unchecked parse result or rounding intermediate Math values before the final calculation.

Activities

1. Calculate the same tip, conversion, or circle result using hand arithmetic and Java, then compare precision.

2. Test zero, a negative value, and a nonnumeric string; document which should be accepted.
3. Compare integer and floating-point division using the same operands.
4. Add a validation message and verify it does not print a misleading result.

Accessibility Practice

Navigate by class, method, and line; inspect braces, parentheses, brackets, semicolons, generic delimiters, and compiler diagnostics with the preferred screen reader or braille display. Compile and run from the terminal so the workflow does not depend on color or a visual project explorer.

Skills Checklist

☐ I can explain the key concept in Java terms.
☐ I can trace and predict the worked example.
☐ I can compile and run the example from the terminal.
☐ I can identify and correct a Java-specific mistake.
☐ I can test a boundary case and navigate the workflow accessibly.

Lesson 2.5: Using the Math Library

Reading References

- Java SE 25 documentation.
- Java Language Specification, Java SE 25.

Learning Objectives

- Explain the purpose and vocabulary of using the math library.
- Apply the relevant Java syntax and APIs in a small, complete program.
- Compile, run, inspect, and adapt the example using keyboard-accessible tools.

Concept Explanation

This lesson develops using the math library through Java's class-based structure and standard library. Identify the input, output, state changes, and relevant type constraints before coding. Keep declarations local where possible, separate computation from presentation, and state preconditions and postconditions. Use numeric types deliberately: integer division truncates, floating-point calculations can round, parsing may throw NumberFormatException, and Math exposes constants and operations such as PI, sqrt, pow, and hypot.

Java is statically typed and class-based. Local variables have declared or inferred compile-time types, reference values may be null, and methods and fields are governed by access and scope rules. The compiler checks syntax and type compatibility before execution; runtime behavior still requires testing. Follow

Java naming conventions, use explicit imports in beginner work, and prefer small methods with clear contracts.

Worked Example

```
public class Lesson_2_5 {
    public static void main(String[] args) {
        double radius = 2.5;
        double area = Math.PI * Math.pow(radius, 2);
        double circumference = 2 * Math.PI * radius;
        double distance = Math.hypot(6, 8);
        System.out.printf("area %.3f; circumference %.3f; distance %.1f%n", area, circumfere
    }
}
```

Expected behavior: The program compiles on JDK 25 and prints a deterministic result illustrating using the math library. Run it from a terminal, compare actual output with your prediction, and change one input to test the effect.

Common Mistakes

- Using integer division where a decimal result is required, or applying an expression in the wrong order.
- Assuming Scanner parsing accepts malformed input or forgetting the newline left by token-based reads.
- Using an unchecked parse result or rounding intermediate Math values before the final calculation.

Activities

1. Calculate the same tip, conversion, or circle result using hand arithmetic and Java, then compare precision.
2. Test zero, a negative value, and a nonnumeric string; document which should be accepted.
3. Compare integer and floating-point division using the same operands.
4. Add a validation message and verify it does not print a misleading result.

Accessibility Practice

Navigate by class, method, and line; inspect braces, parentheses, brackets, semicolons, generic delimiters, and compiler diagnostics with the preferred screen reader or braille display. Compile and run from the terminal so the workflow does not depend on color or a visual project explorer.

Skills Checklist

☐ I can explain the key concept in Java terms.
☐ I can trace and predict the worked example.

☐ I can compile and run the example from the terminal.
☐ I can identify and correct a Java-specific mistake.
☐ I can test a boundary case and navigate the workflow accessibly.

8.5 Unit 3: Control Flow — Decisions and Loops

Overview

This unit develops control flow — decisions and loops through the same shared problem contexts and assessment expectations used across the general-purpose language tracks, expressed with Java 25 syntax, APIs, and static typing.

Unit objectives: explain the core concepts, write and test complete Java programs, diagnose errors, and apply the ideas to accessible projects.

Lesson 3.1: Boolean Expressions and if Statements

Reading References

- Java SE 25 documentation.
- Java Language Specification, Java SE 25.

Learning Objectives

- Explain the purpose and vocabulary of boolean expressions and if statements.
- Apply the relevant Java syntax and APIs in a small, complete program.
- Compile, run, inspect, and adapt the example using keyboard-accessible tools.

Concept Explanation

This lesson develops boolean expressions and if statements through Java's class-based structure and standard library. Identify the input, output, state changes, and relevant type constraints before coding. Keep declarations local where possible, separate computation from presentation, and state preconditions and postconditions. Boolean conditions evaluate to true or false; Java does not implicitly convert numbers or references to booleans. Loops need a clear initialization, continuation condition, and state update. Guard sentinel and input loops against end-of-file and malformed tokens.

Java is statically typed and class-based. Local variables have declared or inferred compile-time types, reference values may be null, and methods and fields are governed by access and scope rules. The compiler checks syntax and type compatibility before execution; runtime behavior still requires testing. Follow

Java naming conventions, use explicit imports in beginner work, and prefer small methods with clear contracts.

Worked Example

```
public class Lesson_3_1 {
    public static void main(String[] args) {
        int score = 82;
        if (score >= 60) {
            System.out.println("Avery passed.");
        } else {
            System.out.println("Avery did not pass.");
        }

        System.out.println(score >= 80 && score < 90);
        System.out.println(score < 60 || score > 100);
        System.out.println(!(score >= 90));

        String label = score >= 90 ? "high" : "not high";
        System.out.println(label);

        String text = null;
        System.out.println(text != null && text.length() > 0);
    }
}
```

Expected behavior: The program compiles on JDK 25 and prints a deterministic result illustrating boolean expressions and if statements. Run it from a terminal, compare actual output with your prediction, and change one input to test the effect.

Common Mistakes

- Using == for String content comparison instead of equals or equalsIgnoreCase.
- Ordering chained score thresholds incorrectly so a higher grade is captured by a lower branch.
- Using an incorrect loop bound, modifying a collection during enhanced iteration, or confusing array length with collection size.

Activities

1. Test each decision threshold immediately below, at, and above the boundary.
2. Process the four shared scores and verify count, total, and average.
3. Normalize a mixed-case sentence containing leading/trailing whitespace and punctuation.

4. Explain the difference between comparing primitive values and comparing String content.

Accessibility Practice

Navigate by class, method, and line; inspect braces, parentheses, brackets, semicolons, generic delimiters, and compiler diagnostics with the preferred screen reader or braille display. Compile and run from the terminal so the workflow does not depend on color or a visual project explorer.

Skills Checklist

- ☐ I can explain the key concept in Java terms.
- ☐ I can trace and predict the worked example.
- ☐ I can compile and run the example from the terminal.
- ☐ I can identify and correct a Java-specific mistake.
- ☐ I can test a boundary case and navigate the workflow accessibly.

Lesson 3.2: Chained Conditionals

Reading References

- Java SE 25 documentation.
- Java Language Specification, Java SE 25.

Learning Objectives

- Explain the purpose and vocabulary of chained conditionals.
- Apply the relevant Java syntax and APIs in a small, complete program.
- Compile, run, inspect, and adapt the example using keyboard-accessible tools.

Concept Explanation

This lesson develops chained conditionals through Java's class-based structure and standard library. Identify the input, output, state changes, and relevant type constraints before coding. Keep declarations local where possible, separate computation from presentation, and state preconditions and postconditions. Boolean conditions evaluate to true or false; Java does not implicitly convert numbers or references to booleans. Loops need a clear initialization, continuation condition, and state update. Guard sentinel and input loops against end-of-file and malformed tokens.

Java is statically typed and class-based. Local variables have declared or inferred compile-time types, reference values may be null, and methods and fields are governed by access and scope rules. The compiler checks syntax and type compatibility before execution; runtime behavior still requires testing. Follow

Java naming conventions, use explicit imports in beginner work, and prefer small methods with clear contracts.

Worked Example

```
public class Lesson_3_2 {
    public static void main(String[] args) {
        int score = 77;
        String grade;
        if (score >= 90) grade = "A";
        else if (score >= 80) grade = "B";
        else if (score >= 70) grade = "C";
        else if (score >= 60) grade = "D";
        else grade = "F";
        System.out.println(grade);
    }
}
```

Expected behavior: The program compiles on JDK 25 and prints a deterministic result illustrating chained conditionals. Run it from a terminal, compare actual output with your prediction, and change one input to test the effect.

Common Mistakes

- Using == for String content comparison instead of equals or equalsIgnoreCase.
- Ordering chained score thresholds incorrectly so a higher grade is captured by a lower branch.
- Using an incorrect loop bound, modifying a collection during enhanced iteration, or confusing array length with collection size.

Activities

1. Test each decision threshold immediately below, at, and above the boundary.
2. Process the four shared scores and verify count, total, and average.
3. Normalize a mixed-case sentence containing leading/trailing whitespace and punctuation.
4. Explain the difference between comparing primitive values and comparing String content.

Accessibility Practice

Navigate by class, method, and line; inspect braces, parentheses, brackets, semicolons, generic delimiters, and compiler diagnostics with the preferred screen reader or braille display. Compile and run from the terminal so the workflow does not depend on color or a visual project explorer.

Skills Checklist

- ☐ I can explain the key concept in Java terms.
- ☐ I can trace and predict the worked example.
- ☐ I can compile and run the example from the terminal.
- ☐ I can identify and correct a Java-specific mistake.
- ☐ I can test a boundary case and navigate the workflow accessibly.

Lesson 3.3: Introduction to Lists

Reading References

- Java SE 25 documentation.
- Java Language Specification, Java SE 25.

Learning Objectives

- Explain the purpose and vocabulary of introduction to lists.
- Apply the relevant Java syntax and APIs in a small, complete program.
- Compile, run, inspect, and adapt the example using keyboard-accessible tools.

Concept Explanation

This lesson develops introduction to lists through Java's class-based structure and standard library. Identify the input, output, state changes, and relevant type constraints before coding. Keep declarations local where possible, separate computation from presentation, and state preconditions and postconditions. Boolean conditions evaluate to true or false; Java does not implicitly convert numbers or references to booleans. Loops need a clear initialization, continuation condition, and state update. Guard sentinel and input loops against end-of-file and malformed tokens.

Java is statically typed and class-based. Local variables have declared or inferred compile-time types, reference values may be null, and methods and fields are governed by access and scope rules. The compiler checks syntax and type compatibility before execution; runtime behavior still requires testing. Follow Java naming conventions, use explicit imports in beginner work, and prefer small methods with clear contracts.

Worked Example

```
public class Lesson_3_3 {
    public static void main(String[] args) {
        java.util.ArrayList<Integer> scores = new java.util.ArrayList<>(java.util.List.of(76
        scores.set(0, 80);
        scores.add(95);
        System.out.println(scores);
```

```
    }
}
```

Expected behavior: The program compiles on JDK 25 and prints a deterministic result illustrating introduction to lists. Run it from a terminal, compare actual output with your prediction, and change one input to test the effect.

Common Mistakes

- Using == for String content comparison instead of equals or equalsIgnoreCase.
- Ordering chained score thresholds incorrectly so a higher grade is captured by a lower branch.
- Using an incorrect loop bound, modifying a collection during enhanced iteration, or confusing array length with collection size.

Activities

1. Test each decision threshold immediately below, at, and above the boundary.
2. Process the four shared scores and verify count, total, and average.
3. Normalize a mixed-case sentence containing leading/trailing whitespace and punctuation.
4. Explain the difference between comparing primitive values and comparing String content.

Accessibility Practice

Navigate by class, method, and line; inspect braces, parentheses, brackets, semicolons, generic delimiters, and compiler diagnostics with the preferred screen reader or braille display. Compile and run from the terminal so the workflow does not depend on color or a visual project explorer.

Skills Checklist

☐ I can explain the key concept in Java terms.
☐ I can trace and predict the worked example.
☐ I can compile and run the example from the terminal.
☐ I can identify and correct a Java-specific mistake.
☐ I can test a boundary case and navigate the workflow accessibly.

Lesson 3.4: Iterating with for Loops

Reading References

- Java SE 25 documentation.
- Java Language Specification, Java SE 25.

Learning Objectives

- Explain the purpose and vocabulary of iterating with for loops.
- Apply the relevant Java syntax and APIs in a small, complete program.
- Compile, run, inspect, and adapt the example using keyboard-accessible tools.

Concept Explanation

This lesson develops iterating with for loops through Java's class-based structure and standard library. Identify the input, output, state changes, and relevant type constraints before coding. Keep declarations local where possible, separate computation from presentation, and state preconditions and postconditions. Boolean conditions evaluate to true or false; Java does not implicitly convert numbers or references to booleans. Loops need a clear initialization, continuation condition, and state update. Guard sentinel and input loops against end-of-file and malformed tokens.

Java is statically typed and class-based. Local variables have declared or inferred compile-time types, reference values may be null, and methods and fields are governed by access and scope rules. The compiler checks syntax and type compatibility before execution; runtime behavior still requires testing. Follow Java naming conventions, use explicit imports in beginner work, and prefer small methods with clear contracts.

Worked Example

```java
public class Lesson_3_4 {
    public static void main(String[] args) {
        int[] scores = {76, 88, 91, 84};
        int total = 0;
        for (int score : scores) total += score;
        double average = (double) total / scores.length;
        System.out.printf("total=%d average=%.2f%n", total, average);
    }
}
```

Expected Behavior or Output

The program prints:

```
Adding score 1: 82
Adding score 2: 91
Adding score 3: 76
Adding score 4: 88
Total: 337
Average: 84.25
Same total with a stream: 337
```

```
Above 85: 91
Above 85: 88
Countdown: 3
Countdown: 2
Countdown: 1
```

Common Mistakes

1. Writing `i <= scores.size()`, which reads one position past the end.
2. Dividing the `int` total by the size and getting a truncated result.
3. Removing from a list inside an enhanced `for`, which throws `ConcurrentModificationException`.

Activities

1. Predict the last six output lines before running.
2. Change the loop to print only the scores above 80.
3. Write a loop that counts from 20 down to 0 by 5.
4. Explain why the average prints `84.25` and not `84`.

Accessibility Practice

Each output line starts with a label and a number, so the running total is read step by step. Count the lines to confirm that the loop ran four times.

Skills Checklist

☐ I can write classic and enhanced `for` loops.
☐ I can write a `while` loop.
☐ I can total and average a list.
☐ I can avoid the off-by-one error.
☐ I can use a stream to sum.

Lesson 3.5: Strings

Reading References

- Java SE 25 API documentation
- Dev.java learning paths
- Matching Python lesson: Lesson 3.5: Basic String Operations

Learning Objectives

- Use `length`, `toLowerCase`, `contains`, `indexOf`, and `strip`.
- Compare strings with `equals`, not `==`.
- Normalize a name and search the shared text sample.

Concept Explanation

A `String` is an immutable sequence of characters. `length()` is its size and `charAt(0)` is the first character. Methods such as `toUpperCase()` return a **new** string and never change the original.

Useful methods: `strip()` (trim whitespace), `contains(text)`, `indexOf(text)` (which returns `-1` when the text is missing), `substring(start, end)`, `replace`, `startsWith`, and `split(regex)`.

Compare strings with `equals`, never `==`. `==` compares whether two variables refer to the *same object*, which can be false even when the text is identical. `equalsIgnoreCase` ignores case.

The shared text sample is "A student studies programming. Programming develops problem-solving skills." It has 75 characters and contains `programming` twice (once in each case), first found at index 18 after lowercasing.

Worked Example

```
import java.util.regex.Pattern;

public class Lesson_3_5 {
    public static void main(String[] args) {
        String rawName = "  avery  ";
        String stripped = rawName.strip();
        String cleanName = stripped.substring(0, 1).toUpperCase() + stripped.substring(1);
        System.out.println("Name: '" + cleanName + "'");

        String text = "A student studies programming. Programming develops problem-solving s
        String lowered = text.toLowerCase();
        System.out.println("Characters: " + text.length());
        System.out.println("Contains 'programming': " + lowered.contains("programming"));
        System.out.println("First index of 'programming': " + lowered.indexOf("programming")
        System.out.println("Times 'programming' appears: " + Pattern.compile("programming").
        System.out.println("Missing word index: " + lowered.indexOf("java"));

        String a = new String("Avery");
        String b = new String("Avery");
        System.out.println(a == b);
        System.out.println(a.equals(b));
        System.out.println(a.equalsIgnoreCase("AVERY"));
        System.out.println(text.toUpperCase().substring(0, 9));
    }
}
```

Expected behavior: The program compiles on JDK 25 and prints a deterministic result illustrating basic string operations. Run it from a terminal, compare

actual output with your prediction, and change one input to test the effect.

Common Mistakes

- Using == for String content comparison instead of equals or equalsIgnoreCase.
- Ordering chained score thresholds incorrectly so a higher grade is captured by a lower branch.
- Using an incorrect loop bound, modifying a collection during enhanced iteration, or confusing array length with collection size.

Activities

1. Test each decision threshold immediately below, at, and above the boundary.
2. Process the four shared scores and verify count, total, and average.
3. Normalize a mixed-case sentence containing leading/trailing whitespace and punctuation.
4. Explain the difference between comparing primitive values and comparing String content.

Accessibility Practice

Navigate by class, method, and line; inspect braces, parentheses, brackets, semicolons, generic delimiters, and compiler diagnostics with the preferred screen reader or braille display. Compile and run from the terminal so the workflow does not depend on color or a visual project explorer.

Skills Checklist

☐ I can explain the key concept in Java terms.
☐ I can trace and predict the worked example.
☐ I can compile and run the example from the terminal.
☐ I can identify and correct a Java-specific mistake.
☐ I can test a boundary case and navigate the workflow accessibly.

8.6 Unit 4: Data Structures — Lists and Dictionaries

Overview

This unit develops data structures — lists and dictionaries through the same shared problem contexts and assessment expectations used across the general-purpose language tracks, expressed with Java 25 syntax, APIs, and static typing.

Unit objectives: explain the core concepts, write and test complete Java programs, diagnose errors, and apply the ideas to accessible projects.

Lesson 4.1: More on Lists

Reading References

- Java SE 25 documentation.
- Java Language Specification, Java SE 25.

Learning Objectives

- Explain the purpose and vocabulary of more on lists.
- Apply the relevant Java syntax and APIs in a small, complete program.
- Compile, run, inspect, and adapt the example using keyboard-accessible tools.

Concept Explanation

This lesson develops more on lists through Java's class-based structure and standard library. Identify the input, output, state changes, and relevant type constraints before coding. Keep declarations local where possible, separate computation from presentation, and state preconditions and postconditions. Arrays have fixed length; List and ArrayList provide collection abstractions; Map associates keys with values. List.of creates an unmodifiable list. Use generics to retain element types and choose an implementation whose ordering and lookup behavior match the task.

Java is statically typed and class-based. Local variables have declared or inferred compile-time types, reference values may be null, and methods and fields are governed by access and scope rules. The compiler checks syntax and type compatibility before execution; runtime behavior still requires testing. Follow Java naming conventions, use explicit imports in beginner work, and prefer small methods with clear contracts.

Worked Example

```
public class Lesson_4_1 {
    public static void main(String[] args) {
        java.util.ArrayList<String> tasks = new java.util.ArrayList<>(java.util.List.of("rea
        tasks.add("review");
        tasks.remove("practice");
        System.out.println(tasks + "; count=" + tasks.size());
    }
}
```

Expected behavior: The program compiles on JDK 25 and prints a deterministic result illustrating more on lists. Run it from a terminal, compare actual output with your prediction, and change one input to test the effect.

Common Mistakes

- Assuming ArrayList and arrays have the same resizing behavior or index limits.
- Trying to modify a List created with List.of, which is unmodifiable.
- Assuming Map iteration order without choosing an ordered implementation, or treating a missing key as a stored zero.

Activities

1. Apply add, update, remove, lookup, and iteration operations to the same inventory data.
2. Compare a fixed-size array, mutable ArrayList, and unmodifiable List.of value.
3. Choose HashMap or LinkedHashMap for a stated ordering requirement and justify the choice.
4. Test a missing key and distinguish null from a legitimate stored value.

Accessibility Practice

Navigate by class, method, and line; inspect braces, parentheses, brackets, semicolons, generic delimiters, and compiler diagnostics with the preferred screen reader or braille display. Compile and run from the terminal so the workflow does not depend on color or a visual project explorer.

Skills Checklist

☐ I can explain the key concept in Java terms.
☐ I can trace and predict the worked example.
☐ I can compile and run the example from the terminal.
☐ I can identify and correct a Java-specific mistake.
☐ I can test a boundary case and navigate the workflow accessibly.

Lesson 4.2: Organizing Lists

Reading References

- Java SE 25 documentation.
- Java Language Specification, Java SE 25.

Learning Objectives

- Explain the purpose and vocabulary of organizing lists.
- Apply the relevant Java syntax and APIs in a small, complete program.
- Compile, run, inspect, and adapt the example using keyboard-accessible tools.

Concept Explanation

This lesson develops organizing lists through Java's class-based structure and standard library. Identify the input, output, state changes, and relevant type constraints before coding. Keep declarations local where possible, separate computation from presentation, and state preconditions and postconditions. Arrays have fixed length; List and ArrayList provide collection abstractions; Map associates keys with values. List.of creates an unmodifiable list. Use generics to retain element types and choose an implementation whose ordering and lookup behavior match the task.

Java is statically typed and class-based. Local variables have declared or inferred compile-time types, reference values may be null, and methods and fields are governed by access and scope rules. The compiler checks syntax and type compatibility before execution; runtime behavior still requires testing. Follow Java naming conventions, use explicit imports in beginner work, and prefer small methods with clear contracts.

Worked Example

```
public class Lesson_4_2 {
    public static void main(String[] args) {
        java.util.ArrayList<Integer> scores = new java.util.ArrayList<>(java.util.List.of(91
        java.util.Collections.sort(scores);
        System.out.println("ascending=" + scores);
        scores.sort(java.util.Comparator.reverseOrder());
        System.out.println("descending=" + scores);
    }
}
```

Expected behavior: The program compiles on JDK 25 and prints a deterministic result illustrating organizing lists. Run it from a terminal, compare actual output with your prediction, and change one input to test the effect.

Common Mistakes

- Assuming ArrayList and arrays have the same resizing behavior or index limits.
- Trying to modify a List created with List.of, which is unmodifiable.
- Assuming Map iteration order without choosing an ordered implementation, or treating a missing key as a stored zero.

Activities

1. Apply add, update, remove, lookup, and iteration operations to the same inventory data.
2. Compare a fixed-size array, mutable ArrayList, and unmodifiable List.of value.

3. Choose HashMap or LinkedHashMap for a stated ordering requirement and justify the choice.
4. Test a missing key and distinguish null from a legitimate stored value.

Accessibility Practice

Navigate by class, method, and line; inspect braces, parentheses, brackets, semicolons, generic delimiters, and compiler diagnostics with the preferred screen reader or braille display. Compile and run from the terminal so the workflow does not depend on color or a visual project explorer.

Skills Checklist

☐ I can explain the key concept in Java terms.
☐ I can trace and predict the worked example.
☐ I can compile and run the example from the terminal.
☐ I can identify and correct a Java-specific mistake.
☐ I can test a boundary case and navigate the workflow accessibly.

Lesson 4.3: Tuples and Immutable Lists

Reading References

- Java SE 25 documentation.
- Java Language Specification, Java SE 25.

Learning Objectives

- Explain the purpose and vocabulary of tuples and immutable lists.
- Apply the relevant Java syntax and APIs in a small, complete program.
- Compile, run, inspect, and adapt the example using keyboard-accessible tools.

Concept Explanation

This lesson develops tuples and immutable lists through Java's class-based structure and standard library. Identify the input, output, state changes, and relevant type constraints before coding. Keep declarations local where possible, separate computation from presentation, and state preconditions and postconditions. Arrays have fixed length; List and ArrayList provide collection abstractions; Map associates keys with values. List.of creates an unmodifiable list. Use generics to retain element types and choose an implementation whose ordering and lookup behavior match the task.

Java is statically typed and class-based. Local variables have declared or inferred compile-time types, reference values may be null, and methods and fields are governed by access and scope rules. The compiler checks syntax and type compatibility before execution; runtime behavior still requires testing. Follow

Java naming conventions, use explicit imports in beginner work, and prefer small methods with clear contracts.

Worked Example

```
public class Lesson_4_3 {
    public static void main(String[] args) {
        java.util.List<String> days = java.util.List.of("Monday", "Tuesday", "Wednesday");
        System.out.println(days.get(0));
        try { days.add("Thursday"); }
        catch (UnsupportedOperationException ex) { System.out.println("List is unmodifiable'
    }
}
```

Expected behavior: The program compiles on JDK 25 and prints a deterministic result illustrating tuples and immutable lists. Run it from a terminal, compare actual output with your prediction, and change one input to test the effect.

Common Mistakes

- Assuming ArrayList and arrays have the same resizing behavior or index limits.
- Trying to modify a List created with List.of, which is unmodifiable.
- Assuming Map iteration order without choosing an ordered implementation, or treating a missing key as a stored zero.

Activities

1. Apply add, update, remove, lookup, and iteration operations to the same inventory data.
2. Compare a fixed-size array, mutable ArrayList, and unmodifiable List.of value.
3. Choose HashMap or LinkedHashMap for a stated ordering requirement and justify the choice.
4. Test a missing key and distinguish null from a legitimate stored value.

Accessibility Practice

Navigate by class, method, and line; inspect braces, parentheses, brackets, semicolons, generic delimiters, and compiler diagnostics with the preferred screen reader or braille display. Compile and run from the terminal so the workflow does not depend on color or a visual project explorer.

Skills Checklist

☐ I can explain the key concept in Java terms.
☐ I can trace and predict the worked example.

☐ I can compile and run the example from the terminal.
☐ I can identify and correct a Java-specific mistake.
☐ I can test a boundary case and navigate the workflow accessibly.

Lesson 4.4: Introducing Dictionaries with Map

Reading References

- Java SE 25 documentation.
- Java Language Specification, Java SE 25.

Learning Objectives

- Explain the purpose and vocabulary of introducing dictionaries with map.
- Apply the relevant Java syntax and APIs in a small, complete program.
- Compile, run, inspect, and adapt the example using keyboard-accessible tools.

Concept Explanation

This lesson develops introducing dictionaries with map through Java's class-based structure and standard library. Identify the input, output, state changes, and relevant type constraints before coding. Keep declarations local where possible, separate computation from presentation, and state preconditions and postconditions. Arrays have fixed length; List and ArrayList provide collection abstractions; Map associates keys with values. List.of creates an unmodifiable list. Use generics to retain element types and choose an implementation whose ordering and lookup behavior match the task.

Java is statically typed and class-based. Local variables have declared or inferred compile-time types, reference values may be null, and methods and fields are governed by access and scope rules. The compiler checks syntax and type compatibility before execution; runtime behavior still requires testing. Follow Java naming conventions, use explicit imports in beginner work, and prefer small methods with clear contracts.

Worked Example

```
public class Lesson_4_4 {
    public static void main(String[] args) {
        java.util.Map<String, Integer> inventory = new java.util.LinkedHashMap<>();
        inventory.put("notebook", 12);
        inventory.put("pen", 30);
        System.out.println("pens=" + inventory.get("pen"));
        System.out.println("missing=" + inventory.getOrDefault("eraser", 0));
    }
}
```

Expected Behavior or Output

The program prints:

```
Avery
{name=Avery, age=17, favoriteSubject=Computer Science, gradeLevel=11}
null
unknown
true
Item[name=Notebook, quantity=3, unitPrice=2.5]
Item[name=Notebook, quantity=5, unitPrice=2.5]
Stock value: 12.5
```

Common Mistakes

1. Using a missing key and expecting an error. `get` returns `null`.
2. Calling a method on the `null` returned by `get`, which throws `NullPointerException`.
3. Using a `HashMap` and expecting the same print order every run.

Activities

1. Predict the printed lines before running.
2. Add a `discount` component to a new record and print the discounted value.
3. Remove `favoriteSubject` and show what `get` returns.
4. Explain why a record is safer than `Map<String, Object>` for a known set of fields.

Accessibility Practice

Print one entry per line when you need to read a map, because a one-line `{a=1, b=2}` is hard to navigate. Choose key names that read well aloud, such as `unitPrice`.

Skills Checklist

- ☐ I can create and change a map.
- ☐ I know `get` can return `null`.
- ☐ I can supply a default with `getOrDefault`.
- ☐ I can choose a map class for the order I need.
- ☐ I can model an item with a record.

Lesson 4.5: Traversing Maps

Reading References

- Java SE 25 API documentation
- Dev.java learning paths
- Matching Python lesson: Lesson 4.5: Looping Through Dictionaries

Learning Objectives

- Visit keys, values, and entries.
- Explain which map classes keep a stable order.
- Sort keys and total the shared inventory.

Concept Explanation

`map.keySet()` gives the keys, `map.values()` gives the values, and `map.entrySet()` gives key-value pairs: `for (Map.Entry<String, Stock> entry : inventory.entrySet())` with `entry.getKey()` and `entry.getValue()`.

For readable output the order must be stable. `LinkedHashMap` keeps insertion order; `new TreeMap<>(map)` gives alphabetical order; `HashMap` gives neither. Do not add or remove entries while looping over a map.

A stream can total the values: `values().stream().mapToDouble(...).sum()`. The shared inventory is Notebook (3 at 2.50), Pencil (10 at 0.50), and Folder (4 at 1.25), worth $7.50 + 5.00 + 5.00 = 17.50$.

Worked Example

```
import java.util.LinkedHashMap;
import java.util.Locale;
import java.util.Map;
import java.util.TreeMap;

public class Lesson_4_5 {
    record Stock(int quantity, double unitPrice) {}

    static String money(double value) {
        return String.format(Locale.US, "%.2f", value);
    }

    public static void main(String[] args) {
        Map<String, Stock> inventory = new LinkedHashMap<>();
        inventory.put("Notebook", new Stock(3, 2.50));
        inventory.put("Pencil", new Stock(10, 0.50));
        inventory.put("Folder", new Stock(4, 1.25));
```

```
        System.out.println("Keys:");
        for (String name : inventory.keySet()) {
            System.out.println(name);
        }

        System.out.println("Quantities:");
        for (Stock stock : inventory.values()) {
            System.out.println(stock.quantity());
        }

        double totalValue = 0;
        for (Map.Entry<String, Stock> entry : inventory.entrySet()) {
            Stock stock = entry.getValue();
            double lineValue = stock.quantity() * stock.unitPrice();
            System.out.println(entry.getKey() + ": " + stock.quantity() + " in stock, value
            totalValue += lineValue;
        }
        System.out.println("Total inventory value: " + money(totalValue));

        System.out.println(new TreeMap<>(inventory).keySet());
        System.out.println(inventory.values().stream().mapToDouble(s -> s.quantity() * s.uni
    }
}
```

Expected behavior: The program compiles on JDK 25 and prints a deterministic result illustrating looping through dictionaries. Run it from a terminal, compare actual output with your prediction, and change one input to test the effect.

Common Mistakes

- Assuming ArrayList and arrays have the same resizing behavior or index limits.
- Trying to modify a List created with List.of, which is unmodifiable.
- Assuming Map iteration order without choosing an ordered implementation, or treating a missing key as a stored zero.

Activities

1. Apply add, update, remove, lookup, and iteration operations to the same inventory data.
2. Compare a fixed-size array, mutable ArrayList, and unmodifiable List.of value.
3. Choose HashMap or LinkedHashMap for a stated ordering requirement and justify the choice.
4. Test a missing key and distinguish null from a legitimate stored value.

Accessibility Practice

Navigate by class, method, and line; inspect braces, parentheses, brackets, semicolons, generic delimiters, and compiler diagnostics with the preferred screen reader or braille display. Compile and run from the terminal so the workflow does not depend on color or a visual project explorer.

Skills Checklist

- ☐ I can explain the key concept in Java terms.
- ☐ I can trace and predict the worked example.
- ☐ I can compile and run the example from the terminal.
- ☐ I can identify and correct a Java-specific mistake.
- ☐ I can test a boundary case and navigate the workflow accessibly.

8.7 Unit 5: User Input and while Loops

Overview

This unit develops user input and while loops through the same shared problem contexts and assessment expectations used across the general-purpose language tracks, expressed with Java 25 syntax, APIs, and static typing.

Unit objectives: explain the core concepts, write and test complete Java programs, diagnose errors, and apply the ideas to accessible projects.

Lesson 5.1: How Console Input Works

Reading References

- Java SE 25 API documentation
- Dev.java learning paths
- Matching Python lesson: Lesson 5.1: How the input() Function Works

Learning Objectives

- Read a line and recognize it as text.
- Convert it safely and handle failure.
- Handle the end of input.

Concept Explanation

`Scanner.nextLine()` returns the next line as a `String`. A program must check `hasNextLine()` first, because at the end of input `nextLine()` throws. It never returns a number: typed digits are text until you convert them. `Integer.parseInt` converts text and throws `NumberFormatException` when the text is not a whole number.

A program that reads input must plan for three cases: a valid value, an invalid value, and no value at all. The example uses a small `next` helper that returns `null` at the end of input, and a `toInt` helper that returns `null` for bad text. With the typed value 16 the age five years later is 21; the second value, `abc`, is rejected.

Worked Example

```
import java.util.Scanner;

public class Lesson_5_1 {
    static String next(Scanner in) {
        return in.hasNextLine() ? in.nextLine() : null;
    }

    static Integer toInt(String text) {
        if (text == null) {
            return null;
        }
        try {
            return Integer.parseInt(text.trim());
        } catch (NumberFormatException e) {
            return null;
        }
    }

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.println("How old are you?");
        String text = next(in);
        System.out.println("Read: " + (text == null ? "nothing" : text.getClass().getSimpleN

        Integer age = toInt(text);
        if (age != null) {
            System.out.println("In 5 years: " + (age + 5));
        } else {
            System.out.println("That was not a whole number.");
        }

        System.out.println("Enter another value:");
        Integer second = toInt(next(in));
        if (second != null) {
            System.out.println("Number: " + second);
        } else {
            System.out.println("Not a number; toInt returned null.");
        }
```

```
    }
}
```

Expected behavior: The program compiles on JDK 25 and prints a deterministic result illustrating how scanner input works. Run it from a terminal, compare actual output with your prediction, and change one input to test the effect.

Common Mistakes

- Mixing nextInt/next with nextLine without consuming the pending line terminator.
- Writing a sentinel loop that fails to stop at the sentinel or does not handle end-of-input.
- Using continue or break in a way that skips required state updates and creates a nonterminating loop.

Activities

1. Feed several lines of input, including an empty line and the sentinel, to the Scanner workflow.
2. Trace the loop condition and state update for every iteration, including end-of-input.
3. Add an attempt limit and a clear exit command to prevent an infinite prompt loop.
4. Test that continue skips only the intended processing step and does not skip required updates.

Accessibility Practice

Navigate by class, method, and line; inspect braces, parentheses, brackets, semicolons, generic delimiters, and compiler diagnostics with the preferred screen reader or braille display. Compile and run from the terminal so the workflow does not depend on color or a visual project explorer.

Skills Checklist

☐ I can explain the key concept in Java terms.
☐ I can trace and predict the worked example.
☐ I can compile and run the example from the terminal.
☐ I can identify and correct a Java-specific mistake.
☐ I can test a boundary case and navigate the workflow accessibly.

Lesson 5.2: The while Loop in Action

Reading References

- Java SE 25 documentation.

- Java Language Specification, Java SE 25.

Learning Objectives

- Explain the purpose and vocabulary of the while loop in action.
- Apply the relevant Java syntax and APIs in a small, complete program.
- Compile, run, inspect, and adapt the example using keyboard-accessible tools.

Concept Explanation

This lesson develops the while loop in action through Java's class-based structure and standard library. Identify the input, output, state changes, and relevant type constraints before coding. Keep declarations local where possible, separate computation from presentation, and state preconditions and postconditions. Boolean conditions evaluate to true or false; Java does not implicitly convert numbers or references to booleans. Loops need a clear initialization, continuation condition, and state update. Guard sentinel and input loops against end-of-file and malformed tokens.

Java is statically typed and class-based. Local variables have declared or inferred compile-time types, reference values may be null, and methods and fields are governed by access and scope rules. The compiler checks syntax and type compatibility before execution; runtime behavior still requires testing. Follow Java naming conventions, use explicit imports in beginner work, and prefer small methods with clear contracts.

Worked Example

```
public class Lesson_5_2 {
    public static void main(String[] args) {
        java.util.Scanner input = new java.util.Scanner("3 2 1 0");
        int count = 0;
        while (input.hasNextInt()) {
            int value = input.nextInt();
            if (value == 0) break;
            count += value;
        }
        System.out.println("sum before sentinel=" + count);
        input.close();
    }
}
```

Expected Behavior or Output

With the following lines typed as standard input (piped input is not echoed, so it does not appear in the output):

```
hello
scores
quit
```

the program prints:

```
1
2
3
4
5
Counting finished!
Enter a message, or type quit to exit:
You said: hello
Enter a message, or type quit to exit:
You said: scores
Enter a message, or type quit to exit:
Goodbye!
```

With no standard input at all, the program ends cleanly and prints:

```
1
2
3
4
5
Counting finished!
Enter a message, or type quit to exit:
Goodbye!
```

Common Mistakes

1. Forgetting to change the loop variable, which creates an endless loop.
2. Testing for `quit` but not for `null`, so the loop reads forever after input ends.
3. Comparing with `message == "quit"` instead of `equals`.

Activities

1. Predict the output for the typed lines hello, scores, and quit.
2. Change the sentinel to `exit` and update the prompt.
3. Count the messages entered and print the count at the end.
4. Rewrite the counting loop as a `for` loop and compare.

Accessibility Practice

Each prompt is on its own line and each response begins with `You said:`, so the conversation can be followed in order by ear. If a loop floods the screen reader, stop it with Ctrl+C.

Skills Checklist

- ☐ I can write a counting `while` loop.
- ☐ I can write a sentinel loop.
- ☐ I test for `null` as well as the sentinel.
- ☐ I update the loop variable.
- ☐ I compare strings with `equals`.

Lesson 5.3: Flags, break, and Labels

Reading References

- Java SE 25 API documentation
- Java Language Specification, Java SE 25
- Dev.java learning paths
- Matching Python lesson: Lesson 5.3: Using Flags and break to Exit Loops

Learning Objectives

- Control a loop with a Boolean flag.
- Leave a loop with `break` and an outer loop with a label.
- Search a list and remember the position.

Concept Explanation

A *flag* is a `boolean` variable that controls a loop: the loop runs while `running` is true, and the body sets it to `false` to stop. `break` leaves the nearest enclosing loop at once.

A search keeps its answer in a variable (here `-1` means "not found"), sets it when the item is found, and uses `break`. After the loop, the variable says whether a match was found. List positions start at 0.

A *label* lets `break` leave an outer loop from inside an inner one: write `outer:` before the outer loop and `break outer;` inside.

The shared scores 82, 91, 76, 88 have their first score of 90 or higher at index 1, with the value 91.

Worked Example

```
import java.util.List;
import java.util.Scanner;

public class Lesson_5_3 {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        boolean running = true;
```

```
        int handled = 0;
        while (running) {
            System.out.println("Say something, or type quit to stop:");
            String message = in.hasNextLine() ? in.nextLine() : null;
            if (message == null || message.equals("quit")) {
                running = false;
            } else {
                handled++;
                System.out.println("You said: " + message);
            }
        }
        System.out.println("Messages handled: " + handled);

        List<Integer> scores = List.of(82, 91, 76, 88);
        int foundIndex = -1;
        for (int i = 0; i < scores.size(); i++) {
            if (scores.get(i) >= 90) {
                foundIndex = i;
                break;
            }
        }
        if (foundIndex >= 0) {
            System.out.println("First score of 90 or higher: " + scores.get(foundIndex) + "
        } else {
            System.out.println("No score of 90 or higher.");
        }

        outer:
        for (int row = 1; row <= 3; row++) {
            for (int col = 1; col <= 3; col++) {
                if (row * col == 4) {
                    System.out.println("Stopped at row " + row + ", column " + col);
                    break outer;
                }
            }
        }
    }
}
```

Expected behavior: The program compiles on JDK 25 and prints a deterministic result illustrating using flags and break to exit loops. Run it from a terminal, compare actual output with your prediction, and change one input to test the effect.

Common Mistakes

- Mixing nextInt/next with nextLine without consuming the pending line terminator.
- Writing a sentinel loop that fails to stop at the sentinel or does not handle end-of-input.
- Using continue or break in a way that skips required state updates and creates a nonterminating loop.

Activities

1. Feed several lines of input, including an empty line and the sentinel, to the Scanner workflow.
2. Trace the loop condition and state update for every iteration, including end-of-input.
3. Add an attempt limit and a clear exit command to prevent an infinite prompt loop.
4. Test that continue skips only the intended processing step and does not skip required updates.

Accessibility Practice

Navigate by class, method, and line; inspect braces, parentheses, brackets, semicolons, generic delimiters, and compiler diagnostics with the preferred screen reader or braille display. Compile and run from the terminal so the workflow does not depend on color or a visual project explorer.

Skills Checklist

☐ I can explain the key concept in Java terms.
☐ I can trace and predict the worked example.
☐ I can compile and run the example from the terminal.
☐ I can identify and correct a Java-specific mistake.
☐ I can test a boundary case and navigate the workflow accessibly.

Lesson 5.4: Using continue in Loops

Reading References

- Java SE 25 documentation.
- Java Language Specification, Java SE 25.

Learning Objectives

- Explain the purpose and vocabulary of using continue in loops.
- Apply the relevant Java syntax and APIs in a small, complete program.
- Compile, run, inspect, and adapt the example using keyboard-accessible tools.

Concept Explanation

This lesson develops using continue in loops through Java's class-based structure and standard library. Identify the input, output, state changes, and relevant type constraints before coding. Keep declarations local where possible, separate computation from presentation, and state preconditions and postconditions. Boolean conditions evaluate to true or false; Java does not implicitly convert numbers or references to booleans. Loops need a clear initialization, continuation condition, and state update. Guard sentinel and input loops against end-of-file and malformed tokens.

Java is statically typed and class-based. Local variables have declared or inferred compile-time types, reference values may be null, and methods and fields are governed by access and scope rules. The compiler checks syntax and type compatibility before execution; runtime behavior still requires testing. Follow Java naming conventions, use explicit imports in beginner work, and prefer small methods with clear contracts.

Worked Example

```
public class Lesson_5_4 {
    public static void main(String[] args) {
        for (int value = 1; value <= 8; value++) {
            if (value % 2 == 0) continue;
            System.out.println("odd=" + value);
        }
    }
}
```

Expected behavior: The program compiles on JDK 25 and prints a deterministic result illustrating using continue in loops. Run it from a terminal, compare actual output with your prediction, and change one input to test the effect.

Common Mistakes

- Mixing nextInt/next with nextLine without consuming the pending line terminator.
- Writing a sentinel loop that fails to stop at the sentinel or does not handle end-of-input.
- Using continue or break in a way that skips required state updates and creates a nonterminating loop.

Activities

1. Feed several lines of input, including an empty line and the sentinel, to the Scanner workflow.
2. Trace the loop condition and state update for every iteration, including end-of-input.

3. Add an attempt limit and a clear exit command to prevent an infinite prompt loop.
4. Test that continue skips only the intended processing step and does not skip required updates.

Accessibility Practice

Navigate by class, method, and line; inspect braces, parentheses, brackets, semicolons, generic delimiters, and compiler diagnostics with the preferred screen reader or braille display. Compile and run from the terminal so the workflow does not depend on color or a visual project explorer.

Skills Checklist

☐ I can explain the key concept in Java terms.
☐ I can trace and predict the worked example.
☐ I can compile and run the example from the terminal.
☐ I can identify and correct a Java-specific mistake.
☐ I can test a boundary case and navigate the workflow accessibly.

Lesson 5.5: Avoiding Infinite Loops

Reading References

- Java SE 25 API documentation
- Java Language Specification, Java SE 25
- Dev.java learning paths
- Matching Python lesson: Lesson 5.5: Avoiding Infinite Loops

Learning Objectives

- Identify what makes a loop stop.
- Add an attempt limit and handle the end of input.
- Use `do ... while`.

Concept Explanation

An *infinite loop* never ends because its condition never becomes false. Common causes are forgetting to change the loop variable, testing the wrong variable, and waiting for input that never arrives. In a terminal, stop a running program with Ctrl+C.

Protect every loop with a *progress rule* and a *limit*: count attempts, and stop when the limit is reached or the input ends.

`do { ... } while (condition);` runs the body first and tests afterward, so it always runs at least once; note the semicolon after the condition.

The example allows three attempts to enter a score from 0 to 100. With the typed lines abc, 105, and 88, the first two are rejected and the third is accepted.

Worked Example

```
import java.util.Scanner;

public class Lesson_5_5 {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        int maxAttempts = 3;
        int attempts = 0;
        Integer score = null;

        while (attempts < maxAttempts && score == null) {
            attempts++;
            System.out.println("Enter a score from 0 to 100 (attempt " + attempts + " of " +
            if (!in.hasNextLine()) {
                System.out.println("No more input.");
                break;
            }
            String text = in.nextLine();
            try {
                int number = Integer.parseInt(text.trim());
                if (number >= 0 && number <= 100) {
                    score = number;
                } else {
                    System.out.println("Invalid score: " + text);
                }
            } catch (NumberFormatException e) {
                System.out.println("Invalid score: " + text);
            }
        }

        if (score != null) {
            System.out.println("Accepted score: " + score);
        } else {
            System.out.println("No valid score was entered.");
        }

        int count = 1;
        do {
            System.out.println(count);
            count++;
        } while (count <= 3);
    }
```

```
}
```

Expected behavior: The program compiles on JDK 25 and prints a deterministic result illustrating avoiding infinite loops. Run it from a terminal, compare actual output with your prediction, and change one input to test the effect.

Common Mistakes

- Mixing nextInt/next with nextLine without consuming the pending line terminator.
- Writing a sentinel loop that fails to stop at the sentinel or does not handle end-of-input.
- Using continue or break in a way that skips required state updates and creates a nonterminating loop.

Activities

1. Feed several lines of input, including an empty line and the sentinel, to the Scanner workflow.
2. Trace the loop condition and state update for every iteration, including end-of-input.
3. Add an attempt limit and a clear exit command to prevent an infinite prompt loop.
4. Test that continue skips only the intended processing step and does not skip required updates.

Accessibility Practice

Navigate by class, method, and line; inspect braces, parentheses, brackets, semicolons, generic delimiters, and compiler diagnostics with the preferred screen reader or braille display. Compile and run from the terminal so the workflow does not depend on color or a visual project explorer.

Skills Checklist

☐ I can explain the key concept in Java terms.
☐ I can trace and predict the worked example.
☐ I can compile and run the example from the terminal.
☐ I can identify and correct a Java-specific mistake.
☐ I can test a boundary case and navigate the workflow accessibly.

8.8 Unit 6: Functions and Methods

Overview

This unit develops functions and methods through the same shared problem contexts and assessment expectations used across the general-purpose language

tracks, expressed with Java 25 syntax, APIs, and static typing.

Unit objectives: explain the core concepts, write and test complete Java programs, diagnose errors, and apply the ideas to accessible projects.

Lesson 6.1: Defining Methods

Reading References

- Java SE 25 API documentation
- Java Language Specification, Java SE 25
- Dev.java learning paths
- Matching Python lesson: Lesson 6.1: Defining Functions

Learning Objectives

- Define and call static methods.
- Pass parameters and return nothing or a value.
- Use a method reference.

Concept Explanation

In Java a function is a **method** inside a class. A *static* method belongs to the class and is called without an object. A method declaration states its return type, its name, and its typed parameters: `static void showTip(double bill, double tipRate)`. `void` means nothing is returned.

The *parameters* are the names in the definition and the *arguments* are the values you pass. Arguments are matched to parameters by position, and Java passes every argument **by value**: the method receives a copy of a number, or a copy of the reference to an object.

A *method reference* such as `Lesson_6_1::showTip` names a method so that it can be passed around and called later through a functional interface like `BiConsumer`.

Worked Example

```
import java.util.Locale;
import java.util.function.BiConsumer;

public class Lesson_6_1 {
    static String money(double value) {
        return String.format(Locale.US, "%.2f", value);
    }

    static void greetUser() {
        System.out.println("Hello, Java programmer!");
    }
```

```
    static void greetName(String name) {
        System.out.println("Hello, " + name + "!");
    }

    static void showTip(double bill, double tipRate) {
        double tip = bill * tipRate;
        System.out.println("Tip: " + money(tip));
        System.out.println("Total: " + money(bill + tip));
    }

    public static void main(String[] args) {
        greetUser();
        greetUser();
        greetName("Avery");
        greetName("Riley");
        showTip(24.5, 0.15);

        BiConsumer<Double, Double> reference = Lesson_6_1::showTip;
        reference.accept(100.0, 0.2);
    }
}
```

Expected Behavior or Output

The program prints:

```
Hello, Java programmer!
Hello, Java programmer!
Hello, Avery!
Hello, Riley!
Tip: 3.68
Total: 28.18
Tip: 20.00
Total: 120.00
```

Common Mistakes

1. Writing `greetUser;` without parentheses, which is not a call.
2. Passing arguments in the wrong order, such as `showTip(0.15, 24.5)`.
3. Forgetting `static` and then calling the method from `main`, which is a compile error.

Activities

1. Call `showTip(100.0, 0.2)` yourself and predict both lines.

2. Write `describeStudent(String name, int age)` that prints one sentence.
3. Call `greetName()` with no argument and read the compiler message.
4. Explain the difference between a parameter and an argument.

Accessibility Practice

Choose method names that read as actions when spoken, such as `showTip`. Print complete sentences from a method so the screen reader announces each call's result separately.

Skills Checklist

- ☐ I can define and call a static method.
- ☐ I can distinguish parameters from arguments.
- ☐ I know Java passes arguments by value.
- ☐ I can write a method reference.
- ☐ I can read a method signature aloud.

Lesson 6.2: Overloading Instead of Default Arguments

Reading References

- Java SE 25 API documentation
- Java Language Specification, Java SE 25
- Dev.java learning paths
- Matching Python lesson: Lesson 6.2: Passing Arguments

Learning Objectives

- Explain that Java has no default or named arguments.
- Use overloading to supply a default.
- Group related values in a record for readable calls.

Concept Explanation

Java has no default parameter values and no named arguments. Two techniques give the same result. **Overloading** declares several methods with the same name and different parameter lists: `calculateTotal(double bill)` calls `calculateTotal(bill, 0.15)`. Java chooses the version by the number and types of the arguments.

When a call needs several values, a **record** groups them under names, so the call site is clear: `new TipRequest(24.5, 0.10)` makes the two numbers visible as a bill and a rate. A record can also supply a second constructor with a default.

With the shared bill of 24.50, the total at the default rate 0.15 is 28.18, at 0.20 it is 29.40, and at 0.10 it is 26.95.

Worked Example

```
import java.util.Locale;

public class Lesson_6_2 {
    static final double DEFAULT_RATE = 0.15;

    static String money(double value) {
        return String.format(Locale.US, "%.2f", value);
    }

    static double calculateTotal(double bill, double tipRate) {
        return bill + bill * tipRate;
    }

    static double calculateTotal(double bill) {
        return calculateTotal(bill, DEFAULT_RATE);
    }

    record TipRequest(double bill, double tipRate) {
        TipRequest(double bill) {
            this(bill, DEFAULT_RATE);
        }

        double total() {
            return bill + bill * tipRate;
        }
    }

    public static void main(String[] args) {
        System.out.println(money(calculateTotal(24.5)));
        System.out.println(money(calculateTotal(24.5, 0.20)));
        System.out.println(money(new TipRequest(24.5, 0.10).total()));
        System.out.println(money(new TipRequest(24.5).total()));
    }
}
```

Expected behavior: The program compiles on JDK 25 and prints a deterministic result illustrating passing arguments. Run it from a terminal, compare actual output with your prediction, and change one input to test the effect.

Common Mistakes

- Confusing a method's parameter list with its return type or omitting a return on one control-flow path.
- Expecting Java to pass primitive variables by reference; arguments are passed by value.
- Placing a varargs parameter before another parameter or using an overload that is ambiguous.

Activities

1. Extract repeated statements into a named method with an explicit parameter and return contract.
2. Call the method with typical, boundary, and empty collection arguments.
3. Add a method that accepts a List or Map and returns a computed result without modifying the input.
4. Overload a method or add a varargs form, then explain overload resolution.

Accessibility Practice

Navigate by class, method, and line; inspect braces, parentheses, brackets, semicolons, generic delimiters, and compiler diagnostics with the preferred screen reader or braille display. Compile and run from the terminal so the workflow does not depend on color or a visual project explorer.

Skills Checklist

☐ I can explain the key concept in Java terms.
☐ I can trace and predict the worked example.
☐ I can compile and run the example from the terminal.
☐ I can identify and correct a Java-specific mistake.
☐ I can test a boundary case and navigate the workflow accessibly.

Lesson 6.3: Return Values

Reading References

- Java SE 25 API documentation
- Dev.java learning paths
- Matching Python lesson: Lesson 6.3: Return Values

Learning Objectives

- Return a computed value.
- Return `OptionalDouble` when a value may be missing.
- Return several values with a record.

Concept Explanation

A method sends a value back with `return`, and its declared return type must match. When a result may not exist, such as the average of an empty list, returning `null` invites a crash. `OptionalDouble` states in the type that a value may be absent: `OptionalDouble.empty()` for no result, `OptionalDouble.of(x)` for a result, and `isPresent()`, `getAsDouble()`, or `orElse(fallback)` to read it.

To return several values, return a record and read its components. The example returns the average of the shared scores, classifies the average, returns a minimum and a maximum together, and shows the empty-list case.

Worked Example

```
import java.util.List;
import java.util.OptionalDouble;

public class Lesson_6_3 {
    record ScoreRange(int low, int high) {}

    static OptionalDouble average(List<Integer> scores) {
        if (scores.isEmpty()) {
            return OptionalDouble.empty();
        }
        int total = 0;
        for (int score : scores) {
            total += score;
        }
        return OptionalDouble.of((double) total / scores.size());
    }

    static String classify(double score) {
        if (score >= 90) return "A";
        if (score >= 80) return "B";
        if (score >= 70) return "C";
        if (score >= 60) return "D";
        return "F";
    }

    static ScoreRange minMax(List<Integer> scores) {
        int low = scores.get(0);
        int high = scores.get(0);
        for (int score : scores) {
            low = Math.min(low, score);
            high = Math.max(high, score);
        }
        return new ScoreRange(low, high);
```

```
    }

    public static void main(String[] args) {
        List<Integer> scores = List.of(82, 91, 76, 88);
        OptionalDouble avg = average(scores);
        System.out.println("Average: " + avg);
        if (avg.isPresent()) {
            System.out.println("Grade for the average: " + classify(avg.getAsDouble()));
        }
        ScoreRange range = minMax(scores);
        System.out.println("Lowest and highest: " + range.low() + " and " + range.high());
        System.out.println(average(List.of()));
        System.out.println(average(List.of()).orElse(0.0));
    }
}
```

Expected Behavior or Output

The program prints:

```
Average: OptionalDouble[84.25]
Grade for the average: B
Lowest and highest: 76 and 91
OptionalDouble.empty
0.0
```

Common Mistakes

1. Printing inside the method and then trying to use the result in a calculation.
2. Calling `getAsDouble()` on an empty `OptionalDouble`, which throws `NoSuchElementException`.
3. Writing code after a `return` in the same block, which the compiler rejects as unreachable.

Activities

1. Predict the six printed lines before running.
2. Write `total(List<Integer> list)` that returns the sum.
3. Write `isPassing(int score)` that returns a `boolean`.
4. Explain how a caller should use a method that returns `OptionalDouble`.

Accessibility Practice

Return values instead of printing inside logic methods, so the caller prints once and the screen reader announces each result a single time.

Skills Checklist

- ☐ I can return a value.
- ☐ I can return `OptionalDouble` for a possibly missing value.
- ☐ I can return several values with a record.
- ☐ I can use a returned value in a calculation.
- ☐ I can classify an average with the shared thresholds.

Lesson 6.4: Methods with Collections

Reading References

- Java SE 25 API documentation
- Dev.java learning paths
- Matching Python lesson: Lesson 6.4: Functions with Lists and Dictionaries

Learning Objectives

- Explain that a list argument is a reference passed by value.
- Change a list in a method and protect the original with a copy.
- Process the shared inventory with a method.

Concept Explanation

Java passes everything **by value**, but for an object the value is a *reference*. A method that receives a `List` can change the caller's list through that reference, so `addScore(scores, 95)` is visible to the caller. Reassigning the parameter inside the method does not change the caller's variable.

To produce a changed list without touching the original, return a *new* list: copy with `new ArrayList<>(list)` and add to the copy. `Collections.unmodifiableList(list)` or `List.copyOf(list)` give read-only versions that protect data from accidental changes.

The shared inventory is Notebook (3 at 2.50), Pencil (10 at 0.50), and Folder (4 at 1.25), worth 17.50 in total.

Worked Example

```
import java.util.ArrayList;
import java.util.List;
import java.util.Locale;

public class Lesson_6_4 {
    record Item(String name, int quantity, double unitPrice) {}
```

```
    static void addScore(List<Integer> scoreList, int newScore) {
        scoreList.add(newScore);
    }

    static List<Integer> withScore(List<Integer> scoreList, int newScore) {
        List<Integer> copy = new ArrayList<>(scoreList);
        copy.add(newScore);
        return copy;
    }

    static double inventoryValue(List<Item> inventory) {
        double total = 0;
        for (Item item : inventory) {
            total += item.quantity() * item.unitPrice();
        }
        return total;
    }

    public static void main(String[] args) {
        List<Integer> scores = new ArrayList<>(List.of(82, 91, 76, 88));
        addScore(scores, 95);
        System.out.println(scores);

        List<Integer> longer = withScore(scores, 100);
        System.out.println(scores.size() + " " + longer.size());

        List<Item> inventory = List.of(
            new Item("Notebook", 3, 2.50),
            new Item("Pencil", 10, 0.50),
            new Item("Folder", 4, 1.25));
        System.out.println("Total inventory value: " + String.format(Locale.US, "%.2f", inve
    }
}
```

Expected behavior: The program compiles on JDK 25 and prints a deterministic result illustrating methods with lists and maps. Run it from a terminal, compare actual output with your prediction, and change one input to test the effect.

Common Mistakes

- Confusing a method's parameter list with its return type or omitting a return on one control-flow path.
- Expecting Java to pass primitive variables by reference; arguments are passed by value.
- Placing a varargs parameter before another parameter or using an overload

that is ambiguous.

Activities

1. Extract repeated statements into a named method with an explicit parameter and return contract.
2. Call the method with typical, boundary, and empty collection arguments.
3. Add a method that accepts a List or Map and returns a computed result without modifying the input.
4. Overload a method or add a varargs form, then explain overload resolution.

Accessibility Practice

Navigate by class, method, and line; inspect braces, parentheses, brackets, semicolons, generic delimiters, and compiler diagnostics with the preferred screen reader or braille display. Compile and run from the terminal so the workflow does not depend on color or a visual project explorer.

Skills Checklist

☐ I can explain the key concept in Java terms.
☐ I can trace and predict the worked example.
☐ I can compile and run the example from the terminal.
☐ I can identify and correct a Java-specific mistake.
☐ I can test a boundary case and navigate the workflow accessibly.

Lesson 6.5: Arbitrary Number of Arguments

Reading References

- Java SE 25 documentation.
- Java Language Specification, Java SE 25.

Learning Objectives

- Explain the purpose and vocabulary of arbitrary number of arguments.
- Apply the relevant Java syntax and APIs in a small, complete program.
- Compile, run, inspect, and adapt the example using keyboard-accessible tools.

Concept Explanation

This lesson develops arbitrary number of arguments through Java's class-based structure and standard library. Identify the input, output, state changes, and relevant type constraints before coding. Keep declarations local where possible, separate computation from presentation, and state preconditions and postconditions. A method signature includes its name and parameter types; Java passes

argument values, including copied reference values. Return types are explicit, overloads differ by parameter lists, and varargs use a final array-like parameter.

Java is statically typed and class-based. Local variables have declared or inferred compile-time types, reference values may be null, and methods and fields are governed by access and scope rules. The compiler checks syntax and type compatibility before execution; runtime behavior still requires testing. Follow Java naming conventions, use explicit imports in beginner work, and prefer small methods with clear contracts.

Worked Example

```
public class Lesson_6_5 {
    public static void main(String[] args) {
        System.out.println(sum(2, 4, 6, 8));
        }
        private static int sum(int... values) {
            int total = 0;
            for (int value : values) total += value;
            return total;
    }
}
```

Expected behavior: The program compiles on JDK 25 and prints a deterministic result illustrating arbitrary number of arguments. Run it from a terminal, compare actual output with your prediction, and change one input to test the effect.

Common Mistakes

- Confusing a method's parameter list with its return type or omitting a return on one control-flow path.
- Expecting Java to pass primitive variables by reference; arguments are passed by value.
- Placing a varargs parameter before another parameter or using an overload that is ambiguous.

Activities

1. Extract repeated statements into a named method with an explicit parameter and return contract.
2. Call the method with typical, boundary, and empty collection arguments.
3. Add a method that accepts a List or Map and returns a computed result without modifying the input.
4. Overload a method or add a varargs form, then explain overload resolution.

Accessibility Practice

Navigate by class, method, and line; inspect braces, parentheses, brackets, semicolons, generic delimiters, and compiler diagnostics with the preferred screen reader or braille display. Compile and run from the terminal so the workflow does not depend on color or a visual project explorer.

Skills Checklist

- ☐ I can explain the key concept in Java terms.
- ☐ I can trace and predict the worked example.
- ☐ I can compile and run the example from the terminal.
- ☐ I can identify and correct a Java-specific mistake.
- ☐ I can test a boundary case and navigate the workflow accessibly.

8.9 Unit 7: Classes and Object-Oriented Programming

Overview

This unit develops classes and object-oriented programming through the same shared problem contexts and assessment expectations used across the general-purpose language tracks, expressed with Java 25 syntax, APIs, and static typing.

Unit objectives: explain the core concepts, write and test complete Java programs, diagnose errors, and apply the ideas to accessible projects.

Lesson 7.1: Why Learn About Classes?

Reading References

- Java SE 25 API documentation
- Java Language Specification, Java SE 25
- Dev.java learning paths
- Matching Python lesson: Lesson 7.1: Why Learn About Classes?

Learning Objectives

- Explain a class as data plus behavior.
- Hide fields with `private` and change them only through methods.
- Describe encapsulation.

Concept Explanation

A *class* describes what a kind of thing knows (its *fields*) and what it can do (its *methods*). An *object* is one actual thing made from a class with `new`. Bundling

data with the methods that work on it is called **encapsulation**. The shared example is an inventory *item* with a name, a quantity, and a unit price.

Marking a field `private` means only code inside the class can read or change it. Other code uses methods such as `getQuantity()` and `restock(amount)`, and `restock` can refuse invalid changes. `final` on a field means it is assigned once, in the constructor.

A source file may contain several classes, but only one may be `public` and it must match the file name. The helper class `Item` below is a normal, package-private class in the same file.

A Notebook with quantity 3 and unit price 2.50 is worth 7.50. Restocking 2 raises that to 12.50, and a negative restock is refused.

Worked Example

```
import java.util.Locale;

class Item {
    private final String name;
    private int quantity;
    private final double unitPrice;

    Item(String name, int quantity, double unitPrice) {
        this.name = name;
        this.quantity = quantity;
        this.unitPrice = unitPrice;
    }

    boolean restock(int amount) {
        if (amount < 0) {
            return false;
        }
        quantity += amount;
        return true;
    }

    int getQuantity() {
        return quantity;
    }

    double totalValue() {
        return quantity * unitPrice;
    }

    void describe() {
```

```
        System.out.println(name + ": " + quantity + " at " + String.format(Locale.US, "%.2f"
            + " each, value " + String.format(Locale.US, "%.2f", totalValue()));
    }
}

public class Lesson_7_1 {
    public static void main(String[] args) {
        Item notebook = new Item("Notebook", 3, 2.50);
        notebook.describe();
        System.out.println(notebook.restock(2));
        notebook.describe();
        System.out.println(notebook.restock(-1));
        System.out.println(notebook.getQuantity());
        // notebook.quantity = 99;   // would not compile: the field is private
    }
}
```

Expected behavior: The program compiles on JDK 25 and prints a deterministic result illustrating why learn about classes?. Run it from a terminal, compare actual output with your prediction, and change one input to test the effect.

Common Mistakes

- Exposing mutable fields and allowing callers to bypass class invariants.
- Using inheritance for a has-a relationship or failing to call the appropriate superclass constructor.
- Confusing an overridden method with an overloaded method, or omitting @Override when overriding.

Activities

1. Model one shared course entity with fields, constructor validation, and behavior methods.
2. Attempt an invalid state transition and verify encapsulation prevents it.
3. Demonstrate a true is-a relationship with overriding and runtime dispatch.
4. Replace an artificial inheritance relationship with composition and compare coupling.

Accessibility Practice

Navigate by class, method, and line; inspect braces, parentheses, brackets, semicolons, generic delimiters, and compiler diagnostics with the preferred screen reader or braille display. Compile and run from the terminal so the workflow does not depend on color or a visual project explorer.

Skills Checklist

- ☐ I can explain the key concept in Java terms.
- ☐ I can trace and predict the worked example.
- ☐ I can compile and run the example from the terminal.
- ☐ I can identify and correct a Java-specific mistake.
- ☐ I can test a boundary case and navigate the workflow accessibly.

Lesson 7.2: Creating and Using a Class

Reading References

- Java SE 25 documentation.
- Java Language Specification, Java SE 25.

Learning Objectives

- Explain the purpose and vocabulary of creating and using a class.
- Apply the relevant Java syntax and APIs in a small, complete program.
- Compile, run, inspect, and adapt the example using keyboard-accessible tools.

Concept Explanation

This lesson develops creating and using a class through Java's class-based structure and standard library. Identify the input, output, state changes, and relevant type constraints before coding. Keep declarations local where possible, separate computation from presentation, and state preconditions and postconditions. Objects combine state and behavior. Encapsulation protects invariants, constructors establish valid initial state, inheritance models a genuine is-a relationship, and composition models has-a collaboration. Prefer interfaces and composition when they reduce coupling.

Java is statically typed and class-based. Local variables have declared or inferred compile-time types, reference values may be null, and methods and fields are governed by access and scope rules. The compiler checks syntax and type compatibility before execution; runtime behavior still requires testing. Follow Java naming conventions, use explicit imports in beginner work, and prefer small methods with clear contracts.

Worked Example

```
public class Lesson_7_2 {
    public static void main(String[] args) {
        Student student = new Student("Avery");
        student.enroll();
        System.out.println(student.name + ": " + student.active);
        }
```

```
        static class Student {
            String name;
            boolean active;
            Student(String name) { this.name = name; }
            void enroll() { active = true; }
    }
}
```

Expected behavior: The program compiles on JDK 25 and prints a deterministic result illustrating creating and using a class. Run it from a terminal, compare actual output with your prediction, and change one input to test the effect.

Common Mistakes

- Exposing mutable fields and allowing callers to bypass class invariants.
- Using inheritance for a has-a relationship or failing to call the appropriate superclass constructor.
- Confusing an overridden method with an overloaded method, or omitting @Override when overriding.

Activities

1. Model one shared course entity with fields, constructor validation, and behavior methods.
2. Attempt an invalid state transition and verify encapsulation prevents it.
3. Demonstrate a true is-a relationship with overriding and runtime dispatch.
4. Replace an artificial inheritance relationship with composition and compare coupling.

Accessibility Practice

Navigate by class, method, and line; inspect braces, parentheses, brackets, semicolons, generic delimiters, and compiler diagnostics with the preferred screen reader or braille display. Compile and run from the terminal so the workflow does not depend on color or a visual project explorer.

Skills Checklist

☐ I can explain the key concept in Java terms.
☐ I can trace and predict the worked example.
☐ I can compile and run the example from the terminal.
☐ I can identify and correct a Java-specific mistake.
☐ I can test a boundary case and navigate the workflow accessibly.

Lesson 7.3: Working with Attributes and Methods

Reading References

- Java SE 25 documentation.
- Java Language Specification, Java SE 25.

Learning Objectives

- Explain the purpose and vocabulary of working with attributes and methods.
- Apply the relevant Java syntax and APIs in a small, complete program.
- Compile, run, inspect, and adapt the example using keyboard-accessible tools.

Concept Explanation

This lesson develops working with attributes and methods through Java's class-based structure and standard library. Identify the input, output, state changes, and relevant type constraints before coding. Keep declarations local where possible, separate computation from presentation, and state preconditions and postconditions. Objects combine state and behavior. Encapsulation protects invariants, constructors establish valid initial state, inheritance models a genuine is-a relationship, and composition models has-a collaboration. Prefer interfaces and composition when they reduce coupling.

Java is statically typed and class-based. Local variables have declared or inferred compile-time types, reference values may be null, and methods and fields are governed by access and scope rules. The compiler checks syntax and type compatibility before execution; runtime behavior still requires testing. Follow Java naming conventions, use explicit imports in beginner work, and prefer small methods with clear contracts.

Worked Example

```
public class Lesson_7_3 {
    public static void main(String[] args) {
        BankAccount account = new BankAccount(100.0);
        account.deposit(25.0);
        System.out.printf("balance=%.2f%n", account.balance());
        }
        static class BankAccount {
            private double balance;
            BankAccount(double opening) { if (opening < 0) throw new IllegalArgumentExceptio
            void deposit(double amount) { if (amount <= 0) throw new IllegalArgumentExceptio
            double balance() { return balance; }
    }
}
```

Expected Behavior or Output

The program prints:

```
Notebook: 3 at 2.50 each, value 7.50
Notebook: 5 at 2.50 each, value 12.50
Notebook: 4 at 2.50 each, value 10.00
Cannot restock a negative amount.
true
```

Common Mistakes

1. Forgetting `@Override`, so a misspelled `tostring()` silently becomes a new method.
2. Giving a field a public setter that skips the validation the object needs.
3. Storing a value that can be computed, so it goes out of date when the inputs change.

Activities

1. Predict the output of all six lines before running.
2. Add a method `discounted(double rate)` that returns the value after a discount without changing the item.
3. Remove the `setQuantity` method and see which lines stop compiling.
4. Explain why `toString` is better than printing fields one by one.

Accessibility Practice

Name methods after actions (`restock`) so a call reads as a sentence. Printing an object with a good `toString` gives the screen reader one complete line per object.

Skills Checklist

- ☐ I can write getters and a computed method.
- ☐ I can validate in a method.
- ☐ I can override `toString`.
- ☐ I know why `@Override` helps.
- ☐ I can tell a stored from a computed value.

Lesson 7.4: Inheritance

Reading References

- Java SE 25 API documentation
- Java Language Specification, Java SE 25
- Dev.java learning paths

- Matching Python lesson: Lesson 7.4: Inheritance

Learning Objectives

- Declare a subclass with `extends`.
- Override a method and call the parent's version with `super`.
- Explain polymorphism.

Concept Explanation

A subclass is declared with `extends`: `class DiscountedItem extends Item`. Its constructor must call the parent's constructor first with `super(...)`, which sets up the inherited fields. A method that replaces the parent's method is marked `@Override`, and `super.totalValue()` calls the parent's version. A class marked `final` cannot be extended, and a `private` field of the parent is not directly visible to the subclass.

Polymorphism means the same method call behaves differently for different classes. A variable of type `Item` can refer to a `DiscountedItem`, and calling `totalValue()` runs the discounted version. `describe` is written once in `Item` and calls `totalValue()`, so it prints the discounted value automatically.

The example discounts the shared Folder (4 at 1.25) by 20 percent, so its value is 4.00 instead of 5.00.

Worked Example

```
import java.util.List;
import java.util.Locale;

class Item {
    protected final String name;
    protected final int quantity;
    protected final double unitPrice;

    Item(String name, int quantity, double unitPrice) {
        this.name = name;
        this.quantity = quantity;
        this.unitPrice = unitPrice;
    }

    double totalValue() {
        return quantity * unitPrice;
    }

    void describe() {
        System.out.println(name + ": " + quantity + " at " + String.format(Locale.US, "%.2f"
            + " each, value " + String.format(Locale.US, "%.2f", totalValue()));
```

```
    }
}

class DiscountedItem extends Item {
    private final double discountRate;

    DiscountedItem(String name, int quantity, double unitPrice, double discountRate) {
        super(name, quantity, unitPrice);
        this.discountRate = discountRate;
    }

    @Override
    double totalValue() {
        return super.totalValue() * (1 - discountRate);
    }
}

public class Lesson_7_4 {
    public static void main(String[] args) {
        DiscountedItem folder = new DiscountedItem("Folder", 4, 1.25, 0.20);
        folder.describe();
        System.out.println(folder instanceof Item);

        List<Item> items = List.of(new Item("Notebook", 3, 2.50), folder);
        for (Item item : items) {
            System.out.println(item.name + ": " + String.format(Locale.US, "%.2f", item.tota
        }
    }
}
```

Expected Behavior or Output

The program prints:

```
Folder: 4 at 1.25 each, value 4.00
true
Notebook: 7.50
Folder: 4.00
```

Common Mistakes

1. Forgetting to call `super(...)` first in the constructor, which is a compile error when the parent has no no-argument constructor.
2. Forgetting `@Override` and accidentally creating a new method instead of replacing the parent's.
3. Using inheritance for a ”has a” relationship; use composition instead (Lesson 7.5).

Activities

1. Predict the output before running.
2. Add a subclass `PerishableItem` with an `expires` field and override `describe`.
3. Explain why `describe` prints the discounted value without being redefined.
4. List the order in which Java looks for `totalValue` on a `DiscountedItem`.

Accessibility Practice

Describe inheritance as an ordered list: the subclass first, then its parent. Speaking the chain aloud ('DiscountedItem, then Item') is clearer than a diagram.

Skills Checklist

- ☐ I can declare a subclass.
- ☐ I can override a method.
- ☐ I can call `super`.
- ☐ I can explain polymorphism.
- ☐ I know `final` stops inheritance.

Lesson 7.5: Composition

Reading References

- Java SE 25 API documentation
- Java Language Specification, Java SE 25
- Dev.java learning paths
- Matching Python lesson: Lesson 7.5: Instances as Attributes (Composition)

Learning Objectives

- Explain composition (”has a”).
- Hold a list of objects inside another object.
- Build an inventory from the shared items.

Concept Explanation

Composition means that one object contains other objects. An inventory *has* items; it is not a kind of item. Composition is usually simpler and safer than inheritance when objects are related by containment.

The container keeps a `private` list of its parts and exposes methods that loop over the parts and call each part's own methods, so the container does not need to know how a part works. Keeping the list private forces other code to use `add`

and `report`. The method `find` returns an `Optional<Item>` because the item may not exist.

The example builds an `Inventory` with the three shared items and reports each quantity and the total value, 17.50.

Worked Example

```
import java.util.ArrayList;
import java.util.List;
import java.util.Locale;
import java.util.Optional;

class Item {
    final String name;
    final int quantity;
    final double unitPrice;

    Item(String name, int quantity, double unitPrice) {
        this.name = name;
        this.quantity = quantity;
        this.unitPrice = unitPrice;
    }

    double totalValue() {
        return quantity * unitPrice;
    }
}

class Inventory {
    private final List<Item> items = new ArrayList<>();

    void add(Item item) {
        items.add(item);
    }

    Optional<Item> find(String name) {
        for (Item item : items) {
            if (item.name.equals(name)) {
                return Optional.of(item);
            }
        }
        return Optional.empty();
    }

    double totalValue() {
```

```
        double total = 0;
        for (Item item : items) {
            total += item.totalValue();
        }
        return total;
    }

    void report() {
        for (Item item : items) {
            System.out.println(item.name + ": " + item.quantity);
        }
        System.out.println("Total value: " + String.format(Locale.US, "%.2f", totalValue()))
    }
}

public class Lesson_7_5 {
    public static void main(String[] args) {
        Inventory inventory = new Inventory();
        inventory.add(new Item("Notebook", 3, 2.50));
        inventory.add(new Item("Pencil", 10, 0.50));
        inventory.add(new Item("Folder", 4, 1.25));
        inventory.report();
        System.out.println(inventory.find("Pencil").map(item -> item.quantity).orElse(-1));
        System.out.println(inventory.find("Eraser").isPresent());
    }
}
```

Expected behavior: The program compiles on JDK 25 and prints a deterministic result illustrating instances as attributes: composition. Run it from a terminal, compare actual output with your prediction, and change one input to test the effect.

Common Mistakes

- Exposing mutable fields and allowing callers to bypass class invariants.
- Using inheritance for a has-a relationship or failing to call the appropriate superclass constructor.
- Confusing an overridden method with an overloaded method, or omitting @Override when overriding.

Activities

1. Model one shared course entity with fields, constructor validation, and behavior methods.
2. Attempt an invalid state transition and verify encapsulation prevents it.
3. Demonstrate a true is-a relationship with overriding and runtime dispatch.

4. Replace an artificial inheritance relationship with composition and compare coupling.

Accessibility Practice

Navigate by class, method, and line; inspect braces, parentheses, brackets, semicolons, generic delimiters, and compiler diagnostics with the preferred screen reader or braille display. Compile and run from the terminal so the workflow does not depend on color or a visual project explorer.

Skills Checklist

- ☐ I can explain the key concept in Java terms.
- ☐ I can trace and predict the worked example.
- ☐ I can compile and run the example from the terminal.
- ☐ I can identify and correct a Java-specific mistake.
- ☐ I can test a boundary case and navigate the workflow accessibly.

8.10 Unit 8: Files and Exceptions

Overview

This unit develops files and exceptions through the same shared problem contexts and assessment expectations used across the general-purpose language tracks, expressed with Java 25 syntax, APIs, and static typing.

Unit objectives: explain the core concepts, write and test complete Java programs, diagnose errors, and apply the ideas to accessible projects.

Lesson 8.1: Reading from a File

Reading References

- Java SE 25 API documentation
- Dev.java learning paths
- Matching Python lesson: Lesson 8.1: Reading from a File

Learning Objectives

- Read a whole file and read it line by line.
- Check that a file exists before reading.
- Report a missing file clearly.

Concept Explanation

`java.nio.file.Path` represents a location and `java.nio.file.Files` does the work. `Files.readString(path)` returns the whole file, `Files.readAllLines(path)` returns a list of lines, and `Files.lines(path)` returns a *stream* of lines that

must be closed, so it is used in try-with-resources. `Files.exists(path)` tests whether a file is there. A relative path such as `notes.txt` starts from the program's working directory.

Most file methods throw the *checked* exception `IOException`, which the compiler forces you to handle: either catch it or declare `throws IOException` on the method. A missing file throws a `NoSuchFileException`.

To keep the lesson self-contained, the example first writes the two-line `notes.txt` that every language track uses, then reads it three ways and checks a file that does not exist.

Worked Example

```
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.stream.Stream;

public class Lesson_8_1 {
    public static void main(String[] args) throws IOException {
        Path notes = Path.of("notes.txt");
        Files.writeString(notes, "Practice Java.\nReview lists and methods.\n");

        System.out.println(Files.readString(notes));

        for (String line : Files.readAllLines(notes)) {
            System.out.println(line);
        }
        System.out.println(Files.readAllLines(notes).size());

        try (Stream<String> lines = Files.lines(notes)) {
            lines.map(String::toUpperCase).forEach(System.out::println);
        }

        Path missing = Path.of("missing_notes.txt");
        if (!Files.exists(missing)) {
            System.out.println("missing_notes.txt was not found.");
        }
    }
}
```

Expected behavior: The program compiles on JDK 25 and prints a deterministic result illustrating reading from a file. Run it from a terminal, compare actual output with your prediction, and change one input to test the effect.

Common Mistakes

- Leaving a stream open instead of using try-with-resources or ignoring IOException.
- Catching Exception broadly and hiding failures that should be reported or propagated.
- Assuming JSON parsing is built into Java SE or constructing JSON with unsafe string concatenation.

Activities

1. Read and write a temporary file, then verify the file content and clean up the resource.
2. Test a missing file and malformed numeric content; distinguish checked from unchecked exceptions.
3. Explain which resources are closed by try-with-resources and which code executes in finally.
4. Document why production JSON requires a parser/serializer library and test malformed JSON with that library when available.

Accessibility Practice

Navigate by class, method, and line; inspect braces, parentheses, brackets, semicolons, generic delimiters, and compiler diagnostics with the preferred screen reader or braille display. Compile and run from the terminal so the workflow does not depend on color or a visual project explorer.

Skills Checklist

☐ I can explain the key concept in Java terms.
☐ I can trace and predict the worked example.
☐ I can compile and run the example from the terminal.
☐ I can identify and correct a Java-specific mistake.
☐ I can test a boundary case and navigate the workflow accessibly.

Lesson 8.2: Writing to a File

Reading References

- Java SE 25 documentation.
- Java Language Specification, Java SE 25.

Learning Objectives

- Explain the purpose and vocabulary of writing to a file.
- Apply the relevant Java syntax and APIs in a small, complete program.
- Compile, run, inspect, and adapt the example using keyboard-accessible tools.

Concept Explanation

This lesson develops writing to a file through Java's class-based structure and standard library. Identify the input, output, state changes, and relevant type constraints before coding. Keep declarations local where possible, separate computation from presentation, and state preconditions and postconditions. File APIs can fail with IOException; use Path and Files for common operations and try-with-resources for closeable streams. Catch only failures you can handle. JSON is not built into the Java SE standard library, so robust JSON parsing requires a documented external library such as Jackson or Gson.

Java is statically typed and class-based. Local variables have declared or inferred compile-time types, reference values may be null, and methods and fields are governed by access and scope rules. The compiler checks syntax and type compatibility before execution; runtime behavior still requires testing. Follow Java naming conventions, use explicit imports in beginner work, and prefer small methods with clear contracts.

Worked Example

```
public class Lesson_8_2 {
    public static void main(String[] args) {
        try {
            java.nio.file.Path file = java.nio.file.Files.createTempFile("java-write", ".txt
            java.nio.file.Files.writeString(file, "Avery,92\\nRiley,85\\n");
            System.out.println(java.nio.file.Files.size(file) + " bytes written");
            java.nio.file.Files.deleteIfExists(file);
        } catch (java.io.IOException ex) { System.out.println(ex.getMessage()); }
    }
}
```

Expected behavior: The program compiles on JDK 25 and prints a deterministic result illustrating writing to a file. Run it from a terminal, compare actual output with your prediction, and change one input to test the effect.

Common Mistakes

- Leaving a stream open instead of using try-with-resources or ignoring IOException.
- Catching Exception broadly and hiding failures that should be reported or propagated.
- Assuming JSON parsing is built into Java SE or constructing JSON with unsafe string concatenation.

Activities

1. Read and write a temporary file, then verify the file content and clean up the resource.

2. Test a missing file and malformed numeric content; distinguish checked from unchecked exceptions.
3. Explain which resources are closed by try-with-resources and which code executes in finally.
4. Document why production JSON requires a parser/serializer library and test malformed JSON with that library when available.

Accessibility Practice

Navigate by class, method, and line; inspect braces, parentheses, brackets, semicolons, generic delimiters, and compiler diagnostics with the preferred screen reader or braille display. Compile and run from the terminal so the workflow does not depend on color or a visual project explorer.

Skills Checklist

☐ I can explain the key concept in Java terms.
☐ I can trace and predict the worked example.
☐ I can compile and run the example from the terminal.
☐ I can identify and correct a Java-specific mistake.
☐ I can test a boundary case and navigate the workflow accessibly.

Lesson 8.3: Exceptions and Handling Errors

Reading References

- Java SE 25 documentation.
- Java Language Specification, Java SE 25.

Learning Objectives

- Explain the purpose and vocabulary of exceptions and handling errors.
- Apply the relevant Java syntax and APIs in a small, complete program.
- Compile, run, inspect, and adapt the example using keyboard-accessible tools.

Concept Explanation

This lesson develops exceptions and handling errors through Java's class-based structure and standard library. Identify the input, output, state changes, and relevant type constraints before coding. Keep declarations local where possible, separate computation from presentation, and state preconditions and postconditions. File APIs can fail with IOException; use Path and Files for common operations and try-with-resources for closeable streams. Catch only failures you can handle. JSON is not built into the Java SE standard library, so robust JSON parsing requires a documented external library such as Jackson or Gson.

Java is statically typed and class-based. Local variables have declared or inferred compile-time types, reference values may be null, and methods and fields are governed by access and scope rules. The compiler checks syntax and type compatibility before execution; runtime behavior still requires testing. Follow Java naming conventions, use explicit imports in beginner work, and prefer small methods with clear contracts.

Worked Example

```java
public class Lesson_8_3 {
    public static void main(String[] args) {
        String raw = "not-a-number";
        try {
            int value = Integer.parseInt(raw);
            System.out.println(value);
        } catch (NumberFormatException ex) {
            System.out.println("Invalid integer: " + raw);
        }
    }
}
```

Expected Behavior or Output

The program prints:

```
Error: missing_notes.txt was not found.
Error: 'abc' is not a valid score.
Error: / by zero
Infinity
raised on purpose
Cleanup runs either way.
```

Common Mistakes

1. Catching `Exception` for everything, which hides mistakes in your own code.
2. Using `Integer.parseInt` on untrusted text without a catch.
3. Expecting `10.0 / 0` to throw. It gives `Infinity`.

Activities

1. Predict each printed line before running.
2. Wrap `Integer.parseInt(scanner.nextLine())` so invalid input prints a clear message and asks again.
3. Catch `ArithmeticException` in a method that computes an average of an empty list.
4. Explain when to catch an exception and when to let it stop the program.

Accessibility Practice

Write error messages as complete sentences that name the problem and the next step, such as 'Enter a whole number from 0 to 100.' Read the exception's class name aloud when you see a stack trace, then jump to the first line that names your own file.

Skills Checklist

- ☐ I can catch specific exceptions.
- ☐ I can use `finally` and multi-catch.
- ☐ I can explain checked versus unchecked.
- ☐ I know integer and decimal division by zero differ.
- ☐ I can write clear error messages.

Lesson 8.4: Cleanup with try-with-resources

Reading References

- Java SE 25 API documentation
- Java Language Specification, Java SE 25
- Dev.java learning paths
- Matching Python lesson: Lesson 8.4: The else and finally Blocks

Learning Objectives

- Close files reliably with try-with-resources.
- Show that a resource is released even after an error.
- Use `finally` for cleanup.

Concept Explanation

A reader or writer holds a resource from the operating system, so it must be closed. *Try-with-resources* declares the resource in parentheses after `try`: `try (BufferedReader reader = ...) { ... }`. Java closes it automatically when the block ends, **even if the block throws an exception**. Since Java 9, an existing variable that is not reassigned can be listed too: `try (reader) { ... }`.

`finally` is the general tool: its block runs after the `try` block whether or not an exception happened. Use try-with-resources for anything that can be closed, and `finally` for other cleanup.

The example writes a file, reads it back, throws an exception inside a second block, and shows that a reader closed by try-with-resources refuses to read.

Worked Example

```
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;

public class Lesson_8_4 {
    public static void main(String[] args) throws IOException {
        Path file = Path.of("cleanup_demo.txt");
        try (BufferedWriter writer = Files.newBufferedWriter(file)) {
            writer.write("Some data.\n");
        }
        System.out.println(Files.readString(file).trim());

        BufferedReader reader = Files.newBufferedReader(file);
        String firstLine;
        try (reader) {
            firstLine = reader.readLine();
        }
        System.out.println(firstLine);

        try (BufferedReader second = Files.newBufferedReader(file)) {
            second.readLine();
            throw new IllegalStateException("failure while reading");
        } catch (IllegalStateException e) {
            System.out.println(e.getMessage());
        }

        try {
            System.out.println("working");
        } finally {
            System.out.println("finally block ran");
        }

        boolean refused = false;
        try {
            reader.readLine();
        } catch (IOException e) {
            refused = true;
        }
        System.out.println(refused);
    }
}
```

Expected behavior: The program compiles on JDK 25 and prints a deterministic result illustrating the catch, finally, and try-with-resources patterns. Run it from a terminal, compare actual output with your prediction, and change one input to test the effect.

Common Mistakes

- Leaving a stream open instead of using try-with-resources or ignoring IOException.
- Catching Exception broadly and hiding failures that should be reported or propagated.
- Assuming JSON parsing is built into Java SE or constructing JSON with unsafe string concatenation.

Activities

1. Read and write a temporary file, then verify the file content and clean up the resource.
2. Test a missing file and malformed numeric content; distinguish checked from unchecked exceptions.
3. Explain which resources are closed by try-with-resources and which code executes in finally.
4. Document why production JSON requires a parser/serializer library and test malformed JSON with that library when available.

Accessibility Practice

Navigate by class, method, and line; inspect braces, parentheses, brackets, semicolons, generic delimiters, and compiler diagnostics with the preferred screen reader or braille display. Compile and run from the terminal so the workflow does not depend on color or a visual project explorer.

Skills Checklist

☐ I can explain the key concept in Java terms.
☐ I can trace and predict the worked example.
☐ I can compile and run the example from the terminal.
☐ I can identify and correct a Java-specific mistake.
☐ I can test a boundary case and navigate the workflow accessibly.

Lesson 8.5: Storing Data with JSON

Reading References

- Java SE 25 API documentation
- JSON specification: https://www.json.org
- Matching Python lesson: Lesson 8.5: Storing Data with JSON

Learning Objectives

- Explain why this lesson includes a JSON helper.
- Encode and decode the shared task records.
- Handle a missing or malformed file.

Concept Explanation

The Java standard library has no JSON support. Real projects add a library such as Jackson or Gson through Maven or Gradle. To keep this course free of build tooling, the example includes a *teaching implementation*, the nested class `Json`, with `encode` and `decode`. It handles strings, numbers, booleans, `null`, lists, and maps, writes object keys in sorted order, and reports malformed text with an exception that includes the position. It is not a replacement for a production library.

JSON text stores data in a standard format: `{ ... }` holds key-value pairs, `[ ... ]` holds ordered items, and text is in double quotation marks. Decoded data is untyped (`Map<String, Object>` and `List<Object>`), so check each field before using it. Numbers without a decimal point decode as `Long`.

The shared to-do record has the fields `id`, `title`, `description`, `done`, and `created_at`. The example saves two tasks, loads them back, and shows the message for a malformed file.

The teaching helper decodes JSON `null` as `null` and numbers without a decimal point as `Long`. It does not handle every number edge case that a full parser must.

Worked Example

```
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.*;

public class Lesson_8_5 {
    static final class Json {
        private Json() {}

        static String encode(Object value) {
            if (value == null) {
                return "null";
            }
            if (value instanceof String text) {
                return quote(text);
            }
            if (value instanceof Boolean || value instanceof Integer || value instanceof Lo
                return value.toString();
```

```
    }
    if (value instanceof Map<?, ?> map) {
        List<String> keys = new ArrayList<>();
        for (Object key : map.keySet()) {
            keys.add(key.toString());
        }
        Collections.sort(keys);
        StringBuilder out = new StringBuilder("{");
        for (int i = 0; i < keys.size(); i++) {
            if (i > 0) {
                out.append(',');
            }
            out.append(quote(keys.get(i))).append(':').append(encode(map.get(keys.ge
        }
        return out.append('}').toString();
    }
    if (value instanceof List<?> list) {
        StringBuilder out = new StringBuilder("[");
        for (int i = 0; i < list.size(); i++) {
            if (i > 0) {
                out.append(',');
            }
            out.append(encode(list.get(i)));
        }
        return out.append(']').toString();
    }
    throw new IllegalArgumentException("cannot encode " + value.getClass().getSimple
}

private static String quote(String text) {
    StringBuilder out = new StringBuilder("\"");
    for (char c : text.toCharArray()) {
        switch (c) {
            case '"' -> out.append("\\\"");
            case '\\' -> out.append("\\\\");
            case '\n' -> out.append("\\n");
            case '\r' -> out.append("\\r");
            case '\t' -> out.append("\\t");
            default -> {
                if (c < ' ') {
                    out.append(String.format("\\u%04x", (int) c));
                } else {
                    out.append(c);
                }
            }
        }
```

```
    }
    return out.append('"').toString();
}

static Object decode(String text) {
    Parser parser = new Parser(text);
    Object result = parser.value();
    parser.skip();
    if (parser.pos < text.length()) {
        throw parser.fail("unexpected extra text");
    }
    return result;
}

private static final class Parser {
    private final String text;
    private int pos = 0;

    Parser(String text) {
        this.text = text;
    }

    IllegalArgumentException fail(String message) {
        return new IllegalArgumentException(message + " at position " + pos);
    }

    void skip() {
        while (pos < text.length() && Character.isWhitespace(text.charAt(pos))) {
            pos++;
        }
    }

    Object value() {
        skip();
        if (pos >= text.length()) {
            throw fail("unexpected end of text");
        }
        char c = text.charAt(pos);
        if (c == '{') return object();
        if (c == '[') return list();
        if (c == '"') return string();
        if (text.startsWith("true", pos)) { pos += 4; return Boolean.TRUE; }
        if (text.startsWith("false", pos)) { pos += 5; return Boolean.FALSE; }
        if (text.startsWith("null", pos)) { pos += 4; return null; }
        if (c == '-' || Character.isDigit(c)) return number();
        throw fail("unexpected character");
```

```
}

Object number() {
    int start = pos;
    if (text.charAt(pos) == '-') {
        pos++;
    }
    while (pos < text.length() && (Character.isDigit(text.charAt(pos)) || ".eE+-
        pos++;
    }
    String token = text.substring(start, pos);
    try {
        return Long.parseLong(token);
    } catch (NumberFormatException notWhole) {
        try {
            return Double.parseDouble(token);
        } catch (NumberFormatException notNumber) {
            throw fail("bad number");
        }
    }
}

String string() {
    pos++;
    StringBuilder out = new StringBuilder();
    while (true) {
        if (pos >= text.length()) {
            throw fail("unterminated string");
        }
        char c = text.charAt(pos++);
        if (c == '"') {
            return out.toString();
        }
        if (c != '\\') {
            out.append(c);
            continue;
        }
        if (pos >= text.length()) {
            throw fail("bad escape");
        }
        char e = text.charAt(pos++);
        switch (e) {
            case 'n' -> out.append('\n');
            case 't' -> out.append('\t');
            case 'r' -> out.append('\r');
            case '"' -> out.append('"');
```

```
                case '\\' -> out.append('\\');
                case '/' -> out.append('/');
                case 'u' -> {
                    if (pos + 4 > text.length()) {
                        throw fail("bad escape");
                    }
                    out.append((char) Integer.parseInt(text.substring(pos, pos + 4),
                    pos += 4;
                }
                default -> throw fail("bad escape");
            }
        }
    }

    Map<String, Object> object() {
        pos++;
        Map<String, Object> map = new LinkedHashMap<>();
        skip();
        if (pos < text.length() && text.charAt(pos) == '}') {
            pos++;
            return map;
        }
        while (true) {
            skip();
            if (pos >= text.length() || text.charAt(pos) != '"') {
                throw fail("expected a key");
            }
            String key = string();
            skip();
            if (pos >= text.length() || text.charAt(pos) != ':') {
                throw fail("expected ':'");
            }
            pos++;
            map.put(key, value());
            skip();
            if (pos >= text.length()) {
                throw fail("unterminated object");
            }
            char d = text.charAt(pos++);
            if (d == '}') {
                return map;
            }
            if (d != ',') {
                pos--;
                throw fail("expected ',' or '}'");
            }
```

```
            }
        }

        List<Object> list() {
            pos++;
            List<Object> items = new ArrayList<>();
            skip();
            if (pos < text.length() && text.charAt(pos) == ']') {
                pos++;
                return items;
            }
            while (true) {
                items.add(value());
                skip();
                if (pos >= text.length()) {
                    throw fail("unterminated list");
                }
                char d = text.charAt(pos++);
                if (d == ']') {
                    return items;
                }
                if (d != ',') {
                    pos--;
                    throw fail("expected ',' or ']'");
                }
            }
        }
    }
}

public static void main(String[] args) throws IOException {
    List<Map<String, Object>> tasks = List.of(
        Map.of("id", 1, "title", "Study programming", "description", "Finish Unit 8",
            "done", false, "created_at", "2026-01-15T09:00:00"),
        Map.of("id", 2, "title", "Review scores", "description", "Check the four scores'
            "done", true, "created_at", "2026-01-15T09:30:00"));

    Path file = Path.of("tasks.json");
    Files.writeString(file, Json.encode(tasks));

    if (!Files.exists(file)) {
        System.out.println("Tasks file not found.");
        return;
    }
    List<?> loaded;
    try {
```

```
            loaded = (List<?>) Json.decode(Files.readString(file));
        } catch (IllegalArgumentException e) {
            System.out.println("Error: could not decode tasks file: " + e.getMessage());
            return;
        }
        for (Object entry : loaded) {
            Map<?, ?> task = (Map<?, ?>) entry;
            String status = Boolean.TRUE.equals(task.get("done")) ? "done" : "not done";
            System.out.println(task.get("id") + ": " + task.get("title") + " (" + status + '
        }

        try {
            Json.decode("[{\"id\": 1,");
        } catch (IllegalArgumentException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Expected behavior: The program compiles on JDK 25 and prints a deterministic result illustrating storing data with json. Run it from a terminal, compare actual output with your prediction, and change one input to test the effect.

Common Mistakes

- Leaving a stream open instead of using try-with-resources or ignoring IOException.
- Catching Exception broadly and hiding failures that should be reported or propagated.
- Assuming JSON parsing is built into Java SE or constructing JSON with unsafe string concatenation.

Activities

1. Read and write a temporary file, then verify the file content and clean up the resource.
2. Test a missing file and malformed numeric content; distinguish checked from unchecked exceptions.
3. Explain which resources are closed by try-with-resources and which code executes in finally.
4. Document why production JSON requires a parser/serializer library and test malformed JSON with that library when available.

Accessibility Practice

Navigate by class, method, and line; inspect braces, parentheses, brackets, semicolons, generic delimiters, and compiler diagnostics with the preferred screen

reader or braille display. Compile and run from the terminal so the workflow does not depend on color or a visual project explorer.

Skills Checklist

- ☐ I can explain the key concept in Java terms.
- ☐ I can trace and predict the worked example.
- ☐ I can compile and run the example from the terminal.
- ☐ I can identify and correct a Java-specific mistake.
- ☐ I can test a boundary case and navigate the workflow accessibly.

8.11 Unit 9: Testing and Data Analysis

Overview

This unit develops testing and data analysis through the same shared problem contexts and assessment expectations used across the general-purpose language tracks, expressed with Java 25 syntax, APIs, and static typing.

Unit objectives: explain the core concepts, write and test complete Java programs, diagnose errors, and apply the ideas to accessible projects.

Lesson 9.1: Testing Functions

Reading References

- Java SE 25 API documentation
- Dev.java learning paths
- Matching Python lesson: Lesson 9.1: Testing Your Code

Learning Objectives

- Write checks with chosen inputs and expected results.
- Compare decimals with a tolerance.
- Test boundary values and report failure with an exit status.

Concept Explanation

A *test* runs code with chosen inputs and compares the result with a value decided in advance. Real Java projects use JUnit through Maven or Gradle. To keep this course free of build tooling, the example uses a tiny `check` method; the habits (inputs, expected values, boundaries) are the same.

Decimal results are compared with a *tolerance*, not `equals`, because of floating-point approximation: the tip for 24.50 at 0.15 is stored as 3.6749999999999998, so the test checks that it is within a tiny distance of 3.675.

The same tests appear in every language track: `add(5, 3)` is 8, `add(-5, -3)` is -8, `add(10, -4)` is 6, the tip is 3.675, and the grade boundaries 90, 89, 60, and 59 give A, B, D, and F.

`System.exit(1)` ends the program with a failure status when any check fails, so other tools can notice.

Worked Example

```
import java.util.Objects;

public class Lesson_9_1 {
    static int passed = 0;
    static int failed = 0;

    static int add(int a, int b) {
        return a + b;
    }

    static double calculateTip(double bill, double rate) {
        return bill * rate;
    }

    static String classify(int score) {
        if (score >= 90) return "A";
        if (score >= 80) return "B";
        if (score >= 70) return "C";
        if (score >= 60) return "D";
        return "F";
    }

    static void check(String name, Object actual, Object expected) {
        if (Objects.equals(actual, expected)) {
            passed++;
            System.out.println("PASS " + name);
        } else {
            failed++;
            System.out.println("FAIL " + name + ": expected " + expected + " but got " + act
        }
    }

    static void checkClose(String name, double actual, double expected, double tolerance) {
        check(name, Math.abs(actual - expected) <= tolerance, true);
    }

    public static void main(String[] args) {
```

```
        check("add 5 and 3", add(5, 3), 8);
        check("add -5 and -3", add(-5, -3), -8);
        check("add 10 and -4", add(10, -4), 6);
        checkClose("tip on 24.50 at 0.15", calculateTip(24.5, 0.15), 3.675, 1e-9);
        check("tip on a zero bill", calculateTip(0, 0.15), 0.0);
        check("classify 90", classify(90), "A");
        check("classify 89", classify(89), "B");
        check("classify 60", classify(60), "D");
        check("classify 59", classify(59), "F");

        System.out.println(passed + " passed, " + failed + " failed");
        if (failed > 0) {
            System.exit(1);
        }
    }
}
```

Expected Behavior or Output

The program prints:

```
PASS add 5 and 3
PASS add -5 and -3
PASS add 10 and -4
PASS tip on 24.50 at 0.15
PASS tip on a zero bill
PASS classify 90
PASS classify 89
PASS classify 60
PASS classify 59
9 passed, 0 failed
```

Common Mistakes

1. Comparing decimal results with `equals` or `==`.
2. Testing only typical values and skipping the boundaries 90, 89, 60, and 59.
3. Writing the expected value after seeing the output, which turns the test into a copy of the bug.

Activities

1. Change one expected value so a check fails, read the message, and restore it.
2. Add checks for the grades at 80, 79, 70, and 69.
3. Write checks for `isPassing(score)` before writing the method.
4. Explain why the tip check uses a tolerance.

Accessibility Practice

Each check prints one complete line that starts with PASS or FAIL, and the last line gives the counts, so the result can be understood from the first word of each line and the final summary.

Skills Checklist

- ☐ I can write a check with an expected value.
- ☐ I can compare decimals with a tolerance.
- ☐ I can choose boundary tests.
- ☐ I can read PASS and FAIL lines.
- ☐ I know how to signal failure with an exit status.

Lesson 9.2: Testing Classes

Reading References

- Java SE 25 API documentation
- Dev.java learning paths
- Matching Python lesson: Lesson 9.2: Testing a Class

Learning Objectives

- Create a fresh object for every test.
- Test a value, a change of state, and an error.
- Test that an exception is thrown.

Concept Explanation

Each test should start from a known state, so create a new object for every test with a *setup* method. Tests that share an object can affect each other and fail in confusing ways.

Test three kinds of behavior: that a method returns the right value (`totalValue` for 3 at 2.50 is 7.5), that a method changes the object correctly (restocking 2 makes the quantity 5), and that invalid use is rejected (restocking a negative amount throws `IllegalArgumentException`).

To test that an exception **does** happen, call the method inside `try`, fail the test if no exception is thrown, and catch the expected exception type. JUnit's `assertThrows` does this in one line.

Worked Example

```
public class Lesson_9_2 {
    static class Item {
```

```
    final String name;
    int quantity;
    final double unitPrice;

    Item(String name, int quantity, double unitPrice) {
        this.name = name;
        this.quantity = quantity;
        this.unitPrice = unitPrice;
    }

    double totalValue() {
        return quantity * unitPrice;
    }

    void restock(int amount) {
        if (amount < 0) {
            throw new IllegalArgumentException("amount must not be negative");
        }
        quantity += amount;
    }
}

static int failures = 0;

static Item setup() {
    return new Item("Notebook", 3, 2.50);
}

static void assertThat(boolean condition, String message) {
    if (!condition) {
        throw new AssertionError(message);
    }
}

static void test(String name, Runnable body) {
    try {
        body.run();
        System.out.println("PASS " + name);
    } catch (AssertionError | RuntimeException e) {
        failures++;
        System.out.println("FAIL " + name + ": " + e.getMessage());
    }
}

public static void main(String[] args) {
    test("total value is 7.5", () ->
```

```
            assertThat(Math.abs(setup().totalValue() - 7.5) < 1e-9, "wrong total"));

        test("restock adds to the quantity", () -> {
            Item item = setup();
            item.restock(2);
            assertThat(item.quantity == 5, "quantity should be 5");
        });

        test("restock rejects a negative amount", () -> {
            boolean thrown = false;
            try {
                setup().restock(-1);
            } catch (IllegalArgumentException e) {
                thrown = true;
            }
            assertThat(thrown, "an exception should have been thrown");
        });

        System.out.println(failures == 0 ? "All tests passed." : failures + " test(s) failed
    }
}
```

Expected behavior: The program compiles on JDK 25 and prints a deterministic result illustrating testing a class. Run it from a terminal, compare actual output with your prediction, and change one input to test the effect.

Common Mistakes

- Relying on Java's assert statements without enabling assertions, or testing implementation details instead of behavior.
- Allowing tests to share mutable state so their order affects results.
- Assuming a simple comma split handles quoted CSV fields or using an unseeded random source in a reproducibility exercise.

Activities

1. Write one passing and one failing test for a pure method, then run the tests repeatedly.
2. Seed a Random instance and verify the exercise is reproducible.
3. Analyze a sentence for word counts after applying a documented normalization rule.
4. Test CSV data with quoted commas and explain why a production parser is required.

Accessibility Practice

Navigate by class, method, and line; inspect braces, parentheses, brackets, semicolons, generic delimiters, and compiler diagnostics with the preferred screen reader or braille display. Compile and run from the terminal so the workflow does not depend on color or a visual project explorer.

Skills Checklist

- ☐ I can explain the key concept in Java terms.
- ☐ I can trace and predict the worked example.
- ☐ I can compile and run the example from the terminal.
- ☐ I can identify and correct a Java-specific mistake.
- ☐ I can test a boundary case and navigate the workflow accessibly.

Lesson 9.3: Introduction to Random Numbers

Reading References

- Java SE 25 documentation.
- Java Language Specification, Java SE 25.

Learning Objectives

- Explain the purpose and vocabulary of introduction to random numbers.
- Apply the relevant Java syntax and APIs in a small, complete program.
- Compile, run, inspect, and adapt the example using keyboard-accessible tools.

Concept Explanation

This lesson develops introduction to random numbers through Java's class-based structure and standard library. Identify the input, output, state changes, and relevant type constraints before coding. Keep declarations local where possible, separate computation from presentation, and state preconditions and postconditions. Separate deterministic logic from random generation, file I/O, and user interaction. Test methods and object invariants with assertions or a test framework such as JUnit. CSV quoting rules mean a comma split is only a teaching illustration, not a production parser.

Java is statically typed and class-based. Local variables have declared or inferred compile-time types, reference values may be null, and methods and fields are governed by access and scope rules. The compiler checks syntax and type compatibility before execution; runtime behavior still requires testing. Follow Java naming conventions, use explicit imports in beginner work, and prefer small methods with clear contracts.

Worked Example

```java
public class Lesson_9_3 {
    public static void main(String[] args) {
        java.util.Random random = new java.util.Random(42);
        for (int i = 0; i < 5; i++) System.out.println(random.nextInt(10));
    }
}
```

Expected Behavior or Output

The program prints:

```
Same seed gives the same rolls: true
All rolls between 1 and 6: true
Decimal in [0, 1): true
Chosen score is from the list: true
```

Common Mistakes

1. Using `nextInt(1, 6)` for a die and never rolling a 6. The upper limit is excluded.
2. Testing for specific random values that may change between versions.
3. Using `Random` for passwords or tokens, which it is not designed for.

Activities

1. Run the program three times and compare.
2. Change the seeds and check that the property lines still print `true`.
3. Simulate 600 rolls and count how many of each face appeared (the counts depend on the seed).
4. Explain why a test should check a property rather than a specific random number.

Accessibility Practice

Print yes or no style results for random behavior, so the announced output is the same on every run and the screen reader does not read unpredictable values.

Skills Checklist

- ☐ I can use `nextInt` with an excluded upper bound.
- ☐ I can set a seed.
- ☐ I can test properties of random code.
- ☐ I know sequences can differ between versions.
- ☐ I can pick a random list item.

Lesson 9.4: Text Analysis

Reading References

- Java SE 25 API documentation
- Dev.java learning paths
- Matching Python lesson: Lesson 9.4: Data Analysis - Text Analysis

Learning Objectives

- Split text into words and count them.
- Count word and letter frequencies with a map.
- Produce stable, sorted output.

Concept Explanation

`text.split(" ")` splits a string into an array of words. Counting uses a map from each word to its count: `counts.merge(word, 1, Integer::sum)` adds one, starting from 1 for a new word.

Map order is only stable if you choose the right class. `LinkedHashMap` keeps the order in which words first appeared. To order by count, copy the entries into a list and sort it with a comparator: count descending first, then letter ascending, so ties print in a fixed order.

The shared text sample is "A student studies programming. Programming develops problem-solving skills." After lower-casing and removing the periods it has 8 words, and `programming` appears twice.

Worked Example

```
import java.util.ArrayList;
import java.util.Comparator;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

public class Lesson_9_4 {
    public static void main(String[] args) {
        String text = "A student studies programming. Programming develops problem-solving s
        String[] words = text.toLowerCase().replace(".", "").split(" ");

        Map<String, Integer> frequencies = new LinkedHashMap<>();
        for (String word : words) {
            frequencies.merge(word, 1, Integer::sum);
        }

        System.out.println("Word count: " + words.length);
```

```
        System.out.println("Word frequencies:");
        for (Map.Entry<String, Integer> entry : frequencies.entrySet()) {
            System.out.println("'" + entry.getKey() + "': " + entry.getValue());
        }

        Map<Character, Integer> letters = new LinkedHashMap<>();
        for (char c : text.toLowerCase().toCharArray()) {
            if (Character.isLetter(c)) {
                letters.merge(c, 1, Integer::sum);
            }
        }
        List<Map.Entry<Character, Integer>> entries = new ArrayList<>(letters.entrySet());
        entries.sort(Comparator.<Map.Entry<Character, Integer>>comparingInt(Map.Entry::getVa
            .thenComparing(Map.Entry::getKey));
        System.out.println("Most common letters:");
        for (int i = 0; i < 3; i++) {
            System.out.println(entries.get(i).getKey() + ": " + entries.get(i).getValue());
        }
    }
}
```

Expected Behavior or Output

The program prints:

```
Word count: 8
Word frequencies:
'a': 1
'student': 1
'studies': 1
'programming': 2
'develops': 1
'problem-solving': 1
'skills': 1
Most common letters:
s: 7
e: 5
g: 5
```

Common Mistakes

1. Counting punctuation as part of a word, so `programming.` and `programming` are different words.
2. Forgetting to lower-case first, so `Programming` and `programming` are counted separately.
3. Relying on `HashMap` order or on an unsorted tie.

Activities

1. Predict the word count and the count for `programming` before running.
2. Count the words longer than 8 letters.
3. Print the longest word.
4. Change the sort so ties are listed in reverse alphabetical order.

Accessibility Practice

Read the frequency list in its fixed order. Because the order is stable, you can learn it and compare runs by ear. Print each pair as a short sentence with the same wording every time.

Skills Checklist

- ☐ I can split text into words.
- ☐ I can count frequencies with a map.
- ☐ I can sort with a tie-break.
- ☐ I can remove punctuation first.
- ☐ I can read a stable report.

Lesson 9.5: Working with CSV Files

Reading References

- Java SE 25 API documentation
- Dev.java learning paths
- Matching Python lesson: Lesson 9.5: Data Analysis - CSV Files

Learning Objectives

- Create and read a CSV file.
- Split lines into fields and convert numbers.
- Compute an average and a count.

Concept Explanation

A CSV (comma-separated values) file stores a table as text. The first line names the columns and each later line is a record. The shared student file has the columns `student_id`, `first_name`, `last_name`, `score`, and `active`, and three records.

The standard library has no CSV reader. `line.split(",")` works only for **simple** files: a value that contains a comma inside quotation marks, such as `"Smith, Jr."`, would be split wrongly. For such files use a maintained library or write a real parser.

Every field is text. Convert the score with `Integer.parseInt` and compare `active` with the text `"true"`. With scores 91, 82, and 76 the average is 83.00, and two students are active.

Worked Example

```
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.List;
import java.util.Locale;

public class Lesson_9_5 {
    public static void main(String[] args) throws IOException {
        Path file = Path.of("students.csv");
        Files.writeString(file, String.join("\n",
            "student_id,first_name,last_name,score,active",
            "1,Avery,Student,91,true",
            "2,Jordan,Lee,82,true",
            "3,Casey,Smith,76,false") + "\n");

        if (!Files.exists(file)) {
            System.out.println("Error: students.csv was not found.");
            return;
        }
        List<String> lines = Files.readAllLines(file);
        System.out.println("Columns: " + String.join(", ", lines.get(0).split(",")));

        int total = 0;
        int active = 0;
        List<String> students = lines.subList(1, lines.size());
        for (String line : students) {
            String[] row = line.split(",");
            int score = Integer.parseInt(row[3]);
            total += score;
            if (row[4].equals("true")) {
                active++;
            }
            System.out.println(row[0] + ": " + row[1] + " " + row[2] + " scored " + score);
        }

        System.out.println("Students: " + students.size());
        System.out.println("Average score: " + String.format(Locale.US, "%.2f", (double) tot
        System.out.println("Active students: " + active);
    }
}
```

Expected behavior: The program compiles on JDK 25 and prints a deterministic result illustrating working with csv data. Run it from a terminal, compare actual output with your prediction, and change one input to test the effect.

Common Mistakes

- Relying on Java's assert statements without enabling assertions, or testing implementation details instead of behavior.
- Allowing tests to share mutable state so their order affects results.
- Assuming a simple comma split handles quoted CSV fields or using an unseeded random source in a reproducibility exercise.

Activities

1. Write one passing and one failing test for a pure method, then run the tests repeatedly.
2. Seed a Random instance and verify the exercise is reproducible.
3. Analyze a sentence for word counts after applying a documented normalization rule.
4. Test CSV data with quoted commas and explain why a production parser is required.

Accessibility Practice

Navigate by class, method, and line; inspect braces, parentheses, brackets, semicolons, generic delimiters, and compiler diagnostics with the preferred screen reader or braille display. Compile and run from the terminal so the workflow does not depend on color or a visual project explorer.

Skills Checklist

☐ I can explain the key concept in Java terms.
☐ I can trace and predict the worked example.
☐ I can compile and run the example from the terminal.
☐ I can identify and correct a Java-specific mistake.
☐ I can test a boundary case and navigate the workflow accessibly.

8.12 Unit 10: Project-Based Learning — Text Applications

Overview

This unit develops project-based learning — text applications through the same shared problem contexts and assessment expectations used across the general-purpose language tracks, expressed with Java 25 syntax, APIs, and static typing.

Unit objectives: explain the core concepts, write and test complete Java programs, diagnose errors, and apply the ideas to accessible projects.

Lesson 10.1: Planning a Text-Based Project

Reading References

- Java SE 25 API documentation
- Dev.java learning paths
- Matching Python lesson: Lesson 10.1: Project Planning and Design

Learning Objectives

- Write a short requirements list.
- Design a command loop with a map of handlers.
- Plan in small steps and test each one.

Concept Explanation

A project is easier to finish when it is planned in small steps. Write down what the program must do (*requirements*), what information it keeps (*data*), and the commands a person can give (*interface*). Build one small piece, test it, then add the next.

Text-based projects share a *command loop*: show a prompt, read a line, find the matching action, and repeat until the user quits. In Java, a `Map<String, Runnable>` holds actions: the keys are command names and the values are *lambdas*, small unnamed methods. A new command is one new entry, and an unknown command is a failed lookup that prints a helpful message.

The loop stops on `quit` and also when input ends. The example is a skeleton with two commands; typing `help`, `look`, an unknown word, and `quit` shows each path.

Worked Example

```
import java.util.Map;
import java.util.Scanner;

public class Lesson_10_1 {
    public static void main(String[] args) {
        Map<String, Runnable> commands = Map.of(
            "help", () -> System.out.println("Commands: help, look, quit"),
            "look", () -> System.out.println("You are in a dark cave."));

        Scanner in = new Scanner(System.in);
        System.out.println("Type a command:");
        String line = in.hasNextLine() ? in.nextLine() : null;
```

```
        while (line != null && !line.equals("quit")) {
            Runnable handler = commands.get(line);
            if (handler != null) {
                handler.run();
            } else {
                System.out.println("I don't understand that command. Type help for a list.")
            }
            System.out.println("Type a command:");
            line = in.hasNextLine() ? in.nextLine() : null;
        }
        System.out.println("Goodbye!");
    }
}
```

Expected Behavior or Output

With the following lines typed as standard input (piped input is not echoed, so it does not appear in the output):

```
help
look
dance
quit
```

the program prints:

```
Type a command:
Commands: help, look, quit
Type a command:
You are in a dark cave.
Type a command:
I don't understand that command. Type help for a list.
Type a command:
Goodbye!
```

With no standard input at all, the program ends cleanly and prints:

```
Type a command:
Goodbye!
```

Common Mistakes

1. Starting to code before listing the requirements.
2. Using a long chain of `if` statements instead of a map of handlers.
3. Forgetting that an unknown command should say what to do next.

Activities

1. Write five requirements for the text adventure and the to-do list.

2. Add a `score` command that prints the shared scores.
3. Add the aliases `h` and `l` without copying the lambdas.
4. Describe the data for the capstone in three sentences.

Accessibility Practice

Prompts and responses are on separate lines and every response is a complete sentence. Test the prompts by listening to them before you add features.

Skills Checklist

- ☐ I can list requirements.
- ☐ I can design a command loop.
- ☐ I can use a map of lambdas.
- ☐ I handle unknown commands.
- ☐ I handle the end of input.

Lesson 10.2: Text Adventure, Part 1: Rooms and Movement

Reading References

- Java SE 25 API documentation
- Dev.java learning paths
- Matching Python lesson: Lesson 10.2: Building a Text-Based Adventure Game (Part 1)

Learning Objectives

- Represent rooms and exits with a record and a map.
- Move the player by command.
- Handle impossible moves and the end of input.

Concept Explanation

A room is a record with a description and an exits map from a direction to another room ID. The player's current room is stored separately. The map contains five rooms: `start_room`, `tunnel`, `clearing`, `locked_door_room`, and `treasure_room`. In this first part, movement is unrestricted so you can verify that every exit points to a room that exists. Part 2 adds the key and locked-door rule.

Worked Example

```
import java.util.Map;
import java.util.Scanner;
```

```java
public class Lesson_10_2 {
    record Room(String description, Map<String, String> exits) {}

    public static void main(String[] args) {
        Map<String, Room> rooms = Map.of(
            "start_room", new Room(
                "A dark cave. A narrow passage leads NORTH.",
                Map.of("north", "tunnel")),
            "tunnel", new Room(
                "A damp tunnel. Passages lead SOUTH and EAST.",
                Map.of("south", "start_room", "east", "clearing")),
            "clearing", new Room(
                "A quiet clearing. A small antechamber lies to the NORTH.",
                Map.of("west", "tunnel", "north", "locked_door_room")),
            "locked_door_room", new Room(
                "A small chamber with a heavy door to the NORTH.",
                Map.of("south", "clearing")),
            "treasure_room", new Room(
                "A glittering room full of treasure. You win!",
                Map.of("south", "locked_door_room")));

        Scanner in = new Scanner(System.in);
        String current = "start_room";
        while (true) {
            System.out.println("---");
            System.out.println(rooms.get(current).description());
            if (current.equals("treasure_room")) {
                break;
            }
            System.out.println("What do you want to do?");
            String command = (in.hasNextLine() ? in.nextLine() : "quit").strip().toLowerCase
            String next = rooms.get(current).exits().get(command);
            if (next != null) {
                current = next;
            } else if (command.equals("quit")) {
                System.out.println("Exiting game. Goodbye!");
                break;
            } else {
                System.out.println("I don't understand that command or you can't go that way
            }
        }
    }
}
```

Expected Behavior or Output

With the following lines supplied as standard input (piped input is not echoed):

```
north
east
north
north
quit
```

The program prints:

```
---
A dark cave. A narrow passage leads NORTH.
What do you want to do?
---
A damp tunnel. Passages lead SOUTH and EAST.
What do you want to do?
---
A quiet clearing. A small antechamber lies to the NORTH.
What do you want to do?
---
A small chamber with a heavy door to the NORTH.
What do you want to do?
I don't understand that command or you can't go that way.
---
A small chamber with a heavy door to the NORTH.
What do you want to do?
Exiting game. Goodbye!
```

With no standard input, the program treats end of input as `quit` and exits cleanly.

Common Mistakes

1. Using a `Map` lookup result without checking for `null`, which throws `NullPointerException` for an unknown direction.
2. Forgetting to treat the end of input as `quit`, so the program loops forever when input is piped in.
3. Misspelling a room name in `exits`, so a move leads to a room that does not exist.

Activities

1. Trace the six commands by hand and write the room after each one.
2. Add a fourth room reached from the clearing and test it.
3. Add the aliases `n`, `s`, `e`, and `w`.
4. Write a loop that checks that every exit names a real room.

Accessibility Practice

Each turn prints a separator line, the description, and the prompt, so a whole turn can be read in order. Listen to one turn from start to finish to confirm that a player could follow it without seeing the screen.

Skills Checklist

- ☐ I can represent rooms with a record.
- ☐ I can move the player by command.
- ☐ I can handle an impossible move.
- ☐ I treat end of input as quit.
- ☐ I can trace a game by hand.

Lesson 10.3: Text Adventure, Part 2: Items and a Locked Door

Reading References

- Java SE 25 API documentation
- Dev.java learning paths
- Matching Python lesson: Lesson 10.3: Building a Text-Based Adventure Game (Part 2)

Learning Objectives

- Keep items in rooms and in an inventory.
- Implement take and use commands.
- Block and then allow a move through a locked door.

Concept Explanation

The completed adventure uses the same five-room graph as the Python, JavaScript, and TypeScript tracks: `start_room` leads north to `tunnel`; `tunnel` leads south to `start_room` and east to `clearing`; `clearing` leads west to `tunnel` and north to `locked_door_room`; the locked room leads south to `clearing` and north to `treasure_room`. The shiny key starts in `start_room`. The player must take it, travel to `locked_door_room`, and use it there before the north exit is allowed. Reaching `treasure_room` ends the game.

Worked Example

```
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.Scanner;
```

```java
public class Lesson_10_3 {
    record Room(String description, Map<String, String> exits, List<String> items) {}

    public static void main(String[] args) {
        Map<String, Room> rooms = Map.of(
            "start_room", new Room("A dark cave.", Map.of("north", "tunnel"),
                new ArrayList<>(List.of("shiny key"))),
            "tunnel", new Room("A damp tunnel. Passages lead SOUTH and EAST.",
                Map.of("south", "start_room", "east", "clearing"), new ArrayList<>()),
            "clearing", new Room("A quiet clearing. A small antechamber lies to the NORTH.",
                Map.of("west", "tunnel", "north", "locked_door_room"), new ArrayList<>()),
            "locked_door_room", new Room("A small chamber with a heavy door to the NORTH.",
                Map.of("south", "clearing", "north", "treasure_room"), new ArrayList<>()),
            "treasure_room", new Room("A glittering room full of treasure. You win!",
                Map.of("south", "locked_door_room"), new ArrayList<>()));

        Scanner in = new Scanner(System.in);
        String current = "start_room";
        List<String> inventory = new ArrayList<>();
        boolean doorLocked = true;

        while (true) {
            Room room = rooms.get(current);
            System.out.println();
            System.out.println(room.description());
            if (current.equals("treasure_room")) {
                break;
            }
            if (!room.items().isEmpty()) {
                System.out.println("You see: " + String.join(", ", room.items()));
            }
            System.out.println("Your inventory: " +
                (inventory.isEmpty() ? "empty" : String.join(", ", inventory)));
            System.out.println("What do you want to do?");

            String command = (in.hasNextLine() ? in.nextLine() : "quit").strip().toLowerCase
            if (command.equals("take shiny key") && room.items().contains("shiny key")) {
                room.items().remove("shiny key");
                inventory.add("shiny key");
                System.out.println("You picked up the shiny key.");
            } else if (command.equals("use shiny key") && current.equals("locked_door_room")
                    && inventory.contains("shiny key")) {
                doorLocked = false;
                System.out.println("You unlocked the door!");
            } else if (room.exits().containsKey(command)) {
```

```
                if (current.equals("locked_door_room") && command.equals("north") && doorLoc
                    System.out.println("The door is locked! You need a key.");
                } else {
                    current = room.exits().get(command);
                }
            } else if (command.equals("quit")) {
                System.out.println("You quit the adventure.");
                break;
            } else {
                System.out.println("Invalid command. Use a direction, 'take shiny key', " +
                    "'use shiny key', or 'quit'.");
            }
        }
    }
}
```

Expected Behavior or Output

With the following lines supplied as standard input (piped input is not echoed):

```
north
east
north
north
south
west
south
take shiny key
north
east
north
use shiny key
north
```

The program prints:

```
A dark cave.
You see: shiny key
Your inventory: empty
What do you want to do?

A damp tunnel. Passages lead SOUTH and EAST.
Your inventory: empty
What do you want to do?

A quiet clearing. A small antechamber lies to the NORTH.
```

```
Your inventory: empty
What do you want to do?

A small chamber with a heavy door to the NORTH.
Your inventory: empty
What do you want to do?
The door is locked! You need a key.

A small chamber with a heavy door to the NORTH.
Your inventory: empty
What do you want to do?

A quiet clearing. A small antechamber lies to the NORTH.
Your inventory: empty
What do you want to do?

A damp tunnel. Passages lead SOUTH and EAST.
Your inventory: empty
What do you want to do?

A dark cave.
You see: shiny key
Your inventory: empty
What do you want to do?
You picked up the shiny key.

A dark cave.
Your inventory: shiny key
What do you want to do?

A damp tunnel. Passages lead SOUTH and EAST.
Your inventory: shiny key
What do you want to do?

A quiet clearing. A small antechamber lies to the NORTH.
Your inventory: shiny key
What do you want to do?

A small chamber with a heavy door to the NORTH.
Your inventory: shiny key
What do you want to do?
You unlocked the door!

A small chamber with a heavy door to the NORTH.
Your inventory: shiny key
What do you want to do?
```

```
A glittering room full of treasure. You win!
```

With no standard input, the program treats end of input as `quit` and exits cleanly.

Common Mistakes

1. Making the door's locked state part of the exits so that the ”locked” message can never be reached.
2. Defining an exit to a room that is missing from the `rooms` map, which crashes when the player arrives.
3. Printing a fixed room description that still mentions an item after the player has taken it.

Activities

1. Play the program with the shown commands and follow each turn on paper.
2. Try `use shiny key` in the cave and describe what happens.
3. Add a second item and require both to open the door.
4. List the game's state (location, inventory, door) and say which command changes each part.

Accessibility Practice

The room description never mentions the key. A separate 'You see:' line lists items, so the description stays true after the key is taken and the screen reader announces the current state.

Skills Checklist

- ☐ I can store items in rooms and an inventory.
- ☐ I can implement take and use commands.
- ☐ I can block and allow a move.
- ☐ I can list the game's state.
- ☐ I can keep descriptions true after state changes.

Lesson 10.4: Command-Line To-Do List

Reading References

- Java SE 25 API documentation
- Dev.java learning paths
- Matching Python lesson: Lesson 10.4: Building a Command-Line To-Do List

Learning Objectives

- Keep records with the shared fields.
- Save and load them as JSON.
- Offer add, view, mark-done, save, load, and quit commands.

Concept Explanation

A to-do list ties together records, loops, methods, and files. Each task has the shared fields `id`, `title`, `description`, `done`, and `created_at`. The program keeps a list of tasks in memory, shows a numbered menu, and saves the list to `todos.json` when asked and on exit.

Saving uses the teaching JSON helper from Lesson 8.5, included here so the example runs on its own. At start-up the program loads the file if it exists. A missing file is normal on first use, and a damaged file must not crash the program: it prints a message and starts with an empty list.

New ids are one more than the highest existing id. The status is shown as the words `done` and `not done`, never with a symbol or color alone.

In the sample session the user adds two tasks, views the list, marks task 2 done, views it again, tries an invalid menu choice, views the list, and quits.

Worked Example

```
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.*;

public class Lesson_10_4 {
    static final class Json {
        private Json() {}

        static String encode(Object value) {
            if (value == null) {
                return "null";
            }
            if (value instanceof String text) {
                return quote(text);
            }
            if (value instanceof Boolean || value instanceof Integer || value instanceof Lo
                return value.toString();
            }
            if (value instanceof Map<?, ?> map) {
```

```
            List<String> keys = new ArrayList<>();
            for (Object key : map.keySet()) {
                keys.add(key.toString());
            }
            Collections.sort(keys);
            StringBuilder out = new StringBuilder("{");
            for (int i = 0; i < keys.size(); i++) {
                if (i > 0) {
                    out.append(',');
                }
                out.append(quote(keys.get(i))).append(':').append(encode(map.get(keys.ge
            }
            return out.append('}').toString();
        }
        if (value instanceof List<?> list) {
            StringBuilder out = new StringBuilder("[");
            for (int i = 0; i < list.size(); i++) {
                if (i > 0) {
                    out.append(',');
                }
                out.append(encode(list.get(i)));
            }
            return out.append(']').toString();
        }
        throw new IllegalArgumentException("cannot encode " + value.getClass().getSimple
    }

    private static String quote(String text) {
        StringBuilder out = new StringBuilder("\"");
        for (char c : text.toCharArray()) {
            switch (c) {
                case '"' -> out.append("\\\"");
                case '\\' -> out.append("\\\\");
                case '\n' -> out.append("\\n");
                case '\r' -> out.append("\\r");
                case '\t' -> out.append("\\t");
                default -> {
                    if (c < ' ') {
                        out.append(String.format("\\u%04x", (int) c));
                    } else {
                        out.append(c);
                    }
                }
            }
        }
        return out.append('"').toString();
```

```
}

static Object decode(String text) {
    Parser parser = new Parser(text);
    Object result = parser.value();
    parser.skip();
    if (parser.pos < text.length()) {
        throw parser.fail("unexpected extra text");
    }
    return result;
}

private static final class Parser {
    private final String text;
    private int pos = 0;

    Parser(String text) {
        this.text = text;
    }

    IllegalArgumentException fail(String message) {
        return new IllegalArgumentException(message + " at position " + pos);
    }

    void skip() {
        while (pos < text.length() && Character.isWhitespace(text.charAt(pos))) {
            pos++;
        }
    }

    Object value() {
        skip();
        if (pos >= text.length()) {
            throw fail("unexpected end of text");
        }
        char c = text.charAt(pos);
        if (c == '{') return object();
        if (c == '[') return list();
        if (c == '"') return string();
        if (text.startsWith("true", pos)) { pos += 4; return Boolean.TRUE; }
        if (text.startsWith("false", pos)) { pos += 5; return Boolean.FALSE; }
        if (text.startsWith("null", pos)) { pos += 4; return null; }
        if (c == '-' || Character.isDigit(c)) return number();
        throw fail("unexpected character");
    }
```

```
Object number() {
    int start = pos;
    if (text.charAt(pos) == '-') {
        pos++;
    }
    while (pos < text.length() && (Character.isDigit(text.charAt(pos)) || ".eE+-
        pos++;
    }
    String token = text.substring(start, pos);
    try {
        return Long.parseLong(token);
    } catch (NumberFormatException notWhole) {
        try {
            return Double.parseDouble(token);
        } catch (NumberFormatException notNumber) {
            throw fail("bad number");
        }
    }
}

String string() {
    pos++;
    StringBuilder out = new StringBuilder();
    while (true) {
        if (pos >= text.length()) {
            throw fail("unterminated string");
        }
        char c = text.charAt(pos++);
        if (c == '"') {
            return out.toString();
        }
        if (c != '\\') {
            out.append(c);
            continue;
        }
        if (pos >= text.length()) {
            throw fail("bad escape");
        }
        char e = text.charAt(pos++);
        switch (e) {
            case 'n' -> out.append('\n');
            case 't' -> out.append('\t');
            case 'r' -> out.append('\r');
            case '"' -> out.append('"');
            case '\\' -> out.append('\\');
            case '/' -> out.append('/');
```

```
                case 'u' -> {
                    if (pos + 4 > text.length()) {
                        throw fail("bad escape");
                    }
                    out.append((char) Integer.parseInt(text.substring(pos, pos + 4),
                    pos += 4;
                }
                default -> throw fail("bad escape");
            }
        }
    }

    Map<String, Object> object() {
        pos++;
        Map<String, Object> map = new LinkedHashMap<>();
        skip();
        if (pos < text.length() && text.charAt(pos) == '}') {
            pos++;
            return map;
        }
        while (true) {
            skip();
            if (pos >= text.length() || text.charAt(pos) != '"') {
                throw fail("expected a key");
            }
            String key = string();
            skip();
            if (pos >= text.length() || text.charAt(pos) != ':') {
                throw fail("expected ':'");
            }
            pos++;
            map.put(key, value());
            skip();
            if (pos >= text.length()) {
                throw fail("unterminated object");
            }
            char d = text.charAt(pos++);
            if (d == '}') {
                return map;
            }
            if (d != ',') {
                pos--;
                throw fail("expected ',' or '}'");
            }
        }
    }
```

```
        List<Object> list() {
            pos++;
            List<Object> items = new ArrayList<>();
            skip();
            if (pos < text.length() && text.charAt(pos) == ']') {
                pos++;
                return items;
            }
            while (true) {
                items.add(value());
                skip();
                if (pos >= text.length()) {
                    throw fail("unterminated list");
                }
                char d = text.charAt(pos++);
                if (d == ']') {
                    return items;
                }
                if (d != ',') {
                    pos--;
                    throw fail("expected ',' or ']'");
                }
            }
        }
    }
}

static final Path TODO_FILE = Path.of("todos.json");
static List<Map<String, Object>> todos = new ArrayList<>();

static void loadTodos() throws IOException {
    if (!Files.exists(TODO_FILE)) {
        System.out.println("No existing tasks found.");
        todos = new ArrayList<>();
        return;
    }
    try {
        List<Map<String, Object>> loaded = new ArrayList<>();
        for (Object entry : (List<?>) Json.decode(Files.readString(TODO_FILE))) {
            Map<String, Object> task = new LinkedHashMap<>();
            for (Map.Entry<?, ?> field : ((Map<?, ?>) entry).entrySet()) {
                task.put(field.getKey().toString(), field.getValue());
            }
            loaded.add(task);
        }
```

```
            todos = loaded;
            System.out.println("Tasks loaded.");
        } catch (IllegalArgumentException | ClassCastException e) {
            System.out.println("Error: todos.json is not valid JSON. Starting with an empty
            todos = new ArrayList<>();
        }
    }

    static void saveTodos() throws IOException {
        Files.writeString(TODO_FILE, Json.encode(todos));
        System.out.println("Tasks saved.");
    }

    static long nextId() {
        long highest = 0;
        for (Map<String, Object> task : todos) {
            highest = Math.max(highest, (Long) task.get("id"));
        }
        return highest + 1;
    }

    static void addTask(String title, String description) {
        if (title.isBlank()) {
            System.out.println("A title is required.");
            return;
        }
        Map<String, Object> task = new LinkedHashMap<>();
        task.put("id", nextId());
        task.put("title", title.strip());
        task.put("description", description.strip());
        task.put("done", false);
        task.put("created_at", LocalDateTime.now().format(DateTimeFormatter.ofPattern("yyyy-
        todos.add(task);
        System.out.println("Task '" + title.strip() + "' added.");
    }

    static void viewTasks() {
        if (todos.isEmpty()) {
            System.out.println("No tasks in your list.");
            return;
        }
        System.out.println("--- Your To-Do List ---");
        for (Map<String, Object> task : todos) {
            String status = Boolean.TRUE.equals(task.get("done")) ? "done" : "not done";
            System.out.println(task.get("id") + ". " + task.get("title") + " (" + status + '
        }
```

```
        System.out.println("----------------------");
    }

    static void markDone(String text) {
        long id;
        try {
            id = Long.parseLong(text.trim());
        } catch (NumberFormatException e) {
            System.out.println("Please enter a whole number.");
            return;
        }
        for (Map<String, Object> task : todos) {
            if (task.get("id").equals(id)) {
                task.put("done", true);
                System.out.println("Task '" + task.get("title") + "' marked as done.");
                return;
            }
        }
        System.out.println("No task has that number.");
    }

    static void showMenu() {
        System.out.println();
        System.out.println("--- To-Do List Menu ---");
        System.out.println("1. Add Task");
        System.out.println("2. View Tasks");
        System.out.println("3. Mark Task Done");
        System.out.println("4. Save Tasks");
        System.out.println("5. Load Tasks");
        System.out.println("6. Quit");
        System.out.println("----------------------");
    }

    static String next(Scanner in, String fallback) {
        return in.hasNextLine() ? in.nextLine() : fallback;
    }

    public static void main(String[] args) throws IOException {
        Scanner in = new Scanner(System.in);
        loadTodos();
        while (true) {
            showMenu();
            System.out.println("Enter your choice (1-6):");
            String choice = next(in, "6");
            switch (choice) {
                case "1" -> {
```

```
                    System.out.println("Enter task title:");
                    String title = next(in, "");
                    System.out.println("Enter task description:");
                    String description = next(in, "");
                    addTask(title, description);
                }
                case "2" -> viewTasks();
                case "3" -> {
                    viewTasks();
                    System.out.println("Enter task number to mark done:");
                    markDone(next(in, ""));
                }
                case "4" -> saveTodos();
                case "5" -> loadTodos();
                case "6" -> {
                    saveTodos();
                    System.out.println("Goodbye!");
                    return;
                }
                default -> System.out.println("Invalid choice. Please try again.");
            }
        }
    }
}
```

Expected Behavior or Output

With the following lines typed as standard input (piped input is not echoed, so it does not appear in the output):

```
1
Study programming
Finish Unit 10
1
Review scores
Check the four scores
2
3
2
3
7
2
6
```

the program prints:

```
No existing tasks found.
```

```
--- To-Do List Menu ---
1. Add Task
2. View Tasks
3. Mark Task Done
4. Save Tasks
5. Load Tasks
6. Quit
----------------------
Enter your choice (1-6):
Enter task title:
Enter task description:
Task 'Study programming' added.

--- To-Do List Menu ---
1. Add Task
2. View Tasks
3. Mark Task Done
4. Save Tasks
5. Load Tasks
6. Quit
----------------------
Enter your choice (1-6):
Enter task title:
Enter task description:
Task 'Review scores' added.

--- To-Do List Menu ---
1. Add Task
2. View Tasks
3. Mark Task Done
4. Save Tasks
5. Load Tasks
6. Quit
----------------------
Enter your choice (1-6):
--- Your To-Do List ---
1. Study programming (not done)
2. Review scores (not done)
----------------------

--- To-Do List Menu ---
1. Add Task
2. View Tasks
3. Mark Task Done
4. Save Tasks
```

```
5. Load Tasks
6. Quit
----------------------
Enter your choice (1-6):
--- Your To-Do List ---
1. Study programming (not done)
2. Review scores (not done)
----------------------
Enter task number to mark done:
Task 'Review scores' marked as done.

--- To-Do List Menu ---
1. Add Task
2. View Tasks
3. Mark Task Done
4. Save Tasks
5. Load Tasks
6. Quit
----------------------
Enter your choice (1-6):
--- Your To-Do List ---
1. Study programming (not done)
2. Review scores (done)
----------------------
Enter task number to mark done:
No task has that number.

--- To-Do List Menu ---
1. Add Task
2. View Tasks
3. Mark Task Done
4. Save Tasks
5. Load Tasks
6. Quit
----------------------
Enter your choice (1-6):
--- Your To-Do List ---
1. Study programming (not done)
2. Review scores (done)
----------------------

--- To-Do List Menu ---
1. Add Task
2. View Tasks
3. Mark Task Done
4. Save Tasks
```

```
5. Load Tasks
6. Quit
----------------------
Enter your choice (1-6):
Tasks saved.
Goodbye!
```

With no standard input at all, the program ends cleanly and prints:

```
No existing tasks found.

--- To-Do List Menu ---
1. Add Task
2. View Tasks
3. Mark Task Done
4. Save Tasks
5. Load Tasks
6. Quit
----------------------
Enter your choice (1-6):
Tasks saved.
Goodbye!
```

Common Mistakes

1. Saving the file but never loading it, so tasks disappear between runs.
2. Reusing a deleted task's id, which can confuse saved references.
3. Letting a damaged `todos.json` crash the program instead of printing a message.

Activities

1. Run the sample session, then run the program again typing `2` and `6` to see the saved tasks.
2. Add a command to delete a task by number.
3. Open `todos.json` and read the saved text. Edit it to remove a bracket and run the program again.
4. Show each task's description in the view command.

Accessibility Practice

Every menu is printed as numbered lines and every prompt is on its own line. The status words are `done` and `not done`, not symbols, so the list reads clearly in speech and braille.

Skills Checklist

- ☐ I can keep records with the shared fields.

- ☐ I can save and load JSON.
- ☐ I handle a missing or damaged file.
- ☐ I keep ids unique.
- ☐ I show status in words.

Lesson 10.5: User Experience and Input Validation

Reading References

- Java SE 25 API documentation
- Dev.java learning paths
- Matching Python lesson: Lesson 10.5: Refining User Experience and Validation

Learning Objectives

- Write reusable validation methods.
- Limit retries and handle the end of input.
- Write clear, accessible messages.

Concept Explanation

Reusable validation methods make every menu and form consistent. `askNumber(in, prompt, low, high, attempts)` prints a prompt, reads a line, converts it, checks the range, and either returns the number or explains the problem and asks again. After the allowed attempts, or at the end of input, it returns `null`. `askYesNo(in, prompt)` accepts `y`, `yes`, `n`, and `no` in any letter case. Returning the wrapper types `Integer` and `Boolean` lets `null` mean "no answer".

Good messages say *what was wrong* and *what to do*: "Enter a whole number from 0 to 100," not "Invalid." They never depend on color, position, or symbols alone, and they use the same wording every time.

The example reads a score with the typed lines abc, 105, and 88, then asks a yes or no question with the lines maybe and y.

Worked Example

```
import java.util.Scanner;

public class Lesson_10_5 {
    static Integer askNumber(Scanner in, String prompt, int low, int high, int attempts) {
        for (int attempt = 1; attempt <= attempts; attempt++) {
            System.out.println(prompt);
            if (!in.hasNextLine()) {
                return null;
```

```
            }
            String text = in.nextLine().trim();
            try {
                int number = Integer.parseInt(text);
                if (number >= low && number <= high) {
                    return number;
                }
            } catch (NumberFormatException e) {
                // fall through to the message below
            }
            System.out.println("Enter a whole number from " + low + " to " + high + ". You h
                + (attempts - attempt) + " attempt(s) left.");
        }
        return null;
    }

    static Boolean askYesNo(Scanner in, String prompt) {
        while (true) {
            System.out.println(prompt + " (yes or no)");
            if (!in.hasNextLine()) {
                return null;
            }
            String text = in.nextLine().trim().toLowerCase();
            if (text.equals("y") || text.equals("yes")) {
                return true;
            }
            if (text.equals("n") || text.equals("no")) {
                return false;
            }
            System.out.println("Please answer yes or no.");
        }
    }

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        Integer score = askNumber(in, "Enter a score from 0 to 100:", 0, 100, 3);
        if (score != null) {
            System.out.println("Accepted score: " + score);
        } else {
            System.out.println("No valid score was entered.");
        }

        Boolean keep = askYesNo(in, "Save this score?");
        System.out.println("Save answer: " + keep);
    }
}
```

Expected Behavior or Output

With the following lines typed as standard input (piped input is not echoed, so it does not appear in the output):

```
abc
105
88
maybe
y
```

the program prints:

```
Enter a score from 0 to 100:
Enter a whole number from 0 to 100. You have 2 attempt(s) left.
Enter a score from 0 to 100:
Enter a whole number from 0 to 100. You have 1 attempt(s) left.
Enter a score from 0 to 100:
Accepted score: 88
Save this score? (yes or no)
Please answer yes or no.
Save this score? (yes or no)
Save answer: true
```

With no standard input at all, the program ends cleanly and prints:

```
Enter a score from 0 to 100:
No valid score was entered.
Save this score? (yes or no)
Save answer: null
```

Common Mistakes

1. Writing the message "Invalid" without saying what is valid.
2. Accepting a decimal such as 88.5 for a field that requires a whole number.
3. Looping forever on bad input instead of limiting attempts.

Activities

1. Run the sample input and read each message aloud.
2. Run with three invalid numbers and read the final message.
3. Add `askText(in, prompt)` that rejects empty answers.
4. Rewrite two messages from earlier lessons so they say what to do next.

Accessibility Practice

Check that every error message ends with an action the user can take, and that the same situation always produces the same words.

Skills Checklist

- ☐ I can write reusable validation methods.
- ☐ I limit retries.
- ☐ I handle the end of input.
- ☐ I write messages that say what to do.
- ☐ I keep wording consistent.

8.13 Unit 11: Capstone — Accessible Personal Data Manager

Overview

This unit develops capstone — accessible personal data manager through the same shared problem contexts and assessment expectations used across the general-purpose language tracks, expressed with Java 25 syntax, APIs, and static typing.

Unit objectives: explain the core concepts, write and test complete Java programs, diagnose errors, and apply the ideas to accessible projects.

Lesson 11.1: Capstone: Requirements and Design

Reading References

- Java SE 25 API documentation
- Dev.java learning paths
- Matching Python lesson: Lesson 11.1: Introduction to NiceGUI (capstone planning)

Learning Objectives

- Write requirements for a personal data manager.
- Define the shared record schema as a Java record.
- Check records against the validation rules before building features.

Concept Explanation

The capstone is a *personal data manager*: a program that keeps a list of records, lets the user add, view, search, change, complete, and delete them, and saves them to a file. In the Java track it is a text program, so it runs from the terminal and is fully usable with a screen reader or braille display.

Shared record schema (all general-purpose language tracks). Every record has `id` (unique whole number), `title`, `description`, `category`, `created_at`, and `updated_at` (ISO 8601 text). A project may add fields, such

as an optional `done` flag, but must not remove these. Java names use camel case (`createdAt`), and the stored JSON uses the shared names. The class is called `Entry` here, because `Record` would hide `java.lang.Record`, the parent of all records.

Validation rules. A record must have a non-empty title and a non-empty category. Timestamps are ISO 8601 text such as `2026-01-15T09:00:00`.

Design before coding: list the operations, then the data, then the messages. The example checks the three shared sample records against the rules and prints them as a plain-text list.

Worked Example

```
import java.util.List;

public class Lesson_11_1 {
    record Entry(int id, String title, String description, String category, String createdAt

    static String validate(String title, String category) {
        if (title == null || title.isBlank()) {
            return "A title is required.";
        }
        if (category == null || category.isBlank()) {
            return "A category is required.";
        }
        return null;
    }

    public static void main(String[] args) {
        List<Entry> sample = List.of(
            new Entry(1, "Notebook", "Course notes", "supplies", "2026-01-15T09:00:00", "202
            new Entry(2, "Pencil", "Box of 10", "supplies", "2026-01-15T09:05:00", "2026-01-
            new Entry(3, "Study plan", "Weekly schedule", "planning", "2026-01-15T09:10:00",

        for (Entry entry : sample) {
            String problem = validate(entry.title(), entry.category());
            System.out.println(entry.id() + ". " + entry.title() + " [" + entry.category() +
                + (problem == null ? "valid" : problem));
        }

        System.out.println(validate("  ", "supplies"));
        System.out.println(validate("Ruler", ""));
    }
}
```

Expected Behavior or Output

The program prints:

```
1. Notebook [supplies] - valid
2. Pencil [supplies] - valid
3. Study plan [planning] - valid
A title is required.
A category is required.
```

Common Mistakes

1. Starting to build features before agreeing on the record fields.
2. Allowing a record with an empty title because the check was left for later.
3. Storing timestamps in different formats in different places.

Activities

1. Write the seven operations as a numbered list of requirements.
2. Add one project-specific field and say how it is validated.
3. Write three sample records for your own project.
4. Describe in plain sentences what each shared field means.

Accessibility Practice

Design the screen output as lines of plain text: one record per line, with the same field order each time. Decide the wording of every message now so that it can be tested.

Skills Checklist

☐ I can write requirements.
☐ I know the shared record fields.
☐ I can write validation rules.
☐ I can describe sample data.
☐ I can plan messages before coding.

Lesson 11.2: Capstone: Data Model and Storage

Reading References

- Java SE 25 API documentation
- Dev.java learning paths
- Matching Python lesson: Lesson 11.2: Data Model and Storage

Learning Objectives

- Store records as record objects.
- Save and load them as JSON.
- Confirm that every field survives a round trip.

Concept Explanation

Records are stored as a list of Java records in memory and as a JSON array on disk. The storage layer has two methods: `saveEntries(path, entries)` and `loadEntries(path)`. Loading returns a small record holding the entries and a message, so the caller decides what to do when the file is missing or damaged.

An `Entry` is converted to a map for saving (`toMap`) and rebuilt from a decoded map when loading (`toEntry`), which checks that each field is present and has the right type. The optional `done` field defaults to `false`.

The example uses the teaching JSON helper from Lesson 8.5 (included so the program runs on its own). After saving and reloading it compares the lists; records compare by value, so this *round-trip test* checks every field of every entry. It is the first acceptance test for storage.

Worked Example

```
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.*;

public class Lesson_11_2 {
    static final class Json {
        private Json() {}

        static String encode(Object value) {
            if (value == null) {
                return "null";
            }
            if (value instanceof String text) {
                return quote(text);
            }
            if (value instanceof Boolean || value instanceof Integer || value instanceof Lo
                return value.toString();
            }
            if (value instanceof Map<?, ?> map) {
                List<String> keys = new ArrayList<>();
                for (Object key : map.keySet()) {
                    keys.add(key.toString());
                }
```

```
            Collections.sort(keys);
            StringBuilder out = new StringBuilder("{");
            for (int i = 0; i < keys.size(); i++) {
                if (i > 0) {
                    out.append(',');
                }
                out.append(quote(keys.get(i))).append(':').append(encode(map.get(keys.ge
            }
            return out.append('}').toString();
        }
        if (value instanceof List<?> list) {
            StringBuilder out = new StringBuilder("[");
            for (int i = 0; i < list.size(); i++) {
                if (i > 0) {
                    out.append(',');
                }
                out.append(encode(list.get(i)));
            }
            return out.append(']').toString();
        }
        throw new IllegalArgumentException("cannot encode " + value.getClass().getSimple
    }

    private static String quote(String text) {
        StringBuilder out = new StringBuilder("\"");
        for (char c : text.toCharArray()) {
            switch (c) {
                case '"' -> out.append("\\\"");
                case '\\' -> out.append("\\\\");
                case '\n' -> out.append("\\n");
                case '\r' -> out.append("\\r");
                case '\t' -> out.append("\\t");
                default -> {
                    if (c < ' ') {
                        out.append(String.format("\\u%04x", (int) c));
                    } else {
                        out.append(c);
                    }
                }
            }
        }
        return out.append('"').toString();
    }

    static Object decode(String text) {
        Parser parser = new Parser(text);
```

```
        Object result = parser.value();
        parser.skip();
        if (parser.pos < text.length()) {
            throw parser.fail("unexpected extra text");
        }
        return result;
    }

    private static final class Parser {
        private final String text;
        private int pos = 0;

        Parser(String text) {
            this.text = text;
        }

        IllegalArgumentException fail(String message) {
            return new IllegalArgumentException(message + " at position " + pos);
        }

        void skip() {
            while (pos < text.length() && Character.isWhitespace(text.charAt(pos))) {
                pos++;
            }
        }

        Object value() {
            skip();
            if (pos >= text.length()) {
                throw fail("unexpected end of text");
            }
            char c = text.charAt(pos);
            if (c == '{') return object();
            if (c == '[') return list();
            if (c == '"') return string();
            if (text.startsWith("true", pos)) { pos += 4; return Boolean.TRUE; }
            if (text.startsWith("false", pos)) { pos += 5; return Boolean.FALSE; }
            if (text.startsWith("null", pos)) { pos += 4; return null; }
            if (c == '-' || Character.isDigit(c)) return number();
            throw fail("unexpected character");
        }

        Object number() {
            int start = pos;
            if (text.charAt(pos) == '-') {
                pos++;
```

```
        }
        while (pos < text.length() && (Character.isDigit(text.charAt(pos)) || ".eE+-
            pos++;
        }
        String token = text.substring(start, pos);
        try {
            return Long.parseLong(token);
        } catch (NumberFormatException notWhole) {
            try {
                return Double.parseDouble(token);
            } catch (NumberFormatException notNumber) {
                throw fail("bad number");
            }
        }
    }

    String string() {
        pos++;
        StringBuilder out = new StringBuilder();
        while (true) {
            if (pos >= text.length()) {
                throw fail("unterminated string");
            }
            char c = text.charAt(pos++);
            if (c == '"') {
                return out.toString();
            }
            if (c != '\\') {
                out.append(c);
                continue;
            }
            if (pos >= text.length()) {
                throw fail("bad escape");
            }
            char e = text.charAt(pos++);
            switch (e) {
                case 'n' -> out.append('\n');
                case 't' -> out.append('\t');
                case 'r' -> out.append('\r');
                case '"' -> out.append('"');
                case '\\' -> out.append('\\');
                case '/' -> out.append('/');
                case 'u' -> {
                    if (pos + 4 > text.length()) {
                        throw fail("bad escape");
                    }
```

```
                    out.append((char) Integer.parseInt(text.substring(pos, pos + 4),
                    pos += 4;
                }
                default -> throw fail("bad escape");
            }
        }
    }

    Map<String, Object> object() {
        pos++;
        Map<String, Object> map = new LinkedHashMap<>();
        skip();
        if (pos < text.length() && text.charAt(pos) == '}') {
            pos++;
            return map;
        }
        while (true) {
            skip();
            if (pos >= text.length() || text.charAt(pos) != '"') {
                throw fail("expected a key");
            }
            String key = string();
            skip();
            if (pos >= text.length() || text.charAt(pos) != ':') {
                throw fail("expected ':'");
            }
            pos++;
            map.put(key, value());
            skip();
            if (pos >= text.length()) {
                throw fail("unterminated object");
            }
            char d = text.charAt(pos++);
            if (d == '}') {
                return map;
            }
            if (d != ',') {
                pos--;
                throw fail("expected ',' or '}'");
            }
        }
    }

    List<Object> list() {
        pos++;
        List<Object> items = new ArrayList<>();
```

```
            skip();
            if (pos < text.length() && text.charAt(pos) == ']') {
                pos++;
                return items;
            }
            while (true) {
                items.add(value());
                skip();
                if (pos >= text.length()) {
                    throw fail("unterminated list");
                }
                char d = text.charAt(pos++);
                if (d == ']') {
                    return items;
                }
                if (d != ',') {
                    pos--;
                    throw fail("expected ',' or ']'");
                }
            }
        }
    }
}

record Entry(int id, String title, String description, String category,
             String createdAt, String updatedAt, boolean done) {}

record Loaded(List<Entry> entries, String problem) {}

static Map<String, Object> toMap(Entry entry) {
    Map<String, Object> map = new LinkedHashMap<>();
    map.put("id", entry.id());
    map.put("title", entry.title());
    map.put("description", entry.description());
    map.put("category", entry.category());
    map.put("created_at", entry.createdAt());
    map.put("updated_at", entry.updatedAt());
    map.put("done", entry.done());
    return map;
}

static String text(Map<?, ?> map, String key) {
    if (!(map.get(key) instanceof String value)) {
        throw new IllegalArgumentException("missing field " + key);
    }
    return value;
```

```
}

static Entry toEntry(Object raw) {
    if (!(raw instanceof Map<?, ?> map)) {
        throw new IllegalArgumentException("a record must be an object");
    }
    if (!(map.get("id") instanceof Long id)) {
        throw new IllegalArgumentException("missing field id");
    }
    boolean done = map.get("done") instanceof Boolean flag && flag;
    return new Entry(id.intValue(), text(map, "title"), text(map, "description"), text(m
        text(map, "created_at"), text(map, "updated_at"), done);
}

static void saveEntries(Path path, List<Entry> entries) throws IOException {
    List<Object> maps = new ArrayList<>();
    for (Entry entry : entries) {
        maps.add(toMap(entry));
    }
    Files.writeString(path, Json.encode(maps));
}

static Loaded loadEntries(Path path) throws IOException {
    if (!Files.exists(path)) {
        return new Loaded(List.of(), "Could not open " + path + ": file not found.");
    }
    try {
        List<Entry> entries = new ArrayList<>();
        for (Object raw : (List<?>) Json.decode(Files.readString(path))) {
            entries.add(toEntry(raw));
        }
        return new Loaded(entries, null);
    } catch (IllegalArgumentException | ClassCastException e) {
        return new Loaded(List.of(), "The data file is not valid: " + e.getMessage());
    }
}

public static void main(String[] args) throws IOException {
    List<Entry> sample = List.of(
        new Entry(1, "Notebook", "Course notes", "supplies", "2026-01-15T09:00:00", "202
        new Entry(2, "Pencil", "Box of 10", "supplies", "2026-01-15T09:05:00", "2026-01-
        new Entry(3, "Study plan", "Weekly schedule", "planning", "2026-01-15T09:10:00",

    Path file = Path.of("records.json");
    saveEntries(file, sample);
    Loaded loaded = loadEntries(file);
```

```
        System.out.println("Round trip preserved all fields: " + loaded.entries().equals(sa
        System.out.println("Records loaded: " + loaded.entries().size());
        System.out.println(loaded.problem());

        System.out.println(loadEntries(Path.of("missing.json")).problem());

        Files.writeString(Path.of("broken.json"), "[{\"id\": 1,");
        System.out.println(loadEntries(Path.of("broken.json")).problem());
    }
}
```

Expected Behavior or Output

The program prints:

```
Round trip preserved all fields: true
Records loaded: 3
null
Could not open missing.json: file not found.
The data file is not valid: expected a key at position 10
```

Common Mistakes

1. Overwriting the data file with an empty list after a failed load.
2. Saving only some of the fields.
3. Assuming a loaded record has every field without checking.

Activities

1. Run the program and open `records.json` to read the saved text.
2. Add a field to `Entry`, save it, and confirm that it survives the round trip.
3. Make `toEntry` reject a record whose title is blank.
4. Explain why `loadEntries` returns a message and does not throw.

Accessibility Practice

Messages that name the file and the problem read clearly in speech. After saving, read the file in the terminal to confirm it contains what you expect.

Skills Checklist

- ☐ I can store records in record objects.
- ☐ I can save and load JSON.
- ☐ I can run a round-trip test.
- ☐ I handle missing and damaged files.
- ☐ I do not overwrite good data after a failed load.

Lesson 11.3: Capstone: Core Operations

Reading References

- Java SE 25 API documentation
- Dev.java learning paths
- Matching Python lesson: Lesson 11.3: Displaying and Managing Items

Learning Objectives

- Add, list, search, change, complete, and delete records.
- Assign new ids and update timestamps correctly.
- Return a message when an operation cannot be done.

Concept Explanation

A `Manager` class holds the entries and offers one method per operation. Each method that can fail returns an `Outcome` (the entry and a message) or a message, so the user interface can show it and decide what to do. New ids are one more than the highest id, so ids are never reused while higher ids exist. Updating an entry sets `updatedAt` but never changes `createdAt`, which is `final`. Completing an entry sets the optional `done` field to `true` and counts as an update.

For repeatable output the example passes in a *fake clock* that returns a new fixed time on each call. A real program passes a supplier that reads the real time.

The example adds the three shared sample records, lists them, searches for "note", changes one description, completes one entry, deletes one, and tries to delete an entry that does not exist.

Worked Example

```
import java.util.*;
import java.util.function.Supplier;

class Entry {
    final int id;
    String title;
    String description;
    String category;
    final String createdAt;
    String updatedAt;
    boolean done;

    Entry(int id, String title, String description, String category, String createdAt) {
        this.id = id;
        this.title = title;
```

```
        this.description = description;
        this.category = category;
        this.createdAt = createdAt;
        this.updatedAt = createdAt;
        this.done = false;
    }

    String describe() {
        return id + ". " + title + " [" + category + "] (" + (done ? "done" : "not done") +
    }
}

record Outcome(Entry entry, String message) {}

class Clocks {
    static Supplier<String> fake() {
        int[] ticks = {0};
        return () -> {
            ticks[0]++;
            return String.format("2026-01-15T10:00:%02d", ticks[0]);
        };
    }
}

class Manager {
    final List<Entry> entries = new ArrayList<>();
    private final Supplier<String> clock;

    Manager(Supplier<String> clock) {
        this.clock = clock;
    }

    static String validate(String title, String category) {
        if (title == null || title.isBlank()) {
            return "A title is required.";
        }
        if (category == null || category.isBlank()) {
            return "A category is required.";
        }
        return null;
    }

    private int nextId() {
        int highest = 0;
        for (Entry entry : entries) {
            highest = Math.max(highest, entry.id);
```

```
    }
    return highest + 1;
}

Entry find(int id) {
    for (Entry entry : entries) {
        if (entry.id == id) {
            return entry;
        }
    }
    return null;
}

Outcome add(String title, String description, String category) {
    String problem = validate(title, category);
    if (problem != null) {
        return new Outcome(null, problem);
    }
    Entry entry = new Entry(nextId(), title.strip(), description.strip(), category.strip
    entries.add(entry);
    return new Outcome(entry, null);
}

Outcome update(int id, String title, String description, String category, Boolean done)
    Entry entry = find(id);
    if (entry == null) {
        return new Outcome(null, "No record has that id.");
    }
    String problem = validate(title != null ? title : entry.title, category != null ? ca
    if (problem != null) {
        return new Outcome(null, problem);
    }
    if (title != null) entry.title = title.strip();
    if (description != null) entry.description = description.strip();
    if (category != null) entry.category = category.strip();
    if (done != null) entry.done = done;
    entry.updatedAt = clock.get();
    return new Outcome(entry, null);
}

Outcome complete(int id) {
    return update(id, null, null, null, true);
}

String delete(int id) {
    Entry entry = find(id);
```

```
        if (entry == null) {
            return "No record has that id.";
        }
        entries.remove(entry);
        return null;
    }

    List<Entry> search(String text) {
        String needle = text.strip().toLowerCase();
        List<Entry> found = new ArrayList<>();
        for (Entry entry : entries) {
            if (entry.title.toLowerCase().contains(needle) || entry.description.toLowerCase(
                found.add(entry);
            }
        }
        return found;
    }
}

public class Lesson_11_3 {
    static void show(Manager manager) {
        for (Entry entry : manager.entries) {
            System.out.println(entry.describe());
        }
    }

    public static void main(String[] args) {
        Manager manager = new Manager(Clocks.fake());
        manager.add("Notebook", "Course notes", "supplies");
        manager.add("Pencil", "Box of 10", "supplies");
        manager.add("Study plan", "Weekly schedule", "planning");
        show(manager);

        System.out.println("Search for 'note':");
        for (Entry entry : manager.search("note")) {
            System.out.println(entry.title);
        }

        manager.update(2, null, "Box of 12", null, null);
        manager.complete(3);
        System.out.println(manager.find(2).description + " " + manager.find(2).updatedAt);

        String first = manager.delete(1);
        System.out.println(first == null ? "Deleted record 1." : first);
        String second = manager.delete(99);
        System.out.println(second == null ? "Deleted record 99." : second);
```

```
        show(manager);
    }
}
```

Expected Behavior or Output

The program prints:

```
1. Notebook [supplies] (not done)
2. Pencil [supplies] (not done)
3. Study plan [planning] (not done)
Search for 'note':
Notebook
Box of 12 2026-01-15T10:00:04
Deleted record 1.
No record has that id.
2. Pencil [supplies] (not done)
3. Study plan [planning] (done)
```

Common Mistakes

1. Reusing a deleted entry's id by counting the list size instead of using the highest id.
2. Changing `createdAt` when updating (the field is `final` to prevent this).
3. Throwing an exception for a missing record where a message to display is more helpful.

Activities

1. Predict the output before running.
2. Add `listByCategory(category)` to `Manager`.
3. Add a rule that blocks completing an entry that is already complete, with a message.
4. Explain how the fake clock makes the output repeatable.

Accessibility Practice

List output uses the same fixed format for every record and the words `done` and `not done`. Search results print only titles, so each line is short.

Skills Checklist

- ☐ I can implement add, list, search, change, complete, and delete.
- ☐ I assign unique ids.
- ☐ I update timestamps correctly.
- ☐ I return clear messages for failures.
- ☐ I can make output repeatable with a fake clock.

Lesson 11.4: Capstone: Validation and Acceptance Tests

Reading References

- Java SE 25 API documentation
- Dev.java learning paths
- Matching Python lesson: Lesson 11.4: Testing and Refinement

Learning Objectives

- Run the shared acceptance tests.
- Verify that validation rejects bad data.
- Read and interpret test results.

Concept Explanation

The shared acceptance tests describe behavior that every language track's capstone must show:

1. Adding a record with an empty title is rejected with a clear message.
2. A new record receives an unused `id`.
3. Updating a record changes `updated_at` but not `created_at`.
4. Deleting a record removes only that record.
5. Saving and reloading preserves all fields and values (shown in Lesson 11.2).
6. Every message and control can be used without color or a mouse (checked by hand in Lesson 11.5).

This lesson automates tests 1 to 4 and also tests completion and the rejection of a blank title in an update. Each test builds a new `Manager`, so tests cannot affect each other. The fake clock keeps timestamps predictable so a test can check that `updatedAt` changed while `createdAt` did not.

Worked Example

```
import java.util.*;
import java.util.function.Supplier;

class Entry {
    final int id;
    String title;
    String description;
    String category;
    final String createdAt;
    String updatedAt;
    boolean done;
```

```
    Entry(int id, String title, String description, String category, String createdAt) {
        this.id = id;
        this.title = title;
        this.description = description;
        this.category = category;
        this.createdAt = createdAt;
        this.updatedAt = createdAt;
        this.done = false;
    }

    String describe() {
        return id + ". " + title + " [" + category + "] (" + (done ? "done" : "not done") +
    }
}

record Outcome(Entry entry, String message) {}

class Clocks {
    static Supplier<String> fake() {
        int[] ticks = {0};
        return () -> {
            ticks[0]++;
            return String.format("2026-01-15T10:00:%02d", ticks[0]);
        };
    }
}

class Manager {
    final List<Entry> entries = new ArrayList<>();
    private final Supplier<String> clock;

    Manager(Supplier<String> clock) {
        this.clock = clock;
    }

    static String validate(String title, String category) {
        if (title == null || title.isBlank()) {
            return "A title is required.";
        }
        if (category == null || category.isBlank()) {
            return "A category is required.";
        }
        return null;
    }

    private int nextId() {
```

```
    int highest = 0;
    for (Entry entry : entries) {
        highest = Math.max(highest, entry.id);
    }
    return highest + 1;
}

Entry find(int id) {
    for (Entry entry : entries) {
        if (entry.id == id) {
            return entry;
        }
    }
    return null;
}

Outcome add(String title, String description, String category) {
    String problem = validate(title, category);
    if (problem != null) {
        return new Outcome(null, problem);
    }
    Entry entry = new Entry(nextId(), title.strip(), description.strip(), category.strip
    entries.add(entry);
    return new Outcome(entry, null);
}

Outcome update(int id, String title, String description, String category, Boolean done)
    Entry entry = find(id);
    if (entry == null) {
        return new Outcome(null, "No record has that id.");
    }
    String problem = validate(title != null ? title : entry.title, category != null ? ca
    if (problem != null) {
        return new Outcome(null, problem);
    }
    if (title != null) entry.title = title.strip();
    if (description != null) entry.description = description.strip();
    if (category != null) entry.category = category.strip();
    if (done != null) entry.done = done;
    entry.updatedAt = clock.get();
    return new Outcome(entry, null);
}

Outcome complete(int id) {
    return update(id, null, null, null, true);
}
```

```
    String delete(int id) {
        Entry entry = find(id);
        if (entry == null) {
            return "No record has that id.";
        }
        entries.remove(entry);
        return null;
    }

    List<Entry> search(String text) {
        String needle = text.strip().toLowerCase();
        List<Entry> found = new ArrayList<>();
        for (Entry entry : entries) {
            if (entry.title.toLowerCase().contains(needle) || entry.description.toLowerCase(
                found.add(entry);
            }
        }
        return found;
    }
}

public class Lesson_11_4 {
    static int failures = 0;

    static void assertThat(boolean condition, String message) {
        if (!condition) {
            throw new AssertionError(message);
        }
    }

    static void test(String name, Runnable body) {
        try {
            body.run();
            System.out.println("PASS " + name);
        } catch (AssertionError | RuntimeException e) {
            failures++;
            System.out.println("FAIL " + name + ": " + e.getMessage());
        }
    }

    static Manager fresh() {
        Manager manager = new Manager(Clocks.fake());
        manager.add("Notebook", "Course notes", "supplies");
        manager.add("Pencil", "Box of 10", "supplies");
        manager.add("Study plan", "Weekly schedule", "planning");
```

```
        return manager;
    }

    public static void main(String[] args) {
        test("1. an empty title is rejected", () -> {
            Manager manager = fresh();
            Outcome outcome = manager.add("   ", "x", "supplies");
            assertThat(outcome.entry() == null && "A title is required.".equals(outcome.mess
            assertThat(manager.entries.size() == 3, "the list should be unchanged");
        });

        test("2. a new record gets an unused id", () -> {
            Manager manager = fresh();
            manager.delete(2);
            Outcome outcome = manager.add("Ruler", "30 cm", "supplies");
            assertThat(outcome.entry() != null && outcome.entry().id == 4, "expected id 4");
        });

        test("3. update changes updatedAt but not createdAt", () -> {
            Manager manager = fresh();
            Entry entry = manager.find(1);
            String createdBefore = entry.createdAt;
            String updatedBefore = entry.updatedAt;
            manager.update(1, null, "Lecture notes", null, null);
            assertThat(entry.createdAt.equals(createdBefore), "createdAt changed");
            assertThat(!entry.updatedAt.equals(updatedBefore), "updatedAt did not change");
        });

        test("4. delete removes only that record", () -> {
            Manager manager = fresh();
            manager.delete(2);
            List<Integer> ids = new ArrayList<>();
            for (Entry entry : manager.entries) {
                ids.add(entry.id);
            }
            assertThat(ids.equals(List.of(1, 3)), "wrong records remain: " + ids);
        });

        test("5. complete marks done and updates the record", () -> {
            Manager manager = fresh();
            Entry entry = manager.complete(3).entry();
            assertThat(entry != null && entry.done && !entry.updatedAt.equals(entry.createdA
        });

        test("6. a blank title cannot be saved by update", () -> {
            Manager manager = fresh();
```

```
            Outcome outcome = manager.update(1, "", null, null, null);
            assertThat(outcome.entry() == null && "A title is required.".equals(outcome.mess
        });

        System.out.println(failures == 0 ? "All acceptance tests passed." : failures + " tes
    }
}
```

Expected Behavior or Output

The program prints:

```
PASS 1. an empty title is rejected
PASS 2. a new record gets an unused id
PASS 3. update changes updatedAt but not createdAt
PASS 4. delete removes only that record
PASS 5. complete marks done and updates the record
PASS 6. a blank title cannot be saved by update
All acceptance tests passed.
```

Common Mistakes

1. Sharing one manager between tests, so one test's deletions change the next test.
2. Checking only that no exception occurred, and not checking the result.
3. Writing the expected message differently from the program, so a correct program fails the test.

Activities

1. Break `Manager.update` so it also changes `createdAt` (make the field non-final) and watch test 3 fail. Restore it.
2. Add a test that searching ignores case.
3. Add a test that deleting an unknown id returns a message.
4. Explain why test 2 deletes record 2 before adding the new one.

Accessibility Practice

The test names begin with the acceptance-test number, so PASS and FAIL lines can be matched to the specification. Read only the first word of each line to hear the overall result quickly.

Skills Checklist

- ☐ I can run the shared acceptance tests.
- ☐ I can test validation failures.
- ☐ I can test timestamps with a fake clock.
- ☐ I can read PASS and FAIL lines.

☐ I use a fresh manager for every test.

Lesson 11.5: Capstone: Documentation and Presentation

Reading References

- Java SE 25 API documentation
- Dev.java learning paths
- Unit P, Lesson P.6: files, data formats, and the shared accessible workflow
- Matching Python lesson: Lesson 11.5: Final Submission

Learning Objectives

- Write a plain-text project summary.
- Report a count of records by category.
- Complete a manual accessibility check.

Concept Explanation

Documentation tells a future reader what the program does and how to run it. Write it as plain text or Markdown with short headings, numbered steps, and complete sentences. Include: the purpose of the program, the compile and run commands, the shared record fields, the commands a user can give, the accessibility checks you carried out, and what you would improve next.

The example generates a summary from the shared sample data. It counts records by category and prints the categories in alphabetical order (a `TreeMap` keeps its keys sorted) so the output is stable. It then prints the manual accessibility checklist. The unchecked boxes `[ ]` are for *you* to complete by hand with your own access technology; the program cannot verify them.

Acceptance test 6 is a manual check: every message and control can be used without color or a mouse. Test it with the keyboard only, and with the screen reader or braille display you actually use. Record what you tested and its version; do not claim compatibility with software you did not test.

Worked Example

```
import java.util.List;
import java.util.Map;
import java.util.TreeMap;

public class Lesson_11_5 {
    record Entry(int id, String title, String category) {}

    public static void main(String[] args) {
        List<Entry> sample = List.of(
```

```
            new Entry(1, "Notebook", "supplies"),
            new Entry(2, "Pencil", "supplies"),
            new Entry(3, "Study plan", "planning"));

        Map<String, Integer> counts = new TreeMap<>();
        for (Entry entry : sample) {
            counts.merge(entry.category(), 1, Integer::sum);
        }

        System.out.println("Personal Data Manager - Summary");
        System.out.println("Records: " + sample.size());
        System.out.println("Categories:");
        for (Map.Entry<String, Integer> category : counts.entrySet()) {
            System.out.println("- " + category.getKey() + ": " + category.getValue());
        }

        System.out.println();
        System.out.println("Manual accessibility checklist (complete this by hand):");
        String[] checks = {
            "[ ] Every command can be used with the keyboard only.",
            "[ ] Every message is a complete sentence that says what to do next.",
            "[ ] No meaning depends on color, position, or symbols alone.",
            "[ ] The output was read with my own screen reader or braille display.",
            "[ ] I recorded the operating system, program, and version I tested."
        };
        for (String check : checks) {
            System.out.println(check);
        }
    }
}
```

Expected Behavior or Output

The program prints:

```
Personal Data Manager - Summary
Records: 3
Categories:
- planning: 1
- supplies: 2

Manual accessibility checklist (complete this by hand):
[ ] Every command can be used with the keyboard only.
[ ] Every message is a complete sentence that says what to do next.
[ ] No meaning depends on color, position, or symbols alone.
[ ] The output was read with my own screen reader or braille display.
```

```
[ ] I recorded the operating system, program, and version I tested.
```

Common Mistakes

1. Writing documentation that only the author can follow.
2. Claiming that a program is accessible without recording what was tested and how.
3. Letting the documentation drift from the program after later changes.

Activities

1. Write the project summary for your own capstone using the headings above.
2. Run the keyboard-only check and record each problem you find.
3. Ask a classmate to follow your run instructions from a fresh folder and note where they get stuck.
4. Write three improvements for the next version, ordered by user benefit.

Accessibility Practice

This is the final accessibility lesson. Read your own documentation aloud from start to finish. Anything you have to look at the screen to understand needs rewriting.

Skills Checklist

- ☐ I can write a plain-text project summary.
- ☐ I can report counts by category.
- ☐ I can run a manual accessibility check.
- ☐ I record what I tested.
- ☐ I do not claim untested compatibility.

AI Policy

AI may be used only for personal conceptual understanding and reinforcement. It must not write, complete, autocomplete, repair, or generate course code or assessment solutions.

Part IX

JAVASCRIPT

Chapter 9

Introduction to Computer Science with JavaScript

Before you start: complete Unit P: Computing Foundations Primer. It explains the language-neutral vocabulary and concepts used in every language track, using non-visual examples.

Accessible beginner curriculum using Node.js

This curriculum teaches introductory programming and computer science through JavaScript, using Node.js for program execution and text-based input/output. It is designed for blind and visually impaired learners who use screen readers, braille displays, magnification, keyboard navigation, or combinations of access technologies. It is also suitable for sighted beginners and mixed-vision classrooms.

JavaScript is widely used in browser applications, server-side systems, and developer tools. This course deliberately starts with Node.js and terminal programs so learners can master programming concepts without needing to interpret a visual web page or graphical interface. Browser-based development can be a later extension.

Course outcomes

By the end of this curriculum, learners should be able to:

- Explain introductory computer science concepts and the software-development cycle.
- Install and verify Node.js and use an editor, terminal, and Node.js REPL.

- Write JavaScript programs using variables, primitive values, expressions, and output.
- Explain dynamic typing, coercion, truthy/falsy values, `undefined`, `null`, and `NaN`.
- Accept and validate terminal input using Node.js input mechanisms.
- Use decisions, loops, arrays, objects, strings, and common collection operations.
- Design functions and use parameters, return values, scope, and closures.
- Explain classes and object-oriented programming at an introductory level.
- Work with files, modules, errors, and asynchronous operations introduced by the curriculum.
- Use Maps, Sets, data processing, and automated tests.
- Design, implement, test, and explain text-based projects and a capstone.
- Evaluate prompts, output order, error messages, and interaction for screen-reader usability.

The goal is a strong introductory foundation, not mastery of the entire JavaScript language or ecosystem.

Prerequisites and intended learners

No previous programming experience is required. Learners need a computer on which a supported Node.js release can be installed, a keyboard-accessible editor, and a terminal. Basic keyboard and file-management skills are useful. Screen-reader proficiency can develop alongside programming.

The course is intended primarily for beginning learners, including students using NVDA, JAWS, VoiceOver, Narrator, braille displays, or magnification. The JavaScript teacher's guide includes strategies for pacing, asynchronous concepts, accessible debugging, and differentiated instruction.

Why JavaScript and Node.js?

JavaScript connects foundational programming concepts to a large software ecosystem. Node.js makes it possible to run JavaScript outside a browser, supporting a text-first course.

- Programs can run in a terminal and provide text output.
- The Node.js REPL supports short experiments with immediate feedback.
- The same language can later be used for accessible web applications and server-side programs.
- Learners can encounter real-world concepts such as asynchronous operations and modules in a controlled sequence.

JavaScript also has behaviors that require explicit teaching. Dynamic typing and coercion can produce surprising results; `undefined`, `null`, and `NaN` are

distinct; arrays and objects have different semantics; and asynchronous code introduces ordering and error-handling questions.

Learning sequence

Unit	Main topics	Evidence of progress
0. Environment setup	Node.js, editor and access technology, workspace, terminal, first file, npm project	Verify Node.js and run a project script
1. Foundations	Computer science, Hello World, variables, primitive values, output	Explain and run a short sequential program
2. Numeric computations	Development process, expressions, input/output, conversion, Math object	Build and test an interactive tip calculator
3. Control flow	Boolean logic, chained conditionals, arrays, for loops, strings	Classify the shared scores and process text
4. Data structures	Array operations, sorting, fixed sequences, objects, Map, iteration	Organize the shared inventory and student profile
5. User input and while loops	readline, sentinel loops, flags, break, continue, termination	Build a validated command loop
6. Functions	Definitions, arguments, returns, arrays/objects, rest parameters	Decompose shared calculations into reusable functions
7. Classes and OOP	Classes, instances, fields, methods, inheritance, composition	Model students and inventory items
8. Files and exceptions	Text files, JSON, exceptions, cleanup	Persist and reload structured records
9. Testing and data analysis	Function/class tests, random values, text frequency, CSV	Test deterministic behavior and summarize data
10. Project-based learning	Text adventure and command-line to-do manager	Implement equivalent project requirements
11. Capstone	Accessible personal data manager, CRUD, search, persistence, tests	Demonstrate an independent, documented solution

Materials and navigation

- Units 0–1: Setup and Foundations
- Units 2–4: Numbers, Input, Control Flow, and Data
- Units 5–8: Strings, Functions, OOP, and Files
- Units 9–11: Modules, Collections, Testing, and Projects
- Teacher's guide: instructor planning, accessible instruction, assessment, and AI guidance
- Standardization audit: Python-to-JavaScript alignment and validation status

Return to the curriculum overview to compare Python, JavaScript, and Java.

Runtime version and modern JavaScript conventions

Use **Node.js 26.13.0 LTS** (the current LTS release as of October 30, 2026) for the course. Node.js 24 is also in LTS maintenance, but new installations should use the current 26.x LTS line. Keep examples compatible with the Node.js 26 LTS runtime and use the latest security/maintenance patch in that line. Check the official Node.js release schedule before installing or upgrading.

The course follows modern JavaScript practices:

- Use const by default and let only when reassignment is required; avoid var in new code.
- Use === and !== for comparisons; convert user input explicitly with Number, String, or other deliberate parsing.
- Prefer descriptive names, small functions, template literals, array methods where they improve clarity, and async/await for asynchronous work.
- Use node: specifiers for built-in modules (for example, node:fs and node:readline/promises).
- Declare module format explicitly in package.json. Use type: commonjs for CommonJS examples and type: module for ES module examples; do not mix require/module.exports with import/export in the same module.
- Handle rejected promises and file-system errors explicitly. Do not assume parsed JSON or external input has the expected shape.
- Use the built-in node:test and node:assert/strict modules for dependency-free introductory tests; Jest remains an optional framework when a project specifically needs it.
- Keep dependencies local to a project, review package provenance, and commit a lockfile for reproducible installations.

Recommended environment

Follow Unit 0 for detailed installation and verification. For a new setup, use a currently supported Node.js Long-Term Support (LTS) release unless course materials specify a different compatible version. Check the official Node.js download page rather than relying on an old version number.

Recommended tools include Node.js and npm, Visual Studio Code or Theia IDE, a terminal, and the learner's preferred access technology. Theia is an alternative with similar keyboard- and screen-reader-oriented workflows; verify language extensions through Open VSX and test the learner's screen reader. Use project-local npm dependencies and `npx`/npm scripts; avoid global installs. Explain each third-party dependency and review its provenance and maintenance before adding it.

Typical verification commands:

```
node --version
npm --version
node
```

Windows Powershell

```
node --version
npm --version
node
```

At the Node.js prompt, enter a short expression and read the returned value. Use `.exit` or the documented keyboard command to leave the REPL.

Accessibility and instructional design

JavaScript uses braces to delimit blocks and semicolons may be optional in many contexts. Teach one consistent style and explain punctuation explicitly. Screen-reader users benefit from short functions, descriptive identifiers, predictable formatting, and output that is meaningful when read linearly.

The curriculum emphasizes keyboard-first editor, terminal, and REPL workflows; explicit reading of braces, parentheses, brackets, commas, operators, and template literals; terminal-based core activities; clear prompts and errors; and step-by-step debugging of coercion and asynchronous behavior.

Accessibility is not guaranteed merely because an application prints text. Consider output order, repeated announcements, asynchronous messages, and whether a learner can review the result after it appears. Adapt to the student's actual technology and preferences.

How to study each lesson

1. Read the lesson overview and identify the new language features.
2. Read each example structurally. Identify blocks, expressions, function calls, and data values.
3. Predict output and note assumptions about types or coercion.
4. Run the example in Node.js and compare actual and expected results.
5. Change one variable or expression and observe behavior.
6. Complete the exercises and record questions.
7. When a result is unexpected, inspect the value and its type, then trace the relevant control flow.
8. For asynchronous examples, identify when an operation begins, what is awaited, and when its result becomes available.
9. Explain the program in your own words and save the work in a portfolio.

Use the REPL for short experiments and files for complete programs. Keep experiments small enough that output can be reviewed without losing context.

Assessment and projects

Assessment can include predicting output, explaining type behavior, tracing a loop, writing a function, refactoring a program, identifying a bug, creating tests, and building a text-based application. Evaluate reasoning, correctness, testing, clarity, and the learner's ability to explain and change their code.

The capstone should include a written problem statement, requirements, input/output design, data representation, implementation milestones, test cases, and a short demonstration or explanation. The learner should be able to discuss known limitations and how to improve the program.

Teaching with AI

AI may be used by students only for personal conceptual explanations and reinforcement. It may not write, autocomplete, repair, complete, or generate course code or assessment solutions. Students must independently produce and explain submitted code. Never enter personal information, student records, credentials, private code, or confidential data into public AI services. The teacher's guide provides the instructor protocol.

Recommended introductory books

- Haverbeke, Marijn. *Eloquent JavaScript: A Modern Introduction to Programming*, 4th edition. No Starch Press, 2024. A current, exercise-rich introduction to JavaScript fundamentals, browser programming, and Node.js. The full text is available online. Book site and publisher page.

Suggested use: this is the primary supplementary book for the JavaScript track. Its examples include browser-based work; learners following this course should prioritize the language fundamentals and Node.js-compatible examples before attempting activities that depend on browser interfaces. Pair it with the course lessons for the specific accessible terminal workflow and assessment expectations.

Further learning

Use MDN's JavaScript Guide for language concepts and the Node.js documentation for runtime APIs. Eloquent JavaScript is a further introductory text. Check version-specific documentation when examples depend on newer features.

The course uses JavaScript to teach transferable problem-solving skills. Learners who can reason about values, control flow, data, functions, errors, and tests are prepared to explore browser programming and other parts of the ecosystem.

GUI, visualization, and workbench accessibility

See the shared GUI, visualization, and workbench accessibility reference for screen-reader support by toolkit, platform-specific activation settings, recommended alternatives, and interfaces that must not be required without an equivalent accessible route.

9.1 JAVASCRIPT Lessons

The curriculum is organized as one Markdown file per unit.

- setting up your javascript development environment
- foundations getting started with javascript
- numeric computations and input output
- control flow decisions lists and strings
- data structures arrays fixed sequences and objects
- user input and while loops
- functions
- classes and object oriented programming
- files and exceptions
- testing and data analysis
- project based learning text based applications
- capstone project accessible personal data manager

9.2 Unit 0: Setting Up Your JavaScript Development Environment

Before you start: complete Unit P: Computing Foundations Primer. It explains the language-neutral vocabulary and concepts used in every language track, using non-visual examples.

Lesson 0.1: Installing Node.js

Node.js 26 LTS lets you run JavaScript on your computer (outside a browser) using the terminal. Install the current LTS line, not an obsolete tutorial version or an experimental prerelease.

Reading References

- Official Node.js downloads and installation documentation.

Learning Objectives

- Install a supported Node.js LTS runtime and npm.
- Verify the runtime from a terminal.

Concept Explanation

Node.js is the JavaScript runtime used for this course. npm is the package manager distributed with Node.js. The terminal must be able to locate both commands through PATH.

Installing Node.js Download **Node.js 26 LTS** from https://nodejs.org/.

Windows

1. Download the `.msi` installer.
2. **CRUCIAL STEP:** Ensure "Add to PATH" is checked during installation.
3. Activate "Install" and "Finish".

macOS

1. Download the `.pkg` installer and follow the prompts.
2. Or use Homebrew: `brew install node`

Linux (Debian/Ubuntu)

```
curl -fsSL https://deb.nodesource.com/setup_26.x | sudo -E bash -
sudo apt install nodejs
```

Windows Powershell

```
winget install --id OpenJS.NodeJS.LTS -e
```

Verify Installation

```
node --version      # Should show v26.x.x (26.13.0 or a newer 26.x maintenance release)
npm --version       # Should show npm version
```

Windows Powershell

```
node --version      # Should show v26.x.x (26.13.0 or a newer 26.x maintenance release)
npm --version       # Should show npm version
```

Worked Example

```
node --version
npm --version
node -e "console.log(2 + 3)"
```

The third command verifies that Node.js can execute a short JavaScript expression.

Expected Behavior or Output

The terminal reports a Node.js version in the documented supported major line and an npm version. The REPL accepts a short expression and prints its result.

Common Mistakes

- Installing a runtime but not reopening the terminal, so PATH changes are not visible.
- Installing a version that is unsupported or inconsistent with the course baseline.
- Confusing Node.js with npm; Node.js runs JavaScript and npm manages packages.

Activities

1. Install Node.js using the official download or package-manager instructions for the operating system.
2. Run `node --version` and `npm --version`.
3. Open the Node.js REPL, evaluate `2 + 3`, and exit.
4. If a command is not found, identify whether the runtime is installed and whether PATH is configured.

Accessibility Practice

Use keyboard navigation in the installer and terminal. Read each installation prompt and verify the runtime with text commands rather than relying on an icon or graphical success message.

Skills Checklist

- ☐ Running `node --version` shows the version number.
- ☐ Running `npm --version` shows the npm version.

Expected Behavior or Output

The terminal reports a Node.js version in the documented supported major line and an npm version. The REPL accepts a short expression and prints its result.

Worked Example

```
node --version
npm --version
node -e "console.log(2 + 3)"
```

The third command verifies that Node.js can execute a short JavaScript expression.

Common Mistakes

- Installing a runtime but not reopening the terminal, so PATH changes are not visible.
- Installing a version that is unsupported or inconsistent with the course baseline.
- Confusing Node.js with npm; Node.js runs JavaScript and npm manages packages.

Activities

1. Install Node.js using the official download or package-manager instructions for the operating system.
2. Run `node --version` and `npm --version`.
3. Open the Node.js REPL, evaluate `2 + 3`, and exit.
4. If a command is not found, identify whether the runtime is installed and whether PATH is configured.

Accessibility Practice

Use keyboard navigation in the installer and terminal. Read each installation prompt and verify the runtime with text commands rather than relying on an

icon or graphical success message.

Lesson 0.2: Setting Up VS Code or Theia IDE

For a new project, create a package manifest with npm init -y. Choose one module system and declare it explicitly: set package.json to { "type": "commonjs" } for the CommonJS examples in the early lessons, or { "type": "module" } for the ES module examples later in the course. Do not combine the two syntaxes in one file.

1. Install Visual Studio Code or Theia IDE. Theia supports many VS Code extensions through Open VSX, but check that the required extensions are available and test them with the learner's screen reader.
2. In VS Code, press `Ctrl + Shift + X` to open Extensions. In Theia, open View → Extensions or run Toggle Extensions View. Install:
   - **JavaScript (ES6) code snippets** — Handy code shortcuts.
   - **ESLint** — Catches common errors.
3. In VS Code, enable Screen Reader Mode: `Ctrl + Shift + P` → "Toggle Screen Reader Optimized". In Theia, enable its corresponding screen-reader setting and verify editor, terminal, diagnostics, and extension announcements.

Reading References

- Visual Studio Code accessibility documentation.
- Theia IDE documentation and extension marketplace guidance.

Learning Objectives

- Configure an editor for JavaScript development.
- Navigate files, editor, terminal, and diagnostics by keyboard.

Concept Explanation

An editor provides source navigation and language tooling; it does not replace the Node.js runtime. Keyboard shortcuts and extension availability can differ between VS Code and Theia.

VS Code Shortcuts for JavaScript (Theia keybindings may differ)

Action	Shortcut
Run file (via terminal)	`Ctrl + \`` then` node filename.js‘
Command Palette	`Ctrl + Shift + P`
Problems View	`Ctrl + Shift + M`
Next Error	`F8`
Format Code	`Shift + Alt + F`

Action	Shortcut
Go to Definition	`F12`

Verify editor tooling and the Node.js project environment Visual Studio Code

- JavaScript language support is built in. Confirm that the built-in JavaScript and TypeScript language features are enabled for the workspace.
- Install **ESLint** from the Extensions view when the project uses ESLint. The JavaScript snippets extension is optional and is not required to compile or run JavaScript.
- Open the project root (the folder containing `package.json`) as the workspace. Open the integrated terminal from that folder and verify that `node --version` and `npm --version` report the expected Node.js 26 LTS and npm versions.

Theia IDE

- Open the Extensions view and use the configured Open VSX registry to find JavaScript/TypeScript language tooling, ESLint integration, and (if needed) Node.js debugging support.
- Verify availability and compatibility with the installed Theia release before depending on an extension. Do not assume every Visual Studio Marketplace extension is available through Open VSX.
- Test syntax diagnostics, completion, navigation, terminal operation, debugging (if used), and screen-reader announcements. The editor extension supplies language tooling; Node.js remains the runtime and must be installed and available to the terminal.

Use the project's local toolchain consistently

- Keep `package.json`, `package-lock.json`, and `node_modules` in the project root. Install project dependencies locally with `npm install` or `npm install --save-dev package-name`; do not install course tools globally.
- Run project tools with `npm run script-name` or `npx tool-name`. Use the integrated terminal opened at the project root so npm resolves the local dependencies.
- Check that the editor's integrated terminal and the system terminal resolve the same Node.js installation and version. If they differ, correct the terminal PATH or runtime configuration before continuing.
- For these terminal-first lessons, run programs with `node filename.js` from the project root. Do not rely on an editor's generic Run button until you have verified that it invokes the same Node.js runtime and working directory.

Advanced editor options Experienced coders may prefer JetBrains IDEs, such as WebStorm. They are advanced alternatives, not the recommended starting point for this beginner curriculum. This course does not provide JetBrains installation or usage instructions.

Worked Example

Use the keyboard to open a project folder, create `hello.js`, move focus to the integrated terminal, and run `node hello.js`.

Expected Behavior or Output

The editor can open a JavaScript project, announce file names and diagnostics, and run the same file from its integrated terminal. The exact keyboard shortcuts may differ between VS Code and Theia.

Common Mistakes

- Assuming VS Code and Theia have identical extensions or keyboard shortcuts.
- Installing extensions without checking compatibility with the chosen editor.
- Relying on a mouse-only run button instead of terminal execution.

Activities

1. Open the Command Palette using the keyboard.
2. Install or verify JavaScript language support and Node.js tooling.
3. Navigate between editor, terminal, and file explorer without a mouse.
4. Confirm that the editor announces syntax errors and file names accessibly.

Accessibility Practice

Test the learner's actual screen reader and braille display configuration. Do not claim universal accessibility; document editor, operating system, and access-technology versions used.

Skills Checklist

- ☐ I can open and navigate a JavaScript workspace.
- ☐ I can use the keyboard to move between editor and terminal.
- ☐ I can locate diagnostics and read their messages.
- ☐ I can verify the installed JavaScript language support.

Lesson 0.3: Your First JavaScript File

Reading References

- MDN: JavaScript first steps.
- Node.js: Running JavaScript files.

Learning Objectives

- Create and save a JavaScript source file.
- Run the file and distinguish script execution from the REPL.

Concept Explanation

A source file stores a repeatable program. The Node.js REPL evaluates individual expressions interactively; running a file executes its statements in sequence.

Hello World Create `hello.js`:

```
console.log("Hello, World!");
console.log("Welcome to JavaScript!");
```

Run it:

```
node hello.js
```

Windows Powershell

```
node hello.js
```

The Node.js REPL For quick experiments, open an interactive session:

```
node
```

Windows Powershell

```
node
```

Type JavaScript directly and press Enter to execute. Type `.exit` to quit.

Key Differences from Python

Python	JavaScript
`print()`	`console.log()`
`#` for comments	`//` for single-line, `/* */` for multi-line
No semicolons required	Semicolons optional but recommended
Indentation defines blocks	Curly braces `{}` define blocks
`def` for functions	`function` or `=>` for functions
`True`/`False`	`true`/`false` (lowercase)
`None`	`null` and `undefined`

Python	JavaScript
`len(x)`	`x.length`

Worked Example

```
console.log("Hello, World!");
console.log("Welcome to JavaScript.");
```

Run the saved file with `node hello.js`.

Expected Behavior or Output

Running the saved file prints the exact greeting text in the terminal, in the order written in the file.

Common Mistakes

- Saving source code with a hidden extra extension such as `.js.txt`.
- Typing smart quotes instead of straight quotation marks.
- Running the REPL command instead of executing the saved file.

Activities

1. Create and save `hello.js`.
2. Run the file from the terminal.
3. Enter a short arithmetic expression in the Node.js REPL.
4. Explain the difference between a saved script and an interactive REPL session.

Accessibility Practice

Ensure the file name, extension, and terminal output are announced in sequence. Keep a terminal path available even when the editor provides a run command.

Skills Checklist

☐ I can create a `.js` file and run it with `node filename.js`.
☐ I can use the Node.js REPL for quick experiments.

Expected Behavior or Output

Running the saved file prints the exact greeting text in the terminal, in the order written in the file.

Worked Example

```
console.log("Hello, World!");
console.log("Welcome to JavaScript.");
```

Run the saved file with `node hello.js`.

Common Mistakes

- Saving source code with a hidden extra extension such as `.js.txt`.
- Typing smart quotes instead of straight quotation marks.
- Running the REPL command instead of executing the saved file.

Activities

1. Create and save `hello.js`.
2. Run the file from the terminal.
3. Enter a short arithmetic expression in the Node.js REPL.
4. Explain the difference between a saved script and an interactive REPL session.

Accessibility Practice

Ensure the file name, extension, and terminal output are announced in sequence. Keep a terminal path available even when the editor provides a run command.

Lesson 0.4: Node.js Projects, Terminal Execution, and Dependencies

Reading References

- Node.js documentation: Getting Started and Command-Line API.
- npm documentation: package.json and npm scripts.

Learning Objectives

- Create a project directory and initialize a Node.js package.
- Run a JavaScript file from a terminal and distinguish the working directory from the file location.
- Explain what package.json records and when a dependency is needed.
- Use npm scripts without relying on mouse-only editor controls.

Concept Explanation

A project is a directory containing source files and supporting configuration. Node.js executes a file with `node filename.js`. The terminal's current working directory determines how relative paths such as `./data.json` are resolved. The `package.json` file records project metadata, scripts, dependencies, and module

mode. For this course, use one explicit module system per project: CommonJS with `"type": "commonjs"`, or ECMAScript modules with `"type": "module"`. Do not mix `require()` and `import` in the same instructional example.

Worked Example

From the terminal, create and enter a project directory:

```
mkdir js-foundations
cd js-foundations
npm init -y
```

Create `hello.js`:

```
"use strict";

const student = "Avery";
console.log(`Hello, ${student}! Your Node.js project is ready.`);
```

Run it:

```
node hello.js
```

Expected output:

```
Hello, Avery! Your Node.js project is ready.
```

Add a script to the `scripts` object in package.json:

```
{
  "type": "commonjs",
  "scripts": {
    "start": "node hello.js"
  }
}
```

Run `npm start`. The script should print the same line.

Expected Behavior or Output

The project can be run with both `node hello.js` and `npm start`; both commands print the same greeting.

Common Mistakes

- Running the command from the parent directory, so Node.js cannot find `hello.js`.
- Naming the file `hello.js.txt` because file extensions are hidden.
- Assuming a package must be installed for built-in Node.js modules such as `node:fs` or `node:readline`.
- Mixing CommonJS and ES module syntax without configuring module mode.

Activities

1. Use `pwd` (macOS/Linux) or `Get-Location` (PowerShell) to announce the current directory, then navigate into the project.
2. Create `profile.js` and print the canonical student name Avery, age 16, and favorite subject Computer Science.
3. Add a `start` npm script and run the same program with `npm start`.
4. Deliberately run Node.js from the wrong directory; read the error, correct the working directory, and rerun.

Accessibility Practice

Use terminal commands and keyboard navigation throughout. Confirm the editor announces the file name and extension. Read the complete terminal error before acting on it. Keep output textual and sequential; do not make successful execution depend on a color change or visual notification.

Skills Checklist

- ☐ I can create and enter a project directory from the terminal.
- ☐ I can initialize and describe a package.json file.
- ☐ I can run a JavaScript file with node and an npm script.
- ☐ I can diagnose a wrong-working-directory error.
- ☐ I can explain why a project should use one explicit module system.

9.3 Unit 1: Foundations — Getting Started with JavaScript

Overview

This unit covers JavaScript fundamentals: variables, data types, and basic output.

Lesson 1.1: Welcome to Computer Science

(See Python Unit 1.1 for conceptual content — hardware, software, algorithms, and problem-solving are language-agnostic.)

Reading References

- Python source counterpart: Unit 1, Lesson 1.1.
- MDN: Introduction to JavaScript.

Learning Objectives

- Define computer science as the study of computational problem-solving.
- Distinguish hardware from software.
- Describe JavaScript's role in browser and server-side programming.

Concept Explanation

Computer science studies computational problem-solving: representing information, designing algorithms, and evaluating solutions. JavaScript is a programming language; Node.js and web browsers are different runtime environments that host JavaScript code.

JavaScript's Role JavaScript was created by Brendan Eich at Netscape in 1995. Originally just for web browsers, it now runs everywhere thanks to Node.js (2009).

Learning Objectives

- Define computer science as the study of computational problem-solving.
- Distinguish hardware from software.
- Describe JavaScript's role in browser and server-side programming.

Worked Example

A simple algorithm for preparing a school day can be written as ordered steps: check the schedule, pack required materials, travel to school, and attend the first class. A computer program expresses a precise algorithm so that a machine can execute it consistently.

Expected Behavior or Output

Students can explain the difference between a physical computer component and a program, and identify that JavaScript code can run in a browser or in Node.js.

Common Mistakes

- Defining computer science as only learning a programming language.
- Confusing a browser with JavaScript itself.
- Assuming JavaScript running in Node.js has browser APIs such as document and window.

Activities

1. Identify three hardware components and three software examples.

2. Describe an everyday task as an ordered algorithm.
3. Compare JavaScript in a browser with JavaScript in Node.js.
4. Explain why this course starts with terminal programs.

Accessibility Practice

Describe hardware and software with textual examples. Do not require interpreting a diagram as the only way to understand system components.

Skills Checklist

- ☐ I can define computer science.
- ☐ I can distinguish hardware and software.
- ☐ I can describe JavaScript's browser and Node.js environments.
- ☐ I can express a simple task as an algorithm.

Lesson 1.2: Your First JavaScript Program — Hello World

Reading References

- Python source counterpart: Unit 1, Lesson 1.2.
- MDN: JavaScript first steps.

Learning Objectives

- Write and run a minimal JavaScript program.
- Explain sequential execution and console output.
- Run the same file from the terminal.

Concept Explanation

A JavaScript program is a sequence of instructions. Node.js executes statements in order. The `console.log()` function writes text to standard output. JavaScript statements are commonly separated by line breaks and may optionally end with semicolons; this course uses semicolons consistently.

Worked Example

Save as `hello.js`:

```
console.log("Hello, World!");
console.log("Welcome to JavaScript.");
```

Run with `node hello.js`.

Expected output:

```
Hello, World!
Welcome to JavaScript.
```

Expected Behavior or Output

Running `node hello.js` prints `Hello, World!` followed by `Welcome to JavaScript.` on the next line.

Common Mistakes

- Saving the file with the wrong extension.
- Running Node.js from a directory that does not contain the file.
- Misspelling the case-sensitive function name `console.log`.

Activities

1. Type and run the two-line program.
2. Add a third line that prints your name.
3. Change the greeting and predict the output before running.
4. Run the program from a different directory and diagnose the error.

Accessibility Practice

Use the terminal and keyboard to create, save, and run the file. Read the output sequentially and confirm the file name and extension.

Skills Checklist

☐ I can create a JavaScript source file.
☐ I can print text with `console.log()`.
☐ I can run a file with `node`.
☐ I can explain sequential execution.

Lesson 1.3: Variables and Simple Data Types

A simple algorithm for preparing a school day can be written as ordered steps: check the schedule, pack required materials, travel to school, and attend the first class. A computer program expresses a precise algorithm so that a machine can execute it consistently.

Expected Behavior or Output

Students can explain the difference between a physical computer component and a program, and identify that JavaScript code can run in a browser or in Node.js.

Common Mistakes

- Defining computer science as only learning a programming language.
- Confusing a browser with JavaScript itself.

- Assuming JavaScript running in Node.js has browser APIs such as document and window.

Activities

1. Identify three hardware components and three software examples.
2. Describe an everyday task as an ordered algorithm.
3. Compare JavaScript in a browser with JavaScript in Node.js.
4. Explain why this course starts with terminal programs.

Accessibility Practice

Describe hardware and software with textual examples. Do not require interpreting a diagram as the only way to understand system components.

Skills Checklist

- ☐ I can define computer science.
- ☐ I can distinguish hardware and software.
- ☐ I can describe JavaScript's browser and Node.js environments.
- ☐ I can express a simple task as an algorithm.

Lesson 1.2: Your First JavaScript Program — Hello World

Reading References

- Python source counterpart: Unit 1, Lesson 1.2.
- MDN: JavaScript first steps.

Learning Objectives

- Write and run a minimal JavaScript program.
- Explain sequential execution and console output.
- Run the same file from the terminal.

Concept Explanation

A JavaScript program is a sequence of instructions. Node.js executes statements in order. The `console.log()` function writes text to standard output. JavaScript statements are commonly separated by line breaks and may optionally end with semicolons; this course uses semicolons consistently.

Worked Example

Save as `hello.js`:

```
console.log("Hello, World!");
console.log("Welcome to JavaScript.");
```

Run with `node hello.js`.

Expected output:

```
Hello, World!
Welcome to JavaScript.
```

Expected Behavior or Output

Running `node hello.js` prints `Hello, World!` followed by `Welcome to JavaScript.` on the next line.

Common Mistakes

- Saving the file with the wrong extension.
- Running Node.js from a directory that does not contain the file.
- Misspelling the case-sensitive function name `console.log`.

Activities

1. Type and run the two-line program.
2. Add a third line that prints your name.
3. Change the greeting and predict the output before running.
4. Run the program from a different directory and diagnose the error.

Accessibility Practice

Use the terminal and keyboard to create, save, and run the file. Read the output sequentially and confirm the file name and extension.

Skills Checklist

☐ I can create a JavaScript source file.
☐ I can print text with `console.log()`.
☐ I can run a file with `node`.
☐ I can explain sequential execution.

Lesson 1.3: Variables and Simple Data Types

Reading References

- Python source counterpart: Unit 1, Lesson 1.3.
- MDN: JavaScript data types and structures.

Learning Objectives

- Declare variables with `const` and `let`.
- Store strings, numbers, and Booleans.
- Explain JavaScript's dynamic typing and primitive values.

Concept Explanation

A variable name refers to a value. Use `const` when the binding will not be reassigned and `let` when reassignment is required. JavaScript is dynamically typed: a variable can refer to values of different types over time.

`console.log()`

```
console.log("Hello, World!");
console.log(42);
console.log(true);
console.log("Value:", 42, "and", "text");
```

Declaring Variables JavaScript has three ways to declare variables:

Keyword	Scope	Reassignable	Use When
`const`	Block	No	Value won't change (default choice)
`let`	Block	Yes	Value will change
`var`	Function	Yes	Legacy — avoid in modern code

```
const PI = 3.14159;                // Cannot be reassigned
let score = 100;                   // Can change
score = 150;                       // OK

// var is function-scoped (legacy)
var oldWay = "avoid this";
```

Best practice: Use `const` by default; use `let` only when you need to reassign.

Template Literals

```
const name = "Alice";
const age = 15;

// Template literal (backticks - preferred)
console.log(`Hello, ${name}! You are ${age} years old.`);

// Concatenation (older style)
console.log("Hello, " + name + "! You are " + age + " years old.");
```

Additional Primitive Types and Type Conversion

Additional Primitive Types and Type Conversion

Reading References

- Python source counterpart: Unit 1, Lesson 1.3.

- MDN: JavaScript data types and structures.

Learning Objectives

- Declare variables with `const` and `let`.
- Store strings, numbers, and Booleans.
- Explain JavaScript's dynamic typing and primitive values.

Concept Explanation

A variable name refers to a value. Use `const` when the binding will not be reassigned and `let` when reassignment is required. JavaScript is dynamically typed: a variable can refer to values of different types over time.

Primitive Types

```
// Number (integers and floats use the same type)
let integer = 42;
let decimal = 3.14;
let negative = -10;

// String
let name = "Alice";
let greeting = 'Hello';
let multiLine = `Line 1
Line 2`;

// Boolean
let isRaining = false;
let isLoggedIn = true;

// null (intentional absence of value)
let emptyValue = null;

// undefined (value not assigned)
let notAssigned;
console.log(notAssigned);  // undefined

// Check type
console.log(typeof 42);           // "number"
console.log(typeof "hello");      // "string"
console.log(typeof true);         // "boolean"
console.log(typeof null);         // "object" (known quirk!)
console.log(typeof undefined);    // "undefined"
```

Arithmetic Operators

```javascript
console.log(5 + 3);   // 8
console.log(10 - 4);  // 6
console.log(7 * 6);   // 42
console.log(15 / 4);  // 3.75  (always decimal, unlike Python 2 or Java)
console.log(10 % 3);  // 1  (remainder)
console.log(2 ** 10); // 1024  (exponentiation)

// Math.floor for integer division equivalent
console.log(Math.floor(15 / 4));  // 3
```

Type Conversion

```javascript
// String to Number
let str = "123";
let num = Number(str);          // 123
let numParsed = parseInt(str);  // 123
let floatParsed = parseFloat("3.14"); // 3.14

// Number to String
let n = 42;
let s = String(n);              // "42"
let s2 = n.toString();          // "42"
let s3 = `${n}`;               // "42" (template literal)

// Checking
console.log(isNaN("abc"));     // true - "not a number"
console.log(isNaN(42));        // false
console.log(Number("abc"));    // NaN
```

Constants

```javascript
const MAX_SIZE = 100;
const APP_NAME = "My App";
const PI = Math.PI;  // Use Math.PI - already defined
```

Examples Example 1.3.1: Personal Information

```javascript
const myName = "Jamie";
const favoriteNumber = 7;
const heightInMeters = 1.75;
const likesPizza = true;

console.log(`Name: ${myName}`);
console.log(`Favorite number: ${favoriteNumber}`);
console.log(`Height: ${heightInMeters} meters`);
console.log(`Likes pizza: ${likesPizza}`);
```

Example 1.3.2: Simple Math

```
const num1 = 15;
const num2 = 7;

console.log(`Sum: ${num1 + num2}`);
console.log(`Difference: ${num1 - num2}`);
console.log(`Product: ${num1 * num2}`);
console.log(`Quotient: ${(num1 / num2).toFixed(2)}`);
console.log(`Remainder: ${num1 % num2}`);
```

Worked Example

```
const student = { name: "Avery", age: 16, heightM: 1.72, favoriteSubject: "Computer Science"
console.log(`${student.name}, age ${student.age}`);
console.log(`Height: ${student.heightM} m`);
```

Expected Behavior or Output

The program prints Avery's name and age, then the height with the unit metres. The profile uses the shared canonical values.

Common Mistakes

- Using `var` when `const` or `let` expresses the intended behavior more clearly.
- Assuming JavaScript has separate integer and floating-point primitive types.
- Confusing `null` with `undefined`.
- Assuming a conversion such as `Number("hello")` throws instead of returning `NaN`.

Activities

Activity 1.3.1: Area Calculator

```
const length = 12;
const width = 5;
const height = 8;

const rectangleArea = length * width;
const boxVolume = length * width * height;
const circleArea = Math.PI * (length / 2) ** 2;

console.log(`Rectangle area: ${rectangleArea}`);
console.log(`Box volume: ${boxVolume}`);
console.log(`Circle area: ${circleArea.toFixed(2)}`);
```

1. Create an Avery profile with name, age 16, height 1.72, and favorite subject Computer Science.
2. Print each property with a descriptive label.
3. Compare `typeof 16`, `typeof 1.72`, `typeof "Avery"`, and `typeof true`.
4. Convert the text `"16"` to a number and test the invalid text `"sixteen"`.

Accessibility Practice

Use meaningful variable names and explicit output labels. Explain punctuation such as braces, commas, quotation marks, and semicolons when reading code aloud.

Skills Checklist

☐ I can use `console.log()` to display output.
☐ I can declare variables using `const` and `let`.
☐ I can use template literals with `${}`.

☐ I can identify JavaScript's primitive data types.
☐ I can perform arithmetic operations in JavaScript.
☐ I understand that JavaScript's `/` always gives a decimal result.
☐ I can convert between types using `Number()`, `String()`, `parseInt()`.
☐ I understand `const` vs. `let`.

Worked Example

```
const student = { name: "Avery", age: 16, heightM: 1.72, favoriteSubject: "Computer Science"
console.log(`${student.name}, age ${student.age}`);
console.log(`Height: ${student.heightM} m`);
```

Common Mistakes

- Using `var` when `const` or `let` expresses the intended behavior more clearly.
- Assuming JavaScript has separate integer and floating-point primitive types.
- Confusing `null` with `undefined`.
- Assuming a conversion such as `Number("hello")` throws instead of returning `NaN`.

Activities

1. Create an Avery profile with name, age 16, height 1.72, and favorite subject Computer Science.
2. Print each property with a descriptive label.
3. Compare `typeof 16`, `typeof 1.72`, `typeof "Avery"`, and `typeof true`.
4. Convert the text `"16"` to a number and test the invalid text `"sixteen"`.

Accessibility Practice

Use meaningful variable names and explicit output labels. Explain punctuation such as braces, commas, quotation marks, and semicolons when reading code aloud.

Expected Behavior or Output

The program prints Avery's name and age, then the height with the unit metres. The profile uses the shared canonical values.

9.4 Unit 2: Numeric Computations and Input/Output

Runtime baseline: Node.js 26.x. Examples use built-in ECMAScript and Node.js APIs; no third-party package is required for these lessons.

Overview

This unit follows the same design, expression, input/output, numeric-conversion, and mathematics sequence as the Python course. Use the shared bill of $24.50 and tip rate of 0.15, the student profile Avery (age 16, height 1.72 m), and the coordinates (3, 4). JavaScript is dynamically typed; input from a terminal arrives as text and must be converted and validated before arithmetic.

Lesson 2.1: The Software Development Process

Reading References

- Python source counterpart: Unit 2, Lesson 2.1.
- MDN: JavaScript Guide, Grammar and Types.

Learning Objectives

- Separate problem analysis, design, implementation, and testing.
- Identify inputs, outputs, assumptions, and boundary cases before coding.
- Write language-neutral pseudocode before JavaScript.

Concept Explanation

Use the same three-phase cycle for every language: **design**, **implement**, and **test/debug**. For a tip calculator, inputs are bill amount and tip rate; outputs are tip amount and total. State assumptions explicitly: the bill is nonnegative, the rate is a decimal fraction, and money is displayed to two decimal places. Pseudocode should describe the calculation without depending on JavaScript syntax.

Worked Example

Pseudocode:

```
GET bill amount
GET tip rate
CALCULATE tip = bill amount * tip rate
CALCULATE total = bill amount + tip
DISPLAY tip and total to two decimal places
```

JavaScript implementation using the canonical values:

```
const bill = 24.50;
const tipRate = 0.15;
const tip = bill * tipRate;
const total = bill + tip;

console.log(`Tip: $${tip.toFixed(2)}`);
console.log(`Total: $${total.toFixed(2)}`);
```

Expected output:

```
Tip: $3.67
Total: $28.18
```

Expected Behavior or Output

The canonical bill of $24.50 at a 0.15 rate produces a displayed tip of $3.67 and total of $28.18 with JavaScript's binary floating-point and `toFixed(2)` behavior.

Common Mistakes

- Coding before deciding what the program should accept and display.
- Treating 15 as equivalent to 0.15 for a percentage rate.
- Forgetting to test zero, negative, and unusually large inputs.
- Rounding intermediate values when only the displayed result needs formatting.

Activities

1. Write input/output tables for the tip calculator before coding.
2. Add a test case for a $0.00 bill and explain its expected result.
3. Change the tip rate to 0.20 and predict the output before running the program.
4. Describe one invalid input and the validation rule that should reject it.

Accessibility Practice

Read the pseudocode and output in order. Use explicit labels for each displayed amount; do not rely on alignment, color, or visual placement to communicate

which value is the tip and which is the total.

Skills Checklist

- ☐ I can identify a program's inputs, outputs, assumptions, and edge cases.
- ☐ I can write pseudocode independently of JavaScript syntax.
- ☐ I can distinguish design, implementation, and testing.
- ☐ I can explain why expected results should be predicted before execution.

Lesson 2.2: Expressions and Assignment

Reading References

- Python source counterpart: Unit 2, Lesson 2.2.
- MDN: Expressions and operators.

Learning Objectives

- Evaluate arithmetic expressions using precedence and parentheses.
- Assign and update variables with clear, predictable statements.
- Explain the difference between `=`, `===`, and arithmetic operators.

Concept Explanation

An expression produces a value. An assignment stores a value in a variable. JavaScript evaluates multiplication and division before addition and subtraction; parentheses make intended grouping explicit. Use `===` for strict equality in beginner examples so that JavaScript does not silently coerce unlike types during comparison.

Worked Example

```
let score = 10;
score = score + 5;
const subtotal = 3 * 4 + 2;
const grouped = 3 * (4 + 2);

console.log(`Score: ${score}`);
console.log(`Subtotal: ${subtotal}`);
console.log(`Grouped: ${grouped}`);
console.log(`Is score 15? ${score === 15}`);
```

Expected output:

```
Score: 15
Subtotal: 14
```

```
Grouped: 18
Is score 15? true
```

Expected Behavior or Output

The program prints score 15, subtotal 14, grouped result 18, and strict comparison result true.

Common Mistakes

- Using `=` when a comparison with `===` is intended.
- Forgetting that `const` prevents reassignment of the binding.
- Assuming arithmetic with a string will always behave like arithmetic with a number.
- Omitting parentheses when they would make a multi-step expression easier to audit.

Activities

1. Predict and then verify `8 + 4 * 2` and `(8 + 4) * 2`.
2. Update a score from 82 by adding 9, then print the new value.
3. Create a bill expression with a 15% tip using the canonical $24.50 bill.
4. Explain why `"15" === 15` evaluates to false.

Accessibility Practice

When reading code aloud, identify parentheses, operators, and assignment separately. Keep one calculation per line when that makes the sequence easier to navigate.

Skills Checklist

☐ I can evaluate expressions using operator precedence.
☐ I can use parentheses to control grouping.
☐ I can update a variable with `let`.
☐ I can explain assignment versus strict equality.

Lesson 2.3: Input and Output Revisited

Reading References

- Python source counterpart: Unit 2, Lesson 2.3.
- Node.js documentation: readline.

Learning Objectives

- Read terminal input with Node.js `readline/promises`.
- Explain that terminal input is returned as a string.
- Display a greeting and the sum of two validated numeric inputs.

Concept Explanation

Unlike Python's built-in `input()`, Node.js uses an input API. The `readline/promises` module provides an asynchronous `question()` method. It returns text, so numeric input must be converted and checked. Always close the interface so the process can exit cleanly.

Worked Example

Save as `sum-input.js` and run with `node sum-input.js`:

```
const readline = require("node:readline/promises");
const { stdin, stdout } = require("node:process");

async function main() {
  const rl = readline.createInterface({ input: stdin, output: stdout });
  try {
    const name = await rl.question("What is your name? ");
    const firstText = await rl.question("Enter the first number: ");
    const secondText = await rl.question("Enter the second number: ");
    const first = Number(firstText);
    const second = Number(secondText);

    if (firstText.trim() === "" || secondText.trim() === ""
        || !Number.isFinite(first) || !Number.isFinite(second)) {
      console.log("Error: enter two valid numbers.");
      return;
    }

    console.log(`Hello, ${name}!`);
    console.log(`Sum: ${first + second}`);
  } finally {
    rl.close();
  }
}

main();
```

Example interaction:

```
What is your name? Avery
Enter the first number: 12
```

```
Enter the second number: 8
Hello, Avery!
Sum: 20
```

Expected Behavior or Output

For inputs Avery, 12, and 8, the program prints `Hello, Avery!` and `Sum: 20`. Blank or nonnumeric input produces the validation message.

Common Mistakes

- Adding two input strings and getting concatenation instead of numeric addition.
- Treating `Number("")` as invalid even though it evaluates to zero; validate blank text explicitly.
- Forgetting to await `question()`.
- Leaving the readline interface open.

Activities

1. Run the example with the name Avery and numbers 12 and 8.
2. Enter a blank value and confirm that the validation message appears.
3. Enter a decimal and a negative number; explain why both are valid numeric values.
4. Change the program to ask for the canonical age 16 and print the year of birth only after asking for a reference year.

Accessibility Practice

Use prompts that state exactly what input is expected. Announce the full error message and re-prompt when extending this example. Keep the interaction linear and keyboard-only.

Skills Checklist

- ☐ I can create a Node.js readline interface.
- ☐ I can read terminal input asynchronously.
- ☐ I can convert and validate numeric text.
- ☐ I can close the interface in a `finally` block.

Lesson 2.4: Computing with Numbers and Type Conversions

Reading References

- Python source counterpart: Unit 2, Lesson 2.4.

- MDN: Number, Number.isFinite, and Number.isInteger.

Learning Objectives

- Distinguish JavaScript numbers from numeric strings.
- Convert text using `Number()` and parse integer text when appropriate.
- Detect invalid numeric values, including `NaN`.

Concept Explanation

JavaScript has one ordinary `Number` type for integers and floating-point values. It uses IEEE 754 double-precision floating-point arithmetic. `Number("16")` returns 16; `Number("sixteen")` returns `NaN`. `parseInt()` parses an integer prefix, so it should not be used as a substitute for strict validation. Use `Number.isFinite()` to reject `NaN` and infinities.

Worked Example

```
const ageText = "16";
const heightText = "1.72";
const age = Number(ageText);
const height = Number(heightText);

console.log(`Age next year: ${age + 1}`);
console.log(`Height in centimeters: ${(height * 100).toFixed(0)}`);
console.log(`Is age valid? ${Number.isInteger(age) && age >= 0}`);
console.log(`Is 'sixteen' valid? ${Number.isFinite(Number("sixteen"))}`);
```

Expected output:

```
Age next year: 17
Height in centimeters: 172
Is age valid? true
Is 'sixteen' valid? false
```

Expected Behavior or Output

The program prints age next year 17, height 172 centimetres, a valid age result of true, and an invalid numeric-text result of false.

Common Mistakes

- Assuming JavaScript separates integers and floats into different primitive types.
- Forgetting that `typeof NaN` is `"number"`.
- Using `parseInt("12abc")` and accepting the partial result 12 without checking the entire input.

- Comparing floating-point calculations for exact equality when a tolerance is needed.

Activities

1. Convert the canonical age text `"16"` and height text `"1.72"` to numbers.
2. Convert `"abc"` and `""`; explain why blank input needs separate validation.
3. Calculate the bill total from $24.50 and a 0.15 tip rate.
4. Test whether 3.5 is an integer and whether `Infinity` is finite.

Accessibility Practice

Use textual labels that name the value and its unit. State why a conversion failed rather than only printing `NaN`.

Skills Checklist

☐ I can convert numeric strings with `Number()`.
☐ I can detect invalid numeric values with `Number.isFinite()`.
☐ I can distinguish integers from non-integer numbers.
☐ I can explain JavaScript's single ordinary Number type.

Lesson 2.5: Using the Math Object

Reading References

- Python source counterpart: Unit 2, Lesson 2.5.
- MDN: Math.

Learning Objectives

- Use `Math.PI`, `Math.sqrt()`, `Math.ceil()`, and `Math.floor()`.
- Calculate circle area and circumference.
- Calculate distance from the origin using the canonical coordinates (3, 4).

Concept Explanation

JavaScript's built-in `Math` object provides mathematical constants and functions. It is used directly; it is not imported. `Math.PI` is the value of pi, `Math.sqrt()` calculates a square root, `Math.ceil()` rounds upward, and `Math.floor()` rounds downward.

Worked Example

```
const radius = 5;
const area = Math.PI * radius ** 2;
```

```
const circumference = 2 * Math.PI * radius;

const x = 3;
const y = 4;
const distance = Math.sqrt(x ** 2 + y ** 2);

console.log(`Area: ${area.toFixed(2)}`);
console.log(`Circumference: ${circumference.toFixed(2)}`);
console.log(`Distance from origin: ${distance.toFixed(2)}`);
console.log(`Ceiling of 4.2: ${Math.ceil(4.2)}`);
console.log(`Floor of 4.8: ${Math.floor(4.8)}`);
```

Expected output:

```
Area: 78.54
Circumference: 31.42
Distance from origin: 5.00
Ceiling of 4.2: 5
Floor of 4.8: 4
```

Expected Behavior or Output

For radius 5 and coordinates (3, 4), the program prints area 78.54, circumference 31.42, distance 5.00, ceiling 5, and floor 4.

Common Mistakes

- Writing `Math.pi` instead of the case-sensitive `Math.PI`.
- Calling `Math.sqrt` without parentheses.
- Confusing `Math.round()` with ceiling or floor.
- Forgetting that `toFixed()` returns a string for display.

Activities

1. Calculate the area and circumference of a circle with radius 5.
2. Calculate the distance from (3, 4) to the origin.
3. Predict the results of `Math.ceil(2.01)` and `Math.floor(2.99)`.
4. Change the radius to 2.5 and report both results to two decimal places.

Accessibility Practice

Read each result with its label. Preserve units in prompts and output. Ensure mathematical notation is also explained in words when students are using a screen reader or braille display.

Skills Checklist

☐ I can use `Math.PI` and `Math.sqrt()`.

☐ I can calculate circle area and circumference.
☐ I can calculate distance from the origin.
☐ I can distinguish ceiling, floor, and rounding.

9.5 Unit 3: Control Flow — Decisions, Lists, and Strings

Overview

This unit follows the Python sequence of Boolean reasoning, chained conditionals, arrays, iteration, and string operations. Use the shared grade thresholds and score list throughout.

Lesson 3.1: Boolean Expressions and `if` Statements

Reading References

- Python source counterpart: Unit 3, Lesson 3.1.
- MDN: Expressions and operators.

Learning Objectives

- Compare values with relational operators.
- Combine conditions with `&&`, `||`, and `!`.
- Use an `if` statement to make a decision.

Concept Explanation

JavaScript Boolean values are `true` and `false`. Use strict equality (`===`) to avoid implicit type conversion during comparisons. Logical AND (`&&`) requires both conditions to be true; logical OR (`||`) requires at least one; logical NOT (`!`) reverses a Boolean value.

Worked Example

```
const score = 82;
const isPassing = score >= 60;
const isValid = score >= 0 && score <= 100;

if (isValid && isPassing) {
  console.log("Valid passing score.");
} else {
  console.log("Score is invalid or not passing.");
}
```

Expected output:

```
Valid passing score.
```

Expected Behavior or Output

With score 82, the program prints `Valid passing score.` because the score is between 0 and 100 and at least 60.

Common Mistakes

- Using `=` instead of `===` in a comparison.
- Confusing `&&` with `||`.
- Forgetting that JavaScript conditions can use truthy/falsy values, not only Booleans.

Activities

1. Test whether each canonical score is at least 60.
2. Write a condition that accepts scores from 0 through 100.
3. Compare `0`, `"0"`, `""`, and `null` using strict equality.
4. Explain the result of `true && false` and `true || false`.

Accessibility Practice

Print the score and decision as a complete sentence. Do not use color alone to indicate pass/fail.

Skills Checklist

- ☐ I can use relational operators.
- ☐ I can combine conditions with logical operators.
- ☐ I can write an if/else statement.
- ☐ I can explain strict equality.

Lesson 3.2: Chained Conditionals (`else if`, `else`)

Reading References

- Python source counterpart: Unit 3, Lesson 3.2.
- MDN: Conditional statements.

Learning Objectives

- Use ordered conditional branches.
- Apply the shared A–F grade thresholds.
- Test branch boundaries.

Concept Explanation

A chained conditional evaluates branches in order and runs the first matching branch. JavaScript uses `else if` for intermediate alternatives. The shared thresholds are A at 90, B at 80, C at 70, D at 60, and F below 60.

Worked Example

Reading References

- Python source counterpart: Unit 3, Lesson 3.2.
- MDN: Conditional statements.

Learning Objectives

- Use ordered conditional branches.
- Apply the shared A–F grade thresholds.
- Test branch boundaries.

Worked Example

```
function getGrade(score) {
  if (score >= 90) {
    return "A";
  } else if (score >= 80) {
    return "B";
  } else if (score >= 70) {
    return "C";
  } else if (score >= 60) {
    return "D";
  } else {
    return "F";
  }
}

for (const score of [82, 91, 76, 88]) {
  console.log(`Score ${score}: ${getGrade(score)}`);
}
```

Expected output:

```
Score 82: B
Score 91: A
Score 76: C
Score 88: B
```

Expected Behavior or Output

The canonical scores are classified as 82:B, 91:A, 76:C, and 88:B.

Common Mistakes

- Testing lower thresholds before higher ones.
- Omitting the final fallback branch.
- Failing to reject scores outside the valid range.

Activities

1. Classify scores 82, 91, 76, and 88.
2. Test 90, 89, 80, 79, 70, 69, 60, and 59.
3. Add validation for scores below 0 or above 100.
4. Explain why branch order matters.

Accessibility Practice

Print the numeric score and grade together, one record per line.

Skills Checklist

☐ I can write chained conditionals.
☐ I can implement the shared grade thresholds.
☐ I can test boundary values.
☐ I can validate a score range.

Lesson 3.3: Introduction to Arrays

Reading References

- Python source counterpart: Unit 3, Lesson 3.3.
- MDN: Indexed collections — Arrays.

Learning Objectives

- Create, access, and modify arrays.
- Explain zero-based indexing and `length`.
- Use the canonical scores [82, 91, 76, 88].

Concept Explanation

JavaScript arrays are ordered and zero-indexed. The first element is at index 0; the last is at `array.length - 1`. A `const` array can still have its elements changed because `const` prevents reassignment of the binding, not mutation of the array.

Worked Example

```
const scores = [82, 91, 76, 88];
console.log(`First score: ${scores[0]}`);
```

```
console.log(`Number of scores: ${scores.length}`);
scores[2] = 80;
console.log(`Updated third score: ${scores[2]}`);
```

Expected output:

```
First score: 82
Number of scores: 4
Updated third score: 80
```

Expected Behavior or Output

The initial array has four values. The first score is 82; after assigning index 2 to 80, the third score is 80.

Common Mistakes

- Using index 1 for the first element.
- Assuming `const` freezes array contents.
- Reading beyond the array and receiving `undefined`.

Activities

1. Read all four canonical scores by index.
2. Change the third score from 76 to 80.
3. Add a fifth score of 95 and report the new length.
4. Explain the difference between an index and a one-based display number.

Accessibility Practice

Identify values by both position and meaning. Print one score per line.

Skills Checklist

- ☐ I can create and access an array.
- ☐ I can explain zero-based indexing.
- ☐ I can update an array element.
- ☐ I can use the length property.

Lesson 3.4: Iterating with `for` Loops

Concept Explanation

A `for...of` loop visits each value in an iterable collection. The loop variable holds the current score, not its array index.

Reading References

- Python source counterpart: Unit 3, Lesson 3.4.
- MDN: Loops and iteration.

Learning Objectives

- Iterate over arrays with `for...of`.
- Calculate a total and average.
- Compare indexed loops with value-based iteration.

Concept Explanation

A `for...of` loop visits each value in an iterable collection. The loop variable holds the current score, not its array index.

Worked Example

Worked Example

```
const scores = [82, 91, 76, 88];
let total = 0;

for (const score of scores) {
  total += score;
}

console.log(`Total: ${total}`);
console.log(`Average: ${(total / scores.length).toFixed(2)}`);
```

Expected output:

```
Total: 337
Average: 84.25
```

Expected Behavior or Output

The canonical score list totals 337 and has an average of 84.25.

Common Mistakes

- Confusing an array index with an array value.
- Modifying the collection while iterating without understanding the effect.
- Using `forEach()` when early termination with `break` is required.

Activities

1. Print each canonical score using `for...of`.
2. Calculate the total and average.

3. Print each score with a one-based display number.
4. Compare `for...of`, indexed `for`, and `forEach()`.

Accessibility Practice

Use one output line per score and include a meaningful label. Do not rely on visual alignment.

Skills Checklist

- ☐ I can iterate over an array.
- ☐ I can calculate a total and average.
- ☐ I can use an indexed loop.
- ☐ I can choose an appropriate loop form.

Lesson 3.5: Basic String Operations

Reading References

- Python source counterpart: Unit 3, Lesson 3.5.
- MDN: String.

Learning Objectives

- Concatenate strings and use template literals.
- Normalize text and search within it.
- Count string length and replace text.

Concept Explanation

JavaScript strings are immutable. Methods such as `trim()`, `toUpperCase()`, and `toLowerCase()` return new strings. Template literals use backticks and `${expression}` interpolation.

Worked Example

Worked Example

```
const firstName = " Avery ";
const lastName = "Student";
const fullName = `${firstName.trim()} ${lastName}`;
const sample = "A student studies programming. Programming develops problem-solving skills."

console.log(`Name: ${fullName}`);
console.log(`Uppercase: ${fullName.toUpperCase()}`);
console.log(`Contains programming: ${sample.toLowerCase().includes("programming")}`);
console.log(`Characters in name: ${fullName.length}`);
```

Expected output:

```
Name: Avery Student
Uppercase: AVERY STUDENT
Contains programming: true
Characters in name: 13
```

Expected Behavior or Output

The example prints `Avery Student`, its uppercase form, true for the programming search, and a character count of 13.

Common Mistakes

- Assuming string methods mutate the original string.
- Forgetting to save the result of `trim()` or case conversion.
- Treating UTF-16 code-unit length as the count of user-perceived characters in all Unicode text.

Activities

1. Trim extra spaces from a name.
2. Convert the canonical text sample to lowercase.
3. Count the occurrences of the word "programming".
4. Replace "skills" with "abilities".

Accessibility Practice

Use complete labels for string length and search results. Explain punctuation and whitespace explicitly.

Skills Checklist

☐ I can concatenate strings and use template literals.
☐ I can trim and change case.
☐ I can search and replace text.
☐ I can describe the Unicode caveat for string length.

9.6 Unit 4: Data Structures — Arrays, Fixed Sequences, and Objects

Overview

This unit maps Python lists, tuples, and dictionaries to idiomatic JavaScript. JavaScript arrays are mutable sequences; a frozen array can express fixed-sequence intent; plain objects represent records; and `Map` represents a dynamic key-value collection.

Lesson 4.1: More on Arrays

Concept Explanation

Array methods such as `push()`, `pop()`, and `splice()` mutate an array. Methods such as `slice()` return a new array. Numeric sorting requires a comparator because the default sort is lexicographic.

Reading References

- Python source counterpart: Unit 4, Lesson 4.1.
- MDN: Array methods.

Learning Objectives

- Add, insert, update, and remove array elements.
- Distinguish mutating methods from methods that return a new array.
- Preserve and process the canonical score data.

Concept Explanation

Array methods such as `push()`, `pop()`, and `splice()` mutate an array. Methods such as `slice()` return a new array. Numeric sorting requires a comparator because the default sort is lexicographic.

Worked Example

```
const scores = [82, 91, 76, 88];
scores.push(95);
scores.splice(1, 0, 85);
const removed = scores.pop();
console.log(`Removed: ${removed}`);
console.log(`Scores: ${scores.join(", ")}`);
```

Expected output:

```
Removed: 95
Scores: 82, 85, 91, 76, 88
```

Expected Behavior or Output

The example removes 95 and prints the remaining scores in the order 82, 85, 91, 76, 88.

Common Mistakes

- Confusing `splice()` (mutates) with `slice()` (returns a copy).
- Assuming `pop()` returns the updated array rather than the removed element.

- Sorting numbers without a numeric comparator.

Activities

1. Append 95 to the canonical scores.
2. Insert 85 at index 1.
3. Remove the final element and report the returned value.
4. Sort a copy numerically with `(a, b) => a - b`.

Accessibility Practice

Announce whether an operation mutates the original array or returns a new one. Read resulting values sequentially.

Skills Checklist

☐ I can add and remove array elements.
☐ I can insert at an index.
☐ I can distinguish mutating and non-mutating methods.
☐ I can sort numeric arrays correctly.

Lesson 4.2: Organizing Arrays

Concept Explanation

Sorting and reversing can mutate their receiver. Spread syntax creates a shallow copy, allowing a separate primitive-value array to be sorted or reversed without changing the original.

Reading References

- Python source counterpart: Unit 4, Lesson 4.2.
- MDN: Array sorting and iteration methods.

Learning Objectives

- Sort and reverse arrays.
- Create filtered and transformed arrays.
- Explain mutation and copying.

Concept Explanation

Sorting and reversing can mutate their receiver. Spread syntax creates a shallow copy, allowing a separate primitive-value array to be sorted or reversed without changing the original.

Worked Example

```
const scores = [82, 91, 76, 88];
const sorted = [...scores].sort((a, b) => a - b);
const reversed = [...sorted].reverse();
console.log(`Original: ${scores.join(", ")}`);
console.log(`Sorted: ${sorted.join(", ")}`);
console.log(`Reversed: ${reversed.join(", ")}`);
```

Expected output:

```
Original: 82, 91, 76, 88
Sorted: 76, 82, 88, 91
Reversed: 91, 88, 82, 76
```

Expected Behavior or Output

The original order remains 82, 91, 76, 88; the sorted copy is 76, 82, 88, 91; the reversed copy is 91, 88, 82, 76.

Common Mistakes

- Calling `sort()` without a comparator for numeric data.
- Reversing the original array when a copy was intended.
- Assuming spread syntax makes a deep copy of nested objects.

Activities

1. Sort the canonical scores from lowest to highest.
2. Reverse a copy and preserve the original order.
3. Filter scores of 80 or higher.
4. Explain why a shallow copy is sufficient for an array of primitive numbers but not necessarily for nested objects.

Accessibility Practice

Label each result as original, sorted, reversed, or filtered. Avoid presenting the distinction only through visual ordering.

Skills Checklist

- ☐ I can sort numbers with a comparator.
- ☐ I can reverse a copy of an array.
- ☐ I can filter an array.
- ☐ I can explain shallow copying.

Lesson 4.3: Tuples — Immutable Sequences in JavaScript

Reading References

- Python source counterpart: Unit 4, Lesson 4.3.
- MDN: Object.freeze().

Learning Objectives

- Explain that JavaScript has no built-in tuple type with Python tuple semantics.
- Use a frozen array to express fixed-sequence intent.
- Distinguish a constant binding from immutable object contents.

Concept Explanation

JavaScript has no native tuple type. For a short fixed sequence, a frozen array can communicate that the values should not be changed. `Object.freeze()` is shallow: nested objects are not automatically frozen. TypeScript has tuple types, but those are outside this JavaScript unit.

Worked Example

```
"use strict";
const coordinates = Object.freeze([3, 4]);
console.log(`x: ${coordinates[0]}`);
console.log(`y: ${coordinates[1]}`);
console.log(`Distance: ${Math.hypot(coordinates[0], coordinates[1])}`);
```

Expected Behavior or Output

```
x: 3
y: 4
Distance: 5
```

Attempting to change a frozen array element in strict mode throws a TypeError.

Common Mistakes

- Assuming `const` freezes array contents.
- Assuming `Object.freeze()` is deep.
- Describing a frozen array as a true language-level tuple.

Activities

1. Create a frozen coordinate pair using the canonical values (3, 4).
2. Read each coordinate by index and by a descriptive label.
3. Attempt to change an element in strict mode and observe the error.
4. Compare the frozen coordinates with a mutable scores array.

Accessibility Practice

Identify each position by semantic name (x or y), not only by index. Read the fixed-sequence contract aloud before attempting mutation.

Skills Checklist

- ☐ I can explain why JavaScript has no direct Python tuple equivalent.
- ☐ I can use a frozen array for fixed-sequence intent.
- ☐ I can distinguish const binding from object immutability.
- ☐ I can explain the shallow behavior of Object.freeze().

Lesson 4.4: Introducing Dictionaries — JavaScript Objects and Map

Reading References

- Python source counterpart: Unit 4, Lesson 4.4.
- MDN: Working with objects and Map.

Learning Objectives

- Represent a record with an object literal.
- Use Map for dynamic key-value storage.
- Add, retrieve, update, and test entries.

Concept Explanation

A plain object is appropriate for a record with named fields such as name, quantity, and unit price. `Map` is appropriate for a dynamic key-value collection and supports keys of any value type. A missing object property or Map key returns `undefined`, not zero.

Worked Example

```
const inventory = [
  { name: "Notebook", quantity: 3, unitPrice: 2.50 },
  { name: "Pencil", quantity: 10, unitPrice: 0.50 },
  { name: "Folder", quantity: 4, unitPrice: 1.25 }
];

const quantities = new Map(inventory.map(item => [item.name, item.quantity]));
console.log(quantities.get("Notebook"));
console.log(quantities.has("Pencil"));
quantities.set("Pencil", 12);
console.log(quantities.get("Pencil"));
```

Expected Behavior or Output

```
3
true
12
```

The canonical inventory contains three records. Its total value is $17.50.

Common Mistakes

- Confusing object properties with Map entries.
- Using bracket notation on a Map instead of `get()` and `set()`.
- Assuming a missing key returns zero rather than `undefined`.

Activities

1. Create the three canonical inventory records.
2. Create a Map from item names to quantities.
3. Update the Pencil quantity from 10 to 12.
4. Calculate the total value of the original inventory and explain the effect of the quantity update.

Accessibility Practice

Print item names, quantities, and prices as labeled text. Use one inventory record per output line.

Skills Checklist

- ☐ I can represent a record with a plain object.
- ☐ I can create and use a Map.
- ☐ I can add, retrieve, update, and test Map entries.
- ☐ I can distinguish objects from Maps.

Lesson 4.5: Looping Through Objects and Maps

Reading References

- Python source counterpart: Unit 4, Lesson 4.5.
- MDN: Object.entries() and Map iteration.

Learning Objectives

- Traverse object keys, values, and key-value pairs.
- Iterate over Map entries.
- Produce readable reports from structured data.

Concept Explanation

Use `Object.keys()`, `Object.values()`, or `Object.entries()` to traverse an object. A Map is directly iterable and yields key-value pairs. Choose the method that matches the data structure rather than assuming the APIs are interchangeable.

Worked Example

```
const student = {
  name: "Avery",
  age: 16,
  heightM: 1.72,
  favoriteSubject: "Computer Science"
};

for (const [key, value] of Object.entries(student)) {
  console.log(`${key}: ${value}`);
}
```

Expected Behavior or Output

```
name: Avery
age: 16
heightM: 1.72
favoriteSubject: Computer Science
```

Common Mistakes

- Iterating over object keys when key-value pairs are needed.
- Modifying a collection during traversal without considering the effect.
- Assuming Map and object iteration APIs are interchangeable.

Activities

1. Print every field in Avery's canonical student profile.
2. Iterate over the inventory Map and print each item and quantity.
3. Calculate the total number of inventory units.
4. Compare `Object.keys()`, `Object.values()`, and `Object.entries()`.

Accessibility Practice

Provide one key-value pair per line and announce when iteration begins and ends. Keep output order stable for instructional examples.

Skills Checklist

☐ I can traverse object keys, values, and entries.

☐ I can traverse Map entries.
☐ I can choose the appropriate traversal API.
☐ I can produce sequential textual reports.

9.7 Unit 5: User Input and `while` Loops

Overview

This unit builds on terminal input and introduces sentinel-controlled repetition, loop flags, `break`, and `continue`. The goal is to create predictable interactive programs with explicit stopping conditions.

Lesson 5.1: How Terminal Input Works

Reading References

- Python source counterpart: Unit 5, Lesson 5.1.
- Node.js documentation: `node:readline/promises`.

Learning Objectives

- Explain why Node.js terminal input is asynchronous.
- Read, trim, and validate a text response.
- Close a readline interface reliably.

Concept Explanation

Node.js does not provide a built-in synchronous `input()` function. Use `readline/promises` and `await question()` to obtain a response. The result is a string; conversion and validation are separate steps.

Worked Example

```
const readline = require("node:readline/promises");
const { stdin, stdout } = require("node:process");

async function main() {
  const rl = readline.createInterface({ input: stdin, output: stdout });
  try {
    const response = (await rl.question("Enter your favorite subject: ")).trim();
    if (response === "") {
      console.log("Error: a subject is required.");
    } else {
      console.log(`Favorite subject: ${response}`);
    }
  } finally {
    rl.close();
```

```
  }
}
main();
```

Example interaction:

```
Enter your favorite subject: Computer Science
Favorite subject: Computer Science
```

Expected Behavior or Output

For the input Computer Science, the program prints `Favorite subject: Computer Science`. Blank input prints the required-field error.

Common Mistakes

- Forgetting `await` before `question()`.
- Treating the response as a number without converting it.
- Failing to close the interface.
- Accepting an empty string when a nonempty response is required.

Activities

1. Ask for the canonical student name Avery and print a greeting.
2. Ask for age, convert it with `Number()`, and reject blank or non-finite input.
3. Ask for a menu choice and normalize it with `trim()`.
4. Explain the purpose of `try/finally` around the readline interface.

Accessibility Practice

Use descriptive prompts, announce validation errors in text, and keep the prompt sequence linear. All steps must work from the terminal without mouse interaction.

Skills Checklist

☐ I can create a readline interface.
☐ I can await terminal input.
☐ I can validate and normalize text input.
☐ I can close the interface reliably.

Lesson 5.2: The `while` Loop in Action

Reading References

- Python source counterpart: Unit 5, Lesson 5.2.
- MDN: `while` statement.

Learning Objectives

- Write a condition-controlled loop.
- Use a sentinel value to terminate repetition.
- Explain the difference between a `for` loop and a `while` loop.

Concept Explanation

A `while` loop repeats as long as its condition is true. A sentinel is a special input value that signals completion. The loop must update state so its condition can eventually become false.

Worked Example

```
const scores = [82, 91, 76, 88];
let index = 0;
let total = 0;

while (index < scores.length) {
  total += scores[index];
  index += 1;
}

console.log(`Total: ${total}`);
console.log(`Average: ${(total / scores.length).toFixed(2)}`);
```

Expected output:

```
Total: 337
Average: 84.25
```

Expected Behavior or Output

The canonical score list produces total 337 and average 84.25.

Common Mistakes

- Forgetting to increment the index.
- Using a condition that is already false before the first iteration.
- Updating the index after a branch that can skip the update.

Activities

1. Count from 1 through 5 with a `while` loop.
2. Sum the canonical scores and calculate their average.
3. Design a sentinel-controlled menu that stops when the user enters "quit".
4. Trace the value of the loop index before and after each iteration.

Accessibility Practice

State the loop condition and termination rule before running the program. Provide a textual completion message.

Skills Checklist

- ☐ I can write a condition-controlled loop.
- ☐ I can use a sentinel to stop repetition.
- ☐ I can update loop state correctly.
- ☐ I can choose between `for` and `while`.

Lesson 5.3: Using Flags and `break` to Exit Loops

Reading References

- Python source counterpart: Unit 5, Lesson 5.3.
- MDN: `break` statement.

Learning Objectives

- Use a Boolean flag to record a condition.
- Stop a loop early with `break`.
- Distinguish normal loop completion from early termination.

Concept Explanation

A flag stores whether a condition has been found or satisfied. `break` exits the nearest loop immediately; code after the loop can then report the final result.

Worked Example

```
const scores = [82, 91, 76, 88];
let found = false;

for (const score of scores) {
  if (score > 90) {
    found = true;
    console.log(`First score above 90: ${score}`);
    break;
  }
}

if (!found) {
  console.log("No score above 90 was found.");
}
```

Expected output:

```
First score above 90: 91
```

Expected Behavior or Output

The canonical scores produce `First score above 90: 91`; if no value matches, the fallback message is printed.

Common Mistakes

- Using a string such as "false" instead of the Boolean value `false`.
- Forgetting to set the flag when the condition is met.
- Placing `break` outside a loop or switch statement.

Activities

1. Search the canonical scores for the first score greater than 90.
2. Repeat the search for a score greater than 95 and report that none exists.
3. Use a flag to validate that a required inventory item exists.
4. Explain when `break` is clearer than changing the loop condition.

Accessibility Practice

Report both the search result and whether a match was found. Do not rely on color or a visual marker.

Skills Checklist

- ☐ I can initialize and update a Boolean flag.
- ☐ I can use `break` to stop a loop.
- ☐ I can report successful and unsuccessful searches.
- ☐ I can explain early termination.

Lesson 5.4: Using `continue` in Loops

Reading References

- Python source counterpart: Unit 5, Lesson 5.4.
- MDN: `continue` statement.

Learning Objectives

- Skip the current iteration with `continue`.
- Validate data before processing it.
- Explain the difference between `continue` and `break`.

Concept Explanation

The `continue` statement skips the remainder of the current iteration and starts the next one. It does not end the loop.

Worked Example

```
const entries = ["82", "", "91", "invalid", "76", "88"];
let total = 0;
let validCount = 0;

for (const entry of entries) {
  const score = Number(entry);
  if (entry.trim() === "" || !Number.isFinite(score)) {
    console.log("Skipping invalid score.");
    continue;
  }
  total += score;
  validCount += 1;
}

console.log(`Valid scores: ${validCount}`);
console.log(`Total: ${total}`);
```

Expected output:

```
Skipping invalid score.
Skipping invalid score.
Valid scores: 4
Total: 337
```

Expected Behavior or Output

The invalid blank and nonnumeric entries are skipped; four valid scores remain and their total is 337.

Common Mistakes

- Using `continue` when the entire loop should end.
- Skipping a loop-control update in a `while` loop.
- Failing to explain why an entry was skipped.

Activities

1. Process the canonical scores while skipping negative values.
2. Add blank and nonnumeric entries and confirm they are rejected.
3. Count accepted and rejected entries separately.
4. Rewrite a short loop using `if/else` instead of `continue`; compare readability.

Accessibility Practice

Print a concise reason for each rejected entry. Keep accepted and rejected counts explicit.

Skills Checklist

- ☐ I can use `continue` to skip an iteration.
- ☐ I can validate data before processing.
- ☐ I can distinguish `continue` from `break`.
- ☐ I can report rejected input clearly.

Lesson 5.5: Avoiding Infinite Loops

Reading References

- Python source counterpart: Unit 5, Lesson 5.5.
- MDN: Loops and iteration.

Learning Objectives

- Identify loop conditions that never become false.
- Trace and repair loop-control logic.
- Test both normal and boundary termination.

Concept Explanation

An infinite loop occurs when the loop condition remains true indefinitely. Before execution, identify the variable that changes, the condition that ends the loop, and every path through the body. For interactive programs, include an explicit quit command.

Worked Example

```
let count = 1;
while (count <= 3) {
  console.log(`Count: ${count}`);
  count += 1;
}
console.log("Loop finished.");
```

Expected output:

```
Count: 1
Count: 2
Count: 3
Loop finished.
```

Expected Behavior or Output

The loop prints Count 1, Count 2, Count 3, then Loop finished. It terminates after exactly three iterations.

Common Mistakes

- Never changing a variable used in the condition.
- Incrementing in the wrong direction.
- Creating a path through the loop that bypasses the update.
- Testing only the normal path and not the stopping boundary.

Activities

1. Trace the worked example by hand before running it.
2. Diagnose a loop that decrements a counter while testing whether it is less than 10.
3. Add a quit command to a menu loop.
4. Test the loop with its initial value already at the boundary.

Accessibility Practice

Announce the expected maximum number of iterations for bounded examples. Stop runaway processes with the terminal interrupt command and describe the recovery procedure.

Skills Checklist

☐ I can identify the termination condition.
☐ I can trace loop state changes.
☐ I can diagnose common infinite-loop causes.
☐ I can test loop boundary cases.

9.8 Unit 6: Functions

Overview

Functions support decomposition, reuse, and testing. JavaScript supports declarations, expressions, arrow functions, default parameters, rest parameters, and closures. Unlike Python, JavaScript does not support Python-style keyword arguments; use an options object when named configuration is useful.

Lesson 6.1: Defining Functions

Reading References

- Python source counterpart: Unit 6, Lesson 6.1.
- MDN: Functions.

Learning Objectives

- Define and call a function.
- Explain parameters and function bodies.
- Choose a function declaration or expression.

Concept Explanation

A function is a reusable block of code. Parameters receive input values, and `return` sends a result back to the caller.

Worked Example

```
function calculateTip(bill, rate) {
  return bill * rate;
}

const tip = calculateTip(24.50, 0.15);
console.log(`Tip: $${tip.toFixed(2)}`);
```

Expected output:

```
Tip: $3.67
```

Expected Behavior or Output

The canonical bill and tip rate produce a displayed tip of $3.67.

Common Mistakes

- Calling a function before understanding its parameter order.
- Omitting `return` when the caller needs a result.
- Confusing a function definition with a function call.

Activities

1. Define `calculateTip` using the canonical bill and tip rate.
2. Define a function that returns the average of the canonical scores.
3. Call each function with at least two different inputs.
4. Explain why returning a value is more reusable than printing inside the function.

Accessibility Practice

Use descriptive function and parameter names. Read the function declaration, call, and returned result as separate steps.

Skills Checklist

- ☐ I can define and call a function.
- ☐ I can identify parameters and arguments.
- ☐ I can return a computed value.
- ☐ I can use a function to reduce repeated code.

Lesson 6.2: Passing Arguments

Reading References

- Python source counterpart: Unit 6, Lesson 6.2.
- MDN: Function parameters.

Learning Objectives

- Pass positional arguments in the correct order.
- Use default parameters.
- Use an options object to model named configuration.

Concept Explanation

JavaScript arguments are positional. It does not have Python's `name=value` call syntax. For functions with several optional settings, pass one object whose properties describe each setting.

Worked Example

```
function calculateTotal(bill, tipRate = 0.15) {
  return bill + bill * tipRate;
}

function describeStudent({ name, age, favoriteSubject }) {
  return `${name}, age ${age}, studies ${favoriteSubject}.`;
}

console.log(calculateTotal(24.50).toFixed(2));
console.log(calculateTotal(24.50, 0.20).toFixed(2));
console.log(describeStudent({
  name: "Avery",
  age: 16,
  favoriteSubject: "Computer Science"
}));
```

Expected output:

```
28.18
29.40
Avery, age 16, studies Computer Science.
```

Expected Behavior or Output

The default tip rate produces 28.18; the 0.20 rate produces 29.40; the student description names Avery and Computer Science.

Common Mistakes

- Swapping positional arguments that have the same primitive type.
- Assuming JavaScript supports Python keyword arguments.
- Forgetting that a default parameter is used for an omitted or `undefined` argument, not for `null`.

Activities

1. Call `calculateTotal` with the default rate and with a 0.20 rate.
2. Create an options object for the canonical student profile.
3. Omit one required property and decide how the function should validate it.
4. Explain the difference between omitted, `undefined`, and `null`.

Accessibility Practice

Prefer clear property names over unexplained positional values when a function accepts several related settings.

Skills Checklist

☐ I can pass positional arguments.
☐ I can define default parameters.
☐ I can use an options object.
☐ I can explain JavaScript's argument semantics.

Lesson 6.3: Return Values

Reading References

- Python source counterpart: Unit 6, Lesson 6.3.
- MDN: return.

Learning Objectives

- Return values from functions.
- Distinguish returning from printing.
- Handle early returns for invalid inputs.

Concept Explanation

A return value is data supplied to the caller. Printing is a side effect and does not substitute for returning a value.

Worked Example

```
function average(scores) {
  if (!Array.isArray(scores) || scores.length === 0) {
    return null;
  }
  const total = scores.reduce((sum, score) => sum + score, 0);
  return total / scores.length;
}

const result = average([82, 91, 76, 88]);
console.log(`Average: ${result.toFixed(2)}`);
console.log(`Empty input: ${average([])}`);
```

Expected output:

```
Average: 84.25
Empty input: null
```

Expected Behavior or Output

The canonical score average is 84.25 and an empty array returns null.

Common Mistakes

- Expecting a function that only calls `console.log()` to return a useful value.
- Forgetting that a function without an explicit return produces `undefined`.
- Returning a formatted string when later calculations require a number.

Activities

1. Write a function that returns the total of the canonical scores.
2. Write a function that returns `null` for an empty list.
3. Use the returned average in a second calculation.
4. Compare a function that prints a result with one that returns it.

Accessibility Practice

Keep calculation results and explanatory output distinct. Use explicit labels when displaying a returned value.

Skills Checklist

- ☐ I can return a value from a function.
- ☐ I can use early returns for invalid cases.
- ☐ I can distinguish `undefined`, `null`, and a computed value.
- ☐ I can reuse returned values in later calculations.

Lesson 6.4: Functions with Arrays and Objects

Reading References

- Python source counterpart: Unit 6, Lesson 6.4.
- MDN: Working with objects and arrays.

Learning Objectives

- Pass arrays and objects to functions.
- Process the canonical score and inventory data.
- Explain that object and array values are passed by sharing their references.

Concept Explanation

Arrays and objects are passed as values that refer to the same underlying object. A function can therefore mutate caller-visible state unless it works on a copy.

Worked Example

```
function inventoryValue(items) {
  return items.reduce(
    (total, item) => total + item.quantity * item.unitPrice,
    0
  );
}

const inventory = [
  { name: "Notebook", quantity: 3, unitPrice: 2.50 },
  { name: "Pencil", quantity: 10, unitPrice: 0.50 },
  { name: "Folder", quantity: 4, unitPrice: 1.25 }
];

console.log(`Inventory value: $${inventoryValue(inventory).toFixed(2)}`);
```

Expected output:

```
Inventory value: $17.50
```

Expected Behavior or Output

The canonical inventory value is $17.50.

Common Mistakes

- Mutating an array or object inside a function without documenting the side effect.
- Assuming a new variable created with `const` makes the referenced object immutable.

- Returning the wrong data shape for the caller.

Activities

1. Write a function that returns the average of the canonical scores.
2. Write a function that returns the total value of the canonical inventory.
3. Add a new inventory item and verify the total changes by its line value.
4. Write a non-mutating function that returns a filtered list of scores at least 80.

Accessibility Practice

Use named object properties and label each returned total. Explain whether a function changes the original collection.

Skills Checklist

☐ I can pass arrays and objects to functions.
☐ I can process a collection with `reduce()` or a loop.
☐ I can explain reference sharing and mutation.
☐ I can return a predictable result shape.

Lesson 6.5: Arbitrary Numbers of Arguments

Reading References

- Python source counterpart: Unit 6, Lesson 6.5.
- MDN: Rest parameters.

Learning Objectives

- Use rest parameters to collect arbitrary positional arguments.
- Use spread syntax to pass array elements as arguments.
- Explain the difference between rest and spread syntax.

Concept Explanation

A rest parameter gathers remaining positional arguments into an array. Spread syntax expands an iterable into individual arguments at a call site.

Worked Example

```
function sumScores(...scores) {
  return scores.reduce((total, score) => total + score, 0);
}

const values = [82, 91, 76, 88];
```

```
console.log(sumScores(...values));
console.log(sumScores(10, 20, 30));
```

Expected output:

```
337
60
```

Expected Behavior or Output

The four canonical scores sum to 337; the values 10, 20, and 30 sum to 60.

Common Mistakes

- Confusing rest parameters in a function definition with spread syntax in a call.
- Placing a rest parameter anywhere other than the final parameter position.
- Passing a whole array when the function expects individual numeric arguments.

Activities

1. Call `sumScores` with the four canonical scores.
2. Call it with three different values.
3. Pass an array using spread syntax.
4. Explain why JavaScript rest parameters do not provide a direct equivalent to Python's `**kwargs`.

Accessibility Practice

Use parameter names that communicate the expected value type. Keep the function's input contract explicit.

Skills Checklist

- ☐ I can define a rest parameter.
- ☐ I can use spread syntax in a function call.
- ☐ I can distinguish rest from spread.
- ☐ I can explain the lack of Python-style keyword-argument collection.

9.9 Unit 7: Classes and Object-Oriented Programming

Overview

This unit uses the same student and inventory domains as the Python course. JavaScript classes use prototypes under the hood; private fields begin with `#`. Composition is often preferable to inheritance when one object contains another.

Lesson 7.1: Why Learn About Classes?

Reading References

- Python source counterpart: Unit 7, Lesson 7.1.
- MDN: Classes.

Learning Objectives

- Explain classes, objects, attributes, and methods.
- Describe encapsulation and abstraction.
- Identify an appropriate domain object.

Concept Explanation

A class defines the structure and behavior shared by its instances. An inventory item can store a name, quantity, and unit price and provide a method that calculates its line value.

Worked Example

```
class InventoryItem {
  constructor(name, quantity, unitPrice) {
    this.name = name;
    this.quantity = quantity;
    this.unitPrice = unitPrice;
  }

  lineValue() {
    return this.quantity * this.unitPrice;
  }
}

const notebook = new InventoryItem("Notebook", 3, 2.50);
console.log(`${notebook.name}: $${notebook.lineValue().toFixed(2)}`);
```

Expected output:

```
Notebook: $7.50
```

Expected Behavior or Output

An InventoryItem for three Notebooks at $2.50 each prints `Notebook: $7.50`.

Common Mistakes

- Confusing the class with an instance.
- Omitting `this` when accessing instance properties.
- Defining behavior that belongs to an object as unrelated global variables.

Activities

1. Create an InventoryItem for each canonical inventory record.
2. Print each item's line value.
3. Create a Student class using Avery's profile fields.
4. Explain which data belongs in an instance and which behavior belongs in a method.

Accessibility Practice

Use descriptive property and method names. Announce object values by property name rather than relying on visual object inspection.

Skills Checklist

- ☐ I can explain a class and an instance.
- ☐ I can define a constructor and method.
- ☐ I can use `this` to access instance properties.
- ☐ I can identify encapsulation in a small class.

Lesson 7.2: Creating and Using a Class

Reading References

- Python source counterpart: Unit 7, Lesson 7.2.
- MDN: constructor.

Learning Objectives

- Define a class with a constructor.
- Instantiate multiple objects.
- Validate constructor arguments.

Concept Explanation

A class constructor initializes each new instance. The `new` operator creates an object and calls the class constructor with the supplied arguments.

Worked Example

```
class Student {
  constructor(name, age, heightM, favoriteSubject) {
    if (typeof name !== "string" || name.trim() === "") {
      throw new TypeError("Student name is required.");
    }
    this.name = name;
    this.age = age;
    this.heightM = heightM;
```

```
    this.favoriteSubject = favoriteSubject;
  }
}

const student = new Student("Avery", 16, 1.72, "Computer Science");
console.log(`${student.name}, age ${student.age}`);
```

Expected output:

```
Avery, age 16
```

Expected Behavior or Output

The canonical Student instance prints `Avery, age 16`.

Common Mistakes

- Forgetting the `new` keyword.
- Using constructor parameters without assigning them to the instance.
- Failing to validate required data.

Activities

1. Create two Student instances with different values.
2. Print each student's name and favorite subject.
3. Add validation that age is a nonnegative integer.
4. Deliberately pass an invalid name and read the error.

Accessibility Practice

Make validation errors specific and textual. Keep constructor parameters in a consistent, documented order.

Skills Checklist

☐ I can define a constructor.
☐ I can instantiate multiple objects.
☐ I can validate constructor inputs.
☐ I can explain the role of `new`.

Lesson 7.3: Working with Attributes and Methods

Reading References

- Python source counterpart: Unit 7, Lesson 7.3.
- MDN: Private class elements.

Learning Objectives

- Read and update object properties.
- Define methods that validate or transform state.
- Use a private field for encapsulated state.

Concept Explanation

Methods define behavior associated with an instance. Private fields beginning with `#` can be accessed only from within the declaring class.

Worked Example

```
class InventoryItem {
  #quantity;

  constructor(name, quantity, unitPrice) {
    this.name = name;
    this.#quantity = quantity;
    this.unitPrice = unitPrice;
  }

  updateQuantity(newQuantity) {
    if (!Number.isInteger(newQuantity) || newQuantity < 0) {
      throw new RangeError("Quantity must be a nonnegative integer.");
    }
    this.#quantity = newQuantity;
  }

  describe() {
    return `${this.name}: quantity ${this.#quantity}`;
  }
}

const pencil = new InventoryItem("Pencil", 10, 0.50);
pencil.updateQuantity(12);
console.log(pencil.describe());
```

Expected output:

```
Pencil: quantity 12
```

Expected Behavior or Output

After updating the Pencil quantity to 12, the method prints `Pencil: quantity 12`.

Common Mistakes

- Accessing a private field from outside its class.
- Updating state without validating it.
- Confusing a method call with a property read.

Activities

1. Add a method that returns the item's line value.
2. Reject a negative quantity.
3. Update the Notebook quantity from 3 to 5.
4. Explain why a private field can protect an object's invariants.

Accessibility Practice

Use methods that return clear text or values. State the validation rule before asking learners to test it.

Skills Checklist

☐ I can read and update public properties.
☐ I can define and call methods.
☐ I can use a private class field.
☐ I can validate state changes.

Lesson 7.4: Inheritance

Reading References

- Python source counterpart: Unit 7, Lesson 7.4.
- MDN: Inheritance and the prototype chain.

Learning Objectives

- Define a subclass with `extends`.
- Call a parent constructor with `super()`.
- Override a method when specialized behavior is required.

Concept Explanation

Inheritance models an is-a relationship. A subclass uses `extends` and calls its parent constructor with `super()` before accessing `this`.

Worked Example

```
class InventoryItem {
  constructor(name, quantity) {
    this.name = name;
```

```
    this.quantity = quantity;
  }

  describe() {
    return `${this.name}: ${this.quantity} in stock`;
  }
}

class PerishableItem extends InventoryItem {
  constructor(name, quantity, expirationDate) {
    super(name, quantity);
    this.expirationDate = expirationDate;
  }

  describe() {
    return `${super.describe()}, expires ${this.expirationDate}`;
  }
}

const milk = new PerishableItem("Milk", 2, "2026-11-10");
console.log(milk.describe());
```

Expected output:

```
Milk: 2 in stock, expires 2026-11-10
```

Expected Behavior or Output

The PerishableItem example prints `Milk: 2 in stock, expires 2026-11-10`.

Common Mistakes

- Calling `this` before `super()` in a derived constructor.
- Forgetting to pass required parent arguments.
- Using inheritance when composition would be simpler.

Activities

1. Create a PerishableItem subclass with an expiration date.
2. Override the description method.
3. Test an instance with the canonical inventory quantity.
4. Identify one situation where composition is preferable.

Accessibility Practice

Describe the parent/child relationship explicitly. Keep overridden method behavior predictable and document what changes.

Skills Checklist

- ☐ I can extend a class.
- ☐ I can call a parent constructor with `super()`.
- ☐ I can override a method.
- ☐ I can explain when inheritance is appropriate.

Lesson 7.5: Instances as Attributes (Composition)

Reading References

- Python source counterpart: Unit 7, Lesson 7.5.
- MDN: Working with objects.

Learning Objectives

- Model a has-a relationship.
- Store one object inside another object.
- Explain composition as an alternative to inheritance.

Concept Explanation

Composition models a has-a relationship by storing one object as a property of another. It often provides a simpler alternative to inheritance.

Worked Example

```
class Student {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }
}

class StudentRecord {
  constructor(student, favoriteSubject) {
    this.student = student;
    this.favoriteSubject = favoriteSubject;
  }

  describe() {
    return `${this.student.name} studies ${this.favoriteSubject}.`;
  }
}

const record = new StudentRecord(
  new Student("Avery", 16),
  "Computer Science"
```

```
);
console.log(record.describe());
```

Expected output:

```
Avery studies Computer Science.
```

Expected Behavior or Output

The StudentRecord example prints `Avery studies Computer Science.`.

Common Mistakes

- Copying an object's fields when a related object should be retained.
- Confusing has-a composition with is-a inheritance.
- Creating circular object references without a clear reason.

Activities

1. Create a Student object for Avery.
2. Place the Student object inside a StudentRecord.
3. Add a second related object, such as a contact record.
4. Explain why StudentRecord is not a subclass of Student.

Accessibility Practice

Use explicit relationship names such as `student` and `contact`. Print nested values with labels.

Skills Checklist

☐ I can model a has-a relationship.
☐ I can store an object as another object's property.
☐ I can explain composition versus inheritance.
☐ I can access nested object properties.

9.10 Unit 8: Files and Exceptions

Overview

This unit uses the same `notes.txt` and structured task/student records as the Python course. Node.js file APIs are asynchronous by default; use promises and `try/catch/finally` to make resource behavior explicit.

Lesson 8.1: Reading from a File

Reading References

- Python source counterpart: Unit 8, Lesson 8.1.

- Node.js documentation: `node:fs/promises`.

Learning Objectives

- Read a UTF-8 text file.
- Distinguish whole-file reads from line-by-line processing.
- Handle a missing file with a clear error.

Concept Explanation

The `node:fs/promises` API returns promises for file operations. UTF-8 decoding returns a string; omitting the encoding returns a Buffer.

Worked Example

Create `notes.txt` containing:

```
Practice JavaScript.
Review arrays and functions.
```

Read it:

```
const fs = require("node:fs/promises");

async function main() {
  try {
    const contents = await fs.readFile("notes.txt", "utf8");
    console.log("File contents:");
    console.log(contents.trimEnd());
  } catch (error) {
    if (error.code === "ENOENT") {
      console.log("Error: notes.txt was not found.");
    } else {
      throw error;
    }
  }
}

main();
```

Expected output:

```
File contents:
Practice JavaScript.
Review arrays and functions.
```

Expected Behavior or Output

With the sample notes.txt file present, both note lines are printed under the File contents label.

Common Mistakes

- Omitting the encoding and receiving a Buffer.
- Using the wrong working directory for a relative path.
- Ignoring the rejected promise from file I/O.

Activities

1. Create and read the canonical notes.txt file.
2. Rename the file and verify the missing-file message.
3. Read the file and count its lines.
4. Explain the difference between a relative and absolute path.

Accessibility Practice

Report the file name and each line in reading order. Use textual errors that identify the missing path.

Skills Checklist

☐ I can read a UTF-8 file with `fs.readFile()`.
☐ I can explain relative paths.
☐ I can handle a missing file.
☐ I can distinguish text from Buffer data.

Lesson 8.2: Writing to a File

Reading References

- Python source counterpart: Unit 8, Lesson 8.2.
- Node.js documentation: `writeFile()` and `appendFile()`.

Learning Objectives

- Create or overwrite a text file.
- Append content without replacing existing content.
- Confirm successful file operations.

Concept Explanation

`writeFile()` creates or replaces file content; `appendFile()` adds content. Await each operation before relying on its result.

Worked Example

```
const fs = require("node:fs/promises");

async function main() {
  await fs.writeFile("notes.txt", "Practice JavaScript.\n", "utf8");
  await fs.appendFile("notes.txt", "Review arrays and functions.\n", "utf8");
  const saved = await fs.readFile("notes.txt", "utf8");
  console.log(saved.trimEnd());
  console.log("Notes saved.");
}

main().catch(error => {
  console.error(`File error: ${error.message}`);
  process.exitCode = 1;
});
```

Expected output:

```
Practice JavaScript.
Review arrays and functions.
Notes saved.
```

Expected Behavior or Output

The two note lines are printed, followed by `Notes saved.`.

Common Mistakes

- Using write mode when the intention is to append.
- Forgetting to await a write before reading the file.
- Assuming a successful call means the path is the intended one.

Activities

1. Write a one-line note to notes.txt.
2. Append a second line.
3. Run the program twice and explain whether the content is overwritten or appended.
4. Verify the saved text by reading it back programmatically.

Accessibility Practice

Print a confirmation that identifies the file and operation. Do not rely on a file icon or color to indicate success.

Skills Checklist

- ☐ I can write a text file.

☐ I can append to a text file.
☐ I can await file operations.
☐ I can verify saved content.

Lesson 8.3: Exceptions — Handling Errors

Reading References

- Python source counterpart: Unit 8, Lesson 8.3.
- MDN: Error and try...catch.

Learning Objectives

- Handle exceptions with `try/catch`.
- Distinguish expected errors from unexpected errors.
- Define a custom error class.

Concept Explanation

An exception interrupts normal control flow. `try/catch` handles expected failures, and custom Error subclasses can carry additional context.

Worked Example

```
class ValidationError extends Error {
  constructor(field, message) {
    super(message);
    this.name = "ValidationError";
    this.field = field;
  }
}

function validateAge(age) {
  if (!Number.isInteger(age) || age < 0) {
    throw new ValidationError("age", "Age must be a nonnegative integer.");
  }
  return age;
}

try {
  console.log(validateAge(16));
  validateAge(-1);
} catch (error) {
  if (error instanceof ValidationError) {
    console.log(`Validation failed for ${error.field}: ${error.message}`);
  } else {
    throw error;
```

```
  }
}
```

Expected output:

```
16
Validation failed for age: Age must be a nonnegative integer.
```

Expected Behavior or Output

The valid age 16 is printed; invalid age -1 produces the field-specific validation message.

Common Mistakes

- Catching every error and silently ignoring it.
- Throwing a string instead of an Error object.
- Failing to rethrow unexpected errors.

Activities

1. Validate that Avery's age is 16.
2. Test negative age, decimal age, and text input.
3. Create a custom error for an invalid inventory quantity.
4. Explain why unexpected errors should not be hidden.

Accessibility Practice

Error messages should identify the field, invalid value when safe, and correction expected. Keep error output textual.

Skills Checklist

☐ I can use `try/catch`.
☐ I can throw an Error object.
☐ I can define a custom error class.
☐ I can distinguish expected and unexpected errors.

Lesson 8.4: The `else` and `finally` Patterns

Reading References

- Python source counterpart: Unit 8, Lesson 8.4.
- MDN: try...catch...finally.

Learning Objectives

- Explain code that runs after success or regardless of success.
- Use `finally` for cleanup.

- Recognize that JavaScript does not have Python's `try/except/else` syntax.

Concept Explanation

JavaScript has `try`, `catch`, and `finally`, but no direct `else` clause attached to a try statement. Put success-only work after the operation inside the try block. Put cleanup in `finally`.

Worked Example

```
async function loadNotes() {
  let contents;
  try {
    contents = await require("node:fs/promises").readFile("notes.txt", "utf8");
  } catch (error) {
    console.log(`Unable to read notes: ${error.message}`);
    return;
  } finally {
    console.log("Read attempt finished.");
  }

  console.log("Read succeeded.");
  console.log(contents.trimEnd());
}

loadNotes();
```

Expected Behavior or Output

The final status message is printed whether the read succeeds or fails. The success message and file contents appear only when the read succeeds.

Common Mistakes

- Assuming JavaScript supports Python's `try/except/else` syntax.
- Placing success-only code in `finally`.
- Returning from `finally` and unintentionally overriding an earlier return or exception.

Activities

1. Run the example with notes.txt present.
2. Run it with notes.txt missing.
3. Identify which messages appear in each case.
4. Explain why cleanup belongs in `finally`.

Accessibility Practice

Use separate success and failure messages. Ensure the final status is announced regardless of outcome.

Skills Checklist

- ☐ I can explain JavaScript's try/catch/finally structure.
- ☐ I can identify success-only operations.
- ☐ I can use finally for cleanup.
- ☐ I can explain the absence of a direct try-else clause.

Lesson 8.5: Storing Data with JSON

Reading References

- Python source counterpart: Unit 8, Lesson 8.5.
- MDN: JSON.

Learning Objectives

- Convert JavaScript objects to JSON text.
- Parse JSON text into JavaScript values.
- Save and reload structured records.

Concept Explanation

JSON is a text representation of structured data. `JSON.stringify()` serializes supported JavaScript values; `JSON.parse()` reconstructs values from valid JSON text.

Worked Example

```
const fs = require("node:fs/promises");

async function main() {
  const tasks = [
    { id: 1, title: "Review arrays", done: false },
    { id: 2, title: "Practice functions", done: true }
  ];

  const jsonText = JSON.stringify(tasks, null, 2);
  await fs.writeFile("tasks.json", jsonText, "utf8");

  const loadedText = await fs.readFile("tasks.json", "utf8");
  const loadedTasks = JSON.parse(loadedText);
  console.log(`Loaded ${loadedTasks.length} tasks.`);
  console.log(`First task: ${loadedTasks[0].title}`);
```

```
}

main().catch(error => {
  console.error(`Storage error: ${error.message}`);
  process.exitCode = 1;
});
```

Expected output:

```
Loaded 2 tasks.
First task: Review arrays
```

Expected Behavior or Output

The example prints `Loaded 2 tasks.` and `First task: Review arrays` after saving and reloading the data.

Common Mistakes

- Passing an already serialized string to `JSON.stringify()` and double-encoding it.
- Parsing malformed JSON without catching SyntaxError.
- Assuming JSON preserves functions, undefined values, or every JavaScript type.

Activities

1. Save and reload the canonical task fields: id, title, description, done, created_at.
2. Add a task and verify it survives a second program run.
3. Deliberately corrupt the JSON and handle the parse error.
4. Compare the in-memory object with the JSON text stored on disk.

Accessibility Practice

Use pretty-printed JSON for manual inspection and provide a textual confirmation after saving or loading.

Skills Checklist

- ☐ I can serialize objects with `JSON.stringify()`.
- ☐ I can parse JSON with `JSON.parse()`.
- ☐ I can save and reload structured data.
- ☐ I can handle malformed JSON.

9.11 Unit 9: Testing and Data Analysis

Overview

This unit follows the Python sequence: test a function, test a class, generate random values, analyze text, and process CSV data. Reuse the canonical score list, text sample, inventory, and CSV column names.

Lesson 9.1: Testing a Function

Reading References

- Python source counterpart: Unit 9, Lesson 9.1.
- Node.js documentation: Test runner (`node:test`).

Learning Objectives

- Write unit tests for deterministic functions.
- Use assertions for expected and actual values.
- Run tests from the terminal.

Concept Explanation

A unit test calls a deterministic function with known inputs and compares the returned value with an expected result. Tests should include ordinary inputs and boundary conditions.

Worked Example

Create `math.js`:

```
function add(a, b) {
  return a + b;
}

function getGrade(score) {
  if (score >= 90) return "A";
  if (score >= 80) return "B";
  if (score >= 70) return "C";
  if (score >= 60) return "D";
  return "F";
}

module.exports = { add, getGrade };
```

Create `math.test.js`:

```
const test = require("node:test");
const assert = require("node:assert/strict");
```

```
const { add, getGrade } = require("./math");

test("add returns the sum", () => {
  assert.equal(add(5, 3), 8);
  assert.equal(add(-5, -3), -8);
  assert.equal(add(0, 7), 7);
});

test("getGrade follows shared thresholds", () => {
  assert.equal(getGrade(90), "A");
  assert.equal(getGrade(82), "B");
  assert.equal(getGrade(76), "C");
  assert.equal(getGrade(60), "D");
  assert.equal(getGrade(59), "F");
});
```

Run with `node --test`.

Expected result: two passing tests and zero failures.

Expected Behavior or Output

The Node.js test runner reports two passing tests and no failures when add() and getGrade() satisfy the stated contracts.

Common Mistakes

- Testing only one typical value and ignoring boundaries.
- Writing assertions that merely repeat the implementation.
- Forgetting to export the function being tested.
- Making tests depend on execution order or shared mutable state.

Activities

1. Add tests for the canonical scores 82, 91, 76, and 88.
2. Add boundary tests for every grade threshold.
3. Add a test for invalid scores and decide what the function contract should be.
4. Run the test file from the terminal and interpret the pass/fail summary.

Accessibility Practice

Read each test name and assertion result. Ensure failures identify the expected and actual values in text.

Skills Checklist

☐ I can write a test with `node:test`.

- ☐ I can assert expected values.
- ☐ I can test normal and boundary cases.
- ☐ I can run tests from the terminal.

Lesson 9.2: Testing a Class

Reading References

- Python source counterpart: Unit 9, Lesson 9.2.
- Node.js documentation: Test runner and assert.

Learning Objectives

- Test object construction, state changes, and methods.
- Verify both valid and invalid operations.
- Keep test cases independent.

Concept Explanation

A class test verifies observable behavior of an instance: construction, method results, valid state changes, and rejected invalid changes.

Worked Example

Assume `InventoryItem` exposes `quantity`, `updateQuantity()`, and `lineValue()`. A test file can verify its public behavior:

```
const test = require("node:test");
const assert = require("node:assert/strict");
const { InventoryItem } = require("./inventory-item");

test("inventory item updates quantity and calculates value", () => {
  const item = new InventoryItem("Notebook", 3, 2.50);
  assert.equal(item.lineValue(), 7.50);
  item.updateQuantity(5);
  assert.equal(item.lineValue(), 12.50);
});

test("inventory item rejects a negative quantity", () => {
  const item = new InventoryItem("Pencil", 10, 0.50);
  assert.throws(() => item.updateQuantity(-1), RangeError);
});
```

Expected result: both tests pass when the class enforces its stated contract.

Expected Behavior or Output

Both tests pass when the item value updates from $7.50 to $12.50 and a negative quantity throws RangeError.

Common Mistakes

- Testing private implementation details instead of public behavior.
- Reusing one mutated instance across unrelated tests.
- Forgetting to test invalid state transitions.

Activities

1. Test a Notebook item with quantity 3 and unit price 2.50.
2. Verify its line value after changing quantity to 5.
3. Verify that a negative quantity is rejected.
4. Add a test for a zero quantity and explain whether it is valid.

Accessibility Practice

Give each test a descriptive name that states the behavior being verified. Avoid ambiguous test labels such as "test 1".

Skills Checklist

☐ I can instantiate an object in a test.
☐ I can test a method's return value.
☐ I can test a state change.
☐ I can assert that invalid operations throw.

Lesson 9.3: Introduction to Random Numbers

Reading References

- Python source counterpart: Unit 9, Lesson 9.3.
- MDN: Math.random.

Learning Objectives

- Generate random floating-point values and bounded integers.
- Select a random element from an array.
- Explain why random tests should not depend on a specific result.

Concept Explanation

Math.random() returns a pseudorandom floating-point value greater than or equal to zero and less than one. Transform it to a bounded integer with multiplication and Math.floor().

Worked Example

```
const randomDecimal = Math.random();
const randomInteger = Math.floor(Math.random() * 10) + 1;
```

```
const colors = ["red", "blue", "green"];
const randomChoice = colors[Math.floor(Math.random() * colors.length)];

console.log(`Decimal in [0, 1): ${randomDecimal >= 0 && randomDecimal < 1}`);
console.log(`Integer in 1..10: ${randomInteger >= 1 && randomInteger <= 10}`);
console.log(`Selected item is valid: ${colors.includes(randomChoice)}`);
```

Expected output: three lines containing `true`; the sampled values vary on each run.

Expected Behavior or Output

Each run prints true for all three range and membership checks; the sampled values may differ.

Common Mistakes

- Assuming `Math.random()` can return exactly 1.
- Forgetting that `Math.floor()` is needed for an integer index.
- Writing a test that expects one particular random result.

Activities

1. Generate a random integer from 1 through 10.
2. Select one of the canonical inventory item names.
3. Run the program several times and observe that values vary.
4. Explain why randomness makes exact-output tests unreliable without a controllable source.

Accessibility Practice

Label the range and meaning of each random value. Never require a visual animation to observe a random result.

Skills Checklist

- ☐ I can use `Math.random()`.
- ☐ I can generate a bounded integer.
- ☐ I can select a random array element.
- ☐ I can test properties of random output rather than one exact sample.

Lesson 9.4: Analyzing Text Data

Reading References

- Python source counterpart: Unit 9, Lesson 9.4.
- MDN: String and Map.

Learning Objectives

- Normalize text before analysis.
- Count words and character frequencies.
- Use a Map to store frequency counts.

Concept Explanation

Text analysis requires explicit normalization and tokenization rules. A Map stores each token and its current frequency; the regular expression in this example treats hyphenated words as separate alphabetic tokens.

Worked Example

```
const sample = "A student studies programming. Programming develops problem-solving skills."

function countWords(text) {
  const words = text.toLowerCase().match(/[a-z]+/g) ?? [];
  const counts = new Map();

  for (const word of words) {
    counts.set(word, (counts.get(word) ?? 0) + 1);
  }
  return counts;
}

const frequencies = countWords(sample);
for (const [word, count] of frequencies) {
  console.log(`${word}: ${count}`);
}
```

Expected output:

```
a: 1
student: 1
studies: 1
programming: 2
develops: 1
problem: 1
solving: 1
skills: 1
```

Expected Behavior or Output

The sample produces nine tokens and eight distinct token keys; programming has frequency 2 and every other displayed token has frequency 1.

Common Mistakes

- Counting uppercase and lowercase forms as separate words.
- Splitting on a single space and retaining punctuation.
- Assuming a simple regular expression handles every language's word boundaries.

Activities

1. Count the words in the canonical text sample.
2. Identify the most frequent word.
3. Count characters after normalizing case.
4. Explain how punctuation and hyphenation affect the result.

Accessibility Practice

Print each word and count as a labeled pair. State the normalization rules so learners can understand why the counts have those values.

Skills Checklist

☐ I can normalize text before counting.
☐ I can tokenize a simple English text sample.
☐ I can store frequencies in a Map.
☐ I can explain limitations of a simple tokenizer.

Lesson 9.5: Working with CSV Data

Reading References

- Python source counterpart: Unit 9, Lesson 9.5.
- npm documentation: `csv-parse`.

Learning Objectives

- Explain rows, columns, headers, and records.
- Read CSV data with a standards-aware parser.
- Compute a summary from the canonical column set.

Concept Explanation

CSV is a text format with quoting and escaping rules. Splitting every line on commas is not a correct general CSV parser because a quoted field may itself contain a comma or newline. Use a maintained parser for real CSV data. The canonical headers are `student_id`, `first_name`, `last_name`, `score`, and `active`.

Worked Example

Install the parser with `npm install csv-parse`. Save a small fixture as `students.csv`:

```
student_id,first_name,last_name,score,active
1,Avery,Student,91,true
2,Jordan,Lee,82,true
3,Casey,Smith,76,false
```

Read it with `csv-parse`:

```
const fs = require("node:fs");
const { parse } = require("csv-parse/sync");

const records = parse(fs.readFileSync("students.csv", "utf8"), {
  columns: true,
  skip_empty_lines: true,
  trim: true
});

const average = records.reduce((sum, row) => sum + Number(row.score), 0)
  / records.length;

console.log(`Loaded students: ${records.length}`);
console.log(`Average score: ${average.toFixed(2)}`);
```

Expected output:

```
Loaded students: 3
Average score: 83.00
```

Expected Behavior or Output

The sample CSV contains three records with scores 91, 82, and 76. The average is 83.00.

Common Mistakes

- Splitting CSV lines on commas without handling quoted fields.
- Forgetting that parsed field values are strings until converted.
- Assuming every row contains every required column.
- Failing to handle a missing or empty file.

Activities

1. Create a CSV with the five canonical column names.
2. Load the records and print each student's name and score.
3. Calculate the average score.

4. Add a quoted field containing a comma and verify that the parser reads it as one field.

Accessibility Practice

Print one record per line with column names. Provide a textual summary of any malformed rows rather than relying on a spreadsheet's visual highlighting.

Skills Checklist

- ☐ I can explain CSV headers and records.
- ☐ I can parse CSV with a standards-aware library.
- ☐ I can convert numeric fields before calculating.
- ☐ I can summarize a dataset with labeled output.

9.12 Unit 10: Project-Based Learning — Text-Based Applications

Overview

The projects preserve the Python track's requirements and user interaction. The text adventure uses the shared five-room graph (`start_room`, `tunnel`, `clearing`, `locked_door_room`, and `treasure_room`) and the shiny-key puzzle. The command-line to-do manager supports add, view, complete, delete, and JSON persistence.

Lesson 10.1: Designing Your Project

Reading References

- Python source counterpart: Unit 10, Lesson 10.1.
- MDN: JavaScript modules and functions.

Learning Objectives

- Apply top-down design to a multi-part program.
- Identify inputs, outputs, data, and edge cases.
- Write pseudocode and acceptance tests before implementation.

Concept Explanation

Top-down design breaks a large task into smaller responsibilities. The project should have separate functions for input, state changes, display, and persistence. Define command names and error behavior before writing code.

Worked Example

Text-adventure pseudocode:

```
CREATE rooms and exits
PLACE items in selected rooms
CREATE player at starting room
REPEAT
  DISPLAY current room and player status
  READ command
  VALIDATE command
  APPLY movement or inventory action
UNTIL player enters quit
```

Expected Behavior or Output

The learner produces pseudocode, a room/command specification, and acceptance tests before implementation.

Common Mistakes

- Starting implementation before defining the project requirements.
- Combining input, state changes, and display in one large function.
- Omitting invalid-command and boundary cases.

Activities

1. Draw or describe the five-room map in text.
2. List every valid movement command and its destination.
3. Define the to-do manager's five menu operations.
4. Write at least five acceptance tests before implementation.

Accessibility Practice

Use keyboard-operable commands, descriptive prompts, and predictable text output. Do not require maps, color, drag-and-drop, or mouse-only interaction.

Skills Checklist

☐ I can decompose a project into functions or modules.
☐ I can define inputs and outputs.
☐ I can write pseudocode.
☐ I can define acceptance tests and error cases.

Lesson 10.2: Building a Text-Based Adventure Game (Part 1)

Reading References

- Python source counterpart: Unit 10, Lesson 10.2.
- Node.js documentation: readline/promises.

Learning Objectives

- Model the five canonical rooms and their exits.
- Move through connected rooms without changing state on invalid directions.
- Explain the locked north exit from locked_door_room.

Shared Project Specification

Use the canonical five-room graph: start_room north to tunnel; tunnel south to start_room and east to clearing; clearing west to tunnel and north to locked_door_room; locked_door_room south to clearing and conditionally north to treasure_room after using shiny key. The player starts in start_room. Valid commands include north, south, east, west, take shiny key, use shiny key, and quit.

Concept Explanation

A room is represented by its identifier, description, and a map of valid exits. Movement looks up the requested direction and changes the player's current room only when the exit exists.

Worked Example

```
const rooms = {
  start_room: { exits: { north: "tunnel" }, items: [] },
  tunnel: { exits: { south: "start_room", east: "clearing" }, items: [] },
  clearing: { exits: { west: "tunnel", north: "locked_door_room" }, items: [] },
  locked_door_room: { exits: { south: "clearing" }, items: [] },
  treasure_room: { exits: {}, items: [] }
};
function move(currentRoom, direction) {
  return rooms[currentRoom].exits[direction] ?? null;
}
console.log(move("start_room", "north"));
console.log(move("start_room", "west"));
```

Expected output:

```
tunnel
null
```

Expected Behavior or Output

Moving north from start_room returns tunnel. Moving west from start_room returns null; the locked-door north exit is added only after the key puzzle is solved.

Common Mistakes

- Adding an exit in one direction but forgetting the reverse connection.
- Accepting an exit that does not exist.
- Confusing a room key with its display name.

Activities

1. Build the five canonical rooms and their unconditional exits.
2. Test north from start_room, east from tunnel, and north from clearing.
3. Test an invalid movement without changing the current room.
4. Add the conditional north exit from locked_door_room only after the key is used.

Accessibility Practice

Announce the current room, description, and available exits in a consistent order. Never make the map available only as a graphic.

Skills Checklist

☐ I can represent rooms and exits as objects.
☐ I can move between connected rooms.
☐ I can reject invalid directions.
☐ I can display room information sequentially.

Lesson 10.3: Building a Text-Based Adventure Game (Part 2)

Reading References

- Python source counterpart: Unit 10, Lesson 10.3.
- MDN: Array methods and object methods.

Learning Objectives

- Place shiny key in start_room and transfer it to inventory exactly once.
- Require the key to be used in locked_door_room.
- Unlock the conditional north exit to treasure_room.

Shared Project Specification

Place only the shiny key in start_room. The player uses `take shiny key` and `use shiny key`. The key must be used while in locked_door_room; this unlocks its north exit to treasure_room. Preserve these item names and rules across language tracks.

Concept Explanation

An item belongs either to a room or to the player's inventory. Picking it up removes it from the room and adds it to inventory as one state transition.

Worked Example

```
const rooms = {
  start_room: { exits: { north: "tunnel" } },
  tunnel: { exits: { south: "start_room", east: "clearing" } },
  clearing: { exits: { west: "tunnel", north: "locked_door_room" } },
  locked_door_room: { exits: { south: "clearing" } },
  treasure_room: { exits: {} }
};
const roomItems = { start_room: ["shiny key"], tunnel: [], clearing: [], locked_door_room:
const inventory = [];
let doorLocked = true;
function takeItem(room, requestedName) {
  const items = roomItems[room];
  const index = items.findIndex(item => item.toLowerCase() === requestedName.toLowerCase());
  if (index < 0) return false;
  inventory.push(items.splice(index, 1)[0]);
  return true;
}
function useKey(room, items) {
  if (room !== "locked_door_room") return "You must be at the locked door to use the key.";
  if (!items.includes("shiny key")) return "The door is locked.";
  doorLocked = false;
  rooms.locked_door_room.exits.north = "treasure_room";
  return "The door unlocks; north now leads to treasure_room.";
}
function runCommand(room, command) {
  if (command.startsWith("take ")) return takeItem(room, command.slice(5));
  if (command === "use shiny key") return useKey(room, inventory);
  return "I don't understand that command.";
}
console.log(runCommand("start_room", "take shiny key"));
console.log(runCommand("start_room", "take shiny key"));
console.log(inventory.join(", "));
console.log(runCommand("clearing", "use shiny key"));
```

```
console.log(runCommand("locked_door_room", "use shiny key"));
console.log(doorLocked);
console.log(rooms.locked_door_room.exits.north);
```

Expected output:

```
true
shiny key
You must be at the locked door to use the key.
The door unlocks; north now leads to treasure_room.
false
treasure_room
```

Expected Behavior or Output

The command dispatcher accepts the literal `take shiny key` and `use shiny key` commands. The second pickup is rejected without duplicating the inventory item. Key use is rejected outside `locked_door_room`; using it there unlocks the north exit.

```
true
false
shiny key
You must be at the locked door to use the key.
The door unlocks; north now leads to treasure_room.
false
treasure_room
```

Common Mistakes

- Allowing an item to be picked up repeatedly after it has been removed.
- Failing to normalize item names consistently.
- Updating the room but not the inventory, or vice versa.

Activities

1. Take shiny key from start_room.
2. Attempt to take it a second time and verify inventory does not change.
3. Attempt to use the key outside locked_door_room.
4. Use the key in locked_door_room and verify the north exit leads to treasure_room.

Accessibility Practice

Confirm each successful pickup by item name. When pickup fails, explain whether the item is absent or the command is invalid.

Skills Checklist

- ☐ I can store items in rooms.
- ☐ I can transfer an item to inventory.
- ☐ I can display inventory contents.
- ☐ I can implement and test a puzzle condition.

Lesson 10.4: Developing a Command-Line To-Do Tool

Reading References

- Python source counterpart: Unit 10, Lesson 10.4.
- Node.js documentation: File system and readline.

Learning Objectives

- Implement add, view, complete, delete, and quit operations.
- Validate a one-based task number.
- Persist task data in JSON.

Concept Explanation

A command-line task manager separates data operations from prompts and display. Task numbers shown to users are one-based, while array indexes are zero-based.

Shared Project Specification Each task has an identifier, description, completion state, and creation timestamp. The menu supports add task, view tasks, mark complete, delete, and quit. Save after every state-changing operation.

Worked Example

Core task operations:

```
function addTask(tasks, description, createdAt = new Date().toISOString()) {
  if (description.trim() === "") {
    throw new Error("Task description cannot be empty.");
  }
  tasks.push({
    id: tasks.length + 1,
    description: description.trim(),
    completed: false,
    createdAt
  });
}

function markComplete(tasks, taskNumber) {
  const index = taskNumber - 1;
```

```
  if (!Number.isInteger(taskNumber) || index < 0 || index >= tasks.length) {
    return false;
  }
  tasks[index].completed = true;
  return true;
}
```

Expected Behavior or Output

A valid task description is added with completed false and a timestamp; a valid task number marks the corresponding task complete.

Common Mistakes

- Confusing the user's one-based task number with a zero-based array index.
- Allowing empty descriptions.
- Saving only when the program exits, so an interrupted run loses changes.
- Reusing task identifiers after deletion without a documented policy.

Activities

1. Add two tasks and view them in numbered order.
2. Mark the first task complete.
3. Delete the second task.
4. Save and reload the task list using JSON and verify the state persists.

Accessibility Practice

Number every task and include an explicit completion label such as ”complete” or ”not complete”. Confirm every add, update, and delete action in text.

Skills Checklist

☐ I can add and display tasks.
☐ I can mark a task complete.
☐ I can delete a task by number.
☐ I can persist tasks as JSON.
☐ I can validate task descriptions and numbers.

Lesson 10.5: Refinement and User Experience

Reading References

- Python source counterpart: Unit 10, Lesson 10.5.
- Node.js documentation: Process exit codes and error handling.

Learning Objectives

- Improve prompt clarity and validation.
- Test invalid commands and boundary cases.
- Review output for predictable keyboard and screen-reader use.

Concept Explanation

A usable text program gives the same kind of response for the same kind of action. Prompts should state valid input, errors should explain the problem, and successful operations should confirm what changed. A help command should list every supported operation.

Expected Behavior or Output

The refined application handles blank input, unknown commands, invalid record numbers, empty lists, and malformed stored data with explicit text messages.

Worked Example

```
function validateMenuChoice(choice) {
  const validChoices = ["add", "view", "search", "edit", "delete", "help", "quit"];
  if (!validChoices.includes(choice.trim().toLowerCase())) {
    return "Invalid choice. Enter add, view, search, edit, delete, help, or quit.";
  }
  return "Command accepted.";
}
console.log(validateMenuChoice("unknown"));
```

Expected Behavior or Output

The refined application handles blank input, unknown commands, invalid record numbers, empty lists, and malformed stored data with explicit text messages.

Common Mistakes

- Printing ”Invalid input” without explaining valid choices.
- Failing to confirm a successful action.
- Letting malformed stored data crash the entire program.
- Relying on visual formatting without textual labels.

Activities

1. Add a help command to the to-do tool.
2. Test blank input, unknown commands, invalid task numbers, and an empty task list.
3. Review every prompt for a clear expected input format.

4. Ask a partner to complete all tasks using keyboard navigation only and record obstacles.

Accessibility Practice

Use consistent separators, descriptive prompts, explicit errors, and sequential output. Do not claim a screen-reader test unless it has actually been performed.

Skills Checklist

☐ I can validate user input.
☐ I can provide specific error messages.
☐ I can confirm successful operations.
☐ I can review a CLI for keyboard and screen-reader usability.

9.13 Unit 11: Capstone Project — Accessible Personal Data Manager

Overview

Build a complete command-line personal data manager in JavaScript and Node.js. The implementation may choose a Contact Manager, Book Tracker, Recipe Collection, or Budget Tracker, but the data model, operations, persistence, accessibility, and acceptance tests must be equivalent across the language tracks. Unlike the Python implementation, which introduces NiceGUI, this JavaScript track uses a terminal interface so the core capstone remains independent of a browser UI.

Lesson 11.1: Project Introduction and Interface Design

Reading References

- Python source counterpart: Unit 11, Lesson 11.1.
- Node.js documentation: readline/promises.

Learning Objectives

- Select a capstone domain.
- Translate requirements into a menu and command set.
- Design a keyboard-first text interface.

Concept Explanation

The capstone is assessed on data operations and accessibility, not visual interface fluency. The terminal interface must support adding, viewing, searching, editing, deleting, and saving records. A help command must list all commands.

Worked Example

A minimal command menu might list: add, view, search, edit, delete, help, and quit. Each command maps to one named function.

Expected Behavior or Output

The project plan contains a selected domain, a record schema, command descriptions, and acceptance tests.

Common Mistakes

- Choosing a domain before identifying required fields and operations.
- Designing a visual-only interaction when the project requires an accessible text interface.
- Omitting help and invalid-command behavior.

Activities

1. Choose one project domain.
2. List the fields each record requires.
3. Design the menu and exact prompt wording.
4. Write acceptance tests for all required operations.

Accessibility Practice

Every action must be possible from the keyboard. Prompts must name the expected field and format. Output must be sequential and understandable without visual layout.

Skills Checklist

☐ I can select a project domain.
☐ I can define a record schema.
☐ I can design a text-only command interface.
☐ I can write acceptance tests.

Lesson 11.2: Data Model and Storage

Reading References

- Python source counterpart: Unit 11, Lesson 11.2.
- Node.js documentation: fs/promises and JSON.

Learning Objectives

- Define a record model.
- Persist records to JSON.
- Handle missing, malformed, and inaccessible files.

Concept Explanation

The data model defines the fields and invariants of each record. JSON persistence stores the records between runs; load and save operations must handle file and parse failures.

Canonical Record Fields Use these shared fields for the generic personal data manager: `id`, `title`, `description`, `category`, `created_at`, and `updated_at`. A selected domain may add fields, but must not remove the shared fields.

Worked Example

A record contains id, title, description, category, created_at, and updated_at. The id is unique; timestamps use ISO 8601 strings.

Expected Behavior or Output

After saving and reloading, all record fields and values are preserved. Missing files can initialize an empty collection; malformed JSON produces a clear error.

Common Mistakes

- Treating parsed JSON as trusted data without validation.
- Reusing record identifiers.
- Overwriting valid data after a parse failure without informing the user.

Activities

1. Define the record shape and document each field.
2. Create a sample JSON file containing at least three records.
3. Implement load and save functions.
4. Test missing files, malformed JSON, and an empty data set.

Accessibility Practice

Use human-readable JSON during development. Provide explicit save/load confirmations and specific error messages.

Skills Checklist

☐ I can define the record schema.
☐ I can load and save JSON.
☐ I can preserve timestamps and identifiers.
☐ I can handle missing and malformed data.

Concept Explanation

The data model defines the fields and invariants of each record. JSON persistence stores the records between runs; load and save operations must handle file and parse failures.

Lesson 11.3: Displaying Items

Reading References

- Python source counterpart: Unit 11, Lesson 11.3.
- Node.js documentation: console output and Array iteration.

Learning Objectives

- Display every record with a one-based number.
- Print each field with an explicit label.
- Handle an empty collection.

Concept Explanation

Displaying records is a read operation. A one-based display number is a temporary selection aid; the stored identifier remains the stable record identity.

Worked Example

```
function displayItems(items) {
  if (items.length === 0) {
    console.log("No items have been saved.");
    return;
  }
  items.forEach((item, index) => {
    console.log(`Item ${index + 1}:`);
    console.log(`  Title: ${item.title}`);
    console.log(`  Category: ${item.category}`);
    console.log(`  Description: ${item.description}`);
  });
}
```

Expected Behavior or Output

Each record is announced with its number and labeled fields. An empty collection prints the no-items message.

Common Mistakes

- Omitting field labels.
- Using zero-based indexes as the displayed number.

- Failing to handle an empty collection.

Activities

1. Display three sample records in order.
2. Display an empty list and confirm the message is clear.
3. Include the record identifier and timestamps in the output.
4. Review output using sequential screen-reader navigation.

Accessibility Practice

Use labels, stable ordering, and one record at a time. Do not rely on columns, color, or icons to convey record fields.

Skills Checklist

☐ I can display all records.
☐ I can number records for selection.
☐ I can handle an empty collection.
☐ I can label every displayed field.

Lesson 11.4: Searching, Adding, and Editing Items

Reading References

- Python source counterpart: Unit 11, Lessons 11.3–11.5.
- Node.js documentation: Array methods and Date.

Learning Objectives

- Add validated records with unique identifiers.
- Search records by keyword.
- Edit selected fields and update timestamps.

Concept Explanation

Searching filters records without changing the stored collection. Editing changes only permitted fields and updates the updated_at timestamp.

Worked Example

```
function searchItems(items, keyword) {
  const query = keyword.trim().toLowerCase();
  return items.filter(item =>
    item.title.toLowerCase().includes(query) ||
    item.description.toLowerCase().includes(query)
  );
}
```

```
function updateItem(item, changes) {
  const allowed = ["title", "description", "category"];
  for (const field of allowed) {
    if (Object.hasOwn(changes, field)) item[field] = changes[field];
  }
  item.updated_at = new Date().toISOString();
}
```

Expected Behavior or Output

Search returns matching titles or descriptions. A successful edit changes the requested field and refreshes updated_at.

Common Mistakes

- Searching case-sensitively without documenting that behavior.
- Allowing edits to immutable identifiers.
- Failing to update the modification timestamp.

Activities

1. Search records by a word in the title.
2. Search by a word in the description.
3. Edit a title and verify that updated_at changes.
4. Reject an edit that contains only blank required fields.

Accessibility Practice

Report the number of matches and identify each result by its title and record number. Confirm every successful edit.

Skills Checklist

☐ I can search records by keyword.
☐ I can add a validated record.
☐ I can update selected fields.
☐ I can maintain the updated_at timestamp.

Lesson 11.5: Deleting Items, Persistence, and Final Verification

Reading References

- Python source counterpart: Unit 11, Lessons 11.3–11.5.
- Node.js documentation: fs/promises and JSON.

Learning Objectives

- Delete a record by its identifier or displayed number.
- Save changes and reload them between program runs.
- Verify the full CRUD workflow and error handling.

Concept Explanation

Deleting removes a selected record from the collection. Persistence must happen after the mutation so the deletion remains effective after the next program launch.

Worked Example

```
function deleteItem(items, id) {
  const index = items.findIndex(item => item.id === id);
  if (index < 0) return false;
  items.splice(index, 1);
  return true;
}
```

Expected Behavior or Output

After deleting and saving a record, reloading the JSON file confirms that the record is absent.

Common Mistakes

- Deleting by a stale array index.
- Failing to save after a deletion.
- Omitting confirmation for an irreversible action.

Activities

1. Delete one record and confirm the title of the deleted item.
2. Save the remaining records to JSON.
3. Restart the application and verify the deleted record is still absent.
4. Run the full acceptance checklist: add, view, search, edit, delete, invalid input, missing file, malformed JSON, and help.

Accessibility Practice

Ask for confirmation before irreversible deletion. Announce success or failure in text and preserve predictable menu navigation.

Runnable Reference Implementation

The repository includes a tested JavaScript domain and JSON-storage reference core in capstone/README.md. Run `node --test JAVASCRIPT/capstone/records.test.js`

from the repository root. The reference core does not replace the requirement to build and evaluate an accessible user interface.

Skills Checklist

- ☐ I can delete a record safely.
- ☐ I can persist and reload records.
- ☐ I can verify the complete CRUD workflow.
- ☐ I can test invalid input and storage failures.

Assessment Criteria

Functionality (60%)

- ☐ All create, read, update, and delete operations work.
- ☐ Data persists to JSON between program runs.
- ☐ Search works by at least one field.
- ☐ Invalid input is handled without crashing.

Code Quality (20%)

- ☐ Functions have focused responsibilities.
- ☐ Names are descriptive and consistent.
- ☐ Code is organized into modules.
- ☐ Key functions include JSDoc comments.

Accessibility and User Experience (15%)

- ☐ Prompts are descriptive and clear.
- ☐ Every state-changing operation has a confirmation.
- ☐ Output is sequential and predictable.
- ☐ A help system is available.
- ☐ Keyboard-only operation is supported.

Documentation and Tests (5%)

- ☐ README includes setup and usage instructions.
- ☐ At least five unit tests cover key functions.
- ☐ README includes screen-reader usage tips and a reflection.

Deliverables

- Source files organized into modules.
- A data folder with example JSON records.
- A README with installation, run instructions, commands, screen-reader tips, and a reflection.
- A test suite with at least five meaningful tests.

JavaScript Module Note

CommonJS and ECMAScript modules are both valid JavaScript module systems. For this curriculum, choose one mode explicitly in package.json and use it consistently. If using CommonJS, use `require()` and `module.exports`; if using ES modules, use `import` and `export`. Do not mix both systems in a single file.

AI Use Policy

AI may be used for personal conceptual explanations and reinforcement only. It may not write, autocomplete, repair, complete, or generate course code or assessment solutions. Students must be able to explain and independently produce their submitted work.

Part X

JULIA

Chapter 10

Julia Curriculum

Before you start: complete Unit P: Computing Foundations Primer. It explains the language-neutral vocabulary and concepts used in every language track, using non-visual examples.Julia is a high-level, dynamically typed, JIT-compiled language for numerical and scientific computing, simulation, optimization, statistics, and data-intensive research. Multiple dispatch, expressive types, array operations, and performance-oriented compilation support both interactive analysis and high-performance numerical kernels.

Course structure

A complete introductory sequence aligned to Python Units 0–11. The course uses the same student profile, calculator, grade thresholds, scores, inventory, coordinates, text sample, CSV columns, task fields, text-adventure requirements, and personal-data-manager record model as the other general-purpose tracks. Julia-specific semantics are taught directly.

Environment

- Baseline: Julia 1.12.
- Verify with `julia --version`.
- Run project scripts with `julia --project=. main.jl`.
- Activate the project and install dependencies with `using Pkg; Pkg.activate("."); Pkg.add("PackageName")`.
- Use JSON3.jl for JSON and CSV.jl for standards-compliant CSV parsing; these are project dependencies, not Julia standard libraries.

Complete the shared IDE setup lesson and terminal prerequisite. The course supports keyboard-first work with screen readers, braille displays, magnification, and other access technology.

- Complete lessons
- Topic index

GUI, visualization, and workbench accessibility

See the shared GUI, visualization, and workbench accessibility reference for screen-reader support by toolkit, platform-specific activation settings, recommended alternatives, and interfaces that must not be required without an equivalent accessible route.

AI Policy

AI may be used only for personal conceptual understanding and reinforcement. It must not write, complete, autocomplete, repair, or generate course code or assessment solutions.

Recommended introductory books

- Lauwens, Ben, and Allen B. Downey. *Think Julia: How to Think Like a Computer Scientist*. O'Reilly Media, 2019. A no-prerequisite introduction to programming with Julia, covering variables, expressions, functions, conditionals, iteration, and data structures. The complete text is freely available online. Online book and publisher page.

Suggested use: *Think Julia* is a natural companion for learners new to programming. It was written against an earlier Julia release; compare package APIs and version-sensitive syntax with the course's Julia baseline and current documentation.

10.1 JULIA Lessons

The curriculum is organized as one Markdown file per unit.

- accessible environment and toolchain
- foundations of computing
- numeric computations
- control flow
- data structures
- input and iteration
- functions
- object oriented programming
- files and exceptions
- testing and data analysis
- project based learning

- capstone

10.2 Unit 0: Accessible Environment and Toolchain

Course overview

Before you start: complete Unit P: Computing Foundations Primer. It explains the language-neutral vocabulary and concepts used in every language track, using non-visual examples.

Course Overview

Julia is a high-level, dynamically typed, JIT-compiled language for numerical and scientific computing, simulation, optimization, statistics, and data-intensive research. Multiple dispatch, expressive types, array operations, and performance-oriented compilation support interactive development and high-performance numerical kernels.

This course follows the Python curriculum's unit sequence and shared problem contexts: Avery's profile, calculator values, grade thresholds, scores, inventory, coordinates, text sample, CSV columns, task fields, text-adventure requirements, and personal-data-manager record model. Julia idioms are taught directly rather than mechanically translated.

Version baseline: Julia 1.12. Run project scripts with `julia --project=. filename.jl`. Examples use the language and standard libraries unless a dependency is identified.

Unit 0: Accessible Environment and Toolchain

Overview

Install Julia, configure an accessible editor and terminal workflow, create a reproducible project environment, and verify source files from the project root. Julia is JIT-compiled; learners run source files with the Julia executable or experiment in the REPL.

Lesson 0.1: Install and Verify Julia

Reading References

- Official Julia downloads
- Julia installation and getting started

Learning Objectives

- Install Julia 1.12 and verify the executable.

- Distinguish the Julia REPL from the operating-system shell.
- Run a saved source file and interpret basic runtime output.

Concept Explanation Install Julia from the official distribution or juliaup. Open a new terminal and run `julia --version`. Start the REPL with `julia`, enter an expression, and exit with `exit()` or Ctrl+D. Run a saved file with `julia main.jl`. Julia compiles functions just in time while executing source code; beginners do not create a separate executable for ordinary scripts.

Worked Example

```
println("Julia version: ", VERSION)
println("Hello, Julia!")
```

Expected Behavior or Output The first line prints the installed Julia version; the second prints `Hello, Julia!`.

Common Mistakes

- Using an existing terminal session whose PATH has not refreshed after installation.
- Entering Julia REPL expressions at the operating-system shell prompt, or shell commands at the REPL prompt.
- Running a filename that is not in the current working directory.

Activities

1. Record the output of `julia --version` and the `VERSION` constant.
2. Run Hello World once in the REPL and once from a saved `.jl` file.
3. Deliberately use an incorrect filename and read the resulting diagnostic.
4. Restart the terminal and verify that Julia remains available.

Accessibility Practice Navigate installer controls and terminal output by keyboard. Confirm that the REPL prompt, punctuation, and error messages are announced by the learner's screen reader or braille display.

Skills Checklist

☐ I can install Julia and identify its version.
☐ I can distinguish the REPL from the system shell.
☐ I can run a saved `.jl` file from a terminal.
☐ I can identify a missing-file or command-path error.
☐ I can read prompts and diagnostics using my access technology.

Lesson 0.2: Workspace and First Run

Reading References

- Julia command-line options
- Julia file and I/O documentation

Learning Objectives

- Create and identify a project root and source file.
- Distinguish source files, generated files, and the process working directory.
- Explain how Julia resolves relative paths.

Concept Explanation Create a course directory and save `main.jl` in its root. Use `pwd()` in Julia (or `pwd` / `Get-Location` in the shell) to confirm the current directory. Julia resolves relative file paths from the process working directory, not automatically from the directory containing the source file. Keep source, test, data, and generated files in clearly named locations.

Worked Example

```
println("Project: Julia course workspace")
println("Working directory: ", pwd())
```

Expected Behavior or Output The example prints a project label and the absolute path of the directory from which Julia is running.

Common Mistakes

- Opening a parent directory instead of the actual project root.
- Assuming relative paths are based on the source file's location.
- Saving the source file with an unintended extension such as `.jl.txt`.

Activities

1. Create the project directory and verify its path with the terminal.
2. Save and run `main.jl` from the project root.
3. Change to a different working directory and observe how a relative path changes.
4. Return to the project root and rerun the program successfully.

Accessibility Practice Use terminal history and directory commands without drag-and-drop. Confirm that the editor exposes file names, indentation, and diagnostics in a linear reading order.

Skills Checklist

- ☐ I can identify the project root and current working directory.
- ☐ I can create and run `main.jl`.
- ☐ I can explain how Julia resolves relative paths.
- ☐ I can organize source and data files predictably.
- ☐ I can navigate the workspace without mouse-only actions.

Lesson 0.3: Editor and Accessibility Workflow

Reading References

- Julia extension for Visual Studio Code
- Shared IDE setup lesson

Learning Objectives

- Configure VS Code, VSCodium, or Theia for Julia development.
- Navigate source, diagnostics, terminal, and project files by keyboard.
- Verify that the editor and terminal use the same Julia installation.

Concept Explanation Install the Julia extension available in the chosen editor, select the same Julia executable used in the terminal, and open the project root. Verify syntax diagnostics, symbol navigation, terminal focus, and file navigation. Marketplace extensions are not automatically available in VSCodium or Theia; check the extension registry for the selected editor. An editor's syntax coloring alone does not establish that the language server is working.

Worked Example

```
function greet(name::AbstractString)
    return "Hello, $(name)!"
end
println(greet("Avery"))
```

Expected Behavior or Output The program prints `Hello, Avery!`. The function accepts an abstract string argument and returns a string.

Common Mistakes

- Selecting a different Julia installation in the editor than the one used by the terminal.
- Treating syntax coloring as proof that language-server diagnostics and navigation work.
- Depending on mouse-only controls or unannounced inline diagnostics.

Activities

1. Open the course project root in the selected editor.
2. Navigate the worked example by line, word, and character.
3. Introduce a syntax error, locate the diagnostic, and correct it.
4. Run the file from both the integrated terminal and the system terminal.

Accessibility Practice Test punctuation verbosity, indentation announcements, completion popups, diagnostics, terminal focus, and editor panels with the learner's access technology. Keep a terminal-first workflow available if an extension is inaccessible.

Skills Checklist

- ☐ I can open the project root and identify the Julia executable.
- ☐ I can navigate and edit Julia source by keyboard.
- ☐ I can locate and correct a language-server diagnostic.
- ☐ I can run a source file without relying on a mouse.
- ☐ I can describe any editor accessibility limitation and use a workable alternative.

Lesson 0.4: Project Environments, Dependencies, and Terminal Usage

Reading References

- Julia package manager
- Project environments

Learning Objectives

- Activate a project-local Julia environment.
- Explain the roles of `Project.toml` and `Manifest.toml`.
- Add and instantiate dependencies in the intended project.
- Run scripts and tests with the project environment active.

Concept Explanation Run `julia --project=.` from the project root. At the REPL use `using Pkg; Pkg.activate(".")`. Add a dependency with `Pkg.add("JSON3")`; direct dependencies are recorded in `Project.toml` and the resolved dependency graph in `Manifest.toml`. Reproduce an environment with `Pkg.instantiate()`. Run scripts with `julia --project=. main.jl` and tests with `julia --project=. test/runtests.jl`. Use a project environment rather than adding course dependencies to the global environment.

Worked Example

```julia
using Pkg
Pkg.activate(".")
println("Active project: ", Base.active_project())
```

Expected Behavior or Output The package manager activates the current directory as the project and prints the active project file path. When no project exists yet, Julia may create the project metadata as part of activation or package operations.

Common Mistakes

- Adding a package before activating the intended project.
- Omitting `--project=.` when running a script that imports project dependencies.
- Deleting or ignoring the manifest when exact dependency reproducibility is required.

Activities

1. Activate the course directory and confirm the active project path.
2. Add JSON3 and inspect the resulting TOML project and manifest files.
3. Start a fresh Julia process and run `Pkg.instantiate()`.
4. Run a dependency-using script with and without the project flag and explain the difference.

Accessibility Practice Inspect TOML files as text and read package-manager output sequentially. Keep installation and test commands in documented terminal steps rather than relying on transient editor notifications.

Skills Checklist

☐ I can activate a project-local environment.
☐ I can explain the purpose of `Project.toml` and `Manifest.toml`.
☐ I can add and instantiate a dependency in the correct project.
☐ I can run scripts and tests from the terminal with the project active.
☐ I can inspect dependency metadata using keyboard-accessible tools.

10.3 Unit 1: Foundations of Computing

Overview

This unit covers computer science, hardware/software, algorithms, first program, variables, primitive types. Follow the same problem progression as Python so that the main comparison is language syntax and semantics. Use the worked examples as starting points, then complete the activities independently.

Lesson 1.1: Computer science and problem solving

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain computer science and problem solving using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

Julia is a high-level, dynamically typed, JIT-compiled language. Values have concrete types and variables are bindings. Define inputs and outputs, choose a finite algorithm, and test ordinary and boundary cases.

Lesson focus: Computer science and problem solving. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
println("Computer science turns a problem into a finite sequence of precise steps.")
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Inspect values with `typeof` rather than assuming a static declaration.
- Do not confuse a variable binding with a `const` binding.
- Read `UndefVarError` and `MethodError` diagnostics before changing code.

Activities

1. Run the example and explain each expression in execution order.
2. Change a canonical profile value and predict the output.
3. Inspect each value with `typeof` and explain the result.
4. Introduce and correct an undefined-name error.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

- ☐ I can explain computer science and problem solving in Julia terminology.
- ☐ I can trace the example and predict its output.
- ☐ I can adapt the example to canonical shared data.
- ☐ I can identify and correct a Julia-specific error or edge case.
- ☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 1.2: Your first program

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain your first program using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

Julia is a high-level, dynamically typed, JIT-compiled language. Values have concrete types and variables are bindings. Define inputs and outputs, choose a finite algorithm, and test ordinary and boundary cases.

Lesson focus: Your first program. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
println("Hello, World!")
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Inspect values with `typeof` rather than assuming a static declaration.
- Do not confuse a variable binding with a `const` binding.
- Read `UndefVarError` and `MethodError` diagnostics before changing code.

Activities

1. Run the example and explain each expression in execution order.
2. Change a canonical profile value and predict the output.
3. Inspect each value with `typeof` and explain the result.
4. Introduce and correct an undefined-name error.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

☐ I can explain your first program in Julia terminology.
☐ I can trace the example and predict its output.
☐ I can adapt the example to canonical shared data.
☐ I can identify and correct a Julia-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 1.3: Variables and data types

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain variables and data types using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

Julia is a high-level, dynamically typed, JIT-compiled language. Values have concrete types and variables are bindings. Define inputs and outputs, choose a finite algorithm, and test ordinary and boundary cases.

Lesson focus: Variables and data types. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
profile = (name="Avery", age=16, height_m=1.72, favorite_subject="Computer Science")
println(profile)
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Inspect values with `typeof` rather than assuming a static declaration.
- Do not confuse a variable binding with a `const` binding.
- Read `UndefVarError` and `MethodError` diagnostics before changing code.

Activities

1. Run the example and explain each expression in execution order.
2. Change a canonical profile value and predict the output.
3. Inspect each value with `typeof` and explain the result.
4. Introduce and correct an undefined-name error.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

☐ I can explain variables and data types in Julia terminology.
☐ I can trace the example and predict its output.
☐ I can adapt the example to canonical shared data.
☐ I can identify and correct a Julia-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

10.4 Unit 2: Numeric Computations

Overview

This unit covers development process, expressions, assignment, input/output, conversions, arithmetic, math library. Follow the same problem progression as Python so that the main comparison is language syntax and semantics. Use the worked examples as starting points, then complete the activities independently.

Lesson 2.1: Design and tracing

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain design and tracing using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

Julia evaluates expressions and binds names with `=`. Numeric text must be converted before arithmetic; use `parse` for trusted input and `tryparse` when conversion may fail. , `hypot`, exponentiation with `^`, and `round(...; digits=...)` support common calculations.

Lesson focus: Design and tracing. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
bill = 24.50; rate = 0.15
println("Tip: ", bill * rate, "; total: ", bill * (1 + rate))
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Use `==` for equality; `=` binds a value.
- Validate uncertain text with `tryparse` before arithmetic.
- Avoid rounding intermediate values before the final result.

Activities

1. Trace each expression and record intermediate values.
2. Calculate the canonical $24.50 bill with a 15% tip.
3. Test a boundary and invalid input; state the intended behavior.
4. Refactor the calculation into a function without changing its result.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

☐ I can explain design and tracing in Julia terminology.
☐ I can trace the example and predict its output.
☐ I can adapt the example to canonical shared data.
☐ I can identify and correct a Julia-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 2.2: Expressions and assignment

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain expressions and assignment using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

Julia evaluates expressions and binds names with `=`. Numeric text must be converted before arithmetic; use `parse` for trusted input and `tryparse` when conversion may fail. , `hypot`, exponentiation with `^`, and `round(...; digits=...)` support common calculations.

Lesson focus: Expressions and assignment. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
subtotal = 18.00 + 6.50
println("Subtotal: ", subtotal, "; tax: ", subtotal * 0.08)
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Use `==` for equality; `=` binds a value.
- Validate uncertain text with `tryparse` before arithmetic.
- Avoid rounding intermediate values before the final result.

Activities

1. Trace each expression and record intermediate values.
2. Calculate the canonical $24.50 bill with a 15% tip.
3. Test a boundary and invalid input; state the intended behavior.
4. Refactor the calculation into a function without changing its result.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

☐ I can explain expressions and assignment in Julia terminology.
☐ I can trace the example and predict its output.
☐ I can adapt the example to canonical shared data.
☐ I can identify and correct a Julia-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 2.3: Input and output

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain input and output using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

Julia evaluates expressions and binds names with `=`. Numeric text must be converted before arithmetic; use `parse` for trusted input and `tryparse` when conversion may fail. , `hypot`, exponentiation with `^`, and `round(...; digits=...)` support common calculations.

Lesson focus: Input and output. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
print("Name: "); name = readline()
println("Hello, ", name)
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Use `==` for equality; `=` binds a value.
- Validate uncertain text with `tryparse` before arithmetic.
- Avoid rounding intermediate values before the final result.

Activities

1. Trace each expression and record intermediate values.
2. Calculate the canonical $24.50 bill with a 15% tip.
3. Test a boundary and invalid input; state the intended behavior.
4. Refactor the calculation into a function without changing its result.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

- ☐ I can explain input and output in Julia terminology.
- ☐ I can trace the example and predict its output.
- ☐ I can adapt the example to canonical shared data.
- ☐ I can identify and correct a Julia-specific error or edge case.
- ☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 2.4: Numeric conversions and mathematics

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain numeric conversions and mathematics using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

Julia evaluates expressions and binds names with `=`. Numeric text must be converted before arithmetic; use `parse` for trusted input and `tryparse` when conversion may fail. , `hypot`, exponentiation with `^`, and `round(...; digits=...)` support common calculations.

Lesson focus: Numeric conversions and mathematics. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
weight_kg = parse(Float64, "68.5")
println("Weight in grams: ", weight_kg * 1000)
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Use `==` for equality; `=` binds a value.
- Validate uncertain text with `tryparse` before arithmetic.
- Avoid rounding intermediate values before the final result.

Activities

1. Trace each expression and record intermediate values.
2. Calculate the canonical $24.50 bill with a 15% tip.
3. Test a boundary and invalid input; state the intended behavior.
4. Refactor the calculation into a function without changing its result.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

☐ I can explain numeric conversions and mathematics in Julia terminology.
☐ I can trace the example and predict its output.
☐ I can adapt the example to canonical shared data.
☐ I can identify and correct a Julia-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 2.5: Using the Math Library

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain using the math library using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

Julia evaluates expressions and binds names with `=`. Numeric text must be converted before arithmetic; use `parse` for trusted input and `tryparse` when conversion may fail. , `hypot`, exponentiation with `^`, and `round(...; digits=...)` support common calculations.

Lesson focus: Using the Math Library. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
radius = 3.0
println("Area: ",  * radius^2, "; circumference: ", 2 * radius, "; distance: ", hypot(3,4))
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Use `==` for equality; `=` binds a value.
- Validate uncertain text with `tryparse` before arithmetic.
- Avoid rounding intermediate values before the final result.

Activities

1. Trace each expression and record intermediate values.
2. Calculate the canonical $24.50 bill with a 15% tip.
3. Test a boundary and invalid input; state the intended behavior.
4. Refactor the calculation into a function without changing its result.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

☐ I can explain using the math library in Julia terminology.
☐ I can trace the example and predict its output.
☐ I can adapt the example to canonical shared data.
☐ I can identify and correct a Julia-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

10.5 Unit 3: Control Flow

Overview

This unit covers Boolean expressions, conditionals, lists or arrays, iteration, string operations. Follow the same problem progression as Python so that the main comparison is language syntax and semantics. Use the worked examples as starting points, then complete the activities independently.

Lesson 3.1: Boolean expressions

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain boolean expressions using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

Julia uses `if`/`elseif`/`else`/`end`, short-circuit Boolean operators, and one-based indexing. `for` iterates over values, `enumerate` supplies indices, and dotted operators broadcast. Strings are UTF-8, so character count and byte indexing differ.

Lesson focus: Boolean expressions. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
score = 82
println(score >= 80 && score <= 100)
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Standard Julia arrays are one-based, not zero-based.
- Use `&&` and `||` for scalar conditions; dotted operators broadcast.
- Do not structurally mutate a collection while iterating over it.

Activities

1. Test scores 82, 91, 76, and 88, including grade boundaries.
2. Test empty and single-element arrays.
3. Trace each loop iteration and collection state.
4. Modify a string and test indexing, normalization, and search.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

- ☐ I can explain boolean expressions in Julia terminology.
- ☐ I can trace the example and predict its output.
- ☐ I can adapt the example to canonical shared data.
- ☐ I can identify and correct a Julia-specific error or edge case.
- ☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 3.2: Chained conditionals

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain chained conditionals using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

Julia uses `if`/`elseif`/`else`/`end`, short-circuit Boolean operators, and one-based indexing. `for` iterates over values, `enumerate` supplies indices, and dotted operators broadcast. Strings are UTF-8, so character count and byte indexing differ.

Lesson focus: Chained conditionals. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
score = 82
grade = score >= 90 ? "A" : score >= 80 ? "B" : score >= 70 ? "C" : score >= 60 ? "D" : "F"
println(grade)
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary

paths and close resources within the callback.

Common Mistakes

- Standard Julia arrays are one-based, not zero-based.
- Use `&&` and `||` for scalar conditions; dotted operators broadcast.
- Do not structurally mutate a collection while iterating over it.

Activities

1. Test scores 82, 91, 76, and 88, including grade boundaries.
2. Test empty and single-element arrays.
3. Trace each loop iteration and collection state.
4. Modify a string and test indexing, normalization, and search.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

☐ I can explain chained conditionals in Julia terminology.
☐ I can trace the example and predict its output.
☐ I can adapt the example to canonical shared data.
☐ I can identify and correct a Julia-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 3.3: Sequences and indexing

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain sequences and indexing using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

Julia uses `if`/`elseif`/`else`/`end`, short-circuit Boolean operators, and one-based indexing. `for` iterates over values, `enumerate` supplies indices, and dotted op-

erators broadcast. Strings are UTF-8, so character count and byte indexing differ.

Lesson focus: Sequences and indexing. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
scores = [82, 91, 76, 88]
println(scores[1], "; ", scores[4])
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Standard Julia arrays are one-based, not zero-based.
- Use `&&` and `||` for scalar conditions; dotted operators broadcast.
- Do not structurally mutate a collection while iterating over it.

Activities

1. Test scores 82, 91, 76, and 88, including grade boundaries.
2. Test empty and single-element arrays.
3. Trace each loop iteration and collection state.
4. Modify a string and test indexing, normalization, and search.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

☐ I can explain sequences and indexing in Julia terminology.
☐ I can trace the example and predict its output.
☐ I can adapt the example to canonical shared data.
☐ I can identify and correct a Julia-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 3.4: For loops and iteration

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain for loops and iteration using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

Julia uses `if`/`elseif`/`else`/`end`, short-circuit Boolean operators, and one-based indexing. `for` iterates over values, `enumerate` supplies indices, and dotted operators broadcast. Strings are UTF-8, so character count and byte indexing differ.

Lesson focus: For loops and iteration. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
scores = [82, 91, 76, 88]
for (i, score) in enumerate(scores)
    println("Score $i: $score")
end
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Standard Julia arrays are one-based, not zero-based.
- Use `&&` and `||` for scalar conditions; dotted operators broadcast.
- Do not structurally mutate a collection while iterating over it.

Activities

1. Test scores 82, 91, 76, and 88, including grade boundaries.
2. Test empty and single-element arrays.
3. Trace each loop iteration and collection state.
4. Modify a string and test indexing, normalization, and search.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

- ☐ I can explain for loops and iteration in Julia terminology.
- ☐ I can trace the example and predict its output.
- ☐ I can adapt the example to canonical shared data.
- ☐ I can identify and correct a Julia-specific error or edge case.
- ☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 3.5: String operations

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain string operations using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

Julia uses `if`/`elseif`/`else`/`end`, short-circuit Boolean operators, and one-based indexing. `for` iterates over values, `enumerate` supplies indices, and dotted operators broadcast. Strings are UTF-8, so character count and byte indexing differ.

Lesson focus: String operations. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
text = "Avery studies Julia programming."
println(lowercase(strip(text)))
println(occursin("Julia", text))
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary

paths and close resources within the callback.

Common Mistakes

- Standard Julia arrays are one-based, not zero-based.
- Use `&&` and `||` for scalar conditions; dotted operators broadcast.
- Do not structurally mutate a collection while iterating over it.

Activities

1. Test scores 82, 91, 76, and 88, including grade boundaries.
2. Test empty and single-element arrays.
3. Trace each loop iteration and collection state.
4. Modify a string and test indexing, normalization, and search.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

☐ I can explain string operations in Julia terminology.
☐ I can trace the example and predict its output.
☐ I can adapt the example to canonical shared data.
☐ I can identify and correct a Julia-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

10.6 Unit 4: Data Structures

Overview

This unit covers sequence mutation, sorting, immutable records, maps, dictionaries, traversal. Follow the same problem progression as Python so that the main comparison is language syntax and semantics. Use the worked examples as starting points, then complete the activities independently.

Lesson 4.1: Mutable sequences

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain mutable sequences using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

Julia provides mutable arrays, immutable tuples and named tuples, and dictionaries for key-value data. Mutating functions conventionally end in `!`. Sort keys or entries when deterministic report order matters.

Lesson focus: Mutable sequences. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
scores = [82, 91, 76, 88]
push!(scores, 95); insert!(scores, 2, 85)
println(scores)
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Tuples and named tuples are immutable; arrays and dictionaries are mutable.
- Use `get` or `haskey` for possibly absent dictionary keys.
- A shallow copy does not duplicate nested mutable values.

Activities

1. Represent the canonical inventory using a dictionary.
2. Add, update, and remove an entry, checking each state.
3. Compare tuple, named tuple, vector, and dictionary semantics.
4. Test a missing key and implement an explicit fallback.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

- ☐ I can explain mutable sequences in Julia terminology.
- ☐ I can trace the example and predict its output.
- ☐ I can adapt the example to canonical shared data.
- ☐ I can identify and correct a Julia-specific error or edge case.
- ☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 4.2: Sorting and organizing data

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain sorting and organizing data using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

Julia provides mutable arrays, immutable tuples and named tuples, and dictionaries for key-value data. Mutating functions conventionally end in `!`. Sort keys or entries when deterministic report order matters.

Lesson focus: Sorting and organizing data. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
scores = [82, 91, 76, 88]
println(sort(scores)); println(sort(scores, rev=true))
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Tuples and named tuples are immutable; arrays and dictionaries are mutable.
- Use `get` or `haskey` for possibly absent dictionary keys.
- A shallow copy does not duplicate nested mutable values.

Activities

1. Represent the canonical inventory using a dictionary.
2. Add, update, and remove an entry, checking each state.
3. Compare tuple, named tuple, vector, and dictionary semantics.
4. Test a missing key and implement an explicit fallback.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

☐ I can explain sorting and organizing data in Julia terminology.
☐ I can trace the example and predict its output.
☐ I can adapt the example to canonical shared data.
☐ I can identify and correct a Julia-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 4.3: Immutable records

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain immutable records using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

Julia provides mutable arrays, immutable tuples and named tuples, and dictionaries for key-value data. Mutating functions conventionally end in `!`. Sort keys or entries when deterministic report order matters.

Lesson focus: Immutable records. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
point = (x=3, y=4)
println(hypot(point.x, point.y))
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Tuples and named tuples are immutable; arrays and dictionaries are mutable.
- Use `get` or `haskey` for possibly absent dictionary keys.
- A shallow copy does not duplicate nested mutable values.

Activities

1. Represent the canonical inventory using a dictionary.
2. Add, update, and remove an entry, checking each state.
3. Compare tuple, named tuple, vector, and dictionary semantics.
4. Test a missing key and implement an explicit fallback.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

☐ I can explain immutable records in Julia terminology.
☐ I can trace the example and predict its output.
☐ I can adapt the example to canonical shared data.
☐ I can identify and correct a Julia-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 4.4: Maps and key-value data

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain maps and key-value data using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

Julia provides mutable arrays, immutable tuples and named tuples, and dictionaries for key-value data. Mutating functions conventionally end in `!`. Sort keys or entries when deterministic report order matters.

Lesson focus: Maps and key-value data. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
item = Dict("name" => "Notebook", "quantity" => 3, "price" => 2.50)
println(item["name"], ": ", item["quantity"])
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Tuples and named tuples are immutable; arrays and dictionaries are mutable.
- Use `get` or `haskey` for possibly absent dictionary keys.
- A shallow copy does not duplicate nested mutable values.

Activities

1. Represent the canonical inventory using a dictionary.
2. Add, update, and remove an entry, checking each state.
3. Compare tuple, named tuple, vector, and dictionary semantics.
4. Test a missing key and implement an explicit fallback.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

- ☐ I can explain maps and key-value data in Julia terminology.
- ☐ I can trace the example and predict its output.
- ☐ I can adapt the example to canonical shared data.
- ☐ I can identify and correct a Julia-specific error or edge case.
- ☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 4.5: Collection traversal

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain collection traversal using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

Julia provides mutable arrays, immutable tuples and named tuples, and dictionaries for key-value data. Mutating functions conventionally end in `!`. Sort keys or entries when deterministic report order matters.

Lesson focus: Collection traversal. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```julia
stock = Dict("Notebook"=>3, "Pencil"=>10, "Folder"=>4)
for key in sort(collect(keys(stock)))
    println(key, ": ", stock[key])
end
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Tuples and named tuples are immutable; arrays and dictionaries are mutable.
- Use `get` or `haskey` for possibly absent dictionary keys.
- A shallow copy does not duplicate nested mutable values.

Activities

1. Represent the canonical inventory using a dictionary.
2. Add, update, and remove an entry, checking each state.
3. Compare tuple, named tuple, vector, and dictionary semantics.
4. Test a missing key and implement an explicit fallback.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

- ☐ I can explain collection traversal in Julia terminology.
- ☐ I can trace the example and predict its output.
- ☐ I can adapt the example to canonical shared data.
- ☐ I can identify and correct a Julia-specific error or edge case.
- ☐ I can run and inspect the program using keyboard-accessible tools.

10.7 Unit 5: Input and Iteration

Overview

This unit covers input validation, sentinel loops, while loops, flags, break/continue, infinite-loop prevention. Follow the same problem progression as Python so that the main comparison is language syntax and semantics. Use the worked examples as starting points, then complete the activities independently.

Lesson 5.1: Input and validation

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain input and validation using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

Interactive input arrives as text through `readline`; validate and convert it before use. A `while` loop repeats while its condition is true, `break` exits the nearest loop, and `continue` skips to the next iteration.

Lesson focus: Input and validation. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
raw = "82"; score = tryparse(Int, raw)
println(score === nothing ? "Invalid score" : "Accepted: $score")
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Convert and validate `readline()` input before numeric operations.
- Every `while` loop needs progress or an explicit exit.
- Place `break` and `continue` at the intended loop nesting level.

Activities

1. Test valid, invalid, and sentinel command input.
2. Trace loop state and condition on each iteration.
3. Stop a search at the first matching score.
4. Prove termination for empty and no-match inputs.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

☐ I can explain input and validation in Julia terminology.
☐ I can trace the example and predict its output.
☐ I can adapt the example to canonical shared data.
☐ I can identify and correct a Julia-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 5.2: While loops

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain while loops using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

Interactive input arrives as text through `readline`; validate and convert it before use. A `while` loop repeats while its condition is true, `break` exits the nearest loop, and `continue` skips to the next iteration.

Lesson focus: While loops. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
function run_commands()
    commands = ["list", "help", "quit"]
    index = 1
    while index <= length(commands)
        println(commands[index])
        commands[index] == "quit" && break
        index += 1
    end
end
run_commands()
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Convert and validate `readline()` input before numeric operations.
- Every `while` loop needs progress or an explicit exit.
- Place `break` and `continue` at the intended loop nesting level.

Activities

1. Test valid, invalid, and sentinel command input.
2. Trace loop state and condition on each iteration.
3. Stop a search at the first matching score.
4. Prove termination for empty and no-match inputs.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

- ☐ I can explain while loops in Julia terminology.
- ☐ I can trace the example and predict its output.
- ☐ I can adapt the example to canonical shared data.
- ☐ I can identify and correct a Julia-specific error or edge case.
- ☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 5.3: Flags and break

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain flags and break using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

Interactive input arrives as text through `readline`; validate and convert it before use. A `while` loop repeats while its condition is true, `break` exits the nearest loop, and `continue` skips to the next iteration.

Lesson focus: Flags and break. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
scores = [82, 91, 76, 88]
for (i, score) in enumerate(scores)
    if score == 91
        println("Found at $i"); break
    end
end
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Convert and validate `readline()` input before numeric operations.
- Every `while` loop needs progress or an explicit exit.
- Place `break` and `continue` at the intended loop nesting level.

Activities

1. Test valid, invalid, and sentinel command input.
2. Trace loop state and condition on each iteration.
3. Stop a search at the first matching score.
4. Prove termination for empty and no-match inputs.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

☐ I can explain flags and break in Julia terminology.
☐ I can trace the example and predict its output.
☐ I can adapt the example to canonical shared data.
☐ I can identify and correct a Julia-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 5.4: Continue and filtering

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain continue and filtering using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

Interactive input arrives as text through `readline`; validate and convert it before use. A `while` loop repeats while its condition is true, `break` exits the nearest loop, and `continue` skips to the next iteration.

Lesson focus: Continue and filtering. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
values = [82, -1, 91]
for value in values
    value < 0 && continue
    println(value)
end
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Convert and validate `readline()` input before numeric operations.
- Every `while` loop needs progress or an explicit exit.
- Place `break` and `continue` at the intended loop nesting level.

Activities

1. Test valid, invalid, and sentinel command input.
2. Trace loop state and condition on each iteration.
3. Stop a search at the first matching score.
4. Prove termination for empty and no-match inputs.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

☐ I can explain continue and filtering in Julia terminology.
☐ I can trace the example and predict its output.
☐ I can adapt the example to canonical shared data.

☐ I can identify and correct a Julia-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 5.5: Infinite-loop prevention

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain infinite-loop prevention using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

Interactive input arrives as text through `readline`; validate and convert it before use. A `while` loop repeats while its condition is true, `break` exits the nearest loop, and `continue` skips to the next iteration.

Lesson focus: Infinite-loop prevention. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```julia
for attempt in 1:3
    println("Attempt $attempt")
end
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Convert and validate `readline()` input before numeric operations.
- Every `while` loop needs progress or an explicit exit.
- Place `break` and `continue` at the intended loop nesting level.

Activities

1. Test valid, invalid, and sentinel command input.
2. Trace loop state and condition on each iteration.

3. Stop a search at the first matching score.
4. Prove termination for empty and no-match inputs.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

☐ I can explain infinite-loop prevention in Julia terminology.
☐ I can trace the example and predict its output.
☐ I can adapt the example to canonical shared data.
☐ I can identify and correct a Julia-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

10.8 Unit 6: Functions

Overview

This unit covers definitions, parameters, arguments, return values, scope, collection parameters, variadic functions. Follow the same problem progression as Python so that the main comparison is language syntax and semantics. Use the worked examples as starting points, then complete the activities independently.

Lesson 6.1: Defining functions

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain defining functions using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

Functions are first-class values. Julia supports positional and keyword arguments, optional defaults, multiple dispatch, and variadic positional arguments with `...`. The final expression is returned implicitly.

Lesson focus: Defining functions. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
tip(bill, rate=0.15) = bill * rate
println(tip(24.50))
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Printing a value is not the same as returning it.
- Distinguish positional arguments from keyword arguments after the semicolon.
- Use `...` deliberately when collecting variadic arguments.

Activities

1. Call a function and inspect its return value independently of printing.
2. Add boundary and invalid-argument tests.
3. Use keyword or variadic arguments where appropriate.
4. Compose two functions without duplicating calculations.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

☐ I can explain defining functions in Julia terminology.
☐ I can trace the example and predict its output.
☐ I can adapt the example to canonical shared data.
☐ I can identify and correct a Julia-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 6.2: Parameters and arguments

Reading References

- Julia Documentation

- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain parameters and arguments using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

Functions are first-class values. Julia supports positional and keyword arguments, optional defaults, multiple dispatch, and variadic positional arguments with `...`. The final expression is returned implicitly.

Lesson focus: Parameters and arguments. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
function invoice_total(bill; rate=0.15, tax=0.0)
    bill + bill * rate + tax
end
println(invoice_total(24.50; tax=1.20))
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Printing a value is not the same as returning it.
- Distinguish positional arguments from keyword arguments after the semicolon.
- Use `...` deliberately when collecting variadic arguments.

Activities

1. Call a function and inspect its return value independently of printing.
2. Add boundary and invalid-argument tests.
3. Use keyword or variadic arguments where appropriate.
4. Compose two functions without duplicating calculations.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

- ☐ I can explain parameters and arguments in Julia terminology.
- ☐ I can trace the example and predict its output.
- ☐ I can adapt the example to canonical shared data.
- ☐ I can identify and correct a Julia-specific error or edge case.
- ☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 6.3: Return values

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain return values using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

Functions are first-class values. Julia supports positional and keyword arguments, optional defaults, multiple dispatch, and variadic positional arguments with `...`. The final expression is returned implicitly.

Lesson focus: Return values. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```julia
function grade(score)
    0 <= score <= 100 || throw(ArgumentError("score out of range"))
    return score >= 90 ? "A" : score >= 80 ? "B" : "C or below"
end
println(grade(82))
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Printing a value is not the same as returning it.
- Distinguish positional arguments from keyword arguments after the semicolon.
- Use `...` deliberately when collecting variadic arguments.

Activities

1. Call a function and inspect its return value independently of printing.
2. Add boundary and invalid-argument tests.
3. Use keyword or variadic arguments where appropriate.
4. Compose two functions without duplicating calculations.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

☐ I can explain return values in Julia terminology.
☐ I can trace the example and predict its output.
☐ I can adapt the example to canonical shared data.
☐ I can identify and correct a Julia-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 6.4: Functions over collections

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain functions over collections using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

Functions are first-class values. Julia supports positional and keyword arguments, optional defaults, multiple dispatch, and variadic positional arguments with `...`. The final expression is returned implicitly.

Lesson focus: Functions over collections. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
average(values::AbstractVector{<:Real}) = sum(values) / length(values)
println(average([82,91,76,88]))
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Printing a value is not the same as returning it.
- Distinguish positional arguments from keyword arguments after the semicolon.
- Use `...` deliberately when collecting variadic arguments.

Activities

1. Call a function and inspect its return value independently of printing.
2. Add boundary and invalid-argument tests.
3. Use keyword or variadic arguments where appropriate.
4. Compose two functions without duplicating calculations.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

☐ I can explain functions over collections in Julia terminology.
☐ I can trace the example and predict its output.
☐ I can adapt the example to canonical shared data.
☐ I can identify and correct a Julia-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 6.5: Scope and variadic arguments

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain scope and variadic arguments using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

Functions are first-class values. Julia supports positional and keyword arguments, optional defaults, multiple dispatch, and variadic positional arguments with `...`. The final expression is returned implicitly.

Lesson focus: Scope and variadic arguments. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```julia
function report(label, scores...)
    println(label, ": ", join(scores, ", "))
end
report("Quiz", 82,91,76,88)
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Printing a value is not the same as returning it.
- Distinguish positional arguments from keyword arguments after the semicolon.
- Use `...` deliberately when collecting variadic arguments.

Activities

1. Call a function and inspect its return value independently of printing.
2. Add boundary and invalid-argument tests.
3. Use keyword or variadic arguments where appropriate.
4. Compose two functions without duplicating calculations.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

- ☐ I can explain scope and variadic arguments in Julia terminology.
- ☐ I can trace the example and predict its output.
- ☐ I can adapt the example to canonical shared data.
- ☐ I can identify and correct a Julia-specific error or edge case.
- ☐ I can run and inspect the program using keyboard-accessible tools.

10.9 Unit 7: Object-Oriented Programming

Overview

This unit covers types/classes, instances, fields, methods, inheritance or interfaces, composition. Follow the same problem progression as Python so that the main comparison is language syntax and semantics. Use the worked examples as starting points, then complete the activities independently.

Lesson 7.1: Types and object models

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain types and object models using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

Declare composite types with `struct` or `mutable struct`. Concrete types do not inherit from other concrete types; abstract types define dispatch hierarchies, and methods specialize on argument types. Composition models related data.

Lesson focus: Types and object models. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
struct Student
    name::String
    age::Int
end
println(Student("Avery",16))
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Concrete Julia structs do not inherit from other concrete structs.
- An immutable `struct` cannot have its fields reassigned.
- Use abstract supertypes and multiple dispatch for polymorphic behavior.

Activities

1. Model Avery or an inventory item with a Julia composite type.
2. Create multiple instances and inspect their fields.
3. Add a validation or display method using multiple dispatch.
4. Explain whether composition, immutability, or mutability fits the model.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

☐ I can explain types and object models in Julia terminology.
☐ I can trace the example and predict its output.
☐ I can adapt the example to canonical shared data.
☐ I can identify and correct a Julia-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 7.2: Creating instances

Reading References

- Julia Documentation
- Julia Base and standard-library reference.

- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain creating instances using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

Declare composite types with `struct` or `mutable struct`. Concrete types do not inherit from other concrete types; abstract types define dispatch hierarchies, and methods specialize on argument types. Composition models related data.

Lesson focus: Creating instances. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
struct Learner
    name::String
    subject::String
end
learners = [Learner("Avery","Computer Science"), Learner("Jordan","Math")]
foreach(println, learners)
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Concrete Julia structs do not inherit from other concrete structs.
- An immutable `struct` cannot have its fields reassigned.
- Use abstract supertypes and multiple dispatch for polymorphic behavior.

Activities

1. Model Avery or an inventory item with a Julia composite type.
2. Create multiple instances and inspect their fields.
3. Add a validation or display method using multiple dispatch.
4. Explain whether composition, immutability, or mutability fits the model.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

- ☐ I can explain creating instances in Julia terminology.
- ☐ I can trace the example and predict its output.
- ☐ I can adapt the example to canonical shared data.
- ☐ I can identify and correct a Julia-specific error or edge case.
- ☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 7.3: Fields and methods

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain fields and methods using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

Declare composite types with `struct` or `mutable struct`. Concrete types do not inherit from other concrete types; abstract types define dispatch hierarchies, and methods specialize on argument types. Composition models related data.

Lesson focus: Fields and methods. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
mutable struct Item
    name::String
    quantity::Int
end
item=Item("Notebook",3); item.quantity += 1
println(item.quantity)
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Concrete Julia structs do not inherit from other concrete structs.
- An immutable `struct` cannot have its fields reassigned.
- Use abstract supertypes and multiple dispatch for polymorphic behavior.

Activities

1. Model Avery or an inventory item with a Julia composite type.
2. Create multiple instances and inspect their fields.
3. Add a validation or display method using multiple dispatch.
4. Explain whether composition, immutability, or mutability fits the model.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

☐ I can explain fields and methods in Julia terminology.
☐ I can trace the example and predict its output.
☐ I can adapt the example to canonical shared data.
☐ I can identify and correct a Julia-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 7.4: Inheritance and interfaces

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain inheritance and interfaces using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

Declare composite types with `struct` or `mutable struct`. Concrete types do not inherit from other concrete types; abstract types define dispatch hierarchies, and methods specialize on argument types. Composition models related data.

Lesson focus: Inheritance and interfaces. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
abstract type Assessment end
struct Quiz <: Assessment; score::Int; end
struct Project <: Assessment; score::Int; end
score(x::Assessment)=x.score
println(score(Quiz(82))+score(Project(91)))
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Concrete Julia structs do not inherit from other concrete structs.
- An immutable `struct` cannot have its fields reassigned.
- Use abstract supertypes and multiple dispatch for polymorphic behavior.

Activities

1. Model Avery or an inventory item with a Julia composite type.
2. Create multiple instances and inspect their fields.
3. Add a validation or display method using multiple dispatch.
4. Explain whether composition, immutability, or mutability fits the model.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

☐ I can explain inheritance and interfaces in Julia terminology.
☐ I can trace the example and predict its output.
☐ I can adapt the example to canonical shared data.

☐ I can identify and correct a Julia-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 7.5: Composition

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain composition using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

Declare composite types with `struct` or `mutable struct`. Concrete types do not inherit from other concrete types; abstract types define dispatch hierarchies, and methods specialize on argument types. Composition models related data.

Lesson focus: Composition. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
struct Address; city::String; end
struct Person; name::String; address::Address; end
println(Person("Avery",Address("Farmington")))
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Concrete Julia structs do not inherit from other concrete structs.
- An immutable `struct` cannot have its fields reassigned.
- Use abstract supertypes and multiple dispatch for polymorphic behavior.

Activities

1. Model Avery or an inventory item with a Julia composite type.
2. Create multiple instances and inspect their fields.

3. Add a validation or display method using multiple dispatch.
4. Explain whether composition, immutability, or mutability fits the model.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

☐ I can explain composition in Julia terminology.
☐ I can trace the example and predict its output.
☐ I can adapt the example to canonical shared data.
☐ I can identify and correct a Julia-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

10.10 Unit 8: Files and Exceptions

Overview

This unit covers read/write text, resource lifetime, error handling, JSON serialization and persistence. Follow the same problem progression as Python so that the main comparison is language syntax and semantics. Use the worked examples as starting points, then complete the activities independently.

Lesson 8.1: Reading text files

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain reading text files using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

Use `open(path) do io ... end` to close files even when operations fail. `eachline` supports line processing. Julia has TOML in its standard library but no general JSON parser; use JSON3.jl for JSON.

Lesson focus: Reading text files. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
mktemp() do path, io
    write(io,"Avery studies Julia.\n"); close(io)
    open(path,"r") do file
        foreach(println, eachline(file))
    end
end
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Relative paths are resolved from the process working directory.
- Use `open(path) do io` to guarantee resource cleanup.
- Catch expected exceptions specifically rather than hiding all failures.

Activities

1. Create a temporary file fixture and read it line by line.
2. Test empty, missing, and malformed files.
3. Verify resources close after successful and exceptional paths.
4. Document the file format and parser limitations.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

- ☐ I can explain reading text files in Julia terminology.
- ☐ I can trace the example and predict its output.
- ☐ I can adapt the example to canonical shared data.
- ☐ I can identify and correct a Julia-specific error or edge case.
- ☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 8.2: Writing and appending files

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain writing and appending files using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

Use `open(path) do io ... end` to close files even when operations fail. `eachline` supports line processing. Julia has TOML in its standard library but no general JSON parser; use JSON3.jl for JSON.

Lesson focus: Writing and appending files. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
mktemp() do path, io
    write(io,"First note\n"); close(io)
    open(path,"a") do file; write(file,"Second note\n"); end
    println(read(path,String))
end
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Relative paths are resolved from the process working directory.
- Use `open(path) do io` to guarantee resource cleanup.
- Catch expected exceptions specifically rather than hiding all failures.

Activities

1. Create a temporary file fixture and read it line by line.
2. Test empty, missing, and malformed files.
3. Verify resources close after successful and exceptional paths.
4. Document the file format and parser limitations.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

- ☐ I can explain writing and appending files in Julia terminology.
- ☐ I can trace the example and predict its output.
- ☐ I can adapt the example to canonical shared data.
- ☐ I can identify and correct a Julia-specific error or edge case.
- ☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 8.3: Errors and exceptions

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain errors and exceptions using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

Use `open(path) do io ... end` to close files even when operations fail. `eachline` supports line processing. Julia has TOML in its standard library but no general JSON parser; use JSON3.jl for JSON.

Lesson focus: Errors and exceptions. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
for raw in ["42","bad"]
    try
        println(parse(Int,raw))
    catch error
        error isa ArgumentError || rethrow()
        println("Invalid integer: $raw")
    end
end
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Relative paths are resolved from the process working directory.
- Use `open(path) do io` to guarantee resource cleanup.
- Catch expected exceptions specifically rather than hiding all failures.

Activities

1. Create a temporary file fixture and read it line by line.
2. Test empty, missing, and malformed files.
3. Verify resources close after successful and exceptional paths.
4. Document the file format and parser limitations.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

☐ I can explain errors and exceptions in Julia terminology.
☐ I can trace the example and predict its output.
☐ I can adapt the example to canonical shared data.
☐ I can identify and correct a Julia-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 8.4: Resource cleanup

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain resource cleanup using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

Use `open(path) do io ... end` to close files even when operations fail. `eachline` supports line processing. Julia has TOML in its standard library but no general JSON parser; use JSON3.jl for JSON.

Lesson focus: Resource cleanup. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
mktemp() do path, io
    write(io,"temporary"); close(io)
    open(path,"r") do file; println(read(file,String)); end
end
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Relative paths are resolved from the process working directory.
- Use `open(path) do io` to guarantee resource cleanup.
- Catch expected exceptions specifically rather than hiding all failures.

Activities

1. Create a temporary file fixture and read it line by line.
2. Test empty, missing, and malformed files.
3. Verify resources close after successful and exceptional paths.
4. Document the file format and parser limitations.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

☐ I can explain resource cleanup in Julia terminology.
☐ I can trace the example and predict its output.
☐ I can adapt the example to canonical shared data.
☐ I can identify and correct a Julia-specific error or edge case.

☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 8.5: JSON persistence

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain json persistence using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

Use `open(path) do io ... end` to close files even when operations fail. `eachline` supports line processing. Julia has TOML in its standard library but no general JSON parser; use JSON3.jl for JSON.

Lesson focus: JSON persistence. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
using TOML
record=Dict("id"=>1,"title"=>"Study Julia","done"=>false)
restored=TOML.parse(sprint(TOML.print,record))
println(restored["title"])
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Relative paths are resolved from the process working directory.
- Use `open(path) do io` to guarantee resource cleanup.
- Catch expected exceptions specifically rather than hiding all failures.

Activities

1. Create a temporary file fixture and read it line by line.
2. Test empty, missing, and malformed files.

3. Verify resources close after successful and exceptional paths.
4. Document the file format and parser limitations.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

- ☐ I can explain json persistence in Julia terminology.
- ☐ I can trace the example and predict its output.
- ☐ I can adapt the example to canonical shared data.
- ☐ I can identify and correct a Julia-specific error or edge case.
- ☐ I can run and inspect the program using keyboard-accessible tools.

10.11 Unit 9: Testing and Data Analysis

Overview

This unit covers unit tests, testing stateful types, randomness, text analysis, CSV processing. Follow the same problem progression as Python so that the main comparison is language syntax and semantics. Use the worked examples as starting points, then complete the activities independently.

Lesson 9.1: Testing functions

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain testing functions using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

The `Test` standard library provides assertions, and `Random` supports seeded sequences. Text analysis uses `split`, regular expressions, and dictionaries. Use CSV.jl for quoted delimiters, escaped quotes, and embedded newlines.

Lesson focus: Testing functions. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
using Test
@test 2 + 3 == 5
println("Arithmetic tests passed")
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Seed random generators when tests require reproducibility.
- Use approximate comparison for floating-point results where appropriate.
- A `split`-based CSV demonstration is not a full quoted-field parser.

Activities

1. Test normal, boundary, and invalid cases.
2. Seed random operations and verify reproducibility.
3. Analyze the canonical text sample and verify word counts.
4. Process the CSV columns and document unsupported quoting cases.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

☐ I can explain testing functions in Julia terminology.
☐ I can trace the example and predict its output.
☐ I can adapt the example to canonical shared data.
☐ I can identify and correct a Julia-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 9.2: Testing stateful objects

Reading References

- Julia Documentation

- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain testing stateful objects using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

The `Test` standard library provides assertions, and `Random` supports seeded sequences. Text analysis uses `split`, regular expressions, and dictionaries. Use CSV.jl for quoted delimiters, escaped quotes, and embedded newlines.

Lesson focus: Testing stateful objects. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
using Test
mutable struct TaskState; title::String; done::Bool; end
t=TaskState("Study",false); @test !t.done
t.done=true; @test t.done
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Seed random generators when tests require reproducibility.
- Use approximate comparison for floating-point results where appropriate.
- A `split`-based CSV demonstration is not a full quoted-field parser.

Activities

1. Test normal, boundary, and invalid cases.
2. Seed random operations and verify reproducibility.
3. Analyze the canonical text sample and verify word counts.
4. Process the CSV columns and document unsupported quoting cases.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and

read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

- ☐ I can explain testing stateful objects in Julia terminology.
- ☐ I can trace the example and predict its output.
- ☐ I can adapt the example to canonical shared data.
- ☐ I can identify and correct a Julia-specific error or edge case.
- ☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 9.3: Random values

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain random values using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

The `Test` standard library provides assertions, and `Random` supports seeded sequences. Text analysis uses `split`, regular expressions, and dictionaries. Use CSV.jl for quoted delimiters, escaped quotes, and embedded newlines.

Lesson focus: Random values. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
using Random
Random.seed!(2026)
println(rand(1:10)); println(rand())
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Seed random generators when tests require reproducibility.
- Use approximate comparison for floating-point results where appropriate.
- A `split`-based CSV demonstration is not a full quoted-field parser.

Activities

1. Test normal, boundary, and invalid cases.
2. Seed random operations and verify reproducibility.
3. Analyze the canonical text sample and verify word counts.
4. Process the CSV columns and document unsupported quoting cases.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

☐ I can explain random values in Julia terminology.
☐ I can trace the example and predict its output.
☐ I can adapt the example to canonical shared data.
☐ I can identify and correct a Julia-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 9.4: Text analysis

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain text analysis using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

The `Test` standard library provides assertions, and `Random` supports seeded sequences. Text analysis uses `split`, regular expressions, and dictionaries. Use CSV.jl for quoted delimiters, escaped quotes, and embedded newlines.

Lesson focus: Text analysis. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
sample="A student studies programming. Programming builds skills."
words=split(lowercase(sample),r"[^a-z]+")
filter!(x -> !isempty(x), words)
println(length(words))
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Seed random generators when tests require reproducibility.
- Use approximate comparison for floating-point results where appropriate.
- A `split`-based CSV demonstration is not a full quoted-field parser.

Activities

1. Test normal, boundary, and invalid cases.
2. Seed random operations and verify reproducibility.
3. Analyze the canonical text sample and verify word counts.
4. Process the CSV columns and document unsupported quoting cases.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

☐ I can explain text analysis in Julia terminology.
☐ I can trace the example and predict its output.
☐ I can adapt the example to canonical shared data.
☐ I can identify and correct a Julia-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 9.5: CSV records

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain csv records using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

The `Test` standard library provides assertions, and `Random` supports seeded sequences. Text analysis uses `split`, regular expressions, and dictionaries. Use CSV.jl for quoted delimiters, escaped quotes, and embedded newlines.

Lesson focus: CSV records. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
csv="id,first_name,score\n1,Avery,91\n2,Jordan,84"
for row in split.(split(chomp(csv),'\n')[2:end], ',')
    println(row[2], ": ", parse(Int,row[3]))
end
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Seed random generators when tests require reproducibility.
- Use approximate comparison for floating-point results where appropriate.
- A `split`-based CSV demonstration is not a full quoted-field parser.

Activities

1. Test normal, boundary, and invalid cases.
2. Seed random operations and verify reproducibility.
3. Analyze the canonical text sample and verify word counts.
4. Process the CSV columns and document unsupported quoting cases.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

- ☐ I can explain csv records in Julia terminology.
- ☐ I can trace the example and predict its output.
- ☐ I can adapt the example to canonical shared data.
- ☐ I can identify and correct a Julia-specific error or edge case.
- ☐ I can run and inspect the program using keyboard-accessible tools.

10.12 Unit 10: Project-Based Learning

Overview

This unit covers requirements, top-down design, text adventure, command-line to-do manager, accessible validation. Follow the same problem progression as Python so that the main comparison is language syntax and semantics. Use the worked examples as starting points, then complete the activities independently.

Lesson 10.1: Requirements and design

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain requirements and design using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

Begin projects with requirements, input/output contracts, and acceptance tests. Keep the text adventure and task manager text-based. Reject invalid commands and communicate status through text, not color alone.

Lesson focus: Requirements and design. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
requirements=["show location","move through valid exits","list inventory","save tasks"]
foreach(println,requirements)
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Write input, output, and acceptance criteria before implementation.
- Reject unknown commands and unavailable room exits explicitly.
- Make status understandable without color or mouse interaction.

Activities

1. Write requirements before implementing a feature.
2. Test valid movement, unavailable exits, and unknown commands.
3. Test inventory and puzzle states with and without required items.
4. Review prompts and errors using keyboard-only, sequential reading.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

☐ I can explain requirements and design in Julia terminology.
☐ I can trace the example and predict its output.
☐ I can adapt the example to canonical shared data.
☐ I can identify and correct a Julia-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 10.2: Text adventure: five rooms and movement

Reading References

- Julia dictionaries
- Julia input and output
- Canonical shared project specification: Python Unit 10, Lesson 10.3.

Learning Objectives

- Represent all five canonical rooms and exits with Julia dictionaries.
- Validate movement before changing the current room.
- Explain why treasure_room is not reachable until the key puzzle is implemented.

Concept Explanation

The shared graph has five rooms: start_room, tunnel, clearing, locked_door_room, and treasure_room. Each room stores a description and an exits dictionary. The player starts in start_room. Part 1 leaves the north exit from locked_door_room unavailable. The get function returns nothing for an absent direction, so an invalid command does not change state.

Worked Example

```
function main()
rooms = Dict(
    "start_room" => (description = "A dimly lit cave. A narrow passage leads north.",
                     exits = Dict("north" => "tunnel")),
    "tunnel" => (description = "A damp tunnel. Passages lead south and east.",
                 exits = Dict("south" => "start_room", "east" => "clearing")),
    "clearing" => (description = "A quiet clearing. An antechamber lies to the north.",
                   exits = Dict("west" => "tunnel", "north" => "locked_door_room")),
    "locked_door_room" => (description = "A chamber with a heavy door to the north.",
                           exits = Dict("south" => "clearing")),
    "treasure_room" => (description = "You found the treasure. You win!",
                        exits = Dict())
)

current_room = "start_room"
while true
    room = rooms[current_room]
    println("---")
    println(room.description)
    println("What do you want to do?")
    command = eof(stdin) ? "quit" : lowercase(strip(readline()))
    if command == "quit"
        println("Exiting game. Goodbye!")
        break
    end
    destination = get(room.exits, command, nothing)
    if destination === nothing
        println("You cannot go that way. Enter a listed direction or quit.")
    else
        current_room = destination
```

```
    end
end
end

main()
```

Expected Behavior or Output

Enter north, east, north, north, quit. The player reaches tunnel, clearing, and locked_door_room. The second north is rejected and the current room does not change. The treasure room exists but has no incoming exit in Part 1.

Common Mistakes

- Indexing a dictionary with an unvalidated direction.
- Updating current_room before checking whether get returned nothing.
- Defining a destination that is not one of the five room identifiers.
- Adding the treasure exit before implementing the key puzzle.
- In Julia, place the interactive loop inside a function such as `main()` so assignments update the function-local `current_room` rather than triggering top-level soft-scope ambiguity.

Activities

1. Trace north, east, north, north and record the current room.
2. Try west from start_room and verify that the room does not change.
3. Check every exit destination against the room dictionary.
4. Explain why treasure_room cannot yet be reached.

Accessibility Practice

Announce the room description and prompt in a consistent order. The invalid-movement response explains what happened and how to continue. Run from the terminal and inspect output sequentially with the learner's access technology.

Skills Checklist

☐ I can model all five canonical rooms.
☐ I can represent exits with dictionaries.
☐ I can reject invalid movement without changing state.
☐ I can explain why the treasure room is not yet reachable.

Lesson 10.3: Text adventure: shiny key and locked door

Reading References

- Julia arrays
- Julia dictionaries
- Canonical shared project specification: Python Unit 10, Lesson 10.3.

Learning Objectives

- Place the shiny key in start_room and transfer it to inventory exactly once.
- Require key use in locked_door_room.
- Unlock the north exit only after the puzzle succeeds.
- End the game when the player reaches treasure_room.

Concept Explanation

Keep the same five-room graph. Each room has a mutable items vector. Taking the shiny key removes it from start_room and adds it to inventory. Using the key succeeds only in locked_door_room when inventory contains it. The north exit remains blocked until the key is used.

Worked Example

```
function main()
rooms = Dict(
    "start_room" => (description = "A dimly lit cave.",
                     exits = Dict("north" => "tunnel"), items = ["shiny key"]),
    "tunnel" => (description = "A damp tunnel. Passages lead south and east.",
                 exits = Dict("south" => "start_room", "east" => "clearing"), items = String
    "clearing" => (description = "A quiet clearing. Antechamber lies to the north.",
                   exits = Dict("west" => "tunnel", "north" => "locked_door_room"), items =
    "locked_door_room" => (description = "A chamber with a heavy door to the north.",
                           exits = Dict("south" => "clearing", "north" => "treasure_room"),
    "treasure_room" => (description = "You found the treasure. You win!",
                        exits = Dict(), items = String[])
)

current_room = "start_room"
inventory = String[]
door_locked = true

while true
    room = rooms[current_room]
    println()
    println(room.description)
```

```
    current_room == "treasure_room" && break
    !isempty(room.items) && println("You see: ", join(room.items, ", "))
    println("Your inventory: ", isempty(inventory) ? "empty" : join(inventory, ", "))
    println("What do you want to do?")
    command = eof(stdin) ? "quit" : lowercase(strip(readline()))

    if command == "quit"
        println("You quit the adventure.")
        break
    elseif command == "take shiny key"
        index = findfirst(==("shiny key"), room.items)
        if index === nothing
            println("There is no shiny key here to take.")
        else
            deleteat!(room.items, index)
            push!(inventory, "shiny key")
            println("You picked up the shiny key.")
        end
    elseif command == "use shiny key"
        if current_room != "locked_door_room"
            println("You must be at the locked door to use the key.")
        elseif !("shiny key" in inventory)
            println("You do not have the shiny key.")
        else
            door_locked = false
            println("You unlocked the door!")
        end
    elseif current_room == "locked_door_room" && command == "north" && door_locked
        println("The door is locked! You need to use the shiny key here.")
    elseif haskey(room.exits, command)
        current_room = room.exits[command]
    else
        println("Invalid command. Enter north, south, east, west, take shiny key, use shiny
    end
end
end

main()
```

Expected Behavior or Output

Enter north, east, north, north, south, west, south, take shiny key, north, east, north, use shiny key, north. The first north from locked_door_room is rejected. Taking the key removes it from the room and adds it to inventory. Using it outside locked_door_room is rejected. Using it there unlocks the door; the final north reaches treasure_room and prints the win description.

Common Mistakes

- Deleting the key from the room without adding it to inventory.
- Unlocking the door outside locked_door_room.
- Unlocking the door without checking inventory.
- Allowing north movement before door_locked becomes false.
- Mutating a copied room collection instead of the stored mutable vector.
- In Julia, place the interactive loop inside a function such as `main()` so assignments update the function-local `current_room` rather than triggering top-level soft-scope ambiguity.

Activities

1. Run the complete puzzle sequence and record room, items, inventory, and door state.
2. Attempt to take the key twice; the second attempt must be rejected.
3. Try to use the key outside locked_door_room.
4. Try north before and after unlocking.
5. Add a help command listing every supported command.

Accessibility Practice

Announce the room, visible items, inventory, and prompt as separate text lines. Error messages explain the failed action. Test the complete interaction with keyboard-only input and sequential screen-reader output.

Skills Checklist

☐ I can model the five canonical rooms.
☐ I can transfer the key from a room to inventory.
☐ I can prevent duplicate pickup.
☐ I can require key use in locked_door_room.
☐ I can block and unlock the north exit.
☐ I can end the game in treasure_room.

Lesson 10.4: Command-line to-do manager

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain command-line to-do manager using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

Begin projects with requirements, input/output contracts, and acceptance tests. Keep the text adventure and task manager text-based. Reject invalid commands and communicate status through text, not color alone.

Lesson focus: Command-line to-do manager. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
tasks=[(id=1,title="Read lesson",done=false),(id=2,title="Run example",done=true)]
foreach(t->println(t.done ? "[x] " : "[ ] ",t.title),tasks)
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Write input, output, and acceptance criteria before implementation.
- Reject unknown commands and unavailable room exits explicitly.
- Make status understandable without color or mouse interaction.

Activities

1. Write requirements before implementing a feature.
2. Test valid movement, unavailable exits, and unknown commands.
3. Test inventory and puzzle states with and without required items.
4. Review prompts and errors using keyboard-only, sequential reading.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

- ☐ I can explain command-line to-do manager in Julia terminology.
- ☐ I can trace the example and predict its output.
- ☐ I can adapt the example to canonical shared data.
- ☐ I can identify and correct a Julia-specific error or edge case.
- ☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 10.5: Accessible UX and validation

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain accessible ux and validation using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

Begin projects with requirements, input/output contracts, and acceptance tests. Keep the text adventure and task manager text-based. Reject invalid commands and communicate status through text, not color alone.

Lesson focus: Accessible UX and validation. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
function validate_title(s)
    t=strip(s); isempty(t) && return (false,"Title cannot be blank")
    return (true,t)
end
println(validate_title("  Study Julia  "))
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Write input, output, and acceptance criteria before implementation.
- Reject unknown commands and unavailable room exits explicitly.

- Make status understandable without color or mouse interaction.

Activities

1. Write requirements before implementing a feature.
2. Test valid movement, unavailable exits, and unknown commands.
3. Test inventory and puzzle states with and without required items.
4. Review prompts and errors using keyboard-only, sequential reading.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

☐ I can explain accessible ux and validation in Julia terminology.
☐ I can trace the example and predict its output.
☐ I can adapt the example to canonical shared data.
☐ I can identify and correct a Julia-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

10.13 Unit 11: Capstone

Overview

This unit covers accessible personal-data manager, model, persistence, CRUD, tests, documentation. Follow the same problem progression as Python so that the main comparison is language syntax and semantics. Use the worked examples as starting points, then complete the activities independently.

Lesson 11.1: Capstone requirements

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain capstone requirements using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

The capstone is an accessible personal-data manager with list/add/search/update/delete/complete operations. Separate data model, validation, persistence, and interaction; test save/reload.

Lesson focus: Capstone requirements. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
println("Capstone: accessible personal data manager")
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Pass application state explicitly rather than hiding it in mutable globals.
- Validate updates and preserve stable record identifiers.
- Do not claim persistence or accessibility without save/reload and keyboard tests.

Activities

1. Specify record fields and validation rules before storage.
2. Test list, add, search, update, delete, and completion.
3. Test duplicate, missing, empty, and invalid values.
4. Create a save/reload test and document known limitations.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

- ☐ I can explain capstone requirements in Julia terminology.
- ☐ I can trace the example and predict its output.
- ☐ I can adapt the example to canonical shared data.
- ☐ I can identify and correct a Julia-specific error or edge case.
- ☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 11.2: Data model and storage

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain data model and storage using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

The capstone is an accessible personal-data manager with list/add/search/update/delete/complete operations. Separate data model, validation, persistence, and interaction; test save/reload.

Lesson focus: Data model and storage. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
struct Record; id::Int; title::String; description::String; category::String; created_at::St
println(Record(1,"Study Julia","Review arrays","learning","2026-11-01","2026-11-01"))
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Pass application state explicitly rather than hiding it in mutable globals.
- Validate updates and preserve stable record identifiers.
- Do not claim persistence or accessibility without save/reload and keyboard tests.

Activities

1. Specify record fields and validation rules before storage.
2. Test list, add, search, update, delete, and completion.
3. Test duplicate, missing, empty, and invalid values.
4. Create a save/reload test and document known limitations.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

- ☐ I can explain data model and storage in Julia terminology.
- ☐ I can trace the example and predict its output.
- ☐ I can adapt the example to canonical shared data.
- ☐ I can identify and correct a Julia-specific error or edge case.
- ☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 11.3: Create/read/update/delete

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain create/read/update/delete using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

The capstone is an accessible personal-data manager with list/add/search/update/delete/complete operations. Separate data model, validation, persistence, and interaction; test save/reload.

Lesson focus: Create/read/update/delete. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
records=[(id=1,title="Study",done=false),(id=2,title="Practice",done=true)]
println(filter(r->!r.done,records))
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Pass application state explicitly rather than hiding it in mutable globals.
- Validate updates and preserve stable record identifiers.
- Do not claim persistence or accessibility without save/reload and keyboard tests.

Activities

1. Specify record fields and validation rules before storage.
2. Test list, add, search, update, delete, and completion.
3. Test duplicate, missing, empty, and invalid values.
4. Create a save/reload test and document known limitations.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

☐ I can explain create/read/update/delete in Julia terminology.
☐ I can trace the example and predict its output.
☐ I can adapt the example to canonical shared data.
☐ I can identify and correct a Julia-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 11.4: Validation and testing

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain validation and testing using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

The capstone is an accessible personal-data manager with list/add/search/update/delete/complete operations. Separate data model, validation, persistence, and interaction; test save/reload.

Lesson focus: Validation and testing. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
record=(id=1,title="Study",done=false)
function rename(r,title)
    cleaned=strip(title); isempty(cleaned) && throw(ArgumentError("blank title"))
    merge(r,(title=cleaned,))
end
println(rename(record,"Study Julia"))
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Pass application state explicitly rather than hiding it in mutable globals.
- Validate updates and preserve stable record identifiers.
- Do not claim persistence or accessibility without save/reload and keyboard tests.

Activities

1. Specify record fields and validation rules before storage.
2. Test list, add, search, update, delete, and completion.
3. Test duplicate, missing, empty, and invalid values.
4. Create a save/reload test and document known limitations.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Skills Checklist

- ☐ I can explain validation and testing in Julia terminology.
- ☐ I can trace the example and predict its output.
- ☐ I can adapt the example to canonical shared data.
- ☐ I can identify and correct a Julia-specific error or edge case.
- ☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 11.5: Documentation and presentation

Reading References

- Julia Documentation
- Julia Base and standard-library reference.
- Corresponding Python curriculum lesson for the shared problem and data.

Learning Objectives

- Explain documentation and presentation using Julia terminology.
- Apply the relevant syntax and standard-library facilities.
- Test normal, boundary, and invalid cases using shared course data.

Concept Explanation

The capstone is an accessible personal-data manager with list/add/search/update/delete/complete operations. Separate data model, validation, persistence, and interaction; test save/reload.

Lesson focus: Documentation and presentation. The worked example is self-contained unless a standard-library import is shown.

Worked Example

```
println("Acceptance: list, add, search, update, delete, complete; document persistence and k
```

Expected Behavior or Output

The program prints a result based on the stated inputs. Predict the result before running it, then compare the observed output. File examples use temporary paths and close resources within the callback.

Common Mistakes

- Pass application state explicitly rather than hiding it in mutable globals.
- Validate updates and preserve stable record identifiers.
- Do not claim persistence or accessibility without save/reload and keyboard tests.

Activities

1. Specify record fields and validation rules before storage.
2. Test list, add, search, update, delete, and completion.
3. Test duplicate, missing, empty, and invalid values.
4. Create a save/reload test and document known limitations.

Accessibility Practice

Navigate source by line, word, and character. Inspect operators, punctuation, mutation markers, and block-ending `end` keywords. Run from the terminal and read output and diagnostics sequentially. Do not rely on syntax color or mouse-only controls.

Runnable Reference Implementation

The repository includes a tested Julia domain and JSON-storage reference core in capstone/README.md. Install the JSON dependency and run the acceptance suite from the repository root using:

```
julia -e 'using Pkg; Pkg.add("JSON3")' && julia JULIA/capstone/test_records.jl
```

The reference core does not replace the requirement to build and evaluate an accessible user interface.

Skills Checklist

- ☐ I can explain documentation and presentation in Julia terminology.
- ☐ I can trace the example and predict its output.
- ☐ I can adapt the example to canonical shared data.
- ☐ I can identify and correct a Julia-specific error or edge case.
- ☐ I can run and inspect the program using keyboard-accessible tools.

Julia-specific package and data-format notes

- The standard library includes `Test`, `Random`, and `TOML`.
- Julia does not include a general JSON parser in its standard library. Use JSON3.jl for JSON serialization and parsing, and record it in the active `Project.toml`.
- Use CSV.jl for quoted delimiters, escaped quotes, or embedded newlines. The introductory `split` approach is limited to simple sample data.
- Use DataFrames.jl only when tabular analysis justifies the dependency; begin with arrays, tuples, named tuples, and dictionaries.
- Standard Julia arrays are one-based. Use `eachindex` or `enumerate` when iteration needs indices.

AI Policy

AI may be used only for personal conceptual understanding and reinforcement. It must not write, complete, autocomplete, repair, or generate course code or assessment solutions.

Part XI

KOTLIN

Chapter 11

Introduction to Kotlin

Before you start: complete Unit P: Computing Foundations Primer. It explains the language-neutral vocabulary and concepts used in every language track, using non-visual examples.

Standardized beginner curriculum

This course is a Kotlin/JVM parallel track aligned to the shared 12-unit, 57-lesson sequence. It assumes no prior Java experience. Kotlin is statically typed, supports nullable and non-nullable types, and interoperates with Java libraries on the JVM.

Toolchain

- Use a supported JDK; the course Java pathway baseline is JDK 25 LTS.
- Use Kotlin compiler and Gradle plugin 1.9.24 (the version used by the example-validation workflow) for the documented course baseline; check the Kotlin release process before changing versions.
- Verify the compiler with `kotlinc -version` and the runtime with `java -version`.
- For a single file: `kotlinc Hello.kt -include-runtime -d hello.jar`, then `java -jar hello.jar`.
- For multi-file work, use the Gradle wrapper and Kotlin/JVM plugin. Commit the wrapper so the build version is reproducible.

Course files

- Unit 0: accessible development environment
- Unit 1: foundations of computing

- Unit 2: numeric computations
- Unit 3: control flow, decisions, and loops
- Unit 4: data structures, lists, and dictionaries
- Unit 5: user input and while loops
- Unit 6: functions
- Unit 7: classes and object-oriented programming
- Unit 8: files and exceptions
- Unit 9: testing and data analysis
- Unit 10: project-based text applications
- Unit 11: accessible personal data manager capstone
- Complete standalone Kotlin reference lessons
- 57-topic index
- Teacher guide
- Expected output fixtures for all 57 examples
- Standardization audit

Kotlin semantics emphasized

Teach `val` versus `var`, static typing and inference, explicit numeric conversions, nullable types, safe calls (`?.`), Elvis expressions (`?:`), and the risk of not-null assertions (`!!`). Kotlin has no general tuple syntax; use named data classes, Pair, or Triple as appropriate. Distinguish read-only collection interfaces from mutable collections. Kotlin classes are final by default; inheritance requires deliberate `open` declarations. Use `use` for resource-safe operations and maintained libraries for JSON and CSV.

Accessibility baseline

Use keyboard-first editor and terminal workflows. Verify the selected Kotlin language server, diagnostics, Gradle tasks, and screen-reader announcements with the learner's actual access technology. Explain punctuation and indentation explicitly. Prefer named intermediate values and short functions over deeply nested expressions. The default capstone interface is text-first; a GUI is not required to demonstrate the shared data operations.

Assessment and AI-use policy

Assess conceptual understanding, independent tracing, testing, explanation, and accessible tool use. AI may support conceptual understanding and reinforcement only; it must not generate, complete, repair, refactor, or write course code or assessment solutions for students or instructors.

The lesson files provide the aligned instructional structure. The audit records what has and has not been verified; structural completeness is not a claim that

every example or full application has passed runtime tests.

GUI, visualization, and workbench accessibility

See the shared GUI, visualization, and workbench accessibility reference for screen-reader support by toolkit, platform-specific activation settings, recommended alternatives, and interfaces that must not be required without an equivalent accessible route.

AI Policy

AI may be used only for personal conceptual understanding and reinforcement. It must not write, complete, autocomplete, repair, or generate course code or assessment solutions.

Verification status

Every lesson has one Kotlin example that uses the shared datasets. All 57 examples compile together with `kotlinc` 1.9.24, run independently with empty standard input, and match the exact output in `examples_expected.json` (`scripts/check_kotlin_examples.py`; checked on JDK 21). The "Expected Behavior or Output" blocks in the lessons are captured from runs of the same examples, with the standard input shown where a program reads input. This has not been tested with screen readers or braille displays.

Recommended introductory books

- Griffiths, Dawn, and David Griffiths. *Head First Kotlin: A Brain-Friendly Guide*. O'Reilly Media, 2019. A hands-on introduction to Kotlin syntax and programming concepts. Publisher catalog.
- Eckel, Bruce, and Svetlana Isakova. *Atomic Kotlin*. Mindview, 2021. Presents Kotlin concepts in small, focused lessons with exercises; intended for both beginning and experienced programmers. Author-maintained learning resources.
- Aigner, Sebastian, Roman Elizarov, Svetlana Isakova, and Dmitry Jemerov. *Kotlin in Action*, 2nd edition. Manning, 2024. A comprehensive treatment aimed primarily at readers familiar with Java or another object-oriented language. Kotlin documentation's book recommendations.

Suggested use: *Head First Kotlin* or *Atomic Kotlin* is the more suitable first companion. Treat *Kotlin in Action* as a follow-on reference, not a prerequisite. Check current Kotlin documentation when book examples differ from the course baseline.

11.1 KOTLIN Lessons

The curriculum is organized as one Markdown file per unit.

- accessible development environment
- foundations of computing
- numeric computations
- control flow decisions and loops
- data structures lists and dictionaries
- user input and while loops
- functions
- classes and object oriented programming
- files and exceptions
- testing and data analysis
- project based learning text based applications
- capstone accessible personal data manager

11.2 Unit 0: Accessible Development Environment

Overview

Set up the language toolchain, a project folder, and a keyboard-first editor workflow that can be read with speech or braille, and learn how projects and dependencies are organized.

Lesson 0.1: Installing the Kotlin Compiler and JDK

Reading References

- Kotlin command-line compiler
- Get started with Kotlin
- Matching Python lesson: Lesson 0.1: Installing Python and uv

Learning Objectives

- Install a JDK and the Kotlin command-line compiler from trusted sources.
- Verify both tools from the terminal.
- Compile and run a first program.

Concept Explanation

Kotlin/JVM programs are compiled to Java bytecode and run on a Java Virtual Machine, so two tools are needed: a **JDK** (Java Development Kit) and the **Kotlin compiler**, `kotlinc`. This course's documented baseline is Kotlin 1.9.24

(the example checks in this repository use it) running on a supported JDK; check the Kotlin release page before changing versions.

Choose one installation route and write down which one you used: a JDK from your operating system's package manager or a vendor build, and the compiler from the official Kotlin release archive, from a version manager such as SDKMAN, or from your package manager. Package managers often install the newest Kotlin rather than 1.9.24, so always verify the version.

Verify both tools from the terminal:

```
java -version
kotlinc -version
```

The second command prints a line of the form `info: kotlinc-jvm 1.9.24 (JRE 21...)`. The numbers depend on what you installed; the point is that both commands run and report a version.

To compile and run the example below, save it as `hello.kt`, then:

```
kotlinc hello.kt -include-runtime -d hello.jar
java -jar hello.jar
```

`-include-runtime` packs the Kotlin library into the jar so that `java -jar` can run it, and `-d` names the output. The first compile of a session can take several seconds.

Worked Example

```
fun main() {
    println("Kotlin toolchain ready.")
}
```

Expected Behavior or Output

The program prints:

```
Kotlin toolchain ready.
```

Common Mistakes

1. Installing a JDK but not the Kotlin compiler (or the reverse) and concluding that nothing works.
2. Assuming the newest Kotlin is the course version. Check `kotlinc -version`.
3. Running `kotlinc` without `-include-runtime` and then being unable to start the jar with `java -jar`.

Activities

1. Install both tools and record the exact output of the two version commands.
2. Compile and run the example from the keyboard and note how long the first compile takes.
3. Rename the file `hello.kt` to `hello.txt`, try to compile it, and write down the error.
4. Explain in two sentences why Kotlin needs a JDK.

Accessibility Practice

Run each command from the keyboard only and read the output line by line with your screen reader. Confirm that the version numbers are announced clearly. Compiler warnings appear before the result, so jump to the last line of output to find whether the compile succeeded.

Skills Checklist

- ☐ I can install a JDK and the Kotlin compiler.
- ☐ I can verify both versions.
- ☐ I can compile and run a first program.
- ☐ I know what `-include-runtime` does.
- ☐ I can say which route I used.

Lesson 0.2: Workspace and First Run

Reading References

- Kotlin command-line compiler
- Basic syntax
- Matching Python lesson: Lesson 0.2: Configuring Your Screen Reader for VS Code or Theia IDE

Learning Objectives

- Create a project folder with a first source file.
- Compile and run it from the terminal.
- Read a compiler error and find the file, line, and column.

Concept Explanation

A *project folder* keeps one program's files together. Create a folder named `kotlin-course`, save a file named `main.kt` inside it, and use a terminal opened in that folder. The folder you are working in is the *working directory*; commands look for relative file names there.

Every Kotlin program starts at a function named `main`. A source file may contain `fun main()` with no parameters, or `fun main(args: Array<String>)` to receive command-line arguments. Statements run in order, and Kotlin does not require semicolons at the ends of lines.

Compile and run from the working directory:

```
kotlinc main.kt -include-runtime -d main.jar
java -jar main.jar
```

If the file contains a mistake, `kotlinc` prints messages of the form `main.kt:3:5: error: ...`. Read each as file name, line, column, then the description. Always go to that line first, and fix the first error before the others, because later errors are often caused by it.

Worked Example

```
fun main() {
    println("Hello, Avery!")
    println("Welcome to Kotlin programming!")
}
```

Expected Behavior or Output

The program prints:

```
Hello, Avery!
Welcome to Kotlin programming!
```

Common Mistakes

1. Running the commands from a different folder, so the compiler cannot find `main.kt`.
2. Fixing the last error message first. Start with the first one.
3. Saving the file with a different extension or name than the one in the command.

Activities

1. Create the folder and file from the keyboard and run the program. Record the working directory.
2. Delete the closing quotation mark on line 2, compile, and write down the file, line, column, and description in the message. Then restore it.
3. Add a third `println` with your own name and predict the output before running.
4. Explain in your own words what the working directory is.

Accessibility Practice

Create the folder and file with terminal commands or your editor's keyboard commands. When the compiler reports an error, use your screen reader's line-by-line review to read the file name, line, and column first, then jump to that line in the editor.

Skills Checklist

- ☐ I can create a project folder and source file.
- ☐ I can compile and run from the terminal.
- ☐ I can read the file, line, and column of an error.
- ☐ I know that Kotlin does not need semicolons.
- ☐ I know what the working directory is.

Lesson 0.3: Editor Extensions and Reading Kotlin Aloud

Reading References

- Null safety
- Basic syntax
- Matching Python lesson: Lesson 0.3: Setting Up Your VS Code or Theia IDE Workspace

Learning Objectives

- Configure an editor so Kotlin source is readable by speech or braille.
- Choose a punctuation level that announces the symbols that carry meaning in Kotlin.
- Read nullable types, safe calls, and the Elvis operator aloud.

Concept Explanation

Complete the shared IDE setup lesson first. Then adapt the editor for Kotlin. A Kotlin language extension is optional; the course works with a plain editor and the terminal. If you install one, take it only from a source you trust and read the permissions it requests.

Kotlin code uses several symbols that a screen reader may skip at low punctuation settings. Learn to say them: `?` after a type means *nullable*, `?.` is a *safe call*, `?:` is the *Elvis operator* that supplies a default, `!!` is a *not-null assertion* that crashes if the value is null, `->` separates a condition from its result in `when`, `{ }` hold blocks, and `$name` or `${expression}` insert a value into a string.

Unlike Python, Kotlin does not use indentation to decide structure. Braces do, and indentation is for people; count the closing braces when you read a nested block.

Read the example aloud: ”val name colon String question mark equals null. println open parenthesis name question-dot length question-colon minus one close parenthesis.” Then say what it prints.

Worked Example

```
fun main() {
    val name: String? = null
    println(name?.length ?: -1)
    val title: String? = "Kotlin"
    println(title?.length ?: -1)
    println("name is $name and title has ${title?.length} characters")
}
```

Expected Behavior or Output

The program prints:

```
-1
6
name is null and title has 6 characters
```

Common Mistakes

1. Setting punctuation so low that `?`, `?.`, and `?:` are never spoken, then misreading nullable code.
2. Relying on indentation alone to see where a block ends. Count the closing braces.
3. Installing several overlapping extensions and getting duplicate diagnostics.

Activities

1. Test your editor with the example and record the punctuation level at which every symbol is announced.
2. Navigate the file by line, word, and character and note the commands you used.
3. Remove a closing brace, save, and find how your editor reports it. Then restore it.
4. Write a half-page note of your edit, compile, and run steps using only the keyboard.

Accessibility Practice

This whole lesson is accessibility practice. Verify that you can open the file, move to a given line, read a line character by character, compile in the terminal, and read the output. Record the operating system, editor, and access technology you actually tested; do not describe the setup as verified for other combinations.

Skills Checklist

- ☐ I can configure my editor for keyboard-only use.
- ☐ I can hear every symbol in a nullable expression.
- ☐ I can read `?.` and `?:` aloud.
- ☐ I know Kotlin structure comes from braces.
- ☐ I can compile and read the output from the keyboard.

Lesson 0.4: Kotlin Toolchain, Terminal, and Command-Line Arguments

Reading References

- Kotlin command-line compiler
- Basic syntax
- Matching Python lesson: Lesson 0.4: Python Environments, Terminal Usage, and Accessibility Tools

Learning Objectives

- Run a program with command-line arguments.
- Describe the project root and where compiled files go.
- Explain when Gradle is needed.

Concept Explanation

A *command-line argument* is an extra word typed after the program name. A `main` function declared as `fun main(args: Array<String>)` receives them in `args`. This is the simplest way to pass input without a prompt, and it is easy to test.

Keep each project in its own folder and run commands from the *project root*, the folder that holds the sources. For a single file the compile-and-run commands from Lesson 0.2 are enough. For several files, or when you need a library, use **Gradle** with the Kotlin/JVM plugin and commit the Gradle wrapper so every machine builds with the same version. Do not add Gradle to a one-file exercise.

`kotlinc` accepts several source files in one command and one `-d` output jar. The program in this lesson can be run with:

```
kotlinc args.kt -include-runtime -d args.jar
java -jar args.jar red blue
```

Worked Example

```
fun main(args: Array<String>) {
    println("Argument count: ${args.size}")
```

```
    for ((index, arg) in args.withIndex()) {
        println("Argument ${index + 1}: $arg")
    }
    if (args.isEmpty()) {
        println("No arguments were given.")
    }
}
```

Expected Behavior or Output

Run with the command-line arguments `red blue`.

The program prints:

```
Argument count: 2
Argument 1: red
Argument 2: blue
```

Common Mistakes

1. Forgetting that `args` is empty when no arguments are typed, then reading `args[0]` and crashing.
2. Counting arguments from 1. The first argument is at index 0.
3. Using Gradle for a single-file exercise, which adds files and steps that are not needed.

Activities

1. Run the program with `red blue` and then with no arguments. Compare the output.
2. Pass three words and a number and print the number plus one.
3. Change the program to print the arguments in reverse order.
4. Explain when a project needs Gradle and when it does not.

Accessibility Practice

Arguments let you test a program without answering prompts, so every run produces plain text that is read in order. Read the output of the two runs aloud and compare them line by line.

Skills Checklist

- ☐ I can run a program with arguments.
- ☐ I know `args` starts at index 0.
- ☐ I can say where the project root is.
- ☐ I know when Gradle is needed.
- ☐ I can compile several files in one command.

11.3 Unit 1: Foundations of Computing

Overview

Connect problem solving and algorithms to a first program, then learn variables, values, and types using the shared student profile.

Lesson 1.1: Welcome to Computer Science

Reading References

- Basic syntax
- Control flow
- Matching Python lesson: Lesson 1.1: Welcome to Computer Science

Learning Objectives

- Explain hardware, software, and algorithm in your own words.
- Turn a written algorithm into a short Kotlin program.
- Predict a program's output by hand before running it.

Concept Explanation

Computers follow instructions exactly. *Hardware* is the machine, *software* is the set of instructions it follows, and an *algorithm* is an ordered, finite list of clear steps that solves a problem. Unit P of this repository covers these ideas in detail.

A reliable way to solve a problem is to understand it, break it into steps, write the steps in plain language, translate them into code, and test the result.

The plan here is: set a total to zero; for each score, add it to the total; divide the total by the number of scores; report both. In Kotlin, `val` makes a read-only name and `var` makes a name that can be reassigned, `for (score in scores)` visits a list in order, and `scores.size` is the number of items.

The shared scores are 82, 91, 76, and 88. Their total is 337 and their average is 84.25. A hand trace should reach the same numbers: 82, 173, 249, then 337.

Worked Example

```
fun main() {
    val scores = listOf(82, 91, 76, 88)
    var total = 0
    for (score in scores) {
        total += score
    }
    val average = total.toDouble() / scores.size
```

```
    println("Total: $total")
    println("Average: $average")
}
```

Expected Behavior or Output

The program prints:

```
Total: 337
Average: 84.25
```

Common Mistakes

1. Writing code before writing the steps, then forgetting to divide by the number of scores.
2. Dividing two integers and expecting a decimal. `337 / 4` is `84`; convert one side with `toDouble()`.
3. Declaring `total` with `val` and then trying to add to it, which is a compile error.

Activities

1. Trace the loop by hand and write the running total after each score.
2. Change the list to 90 and 80 and predict the total and average before running.
3. Add a loop that finds the highest score and prints it.
4. Explain the difference between the algorithm and the program in two sentences.

Accessibility Practice

Write the hand trace as numbered sentences, one for each loop pass, and read it back with your screen reader. Compare it with the two printed lines in order.

Skills Checklist

- ☐ I can define hardware, software, and algorithm.
- ☐ I can describe an algorithm as ordered steps.
- ☐ I can trace a loop by hand.
- ☐ I can explain `val` versus `var`.
- ☐ I can run the program and compare it with my trace.

Lesson 1.2: Your First Kotlin Program

Reading References

- Basic syntax

- Matching Python lesson: Lesson 1.2: Your First Python Program - Hello World

Learning Objectives

- Write and run a program that prints text.
- Use comments and string templates.
- Explain that statements run in order.

Concept Explanation

A Kotlin program is a list of *statements* that run in order inside `main`. `println` writes its argument and ends the line; `print` writes without ending the line. Text in double quotation marks is a *string*.

A *string template* inserts a value into text: `$name` inserts a variable, and `${expression}` inserts the result of any expression. A line comment starts with `//` and a block comment is written between `/*` and `*/`. Triple quotation marks, `"""..."""`, make a *raw string* that can span lines.

Because Kotlin is a compiled language, a mistake such as a missing quotation mark is reported at compile time and no program is produced.

Worked Example

```
fun main() {
    // This program greets the shared student.
    val name = "Avery"
    println("Hello, $name!")
    print("No new line here, ")
    println("then one here.")
    println("Name has ${name.length} letters.")
    println("""Two lines
in one raw string.""")
}
```

Expected Behavior or Output

The program prints:

```
Hello, Avery!
No new line here, then one here.
Name has 5 letters.
Two lines
in one raw string.
```

Common Mistakes

1. Using a curly quotation mark pasted from a document instead of a straight one, which is a syntax error.
2. Writing `$name` where a longer expression needs `${...}`, such as `$name.length`, which inserts the name and then the text `.length`.
3. Forgetting that `print` does not end the line.

Activities

1. Run the program and describe how the second and third lines differ from the others.
2. Change the greeting to use your own name and predict the output.
3. Print three lines: your name, the course name, and today's task.
4. Add a comment above every line and read the file aloud with the comments.

Accessibility Practice

Run from the terminal and read the output line by line. Check that your screen reader announces the dollar sign and the braces in a string template, since they decide what the program prints.

Skills Checklist

☐ I can write and run a program that prints text.
☐ I can use string templates.
☐ I can write comments.
☐ I know how `print` and `println` differ.
☐ I can read a first compile error.

Lesson 1.3: Variables and Simple Data Types

Reading References

- Basic types
- Basic syntax
- Matching Python lesson: Lesson 1.3: Variables and Simple Data Types

Learning Objectives

- Declare read-only and changeable variables.
- Name Kotlin's basic types and check a value's type.
- Explain type inference.

Concept Explanation

Kotlin is *statically typed*: every value has a type that the compiler knows. You may write the type, as in `val age: Int = 16`, or let the compiler *infer* it from the value, as in `val age = 16`. The type is still fixed after inference; Kotlin is not dynamically typed.

`val` declares a read-only name that cannot be reassigned. `var` declares a name that can. Prefer `val`, and use `var` only when a value must change.

The basic types are `Int` and `Long` (whole numbers), `Double` and `Float` (decimals), `Boolean`, `Char`, and `String`. A whole-number literal such as 16 is an `Int`, and a literal with a decimal point such as 1.72 is a `Double`. `value::class.simpleName` reports a value's type name.

The shared student profile is name Avery, age 16, height 1.72 meters, and favorite subject Computer Science. The shared Notebook item has quantity 3 and unit price 2.50, so three notebooks cost 7.5.

Worked Example

```
fun main() {
    val name = "Avery"
    val age = 16
    val height = 1.72
    val favoriteSubject = "Computer Science"
    println("$name, $age, $height, $favoriteSubject")
    println("${name::class.simpleName} ${age::class.simpleName} ${height::class.simpleName}")

    val notebookPrice = 2.50
    val notebookQuantity = 3
    println("Notebook total: ${notebookPrice * notebookQuantity}")

    var visits = 1
    visits += 1
    println("Visits: $visits")
}
```

Expected Behavior or Output

The program prints:

```
Avery, 16, 1.72, Computer Science
String Int Double
Notebook total: 7.5
Visits: 2
```

Common Mistakes

1. Declaring with `val` and then reassigning, which is a compile error.
2. Assuming inference makes Kotlin dynamic. `val x = 16` can never hold text later.
3. Mixing `Int` and `Double` without conversion in a declaration that names the type, such as `val d: Double = 16`, which does not compile.

Activities

1. Run the example and write each type name in the order printed.
2. Change `age` to `16L` and predict the type name.
3. Add a Boolean `likesCs` and print its type.
4. Explain, without code, when to choose `var` over `val`.

Accessibility Practice

Say names with their capital letters aloud (`favoriteSubject`), and print each value inside a labelled sentence so that the screen reader announces the name and value together.

Skills Checklist

☐ I can declare `val` and `var`.
☐ I can name Kotlin's basic types.
☐ I can explain type inference.
☐ I can check a value's type name.
☐ I can explain why `val` is preferred.

11.4 Unit 2: Numeric Computations

Overview

Plan and build the tip calculator, learn the language's numeric operators and conversions, read typed input safely, and use the math library.

Lesson 2.1: The Software Development Process

Reading References

- Basic syntax
- Basic types
- Matching Python lesson: Lesson 2.1: The Software Development Process

Learning Objectives

- Describe the design, implement, and test cycle.
- Write pseudocode for the tip calculator and implement it.
- Explain why money is formatted to two decimals.

Concept Explanation

Software is built in a cycle: understand the problem, design a solution, implement it, test it, and refine it. Design comes first. For the tip calculator the plan is: get the bill, calculate 15 percent of it as the tip, add the tip to the bill for the total, and print the tip and the total.

With the shared values (bill 24.50, tip rate 0.15) the exact tip is 3.675 and the total is 28.175. `"%.2f".format(x)` formats a number with two decimals. On the JVM, formatting rounds half up using the decimal digits of the `Double`, so Kotlin and Java display the tip as **3.68**; Python, C, Go, and several other languages display 3.67 for the same stored number. Both are correct displays of the same value.

Formatting uses the default *locale*. In a locale that writes decimals with a comma, `"%.2f".format(x)` prints a comma. Pass `java.util.Locale.US` as the first argument for fixed output.

Worked Example

```
fun main() {
    val bill = 24.5
    val tipRate = 0.15

    val tip = bill * tipRate
    val total = bill + tip

    println("Bill: " + "%.2f".format(bill))
    println("Tip: " + "%.2f".format(tip))
    println("Total: " + "%.2f".format(total))
    println("Tip with an explicit locale: " + "%.2f".format(java.util.Locale.US, tip))
    println("Unformatted tip: $tip")
}
```

Expected Behavior or Output

The program prints:

```
Bill: 24.50
Tip: 3.68
Total: 28.18
Tip with an explicit locale: 3.68
Unformatted tip: 3.675
```

Common Mistakes

1. Writing the rate as 15 instead of 0.15, which makes the tip fifteen times too large.
2. Rounding the tip first and then adding it, which can change the total by a cent.
3. Assuming every language prints the same two-decimal text for 3.675.

Activities

1. Trace the plan with the shared values and write each result as a sentence.
2. Change the rate to 0.20 and predict the tip and total (4.90 and 29.40).
3. Add a test for a bill of 100 with a written expected result before running.
4. Explain in two sentences why the program prints 3.68 even though the stored value is just below 3.675.

Accessibility Practice

Print each result on its own labelled line (`Tip: 3.68`) so a screen reader announces one clear unit per line. Never use alignment or spacing to carry meaning.

Skills Checklist

- ☐ I can name the stages of the development cycle.
- ☐ I can write pseudocode for a calculation.
- ☐ I can format a number with `format`.
- ☐ I know formatting depends on the locale.
- ☐ I can explain why displays can differ between languages.

Lesson 2.2: Expressions and Assignment

Reading References

- Basic types
- Basic syntax
- Matching Python lesson: Lesson 2.2: Elements of Programs - Expressions and Assignment

Learning Objectives

- Evaluate expressions using operator precedence.
- Use integer division, remainder, and compound assignment.
- Join strings with templates and `+`.

Concept Explanation

An *expression* produces a value. The arithmetic operators are `+`, `-`, `*`, `/`, and `%` (remainder). Multiplication, division, and remainder happen before addition and subtraction, and parentheses change the order, so `8 + 4 * 2` is 16 and `(8 + 4) * 2` is 24.

Division depends on the types. Two `Int` values give an `Int`, so `7 / 2` is `3`. If either side is a `Double`, the result is a `Double`, so `7 / 2.0` is `3.5`. The remainder `7 % 2` is `1`.

Assignment stores a value: `score = score + 9`, shortened to `score += 9`. Only a `var` can be assigned again. Comparison uses `==`, `!=`, `<`, `<=`, `>`, and `>=`. In Kotlin `==` compares values, not identities.

Kotlin does not convert numbers automatically: you cannot add an `Int` variable to a `Long` variable without a conversion. This catches mistakes at compile time.

Worked Example

```
fun main() {
    println(8 + 4 * 2)
    println((8 + 4) * 2)

    var score = 82
    score += 9
    println("New score: $score")

    println(7 / 2)
    println(7 / 2.0)
    println(7 % 2)
    println(-7 / 2)
    println(5 != 6)
    println("Avery" + " " + "Student")
}
```

Expected Behavior or Output

The program prints:

```
16
24
New score: 91
3
3.5
1
-3
true
Avery Student
```

Common Mistakes

1. Expecting `7 / 2` to be `3.5`. With two `Int` values the fraction is dropped.
2. Reassigning a `val`, which is a compile error.
3. Writing `score =+ 9` and storing positive nine instead of adding.

Activities

1. Predict the ten output lines before running, then compare.
2. Write an expression for the area and the perimeter of a 12 by 8 rectangle.
3. Explain why `-7 / 2` prints `-3` rather than `-4`.
4. Join your name, a space, and your age into one string in two different ways.

Accessibility Practice

Say operators by name when reading code aloud ('slash', 'percent', 'plus equals'). Raise your punctuation level while checking expressions so that parentheses and operators are all announced.

Skills Checklist

- ☐ I can apply operator precedence.
- ☐ I can predict `Int` and `Double` division.
- ☐ I can use `+=` and `%`.
- ☐ I know only `var` can be reassigned.
- ☐ I can join strings.

Lesson 2.3: Console Input and Output

Reading References

- Basic syntax
- Null safety
- Matching Python lesson: Lesson 2.3: Input and Output Revisited

Learning Objectives

- Read a line with `readlnOrNull`.
- Convert text with `toIntOrNull`.
- Handle missing and invalid input.

Concept Explanation

`readlnOrNull()` reads one line from standard input and returns a `String?`: the text without the line ending, or `null` at the end of input. `readln()` is similar but throws an exception at the end of input, so this course prefers `readlnOrNull()`.

Typed text is always a string. `toIntOrNull()` returns an `Int?`: the number, or `null` when the text is not a valid whole number. The safe-call operator `?.` stops a chain at `null`, so `readlnOrNull()?.trim()?.toIntOrNull()` reads, trims, and converts, and gives `null` if any step has nothing to work on.

After a null check on a `val`, Kotlin treats the value as non-null: this is a *smart cast*. The example reads a name and two numbers and prints a greeting and their sum. With Avery, 12, and 8 the sum is 20. Prompts are printed on their own lines so the screen reader reads each cleanly.

Worked Example

```
fun main() {
    println("Enter your name:")
    val name = readlnOrNull()
    println("Enter the first number:")
    val first = readlnOrNull()?.trim()?.toIntOrNull()
    println("Enter the second number:")
    val second = readlnOrNull()?.trim()?.toIntOrNull()

    if (name != null && first != null && second != null) {
        println("Hello, $name!")
        println("The sum of $first and $second is ${first + second}.")
    } else {
        println("Input was missing or not a number.")
    }
}
```

Expected Behavior or Output

With the following lines typed as standard input (piped input is not echoed, so it does not appear in the output):

```
Avery
12
8
```

the program prints:

```
Enter your name:
Enter the first number:
Enter the second number:
Hello, Avery!
The sum of 12 and 8 is 20.
```

With no standard input at all, the program ends cleanly and prints:

```
Enter your name:
Enter the first number:
```

```
Enter the second number:
Input was missing or not a number.
```

Common Mistakes

1. Using `readln()` and crashing at the end of input.
2. Doing arithmetic on a `String` read from input without converting it.
3. Using `!!` to silence the null check, which crashes on exactly the input the program should handle.

Activities

1. Run with Avery, 12, and 8 and confirm the output.
2. Type `twelve` for the first number and explain the result.
3. Run with no input and describe the output.
4. Change the program to subtract the second number from the first.

Accessibility Practice

Every prompt is a complete sentence on its own line and every result is a complete sentence. When you test, listen for whether the prompt is read in full before you begin to type.

Skills Checklist

- ☐ I can read a line with `readlnOrNull`.
- ☐ I can convert text with `toIntOrNull`.
- ☐ I understand `String?` and `?.`.
- ☐ I can write clear prompts.
- ☐ I handle missing and invalid input.

Lesson 2.4: Numbers and Type Conversions

Reading References

- Basic types
- Basic syntax
- Matching Python lesson: Lesson 2.4: Computing with Numbers and Type Conversions

Learning Objectives

- Convert explicitly between numbers and text.
- Explain truncation and rounding.
- Explain why decimal arithmetic is approximate.

Concept Explanation

Kotlin never converts numeric types for you. Use the conversion functions: `toInt()`, `toLong()`, `toDouble()`, and `toString()`. `"16".toInt()` converts text and throws an exception if the text is invalid, so for typed input prefer `toIntOrNull()` and `toDoubleOrNull()`, which return `null` instead.

`toInt()` on a `Double` **truncates** toward zero: `3.9.toInt()` is `3`. To round, use `kotlin.math.roundToInt()`, which rounds halves up: `3.5.roundToInt()` is `4`.

A `Double` is a binary approximation. `0.1 + 0.2` is `0.30000000000000004`, not `0.3`; this is correct behavior. Compare decimals with a tolerance and format them for display.

The shared student has age 16 and height 1.72 meters; the example converts text to those values and computes the height in centimeters.

Worked Example

```
import kotlin.math.roundToInt

fun main() {
    val age = "16".toInt()
    val height = "1.72".toDouble()
    println("$age ${age::class.simpleName}")
    println("$height ${height::class.simpleName}")

    println("abc".toIntOrNull())
    println("".toIntOrNull())
    println(42.toString() + " is now text")

    println(3.9.toInt())
    println(3.5.roundToInt())
    println(0.1 + 0.2)

    val heightCm = height * 100
    println("Height in centimeters: " + "%.1f".format(heightCm))
}
```

Expected Behavior or Output

The program prints:

```
16 Int
1.72 Double
null
null
42 is now text
3
```

```
4
0.30000000000000004
Height in centimeters: 172.0
```

Common Mistakes

1. Using `toInt()` on untrusted text and crashing with `NumberFormatException`.
2. Expecting `toInt()` to round. It truncates.
3. Comparing decimals with `==` after arithmetic.

Activities

1. Predict each of the ten lines before running, then compare.
2. Check what `" 16 ".toIntOrNull()` and `"1e3".toDoubleOrNull()` return and explain each.
3. Write code that reads a line and prints a clear message when the conversion fails.
4. Explain the difference between `3.5.toInt()` and `3.5.roundToInt()`.

Accessibility Practice

Format floating-point results so that the screen reader reads short numbers. Read the type names aloud and note that `Int` and `Double` are announced as words, not symbols.

Skills Checklist

- ☐ I can convert between text and numbers.
- ☐ I know `toInt()` truncates.
- ☐ I can use `roundToInt()`.
- ☐ I can use `toIntOrNull()`.
- ☐ I can explain why decimals are approximate.

Lesson 2.5: Using the Math Library

Reading References

- Basic types
- Kotlin standard library: kotlin.math
- Matching Python lesson: Lesson 2.5: Using the Math Library

Learning Objectives

- Use `PI`, `sqrt`, `pow`, and `hypot` from `kotlin.math`.
- Compute circle area and circumference.
- Compute the distance from the origin.

Concept Explanation

The `kotlin.math` package supplies `PI`, `sqrt`, `abs`, `max`, `min`, `floor`, `ceil`, `hypot`, and the `pow` extension function. Import what you use: `import kotlin.math.PI`, or `import kotlin.math.*` for everything.

For a circle of radius 5 the area is `PI * radius.pow(2)` (about 78.54) and the circumference is `2 * PI * radius` (about 31.42). Kotlin has a built-in hypotenuse function, so the distance from the origin to the shared point (3, 4) is `hypot(3.0, 4.0)`, which is 5.0.

The math functions take `Double` values. `sqrt(81.0)` returns `9.0`, a `Double`, so it prints with a decimal point.

Worked Example

```
import kotlin.math.PI
import kotlin.math.abs
import kotlin.math.hypot
import kotlin.math.max
import kotlin.math.min
import kotlin.math.pow
import kotlin.math.sqrt

fun main() {
    val radius = 5.0
    val area = PI * radius.pow(2)
    val circumference = 2 * PI * radius
    println("Area: " + "%.2f".format(area))
    println("Circumference: " + "%.2f".format(circumference))

    println(sqrt(81.0))
    println("Distance from origin: " + "%.2f".format(hypot(3.0, 4.0)))
    println("${max(82, 91)} ${min(76, 88)} ${abs(-5)}")
}
```

Expected Behavior or Output

The program prints:

```
Area: 78.54
Circumference: 31.42
9.0
Distance from origin: 5.00
91 76 5
```

Common Mistakes

1. Forgetting the import, so `sqrt` is an unresolved reference.

2. Passing an `Int` to `sqrt`, which needs a `Double`.
3. Writing the area as `PI * radius * 2`; the radius must be squared.

Activities

1. Predict each printed line and then run the program.
2. Compute the area for radius 2.5 by hand and with the program.
3. Change the point to (6, 8) and predict the distance before running.
4. Print the larger of two shared scores using `max`.

Accessibility Practice

Round results with `format` so that the screen reader announces `78.54` rather than fifteen digits. Say 'dot' aloud in `kotlin.math` and `radius.pow`.

Skills Checklist

- ☐ I can import and use `kotlin.math`.
- ☐ I can compute circle area and circumference.
- ☐ I can compute distance with `hypot`.
- ☐ I can format results to two decimals.
- ☐ I know math functions take `Double`.

11.5 Unit 3: Control Flow — Decisions and Loops

Overview

Make decisions, classify the shared scores with the shared grade thresholds, and learn collections, loops, and strings.

Lesson 3.1: Boolean Expressions and if

Reading References

- Control flow
- Matching Python lesson: Lesson 3.1: Boolean Expressions and if Statements

Learning Objectives

- Write comparisons and combine them with `&&`, `||`, and `!`.
- Use `if` as a statement and as an expression.
- Test a range with `in`.

Concept Explanation

A *Boolean expression* is `true` or `false`. Comparisons use `==`, `!=`, `<`, `<=`, `>`, `>=`, and conditions combine with `&&` (and), `||` (or), and `!` (not). A condition must be a real `Boolean`: Kotlin never treats 0, an empty string, or `null` as false.

`if` can be a *statement* that chooses what to run, or an *expression* that produces a value: `val label = if (score >= 90) "high" else "not high"`. When `if` is used as an expression, `else` is required. `in 80..89` tests whether a value is inside a range, both ends included.

The shared score is 82. It passes (at least 60), it is in the 80 to 89 range, and it is not at least 90.

Worked Example

```
fun main() {
    val score = 82

    if (score >= 60) {
        println("Avery passed.")
    } else {
        println("Avery did not pass.")
    }

    println(score >= 80 && score < 90)
    println(score < 60 || score > 100)
    println(!(score >= 90))
    println(score in 80..89)

    val label = if (score >= 90) "high" else "not high"
    println(label)
}
```

Expected Behavior or Output

The program prints:

```
Avery passed.
true
false
true
true
not high
```

Common Mistakes

1. Writing `=` where `==` is meant, which does not compile as a condition.

2. Leaving out `else` when `if` is used as an expression.
3. Expecting a number or text to act as a condition. It must be a `Boolean`.

Activities

1. Change the score to 59 and predict which lines change.
2. Write a condition that is true only for scores from 70 to 79 inclusive, in two ways.
3. Rewrite the `label` line as an `if` statement.
4. Explain why `if (score)` is not allowed in Kotlin.

Accessibility Practice

Read each condition aloud with 'greater than or equal to', 'and', 'or', and 'not'. Check that your screen reader announces `&&` and `||` so you do not confuse them with single symbols.

Skills Checklist

☐ I can write comparisons and combine them.
☐ I can write `if` and `else`.
☐ I can use `if` as an expression.
☐ I can test a range with `in`.
☐ I know conditions must be `Boolean`.

Lesson 3.2: Chained Conditions with when

Reading References

- Control flow: when
- Matching Python lesson: Lesson 3.2: Chained Conditionals (elif, else)

Learning Objectives

- Use `when` to choose among several conditions.
- Classify the shared grade thresholds.
- Test boundary scores.

Concept Explanation

Kotlin has no `elif`. Use `else if` for a short chain, or `when`, which tests conditions from top to bottom and runs the first branch that is true. `when` without an argument takes a Boolean condition for each branch, and it is an *expression* when you use its value; then an `else` branch is required.

The shared grade thresholds are 90 or more for A, 80 or more for B, 70 or more for C, 60 or more for D, and anything lower for F. Because branches are tested in order, each later test needs only a lower bound.

Boundary scores sit exactly on or just below a threshold. The scores 90, 89, 80, 79, 70, 69, 60, and 59 check every boundary.

Worked Example

```
fun classify(score: Int): String = when {
    score >= 90 -> "A"
    score >= 80 -> "B"
    score >= 70 -> "C"
    score >= 60 -> "D"
    else -> "F"
}

fun main() {
    for (score in listOf(90, 89, 80, 79, 70, 69, 60, 59)) {
        println("$score ${classify(score)}")
    }
}
```

Expected Behavior or Output

The program prints:

```
90 A
89 B
80 B
79 C
70 C
69 D
60 D
59 F
```

Common Mistakes

1. Putting the branches in the wrong order, such as testing `score >= 60` first so every passing score is a D.
2. Using `>` instead of `>=`, so exactly 90 receives a B.
3. Forgetting the `else` branch in a `when` expression, which does not compile.

Activities

1. Trace the score 82 through each branch and say which one runs.
2. Add the scores 100 and 0 to the list and predict their grades.

3. Reorder the branches so `score >= 60` is first, run it, and explain the wrong output.
4. Write `isPassing(score)` that returns a `Boolean` using `when`.

Accessibility Practice

Read the eight output lines in order. Each is a number, a space, and a letter, so they are easy to follow by ear and in braille.

Skills Checklist

- ☐ I can write a `when` chain.
- ☐ I know branch order matters.
- ☐ I can classify with the shared thresholds.
- ☐ I can choose boundary tests.
- ☐ I know a `when` expression needs `else`.

Lesson 3.3: Lists

Reading References

- Collections overview
- List-specific operations
- Matching Python lesson: Lesson 3.3: Introduction to Lists

Learning Objectives

- Create read-only and mutable lists.
- Index from 0 and use `size`, `last()`, and `getOrNull`.
- Explain the difference between a read-only view and a copy.

Concept Explanation

`listOf(...)` creates a **read-only** list and `mutableListOf(...)` creates a list you can change. Indexes start at 0, so `scores[0]` is the first item, `scores.last()` is the last, and `scores.size` is the length. Reading outside the list with `[]` throws an `IndexOutOfBoundsException`; `getOrNull(index)` returns `null` instead.

Read-only is a property of the **type**, not of the data. A `List<Int>` variable cannot change the list, but if it refers to the same list as a `MutableList<Int>`, the other reference can still change it. To protect a list, make a copy with `toList()`.

The shared list is 82, 91, 76, 88. Python uses the same 0-based indexes.

Worked Example

```
fun main() {
    val scores = mutableListOf(82, 91, 76, 88)
    println(scores[0])
    println(scores.last())
    println(scores.size)

    scores[2] = 80
    println(scores)

    val readOnlyView: List<Int> = scores
    val copy = scores.toList()
    scores[0] = 100
    println("view: ${readOnlyView[0]}, copy: ${copy[0]}")
    println(scores.getOrNull(10))
}
```

Expected Behavior or Output

The program prints:

```
82
88
4
[82, 91, 80, 88]
view: 100, copy: 82
null
```

Common Mistakes

1. Using index 1 for the first item. The first item is at index 0.
2. Using `[10]` on a short list and crashing instead of using `getOrNull`.
3. Assuming a `List` variable guarantees the data cannot change.

Activities

1. Predict each of the seven outputs before running.
2. Replace the second score with 95 and print the list.
3. Make a copy of the list and show that changing the original does not change the copy.
4. Explain, in two sentences, why `List` is called read-only rather than immutable.

Accessibility Practice

Print a whole list on one line and its size on another, so the screen reader announces a complete state in one pass. When reading `scores[0]` aloud, say

'scores, open bracket, zero, close bracket'.

Skills Checklist

- ☐ I can create `List` and `MutableList` values.
- ☐ I can index from 0.
- ☐ I can use `getOrNull`.
- ☐ I can make a copy with `toList()`.
- ☐ I can explain read-only versus immutable.

Lesson 3.4: Loops

Reading References

- Control flow: for loops
- Matching Python lesson: Lesson 3.4: Iterating with for Loops

Learning Objectives

- Use `for` with ranges, `until`, `downTo`, and `step`.
- Visit a list by value and with its index.
- Compute the total and average of the shared scores.

Concept Explanation

`for (x in collection)` visits every element. A range `1..3` includes both ends, `1 until 3` excludes the end, `10 downTo 1` counts down, and `step 3` skips values: `for (i in 10 downTo 1 step 3)`. `withIndex()` gives each element with its position: `for ((index, value) in scores.withIndex())`.

The standard library can do common work without a loop: `sum()` adds a list of numbers and `average()` returns the mean as a `Double`. A loop is still the way to learn what those functions do.

The shared scores 82, 91, 76, 88 total 337 and average 84.25.

Worked Example

```
fun main() {
    val scores = listOf(82, 91, 76, 88)
    var total = 0
    for ((index, score) in scores.withIndex()) {
        println("Adding score ${index + 1}: $score")
        total += score
    }
    println("Total: $total")
    println("Average: ${total.toDouble() / scores.size}")
```

```
    println("Same results: ${scores.sum()} and ${scores.average()}")

    for (i in 1..3) println("Count: $i")
    for (i in 1 until 3) println("Until: $i")
    for (i in 10 downTo 1 step 3) println("Step: $i")
}
```

Expected Behavior or Output

The program prints:

```
Adding score 1: 82
Adding score 2: 91
Adding score 3: 76
Adding score 4: 88
Total: 337
Average: 84.25
Same results: 337 and 84.25
Count: 1
Count: 2
Count: 3
Until: 1
Until: 2
Step: 10
Step: 7
Step: 4
Step: 1
```

Common Mistakes

1. Using `1..scores.size` as indexes, which reads one position past the end of a 0-based list.
2. Forgetting that `..` includes the end while `until` excludes it.
3. Dividing the `Int` total by the size and getting a truncated result.

Activities

1. Predict the last seven output lines before running.
2. Change the loop to print only the scores above 80.
3. Write a loop that counts from 20 down to 0 by 5.
4. Explain why `scores.average()` prints `84.25` and not `84`.

Accessibility Practice

Each output line starts with a label and a number, so the running total is read step by step. Count the lines to confirm that the loop ran four times.

Skills Checklist

- ☐ I can use ranges, `until`, `downTo`, and `step`.
- ☐ I can visit a list with its index.
- ☐ I can total and average a list.
- ☐ I know `..` includes its end.
- ☐ I can use `sum()` and `average()`.

Lesson 3.5: Strings

Reading References

- Strings
- Basic types
- Matching Python lesson: Lesson 3.5: Basic String Operations

Learning Objectives

- Use `length`, `lowercase`, `contains`, `indexOf`, and `trim`.
- Normalize a name and search the shared text sample.
- Explain that strings are immutable.

Concept Explanation

A Kotlin `String` is an immutable sequence of characters. `length` is its size, and `text[0]` is the first character. Functions such as `lowercase()` and `uppercase()` return a **new** string; they never change the original.

Useful functions: `trim()`, `contains(text)`, `indexOf(text)` (which returns `-1` when the text is missing), `replace`, `startsWith`, and `replaceFirstChar { it.uppercase() }` to capitalize. `Regex("programming").findAll(text).count()` counts non-overlapping matches.

The shared text sample is "A student studies programming. Programming develops problem-solving skills." It has 75 characters and contains `programming` twice (once in each case), found at index 18 after lowercasing.

Worked Example

```
fun main() {
    val rawName = "  avery  "
    val cleanName = rawName.trim().replaceFirstChar { it.uppercase() }
    println("Name: '$cleanName'")

    val text = "A student studies programming. Programming develops problem-solving skills."
    val lowered = text.lowercase()
    println("Characters: ${text.length}")
```

```
    println("Contains 'programming': ${lowered.contains("programming")}")
    println("First index of 'programming': ${lowered.indexOf("programming")}")
    println("Times 'programming' appears: ${Regex("programming").findAll(lowered).count()}")
    println("Missing word index: ${lowered.indexOf("kotlin")}")
    println(text.uppercase().take(9))
}
```

Expected Behavior or Output

The program prints:

```
Name: 'Avery'
Characters: 75
Contains 'programming': true
First index of 'programming': 18
Times 'programming' appears: 2
Missing word index: -1
A STUDENT
```

Common Mistakes

1. Expecting `text.uppercase()` to change `text`. Assign or print the result.
2. Treating `indexOf` as an error when it returns `-1`.
3. Counting positions from 1. The first character is at index 0.

Activities

1. Predict the printed values, then run and compare.
2. Find the index of the word `student` in the sample text.
3. Replace every `programming` with `coding` and print the result.
4. Write code that trims and capitalizes `"  riley  "`.

Accessibility Practice

Strings with many quotation marks are hard to read by ear. Say 'single quote' and 'double quote' aloud when checking a line, and print results inside labelled sentences so values are not run together.

Skills Checklist

- ☐ I can use common `String` functions.
- ☐ I can search text and find positions.
- ☐ I can trim and capitalize a name.
- ☐ I know strings are immutable.
- ☐ I can count matches with `Regex`.

11.6 Unit 4: Data Structures — Lists and Dictionaries

Overview

Work with lists, sorting, fixed records, dictionaries, and stable traversal of the shared inventory.

Lesson 4.1: More on Mutable Lists

Reading References

- Collections overview
- Adding and removing elements
- Matching Python lesson: Lesson 4.1: More on Lists

Learning Objectives

- Add, insert, replace, and remove list items.
- Remove by value and by position without confusing them.
- Search a list with `in` and `indexOf`.

Concept Explanation

A `MutableList` changes with `add(value)` (append), `add(index, value)` (insert and shift), `set` or `list[i] = value` (replace), `remove(value)` (remove the first matching **value**), and `removeAt(index)` (remove by **position**). For a list of `Int`, `remove(76)` removes the number 76, while `removeAt(76)` would try to remove position 76. `removeAt` returns the removed item.

`value in list` tests membership, and `indexOf(value)` returns the position or `-1`. These searches look at each item in turn, so they take longer for longer lists.

Avoid `removeLast()` in code that must run on several JDK versions: newer JDKs add a method with the same name to `List`, and the behavior depends on the JDK. `removeAt(lastIndex)` is portable.

The example applies the same operations as the Python course to the shared scores, from [82, 91, 76, 88] to [82, 85, 93, 88].

Worked Example

```
fun main() {
    val scores = mutableListOf(82, 91, 76, 88)

    scores.add(95)
    scores.add(1, 85)
```

```
    scores[2] = 93
    scores.remove(76)
    val last = scores.removeAt(scores.lastIndex)

    println(scores)
    println("${scores.size} items; removed $last")
    println(88 in scores)
    println(scores.indexOf(88))
    println(scores.indexOf(100))
}
```

Expected Behavior or Output

The program prints:

```
[82, 85, 93, 88]
4 items; removed 95
true
3
-1
```

Common Mistakes

1. Confusing `remove(76)` (by value) with `removeAt(76)` (by position).
2. Removing items inside a `for` loop over the same list, which throws `ConcurrentModificationException`.
3. Calling `removeLast()` and getting different behavior on different JDKs.

Activities

1. Trace the list after every operation and write each version as a sentence.
2. Remove the first score with `removeAt(0)` and describe what happens to the others.
3. Remove every score below 80 safely, using `removeAll { ... }`.
4. Print the position of the first score above 90 with `indexOfFirst`.

Accessibility Practice

After every change, print the whole list on one line so the screen reader reads a complete state in one pass. Say the method names aloud, and notice how `remove` and `removeAt` sound different.

Skills Checklist

- ☐ I can add, insert, replace, and remove items.
- ☐ I can tell `remove` from `removeAt`.
- ☐ I can search with `in` and `indexOf`.
- ☐ I know why `removeLast()` is avoided.

☐ I can show a list in one line.

Lesson 4.2: Sorting and Organizing Lists

Reading References

- Ordering
- Collections overview
- Matching Python lesson: Lesson 4.2: Organizing Lists

Learning Objectives

- Sort a list ascending and descending.
- Tell `sorted()` (a new list) from `sort()` (in place).
- Sort by a computed key.

Concept Explanation

Kotlin offers two families. **Returning** functions leave the original alone and give a new list: `sorted()`, `sortedDescending()`, `sortedBy { ... }`, and `reversed()`. **In-place** functions change a `MutableList`: `sort()`, `sortDescending()`, and `reverse()`. A read-only `List` has only the returning kind.

`sortedBy { it.length }` sorts by a computed key, here the length of each word, and keeps the original order of items with equal keys because the sort is *stable*.

The shared scores sort to 76, 82, 88, 91 in ascending order and 91, 88, 82, 76 in descending order.

Worked Example

```
fun main() {
    val scores = mutableListOf(82, 91, 76, 88)

    println("Original: $scores")
    println("Ascending: ${scores.sorted()}")
    println("Descending: ${scores.sortedDescending()}")
    println("Still original: $scores")

    scores.sort()
    println("Sorted in place: $scores")
    scores.reverse()
    println("Reversed in place: $scores")

    val words = listOf("pencil", "folder", "notebook")
```

```
    println(words.sorted())
    println(words.sortedBy { it.length })
}
```

Expected Behavior or Output

The program prints:

```
Original: [82, 91, 76, 88]
Ascending: [76, 82, 88, 91]
Descending: [91, 88, 82, 76]
Still original: [82, 91, 76, 88]
Sorted in place: [76, 82, 88, 91]
Reversed in place: [91, 88, 82, 76]
[folder, notebook, pencil]
[pencil, folder, notebook]
```

Common Mistakes

1. Expecting `sorted()` to change the list. It returns a new list.
2. Calling `sort()` on a read-only `List`, which does not compile.
3. Expecting `reverse()` to sort. It only flips the current order.

Activities

1. Predict each printed line before running.
2. Print the scores in descending order without changing the original.
3. Sort the words by their last letter with `sortedBy`.
4. Explain the difference between `sorted()` and `sort()` in two sentences.

Accessibility Practice

Print each result on its own labelled line (`Ascending:`, `Descending:`). The first and last numbers are the quickest to check by ear.

Skills Checklist

- ☐ I can sort ascending and descending.
- ☐ I know which functions change the list.
- ☐ I can use `sortedBy`.
- ☐ I can reverse a list.
- ☐ I can keep the original order.

Lesson 4.3: Pairs, Triples, and Data Classes

Reading References

- Data classes
- Destructuring declarations
- Matching Python lesson: Lesson 4.3: Tuples - Immutable Lists

Learning Objectives

- Use `Pair` and `Triple` for small fixed groups.
- Declare a data class with `val` properties.
- Unpack values with destructuring.

Concept Explanation

Kotlin has no general tuple type. For two or three values, use `Pair(a, b)` and `Triple(a, b, c)`, whose parts are `first`, `second`, and `third`. For anything you will use more than briefly, a **data class** with named properties is clearer: `data class Point(val x: Int, val y: Int)`.

A data class automatically provides `toString()`, `equals()` (so two points with the same values are equal), `copy()` (a new object with some values changed), and *destructuring*: `val (x, y) = point`. With `val` properties, the object cannot change after it is created, so use `copy(y = 10)` to get a changed version.

The shared coordinates are x = 3 and y = 4.

Worked Example

```kotlin
data class Point(val x: Int, val y: Int)

fun minMax(values: List<Int>): Pair<Int, Int> = Pair(values.min(), values.max())

fun main() {
    val (x, y) = Pair(3, 4)
    println("x=$x y=$y")

    val point = Point(3, 4)
    println(point)
    val moved = point.copy(y = 10)
    println("$point -> $moved")
    println(point == Point(3, 4))

    val (low, high) = minMax(listOf(82, 91, 76, 88))
    println("low=$low high=$high")

    val profile = Triple("Avery", 16, 1.72)
```

```
    println("${profile.first} ${profile.second} ${profile.third}")
}
```

Expected Behavior or Output

The program prints:

```
x=3 y=4
Point(x=3, y=4)
Point(x=3, y=4) -> Point(x=3, y=10)
true
low=76 high=91
Avery 16 1.72
```

Common Mistakes

1. Using `var` properties in a data class and then being surprised when values change.
2. Expecting `copy()` to change the original. It returns a new object.
3. Using `Pair` with unclear meanings (`first`, `second`) in code that others must read.

Activities

1. Predict the output, then run.
2. Create a `Student` data class for the shared profile and print it.
3. Use `copy` to make a student one year older and print both.
4. Explain when a data class is a better choice than a `Pair`.

Accessibility Practice

Named properties read better aloud than `first` and `second`. Printing a data class shows its name and every property, so the screen reader announces `Point(x=3, y=4)` as a complete sentence.

Skills Checklist

- ☐ I can use `Pair` and `Triple`.
- ☐ I can declare a data class.
- ☐ I can destructure values.
- ☐ I can use `copy()`.
- ☐ I can choose between a pair and a data class.

Lesson 4.4: Maps

Reading References

- Map-specific operations
- Collections overview
- Matching Python lesson: Lesson 4.4: Introducing Dictionaries

Learning Objectives

- Create a map and read, add, change, and remove entries.
- Handle a missing key with `?:`.
- Model the shared inventory item with a data class.

Concept Explanation

A `Map` pairs keys with values. `mapOf(...)` is read-only and `mutableMapOf(...)` can change. Read with `map[key]`, which returns a **nullable** value: it is `null` when the key is missing. Combine it with the Elvis operator to supply a default: `map[key] ?: "unknown"`.

`map[key] = value` adds or replaces an entry, `remove(key)` deletes one, `containsKey(key)` tests for a key, and `size` counts the entries. `mutableMapOf` keeps insertion order.

A map with values of different types, such as `MutableMap<String, Any>`, is flexible but loses type checking. When the fields are known, a data class is safer, so the shared Notebook item is a data class.

Worked Example

```
data class Item(val name: String, var quantity: Int, val unitPrice: Double)

fun main() {
    val profile = mutableMapOf<String, Any>(
        "name" to "Avery",
        "age" to 16,
        "heightM" to 1.72,
        "favoriteSubject" to "Computer Science",
    )
    println(profile["name"])
    profile["age"] = 17
    profile["gradeLevel"] = 11
    profile.remove("heightM")
    println(profile)
    println(profile["heightM"])
    println(profile["heightM"] ?: "unknown")
    println(profile.containsKey("gradeLevel"))
```

```
    val item = Item("Notebook", 3, 2.50)
    item.quantity += 2
    println(item)
    println("Stock value: ${item.quantity * item.unitPrice}")
}
```

Expected Behavior or Output

The program prints:

```
Avery
{name=Avery, age=17, favoriteSubject=Computer Science, gradeLevel=11}
null
unknown
true
Item(name=Notebook, quantity=5, unitPrice=2.5)
Stock value: 12.5
```

Common Mistakes

1. Using a missing key and expecting an error. The result is `null`.
2. Using `!!` on a map lookup, which crashes when the key is missing.
3. Storing mixed types in `Map<String, Any>` and then needing casts everywhere.

Activities

1. Predict the printed lines before running.
2. Add a `discount` key to the item as a data class property and print the discounted value.
3. Remove `favoriteSubject` and show what the lookup returns.
4. Explain why a data class is safer than `Map<String, Any>` for a record.

Accessibility Practice

Print one entry per line when you need to read a map, because a one-line `{a=1, b=2}` is hard to navigate. Choose key names that read well aloud, such as `unitPrice`.

Skills Checklist

- ☐ I can create and change a map.
- ☐ I know a lookup returns a nullable value.
- ☐ I can supply a default with `?:`.
- ☐ I can model an item with a data class.
- ☐ I can test for a key.

Lesson 4.5: Traversing Maps

Reading References

- Map-specific operations
- Collections overview
- Matching Python lesson: Lesson 4.5: Looping Through Dictionaries

Learning Objectives

- Visit keys, values, and key-value pairs.
- Explain which maps keep insertion order.
- Produce sorted output and total the shared inventory.

Concept Explanation

`for (key in map.keys)` visits keys, `for (value in map.values)` visits values, and `for ((key, value) in map)` visits pairs with destructuring.

Order matters for readable output. `mutableMapOf`, `linkedMapOf`, and `mapOf` keep **insertion order**, so output is stable. `hashMapOf` makes no promise about order. To list keys alphabetically, use `toSortedMap()`.

`sumOf { ... }` adds a computed number for every element, which is the compact way to total the shared inventory: Notebook (3 at 2.50), Pencil (10 at 0.50), and Folder (4 at 1.25) are worth $7.50 + 5.00 + 5.00 = 17.50$.

Worked Example

```
data class Stock(val quantity: Int, val unitPrice: Double)

fun main() {
    val inventory = linkedMapOf(
        "Notebook" to Stock(3, 2.50),
        "Pencil" to Stock(10, 0.50),
        "Folder" to Stock(4, 1.25),
    )

    println("Keys:")
    for (name in inventory.keys) println(name)

    println("Quantities:")
    for (stock in inventory.values) println(stock.quantity)

    var totalValue = 0.0
    for ((name, stock) in inventory) {
```

```
        val lineValue = stock.quantity * stock.unitPrice
        println("$name: ${stock.quantity} in stock, value ${"%.2f".format(lineValue)}")
        totalValue += lineValue
    }
    println("Total inventory value: ${"%.2f".format(totalValue)}")

    println(inventory.toSortedMap().keys)
    println(inventory.values.sumOf { it.quantity * it.unitPrice })
}
```

Expected Behavior or Output

The program prints:

```
Keys:
Notebook
Pencil
Folder
Quantities:
3
10
4
Notebook: 3 in stock, value 7.50
Pencil: 10 in stock, value 5.00
Folder: 4 in stock, value 5.00
Total inventory value: 17.50
[Folder, Notebook, Pencil]
17.5
```

Common Mistakes

1. Using a `HashMap` and expecting the same order every run.
2. Adding or removing entries while looping over the map, which throws `ConcurrentModificationException`.
3. Forgetting the parentheses in `for ((name, stock) in inventory)` and iterating over entries without unpacking them.

Activities

1. Run the example and write the three key-value lines in order.
2. Print only the items with a quantity below 5.
3. Print the items from the highest line value to the lowest.
4. Explain how `toSortedMap()` differs from the default order.

Accessibility Practice

Printing each inventory line as one sentence lets the screen reader announce the name, quantity, and value together. Because the order is stable, you can learn it and compare runs by ear.

Skills Checklist

- ☐ I can visit keys, values, and pairs.
- ☐ I know which maps keep insertion order.
- ☐ I can sort keys.
- ☐ I can total with `sumOf`.
- ☐ I can total the shared inventory.

11.7 Unit 5: User Input and while Loops

Overview

Read and validate input, write sentinel and flag loops, skip entries, and prevent endless loops.

Lesson 5.1: How Console Input Works

Reading References

- Basic syntax
- Null safety
- Matching Python lesson: Lesson 5.1: How the input() Function Works

Learning Objectives

- Read a line and recognize it as text.
- Convert it safely and handle failure.
- Handle the end of input.

Concept Explanation

`readlnOrNull()` returns the next line as a `String?`, or `null` when the input has ended. It never returns a number: typed digits are text until you convert them. `toIntOrNull()` returns an `Int?`, and a `null` result means the text was not a whole number.

A program that reads input must plan for three cases: a valid value, an invalid value, and no value at all. The example reads an age and then a second value,

so you can see the valid, invalid, and missing cases. With the typed value 16 the age five years later is 21; the second value, `abc`, is rejected.

Worked Example

```
fun main() {
    println("How old are you?")
    val text = readlnOrNull()
    println("Read: ${if (text == null) "nothing" else text::class.simpleName}")

    val age = text?.trim()?.toIntOrNull()
    if (age != null) {
        println("In 5 years: ${age + 5}")
    } else {
        println("That was not a whole number.")
    }

    println("Enter another value:")
    val second = readlnOrNull()?.trim()?.toIntOrNull()
    if (second != null) {
        println("Number: $second")
    } else {
        println("Not a number; toIntOrNull returned null.")
    }
}
```

Expected Behavior or Output

With the following lines typed as standard input (piped input is not echoed, so it does not appear in the output):

```
16
abc
```

the program prints:

```
How old are you?
Read: String
In 5 years: 21
Enter another value:
Not a number; toIntOrNull returned null.
```

With no standard input at all, the program ends cleanly and prints:

```
How old are you?
Read: nothing
That was not a whole number.
Enter another value:
Not a number; toIntOrNull returned null.
```

Common Mistakes

1. Adding to the text without converting it, which does not compile or gives joined text.
2. Using `toInt()` on user input and crashing on `abc`.
3. Forgetting that an empty answer is not a number, so `toIntOrNull()` is `null`.

Activities

1. Run with 16 and then `abc` and compare with the shown output.
2. Run with no input and explain every line printed.
3. Accept an age only from 0 to 120.
4. Rewrite the program using `readln()` and describe what happens at the end of input.

Accessibility Practice

Each prompt is a complete sentence on its own line. Run the program twice, once typing and once with piped input, and compare what your screen reader announces.

Skills Checklist

- ☐ I can read a line of input.
- ☐ I can convert and validate it.
- ☐ I handle `null` at the end of input.
- ☐ I know typed input is text.
- ☐ I can write a clear prompt.

Lesson 5.2: The while Loop in Action

Reading References

- Control flow: while loops
- Matching Python lesson: Lesson 5.2: The while Loop in Action

Learning Objectives

- Write a counting `while` loop.
- Write a sentinel loop that ends on a command.
- Stop at the end of input.

Concept Explanation

`while (condition) { ... }` repeats while the condition is true, testing **before** each pass. A loop needs a way to make its condition false, normally by

changing a variable inside the body: `count++` adds one.

A *sentinel* is a special input that tells the loop to stop. Here it is the word `quit`. The standard pattern reads once before the loop and again at the end of each pass, and tests for `null` as well, so the loop ends when input runs out even without `quit`.

The example counts from 1 to 5, then repeats a message prompt until the user types `quit`.

Worked Example

```
fun main() {
    var count = 1
    while (count <= 5) {
        println(count)
        count++
    }
    println("Counting finished!")

    println("Enter a message, or type quit to exit:")
    var message = readlnOrNull()
    while (message != null && message != "quit") {
        println("You said: $message")
        println("Enter a message, or type quit to exit:")
        message = readlnOrNull()
    }
    println("Goodbye!")
}
```

Expected Behavior or Output

With the following lines typed as standard input (piped input is not echoed, so it does not appear in the output):

```
hello
scores
quit
```

the program prints:

```
1
2
3
4
5
Counting finished!
Enter a message, or type quit to exit:
You said: hello
```

```
Enter a message, or type quit to exit:
You said: scores
Enter a message, or type quit to exit:
Goodbye!
```

With no standard input at all, the program ends cleanly and prints:

```
1
2
3
4
5
Counting finished!
Enter a message, or type quit to exit:
Goodbye!
```

Common Mistakes

1. Forgetting to change the loop variable, which creates an endless loop.
2. Testing for `quit` but not for `null`, so the loop reads forever after input ends.
3. Reading input only before the loop and never again inside it.

Activities

1. Predict the output for the typed lines hello, scores, and quit.
2. Change the sentinel to `exit` and update the prompt.
3. Count the messages entered and print the count at the end.
4. Rewrite the counting loop with `for (count in 1..5)` and compare.

Accessibility Practice

Each prompt is on its own line and each response begins with `You said:`, so the conversation can be followed in order by ear. If a loop floods the screen reader, stop it with Ctrl+C.

Skills Checklist

- ☐ I can write a counting `while` loop.
- ☐ I can write a sentinel loop.
- ☐ I test for `null` as well as the sentinel.
- ☐ I update the loop variable.
- ☐ I can compare `while` with `for`.

Lesson 5.3: Flags, break, and Labels

Reading References

- Returns and jumps
- Matching Python lesson: Lesson 5.3: Using Flags and break to Exit Loops

Learning Objectives

- Control a loop with a Boolean flag.
- Leave a loop with `break` and an outer loop with a label.
- Search a list and remember the position.

Concept Explanation

A *flag* is a Boolean variable that controls a loop: the loop runs while `running` is true, and the body sets it to `false` to stop. `break` leaves the nearest enclosing loop at once.

A search keeps its answer in a variable (here `-1` means "not found"), sets it when the item is found, and uses `break`. After the loop, the variable says whether a match was found. List positions start at 0.

A *label* lets `break` leave an outer loop from inside an inner one: write `outer@ for (...)` and `break@outer`.

The shared scores 82, 91, 76, 88 have their first score of 90 or higher at index 1, with the value 91.

Worked Example

```
fun main() {
    var running = true
    var handled = 0
    while (running) {
        println("Say something, or type quit to stop:")
        val message = readlnOrNull()
        if (message == null || message == "quit") {
            running = false
        } else {
            handled++
            println("You said: $message")
        }
    }
    println("Messages handled: $handled")

    val scores = listOf(82, 91, 76, 88)
    var foundIndex = -1
    for ((index, score) in scores.withIndex()) {
```

```
        if (score >= 90) {
            foundIndex = index
            break
        }
    }
    if (foundIndex >= 0) {
        println("First score of 90 or higher: ${scores[foundIndex]} at index $foundIndex")
    } else {
        println("No score of 90 or higher.")
    }

    outer@ for (row in 1..3) {
        for (col in 1..3) {
            if (row * col == 4) {
                println("Stopped at row $row, column $col")
                break@outer
            }
        }
    }
}
```

Expected Behavior or Output

With the following lines typed as standard input (piped input is not echoed, so it does not appear in the output):

```
hello
quit
ignored
```

the program prints:

```
Say something, or type quit to stop:
You said: hello
Say something, or type quit to stop:
Messages handled: 1
First score of 90 or higher: 91 at index 1
Stopped at row 2, column 2
```

With no standard input at all, the program ends cleanly and prints:

```
Say something, or type quit to stop:
Messages handled: 0
First score of 90 or higher: 91 at index 1
Stopped at row 2, column 2
```

Common Mistakes

1. Setting the flag but running more code in the same pass that should be skipped.
2. Forgetting to give the result variable a starting value, then misreading it after the loop.
3. Using `break` in an inner loop and expecting it to leave the outer loop.

Activities

1. Run with hello and quit and confirm the count is 1.
2. Change the search to the first score below 80 and predict the index.
3. Rewrite the first loop without a flag, using `break`.
4. Explain in two sentences when a flag is clearer than `break`.

Accessibility Practice

Report search results as complete sentences that include both the value and its position. Remember that Kotlin indexes start at 0, and say 'index 1' aloud to avoid confusing it with 'the first'.

Skills Checklist

- ☐ I can control a loop with a flag.
- ☐ I can use `break`.
- ☐ I can use a label to leave an outer loop.
- ☐ I can search a list and keep the position.
- ☐ I can explain flag versus `break`.

Lesson 5.4: Skipping Entries with continue

Reading References

- Returns and jumps
- Matching Python lesson: Lesson 5.4: Using continue in Loops

Learning Objectives

- Skip one pass of a loop with `continue`.
- Skip invalid scores in a list.
- Compare `continue` with `filter`.

Concept Explanation

`continue` skips the rest of the current pass and goes straight to the next one. It is useful for ignoring invalid entries while still reporting them. In a `while` loop,

make sure that the loop variable is updated *before* `continue`, or the loop never ends.

Kotlin also lets you filter first: `rawScores.filter { it in 0..100 }` builds a list of the valid scores, which is shorter when you do not need to report each skipped entry.

The example skips the invalid scores -5 and 105 in the list 82, -5, 91, 105, 76, 88. The four valid scores total 337.

Worked Example

```
fun main() {
    val rawScores = listOf(82, -5, 91, 105, 76, 88)
    var validTotal = 0
    var validCount = 0

    for (raw in rawScores) {
        if (raw < 0 || raw > 100) {
            println("Skipping invalid score: $raw")
            continue
        }
        validTotal += raw
        validCount++
    }
    println("Valid scores: $validCount, total: $validTotal")

    for (number in 1..9) {
        if (number % 2 == 0) continue
        println(number)
    }

    println(rawScores.filter { it in 0..100 }.sum())
}
```

Expected Behavior or Output

The program prints:

```
Skipping invalid score: -5
Skipping invalid score: 105
Valid scores: 4, total: 337
1
3
5
7
9
337
```

Common Mistakes

1. Putting statements that must always run after `continue`, so they are skipped.
2. Using `continue` in a `while` loop before updating the counter, which creates an endless loop.
3. Confusing `continue` (skip this pass) with `break` (leave the loop).

Activities

1. Predict the printed lines and the final total.
2. Skip scores below 80 instead.
3. Rewrite the first loop with an `if`/`else` and no `continue`.
4. Rewrite it with `filter` and explain what you no longer report.

Accessibility Practice

Print a line for each skipped entry, so the screen reader announces what was ignored. Never skip silently when a user needs to know what happened to their data.

Skills Checklist

☐ I can use `continue`.
☐ I can skip invalid entries.
☐ I know where the counter must be updated.
☐ I can use `filter` as an alternative.
☐ I can tell `continue` from `break`.

Lesson 5.5: Avoiding Infinite Loops

Reading References

- Control flow: do-while
- Matching Python lesson: Lesson 5.5: Avoiding Infinite Loops

Learning Objectives

- Identify what makes a loop stop.
- Add an attempt limit and handle the end of input.
- Use `do ... while`.

Concept Explanation

An *infinite loop* never ends because its condition never becomes false. Common causes are forgetting to change the loop variable, testing the wrong variable,

and waiting for input that never arrives. In a terminal, stop a running program with Ctrl+C.

Protect every loop with a *progress rule* and a *limit*: count attempts, and stop when the limit is reached or the input ends.

`do { ... } while (condition)` runs the body first and tests afterward, so it always runs at least once. A variable declared inside the body is visible in the `while` condition.

The example allows three attempts to enter a score from 0 to 100. With the typed lines abc, 105, and 88, the first two are rejected and the third is accepted.

Worked Example

```
fun main() {
    val maxAttempts = 3
    var attempts = 0
    var score: Int? = null

    while (attempts < maxAttempts && score == null) {
        attempts++
        println("Enter a score from 0 to 100 (attempt $attempts of $maxAttempts):")
        val text = readlnOrNull()
        if (text == null) {
            println("No more input.")
            break
        }
        val number = text.trim().toIntOrNull()
        if (number != null && number in 0..100) {
            score = number
        } else {
            println("Invalid score: $text")
        }
    }

    if (score != null) {
        println("Accepted score: $score")
    } else {
        println("No valid score was entered.")
    }

    var count = 1
    do {
        println(count)
        count++
    } while (count <= 3)
```

```
}
```

Expected Behavior or Output

With the following lines typed as standard input (piped input is not echoed, so it does not appear in the output):

```
abc
105
88
```

the program prints:

```
Enter a score from 0 to 100 (attempt 1 of 3):
Invalid score: abc
Enter a score from 0 to 100 (attempt 2 of 3):
Invalid score: 105
Enter a score from 0 to 100 (attempt 3 of 3):
Accepted score: 88
1
2
3
```

With no standard input at all, the program ends cleanly and prints:

```
Enter a score from 0 to 100 (attempt 1 of 3):
No more input.
No valid score was entered.
1
2
3
```

Common Mistakes

1. Forgetting to increase `attempts`, so the limit is never reached.
2. Not handling `null` from `readlnOrNull()`, so the loop never ends or crashes when input closes.
3. Writing `while (count < 3)` after a `do` body and stopping one pass early.

Activities

1. Run with abc, 105, and 88 and confirm the accepted score.
2. Run with three invalid lines and read the final message.
3. Change the limit to 5 and test it.
4. For each loop in the example, write down what makes it stop.

Accessibility Practice

A loop that cannot be stopped is an accessibility problem, because it can flood a screen reader with output. Learn Ctrl+C before running loops, give every input loop a limit, and write error messages that say what is valid.

Skills Checklist

- ☐ I can name what ends each loop.
- ☐ I can add an attempt limit.
- ☐ I handle the end of input.
- ☐ I can use `do ... while`.
- ☐ I know how to interrupt a program.

11.8 Unit 6: Functions

Overview

Define functions, pass arguments, return results, process collections in functions, and accept a variable number of arguments.

Lesson 6.1: Defining Functions

Reading References

- Functions
- Matching Python lesson: Lesson 6.1: Defining Functions

Learning Objectives

- Define and call functions.
- Pass parameters and return nothing or a value.
- Treat a function as a value.

Concept Explanation

A function is declared with `fun name(parameters): ReturnType { ... }`. Each parameter names its type: `fun showTip(bill: Double, tipRate: Double)`. A function with no result returns `Unit`, which you usually leave out. A function whose body is one expression can use `=`: `fun greetName(name: String) = println("Hello, $name!")`.

The *parameters* are the names in the definition and the *arguments* are the values you pass. Arguments are matched to parameters by position.

Functions are values. `::showTip` is a reference to the function, which can be stored in a variable and called like the function itself. Top-level functions are declared outside any class and can be called from anywhere in the same package.

Worked Example

```
fun greetUser() {
    println("Hello, Kotlin programmer!")
}

fun greetName(name: String) = println("Hello, $name!")

fun showTip(bill: Double, tipRate: Double) {
    val tip = bill * tipRate
    println("Tip: " + "%.2f".format(tip))
    println("Total: " + "%.2f".format(bill + tip))
}

fun main() {
    greetUser()
    greetUser()
    greetName("Avery")
    greetName("Riley")
    showTip(24.5, 0.15)

    val reference = ::showTip
    reference(100.0, 0.2)
}
```

Expected Behavior or Output

The program prints:

```
Hello, Kotlin programmer!
Hello, Kotlin programmer!
Hello, Avery!
Hello, Riley!
Tip: 3.68
Total: 28.18
Tip: 20.00
Total: 120.00
```

Common Mistakes

1. Writing `greetUser` without parentheses and expecting the function to run.
2. Passing arguments in the wrong order, such as `showTip(0.15, 24.5)`.
3. Omitting a parameter's type, which is a compile error.

Activities

1. Call `showTip(100.0, 0.2)` yourself and predict both lines.
2. Write `describeStudent(name: String, age: Int)` that prints one sentence.
3. Call `greetName()` with no argument and read the compiler message.
4. Explain the difference between a parameter and an argument.

Accessibility Practice

Choose function names that read as actions when spoken, such as `showTip`. Print complete sentences from a function so the screen reader announces each call's result separately.

Skills Checklist

- ☐ I can define and call a function.
- ☐ I can distinguish parameters from arguments.
- ☐ I can write an expression-bodied function.
- ☐ I can store a function in a variable.
- ☐ I can read a function signature aloud.

Lesson 6.2: Default and Named Arguments

Reading References

- Functions: default and named arguments
- Matching Python lesson: Lesson 6.2: Passing Arguments

Learning Objectives

- Give a parameter a default value.
- Call a function with named arguments.
- Mix positional and named arguments correctly.

Concept Explanation

Kotlin supports *default values*, written in the parameter list: `fun calculateTotal(bill: Double, tipRate: Double = 0.15)`. A caller may leave out an argument that has a default. It also supports *named arguments*: `calculateTotal(tipRate = 0.10, bill = 24.5)`. Named arguments make a call read like a sentence and let you give them in any order.

You may mix the two: positional arguments first, then named ones. With the shared bill of 24.50, the total at the default rate 0.15 is 28.18, at 0.20 it is 29.40, and at 0.10 it is 26.95.

Worked Example

```
fun calculateTotal(bill: Double, tipRate: Double = 0.15): Double = bill + bill * tipRate

fun main() {
    println("%.2f".format(calculateTotal(24.5)))
    println("%.2f".format(calculateTotal(24.5, 0.20)))
    println("%.2f".format(calculateTotal(tipRate = 0.10, bill = 24.5)))
    println("%.2f".format(calculateTotal(24.5, tipRate = 0.0)))
}
```

Expected Behavior or Output

The program prints:

```
28.18
29.40
26.95
24.50
```

Common Mistakes

1. Putting a positional argument after a named one in a way the compiler cannot match.
2. Changing a default value and forgetting that every caller that omitted it now gets the new value.
3. Using a very long list of positional parameters instead of named arguments.

Activities

1. Predict the four totals before running.
2. Add a `roundTotal: Boolean = false` parameter and use it.
3. Call the function with the arguments in a different order using names.
4. Explain why named arguments help a screen reader user.

Accessibility Practice

Named arguments make a call read as a sentence: 'calculate total, bill equals 24.5, tip rate equals 0.10'. Say the equals signs aloud and check that your screen reader announces them.

Skills Checklist

- ☐ I can write default values.
- ☐ I can call with named arguments.
- ☐ I can mix positional and named arguments.
- ☐ I know defaults apply to omitted arguments.
- ☐ I can read a named call aloud.

Lesson 6.3: Return Values

Reading References

- Functions
- Null safety
- Matching Python lesson: Lesson 6.3: Return Values

Learning Objectives

- Return a computed value.
- Return a nullable result when a value may be missing.
- Return several values with a data class.

Concept Explanation

A function sends a value back with `return`, or with `=` for a single expression. The return type follows the parameters: `fun average(scores: List<Int>): Double?`. A `?` after the type means the function may return `null`; that is the Kotlin way to say "no result", for example the average of an empty list.

The caller must deal with `null` before using the value, usually with a check or with `?:`. To return several values, return a `Pair` or a small data class and destructure it.

The example returns the average of the shared scores, classifies that average, returns a minimum and maximum together, and shows the empty-list case.

Worked Example

```kotlin
data class ScoreRange(val low: Int, val high: Int)

fun average(scores: List<Int>): Double? {
    if (scores.isEmpty()) return null
    return scores.sum().toDouble() / scores.size
}

fun classify(score: Double): String = when {
    score >= 90 -> "A"
    score >= 80 -> "B"
    score >= 70 -> "C"
    score >= 60 -> "D"
    else -> "F"
}

fun minMax(scores: List<Int>) = ScoreRange(scores.min(), scores.max())
```

```
fun main() {
    val scores = listOf(82, 91, 76, 88)
    val avg = average(scores)
    println("Average: $avg")
    if (avg != null) {
        println("Grade for the average: ${classify(avg)}")
    }
    val (low, high) = minMax(scores)
    println("Lowest and highest: $low and $high")
    println(average(emptyList()))
}
```

Expected Behavior or Output

The program prints:

```
Average: 84.25
Grade for the average: B
Lowest and highest: 76 and 91
null
```

Common Mistakes

1. Printing inside the function and then trying to use the result in a calculation.
2. Ignoring the `null` case and crashing with `!!`.
3. Writing code after a `return` in the same block. It never runs.

Activities

1. Predict the four printed lines before running.
2. Write `total(list: List<Int>)` that returns the sum.
3. Write `isPassing(score: Int)` that returns a `Boolean`.
4. Explain how a caller should use a function that returns `Double?`.

Accessibility Practice

Return values instead of printing inside logic functions, so the caller prints once and the screen reader announces each result a single time.

Skills Checklist

- ☐ I can return a value.
- ☐ I can return a nullable result.
- ☐ I can return several values with a data class.
- ☐ I can use a returned value in a calculation.
- ☐ I can classify an average with the shared thresholds.

Lesson 6.4: Functions with Collections

Reading References

- Collections overview
- Functions
- Matching Python lesson: Lesson 6.4: Functions with Lists and Dictionaries

Learning Objectives

- Explain that a collection argument refers to the same object.
- Change a list in a function and protect the original with a copy or a new list.
- Process the shared inventory with a function.

Concept Explanation

A `List` or `MutableList` passed to a function is a *reference* to the same object. A function that takes a `MutableList` can change the caller's list; a function that takes a read-only `List` cannot change it through that parameter. Numbers and strings are copied or immutable, so they cannot be changed this way.

To produce a changed list without touching the original, return a *new* list: `scoreList + newScore` builds one. Choose the parameter type to say what the function does: `MutableList<Int>` means "I will change this", `List<Int>` means "I will only read this".

The shared inventory is Notebook (3 at 2.50), Pencil (10 at 0.50), and Folder (4 at 1.25), worth 17.50 in total.

Worked Example

```
data class Item(val name: String, val quantity: Int, val unitPrice: Double)

fun addScore(scoreList: MutableList<Int>, newScore: Int) {
    scoreList.add(newScore)
}

fun withScore(scoreList: List<Int>, newScore: Int): List<Int> = scoreList + newScore

fun inventoryValue(inventory: List<Item>): Double =
    inventory.sumOf { it.quantity * it.unitPrice }

fun main() {
    val scores = mutableListOf(82, 91, 76, 88)
```

```
    addScore(scores, 95)
    println(scores)

    val longer = withScore(scores, 100)
    println("${scores.size} ${longer.size}")

    val inventory = listOf(
        Item("Notebook", 3, 2.50),
        Item("Pencil", 10, 0.50),
        Item("Folder", 4, 1.25),
    )
    println("Total inventory value: " + "%.2f".format(inventoryValue(inventory)))
}
```

Expected Behavior or Output

The program prints:

```
[82, 91, 76, 88, 95]
5 6
Total inventory value: 17.50
```

Common Mistakes

1. Expecting a function to receive its own copy of a list.
2. Changing a list inside a function that was meant only to read it.
3. Declaring a parameter `MutableList` when `List` is enough, which allows accidental changes.

Activities

1. Predict the three printed lines before running.
2. Write `scalePrices(inventory, factor)` that returns a new list with changed prices and leaves the original unchanged.
3. Write a version that changes a `MutableList<Item>` in place and compare the two designs.
4. Explain the difference between passing a number and passing a list.

Accessibility Practice

State aloud before each call whether the function changes its argument, and name it accordingly (`addScore` changes, `withScore` returns a new list). Print the list before and after so that the effect is heard.

Skills Checklist

- ☐ I know collections are passed by reference.
- ☐ I can change a list in a function.

- ☐ I can return a new list instead.
- ☐ I can choose `List` or `MutableList` for a parameter.
- ☐ I can total the shared inventory in a function.

Lesson 6.5: Variable Numbers of Arguments

Reading References

- Functions: variable number of arguments
- Matching Python lesson: Lesson 6.5: Arbitrary Number of Arguments

Learning Objectives

- Write a function with `vararg`.
- Pass an existing array with the spread operator `*`.
- Handle the case of no arguments.

Concept Explanation

A parameter marked `vararg` accepts any number of arguments, and inside the function it is an array: `vararg scores: Int` is an `IntArray`. Pass an existing array with the *spread operator*: `average(*list.toIntArray())`. A list must be converted to an array first.

Handle the case of zero arguments explicitly. Dividing a sum by a size of 0 would give a meaningless result, so the example returns `null`.

The shared scores 82, 91, 76, 88 average 84.25, and the average of 90 and 80 is 85.0.

Worked Example

```
fun average(vararg scores: Int): Double? =
    if (scores.isEmpty()) null else scores.sum().toDouble() / scores.size

fun describe(label: String, vararg values: Int): String =
    "$label: ${values.joinToString(", ")}"

fun main() {
    println(average(82, 91, 76, 88))
    println(average(90, 80))
    println(average())

    val list = listOf(82, 91, 76, 88)
    println(average(*list.toIntArray()))

    println(describe("scores", 82, 91, 76, 88))
```

```
    println(describe("nothing"))
}
```

Expected Behavior or Output

The program prints:

```
84.25
85.0
null
84.25
scores: 82, 91, 76, 88
nothing:
```

Common Mistakes

1. Calling `average()` with no arguments and dividing by zero.
2. Passing a list directly to a `vararg` parameter instead of spreading an array.
3. Placing `vararg` before other parameters and then needing named arguments for the rest.

Activities

1. Predict each printed line.
2. Write `maximum(vararg numbers: Int)` that returns the largest or `null`.
3. Write `joinWords(separator: String, vararg words: String)`.
4. Explain what the `*` does in a call.

Accessibility Practice

Read variadic signatures aloud: 'fun average, open parenthesis, vararg scores colon Int, close parenthesis'. Check that your screen reader announces `vararg` and the star.

Skills Checklist

- ☐ I can write a `vararg` function.
- ☐ I can use the spread operator.
- ☐ I handle the no-argument case.
- ☐ I know `vararg` is an array inside the function.
- ☐ I can convert a list with `toIntArray()`.

11.9 Unit 7: Classes and Object-Oriented Programming

Overview

Model the shared inventory item as an object: constructors, methods, inheritance, and composition.

Lesson 7.1: Why Learn About Classes?

Reading References

- Classes
- Visibility modifiers
- Matching Python lesson: Lesson 7.1: Why Learn About Classes?

Learning Objectives

- Explain a class as data plus behavior.
- Hide a property's setter to protect it.
- Describe encapsulation.

Concept Explanation

A *class* describes what a kind of thing knows (its *properties*) and what it can do (its *functions*). An *object* is one actual thing made from a class. Bundling data with the functions that work on it is called **encapsulation**. The shared example is an inventory *item* with a name, a quantity, and a unit price.

Kotlin lets you protect a property with a *private setter*: `var quantity: Int = quantity` followed by `private set` on the next line means code outside the class can read `quantity` but cannot assign to it. Changes must go through a function such as `restock`, which can check them.

Kotlin does not have a `new` keyword: you create an object by calling the class like a function, `Item("Notebook", 3, 2.50)`.

A Notebook with quantity 3 and unit price 2.50 is worth 7.50. Restocking 2 raises that to 12.50, and a negative restock is refused.

Worked Example

```
class Item(val name: String, quantity: Int, val unitPrice: Double) {
    var quantity: Int = quantity
        private set

    fun restock(amount: Int): Boolean {
```

```
        if (amount < 0) return false
        quantity += amount
        return true
    }

    fun totalValue(): Double = quantity * unitPrice

    fun describe() {
        println("$name: $quantity at ${"%.2f".format(unitPrice)} each, value ${"%.2f".format
    }
}

fun main() {
    val notebook = Item("Notebook", 3, 2.50)
    notebook.describe()
    println(notebook.restock(2))
    notebook.describe()
    println(notebook.restock(-1))
    println(notebook.quantity)
    // notebook.quantity = 99   // would not compile: the setter is private
}
```

Expected Behavior or Output

The program prints:

```
Notebook: 3 at 2.50 each, value 7.50
true
Notebook: 5 at 2.50 each, value 12.50
false
5
```

Common Mistakes

1. Treating a class as one single thing. A class is a blueprint; each call such as `Item(...)` makes a separate object.
2. Making every property public and changeable, so outside code can put the object into an invalid state.
3. Looking for a `new` keyword, which Kotlin does not have.

Activities

1. Predict the five printed lines before running.
2. Add a function `isLowStock()` that returns true when the quantity is below 5.
3. Uncomment the assignment, read the compiler message, and restore the comment.

4. Explain what encapsulation means and why a private setter helps.

Accessibility Practice

Describe a class as a list read in order: the name, then each property, then each function. Reading a class aloud works best by saying 'class Item, open parenthesis, val name colon String...' and listening for the braces that show structure.

Skills Checklist

- ☐ I can explain data plus behavior.
- ☐ I can use a private setter.
- ☐ I can create an object without `new`.
- ☐ I can describe encapsulation.
- ☐ I can read a class definition aloud.

Lesson 7.2: Creating Classes and Objects

Reading References

- Classes
- Exceptions: require
- Matching Python lesson: Lesson 7.2: Creating and Using a Class

Learning Objectives

- Declare a class with a primary constructor.
- Validate in an `init` block with `require`.
- Create several objects with the same properties.

Concept Explanation

A class declares its constructor right after its name: `class Item(val name: String, val quantity: Int, val unitPrice: Double)`. Writing `val` or `var` in front of a constructor parameter makes it a property in one step.

An `init` block runs whenever an object is created, so it is the place to check the values. `require(condition) { message }` throws an `IllegalArgumentException` with the message when the condition is false. An object that cannot be created invalid never needs to be checked again.

The example creates the three shared inventory items. Each has the same properties with different values. A class is a blueprint and each object is one *instance*.

Worked Example

```
class Item(val name: String, val quantity: Int, val unitPrice: Double) {
    init {
        require(quantity >= 0) { "quantity must not be negative" }
        require(unitPrice >= 0.0) { "unit price must not be negative" }
    }
}

fun main() {
    val notebook = Item("Notebook", 3, 2.50)
    val pencil = Item("Pencil", 10, 0.50)
    val folder = Item("Folder", 4, 1.25)

    for (item in listOf(notebook, pencil, folder)) {
        println("${item.name}: ${item.quantity} at ${"%.2f".format(item.unitPrice)}")
    }

    val attempt = runCatching { Item("Broken", -1, 1.0) }
    println(attempt.exceptionOrNull()?.message)
}
```

Expected Behavior or Output

The program prints:

```
Notebook: 3 at 2.50
Pencil: 10 at 0.50
Folder: 4 at 1.25
quantity must not be negative
```

Common Mistakes

1. Forgetting `val` in the constructor, so the parameter is not a property and cannot be read later.
2. Validating in each caller instead of once in `init`.
3. Catching the `IllegalArgumentException` and ignoring it, so the program continues with no object.

Activities

1. Create a fourth item with your own values and print it.
2. Add a rule that the name must not be blank.
3. Create two items with the same name and different quantities and show that they are separate objects.
4. Explain what `init` does and when it runs.

Accessibility Practice

Say the constructor aloud in order and check that the closing parenthesis and brace match. Print each object on its own line with its name so the output can be matched to the objects by ear.

Skills Checklist

- ☐ I can write a primary constructor.
- ☐ I can validate with `init` and `require`.
- ☐ I can create several objects.
- ☐ I know what an instance is.
- ☐ I can read a class header aloud.

Lesson 7.3: Properties and Functions

Reading References

- Properties
- Classes
- Matching Python lesson: Lesson 7.3: Working with Attributes and Methods

Learning Objectives

- Define a computed property and member functions.
- Change properties directly and through functions.
- Validate in a function.

Concept Explanation

A *property* can be stored (`var quantity`) or *computed*: `val totalValue: Double get() = quantity * unitPrice` has no stored value and is calculated each time it is read. Callers use it like any property, without parentheses.

A *member function* works on the object: `item.restock(2)`. A function that validates its input protects the object from invalid data. Directly changing a public `var` skips that protection, so make a property `var` only when any value is acceptable.

The example adds a computed value, a validity check, a restock function, and a describe function to the shared Notebook.

Worked Example

```
class Item(val name: String, var quantity: Int, val unitPrice: Double) {
    val totalValue: Double
```

```
        get() = quantity * unitPrice

    fun isValid(): Boolean = name.isNotBlank() && quantity >= 0 && unitPrice >= 0.0

    fun restock(amount: Int) {
        if (amount < 0) {
            println("Cannot restock a negative amount.")
            return
        }
        quantity += amount
    }

    fun describe() {
        println("$name: $quantity at ${"%.2f".format(unitPrice)} each, value ${"%.2f".format
    }
}

fun main() {
    val notebook = Item("Notebook", 3, 2.50)
    notebook.describe()
    notebook.restock(2)
    notebook.describe()
    notebook.quantity = 4
    notebook.describe()
    notebook.restock(-1)
    println(notebook.isValid())
}
```

Expected Behavior or Output

The program prints:

```
Notebook: 3 at 2.50 each, value 7.50
Notebook: 5 at 2.50 each, value 12.50
Notebook: 4 at 2.50 each, value 10.00
Cannot restock a negative amount.
true
```

Common Mistakes

1. Writing `totalValue()` with parentheses for a property, or `isValid` without them for a function.
2. Changing a public `var` directly and skipping the validation that a function would perform.
3. Storing a value that can be computed, so it goes out of date when the inputs change.

Activities

1. Predict the output of all six calls before running.
2. Add a function `discounted(rate: Double): Double` that returns the value after a discount without changing the item.
3. Make `quantity` have a private setter and see which lines stop compiling.
4. Explain the difference between a stored and a computed property.

Accessibility Practice

Name functions after actions (`restock`, `describe`) so a call reads as a sentence. Print one line per call, so the screen reader announces the effect of each step in order.

Skills Checklist

- ☐ I can write a computed property.
- ☐ I can write member functions.
- ☐ I can validate in a function.
- ☐ I know when to use a private setter.
- ☐ I can tell a property from a function when reading.

Lesson 7.4: Inheritance

Reading References

- Inheritance
- Matching Python lesson: Lesson 7.4: Inheritance

Learning Objectives

- Declare an `open` class and a subclass.
- Override a function and call the parent's version.
- Explain polymorphism.

Concept Explanation

Kotlin classes are **final by default**: nothing can inherit from them. Mark a class `open` to allow subclasses, and mark each function that may be replaced `open` as well. A subclass names its parent after a colon and passes constructor values up: `class DiscountedItem(...) : Item(name, quantity, unitPrice)`. A function that replaces a parent's function is marked `override`, and `super.totalValue()` calls the parent's version.

Polymorphism means that the same function name behaves differently for different classes. Here `describe`, written once in `Item`, calls `totalValue()`, so a `DiscountedItem` prints its discounted value automatically.

The example discounts the shared Folder (4 at 1.25) by 20 percent, so its value is 4.00 instead of 5.00.

Worked Example

```
open class Item(val name: String, val quantity: Int, val unitPrice: Double) {
    open fun totalValue(): Double = quantity * unitPrice

    fun describe() {
        println("$name: $quantity at ${"%.2f".format(unitPrice)} each, value ${"%.2f".format
    }
}

class DiscountedItem(name: String, quantity: Int, unitPrice: Double, val discountRate: Doubl
    Item(name, quantity, unitPrice) {
    override fun totalValue(): Double = super.totalValue() * (1 - discountRate)
}

fun main() {
    val folder = DiscountedItem("Folder", 4, 1.25, 0.20)
    folder.describe()
    println(folder is Item)

    val items: List<Item> = listOf(Item("Notebook", 3, 2.50), folder)
    for (item in items) println("${item.name}: ${"%.2f".format(item.totalValue())}")
}
```

Expected Behavior or Output

The program prints:

```
Folder: 4 at 1.25 each, value 4.00
true
Notebook: 7.50
Folder: 4.00
```

Common Mistakes

1. Forgetting `open`, so the compiler refuses to let another class inherit or override.
2. Forgetting `override` on a function that replaces a parent's function, which is a compile error.
3. Using inheritance for a ”has a” relationship; use composition instead (Lesson 7.5).

Activities

1. Predict the output before running.
2. Add a subclass `PerishableItem` with an `expires` property and override `describe` by making it `open`.
3. Explain why `describe` prints the discounted value without being redefined.
4. List the order in which Kotlin looks for `totalValue` on a `DiscountedItem`.

Accessibility Practice

Describe inheritance as an ordered list: the subclass first, then its parent. Speaking the chain aloud ('DiscountedItem, then Item') is clearer than a diagram.

Skills Checklist

- ☐ I can declare an open class and a subclass.
- ☐ I can override a function.
- ☐ I can call `super`.
- ☐ I can explain polymorphism.
- ☐ I know classes are final by default.

Lesson 7.5: Composition

Reading References

- Classes
- Matching Python lesson: Lesson 7.5: Instances as Attributes (Composition)

Learning Objectives

- Explain composition (”has a”).
- Hold a list of objects inside another object.
- Build an inventory from the shared items.

Concept Explanation

Composition means that one object contains other objects. An inventory *has* items; it is not a kind of item. Composition is usually simpler and safer than inheritance when objects are related by containment.

The container keeps a private list of its parts and exposes functions that loop over the parts and call each part's own functions, so the container does not need to know how a part works. Keeping the list `private` forces other code to use `add` and `report`.

The example builds an `Inventory` with the three shared items and reports each quantity and the total value, 17.50.

Worked Example

```
class Item(val name: String, val quantity: Int, val unitPrice: Double) {
    fun totalValue(): Double = quantity * unitPrice
}

class Inventory {
    private val items = mutableListOf<Item>()

    fun add(item: Item) {
        items.add(item)
    }

    fun find(name: String): Item? = items.firstOrNull { it.name == name }

    fun totalValue(): Double = items.sumOf { it.totalValue() }

    fun report() {
        for (item in items) {
            println("${item.name}: ${item.quantity}")
        }
        println("Total value: " + "%.2f".format(totalValue()))
    }
}

fun main() {
    val inventory = Inventory()
    inventory.add(Item("Notebook", 3, 2.50))
    inventory.add(Item("Pencil", 10, 0.50))
    inventory.add(Item("Folder", 4, 1.25))
    inventory.report()
    println(inventory.find("Pencil")?.quantity)
    println(inventory.find("Eraser"))
}
```

Expected Behavior or Output

The program prints:

```
Notebook: 3
Pencil: 10
Folder: 4
Total value: 17.50
```

```
10
null
```

Common Mistakes

1. Using inheritance where a ”has a” relationship is meant.
2. Making the list public so other code changes it without going through `add`.
3. Letting the container read each item's properties directly instead of calling the item's functions.

Activities

1. Predict the output.
2. Add `remove(name)` to `Inventory` and test it.
3. Add a function that returns the names of items with a quantity below 5.
4. Explain ”is a” versus ”has a” with two examples.

Accessibility Practice

Report the inventory as one labelled line per item and a final total line, so the screen reader reads a complete summary in order.

Skills Checklist

☐ I can explain composition.
☐ I can hold a list of objects in an object.
☐ I can keep a list private.
☐ I can say ”is a” versus ”has a”.
☐ I can build the shared inventory.

11.10 Unit 8: Files and Exceptions

Overview

Read and write files, handle errors, release resources reliably, and store records as JSON.

Lesson 8.1: Reading from a File

Reading References

- Kotlin standard library: kotlin.io
- Exceptions
- Matching Python lesson: Lesson 8.1: Reading from a File

Learning Objectives

- Read a whole file and read it line by line.
- Check that a file exists before reading.
- Report a missing file clearly.

Concept Explanation

`java.io.File` represents a path. `readText()` returns the whole file as a string, `readLines()` returns a list of lines, and `forEachLine { ... }` processes one line at a time. A relative path such as `notes.txt` starts from the program's working directory, so a program run from another folder will not find it.

`File.exists()` tests whether a file is there. Reading a file that does not exist throws a `FileNotFoundException`, which Lesson 8.3 handles.

To keep the lesson self-contained, the example first writes the two-line `notes.txt` that every language track uses, then reads it three ways and checks a file that does not exist.

Worked Example

```
import java.io.File

fun main() {
    val notes = File("notes.txt")
    notes.writeText("Practice Kotlin.\nReview lists and functions.\n")

    println(notes.readText())

    for (line in notes.readLines()) {
        println(line)
    }
    println(notes.readLines().size)

    notes.forEachLine { println(it.uppercase()) }

    val missing = File("missing_notes.txt")
    if (!missing.exists()) {
        println("missing_notes.txt was not found.")
    }
}
```

Expected Behavior or Output

The program prints:

```
Practice Kotlin.
Review lists and functions.
```

```
Practice Kotlin.
Review lists and functions.
2
PRACTICE KOTLIN.
REVIEW LISTS AND FUNCTIONS.
missing_notes.txt was not found.
```

Common Mistakes

1. Reading a file that does not exist without checking or catching the error.
2. Running from a different folder, so the relative path points to the wrong place.
3. Forgetting that `readText()` keeps the final newline, so `println` adds a blank line.

Activities

1. Run the program and describe each block of output.
2. Add a third line to the file text and predict the new count.
3. Print each line with its line number.
4. Print a friendly message that includes the missing file's name.

Accessibility Practice

Read the output aloud in order. Make an error message a complete sentence that includes the file name so that it is clear without seeing the screen.

Skills Checklist

- ☐ I can read a file whole and by line.
- ☐ I can check that a file exists.
- ☐ I know relative paths use the working directory.
- ☐ I can report a missing file.
- ☐ I can write a file for the example.

Lesson 8.2: Writing to a File

Reading References

- Kotlin standard library: kotlin.io
- Matching Python lesson: Lesson 8.2: Writing to a File

Learning Objectives

- Write with `writeText` and append with `appendText`.

- Include line endings correctly.
- Read the file back to verify it.

Concept Explanation

`writeText(text)` creates the file or **replaces** its contents. `appendText(text)` adds to the end. Neither adds a line ending for you, so end each line with `\n`. `printWriter()` and `bufferedWriter()` write in pieces and should be closed with `use` (Lesson 8.4).

Reading the file back is the simplest way to confirm what was saved. The example writes the first notes line, appends the second, and reads the file back.

Worked Example

```
import java.io.File

fun main() {
    val file = File("notes.txt")
    file.writeText("Practice Kotlin.\n")
    file.appendText("Review lists and functions.\n")
    print(file.readText())
    println("Lines: ${file.readLines().size}")
}
```

Expected Behavior or Output

The program prints:

```
Practice Kotlin.
Review lists and functions.
Lines: 2
```

Common Mistakes

1. Using `writeText` when `appendText` was intended, which erases earlier content.
2. Forgetting `\n`, so all written lines run together.
3. Not reading the file back to check what was actually saved.

Activities

1. Predict the output, then run.
2. Replace the second call with `writeText` and explain the difference.
3. Append a third line with the current number of lines.
4. Write the shared scores, one per line, then read them back and total them.

Accessibility Practice

Always read a written file back in the terminal to confirm it, using line navigation, so you do not depend on assumptions about what was saved.

Skills Checklist

- ☐ I can write and append.
- ☐ I know `writeText` replaces.
- ☐ I can include line endings.
- ☐ I can verify by reading back.
- ☐ I can count lines in a file.

Lesson 8.3: Exceptions

Reading References

- Exceptions
- Matching Python lesson: Lesson 8.3: Exceptions - Handling Errors

Learning Objectives

- Catch specific exceptions with `try` and `catch`.
- Use `finally` and `runCatching`.
- Raise errors with `require` and `error`.

Concept Explanation

An *exception* is an error that stops the program unless it is caught. `try { ... } catch (e: SomeException) { ... }` runs the risky code and handles the failure. Catch the **specific** exception you expect (`FileNotFoundException`, `NumberFormatException`, `ArithmeticException`), never everything, so mistakes in your own code are not hidden. A `finally` block runs whether or not an exception happened.

Kotlin has no checked exceptions: the compiler does not force you to catch anything. `runCatching { ... }` runs a block and returns a `Result` that holds either the value or the exception, and `exceptionOrNull()?.message` reads the message.

`require(condition) { message }` rejects a bad argument and `error(message)` throws an `IllegalStateException`. Integer division by zero throws an `ArithmeticException`; division of `Double` values by zero gives infinity instead.

Worked Example

```
import java.io.File
import java.io.FileNotFoundException
```

```
fun main() {
    try {
        File("missing_notes.txt").readText()
    } catch (e: FileNotFoundException) {
        println("Error: missing_notes.txt was not found.")
    }

    try {
        "abc".toInt()
    } catch (e: NumberFormatException) {
        println("Error: 'abc' is not a valid score.")
    }

    val zero = 0
    try {
        println(10 / zero)
    } catch (e: ArithmeticException) {
        println("Error: ${e.message}")
    }
    println(10.0 / zero)

    val result = runCatching { require(false) { "custom failure" } }
    println(result.exceptionOrNull()?.message)

    try {
        error("raised by error()")
    } catch (e: IllegalStateException) {
        println(e.message)
    } finally {
        println("Cleanup runs either way.")
    }
}
```

Expected Behavior or Output

The program prints:

```
Error: missing_notes.txt was not found.
Error: 'abc' is not a valid score.
Error: / by zero
Infinity
custom failure
raised by error()
Cleanup runs either way.
```

Common Mistakes

1. Catching `Exception` for everything, which hides mistakes in your own code.
2. Using `toInt()` on untrusted text without a catch or `toIntOrNull()`.
3. Expecting `10.0 / 0` to throw. It gives `Infinity`.

Activities

1. Predict each printed line before running.
2. Wrap `readln().toInt()` so invalid input prints a clear message and asks again.
3. Catch `ArithmeticException` in a function that computes an average of an empty list.
4. Explain when to catch an exception and when to let it stop the program.

Accessibility Practice

Write error messages as complete sentences that name the problem and the next step, such as 'Enter a whole number from 0 to 100.' Read the exception's class name aloud when you see a stack trace, then jump to the first line that names your own file.

Skills Checklist

☐ I can catch specific exceptions.
☐ I can use `finally` and `runCatching`.
☐ I can raise errors with `require` and `error`.
☐ I know integer and decimal division by zero differ.
☐ I can write clear error messages.

Lesson 8.4: Cleanup with use and finally

Reading References

- Exceptions: finally
- Matching Python lesson: Lesson 8.4: The else and finally Blocks

Learning Objectives

- Close files reliably with `use`.
- Show that a resource is released even after an error.
- Use `finally` for cleanup.

Concept Explanation

A file reader or writer holds a resource from the operating system, so it must be closed. The extension function `use { ... }` runs a block and then closes the resource, **even if the block throws an exception**. It is the Kotlin equivalent of try-with-resources in Java.

`finally` is the general tool: its block runs after the `try` block whether or not an exception happened. Use `use` for anything that can be closed, and `finally` for other cleanup.

The example writes a file with `use`, reads it back, throws an exception inside a second `use` block, and shows that a reader that was closed by `use` refuses to read.

Worked Example

```
import java.io.File

fun main() {
    val file = File("cleanup_demo.txt")
    file.bufferedWriter().use { it.write("Some data.\n") }
    println(file.readText().trim())

    val reader = file.bufferedReader()
    val firstLine = reader.use { it.readLine() }
    println(firstLine)

    val result = runCatching {
        file.bufferedReader().use { r ->
            r.readLine()
            throw IllegalStateException("failure while reading")
        }
    }
    println(result.exceptionOrNull()?.message)

    try {
        println("working")
    } finally {
        println("finally block ran")
    }

    println(runCatching { reader.readLine() }.exceptionOrNull() != null)
}
```

Expected Behavior or Output

The program prints:

```
Some data.
Some data.
failure while reading
working
finally block ran
true
```

Common Mistakes

1. Opening a reader or writer and forgetting to close it.
2. Putting `return` in a `finally` block, which can hide an earlier exception.
3. Using the reader after `use` has closed it.

Activities

1. Predict the six printed lines.
2. Rewrite the first example with an explicit `try` and `finally` and compare the length.
3. Throw an exception before the write and confirm that the file is still closed.
4. Explain why cleanup matters even if the program is about to end.

Accessibility Practice

Print a short labelled line after each step, so the screen reader announces which path ran. Labelled lines make the order of cleanup audible.

Skills Checklist

- ☐ I can close files with `use`.
- ☐ I can use `finally`.
- ☐ I know `use` closes after an error.
- ☐ I can show a closed reader refuses to read.
- ☐ I can explain why cleanup matters.

Lesson 8.5: Storing Data with JSON

Reading References

- Kotlin serialization documentation (kotlinlang.org/docs/serialization.html)
- JSON specification: https://www.json.org
- Matching Python lesson: Lesson 8.5: Storing Data with JSON

Learning Objectives

- Explain why this lesson includes a JSON helper.
- Encode and decode the shared task records.

- Handle a missing or malformed file.

Concept Explanation

The Kotlin standard library has no JSON support. Real projects add the `kotlinx.serialization` library through Gradle, which also requires a compiler plugin. To keep this course free of build tooling, the example includes a *teaching implementation*, `object Json`, with `encode` and `decode`. It handles strings, numbers, booleans, `null`, lists, and maps, writes object keys in sorted order, and reports malformed text with an exception that includes the position. It is not a replacement for a production library.

JSON text stores data in a standard format: `{ ... }` holds key-value pairs, `[ ... ]` holds ordered items, and text is in double quotation marks. Decoded data is untyped (`Map<String, Any?>`), so check each field before using it. Numbers without a decimal point decode as `Long`.

The shared to-do record has the fields `id`, `title`, `description`, `done`, and `created_at`. The example saves two tasks and loads them back, then shows the message for a malformed file.

The teaching helper decodes JSON `null` as `null` and numbers without a decimal point as `Long`. It does not handle every number edge case that a full parser must.

Worked Example

```
object Json {
    fun encode(value: Any?): String = when (value) {
        null -> "null"
        is String -> quote(value)
        is Boolean, is Int, is Long, is Double -> value.toString()
        is Map<*, *> -> value.entries.sortedBy { it.key.toString() }
            .joinToString(",", "{", "}") { quote(it.key.toString()) + ":" + encode(it.value)
        is List<*> -> value.joinToString(",", "[", "]") { encode(it) }
        else -> throw IllegalArgumentException("cannot encode ${value::class.simpleName}")
    }

    private fun quote(text: String): String {
        val out = StringBuilder("\"")
        for (c in text) {
            when {
                c == '"' -> out.append("\\\"")
                c == '\\' -> out.append("\\\\")
                c == '\n' -> out.append("\\n")
                c == '\r' -> out.append("\\r")
                c == '\t' -> out.append("\\t")
                c < ' ' -> out.append("\\u%04x".format(c.code))
                else -> out.append(c)
```

```
            }
        }
        return out.append('"').toString()
    }

    fun decode(text: String): Any? = Parser(text).parse()

    private class Parser(private val text: String) {
        private var pos = 0

        private fun fail(message: String): Nothing =
            throw IllegalArgumentException("$message at position $pos")

        private fun skip() {
            while (pos < text.length && text[pos].isWhitespace()) pos++
        }

        fun parse(): Any? {
            val result = value()
            skip()
            if (pos < text.length) fail("unexpected extra text")
            return result
        }

        private fun value(): Any? {
            skip()
            if (pos >= text.length) fail("unexpected end of text")
            val c = text[pos]
            return when {
                c == '{' -> obj()
                c == '[' -> list()
                c == '"' -> str()
                text.startsWith("true", pos) -> { pos += 4; true }
                text.startsWith("false", pos) -> { pos += 5; false }
                text.startsWith("null", pos) -> { pos += 4; null }
                c == '-' || c.isDigit() -> number()
                else -> fail("unexpected character")
            }
        }

        private fun number(): Any {
            val start = pos
            if (text[pos] == '-') pos++
            while (pos < text.length && (text[pos].isDigit() || text[pos] in ".eE+-")) pos++
            val token = text.substring(start, pos)
            return token.toLongOrNull() ?: token.toDoubleOrNull() ?: fail("bad number")
```

```
}

private fun str(): String {
    pos++
    val out = StringBuilder()
    while (true) {
        if (pos >= text.length) fail("unterminated string")
        val c = text[pos++]
        if (c == '"') return out.toString()
        if (c != '\\') { out.append(c); continue }
        if (pos >= text.length) fail("bad escape")
        when (text[pos++]) {
            'n' -> out.append('\n')
            't' -> out.append('\t')
            'r' -> out.append('\r')
            '"' -> out.append('"')
            '\\' -> out.append('\\')
            '/' -> out.append('/')
            'u' -> {
                if (pos + 4 > text.length) fail("bad escape")
                out.append(text.substring(pos, pos + 4).toInt(16).toChar())
                pos += 4
            }
            else -> fail("bad escape")
        }
    }
}

private fun obj(): Map<String, Any?> {
    pos++
    val map = LinkedHashMap<String, Any?>()
    skip()
    if (pos < text.length && text[pos] == '}') { pos++; return map }
    while (true) {
        skip()
        if (pos >= text.length || text[pos] != '"') fail("expected a key")
        val key = str()
        skip()
        if (pos >= text.length || text[pos] != ':') fail("expected ':'")
        pos++
        map[key] = value()
        skip()
        if (pos >= text.length) fail("unterminated object")
        val d = text[pos++]
        if (d == '}') return map
        if (d != ',') { pos--; fail("expected ',' or '}'") }
```

```
            }
        }

        private fun list(): List<Any?> {
            pos++
            val items = ArrayList<Any?>()
            skip()
            if (pos < text.length && text[pos] == ']') { pos++; return items }
            while (true) {
                items.add(value())
                skip()
                if (pos >= text.length) fail("unterminated list")
                val d = text[pos++]
                if (d == ']') return items
                if (d != ',') { pos--; fail("expected ',' or ']'") }
            }
        }
    }
}

fun main() {
    val tasks = listOf(
        mapOf("id" to 1, "title" to "Study programming", "description" to "Finish Unit 8",
            "done" to false, "created_at" to "2026-01-15T09:00:00"),
        mapOf("id" to 2, "title" to "Review scores", "description" to "Check the four scores
            "done" to true, "created_at" to "2026-01-15T09:30:00"),
    )

    val file = java.io.File("tasks.json")
    file.writeText(Json.encode(tasks))

    if (!file.exists()) {
        println("Tasks file not found.")
        return
    }
    val loaded = try {
        Json.decode(file.readText()) as List<*>
    } catch (e: IllegalArgumentException) {
        println("Error: could not decode tasks file: ${e.message}")
        return
    }
    for (entry in loaded) {
        val task = entry as Map<*, *>
        val status = if (task["done"] == true) "done" else "not done"
        println("${task["id"]}: ${task["title"]} ($status)")
    }
```

```
    println(runCatching { Json.decode("[{\"id\": 1,") }.exceptionOrNull()?.message)
}
```

Expected Behavior or Output

The program prints:

```
1: Study programming (not done)
2: Review scores (done)
expected a key at position 10
```

Common Mistakes

1. Assuming Kotlin has built-in JSON support.
2. Trusting decoded data without checking that each field exists and has the right type.
3. Overwriting a good data file with an empty list after a failed load.

Activities

1. Run the program and read the saved JSON text from `tasks.json`.
2. Edit `tasks.json` by hand to remove a closing bracket and run the loading part again. Read the message.
3. Add a third task and confirm that it is saved and loaded.
4. Explain why a production project should use `kotlinx.serialization`.

Accessibility Practice

Open `tasks.json` in the terminal and read it as text. JSON in a single line is difficult to navigate, so practise stepping through it by word or character.

Skills Checklist

- ☐ I know Kotlin has no built-in JSON.
- ☐ I can encode and decode records.
- ☐ I handle missing and malformed files.
- ☐ I check decoded fields.
- ☐ I know when to use a library.

11.11 Unit 9: Testing and Data Analysis

Overview

Write tests with expected results, test random code by its properties, analyze the shared text sample, and read the shared CSV file.

Lesson 9.1: Testing Functions

Reading References

- Functions
- Functions
- Matching Python lesson: Lesson 9.1: Testing Your Code

Learning Objectives

- Write checks with chosen inputs and expected results.
- Compare decimals with a tolerance.
- Test boundary values and report failure with an exit status.

Concept Explanation

A *test* runs code with chosen inputs and compares the result with a value decided in advance. Real Kotlin projects use the `kotlin.test` library with JUnit, added through Gradle. To keep this course free of build tooling, the example uses a tiny `check` function; the habits (inputs, expected values, boundaries) are the same.

Decimal results are compared with a *tolerance*, not `==`, because of floating-point approximation: the tip for 24.50 at 0.15 is stored as 3.6749999999999998, so the test checks that it is within a tiny distance of 3.675.

The same tests appear in every language track: `add(5, 3)` is 8, `add(-5, -3)` is -8, `add(10, -4)` is 6, the tip is 3.675, and the grade boundaries 90, 89, 60, and 59 give A, B, D, and F.

`exitProcess(1)` ends the program with a failure status when any check fails, so other tools can notice.

Worked Example

```
import kotlin.math.abs
import kotlin.system.exitProcess

fun add(a: Int, b: Int) = a + b

fun calculateTip(bill: Double, rate: Double) = bill * rate

fun classify(score: Int): String = when {
    score >= 90 -> "A"
    score >= 80 -> "B"
    score >= 70 -> "C"
    score >= 60 -> "D"
```

```
    else -> "F"
}

var passed = 0
var failed = 0

fun <T> check(name: String, actual: T, expected: T) {
    if (actual == expected) {
        passed++
        println("PASS $name")
    } else {
        failed++
        println("FAIL $name: expected $expected but got $actual")
    }
}

fun checkClose(name: String, actual: Double, expected: Double, tolerance: Double = 1e-9) {
    check(name, abs(actual - expected) <= tolerance, true)
}

fun main() {
    check("add 5 and 3", add(5, 3), 8)
    check("add -5 and -3", add(-5, -3), -8)
    check("add 10 and -4", add(10, -4), 6)
    checkClose("tip on 24.50 at 0.15", calculateTip(24.5, 0.15), 3.675)
    check("tip on a zero bill", calculateTip(0.0, 0.15), 0.0)
    check("classify 90", classify(90), "A")
    check("classify 89", classify(89), "B")
    check("classify 60", classify(60), "D")
    check("classify 59", classify(59), "F")

    println("$passed passed, $failed failed")
    if (failed > 0) exitProcess(1)
}
```

Expected Behavior or Output

The program prints:

```
PASS add 5 and 3
PASS add -5 and -3
PASS add 10 and -4
PASS tip on 24.50 at 0.15
PASS tip on a zero bill
PASS classify 90
PASS classify 89
```

```
PASS classify 60
PASS classify 59
9 passed, 0 failed
```

Common Mistakes

1. Comparing decimal results with `==`.
2. Testing only typical values and skipping the boundaries 90, 89, 60, and 59.
3. Writing the expected value after seeing the output, which turns the test into a copy of the bug.

Activities

1. Change one expected value so a check fails, read the message, and restore it.
2. Add checks for the grades at 80, 79, 70, and 69.
3. Write checks for `isPassing(score)` before writing the function.
4. Explain why the tip check uses a tolerance.

Accessibility Practice

Each check prints one complete line that starts with PASS or FAIL, and the last line gives the counts, so the result can be understood from the first word of each line and the final summary.

Skills Checklist

☐ I can write a check with an expected value.
☐ I can compare decimals with a tolerance.
☐ I can choose boundary tests.
☐ I can read PASS and FAIL lines.
☐ I know how to signal failure with an exit status.

Lesson 9.2: Testing Classes

Reading References

- Classes
- Exceptions
- Matching Python lesson: Lesson 9.2: Testing a Class

Learning Objectives

- Create a fresh object for every test.
- Test a value, a change of state, and an error.

- Use `runCatching` to test that an error occurs.

Concept Explanation

Each test should start from a known state, so create a new object for every test with a *setup* function. Tests that share an object can affect each other and fail in confusing ways.

Test three kinds of behavior: that a function returns the right value (`totalValue` for 3 at 2.50 is 7.5), that a function changes the object correctly (restocking 2 makes the quantity 5), and that invalid use is rejected (restocking a negative amount throws).

`runCatching { ... }.isFailure` is true when the block throws, so a test can assert that an error *does* happen.

Worked Example

```
import kotlin.math.abs

class Item(val name: String, var quantity: Int, val unitPrice: Double) {
    fun totalValue(): Double = quantity * unitPrice

    fun restock(amount: Int) {
        require(amount >= 0) { "amount must not be negative" }
        quantity += amount
    }
}

fun setup() = Item("Notebook", 3, 2.50)

var failures = 0

fun test(name: String, body: () -> Unit) {
    val result = runCatching(body)
    if (result.isSuccess) {
        println("PASS $name")
    } else {
        failures++
        println("FAIL $name: ${result.exceptionOrNull()?.message}")
    }
}

fun assertThat(condition: Boolean, message: String) {
    if (!condition) throw AssertionError(message)
}
```

```
fun main() {
    test("total value is 7.5") {
        assertThat(abs(setup().totalValue() - 7.5) < 1e-9, "wrong total")
    }

    test("restock adds to the quantity") {
        val item = setup()
        item.restock(2)
        assertThat(item.quantity == 5, "quantity should be 5")
    }

    test("restock rejects a negative amount") {
        val failed = runCatching { setup().restock(-1) }.isFailure
        assertThat(failed, "an error should have been thrown")
    }

    println(if (failures == 0) "All tests passed." else "$failures test(s) failed.")
}
```

Expected Behavior or Output

The program prints:

```
PASS total value is 7.5
PASS restock adds to the quantity
PASS restock rejects a negative amount
All tests passed.
```

Common Mistakes

1. Sharing one object between tests, so an earlier test changes the later ones.
2. Testing only the successful path and never the error path.
3. Writing an `assertThat` with no message, so a failure says nothing about what went wrong.

Activities

1. Add a test that `restock(0)` leaves the quantity unchanged.
2. Add a test for an `isValid` function.
3. Break `restock` on purpose and read the failure line.
4. Explain why `setup` creates a new item each time.

Accessibility Practice

Name each test as a short sentence that says what it checks, so the PASS and FAIL lines read as clear statements. Read the final summary line first.

Skills Checklist

- ☐ I can create a fresh object per test.
- ☐ I can test a return value.
- ☐ I can test a change of state.
- ☐ I can test that an error is thrown.
- ☐ I can write a clear test name.

Lesson 9.3: Random Numbers

Reading References

- Kotlin standard library: kotlin.random
- Matching Python lesson: Lesson 9.3: Random Numbers and Simulation

Learning Objectives

- Generate random integers and decimals.
- Use a seed to make results repeatable.
- Test random code by its properties.

Concept Explanation

`kotlin.random.Random` produces random numbers. `Random.nextInt(from, until)` returns an integer from `from` up to but **not including** `until`, so a die roll is `nextInt(1, 7)`. `Random.nextDouble()` returns a decimal from 0.0 up to but not including 1.0, and `list.random(random)` picks a list element.

`Random(42)` creates a generator with a fixed *seed*: two generators with the same seed produce the same sequence. Without a seed the results change on every run. Because the exact sequence may differ between Kotlin versions, tests should check *properties* (every roll is between 1 and 6) and repeatability, not particular values.

Worked Example

```
import kotlin.random.Random

fun main() {
    val first = Random(42)
    val second = Random(42)
    val firstRolls = List(5) { first.nextInt(1, 7) }
    val secondRolls = List(5) { second.nextInt(1, 7) }
    println("Same seed gives the same rolls: ${firstRolls == secondRolls}")

    val random = Random(7)
    val inRange = List(1000) { random.nextInt(1, 7) }.all { it in 1..6 }
```

```
    println("All rolls between 1 and 6: $inRange")

    val decimal = random.nextDouble()
    println("Decimal in [0, 1): ${decimal >= 0.0 && decimal < 1.0}")

    val scores = listOf(82, 91, 76, 88)
    val chosen = scores.random(random)
    println("Chosen score is from the list: ${chosen in scores}")
}
```

Expected Behavior or Output

The program prints:

```
Same seed gives the same rolls: true
All rolls between 1 and 6: true
Decimal in [0, 1): true
Chosen score is from the list: true
```

Common Mistakes

1. Using `nextInt(1, 6)` for a die and never rolling a 6. The upper limit is excluded.
2. Testing for specific random values that may change between versions.
3. Creating a new unseeded generator for every roll, which wastes work and can repeat values.

Activities

1. Run the program three times and compare.
2. Change the seeds and check that the property lines still print `true`.
3. Simulate 600 rolls and count how many of each face appeared (the counts will differ with the seed).
4. Explain why a test should check a property rather than a specific random number.

Accessibility Practice

Print yes or no style results for random behavior, so the announced output is the same on every run and the screen reader does not read unpredictable values.

Skills Checklist

- ☐ I can use `nextInt` with an excluded upper limit.
- ☐ I can set a seed.
- ☐ I can test properties of random code.
- ☐ I know sequences can differ between versions.
- ☐ I can pick a random list item.

Lesson 9.4: Text Analysis

Reading References

- Strings
- Grouping
- Matching Python lesson: Lesson 9.4: Data Analysis - Text Analysis

Learning Objectives

- Split text into words and count them.
- Count word and letter frequencies with `groupingBy`.
- Produce stable, sorted output.

Concept Explanation

Splitting uses `split`: `cleaned.split(" ")` or a regular expression such as `Regex("\\s+")` for any run of spaces. Counting is a one-liner with `groupingBy { it }.eachCount()`, which returns a map from each item to its count.

Map order is only stable if you make it so. `eachCount()` keeps the order in which items first appeared, and sorting with `sortedWith(compareBy({ -it.value }, { it.key }))` orders by count (highest first) and then alphabetically, so ties print in a fixed order.

The shared text sample is "A student studies programming. Programming develops problem-solving skills." After lower-casing and removing the periods it has 8 words, and `programming` appears twice.

Worked Example

```
fun main() {
    val text = "A student studies programming. Programming develops problem-solving skills.'
    val words = text.lowercase().replace(".", "").split(" ")

    println("Word count: ${words.size}")
    println("Word frequencies:")
    for ((word, count) in words.groupingBy { it }.eachCount()) {
        println("'$word': $count")
    }

    val letters = text.lowercase().filter { it.isLetter() }.groupingBy { it }.eachCount()
    val topThree = letters.entries.sortedWith(compareBy({ -it.value }, { it.key })).take(3)
    println("Most common letters:")
    for ((letter, count) in topThree) {
        println("$letter: $count")
```

```
    }
}
```

Expected Behavior or Output

The program prints:

```
Word count: 8
Word frequencies:
'a': 1
'student': 1
'studies': 1
'programming': 2
'develops': 1
'problem-solving': 1
'skills': 1
Most common letters:
s: 7
e: 5
g: 5
```

Common Mistakes

1. Counting punctuation as part of a word, so `programming.` and `programming` are different words.
2. Forgetting to lower-case first, so `Programming` and `programming` are counted separately.
3. Relying on map order for tied counts without a tie-break.

Activities

1. Predict the word count and the count for `programming` before running.
2. Count the words longer than 8 letters.
3. Print the longest word.
4. Change the sort so ties are listed in reverse alphabetical order.

Accessibility Practice

Read the frequency list in its fixed order. Because the order is stable, you can learn it and compare runs by ear. Print each pair as a short sentence with the same wording every time.

Skills Checklist

- ☐ I can split text into words.
- ☐ I can count frequencies with `groupingBy`.
- ☐ I can sort with a tie-break.
- ☐ I can remove punctuation first.

☐ I can read a stable report.

Lesson 9.5: Working with CSV Files

Reading References

- Kotlin standard library: kotlin.io
- Strings
- Matching Python lesson: Lesson 9.5: Data Analysis - CSV Files

Learning Objectives

- Create and read a CSV file.
- Split lines into fields and convert numbers.
- Compute an average and a count.

Concept Explanation

A CSV (comma-separated values) file stores a table as text. The first line names the columns and each later line is a record. The shared student file has the columns `student_id`, `first_name`, `last_name`, `score`, and `active`, and three records.

The standard library has no CSV reader. `line.split(",")` works only for **simple** files: a value that contains a comma inside quotation marks, such as `"Smith, Jr."`, would be split wrongly. For such files use a maintained library or write a real parser.

Every field is text. Convert the score with `toInt()` and compare `active` with the text `"true"`. With scores 91, 82, and 76 the average is 83.00, and two students are active.

Worked Example

```
import java.io.File

fun main() {
    val file = File("students.csv")
    file.writeText(
        "student_id,first_name,last_name,score,active\n" +
            "1,Avery,Student,91,true\n" +
            "2,Jordan,Lee,82,true\n" +
            "3,Casey,Smith,76,false\n"
    )

    if (!file.exists()) {
        println("Error: students.csv was not found.")
```

```
        return
    }
    val lines = file.readLines()
    println("Columns: " + lines.first().split(",").joinToString(", "))

    var total = 0
    var active = 0
    val students = lines.drop(1)
    for (line in students) {
        val row = line.split(",")
        val score = row[3].toInt()
        total += score
        if (row[4] == "true") active++
        println("${row[0]}: ${row[1]} ${row[2]} scored $score")
    }

    println("Students: ${students.size}")
    println("Average score: " + "%.2f".format(total.toDouble() / students.size))
    println("Active students: $active")
}
```

Expected Behavior or Output

The program prints:

```
Columns: student_id, first_name, last_name, score, active
1: Avery Student scored 91
2: Jordan Lee scored 82
3: Casey Smith scored 76
Students: 3
Average score: 83.00
Active students: 2
```

Common Mistakes

1. Splitting a CSV line at every comma when a value contains a quoted comma.
2. Forgetting to convert the text in the score column to a number.
3. Not skipping the header line, so `toInt()` fails on the text `score`.

Activities

1. Predict the printed lines before running.
2. Add a fourth student and check the new average.
3. Print only the active students.
4. Explain, with an example, why `"Smith, Jr."` breaks `split(",")`.

Accessibility Practice

Print one student per line with the same wording each time, so the screen reader announces a repeating pattern that is easy to compare. Read the column names aloud once at the start.

Skills Checklist

- ☐ I can write and read a CSV file.
- ☐ I can split a line into fields.
- ☐ I convert text fields to numbers.
- ☐ I can compute an average and a count.
- ☐ I know when `split` is not enough.

11.12 Unit 10: Project-Based Learning — Text-Based Applications

Overview

Plan a text project, build the shared text adventure in two parts, build the command-line to-do manager, and refine input validation.

Lesson 10.1: Planning a Text-Based Project

Reading References

- Functions: function types
- Collections overview
- Matching Python lesson: Lesson 10.1: Project Planning and Design

Learning Objectives

- Write a short requirements list.
- Design a command loop with a map of handlers.
- Plan in small steps and test each one.

Concept Explanation

A project is easier to finish when it is planned in small steps. Write down what the program must do (*requirements*), what information it keeps (*data*), and the commands a person can give (*interface*). Build one small piece, test it, then add the next.

Text-based projects share a *command loop*: show a prompt, read a line, find the matching action, and repeat until the user quits. In Kotlin, a `Map<String, () -> Unit>` holds actions: the keys are command names and the values are *lambdas*, small functions with no name. A new command is one new entry, and an unknown command is a failed lookup that prints a helpful message.

The loop stops on `quit` and also when input ends (`null`). The example is a skeleton with two commands; typing `help`, `look`, an unknown word, and `quit` shows each path.

Worked Example

```
fun main() {
    val commands: Map<String, () -> Unit> = mapOf(
        "help" to { println("Commands: help, look, quit") },
        "look" to { println("You are in a dark cave.") },
    )

    println("Type a command:")
    var line = readlnOrNull()
    while (line != null && line != "quit") {
        val handler = commands[line]
        if (handler != null) {
            handler()
        } else {
            println("I don't understand that command. Type help for a list.")
        }
        println("Type a command:")
        line = readlnOrNull()
    }
    println("Goodbye!")
}
```

Expected Behavior or Output

With the following lines typed as standard input (piped input is not echoed, so it does not appear in the output):

```
help
look
dance
quit
```

the program prints:

```
Type a command:
Commands: help, look, quit
Type a command:
```

```
You are in a dark cave.
Type a command:
I don't understand that command. Type help for a list.
Type a command:
Goodbye!
```

With no standard input at all, the program ends cleanly and prints:

```
Type a command:
Goodbye!
```

Common Mistakes

1. Starting to code before listing the requirements.
2. Using a long chain of `if` statements instead of a map of handlers.
3. Forgetting that an unknown command should say what to do next.

Activities

1. Write five requirements for the text adventure and the to-do list.
2. Add a `score` command that prints the shared scores.
3. Add the aliases `h` and `l` without copying the lambdas.
4. Describe the data for the capstone in three sentences.

Accessibility Practice

Prompts and responses are on separate lines and every response is a complete sentence. Test the prompts by listening to them before you add features.

Skills Checklist

- ☐ I can list requirements.
- ☐ I can design a command loop.
- ☐ I can use a map of lambdas.
- ☐ I handle unknown commands.
- ☐ I handle the end of input.

Lesson 10.2: Text Adventure, Part 1: Five Rooms and Movement

Reading References

- Data classes
- Collections overview
- Canonical shared project specification: Python Unit 10, Lesson 10.3

Learning Objectives

- Represent the five canonical rooms and their exits with Kotlin data.
- Move through valid exits without changing location after an invalid command.
- Explain why the north exit from the locked-door room is not available yet.

Concept Explanation

The shared adventure has five rooms: start_room, tunnel, clearing, locked_door_room, and treasure_room. The unconditional connections are start north to tunnel; tunnel south to start and east to clearing; clearing west to tunnel and north to the locked-door room; and locked-door room south to clearing. The north exit to the treasure room is conditional and is added in Part 2 only after the shiny-key puzzle is solved.

A Room data class stores a description and a map of exits. Looking up a missing direction with square brackets returns null; it does not change the current room.

Worked Example

```
data class Room(val description: String, val exits: Map<String, String>)

fun main() {
    val rooms = mapOf(
        "start_room" to Room("A dimly lit cave. A narrow passage leads north.", mapOf("north
        "tunnel" to Room("A damp tunnel. Passages lead south and east.", mapOf("south" to "s
        "clearing" to Room("A quiet clearing. An antechamber lies to the north.", mapOf("wes
        "locked_door_room" to Room("A chamber with a heavy door to the north.", mapOf("south
        "treasure_room" to Room("You found the treasure. You win!", emptyMap()),
    )

    var currentRoom = "start_room"
    while (true) {
        println("---")
        println(rooms.getValue(currentRoom).description)
        println("What do you want to do?")
        val command = (readlnOrNull() ?: "quit").trim().lowercase()
        val destination = rooms.getValue(currentRoom).exits[command]
        when {
            command == "quit" -> {
                println("Exiting game. Goodbye!")
                break
            }
            destination != null -> currentRoom = destination
            else -> println("You cannot go that way. Enter a listed direction or quit.")
        }
```

```
    }
}
```

Expected Behavior or Output

With north, east, north, north, quit entered as separate input lines, the player visits tunnel, clearing, and locked_door_room. The second north from locked_door_room is rejected and the current room does not change. The treasure room exists in the model but cannot be reached in Part 1.

Common Mistakes

1. Adding the treasure-room north exit before implementing the key puzzle.
2. Updating currentRoom before confirming that the requested exit exists.
3. Using getValue on an exit map when the direction may be absent.
4. Defining an exit whose destination is not one of the five room identifiers.

Activities

1. Trace north, east, north, north and record the room after each command.
2. Try west from start_room; confirm that the location does not change.
3. Check that every destination in every unconditional exit exists in rooms.
4. Explain why treasure_room has no incoming exit in Part 1.

Accessibility Practice

Read the separator, room description, and prompt in order. The invalid-movement response explains what happened and how to continue. Test the sequence by listening to terminal output; do not rely on a visual map.

Skills Checklist

☐ I can model all five canonical rooms.
☐ I can represent exits with a map.
☐ I can move only when an exit exists.
☐ I can reject an invalid direction without changing state.
☐ I can explain why the treasure room is not yet reachable.

Lesson 10.3: Text Adventure, Part 2: Shiny Key and Locked Door

Reading References

- Collections overview
- Control flow: when
- Canonical shared project specification: Python Unit 10, Lesson 10.3

Learning Objectives

- Place the shiny key in start_room and transfer it to inventory exactly once.
- Require the player to use the key while in locked_door_room.
- Unlock the north exit to treasure_room only after the puzzle succeeds.
- End the game when the player reaches the treasure room.

Concept Explanation

Part 2 keeps the same five-room graph. The shiny key begins in start_room. The take shiny key command removes it from that room and adds it to inventory. The use shiny key command works only in locked_door_room and only when the key is in inventory. Attempting north from the locked-door room before that action prints a locked-door message and leaves the player in place. After the key is used, north leads to treasure_room.

Room descriptions do not claim that an item is still present. Room items and inventory are announced on separate lines.

Worked Example

```
class Room(
    val description: String,
    val exits: Map<String, String>,
    val items: MutableList<String> = mutableListOf(),
)

fun main() {
    val rooms = mapOf(
        "start_room" to Room("A dimly lit cave.", mapOf("north" to "tunnel"), mutableListOf(
        "tunnel" to Room("A damp tunnel. Passages lead south and east.", mapOf("south" to "s
        "clearing" to Room("A quiet clearing. An antechamber lies to the north.", mapOf("wes
        "locked_door_room" to Room("A chamber with a heavy door to the north.", mapOf("south
        "treasure_room" to Room("You found the treasure. You win!", emptyMap()),
    )
    var currentRoom = "start_room"
    val inventory = mutableListOf<String>()
    var doorLocked = true

    while (true) {
        val room = rooms.getValue(currentRoom)
        println()
        println(room.description)
        if (currentRoom == "treasure_room") break
        if (room.items.isNotEmpty()) println("You see: " + room.items.joinToString(", "))
        println("Your inventory: " + if (inventory.isEmpty()) "empty" else inventory.joinToS
```

```
        println("What do you want to do?")
        val command = (readlnOrNull() ?: "quit").trim().lowercase()

        when {
            command == "quit" -> {
                println("You quit the adventure.")
                break
            }
            command == "take shiny key" -> {
                if (room.items.remove("shiny key")) {
                    inventory.add("shiny key")
                    println("You picked up the shiny key.")
                } else println("There is no shiny key here to take.")
            }
            command == "use shiny key" -> when {
                currentRoom != "locked_door_room" -> println("You must be at the locked door
                "shiny key" !in inventory -> println("You do not have the shiny key.")
                else -> {
                    doorLocked = false
                    println("You unlocked the door!")
                }
            }
            currentRoom == "locked_door_room" && command == "north" && doorLocked ->
                println("The door is locked! You need to use the shiny key here.")
            command in room.exits -> currentRoom = room.exits.getValue(command)
            else -> println("Invalid command. Enter north, south, east, west, take shiny key
        }
    }
}
```

Expected Behavior or Output

Use this sequence to verify the puzzle: north, east, north, north, south, west, south, take shiny key, north, east, north, use shiny key, north.

The first north attempt from locked_door_room must be rejected. Taking the key removes it from start_room and adds it to inventory. Using it outside locked_door_room must be rejected. Using it there prints You unlocked the door!; the next north reaches treasure_room and prints You found the treasure. You win!. End-of-file is treated as quit.

Common Mistakes

1. Leaving the shiny key in the room after adding it to inventory.
2. Unlocking the door outside locked_door_room.
3. Unlocking the door without checking inventory.
4. Allowing north movement before doorLocked becomes false.

5. Describing the key as still on the floor after it has been collected.

Activities

1. Run the full sequence and record room, room items, inventory, and door state after every command.
2. Try take shiny key twice; the second attempt must not duplicate the item.
3. Try use shiny key in start_room and clearing; both attempts must be rejected.
4. Try north from locked_door_room before and after unlocking.
5. Add a look command that announces room, visible items, inventory, and exits in a consistent order.

Accessibility Practice

Announce room description, visible items, inventory, and prompt as separate text lines. Every failed action explains why it failed. Listen to the full puzzle sequence and verify that key location and door state are unambiguous.

Skills Checklist

☐ I can model the five canonical rooms.
☐ I can transfer the shiny key from a room to inventory.
☐ I can prevent duplicate pickup.
☐ I can require key use in the locked-door room.
☐ I can block and unlock the north exit.
☐ I can end the game when the treasure room is reached.

Lesson 10.4: Command-Line To-Do List

Reading References

- Collections overview
- Exceptions
- Matching Python lesson: Lesson 10.4: Building a Command-Line To-Do List

Learning Objectives

- Keep records with the shared fields.
- Save and load them as JSON.
- Offer add, view, mark-done, save, load, and quit commands.

Concept Explanation

A to-do list ties together records, loops, functions, and files. Each task has the shared fields `id`, `title`, `description`, `done`, and `created_at`. The program keeps a list of tasks in memory, shows a numbered menu, and saves the list to `todos.json` when asked and on exit.

Saving uses the teaching JSON helper from Lesson 8.5, included here so the example runs on its own. At start-up the program loads the file if it exists. A missing file is normal on first use, and a damaged file must not crash the program: it prints a message and starts with an empty list.

New ids are one more than the highest existing id. The status is shown as the words `done` and `not done`, never with a symbol or color alone.

In the sample session the user adds two tasks, views the list, marks task 2 done, views it again, tries an invalid menu choice, views the list, and quits.

Worked Example

```
object Json {
    fun encode(value: Any?): String = when (value) {
        null -> "null"
        is String -> quote(value)
        is Boolean, is Int, is Long, is Double -> value.toString()
        is Map<*, *> -> value.entries.sortedBy { it.key.toString() }
            .joinToString(",", "{", "}") { quote(it.key.toString()) + ":" + encode(it.value)
        is List<*> -> value.joinToString(",", "[", "]") { encode(it) }
        else -> throw IllegalArgumentException("cannot encode ${value::class.simpleName}")
    }

    private fun quote(text: String): String {
        val out = StringBuilder("\"")
        for (c in text) {
            when {
                c == '"' -> out.append("\\\"")
                c == '\\' -> out.append("\\\\")
                c == '\n' -> out.append("\\n")
                c == '\r' -> out.append("\\r")
                c == '\t' -> out.append("\\t")
                c < ' ' -> out.append("\\u%04x".format(c.code))
                else -> out.append(c)
            }
        }
        return out.append('"').toString()
    }

    fun decode(text: String): Any? = Parser(text).parse()
```

```
private class Parser(private val text: String) {
    private var pos = 0

    private fun fail(message: String): Nothing =
        throw IllegalArgumentException("$message at position $pos")

    private fun skip() {
        while (pos < text.length && text[pos].isWhitespace()) pos++
    }

    fun parse(): Any? {
        val result = value()
        skip()
        if (pos < text.length) fail("unexpected extra text")
        return result
    }

    private fun value(): Any? {
        skip()
        if (pos >= text.length) fail("unexpected end of text")
        val c = text[pos]
        return when {
            c == '{' -> obj()
            c == '[' -> list()
            c == '"' -> str()
            text.startsWith("true", pos) -> { pos += 4; true }
            text.startsWith("false", pos) -> { pos += 5; false }
            text.startsWith("null", pos) -> { pos += 4; null }
            c == '-' || c.isDigit() -> number()
            else -> fail("unexpected character")
        }
    }

    private fun number(): Any {
        val start = pos
        if (text[pos] == '-') pos++
        while (pos < text.length && (text[pos].isDigit() || text[pos] in ".eE+-")) pos++
        val token = text.substring(start, pos)
        return token.toLongOrNull() ?: token.toDoubleOrNull() ?: fail("bad number")
    }

    private fun str(): String {
        pos++
        val out = StringBuilder()
        while (true) {
```

```
            if (pos >= text.length) fail("unterminated string")
            val c = text[pos++]
            if (c == '"') return out.toString()
            if (c != '\\') { out.append(c); continue }
            if (pos >= text.length) fail("bad escape")
            when (text[pos++]) {
                'n' -> out.append('\n')
                't' -> out.append('\t')
                'r' -> out.append('\r')
                '"' -> out.append('"')
                '\\' -> out.append('\\')
                '/' -> out.append('/')
                'u' -> {
                    if (pos + 4 > text.length) fail("bad escape")
                    out.append(text.substring(pos, pos + 4).toInt(16).toChar())
                    pos += 4
                }
                else -> fail("bad escape")
            }
        }
    }

    private fun obj(): Map<String, Any?> {
        pos++
        val map = LinkedHashMap<String, Any?>()
        skip()
        if (pos < text.length && text[pos] == '}') { pos++; return map }
        while (true) {
            skip()
            if (pos >= text.length || text[pos] != '"') fail("expected a key")
            val key = str()
            skip()
            if (pos >= text.length || text[pos] != ':') fail("expected ':'")
            pos++
            map[key] = value()
            skip()
            if (pos >= text.length) fail("unterminated object")
            val d = text[pos++]
            if (d == '}') return map
            if (d != ',') { pos--; fail("expected ',' or '}'") }
        }
    }

    private fun list(): List<Any?> {
        pos++
        val items = ArrayList<Any?>()
```

```
            skip()
            if (pos < text.length && text[pos] == ']') { pos++; return items }
            while (true) {
                items.add(value())
                skip()
                if (pos >= text.length) fail("unterminated list")
                val d = text[pos++]
                if (d == ']') return items
                if (d != ',') { pos--; fail("expected ',' or ']'") }
            }
        }
    }
}

val todoFile = java.io.File("todos.json")
var todos = mutableListOf<MutableMap<String, Any?>>()

fun loadTodos() {
    if (!todoFile.exists()) {
        println("No existing tasks found.")
        todos = mutableListOf()
        return
    }
    try {
        val data = Json.decode(todoFile.readText()) as List<*>
        todos = data.map { (it as Map<*, *>).entries.associate { e -> e.key.toString() to e.
        println("Tasks loaded.")
    } catch (e: Exception) {
        println("Error: todos.json is not valid JSON. Starting with an empty list.")
        todos = mutableListOf()
    }
}

fun saveTodos() {
    todoFile.writeText(Json.encode(todos))
    println("Tasks saved.")
}

fun nextId(): Long = (todos.maxOfOrNull { it["id"] as Long } ?: 0L) + 1

fun addTask(title: String, description: String) {
    if (title.isBlank()) {
        println("A title is required.")
        return
    }
    val stamp = java.time.LocalDateTime.now().format(java.time.format.DateTimeFormatter.ofPa
```

```
    todos.add(mutableMapOf("id" to nextId(), "title" to title.trim(), "description" to desc
    println("Task '${title.trim()}' added.")
}

fun viewTasks() {
    if (todos.isEmpty()) {
        println("No tasks in your list.")
        return
    }
    println("--- Your To-Do List ---")
    for (task in todos) {
        val status = if (task["done"] == true) "done" else "not done"
        println("${task["id"]}. ${task["title"]} ($status)")
    }
    println("-----------------------")
}

fun markDone(text: String?) {
    val id = text?.trim()?.toLongOrNull()
    if (id == null) {
        println("Please enter a whole number.")
        return
    }
    val task = todos.firstOrNull { it["id"] == id }
    if (task == null) {
        println("No task has that number.")
        return
    }
    task["done"] = true
    println("Task '${task["title"]}' marked as done.")
}

fun showMenu() {
    println()
    println("--- To-Do List Menu ---")
    println("1. Add Task")
    println("2. View Tasks")
    println("3. Mark Task Done")
    println("4. Save Tasks")
    println("5. Load Tasks")
    println("6. Quit")
    println("-----------------------")
}

fun main() {
    loadTodos()
```

```
    while (true) {
        showMenu()
        println("Enter your choice (1-6):")
        val choice = readlnOrNull() ?: "6"
        when (choice) {
            "1" -> {
                println("Enter task title:")
                val title = readlnOrNull() ?: ""
                println("Enter task description:")
                val description = readlnOrNull() ?: ""
                addTask(title, description)
            }
            "2" -> viewTasks()
            "3" -> {
                viewTasks()
                println("Enter task number to mark done:")
                markDone(readlnOrNull())
            }
            "4" -> saveTodos()
            "5" -> loadTodos()
            "6" -> {
                saveTodos()
                println("Goodbye!")
                return
            }
            else -> println("Invalid choice. Please try again.")
        }
    }
}
```

Expected Behavior or Output

With the following lines typed as standard input (piped input is not echoed, so it does not appear in the output):

```
1
Study programming
Finish Unit 10
1
Review scores
Check the four scores
2
3
2
3
7
```

```
2
6
```

the program prints:

```
No existing tasks found.

--- To-Do List Menu ---
1. Add Task
2. View Tasks
3. Mark Task Done
4. Save Tasks
5. Load Tasks
6. Quit
----------------------
Enter your choice (1-6):
Enter task title:
Enter task description:
Task 'Study programming' added.

--- To-Do List Menu ---
1. Add Task
2. View Tasks
3. Mark Task Done
4. Save Tasks
5. Load Tasks
6. Quit
----------------------
Enter your choice (1-6):
Enter task title:
Enter task description:
Task 'Review scores' added.

--- To-Do List Menu ---
1. Add Task
2. View Tasks
3. Mark Task Done
4. Save Tasks
5. Load Tasks
6. Quit
----------------------
Enter your choice (1-6):
--- Your To-Do List ---
1. Study programming (not done)
2. Review scores (not done)
----------------------
```

```
--- To-Do List Menu ---
1. Add Task
2. View Tasks
3. Mark Task Done
4. Save Tasks
5. Load Tasks
6. Quit
----------------------
Enter your choice (1-6):
--- Your To-Do List ---
1. Study programming (not done)
2. Review scores (not done)
----------------------
Enter task number to mark done:
Task 'Review scores' marked as done.

--- To-Do List Menu ---
1. Add Task
2. View Tasks
3. Mark Task Done
4. Save Tasks
5. Load Tasks
6. Quit
----------------------
Enter your choice (1-6):
--- Your To-Do List ---
1. Study programming (not done)
2. Review scores (done)
----------------------
Enter task number to mark done:
No task has that number.

--- To-Do List Menu ---
1. Add Task
2. View Tasks
3. Mark Task Done
4. Save Tasks
5. Load Tasks
6. Quit
----------------------
Enter your choice (1-6):
--- Your To-Do List ---
1. Study programming (not done)
2. Review scores (done)
----------------------
```

```
--- To-Do List Menu ---
1. Add Task
2. View Tasks
3. Mark Task Done
4. Save Tasks
5. Load Tasks
6. Quit
----------------------
Enter your choice (1-6):
Tasks saved.
Goodbye!
```

With no standard input at all, the program ends cleanly and prints:

```
No existing tasks found.

--- To-Do List Menu ---
1. Add Task
2. View Tasks
3. Mark Task Done
4. Save Tasks
5. Load Tasks
6. Quit
----------------------
Enter your choice (1-6):
Tasks saved.
Goodbye!
```

Common Mistakes

1. Saving the file but never loading it, so tasks disappear between runs.
2. Reusing a deleted task's id, which can confuse saved references.
3. Letting a damaged `todos.json` crash the program instead of printing a message.

Activities

1. Run the sample session, then run the program again typing `2` and `6` to see the saved tasks.
2. Add a command to delete a task by number.
3. Open `todos.json` and read the saved text. Edit it to remove a bracket and run the program again.
4. Show each task's description in the view command.

Accessibility Practice

Every menu is printed as numbered lines and every prompt is on its own line. The status words are `done` and `not done`, not symbols, so the list reads clearly

in speech and braille.

Skills Checklist

- ☐ I can keep records with the shared fields.
- ☐ I can save and load JSON.
- ☐ I handle a missing or damaged file.
- ☐ I keep ids unique.
- ☐ I show status in words.

Lesson 10.5: User Experience and Input Validation

Reading References

- Functions
- Null safety
- Matching Python lesson: Lesson 10.5: Refining User Experience and Validation

Learning Objectives

- Write reusable validation functions.
- Limit retries and handle the end of input.
- Write clear, accessible messages.

Concept Explanation

Reusable validation functions make every menu and form consistent. `askNumber(prompt, low, high, attempts)` prints a prompt, reads a line, converts it, checks the range, and either returns the number or explains the problem and asks again. After the allowed attempts, or at the end of input, it returns `null`. `askYesNo(prompt)` accepts `y`, `yes`, `n`, and `no` in any letter case.

Good messages say *what was wrong* and *what to do*: "Enter a whole number from 0 to 100," not "Invalid." They never depend on color, position, or symbols alone, and they use the same wording every time. The return type `Int?` makes the caller deal with the failure case.

The example reads a score with the typed lines abc, 105, and 88, then asks a yes or no question with the lines maybe and y.

Worked Example

```
fun askNumber(prompt: String, low: Int, high: Int, attempts: Int): Int? {
    for (attempt in 1..attempts) {
        println(prompt)
```

```
        val text = readlnOrNull() ?: return null
        val number = text.trim().toIntOrNull()
        if (number != null && number in low..high) return number
        println("Enter a whole number from $low to $high. You have ${attempts - attempt} att
    }
    return null
}

fun askYesNo(prompt: String): Boolean? {
    while (true) {
        println("$prompt (yes or no)")
        val text = readlnOrNull()?.trim()?.lowercase() ?: return null
        if (text == "y" || text == "yes") return true
        if (text == "n" || text == "no") return false
        println("Please answer yes or no.")
    }
}

fun main() {
    val score = askNumber("Enter a score from 0 to 100:", 0, 100, 3)
    if (score != null) {
        println("Accepted score: $score")
    } else {
        println("No valid score was entered.")
    }

    val keep = askYesNo("Save this score?")
    println("Save answer: $keep")
}
```

Expected Behavior or Output

With the following lines typed as standard input (piped input is not echoed, so it does not appear in the output):

```
abc
105
88
maybe
y
```

the program prints:

```
Enter a score from 0 to 100:
Enter a whole number from 0 to 100. You have 2 attempt(s) left.
Enter a score from 0 to 100:
Enter a whole number from 0 to 100. You have 1 attempt(s) left.
```

```
Enter a score from 0 to 100:
Accepted score: 88
Save this score? (yes or no)
Please answer yes or no.
Save this score? (yes or no)
Save answer: true
```

With no standard input at all, the program ends cleanly and prints:

```
Enter a score from 0 to 100:
No valid score was entered.
Save this score? (yes or no)
Save answer: null
```

Common Mistakes

1. Writing the message "Invalid" without saying what is valid.
2. Accepting a decimal such as 88.5 for a field that requires a whole number.
3. Looping forever on bad input instead of limiting attempts.

Activities

1. Run the sample input and read each message aloud.
2. Run with three invalid numbers and read the final message.
3. Add `askText(prompt)` that rejects empty answers.
4. Rewrite two messages from earlier lessons so they say what to do next.

Accessibility Practice

Check that every error message ends with an action the user can take, and that the same situation always produces the same words.

Skills Checklist

- ☐ I can write reusable validation functions.
- ☐ I limit retries.
- ☐ I handle the end of input.
- ☐ I write messages that say what to do.
- ☐ I keep wording consistent.

11.13 Unit 11: Capstone — Accessible Personal Data Manager

Overview

Build a personal data manager with the shared record schema, storage, search and completion, acceptance tests, and a documentation and accessibility check.

Lesson 11.1: Capstone: Requirements and Design

Reading References

- Data classes
- Collections overview
- Matching Python lesson: Lesson 11.1: Introduction to NiceGUI (capstone planning)

Learning Objectives

- Write requirements for a personal data manager.
- Define the shared record schema as a data class.
- Check records against the validation rules before building features.

Concept Explanation

The capstone is a *personal data manager*: a program that keeps a list of records, lets the user add, view, search, change, complete, and delete them, and saves them to a file. In the Kotlin track it is a text program, so it runs from the terminal and is fully usable with a screen reader or braille display.

Shared record schema (all general-purpose language tracks). Every record has `id` (unique whole number), `title`, `description`, `category`, `created_at`, and `updated_at` (ISO 8601 text). A project may add fields, such as an optional `done` flag, but must not remove these. Kotlin property names use camel case (`createdAt`), and the stored JSON uses the shared names.

Validation rules. A record must have a non-empty title and a non-empty category. Timestamps are ISO 8601 text such as `2026-01-15T09:00:00`.

Design before coding: list the operations, then the data, then the messages. The example checks the three shared sample records against the rules and prints them as a plain-text list.

Worked Example

```
data class Record(
    val id: Int,
```

```
    val title: String,
    val description: String,
    val category: String,
    val createdAt: String,
    val updatedAt: String,
)

fun validate(title: String?, category: String?): String? = when {
    title == null || title.isBlank() -> "A title is required."
    category == null || category.isBlank() -> "A category is required."
    else -> null
}

fun main() {
    val sample = listOf(
        Record(1, "Notebook", "Course notes", "supplies", "2026-01-15T09:00:00", "2026-01-15
        Record(2, "Pencil", "Box of 10", "supplies", "2026-01-15T09:05:00", "2026-01-15T09:0
        Record(3, "Study plan", "Weekly schedule", "planning", "2026-01-15T09:10:00", "2026-
    )

    for (record in sample) {
        val problem = validate(record.title, record.category)
        println("${record.id}. ${record.title} [${record.category}] - ${problem ?: "valid"}"
    }

    println(validate("  ", "supplies"))
    println(validate("Ruler", ""))
}
```

Expected Behavior or Output

The program prints:

```
1. Notebook [supplies] - valid
2. Pencil [supplies] - valid
3. Study plan [planning] - valid
A title is required.
A category is required.
```

Common Mistakes

1. Starting to build features before agreeing on the record fields.
2. Allowing a record with an empty title because the check was left for later.
3. Storing timestamps in different formats in different places.

Activities

1. Write the seven operations as a numbered list of requirements.
2. Add one project-specific property and say how it is validated.
3. Write three sample records for your own project.
4. Describe in plain sentences what each shared field means.

Accessibility Practice

Design the screen output as lines of plain text: one record per line, with the same field order each time. Decide the wording of every message now so that it can be tested.

Skills Checklist

☐ I can write requirements.
☐ I know the shared record fields.
☐ I can write validation rules.
☐ I can describe sample data.
☐ I can plan messages before coding.

Lesson 11.2: Capstone: Data Model and Storage

Reading References

- Data classes
- Exceptions
- Matching Python lesson: Lesson 11.2: Data Model and Storage

Learning Objectives

- Store records as data class objects.
- Save and load them as JSON.
- Confirm that every field survives a round trip.

Concept Explanation

Records are stored as a list of data class objects in memory and as a JSON array on disk. The storage layer has two functions: `saveRecords(path, records)` and `loadRecords(path)`. Loading returns a pair of the records and a message, so the caller decides what to do when the file is missing or damaged.

A `Record` is converted to a map for saving (`toMap`) and rebuilt from a decoded map when loading (`toRecord`), which checks that each field is present and has the right type. The optional `done` field defaults to `false`.

The example uses the teaching JSON helper from Lesson 8.5 (included so the program runs on its own). After saving and reloading it compares the lists;

data classes compare by value, so this *round-trip test* checks every field of every record. It is the first acceptance test for storage.

Worked Example

```
object Json {
    fun encode(value: Any?): String = when (value) {
        null -> "null"
        is String -> quote(value)
        is Boolean, is Int, is Long, is Double -> value.toString()
        is Map<*, *> -> value.entries.sortedBy { it.key.toString() }
            .joinToString(",", "{", "}") { quote(it.key.toString()) + ":" + encode(it.value)
        is List<*> -> value.joinToString(",", "[", "]") { encode(it) }
        else -> throw IllegalArgumentException("cannot encode ${value::class.simpleName}")
    }

    private fun quote(text: String): String {
        val out = StringBuilder("\"")
        for (c in text) {
            when {
                c == '"' -> out.append("\\\"")
                c == '\\' -> out.append("\\\\")
                c == '\n' -> out.append("\\n")
                c == '\r' -> out.append("\\r")
                c == '\t' -> out.append("\\t")
                c < ' ' -> out.append("\\u%04x".format(c.code))
                else -> out.append(c)
            }
        }
        return out.append('"').toString()
    }

    fun decode(text: String): Any? = Parser(text).parse()

    private class Parser(private val text: String) {
        private var pos = 0

        private fun fail(message: String): Nothing =
            throw IllegalArgumentException("$message at position $pos")

        private fun skip() {
            while (pos < text.length && text[pos].isWhitespace()) pos++
        }

        fun parse(): Any? {
            val result = value()
```

```
    skip()
    if (pos < text.length) fail("unexpected extra text")
    return result
}

private fun value(): Any? {
    skip()
    if (pos >= text.length) fail("unexpected end of text")
    val c = text[pos]
    return when {
        c == '{' -> obj()
        c == '[' -> list()
        c == '"' -> str()
        text.startsWith("true", pos) -> { pos += 4; true }
        text.startsWith("false", pos) -> { pos += 5; false }
        text.startsWith("null", pos) -> { pos += 4; null }
        c == '-' || c.isDigit() -> number()
        else -> fail("unexpected character")
    }
}

private fun number(): Any {
    val start = pos
    if (text[pos] == '-') pos++
    while (pos < text.length && (text[pos].isDigit() || text[pos] in ".eE+-")) pos++
    val token = text.substring(start, pos)
    return token.toLongOrNull() ?: token.toDoubleOrNull() ?: fail("bad number")
}

private fun str(): String {
    pos++
    val out = StringBuilder()
    while (true) {
        if (pos >= text.length) fail("unterminated string")
        val c = text[pos++]
        if (c == '"') return out.toString()
        if (c != '\\') { out.append(c); continue }
        if (pos >= text.length) fail("bad escape")
        when (text[pos++]) {
            'n' -> out.append('\n')
            't' -> out.append('\t')
            'r' -> out.append('\r')
            '"' -> out.append('"')
            '\\' -> out.append('\\')
            '/' -> out.append('/')
            'u' -> {
```

```
                        if (pos + 4 > text.length) fail("bad escape")
                        out.append(text.substring(pos, pos + 4).toInt(16).toChar())
                        pos += 4
                    }
                    else -> fail("bad escape")
                }
            }
        }

        private fun obj(): Map<String, Any?> {
            pos++
            val map = LinkedHashMap<String, Any?>()
            skip()
            if (pos < text.length && text[pos] == '}') { pos++; return map }
            while (true) {
                skip()
                if (pos >= text.length || text[pos] != '"') fail("expected a key")
                val key = str()
                skip()
                if (pos >= text.length || text[pos] != ':') fail("expected ':'")
                pos++
                map[key] = value()
                skip()
                if (pos >= text.length) fail("unterminated object")
                val d = text[pos++]
                if (d == '}') return map
                if (d != ',') { pos--; fail("expected ',' or '}'") }
            }
        }

        private fun list(): List<Any?> {
            pos++
            val items = ArrayList<Any?>()
            skip()
            if (pos < text.length && text[pos] == ']') { pos++; return items }
            while (true) {
                items.add(value())
                skip()
                if (pos >= text.length) fail("unterminated list")
                val d = text[pos++]
                if (d == ']') return items
                if (d != ',') { pos--; fail("expected ',' or ']'") }
            }
        }
    }
}
```

```
data class Record(
    val id: Int,
    val title: String,
    val description: String,
    val category: String,
    val createdAt: String,
    val updatedAt: String,
    val done: Boolean = false,
)

fun Record.toMap(): Map<String, Any?> = mapOf(
    "id" to id, "title" to title, "description" to description, "category" to category,
    "created_at" to createdAt, "updated_at" to updatedAt, "done" to done,
)

fun toRecord(entry: Any?): Record {
    val map = entry as? Map<*, *> ?: throw IllegalArgumentException("a record must be an ob
    fun text(key: String) = map[key] as? String ?: throw IllegalArgumentException("missing f
    val id = (map["id"] as? Long)?.toInt() ?: throw IllegalArgumentException("missing field
    return Record(id, text("title"), text("description"), text("category"),
        text("created_at"), text("updated_at"), map["done"] as? Boolean ?: false)
}

fun saveRecords(path: String, records: List<Record>) {
    java.io.File(path).writeText(Json.encode(records.map { it.toMap() }))
}

fun loadRecords(path: String): Pair<List<Record>, String?> {
    val file = java.io.File(path)
    if (!file.exists()) return Pair(emptyList(), "Could not open $path: file not found.")
    return try {
        Pair((Json.decode(file.readText()) as List<*>).map { toRecord(it) }, null)
    } catch (e: Exception) {
        Pair(emptyList(), "The data file is not valid: ${e.message}")
    }
}

fun main() {
    val sample = listOf(
        Record(1, "Notebook", "Course notes", "supplies", "2026-01-15T09:00:00", "2026-01-15
        Record(2, "Pencil", "Box of 10", "supplies", "2026-01-15T09:05:00", "2026-01-15T09:0
        Record(3, "Study plan", "Weekly schedule", "planning", "2026-01-15T09:10:00", "2026-
    )

    saveRecords("records.json", sample)
```

```
    val (loaded, problem) = loadRecords("records.json")
    println("Round trip preserved all fields: ${loaded == sample}")
    println("Records loaded: ${loaded.size}")
    println(problem)

    println(loadRecords("missing.json").second)

    java.io.File("broken.json").writeText("[{\"id\": 1,")
    println(loadRecords("broken.json").second)
}
```

Expected Behavior or Output

The program prints:

```
Round trip preserved all fields: true
Records loaded: 3
null
Could not open missing.json: file not found.
The data file is not valid: expected a key at position 10
```

Common Mistakes

1. Overwriting the data file with an empty list after a failed load.
2. Saving only some of the fields.
3. Assuming a loaded record has every field without checking.

Activities

1. Run the program and open `records.json` to read the saved text.
2. Add a field to `Record`, save it, and confirm that it survives the round trip.
3. Make `toRecord` reject a record whose title is blank.
4. Explain why `loadRecords` returns a message and does not throw.

Accessibility Practice

Messages that name the file and the problem read clearly in speech. After saving, read the file in the terminal to confirm it contains what you expect.

Skills Checklist

- ☐ I can store records in data class objects.
- ☐ I can save and load JSON.
- ☐ I can run a round-trip test.
- ☐ I handle missing and damaged files.
- ☐ I do not overwrite good data after a failed load.

Lesson 11.3: Capstone: Core Operations

Reading References

- Classes
- Collections overview
- Matching Python lesson: Lesson 11.3: Displaying and Managing Items

Learning Objectives

- Add, list, search, change, complete, and delete records.
- Assign new ids and update timestamps correctly.
- Return a message when an operation cannot be done.

Concept Explanation

A `Manager` class holds the records and offers one function per operation. Each function that can fail returns a pair of the result and a message, so the user interface can show the message and decide what to do. New ids are one more than the highest id, so ids are never reused while higher ids exist. Updating a record sets `updatedAt` but never changes `createdAt`, which is a `val`. Completing a record sets the optional `done` field to `true` and counts as an update.

For repeatable output the example passes in a *fake clock* that returns a new fixed time on each call. A real program passes a function that reads the real time.

The example adds the three shared sample records, lists them, searches for "note", changes one description, completes one record, deletes one, and tries to delete a record that does not exist.

Worked Example

```
data class Record(
    val id: Int,
    var title: String,
    var description: String,
    var category: String,
    val createdAt: String,
    var updatedAt: String,
    var done: Boolean = false,
)

fun validate(title: String?, category: String?): String? = when {
    title == null || title.isBlank() -> "A title is required."
    category == null || category.isBlank() -> "A category is required."
    else -> null
}
```

```
class Manager(private val clock: () -> String) {
    val records = mutableListOf<Record>()

    private fun nextId(): Int = (records.maxOfOrNull { it.id } ?: 0) + 1

    fun find(id: Int): Record? = records.firstOrNull { it.id == id }

    fun add(title: String, description: String, category: String): Pair<Record?, String?> {
        val problem = validate(title, category)
        if (problem != null) return Pair(null, problem)
        val stamp = clock()
        val record = Record(nextId(), title.trim(), description.trim(), category.trim(), sta
        records.add(record)
        return Pair(record, null)
    }

    fun update(
        id: Int,
        title: String? = null,
        description: String? = null,
        category: String? = null,
        done: Boolean? = null,
    ): Pair<Record?, String?> {
        val record = find(id) ?: return Pair(null, "No record has that id.")
        val problem = validate(title ?: record.title, category ?: record.category)
        if (problem != null) return Pair(null, problem)
        if (title != null) record.title = title.trim()
        if (description != null) record.description = description.trim()
        if (category != null) record.category = category.trim()
        if (done != null) record.done = done
        record.updatedAt = clock()
        return Pair(record, null)
    }

    fun complete(id: Int) = update(id, done = true)

    fun delete(id: Int): Pair<Boolean, String?> {
        val record = find(id) ?: return Pair(false, "No record has that id.")
        records.remove(record)
        return Pair(true, null)
    }

    fun search(text: String): List<Record> {
        val needle = text.trim().lowercase()
        return records.filter { it.title.lowercase().contains(needle) || it.description.lowe
```

```
    }
}

fun describe(record: Record): String =
    "${record.id}. ${record.title} [${record.category}] (${if (record.done) "done" else "not

fun makeClock(): () -> String {
    var ticks = 0
    return {
        ticks++
        "2026-01-15T10:00:%02d".format(ticks)
    }
}

fun show(manager: Manager) {
    for (record in manager.records) println(describe(record))
}

fun main() {
    val manager = Manager(makeClock())
    manager.add("Notebook", "Course notes", "supplies")
    manager.add("Pencil", "Box of 10", "supplies")
    manager.add("Study plan", "Weekly schedule", "planning")
    show(manager)

    println("Search for 'note':")
    for (record in manager.search("note")) println(record.title)

    manager.update(2, description = "Box of 12")
    manager.complete(3)
    println("${manager.find(2)?.description} ${manager.find(2)?.updatedAt}")
    println(manager.delete(1))
    println(manager.delete(99))
    show(manager)
}
```

Expected Behavior or Output

The program prints:

```
1. Notebook [supplies] (not done)
2. Pencil [supplies] (not done)
3. Study plan [planning] (not done)
Search for 'note':
Notebook
Box of 12 2026-01-15T10:00:04
```

```
(true, null)
(false, No record has that id.)
2. Pencil [supplies] (not done)
3. Study plan [planning] (done)
```

Common Mistakes

1. Reusing a deleted record's id by counting the list size instead of using the highest id.
2. Changing `createdAt` when updating a record (the property is a `val` to prevent this).
3. Throwing an exception for a missing record where a message to display is more helpful.

Activities

1. Predict the output before running.
2. Add `listByCategory(category)` to `Manager`.
3. Add a rule that blocks completing a record that is already complete, with a message.
4. Explain how the fake clock makes the output repeatable.

Accessibility Practice

List output uses the same fixed format for every record and the words `done` and `not done`. Search results print only titles, so each line is short.

Skills Checklist

- ☐ I can implement add, list, search, change, complete, and delete.
- ☐ I assign unique ids.
- ☐ I update timestamps correctly.
- ☐ I return clear messages for failures.
- ☐ I can make output repeatable with a fake clock.

Lesson 11.4: Capstone: Validation and Acceptance Tests

Reading References

- Exceptions
- Functions
- Matching Python lesson: Lesson 11.4: Testing and Refinement

Learning Objectives

- Run the shared acceptance tests.

- Verify that validation rejects bad data.
- Read and interpret test results.

Concept Explanation

The shared acceptance tests describe behavior that every language track's capstone must show:

1. Adding a record with an empty title is rejected with a clear message.
2. A new record receives an unused `id`.
3. Updating a record changes `updated_at` but not `created_at`.
4. Deleting a record removes only that record.
5. Saving and reloading preserves all fields and values (shown in Lesson 11.2).
6. Every message and control can be used without color or a mouse (checked by hand in Lesson 11.5).

This lesson automates tests 1 to 4 and also tests completion and the rejection of a blank title in an update. Each test builds a new `Manager`, so tests cannot affect each other. The fake clock keeps timestamps predictable so a test can check that `updatedAt` changed while `createdAt` did not.

Worked Example

```
data class Record(
    val id: Int,
    var title: String,
    var description: String,
    var category: String,
    val createdAt: String,
    var updatedAt: String,
    var done: Boolean = false,
)

fun validate(title: String?, category: String?): String? = when {
    title == null || title.isBlank() -> "A title is required."
    category == null || category.isBlank() -> "A category is required."
    else -> null
}

class Manager(private val clock: () -> String) {
    val records = mutableListOf<Record>()

    private fun nextId(): Int = (records.maxOfOrNull { it.id } ?: 0) + 1

    fun find(id: Int): Record? = records.firstOrNull { it.id == id }

    fun add(title: String, description: String, category: String): Pair<Record?, String?> {
```

```
        val problem = validate(title, category)
        if (problem != null) return Pair(null, problem)
        val stamp = clock()
        val record = Record(nextId(), title.trim(), description.trim(), category.trim(), sta
        records.add(record)
        return Pair(record, null)
    }

    fun update(
        id: Int,
        title: String? = null,
        description: String? = null,
        category: String? = null,
        done: Boolean? = null,
    ): Pair<Record?, String?> {
        val record = find(id) ?: return Pair(null, "No record has that id.")
        val problem = validate(title ?: record.title, category ?: record.category)
        if (problem != null) return Pair(null, problem)
        if (title != null) record.title = title.trim()
        if (description != null) record.description = description.trim()
        if (category != null) record.category = category.trim()
        if (done != null) record.done = done
        record.updatedAt = clock()
        return Pair(record, null)
    }

    fun complete(id: Int) = update(id, done = true)

    fun delete(id: Int): Pair<Boolean, String?> {
        val record = find(id) ?: return Pair(false, "No record has that id.")
        records.remove(record)
        return Pair(true, null)
    }

    fun search(text: String): List<Record> {
        val needle = text.trim().lowercase()
        return records.filter { it.title.lowercase().contains(needle) || it.description.lowe
    }
}

fun describe(record: Record): String =
    "${record.id}. ${record.title} [${record.category}] (${if (record.done) "done" else "not

fun makeClock(): () -> String {
    var ticks = 0
    return {
```

```
        ticks++
        "2026-01-15T10:00:%02d".format(ticks)
    }
}

var failures = 0

fun test(name: String, body: () -> Unit) {
    val result = runCatching(body)
    if (result.isSuccess) {
        println("PASS $name")
    } else {
        failures++
        println("FAIL $name: ${result.exceptionOrNull()?.message}")
    }
}

fun assertThat(condition: Boolean, message: String) {
    if (!condition) throw AssertionError(message)
}

fun fresh(): Manager {
    val manager = Manager(makeClock())
    manager.add("Notebook", "Course notes", "supplies")
    manager.add("Pencil", "Box of 10", "supplies")
    manager.add("Study plan", "Weekly schedule", "planning")
    return manager
}

fun main() {
    test("1. an empty title is rejected") {
        val manager = fresh()
        val (record, message) = manager.add("   ", "x", "supplies")
        assertThat(record == null && message == "A title is required.", "expected the title
        assertThat(manager.records.size == 3, "the list should be unchanged")
    }

    test("2. a new record gets an unused id") {
        val manager = fresh()
        manager.delete(2)
        val (record, _) = manager.add("Ruler", "30 cm", "supplies")
        assertThat(record?.id == 4, "expected id 4 but got ${record?.id}")
    }

    test("3. update changes updatedAt but not createdAt") {
        val manager = fresh()
```

```
    val before = manager.find(1)!!.copy()
    manager.update(1, description = "Lecture notes")
    val after = manager.find(1)!!
    assertThat(after.createdAt == before.createdAt, "createdAt changed")
    assertThat(after.updatedAt != before.updatedAt, "updatedAt did not change")
}

test("4. delete removes only that record") {
    val manager = fresh()
    manager.delete(2)
    val ids = manager.records.map { it.id }
    assertThat(ids == listOf(1, 3), "wrong records remain: $ids")
}

test("5. complete marks done and updates the record") {
    val manager = fresh()
    val (record, _) = manager.complete(3)
    assertThat(record?.done == true && record.updatedAt != record.createdAt, "completion
}

test("6. a blank title cannot be saved by update") {
    val manager = fresh()
    val (record, message) = manager.update(1, title = "")
    assertThat(record == null && message == "A title is required.", "update accepted a b
}

println(if (failures == 0) "All acceptance tests passed." else "$failures test(s) failed
}
```

Expected Behavior or Output

The program prints:

```
PASS 1. an empty title is rejected
PASS 2. a new record gets an unused id
PASS 3. update changes updatedAt but not createdAt
PASS 4. delete removes only that record
PASS 5. complete marks done and updates the record
PASS 6. a blank title cannot be saved by update
All acceptance tests passed.
```

Common Mistakes

1. Sharing one manager between tests, so one test's deletions change the next test.
2. Checking only that no exception occurred, and not checking the result.

3. Writing the expected message differently from the program, so a correct program fails the test.

Activities

1. Break `Manager.update` so it also changes `createdAt` (make the property a `var`) and watch test 3 fail. Restore it.
2. Add a test that searching ignores case.
3. Add a test that deleting an unknown id returns `false` and a message.
4. Explain why test 2 deletes record 2 before adding the new one.

Accessibility Practice

The test names begin with the acceptance-test number, so PASS and FAIL lines can be matched to the specification. Read only the first word of each line to hear the overall result quickly.

Skills Checklist

☐ I can run the shared acceptance tests.
☐ I can test validation failures.
☐ I can test timestamps with a fake clock.
☐ I can read PASS and FAIL lines.
☐ I use a fresh manager for every test.

Lesson 11.5: Capstone: Documentation and Presentation

Reading References

- Basic syntax
- Unit P, Lesson P.6: files, data formats, and the shared accessible workflow
- Matching Python lesson: Lesson 11.5: Final Submission

Learning Objectives

- Write a plain-text project summary.
- Report a count of records by category.
- Complete a manual accessibility check.

Concept Explanation

Documentation tells a future reader what the program does and how to run it. Write it as plain text or Markdown with short headings, numbered steps, and complete sentences. Include: the purpose of the program, the compile and run commands, the shared record fields, the commands a user can give, the accessibility checks you carried out, and what you would improve next.

The example generates a summary from the shared sample data. It counts records by category and prints the categories in alphabetical order so the output is stable. It then prints the manual accessibility checklist. The unchecked boxes `[ ]` are for *you* to complete by hand with your own access technology; the program cannot verify them.

Acceptance test 6 is a manual check: every message and control can be used without color or a mouse. Test it with the keyboard only, and with the screen reader or braille display you actually use. Record what you tested and its version; do not claim compatibility with software you did not test.

Worked Example

```
data class Record(val id: Int, val title: String, val category: String)

fun main() {
    val sample = listOf(
        Record(1, "Notebook", "supplies"),
        Record(2, "Pencil", "supplies"),
        Record(3, "Study plan", "planning"),
    )

    val counts = sample.groupingBy { it.category }.eachCount().toSortedMap()

    println("Personal Data Manager - Summary")
    println("Records: ${sample.size}")
    println("Categories:")
    for ((category, count) in counts) {
        println("- $category: $count")
    }

    println()
    println("Manual accessibility checklist (complete this by hand):")
    val checks = listOf(
        "[ ] Every command can be used with the keyboard only.",
        "[ ] Every message is a complete sentence that says what to do next.",
        "[ ] No meaning depends on color, position, or symbols alone.",
        "[ ] The output was read with my own screen reader or braille display.",
        "[ ] I recorded the operating system, program, and version I tested.",
    )
    for (check in checks) println(check)
}
```

Expected Behavior or Output

The program prints:

```
Personal Data Manager - Summary
Records: 3
Categories:
- planning: 1
- supplies: 2

Manual accessibility checklist (complete this by hand):
[ ] Every command can be used with the keyboard only.
[ ] Every message is a complete sentence that says what to do next.
[ ] No meaning depends on color, position, or symbols alone.
[ ] The output was read with my own screen reader or braille display.
[ ] I recorded the operating system, program, and version I tested.
```

Common Mistakes

1. Writing documentation that only the author can follow.
2. Claiming that a program is accessible without recording what was tested and how.
3. Letting the documentation drift from the program after later changes.

Activities

1. Write the project summary for your own capstone using the headings above.
2. Run the keyboard-only check and record each problem you find.
3. Ask a classmate to follow your run instructions from a fresh folder and note where they get stuck.
4. Write three improvements for the next version, ordered by user benefit.

Accessibility Practice

This is the final accessibility lesson. Read your own documentation aloud from start to finish. Anything you have to look at the screen to understand needs rewriting.

Skills Checklist

- ☐ I can write a plain-text project summary.
- ☐ I can report counts by category.
- ☐ I can run a manual accessibility check.
- ☐ I record what I tested.
- ☐ I do not claim untested compatibility.

AI Policy

AI may be used only for personal conceptual understanding and reinforcement. It must not write, complete, autocomplete, repair, or generate course code or assessment solutions.

Part XII

LUA

Chapter 12

Lua Curriculum

Lua is a lightweight, embeddable scripting language used in game scripting, application extension, configuration, automation, and embedded environments. This track teaches general programming concepts while making Lua's table-centric data model and compact runtime explicit.

Before you start

Complete the Computing Foundations Primer and the Terminal Skills prerequisite. The course is designed for beginners and does not assume prior programming experience.

Course sequence

- Unit 0: interpreter and toolchain concepts, workspace, accessible editor workflow, and project dependencies.
- Unit 1: computing foundations and first Lua programs.
- Units 2–4: values, numeric computation, input/output, control flow, and tables.
- Units 5–8: iteration, functions, modularity, object patterns, files, and errors.
- Units 9–11: testing, data processing, projects, and capstone work.

See the complete Lua lessons, the Lua topic index, and the existing Lua course overview and toolchain notes.

Lua language concepts

Lua is dynamically typed. Tables can represent sequences, maps, records, and objects. Sequence indexing conventionally begins at 1. ipairs iterates sequence entries; pairs iterates key/value pairs without guaranteeing traversal order. Only nil and false are falsey. Functions are values and can return multiple results. Metatables support customized lookup and operator behavior.

File operations return values that should be checked. JSON and CSV support are provided through external packages rather than the core standard library. The course identifies external dependencies where used.

Toolchain baseline

The examples target Lua 5.4 and the standard library. LuaRocks is introduced as an optional package manager, not a prerequisite for the core course. Refer to the Lua overview for the validation command and current verification notes.

Accessibility

Use keyboard-first editing and terminal workflows. Verify screen-reader output for punctuation, nesting, diagnostics, paths, and program output. The core course should not depend on visual-only game editors or graphical tools.

Recommended introductory books

- Burton, Brian. *Learning Lua: A Guide for Getting Started*. Burtons Media Group, 2023. A beginner-oriented introduction to Lua scripting, with examples relevant to embedded applications and game scripting. Publisher page.
- Ierusalimschy, Roberto. *Programming in Lua*, 4th edition. Lua.org, 2016. The authoritative book by Lua's chief architect, covering the language and its C API. It assumes general programming experience, though it does not assume prior Lua knowledge. Official book page.

Suggested use: *Learning Lua* is the more direct first book for this track. Use *Programming in Lua* as the detailed language reference once basic programming concepts are familiar. Its fourth edition targets Lua 5.3; consult the current Lua 5.4 reference manual for version-specific differences.

12.1 LUA Lessons

The curriculum is organized as one Markdown file per unit.

- accessible environment and toolchain

- foundations of computing
- numeric computations
- control flow decisions and loops
- data structures lists and dictionaries
- user input and while loops
- functions
- classes and object oriented programming
- files and exceptions
- testing and data analysis
- project based learning text based applications
- capstone accessible personal data manager

12.2 Unit 0: Accessible Environment and Toolchain

Course overview

Course alignment and implementation notes

This course follows the Python canonical sequence in `lesson_standardization.json`. The shared problem contexts, inputs, records, boundaries, and expected results are preserved; Lua syntax and idioms are taught directly rather than transliterated from Python. Lua tables represent sequences, maps, and records; sequences are one-indexed; only `false` and `nil` are false; `pairs` does not guarantee order; `ipairs` traverses a consecutive sequence. Lua has no built-in class, tuple, continue statement, or JSON library. These differences are taught explicitly.

Runtime: Lua 5.4. Run a file with `lua5.4 filename.lua`; syntax-check with `luac5.4 -p filename.lua`. Use local variables by default. The Lua standard library does not include a JSON or general CSV parser; capstone persistence must declare and document an approved dependency (for example, dkjson), and CSV parsing must use a tested parser when quoted fields are in scope.

Accessibility and lesson conventions

Use the terminal and keyboard for every exercise. Read code and output sequentially; describe braces, brackets, quotes, punctuation, and indentation in prose. Do not communicate status through color alone. Provide explicit prompts, errors, confirmations, and textual equivalents for any visual output. The course does not claim screen-reader or braille verification until those workflows have been tested on named platforms.

Each lesson includes a focused explanation, runnable worked example, expected behavior, common mistakes, practice, accessibility practice, and a skills check. Save each fenced example to a separate file or copy it into the Lua REPL.

Overview

Install Lua 5.4, create a project folder, set up a keyboard-first editor workflow, and learn how Lua finds modules. No programming experience is needed beyond Unit P.

Lesson 0.1: Install and Verify Lua

Reading References

- Programming in Lua, 4th edition (Roberto Ierusalimschy): Chapter 1 (Getting Started)
- Lua 5.4 Reference Manual: Section 7 (Lua Standalone)
- Matching Python lesson: Lesson 0.1 Installing Python and uv

Learning Objectives

- Install Lua 5.4 using a trusted source for your operating system.
- Verify the installed version from the terminal.
- Run a one-line program with the interpreter before creating any file.

Concept Explanation

Lua is a small interpreted language. The program that reads and runs Lua source is called the *interpreter*. Installing Lua therefore means installing one command-line program. This course targets **Lua 5.4**; features such as integer and float number subtypes, `//` integer division, and `<close>` variables belong to Lua 5.3 and 5.4, so an older 5.1 or 5.2 interpreter will not run every example.

Choose one installation route and write down which one you used:

- Debian or Ubuntu Linux: the package manager provides `lua5.4`, and the command is `lua5.4`.
- macOS: a package manager such as Homebrew provides a `lua` formula. After installing, check the version, because the command is usually named `lua`.
- Windows: use a Lua 5.4 binary distribution or a package manager you already trust, then confirm the command name and that it is on your PATH.
- Any system: Lua can be built from the source archive at lua.org if you prefer.

The command name differs by system (`lua5.4` or `lua`). This course writes `lua5.4`; substitute your own command everywhere. Always confirm the version with `-v` rather than assuming, because a computer can have an old Lua installed alongside a new one.

The `-e` option runs a statement given on the command line, which proves the interpreter works before you create a project.

Worked Example

```
lua5.4 -v
lua5.4 -e "print('Hello from Lua')"
```

Expected Behavior or Output

Expected output (the patch number and year depend on the version you installed; this was observed with Lua 5.4.6):

```
Lua 5.4.6  Copyright (C) 1994-2023 Lua.org, PUC-Rio
Hello from Lua
```

Common Mistakes

1. Installing a package named `lua` and assuming it is Lua 5.4. Run the version command to confirm.
2. Mixing up the command name: typing `lua` when the installed command is `lua5.4`, then concluding the install failed.
3. Running a Lua statement in a shell without noticing that the shell's own quoting rules changed the quotation marks.

Activities

1. Install Lua and record the exact output of the version command.
2. Run `lua5.4 -e "print(1 + 1)"` and explain each part of the command in your own words.
3. Find out which command name your system uses and write it on the first line of your course notes file.
4. Explain the difference between the interpreter and the source file it reads.

Accessibility Practice

Run each command from the keyboard only. After pressing Enter, use your screen reader's review commands to read the output line by line, and confirm that the version number is announced. If you use a braille display, check that the quotation marks in the second command are shown.

Skills Checklist

- ☐ I can install Lua 5.4 and say which route I used.
- ☐ I can run the version command and read the result.
- ☐ I know which command name my computer uses.
- ☐ I can run a one-line Lua statement from the terminal.
- ☐ I can say what the interpreter does.

Lesson 0.2: Workspace and First Run

Reading References

- Programming in Lua, 4th edition (Roberto Ierusalimschy): Chapter 1 (Getting Started)
- Lua 5.4 Reference Manual: Section 7 (Lua Standalone)
- Matching Python lesson: Lesson 0.2 Installing and configuring VS Code or Theia IDE

Learning Objectives

- Create a course folder with a first source file.
- Run the file from the terminal and read the result.
- Read a Lua error message and find the file name and line number.

Concept Explanation

A *project folder* keeps one program's files together. Create a folder named `lua-course`, and inside it save a file named `main.lua`. Lua source files conventionally end in `.lua`. From the terminal, change into the folder (this is your *working directory*) and run the file by giving its name to the interpreter: `lua5.4 main.lua`.

Lua runs the file from top to bottom as one unit called a *chunk*. `print` writes its arguments to the terminal and ends with a new line. The global variable `_VERSION` holds the language version as text, which is a quick way to confirm that the file ran under Lua 5.4.

If a file contains a mistake, the interpreter prints a message of the form `lua5.4: main.lua:3: ...`. Read it in three parts: the program name, the file name and line number (`main.lua:3`), and a description of the problem. Always go to that line first.

To check syntax without running a program, use the compiler front end: `luac5.4 -p main.lua`. It prints nothing when the file is syntactically valid.

Worked Example

```lua
-- main.lua: the first program in the course project folder
print("Hello, Lua!")
print(_VERSION)
```

Expected Behavior or Output

The program prints:

```
Hello, Lua!
Lua 5.4
```

Common Mistakes

1. Running the command from the wrong folder, so the interpreter reports that it cannot open `main.lua`.
2. Saving the file with a hidden extra extension such as `main.lua.txt`, so the file name does not match.
3. Ignoring the line number in an error message and re-reading the whole file instead.

Activities

1. Create the folder and file, then run them. Record the working directory shown by `pwd` (macOS and Linux) or `cd` (Windows).
2. Remove the closing quotation mark on line 2, run the file, and write down the file name, line number, and description in the error message. Then restore the file.
3. Run `luac5.4 -p main.lua` before and after the mistake and compare what is printed.
4. Add a third line that prints your own name, and predict the output before running it.

Accessibility Practice

Use a keyboard-only route to create the folder and file (terminal commands or your editor's keyboard commands). Read your error message with the screen reader's line-by-line review, and find the line number without relying on a mouse or on color.

Skills Checklist

☐ I can create a project folder and a `.lua` file.
☐ I can run the file from the terminal.
☐ I can say what the working directory is.
☐ I can read the file name and line number in an error.
☐ I can check syntax with `luac5.4 -p`.

Lesson 0.3: Editor and Accessibility Workflow

Reading References

- Programming in Lua, 4th edition (Roberto Ierusalimschy): Chapter 1 (Getting Started)
- This repository: IDE setup lesson
- Matching Python lesson: Lesson 0.3 Editor extensions and project workspace

Learning Objectives

- Configure an editor so Lua source can be read by speech or braille.
- Choose a punctuation announcement level that distinguishes brackets, braces, and quotation marks.
- Practice navigating a Lua file by line, word, and character.

Concept Explanation

Complete the shared IDE setup lesson first. Then adapt the editor for Lua:

- Install only a Lua language extension from a source you trust, such as a Lua language server, and read what permissions it requests. An extension is optional; the course works with a plain text editor and the terminal.
- Set your screen reader's punctuation level high enough to hear the symbols that carry meaning in Lua: braces `{ }` that surround tables, square brackets `[ ]` for indexing, the length operator `#`, the concatenation operator `..`, the not-equal operator `~=`, and the comment marker `--`.
- Lua does not use indentation to decide structure. Blocks end with the keyword `end`, so each `if`, `for`, `while`, and `function` needs a matching `end`. Indentation is for people, and your editor's indentation guides or announcements still help you count nesting levels.

Practise reading the example below aloud, saying each symbol: "local scores equals open brace eighty-two comma ninety-one comma seventy-six comma eighty-eight close brace." Then say what `#scores` and `scores[1]` mean.

Worked Example

```
local scores = {82, 91, 76, 88}
print(string.format("count=%d first=%d last=%d", #scores, scores[1], scores[#scores]))
```

Expected Behavior or Output

The program prints:

```
count=4 first=82 last=88
```

Common Mistakes

1. Setting punctuation announcement so low that `{`, `}`, and `..` are never spoken, then misreading a table or a string join.
2. Forgetting that a Lua block closes with `end`, and relying on indentation alone.
3. Installing several overlapping extensions and getting duplicate, confusing diagnostics.

Activities

1. Test your editor with the example. Record the punctuation level at which every symbol in line 1 is announced.
2. Navigate the file by line, then by word, then by character, and note the keyboard commands you used.
3. Introduce a missing `end` in a small `if` block, save, and find how your editor reports the problem. Then fix it.
4. Write a half-page note describing your edit, run, and fix cycle using only the keyboard.

Accessibility Practice

This whole lesson is accessibility practice. Verify that you can open the file, move to a given line number, read a line character by character, run the program in the terminal, and read the output. Do not describe this workflow as screen-reader tested for others; record only what you tested yourself, with the names of your operating system, editor, and access technology.

Skills Checklist

- ☐ I can configure my editor for keyboard-only use.
- ☐ I can hear or feel every symbol in a Lua table and string join.
- ☐ I can explain why Lua blocks end with `end`.
- ☐ I can navigate a file by line, word, and character.
- ☐ I can run and read a program entirely from the keyboard.

Lesson 0.4: Project Environments, Dependencies, and Terminal Usage

Reading References

- Programming in Lua, 4th edition (Roberto Ierusalimschy): Chapter 17 (Modules and Packages)
- Lua 5.4 Reference Manual: Section 6.3 (Modules)
- Matching Python lesson: Lesson 0.4 Python environments and terminal usage

Learning Objectives

- Keep each project in its own folder and run commands from its root.
- Load code from a second file with `require` and explain `package.path`.
- Explain what LuaRocks is and when it is needed.

Concept Explanation

Run commands from the *project root*, the folder that holds your main file. Keep different projects in different folders so their files never mix.

A Lua *module* is a file that returns a value, usually a table of functions. The function `require("name")` finds the file `name.lua`, runs it once, and returns what it returned. Lua searches the places listed in `package.path`, a text value where each template uses `?` as a stand-in for the module name. Adding `./?.lua;` to the front tells Lua to look in the current folder first.

The standard library is enough for this course. **LuaRocks** is Lua's package manager for external packages; it records a package in a *rockspec* file and can install either system-wide or into a local tree. Use it only when a project needs an external package, and prefer a local tree so a project does not change the whole computer.

The example below is self-contained: it writes a tiny module file, then loads it with `require`, so the same lesson works in an empty folder.

Worked Example

```
-- Create a small module file, then load it with require.
local file = assert(io.open("pricing.lua", "w"))
file:write([[
local M = {}
function M.line_total(quantity, unit_price)
  return quantity * unit_price
end
return M
]])
file:close()

package.path = "./?.lua;" .. package.path
local pricing = require("pricing")
print(pricing.line_total(3, 2.50))
```

Expected Behavior or Output

The program prints:

```
7.5
```

Common Mistakes

1. Running a program from a different folder, so `require` cannot find a module in the current folder.
2. Forgetting that a module file must `return` its table; without it `require` returns `true` and calls such as `pricing.line_total` fail.
3. Installing every package system-wide instead of keeping dependencies with the project.

Activities

1. Run the example, then open the new `pricing.lua` file and read it aloud, line by line.
2. Add a second function, `line_value_with_discount`, to the module and call it from the main file.
3. Run the program from a different folder and record the error message. Explain why it happened.
4. Print `package.path` and write down, in your own words, what the first template means.

Accessibility Practice

Keep one project per folder and begin every session by confirming your working directory with a command and reading the result. Read the long text in `package.path` with character navigation, noting where each semicolon separates one template from the next.

Skills Checklist

- ☐ I can identify the project root and run commands from it.
- ☐ I can load a module with `require`.
- ☐ I can explain what `package.path` contains.
- ☐ I can say when LuaRocks is needed.
- ☐ I can run a program from the terminal and read its output.

12.3 Unit 1: Foundations of Computing

Overview

Turn a written algorithm into Lua, run a first program, and learn variables, values, and Lua's dynamic types. The shared student profile appears here for the first time.

Lesson 1.1: Computer Science and Problem Solving

Reading References

- Programming in Lua, 4th edition (Roberto Ierusalimschy): Chapter 1 (Getting Started)
- Zelle: Chapter 1, pages 1-7 (shared with the Python track)
- Matching Python lesson: Lesson 1.1 Welcome to Computer Science

Learning Objectives

- Explain hardware, software, and algorithms in your own words.
- Turn a written algorithm into a short Lua program.

- Predict a program's output by hand before running it.

Concept Explanation

Computers follow instructions exactly. *Hardware* is the physical machine; *software* is the set of instructions it follows; an *algorithm* is an ordered, finite list of clear steps that solves a problem. The foundations unit of this repository (Unit P) covers these ideas in detail.

Computer science is mainly about solving problems, and programming is how a solution is carried out. A reliable approach is to understand the problem, break it into small steps, write the algorithm in plain language, translate it into code, and test it.

Here the algorithm is: "set a total to zero; for each score, add it to the total; divide the total by the number of scores; report both." Lua's `local` keyword creates a variable that exists only in its block, `for ... in ipairs(...)` visits a list in order, and `#` gives the length of a list.

The scores 82, 91, 76, and 88 are the course's shared data. Their total is 337 and their average is 84.25. A hand trace should reach the same numbers before the program is run.

Worked Example

```lua
-- Algorithm: total and average of the shared scores.
local scores = {82, 91, 76, 88}
local total = 0
for _, score in ipairs(scores) do
  total = total + score
end
local average = total / #scores
print(string.format("Total: %d", total))
print(string.format("Average: %.2f", average))
```

Expected Behavior or Output

The program prints:

```
Total: 337
Average: 84.25
```

Common Mistakes

1. Starting to code before writing the steps, then forgetting the step that divides by the number of scores.
2. Writing `local total` without giving it a value, so the first addition fails on `nil`.

3. Using `/` and expecting a whole number. In Lua 5.4, `/` always produces a float.

Activities

1. Trace the program by hand, writing the running total after each score: 82, 173, 249, 337.
2. Change the list to {90, 80} and predict the total and average before running.
3. Add a line that reports the highest score using a second loop.
4. Explain, in two sentences, the difference between the algorithm and the Lua program.

Accessibility Practice

Write your hand trace as numbered sentences, one per loop step, rather than as a table, and read it back with your screen reader. Compare it with the program's printed lines in order.

Skills Checklist

☐ I can define hardware, software, and algorithm.
☐ I can describe an algorithm as ordered steps.
☐ I can trace a loop by hand.
☐ I can read a short Lua program aloud.
☐ I can run it and compare the output with my trace.

Lesson 1.2: Your First Lua Program

Reading References

- Programming in Lua, 4th edition (Roberto Ierusalimschy): Chapter 1 (Getting Started)
- Lua 5.4 Reference Manual: Section 3.1 (Blocks) and Section 3.3.5 (For Statement)
- Matching Python lesson: Lesson 1.2 Your First Python Program - Hello World

Learning Objectives

- Write and run a program that prints text.
- Use comments to document code.
- Explain that Lua executes statements in order, top to bottom.

Concept Explanation

A Lua program is a sequence of *statements* that run in order. `print` is a built-in function that writes its arguments to the terminal. Text in quotation marks is

a *string*. Single or double quotes both work, and a string written with double square brackets, `[[ ... ]]`, can span several lines.

A line comment starts with `--` and runs to the end of the line; Lua ignores it. A long comment starts with `--[[` and ends with `]]`.

`print` separates multiple arguments with a tab character. `io.write` writes exactly what you give it with nothing added, so a new line must be written as the escape sequence `\n`. Knowing the difference matters because a screen reader may read a tab as a pause, and because exact output is easier to test.

Worked Example

```
-- This program prints a greeting for the shared student.
print("Hello, Avery!")
print("Welcome to Lua programming!")
io.write("No new line here, ")
io.write("then one here.\n")
print("a", "b", "c")
```

Expected Behavior or Output

The program prints:

```
Hello, Avery!
Welcome to Lua programming!
No new line here, then one here.
a   b   c
```

Common Mistakes

1. Using a curly quotation mark pasted from a document instead of a straight quotation mark, which causes a syntax error.
2. Expecting `io.write` to add a new line the way `print` does.
3. Writing `print 'Hello'` with a missing space or quotation mark and misreading the resulting error.

Activities

1. Run the program and describe in words how the last line differs from the line before it.
2. Change the greeting to use your own name and predict the output.
3. Write a program that prints three lines: your name, the course name, and today's task.
4. Add a comment above each line and read the file aloud with comments included.

Accessibility Practice

Run the file from the terminal and read the output in order. Note that `a`, `b`, and `c` on the last line are separated by tab characters; check whether your screen reader announces them.

Skills Checklist

- ☐ I can write and run a program that prints text.
- ☐ I can write a line comment.
- ☐ I can explain how `print` and `io.write` differ.
- ☐ I can say that statements run in order.
- ☐ I can read a first error message.

Lesson 1.3: Variables, Values, and Dynamic Types

Reading References

- Programming in Lua, 4th edition (Roberto Ierusalimschy): Chapter 3 (Numbers), Chapter 4 (Strings)
- Lua 5.4 Reference Manual: Section 2.1 (Values and Types)
- Matching Python lesson: Lesson 1.3 Variables and Simple Data Types

Learning Objectives

- Create local variables and assign values.
- Name the basic Lua types and check a value's type.
- Explain `nil` and the difference between local and global variables.

Concept Explanation

A *variable* is a name that refers to a value. In Lua, variables do not have types; *values* do. The same variable may refer to a number now and a string later. The basic types are `nil`, `boolean`, `number`, `string`, `table`, and `function`. The function `type(value)` returns the type name as a string.

Lua 5.4 numbers have two subtypes: *integer* (whole numbers such as 16) and *float* (numbers with a decimal point such as 1.72). `math.type(value)` reports `integer` or `float`.

Always declare variables with `local`. A variable assigned without `local` becomes a *global* variable visible to the whole program, which is a common source of hard-to-find bugs. A variable that has never been assigned has the value `nil`, which means "no value."

The shared student profile is name Avery, age 16, height 1.72 meters, and favorite subject Computer Science. The shared inventory item Notebook has quantity 3 and unit price 2.50, so three notebooks cost 7.5.

Worked Example

```lua
local name = "Avery"
local age = 16
local height = 1.72
local favorite_subject = "Computer Science"

print(name, age, height, favorite_subject)
print(type(name), type(age), type(height), type(favorite_subject))
print(math.type(age), math.type(height))

local notebook_price = 2.50
local notebook_quantity = 3
print("Notebook total:", notebook_price * notebook_quantity)

local missing
print(missing, type(missing))
```

Expected Behavior or Output

The program prints:

```
Avery    16  1.72    Computer Science
string  number  number  string
integer float
Notebook total: 7.5
nil nil
```

Common Mistakes

1. Forgetting `local`, which silently creates a global variable that other code can change.
2. Expecting Lua to report an error for an unassigned variable. It simply has the value `nil`.
3. Confusing `=` (assignment) with `==` (comparison).

Activities

1. Run the example and write down each type name in the order it was printed.
2. Change `age` to 16.0 and predict what `math.type(age)` prints.
3. Add a boolean variable `likes_cs` set to `true` and print its type.
4. Explain, without code, why `type(nil)` is the string `nil` while a missing variable's value is the value `nil`.

Accessibility Practice

Read the variable names aloud as part of each line, for example 'local favorite underscore subject'. Check that your screen reader pauses at the underscore so you can tell `favorite_subject` from two words.

Skills Checklist

- ☐ I can declare local variables and assign values.
- ☐ I can name Lua's basic types.
- ☐ I can use `type` and `math.type`.
- ☐ I can explain `nil`.
- ☐ I can explain why `local` matters.

12.4 Unit 2: Numeric Computations

Overview

This unit develops the concepts in the same order and with the same problem context as the Python reference. Predict the output before running each example. Use Lua 5.4 semantics, not assumptions imported from Python.

Lesson 2.1 The Software Development Process

Reading References

- Lua 5.4 Reference Manual: https://www.lua.org/manual/5.4/
- Programming in Lua: https://www.lua.org/pil/
- Corresponding Python lesson in the canonical course map.

Learning Objectives

- Explain the concept using Lua terminology.
- Trace the worked example and predict its behavior.
- Modify and test the example while preserving the shared data contract.

Concept Explanation

Plan, implement, and test the same tip-calculator problem. Separate input values from the calculation and presentation.

Worked Example

```lua
local bill_cents = 2450
local tip_rate_percent = 15
-- Integer arithmetic avoids binary floating-point rounding errors.
local tip_cents = (bill_cents * tip_rate_percent + 50) // 100
local total_cents = bill_cents + tip_cents
```

```lua
print(string.format("Tip: $%.2f", tip_cents / 100))
print(string.format("Total: $%.2f", total_cents / 100))
```

Expected Behavior or Output

```
Tip: $3.68
Total: $28.18
```
```8.18
```

Common Mistakes

- Using binary floating-point display rounding as a substitute for a currency-rounding policy.
- Skipping parentheses when operator precedence is not obvious.
- Assuming tonumber succeeds for every string.

Activities

1. Calculate the shared $24.50 bill and 15% tip using integer cents and half-up rounding.
2. Trace each arithmetic intermediate and compare the displayed tip and total with $3.68 and $28.18.
3. Try a nonnumeric string and handle a nil conversion result.
4. Use coordinates (3, 4) to calculate distance 5 and explain the math-library call.

Accessibility Practice

Navigate by line, word, and character; identify delimiters and nesting verbally; execute from the terminal; and read the output and any error messages sequentially. Do not rely on syntax highlighting or color to understand the example.

Skills Checklist

☐ I can explain the concept in Lua terminology.
☐ I can trace and predict the example's behavior.
☐ I can run and modify the example from the terminal.
☐ I can test a boundary or invalid case.
☐ I can explain the relevant Lua-specific semantic difference.

Lesson 2.2 Expressions and Assignment

Reading References

- Lua 5.4 Reference Manual: https://www.lua.org/manual/5.4/
- Programming in Lua: https://www.lua.org/pil/

- Corresponding Python lesson in the canonical course map.

Learning Objectives

- Explain the concept using Lua terminology.
- Trace the worked example and predict its behavior.
- Modify and test the example while preserving the shared data contract.

Concept Explanation

Lua uses arithmetic operators and parentheses for precedence. Assignment updates a variable; it is not an equality test.

Worked Example

```lua
local score = 50
score = score + 20
local expression = 10 / 2 + 3
print("Score:", score)
print("Expression:", expression)
```

Expected Behavior or Output

```
Score: 70
Expression: 8.0
```

Common Mistakes

- Using binary floating-point display rounding as a substitute for a currency-rounding policy.
- Skipping parentheses when operator precedence is not obvious.
- Assuming tonumber succeeds for every string.

Activities

1. Calculate the shared $24.50 bill and 15% tip using integer cents and half-up rounding.
2. Trace each arithmetic intermediate and compare the displayed tip and total with $3.68 and $28.18.
3. Try a nonnumeric string and handle a nil conversion result.
4. Use coordinates (3, 4) to calculate distance 5 and explain the math-library call.

Accessibility Practice

Navigate by line, word, and character; identify delimiters and nesting verbally; execute from the terminal; and read the output and any error messages sequentially. Do not rely on syntax highlighting or color to understand the example.

Skills Checklist

- ☐ I can explain the concept in Lua terminology.
- ☐ I can trace and predict the example's behavior.
- ☐ I can run and modify the example from the terminal.
- ☐ I can test a boundary or invalid case.
- ☐ I can explain the relevant Lua-specific semantic difference.

Lesson 2.3 Input and Output Revisited

Reading References

- Lua 5.4 Reference Manual: https://www.lua.org/manual/5.4/
- Programming in Lua: https://www.lua.org/pil/
- Corresponding Python lesson in the canonical course map.

Learning Objectives

- Explain the concept using Lua terminology.
- Trace the worked example and predict its behavior.
- Modify and test the example while preserving the shared data contract.

Concept Explanation

io.read returns text or nil at end-of-file. Convert numeric text with tonumber and validate the conversion.

Worked Example

```lua
io.write("Enter your name: ")
local name = io.read("*l")
if name == nil or name == "" then name = "Avery" end
print("Hello, " .. name .. "!")
```

Expected Behavior or Output

```
Enter your name: Hello, Avery!
```

Common Mistakes

- Assuming io.read always returns a non-empty string; it can return nil at end-of-file.
- Calling tonumber and immediately doing arithmetic without checking for nil.
- Treating terminal input as already numeric.

Activities

1. Run the prompt with a valid numeric string and record the converted Lua type.
2. Run it with alphabetic input and confirm that the program prints a useful error rather than crashing.
3. Simulate end-of-file and handle the nil result explicitly.
4. Explain the difference between the raw string and the converted number.

Accessibility Practice

Navigate by line, word, and character; identify delimiters and nesting verbally; execute from the terminal; and read the output and any error messages sequentially. Do not rely on syntax highlighting or color to understand the example.

Skills Checklist

☐ I can explain the concept in Lua terminology.
☐ I can trace and predict the example's behavior.
☐ I can run and modify the example from the terminal.
☐ I can test a boundary or invalid case.
☐ I can explain the relevant Lua-specific semantic difference.

Lesson 2.4 Computing with Numbers and Type Conversions

Reading References

- Lua 5.4 Reference Manual: https://www.lua.org/manual/5.4/
- Programming in Lua: https://www.lua.org/pil/
- Corresponding Python lesson in the canonical course map.

Learning Objectives

- Explain the concept using Lua terminology.
- Trace the worked example and predict its behavior.
- Modify and test the example while preserving the shared data contract.

Concept Explanation

tonumber converts valid numeric text and returns nil for invalid text. tostring converts values to text.

Worked Example

```lua
local raw = "16"
local age = tonumber(raw)
```

```lua
if age then
  print(age + 1)
else
  print("Enter a valid number.")
end
print(tostring(16) .. " years")
```

Expected Behavior or Output

```
17
16 years
```

Common Mistakes

- Using binary floating-point display rounding as a substitute for a currency-rounding policy.
- Skipping parentheses when operator precedence is not obvious.
- Assuming tonumber succeeds for every string.

Activities

1. Calculate the shared $24.50 bill and 15% tip using integer cents and half-up rounding.
2. Trace each arithmetic intermediate and compare the displayed tip and total with $3.68 and $28.18.
3. Try a nonnumeric string and handle a nil conversion result.
4. Use coordinates (3, 4) to calculate distance 5 and explain the math-library call.

Accessibility Practice

Navigate by line, word, and character; identify delimiters and nesting verbally; execute from the terminal; and read the output and any error messages sequentially. Do not rely on syntax highlighting or color to understand the example.

Skills Checklist

☐ I can explain the concept in Lua terminology.
☐ I can trace and predict the example's behavior.
☐ I can run and modify the example from the terminal.
☐ I can test a boundary or invalid case.
☐ I can explain the relevant Lua-specific semantic difference.

Lesson 2.5 Using the Math Library

Reading References

- Lua 5.4 Reference Manual: https://www.lua.org/manual/5.4/
- Programming in Lua: https://www.lua.org/pil/
- Corresponding Python lesson in the canonical course map.

Learning Objectives

- Explain the concept using Lua terminology.
- Trace the worked example and predict its behavior.
- Modify and test the example while preserving the shared data contract.

Concept Explanation

The math library supplies sqrt and pi. Use the canonical coordinates (3, 4) to calculate distance from the origin.

Worked Example

```lua
local x, y = 3, 4
local distance = math.sqrt(x * x + y * y)
local radius = 3
local circumference = 2 * math.pi * radius
print(distance)
print(string.format("%.2f", circumference))
```

Expected Behavior or Output

```
5.0
18.85
```

Common Mistakes

- Using binary floating-point display rounding as a substitute for a currency-rounding policy.
- Skipping parentheses when operator precedence is not obvious.
- Assuming tonumber succeeds for every string.

Activities

1. Calculate the shared $24.50 bill and 15% tip using integer cents and half-up rounding.
2. Trace each arithmetic intermediate and compare the displayed tip and total with $3.68 and $28.18.
3. Try a nonnumeric string and handle a nil conversion result.
4. Use coordinates (3, 4) to calculate distance 5 and explain the math-library call.

Accessibility Practice

Navigate by line, word, and character; identify delimiters and nesting verbally; execute from the terminal; and read the output and any error messages sequentially. Do not rely on syntax highlighting or color to understand the example.

Skills Checklist

- ☐ I can explain the concept in Lua terminology.
- ☐ I can trace and predict the example's behavior.
- ☐ I can run and modify the example from the terminal.
- ☐ I can test a boundary or invalid case.
- ☐ I can explain the relevant Lua-specific semantic difference.

12.5 Unit 3: Control Flow — Decisions and Loops

Overview

Make decisions with `if`/`elseif`, classify the shared scores with the shared grade thresholds, and learn tables as lists, loops, and strings.

Lesson 3.1: Boolean Expressions and if Statements

Reading References

- Programming in Lua, 4th edition (Roberto Ierusalimschy): Chapter 1 (Getting Started), the sections on control structures
- Lua 5.4 Reference Manual: Section 3.3.4 (Control Structures)
- Matching Python lesson: Lesson 3.1 Boolean Expressions and if Statements

Learning Objectives

- Write comparisons and combine them with `and`, `or`, and `not`.
- Use `if` ... `then` ... `else` ... `end`.
- Explain which Lua values count as false.

Concept Explanation

A *Boolean expression* is true or false. Comparison operators are `==`, `~=`, `<`, `<=`, `>`, and `>=`. Conditions combine with `and`, `or`, and `not`.

An `if` statement runs a block when its condition is true: `if condition then ... end`, with an optional `else`. Every `if` closes with `end`.

Lua's rule for truth is unusual and important: **only `nil` and `false` count as false.** The number 0, the empty string `""`, and an empty table are all *true*. Programmers used to other languages often expect 0 to be false.

`and` and `or` return one of their operands rather than always `true` or `false`. `a and b` returns `a` if `a` is false, otherwise `b`. `a or b` returns `a` if `a` is true, otherwise `b`. The idiom `condition and "yes" or "no"` therefore works as a one-line choice, provided the "yes" value is never `nil` or `false`.

The shared score is 82. It passes (at least 60), it falls between 80 and 89, and it is not at least 90.

Worked Example

```
local score = 82

if score >= 60 then
  print("Avery passed.")
else
  print("Avery did not pass.")
end

print(score >= 80 and score < 90)
print(score < 60 or score > 100)
print(not (score >= 90))

if 0 then print("0 counts as true") end
if "" then print("the empty string counts as true") end
if not nil then print("nil counts as false") end
print(score > 90 and "high" or "not high")
```

Expected Behavior or Output

The program prints:

```
Avery passed.
true
false
true
0 counts as true
the empty string counts as true
nil counts as false
not high
```

Common Mistakes

1. Expecting `0` or `""` to be false.
2. Forgetting `then` after the condition, or forgetting the closing `end`.

3. Writing `score >= 80 and < 90` instead of repeating the variable in each comparison.

Activities

1. Change the score to 59 and predict which lines change.
2. Write a condition that is true only for scores from 70 to 79 inclusive.
3. Predict `print(nil == false)` and `print(not nil)` before running them.
4. Explain why `x and "yes" or "no"` can give a wrong answer when the middle value is `false`.

Accessibility Practice

Read each condition aloud with the operator names (`greater than or equal to`, `and`, `not`), and check that your screen reader announces `~=` so you do not confuse it with `==`.

Skills Checklist

- ☐ I can write comparisons and combine them.
- ☐ I can write a complete `if` statement with `end`.
- ☐ I know that only `nil` and `false` are false.
- ☐ I can use `and`/`or` to choose a value.
- ☐ I can read a condition aloud.

Lesson 3.2: Chained Conditionals and Grade Classification

Reading References

- Programming in Lua, 4th edition (Roberto Ierusalimschy): Chapter 1 (control structures)
- Lua 5.4 Reference Manual: Section 3.3.4 (Control Structures)
- Matching Python lesson: Lesson 3.2 Chained Conditionals (elif, else)

Learning Objectives

- Use `elseif` to test conditions in order.
- Classify the shared grade thresholds.
- Test boundary scores.

Concept Explanation

Lua chains conditions with `elseif` (one word) and finishes with an optional `else` and a single `end`. Lua tests the conditions from top to bottom and runs only the first block whose condition is true.

The shared grade thresholds are: 90 or more is A, 80 or more is B, 70 or more is C, 60 or more is D, and anything lower is F. Because the conditions are tested in order, each later test only needs a lower bound.

Boundary scores sit exactly on or just below a threshold. The test scores 90, 89, 80, 79, 70, 69, 60, and 59 check every boundary. A function that returns a value is used here so the same rule can be reused; functions are explained fully in Unit 6.

Writing `else if` as two words is legal Lua but opens a new nested `if`, which then needs its own `end`; write `elseif`.

Worked Example

```
local function classify(score)
  if score >= 90 then
    return "A"
  elseif score >= 80 then
    return "B"
  elseif score >= 70 then
    return "C"
  elseif score >= 60 then
    return "D"
  else
    return "F"
  end
end

for _, score in ipairs({90, 89, 80, 79, 70, 69, 60, 59}) do
  print(string.format("%d %s", score, classify(score)))
end
```

Expected Behavior or Output

```
90 A
89 B
80 B
79 C
70 C
69 D
60 D
59 F
```

Common Mistakes

- Checking lower grade thresholds before higher thresholds, causing scores to match the wrong branch.
- Using > where the shared boundary requires >=.

- Testing only the sample scores and omitting exact boundary values 90, 80, 70, 60, and 59.

Activities

1. Trace each shared score (82, 91, 76, 88) through the conditional chain and record the selected grade.
2. Add tests for 90, 80, 70, 60, and 59; explain each branch taken.
3. Refactor the classifier into a local function and verify the four canonical outputs.
4. Compare the sum and average with 337 and 84.25; explain why the threshold order matters.

Accessibility Practice

Navigate by line, word, and character; identify delimiters and nesting verbally; execute from the terminal; and read the output and any error messages sequentially. Do not rely on syntax highlighting or color to understand the example.

Skills Checklist

☐ I can explain the concept in Lua terminology.
☐ I can trace and predict the example's behavior.
☐ I can run and modify the example from the terminal.
☐ I can test a boundary or invalid case.
☐ I can explain the relevant Lua-specific semantic difference.

Lesson 3.3 Introduction to Lists

Reading References

- Lua 5.4 Reference Manual: https://www.lua.org/manual/5.4/
- Programming in Lua: https://www.lua.org/pil/
- Corresponding Python lesson in the canonical course map.

Learning Objectives

- Explain the concept using Lua terminology.
- Trace the worked example and predict its behavior.
- Modify and test the example while preserving the shared data contract.

Concept Explanation

Lua uses one-based sequence indices. Store the shared score sequence in an array-like table and update an element by index.

Worked Example

```lua
local scores = {82, 91, 76, 88}
print(scores[1], scores[4], #scores)
scores[3] = 77
print(scores[3])
```

Expected Behavior or Output

```
82 88  4
77
```

Common Mistakes

- Using index 0 for the first sequence element; Lua sequences start at index 1.
- Assuming #table counts sparse or map keys; it is reliable for a contiguous sequence.
- Using pairs where sequence order is required, or ipairs where arbitrary map keys are required.

Activities

1. Print the first and last canonical score and explain the one-based indices.
2. Insert and remove one score, then show the sequence and its length.
3. Calculate the total and average with ipairs and compare with 337 and 84.25.
4. Create a sparse table and explain why # and ipairs do not provide a general map traversal.

Accessibility Practice

Navigate by line, word, and character; identify delimiters and nesting verbally; execute from the terminal; and read the output and any error messages sequentially. Do not rely on syntax highlighting or color to understand the example.

Skills Checklist

☐ I can explain the concept in Lua terminology.
☐ I can trace and predict the example's behavior.
☐ I can run and modify the example from the terminal.
☐ I can test a boundary or invalid case.
☐ I can explain the relevant Lua-specific semantic difference.

Lesson 3.4 Iterating with for Loops

Reading References

- Lua 5.4 Reference Manual: https://www.lua.org/manual/5.4/
- Programming in Lua: https://www.lua.org/pil/
- Corresponding Python lesson in the canonical course map.

Learning Objectives

- Explain the concept using Lua terminology.
- Trace the worked example and predict its behavior.
- Modify and test the example while preserving the shared data contract.

Concept Explanation

ipairs visits consecutive integer keys starting at 1. Use it to compute the shared score total and average.

Worked Example

```lua
local scores = {82, 91, 76, 88}
local total = 0
for _, score in ipairs(scores) do total = total + score end
print("Total:", total)
print("Average:", total / #scores)
```

Expected Behavior or Output

```
Total: 337
Average:    84.25
```

Common Mistakes

- Using index 0 for the first sequence element; Lua sequences start at index 1.
- Assuming #table counts sparse or map keys; it is reliable for a contiguous sequence.
- Using pairs where sequence order is required, or ipairs where arbitrary map keys are required.

Activities

1. Print the first and last canonical score and explain the one-based indices.
2. Insert and remove one score, then show the sequence and its length.
3. Calculate the total and average with ipairs and compare with 337 and 84.25.

4. Create a sparse table and explain why # and ipairs do not provide a general map traversal.

Accessibility Practice

Navigate by line, word, and character; identify delimiters and nesting verbally; execute from the terminal; and read the output and any error messages sequentially. Do not rely on syntax highlighting or color to understand the example.

Skills Checklist

☐ I can explain the concept in Lua terminology.
☐ I can trace and predict the example's behavior.
☐ I can run and modify the example from the terminal.
☐ I can test a boundary or invalid case.
☐ I can explain the relevant Lua-specific semantic difference.

Lesson 3.5 Basic String Operations

Reading References

- Lua 5.4 Reference Manual: https://www.lua.org/manual/5.4/
- Programming in Lua: https://www.lua.org/pil/
- Corresponding Python lesson in the canonical course map.

Learning Objectives

- Explain the concept using Lua terminology.
- Trace the worked example and predict its behavior.
- Modify and test the example while preserving the shared data contract.

Concept Explanation

Lua strings support methods through the string library. Normalize the canonical sentence before splitting it into words.

Worked Example

```lua
local sentence = "A student studies programming. Programming develops problem-solving skills
local normalized = string.lower(sentence)
local _, count = string.gsub(normalized, "[%w%-]+", "")
local _, programming_count = string.gsub(normalized, "programming", "")
print("Words:", count)
print("Programming:", programming_count)
```

Expected Behavior or Output

```
Words: 8
Programming:    2
```

Common Mistakes

- Assuming Lua patterns are full regular expressions.
- Forgetting that string functions return additional values such as match counts.
- Changing case or punctuation inconsistently before counting words.

Activities

1. Normalize the exact canonical text sample and list the tokens in order.
2. Count all eight words and verify that programming occurs twice.
3. Add punctuation and a hyphenated word; explain the chosen pattern behavior.
4. Read the output as plain text and explain how the counts were obtained.

Accessibility Practice

Navigate by line, word, and character; identify delimiters and nesting verbally; execute from the terminal; and read the output and any error messages sequentially. Do not rely on syntax highlighting or color to understand the example.

Skills Checklist

☐ I can explain the concept in Lua terminology.
☐ I can trace and predict the example's behavior.
☐ I can run and modify the example from the terminal.
☐ I can test a boundary or invalid case.
☐ I can explain the relevant Lua-specific semantic difference.

12.6 Unit 4: Data Structures — Lists and Dictionaries

Overview

Work with sequence tables, sorting, fixed and read-only records, dictionaries, and stable traversal of the shared inventory.

Lesson 4.1: More on Sequence Tables

Reading References

- Programming in Lua, 4th edition (Roberto Ierusalimschy): Chapter 5 (Tables) and Chapter 14 (Data Structures)
- Lua 5.4 Reference Manual: Section 6.6 (Table Manipulation)
- Matching Python lesson: Lesson 4.1 More on Lists

Learning Objectives

- Append, insert, replace, and remove list items.
- Search a list for a value.
- Use `table.insert` and `table.remove` correctly.

Concept Explanation

`table.insert(list, value)` appends. `table.insert(list, position, value)` inserts at a position and shifts later items up. `table.remove(list, position)` removes the item at a position, shifts later items down, and returns the removed item; `table.remove(list)` removes and returns the last item. Assigning `list[i] = value` replaces an item without shifting.

Lua has no built-in "remove by value," so write a short loop that finds the index with `ipairs` and then calls `table.remove`. Searching a list this way takes time proportional to its length.

Because Lua lists start at 1, positions in this lesson are one more than the matching Python index.

The example applies the same operations as the Python course to the shared scores.

Worked Example

```lua
local scores = {82, 91, 76, 88}

table.insert(scores, 95)
table.insert(scores, 2, 85)
scores[3] = 93

local function remove_value(list, value)
  for index, item in ipairs(list) do
    if item == value then
      return table.remove(list, index)
    end
  end
  return nil
end
```

```
remove_value(scores, 76)
local last = table.remove(scores)

print(table.concat(scores, ", "))
print(#scores, last)

local function contains(list, value)
  for _, item in ipairs(list) do
    if item == value then return true end
  end
  return false
end
print(contains(scores, 88), contains(scores, 100))
```

Expected Behavior or Output

The program prints:

```
82, 85, 93, 88
4   95
true    false
```

Common Mistakes

1. Calling `table.insert(list, position, value)` with the arguments in the wrong order.
2. Using `list[#list + 2] = value`, which leaves a gap and breaks `#`.
3. Removing items while looping forward over the same list, which skips the item after each removal.

Activities

1. Trace the list after every operation and write each version.
2. Use `table.remove(scores, 1)` and explain what happens to the other items.
3. Write `remove_all(list, value)` that removes every matching item safely.
4. Print the position of the first score above 90.

Accessibility Practice

After each change, print the whole list with `table.concat` on one line so a screen reader reads a complete state in one pass.

Skills Checklist

- ☐ I can append, insert, replace, and remove items.

- ☐ I can return a removed item from `table.remove`.
- ☐ I can search a list with a loop.
- ☐ I know positions start at 1.
- ☐ I can show a list in one line.

Lesson 4.2: Sorting and Organizing Tables

Reading References

- Programming in Lua, 4th edition (Roberto Ierusalimschy): Chapter 6 (Functions), the section on sort
- Lua 5.4 Reference Manual: Section 6.6 (table.sort)
- Matching Python lesson: Lesson 4.2 Organizing Lists

Learning Objectives

- Sort a list ascending and descending.
- Reverse a list and sort a copy so the original is unchanged.
- Write a comparison function.

Concept Explanation

`table.sort(list)` sorts the list *in place* from smallest to largest. For another order, pass a *comparison function* that returns `true` when its first argument must come before its second: `table.sort(list, function(a, b) return a > b end)` sorts from largest to smallest.

The comparison function must be consistent. Using `>=` can cause the error "invalid order function for sorting". Use strict `<` or `>`.

`table.sort` has no stable-order guarantee for equal items. To keep the original order, sort a *copy*. `table.move(source, 1, #source, 1, {})` copies a list into a new table.

Lua has no built-in reverse for lists; reverse with two indexes that swap and move toward each other.

The example sorts the shared scores: ascending is 76, 82, 88, 91 and descending is 91, 88, 82, 76.

Worked Example

```lua
local scores = {82, 91, 76, 88}

local sorted_copy = table.move(scores, 1, #scores, 1, {})
table.sort(sorted_copy)
print("Original:", table.concat(scores, ", "))
print("Ascending:", table.concat(sorted_copy, ", "))
```

```lua
table.sort(scores, function(a, b) return a > b end)
print("Descending:", table.concat(scores, ", "))

local function reverse(list)
  local i, j = 1, #list
  while i < j do
    list[i], list[j] = list[j], list[i]
    i = i + 1
    j = j - 1
  end
end
reverse(scores)
print("Reversed:", table.concat(scores, ", "))
```

Expected Behavior or Output

The program prints:

```
Original:    82, 91, 76, 88
Ascending:  76, 82, 88, 91
Descending: 91, 88, 82, 76
Reversed:   76, 82, 88, 91
```

Common Mistakes

1. Using `>=` in a comparison function, which can raise an invalid-order error.
2. Expecting `table.sort` to return the sorted list. It returns nothing and sorts in place.
3. Sorting a list that mixes numbers and strings, which raises a comparison error.

Activities

1. Predict the four printed lines before running.
2. Sort the scores descending without changing the original by sorting a copy.
3. Sort the words `{"pencil", "folder", "notebook"}` and print them.
4. Explain why `list[i], list[j] = list[j], list[i]` swaps without a temporary variable.

Accessibility Practice

Print each result on its own labelled line. Compare the lines by listening for the first and last numbers, which are quickest to check.

Skills Checklist

- ☐ I can sort in place and sort a copy.
- ☐ I can write a comparison function.

☐ I can reverse a list with a loop.
☐ I know `table.sort` returns nothing.
☐ I know why comparison functions must be strict.

Lesson 4.3: Fixed Records and Read-Only Tables

Reading References

- Programming in Lua, 4th edition (Roberto Ierusalimschy): Chapter 6 (Functions), multiple results; Chapter 20 (Metatables and Metamethods)
- Lua 5.4 Reference Manual: Section 2.4 (Metatables and Metamethods)
- Matching Python lesson: Lesson 4.3 Tuples - Immutable Lists

Learning Objectives

- Return and receive several values from a function.
- Explain why Lua has no tuple type.
- Create a read-only table with a metatable.

Concept Explanation

Lua has no tuple type. Two idioms cover the same needs. First, a function can return *multiple values*, and the caller can receive them with `local x, y = point()`. Second, a fixed collection can be a small table such as `{3, 4}` or `{x = 3, y = 4}`.

A plain table can always be changed. To make a table *read-only*, return an empty proxy table whose metatable forwards reads to the real table with `__index` and rejects writes with `__newindex`. The length operator needs a `__len` metamethod because the proxy itself is empty.

`pcall(f)` (protected call) runs a function and returns `false` plus an error message instead of stopping the program. The error here uses level 0 so the message has no file name or line number and prints the same on every computer.

The shared coordinates are x = 3 and y = 4.

Worked Example

```lua
local function point()
  return 3, 4
end
local x, y = point()
print(x, y)

local function readonly(source)
  return setmetatable({}, {
    __index = source,
    __len = function() return #source end,
```

```
    __newindex = function()
      error("attempt to modify a read-only table", 0)
    end,
  })
end

local coordinates = readonly({3, 4})
print(coordinates[1], coordinates[2], #coordinates)

local ok, message = pcall(function() coordinates[1] = 10 end)
print(ok, message)
```

Expected Behavior or Output

The program prints:

```
3    4
3   4   2
false   attempt to modify a read-only table
```

Common Mistakes

1. Expecting `coordinates[1] = 10` to fail on a normal table. Only the proxy rejects writes.
2. Forgetting `__len`, so `#coordinates` is 0 on the empty proxy.
3. Using a read-only proxy for iteration with `pairs` and being surprised that it appears empty.

Activities

1. Run the program and explain each of the five printed values.
2. Write a function that returns a name, an age, and a height, and receive all three.
3. Add `__newindex` rejection for a table of the shared inventory item and test it with `pcall`.
4. Explain in two sentences why a tuple is not needed in Lua.

Accessibility Practice

Read the proxy code in short pieces: the real table, the metatable, `__index`, `__len`, `__newindex`. Check closing braces by counting the levels aloud.

Skills Checklist

- ☐ I can return and receive multiple values.
- ☐ I can explain why Lua has no tuples.
- ☐ I can make a read-only proxy table.
- ☐ I can use `pcall` to catch an error.

☐ I can read nested braces aloud.

Lesson 4.4: Tables as Dictionaries

Reading References

- Programming in Lua, 4th edition (Roberto Ierusalimschy): Chapter 5 (Tables)
- Lua 5.4 Reference Manual: Section 3.4.9 (Table Constructors)
- Matching Python lesson: Lesson 4.4 Introducing Dictionaries

Learning Objectives

- Create records with named keys.
- Add, change, and remove a key.
- Model the shared inventory item.

Concept Explanation

A table with named keys works as a dictionary or a record. Write `{name = "Avery", age = 16}` and read a field with `profile.name` or `profile["name"]`. The two forms are equivalent for plain names; use brackets when the key is in a variable or contains spaces.

Assigning to a new key adds it, assigning to an existing key changes it, and **assigning `nil` removes it**. Reading a key that does not exist yields `nil`, not an error.

The length operator `#` counts only list items, so it does not count named keys. To count keys, loop with `pairs`.

The shared student profile has name Avery, age 16, height 1.72 meters, and favorite subject Computer Science. The shared inventory item is Notebook with quantity 3 and unit price 2.50.

Worked Example

```lua
local student_profile = {
  name = "Avery",
  age = 16,
  height_m = 1.72,
  favorite_subject = "Computer Science",
}

student_profile.age = 17
student_profile.grade_level = 11
student_profile.height_m = nil
print(student_profile.name, student_profile.age, student_profile.grade_level, student_profil
```

```lua
local function count_keys(t)
  local n = 0
  for _ in pairs(t) do n = n + 1 end
  return n
end
print("Keys:", count_keys(student_profile), "Length operator:", #student_profile)

local item = {name = "Notebook", quantity = 3, unit_price = 2.50}
item.quantity = item.quantity + 2
item.location = "Shelf A"
print(string.format("%s: %d at %.2f = %.2f", item.name, item.quantity, item.unit_price, item
```

Expected Behavior or Output

The program prints:

```
Avery    17  11  nil
Keys:   4   Length operator:    0
Notebook: 5 at 2.50 = 12.50
```

Common Mistakes

1. Using `#` to count keys. It returns 0 for a table with only named keys.
2. Assuming a missing key causes an error. It returns `nil`.
3. Using a number as a key by accident (`t[1]`) when the intended key is the text `"1"`.

Activities

1. Predict the printed values, then run.
2. Remove `favorite_subject` and show that reading it gives `nil`.
3. Add `discount = 0.20` to the item and print the discounted line value.
4. Write a function that returns `true` if a record has all of the keys `name`, `quantity`, and `unit_price`.

Accessibility Practice

Use field names that read well aloud, such as `unit_price`. Read records one key per line so a screen reader announces each key with its value.

Skills Checklist

- ☐ I can create a record with named keys.
- ☐ I can add, change, and remove keys.
- ☐ I know `nil` assignment removes a key.
- ☐ I can count keys with `pairs`.
- ☐ I can model the shared inventory item.

Lesson 4.5: Traversing Tables with pairs and ipairs

Reading References

- Programming in Lua, 4th edition (Roberto Ierusalimschy): Chapter 18 (Iterators and the Generic for)
- Lua 5.4 Reference Manual: Section 6.1 (pairs and ipairs)
- Matching Python lesson: Lesson 4.5 Looping Through Dictionaries

Learning Objectives

- Traverse keys and values with `pairs`.
- Explain why `pairs` order is not guaranteed.
- Produce stable, sorted output from a table.

Concept Explanation

`pairs(table)` visits every key and value, but its iteration order is not defined. Do not use that order for output that must be stable. Collect the keys into a sequence, sort the sequence with `table.sort`, and then visit the values in that order. Use `ipairs` for a contiguous sequence whose order is defined from index 1 upward.

The shared inventory contains Notebook (quantity 3, unit price 2.50), Pencil (quantity 10, unit price 0.50), and Folder (quantity 4, unit price 1.25). Its total value is 17.50.

Worked Example

```lua
local inventory = {
  Notebook = {quantity = 3, unit_price = 2.50},
  Pencil = {quantity = 10, unit_price = 0.50},
  Folder = {quantity = 4, unit_price = 1.25},
}

local names = {}
for name in pairs(inventory) do
  names[#names + 1] = name
end
table.sort(names)

local total_cents = 0
for _, name in ipairs(names) do
  local item = inventory[name]
  local line_cents = item.quantity * math.floor(item.unit_price * 100 + 0.5)
  total_cents = total_cents + line_cents
  print(string.format("%s\t%d\t$%.2f\t$%.2f",
    name, item.quantity, item.unit_price, line_cents / 100))
```

```lua
end
print(string.format("Inventory total: $%.2f", total_cents / 100))
```

Expected Behavior or Output

```
Folder 4   $1.25   $5.00
Notebook    3   $2.50   $7.50
Pencil  10  $0.50   $5.00
Inventory total: $17.50
```

Common Mistakes

1. Assuming `pairs` visits keys in alphabetical or insertion order.
2. Printing values directly from `pairs` when exact output order matters.
3. Using `#` to count named keys; it is intended for sequence length, not arbitrary maps.

Activities

1. Run the example and explain why the keys are sorted before printing.
2. Add a fourth item and verify the total.
3. Remove the sort and explain why output order is no longer guaranteed.
4. Explain the difference between `pairs` and `ipairs`.

Accessibility Practice

Keep each inventory item on one line with its name, quantity, unit price, and line value. Read the final total after all item lines so the result is clear in sequential screen-reader output.

Skills Checklist

- ☐ I can traverse a table with `pairs`.
- ☐ I know that `pairs` order is not guaranteed.
- ☐ I can sort keys for stable output.
- ☐ I can traverse a sequence with `ipairs`.
- ☐ I can calculate the shared inventory total.

12.7 Unit 5: User Input and while Loops

Overview

Read and validate input, write sentinel and flag loops, skip entries without a `continue` keyword, and prevent endless loops.

Lesson 5.1: How io.read Works

Reading References

- Programming in Lua, 4th edition (Roberto Ierusalimschy): Chapter 7 (The External World)
- Lua 5.4 Reference Manual: Section 6.8 (io.read)
- Matching Python lesson: Lesson 5.1 How the input() Function Works

Learning Objectives

- Read a line and recognize that it is text.
- Convert it with `tonumber` and test for failure.
- Handle the end of input.

Concept Explanation

`io.read()` (the same as `io.read("l")`) reads one line and returns it as a string. Other formats exist: `io.read("n")` reads a number directly and `io.read("a")` reads everything that remains. This course uses line reading with explicit conversion because it is easier to validate.

`io.read` returns `nil` when there is no more input. A program that keeps reading must check for `nil`, or it will crash on the first run with no input.

Converting text to a number is a separate step: `tonumber(text)` returns the number or `nil`. Test the result before using it.

With the typed value 16, the program converts the text to the number 16 and prints the age five years later, 21. A second value, `abc`, is rejected because `tonumber` returns `nil`.

Worked Example

```lua
print("How old are you?")
local text = io.read()
print("Type:", type(text))

local age = tonumber(text)
if age then
  print("In 5 years:", age + 5)
else
  print("That was not a number.")
end

print("Enter another value:")
local second = tonumber(io.read())
if second then
  print("Number:", second)
```

```lua
else
  print("Not a number; tonumber returned nil.")
end
```

Expected Behavior or Output

When the standard input is the following lines (the program's own prompts and results are shown separately because piped input is not echoed):

```
16
abc
```

the program prints:

```
How old are you?
Type:   string
In 5 years: 21
Enter another value:
Not a number; tonumber returned nil.
```

Common Mistakes

1. Calling arithmetic on the text without converting it first.
2. Not checking for `nil` from `io.read` at the end of input.
3. Forgetting that `tonumber("")` is `nil`, so an empty answer is invalid.

Activities

1. Run with 16 and then `abc`. Compare with the shown output.
2. Run with no input and explain each line printed.
3. Make the program accept an age only between 0 and 120.
4. Write the program using `io.read("n")` and explain what happens when the user types a word.

Accessibility Practice

Prompts are on their own lines. Run the program twice, once typing and once with piped input, and compare what your screen reader announces.

Skills Checklist

- ☐ I can read a line of input.
- ☐ I can convert and validate it.
- ☐ I handle `nil` at end of input.
- ☐ I can explain that typed input is text.
- ☐ I can use `io.read` formats.

Lesson 5.2: The while Loop in Action

Reading References

- Programming in Lua, 4th edition (Roberto Ierusalimschy): Chapter 1 (control structures)
- Lua 5.4 Reference Manual: Section 3.3.4 (While Statement)
- Matching Python lesson: Lesson 5.2 The while Loop in Action

Learning Objectives

- Write a counting `while` loop.
- Write a sentinel loop that ends on a command.
- Stop at the end of input.

Concept Explanation

A `while` loop repeats its block as long as the condition is true: `while condition do ... end`. The condition is tested *before* each pass. A loop needs a way to make the condition false, normally by changing a variable inside the body.

A *sentinel* is a special value that tells the loop to stop. Here the sentinel is the word `quit`. The standard pattern reads once before the loop and again at the end of each pass, and checks for `nil` as well, so the loop ends when input runs out.

The example first counts from 1 to 5, then repeats a message prompt until the user types `quit`.

Worked Example

```lua
local count = 1
while count <= 5 do
  print(count)
  count = count + 1
end
print("Counting finished!")

print("Enter a message, or type quit to exit:")
local message = io.read()
while message ~= nil and message ~= "quit" do
  print("You said: " .. message)
  print("Enter a message, or type quit to exit:")
  message = io.read()
end
print("Goodbye!")
```

Expected Behavior or Output

When the standard input is the following lines (the program's own prompts and results are shown separately because piped input is not echoed):

```
hello
scores
quit
```

the program prints:

```
1
2
3
4
5
Counting finished!
Enter a message, or type quit to exit:
You said: hello
Enter a message, or type quit to exit:
You said: scores
Enter a message, or type quit to exit:
Goodbye!
```

Common Mistakes

1. Forgetting to change the loop variable, creating an endless loop.
2. Testing `message ~= "quit"` without testing for `nil`, so input that ends without `quit` crashes the loop.
3. Reading input only before the loop and never again inside it.

Activities

1. Predict the output for the typed lines hello, scores, quit.
2. Change the sentinel to `exit` and update the prompt.
3. Count how many messages were entered and print the count at the end.
4. Rewrite the counting loop as a numeric `for` loop and compare.

Accessibility Practice

Each prompt appears on its own line and each response begins with the label `You said:`, so the conversation can be followed in order by ear.

Skills Checklist

- ☐ I can write a counting `while` loop.
- ☐ I can write a sentinel loop.
- ☐ I check for `nil` as well as the sentinel.
- ☐ I update the loop variable.

☐ I can compare `while` with numeric `for`.

Lesson 5.3: Using Flags and break to Exit Loops

Reading References

- Programming in Lua, 4th edition (Roberto Ierusalimschy): Chapter 1 (control structures)
- Lua 5.4 Reference Manual: Section 3.3.4 (Break and Goto)
- Matching Python lesson: Lesson 5.3 Using Flags and break to Exit Loops

Learning Objectives

- Use a Boolean flag to control a loop.
- Use `break` to leave a loop early.
- Search a list and remember where the item was found.

Concept Explanation

A *flag* is a Boolean variable that controls a loop: the loop runs while `running` is true, and the body sets it to `false` when it is time to stop. `break` leaves the nearest enclosing loop immediately.

A flag makes the stopping rule visible in one place. `break` is shorter and is clearest for searches that can stop as soon as the answer is found.

For a search, keep a variable for the result (initially `nil`), set it when the item is found, then `break`. After the loop, test the variable to see whether a match was found. Positions in Lua lists start at 1.

The shared scores are 82, 91, 76, 88. The first score of 90 or higher is 91 at position 2.

Worked Example

```lua
local running = true
local handled = 0
while running do
  print("Say something, or type quit to stop:")
  local message = io.read()
  if message == nil or message == "quit" then
    running = false
  else
    handled = handled + 1
    print("You said: " .. message)
  end
end
print("Messages handled: " .. handled)
```

```lua
local scores = {82, 91, 76, 88}
local found_index
for index, score in ipairs(scores) do
  if score >= 90 then
    found_index = index
    break
  end
end
if found_index then
  print(string.format("First score of 90 or higher: %d at position %d", scores[found_index],
else
  print("No score of 90 or higher.")
end
```

Expected Behavior or Output

When the standard input is `hello` followed by `quit` (piped input is not echoed), the program prints:

```
Say something, or type quit to stop:
You said: hello
Say something, or type quit to stop:
Messages handled: 1
First score of 90 or higher: 91 at position 2
```

Common Mistakes

1. Setting the flag but then running more code in the same pass that should be skipped.
2. Forgetting to initialize the result variable and misreading an old value after the loop.
3. Using `break` to leave an outer loop from an inner loop. It only leaves the innermost loop.

Activities

1. Run with the lines `hello` and `quit` and confirm the count is 1.
2. Change the search to the first score below 80 and predict the position.
3. Rewrite the first loop without a flag, using `break`.
4. Explain when a flag is clearer than `break`.

Accessibility Practice

Announce results with complete sentences. The search result includes both the value and its position so the sentence is understandable without layout.

Skills Checklist

- ☐ I can control a loop with a flag.
- ☐ I can leave a loop with `break`.
- ☐ I can search a list and keep the position.
- ☐ I know `break` leaves one loop.
- ☐ I can explain flag versus `break`.

Lesson 5.4: Skipping Entries with goto continue

Reading References

- Programming in Lua, 4th edition (Roberto Ierusalimschy): Chapter 8 (Filling some Gaps), goto
- Lua 5.4 Reference Manual: Section 3.3.4 (Goto Statement)
- Matching Python lesson: Lesson 5.4 Using continue in Loops

Learning Objectives

- Skip a loop pass with `goto continue`.
- Write the same skip with a nested `if`.
- Skip invalid scores in a list.

Concept Explanation

Lua has no `continue` statement. Two idioms skip the rest of one pass. The first is `goto continue` with a label `::continue::` at the very end of the loop body. The label name is a convention, not a keyword. A `goto` cannot jump into the scope of a local variable, so the label belongs at the end of the block.

The second idiom avoids `goto` entirely: wrap the work in an `if` that handles only the wanted case. For short loops this is usually clearer.

The example skips invalid scores (below 0 or above 100) in the list 82, -5, 91, 105, 76, 88, then lists the odd numbers from 1 to 9 with the `if` form. The four valid scores total 337.

Worked Example

```
local raw_scores = {82, -5, 91, 105, 76, 88}
local valid_total, valid_count = 0, 0

for _, raw in ipairs(raw_scores) do
  if raw < 0 or raw > 100 then
    print("Skipping invalid score: " .. raw)
    goto continue
  end
```

```
  valid_total = valid_total + raw
  valid_count = valid_count + 1
  ::continue::
end
print("Valid scores: " .. valid_count .. ", total: " .. valid_total)

for number = 1, 9 do
  if number % 2 ~= 0 then
    print(number)
  end
end
```

Expected Behavior or Output

The program prints:

```
Skipping invalid score: -5
Skipping invalid score: 105
Valid scores: 4, total: 337
1
3
5
7
9
```

Common Mistakes

1. Looking for a `continue` keyword. It does not exist in Lua.
2. Declaring a local variable after the `goto` and before the label in the same block, which is a compile error.
3. Placing the label in the middle of the body so statements after it still run.

Activities

1. Predict the printed lines and the final total.
2. Skip the scores below 80 instead.
3. Rewrite the first loop with the nested `if` form and compare readability.
4. Explain why the label must come at the end of the loop body.

Accessibility Practice

Each skipped entry prints its own line, so the screen reader announces what was ignored. Never skip silently when accessibility depends on knowing what happened.

Skills Checklist

- ☐ I know Lua has no `continue`.

- ☐ I can use `goto continue` correctly.
- ☐ I can use a nested `if` instead.
- ☐ I can skip invalid entries.
- ☐ I can explain where the label goes.

Lesson 5.5: Avoiding Infinite Loops

Reading References

- Programming in Lua, 4th edition (Roberto Ierusalimschy): Chapter 1 (control structures)
- Lua 5.4 Reference Manual: Section 3.3.4 (Repeat Statement)
- Matching Python lesson: Lesson 5.5 Avoiding Infinite Loops

Learning Objectives

- Identify what makes a loop stop.
- Add an attempt limit to an input loop.
- Use `repeat ... until`.

Concept Explanation

An *infinite loop* never ends because its condition never becomes false. Typical causes are forgetting to change the loop variable, testing the wrong variable, or waiting for input that never arrives. In a terminal, stop a running program with Ctrl+C.

Defend against endless loops by giving every loop a *progress rule* and a *limit*. Count attempts, and stop when the limit is reached or the input ends.

`repeat ... until condition` runs the body first and tests afterward, so the body always runs at least once. Unlike other loops, variables declared inside the body are visible in the `until` test.

The example allows three attempts to enter a score from 0 to 100. With the typed lines abc, 105, and 88 the first two fail validation and the third is accepted.

Worked Example

```lua
local MAX_ATTEMPTS = 3
local attempts = 0
local score

while attempts < MAX_ATTEMPTS do
  attempts = attempts + 1
  print(string.format("Enter a score from 0 to 100 (attempt %d of %d):", attempts, MAX_ATTE
  local text = io.read()
  if text == nil then
    print("No more input.")
```

```
    break
  end
  local number = tonumber(text)
  if number and number >= 0 and number <= 100 then
    score = number
    break
  end
  print("Invalid score: " .. text)
end

if score then
  print("Accepted score: " .. score)
else
  print("No valid score was entered.")
end

local count = 1
repeat
  print(count)
  count = count + 1
until count > 3
```

Expected Behavior or Output

When the standard input is the following lines (the program's own prompts and results are shown separately because piped input is not echoed):

```
abc
105
88
```

the program prints:

```
Enter a score from 0 to 100 (attempt 1 of 3):
Invalid score: abc
Enter a score from 0 to 100 (attempt 2 of 3):
Invalid score: 105
Enter a score from 0 to 100 (attempt 3 of 3):
Accepted score: 88
1
2
3
```

Common Mistakes

1. Forgetting to increase `attempts`, so the limit is never reached.
2. Waiting for input without handling `nil`, so a closed input stream ends in an error instead of a message.

3. Writing `until count < 3` and ending after one pass because the condition is true at once.

Activities

1. Run with abc, 105, and 88 and confirm the accepted score.
2. Run with three invalid lines and read the final message.
3. Change the limit to 5 and test it.
4. Write down, for each loop in the example, what makes it stop.

Accessibility Practice

A loop that cannot be stopped is an accessibility problem as well as a bug, because output may flood a screen reader. Learn the interrupt key (Ctrl+C) before you run loops, and give every input loop a limit.

Skills Checklist

☐ I can name what ends each loop.
☐ I can add an attempt limit.
☐ I handle the end of input.
☐ I can use `repeat ... until`.
☐ I know how to interrupt a running program.

12.8 Unit 6: Functions

Overview

This unit develops the concepts in the same order and with the same problem context as the Python reference. Predict the output before running each example. Use Lua 5.4 semantics, not assumptions imported from Python.

Lesson 6.1 Defining Functions

Reading References

- Lua 5.4 Reference Manual: https://www.lua.org/manual/5.4/
- Programming in Lua: https://www.lua.org/pil/
- Corresponding Python lesson in the canonical course map.

Learning Objectives

- Explain the concept using Lua terminology.
- Trace the worked example and predict its behavior.
- Modify and test the example while preserving the shared data contract.

Concept Explanation

Functions package a named operation. The function below calculates a tip and total from the canonical bill.

Worked Example

```lua
local function calculate_total(bill_cents, rate_basis_points)
  -- 10,000 basis points = 100%; add 5,000 before integer division
  -- to implement half-up rounding to the nearest cent.
  local tip_cents = (bill_cents * rate_basis_points + 5000) // 10000
  return tip_cents, bill_cents + tip_cents
end
local tip_cents, total_cents = calculate_total(2450, 1500)
print(string.format("Tip: $%.2f", tip_cents / 100))
print(string.format("Total: $%.2f", total_cents / 100))
```

Expected Behavior or Output

```
Tip: $3.68
Total: $28.18
```8.18

### Common Mistakes

- Forgetting that a Lua function returns nil when it reaches the end without return.
- Confusing multiple return values with a single table result.
- Assuming Python keyword arguments or default-parameter syntax exists in Lua.

### Activities

1. Call the function with the canonical values and record every returned value.
2. Add a second return value and capture it explicitly at the call site.
3. Test a missing optional argument and apply a deliberate default.
4. Pass a table to the function, mutate it, and explain the shared table identity.

### Accessibility Practice

Navigate by line, word, and character; identify delimiters and nesting verbally; execute from the terminal; and read the output and any error messages sequentially. Do not rely on syntax highlighting or color to understand the example.

### Skills Checklist

- ☐ I can explain the concept in Lua terminology.
- ☐ I can trace and predict the example's behavior.
- ☐ I can run and modify the example from the terminal.
- ☐ I can test a boundary or invalid case.
- ☐ I can explain the relevant Lua-specific semantic difference.

---

## Lesson 6.2 Passing Arguments

### Reading References

- Lua 5.4 Reference Manual: https://www.lua.org/manual/5.4/
- Programming in Lua: https://www.lua.org/pil/
- Corresponding Python lesson in the canonical course map.

### Learning Objectives

- Explain the concept using Lua terminology.
- Trace the worked example and predict its behavior.
- Modify and test the example while preserving the shared data contract.

### Concept Explanation

Lua uses positional arguments rather than Python-style keyword/default arguments. A nil parameter can be replaced with an explicit default.

### Worked Example

```lua
local function grade_for(score, passing_grade)
  passing_grade = passing_grade or "D"
  if score >= 60 then return passing_grade end
  return "F"
end
print(grade_for(75))
print(grade_for(75, "Pass"))
```

Expected Behavior or Output

```
D
Pass
```

Common Mistakes

- Forgetting that a Lua function returns nil when it reaches the end without return.
- Confusing multiple return values with a single table result.

- Assuming Python keyword arguments or default-parameter syntax exists in Lua.

Activities

1. Call the function with the canonical values and record every returned value.
2. Add a second return value and capture it explicitly at the call site.
3. Test a missing optional argument and apply a deliberate default.
4. Pass a table to the function, mutate it, and explain the shared table identity.

Accessibility Practice

Navigate by line, word, and character; identify delimiters and nesting verbally; execute from the terminal; and read the output and any error messages sequentially. Do not rely on syntax highlighting or color to understand the example.

Skills Checklist

☐ I can explain the concept in Lua terminology.
☐ I can trace and predict the example's behavior.
☐ I can run and modify the example from the terminal.
☐ I can test a boundary or invalid case.
☐ I can explain the relevant Lua-specific semantic difference.

Lesson 6.3 Return Values

Reading References

- Lua 5.4 Reference Manual: https://www.lua.org/manual/5.4/
- Programming in Lua: https://www.lua.org/pil/
- Corresponding Python lesson in the canonical course map.

Learning Objectives

- Explain the concept using Lua terminology.
- Trace the worked example and predict its behavior.
- Modify and test the example while preserving the shared data contract.

Concept Explanation

return sends values to the caller. Lua functions can return multiple values; capture them in matching local variables.

Worked Example

```lua
local function summarize(values)
  local total = 0
  for _, value in ipairs(values) do total = total + value end
  return total, total / #values
end
local total, average = summarize({82, 91, 76, 88})
print(total, average)
```

Expected Behavior or Output

```
337    84.25
```

Common Mistakes

- Forgetting that a Lua function returns nil when it reaches the end without return.
- Confusing multiple return values with a single table result.
- Assuming Python keyword arguments or default-parameter syntax exists in Lua.

Activities

1. Call the function with the canonical values and record every returned value.
2. Add a second return value and capture it explicitly at the call site.
3. Test a missing optional argument and apply a deliberate default.
4. Pass a table to the function, mutate it, and explain the shared table identity.

Accessibility Practice

Navigate by line, word, and character; identify delimiters and nesting verbally; execute from the terminal; and read the output and any error messages sequentially. Do not rely on syntax highlighting or color to understand the example.

Skills Checklist

☐ I can explain the concept in Lua terminology.
☐ I can trace and predict the example's behavior.
☐ I can run and modify the example from the terminal.
☐ I can test a boundary or invalid case.
☐ I can explain the relevant Lua-specific semantic difference.

Lesson 6.4 Functions with Tables

Reading References

- Lua 5.4 Reference Manual: https://www.lua.org/manual/5.4/
- Programming in Lua: https://www.lua.org/pil/
- Corresponding Python lesson in the canonical course map.

Learning Objectives

- Explain the concept using Lua terminology.
- Trace the worked example and predict its behavior.
- Modify and test the example while preserving the shared data contract.

Concept Explanation

Tables are passed by reference-like identity: mutating a table inside a function changes the same table the caller holds.

Worked Example

```lua
local function add_score(scores, score)
  table.insert(scores, score)
end
local scores = {82, 91, 76, 88}
add_score(scores, 95)
print(#scores, scores[5])
```

Expected Behavior or Output

```
5  95
```

Common Mistakes

- Forgetting that a Lua function returns nil when it reaches the end without return.
- Confusing multiple return values with a single table result.
- Assuming Python keyword arguments or default-parameter syntax exists in Lua.

Activities

1. Call the function with the canonical values and record every returned value.
2. Add a second return value and capture it explicitly at the call site.
3. Test a missing optional argument and apply a deliberate default.
4. Pass a table to the function, mutate it, and explain the shared table identity.

Accessibility Practice

Navigate by line, word, and character; identify delimiters and nesting verbally; execute from the terminal; and read the output and any error messages sequentially. Do not rely on syntax highlighting or color to understand the example.

Skills Checklist

- ☐ I can explain the concept in Lua terminology.
- ☐ I can trace and predict the example's behavior.
- ☐ I can run and modify the example from the terminal.
- ☐ I can test a boundary or invalid case.
- ☐ I can explain the relevant Lua-specific semantic difference.

Lesson 6.5 Variadic Functions

Reading References

- Lua 5.4 Reference Manual: https://www.lua.org/manual/5.4/
- Programming in Lua: https://www.lua.org/pil/
- Corresponding Python lesson in the canonical course map.

Learning Objectives

- Explain the concept using Lua terminology.
- Trace the worked example and predict its behavior.
- Modify and test the example while preserving the shared data contract.

Concept Explanation

The ... expression captures variable arguments. Use select or a table pack when processing an arbitrary argument list.

Worked Example

```lua
local function sum_all(...)
  local values = {...}
  local total = 0
  for _, value in ipairs(values) do total = total + value end
  return total
end
print(sum_all(82, 91, 76, 88))
```

Expected Behavior or Output

```
337
```

Common Mistakes

- Forgetting that a Lua function returns nil when it reaches the end without return.
- Confusing multiple return values with a single table result.
- Assuming Python keyword arguments or default-parameter syntax exists in Lua.

Activities

1. Call the function with the canonical values and record every returned value.
2. Add a second return value and capture it explicitly at the call site.
3. Test a missing optional argument and apply a deliberate default.
4. Pass a table to the function, mutate it, and explain the shared table identity.

Accessibility Practice

Navigate by line, word, and character; identify delimiters and nesting verbally; execute from the terminal; and read the output and any error messages sequentially. Do not rely on syntax highlighting or color to understand the example.

Skills Checklist

☐ I can explain the concept in Lua terminology.
☐ I can trace and predict the example's behavior.
☐ I can run and modify the example from the terminal.
☐ I can test a boundary or invalid case.
☐ I can explain the relevant Lua-specific semantic difference.

12.9 Unit 7: Classes and Object-Oriented Programming

Overview

This unit develops the concepts in the same order and with the same problem context as the Python reference. Predict the output before running each example. Use Lua 5.4 semantics, not assumptions imported from Python.

Lesson 7.1 Why Learn About Classes?

Reading References

- Lua 5.4 Reference Manual: https://www.lua.org/manual/5.4/

- Programming in Lua: https://www.lua.org/pil/
- Corresponding Python lesson in the canonical course map.

Learning Objectives

- Explain the concept using Lua terminology.
- Trace the worked example and predict its behavior.
- Modify and test the example while preserving the shared data contract.

Concept Explanation

Lua has no class keyword. Tables and functions can model state and behavior; a metatable can provide method lookup.

Worked Example

```lua
local student = {name = "Avery", age = 16}
local function describe(record)
  return record.name .. " is " .. record.age
end
print(describe(student))
```

Expected Behavior or Output

```
Avery is 16
```

Common Mistakes

- Describing Lua as having a built-in class keyword; this course uses tables and metatables.
- Forgetting that colon method syntax passes self as the first argument.
- Creating an __index chain without explaining which table supplies the missing method.

Activities

1. Create two independent records using the constructor and compare their fields.
2. Add one method and call it with colon syntax, then show the equivalent explicit self call.
3. Extend the example with a composed record and read a nested field.
4. Trace method lookup through the metatable chain and explain the result.

Accessibility Practice

Navigate by line, word, and character; identify delimiters and nesting verbally; execute from the terminal; and read the output and any error messages sequentially. Do not rely on syntax highlighting or color to understand the example.

Skills Checklist

- ☐ I can explain the concept in Lua terminology.
- ☐ I can trace and predict the example's behavior.
- ☐ I can run and modify the example from the terminal.
- ☐ I can test a boundary or invalid case.
- ☐ I can explain the relevant Lua-specific semantic difference.

Lesson 7.2 Creating and Using a Class

Reading References

- Lua 5.4 Reference Manual: https://www.lua.org/manual/5.4/
- Programming in Lua: https://www.lua.org/pil/
- Corresponding Python lesson in the canonical course map.

Learning Objectives

- Explain the concept using Lua terminology.
- Trace the worked example and predict its behavior.
- Modify and test the example while preserving the shared data contract.

Concept Explanation

A constructor function creates records with a shared shape. This is a common lightweight Lua object pattern.

Worked Example

```lua
local function Student(name, age)
  return {name = name, age = age}
end
local a = Student("Avery", 16)
local b = Student("Riley", 17)
print(a.name, b.name)
```

Expected Behavior or Output

```
Avery  Riley
```

Common Mistakes

- Describing Lua as having a built-in class keyword; this course uses tables and metatables.
- Forgetting that colon method syntax passes self as the first argument.
- Creating an __index chain without explaining which table supplies the missing method.

Activities

1. Create two independent records using the constructor and compare their fields.
2. Add one method and call it with colon syntax, then show the equivalent explicit self call.
3. Extend the example with a composed record and read a nested field.
4. Trace method lookup through the metatable chain and explain the result.

Accessibility Practice

Navigate by line, word, and character; identify delimiters and nesting verbally; execute from the terminal; and read the output and any error messages sequentially. Do not rely on syntax highlighting or color to understand the example.

Skills Checklist

- ☐ I can explain the concept in Lua terminology.
- ☐ I can trace and predict the example's behavior.
- ☐ I can run and modify the example from the terminal.
- ☐ I can test a boundary or invalid case.
- ☐ I can explain the relevant Lua-specific semantic difference.

Lesson 7.3 Attributes and Methods

Reading References

- Lua 5.4 Reference Manual: https://www.lua.org/manual/5.4/
- Programming in Lua: https://www.lua.org/pil/
- Corresponding Python lesson in the canonical course map.

Learning Objectives

- Explain the concept using Lua terminology.
- Trace the worked example and predict its behavior.
- Modify and test the example while preserving the shared data contract.

Concept Explanation

A method is a function stored on a table. The colon syntax passes self as the first argument.

Worked Example

```lua
local Item = {}
function Item:new(name, quantity)
  return setmetatable({name = name, quantity = quantity}, {__index = self})
```

```lua
end
function Item:display()
  print(self.name .. ": " .. self.quantity)
end
local notebook = Item:new("Notebook", 3)
notebook:display()
```

Expected Behavior or Output

```
Notebook: 3
```

Common Mistakes

- Describing Lua as having a built-in class keyword; this course uses tables and metatables.
- Forgetting that colon method syntax passes self as the first argument.
- Creating an __index chain without explaining which table supplies the missing method.

Activities

1. Create two independent records using the constructor and compare their fields.
2. Add one method and call it with colon syntax, then show the equivalent explicit self call.
3. Extend the example with a composed record and read a nested field.
4. Trace method lookup through the metatable chain and explain the result.

Accessibility Practice

Navigate by line, word, and character; identify delimiters and nesting verbally; execute from the terminal; and read the output and any error messages sequentially. Do not rely on syntax highlighting or color to understand the example.

Skills Checklist

☐ I can explain the concept in Lua terminology.
☐ I can trace and predict the example's behavior.
☐ I can run and modify the example from the terminal.
☐ I can test a boundary or invalid case.
☐ I can explain the relevant Lua-specific semantic difference.

Lesson 7.4 Inheritance and Metatables

Reading References

- Lua 5.4 Reference Manual: https://www.lua.org/manual/5.4/
- Programming in Lua: https://www.lua.org/pil/
- Corresponding Python lesson in the canonical course map.

Learning Objectives

- Explain the concept using Lua terminology.
- Trace the worked example and predict its behavior.
- Modify and test the example while preserving the shared data contract.

Concept Explanation

Metatable __index can delegate missing method lookups to a parent table. Explain this lookup behavior rather than implying Lua has built-in classes.

Worked Example

```lua
local Base = {kind = "record"}
function Base:describe() return self.name .. " (" .. self.kind .. ")" end
local Student = setmetatable({}, {__index = Base})
local a = setmetatable({name = "Avery", kind = "student"}, {__index = Student})
print(a:describe())
```

Expected Behavior or Output

```
Avery (student)
```

Common Mistakes

- Describing Lua as having a built-in class keyword; this course uses tables and metatables.
- Forgetting that colon method syntax passes self as the first argument.
- Creating an __index chain without explaining which table supplies the missing method.

Activities

1. Create two independent records using the constructor and compare their fields.
2. Add one method and call it with colon syntax, then show the equivalent explicit self call.
3. Extend the example with a composed record and read a nested field.
4. Trace method lookup through the metatable chain and explain the result.

Accessibility Practice

Navigate by line, word, and character; identify delimiters and nesting verbally; execute from the terminal; and read the output and any error messages sequentially. Do not rely on syntax highlighting or color to understand the example.

Skills Checklist

- ☐ I can explain the concept in Lua terminology.
- ☐ I can trace and predict the example's behavior.
- ☐ I can run and modify the example from the terminal.
- ☐ I can test a boundary or invalid case.
- ☐ I can explain the relevant Lua-specific semantic difference.

Lesson 7.5 Composition

Reading References

- Lua 5.4 Reference Manual: https://www.lua.org/manual/5.4/
- Programming in Lua: https://www.lua.org/pil/
- Corresponding Python lesson in the canonical course map.

Learning Objectives

- Explain the concept using Lua terminology.
- Trace the worked example and predict its behavior.
- Modify and test the example while preserving the shared data contract.

Concept Explanation

Composition stores one record inside another. This is often simpler than building a deep inheritance hierarchy.

Worked Example

```lua
local profile = {name = "Avery", contact = {email = "avery@example.test"}}
print(profile.name)
print(profile.contact.email)
```

Expected Behavior or Output

```
Avery
avery@example.test
```

Common Mistakes

- Describing Lua as having a built-in class keyword; this course uses tables and metatables.
- Forgetting that colon method syntax passes self as the first argument.
- Creating an __index chain without explaining which table supplies the missing method.

Activities

1. Create two independent records using the constructor and compare their fields.
2. Add one method and call it with colon syntax, then show the equivalent explicit self call.
3. Extend the example with a composed record and read a nested field.
4. Trace method lookup through the metatable chain and explain the result.

Accessibility Practice

Navigate by line, word, and character; identify delimiters and nesting verbally; execute from the terminal; and read the output and any error messages sequentially. Do not rely on syntax highlighting or color to understand the example.

Skills Checklist

☐ I can explain the concept in Lua terminology.
☐ I can trace and predict the example's behavior.
☐ I can run and modify the example from the terminal.
☐ I can test a boundary or invalid case.
☐ I can explain the relevant Lua-specific semantic difference.

12.10 Unit 8: Files and Exceptions

Overview

Read and write files, handle errors with `pcall`, close resources reliably, and store records as JSON with a small teaching helper.

Lesson 8.1: Reading from a File

Reading References

- Programming in Lua, 4th edition (Roberto Ierusalimschy): Chapter 7 (The External World)
- Lua 5.4 Reference Manual: Section 6.8 (File Operations)
- Matching Python lesson: Lesson 8.1 Reading from a File

Learning Objectives

- Open a file for reading and check the result.
- Read the whole file or one line at a time.
- Report a missing file clearly.

Concept Explanation

`io.open(path, mode)` returns a file handle, or `nil` plus an error message and an error number when it fails. Always check the result. Mode `"r"` reads. Methods are called with a colon on the handle: `file:read("a")` reads everything, `file:read("l")` reads one line, and `file:lines()` gives an iterator over the lines.

A relative path starts from the working directory, so a program run from a different folder will not find the file. A file handle uses a system resource, so close it with `file:close()` when finished.

To keep this lesson self-contained, the example first writes the two-line `notes.txt` used by every language track, then reads it back three ways. The missing-file case prints the message that `io.open` returns.

Worked Example

```lua
local out = assert(io.open("notes.txt", "w"))
out:write("Practice Lua.\nReview tables and functions.\n")
out:close()

local file, err = io.open("notes.txt", "r")
if not file then
  print("Could not open notes.txt: " .. err)
  return
end
local contents = file:read("a")
file:close()
print(contents)

for line in io.lines("notes.txt") do
  print(line)
end

local count = 0
for _ in io.lines("notes.txt") do count = count + 1 end
print(count)

local missing, message = io.open("missing_notes.txt", "r")
print(missing, message)
```

Expected Behavior or Output

The program prints:

```
Practice Lua.
Review tables and functions.

Practice Lua.
Review tables and functions.
2
nil missing_notes.txt: No such file or directory
```

Common Mistakes

1. Not checking the result of `io.open` and then calling a method on `nil`.
2. Forgetting to close the file.
3. Using a relative path from a different working directory.

Activities

1. Run the program and describe each block of output.
2. Add a third line to `notes.txt` and predict the new count.
3. Print each line with its line number.
4. Open a file that does not exist and print a friendly message that includes the file name.

Accessibility Practice

Read the output aloud in order. Make error messages complete sentences that include the file name, so they are clear without seeing the screen.

Skills Checklist

- ☐ I can open a file and check the result.
- ☐ I can read it whole or by line.
- ☐ I can close a file.
- ☐ I can report a missing file.
- ☐ I can explain the working directory.

Lesson 8.2: Writing to a File

Reading References

- Programming in Lua, 4th edition (Roberto Ierusalimschy): Chapter 7 (The External World)
- Lua 5.4 Reference Manual: Section 6.8 (file:write)
- Matching Python lesson: Lesson 8.2 Writing to a File

Learning Objectives

- Write with mode `"w"` and append with mode `"a"`.
- Check for failure when writing.
- Read the file back to verify.

Concept Explanation

Mode `"w"` creates the file or *overwrites* an existing one. Mode `"a"` adds to the end. `file:write(text)` writes exactly the text you give, so include `\n` where a line should end. `file:write` returns the file handle on success and `nil` plus a message on failure, so it can also be checked.

A closed file is the only way to be certain that the data reached storage. Always close, then reopen to verify.

The example writes the first notes line, appends the second, and reads the file back.

Worked Example

```
local file = assert(io.open("notes.txt", "w"))
file:write("Practice Lua.\n")
file:close()

file = assert(io.open("notes.txt", "a"))
file:write("Review tables and functions.\n")
file:close()

file = assert(io.open("notes.txt", "r"))
print(file:read("a"))
file:close()
```

Expected Behavior or Output

The program prints:

```
Practice Lua.
Review tables and functions.
```

Common Mistakes

1. Using `"w"` when `"a"` was intended, which erases earlier content.
2. Forgetting `\n`, so all lines run together.
3. Not closing the file before reading it back.

Activities

1. Predict the output, then run.

2. Change the second open to mode `"w"` and explain the difference.
3. Append a third line with the current count of lines.
4. Write the shared scores, one per line, then read them back and total them.

Accessibility Practice

Always read a written file back in the terminal to confirm it, using line navigation, so you do not rely on assumptions about what was saved.

Skills Checklist

- ☐ I can write and append.
- ☐ I know `"w"` overwrites.
- ☐ I can include new lines.
- ☐ I can verify by reading back.
- ☐ I can check for a write failure.

Lesson 8.3: Errors and pcall

Reading References

- Programming in Lua, 4th edition (Roberto Ierusalimschy): Chapter 16 (Compilation, Execution, and Errors)
- Lua 5.4 Reference Manual: Section 2.3 (Error Handling) and Section 6.1 (pcall, error, assert)
- Matching Python lesson: Lesson 8.3 Exceptions - Handling Errors

Learning Objectives

- Raise an error with `error` and `assert`.
- Catch an error with `pcall`.
- Handle a missing file and malformed data.

Concept Explanation

Lua reports problems in two ways. Library functions such as `io.open` and `tonumber` usually *return* `nil` plus a message. Other problems *raise an error*, which stops the program unless it is caught. `error(message, level)` raises an error. `assert(value, message)` raises an error when `value` is false or `nil`.

`pcall(function)` runs a function in *protected mode*. It returns `true` plus the function's results on success, or `false` plus the error value on failure, and the program continues.

Error messages from `error` normally start with a file name and line number. Pass level `0` to leave the position out, which makes a message easier to test. An error value may be any value, including a table.

Integer division by zero (`10 // 0`) raises an error, while float division (`10 / 0`) returns `inf`. Runtime error messages from Lua include the file name and line number, and their exact wording can differ between Lua versions, so the example prints only the type of that error value. The example catches a missing file, an invalid number, and an integer division by zero.

Worked Example

```
local file, message = io.open("missing_notes.txt", "r")
if not file then
  print("Error: missing_notes.txt was not found.")
end

local score = tonumber("abc")
if not score then
  print("Error: 'abc' is not a valid score.")
end

local ok, err = pcall(function() return 10 // 0 end)
print(ok, type(err))
print(10 / 0)

local ok2, err2 = pcall(function() error("custom failure", 0) end)
print(ok2, err2)

local ok3, err3 = pcall(assert, false, "assertion message")
print(ok3, err3)
```

Expected Behavior or Output

The program prints:

```
Error: missing_notes.txt was not found.
Error: 'abc' is not a valid score.
false   string
inf
false   custom failure
false   assertion message
```

Common Mistakes

1. Using `pcall` to hide errors without reporting them.
2. Forgetting that `error` messages include a position unless the level is 0.
3. Expecting `10 / 0` to raise an error. It returns `inf`; only integer division by zero raises one.

Activities

1. Predict each printed line before running, then print `err` to see the full message including the file name and line.
2. Wrap a call to `tonumber` and arithmetic so invalid input prints a clear message.
3. Raise an error with a table value and read its fields after catching it.
4. Explain when to return `nil, message` and when to call `error`.

Accessibility Practice

Write error messages as complete sentences that name the problem and the next step, such as 'Enter a whole number from 0 to 100.'

Skills Checklist

- ☐ I can raise and catch errors.
- ☐ I can use `assert` and `pcall`.
- ☐ I know `io.open` returns a message instead of raising.
- ☐ I know integer and float division by zero differ.
- ☐ I can write clear error messages.

Lesson 8.4: Cleanup and Resource Safety

Reading References

- Programming in Lua, 4th edition (Roberto Ierusalimschy): Chapter 16 (Compilation, Execution, and Errors)
- Lua 5.4 Reference Manual: Section 3.3.8 (To-be-closed Variables)
- Matching Python lesson: Lesson 8.4 The else and finally Blocks

Learning Objectives

- Close files reliably, even when an error occurs.
- Use a to-be-closed variable (`<close>`).
- Use `pcall` to guarantee cleanup.

Concept Explanation

Lua has no `try`/`finally`. Two techniques make cleanup reliable. The manual technique runs the risky work inside `pcall`, closes the resource afterward, then re-raises the error if there was one.

Lua 5.4 adds *to-be-closed variables*: `local file <close> = io.open(...)`. When the variable goes out of scope (normally or through an error) Lua calls its `__close` metamethod. File handles already provide it, so the file is closed automatically. `<close>` needs Lua 5.4.

`io.type(handle)` reports `"file"`, `"closed file"`, or `nil`. The example keeps a reference to each handle so it can check that the file is closed after the block ends and after an error.

Worked Example

```
local saved
do
  local file <close> = assert(io.open("cleanup_demo.txt", "w"))
  saved = file
  file:write("Some data.\n")
end
print(io.type(saved))

local saved2
local ok, err = pcall(function()
  local file <close> = assert(io.open("cleanup_demo.txt", "w"))
  saved2 = file
  error("failure while writing", 0)
end)
print(ok, err, io.type(saved2))

local manual = assert(io.open("cleanup_demo.txt", "r"))
local ok3, content = pcall(function() return manual:read("a") end)
manual:close()
print(ok3, content, io.type(manual))
```

Expected Behavior or Output

The program prints:

```
closed file
false   failure while writing   closed file
true        closed file
```

Common Mistakes

1. Using `<close>` on Lua 5.3 or earlier, where it is a syntax error.
2. Returning from a function before closing a manually opened file.
3. Closing a file twice and reading the error.

Activities

1. Predict the three printed lines.
2. Rewrite the manual example with `<close>`.
3. Raise an error before the write in the second example and confirm the file is still closed.

4. Explain why cleanup matters even if the program is about to end.

Accessibility Practice

Print the file state (`io.type`) after each step. Short, labelled status lines are easy to follow with a screen reader.

Skills Checklist

- ☐ I can close files reliably.
- ☐ I can use `<close>`.
- ☐ I can protect cleanup with `pcall`.
- ☐ I can check a handle with `io.type`.
- ☐ I know `<close>` needs Lua 5.4.

Lesson 8.5: Storing Data with JSON

Reading References

- Programming in Lua, 4th edition: Chapter 15 (Data Files and Serialization)
- Lua 5.4 Reference Manual: Section 6.8 (Input and Output Facilities)
- JSON specification: https://www.json.org
- Matching Python lesson: Lesson 8.5 Storing Data with JSON

Learning Objectives

- Explain why Lua needs a JSON helper.
- Encode and decode the shared task records.
- Handle a missing or malformed file.

Concept Explanation

Lua's standard library has no JSON support. Real projects install a maintained library through LuaRocks, such as `dkjson` or `lua-cjson`. To keep this course free of external dependencies, the example includes a *teaching implementation* of `json.encode` and `json.decode`. It is intentionally small: it handles strings, numbers, booleans, `null`, lists, and objects, writes object keys in sorted order, and reports malformed text with an error that includes the position. It is not a replacement for a production library.

JSON text stores data in a standard format. An object `{ ... }` holds key-value pairs, a list `[ ... ]` holds ordered items, and text is in double quotation marks. Decoding turns text into Lua tables, so decoded data should be validated before use.

The shared to-do record has the fields `id`, `title`, `description`, `done`, and `created_at`. The example saves two tasks and loads them back. It also shows what a malformed file produces.

Language-Specific Semantics

The helper's `json.decode` turns JSON `null` into `nil`, so a key with a `null` value is simply absent from the decoded table. It does not handle every escape or number edge case that a full parser must, and it assumes the text is valid UTF-8.

Worked Example

```
local json = {}

local escapes = {['"'] = '\\"', ['\\'] = '\\\\', ['\n'] = '\\n', ['\r'] = '\\r', ['\t'] = '\

function json.encode(value)
  local kind = type(value)
  if kind == "string" then
    return '"' .. value:gsub('[%c"\\]', function(c)
      return escapes[c] or string.format("\\u%04x", c:byte())
    end) .. '"'
  elseif kind == "number" then
    return math.type(value) == "integer" and tostring(value) or string.format("%.14g", value
  elseif kind == "boolean" then
    return tostring(value)
  elseif kind == "nil" then
    return "null"
  elseif kind == "table" then
    local parts = {}
    if value[1] ~= nil or next(value) == nil then
      for _, item in ipairs(value) do parts[#parts + 1] = json.encode(item) end
      return "[" .. table.concat(parts, ",") .. "]"
    end
    local keys = {}
    for key in pairs(value) do keys[#keys + 1] = key end
    table.sort(keys)
    for _, key in ipairs(keys) do
      parts[#parts + 1] = json.encode(tostring(key)) .. ":" .. json.encode(value[key])
    end
    return "{" .. table.concat(parts, ",") .. "}"
  end
  error("cannot encode a value of type " .. kind, 0)
end

function json.decode(text)
  local position = 1
  local read_value
```

```
local function skip()
  position = text:find("%S", position) or #text + 1
end

local function fail(message)
  error(message .. " at position " .. position, 0)
end

local function read_string()
  local pieces = {}
  position = position + 1
  while true do
    local c = text:sub(position, position)
    if c == "" then fail("unterminated string") end
    if c == '"' then
      position = position + 1
      return table.concat(pieces)
    elseif c == "\\" then
      local n = text:sub(position + 1, position + 1)
      local simple = {n = "\n", t = "\t", r = "\r", ['"'] = '"', ["\\"] = "\\", ["/"] = "/
      if n == "u" then
        pieces[#pieces + 1] = utf8.char(tonumber(text:sub(position + 2, position + 5), 16)
        position = position + 6
      elseif simple[n] then
        pieces[#pieces + 1] = simple[n]
        position = position + 2
      else
        fail("bad escape")
      end
    else
      pieces[#pieces + 1] = c
      position = position + 1
    end
  end
end

function read_value()
  skip()
  local c = text:sub(position, position)
  if c == "{" then
    local object = {}
    position = position + 1
    skip()
    if text:sub(position, position) == "}" then position = position + 1; return object end
    while true do
      skip()
```

```
        if text:sub(position, position) ~= '"' then fail("expected a key") end
        local key = read_string()
        skip()
        if text:sub(position, position) ~= ":" then fail("expected ':'") end
        position = position + 1
        object[key] = read_value()
        skip()
        local d = text:sub(position, position)
        position = position + 1
        if d == "}" then return object end
        if d ~= "," then position = position - 1; fail("expected ',' or '}'") end
      end
    elseif c == "[" then
      local list = {}
      position = position + 1
      skip()
      if text:sub(position, position) == "]" then position = position + 1; return list end
      while true do
        list[#list + 1] = read_value()
        skip()
        local d = text:sub(position, position)
        position = position + 1
        if d == "]" then return list end
        if d ~= "," then position = position - 1; fail("expected ',' or ']'") end
      end
    elseif c == '"' then
      return read_string()
    elseif text:find("^true", position) then
      position = position + 4; return true
    elseif text:find("^false", position) then
      position = position + 5; return false
    elseif text:find("^null", position) then
      position = position + 4; return nil
    end
    local number = text:match("^-?%d+%.?%d*[eE]?[+-]?%d*", position)
    if not number or number == "" then fail("unexpected character") end
    position = position + #number
    return tonumber(number)
  end

  local result = read_value()
  skip()
  if position <= #text then fail("unexpected extra text") end
  return result
end
```

```lua
local tasks = {
  {id = 1, title = "Study programming", description = "Finish Unit 8", done = false, created
  {id = 2, title = "Review scores", description = "Check the four scores", done = true, crea
}

local file = assert(io.open("tasks.json", "w"))
file:write(json.encode(tasks))
file:close()

local input = io.open("tasks.json", "r")
if not input then
  print("Tasks file not found.")
  return
end
local text = input:read("a")
input:close()

local ok, loaded = pcall(json.decode, text)
if not ok then
  print("Error: could not decode tasks file: " .. loaded)
  return
end
for _, task in ipairs(loaded) do
  print(string.format("%d: %s (%s)", task.id, task.title, task.done and "done" or "not done"
end

print(pcall(json.decode, '[{"id": 1,'))
```

Expected Behavior or Output

The program prints:

```
1: Study programming (not done)
2: Review scores (done)
false   expected a key at position 11
```

Common Mistakes

1. Assuming Lua has built-in JSON support.
2. Trusting decoded data without checking that each field exists and has the right type.
3. Overwriting a good data file with an empty table after a failed load.

Activities

1. Run the program and read the saved JSON text from `tasks.json`.

2. Edit `tasks.json` by hand to remove a closing bracket and rerun. Read the error message.
3. Add a third task and confirm it is saved and loaded.
4. Explain why a production project should use a maintained JSON library.

Accessibility Practice

Open `tasks.json` in the terminal and read it as text. JSON in a single line is difficult to navigate, so practise stepping through it by word or character.

Skills Checklist

☐ I know Lua has no built-in JSON.
☐ I can encode and decode records.
☐ I handle missing and malformed files.
☐ I validate decoded data.
☐ I know when to use a library.

12.11 Unit 9: Testing and Data Analysis

Overview

This unit develops the concepts in the same order and with the same problem context as the Python reference. Predict the output before running each example. Use Lua 5.4 semantics, not assumptions imported from Python.

Lesson 9.1 Testing a Function

Reading References

- Lua 5.4 Reference Manual: https://www.lua.org/manual/5.4/
- Programming in Lua: https://www.lua.org/pil/
- Corresponding Python lesson in the canonical course map.

Learning Objectives

- Explain the concept using Lua terminology.
- Trace the worked example and predict its behavior.
- Modify and test the example while preserving the shared data contract.

Concept Explanation

Assertions make expected behavior executable. Test the shared score average and the exact tip calculation.

Worked Example

```lua
local function average(values)
  local total = 0
  for _, value in ipairs(values) do total = total + value end
  return total / #values
end
assert(average({82, 91, 76, 88}) == 84.25)
print("average test passed")
```

Expected Behavior or Output

```
average test passed
```

Common Mistakes

- Forgetting that a Lua function returns nil when it reaches the end without return.
- Confusing multiple return values with a single table result.
- Assuming Python keyword arguments or default-parameter syntax exists in Lua.

Activities

1. Call the function with the canonical values and record every returned value.
2. Add a second return value and capture it explicitly at the call site.
3. Test a missing optional argument and apply a deliberate default.
4. Pass a table to the function, mutate it, and explain the shared table identity.

Accessibility Practice

Navigate by line, word, and character; identify delimiters and nesting verbally; execute from the terminal; and read the output and any error messages sequentially. Do not rely on syntax highlighting or color to understand the example.

Skills Checklist

☐ I can explain the concept in Lua terminology.
☐ I can trace and predict the example's behavior.
☐ I can run and modify the example from the terminal.
☐ I can test a boundary or invalid case.
☐ I can explain the relevant Lua-specific semantic difference.

Lesson 9.2 Testing a Table-Based Object

Reading References

- Lua 5.4 Reference Manual: https://www.lua.org/manual/5.4/
- Programming in Lua: https://www.lua.org/pil/
- Corresponding Python lesson in the canonical course map.

Learning Objectives

- Explain the concept using Lua terminology.
- Trace the worked example and predict its behavior.
- Modify and test the example while preserving the shared data contract.

Concept Explanation

Test observable state changes on an object-like table. Include both normal and invalid cases.

Worked Example

```lua
local function new_task(id, title, description, created_at)
  assert(type(id) == "number", "id must be numeric")
  assert(type(title) == "string" and title:match("%S"), "title is required")
  assert(type(description) == "string", "description must be text")
  assert(type(created_at) == "string", "created_at must be text")
  return {
    id = id,
    title = title,
    description = description,
    done = false,
    created_at = created_at
  }
end
local task = new_task(1, "Practice Lua", "Complete the shared exercises",
  "2025-01-15T09:00:00Z")
assert(task.done == false)
task.done = true
assert(task.done == true)
print("task tests passed")
```

Expected Behavior or Output

```
task tests passed
```

Common Mistakes

- Describing Lua as having a built-in class keyword; this course uses tables and metatables.
- Forgetting that colon method syntax passes self as the first argument.
- Creating an __index chain without explaining which table supplies the missing method.

Activities

1. Create two independent records using the constructor and compare their fields.
2. Add one method and call it with colon syntax, then show the equivalent explicit self call.
3. Extend the example with a composed record and read a nested field.
4. Trace method lookup through the metatable chain and explain the result.

Accessibility Practice

Navigate by line, word, and character; identify delimiters and nesting verbally; execute from the terminal; and read the output and any error messages sequentially. Do not rely on syntax highlighting or color to understand the example.

Skills Checklist

☐ I can explain the concept in Lua terminology.
☐ I can trace and predict the example's behavior.
☐ I can run and modify the example from the terminal.
☐ I can test a boundary or invalid case.
☐ I can explain the relevant Lua-specific semantic difference.

Lesson 9.3 Random Numbers

Reading References

- Lua 5.4 Reference Manual: https://www.lua.org/manual/5.4/
- Programming in Lua: https://www.lua.org/pil/
- Corresponding Python lesson in the canonical course map.

Learning Objectives

- Explain the concept using Lua terminology.
- Trace the worked example and predict its behavior.
- Modify and test the example while preserving the shared data contract.

Concept Explanation

math.random generates pseudorandom values. math.randomseed can make a sequence repeatable for a lesson or test.

Worked Example

```lua
math.randomseed(12345)
local die = math.random(1, 6)
print("Die roll is in range:", die >= 1 and die <= 6)
```

Expected Behavior or Output

```
Die roll is in range:  true
```

Common Mistakes

- Expecting an unseeded pseudorandom sequence to reproduce exactly across runs.
- Using a random value without checking the required range.
- Seeding repeatedly inside a tight loop, which can reduce randomness.

Activities

1. Generate an integer in the range 1 through 6 and assert both bounds.
2. Seed once before a reproducibility demonstration and compare two controlled runs.
3. Generate a random selection from the canonical score list.
4. Explain why a statistical test should not require one specific unseeded value.

Accessibility Practice

Navigate by line, word, and character; identify delimiters and nesting verbally; execute from the terminal; and read the output and any error messages sequentially. Do not rely on syntax highlighting or color to understand the example.

Skills Checklist

☐ I can explain the concept in Lua terminology.
☐ I can trace and predict the example's behavior.
☐ I can run and modify the example from the terminal.
☐ I can test a boundary or invalid case.
☐ I can explain the relevant Lua-specific semantic difference.

Lesson 9.4 Analyzing Text Data

Reading References

- Lua 5.4 Reference Manual: https://www.lua.org/manual/5.4/
- Programming in Lua: https://www.lua.org/pil/
- Corresponding Python lesson in the canonical course map.

Learning Objectives

- Explain the concept using Lua terminology.
- Trace the worked example and predict its behavior.
- Modify and test the example while preserving the shared data contract.

Concept Explanation

Count tokens in the canonical text sample. Lua patterns are not full regular expressions; document the pattern being used.

Worked Example

```lua
local text = "A student studies programming. Programming develops problem-solving skills."
local normalized = string.lower(text)
local words = {}
for word in string.gmatch(normalized, "[%w%-]+") do
  words[#words + 1] = word
end
local count = 0
for _, word in ipairs(words) do if word == "programming" then count = count + 1 end end
print("Words:", #words)
print("Programming:", count)
```

Expected Behavior or Output

```
Words: 8
Programming:    2
```

Common Mistakes

- Assuming Lua patterns are full regular expressions.
- Forgetting that string functions return additional values such as match counts.
- Changing case or punctuation inconsistently before counting words.

Activities

1. Normalize the exact canonical text sample and list the tokens in order.
2. Count all eight words and verify that programming occurs twice.

3. Add punctuation and a hyphenated word; explain the chosen pattern behavior.
4. Read the output as plain text and explain how the counts were obtained.

Accessibility Practice

Navigate by line, word, and character; identify delimiters and nesting verbally; execute from the terminal; and read the output and any error messages sequentially. Do not rely on syntax highlighting or color to understand the example.

Skills Checklist

- ☐ I can explain the concept in Lua terminology.
- ☐ I can trace and predict the example's behavior.
- ☐ I can run and modify the example from the terminal.
- ☐ I can test a boundary or invalid case.
- ☐ I can explain the relevant Lua-specific semantic difference.

Lesson 9.5 Working with CSV Data

Reading References

- Lua 5.4 Reference Manual: https://www.lua.org/manual/5.4/
- Programming in Lua: https://www.lua.org/pil/
- Corresponding Python lesson in the canonical course map.

Learning Objectives

- Explain the concept using Lua terminology.
- Trace the worked example and predict its behavior.
- Modify and test the example while preserving the shared data contract.

Concept Explanation

CSV fields can contain quoted commas and escaped quotes; naive string splitting is not a correct general CSV parser. Use a vetted CSV library for production data.

Worked Example

```lua
local headers = {"student_id", "first_name", "last_name", "score", "active"}
local rows = {
  {"1001", "Avery", "Jordan", "82", "true"},
  {"1002", "Riley", "Chen", "91", "true"},
  {"1003", "Sam", "Patel", "76", "false"},
  {"1004", "Morgan", "Diaz", "88", "true"}
```

```lua
}
print(table.concat(headers, ","))
for _, row in ipairs(rows) do
  print(table.concat(row, ","))
end
```

Expected Behavior or Output

```
student_id,first_name,last_name,score,active
1001,Avery,Jordan,82,true
1002,Riley,Chen,91,true
1003,Sam,Patel,76,false
1004,Morgan,Diaz,88,true
```

Common Mistakes

- Ignoring the second return value from io.open when a file cannot be opened.
- Leaving a file handle open after a successful or failed operation.
- Assuming Lua includes a standard JSON or full CSV parser.

Activities

1. Test the file operation with an existing file and a missing-file path.
2. Add an explicit failure message using the error returned by io.open.
3. Close the file on every control path and explain the resource-lifetime rule.
4. Document the selected JSON/CSV dependency and test malformed and quoted-field inputs.

Accessibility Practice

Navigate by line, word, and character; identify delimiters and nesting verbally; execute from the terminal; and read the output and any error messages sequentially. Do not rely on syntax highlighting or color to understand the example.

Skills Checklist

☐ I can explain the concept in Lua terminology.
☐ I can trace and predict the example's behavior.
☐ I can run and modify the example from the terminal.
☐ I can test a boundary or invalid case.
☐ I can explain the relevant Lua-specific semantic difference.

12.12 Unit 10: Project-Based Learning — Text-Based Applications

Overview

This unit develops the concepts in the same order and with the same problem context as the Python reference. Predict the output before running each example. Use Lua 5.4 semantics, not assumptions imported from Python.

Lesson 10.1 Project Design and Requirements

Reading References

- Lua 5.4 Reference Manual: https://www.lua.org/manual/5.4/
- Programming in Lua: https://www.lua.org/pil/
- Corresponding Python lesson in the canonical course map.

Learning Objectives

- Explain the concept using Lua terminology.
- Trace the worked example and predict its behavior.
- Modify and test the example while preserving the shared data contract.

Concept Explanation

Translate requirements into data structures, commands, and acceptance tests before coding. This room graph is the common adventure map.

Worked Example

```lua
local rooms = {
  start_room = {exits = {north = "tunnel"}, items = {"shiny key"}},
  tunnel = {exits = {south = "start_room", east = "clearing"}, items = {}},
  clearing = {exits = {west = "tunnel", north = "locked_door_room"}, items = {}},
  locked_door_room = {exits = {south = "clearing", north = "treasure_room"}, items = {}},
  treasure_room = {exits = {}, items = {}}
}
print(rooms.tunnel.exits.east)
```

Expected Behavior or Output

```
clearing
```

Common Mistakes

- Dropping one of the six required capstone record fields during serialization.
- Updating a record without refreshing updated_at.

- Claiming screen-reader compatibility without testing keyboard and assistive-technology workflows.

Activities

1. Create a record containing all six canonical capstone fields and validate each field.
2. Exercise add, list, search, update, and delete with both existing and missing IDs.
3. Save and reload a record; compare every field before and after persistence.
4. Write a text-only keyboard interaction guide and list the accessibility tests still required.

Accessibility Practice

Navigate by line, word, and character; identify delimiters and nesting verbally; execute from the terminal; and read the output and any error messages sequentially. Do not rely on syntax highlighting or color to understand the example.

Skills Checklist

☐ I can explain the concept in Lua terminology.
☐ I can trace and predict the example's behavior.
☐ I can run and modify the example from the terminal.
☐ I can test a boundary or invalid case.
☐ I can explain the relevant Lua-specific semantic difference.

Lesson 10.2: Text Adventure, Part 1: Five Rooms and Movement

Reading References

- Lua 5.4 Reference Manual
- Programming in Lua
- Canonical shared project specification: Python Unit 10, Lesson 10.3

Learning Objectives

- Represent the five canonical rooms and their exits with Lua tables.
- Move through valid exits without changing location after an invalid command.
- Explain why the north exit from the locked-door room is not available yet.

Concept Explanation

The shared adventure has five rooms: start_room, tunnel, clearing, locked_door_room, and treasure_room. The unconditional connections are start north to tunnel; tunnel south to start and east to clearing; clearing west to tunnel and north to the locked-door room; and locked-door room south to clearing. The north exit to the treasure room is conditional and is added in Part 2 only after the shiny-key puzzle is solved.

Each room is a Lua table with a description and an exits table. A missing key lookup returns nil, so invalid movement can be rejected without changing the current room.

Worked Example

```
local rooms = {
  start_room = {
    description = "A dimly lit cave. A narrow passage leads north.",
    exits = {north = "tunnel"}
  },
  tunnel = {
    description = "A damp tunnel. Passages lead south and east.",
    exits = {south = "start_room", east = "clearing"}
  },
  clearing = {
    description = "A quiet clearing. An antechamber lies to the north.",
    exits = {west = "tunnel", north = "locked_door_room"}
  },
  locked_door_room = {
    description = "A chamber with a heavy door to the north.",
    exits = {south = "clearing"}
  },
  treasure_room = {
    description = "You found the treasure. You win!",
    exits = {}
  }
}

local current_room = "start_room"
while true do
  local room = rooms[current_room]
  print("---")
  print(room.description)
  print("What do you want to do?")
  local input = io.read("*l")
  local command = (input or "quit"):match("^%s*(.-)%s*$"):lower()
```

```
  if command == "quit" then
    print("Exiting game. Goodbye!")
    break
  end

  local destination = room.exits[command]
  if destination then
    current_room = destination
  else
    print("You cannot go that way. Enter a listed direction or quit.")
  end
end
```

Expected Behavior or Output

With north, east, north, north, quit entered as separate input lines, the player visits tunnel, clearing, and locked_door_room. The second north from locked_door_room is rejected and the current room does not change. The treasure room exists in the model but cannot be reached in Part 1.

Common Mistakes

- Adding the treasure-room north exit before implementing the key puzzle.
- Updating current_room before confirming that the requested exit exists.
- Treating a missing table key as a valid destination.
- Defining an exit whose destination is not one of the five room identifiers.

Activities

1. Trace north, east, north, north and record the room after each command.
2. Try west from start_room; confirm that the location does not change.
3. Check that every destination in every unconditional exit exists in rooms.
4. Explain why treasure_room has no incoming exit in Part 1.

Accessibility Practice

Read the separator, room description, and prompt in order. The invalid-movement response explains what happened and how to continue. Test the sequence by listening to terminal output; do not rely on a visual map.

Skills Checklist

☐ I can model all five canonical rooms.
☐ I can represent exits with Lua tables.
☐ I can move only when an exit exists.
☐ I can reject an invalid direction without changing state.
☐ I can explain why the treasure room is not yet reachable.

Lesson 10.3: Text Adventure, Part 2: Shiny Key and Locked Door

Reading References

- Lua 5.4 Reference Manual
- Programming in Lua
- Canonical shared project specification: Python Unit 10, Lesson 10.3

Learning Objectives

- Place the shiny key in start_room and transfer it to inventory exactly once.
- Require the player to use the key while in locked_door_room.
- Unlock the north exit to treasure_room only after the puzzle succeeds.
- End the game when the player reaches the treasure room.

Concept Explanation

Part 2 keeps the same five-room graph. The shiny key begins in start_room. The take shiny key command removes it from that room and adds it to inventory. The use shiny key command works only in locked_door_room and only when the key is in inventory. Attempting north from the locked-door room before that action prints a locked-door message and leaves the player in place. After the key is used, north leads to treasure_room.

Room descriptions do not claim that an item is still present. Room items and inventory are announced on separate lines.

Worked Example

```
local rooms = {
  start_room = {
    description = "A dimly lit cave.",
    exits = {north = "tunnel"},
    items = {"shiny key"}
  },
  tunnel = {
    description = "A damp tunnel. Passages lead south and east.",
    exits = {south = "start_room", east = "clearing"},
    items = {}
  },
  clearing = {
    description = "A quiet clearing. Antechamber lies to the north.",
    exits = {west = "tunnel", north = "locked_door_room"},
```

```
    items = {}
  },
  locked_door_room = {
    description = "A chamber with a heavy door to the north.",
    exits = {south = "clearing"},
    items = {}
  },
  treasure_room = {
    description = "You found the treasure. You win!",
    exits = {},
    items = {}
  }
}

local current_room = "start_room"
local inventory = {}
local door_locked = true

local function inventory_has(item_name)
  for _, item in ipairs(inventory) do
    if item == item_name then return true end
  end
  return false
end

while true do
  local room = rooms[current_room]
  print()
  print(room.description)
  if current_room == "treasure_room" then break end
  if #room.items > 0 then
    print("You see: " .. table.concat(room.items, ", "))
  end
  print("Your inventory: " .. (#inventory == 0 and "empty" or table.concat(inventory, ", "))
  print("What do you want to do?")

  local input = io.read("*l")
  local command = (input or "quit"):match("^%s*(.-)%s*$"):lower()

  if command == "quit" then
    print("You quit the adventure.")
    break
  elseif command == "take shiny key" then
    local found_index
    for index, item in ipairs(room.items) do
      if item == "shiny key" then
```

```
        found_index = index
        break
      end
    end
    if found_index then
      table.remove(room.items, found_index)
      inventory[#inventory + 1] = "shiny key"
      print("You picked up the shiny key.")
    else
      print("There is no shiny key here to take.")
    end
  elseif command == "use shiny key" then
    if current_room ~= "locked_door_room" then
      print("You must be at the locked door to use the key.")
    elseif not inventory_has("shiny key") then
      print("You do not have the shiny key.")
    else
      door_locked = false
      rooms.locked_door_room.exits.north = "treasure_room"
      print("You unlocked the door!")
    end
  elseif current_room == "locked_door_room" and command == "north" and door_locked then
    print("The door is locked! You need to use the shiny key here.")
  elseif room.exits[command] then
    current_room = room.exits[command]
  else
    print("Invalid command. Enter north, south, east, west, take shiny key, use shiny key, c
  end
end
```

Expected Behavior or Output

Use this sequence to verify the puzzle: north, east, north, north, south, west, south, take shiny key, north, east, north, use shiny key, north.

The first north attempt from locked_door_room must be rejected. Taking the key removes it from start_room and adds it to inventory. Using it outside locked_door_room must be rejected. Using it there prints You unlocked the door!; the next north reaches treasure_room and prints You found the treasure. You win!. End-of-file is treated as quit.

Common Mistakes

- Leaving the shiny key in the room after adding it to inventory.
- Unlocking the door outside locked_door_room.
- Unlocking the door without checking inventory.
- Allowing north movement before door_locked becomes false.

- Describing the key as still on the floor after it has been collected.

Activities

1. Run the full sequence and record room, room items, inventory, and door state after every command.
2. Try take shiny key twice; the second attempt must not duplicate the item.
3. Try use shiny key in start_room and clearing; both attempts must be rejected.
4. Try north from locked_door_room before and after unlocking.
5. Add a look command that announces room, visible items, inventory, and exits in a consistent order.

Accessibility Practice

Announce room description, visible items, inventory, and prompt as separate text lines. Every failed action explains why it failed. Listen to the full puzzle sequence and verify that key location and door state are unambiguous.

Skills Checklist

☐ I can model the five canonical rooms.
☐ I can transfer the shiny key from a room to inventory.
☐ I can prevent duplicate pickup.
☐ I can require key use in the locked-door room.
☐ I can block and unlock the north exit.
☐ I can end the game when the treasure room is reached.

Lesson 10.4 Command-Line To-Do Manager

Reading References

- Lua 5.4 Reference Manual: https://www.lua.org/manual/5.4/
- Programming in Lua: https://www.lua.org/pil/
- Corresponding Python lesson in the canonical course map.

Learning Objectives

- Explain the concept using Lua terminology.
- Trace the worked example and predict its behavior.
- Modify and test the example while preserving the shared data contract.

Concept Explanation

The task record must include id, title, description, done, and created_at. Keep CRUD operations separate from the input loop.

Worked Example

```lua
local tasks = {}
local function add_task(id, title, description, created_at)
  assert(title ~= nil and title ~= "", "title is required")
  tasks[#tasks + 1] = {id = id, title = title, description = description,
    done = false, created_at = created_at}
end
add_task(1, "Practice Lua", "Complete the shared examples", "2025-01-15T09:00:00Z")
print(tasks[1].id, tasks[1].title, tasks[1].done)
```

Expected Behavior or Output

```
1  Practice Lua    false
```

Common Mistakes

- Omitting one or more required task fields from a stored record.
- Using a list position as a permanent task identifier after deletion or reordering.
- Treating malformed persistence data as an empty task list without reporting the error.

Activities

1. Create a task with all five canonical fields and list it in a stable format.
2. Update its done state, then search for it by identifier.
3. Delete an existing ID and attempt to delete a missing ID; compare the messages.
4. Save and reload the records, then verify every field is preserved.

Accessibility Practice

Navigate by line, word, and character; identify delimiters and nesting verbally; execute from the terminal; and read the output and any error messages sequentially. Do not rely on syntax highlighting or color to understand the example.

Skills Checklist

- ☐ I can explain the concept in Lua terminology.
- ☐ I can trace and predict the example's behavior.
- ☐ I can run and modify the example from the terminal.
- ☐ I can test a boundary or invalid case.
- ☐ I can explain the relevant Lua-specific semantic difference.

Lesson 10.5 Refinement and Accessibility

Reading References

- Lua 5.4 Reference Manual: https://www.lua.org/manual/5.4/
- Programming in Lua: https://www.lua.org/pil/
- Corresponding Python lesson in the canonical course map.

Learning Objectives

- Explain the concept using Lua terminology.
- Trace the worked example and predict its behavior.
- Modify and test the example while preserving the shared data contract.

Concept Explanation

A command-line application needs explicit help, validation, confirmations, and readable errors. Keep all critical information in text.

Worked Example

```lua
local function validate_title(title)
  if title == nil or title:match("^%s*$") then
    return false, "Title is required; enter at least one non-space character."
  end
  return true, "Title accepted."
end
local ok, message = validate_title("   ")
print(message)
```

Expected Behavior or Output

```
Title is required; enter at least one non-space character.
```

Common Mistakes

- Copying a Python assumption into Lua without checking the Lua 5.4 semantics.
- Changing the canonical input values and then comparing output against the shared fixture.
- Skipping a negative or boundary case for the lesson's main operation.

Activities

1. Trace the 10.5 Refinement and Accessibility worked example and explain each statement in Lua terms.
2. Change one noncanonical test value while keeping the shared fixture intact; predict the result before execution.

3. Add a boundary or invalid-input test for the main operation in this lesson.
4. Explain one semantic difference between Lua and Python that affects 10.5 refinement and accessibility.

Accessibility Practice

Navigate by line, word, and character; identify delimiters and nesting verbally; execute from the terminal; and read the output and any error messages sequentially. Do not rely on syntax highlighting or color to understand the example.

Skills Checklist

☐ I can explain the concept in Lua terminology.
☐ I can trace and predict the example's behavior.
☐ I can run and modify the example from the terminal.
☐ I can test a boundary or invalid case.
☐ I can explain the relevant Lua-specific semantic difference.

Overview

Plan a text project, build the shared text adventure in two parts, build the command-line to-do manager, and refine input validation.

Lesson 10.1: Planning a Text-Based Project

Reading References

- Programming in Lua, 4th edition (Roberto Ierusalimschy): Chapter 6 (Functions) and Chapter 5 (Tables)
- Lua 5.4 Reference Manual: Section 3.3 (Statements)
- Matching Python lesson: Lesson 10.1 Project Planning and Design

Learning Objectives

- Write a short requirements list.
- Design a command loop with a table of handlers.
- Plan in small steps and test each one.

Concept Explanation

A project is easier to finish when you plan it in small steps. Write down *what the program must do* (requirements), *what information it keeps* (data), and *the commands a person can give* (interface). Build one small piece, test it, and then add the next.

Text-based projects share a *command loop*: show a prompt, read a line, find the matching action, and repeat until the user quits. A good Lua design stores

actions in a table whose keys are command names and whose values are functions. A new command is then one new entry, and an unknown command is a failed lookup that prints a helpful message.

The loop must stop when the input ends (`nil`) as well as on `quit`.

The example is a skeleton with three commands. Typing `help`, `look`, an unknown word, and `quit` shows each path.

Worked Example

```
local commands = {
  help = function() print("Commands: help, look, quit") end,
  look = function() print("You are in a dark cave.") end,
}

print("Type a command:")
local line = io.read()
while line ~= nil and line ~= "quit" do
  local handler = commands[line]
  if handler then
    handler()
  else
    print("I don't understand that command. Type help for a list.")
  end
  print("Type a command:")
  line = io.read()
end
print("Goodbye!")
```

Expected Behavior or Output

When the standard input is the following lines (the program's own prompts and results are shown separately because piped input is not echoed):

```
help
look
dance
quit
```

the program prints:

```
Type a command:
Commands: help, look, quit
Type a command:
You are in a dark cave.
Type a command:
I don't understand that command. Type help for a list.
```

```
Type a command:
Goodbye!
```

Common Mistakes

1. Starting to code before listing the requirements.
2. Using a long chain of `if` statements for commands instead of a table of handlers.
3. Forgetting that an unknown command should explain what to do next.

Activities

1. Write five requirements for the text adventure and the to-do list.
2. Add a `score` command that prints the shared scores.
3. Add the command aliases `h` and `l` without copying the functions.
4. Describe the data for the capstone in three sentences.

Accessibility Practice

Prompts and responses are on separate lines and every response is a complete sentence. Test the prompts by listening to them before you add features.

Skills Checklist

☐ I can list requirements.
☐ I can design a command loop.
☐ I can use a table of handlers.
☐ I handle unknown commands.
☐ I handle the end of input.

Lesson 10.2: Text Adventure, Part 1: Rooms and Movement

Reading References

- Programming in Lua, 4th edition (Roberto Ierusalimschy): Chapter 5 (Tables)
- Lua 5.4 Reference Manual: Section 3.4.9 (Table Constructors)
- Matching Python lesson: Lesson 10.2 Building a Text-Based Adventure Game (Part 1)

Learning Objectives

- Represent rooms and exits with tables.
- Move the player by command.
- Handle impossible moves and the end of input.

Concept Explanation

A room is a table with a `description` and an `exits` table that maps a direction to the name of another room. The program remembers the player's location in one variable, `current_room`. A move looks up the command in the current room's exits; if it is found, `current_room` changes, otherwise the program explains that the move is impossible.

This is the same three-room layout as the Python course: a cave, a tunnel, and a clearing. Part 2 adds items and a locked door.

The example is played with the commands north, east, west, up, south, and quit. The command `up` is not an exit anywhere, so it prints the help message.

Worked Example

```
local rooms = {
  start_room = {
    description = "You are in a dimly lit cave. There's a narrow passage to the NORTH.",
    exits = {north = "tunnel"},
  },
  tunnel = {
    description = "The tunnel is damp and cold. You can go SOUTH back to the cave or EAST to
    exits = {south = "start_room", east = "clearing"},
  },
  clearing = {
    description = "You've reached a small, sunny clearing. A faint path leads WEST.",
    exits = {west = "tunnel"},
  },
}

local current_room = "start_room"

while true do
  print("---")
  print(rooms[current_room].description)
  print("What do you want to do?")
  local command = io.read()
  if command == nil then command = "quit" end
  command = command:lower()

  if rooms[current_room].exits[command] then
    current_room = rooms[current_room].exits[command]
  elseif command == "quit" then
    print("Exiting game. Goodbye!")
    break
  else
```

```
    print("I don't understand that command or you can't go that way.")
  end
end
```

Expected Behavior or Output

When the standard input is the following lines (the program's own prompts and results are shown separately because piped input is not echoed):

```
north
east
west
up
south
quit
```

the program prints:

```
---
You are in a dimly lit cave. There's a narrow passage to the NORTH.
What do you want to do?
---
The tunnel is damp and cold. You can go SOUTH back to the cave or EAST to a clearing.
What do you want to do?
---
You've reached a small, sunny clearing. A faint path leads WEST.
What do you want to do?
---
The tunnel is damp and cold. You can go SOUTH back to the cave or EAST to a clearing.
What do you want to do?
I don't understand that command or you can't go that way.
---
The tunnel is damp and cold. You can go SOUTH back to the cave or EAST to a clearing.
What do you want to do?
---
You are in a dimly lit cave. There's a narrow passage to the NORTH.
What do you want to do?
Exiting game. Goodbye!
```

Common Mistakes

1. Looking up a direction that is not in `exits` and expecting an error. The lookup returns `nil`, which is the signal to explain the move is impossible.
2. Forgetting to treat the end of input as `quit`, so the program loops forever when input is piped in.
3. Misspelling a room name in `exits`, so the move leads to a room that does not exist.

Activities

1. Trace the six commands by hand and write the room after each one.
2. Add a fourth room reached from the clearing and test it.
3. Add the aliases `n`, `s`, `e`, and `w`.
4. Check that every exit names a real room by writing a loop that tests the whole table.

Accessibility Practice

Each turn prints a separator line, the description, and the prompt on its own line. Listen to a whole turn to confirm that a player could follow it without seeing the screen.

Skills Checklist

- ☐ I can represent rooms with tables.
- ☐ I can move the player by command.
- ☐ I can handle an impossible move.
- ☐ I treat end of input as quit.
- ☐ I can trace a game by hand.

Lesson 10.3: Text Adventure, Part 2: Items and a Locked Door

Reading References

- Programming in Lua, 4th edition (Roberto Ierusalimschy): Chapter 5 (Tables) and Chapter 6 (Functions)
- Lua 5.4 Reference Manual: Section 6.6 (Table Manipulation)
- Matching Python lesson: Lesson 10.3 Building a Text-Based Adventure Game (Part 2)

Learning Objectives

- Keep items in rooms and in an inventory.
- Implement take and use commands.
- Block and then allow a move with a locked door.

Concept Explanation

Part 2 adds *state*: items in rooms, an inventory, and a door that is locked until the player uses the key. This is the shared specification used by every language track: three rooms (`start`, `locked_door_room`, `treasure_room`); the commands `north`, `south`, `east`, `west`, `take shiny key`, `use shiny key`, and `quit`; the door between the chamber and the treasure room stays locked until the key is used there; reaching the treasure room ends the game.

Items are lists of names. Taking an item removes it from the room's list and appends it to the inventory. The locked door is a Boolean variable, `door_locked`. A move through the door checks it first.

The program is played with: north (into the chamber), north (the locked door refuses), south, take shiny key, north, use shiny key, north (the treasure room).

Worked Example

```
local rooms = {
  start = {
    description = "A dark cave.",
    exits = {north = "locked_door_room"},
    items = {"shiny key"},
  },
  locked_door_room = {
    description = "A small chamber with a heavy door to the NORTH.",
    exits = {south = "start", north = "treasure_room"},
    items = {},
  },
  treasure_room = {
    description = "A glittering room full of treasure. You win!",
    exits = {south = "locked_door_room"},
    items = {},
  },
}

local function index_of(list, value)
  for index, item in ipairs(list) do
    if item == value then return index end
  end
  return nil
end

local current_room = "start"
local inventory = {}
local door_locked = true

while true do
  local room = rooms[current_room]
  print("")
  print(room.description)
  if current_room == "treasure_room" then break end
  if #room.items > 0 then print("You see: " .. table.concat(room.items, ", ")) end
  print("Your inventory: " .. (#inventory > 0 and table.concat(inventory, ", ") or "empty"))
  print("What do you want to do?")
```

```
  local command = io.read()
  if command == nil then command = "quit" end
  command = command:lower()

  if command == "take shiny key" and index_of(room.items, "shiny key") then
    table.remove(room.items, index_of(room.items, "shiny key"))
    inventory[#inventory + 1] = "shiny key"
    print("You picked up the shiny key.")
  elseif command == "use shiny key" and current_room == "locked_door_room" and index_of(inve
    door_locked = false
    print("You unlocked the door!")
  elseif room.exits[command] then
    if current_room == "locked_door_room" and command == "north" and door_locked then
      print("The door is locked! You need a key.")
    else
      current_room = room.exits[command]
    end
  elseif command == "quit" then
    print("Exiting game. Goodbye!")
    break
  else
    print("I don't understand that command.")
  end
end
```

Expected Behavior or Output

When the standard input is the following lines (the program's own prompts and results are shown separately because piped input is not echoed):

```
north
north
south
take shiny key
north
use shiny key
north
```

the program prints:

```
A dark cave.
You see: shiny key
Your inventory: empty
What do you want to do?
```

```
A small chamber with a heavy door to the NORTH.
Your inventory: empty
What do you want to do?
The door is locked! You need a key.

A small chamber with a heavy door to the NORTH.
Your inventory: empty
What do you want to do?

A dark cave.
You see: shiny key
Your inventory: empty
What do you want to do?
You picked up the shiny key.

A dark cave.
Your inventory: shiny key
What do you want to do?

A small chamber with a heavy door to the NORTH.
Your inventory: shiny key
What do you want to do?
You unlocked the door!

A small chamber with a heavy door to the NORTH.
Your inventory: shiny key
What do you want to do?

A glittering room full of treasure. You win!
```

Common Mistakes

1. Making the door's locked state part of the exits so that the ”locked” message can never be reached.
2. Defining an exit to a room that is missing from the `rooms` table, which crashes when the player arrives.
3. Printing a fixed room description that still mentions an item after the player has taken it.

Activities

1. Play the program with the shown commands and follow each turn on paper.
2. Try `use shiny key` in the cave and describe what happens.
3. Add a second item and require both to open the door.
4. Write a short list of the game's states (location, inventory, door) and say

which command changes each.

Accessibility Practice

The room description never mentions the key. The separate line 'You see:' lists items, so the description stays true after the key is taken and the screen reader announces the current state.

Skills Checklist

- ☐ I can store items in rooms and an inventory.
- ☐ I can implement take and use commands.
- ☐ I can block and allow a move.
- ☐ I can list the game's state variables.
- ☐ I can keep descriptions true after state changes.

Lesson 10.4: Command-Line To-Do List

Reading References

- Programming in Lua, 4th edition (Roberto Ierusalimschy): Chapter 15 (Data Files and Serialization)
- Lua 5.4 Reference Manual: Section 6.8 (Input and Output Facilities)
- Matching Python lesson: Lesson 10.4 Building a Command-Line To-Do List

Learning Objectives

- Keep records with the shared fields.
- Save and load them as JSON.
- Offer add, view, mark-done, save, load, and quit commands.

Concept Explanation

A to-do list ties together records, loops, functions, and files. Each task has the shared fields `id`, `title`, `description`, `done`, and `created_at`. The program keeps a list of tasks in memory, shows a numbered menu, and saves the list to `todos.json` when asked and on exit.

Saving uses the teaching JSON helper from Lesson 8.5 (included here so the example runs on its own). On start-up the program loads the file if it exists. A missing file is normal on first use, and a damaged file must not crash the program: it prints a message and starts with an empty list.

New ids are one more than the highest existing id, so ids stay unique even after a task is removed. The status is shown as the words `done` and `not done`, never with a symbol or color alone.

In the sample session the user adds two tasks, views the list, marks task 2 as done, views it again, tries an invalid menu choice, views the list, and quits.

Worked Example

```
local json = {}

local escapes = {['"'] = '\\"', ['\\'] = '\\\\', ['\n'] = '\\n', ['\r'] = '\\r', ['\t'] = '\

function json.encode(value)
  local kind = type(value)
  if kind == "string" then
    return '"' .. value:gsub('[%c"\\]', function(c)
      return escapes[c] or string.format("\\u%04x", c:byte())
    end) .. '"'
  elseif kind == "number" then
    return math.type(value) == "integer" and tostring(value) or string.format("%.14g", value
  elseif kind == "boolean" then
    return tostring(value)
  elseif kind == "nil" then
    return "null"
  elseif kind == "table" then
    local parts = {}
    if value[1] ~= nil or next(value) == nil then
      for _, item in ipairs(value) do parts[#parts + 1] = json.encode(item) end
      return "[" .. table.concat(parts, ",") .. "]"
    end
    local keys = {}
    for key in pairs(value) do keys[#keys + 1] = key end
    table.sort(keys)
    for _, key in ipairs(keys) do
      parts[#parts + 1] = json.encode(tostring(key)) .. ":" .. json.encode(value[key])
    end
    return "{" .. table.concat(parts, ",") .. "}"
  end
  error("cannot encode a value of type " .. kind, 0)
end

function json.decode(text)
  local position = 1
  local read_value

  local function skip()
    position = text:find("%S", position) or #text + 1
  end
```

```
local function fail(message)
  error(message .. " at position " .. position, 0)
end

local function read_string()
  local pieces = {}
  position = position + 1
  while true do
    local c = text:sub(position, position)
    if c == "" then fail("unterminated string") end
    if c == '"' then
      position = position + 1
      return table.concat(pieces)
    elseif c == "\\" then
      local n = text:sub(position + 1, position + 1)
      local simple = {n = "\n", t = "\t", r = "\r", ['"'] = '"', ["\\"] = "\\", ["/"] = "/
      if n == "u" then
        pieces[#pieces + 1] = utf8.char(tonumber(text:sub(position + 2, position + 5), 16)
        position = position + 6
      elseif simple[n] then
        pieces[#pieces + 1] = simple[n]
        position = position + 2
      else
        fail("bad escape")
      end
    else
      pieces[#pieces + 1] = c
      position = position + 1
    end
  end
end

function read_value()
  skip()
  local c = text:sub(position, position)
  if c == "{" then
    local object = {}
    position = position + 1
    skip()
    if text:sub(position, position) == "}" then position = position + 1; return object end
    while true do
      skip()
      if text:sub(position, position) ~= '"' then fail("expected a key") end
      local key = read_string()
      skip()
      if text:sub(position, position) ~= ":" then fail("expected ':'") end
```

```
        position = position + 1
        object[key] = read_value()
        skip()
        local d = text:sub(position, position)
        position = position + 1
        if d == "}" then return object end
        if d ~= "," then position = position - 1; fail("expected ',' or '}'") end
      end
    elseif c == "[" then
      local list = {}
      position = position + 1
      skip()
      if text:sub(position, position) == "]" then position = position + 1; return list end
      while true do
        list[#list + 1] = read_value()
        skip()
        local d = text:sub(position, position)
        position = position + 1
        if d == "]" then return list end
        if d ~= "," then position = position - 1; fail("expected ',' or ']'") end
      end
    elseif c == '"' then
      return read_string()
    elseif text:find("^true", position) then
      position = position + 4; return true
    elseif text:find("^false", position) then
      position = position + 5; return false
    elseif text:find("^null", position) then
      position = position + 4; return nil
    end
    local number = text:match("^-?%d+%.?%d*[eE]?[+-]?%d*", position)
    if not number or number == "" then fail("unexpected character") end
    position = position + #number
    return tonumber(number)
  end

  local result = read_value()
  skip()
  if position <= #text then fail("unexpected extra text") end
  return result
end

local TODO_FILE = "todos.json"
local todos = {}

local function load_todos()
```

```
  local file = io.open(TODO_FILE, "r")
  if not file then
    print("No existing tasks found.")
    todos = {}
    return
  end
  local text = file:read("a")
  file:close()
  local ok, data = pcall(json.decode, text)
  if ok and type(data) == "table" then
    todos = data
    print("Tasks loaded.")
  else
    print("Error: todos.json is not valid JSON. Starting with an empty list.")
    todos = {}
  end
end

local function save_todos()
  local file = assert(io.open(TODO_FILE, "w"))
  file:write(json.encode(todos))
  file:close()
  print("Tasks saved.")
end

local function next_id()
  local highest = 0
  for _, task in ipairs(todos) do
    if task.id > highest then highest = task.id end
  end
  return highest + 1
end

local function add_task(title, description)
  title = (title or ""):match("^%s*(.-)%s*$")
  if title == "" then
    print("A title is required.")
    return
  end
  todos[#todos + 1] = {
    id = next_id(),
    title = title,
    description = (description or ""):match("^%s*(.-)%s*$"),
    done = false,
    created_at = os.date("!%Y-%m-%dT%H:%M:%S"),
  }
```

```
  print("Task '" .. title .. "' added.")
end

local function view_tasks()
  if #todos == 0 then
    print("No tasks in your list.")
    return
  end
  print("--- Your To-Do List ---")
  for _, task in ipairs(todos) do
    print(string.format("%d. %s (%s)", task.id, task.title, task.done and "done" or "not do
  end
  print("-----------------------")
end

local function mark_done(text)
  local id = tonumber(text)
  if not id then
    print("Please enter a whole number.")
    return
  end
  for _, task in ipairs(todos) do
    if task.id == id then
      task.done = true
      print("Task '" .. task.title .. "' marked as done.")
      return
    end
  end
  print("No task has that number.")
end

local function show_menu()
  print("")
  print("--- To-Do List Menu ---")
  print("1. Add Task")
  print("2. View Tasks")
  print("3. Mark Task Done")
  print("4. Save Tasks")
  print("5. Load Tasks")
  print("6. Quit")
  print("-----------------------")
end

load_todos()
while true do
  show_menu()
```

```
  print("Enter your choice (1-6):")
  local choice = io.read()
  if choice == nil then choice = "6" end

  if choice == "1" then
    print("Enter task title:")
    local title = io.read()
    print("Enter task description:")
    local description = io.read()
    add_task(title, description)
  elseif choice == "2" then
    view_tasks()
  elseif choice == "3" then
    view_tasks()
    print("Enter task number to mark done:")
    mark_done(io.read())
  elseif choice == "4" then
    save_todos()
  elseif choice == "5" then
    load_todos()
  elseif choice == "6" then
    save_todos()
    print("Goodbye!")
    break
  else
    print("Invalid choice. Please try again.")
  end
end
```

Expected Behavior or Output

When the standard input is the following lines (the program's own prompts and results are shown separately because piped input is not echoed):

```
1
Study programming
Finish Unit 10
1
Review scores
Check the four scores
2
3
2
3
7
2
```

```
6
```

the program prints:

```
No existing tasks found.

--- To-Do List Menu ---
1. Add Task
2. View Tasks
3. Mark Task Done
4. Save Tasks
5. Load Tasks
6. Quit
-----------------------
Enter your choice (1-6):
Enter task title:
Enter task description:
Task 'Study programming' added.

--- To-Do List Menu ---
1. Add Task
2. View Tasks
3. Mark Task Done
4. Save Tasks
5. Load Tasks
6. Quit
-----------------------
Enter your choice (1-6):
Enter task title:
Enter task description:
Task 'Review scores' added.

--- To-Do List Menu ---
1. Add Task
2. View Tasks
3. Mark Task Done
4. Save Tasks
5. Load Tasks
6. Quit
-----------------------
Enter your choice (1-6):
--- Your To-Do List ---
1. Study programming (not done)
2. Review scores (not done)
-----------------------

--- To-Do List Menu ---
```

```
1. Add Task
2. View Tasks
3. Mark Task Done
4. Save Tasks
5. Load Tasks
6. Quit
----------------------
Enter your choice (1-6):
--- Your To-Do List ---
1. Study programming (not done)
2. Review scores (not done)
----------------------
Enter task number to mark done:
Task 'Review scores' marked as done.

--- To-Do List Menu ---
1. Add Task
2. View Tasks
3. Mark Task Done
4. Save Tasks
5. Load Tasks
6. Quit
----------------------
Enter your choice (1-6):
--- Your To-Do List ---
1. Study programming (not done)
2. Review scores (done)
----------------------
Enter task number to mark done:
No task has that number.

--- To-Do List Menu ---
1. Add Task
2. View Tasks
3. Mark Task Done
4. Save Tasks
5. Load Tasks
6. Quit
----------------------
Enter your choice (1-6):
--- Your To-Do List ---
1. Study programming (not done)
2. Review scores (done)
----------------------

--- To-Do List Menu ---
```

```
1. Add Task
2. View Tasks
3. Mark Task Done
4. Save Tasks
5. Load Tasks
6. Quit
----------------------
Enter your choice (1-6):
Tasks saved.
Goodbye!
```

Common Mistakes

1. Saving the file but never loading it, so tasks disappear between runs.
2. Reusing a deleted task's id, which can confuse saved references.
3. Letting a damaged `todos.json` crash the program instead of printing a message.

Activities

1. Run the sample session and then run the program again with the input `2` then `6` to see the saved tasks.
2. Add a command to delete a task by number.
3. Open `todos.json` and read the saved text. Edit it to remove a bracket and rerun.
4. Add a `description` display to the view command.

Accessibility Practice

Every menu is printed as numbered lines and each prompt is on its own line. The status words are `done` and `not done`, not symbols, so the list reads clearly in speech and braille.

Skills Checklist

☐ I can keep records with the shared fields.
☐ I can save and load JSON.
☐ I handle a missing or damaged file.
☐ I keep ids unique.
☐ I show status in words.

Lesson 10.5: User Experience and Input Validation

Reading References

- Programming in Lua, 4th edition (Roberto Ierusalimschy): Chapter 6 (Functions) and Chapter 16 (Compilation, Execution, and Errors)

- Lua 5.4 Reference Manual: Section 6.8 (io.read)
- Matching Python lesson: Lesson 10.5 Refining User Experience and Validation

Learning Objectives

- Write reusable validation functions.
- Limit retries and handle the end of input.
- Write clear, accessible messages.

Concept Explanation

Reusable validation functions make every menu and form consistent. `ask_number(prompt, low, high, attempts)` prints a prompt, reads a line, converts it, checks the range, and either returns the number or explains the problem and asks again. After the allowed attempts, or at the end of input, it returns `nil` and a reason. `ask_yes_no(prompt)` accepts `y`, `yes`, `n`, and `no`, in any letter case.

Good messages say *what was wrong* and *what to do*: "Enter a whole number from 0 to 100," not "Invalid." They never depend on color, position, or symbols alone, and they use the same wording every time.

The example reads a score with the typed lines abc, 105, and 88, then asks a yes or no question with the lines maybe and y.

Worked Example

```
local function ask_number(prompt, low, high, attempts)
  for attempt = 1, attempts do
    print(prompt)
    local text = io.read()
    if text == nil then return nil, "No more input." end
    local number = tonumber(text)
    if number and number == math.floor(number) and number >= low and number <= high then
      return math.tointeger(number)
    end
    print(string.format("Enter a whole number from %d to %d. You have %d attempt(s) left.",
  end
  return nil, "Too many invalid attempts."
end

local function ask_yes_no(prompt)
  while true do
    print(prompt .. " (yes or no)")
    local text = io.read()
    if text == nil then return nil end
    text = text:lower()
```

```
    if text == "y" or text == "yes" then return true end
    if text == "n" or text == "no" then return false end
    print("Please answer yes or no.")
  end
end

local score, reason = ask_number("Enter a score from 0 to 100:", 0, 100, 3)
if score then
  print("Accepted score: " .. score)
else
  print("No score: " .. reason)
end

local keep = ask_yes_no("Save this score?")
print("Save answer:", keep)
```

Expected Behavior or Output

When the standard input is the following lines (the program's own prompts and results are shown separately because piped input is not echoed):

```
abc
105
88
maybe
y
```

the program prints:

```
Enter a score from 0 to 100:
Enter a whole number from 0 to 100. You have 2 attempt(s) left.
Enter a score from 0 to 100:
Enter a whole number from 0 to 100. You have 1 attempt(s) left.
Enter a score from 0 to 100:
Accepted score: 88
Save this score? (yes or no)
Please answer yes or no.
Save this score? (yes or no)
Save answer:    true
```

Common Mistakes

1. Writing the message ”Invalid” without saying what is valid.
2. Accepting a decimal such as 88.5 for a field that requires a whole number.
3. Looping forever on bad input instead of limiting attempts.

Activities

1. Run the sample input and read each message aloud.
2. Run with three invalid numbers and read the final message.
3. Add a function `ask_text(prompt)` that rejects empty answers.
4. Rewrite two messages from earlier lessons so they say what to do next.

Accessibility Practice

Check that every error message ends with an action the user can take, and that the same situation always produces the same words.

Skills Checklist

- ☐ I can write reusable validation functions.
- ☐ I limit retries.
- ☐ I handle the end of input.
- ☐ I write messages that say what to do.
- ☐ I keep wording consistent.

12.13 Unit 11: Capstone — Accessible Personal Data Manager

Overview

This unit develops the concepts in the same order and with the same problem context as the Python reference. Predict the output before running each example. Use Lua 5.4 semantics, not assumptions imported from Python.

Lesson 11.1 Requirements and Interface Design

Reading References

- Lua 5.4 Reference Manual: https://www.lua.org/manual/5.4/
- Programming in Lua: https://www.lua.org/pil/
- Corresponding Python lesson in the canonical course map.

Learning Objectives

- Explain the concept using Lua terminology.
- Trace the worked example and predict its behavior.
- Modify and test the example while preserving the shared data contract.

Concept Explanation

Separate the core data model from the interface. A terminal interface is an appropriate Lua implementation while preserving the shared user-facing require-

ments.

Worked Example

```lua
local required_fields = {"id", "title", "description", "category", "created_at", "updated_at
for _, field in ipairs(required_fields) do print(field) end
```

Expected Behavior or Output

```
id
title
description
category
created_at
updated_at
```

Common Mistakes

- Dropping one of the six required capstone record fields during serialization.
- Updating a record without refreshing updated_at.
- Claiming screen-reader compatibility without testing keyboard and assistive-technology workflows.

Activities

1. Create a record containing all six canonical capstone fields and validate each field.
2. Exercise add, list, search, update, and delete with both existing and missing IDs.
3. Save and reload a record; compare every field before and after persistence.
4. Write a text-only keyboard interaction guide and list the accessibility tests still required.

Accessibility Practice

Navigate by line, word, and character; identify delimiters and nesting verbally; execute from the terminal; and read the output and any error messages sequentially. Do not rely on syntax highlighting or color to understand the example.

Skills Checklist

☐ I can explain the concept in Lua terminology.
☐ I can trace and predict the example's behavior.
☐ I can run and modify the example from the terminal.
☐ I can test a boundary or invalid case.
☐ I can explain the relevant Lua-specific semantic difference.

Lesson 11.2 Data Model and Storage

Reading References

- Lua 5.4 Reference Manual: https://www.lua.org/manual/5.4/
- Programming in Lua: https://www.lua.org/pil/
- Corresponding Python lesson in the canonical course map.

Learning Objectives

- Explain the concept using Lua terminology.
- Trace the worked example and predict its behavior.
- Modify and test the example while preserving the shared data contract.

Concept Explanation

The capstone record has six required fields. Validate them before storage; keep timestamps in a consistent documented format.

Worked Example

```lua
local record = {
  id = 1, title = "Programming notes",
  description = "Review lists and dictionaries",
  category = "Study",
  created_at = "2025-01-15T09:00:00Z",
  updated_at = "2025-01-15T09:00:00Z"
}
print(record.id, record.title, record.category)
```

Expected Behavior or Output

```
1  Programming notes   Study
```

Common Mistakes

- Ignoring the second return value from io.open when a file cannot be opened.
- Leaving a file handle open after a successful or failed operation.
- Assuming Lua includes a standard JSON or full CSV parser.

Activities

1. Test the file operation with an existing file and a missing-file path.
2. Add an explicit failure message using the error returned by io.open.
3. Close the file on every control path and explain the resource-lifetime rule.
4. Document the selected JSON/CSV dependency and test malformed and quoted-field inputs.

Accessibility Practice

Navigate by line, word, and character; identify delimiters and nesting verbally; execute from the terminal; and read the output and any error messages sequentially. Do not rely on syntax highlighting or color to understand the example.

Skills Checklist

- ☐ I can explain the concept in Lua terminology.
- ☐ I can trace and predict the example's behavior.
- ☐ I can run and modify the example from the terminal.
- ☐ I can test a boundary or invalid case.
- ☐ I can explain the relevant Lua-specific semantic difference.

Lesson 11.3 Create, Read, Update, Delete

Reading References

- Lua 5.4 Reference Manual: https://www.lua.org/manual/5.4/
- Programming in Lua: https://www.lua.org/pil/
- Corresponding Python lesson in the canonical course map.

Learning Objectives

- Explain the concept using Lua terminology.
- Trace the worked example and predict its behavior.
- Modify and test the example while preserving the shared data contract.

Concept Explanation

Implement CRUD as separate functions. Updating a record should also update updated_at; deleting a missing ID should return a clear status.

Worked Example

```lua
local records = {{
  id = 1,
  title = "Programming notes",
  description = "Review lists and dictionaries",
  category = "Study",
  created_at = "2025-01-15T09:00:00Z",
  updated_at = "2025-01-15T09:00:00Z"
}}
local function find_record(id)
  for _, record in ipairs(records) do
    if record.id == id then return record end
```

```
  end
  return nil
end
local function update_title(id, title, updated_at)
  local record = find_record(id)
  if not record then return false end
  record.title = title
  record.updated_at = updated_at
  return true
end
local function delete_record(id)
  for index, record in ipairs(records) do
    if record.id == id then
      table.remove(records, index)
      return true
    end
  end
  return false
end
local found = find_record(1)
print("Found:", found.title, found.category)
print("Updated:", update_title(1, "Revised notes", "2025-01-16T09:00:00Z"))
print("Updated title:", records[1].title)
print("Deleted:", delete_record(1))
print("Missing delete:", delete_record(9))
```

Expected Behavior or Output

```
Found: Programming notes   Study
Updated:    true
Updated title:  Revised notes
Deleted:    true
Missing delete: false
```

Common Mistakes

- Copying a Python assumption into Lua without checking the Lua 5.4 semantics.
- Changing the canonical input values and then comparing output against the shared fixture.
- Skipping a negative or boundary case for the lesson's main operation.

Activities

1. Trace the 11.3 Create, Read, Update, Delete worked example and explain each statement in Lua terms.

2. Change one noncanonical test value while keeping the shared fixture intact; predict the result before execution.
3. Add a boundary or invalid-input test for the main operation in this lesson.
4. Explain one semantic difference between Lua and Python that affects 11.3 create, read, update, delete.

Accessibility Practice

Navigate by line, word, and character; identify delimiters and nesting verbally; execute from the terminal; and read the output and any error messages sequentially. Do not rely on syntax highlighting or color to understand the example.

Skills Checklist

☐ I can explain the concept in Lua terminology.
☐ I can trace and predict the example's behavior.
☐ I can run and modify the example from the terminal.
☐ I can test a boundary or invalid case.
☐ I can explain the relevant Lua-specific semantic difference.

Lesson 11.4 Validation and Tests

Reading References

- Lua 5.4 Reference Manual: https://www.lua.org/manual/5.4/
- Programming in Lua: https://www.lua.org/pil/
- Corresponding Python lesson in the canonical course map.

Learning Objectives

- Explain the concept using Lua terminology.
- Trace the worked example and predict its behavior.
- Modify and test the example while preserving the shared data contract.

Concept Explanation

Test required fields, duplicate identifiers, not-found behavior, and persistence failures. Assertions should verify results, not merely execute examples.

Worked Example

```lua
local function valid_record(record)
  return type(record.id) == "number"
    and type(record.title) == "string" and record.title ~= ""
    and type(record.description) == "string"
    and type(record.category) == "string"
```

```
    and type(record.created_at) == "string"
    and type(record.updated_at) == "string"
end
assert(valid_record({id=1,title="A",description="B",category="C",created_at="now",updated_at
print("record validation passed")
```

Expected Behavior or Output

```
record validation passed
```

Common Mistakes

- Dropping one of the six required capstone record fields during serialization.
- Updating a record without refreshing updated_at.
- Claiming screen-reader compatibility without testing keyboard and assistive-technology workflows.

Activities

1. Create a record containing all six canonical capstone fields and validate each field.
2. Exercise add, list, search, update, and delete with both existing and missing IDs.
3. Save and reload a record; compare every field before and after persistence.
4. Write a text-only keyboard interaction guide and list the accessibility tests still required.

Accessibility Practice

Navigate by line, word, and character; identify delimiters and nesting verbally; execute from the terminal; and read the output and any error messages sequentially. Do not rely on syntax highlighting or color to understand the example.

Skills Checklist

☐ I can explain the concept in Lua terminology.
☐ I can trace and predict the example's behavior.
☐ I can run and modify the example from the terminal.
☐ I can test a boundary or invalid case.
☐ I can explain the relevant Lua-specific semantic difference.

Lesson 11.5 Documentation and Presentation

Reading References

- Lua 5.4 Reference Manual: https://www.lua.org/manual/5.4/

- Programming in Lua: https://www.lua.org/pil/
- Corresponding Python lesson in the canonical course map.

Learning Objectives

- Explain the concept using Lua terminology.
- Trace the worked example and predict its behavior.
- Modify and test the example while preserving the shared data contract.

Concept Explanation

Document installation, launch commands, data format, keyboard interaction, errors, and known limitations. A screen-reader claim requires hands-on testing.

Worked Example

```lua
local help = {
  "Commands: list, add, search, update, delete, save, load, help, quit",
  "All actions report success or a textual error.",
  "Records use the shared six-field capstone schema."
}
for _, line in ipairs(help) do print(line) end
```

Expected Behavior or Output

```
Commands: list, add, search, update, delete, save, load, help, quit
All actions report success or a textual error.
Records use the shared six-field capstone schema.
```

Common Mistakes

- Dropping one of the six required capstone record fields during serialization.
- Updating a record without refreshing updated_at.
- Claiming screen-reader compatibility without testing keyboard and assistive-technology workflows.

Activities

1. Create a record containing all six canonical capstone fields and validate each field.
2. Exercise add, list, search, update, and delete with both existing and missing IDs.
3. Save and reload a record; compare every field before and after persistence.
4. Write a text-only keyboard interaction guide and list the accessibility tests still required.

Accessibility Practice

Navigate by line, word, and character; identify delimiters and nesting verbally; execute from the terminal; and read the output and any error messages sequentially. Do not rely on syntax highlighting or color to understand the example.

Runnable Reference Implementation

The repository includes a tested Lua domain and JSON-storage reference core in capstone/README.md. Install the JSON dependency and run the acceptance suite from the repository root using:

```
luarocks install dkjson && lua LUA/capstone/test_records.lua
```

The reference core does not replace the requirement to build and evaluate an accessible user interface.

Skills Checklist

- ☐ I can explain the concept in Lua terminology.
- ☐ I can trace and predict the example's behavior.
- ☐ I can run and modify the example from the terminal.
- ☐ I can test a boundary or invalid case.
- ☐ I can explain the relevant Lua-specific semantic difference.

Canonical shared data contract

The following values must remain identical to the Python source and the other general-purpose tracks:

- Student: Avery, age 16, height 1.72 meters, favorite subject Computer Science.
- Calculator: bill 24.50, tip rate 0.15; round half-up to the nearest cent; tip displays as $3.68 and total as $28.18.
- Grade thresholds: A 90, B 80, C 70, D 60, otherwise F. Scores: 82, 91, 76, 88; total 337; average 84.25.
- Inventory: Notebook (3 at $2.50), Pencil (10 at $0.50), Folder (4 at $1.25); total $17.50.
- Coordinates: (3, 4); distance from origin 5.
- Text: `A student studies programming. Programming develops problem-solving skills.`
- CSV fields: `student_id, first_name, last_name, score, active`.
- To-do fields: `id, title, description, done, created_at`.
- Capstone fields: `id, title, description, category, created_at, updated_at`.

Course projects and acceptance criteria

The text adventure uses the shared five-room graph: `start_room` (north → `tunnel`, shiny key), `tunnel` (south → `start_room`, east → `clearing`), `clearing` (west → `tunnel`, north → `locked_door_room`), `locked_door_room` (south → `clearing`, north → `treasure_room` only after unlocking), and terminal `treasure_room`. The player takes the shiny key in `start_room`, travels to `locked_door_room`, uses the key there, and enters `treasure_room` to win. Commands are `north`, `south`, `east`, `west`, `take shiny key`, `use shiny key`, and `quit`. The command-line to-do manager must implement create, list, update/complete, delete, save, and load with all five canonical fields. The capstone must implement list/display, add, search, update, delete, and persistence for the six-field record. A terminal interface is acceptable for Lua, but the data model, operations, acceptance tests, and keyboard/text accessibility requirements remain equivalent.

AI and assessment integrity

AI must not write, complete, autocomplete, repair, or generate course code or assessment solutions. Students may use only instructor-authorized conceptual explanations that do not produce deliverable code. Do not recommend AI-generated code as a workflow or as a first draft.

Validation status

The earlier smoke-execution result applies only to the previous version of this file. The current checker now compares exact stdout for all 53 lesson examples; run the updated Lua 5.4 workflow and complete end-to-end project/persistence tests before marking the projects verified.

Part XIII

PYTHON

Chapter 13

Introduction to Computer Science with Python

Before you start: complete Unit P: Computing Foundations Primer. It explains the language-neutral vocabulary and concepts used in every language track, using non-visual examples.

Accessible beginner curriculum

The Python curriculum is a complete introductory pathway in programming and core computer science. It is designed to be usable by blind and visually impaired learners who use screen readers, braille displays, magnification, keyboard navigation, or combinations of access technologies. It is also suitable for sighted beginners and mixed-vision classrooms.

Python is a practical first language for many learners because it has compact syntax, an interactive interpreter, and a broad standard library. The course uses those strengths to teach problem-solving rather than treating programming as memorizing commands.

Recommended sequence: complete Units 0–11 in order. The unit files group related units for navigation; use the headings and lesson numbers within each file.

Course outcomes

By the end of the curriculum, learners should be able to:

- Explain basic computer science concepts and the iterative software-development process.

- Configure a Python environment and use an editor, terminal, and interactive interpreter.
- Write, run, read, and modify programs using variables, built-in types, expressions, and input/output.
- Use Boolean expressions, conditionals, and loops to control program behavior.
- Work with strings, lists, tuples, dictionaries, and common collection operations.
- Design and call functions with parameters and return values.
- Explain and implement introductory classes, objects, attributes, methods, and inheritance.
- Read and write files, handle exceptions, and organize program behavior.
- Apply introductory testing and data-analysis techniques.
- Plan, implement, test, and explain text-based projects and a capstone application.
- Evaluate prompts, messages, and output for clarity when consumed sequentially with assistive technology.

These outcomes represent introductory proficiency, not mastery of every part of Python.

Prerequisites and intended learners

No previous programming experience is required. Learners need access to a computer on which Python can be installed or provided, a keyboard-accessible development environment, and a way to run terminal commands. Basic file management and keyboard navigation are helpful.

Screen-reader fluency and programming readiness are separate skills. A student who needs additional time to navigate an editor may still understand algorithms and programming concepts. Teachers can teach access workflows alongside programming; see the Python teacher's guide.

Learning sequence

Unit	Main topics	Evidence of progress
0. Accessible setup	Python installation, Visual Studio Code, screen-reader configuration, workspace, terminal, and environments	Open a project, create a file, run a program, and locate output

Unit	Main topics	Evidence of progress
1. Foundations	Computer science, hardware and software, problem solving, first program, variables, and basic types	Explain a simple program and identify its values and statements
2. Numeric programs	Software-development cycle, expressions, assignment, input/output, conversion, math operations, and formatted output	Design and run a small interactive calculation
3. Control flow	Boolean expressions, conditionals, lists, loops, and string operations	Express decisions and repetition and predict execution
4. Data structures	List operations, tuples, dictionaries, and organizing information	Select and manipulate a suitable collection
5. Input and iteration	Validating user input and using while loops for repeated interaction	Build a robust text-based interaction loop
6. Functions	Decomposition, parameters, return values, scope, and reusable logic	Divide a problem into named, testable operations
7. Object-oriented programming	Classes, objects, attributes, methods, and inheritance	Model a simple domain with related objects
8. Files and exceptions	Persistent data, file operations, exceptions, and safe resource handling	Read/write data and respond to common failures
9. Testing and data analysis	Testing program behavior and analyzing structured data	Validate behavior with repeatable checks
10. Project-based learning	Requirements, top-down design, text-based applications, and incremental implementation	Plan and build a multi-feature program
11. Capstone	Integration, testing, documentation, and presentation	Demonstrate and explain an independent solution

Materials and navigation

- Unit 0: Setting Up Your Accessible Python Development Environment
- Unit 1: Foundations of Computing and Getting Started with Python
- Unit 2: Writing Simple Programs and Numeric Computations
- Unit 3: Control Flow — Decisions and Loops
- Units 4–6: Data Structures and Functions
- Units 7–9: Object-Oriented Programming and Files
- Units 10–11: Projects and Capstone
- Teacher's guide: instructor planning, accessible instruction, assessment, and AI guidance

Return to the curriculum overview to compare the three language pathways.

Recommended environment

The setup unit contains the Python-specific runtime and project instructions. For a cross-language comparison and detailed installation steps for Visual Studio Code, VSCodium, and Theia IDE, see the standalone IDE setup lesson. In general, learners need:

- A currently supported Python 3 release compatible with the course examples.
- Visual Studio Code or Theia IDE, or another keyboard-accessible editor that supports Python and the learner's screen reader.
- Python language tooling for the selected editor.
- A screen reader such as NVDA, JAWS, VoiceOver, or Narrator, a braille display, magnification, or the access technology used by the learner.
- A terminal for running programs and reading text output.

Use a project-specific virtual environment for packages. Keep dependencies limited in the introductory sequence; when a third-party package is necessary, explain why it is needed, how it is installed, and how to verify it.

Install `uv` using its official installation instructions. In a new project folder, use the same project workflow on macOS, Linux, and Windows:

```
uv init --python 3.12
uv add requests
uv run python main.py
uv tree
```

Windows Powershell

```
uv init --python 3.12
uv add requests
uv run python main.py
uv tree
```

`uv` creates and manages the project-local `.venv`, records dependencies in `pyproject.toml` and `uv.lock`, and runs commands in that environment. Students do not need to activate the environment manually. Use `uv add package` rather than global `pip install package`.

Python version and coding conventions

All examples target **Python 3.12+**. The course uses syntax and standard-library APIs available in Python 3.12; learners may use a newer stable interpreter without changing the lessons. Do not assume that a feature available in the newest Python release is available to every learner.

Use current, readable Python conventions:

- Follow PEP 8 for naming, four-space indentation, whitespace, imports, and readable line length. Use snake_case for variables and functions, PascalCase for classes, and UPPER_SNAKE_CASE for constants.
- Prefer descriptive names, f-strings, and explicit intermediate steps over dense one-liners.
- Use pathlib.Path for filesystem paths, with statements for files and other managed resources, and if **name** == "**main**" for runnable module entry points.
- Add type annotations to function parameters and return values as concepts become appropriate. Built-in generics such as list[str], dict[str, int], and tuple[float, float] are supported by the 3.12 baseline; avoid legacy typing.List/typing.Dict forms in new examples.
- Catch specific exceptions, keep try blocks narrow, and do not silently ignore errors. Use None for an intentional absence of a value and compare it with is None.
- Use `uv` to manage project dependencies: `uv add package` records a dependency in `pyproject.toml`; `uv run` executes in the managed project environment. Do not use global pip installs.
- Keep dependencies minimal. Optional quality tools such as Ruff and pytest can be introduced after learners understand the standard-library workflow.

The curriculum favors clarity and compatibility over novelty. It does not require Python 3.13–3.15-only syntax or APIs.

Accessibility and instructional design

Python uses indentation to define code blocks. Consistent indentation, whitespace navigation, and screen-reader announcement settings are especially important. Learners should practice reading a block's start and end, identifying which statements belong to a condition or loop, and moving between indentation levels.

The curriculum emphasizes keyboard-first editor and terminal workflows, explicit reading of punctuation and operators, text-based exercises, clear prompts and errors, debugging, descriptive names, and manageable line lengths.

Do not assume all learners use the same screen reader or prefer speech. Braille, magnification, speech, switch access, and other configurations may require different pacing and navigation strategies. Ask learners what works for them and adapt without changing the learning objective.

How to study each lesson

1. Read the overview and identify the new concepts.
2. Trace each example line by line; describe the role of indentation and punctuation.
3. Predict the output before running the program.
4. Run the example and compare actual output with the prediction.
5. Change one value or statement and observe the effect.
6. Complete the activities independently before consulting a worked solution.
7. When an error occurs, read the traceback, identify the exception type and line, inspect nearby code, and test one correction at a time.
8. Explain the solution in plain language and record unresolved questions.

Keep a portfolio. Save small experiments as well as completed projects; revision is evidence of learning.

Assessment and project work

A useful portfolio includes short programs, input/output examples, debugging notes, tests, and project milestones. Instructors can assess algorithmic reasoning, code correctness, use of appropriate data structures, testing, clarity, accessibility of interaction, and the learner's ability to explain and modify the program.

The capstone should have a clear problem statement, user needs, a manageable feature list, text-based interaction, a test plan, and a short explanation or demonstration. Success means the learner can explain the design and limitations, not merely that the code runs.

Teaching with AI

AI assistants can explain a Python traceback, suggest test cases, compare alternatives, or provide a second explanation. They can also hallucinate APIs, invent package names, produce insecure file handling, or generate dense code that is difficult to navigate with speech or braille.

Students should follow the instructor's activity-specific policy, disclose assistance where required, and explain and modify every submitted line. Do not enter student records, credentials, private code, or sensitive data into public AI services. AI must not replace independent work in assessments designed to measure individual skills. The teacher's guide contains a fuller protocol.

Recommended introductory books

- Matthes, Eric. *Python Crash Course: A Hands-On, Project-Based Introduction to Programming*, 3rd edition. No Starch Press, 2023. A practical introduction organized around core language concepts and substantial projects. Publisher page.
- Zelle, John M. *Python Programming: An Introduction to Computer Science*, 4th edition. Franklin, Beedle & Associates, 2024. A first-course computer science textbook that develops problem solving and programming through Python. Book information.
- Zinoviev, Dmitry, Paul Gries, Jennifer Campbell, and Jason Montojo. *Practical Programming: An Introduction to Computer Science Using Python*, 4th edition. Pragmatic Bookshelf, 2026. A current, no-prior-experience introduction to computer science and programming, with emphasis on design, algorithms, testing, and debugging. Publisher page.
- Downey, Allen B. *Think Python: How to Think Like a Computer Scientist*, 3rd edition. Green Tea Press, 2024. A free, concept-oriented introduction emphasizing computational thinking, functions, data structures, and problem solving. Author's book site.
- Sweigart, Al. *Automate the Boring Stuff with Python: Practical Programming for Total Beginners*, 3rd edition. No Starch Press, 2025. A task-oriented introduction that teaches programming through useful automation projects. Publisher page.

Suggested use: select one primary text rather than attempting all three at once. *Think Python* is useful for conceptual reinforcement; *Python Crash Course* offers a project-based sequence; *Automate the Boring Stuff* connects fundamentals to practical tasks. The curriculum's own examples and exercises remain authoritative for course requirements.

Further learning

Use the official Python documentation as the reference for language behavior and standard-library APIs. The curriculum also draws on *Python Crash Course* by Eric Matthes and *Python Programming: An Introduction to Computer Science* by John Zelle. See the unit materials and teacher's guide for additional references.

The objective is to develop a repeatable approach to understanding problems, writing clear programs, testing behavior, and learning new tools independently.

GUI, visualization, and workbench accessibility

See the shared GUI, visualization, and workbench accessibility reference for screen-reader support by toolkit, platform-specific activation settings, recommended alternatives, and interfaces that must not be required without an equivalent accessible route.

AI Policy

AI may be used only for personal conceptual understanding and reinforcement. It must not write, complete, autocomplete, repair, or generate course code or assessment solutions.

Capstone code

The verified Unit 11 code (logic module, NiceGUI pages, and tests) is in capstone/.

13.1 PYTHON Lessons

The curriculum is organized as one Markdown file per unit.

- setting up your accessible python development environme
- foundations of computing and getting started with pytho
- writing simple programs and numeric computations
- control flow decisions and loops
- data structures lists and dictionaries
- user input and while loops
- functions
- classes and object oriented programming
- files and exceptions
- testing and data analysis
- project based learning building text based applications
- capstone project accessible personal data manager

13.2 Unit 0: Setting Up Your Accessible Python Development Environment

Before you start: complete Unit P: Computing Foundations Primer. It explains the language-neutral vocabulary and concepts used in every language

track, using non-visual examples.

Overview

A detailed environment setup is not merely a technical prerequisite; it empowers students and reduces dependency on external assistance. For visually impaired students, relying on external help for every setup or environment issue can be disempowering and slow down the learning process. This unit provides extremely detailed, screen-reader-friendly instructions for environment setup.

Target Audience: Students with no prior programming experience, relying on screen readers for all computer interactions.

Lesson 0.1: Installing Python and uv

Reading References

- Matthes: Chapter 1, pages 3–14
- Matthes: Appendix A, pages 463–468

Learning Objectives

- Successfully download the Python 3 installer for my operating system.
- Run the Python installer and correctly select the "Add Python to PATH" option.
- Successfully download the VS Code installer for my operating system.
- Run the VS Code installer and correctly select the "Add to PATH" option.
- Open my computer's system terminal.
- Verify Python's installation by running `python3 --version` in the terminal.
- Verify VS Code's installation by running `code --version` in the terminal.

Concept Explanation

Installing Python 3 Python is not always installed on a computer by default. You may install **Python 3.12 or newer** from the official Python downloads page, or let `uv` install and manage the course interpreter after you install uv. All course examples remain compatible with Python 3.12.

Windows

1. Locate the downloaded `.exe` file (usually in your Downloads folder). Activate it.
2. **CRUCIAL STEP:** When the installer window appears, check the **"Add Python to PATH"** checkbox. Use Tab to navigate to this checkbox and press Spacebar to check it.

3. Navigate to ”Install Now” and activate it.
4. Follow any further prompts. Activate ”Close” when finished.

macOS

1. Locate the downloaded `.pkg` file. Activate it.
2. Activate ”Continue” through the steps.
3. Agree to the license terms. Choose the installation location (default is fine).
4. Enter your administrator password when prompted.
5. Activate ”Install” and then ”Close” when finished.

Linux Python 3 is often pre-installed. Verify in a terminal:

```
python3 --version
```

Windows PowerShell (Windows 11):

```
py -3 --version
```

If not installed (Debian/Ubuntu):

```
sudo apt update
sudo apt install python3 curl
python3 --version  # Confirm this is 3.12 or newer
```

Windows PowerShell (Windows 11):

```
# Windows uses winget; no apt package-index update is needed.
winget install --id Python.Python.3.12 -e
py -3 --version  # Confirm this is 3.12 or newer
```

Installing uv and a managed Python interpreter `uv` is the recommended Python project and environment manager. It installs and manages Python interpreters, creates a project-local `.venv`, records dependencies, and runs commands in the correct environment. Install uv using the official standalone installer; this does not require Python or pip to be installed first.

macOS and Linux:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
uv --version
uv python install 3.12
uv python list
```

Windows PowerShell (Windows 11):

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
uv --version
uv python install 3.12
uv python list
```

If the `uv` command is not found immediately after installation, open a new terminal as instructed by the installer. The managed interpreter must be Python 3.12 or newer for this course.

Installing Visual Studio Code or Theia IDE

1. Install Visual Studio Code or Theia IDE. Theia supports many VS Code extensions through Open VSX, but check that the required extensions are available and test them with the learner's screen reader.

Windows

- **CRUCIAL STEP:** On the ”Select Additional Tasks” screen, check **”Add to PATH”**.
- Activate ”Next” and then ”Install”.

macOS

- Drag the ”Visual Studio Code” application to your ”Applications” folder.

Linux (Debian/Ubuntu)

```
cd Downloads
sudo dpkg -i code_*.deb
```

Windows PowerShell (Windows 11):

```
Set-Location "$HOME\Downloads"
winget install --id Microsoft.VisualStudioCode -e
```

Verify VS Code

```
code --version
```

Windows PowerShell (Windows 11):

```
code --version
```

Editor alternative: Theia IDE Theia IDE is another option instead of Visual Studio Code. It supports the keyboard- and screen-reader-oriented accessibility workflows used in this course. Its interface is similar in many respects and it supports many VS Code extensions through the Open VSX registry, but not every extension or shortcut is identical. Install the Python and language-support extensions available for Theia, then test navigation, diagnostics, terminal access, and screen-reader announcements with the learner's access technology.

Worked Example

Run these commands in a terminal after installing Python and uv. On Windows, `py` replaces `python3` if that is how your installation was set up.

```
python3 --version
uv --version
python3 -c "print('Hello from Python')"
```

Expected Behavior or Output

The first two commands print version numbers, and the third prints one line. The numbers depend on what you installed; this was observed with Python 3.12.3 and uv 0.11.7 on Linux:

```
Python 3.12.3
uv 0.11.7
Hello from Python
```

If a command is not found, the installation or the PATH setting is incomplete; reopen the terminal and try again before changing anything else.

Common Mistakes

1. Installing Python but not opening a new terminal, so the old PATH is still in use and the command is reported as not found.
2. Using the system Python for everything instead of a project environment, which later causes version and package conflicts.
3. Copying an installer command from an unknown website. Use the official download pages and read what a command does before running it.

Activities

1. Run the three verification commands and record the exact output in your notes file.
2. Find which command name your system uses for Python (`python`, `python3`, or `py`) and write it down.
3. Explain in two sentences what `uv` is used for in this course.
4. Close and reopen the terminal and repeat the commands; note whether anything changed.

Accessibility Practice

Run every command from the keyboard and read the output with your screen reader's line-by-line review. Confirm that the version numbers are announced clearly, digit by digit. Prefer the terminal route to any graphical installer step that is hard to operate with your access technology, and ask a sighted or experienced helper only for steps you cannot perform yourself.

Skills Checklist

- ☐ I can successfully download the Python 3 installer for my operating system.
- ☐ I can run the Python installer and correctly select the "Add Python to PATH" option.
- ☐ I can successfully download the VS Code installer for my operating system.
- ☐ I can run the VS Code installer and correctly select the "Add to PATH" option.
- ☐ I can open my computer's system terminal.
- ☐ I can verify Python's installation by running `python3 --version` in the terminal.
- ☐ I can verify VS Code's installation by running `code --version` in the terminal.

Lesson 0.2: Configuring Your Screen Reader for VS Code or Theia IDE

Reading References

- Matthes: Appendix B, pages 469–476
- Matthes: Appendix C, pages 477–482

Learning Objectives

- Identify and toggle VS Code's "Screen Reader Optimized Mode".
- Open and close the "Accessible View".
- Navigate between VS Code panes using Tab and Shift+Tab.
- Navigate line by line and character by character within the editor.
- Locate and listen to information in the VS Code Status Bar.
- Configure my screen reader to announce indentation changes.

Concept Explanation

General Screen Reader Tips for VS Code 1. Use Screen Reader Mode

- Press `Ctrl + Shift + P` to open the Command Palette.
- Type "Toggle Screen Reader Optimized" and press Enter.

2. Accessible View

- Press `Shift + Alt + F2` (Windows/Linux) or `Shift + Option + F2` (macOS).
- Opens a flat, text-based representation of complex UI elements.
- Press `Esc` to close.

3. Keyboard-First Navigation

- `Tab` and `Shift+Tab`: Move focus between UI elements.
- Arrow Keys: Navigate within active panes.

4. Status Bar The status bar at the bottom provides current line/column number, file type, active Python environment, and error/warning counts.

Screen Reader Indentation Configuration Python relies heavily on indentation. Configure your screen reader to announce indentation changes.

NVDA (Windows)

1. Press `NVDA Key + N` → Preferences → Settings.
2. Find "Document Formatting".
3. Locate "Indentation" and set it to **"Say all"** or **"Beeps"**.
4. Activate "OK".

JAWS (Windows)

1. Press `JAWS Key + J` → Alt+O for Options → B for Basics.
2. Find "Text Processing" settings.
3. Configure indentation announcement.

VoiceOver (macOS) VoiceOver generally handles indentation well by default, announcing "4 spaces" or "tab" as you navigate.

Worked Example

Create a file named `indent_test.py` with exactly these lines, using four spaces for each indentation level:

```
score = 82
if score >= 60:
    print("Passed")
    if score >= 80:
        print("Strong pass")
print("Done")
```

Move through it line by line and have your screen reader announce the indentation of lines 3 to 5.

Expected Behavior or Output

When the file is read line by line, line 3 and line 5 are announced one indentation level in, line 4 two levels in, and the last line at the left margin. Running `python3 indent_test.py` prints:

```
Passed
Strong pass
Done
```

If the screen reader does not report indentation, adjust the settings described above until it does; Python uses indentation to decide which lines belong together.

Common Mistakes

1. Leaving the indentation announcement turned off and then misreading which lines belong to an `if` block.
2. Mixing tabs and spaces in one file, which Python rejects with an `IndentationError`.
3. Changing several screen reader settings at once, so it is impossible to tell which change helped.

Activities

1. Change one setting at a time and record how the reading of `indent_test.py` changes.
2. Add a fifth indented line inside the inner `if` and predict the output before running.
3. Read the file once by character and once by word; note which is faster for checking indentation.
4. Write the keyboard commands you use to jump to the next line with the same indentation as the current line, if your editor has one.

Accessibility Practice

This lesson is the accessibility foundation for the course. Record, in your own notes, the operating system, editor, screen reader, and version you tested with. Do not describe a setup as verified for other users; it is verified only for the combination you actually ran.

Skills Checklist

☐ I can identify and toggle VS Code's "Screen Reader Optimized Mode".
☐ I can open and close the "Accessible View".
☐ I can navigate between VS Code panes using Tab and Shift+Tab.
☐ I can navigate line by line and character by character within the editor.
☐ I can locate and listen to information in the VS Code Status Bar.
☐ I can configure my screen reader to announce indentation changes.

Lesson 0.3: Setting Up Your VS Code or Theia IDE Workspace

Reading References

- Matthes: Appendix B, pages 469–476

- Matthes: Chapter 8, pages 149–152

Learning Objectives

- Open the Extensions view in my selected editor.
- Verify that Python language support and its language server are installed and enabled.
- Create a new folder for Python projects.
- Open the project folder as the editor workspace.
- Navigate the editor, file explorer, terminal, and diagnostics using the keyboard.
- Use the Command Palette or equivalent command interface.

Concept Explanation

Install and verify Python language extensions Choose the extension instructions for the editor you are using. The Python interpreter and project environment are separate from the editor extensions; installing an extension does not install Python or project dependencies.

Visual Studio Code

1. Open the Extensions view with `Ctrl + Shift + X` (Windows/Linux) or `Command + Shift + X` (macOS).
2. Install **Python** by Microsoft. This provides Python environment discovery, run/debug integration, and interpreter selection.
3. Install **Pylance** for language intelligence, completion, navigation, and type diagnostics.
4. Confirm both extensions are enabled for this workspace. Do not use Code Runner for course projects: it can run a file outside the project's managed environment and bypass the standard `uv run` workflow.

Theia IDE

1. Open View → Extensions (or the Extensions view available in your Theia distribution).
2. Search the configured Open VSX registry for a Python language extension and a compatible Python language server.
3. Check the extension publisher, supported Theia version, and extension documentation. Do not assume that Microsoft's Python or Pylance extensions are available or interchangeable in Theia.
4. Confirm that completion, go-to-definition, diagnostics, interpreter discovery, terminal access, and debugging work with the selected Python installation. If a required capability is unavailable, use a verified supported extension or choose VS Code for this course.

In either editor, install only extensions needed for the course, keep them enabled for the project workspace, and test their announcements and keyboard navigation with the learner's screen reader before relying on them.

Creating Your First Workspace Folder **Windows:** Open File Explorer (`Windows Key + E`). Navigate to Documents. Press `Ctrl + Shift + N` to create a new folder named `PythonProjects`.

macOS: Open Finder. Navigate to Documents. Press `Command + Shift + N` to create `PythonProjects`.

Opening the Folder in VS Code **Method 1 (Recommended):** Open terminal, navigate to the folder, then:

```
code .
```

Windows PowerShell (Windows 11):

```
code .
```

Method 2 (VS Code): Press `Ctrl + K, Ctrl + O` to open a folder.

Theia IDE: Use the File → Open Folder command (or the equivalent workspace-opening command in your Theia distribution) and select the project folder. Confirm that the folder containing `pyproject.toml` is the workspace root; opening only `hello.py` can prevent the editor from discovering the project's `.venv` and configuration.

VS Code Navigation Shortcuts

Area	Shortcut
Explorer	`Ctrl + Shift + E`
Search	`Ctrl + Shift + F`
Extensions	`Ctrl + Shift + X`
Toggle Sidebar	`Ctrl + B`
Command Palette	`Ctrl + Shift + P`
Problems View	`Ctrl + Shift + M`
Next Error	`F8`
Previous Error	`Shift + F8`
Integrated Terminal	‘Ctrl + “

Advanced editor options Experienced coders may prefer JetBrains IDEs such as PyCharm. These are advanced alternatives, not the recommended starting point for this beginner curriculum. This course does not provide JetBrains installation or usage instructions.

Worked Example

Create a project folder and a first file, then run it from the terminal as well as from the editor:

```
mkdir python-course
cd python-course
code .
```

In the editor, create `hello.py` containing `print("Hello, Avery!")`, save it, and then run it from the terminal with `python3 hello.py`.

Expected Behavior or Output

The terminal prints one line:

```
Hello, Avery!
```

The editor's Run command should produce the same line in its integrated terminal. If the two differ, the editor is using a different Python interpreter from the terminal; check which interpreter is selected.

Common Mistakes

1. Opening a single file instead of the project folder, so the editor does not know the project's interpreter or files.
2. Selecting a different Python interpreter in the editor from the one used in the terminal, so the same program behaves differently.
3. Depending only on the editor's Run button, which hides the exact command being run.

Activities

1. Create the folder and file from the keyboard only and write down each keyboard command you used.
2. Open the folder in the editor and list the files using the keyboard.
3. Run `hello.py` from the editor and from the terminal, and compare the output.
4. Change the greeting to use your own name and rerun it.

Accessibility Practice

Use the keyboard to open the folder, move between the file list, the editor, and the terminal, and return. Note which shortcuts your screen reader announces and which you must learn by heart. Always keep one terminal route available, since it does not depend on any editor feature.

Skills Checklist

- ☐ I can open the Extensions view in my selected editor.
- ☐ I can verify that Python language support and its language server are installed and enabled.
- ☐ I can create a new folder for Python projects.
- ☐ I can open the project folder as the editor workspace.

- ☐ I can navigate the editor, file explorer, terminal, and diagnostics using the keyboard.
- ☐ I can use the Command Palette or equivalent command interface.

Lesson 0.4: Python Environments, Terminal Usage, and Accessibility Tools

Reading References

- Matthes: Chapter 1, pages 3–14
- Matthes: Chapter 8, pages 149–152
- Matthes: Chapter 11, pages 210–211

Learning Objectives

- Create a new Python file (.py) in VS Code.
- Write a simple `print()` statement.
- Run a Python script from the system terminal.
- Run a Python script from the integrated terminal in VS Code.
- Understand why virtual environments are useful.
- Initialize a Python project with `uv init`.
- Add a project dependency with `uv add` and inspect it with `uv tree`.
- Run a program with `uv run` and select the project's `.venv` interpreter in my editor.

Concept Explanation

Creating Your First Python File

1. Ensure Explorer is focused: `Ctrl + Shift + E`.
2. Create new file: `Ctrl + N`.
3. Save: `Ctrl + S`. Name it `hello.py`.
4. Type:

```
print("Hello from VS Code!")
```

Running Python Scripts From System Terminal:

```
cd ~/Documents/PythonProjects
python3 hello.py
```

Windows PowerShell (Windows 11):

```
Set-Location "$HOME\Documents\PythonProjects"
py -3 hello.py
```

From VS Code Integrated Terminal:

1. Open terminal: ‘Ctrl + “
2. The terminal opens in your project folder automatically.
3. Run: `python3 hello.py`

Project environments and dependencies with uv Use one isolated environment per Python project. The recommended course workflow uses `uv`; it creates and manages the project's `.venv` automatically, so learners do not need to activate it manually. Run these commands from the project folder. If the folder does not yet contain a project, initialize it first.

Create the project environment, add a dependency, and run the program:

```
uv init --python 3.12
uv add requests
uv run python hello.py
uv tree
```

Windows PowerShell (Windows 11):

```
uv init --python 3.12
uv add requests
uv run python hello.py
uv tree
```

- `uv init` creates the project configuration.
- `uv add requests` installs the package into the project environment and records it in `pyproject.toml` and `uv.lock`.
- `uv run python hello.py` runs the program with the project's managed interpreter and dependencies.
- `uv tree` lists the dependency tree.

Run without installing a project dependency: for a one-off tool or script dependency, use `uv run --with package command` or `uvx tool-name`. Do not use bare/global `pip install` or `python -m pip install` for course packages.

Select and verify the project interpreter in VS Code or Theia IDE: open the project folder (the folder containing `pyproject.toml`) as the workspace, then choose the interpreter inside that project's `.venv` directory: `.venv/bin/python` on macOS/Linux or `.venv\\Scripts\\python.exe` on Windows. In VS Code, use **Python: Select Interpreter** and select the workspace environment. In Theia, use the Python extension's interpreter-selection command or setting. If the environment is not listed, create it with `uv init --python 3.12`, reopen the project folder, and select it again.

Build the habit of checking the active environment before running or debugging code. The editor's selected interpreter, its integrated terminal, and the command-line workflow must all point to the same project. Run course programs with `uv run python filename.py`; add packages with `uv add package-name`.

Do not install course dependencies globally. If an editor's Run or Debug action does not use the project interpreter, correct its interpreter/launch configuration or use the terminal command rather than accepting a run from the wrong environment.

Optional venv concepts: `uv` uses the standard `.venv` directory. Students may inspect it and learn that it isolates project dependencies, but routine manual activation is not required when commands are run with `uv run`.

VS Code and Theia IDE Accessibility Features The following shortcuts are VS Code defaults. Theia provides comparable keyboard and screen-reader workflows, but verify command names and keybindings in the installed version.

- **Accessible View** (`Shift+Alt+F2`): Reveals hover information and error details.
- **Focus on Errors** (`F8` / `Shift+F8`): Jump between errors without visual scanning.
- **Go to Definition** (`F12`): Navigate to where a function or variable is defined.
- **Go to Reference** (`Shift+F12`): Find all usages of a symbol.
- **Go to Line/Column** (`Ctrl+G`): Jump to a specific line number.

Python Documentation The official Python documentation is well-structured and accessible: https://docs.python.org/3/

Unit 0 Completion Checklist

- ☐ uv is installed and `uv run python --version` reports Python 3.12 or newer.
- ☐ VS Code or Theia IDE is installed and I can open the course project.
- ☐ Your screen reader is configured to announce indentation changes in your chosen editor.
- ☐ You can open your PythonProjects folder in VS Code or Theia IDE.
- ☐ You have installed Python language support in your chosen editor (the Python extension in VS Code or its compatible Theia alternative).
- ☐ You can create a new .py file, write `print("Hello!")`, and save it.
- ☐ You can run your hello.py file from both terminals.
- ☐ You can create a project environment with `uv init` and add dependencies with `uv add`.
- ☐ You can run project code with `uv run` and select the project's `.venv` interpreter in your editor.
- ☐ You know how to use `Ctrl + Shift + P` (Command Palette) and `F8` (Go to next error).

Congratulations! You now have a powerful and accessible development environment ready for your Python journey.

Worked Example

Create a project environment with uv, add nothing, and run a script inside it:

```
mkdir env-demo
cd env-demo
uv init
uv run python -c "print('Running inside the project environment')"
```

Expected Behavior or Output

`uv init` creates project files and the last command prints:

```
Running inside the project environment
```

Use `uv run` (or activate the environment) every time you run the project, so the same Python and the same packages are used.

Common Mistakes

1. Installing packages into the system Python instead of the project environment.
2. Running a script from the wrong working directory, so relative file names are not found.
3. Forgetting which environment is active and getting different results for the same code.

Activities

1. Run `pwd` (macOS and Linux) or `cd` (Windows) and write down the working directory.
2. List the files that `uv init` created and say what each one is for.
3. Create a second project folder and confirm that its environment is separate.
4. Write the three terminal commands you need to start work on a project each day.

Accessibility Practice

Terminal output is plain text and works well with a screen reader or braille display. Learn the keys that jump to the start and end of the last command's output, and confirm the working directory before every run. Avoid long one-line commands you cannot navigate; break them with line continuation or run them in steps.

Skills Checklist

- ☐ I can create a new Python file (.py) in VS Code.
- ☐ I can write a simple `print()` statement.

- ☐ I can run a Python script from the system terminal.
- ☐ I can run a Python script from the integrated terminal in VS Code.
- ☐ I understand why virtual environments are useful.
- ☐ I can initialize a Python project with `uv init`.
- ☐ I can add a project dependency with `uv add` and inspect it with `uv tree`.
- ☐ I can run a program with `uv run` and select the project's `.venv` interpreter in my editor.

13.3 Unit 1: Foundations of Computing and Getting Started with Python

Overview

Compatibility: All code examples in this curriculum target Python 3.12 and newer. The course avoids syntax and APIs introduced after Python 3.12.

This unit covers basic computer science concepts, setting up the Python environment, and writing and executing simple Python programs.

Objectives:

- Understand what computer science is and how hardware relates to software.
- Write and run your first Python program.
- Use variables to store and manipulate simple data types.

Lesson 1.1: Welcome to Computer Science

Reading References

- Zelle: Chapter 1, pages 1–7
- Matthes: Chapter 1, pages 3–14

Learning Objectives

- Define what computer science is.
- Distinguish between computer hardware and software.
- Explain what a computer program is.
- Identify Python as a programming language.

Concept Explanation

What Is Computer Science? A computer is a **universal machine** — it can be programmed to solve any problem that can be broken down into a series

of precise, logical steps. The true power of a computer doesn't come from its hardware but from the software we create.

Computer science is the study of problem-solving, not just programming. It's an interdisciplinary field involving breaking down complex problems, designing algorithms, and implementing efficient solutions.

Hardware vs. Software

Component	Description
Hardware	Physical components you can touch: keyboard, screen, processor, memory
Software	Instructions that direct hardware: programs, operating systems, apps
CPU	The processor that executes instructions
Memory (RAM)	Temporary storage for active programs and data
Storage	Permanent storage for files

Analogy: Think of a kitchen. The stove, refrigerator, and pots are **hardware**. A cookie recipe is **software** — it provides precise steps for the "hardware" (the cook) to follow.

Why Python? Python is an ideal language for beginners because of its:

- **Readability** — English-like syntax makes code easy to understand.
- **Simplicity** — Less "boilerplate" code than other languages.
- **Versatility** — Used in web development, data science, AI, and more.

The Problem-Solving Approach

1. Understand the problem.
2. Break it down into smaller pieces.
3. Design a step-by-step solution (an algorithm).
4. Implement the solution in Python.
5. Test and refine your code.

Worked Example

The problem-solving approach applied to a small task: find the total and average of the shared scores 82, 91, 76, and 88.

Algorithm in plain steps: set the total to 0; for each score, add it to the total; divide the total by the number of scores; report both.

```
scores = [82, 91, 76, 88]
total = 0
for score in scores:
    total = total + score
average = total / len(scores)
print("Total:", total)
print("Average:", average)
```

Expected Behavior or Output

Output of example 1:

```
Total: 337
Average: 84.25
```

Common Mistakes

1. Writing code before writing the steps, then forgetting to divide by the number of scores.
2. Confusing the program with the algorithm. The algorithm is the plan; the program is one way of carrying it out.
3. Expecting a computer to fill in a step you did not state.

Activities

Activity 1.1.1: "Human Computer" Role-Play

- In pairs, one student is the "computer" and the other is the "programmer."
- The programmer provides simple, precise instructions for a common task (e.g., "make a peanut butter and jelly sandwich").
- The "computer" follows only what is literally said.
- Purpose: Illustrate the need for precise, unambiguous instructions.

Activity 1.1.2: Identify Hardware and Software

- List five items you use daily.
- For each, identify whether it is hardware or software and explain your choice.

Accessibility Practice

Write the plain-language steps as a numbered list and read it back with your screen reader before coding. Trace the loop by hand as numbered sentences (82, then 173, then 249, then 337) rather than as a table, so every state is read in order.

Skills Checklist

☐ I can define what computer science is.

☐ I can distinguish between computer hardware and software.
☐ I can explain what a computer program is.
☐ I can identify Python as a programming language.

Lesson 1.2: Your First Python Program — "Hello World!"

Reading References

- Zelle: Chapter 1, pages 7–13
- Matthes: Chapter 1, pages 3–14

Learning Objectives

- Write a `print()` statement in Python.
- Save a Python file with the `.py` extension.
- Execute a Python script from the terminal.
- Understand that the output of `print()` appears in the terminal.

Concept Explanation

The `print()` Function The `print()` function is Python's primary method for displaying output to the user.

```
print("Hello, World!")
```

Expected output:

```
Hello, World!
```

Key Concepts

- **Strings** are text data enclosed in quotation marks (single or double).
- **Comments** start with `#` and are ignored by Python.
- Python executes code **line by line**, from top to bottom.

Worked Example

Example 1.2.1: Simple Greeting

```
# This program prints a classic greeting.
print("Hello, World!")
```

Example 1.2.2: Personalized Message

```
# This program prints a message with your name.
print("Hello, Avery!")
print("Welcome to Python programming!")
```

Expected Behavior or Output

Expected output:

```
Hello, Avery!
Welcome to Python programming!
```

Common Mistakes

```
print("Hello, World"        # Missing closing parenthesis - SyntaxError
print "Hello"               # Python 2 style - SyntaxError in Python 3
Print("Hello")              # Capital P - NameError
```

Activities

Activity 1.2.1: My First Message

1. Open VS Code and confirm the PythonProjects folder is open.
2. Create a new file as `my_message.py`.
3. Type:
   ```
   print("My favorite color is blue.")
   print("My favorite animal is a dog.")
   ```
4. Save and run the file.

Activity 1.2.2: Exploring Print Statements

1. Experiment with:
   - Printing numbers: `print(123)`
   - Printing sentences with punctuation.
   - Omitting quotes: `print(Hello)` — expect a `NameError`.

Accessibility Practice

Run the file from the terminal and read the output line by line. Check that your screen reader announces the quotation marks and parentheses in `print("Hello, Avery!")`, because a missing closing quotation mark is the most common first error. If the error message is long, jump to its last line, which names the problem.

Skills Checklist

☐ I can write a `print()` statement in Python.
☐ I can save a Python file with the `.py` extension.
☐ I can execute a Python script from the terminal.
☐ I understand that the output of `print()` appears in the terminal.

Lesson 1.3: Variables and Simple Data Types

Reading References

- Zelle: Chapter 2, pages 24–26
- Matthes: Chapter 2, pages 15–32

Learning Objectives

- Define what a variable is in programming.
- Create a valid variable name in Python.
- Assign string, integer, and float values to variables.
- Use `print()` to display the value of a variable.
- Perform basic arithmetic operations with numbers in Python.

Concept Explanation

What Is a Variable? A **variable** is a name that refers to a value stored in the computer's memory. The **assignment operator** `=` creates a connection between a name and a value.

```
name = "Avery"
age = 16
height = 1.72
favorite_subject = "Computer Science"
```

Rules for variable names:

- Must start with a letter or underscore
- Can only contain letters, digits, and underscores
- Are case-sensitive (`age` is different from `Age`)
- Cannot be a Python keyword (`if`, `for`, `while`, etc.)

Best practices:

- Use descriptive names: `student_count`, not `sc`
- Use lowercase with underscores: `first_name`

Essential Data Types

Type	Description	Examples
`str` (String)	Text data	`"hello"`, `"Python"`
`int` (Integer)	Whole numbers	`10`, `-5`, `0`
`float` (Float)	Numbers with decimals	`3.14`, `9.99`
`bool` (Boolean)	True or False	`True`, `False`

Key Table: Python Data Types and Operations

Data Type	Description	Example Value	Common Operations
`str`	Text, words, sentences	`"Hello"`, `"Python"`	`+` (concatenation), `len()`, `.upper()`
`int`	Whole numbers	`10`, `-5`, `0`	`+`, `-`, `*`, `/`, `%`
`float`	Numbers with decimals	`3.14`, `9.99`, `0.0`	`+`, `-`, `*`, `/`

Worked Example

Example 1.3.1: Storing and Printing a String

```
student_name = "Avery"
print("Hello, " + student_name + "!")
```

Example 1.3.2: The Shared Student Profile and Notebook Cost The same student profile and inventory item are used in every language track, so you can compare how each language declares the same data.

```
name = "Avery"
age = 16
height = 1.72
favorite_subject = "Computer Science"

print("Name:", name)
print("Age:", age)
print("Height in meters:", height)
print("Favorite subject:", favorite_subject)
print(type(name), type(age), type(height), type(favorite_subject))

notebook_price = 2.50
notebook_quantity = 3
notebook_total = notebook_price * notebook_quantity
print("Notebook total:", notebook_total)
```

Python decides each variable's type from the value you assign (dynamic typing). `type()` reports it: `str`, `int`, `float`, `str`.

Expected Behavior or Output

Output:

```
Hello, Avery!
```

Expected output:

```
Name: Avery
Age: 16
Height in meters: 1.72
```

```
Favorite subject: Computer Science
<class 'str'> <class 'int'> <class 'float'> <class 'str'>
Notebook total: 7.5
```

Common Mistakes

```
2nd_number = 10          # Invalid variable name - starts with digit
print("Result: " + 42)   # TypeError - can't concatenate str and int
```

Activities

Activity 1.3.1: Personal Information Variables

```
my_name = "Avery"
favorite_number = 7
height_in_meters = 1.72
favorite_food = "pizza"

print("My name is:", my_name)
print("My favorite number is:", favorite_number)
print("My height is:", height_in_meters, "meters")
print("My favorite food is:", favorite_food)
```

Activity 1.3.2: Simple Math Challenge

```
num1 = 15
num2 = 7
sum_result = num1 + num2
difference = num1 - num2
product = num1 * num2
quotient = num1 / num2

print("Sum:", sum_result)
print("Difference:", difference)
print("Product:", product)
print("Quotient:", quotient)
```

Accessibility Practice

Say variable names aloud with their underscores (`favorite_subject`) and listen for the pause so you can tell one name from two words. Print each value on its own labelled line so the screen reader announces the name and the value together, and avoid relying on the spacing between printed items.

Skills Checklist

- ☐ I can define what a variable is in programming.
- ☐ I can create a valid variable name in Python.

☐ I can assign string, integer, and float values to variables.
☐ I can use `print()` to display the value of a variable.
☐ I can perform basic arithmetic operations with numbers in Python.

Canonical Shared Dataset — Student Profile

The parallel-language courses use the same student record. Keep these values unchanged when translating the example into another language.

```
student_profile = {
    "name": "Avery",
    "age": 16,
    "height_m": 1.72,
    "favorite_subject": "Computer Science",
}
print(student_profile["name"])
print(student_profile["age"])
print(student_profile["height_m"])
print(student_profile["favorite_subject"])
```

Expected output:

```
Avery
16
1.72
Computer Science
```

Use this record in place of unrelated personal-profile values in parallel-track comparison exercises. The earlier Hello World example remains a minimal first-program demonstration.

13.4 Unit 2: Writing Simple Programs and Numeric Computations

Overview

Compatibility: All code examples in this curriculum target Python 3.12 and newer. The course avoids syntax and APIs introduced after Python 3.12.

This unit explores the software development process, more complex numeric operations, and how to obtain and use input from users.

Objectives:

- Understand the phases of software development.
- Work with different numeric data types (integers, floats).
- Perform type conversions.
- Use mathematical operators and the math library.

- Get user input with `input()`.

Lesson 2.1: The Software Development Process

Reading References

- Zelle: Chapter 2, pages 21–22 and pages 35–36

Learning Objectives

- Describe the purpose of the software development process.
- List and briefly explain the main phases of software development.
- Understand that problem-solving and design happen before writing code.

Concept Explanation

The Three-Phase Development Process **1. Design (Plan first)** Before writing any code, fully understand the problem:

- Identify **inputs** (information the program needs) and **outputs** (results it must produce).
- Write an **algorithm** — a step-by-step plan in plain English (**pseudocode**).
- Consider **edge cases** — unusual inputs that might break your program.

2. Implement (Write the code)

- Translate your design into Python syntax.
- Code **incrementally** — build in small pieces and test as you go.
- Follow coding standards and use descriptive variable names.

3. Test and Debug (Verify it works)

- Run with a variety of inputs, including normal values and edge cases.
- Read error messages carefully — they tell you the type of error and the line number.
- Use `print()` statements to check variable values during debugging.

Why This Process Matters

- **Reduces frustration** — Having a plan prevents getting stuck.
- **Improves code quality** — A thoughtful design leads to cleaner code.
- **Simplifies debugging** — Test each component in isolation.
- **Saves time** — Good design prevents time-consuming rewrites.

Worked Example

Implement the pseudocode from the activity below with the shared values (bill 24.50, tip rate 0.15):

```
bill = 24.5
tip_rate = 0.15

tip = bill * tip_rate
total = bill + tip

print(f"Bill: {bill:.2f}")
print(f"Tip: {tip:.2f}")
print(f"Total: {total:.2f}")
```

Expected Behavior or Output

Output of example 1:

```
Bill: 24.50
Tip: 3.67
Total: 28.18
```

Common Mistakes

1. Writing the rate as 15 instead of 0.15, which makes the tip fifteen times too large.
2. Rounding the tip before adding it to the bill, which can change the total by a cent.
3. Skipping tests for a zero bill or a negative bill.

Activities

Activity 2.1.1: Planning a Tip Calculator Write pseudocode before coding:

```
1. GET bill amount from user
2. CALCULATE tip as 15% of bill amount
3. CALCULATE total amount (bill + tip)
4. PRINT tip amount
5. PRINT total amount
```

Trace this plan with the shared calculator values: the bill is 24.50 and the tip rate is 0.15. The tip is $24.50 \times 0.15 = 3.675$ and the total is $24.50 + 3.675 = 28.175$. Shown with two decimal places, Python displays the tip as 3.67 and the total as 28.18, because 3.675 cannot be stored exactly in binary floating point. You will implement this plan in Lesson 2.4.

Activity 2.1.2: Tracing a Simple Program Without running the code, trace the execution and predict output:

```
a = 10
b = 5
c = a * b
a = a + 2
```

```
print(c)
print(a)
```

Accessibility Practice

Write the design steps as a numbered list and print every result on its own labelled line (`Tip: 3.67`), so a screen reader reads each result as one clear unit. Do not use alignment or spacing to convey meaning.

Skills Checklist

- ☐ I can describe the purpose of the software development process.
- ☐ I can list and briefly explain the main phases of software development.
- ☐ I understand that problem-solving and design happen before writing code.

Lesson 2.2: Elements of Programs — Expressions and Assignment

Reading References

- Zelle: Chapter 2, pages 24–26

Learning Objectives

- Identify an expression in Python code.
- Predict the result of simple Python expressions.
- Explain how the assignment operator `=` works.
- Demonstrate how to update a variable's value.

Concept Explanation

Expressions and Assignment An **expression** is any combination of values, variables, and operators that evaluates to a single result. Python follows the standard mathematical order of operations (PEMDAS).

```
result1 = 10 / 2 + 3  # 10/2 is 5, then 5+3 = 8.0
message = "Python" + " " + "is fun!"
```

Assignment:

```
score = 82
score = score + 9  # score is now 91
```

Worked Example

Example 2.2.1: Expression Evaluation

```
result1 = 10 / 2 + 3
print("Result 1:", result1)   # Output: Result 1: 8.0
print("Precedence:", 8 + 4 * 2)   # Output: Precedence: 16
print("Grouped:", (8 + 4) * 2)    # Output: Grouped: 24

message = "Python" + " " + "is fun!"
print("Message:", message)    # Output: Message: Python is fun!
```

Example 2.2.2: Reassigning Variables

```
score = 82
print("Initial score:", score)
score = score + 9
print("New score:", score)     # Output: New score: 91
```

Expected Behavior or Output

Output of example 1:

```
Result 1: 8.0
Precedence: 16
Grouped: 24
Message: Python is fun!
```

Output of example 2:

```
Initial score: 82
New score: 91
```

Common Mistakes

1. Reading `=` as "equals". In an assignment it means "store the value on the right in the name on the left."
2. Forgetting that `/` always gives a float, so `10 / 2` is `5.0`, not `5`.
3. Expecting `8 + 4 * 2` to be 24. Multiplication happens first, so it is 16; use parentheses to change the order.

Activities

1. Predict the value of `8 + 4 * 2`, `(8 + 4) * 2`, `7 // 2`, `7 % 2`, and `2 ** 3`, then check each in Python.
2. Start with `score = 82` and write two statements that raise it by 9 and then double it. Predict the result.
3. Join the strings `"Avery"` and `"Student"` with a space between them using `+`.
4. Explain, in two sentences, why `score = score + 9` is valid even though it looks like an impossible equation.

Accessibility Practice

Say operators by name when reading code aloud: 'double star' for `**`, 'double slash' for `//`, 'percent' for `%`. Raise your screen reader's punctuation level while reading expressions so that parentheses and operators are all announced.

Skills Checklist

- ☐ I can identify an expression in Python code.
- ☐ I can predict the result of simple Python expressions.
- ☐ I can explain how the assignment operator `=` works.
- ☐ I can demonstrate how to update a variable's value.

Lesson 2.3: Input and Output Revisited

Reading References

- Zelle: Chapter 2, pages 27–33
- Matthes: Chapter 7, pages 114–116

Learning Objectives

- Use the `input()` function to ask the user for information.
- Provide a clear prompt message when using `input()`.
- Remember that `input()` returns a string, even for numbers.
- Combine text and variable values in a `print()` statement using f-strings.

Concept Explanation

The `input()` Function The `input()` function pauses program execution and waits for the user to type something and press Enter. **It always returns a string.**

```
user_name = input("What is your name? ")
print(f"Hello, {user_name}! Nice to meet you.")
```

F-Strings (Formatted String Literals) F-strings provide a clean way to embed variable values directly into a string:

```
name = "Avery"
age = 16
print(f"Hello, {name}! You are {age} years old.")
```

Converting Input for Calculations Since `input()` always returns a string, you must convert it for math:

```
age_str = input("How old are you? ")
age_int = int(age_str)
print(f"Next year you will be {age_int + 1} years old.")
```

Worked Example

Example 2.3.1: Basic Input

```
user_name = input("What is your name? ")
print(f"Hello, {user_name}! Nice to meet you.")
```

Example 2.3.2: Numeric Input and Calculation

```
user_name = input("Enter your name: ")

num1_str = input("Enter the first number: ")
num1 = int(num1_str)

num2_str = input("Enter the second number: ")
num2 = int(num2_str)

sum_result = num1 + num2
print(f"Hello, {user_name}!")
print(f"The sum of {num1} and {num2} is {sum_result}.")
```

If the user types text that is not a whole number, `int()` raises a `ValueError`. Lesson 8.3 shows how to handle that.

Expected Behavior or Output

Sample run (typed input shown after each prompt):

```
What is your name? Avery
Hello, Avery! Nice to meet you.
```

Sample run (typed input shown after each prompt):

```
Enter your name: Avery
Enter the first number: 12
Enter the second number: 8
Hello, Avery!
The sum of 12 and 8 is 20.
```

Common Mistakes

1. Forgetting that `input()` always returns text. `"12" + "8"` joins the text to make `"128"`, and does not add.
2. Calling `int()` on text that is not a whole number, which raises a `ValueError`.
3. Writing a prompt that does not say what to type, such as `input("> ")`.

Activities

Activity 2.3.1: Temperature Converter

```
fahrenheit_str = input("Enter temperature in Fahrenheit: ")
fahrenheit = float(fahrenheit_str)
celsius = (fahrenheit - 32) * 5 / 9
print(f"{fahrenheit}°F is {celsius:.2f}°C")
```

Accessibility Practice

Write each prompt as a complete sentence that says what to enter, such as `Enter the first number:` , and end it with a space so the answer starts after it. Test the program with your screen reader to confirm the prompt is read before you start typing.

Skills Checklist

- ☐ I can use the `input()` function to ask the user for information.
- ☐ I can provide a clear prompt message when using `input()`.
- ☐ I remember that `input()` returns a string, even for numbers.
- ☐ I can combine text and variable values in a `print()` statement using f-strings.

Lesson 2.4: Computing with Numbers and Type Conversions

Reading References

- Zelle: Chapter 3, pages 41–48
- Matthes: Chapter 2, pages 26–28

Learning Objectives

- Identify when to use an integer versus a float.
- Convert a string to an integer using `int()`.
- Convert a string to a float using `float()`.
- Convert a number to a string using `str()`.
- Apply type conversions to perform calculations on user input.
- Understand and can apply the order of operations.

Concept Explanation

Integers and Floats

Type	Description	Example
`int`	Whole numbers (exact)	`5`, `-20`, `1000000`
`float`	Numbers with decimals	`3.14`, `-2.5`, `6.0`

Key behaviors:

- Division `/` always returns a float: `10 / 2` → `5.0`
- Floor division `//` returns the integer part: `10 // 3` → `3`
- Mixed operations promote to float: `10 + 3.5` → `13.5`

Type Conversion Functions

```
int("123")       # → 123  (truncates: int(3.9) → 3)
float("3.14")    # → 3.14
str(42)          # → "42"
```

Worked Example

Example 2.4.1: Type Conversion

```
num_str = "123"
num_int = int(num_str)
print(f"String '{num_str}' as integer: {num_int}, type: {type(num_int)}")

num_float = float(num_str)
print(f"String '{num_str}' as float: {num_float}, type: {type(num_float)}")

my_number = 42
my_text = str(my_number)
print(f"Integer {my_number} as string: '{my_text}', type: {type(my_text)}")
```

Common Type Error:

```
# This will cause a TypeError:
# print("My age is " + 14)
# Fix: print("My age is " + str(14))
# Or: print(f"My age is {14}")
```

Example 2.4.2: Tip Calculator (shared calculator values)

```
bill = 24.5
tip_rate = 0.15

tip = bill * tip_rate
total = bill + tip

print(f"Bill: {bill:.2f}")
```

```
print(f"Tip: {tip:.2f}")
print(f"Total: {total:.2f}")
```

The unrounded values are 3.675 and 28.175. Python's `:.2f` formatting displays 3.67 and 28.18. This is binary floating-point behavior, not a mistake in the program; other languages may round the tip to 3.68.

Expected Behavior or Output

Expected output:

```
Bill: 24.50
Tip: 3.67
Total: 28.18
```

Common Mistakes

1. Converting with `int()` and expecting rounding. `int(3.9)` is `3` because it removes the fraction.
2. Expecting `0.1 + 0.2` to equal exactly `0.3`. Floating-point numbers are approximations.
3. Forgetting to convert typed text before arithmetic, then seeing `TypeError` or joined text instead of a sum.

Activities

Activity 2.4.1: Mixed Bag Calculator

```
age_str = input("Enter your age: ")
age = int(age_str)

height_str = input("Enter your height in meters: ")
height = float(height_str)

phrase = input("Enter a short phrase: ")

print(f"Age doubled: {age * 2}")
print(f"Height in centimeters: {height * 100:.1f}")
print(f"Phrase length: {len(phrase)} characters")
```

Sample run (typed input shown after each prompt):

```
Enter your age: 16
Enter your height in meters: 1.72
Enter a short phrase: Computer Science
Age doubled: 32
Height in centimeters: 172.0
Phrase length: 16 characters
```

Accessibility Practice

When printing floating-point results, format them (`:.2f`) so the screen reader reads a short number instead of a long unrounded one. Read the type names printed by `type()` aloud, and note that `<class 'int'>` is announced with its angle brackets.

Skills Checklist

- ☐ I can identify when to use an integer versus a float.
- ☐ I can convert a string to an integer using `int()`.
- ☐ I can convert a string to a float using `float()`.
- ☐ I can convert a number to a string using `str()`.
- ☐ I can apply type conversions to perform calculations on user input.
- ☐ I understand and can apply the order of operations.

Lesson 2.5: Using the Math Library

Reading References

- Zelle: Chapter 3, pages 49–51

Learning Objectives

- Explain what a Python module is.
- Import the `math` module into my program.
- Use `math.pi` and `math.sqrt()`.
- Use `math.ceil()` to round up and `math.floor()` to round down.

Concept Explanation

Importing the Math Module

```
import math
```

Key Math Functions and Constants

Function/Constant	Description	Example
`math.pi`	3.14159	`math.pi`
`math.e`	e 2.71828	`math.e`
`math.sqrt(x)`	Square root of x	`math.sqrt(25)` → `5.0`
`math.pow(x, y)`	x raised to power y	`math.pow(2, 4)` → `16.0`
`math.ceil(x)`	Round up	`math.ceil(4.2)` → `5`

Function/Constant	Description	Example
`math.floor(x)`	Round down	`math.floor(4.8)` → `4`
`math.log(x, base)`	Logarithm	`math.log(100, 10)` → `2.0`
`math.sin(x)`	Sine (in radians)	`math.sin(math.pi/2)` → `1.0`
`math.radians(deg)`	Convert degrees to radians	`math.radians(90)`

Worked Example

Example 2.5.1: Circle Calculations

```
import math

radius = 5
area = math.pi * radius ** 2
circumference = 2 * math.pi * radius

print(f"Area of circle with radius {radius}: {area:.2f}")
print(f"Circumference: {circumference:.2f}")

number = 81
square_root = math.sqrt(number)
print(f"Square root of {number}: {square_root}")
```

Expected Behavior or Output

Output of example 1:

```
Area of circle with radius 5: 78.54
Circumference: 31.42
Square root of 81: 9.0
```

Common Mistakes

1. Forgetting `import math`, so `math.sqrt` raises a `NameError`.
2. Writing `math.pi * radius * 2` for the area. The radius must be squared: `math.pi * radius ** 2`.
3. Expecting `math.sqrt(81)` to print `9`. It prints `9.0` because square roots are floats.

Activities

Activity 2.5.1: Circle Calculator

```
import math

radius_str = input("Enter the radius: ")
radius = float(radius_str)

area = math.pi * radius ** 2
circumference = 2 * math.pi * radius

print(f"Area: {area:.2f}")
print(f"Circumference: {circumference:.2f}")
```

Activity 2.5.2: Distance Finder

```
import math

x = float(input("Enter x coordinate: "))
y = float(input("Enter y coordinate: "))
distance = math.sqrt(x**2 + y**2)
print(f"Distance from origin: {distance:.2f}")
```

Sample run with the shared coordinates x = 3 and y = 4 (typed input shown after each prompt):

```
Enter x coordinate: 3
Enter y coordinate: 4
Distance from origin: 5.00
```

Accessibility Practice

Round results with `:.2f` so the screen reader announces `78.54` rather than fifteen digits. Say the dotted names aloud ('math dot sqrt') and listen for the dot, since a missing dot is a common transcription mistake when reading code by ear.

Skills Checklist

- ☐ I can explain what a Python module is.
- ☐ I can import the `math` module into my program.
- ☐ I can use `math.pi` and `math.sqrt()`.
- ☐ I can use `math.ceil()` to round up and `math.floor()` to round down.

Canonical Shared Dataset — Calculator and Coordinates

Use the canonical bill and tip rate in every parallel-language implementation.

```
from decimal import Decimal, ROUND_HALF_UP

bill = Decimal("24.50")
tip_rate = Decimal("0.15")
```

```
tip = (bill * tip_rate).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)
total = bill + tip
print(f"Tip: ${tip:.2f}")
print(f"Total: ${total:.2f}")
```

Expected output:

```
Tip: $3.68
Total: $28.18
```

Use the same coordinate values for the math-library example:

```
import math
x, y = 3, 4
distance = math.sqrt(x**2 + y**2)
print(distance)
```

Expected output: `5.0`. The shared monetary contract rounds the tip to cents using decimal half-up rounding before adding it to the bill. Use decimal arithmetic rather than binary floating-point formatting for currency.

13.5 Unit 3: Control Flow — Decisions and Loops

Overview

Compatibility: All examples target Python 3.12 and newer; they avoid syntax and standard-library APIs introduced after Python 3.12.

This unit introduces how programs make decisions using `if` statements and perform repetitive tasks with `for` loops, along with fundamental string manipulation techniques.

Objectives:

- Control program flow with `if`, `elif`, and `else`.
- Iterate over sequences using `for` loops.
- Understand and manipulate strings.

Lesson 3.1: Boolean Expressions and `if` Statements

Reading References

- Zelle: Chapter 7, pages 165–173
- Matthes: Chapter 5, pages 71–78

Learning Objectives

- Identify `True` and `False` as Boolean values.
- Use comparison operators to create Boolean expressions.
- Write a basic `if` statement.
- Correctly indent the code block belonging to an `if` statement.

Concept Explanation

Comparison and Boolean Operators

Operator	Meaning	Example	Result
`==`	Equal to	`5 == 5`	`True`
`!=`	Not equal to	`5 != 10`	`True`
`<`	Less than	`5 < 10`	`True`
`>`	Greater than	`10 > 5`	`True`
`<=`	Less than or equal to	`5 <= 5`	`True`
`>=`	Greater than or equal	`10 >= 5`	`True`

Boolean Operator	Description	Example	Result
`and`	True if both are true	`True and False`	`False`
`or`	True if at least one is true	`True or False`	`True`
`not`	Inverts the truth value	`not True`	`False`

The `if` Statement

```
score = 59
if score >= 60:
    print("Passing score.")
print("Program finished.")  # This runs regardless
```

Indentation defines which lines belong to the `if` block.

Worked Example

Example 3.1.1: Comparing a Score

```
score = 82

if score >= 60:
    print("Avery passed.")
else:
    print("Avery did not pass.")

print(score >= 80 and score < 90)   # both conditions must be true
```

```
print(score < 60 or score > 100)    # at least one condition must be true
print(not score >= 90)              # reverses True and False
```

Example 3.1.2: Password Length Check

```
password = "mysecret"
if len(password) < 8:
    print("Weak password: Must be at least 8 characters.")
```

Expected Behavior or Output

Expected output:

```
Avery passed.
True
False
True
```

Common Mistakes

1. Writing `=` where `==` is needed in a condition, which is a syntax error in Python.
2. Forgetting the colon at the end of the `if` line or the indentation of its block.
3. Writing `score >= 80 and < 90` instead of repeating the variable in each comparison.

Activities

Activity 3.1.1: Number Comparison Game

```
num1 = int(input("Enter the first number: "))
num2 = int(input("Enter the second number: "))

if num1 > num2:
    print(f"{num1} is greater than {num2}.")
if num1 == num2:
    print("The numbers are equal.")
if num1 < num2:
    print(f"{num1} is less than {num2}.")
```

Accessibility Practice

Read each condition aloud using the words 'greater than or equal to', 'and', 'or', and 'not'. Confirm that your screen reader announces the colon at the end of the `if` line and the indentation of the next line, because these decide which statements run.

Skills Checklist

- ☐ I can identify `True` and `False` as Boolean values.
- ☐ I can use comparison operators to create Boolean expressions.
- ☐ I can write a basic `if` statement.
- ☐ I can correctly indent the code block belonging to an `if` statement.

Lesson 3.2: Chained Conditionals (`elif`, `else`)

Reading References

- Zelle: Chapter 7, pages 173–176
- Matthes: Chapter 5, pages 79–84

Learning Objectives

- Use an `else` statement to provide an alternative path.
- Use `elif` to check multiple conditions in order.
- Use `and`, `or`, and `not` operators to combine conditions.

Concept Explanation

`if-elif-else` Structure Python checks each condition in **sequential order**. As soon as a condition is `True`, it runs that block and skips the rest.

```
score = 82

if score >= 90:
    print("Grade: A")
elif score >= 80:
    print("Grade: B")
elif score >= 70:
    print("Grade: C")
elif score >= 60:
    print("Grade: D")
else:
    print("Grade: F")
```

Output: `Grade: B`

Worked Example

Example 3.2.1: Grade Classification and Boundary Scores Every language track uses these same thresholds: A at 90, B at 80, C at 70, D at 60, otherwise F. The test scores sit exactly on and just below each boundary. (The `def` and `return` lines are explained fully in Unit 6; for now read `classify(score)` as "the grade for this score.")

```
def classify(score):
    if score >= 90:
        return "A"
    elif score >= 80:
        return "B"
    elif score >= 70:
        return "C"
    elif score >= 60:
        return "D"
    else:
        return "F"

for score in [90, 89, 80, 79, 70, 69, 60, 59]:
    print(score, classify(score))
```

Expected Behavior or Output

Expected output:

```
90 A
89 B
80 B
79 C
70 C
69 D
60 D
59 F
```

Common Mistakes

1. Putting the conditions in the wrong order, such as testing `score >= 60` first so every passing score is reported as a D.
2. Using `>` instead of `>=`, so exactly 90 receives a B.
3. Using a separate `if` for each grade instead of `elif`, so more than one message can print.

Activities

Activity 3.2.1: Grade Checker

```
score = int(input("Enter a score from 0 to 100: "))

if score >= 90:
    print("Grade: A")
elif score >= 80:
    print("Grade: B")
elif score >= 70:
    print("Grade: C")
```

```
elif score >= 60:
    print("Grade: D")
else:
    print("Grade: F")
```

Sample run (typed input shown after each prompt):

```
Enter a score from 0 to 100: 88
Grade: B
```

Run it again with 90, 89, 60, and 59 and confirm the grades A, B, D, and F.

Accessibility Practice

Read the boundary results aloud in order (`90 A`, `89 B`, `80 B`, `79 C`...). Each line is a number, a space, and a letter, so it is easy to follow by ear and in braille. Do not rely on indentation alone to see which `elif` belongs to which `if`; count the lines that start at the left margin.

Skills Checklist

- ☐ I can use an `else` statement to provide an alternative path.
- ☐ I can use `elif` to check multiple conditions in order.
- ☐ I can use `and`, `or`, and `not` operators to combine conditions.

Lesson 3.3: Introduction to Lists

Reading References

- Zelle: Chapter 5, pages 105–106
- Matthes: Chapter 3, pages 33–47

Learning Objectives

- Create a list in Python.
- Access individual items in a list using their index.
- Access list items from the end using negative indices.
- Change the value of an item at a specific index.

Concept Explanation

A *list* is an ordered collection of values written between square brackets. Each value has a position called an *index*, and **Python starts counting at 0**. Negative indexes count from the end, so `-1` is the last item. Lists are *mutable*: you can replace an item without creating a new list.

Creating and Accessing Lists

```
scores = [82, 91, 76, 88]

print(scores[0])   # 82  (first item)
print(scores[-1])  # 88  (last item)
print(len(scores)) # 4
```

Zero-based indexing: The first item is at index `0`.

Modifying Lists

Worked Example

```
scores = [82, 91, 76, 88]
scores[2] = 80  # Change the third score (index 2) from 76 to 80
print(scores)   # [82, 91, 80, 88]
```

Expected Behavior or Output

Output of example 1:

```
[82, 91, 80, 88]
```

Common Mistakes

1. Using `scores[1]` for the first item. The first item is at index 0.
2. Using an index that is too large, which raises an `IndexError`. The last valid index is `len(scores) - 1`.
3. Assuming `copy = scores` makes a separate list. Both names refer to the same list.

Activities

Activity 3.3.1: My Top 3 Hobbies

```
hobbies = ["reading", "coding", "hiking"]
print("My hobbies:", hobbies)
print("My second hobby:", hobbies[1])
hobbies[2] = "swimming"
print("Updated hobbies:", hobbies)
```

Accessibility Practice

When reading `scores[-1]` aloud, say 'scores, open bracket, minus one, close bracket' so the sign is not lost. Print a whole list on one line, and read its length separately, so the screen reader announces the state of the list in a single pass.

Skills Checklist

- ☐ I can create a list in Python.
- ☐ I can access individual items in a list using their index.
- ☐ I can access list items from the end using negative indices.
- ☐ I can change the value of an item at a specific index.

Lesson 3.4: Iterating with `for` Loops

Reading References

- Zelle: Chapter 2, pages 34–36
- Matthes: Chapter 4, pages 49–60

Learning Objectives

- Explain the purpose of a `for` loop.
- Write a `for` loop to iterate over items in a list.
- Write a `for` loop to iterate over characters in a string.
- Use the `range()` function to control how many times a loop runs.

Concept Explanation

The `for` Loop Structure

```
for item in sequence:
    # code that runs for each item
```

Worked Example

Example 3.4.1: Total and Average of the Shared Scores

```
scores = [82, 91, 76, 88]
total = 0

for score in scores:
    print(f"Adding {score}")
    total = total + score

average = total / len(scores)
print(f"Total: {total}")
print(f"Average: {average}")
```

Example 3.4.2: Looping Through a String

```
word = "Python"
for char in word:
    print(char)
```

Example 3.4.3: Using `range()`

```
for i in range(5):      # 0, 1, 2, 3, 4
    print(i)

for i in range(2, 8):   # 2, 3, 4, 5, 6, 7
    print(i)

for i in range(0, 10, 2):  # 0, 2, 4, 6, 8
    print(i)
```

Expected Behavior or Output

Expected output:

```
Adding 82
Adding 91
Adding 76
Adding 88
Total: 337
Average: 84.25
```

Common Mistakes

1. Forgetting the colon after the `for` line or indenting the loop body inconsistently.
2. Changing a list while looping over it, which can skip items.
3. Using `range(1, 4)` and expecting it to include 4. The stop value is not included.

Activities

Activity 3.4.1: My Favorite Things

```
favorite_things = ["sunshine", "coffee", "books", "music", "mountains"]
for thing in favorite_things:
    print(f"I love {thing}!")
```

Activity 3.4.2: Countdown Timer

```
start = int(input("Enter a starting number: "))
for i in range(start, 0, -1):
    print(i)
print("Blast off!")
```

Accessibility Practice

Print one labelled line per loop pass (`Adding 82`), so your screen reader reads each step. Count the lines to confirm the loop ran the expected number of times,

since the loop's structure is carried by indentation and not by visible braces.

Skills Checklist

- ☐ I can explain the purpose of a `for` loop.
- ☐ I can write a `for` loop to iterate over items in a list.
- ☐ I can write a `for` loop to iterate over characters in a string.
- ☐ I can use the `range()` function to control how many times a loop runs.

Lesson 3.5: Basic String Operations

Reading References

- Zelle: Chapter 5, pages 99–115
- Matthes: Chapter 2, pages 19–25

Learning Objectives

- Combine strings using concatenation.
- Find the length of a string using `len()`.
- Convert a string to uppercase/lowercase.
- Remove extra spaces using `.strip()`.
- Replace parts of a string using `.replace()`.

Concept Explanation

Common String Methods

```
message = "  Hello Python!  "

message.strip()           # "Hello Python!"   - removes whitespace
message.upper()           # "  HELLO PYTHON!  "
message.lower()           # "  hello python!  "
message.replace("Hello", "Hi")  # "  Hi Python!  "
len(message)              # 17
```

Important: String methods return a **new string** — they do not modify the original.

Worked Example

Example 3.5.1: String Manipulation

```
message = "  Hello Python  "
print(f"Original: '{message}'")
print(f"Length: {len(message)}")
print(f"Uppercase: '{message.upper()}'")
print(f"Stripped: '{message.strip()}'")
```

Example 3.5.2: Normalizing a Name and Searching the Shared Text Sample

```
raw_name = "  avery  "
clean_name = raw_name.strip().title()
print(f"Name: '{clean_name}'")

text = "A student studies programming. Programming develops problem-solving skills."
lowered = text.lower()
print("Characters:", len(text))
print("Contains 'programming':", "programming" in lowered)
print("First position of 'programming':", lowered.find("programming"))
print("Times 'programming' appears:", lowered.count("programming"))
```

Expected Behavior or Output

Expected output:

```
Name: 'Avery'
Characters: 75
Contains 'programming': True
First position of 'programming': 18
Times 'programming' appears: 2
```

Common Mistakes

1. Expecting `text.upper()` to change `text`. Strings are immutable; assign the result: `text = text.upper()`.
2. Forgetting that `find` returns `-1`, not an error, when the text is missing.
3. Counting positions from 1. The first character is at index 0.

Activities

Activity 3.5.1: Name Formatter

```
first_name = input("Enter your first name: ")
last_name = input("Enter your last name: ")
full_name = first_name + " " + last_name
print(f"Full name: {full_name}")
print(f"Uppercase: {full_name.upper()}")
print(f"Total characters: {len(full_name)}")
```

Accessibility Practice

Strings with many quotation marks are hard to read by ear. Say 'single quote' and 'double quote' aloud when checking a line, and print results inside labelled sentences such as `Characters: 75` so the screen reader does not run values together.

Skills Checklist

- ☐ I can combine strings using concatenation.
- ☐ I can find the length of a string using `len()`.
- ☐ I can convert a string to uppercase/lowercase.
- ☐ I can remove extra spaces using `.strip()`.
- ☐ I can replace parts of a string using `.replace()`.

Canonical Shared Dataset — Grade Thresholds and Scores

All tracks must classify scores using A 90, B 80, C 70, D 60, and F otherwise. Use the same score sequence.

```
scores = [82, 91, 76, 88]

def grade_for(score):
    if score >= 90:
        return "A"
    if score >= 80:
        return "B"
    if score >= 70:
        return "C"
    if score >= 60:
        return "D"
    return "F"

for score in scores:
    print(score, grade_for(score))

total = sum(scores)
average = total / len(scores)
print("Total:", total)
print("Average:", average)
```

Expected output:

```
82 B
91 A
76 C
88 B
Total: 337
Average: 84.25
```

Boundary tests must include 90, 80, 70, 60, and 59. Do not silently change a threshold or replace the shared scores with a different set in a parallel example.

13.6 Unit 4: Data Structures — Lists and Dictionaries

Overview

Compatibility: All examples target Python 3.12 and newer; they avoid syntax and standard-library APIs introduced after Python 3.12.

This unit deepens understanding of lists and introduces dictionaries as powerful mechanisms for organizing data.

Objectives:

- Add, remove, and organize list elements.
- Work with tuples (immutable lists).
- Use dictionaries for key-value pair storage.
- Loop through dictionaries.

Lesson 4.1: More on Lists

Reading References

- Matthes: Chapter 3 (Introducing Lists) and Chapter 4 (Working with Lists)

Learning Objectives

- Add an item to the end of a list using `.append()`.
- Insert an item at a specific position using `.insert()`.
- Remove an item by index using `del`.
- Remove an item and retrieve its value using `.pop()`.
- Remove the first occurrence of a value using `.remove()`.

Concept Explanation

Adding and Removing Elements

```
my_list = [10, 20, 30]

my_list.append(40)          # Add to end → [10, 20, 30, 40]
my_list.insert(1, 15)       # Insert at index 1 → [10, 15, 20, 30, 40]
del my_list[0]              # Delete by index → [15, 20, 30, 40]
removed = my_list.pop()     # Remove and return last → removed=40, list=[15, 20, 30]
my_list.remove(20)          # Remove by value → [15, 30]
```

Worked Example

```
scores = [82, 91, 76, 88]

scores.append(95)         # [82, 91, 76, 88, 95]
scores.insert(1, 85)      # [82, 85, 91, 76, 88, 95]
scores[2] = 93            # [82, 85, 93, 76, 88, 95]
scores.remove(76)         # [82, 85, 93, 88, 95]
last_score = scores.pop()  # last_score is 95

print(scores)
print(len(scores))
print(88 in scores)
```

Expected Behavior or Output

Expected output:

```
[82, 85, 93, 88]
4
True
```

Common Mistakes

1. Using `list.insert(value, position)`. The position comes first: `scores.insert(1, 85)`.
2. Calling `remove()` with a value that is not in the list, which raises a `ValueError`; check with `in` first.
3. Removing items from a list while looping over it, which skips the item after each removal.

Activities

1. Trace the list after each operation in the example and write each version as one sentence.
2. Remove the first score with `pop(0)` and explain what happens to the other scores.
3. Write code that removes every score below 80 safely.
4. Print the position of the first score above 90 with `index()`, and explain what happens if none exists.

Accessibility Practice

After every change, print the whole list on one line so a screen reader reads a complete state in one pass. Say the comments aloud as part of each line, since they hold the expected value after the change.

Skills Checklist

- ☐ I can add an item to the end of a list using `.append()`.
- ☐ I can insert an item at a specific position using `.insert()`.
- ☐ I can remove an item by index using `del`.
- ☐ I can remove an item and retrieve its value using `.pop()`.
- ☐ I can remove the first occurrence of a value using `.remove()`.

Lesson 4.2: Organizing Lists

Reading References

- Matthes: Chapter 3 (Introducing Lists) and Chapter 4 (Working with Lists)

Learning Objectives

- Sort a list permanently using `.sort()`.
- Get a temporarily sorted version using `sorted()`.
- Reverse a list using `.reverse()`.
- Find how many items are in a list using `len()`.

Concept Explanation

Lists can be reordered in two ways. `sort()` changes the list *in place* and returns nothing. `sorted()` returns a *new* sorted list and leaves the original unchanged. Both accept `reverse=True` for descending order.

Sorting and Reversing

Worked Example

```
scores = [82, 91, 76, 88]
scores.sort()                           # Permanent sort in-place
print(scores)                           # [76, 82, 88, 91]

fruits = ["orange", "apple", "banana"]
sorted_fruits = sorted(fruits)          # Temporary sort - original unchanged
print(fruits)                           # ['orange', 'apple', 'banana']
print(sorted_fruits)                    # ['apple', 'banana', 'orange']

scores.reverse()                        # Reverse in-place -> [91, 88, 82, 76]
scores.sort(reverse=True)               # Sort descending -> [91, 88, 82, 76]
print(scores)                           # [91, 88, 82, 76]
print(len(scores))                      # Length: 4
```

Expected Behavior or Output

Output of example 1:

```
[76, 82, 88, 91]
['orange', 'apple', 'banana']
['apple', 'banana', 'orange']
[91, 88, 82, 76]
4
```

Common Mistakes

1. Writing `scores = scores.sort()`. `sort()` returns `None`, so the list is lost.
2. Sorting a list that mixes numbers and text, which raises a `TypeError`.
3. Expecting `reverse()` to sort. It only flips the current order.

Activities

1. Predict the result of `scores.sort()` and `scores.sort(reverse=True)` on the shared scores, then check.
2. Use `sorted(scores)` and show that the original list is unchanged.
3. Sort the words `["pencil", "folder", "notebook"]` and print them.
4. Explain the difference between `sort()` and `sorted()` in two sentences.

Accessibility Practice

Print each sorted version on its own labelled line (`Ascending:`, `Descending:`). The first and last numbers are the quickest to check by ear.

Skills Checklist

- ☐ I can sort a list permanently using `.sort()`.
- ☐ I can get a temporarily sorted version using `sorted()`.
- ☐ I can reverse a list using `.reverse()`.
- ☐ I can find how many items are in a list using `len()`.

Lesson 4.3: Tuples — Immutable Lists

Reading References

- Matthes: Chapter 4 (Working with Lists), the section on tuples

Learning Objectives

- Create a tuple in Python.
- Explain the main difference between a list and a tuple (mutability).

- Access items in a tuple by their index.

Concept Explanation

A *tuple* is an ordered collection like a list, written with parentheses, but it cannot be changed after it is created. Use a tuple for a fixed group of values such as the shared coordinates (3, 4). Because it cannot change, a tuple can also be used where Python needs a value that never changes, such as a dictionary key.

Tuples Tuples are defined with parentheses and **cannot be changed** after creation.

When to use tuples: For data that should remain constant (coordinates, RGB values, fixed settings).

Worked Example

```
coordinates = (3, 4)
x, y = coordinates          # tuple unpacking
print(coordinates[0])       # 3
print(coordinates[-1])      # 4
print(x, y)                 # 3 4

# This would cause a TypeError:
# coordinates[1] = 5
```

Expected Behavior or Output

Output of example 1:

```
3
4
3 4
```

Common Mistakes

1. Writing `(5)` for a one-item tuple. A one-item tuple needs a comma: `(5,)`.
2. Trying to assign to an item (`coordinates[1] = 5`), which raises a `TypeError`.
3. Using a list when the data should never change, so an accidental change goes unnoticed.

Activities

1. Create the tuple `(3, 4)`, unpack it into `x` and `y`, and print both.
2. Write a function that returns a name, an age, and a height as a tuple and unpack all three.
3. Try to change one item of a tuple and read the error message aloud.

4. Explain, in two sentences, when a tuple is a better choice than a list.

Accessibility Practice

Say the parentheses and the commas aloud when reading a tuple, because a one-item tuple differs from a plain value only by its trailing comma. Read error messages from the last line upward to find the problem quickly.

Skills Checklist

☐ I can create a tuple in Python.
☐ I can explain the main difference between a list and a tuple (mutability).
☐ I can access items in a tuple by their index.

Lesson 4.4: Introducing Dictionaries

Reading References

- Zelle: Chapter 11, pages 307–309
- Matthes: Chapter 6, pages 91–98

Learning Objectives

- Create a dictionary in Python.
- Explain what a key-value pair is.
- Access a value in a dictionary using its key.
- Add, modify, and remove key-value pairs.

Concept Explanation

Dictionary Basics A dictionary stores **key-value pairs**. Keys must be unique and immutable (strings, numbers, tuples). Values can be anything.

```
student_profile = {
    "name": "Avery",
    "age": 16,
    "height_m": 1.72,
    "favorite_subject": "Computer Science",
}

print(student_profile["name"])
print(student_profile["favorite_subject"])

# Demonstrate adding and removing a temporary key without changing
# the four-field canonical student record.
student_profile["temporary_note"] = "Dictionary example"
```

```
del student_profile["temporary_note"]
print(sorted(student_profile))
```

Expected output:

```
Avery
Computer Science
['age', 'favorite_subject', 'height_m', 'name']
```

Worked Example

Example 4.4.1: Inventory Item Dictionary

```
item = {
    "name": "Notebook",
    "quantity": 3,
    "unit_price": 2.50
}

print(f"Item: {item['name']}")
print(f"Quantity: {item['quantity']}")

item["quantity"] = item["quantity"] + 2    # restock two more
item["location"] = "Shelf A"               # add a new key
print(item)
print("Stock value:", item["quantity"] * item["unit_price"])
```

Expected Behavior or Output

Expected output:

```
Item: Notebook
Quantity: 3
{'name': 'Notebook', 'quantity': 5, 'unit_price': 2.5, 'location': 'Shelf A'}
Stock value: 12.5
```

Common Mistakes

1. Using a key that does not exist (`profile["grade"]`), which raises a `KeyError`; use `profile.get("grade")` to receive `None` instead.
2. Forgetting the quotation marks around a text key, so Python looks for a variable.
3. Expecting dictionary order to be sorted. Dictionaries keep insertion order, not alphabetical order.

Activities

1. Predict the printed values before running the example, then run it.
2. Add a `"discount"` key to the item and print the discounted line value.

3. Remove `"favorite_subject"` from the profile with `del` and show what `get` returns afterward.
4. Write a check that tells whether an item has all of the keys `name`, `quantity`, and `unit_price`.

Accessibility Practice

Choose key names that read well aloud, such as `unit_price`. Print one key and value per line, so the screen reader announces each pair separately, and avoid relying on curly braces alone to see where a dictionary ends.

Skills Checklist

- ☐ I can create a dictionary in Python.
- ☐ I can explain what a key-value pair is.
- ☐ I can access a value in a dictionary using its key.
- ☐ I can add, modify, and remove key-value pairs.

Lesson 4.5: Looping Through Dictionaries

Reading References

- Matthes: Chapter 6 (Dictionaries), the sections on looping through a dictionary

Learning Objectives

- Loop through all the keys in a dictionary.
- Loop through all the values in a dictionary.
- Loop through both keys and values at the same time.

Concept Explanation

Dictionary Iteration Methods

Worked Example

```
inventory = {
    "Notebook": {"quantity": 3, "unit_price": 2.50},
    "Pencil": {"quantity": 10, "unit_price": 0.50},
    "Folder": {"quantity": 4, "unit_price": 1.25},
}

# Loop through keys (default)
for item_name in inventory:
    print(item_name)
```

```python
# Loop through values only
for details in inventory.values():
    print(details["quantity"])

# Loop through key-value pairs
total_value = 0
for item_name, details in inventory.items():
    line_value = details["quantity"] * details["unit_price"]
    print(f"{item_name}: {details['quantity']} in stock, value {line_value:.2f}")
    total_value += line_value
print(f"Total inventory value: {total_value:.2f}")
```

Expected Behavior or Output

Expected output:

```
Notebook
Pencil
Folder
3
10
4
Notebook: 3 in stock, value 7.50
Pencil: 10 in stock, value 5.00
Folder: 4 in stock, value 5.00
Total inventory value: 17.50
```

Common Mistakes

1. Looping with `for item in inventory` and expecting the values. The default loop gives the keys.
2. Changing the dictionary's size while looping over it, which raises a `RuntimeError`.
3. Forgetting `.items()` when both the key and the value are needed.

Activities

1. Run the example and write the three key-value lines in order.
2. Print only the items with a quantity below 5.
3. Print the inventory sorted by the line value, highest first.
4. Explain why `.items()` returns pairs and how the loop unpacks them.

Accessibility Practice

Print each inventory item as one complete sentence so the screen reader reads the name, quantity, and value together. Python dictionaries keep insertion order,

so the order is stable and can be learned by ear.

Skills Checklist

- ☐ I can loop through all the keys in a dictionary.
- ☐ I can loop through all the values in a dictionary.
- ☐ I can loop through both keys and values at the same time.

13.7 Unit 5: User Input and `while` Loops

Overview

This unit focuses on obtaining diverse input from users and controlling program execution through indefinite loops.

Objectives:

- Implement indefinite loops using `while` statements.
- Control `while` loop execution with `break` and `continue`.
- Avoid infinite loops.

Lesson 5.1: How the `input()` Function Works

Reading References

- Matthes: Chapter 7 (User Input and while Loops), the section on how the `input()` function works
- Zelle: Chapter 2 (Writing Simple Programs), the section on input

Learning Objectives

- Effectively use `input()` to get user data.
- Confirm that `input()`'s return value is always a string.
- Convert user input to a numeric type when necessary.

Concept Explanation

`input(prompt)` shows the prompt, waits for the user to press Enter, and returns what was typed as a **string**, even when the user typed digits. To use the value as a number, convert it with `int()` or `float()`. Checking first with `isdigit()` avoids an error for text that is not a whole number; Lesson 8.3 shows the general solution, exceptions.

Key Reminder `input()` **always returns a string**. Convert to numeric types as needed.

```
age_input = input("How old are you? ")
print(f"Type: {type(age_input)}")    # <class 'str'>

age_int = int(age_input)
print(f"In 5 years: {age_int + 5}")
```

Worked Example

```
text = input("How old are you? ")
print("Type:", type(text).__name__)
if text.isdigit():
    age = int(text)
    print("In 5 years:", age + 5)
else:
    print("That was not a whole number.")
```

Expected Behavior or Output

Sample run typing `16`:

```
How old are you? 16
Type: str
In 5 years: 21
```

Sample run typing `abc`:

```
How old are you? abc
Type: str
That was not a whole number.
```

Common Mistakes

1. Adding to the value returned by `input()` without converting it. `"16" + 5` raises a `TypeError`.
2. Calling `int()` on text such as `"abc"` or an empty answer, which raises a `ValueError`.
3. Using `isdigit()` for numbers with a minus sign or a decimal point, which it rejects.

Activities

1. Run the program typing `16`, then `abc`, then just Enter. Compare the three outputs.
2. Change the program so that it accepts an age only from 0 to 120.
3. Write a program that asks for a name and a number and prints one sentence using both.

4. Explain why `type(text)` is `str` even when you typed 16.

Accessibility Practice

Each prompt ends with a space and says what to type. When you test, listen to whether your screen reader reads the prompt in full before you start typing, and write the prompt text so it still makes sense when read on its own.

Skills Checklist

☐ I can effectively use `input()` to get user data.
☐ I can confirm that `input()`'s return value is always a string.
☐ I can convert user input to a numeric type when necessary.

Lesson 5.2: The `while` Loop in Action

Reading References

- Matthes: Chapter 7 (User Input and while Loops)
- Zelle: Chapter 8 (Loop Structures and Booleans)

Learning Objectives

- Explain the difference between a `for` loop and a `while` loop.
- Write a basic `while` loop.
- Make sure my `while` loop has a way to stop.

Concept Explanation

`for` vs. `while` Loop Comparison

Feature	`for`	Loop
Purpose	Definite iteration (known count)	Indefinite iteration (condition-based)
When to Use	Iterating over lists, ranges	Repeating until a condition is met
Control Variable	Automatically updated	Must be manually updated
Risk	Less prone to infinite loops	High risk if condition never becomes false

Worked Example

```
count = 1
while count <= 5:
    print(count)
```

```
    count += 1
print("Counting finished!")

message = input("Enter a message, or type quit to exit: ")
while message != "quit":
    print("You said: " + message)
    message = input("Enter a message, or type quit to exit: ")
print("Goodbye!")
```

Expected Behavior or Output

Sample run typing `hello`, `scores`, and `quit`:

```
1
2
3
4
5
Counting finished!
Enter a message, or type quit to exit: hello
You said: hello
Enter a message, or type quit to exit: scores
You said: scores
Enter a message, or type quit to exit: quit
Goodbye!
```

Common Mistakes

1. Forgetting to change the loop variable, which creates an endless loop.
2. Reading input only before the loop and never again inside it, so the same message repeats forever.
3. Comparing the sentinel with different capitalization (`Quit` and `quit`) and never stopping.

Activities

1. Predict the output for the typed lines `hello`, `scores`, and `quit`.
2. Change the sentinel to `exit` and update the prompt.
3. Count how many messages were entered and print the count at the end.
4. Rewrite the counting loop as a `for` loop with `range` and compare the two.

Accessibility Practice

Each prompt is a full sentence that names the way to stop. The response begins with the label `You said:` so the conversation can be followed in order by ear. If a loop floods the screen reader with output, stop it with Ctrl+C.

Skills Checklist

- ☐ I can explain the difference between a `for` loop and a `while` loop.
- ☐ I can write a basic `while` loop.
- ☐ I can make sure my `while` loop has a way to stop.

Lesson 5.3: Using Flags and `break` to Exit Loops

Reading References

- Matthes: Chapter 7 (User Input and while Loops)

Learning Objectives

- Use a Boolean "flag" variable to control a `while` loop.
- Use the `break` statement to exit a loop early.

Concept Explanation

Flag Variable

```
active = True
while active:
    message = input("Say something (or 'quit' to stop): ")
    if message == 'quit':
        active = False
    else:
        print(f"You said: {message}")
```

`break` Statement

```
while True:
    city = input("Enter a city (or 'done' to finish): ")
    if city == 'done':
        break
    print(f"I've heard {city} is a great place!")
```

Worked Example

Example: Stopping a Search with a Flag and `break` (shared scores)

```
scores = [82, 91, 76, 88]
found = False
index = 0

while index < len(scores):
    if scores[index] >= 90:
```

```
        found = True
        break
    index += 1

if found:
    print(f"First score of 90 or higher: {scores[index]} at index {index}")
else:
    print("No score of 90 or higher.")
```

Expected Behavior or Output

Expected output:

```
First score of 90 or higher: 91 at index 1
```

Common Mistakes

1. Setting the flag but running more code in the same pass that should have been skipped.
2. Forgetting to set the result variable before the loop and misreading an old value afterward.
3. Using `break` in an inner loop and expecting it to leave the outer loop. It leaves only the innermost loop.

Activities

1. Change the search to find the first score below 80 and predict the index.
2. Rewrite a flag-controlled loop so that it uses `break` and compare the two versions.
3. Write a loop that stops when the user types `quit` and also stops after 5 attempts.
4. Explain in two sentences when a flag is clearer than `break`.

Accessibility Practice

Report search results as complete sentences that include both the value and its position, so the sentence is understood without any layout. Remember that Python indexes start at 0, and say 'index 1' aloud rather than 'second' to avoid confusion.

Skills Checklist

- ☐ I can use a Boolean "flag" variable to control a `while` loop.
- ☐ I can use the `break` statement to exit a loop early.

Lesson 5.4: Using `continue` in Loops

Reading References

- Matthes: Chapter 7 (User Input and while Loops), the section on using `continue`

Learning Objectives

- Use `continue` to skip the rest of the current loop iteration.
- Identify scenarios where `continue` would be useful.

Concept Explanation

`continue` — Skip the Current Iteration

```
number = 0
while number < 10:
    number += 1
    if number % 2 == 0:
        continue        # Skip even numbers
    print(number)       # Prints 1, 3, 5, 7, 9
```

`continue` vs. `break`

- `break` — **exits the loop entirely**
- `continue` — **skips only the current iteration**, loop continues

Worked Example

Example: Skipping Invalid Entries with `continue`

```
raw_scores = [82, -5, 91, 105, 76, 88]
valid_total = 0
valid_count = 0

for raw in raw_scores:
    if raw < 0 or raw > 100:
        print(f"Skipping invalid score: {raw}")
        continue
    valid_total += raw
    valid_count += 1

print(f"Valid scores: {valid_count}, total: {valid_total}")
```

Expected Behavior or Output

Expected output:

```
Skipping invalid score: -5
Skipping invalid score: 105
Valid scores: 4, total: 337
```

Common Mistakes

1. Placing statements that must always run after `continue`, so they are skipped.
2. Using `continue` in a `while` loop before changing the loop variable, which creates an endless loop.
3. Confusing `continue` (skip this pass) with `break` (leave the loop).

Activities

1. Predict the printed lines and the final total of the invalid-scores example.
2. Skip scores below 80 instead.
3. Rewrite the invalid-scores loop with an `if`/`else` and no `continue`, and compare the readability.
4. Write a loop over `range(1, 10)` that prints only odd numbers using `continue`.

Accessibility Practice

Print a line for every skipped entry, so the screen reader announces what was ignored. Never skip silently when a user needs to know what the program did with their data.

Skills Checklist

☐ I can use `continue` to skip the rest of the current loop iteration.
☐ I can identify scenarios where `continue` would be useful.

Lesson 5.5: Avoiding Infinite Loops

Reading References

- Matthes: Chapter 7 (User Input and while Loops), the sections on avoiding infinite loops
- Zelle: Chapter 8 (Loop Structures and Booleans)

Learning Objectives

- Define what an infinite loop is.
- Identify common causes of infinite loops.
- Stop a running program in an infinite loop.

Concept Explanation

An **infinite loop** never ends because its condition always remains `True`.

Common causes:

1. Forgetting to update the loop variable.
2. Updating the variable in the wrong direction.
3. Using an incorrect logical condition.

```
# INFINITE LOOP - Don't run!
count = 1
while count <= 5:
    print(count)
    # Missing: count += 1

# CORRECT
count = 1
while count <= 5:
    print(count)
    count += 1
```

To stop a running infinite loop: Press `Ctrl + C` in the terminal.

Worked Example

```
MAX_ATTEMPTS = 3
attempts = 0
score = None

while attempts < MAX_ATTEMPTS:
    attempts += 1
    text = input(f"Enter a score from 0 to 100 (attempt {attempts} of {MAX_ATTEMPTS}): ")
    if text.isdigit() and 0 <= int(text) <= 100:
        score = int(text)
        break
    print("Invalid score: " + text)

if score is not None:
    print("Accepted score:", score)
else:
    print("No valid score was entered.")
```

Expected Behavior or Output

Sample run typing `abc`, `105`, and `88`:

```
Enter a score from 0 to 100 (attempt 1 of 3): abc
Invalid score: abc
```

```
Enter a score from 0 to 100 (attempt 2 of 3): 105
Invalid score: 105
Enter a score from 0 to 100 (attempt 3 of 3): 88
Accepted score: 88
```

Sample run typing three invalid lines (`x`, `y`, `z`):

```
Enter a score from 0 to 100 (attempt 1 of 3): x
Invalid score: x
Enter a score from 0 to 100 (attempt 2 of 3): y
Invalid score: y
Enter a score from 0 to 100 (attempt 3 of 3): z
Invalid score: z
No valid score was entered.
```

Common Mistakes

1. Forgetting to increase `attempts`, so the limit is never reached.
2. Testing the wrong variable in the loop condition, so it never becomes false.
3. Not knowing the interrupt key. Press Ctrl+C in the terminal to stop a program that will not end.

Activities

1. Run the sample with `abc`, `105`, and `88` and confirm the accepted score.
2. Run it with three invalid lines and read the final message.
3. Change the limit to 5 and test it.
4. For each loop you have written so far, write down what makes it stop.

Accessibility Practice

A loop that cannot be stopped is an accessibility problem as well as a bug, because it can flood a screen reader with output. Learn Ctrl+C before running loops, give every input loop a limit, and write error messages that say what is valid.

Skills Checklist

- ☐ I can define what an infinite loop is.
- ☐ I can identify common causes of infinite loops.
- ☐ I know how to stop a running program in an infinite loop.

13.8 Unit 6: Functions

Overview

This unit introduces functions, enabling organization of code into reusable blocks, passing arguments, and returning values.

Objectives:

- Define and call functions.
- Pass arguments to functions.
- Return values from functions.
- Understand the concept of scope.

Lesson 6.1: Defining Functions

Reading References

- Zelle: Chapter 6, pages 137–142
- Matthes: Chapter 8, pages 129–130

Learning Objectives

- Explain why functions are useful.
- Define a simple function in Python.
- Use parameters to specify function inputs.
- Call a function by its name.

Concept Explanation

Function Syntax

```
def function_name(parameter1, parameter2):
    """Docstring describing what the function does."""
    # Function body
    print(f"Hello, {parameter1}!")
```

Worked Example

Example 6.1.1: Simple Greeting Function

```
def greet_user():
    """Prints a simple greeting message."""
    print("Hello, Python programmer!")

greet_user()   # Call the function
greet_user()   # Call it again
```

Example 6.1.2: Function with a Parameter

```
def greet_name(name):
    """Prints a personalized greeting."""
    print(f"Hello, {name}!")

greet_name("Avery")
greet_name("Riley")
```

Example 6.1.3: A Reusable Tip Calculator (shared calculator values)

```
def show_tip(bill, tip_rate):
    """Print the tip and the total for a bill."""
    tip = bill * tip_rate
    print(f"Tip: {tip:.2f}")
    print(f"Total: {bill + tip:.2f}")

show_tip(24.5, 0.15)
```

Expected Behavior or Output

Expected output:

```
Tip: 3.67
Total: 28.18
```

Common Mistakes

1. Writing `greet_user` without parentheses and expecting the function to run.
2. Forgetting the colon after the `def` line, or the indentation of the body.
3. Passing arguments in the wrong order, such as `show_tip(0.15, 24.5)`.

Activities

1. Call `show_tip(100, 0.2)` and predict both printed lines.
2. Write a function `describe_student(name, age)` that prints one sentence.
3. Call `greet_name()` with no argument and read the error message.
4. Explain the difference between a parameter and an argument using the example.

Accessibility Practice

Choose function names that read as actions when spoken, such as `show_tip`. Print complete sentences from a function so the screen reader announces the result of each call separately, and say 'open parenthesis, close parenthesis' for an empty parameter list.

Skills Checklist

- ☐ I can explain why functions are useful.
- ☐ I can define a simple function in Python.
- ☐ I can use parameters to specify function inputs.
- ☐ I can call a function by its name.

Lesson 6.2: Passing Arguments

Reading References

- Matthes: Chapter 8 (Functions), the sections on passing arguments
- Zelle: Chapter 6 (Defining Functions)

Learning Objectives

- Differentiate between a parameter and an argument.
- Pass arguments using positional order.
- Pass arguments using keyword arguments.

Concept Explanation

Positional vs. Keyword Arguments

```
def describe_animal(animal_type, animal_name):
    print(f"I have a {animal_type} named {animal_name}.")

# Positional arguments (order matters)
describe_animal("dog", "Buddy")

# Keyword arguments (order doesn't matter)
describe_animal(animal_name="Max", animal_type="cat")
```

Default Values

```
def make_shirt(size, message="I love Python"):
    print(f"Making a {size}-sized shirt: '{message}'")

make_shirt("large")                         # Uses default message
make_shirt("medium", "Hello World")         # Custom message
```

Worked Example

Example: Default, Positional, and Keyword Arguments (shared calculator values)

```python
def calculate_total(bill, tip_rate=0.15):
    return bill + bill * tip_rate

print(f"{calculate_total(24.5):.2f}")                       # default rate
print(f"{calculate_total(24.5, 0.20):.2f}")                 # positional
print(f"{calculate_total(tip_rate=0.10, bill=24.5):.2f}")   # keyword
```

Expected Behavior or Output

Expected output:

```
28.18
29.40
26.95
```

Common Mistakes

1. Putting a parameter with a default before a parameter without one in the definition, which is a syntax error.
2. Mixing positional and keyword arguments in an order Python does not allow (positional first).
3. Using a mutable default such as `[]`, which is shared between calls.

Activities

1. Predict the three totals in the example before running it.
2. Add a third parameter `round_total` with the default `False` and use it.
3. Call the function with the arguments in a different order using keywords.
4. Explain why `tip_rate=0.15` is called a default value.

Accessibility Practice

Keyword arguments make a call read as a sentence: 'calculate total, bill equals 24.5, tip rate equals 0.10'. Say the equals signs aloud and check that your screen reader announces them.

Skills Checklist

- ☐ I can differentiate between a parameter and an argument.
- ☐ I can pass arguments using positional order.
- ☐ I can pass arguments using keyword arguments.

Lesson 6.3: Return Values

Reading References

- Matthes: Chapter 8 (Functions), the section on return values

- Zelle: Chapter 6 (Defining Functions)

Learning Objectives

- Use `return` to send a value back from a function.
- Explain the difference between `print()` and `return`.
- Store the returned value of a function in a variable.

Concept Explanation

The `return` Statement

```
def add_numbers(num1, num2):
    """Adds two numbers and returns the result."""
    total = num1 + num2
    return total

result = add_numbers(5, 3)
print(f"The sum is: {result}")   # 8
```

Key difference: `print()` displays a value; `return` sends a value back to the caller for further use.

Worked Example

Example: Returning Computed Results (shared scores)

```
def average(scores):
    """Return the average of a list of scores."""
    return sum(scores) / len(scores)

def classify(score):
    if score >= 90:
        return "A"
    elif score >= 80:
        return "B"
    elif score >= 70:
        return "C"
    elif score >= 60:
        return "D"
    return "F"

avg = average([82, 91, 76, 88])   # returned, not just printed
print(f"Average: {avg}")
print(f"Grade for the average: {classify(avg)}")
```

Expected Behavior or Output

Expected output:

```
Average: 84.25
Grade for the average: B
```

Common Mistakes

1. Printing inside a function and then trying to use the result in a calculation. The function returned `None`.
2. Writing statements after `return` in the same block. They never run.
3. Ignoring a returned value, so the work of the function is lost.

Activities

1. Predict the two printed results of the example before running.
2. Write `total(list)` that returns the sum of a list of numbers.
3. Write `is_passing(score)` that returns `True` or `False` using the shared thresholds.
4. Explain the difference between `print` and `return` in two sentences.

Accessibility Practice

Return values from functions and print them in one place. A function that prints and returns at the same time can announce a result twice. Read the single printed line aloud and check it matches the returned value.

Skills Checklist

☐ I can use `return` to send a value back from a function.
☐ I can explain the difference between `print()` and `return`.
☐ I can store the returned value of a function in a variable.

Lesson 6.4: Functions with Lists and Dictionaries

Reading References

- Matthes: Chapter 8 (Functions), the section on passing a list to a function

Learning Objectives

- Pass a list or dictionary as an argument to a function.
- Understand that modifying a list inside a function affects the original.
- Create a copy of a list to avoid unintended modifications.

Concept Explanation

Since lists are **mutable**, modifying them inside a function changes the original:

```
def add_score(score_list, new_score):
    score_list.append(new_score)

scores = [82, 91, 76, 88]
add_score(scores, 95)
print(scores)   # [82, 91, 76, 88, 95]  - the original list changed
```

Expected output:

```
[82, 91, 76, 88, 95]
```

To prevent modification, pass a copy:

```
add_score(scores[:], 100)   # Original unchanged
```

Worked Example

Example: Processing the Shared Inventory with a Function

```
def inventory_value(inventory):
    total = 0
    for details in inventory.values():
        total += details["quantity"] * details["unit_price"]
    return total

inventory = {
    "Notebook": {"quantity": 3, "unit_price": 2.50},
    "Pencil": {"quantity": 10, "unit_price": 0.50},
    "Folder": {"quantity": 4, "unit_price": 1.25},
}
print(f"Total inventory value: {inventory_value(inventory):.2f}")
```

Expected Behavior or Output

Expected output:

```
Total inventory value: 17.50
```

Common Mistakes

1. Expecting a function to receive its own copy of a list. It receives the same list and can change it.
2. Changing a list inside a function that is meant only to read it.
3. Making a copy with `copy = original` instead of `original[:]` or `list(original)`.

Activities

1. Predict the printed lists in the example before running it.
2. Write `scale_prices(inventory, factor)` that changes the prices in place and show the effect.
3. Rewrite it so it returns a new inventory and leaves the original unchanged.
4. Explain the difference between passing a number and passing a list to a function.

Accessibility Practice

State aloud before each call whether the function changes its argument. Print the list before and after the call so a screen reader user hears the effect, and use names such as `add_score` that say a change will happen.

Skills Checklist

- ☐ I can pass a list or dictionary as an argument to a function.
- ☐ I understand that modifying a list inside a function affects the original.
- ☐ I can create a copy of a list to avoid unintended modifications.

Lesson 6.5: Arbitrary Number of Arguments

Reading References

- Matthes: Chapter 8 (Functions), the section on passing an arbitrary number of arguments

Learning Objectives

- Define a function using `*args` for arbitrary positional arguments.
- Define a function using `**kwargs` for arbitrary keyword arguments.

Concept Explanation

`*args` — Arbitrary Positional Arguments

```
def make_pizza(size, *toppings):
    print(f"Making a {size}-inch pizza with:")
    for topping in toppings:
        print(f"  - {topping}")

make_pizza(12, 'pepperoni')
make_pizza(16, 'mushrooms', 'green peppers', 'extra cheese')
```

Example: Variable Number of Scores

```
def average(*scores):
    return sum(scores) / len(scores)

print(average(82, 91, 76, 88))   # any number of arguments
print(average(90, 80))
```

Expected output:

```
84.25
85.0
```

`**kwargs` — Arbitrary Keyword Arguments

Worked Example

```
def build_profile(first, last, **user_info):
    user_info['first_name'] = first
    user_info['last_name'] = last
    return user_info

profile = build_profile('albert', 'einstein',
                        location='princeton',
                        field='physics')
print(profile)
```

Expected Behavior or Output

Output of example 1:

```
{'location': 'princeton', 'field': 'physics', 'first_name': 'albert', 'last_name': 'einstein
```

Common Mistakes

1. Calling `average()` with no arguments, which divides by zero. Check the count first.
2. Putting `*args` before a normal parameter and being unable to pass the normal one by position.
3. Forgetting that `*args` is a tuple, not a list, so it cannot be changed.

Activities

1. Predict the result of `average(82, 91, 76, 88)` and `average(90, 80)`.
2. Write `maximum(*numbers)` that returns the largest argument.
3. Write `join_words(separator, *words)` that joins any number of words.
4. Explain what the `*` does in a call such as `average(*scores)`.

Accessibility Practice

Read variadic signatures aloud: 'define average, open parenthesis, star scores, close parenthesis'. Make sure your screen reader announces the star, since it changes the meaning of the parameter.

Skills Checklist

- ☐ I can define a function using `*args` for arbitrary positional arguments.
- ☐ I can define a function using `**kwargs` for arbitrary keyword arguments.

Canonical Shared Dataset — Inventory, Collections, and Functions

Use the same inventory records for list, dictionary, iteration, and function examples.

```
inventory = [
    {"name": "Notebook", "quantity": 3, "unit_price": 2.50},
    {"name": "Pencil", "quantity": 10, "unit_price": 0.50},
    {"name": "Folder", "quantity": 4, "unit_price": 1.25},
]

def item_value(item):
    return item["quantity"] * item["unit_price"]

for item in inventory:
    print(item["name"], item["quantity"], f"${item['unit_price']:.2f}",
          f"${item_value(item):.2f}")

grand_total = sum(item_value(item) for item in inventory)
print(f"Inventory total: ${grand_total:.2f}")
```

Expected output:

```
Notebook 3 $2.50 $7.50
Pencil 10 $0.50 $5.00
Folder 4 $1.25 $5.00
Inventory total: $17.50
```

For sequence exercises, use the canonical scores `[82, 91, 76, 88]`. For immutable-sequence exercises, use coordinates `(3, 4)`. Keep language-specific collection types idiomatic while preserving record values and calculation results.

13.9 Unit 7: Classes and Object-Oriented Programming

Overview

Compatibility: All examples target Python 3.12 and newer; they avoid syntax and standard-library APIs introduced after Python 3.12.

This unit introduces the fundamental concepts of Object-Oriented Programming (OOP) in Python, focusing on defining and using classes, attributes, methods, and inheritance.

Objectives:

- Define and instantiate classes.
- Work with attributes and methods.
- Understand inheritance and composition.

Lesson 7.1: Why Learn About Classes?

Reading References

- Zelle: Chapter 4, pages 65–67; Chapter 10, pages 247–248
- Matthes: Chapter 9, page 157

Learning Objectives

- Explain the basic idea of Object-Oriented Programming.
- Describe a class as a blueprint and an object as an instance.

Concept Explanation

A *class* describes what a kind of thing knows (its *attributes*) and what it can do (its *methods*). An *object* is one actual thing made from a class. Bundling data with the functions that work on it is called **encapsulation**, and it keeps related code together so a change in one place does not break unrelated code.

Classes as Blueprints A **class** is a blueprint or template for creating objects. An **object** is a concrete instance created from that blueprint.

Analogy: An "Item" class defines that every inventory item has a `name`, a `quantity`, and a `unit_price` (attributes) and can report its `total_value()` (a method). A Notebook and a Pencil are two separate **objects** of the same "Item" class. This course models the same inventory item in every language.

Key OOP Concepts

- **Encapsulation** — Bundling data and methods into a single unit.
- **Abstraction** — Hiding complexity from the outside.
- **Inheritance** — New class reuses functionality of existing class.
- **Polymorphism** — Same method name behaves differently for different classes.

Worked Example

A student profile kept as loose variables is easy to mix up. As an object, the same data and the operations on it travel together:

```
class Item:
    def __init__(self, name, quantity, unit_price):
        self.name = name
        self.quantity = quantity
        self.unit_price = unit_price

    def total_value(self):
        return self.quantity * self.unit_price


notebook = Item("Notebook", 3, 2.50)
print(notebook.name, notebook.total_value())
```

Expected Behavior or Output

Output of example 1:

```
Notebook 7.5
```

Common Mistakes

1. Treating a class as one single thing. A class is a blueprint, and each call such as `Item(...)` makes a separate object.
2. Putting unrelated data into one class so that it has no clear purpose.
3. Skipping the design step and creating classes before knowing what data and operations are needed.

Activities

1. List the attributes and methods of a `Student` class for the shared profile (Avery, 16, 1.72, Computer Science).
2. Describe in two sentences how a class differs from an object, using `Item` and `notebook`.
3. Name three real things and for each write one attribute and one method.

4. Explain what encapsulation means and why it helps when a program grows.

Accessibility Practice

Describe a class as a list read in order: its name, then its attributes, then its methods. Reading a class definition aloud works best by saying 'class Item, colon, define init, open parenthesis, self, comma, name...' and listening for the colons and indentation that show structure.

Skills Checklist

☐ I can explain the basic idea of Object-Oriented Programming.
☐ I can describe a class as a blueprint and an object as an instance.

Lesson 7.2: Creating and Using a Class

Reading References

- Zelle: Chapter 10, pages 248–254
- Matthes: Chapter 9, pages 158–162

Learning Objectives

- Define a basic Python class.
- Define the `__init__` method within a class.
- Understand the purpose of the `self` parameter.
- Create an instance (object) of a class.

Concept Explanation

Class Syntax The `__init__()` method runs automatically when a new object is created. `self` refers to the specific instance being created. Each of the three objects above has the same fields (`name`, `quantity`, `unit_price`) with different values. Keeping the attributes inside the object, and changing them only through methods, is called **encapsulation**.

Worked Example

```
class Item:
    """An inventory item with a name, a quantity, and a unit price."""

    def __init__(self, name, quantity, unit_price):
        """Initialize the item's attributes."""
        self.name = name
        self.quantity = quantity
```

```
        self.unit_price = unit_price


notebook = Item("Notebook", 3, 2.50)
pencil = Item("Pencil", 10, 0.50)
folder = Item("Folder", 4, 1.25)

print(f"{notebook.name}: {notebook.quantity} at {notebook.unit_price}")
print(f"{pencil.name}: {pencil.quantity} at {pencil.unit_price}")
print(f"{folder.name}: {folder.quantity} at {folder.unit_price}")
```

Expected Behavior or Output

Expected output:

```
Notebook: 3 at 2.5
Pencil: 10 at 0.5
Folder: 4 at 1.25
```

Common Mistakes

1. Forgetting `self` as the first parameter of `__init__`, which raises a `TypeError` when an object is created.
2. Writing `name = name` instead of `self.name = name`, so the value is lost when `__init__` ends.
3. Writing `__init__` with one underscore on each side, so it is not recognized.

Activities

1. Create a fourth item with your own values and print its fields.
2. Add a check in `__init__` that rejects a negative quantity by printing a message.
3. Create two items with the same name and different quantities and show that changing one does not change the other.
4. Explain what `self` refers to in your own words.

Accessibility Practice

Say the double underscores aloud ('dunder init') and confirm that your screen reader announces them, because `_init_` with a single underscore fails silently. Print each object on its own line with its name, so the output can be matched to the objects by ear.

Skills Checklist

- ☐ I can define a basic Python class.
- ☐ I can define the `__init__` method within a class.
- ☐ I understand the purpose of the `self` parameter.

☐ I can create an instance (object) of a class.

Lesson 7.3: Working with Attributes and Methods

Reading References

- Matthes: Chapter 9, pages 162–166

Learning Objectives

- Access an object's attributes using dot notation.
- Define and call methods within a class.
- Use a method to change an object's attribute.

Concept Explanation

Calling Methods

Worked Example

```
class Item:
    """An inventory item with a name, a quantity, and a unit price."""

    def __init__(self, name, quantity, unit_price):
        """Initialize the item's attributes."""
        self.name = name
        self.quantity = quantity
        self.unit_price = unit_price

    def total_value(self):
        """Return quantity multiplied by unit price."""
        return self.quantity * self.unit_price

    def is_valid(self):
        """An item is valid if it has a name and non-negative numbers."""
        return bool(self.name) and self.quantity >= 0 and self.unit_price >= 0

    def restock(self, amount):
        """Add stock; reject a negative amount."""
        if amount < 0:
            print("Cannot restock a negative amount.")
            return
        self.quantity += amount

    def describe(self):
        print(f"{self.name}: {self.quantity} at {self.unit_price:.2f} each, "
```

```
                f"value {self.total_value():.2f}")


notebook = Item("Notebook", 3, 2.50)
notebook.describe()
notebook.restock(2)          # method-based modification
notebook.describe()
notebook.quantity = 4        # direct attribute modification
notebook.describe()
notebook.restock(-1)         # validation message
print(notebook.is_valid())
```

Expected Behavior or Output

Expected output:

```
Notebook: 3 at 2.50 each, value 7.50
Notebook: 5 at 2.50 each, value 12.50
Notebook: 4 at 2.50 each, value 10.00
Cannot restock a negative amount.
True
```

Common Mistakes

1. Calling a method without parentheses (`notebook.describe`), which refers to the method but does not run it.
2. Changing an attribute directly and skipping the validation that a method would have performed.
3. Forgetting `self.` when a method uses an attribute, which raises a `NameError`.

Activities

1. Predict the output of each call in the example before running it.
2. Add a method `discount(rate)` that returns the value after a discount without changing the item.
3. Call `restock(-1)` and read the message. Then change the method to reject zero as well.
4. Explain the difference between changing `quantity` directly and calling `restock`.

Accessibility Practice

Name methods after actions (`restock`, `describe`) so the call reads as a sentence. Print one line per method call, so a screen reader announces the effect of every step in order.

Skills Checklist

- ☐ I can access an object's attributes using dot notation.
- ☐ I can define and call methods within a class.
- ☐ I can use a method to change an object's attribute.

Lesson 7.4: Inheritance

Reading References

- Zelle: Chapter 12, pages 351–352
- Matthes: Chapter 9, pages 167–170

Learning Objectives

- Define a child class that inherits from a parent class.
- Use `super().__init__()` to properly initialize inherited attributes.
- Add new attributes and methods to a child class.
- Override a parent method in a child class.

Concept Explanation

Creating a Child Class

`super()` Function `super().__init__()` calls the parent class's `__init__()` method, ensuring inherited attributes are properly initialized. In the example, `DiscountedItem` overrides `total_value()`; the inherited `describe()` automatically uses the new version. This is **polymorphism**: the same method name behaves differently for different classes.

Worked Example

```
class Item:
    """An inventory item with a name, a quantity, and a unit price."""

    def __init__(self, name, quantity, unit_price):
        """Initialize the item's attributes."""
        self.name = name
        self.quantity = quantity
        self.unit_price = unit_price

    def total_value(self):
        return self.quantity * self.unit_price

    def describe(self):
        print(f"{self.name}: {self.quantity} at {self.unit_price:.2f} each, "
```

```
            f"value {self.total_value():.2f}")


class DiscountedItem(Item):
    """An item sold at a discount; inherits from Item."""

    def __init__(self, name, quantity, unit_price, discount_rate):
        super().__init__(name, quantity, unit_price)   # Call parent's __init__
        self.discount_rate = discount_rate

    def total_value(self):                              # Override
        return super().total_value() * (1 - self.discount_rate)


folder = DiscountedItem("Folder", 4, 1.25, 0.20)
folder.describe()        # inherited describe() calls the overridden total_value()
print(isinstance(folder, Item))
```

Expected Behavior or Output

Expected output:

```
Folder: 4 at 1.25 each, value 4.00
True
```

Common Mistakes

1. Forgetting to call `super().__init__()`, so the parent's attributes are never created.
2. Overriding a method but changing what it returns, so code that uses the parent version breaks.
3. Using inheritance for a "has a" relationship; use composition (Lesson 7.5) instead.

Activities

1. Predict the printed value and the final `True` of the example before running it.
2. Add a subclass `PerishableItem` with an `expires` attribute and override `describe`.
3. Explain why `describe` prints the discounted value without being redefined.
4. List the lookup order Python uses to find `describe` for a `DiscountedItem`.

Accessibility Practice

Describe inheritance as an ordered list: the object's own class first, then its parent. Speaking the chain aloud ('DiscountedItem, then Item') is clearer than a diagram.

Skills Checklist

- ☐ I can define a child class that inherits from a parent class.
- ☐ I can use `super().__init__()` to properly initialize inherited attributes.
- ☐ I can add new attributes and methods to a child class.
- ☐ I can override a parent method in a child class.

Lesson 7.5: Instances as Attributes (Composition)

Reading References

- Matthes: Chapter 9 (Classes), the section on instances as attributes
- Zelle: Chapter 10 (Defining Classes)

Learning Objectives

- Explain the concept of composition in OOP.
- Create an instance of one class and assign it as an attribute of another.
- Access attributes and methods of a nested object.

Concept Explanation

Composition — "Has-A" Relationship Instead of inheriting, a class **contains an instance of another class** as an attribute.

Worked Example

```
class Item:
    """An inventory item with a name, a quantity, and a unit price."""

    def __init__(self, name, quantity, unit_price):
        """Initialize the item's attributes."""
        self.name = name
        self.quantity = quantity
        self.unit_price = unit_price

    def total_value(self):
        return self.quantity * self.unit_price
```

```
class Inventory:
    """An Inventory HAS-A list of Item objects (composition)."""

    def __init__(self):
        self.items = []

    def add(self, item):
        self.items.append(item)

    def total_value(self):
        return sum(item.total_value() for item in self.items)

    def report(self):
        for item in self.items:
            print(f"{item.name}: {item.quantity}")
        print(f"Total value: {self.total_value():.2f}")


inventory = Inventory()
inventory.add(Item("Notebook", 3, 2.50))
inventory.add(Item("Pencil", 10, 0.50))
inventory.add(Item("Folder", 4, 1.25))
inventory.report()
```

Expected Behavior or Output

Expected output:

```
Notebook: 3
Pencil: 10
Folder: 4
Total value: 17.50
```

Common Mistakes

1. Using inheritance where a ”has a” relationship is meant.
2. Letting the container reach into each item's attributes instead of calling the item's own methods.
3. Creating the list in a default argument or at class level, so every `Inventory` shares one list.

Activities

1. Predict the output of the example.
2. Add `Inventory.find(name)` that returns the item with that name or `None`.
3. Add `Inventory.remove(name)` and test it.

4. Explain the difference between ”is a” and ”has a” with two examples.

Accessibility Practice

Report the inventory as one labelled line per item and a final total line, so a screen reader reads a complete summary in order.

Skills Checklist

☐ I can explain the concept of composition in OOP.
☐ I can create an instance of one class and assign it as an attribute of another.
☐ I can access attributes and methods of a nested object.

13.10 Unit 8: Files and Exceptions

Overview

This unit covers interacting with text files for reading and writing data, handling common errors gracefully using exceptions, and storing data using JSON.

Objectives:

- Read from and write to files.
- Handle exceptions with `try-except` blocks.
- Store and load data using JSON.

Lesson 8.1: Reading from a File

Reading References

- Zelle: Chapter 5, pages 125–127
- Matthes: Chapter 10, pages 184–189

Learning Objectives

- Open a file for reading using `with open()`.
- Read the entire content of a file into a string.
- Read a file line by line using a `for` loop.
- Remove whitespace/newlines using `.strip()`.

Concept Explanation

Reading Files Create a file named `notes.txt` in your project folder containing exactly these two lines:

```
Practice Python.
Review lists and functions.
```

The `with` statement **automatically closes the file** even if an error occurs.

File Modes

Mode	Description
`'r'`	Read (default)
`'w'`	Write (creates new or overwrites)
`'a'`	Append (adds to end)

Worked Example

```
# Reading entire file
with open('notes.txt', 'r') as file_object:
    contents = file_object.read()
print(contents)

# Reading line by line
with open('notes.txt', 'r') as file_object:
    for line in file_object:
        print(line.strip())

# Reading all lines into a list
with open('notes.txt', 'r') as file_object:
    lines = file_object.readlines()
print(len(lines))
```

Expected Behavior or Output

Expected output:

```
Practice Python.
Review lists and functions.

Practice Python.
Review lists and functions.
2
```

Common Mistakes

1. Opening a file that does not exist without handling the `FileNotFoundError`.
2. Forgetting that a relative path starts from the working directory, not from the program's folder.
3. Forgetting that `readlines()` keeps the newline at the end of each line.

Activities

1. Run the example and describe each block of output.
2. Add a third line to `notes.txt` and predict the new line count.
3. Print each line with its line number.
4. Open a file that does not exist and print a friendly message that includes the file name.

Accessibility Practice

Read the output aloud in order. Make any error message a complete sentence that includes the file name, so it is clear without seeing the screen.

Skills Checklist

- ☐ I can open a file for reading using `with open()`.
- ☐ I can read the entire content of a file into a string.
- ☐ I can read a file line by line using a `for` loop.
- ☐ I can remove whitespace/newlines using `.strip()`.

Lesson 8.2: Writing to a File

Reading References

- Zelle: Chapter 5, pages 127–128
- Matthes: Chapter 10, pages 190–192

Learning Objectives

- Open a file for writing using `"w"` mode.
- Open a file for appending using `"a"` mode.
- Write a string to a file.
- Add newlines to a file using `\n`.

Concept Explanation

Important: The `.write()` method does not add newlines automatically. Use `\n`.

Worked Example

```
# Write mode (overwrites existing content)
with open('notes.txt', 'w') as f:
    f.write("Practice Python.\n")

# Append mode (adds to end)
with open('notes.txt', 'a') as f:
```

```
    f.write("Review lists and functions.\n")

with open('notes.txt', 'r') as f:
    print(f.read())
```

Expected Behavior or Output

Expected output:

```
Practice Python.
Review lists and functions.
```

Common Mistakes

1. Using mode `'w'` when `'a'` was intended, which erases earlier content.
2. Forgetting `\n`, so all written lines run together.
3. Not using `with`, so the file may stay open if an error occurs.

Activities

1. Predict the output, then run.
2. Change the second open to mode `'w'` and explain the difference.
3. Append a third line with the current number of lines.
4. Write the shared scores, one per line, then read them back and total them.

Accessibility Practice

Always read a written file back in the terminal to confirm it, using line navigation, so you do not depend on assumptions about what was saved.

Skills Checklist

☐ I can open a file for writing using `"w"` mode.
☐ I can open a file for appending using `"a"` mode.
☐ I can write a string to a file.
☐ I can add newlines to a file using `\n`.

Lesson 8.3: Exceptions — Handling Errors

Reading References

- Zelle: Chapter 7, pages 177–179
- Matthes: Chapter 10, pages 192–199

Learning Objectives

- Use a `try-except` block to handle errors.
- Name common exception types and what causes them.
- Catch specific exception types.

Concept Explanation

`try-except` Block

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Error: You cannot divide by zero!")
print("Program continues after error handling.")
```

Common Exception Types

Exception	When it occurs
`ValueError`	Invalid value (e.g., `int("abc")`)
`ZeroDivisionError`	Division by zero
`FileNotFoundError`	File doesn't exist
`IndexError`	List index out of range
`KeyError`	Dictionary key doesn't exist
`TypeError`	Wrong type (e.g., `"5" + 5`)

Worked Example

Example 8.3.1: Handling ZeroDivisionError

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Error: Cannot divide by zero!")
```

Example 8.3.3: Missing File and Malformed Data

```
try:
    with open("missing_notes.txt", "r") as f:
        print(f.read())
except FileNotFoundError:
    print("Error: missing_notes.txt was not found.")

try:
    score = int("abc")
except ValueError:
    print("Error: 'abc' is not a valid score.")
```

Example 8.3.2: Handling ValueError

```
try:
    num = int(input("Enter a number: "))
    print(f"You entered: {num}")
except ValueError:
    print("Error: That was not a valid number.")
```

Expected Behavior or Output

Expected output:

```
Error: missing_notes.txt was not found.
Error: 'abc' is not a valid score.
```

Common Mistakes

1. Using a bare `except:` that hides every error, including mistakes in your own code.
2. Putting too much code inside `try`, so it is unclear which line failed.
3. Catching an error and printing nothing, so the user does not know what happened.

Activities

1. Predict each printed line of the missing-file example before running it.
2. Wrap `int(input(...))` so that invalid input prints a clear message and asks again.
3. Catch `ZeroDivisionError` in a function that computes an average of an empty list.
4. Explain when to catch an exception and when to let it stop the program.

Accessibility Practice

Write error messages as complete sentences that name the problem and the next step, such as 'Enter a whole number from 0 to 100.' Read the exception name aloud when you see it in a traceback, then jump to the last line of the traceback.

Skills Checklist

- ☐ I can use a `try-except` block to handle errors.
- ☐ I can name common exception types and what causes them.
- ☐ I can catch specific exception types.

Lesson 8.4: The `else` and `finally` Blocks

Reading References

- Matthes: Chapter 10 (Files and Exceptions), the sections on the `else` block
- Zelle: Chapter 5 (Sequences: Strings, Lists, and Files)

Learning Objectives

- Use an `else` block for code that runs only on success.
- Use a `finally` block for code that always runs.

Concept Explanation

`else` Block — Runs only on success

```
try:
    num = int(input("Enter a number: "))
except ValueError:
    print("That's not a number.")
else:
    print(f"You entered {num}.")    # Only runs if no exception
```

`finally` Block — Always runs (cleanup)

Worked Example

```
# For file I/O, prefer a with statement. This example demonstrates
# finally for cleanup of a resource managed manually.
file_handle = None
try:
    file_handle = open('cleanup_demo.txt', 'w')
    file_handle.write("Some data.")
except TypeError:
    print("A type error occurred.")
finally:
    if file_handle:
        file_handle.close()     # Always closes the file
        print("File closed.")
```

Expected Behavior or Output

Output of example 1:

```
File closed.
```

Common Mistakes

1. Putting code in `else` that can itself fail, so its errors are not handled.
2. Relying on `finally` to return a value, which can hide an earlier error.
3. Closing a file manually and forgetting to do so when an error occurs; prefer `with`.

Activities

1. Predict the output of both runs of the example.
2. Add a `finally` block that prints a message even when an error occurs.
3. Rewrite the example with `with` and explain why it is shorter and safer.
4. Explain in two sentences what `else` adds to `try` and `except`.

Accessibility Practice

Print a status line in each block (`try`, `except`, `else`, `finally`), so a screen reader announces which path ran. Short, labelled lines make the control flow audible.

Skills Checklist

☐ I can use an `else` block for code that runs only on success.
☐ I can use a `finally` block for code that always runs.

Lesson 8.5: Storing Data with JSON

Reading References

- Matthes: Chapter 10, pages 200–204

Learning Objectives

- Explain what JSON is and why it's used.
- Use `json.dump()` to save Python data to a JSON file.
- Use `json.load()` to load JSON data from a file.
- Save and load lists of dictionaries using JSON.

Concept Explanation

The `json` Module JSON (JavaScript Object Notation) is a lightweight format for storing and exchanging data. Python's `json` module converts between Python objects and JSON.

```
import json

# Save data
```

```
task = {
    "id": 1,
    "title": "Practice Python",
    "description": "Complete the shared dataset exercises",
    "done": False,
    "created_at": "2025-01-15T09:00:00Z",
}

with open("tasks.json", "w", encoding="utf-8") as file:
    json.dump([task], file, indent=4)

# Load and verify all five canonical fields.
try:
    with open("tasks.json", "r", encoding="utf-8") as file:
        loaded_tasks = json.load(file)
    loaded_task = loaded_tasks[0]
    required_fields = {"id", "title", "description", "done", "created_at"}
    if set(loaded_task) != required_fields:
        raise ValueError("Task record does not match the five-field schema")
    print(loaded_task["id"], loaded_task["title"], loaded_task["done"])
except FileNotFoundError:
    print("Task file not found.")
except (json.JSONDecodeError, ValueError) as error:
    print(f"Invalid task data: {error}")
```

JSON Python Mapping

JSON	Python
Object `{}`	Dictionary `{}`
Array `[]`	List `[]`
String	`str`
Number	`int` or `float`
Boolean	`bool`
null	`None`

Worked Example

```
import json

tasks = [
    {"id": 1, "title": "Study programming", "description": "Finish Unit 8",
     "done": False, "created_at": "2026-01-15T09:00:00"},
    {"id": 2, "title": "Review scores", "description": "Check the four scores",
     "done": True, "created_at": "2026-01-15T09:30:00"},
]
```

```
with open('tasks.json', 'w') as f:
    json.dump(tasks, f, indent=4)

try:
    with open('tasks.json', 'r') as f:
        loaded = json.load(f)
    for task in loaded:
        status = "done" if task["done"] else "not done"
        print(f"{task['id']}: {task['title']} ({status})")
except FileNotFoundError:
    print("Tasks file not found.")
except json.JSONDecodeError:
    print("Error: Could not decode tasks file.")
```

Expected Behavior or Output

Expected output:

```
1: Study programming (not done)
2: Review scores (done)
```

Common Mistakes

1. Calling `json.dump` with a file opened in read mode.
2. Trying to save an object that JSON cannot represent, such as a custom class instance.
3. Not handling `json.JSONDecodeError`, so a damaged file crashes the program.

Activities

1. Run the example and read the saved JSON text from `tasks.json`.
2. Edit `tasks.json` by hand to remove a closing bracket and run the program again.
3. Add a third task and confirm that it is saved and loaded.
4. Explain why `True` in Python becomes `true` in the JSON file.

Accessibility Practice

Open `tasks.json` in the terminal and read it as text. Because `indent=4` puts one field per line, the file reads line by line with a screen reader. Practise moving through it by word or character.

Skills Checklist

- ☐ I can explain what JSON is and why it's used.
- ☐ I can use `json.dump()` to save Python data to a JSON file.

☐ I can use `json.load()` to load JSON data from a file.
☐ I can save and load lists of dictionaries using JSON.

13.11 Unit 9: Testing and Data Analysis

Overview

This unit introduces testing code for correctness and explores basic text-based data analysis.

Lesson 9.1: Testing a Function

Reading References

- Matthes: Chapter 11, pages 209–217

Learning Objectives

- Create a basic test class using `unittest.TestCase`.
- Define test methods within a test class.
- Use `assertEqual()` to check for expected results.
- Run unit tests from the terminal.

Concept Explanation

Unit Testing with `unittest`

```
# my_functions.py
def add(a, b):
    return a + b


def calculate_tip(bill, rate):
    return bill * rate


def classify(score):
    if score >= 90:
        return "A"
    elif score >= 80:
        return "B"
    elif score >= 70:
        return "C"
    elif score >= 60:
```

```
        return "D"
    return "F"
```

```
# test_my_functions.py
import unittest
from my_functions import add, calculate_tip, classify


class TestFunctions(unittest.TestCase):
    def test_add(self):
        self.assertEqual(add(5, 3), 8)
        self.assertEqual(add(-5, -3), -8)
        self.assertEqual(add(10, -4), 6)

    def test_tip(self):
        # Decimal results are compared with a tolerance, not exact equality.
        self.assertAlmostEqual(calculate_tip(24.5, 0.15), 3.675)
        self.assertEqual(calculate_tip(0, 0.15), 0)

    def test_classify_boundaries(self):
        self.assertEqual(classify(90), "A")
        self.assertEqual(classify(89), "B")
        self.assertEqual(classify(60), "D")
        self.assertEqual(classify(59), "F")


if __name__ == '__main__':
    unittest.main()
```

Run the tests from the terminal with `python test_my_functions.py` or `python -m unittest`. Expected output (the timing line will differ):

```
...
----------------------------------------------------------------------
Ran 3 tests in 0.001s

OK
```

The three tests check the same inputs and expected results in every language track: `add(5, 3)` is 8, the tip for a bill of 24.50 at 0.15 is 3.675, and the grade boundaries 90, 89, 60, and 59 give A, B, D, and F.

Worked Example

```
# my_functions.py
def add(a, b):
    return a + b
```

```
def calculate_tip(bill, rate):
    return bill * rate


def classify(score):
    if score >= 90:
        return "A"
    elif score >= 80:
        return "B"
    elif score >= 70:
        return "C"
    elif score >= 60:
        return "D"
    return "F"
```

```
# test_my_functions.py
import unittest
from my_functions import add, calculate_tip, classify


class TestFunctions(unittest.TestCase):
    def test_add(self):
        self.assertEqual(add(5, 3), 8)
        self.assertEqual(add(-5, -3), -8)
        self.assertEqual(add(10, -4), 6)

    def test_tip(self):
        # Decimal results are compared with a tolerance, not exact equality.
        self.assertAlmostEqual(calculate_tip(24.5, 0.15), 3.675)
        self.assertEqual(calculate_tip(0, 0.15), 0)

    def test_classify_boundaries(self):
        self.assertEqual(classify(90), "A")
        self.assertEqual(classify(89), "B")
        self.assertEqual(classify(60), "D")
        self.assertEqual(classify(59), "F")


if __name__ == '__main__':
    unittest.main()
```

Run the tests from the terminal with `python test_my_functions.py` or `python -m unittest`.

Expected Behavior or Output

Expected output (the timing line differs from run to run):

```
...
----------------------------------------------------------------------
Ran 3 tests in 0.001s

OK
```

The three tests check the same inputs and expected results in every language track: `add(5, 3)` is 8, the tip for a bill of 24.50 at 0.15 is 3.675, and the grade boundaries 90, 89, 60, and 59 give A, B, D, and F.

Common Mistakes

1. Comparing decimal results with `assertEqual`. Use `assertAlmostEqual`, because floating-point results are approximations.
2. Naming a test method without the `test_` prefix, so `unittest` never runs it.
3. Testing only typical values and skipping the boundaries 90, 89, 60, and 59.

Activities

1. Change one expected value so a test fails, read the failure message, and then restore it.
2. Add tests for the grades at 80, 79, 70, and 69.
3. Write the tests for `is_passing(score)` before you write the function.
4. Explain why the tip test uses a tolerance.

Accessibility Practice

A passing run prints a short summary (`Ran 3 tests ... OK`) that is quick to read. A failing run prints the test name and the expected and actual values on separate lines; jump to those lines first. Name each test as a sentence that completes 'This test checks that...'.

Skills Checklist

- ☐ I can create a basic test class using `unittest.TestCase`.
- ☐ I can define test methods within a test class.
- ☐ I can use `assertEqual()` to check for expected results.
- ☐ I can run unit tests from the terminal.

Lesson 9.2: Testing a Class

Reading References

- Matthes: Chapter 11 (Testing Your Code), the sections on testing a class

Learning Objectives

- Write tests for a Python class.
- Use `setUp()` to prepare my tests.

Concept Explanation

The `setUp()` Method `setUp()` runs **before every test method**, providing a fresh instance each time:

```
# item.py  (the Item class from Unit 7, with a restock method)
class Item:
    def __init__(self, name, quantity, unit_price):
        self.name = name
        self.quantity = quantity
        self.unit_price = unit_price

    def total_value(self):
        return self.quantity * self.unit_price

    def restock(self, amount):
        if amount < 0:
            raise ValueError("amount must not be negative")
        self.quantity += amount
```

```
# test_item.py
import unittest
from item import Item


class TestItem(unittest.TestCase):
    def setUp(self):
        """Create a fresh Notebook for every test method."""
        self.item = Item("Notebook", 3, 2.50)

    def test_total_value(self):
        self.assertAlmostEqual(self.item.total_value(), 7.5)

    def test_restock(self):
        self.item.restock(2)
        self.assertEqual(self.item.quantity, 5)
```

```
    def test_restock_rejects_negative_amount(self):
        with self.assertRaises(ValueError):
            self.item.restock(-1)


if __name__ == '__main__':
    unittest.main()
```

Expected output (the timing line will differ):

```
...
----------------------------------------------------------------------
Ran 3 tests in 0.001s

OK
```

Because `setUp()` runs before each test, `test_restock` changing the quantity to 5 does not affect `test_total_value`, which still sees 3 notebooks.

Worked Example

```
# item.py  (the Item class from Unit 7, with a restock method)
class Item:
    def __init__(self, name, quantity, unit_price):
        self.name = name
        self.quantity = quantity
        self.unit_price = unit_price

    def total_value(self):
        return self.quantity * self.unit_price

    def restock(self, amount):
        if amount < 0:
            raise ValueError("amount must not be negative")
        self.quantity += amount
```

```
# test_item.py
import unittest
from item import Item


class TestItem(unittest.TestCase):
    def setUp(self):
        """Create a fresh Notebook for every test method."""
        self.item = Item("Notebook", 3, 2.50)

    def test_total_value(self):
        self.assertAlmostEqual(self.item.total_value(), 7.5)
```

```
    def test_restock(self):
        self.item.restock(2)
        self.assertEqual(self.item.quantity, 5)

    def test_restock_rejects_negative_amount(self):
        with self.assertRaises(ValueError):
            self.item.restock(-1)


if __name__ == '__main__':
    unittest.main()
```

Expected Behavior or Output

Expected output (the timing line differs):

```
...
----------------------------------------------------------------------
Ran 3 tests in 0.001s

OK
```

Because `setUp()` runs before each test, `test_restock` changing the quantity to 5 does not affect `test_total_value`, which still sees 3 notebooks.

Common Mistakes

1. Sharing one object between tests so that an earlier test changes the later ones; use `setUp`.
2. Testing only the successful path and never the error path.
3. Forgetting `self.` inside a test method, so the fixture object is not found.

Activities

1. Add a test that `restock(0)` leaves the quantity unchanged.
2. Add a test for the `is_valid` method from Unit 7.
3. Break `restock` on purpose and read the failure report.
4. Explain why `setUp` creates a new item for each test.

Accessibility Practice

Name each test as a short sentence that says what behavior it checks, so the PASS or FAIL report reads as a clear statement. Read the summary line first, and open the failure details only when it says FAILED.

Skills Checklist

- ☐ I can write tests for a Python class.
- ☐ I can use `setUp()` to prepare my tests.

Lesson 9.3: Introduction to Random Numbers

Reading References

- Matthes: Chapter 9 (Classes), the section on the `random` module
- Zelle: Chapter 9 (Simulation and Design)

Learning Objectives

- Import the `random` module.
- Generate a random integer within a range.
- Select a random item from a list.

Concept Explanation

Without the `random.seed(...)` line the values change on every run. Use a seed when you need to test or reproduce a result.

Worked Example

```
import random

random.seed(42)   # a fixed seed makes the "random" results repeatable

die_roll = random.randint(1, 6)          # random integer (inclusive)
print(f"You rolled: {die_roll}")

rand_float = random.random()             # random float from 0.0 up to 1.0
print(f"Random decimal: {rand_float:.4f}")

scores = [82, 91, 76, 88]
chosen = random.choice(scores)           # random choice from a list
print(f"Chosen score: {chosen}")
```

Expected Behavior or Output

Expected output with `random.seed(42)` on Python 3.12:

```
You rolled: 6
Random decimal: 0.1113
Chosen score: 76
```

Common Mistakes

1. Testing for specific random values that change between Python versions; test properties such as the range instead.
2. Calling `random.seed()` in every loop pass, which repeats the same value.
3. Forgetting that `randint(1, 6)` includes both ends, while `randrange(1, 6)` does not include 6.

Activities

1. Run the example three times with the seed line and then three times without it; compare.
2. Change the seed and check that the ranges still hold.
3. Simulate 600 rolls and count how many of each face appeared (the counts will differ).
4. Explain why a test should check a property rather than a specific random number.

Accessibility Practice

When output is random, print yes or no results about properties (`All rolls between 1 and 6: True`) so the screen reader announces the same result on every run and does not read unpredictable values.

Skills Checklist

☐ I can import the `random` module.
☐ I can generate a random integer within a range.
☐ I can select a random item from a list.

Lesson 9.4: Analyzing Text Data

Reading References

- Matthes: Chapter 6 (Dictionaries), the section on counting with dictionaries
- Zelle: Chapter 5 (Sequences: Strings, Lists, and Files)

Learning Objectives

- Read text content from a file.
- Split a string of text into individual words.
- Use a dictionary to count the frequency of words.

Concept Explanation

Worked Example

```
sentence = "A student studies programming. Programming develops problem-solving skills."
words = sentence.lower().replace(".", "").split()

word_frequencies = {}
for word in words:
    word_frequencies[word] = word_frequencies.get(word, 0) + 1

print("Word count:", len(words))
print("Word Frequencies:")
for word, count in word_frequencies.items():
    print(f"'{word}': {count}")

letter_counts = {}
for ch in text.lower():
    if ch.isalpha():
        letter_counts[ch] = letter_counts.get(ch, 0) + 1

top_three = sorted(letter_counts.items(), key=lambda pair: (-pair[1], pair[0]))[:3]
print("Most common letters:", top_three)
```

Expected Behavior or Output

Expected output:

```
Word count: 8
Word Frequencies:
'a': 1
'student': 1
'studies': 1
'programming': 2
'develops': 1
'problem-solving': 1
'skills': 1
Most common letters: [('s', 7), ('e', 5), ('g', 5)]
```

Common Mistakes

1. Counting punctuation as part of a word, so `programming.` and `programming` become different words.
2. Forgetting to convert text to lower case before counting, so `Programming` and `programming` are counted separately.
3. Sorting a dictionary by key when the count was intended.

Activities

1. Predict the word count and the count for `programming` before running.
2. Count the words longer than 8 letters.
3. Print the longest word.
4. Change the sort so ties are listed in reverse alphabetical order.

Accessibility Practice

Read the frequency list in its fixed order. Because the order is stable, you can learn it and compare runs by ear. Print each pair as a short sentence (`'programming': 2`) with the same wording every time.

Skills Checklist

- ☐ I can read text content from a file.
- ☐ I can split a string of text into individual words.
- ☐ I can use a dictionary to count the frequency of words.

Lesson 9.5: Working with CSV Data

Reading References

- Matthes: Chapter 10 (Files and Exceptions), the section on CSV files
- Zelle: Chapter 5 (Sequences: Strings, Lists, and Files)

Learning Objectives

- Explain what CSV data is.
- Read a CSV file line by line.
- Split a line of CSV data into individual values.

Concept Explanation

Save this as `students.csv` (the same file is used in every language track):

```
student_id,first_name,last_name,score,active
1,Avery,Student,91,true
2,Jordan,Lee,82,true
3,Casey,Smith,76,false
```

The `csv` module handles quoted fields and embedded commas correctly. Splitting each line on commas with `line.split(',')` works only for simple files where no value contains a comma.

Worked Example

```
import csv

filename = 'students.csv'
try:
    with open(filename, 'r', newline='') as file_object:
        reader = csv.DictReader(file_object)
        print(f"Columns: {reader.fieldnames}")
        scores = []
        active_count = 0
        for row in reader:
            scores.append(int(row["score"]))
            if row["active"] == "true":
                active_count += 1
            print(f"{row['student_id']}: {row['first_name']} {row['last_name']} scored {row
    print(f"Students: {len(scores)}")
    print(f"Average score: {sum(scores) / len(scores):.2f}")
    print(f"Active students: {active_count}")
except FileNotFoundError:
    print(f"Error: The file {filename} was not found.")
```

Reading the Canonical CSV Schema

Use the standard `csv` module rather than splitting rows on commas. A quoted field may itself contain a comma.

```
import csv
from io import StringIO

csv_text = """student_id,first_name,last_name,score,active
1001,Avery,Jordan,82,true
1002,Riley,Chen,91,true
1003,Sam,Patel,76,false
1004,Morgan,Diaz,88,true
"""

reader = csv.DictReader(StringIO(csv_text))
for row in reader:
    print(row["student_id"], row["first_name"], row["last_name"],
          row["score"], row["active"])
```

Expected output:

```
1001 Avery Jordan 82 true
1002 Riley Chen 91 true
1003 Sam Patel 76 false
1004 Morgan Diaz 88 true
```

Common Mistakes

1. Splitting a CSV line at every comma when a value itself contains a quoted comma; use the `csv` module.
2. Forgetting to convert the text in the score column to a number.
3. Opening a CSV file without `newline=''`, which can produce blank rows on some systems.

Activities

1. Predict the printed lines before running.
2. Add a fourth student and check the new average.
3. Print only the active students.
4. Explain, with an example, why `"Smith, Jr."` breaks `line.split(',')`.

Accessibility Practice

Print one student per line with the same wording each time, so the screen reader announces a repeating pattern that is easy to compare line by line. Read the column names aloud once at the start so the values that follow have meaning.

Skills Checklist

- ☐ I can explain what CSV data is.
- ☐ I can read a CSV file line by line.
- ☐ I can split a line of CSV data into individual values.

Canonical Shared Dataset — Text, CSV, Tasks, and Tests

Text analysis

Use this exact sentence for word-count and character-processing exercises:

```
text_sample = "A student studies programming. Programming develops problem-solving skills."
words = text_sample.lower().replace(".", "").split()
print("Words:", len(words))
print("Programming:", words.count("programming"))
```

Expected output:

```
Words: 8
Programming: 2
```

CSV schema

The shared CSV header is exactly `student_id,first_name,last_name,score,active`. Example rows:

```
student_id,first_name,last_name,score,active
1001,Avery,Jordan,82,true
```

```
1002,Riley,Chen,91,true
1003,Sam,Patel,76,false
1004,Morgan,Diaz,88,true
```

Use the `csv` module and named column positions; do not substitute a different three-column name/age/grade schema.

```
import csv
from io import StringIO

csv_text = """student_id,first_name,last_name,score,active
1001,Avery,Jordan,82,true
1002,Riley,Chen,91,true
1003,Sam,Patel,76,false
1004,Morgan,Diaz,88,true
"""
for row in csv.DictReader(StringIO(csv_text)):
    print(row["student_id"], row["first_name"], row["last_name"],
          row["score"], row["active"])
```

Expected output:

```
1001 Avery Jordan 82 true
1002 Riley Chen 91 true
1003 Sam Patel 76 false
1004 Morgan Diaz 88 true
```

To-do record schema

Use all five fields in every serialized task record: `id`, `title`, `description`, `done`, and `created_at`.

```
task = {
    "id": 1,
    "title": "Practice Python",
    "description": "Complete the shared dataset exercises",
    "done": False,
    "created_at": "2025-01-15T09:00:00Z",
}
```

```
import json

task = {
    "id": 1,
    "title": "Practice Python",
    "description": "Complete the shared dataset exercises",
    "done": False,
    "created_at": "2025-01-15T09:00:00Z",
}
```

```
serialized = json.dumps([task])
loaded_tasks = json.loads(serialized)
loaded_task = loaded_tasks[0]
required_fields = {"id", "title", "description", "done", "created_at"}
assert set(loaded_task) == required_fields
print(loaded_task["id"], loaded_task["title"], loaded_task["done"])
```

Expected output:

```
1 Practice Python False
```

Tests should assert deterministic results for the shared tip calculator, grade classifier, inventory total, and text sample. Random-number lessons must seed the generator when an exact result is asserted.

13.12 Unit 10: Project-Based Learning — Building Text-Based Applications

Overview

Compatibility: All examples target Python 3.12 and newer; they avoid syntax and standard-library APIs introduced after Python 3.12.

This unit guides students through designing and building larger, interactive text-based applications, integrating all previously learned concepts.

Objectives:

- Practice modular programming and code organization.
- Develop a text-based interactive program.
- Apply top-down design principles.

Lesson 10.1: Designing Your Project

Reading References

- Zelle: Chapter 9, pages 225–235
- Matthes: Part II: Projects, pages 225–226

Learning Objectives

- Break down a larger problem into smaller, manageable parts.
- Identify the key features required for a project.
- Plan which data structures will be most suitable for my project's data.
- Outline the main steps or flow of my program.

Concept Explanation

Top-Down Design **Top-down design** (stepwise refinement) breaks a complex problem into smaller, more manageable subproblems.

The Design Process:

1. Analyze the Problem

- Identify **inputs** (information the program needs).
- Identify **outputs** (results it must produce).
- Define the scope of the project.

2. Determine the Major Components Write a high-level algorithm in plain English:

```
show_greeting()
get_user_input()
calculate_cost()
print_ticket()
```

3. Refine Each Component Repeat the design process for each component, adding detail until simple enough to code.

Benefits of Top-Down Design

- **Reduces Complexity** — Focus on one piece at a time.
- **Encourages Reusability** — Functions written for one part may be useful elsewhere.
- **Simplifies Debugging** — Test each component in isolation.
- **Improves Collaboration** — Clear plan for assigning work.

Accessibility Considerations for I/O Design

- Provide **clear, descriptive prompts** for all input.
- Provide **explicit, labeled output** — avoid relying on visual formatting.
- Print **confirmation messages** after every action.
- Use consistent **output separators** like `---` to group information.

Worked Example

Top-down design applied to the command-line to-do tool. The top-level plan names the major components, and the first runnable version contains only the first component:

```
show_menu()
get_choice()
add_task() / view_tasks() / mark_done() / save_tasks()
repeat until the user chooses to quit
```

```
def show_menu():
    print("1. Add task")
    print("2. View tasks")
    print("3. Quit")


def run():
    show_menu()
    print("(The choices are not implemented yet.)")


run()
```

Expected Behavior or Output

Output of example 1:

```
1. Add task
2. View tasks
3. Quit
(The choices are not implemented yet.)
```

Common Mistakes

1. Starting to code the whole program at once instead of building and testing one component at a time.
2. Writing a design that names steps too vague to turn into code, such as "handle the data".
3. Forgetting the messages: a design that lists operations but not what the user sees after each one.

Activities

1. Write the top-level design for the text adventure as a numbered list of components.
2. Refine one component of your design into three smaller steps.
3. Write the exact wording of five prompts and messages that your project will use.
4. Explain in two sentences how the design lets you test one function in isolation.

Accessibility Practice

Write the design as a numbered list and read it aloud before coding. Decide the wording of every prompt and message now, and check that none depends on color, layout, or symbols alone.

Skills Checklist

- ☐ I can break down a larger problem into smaller, manageable parts.
- ☐ I can identify the key features required for a project.
- ☐ I can plan which data structures will be most suitable for my project's data.
- ☐ I can outline the main steps or flow of my program.

Lesson 10.2: Building a Text-Based Adventure Game (Part 1)

Reading References

- Matthes: Chapter 6, pages 105–111

Learning Objectives

- Design a way to represent different locations in a text-based game.
- Implement player movement based on user commands.
- Update and track the player's current location.
- Parse simple commands from user input.

Concept Explanation

Room Definition and Movement

Worked Example

```
room_descriptions = {
    "start_room": {
        "description": "A dimly lit cave. A narrow passage leads north.",
        "exits": {"north": "tunnel"},
        "items": [],
    },
    "tunnel": {
        "description": "A damp tunnel. Passages lead south and east.",
        "exits": {"south": "start_room", "east": "clearing"},
        "items": [],
    },
    "clearing": {
        "description": "A quiet clearing. A small antechamber lies to the north.",
        "exits": {"west": "tunnel", "north": "locked_door_room"},
        "items": [],
    },
    "locked_door_room": {
        "description": "A chamber with a heavy locked door to the north.",
```

```
        "exits": {"south": "clearing"},
        "items": [],
    },
    "treasure_room": {
        "description": "You found the treasure. You win!",
        "exits": {},
        "items": [],
    },
}

current_room = "start_room"

while True:
    print("---")
    print(room_descriptions[current_room]["description"])
    command = input("What do you want to do? ").lower()

    if command in room_descriptions[current_room]["exits"]:
        current_room = room_descriptions[current_room]["exits"][command]
    elif command == "quit":
        print("Exiting game. Goodbye!")
        break
    else:
        print("I don't understand that command or you can't go that way.")
```

Expected Behavior or Output

Sample run typing `north`, `east`, `north`, `north`, `quit` (typed input shown after each prompt):

```
---
A dimly lit cave. A narrow passage leads north.
What do you want to do? north
---
A damp tunnel. Passages lead south and east.
What do you want to do? east
---
A quiet clearing. A small antechamber lies to the north.
What do you want to do? north
---
A chamber with a heavy locked door to the north.
What do you want to do? north
I don't understand that command or you can't go that way.
---
A chamber with a heavy locked door to the north.
What do you want to do? quit
```

```
Exiting game. Goodbye!
```

Common Mistakes

1. Looking up a direction that is not in `exits` with square brackets, which raises a `KeyError`. Use `in` to test first.
2. Forgetting that `input()` raises `EOFError` at the end of piped input; scripted tests should end with `quit`.
3. Misspelling a room name in `exits`, so a move leads to a room that does not exist.

Activities

1. Trace north, east, north, north, quit and record the room after each command.
2. Try west from start_room and verify that the location does not change.
3. Verify that north from locked_door_room is rejected and leaves the player in that room.
4. Write a loop that checks that every exit names a real room.

Accessibility Practice

Each turn prints a separator line, the description, and the prompt, so a whole turn can be read in order. Listen to one turn from start to finish to confirm that a player could follow it without seeing the screen.

Skills Checklist

☐ I can design a way to represent different locations in a text-based game.
☐ I can implement player movement based on user commands.
☐ I can update and track the player's current location.
☐ I can parse simple commands from user input.

Lesson 10.3: Building a Text-Based Adventure Game (Part 2)

Reading References

- Matthes: Chapter 6 (Dictionaries) and Chapter 8 (Functions)

Learning Objectives

- Add items to locations and manage a player's inventory.
- Implement commands for picking up and dropping items.
- Design and implement a simple puzzle.
- Define win/lose conditions for the game.

Concept Explanation

Adding Items, Inventory, and Puzzles These rules are the shared text-adventure specification for every language track: five rooms (`start_room`, `tunnel`, `clearing`, `locked_door_room`, and `treasure_room`). The player starts in `start_room`; its north exit leads to `tunnel`. From `tunnel`, south returns to `start_room` and east leads to `clearing`. From `clearing`, west returns to `tunnel` and north leads to `locked_door_room`. From `locked_door_room`, south returns to `clearing`; north leads to `treasure_room` only after the player has taken the shiny key from `start_room` and used it in `locked_door_room`. The supported commands are `north`, `south`, `east`, `west`, `take shiny key`, `use shiny key`, and `quit`. Reaching `treasure_room` ends the game.

Worked Example

```
rooms = {
    "start_room": {
        "description": "A dimly lit cave. A shiny key rests on the ground.",
        "exits": {"north": "tunnel"},
        "items": ["shiny key"],
    },
    "tunnel": {
        "description": "A damp tunnel. Passages lead south and east.",
        "exits": {"south": "start_room", "east": "clearing"},
        "items": [],
    },
    "clearing": {
        "description": "A quiet clearing. A small antechamber lies to the north.",
        "exits": {"west": "tunnel", "north": "locked_door_room"},
        "items": [],
    },
    "locked_door_room": {
        "description": "A chamber with a heavy locked door to the north.",
        "exits": {"south": "clearing", "north": "treasure_room"},
        "items": [],
    },
    "treasure_room": {
        "description": "You found the treasure. You win!",
        "exits": {},
        "items": [],
    },
}

current_room = "start_room"
player_inventory = []
door_locked = True
```

```
while True:
    print("\n" + rooms[current_room]["description"])
    if current_room == "treasure_room":
        break
    if rooms[current_room]["items"]:
        print(f"You see: {', '.join(rooms[current_room]['items'])}")
    print(f"Your inventory: {', '.join(player_inventory) if player_inventory else 'empty'}")

    if current_room == "treasure_room":
        break

    command = input("What do you want to do? ").strip().lower()

    if command == "take shiny key" and "shiny key" in rooms[current_room]["items"]:
        player_inventory.append("shiny key")
        rooms[current_room]["items"].remove("shiny key")
        print("You picked up the shiny key.")
    elif command == "use shiny key" and current_room == "locked_door_room" and "shiny key" i
        door_locked = False
        print("You unlocked the door!")
    elif command in rooms[current_room]["exits"]:
        if current_room == "locked_door_room" and command == "north" and door_locked:
            print("The door is locked! You need a key.")
        else:
            current_room = rooms[current_room]["exits"][command]
    elif command == "quit":
        print("You quit the adventure.")
        break
    else:
        print("Invalid command. Use a direction, 'take shiny key', 'use shiny key', or 'quit
```

Expected Behavior or Output

Sample play (typed input shown after each prompt). The player reaches the chamber, finds the door locked, returns to take the key, then unlocks the door and wins:

```
A dark cave.
You see: shiny key
Your inventory: empty
What do you want to do? north

A small chamber with a heavy door to the NORTH.
Your inventory: empty
```

```
What do you want to do? north
The door is locked! You need a key.

A small chamber with a heavy door to the NORTH.
Your inventory: empty
What do you want to do? south

A dark cave.
You see: shiny key
Your inventory: empty
What do you want to do? take shiny key
You picked up the shiny key.

A dark cave.
Your inventory: shiny key
What do you want to do? north

A small chamber with a heavy door to the NORTH.
Your inventory: shiny key
What do you want to do? use shiny key
You unlocked the door!

A small chamber with a heavy door to the NORTH.
Your inventory: shiny key
What do you want to do? north

A glittering room full of treasure. You win!
```

Common Mistakes

1. Making the door's locked state part of the exits so the ”locked” message can never be reached.
2. Defining an exit to a room that is missing from the `rooms` dictionary, which crashes when the player arrives.
3. Printing a fixed room description that still mentions an item after the player has taken it.

Activities

1. Play the program with the sample commands and follow each turn on paper.
2. Try `use shiny key` in the cave and describe what happens.
3. Add a second item and require both to open the door.
4. List the game's state (location, inventory, door) and say which command changes each part.

Accessibility Practice

The room description never mentions the key. A separate 'You see:' line lists items, so the description stays true after the key is taken and the screen reader announces the current state.

Skills Checklist

- ☐ I can add items to locations and manage a player's inventory.
- ☐ I can implement commands for picking up and dropping items.
- ☐ I can design and implement a simple puzzle.
- ☐ I can define win/lose conditions for the game.

Lesson 10.4: Developing a Command-Line Tool

Canonical To-Do Record and Required Operations

Every task must preserve exactly these fields:

```
task = {
    "id": 1,
    "title": "Practice Python",
    "description": "Complete the shared dataset exercises",
    "done": False,
    "created_at": "2025-01-15T09:00:00Z",
}
```

Design the command-line interface around these operations:

Operation	Required behavior
Add	Reject a blank title; assign a unique integer ID; initialize `done` to `False`; preserve all five fields
List	Display tasks in ID order and state whether each is complete
Complete/update	Find by ID, update `done`, and report a missing or invalid ID
Delete	Remove by ID and report a missing or invalid ID
Save	Serialize the complete list as JSON
Load	Validate that the top level is a list and every task has exactly the five required fields and appropriate value types

Use the following function boundaries as a design scaffold. Implement and test each function as a separate exercise; this outline intentionally does not provide the project solution.

```
def add_task(tasks, title, description, created_at):
    # TODO: validate title, allocate a unique ID, and append a five-field record
    raise NotImplementedError


def list_tasks(tasks):
    # TODO: display records in a stable order
    raise NotImplementedError


def update_task(tasks, task_id, *, done=None, title=None, description=None):
    # TODO: locate by ID, validate updates, and report a missing ID
    raise NotImplementedError


def delete_task(tasks, task_id):
    # TODO: remove by ID and report a missing ID
    raise NotImplementedError


def save_tasks(tasks, path):
    # TODO: write all five fields as JSON
    raise NotImplementedError


def load_tasks(path):
    # TODO: handle missing/malformed JSON and validate every record
    raise NotImplementedError
```

Do not substitute list positions for IDs. Deleting or sorting a task must not change the identity of any remaining task. Use a deterministic timestamp fixture in tests; do not call the wall clock in expected-output tests.

Skills Checklist

- ☐ I can design an interactive menu system for a tool.
- ☐ I can use a `while` loop to maintain continuous user interaction.
- ☐ I can break down the tool's functionality into separate functions.
- ☐ I can implement file I/O to save and load data.

Lesson 10.5: Refinement and User Experience

Reading References

- Matthes: Chapter 10 (Files and Exceptions), the sections on exceptions
- Zelle: Chapter 8 (Loop Structures and Booleans)

Learning Objectives

- Write clear, descriptive prompts for user input.
- Handle invalid input gracefully using `try-except`.
- Provide helpful error messages.

Concept Explanation

Principles for Accessible Text Programs

1. **Clear prompts** — Tell the user exactly what to enter and in what format.
2. **Consistent output** — Use the same format for similar information.
3. **Graceful error handling** — Never let the program crash on bad input.
4. **Helpful error messages** — Tell the user what went wrong and how to fix it.
5. **"Help" commands** — Provide a way to list available commands.
6. **Confirmation messages** — Confirm every action taken.

Robust Input Validation

Worked Example

```
def get_positive_number(prompt):
    """Prompts user until a positive integer is entered."""
    while True:
        try:
            num = int(input(prompt))
            if num > 0:
                return num
            else:
                print("Please enter a positive number.")
        except ValueError:
            print("Invalid input. Please enter a whole number.")


age = get_positive_number("Enter your age: ")
print(f"You entered age: {age}")
```

Expected Behavior or Output

Sample run typing `abc`, `-4`, `0`, `16` (typed input shown after each prompt):

```
Enter your age: abc
Invalid input. Please enter a whole number.
Enter your age: -4
Please enter a positive number.
Enter your age: 0
Please enter a positive number.
Enter your age: 16
You entered age: 16
```

Common Mistakes

1. Writing the message "Invalid" without saying what is valid.
2. Accepting a decimal such as 88.5 for a field that requires a whole number.
3. Looping forever on bad input instead of limiting attempts or providing a way to quit.

Activities

1. Run the example typing `abc`, `-4`, `0`, and `16`, and read each message aloud.
2. Add an upper limit of 120 and a message that names it.
3. Write `get_yes_no(prompt)` that accepts only yes or no in any letter case.
4. Rewrite two messages from earlier lessons so they say what to do next.

Accessibility Practice

Check that every error message ends with an action the user can take, and that the same situation always produces the same words. Test with your screen reader that a message is announced before the next prompt appears.

Skills Checklist

☐ I can write clear, descriptive prompts for user input.
☐ I can handle invalid input gracefully using `try-except`.
☐ I can provide helpful error messages.

13.13 Unit 11: Capstone Project — Accessible Personal Data Manager

Overview

Students develop a fully functional personal data manager that is built to work with a screen reader and keyboard. The program keeps a list of structured records, lets the user add, view, search, change, complete, and delete them, and saves them to a JSON file. The Python track builds it as a small local web

application with NiceGUI. The other language tracks build a text-based version with the same records, rules, and tests.

Project Goal: Build an application to manage structured records with list, add, search, update, delete, and JSON persistence operations. The interface may be GUI-based in Python; the same data contract and operation semantics apply to the Lua terminal implementation.

Project Options:

- Contact Manager
- Book/Movie Tracker
- Study Notes Manager
- Personal Inventory

Every option uses the same six-field record schema (`id`, `title`, `description`, `category`, `created_at`, `updated_at`). The selected project domain changes the meaning of the field values, not the record shape or required operations.

Lesson 11.1: Introduction to NiceGUI

1. **Add New Item** — A form to input details for a new item.
2. **View All Items** — Display items in a clear, accessible list format.
3. **Edit Item** — Select an existing item and modify its details.
4. **Search Items** — Find records by ID and case-insensitive text in title, description, or category; report no matches explicitly.
5. **Delete Item** — Remove an item from the collection and confirm the result.
6. **Save/Load Data** — Persist the data to a JSON file and validate the complete six-field schema on load.

Learning Objectives

- Install NiceGUI with `uv` and start a page from the terminal.
- Explain that each `ui.` call creates an element and that an event handler runs when the user acts.
- Give every input a label and report results in a plain text status line.

Concept Explanation

NiceGUI lets you build a web page in Python. When the program runs it starts a small web server on your own computer and prints an address, normally `http://localhost:8080`. You open that address in a browser. Nothing is sent to other people; the page is served only from your computer unless you choose otherwise.

Setup. Install the package into the project environment:

```
uv add nicegui
```

Windows PowerShell uses the same command:

```
uv add nicegui
```

Elements and handlers. Each call such as `ui.label(...)`, `ui.input(...)`, or `ui.button(...)` creates an element on the page. A button takes an `on_click` function, called a *handler*, that runs when the user presses it. The code inside `@ui.page("/")` builds the page each time someone opens it.

Starting the server. `ui.run(...)` starts the server. Guard it with `if __name__ in {"__main__", "__mp_main__"}:` as NiceGUI requires. Pass `show=False` so that no browser is opened automatically, and `reload=False` while learning.

Accessibility. NiceGUI pages are ordinary web pages, so a screen reader reads them as any other page. Three habits make them usable: give every input a clear `label`, because the label text is what a screen reader announces for the field; report every result in a plain text label as well as in a pop-up, because a pop-up message disappears; and never use color as the only signal. The status label below uses the standard attributes `role="status"` and `aria-live="polite"`, which ask assistive technology to announce changes. Whether your screen reader announces them as intended must be tested with your own setup.

Worked Example

Save this as `hello_app.py`:

```
from nicegui import ui


@ui.page("/")
def main_page():
    ui.page_title("Personal Data Manager")
    ui.label("Personal Data Manager")
    name = ui.input(label="Your name")
    greeting = ui.label("Type a name and press the button.").props('role="status" aria-live=

    def greet():
        if name.value.strip():
            greeting.set_text(f"Hello, {name.value.strip()}!")
        else:
            greeting.set_text("Please enter a name.")

    ui.button("Greet", on_click=greet)
```

```
if __name__ in {"__main__", "__mp_main__"}:
    ui.run(title="Personal Data Manager", reload=False, show=False)
```

Start it from the terminal and open the address that it prints:

```
uv run python hello_app.py
```

The repository also includes a user-level test, `test_hello_ui.py`, which needs `uv add --dev pytest pytest-asyncio`:

```
from nicegui.testing import User

pytest_plugins = ["nicegui.testing.user_plugin"]


async def test_greeting(user: User) -> None:
    await user.open("/")
    await user.should_see("Personal Data Manager")
    user.find("Greet").click()
    await user.should_see("Please enter a name.")
    user.find("Your name").type("Avery")
    user.find("Greet").click()
    await user.should_see("Hello, Avery!")
```

```
# pytest.ini
[pytest]
asyncio_mode = auto
asyncio_default_fixture_loop_scope = function
main_file = hello_app.py
```

Expected Behavior or Output

When the program starts, the terminal prints a line like this one (the address and port depend on your computer; the default port is 8080):

```
NiceGUI ready to go on http://localhost:8080, and http://192.0.2.2:8080
```

The page title is Personal Data Manager. Pressing **Greet** with an empty field sets the status line to `Please enter a name.`; after typing `Avery` it says `Hello, Avery!`. Running the test with `python -m pytest -q test_hello_ui.py` prints:

```
1 passed
```

Common Mistakes

1. Calling `ui.run()` without the `__name__` guard, which can start the server twice.
2. Relying only on `ui.notify`, whose message disappears, instead of also updating a text status line.

3. Leaving an input without a label, so the screen reader announces only "edit text".
4. Starting a second copy of the program while the first still holds the port.

Activities

1. Start `hello_app.py` and read the page title and the status line with your screen reader.
2. Change the greeting to include the time of day using a second input.
3. Run the test file and then break the greeting text on purpose to watch the test fail.
4. Add a second button that clears the name and sets the status line to `Name cleared.`

Accessibility Practice

Open the page with the keyboard only: press Tab to move between the input and the button and read each stop. Check that the input's label is announced and that the status line is read after you press the button. Record the browser, screen reader, and version you tested, and do not claim the page is accessible to other setups.

Skills Checklist

☐ I can install NiceGUI and start a page from the terminal.
☐ I can explain elements and event handlers.
☐ I can give every input a label.
☐ I can report results in a plain text status line.
☐ I can run a user-level test.

Lesson 11.2: Data Model and Storage

Reading References

- Matthes: Chapter 6 (Dictionaries) and Chapter 10 (Files and Exceptions)
- Python documentation: the `json` and `datetime` modules

Learning Objectives

- Represent each record as a dictionary with the shared fields.
- Validate input and return a clear message when it is invalid.
- Save to and load from a JSON file, and handle a missing or damaged file.

Concept Explanation

The data and the rules live in one module, `records.py`, that contains no user-interface code. Keeping logic separate from the page has two benefits: the rules can be tested without a browser, and the same logic can serve a different front end later.

Each record is a dictionary with the shared fields (`id`, `title`, `description`, `category`, `created_at`, `updated_at`) and the optional `done` field. Dictionaries convert directly to JSON.

Rules. A title and a category are required. A new record gets an `id` one higher than the highest in use. `created_at` is set once and never changes. `updated_at` changes whenever a record is updated, completed, or edited. Every function that can fail returns a clear message instead of raising an error that the user must interpret; functions return a pair `(result, message)`.

Storage. `save_records` writes the list as JSON. `load_records` treats a missing file as an empty list (this is normal on first use), and returns a message when the file is damaged or does not contain valid records, so the program never overwrites good data with an empty list without telling the user.

Testable time. Functions that stamp a record take a `clock` argument that defaults to the real time. Tests pass a fake clock so that timestamps are predictable.

Worked Example

Save this as `records.py`:

```
"""Logic for the Personal Data Manager. No user-interface code lives here."""
import json
from datetime import datetime

REQUIRED_FIELDS = ("id", "title", "description", "category", "created_at", "updated_at")


def now():
    """Return the current time as ISO 8601 text, for example 2026-01-15T09:00:00."""
    return datetime.now().isoformat(timespec="seconds")


def validate(title, category):
    """Return an error message for invalid input, or None when it is valid."""
    if not title or not title.strip():
        return "A title is required."
    if not category or not category.strip():
        return "A category is required."
    return None
```

```
def next_id(records):
    """Return one more than the highest id in use (1 for an empty list)."""
    return max((record["id"] for record in records), default=0) + 1


def find(records, record_id):
    """Return the record with this id, or None."""
    for record in records:
        if record["id"] == record_id:
            return record
    return None


def add_record(records, title, description, category, clock=now):
    """Add a record. Return (record, None) on success or (None, message) on failure."""
    problem = validate(title, category)
    if problem:
        return None, problem
    stamp = clock()
    record = {
        "id": next_id(records),
        "title": title.strip(),
        "description": (description or "").strip(),
        "category": category.strip(),
        "created_at": stamp,
        "updated_at": stamp,
        "done": False,
    }
    records.append(record)
    return record, None


def update_record(records, record_id, changes, clock=now):
    """Change fields of a record. created_at and id never change."""
    record = find(records, record_id)
    if record is None:
        return None, "No record has that id."
    problem = validate(changes.get("title", record["title"]), changes.get("category", record
    if problem:
        return None, problem
    for key, value in changes.items():
        if key not in ("id", "created_at"):
            record[key] = value.strip() if isinstance(value, str) else value
    record["updated_at"] = clock()
```

```
    return record, None


def complete_record(records, record_id, clock=now):
    """Mark a record done; this counts as an update."""
    return update_record(records, record_id, {"done": True}, clock)


def delete_record(records, record_id):
    """Remove one record. Return (True, None) or (False, message)."""
    record = find(records, record_id)
    if record is None:
        return False, "No record has that id."
    records.remove(record)
    return True, None


def search(records, text):
    """Return records whose title or description contains the text (ignoring case)."""
    needle = text.strip().lower()
    return [r for r in records if needle in r["title"].lower() or needle in r["description"]


def describe(record):
    """One line of plain text for a record; status is shown in words, never by color."""
    status = "done" if record.get("done") else "not done"
    return f"{record['id']}. {record['title']} [{record['category']}] ({status})"


def save_records(path, records):
    """Write records to a JSON file."""
    with open(path, "w", encoding="utf-8") as file:
        json.dump(records, file, indent=4)


def load_records(path):
    """Return (records, None) or ([], message). A missing file is not an error."""
    try:
        with open(path, "r", encoding="utf-8") as file:
            data = json.load(file)
    except FileNotFoundError:
        return [], None
    except json.JSONDecodeError:
        return [], f"{path} is not valid JSON, so the list starts empty."
    if not isinstance(data, list) or not all(isinstance(r, dict) and all(f in r for f in REC
        return [], f"{path} does not contain valid records, so the list starts empty."
```

```
    return data, None
```

Try it with this short script, `demo.py`, in the same folder:

```
import records

ticks = iter(range(1, 100))


def clock():
    return f"2026-01-15T10:00:{next(ticks):02d}"


items = []
records.add_record(items, "Notebook", "Course notes", "supplies", clock)
records.add_record(items, "Pencil", "Box of 10", "supplies", clock)
print(records.add_record(items, "  ", "x", "supplies", clock))
for item in items:
    print(records.describe(item))

records.save_records("data.json", items)
loaded, problem = records.load_records("data.json")
print(loaded == items, problem)
print(records.load_records("missing.json"))
```

Expected Behavior or Output

Running `python demo.py` prints:

```
(None, 'A title is required.')
1. Notebook [supplies] (not done)
2. Pencil [supplies] (not done)
True None
([], None)
```

The invalid record is rejected with `(None, 'A title is required.')`. The saved and reloaded list equals the original (`True None`), and a missing file gives an empty list with no message.

Common Mistakes

1. Letting the user interface and the rules share the same code, so neither can be tested alone.
2. Overwriting `data.json` with an empty list after a failed load.
3. Changing `created_at` when a record is edited.
4. Reusing an old `id` by counting the list length instead of using the highest `id`.

Activities

1. Run `demo.py` and open the saved `data.json`. Read the text aloud.
2. Add a rule that a title may not be longer than 60 characters, with a clear message.
3. Add an `is_done` filter function that returns only completed records.
4. Damage `data.json` by deleting a bracket and run `load_records` again; read the message it returns.

Accessibility Practice

Messages are complete sentences that say what is wrong (`A title is required.`) so they read clearly in speech and braille. After saving, read `data.json` in the terminal; `indent=4` puts one field per line, which reads well line by line.

Skills Checklist

☐ I can represent records as dictionaries.
☐ I can validate input and return clear messages.
☐ I can save and load JSON.
☐ I can handle a missing or damaged file.
☐ I can keep logic separate from the user interface.

Lesson 11.3: Displaying and Searching Items

Reading References

- NiceGUI documentation (official site, nicegui.io); check the documentation for the version you installed

Learning Objectives

- Build a page that adds a record and lists all records.
- Filter the list as the user types in a search box.
- Show errors in a status line that is also announced.

Concept Explanation

The page uses the functions from `records.py`. It builds a form for a new record, a search box, and a list. The list is rebuilt by one function, `refresh()`, that clears the list column and then creates one label per record. Calling `refresh()` after every change keeps the page and the data in step.

Each record is shown as one plain sentence produced by `describe()`, for example `1. Notebook [supplies] (not done)`. The status is the words *done* and

not done, never a color or an icon alone. The description appears on its own line prefixed `Details:`.

The search box filters as the user types: `search_text.on_value_change(...)` calls `refresh()`, which uses `search()` when the box is not empty.

Every input has a unique label (`Record title`, `Record description`, `Record category`, `Search title or description`). Unique labels let a screen reader user tell the fields apart and let the tests find them.

Worked Example

Save this as `app_v1.py` (it uses `records.py` from Lesson 11.2):

```
"""Personal Data Manager, version for Lesson 11.3: add, list, and search."""
import os
from pathlib import Path

from nicegui import ui

import records as store

DATA_FILE = Path(os.environ.get("PDM_DATA_FILE", "data.json"))
items, load_problem = store.load_records(DATA_FILE)


@ui.page("/")
def main_page():
    ui.page_title("Personal Data Manager")
    ui.label("Personal Data Manager").classes("text-2xl")

    # A plain-text status line. aria-live asks assistive technology to announce changes.
    status = ui.label(load_problem or "Ready.").props('role="status" aria-live="polite"')

    def say(message, ok=True):
        status.set_text(message)
        ui.notify(message, type="positive" if ok else "negative")

    ui.label("Add a record")
    title = ui.input(label="Record title")
    description = ui.input(label="Record description")
    category = ui.input(label="Record category")

    ui.label("Find records")
    search_text = ui.input(label="Search title or description")

    ui.label("Records")
```

```
    list_column = ui.column()

    def refresh():
        shown = store.search(items, search_text.value) if search_text.value else items
        list_column.clear()
        with list_column:
            if not shown:
                ui.label("No records to show.")
            for record in shown:
                ui.label(store.describe(record))
                if record["description"]:
                    ui.label("Details: " + record["description"])

    def add_clicked():
        record, problem = store.add_record(items, title.value, description.value, category.v
        if problem:
            say(problem, ok=False)
            return
        store.save_records(DATA_FILE, items)
        title.set_value("")
        description.set_value("")
        category.set_value("")
        refresh()
        say(f"Added record {record['id']}: {record['title']}.")

    ui.button("Add record", on_click=add_clicked)
    search_text.on_value_change(lambda _: refresh())
    refresh()


if __name__ in {"__main__", "__mp_main__"}:
    ui.run(title="Personal Data Manager", reload=False, show=False)
```

The user-level test `test_app_v1_ui.py`:

```
import os
import tempfile

from nicegui.testing import User

pytest_plugins = ["nicegui.testing.user_plugin"]
os.environ["PDM_DATA_FILE"] = os.path.join(tempfile.mkdtemp(), "data.json")


async def test_add_and_search(user: User) -> None:
    await user.open("/")
    await user.should_see("No records to show.")
```

```
    for title, description, category in [("Notebook", "Course notes", "supplies"), ("Pencil'
        user.find("Record title").type(title)
        user.find("Record description").type(description)
        user.find("Record category").type(category)
        user.find("Add record").click()
    await user.should_see("2. Pencil [supplies] (not done)")
    user.find("Search title or description").type("note")
    await user.should_see("1. Notebook [supplies] (not done)")
    await user.should_not_see("2. Pencil [supplies] (not done)")
    user.find("Add record").click()
    await user.should_see("A title is required.")
```

```
# pytest.ini
[pytest]
asyncio_mode = auto
asyncio_default_fixture_loop_scope = function
main_file = app_v1.py
```

Expected Behavior or Output

Start the app with `uv run python app_v1.py` and open the printed address. With no records the list says `No records to show.` After adding Notebook and Pencil, the list shows `1. Notebook [supplies] (not done)` and `2. Pencil [supplies] (not done)`. Typing `note` in the search box leaves only the Notebook line. Pressing **Add record** with an empty title sets the status line to `A title is required.` Running the test prints:

```
1 passed
```

Common Mistakes

1. Building the list outside `refresh()`, so it is not updated when the data changes.
2. Using the same label text for two inputs, so the screen reader cannot tell them apart.
3. Showing a record's status only by color.
4. Forgetting to clear the form after a successful add, so the next record starts with old text.

Activities

1. Add the three shared sample records through the page and compare the list with the sample data.
2. Add a category filter next to the search box.
3. Show the number of records after the list, for example `3 records.`
4. Break the `describe()` text on purpose and see which test fails.

Accessibility Practice

Tab through the form and confirm that each field is announced with its label. After pressing **Add record**, listen for the status line. If your screen reader does not announce it, note the browser and screen reader you used, then add a visible sentence under the button as a fallback.

Skills Checklist

- ☐ I can build a form and a list in NiceGUI.
- ☐ I can refresh the list after every change.
- ☐ I can filter the list with a search box.
- ☐ I can show status in words.
- ☐ I can give each input a unique label.

Lesson 11.4: Changing, Completing, and Deleting Items

Reading References

- NiceGUI documentation (official site, nicegui.io); check the documentation for the version you installed

Learning Objectives

- Select a record from a list and change its title or category.
- Mark a record done and delete a record.
- Save after every successful change and report each result.

Concept Explanation

The finished page adds a *chooser* (`ui.select`) that lists every record by its descriptive sentence, plus buttons to change, complete, or delete the chosen record. All three handlers follow the same pattern: check that a record is chosen, call the matching function in `records.py`, show the message if it failed, and otherwise save the file, refresh the list, and report what happened.

`update_record` changes only the fields that are supplied and always updates `updated_at`. `complete_record` is an update that sets `done` to `True`. `delete_record` removes exactly one record. After a delete the chooser is cleared so a stale choice cannot be used.

Notice that the handlers contain no rules. A rule such as "a title is required" exists once, in `records.py`, and applies the same way no matter which control triggers it.

Worked Example

The finished `app.py` (it uses `records.py` from Lesson 11.2):

```python
"""Personal Data Manager: a NiceGUI front end for records.py."""
import os
from pathlib import Path

from nicegui import ui

import records as store

DATA_FILE = Path(os.environ.get("PDM_DATA_FILE", "data.json"))
items, load_problem = store.load_records(DATA_FILE)


@ui.page("/")
def main_page():
    ui.page_title("Personal Data Manager")
    ui.label("Personal Data Manager").classes("text-2xl")

    # A plain-text status line. aria-live asks assistive technology to announce changes.
    status = ui.label(load_problem or "Ready.").props('role="status" aria-live="polite"')

    def say(message, ok=True):
        status.set_text(message)
        ui.notify(message, type="positive" if ok else "negative")

    ui.label("Add a record")
    title = ui.input(label="Record title")
    description = ui.input(label="Record description")
    category = ui.input(label="Record category")

    ui.label("Find records")
    search_text = ui.input(label="Search title or description")

    ui.label("Records")
    list_column = ui.column()

    ui.label("Change one record")
    chooser = ui.select(options={}, label="Choose a record")
    new_title = ui.input(label="New title (leave blank to keep)")
    new_category = ui.input(label="New category (leave blank to keep)")

    def refresh():
        shown = store.search(items, search_text.value) if search_text.value else items
        list_column.clear()
        with list_column:
            if not shown:
                ui.label("No records to show.")
```

```
            for record in shown:
                ui.label(store.describe(record))
                if record["description"]:
                    ui.label("Details: " + record["description"])
        chooser.set_options({r["id"]: store.describe(r) for r in items})

    def add_clicked():
        record, problem = store.add_record(items, title.value, description.value, category.v
        if problem:
            say(problem, ok=False)
            return
        store.save_records(DATA_FILE, items)
        title.set_value("")
        description.set_value("")
        category.set_value("")
        refresh()
        say(f"Added record {record['id']}: {record['title']}.")

    def change_clicked():
        if chooser.value is None:
            say("Choose a record first.", ok=False)
            return
        changes = {}
        if new_title.value.strip():
            changes["title"] = new_title.value
        if new_category.value.strip():
            changes["category"] = new_category.value
        if not changes:
            say("Enter a new title or a new category.", ok=False)
            return
        record, problem = store.update_record(items, chooser.value, changes)
        if problem:
            say(problem, ok=False)
            return
        store.save_records(DATA_FILE, items)
        new_title.set_value("")
        new_category.set_value("")
        refresh()
        say(f"Updated record {record['id']}.")

    def complete_clicked():
        if chooser.value is None:
            say("Choose a record first.", ok=False)
            return
        record, problem = store.complete_record(items, chooser.value)
        if problem:
```

```
            say(problem, ok=False)
            return
        store.save_records(DATA_FILE, items)
        refresh()
        say(f"Marked record {record['id']} as done.")

    def delete_clicked():
        if chooser.value is None:
            say("Choose a record first.", ok=False)
            return
        record_id = chooser.value
        ok, problem = store.delete_record(items, record_id)
        if not ok:
            say(problem, ok=False)
            return
        store.save_records(DATA_FILE, items)
        chooser.set_value(None)
        refresh()
        say(f"Deleted record {record_id}.")

    ui.button("Add record", on_click=add_clicked)
    search_text.on_value_change(lambda _: refresh())
    ui.button("Change selected record", on_click=change_clicked)
    ui.button("Mark selected record done", on_click=complete_clicked)
    ui.button("Delete selected record", on_click=delete_clicked)
    refresh()


if __name__ in {"__main__", "__mp_main__"}:
    ui.run(title="Personal Data Manager", reload=False, show=False)
```

The user-level test `test_app_ui.py`:

```
"""User-level tests for app.py. They need: pip install nicegui pytest pytest-asyncio"""
import os
import tempfile

from nicegui.testing import User

pytest_plugins = ["nicegui.testing.user_plugin"]
os.environ["PDM_DATA_FILE"] = os.path.join(tempfile.mkdtemp(), "data.json")


async def test_add_search_complete_delete(user: User) -> None:
    await user.open("/")
    await user.should_see("Personal Data Manager")
    await user.should_see("No records to show.")
```

```
    user.find("Record title").type("Notebook")
    user.find("Record description").type("Course notes")
    user.find("Record category").type("supplies")
    user.find("Add record").click()
    await user.should_see("1. Notebook [supplies] (not done)")
    await user.should_see("Added record 1: Notebook.")

    user.find("Add record").click()
    await user.should_see("A title is required.")

    user.find("Choose a record").click()
    user.find("1. Notebook [supplies] (not done)").click()
    user.find("Mark selected record done").click()
    await user.should_see("1. Notebook [supplies] (done)")

    user.find("Delete selected record").click()
    await user.should_see("Deleted record 1.")
    await user.should_see("No records to show.")
```

```
# pytest.ini
[pytest]
asyncio_mode = auto
asyncio_default_fixture_loop_scope = function
main_file = app.py
```

Expected Behavior or Output

In the page: choose `1. Notebook [supplies] (not done)` and press **Mark selected record done**; the list shows `1. Notebook [supplies] (done)` and the status line says `Marked record 1 as done.` Pressing **Delete selected record** says `Deleted record 1.` Pressing a change button with nothing chosen says `Choose a record first.` The test prints:

```
1 passed
```

Common Mistakes

1. Using a stale choice after deleting, so a later action targets a record that no longer exists.
2. Putting rules in the handlers, so two buttons enforce the same rule differently.
3. Forgetting to save after a change, so the change is lost when the program stops.
4. Deleting without telling the user, which gives no way to confirm that the action happened.

Activities

1. Change the title and category of record 2 through the page and read `data.json` to confirm.
2. Add a confirmation step before a delete (for example a second button that must be pressed).
3. Add a button that deletes all completed records and reports how many were removed.
4. Write a user-level test for the change handler.

Accessibility Practice

Use the keyboard to choose a record in the chooser and press each button. Confirm that the result of every action is spoken or readable in the status line without using a mouse. Check that buttons have text labels that say what they do; avoid icon-only buttons.

Skills Checklist

☐ I can select a record and change its fields.
☐ I can mark a record done.
☐ I can delete one record.
☐ I save and refresh after each change.
☐ I keep rules in one place.

Lesson 11.5: Testing, Documentation, and Final Submission

Reading References

- Matthes: Chapter 11 (Testing Your Code)
- Python documentation: the `unittest` module
- NiceGUI documentation (official site, nicegui.io); check the documentation for the version you installed

Learning Objectives

- Run the shared acceptance tests for the logic module.
- Check the page by hand with the keyboard and your own screen reader.
- Write the README and submit the deliverables.

Concept Explanation

The shared acceptance tests (listed in the unit overview) are written once for the logic in `records.py`. Tests 1 to 5 are automated below; test 6, using every control without color or a mouse, must be checked by hand. The user-level page

test from Lesson 11.4 adds a simulated walk through the page, but it runs in Python, so it cannot tell you how a real browser and screen reader behave.

A **README** tells the next reader what the program does and how to run it. Write it in plain text or Markdown with short headings and complete sentences. Include the project title and your name, a description, the run command (`uv run python app.py`), the keyboard and screen reader instructions that you actually tested, the shared record fields, and a reflection on what was hard.

Version control. Keep your project in Git and commit after each working step so you can return to a working version (see the supplemental section at the end of this unit).

Worked Example

The test file `test_records.py` (put it next to `records.py`):

```
import json
import tempfile
import unittest
from pathlib import Path

import records


def make_clock():
    state = {"n": 0}

    def clock():
        state["n"] += 1
        return f"2026-01-15T10:00:{state['n']:02d}"
    return clock


class TestRecords(unittest.TestCase):
    def setUp(self):
        self.clock = make_clock()
        self.items = []
        records.add_record(self.items, "Notebook", "Course notes", "supplies", self.clock)
        records.add_record(self.items, "Pencil", "Box of 10", "supplies", self.clock)
        records.add_record(self.items, "Study plan", "Weekly schedule", "planning", self.clo

    def test_1_empty_title_is_rejected(self):
        record, message = records.add_record(self.items, "   ", "x", "supplies", self.clock)
        self.assertIsNone(record)
        self.assertEqual(message, "A title is required.")
        self.assertEqual(len(self.items), 3)
```

```
    def test_2_new_record_gets_an_unused_id(self):
        records.delete_record(self.items, 2)
        record, _ = records.add_record(self.items, "Ruler", "30 cm", "supplies", self.clock)
        self.assertEqual(record["id"], 4)

    def test_3_update_changes_updated_at_but_not_created_at(self):
        before = dict(records.find(self.items, 1))
        records.update_record(self.items, 1, {"description": "Lecture notes"}, self.clock)
        after = records.find(self.items, 1)
        self.assertEqual(after["created_at"], before["created_at"])
        self.assertNotEqual(after["updated_at"], before["updated_at"])

    def test_4_delete_removes_only_that_record(self):
        ok, _ = records.delete_record(self.items, 2)
        self.assertTrue(ok)
        self.assertEqual([r["id"] for r in self.items], [1, 3])

    def test_5_save_and_reload_preserves_all_fields(self):
        with tempfile.TemporaryDirectory() as folder:
            path = Path(folder) / "data.json"
            records.save_records(path, self.items)
            loaded, message = records.load_records(path)
        self.assertIsNone(message)
        self.assertEqual(loaded, self.items)

    def test_damaged_file_gives_an_empty_list_and_a_message(self):
        with tempfile.TemporaryDirectory() as folder:
            path = Path(folder) / "data.json"
            path.write_text('[{"id": 1,', encoding="utf-8")
            loaded, message = records.load_records(path)
        self.assertEqual(loaded, [])
        self.assertIn("not valid JSON", message)

    def test_complete_marks_done_and_updates(self):
        record, _ = records.complete_record(self.items, 3, self.clock)
        self.assertTrue(record["done"])
        self.assertNotEqual(record["updated_at"], record["created_at"])

    def test_search_ignores_case_and_checks_description(self):
        self.assertEqual([r["title"] for r in records.search(self.items, "NOTE")], ["Notebo
        self.assertEqual([r["title"] for r in records.search(self.items, "box")], ["Pencil"]

    def test_blank_title_cannot_be_saved_by_update(self):
        record, message = records.update_record(self.items, 1, {"title": ""}, self.clock)
        self.assertIsNone(record)
```

```
        self.assertEqual(message, "A title is required.")


if __name__ == "__main__":
    unittest.main()
```

Run it from the terminal:

```
uv run python -m unittest -v test_records
```

A starting point for `README.md`:

```
# Personal Data Manager

By: (your name)

## What it does
A small web application that keeps a list of records and saves them to data.json.

## How to run
    uv run python app.py
Then open the address that the terminal prints.

## Keyboard and screen reader notes
(Describe what you tested: operating system, browser, screen reader, and versions.)

## Record fields
id, title, description, category, created_at, updated_at, done

## Reflection
(What was hard, and what would you improve?)
```

Expected Behavior or Output

The command prints one line per test and a summary (the timing differs from run to run):

```
test_1_empty_title_is_rejected (test_records.TestRecords.test_1_empty_title_is_rejected) ...
test_2_new_record_gets_an_unused_id (test_records.TestRecords.test_2_new_record_gets_an_unus
test_3_update_changes_updated_at_but_not_created_at (test_records.TestRecords.test_3_update_
test_4_delete_removes_only_that_record (test_records.TestRecords.test_4_delete_removes_only_
test_5_save_and_reload_preserves_all_fields (test_records.TestRecords.test_5_save_and_reloac
test_blank_title_cannot_be_saved_by_update (test_records.TestRecords.test_blank_title_cannot
test_complete_marks_done_and_updates (test_records.TestRecords.test_complete_marks_done_and_
test_damaged_file_gives_an_empty_list_and_a_message (test_records.TestRecords.test_damaged_f
test_search_ignores_case_and_checks_description (test_records.TestRecords.test_search_ignore

----------------------------------------------------------------------
```

```
Ran 9 tests in 0.001s

OK
```

Common Mistakes

1. Testing only that nothing crashed, and not checking the values returned.
2. Sharing one list of records between tests, so one test's changes affect the next.
3. Writing a README that only the author can follow, or claiming accessibility that was never tested.
4. Skipping test 6 because it cannot be automated.

Activities

1. Run the tests, then break `update_record` so it also changes `created_at` and watch test 3 fail. Restore it.
2. Add a test that deleting an unknown id returns `False` and a message.
3. Ask a classmate to follow your README from an empty folder and note where they get stuck.
4. Complete test 6 by hand: use every control with the keyboard only and record what you found.

Accessibility Practice

Run the tests from the terminal and read the summary line first. For the hand check, use the keyboard only, then repeat with your screen reader or braille display. Record the operating system, browser, screen reader, and versions that you tested, and describe only what you actually tried.

Skills Checklist

- ☐ I can run the acceptance tests and read the results.
- ☐ I can complete a hand check of keyboard and screen reader use.
- ☐ I can write a README another person can follow.
- ☐ I know what the page test does and does not show.
- ☐ I can submit the listed deliverables.

Assessment Criteria

Functionality (60%)

- ☐ Ability to add new items with all specified attributes.
- ☐ Correct display of all current items.
- ☐ Correct editing of existing items.
- ☐ Correct deletion of items.

- ☐ Accurate saving and loading to and from `data.json`.
- ☐ Search and completion work as described.
- ☐ No major bugs or crashes during typical usage.

Code Quality and Structure (20%)

- ☐ Clear and concise Python code.
- ☐ Appropriate use of functions, with rules kept in `records.py`.
- ☐ Meaningful variable and function names.
- ☐ Adequate comments where logic is complex.
- ☐ Proper indentation.

Accessibility and User Experience (15%)

- ☐ All interactive elements are correctly labeled for screen readers.
- ☐ Clear and consistent feedback in a text status line.
- ☐ Logical flow and presentation of information.
- ☐ No visual-only cues for critical information.

README and Reflection (5%)

- ☐ Complete and clear README.
- ☐ Thoughtful reflection on development and accessibility challenges.

Deliverables

1. `app.py` and `records.py`: the complete application code.
2. `test_records.py`: the acceptance tests.
3. `data.json`: an example file that shows saved data.
4. `README.md`, including: the project title and student name; a description of the application; instructions on how to run it (`uv run python app.py`); instructions for interacting with a screen reader; and a reflection on challenges faced.

Supplemental: AI Tools and Version Control

Academic Integrity: AI and Assessment

AI tools must not write, complete, autocomplete, repair, or generate course code or assessment solutions. Students may use only instructor-authorized conceptual explanations that do not produce deliverable code. Git and version control are taught separately and are not a workaround for the assessment-integrity rule.

Git and Version Control

Git is the most widely used distributed version control system, invented in 2005 by Linus Torvalds for the Linux kernel project.

Core workflow:

1. **Working directory** — Make changes to files.
2. **Staging area** — `git add` — Stage changes.
3. **Repository** — `git commit` — Save a permanent snapshot.
4. **Remote** — `git push` / `git pull` — Sync with GitHub.

Why Git is valuable:

- **Rollback** — If a change breaks your app, revert instantly.
- **`git blame`** — See exactly which commit introduced a bug.
- **Branches** — Try experiments in isolation without affecting main code.
- **Pull requests** — Review changes with others before merging.

Basic commands:

```
git init                    # Initialize a repository
git add .                   # Stage all changes
git commit -m "Add feature" # Save a snapshot
git log                     # View history
git diff                    # See what changed
git checkout -- file.py     # Discard changes to a file
```

Windows Powershell

```
git init                    # Initialize a repository
git add .                   # Stage all changes
git commit -m "Add feature" # Save a snapshot
git log                     # View history
git diff                    # See what changed
git restore -- file.py     # Discard changes to a file
```

Canonical Shared Project Contracts

Unit 10: Adventure and command-line to-do projects

All parallel-language versions must use this exact adventure graph and command set.

Room	Exits	Items / state
`start_room`	north → `tunnel`	shiny key
`tunnel`	south → `start_room`; east → `clearing`	none
`clearing`	west → `tunnel`; north → `locked_door_room`	north exit is blocked until unlocked

Room	Exits	Items / state
`locked_door_room`	south → `clearing`; north → `treasure_room` after unlocking	locked door
`treasure_room`	none	terminal winning room

Commands are `north`, `south`, `east`, `west`, `take shiny key`, `use shiny key`, and `quit`. The key begins in `start_room`; the player must take it, travel to `locked_door_room`, use it there, and then enter `treasure_room` to win. Invalid movement must not change the current room.

The CLI to-do manager must store these five fields on every task: `id`, `title`, `description`, `done`, and `created_at`. It must support create, list, complete/update, delete, save, and load. Invalid identifiers, empty titles, and malformed persistence data require explicit textual errors.

Unit 11: Accessible Personal Data Manager

The shared capstone record has exactly these fields:

```
record = {
    "id": 1,
    "title": "Programming notes",
    "description": "Review lists and dictionaries",
    "category": "Study",
    "created_at": "2025-01-15T09:00:00Z",
    "updated_at": "2025-01-15T09:00:00Z",
}
```

Required operations are list/display, add, search, update, delete, and JSON save/load. Search is case-insensitive across title, description, and category and must report an empty result set clearly. Updating any field other than the ID must preserve the ID and refresh updated_at using the supplied timestamp. Reject blank titles, duplicate/non-positive IDs, incorrect field types, missing fields, and extra fields. A GUI framework is a Python-specific implementation choice; the data model, operations, textual status/error messages, keyboard access, and acceptance tests are not optional or language-specific.

Academic integrity and AI policy

AI tools must not write, complete, autocomplete, repair, or generate course code or assessment solutions. Students may use only instructor-authorized conceptual explanations that do not produce deliverable code. Git and version control remain appropriate topics when taught independently of AI code generation.

Part XIV

RUST

Chapter 14

Rust Curriculum

Before you start: complete Unit P: Computing Foundations Primer. It explains the language-neutral vocabulary and concepts used in every language track, using non-visual examples.Rust is used for systems software, command-line tools, networking, embedded development, WebAssembly, and performance-sensitive services.

A complete introductory course aligned to Python Units 0–11. Familiar problem contexts recur so learners focus on Rust's ownership, borrowing, types, collections, error handling, traits, and idioms.

Environment

Install Rust using rustup. Verify `rustc --version` and `cargo --version`. Create a project with `cargo new course_app`, then run `cargo run`.

Complete the shared IDE setup lesson and terminal prerequisite.

- Complete lessons
- Topic index

Lesson-quality refinement

The Unit 0–11 manuscript now has four complete setup lessons and 53 numbered lessons, including the missing Unit 2 Lesson 2.5 math counterpart. Each numbered lesson includes Rust-specific concepts, a distinct standard-library worked example, three common mistakes, four activities, accessibility practice, and five measurable checklist criteria. The course targets stable Rust with the Rust 2024 edition.

Validation

scripts/check_rust_examples.py compiles and smoke-runs every fenced Rust example with rustc --edition=2024 -D warnings. The GitHub Actions workflow .github/workflows/rust-examples.yml runs this check on relevant changes. Passing establishes compilation and successful process exit, not expected-output correctness, complete capstone behavior, or assistive-technology certification. See VALIDATION.md for current status.

GUI, visualization, and workbench accessibility

See the shared GUI, visualization, and workbench accessibility reference for screen-reader support by toolkit, platform-specific activation settings, recommended alternatives, and interfaces that must not be required without an equivalent accessible route.

AI Policy

AI may be used only for personal conceptual understanding and reinforcement. It must not write, complete, autocomplete, repair, or generate course code or assessment solutions.

Recommended introductory books

- Klabnik, Steve, Carol Nichols, and Chris Krycho, with contributions from the Rust community. *The Rust Programming Language*. No Starch Press; continuously maintained online edition. The official Rust book introduces ownership, borrowing, types, structs, enums, pattern matching, error handling, generics, and Cargo through sequential explanations and projects. Free official online edition.

Prerequisite note: the official Rust book explicitly recommends that readers who are entirely new to programming first study a general programming introduction. Complete the shared computing foundations and an introductory language pathway if concepts such as variables, functions, collections, and control flow are unfamiliar. Then use this book alongside the course's Rust lessons. Check its stated Rust version against the course toolchain.

14.1 RUST Lessons

The curriculum is organized as one Markdown file per unit.

- accessible environment and toolchain
- foundations of computing
- numeric computations

- control flow
- data structures
- input and iteration
- functions
- data modeling and traits
- files and error handling
- testing and data analysis
- project based learning
- capstone

14.2 Unit 0: Accessible Environment and Toolchain

Before you start: complete Unit P: Computing Foundations Primer. It explains the language-neutral vocabulary and concepts used in every language track, using non-visual examples.

Overview

Rust is a systems programming language designed for memory safety, concurrency, and performance without a garbage collector. It is used for command-line tools, operating-system components, networking, embedded software, WebAssembly, and performance-sensitive services. This course uses the Python curriculum's canonical examples and datasets while teaching Rust idioms: ownership, borrowing, lifetimes, enums, pattern matching, Option, Result, traits, iterators, and RAII.

Toolchain baseline: stable Rust with the Rust 2024 edition. Install through rustup; use Cargo for project creation, dependency management, formatting, checking, testing, and builds. The worked examples use the standard library unless a lesson explicitly discusses an ecosystem crate.

Verification note from the prior curriculum pass: the earlier author reported that all 57 Rust programs compiled with `rustc` using edition 2021 and `-D warnings`, ran independently, and produced the documented output; two embedded test modules were also run. This report does not establish compatibility with the Rust 2024 edition specified for this course, and it has not been independently rerun in this repair. Screen-reader and braille-display testing was not reported. Rust formats the shared tip as 3.67 with `{:.2}`; Java and Kotlin format the same stored value as 3.68.

Install Rust, understand Cargo projects, configure an editor with rust-analyzer, and establish an accessible keyboard/screen-reader workflow.

Lesson 0.1: Install and Verify Rust

Reading References

- The Rust Programming Language: Installation
- Rust standard library documentation
- Matching Python lesson: Lesson 0.1: Installing Python and uv

Learning Objectives

- Install Rust with `rustup` from the official source.
- Verify `rustc` and `cargo` from the terminal.
- Compile and run a first program.

Concept Explanation

Rust is a compiled language. The compiler is `rustc`, and **Cargo** is Rust's build tool and package manager. Both are installed by **rustup**, the official installer and version manager. This course targets stable Rust with the 2024 edition (Rust 1.85 or later); check `rustc --version` rather than assuming.

Install from the official Rust site, and write down which route you used. Then verify:

```
rustc --version
cargo --version
```

Each prints a version line. If a command is not found, open a new terminal so that the updated PATH is read.

To compile and run the example below without Cargo, save it as `main.rs`, then:

```
rustc main.rs
./main
```

(on Windows the program is `main.exe`). `rustc` prints nothing when the compile succeeds and there are no warnings.

Worked Example

```rust
fn main() {
    println!("Rust toolchain ready.");
}
```

Expected Behavior or Output

The program prints:

```
Rust toolchain ready.
```

Common Mistakes

1. Installing from an unofficial source or running an installer command without reading what it does.
2. Not opening a new terminal after installing, so the tools are reported as not found.
3. Assuming an old `rustc` supports the 2024 edition. Check the version.

Activities

1. Install Rust and record the exact output of both version commands.
2. Compile and run the example from the keyboard and list the files `rustc` created.
3. Remove the semicolon after the `println!` call, compile, and write down the message (it is still valid Rust, so explain why).
4. Explain in two sentences what rustup does that rustc alone does not.

Accessibility Practice

Run each command from the keyboard and read the output line by line with your screen reader. Confirm that the version numbers are announced digit by digit. Compiler messages are long, so jump to the first line, which names the problem and the file.

Skills Checklist

☐ I can install Rust with rustup.
☐ I can verify `rustc` and `cargo`.
☐ I can compile and run a first program.
☐ I know what Cargo is for.
☐ I can say which route I used.

Lesson 0.2: Cargo Workspace and First Run

Reading References

- The Rust Programming Language: Hello, Cargo!
- Rust standard library documentation
- Matching Python lesson: Lesson 0.2: Configuring Your Screen Reader for VS Code or Theia IDE

Learning Objectives

- Create a Cargo project and run it.
- Name the files Cargo creates.
- Read a compiler error and find the file, line, and message.

Concept Explanation

A **Cargo project** keeps one program's files together. `cargo new rust-course` creates a folder with `Cargo.toml` (the project's name, version, and dependencies) and `src/main.rs` (the source). From inside that folder, `cargo run` compiles and runs the program, `cargo check` checks it without producing a program (fast), `cargo build` compiles it, `cargo fmt` formats the source, and `cargo clippy` suggests improvements.

Every Rust program starts at `fn main()`. Statements end with semicolons and blocks are enclosed in braces. `println!` is a *macro*, which is why its name ends with an exclamation mark.

Rust's compiler messages are detailed: each starts with `error[E0425]: ...` or `warning: ...`, then an arrow line `--> src/main.rs:3:5` giving the file, line, and column, then the code with a caret under the problem and often a `help:` suggestion. Read the first message first, because later ones are often consequences of it.

Worked Example

```
fn main() {
    println!("Hello, Avery!");
    println!("Welcome to Rust programming!");
}
```

Expected Behavior or Output

The program prints:

```
Hello, Avery!
Welcome to Rust programming!
```

Common Mistakes

1. Running `cargo` commands outside the project folder, so no `Cargo.toml` is found.
2. Fixing the last error message first. Start with the first one.
3. Ignoring `help:` lines, which often contain the exact fix.

Activities

1. Create a Cargo project from the keyboard, run it, and record the files it created.
2. Misspell `println` as `printn`, run `cargo check`, and write down the code, file, line, and help text. Then restore it.
3. Add a third `println!` with your own name and predict the output before running.
4. Explain the difference between `cargo check` and `cargo run`.

Accessibility Practice

Create the project with terminal commands. When the compiler reports an error, read the `-->` line first (it holds the file, line, and column), then jump to that line in the editor. Ask your screen reader to read the caret line by character, because the caret marks the exact position.

Skills Checklist

- ☐ I can create and run a Cargo project.
- ☐ I know what `Cargo.toml` is.
- ☐ I can use `cargo check`.
- ☐ I can read the location line of an error.
- ☐ I know why `println!` ends with `!`.

Lesson 0.3: Editor and Accessibility Workflow

Reading References

- rust-analyzer manual
- The Rust Programming Language: The match Control Flow Construct
- Rust standard library documentation
- Matching Python lesson: Lesson 0.3: Setting Up Your VS Code or Theia IDE Workspace

Learning Objectives

- Configure an editor so Rust source is readable by speech or braille.
- Choose a punctuation level that announces the symbols Rust depends on.
- Read `Option`, references, closures, and macros aloud.

Concept Explanation

Complete the shared IDE setup lesson first. Then adapt the editor for Rust. **rust-analyzer** is the usual language server; it is optional, and the course works with a plain editor and the terminal. Install extensions only from a source you trust and read the permissions they request.

Rust depends on punctuation that a screen reader may skip at low settings. Learn to say these: `&` is a *reference* (a borrow), `&mut` is a *mutable reference*, `::` separates a path (`std::env::args`), `->` introduces a return type, `?` passes an error up, `!` after a name marks a *macro*, `'a` is a *lifetime*, `|x| x * 2` is a *closure* (the bars hold its parameters), `{}` inside a format string is a placeholder, and `<T>` holds a type parameter.

Rust structure comes from braces, not indentation, so count the closing braces when you read a nested block.

Read the example aloud: ”let name colon Option open angle bracket ampersand str close angle bracket equals None semicolon.” Then say what it prints.

Worked Example

```
fn main() {
    let name: Option<&str> = None;
    println!("{}", name.unwrap_or("no value"));

    let title: Option<&str> = Some("Rust");
    if let Some(t) = title {
        println!("title has {} characters", t.len());
    }

    let numbers = vec![1, 2, 3];
    let doubled: Vec<i32> = numbers.iter().map(|n| n * 2).collect();
    println!("{:?}", doubled);
}
```

Expected Behavior or Output

The program prints:

```
no value
title has 4 characters
[2, 4, 6]
```

Common Mistakes

1. Setting punctuation so low that `&`, `::`, and `|` are never spoken, then misreading references or closures.
2. Relying on indentation alone to see where a block ends. Count the closing braces.
3. Installing several overlapping extensions and getting duplicate diagnostics.

Activities

1. Test your editor with the example and record the punctuation level at which every symbol is announced.
2. Navigate the file by line, word, and character and note the commands you used.
3. Remove a closing brace, save, and find how your editor reports it. Then restore it.
4. Write a half-page note of your edit, check, and run steps using only the keyboard.

Accessibility Practice

This whole lesson is accessibility practice. Verify that you can open the file, move to a given line, read a line character by character, run `cargo check` in the terminal, and read the output. Record the operating system, editor, and access technology you actually tested; do not describe the setup as verified for other combinations.

Skills Checklist

- ☐ I can configure my editor for keyboard-only use.
- ☐ I can hear `&`, `::`, and `|` announced.
- ☐ I can read `Option` and a closure aloud.
- ☐ I know Rust structure comes from braces.
- ☐ I can check and read the output from the keyboard.

Lesson 0.4: Project Environments, Dependencies, and Terminal Usage

Reading References

- The Rust Programming Language: Accepting Command Line Arguments
- The Cargo Book
- Rust standard library documentation
- Matching Python lesson: Lesson 0.4: Python Environments, Terminal Usage, and Accessibility Tools

Learning Objectives

- Run a program with command-line arguments.
- Describe the project root and what Cargo adds.
- Explain when a dependency is needed.

Concept Explanation

A *command-line argument* is an extra word typed after the program name. `std::env::args()` returns an iterator over them; the first item is the program's own name, so `.skip(1)` leaves only the arguments you typed. This is the simplest way to pass input without a prompt, and it is easy to test.

Keep each project in its own folder and run commands from the *project root*, the folder that contains `Cargo.toml`. With Cargo, pass arguments after `--`: `cargo run -- red blue`. **Dependencies** (called *crates*) are listed in `Cargo.toml` and downloaded from crates.io; `Cargo.lock` records the exact versions so that every machine builds the same way. The standard library is enough for this course, so do not add a crate to a small exercise.

Worked Example

```rust
use std::env;

fn main() {
    let args: Vec<String> = env::args().skip(1).collect();
    println!("Argument count: {}", args.len());
    for (index, arg) in args.iter().enumerate() {
        println!("Argument {}: {}", index + 1, arg);
    }
    if args.is_empty() {
        println!("No arguments were given.");
    }
}
```

Expected Behavior or Output

Run with the command-line arguments `red blue`.

The program prints:

```
Argument count: 2
Argument 1: red
Argument 2: blue
```

With no arguments it prints:

```
Argument count: 0
No arguments were given.
```

Common Mistakes

1. Forgetting `--`, so Cargo treats `red` as its own option.
2. Forgetting that the first item of `args()` is the program name.
3. Adding a crate for something the standard library already does.

Activities

1. Run the program with `red blue` and then with no arguments. Compare the output.
2. Pass three words and a number and print the number plus one (use `parse`).
3. Print the arguments in reverse order.
4. Explain what `Cargo.lock` is for.

Accessibility Practice

Arguments let you test a program without answering prompts, so every run produces plain text read in order. Read the output of the two runs aloud and compare them line by line.

Skills Checklist

- ☐ I can run a program with arguments.
- ☐ I know the first item is the program name.
- ☐ I can pass arguments through Cargo.
- ☐ I know what a crate is.
- ☐ I can say where the project root is.

14.3 Unit 1: Foundations of Computing

Overview

This unit develops foundations of computing through Rust's type system and standard library. Prerequisites: prior units, a working Cargo project, and the ability to read compiler diagnostics. Preserve shared course datasets and test boundary conditions explicitly.

Lesson 1.1: Computer science and problem solving

Reading References

- The Rust Programming Language
- Rust standard library
- Cargo Book
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain Computer science and problem solving using Rust's types, ownership model, and standard-library conventions.
- Select an idiomatic Rust construct rather than transliterating Python syntax.
- Predict, compile, and verify the example's behavior.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

Rust is compiled and statically typed. Bindings are immutable by default. Ownership determines which binding is responsible for a value; borrowing provides temporary access without transferring ownership. Compiler diagnostics are part of the design feedback loop. This lesson retains the canonical course data so learners practice Rust rather than a new domain problem. Use cargo fmt, cargo check, and cargo test as routine development tools.

Worked Example

```rust
fn main() { println!("A program is a precise set of instructions."); }
```

Expected behavior: Runs independently with the standard library and prints the result of this lesson's focused operation. Predict the output, then compare it with the terminal output.

Common Mistakes

1. Changing a binding declared with let instead of declaring it mutable.
2. Assuming every value is copied; String moves unless borrowed or cloned.
3. Treating a type annotation as a runtime conversion.

Activities

1. Run the worked example unchanged; trace each binding, expression, ownership transfer, and output line.
2. Change the canonical Avery profile, calculator bill, or grade thresholds and predict the result before running it.
3. Add one boundary case and one invalid or missing-data case; state expected behavior before implementing the change.
4. Refactor one part using the Rust feature named in this lesson (computer science and problem solving); run cargo check and explain the diagnostic output.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect punctuation in ownership and borrowing expressions (&, &mut, ::, ->, ?). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order and verify that output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain computer science and problem solving using Rust terminology.
- ☐ I can compile and run the example with cargo run.
- ☐ I can explain the ownership, borrowing, and type decisions used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 1.2: Your first program and sequential execution

Reading References

- The Rust Programming Language
- Rust standard library
- Cargo Book
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain Your first program and sequential execution using Rust's types, ownership model, and standard-library conventions.
- Select an idiomatic Rust construct rather than transliterating Python syntax.
- Predict, compile, and verify the example's behavior.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

Rust is compiled and statically typed. Bindings are immutable by default. Ownership determines which binding is responsible for a value; borrowing provides temporary access without transferring ownership. Compiler diagnostics are part of the design feedback loop. This lesson retains the canonical course data so learners practice Rust rather than a new domain problem. Use cargo fmt, cargo check, and cargo test as routine development tools.

Worked Example

```
fn main() { let name = "Avery"; println!("Hello, {name}!"); println!("This line runs second.
```

Expected behavior: Runs independently with the standard library and prints the result of this lesson's focused operation. Predict the output, then compare it with the terminal output.

Common Mistakes

1. Changing a binding declared with let instead of declaring it mutable.
2. Assuming every value is copied; String moves unless borrowed or cloned.
3. Treating a type annotation as a runtime conversion.

Activities

1. Run the worked example unchanged; trace each binding, expression, ownership transfer, and output line.
2. Change the canonical Avery profile, calculator bill, or grade thresholds and predict the result before running it.
3. Add one boundary case and one invalid or missing-data case; state expected behavior before implementing the change.

4. Refactor one part using the Rust feature named in this lesson (your first program and sequential execution); run cargo check and explain the diagnostic output.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect punctuation in ownership and borrowing expressions (&, &mut, ::, ->, ?). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order and verify that output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain your first program and sequential execution using Rust terminology.
- ☐ I can compile and run the example with cargo run.
- ☐ I can explain the ownership, borrowing, and type decisions used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 1.3: Variables, bindings, mutability, and primitive types

Reading References

- The Rust Programming Language: Variables and Mutability
- The Rust Programming Language: Data Types
- Rust standard library documentation
- Matching Python lesson: Lesson 1.3: Variables and Simple Data Types

Learning Objectives

- Declare immutable and mutable bindings and use shadowing.
- Name Rust's basic types and check a value's type.
- Use `const` for fixed values.

Concept Explanation

`let` creates a *binding*. Bindings are **immutable by default**: `let age = 16;` cannot be reassigned. `let mut visits = 1;` can. *Shadowing* declares a new binding with the same name, `let visits = visits * 10;`, which may even change the type; it is not the same as mutation.

Rust is statically typed with inference. A whole-number literal is an `i32` and a decimal literal is an `f64` unless the context says otherwise; you can write

the type (`let big: i64 = 3_000_000_000;`). The basic types are the integers `i8`..`i128` and `u8`..`u128` (`i` signed, `u` unsigned), `f32` and `f64`, `bool`, `char`, and `&str` (a string slice). `const NAME: Type = value;` declares a fixed value that must have a type.

`std::any::type_name::<T>()` reports a type's name. The shared student profile is Avery, 16, 1.72 meters, and Computer Science. Three notebooks at 2.50 cost 7.5.

Worked Example

```
fn type_of<T>(_value: &T) -> &'static str {
    std::any::type_name::<T>()
}

fn main() {
    let name = "Avery";
    let age = 16;
    let height = 1.72;
    let favorite_subject = "Computer Science";
    let likes_cs = true;
    println!("{name}, {age}, {height}, {favorite_subject}, {likes_cs}");
    println!("{} {} {} {}", type_of(&name), type_of(&age), type_of(&height), type_of(&likes_

    let mut visits = 1;
    visits += 1;
    println!("Visits: {visits}");
    let visits = visits * 10;
    println!("Shadowed visits: {visits}");

    const NOTEBOOK_PRICE: f64 = 2.50;
    let notebook_quantity = 3;
    println!("Notebook total: {}", NOTEBOOK_PRICE * notebook_quantity as f64);

    let big: i64 = 3_000_000_000;
    println!("{big}");
}
```

Expected Behavior or Output

The program prints:

```
Avery, 16, 1.72, Computer Science, true
&str i32 f64 bool
Visits: 2
Shadowed visits: 20
Notebook total: 7.5
```

```
3000000000
```

Common Mistakes

1. Reassigning a binding declared without `mut`, which is a compile error.
2. Confusing shadowing with mutation. Shadowing makes a new binding.
3. Writing `let big = 3000000000;` and getting an out-of-range error for `i32`.

Activities

1. Run the example and write each type name in the order printed.
2. Change `age` to `16u8` and predict the type name.
3. Add a `char` initial for the student and print its type.
4. Explain the difference between `let mut x` and shadowing with `let x`.

Accessibility Practice

Say names with their underscores aloud (`favorite_subject`) and print each value inside a labelled sentence so the screen reader announces the name and the value together.

Skills Checklist

- ☐ I can declare immutable and mutable bindings.
- ☐ I can use shadowing.
- ☐ I can name Rust's basic types.
- ☐ I can use `const`.
- ☐ I can explain why bindings are immutable by default.

14.4 Unit 2: Numeric Computations

Overview

This unit develops numeric computations through Rust's type system and standard library. Prerequisites: prior units, a working Cargo project, and the ability to read compiler diagnostics. Preserve shared course datasets and test boundary conditions explicitly.

Lesson 2.1: Software design and tracing

Reading References

- The Rust Programming Language
- Rust standard library
- Cargo Book

- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain Software design and tracing using Rust's types, ownership model, and standard-library conventions.
- Select an idiomatic Rust construct rather than transliterating Python syntax.
- Predict, compile, and verify the example's behavior.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

Rust distinguishes integer and floating-point types and does not silently coerce between them. Expressions have values; mutable assignment updates a place. Input is read as text and parsed into a requested type through Result. For real financial systems, use integer minor units or a decimal type rather than binary floating point. This lesson retains the canonical course data so learners practice Rust rather than a new domain problem. Use cargo fmt, cargo check, and cargo test as routine development tools.

Worked Example

```
fn main() { let bill=24.50_f64; let rate=0.15_f64; let tip=bill*rate; println!("Tip: {tip:.2
```

Expected behavior: Runs independently with the standard library and prints the result of this lesson's focused operation. Predict the output, then compare it with the terminal output.

Common Mistakes

1. Using integer division when a fractional result is required.
2. Ignoring the Result returned by parsing user input.
3. Mixing numeric types without explicit conversions.

Activities

1. Run the worked example unchanged; trace each binding, expression, ownership transfer, and output line.
2. Change the canonical Avery profile, calculator bill, or grade thresholds and predict the result before running it.
3. Add one boundary case and one invalid or missing-data case; state expected behavior before implementing the change.
4. Refactor one part using the Rust feature named in this lesson (software design and tracing); run cargo check and explain the diagnostic output.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect punctuation in ownership and borrowing expressions (&, &mut, ::, ->, ?). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order and verify that output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain software design and tracing using Rust terminology.
- ☐ I can compile and run the example with cargo run.
- ☐ I can explain the ownership, borrowing, and type decisions used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 2.5: Mathematical functions, circle geometry, and distance

Reading References

- The Rust Programming Language
- Rust standard library
- Cargo Book
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain Mathematical functions, circle geometry, and distance using Rust's types, ownership model, and standard-library conventions.
- Select an idiomatic Rust construct rather than transliterating Python syntax.
- Predict, compile, and verify the example's behavior.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

Rust distinguishes integer and floating-point types and does not silently coerce between them. Expressions have values; mutable assignment updates a place. Input is read as text and parsed into a requested type through Result. For real financial systems, use integer minor units or a decimal type rather than binary floating point. This lesson retains the canonical course data so learners practice Rust rather than a new domain problem. Use cargo fmt, cargo check, and cargo test as routine development tools.

Worked Example

```
fn main() { let d=3.0_f64.hypot(4.0); println!("Distance from origin: {d}"); }
```

Expected behavior: Runs independently with the standard library and prints the result of this lesson's focused operation. Predict the output, then compare it with the terminal output.

Common Mistakes

1. Using integer division when a fractional result is required.
2. Ignoring the Result returned by parsing user input.
3. Mixing numeric types without explicit conversions.

Activities

1. Run the worked example unchanged; trace each binding, expression, ownership transfer, and output line.
2. Change the canonical Avery profile, calculator bill, or grade thresholds and predict the result before running it.
3. Add one boundary case and one invalid or missing-data case; state expected behavior before implementing the change.
4. Refactor one part using the Rust feature named in this lesson (mathematical functions, circle geometry, and distance); run cargo check and explain the diagnostic output.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect punctuation in ownership and borrowing expressions (&, &mut, ::, ->, ?). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order and verify that output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain mathematical functions, circle geometry, and distance using Rust terminology.
- ☐ I can compile and run the example with cargo run.
- ☐ I can explain the ownership, borrowing, and type decisions used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 2.2: Expressions, operators, and assignment

Reading References

- The Rust Programming Language
- Rust standard library
- Cargo Book
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain Expressions, operators, and assignment using Rust's types, ownership model, and standard-library conventions.
- Select an idiomatic Rust construct rather than transliterating Python syntax.
- Predict, compile, and verify the example's behavior.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

Rust distinguishes integer and floating-point types and does not silently coerce between them. Expressions have values; mutable assignment updates a place. Input is read as text and parsed into a requested type through Result. For real financial systems, use integer minor units or a decimal type rather than binary floating point. This lesson retains the canonical course data so learners practice Rust rather than a new domain problem. Use cargo fmt, cargo check, and cargo test as routine development tools.

Worked Example

```
fn main() { let subtotal=24.50_f64; let tax=subtotal*0.075; println!("Subtotal {subtotal:.2]
```

Expected behavior: Runs independently with the standard library and prints the result of this lesson's focused operation. Predict the output, then compare it with the terminal output.

Common Mistakes

1. Using integer division when a fractional result is required.
2. Ignoring the Result returned by parsing user input.
3. Mixing numeric types without explicit conversions.

Activities

1. Run the worked example unchanged; trace each binding, expression, ownership transfer, and output line.
2. Change the canonical Avery profile, calculator bill, or grade thresholds and predict the result before running it.

3. Add one boundary case and one invalid or missing-data case; state expected behavior before implementing the change.
4. Refactor one part using the Rust feature named in this lesson (expressions, operators, and assignment); run cargo check and explain the diagnostic output.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect punctuation in ownership and borrowing expressions (&, &mut, ::, ->, ?). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order and verify that output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain expressions, operators, and assignment using Rust terminology.
- ☐ I can compile and run the example with cargo run.
- ☐ I can explain the ownership, borrowing, and type decisions used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 2.3: Input and output

Reading References

- The Rust Programming Language: Programming a Guessing Game
- Rust standard library documentation
- Matching Python lesson: Lesson 2.3: Input and Output Revisited

Learning Objectives

- Read a line from standard input.
- Convert text with `parse` and handle failure.
- Handle missing and invalid input.

Concept Explanation

`std::io::stdin().read_line(&mut line)` appends the next line to a `String` you own and returns a `Result` holding the number of bytes read. A result of `Ok(0)` means the input has ended. The line keeps its line ending, so call `trim()`.

Typed text is always text. `text.parse::<i32>()` returns a `Result`: `Ok(number)` or `Err(error)`. `.ok()` turns a `Result` into an `Option`, dis-

carding the error, and the `?` operator in a function that returns `Option` stops early with `None`. Rust has no null; a value that may be missing is an `Option`.

The example reads a name and two numbers and prints a greeting and their sum. With Avery, 12, and 8 the sum is 20. Prompts are printed on their own lines so the screen reader reads each cleanly.

Worked Example

```
use std::io;

fn read_line() -> Option<String> {
    let mut line = String::new();
    match io::stdin().read_line(&mut line) {
        Ok(0) | Err(_) => None,
        Ok(_) => Some(line.trim().to_string()),
    }
}

fn read_number() -> Option<i32> {
    read_line()?.parse().ok()
}

fn main() {
    println!("Enter your name:");
    let name = read_line();
    println!("Enter the first number:");
    let first = read_number();
    println!("Enter the second number:");
    let second = read_number();

    match (name, first, second) {
        (Some(name), Some(first), Some(second)) => {
            println!("Hello, {name}!");
            println!("The sum of {first} and {second} is {}.", first + second);
        }
        _ => println!("Input was missing or not a number."),
    }
}
```

Expected Behavior or Output

With the following lines typed as standard input (piped input is not echoed, so it does not appear in the output):

```
Avery
12
```

```
8
```

the program prints:

```
Enter your name:
Enter the first number:
Enter the second number:
Hello, Avery!
The sum of 12 and 8 is 20.
```

With no standard input at all, the program ends cleanly and prints:

```
Enter your name:
Enter the first number:
Enter the second number:
Input was missing or not a number.
```

Common Mistakes

1. Forgetting `trim()`, so the line ending makes `parse` fail.
2. Calling `unwrap()` on a `parse` result and crashing on `abc`.
3. Forgetting that `read_line` returns `Ok(0)` at the end of input and looping forever.

Activities

1. Run with Avery, 12, and 8 and confirm the output.
2. Type `twelve` for the first number and explain the result.
3. Run with no input and describe the output.
4. Change the program to subtract the second number from the first.

Accessibility Practice

Every prompt is a complete sentence on its own line and every result is a complete sentence. When you test, listen for whether the prompt is read in full before you begin to type.

Skills Checklist

- ☐ I can read a line with `read_line`.
- ☐ I can convert text with `parse`.
- ☐ I handle `Ok(0)` at the end of input.
- ☐ I can write clear prompts.
- ☐ I handle missing and invalid input.

Lesson 2.4: Numeric types, conversions, and math

Reading References

- The Rust Programming Language: Data Types
- Rust by Example: Casting
- Rust standard library documentation
- Matching Python lesson: Lesson 2.4: Computing with Numbers and Type Conversions

Learning Objectives

- Convert between text and numbers with `parse`.
- Explain what `as` does and when `try_from` is safer.
- Explain why decimal arithmetic is approximate.

Concept Explanation

`"16".parse::<i32>()` converts text and returns a `Result`; invalid text gives an `Err` whose message is readable, for example `invalid digit found in string`. `to_string()` converts a number to text.

`as` converts between number types and **never fails**: `3.9_f64 as i32` is `3` (the fraction is truncated), and `300_i32 as u8` is `44` (the value wraps). When a lossy conversion would be a bug, use `i32::try_from(value)` or `u8::try_from(value)`, which return a `Result`. `f64::round()` rounds halves away from zero.

A float is a binary approximation. `0.1 + 0.2` prints `0.30000000000000004`, not `0.3`; this is correct behavior. Compare decimals with a tolerance and format them for display.

The shared student has age 16 and height 1.72 meters; the example converts text to those values and computes the height in centimeters.

Worked Example

```rust
fn main() {
    let age: i32 = "16".parse().unwrap_or(0);
    let height: f64 = "1.72".parse().unwrap_or(0.0);
    println!("{age} {height}");

    match "abc".parse::<i32>() {
        Ok(number) => println!("number {number}"),
        Err(error) => println!("abc is not a whole number: {error}"),
    }
    println!("{} is now text", 42.to_string());

    println!("{}", 3.9_f64 as i32);
```

```
    println!("{}", 3.5_f64.round());
    println!("{}", 300_i32 as u8);
    println!("{:?}", u8::try_from(300_i32).is_err());
    println!("{}", 0.1 + 0.2);

    let height_cm = height * 100.0;
    println!("Height in centimeters: {height_cm:.1}");
}
```

Expected Behavior or Output

The program prints:

```
16 1.72
abc is not a whole number: invalid digit found in string
42 is now text
3
4
44
true
0.30000000000000004
Height in centimeters: 172.0
```

Common Mistakes

1. Using `as` for a lossy conversion and not noticing that the value was truncated or wrapped.
2. Calling `unwrap()` on untrusted text and crashing on `abc`.
3. Comparing decimals with `==` after arithmetic.

Activities

1. Predict each output line before running, then compare.
2. Check what `" 16 ".parse::<i32>()` does and explain the result.
3. Write a function that returns `Option<i32>` for text that may not be a number.
4. Explain the difference between `3.5 as i32` and `3.5_f64.round()`.

Accessibility Practice

Format floating-point results so the screen reader reads short numbers. Read the error text aloud when a parse fails; it is written to be understood in words.

Skills Checklist

- ☐ I can convert between text and numbers.
- ☐ I know `as` never fails.
- ☐ I can use `try_from` for safe conversion.

☐ I can handle a parse error.
☐ I can explain why decimals are approximate.

Lesson 2.5: Mathematical functions, circle geometry, and distance

Reading References

- f64 documentation
- Rust standard library documentation
- Matching Python lesson: Lesson 2.5: Using the Math Library

Learning Objectives

- Use `PI`, `powi`, `sqrt`, and `hypot`.
- Compute circle area and circumference.
- Compute the distance from the origin.

Concept Explanation

Rust has no separate math library to import for common work: numeric types have methods. `f64` provides `sqrt()`, `powi(n)` (an integer power), `powf(x)`, `abs()`, `floor()`, `ceil()`, `round()`, `hypot(other)`, `max(other)`, and `min(other)`. Constants such as `PI` live in `std::f64::consts`. Integers have `abs()`, `max`, `min`, and `pow`.

For a circle of radius 5 the area is `PI * radius.powi(2)` (about 78.54) and the circumference is `2.0 * PI * radius` (about 31.42). `x.hypot(y)` computes the distance from the origin, so the shared point (3, 4) is exactly 5.0 away. `81.0_f64.sqrt()` is `9`, which prints as `9` with `{}`.

Literals need the right type: `2.0 * PI` is valid but `2 * PI` is not, because an integer cannot be multiplied by a float.

Worked Example

```rust
use std::f64::consts::PI;

fn main() {
    let radius = 5.0_f64;
    let area = PI * radius.powi(2);
    let circumference = 2.0 * PI * radius;
    println!("Area: {area:.2}");
    println!("Circumference: {circumference:.2}");

    println!("{}", 81.0_f64.sqrt());
```

```
    println!("Distance from origin: {:.2}", 3.0_f64.hypot(4.0));
    println!("{} {} {}", 82.max(91), 76.min(88), (-5_i32).abs());
}
```

Expected Behavior or Output

The program prints:

```
Area: 78.54
Circumference: 31.42
9
Distance from origin: 5.00
91 76 5
```

Common Mistakes

1. Writing `2 * PI`, which does not compile because an integer cannot multiply a float.
2. Writing the area as `PI * radius * 2`; the radius must be squared.
3. Forgetting `use std::f64::consts::PI;` so `PI` is not found.

Activities

1. Predict each printed line and then run the program.
2. Compute the area for radius 2.5 by hand and with the program.
3. Change the point to (6, 8) and predict the distance before running.
4. Print the larger of two shared scores using `max`.

Accessibility Practice

Round results with `{:.2}` so the screen reader announces `78.54` rather than fifteen digits. Say 'colon colon' aloud in `std::f64::consts::PI` and check that your screen reader announces both colons.

Skills Checklist

- ☐ I can use `f64` methods.
- ☐ I can compute circle area and circumference.
- ☐ I can compute distance with `hypot`.
- ☐ I can format results to two decimals.
- ☐ I know literals need matching types.

14.5 Unit 3: Control Flow

Overview

This unit develops control flow through Rust's type system and standard library. Prerequisites: prior units, a working Cargo project, and the ability to read compiler diagnostics. Preserve shared course datasets and test boundary conditions explicitly.

Lesson 3.1: Boolean expressions and if expressions

Reading References

- The Rust Programming Language
- Rust standard library
- Cargo Book
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain Boolean expressions and if expressions using Rust's types, ownership model, and standard-library conventions.
- Select an idiomatic Rust construct rather than transliterating Python syntax.
- Predict, compile, and verify the example's behavior.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

Conditions are expressions and branches must have compatible types. Arrays have fixed length; vectors grow dynamically. Prefer iteration or checked get access over unchecked indexing. String owns UTF-8 bytes and &str borrows a slice; use chars when counting Unicode scalar values. This lesson retains the canonical course data so learners practice Rust rather than a new domain problem. Use cargo fmt, cargo check, and cargo test as routine development tools.

Worked Example

```rust
fn main() { let score=82; println!("score={score}; passing={}",score>=60); }
```

Expected behavior: Runs independently with the standard library and prints the result of this lesson's focused operation. Predict the output, then compare it with the terminal output.

Common Mistakes

1. Using an out-of-range index instead of checked access with get.

2. Returning incompatible types from if branches.
3. Confusing UTF-8 byte length with Unicode character count.

Activities

1. Run the worked example unchanged; trace each binding, expression, ownership transfer, and output line.
2. Change the canonical Avery profile, calculator bill, or grade thresholds and predict the result before running it.
3. Add one boundary case and one invalid or missing-data case; state expected behavior before implementing the change.
4. Refactor one part using the Rust feature named in this lesson (boolean expressions and if expressions); run cargo check and explain the diagnostic output.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect punctuation in ownership and borrowing expressions (&, &mut, ::, ->, ?). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order and verify that output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain boolean expressions and if expressions using Rust terminology.
- ☐ I can compile and run the example with cargo run.
- ☐ I can explain the ownership, borrowing, and type decisions used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 3.2: Chained conditionals and grade classification

Reading References

- The Rust Programming Language
- Rust standard library
- Cargo Book
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain Chained conditionals and grade classification using Rust's types, ownership model, and standard-library conventions.

- Select an idiomatic Rust construct rather than transliterating Python syntax.
- Predict, compile, and verify the example's behavior.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

Conditions are expressions and branches must have compatible types. Arrays have fixed length; vectors grow dynamically. Prefer iteration or checked get access over unchecked indexing. String owns UTF-8 bytes and &str borrows a slice; use chars when counting Unicode scalar values. This lesson retains the canonical course data so learners practice Rust rather than a new domain problem. Use cargo fmt, cargo check, and cargo test as routine development tools.

Worked Example

```
fn main() { for s in [82,91,76,88,59] { let g=if s>=90 {"A"} else if s>=80 {"B"} else if s>=
```

Expected behavior: Runs independently with the standard library and prints the result of this lesson's focused operation. Predict the output, then compare it with the terminal output.

Common Mistakes

1. Using an out-of-range index instead of checked access with get.
2. Returning incompatible types from if branches.
3. Confusing UTF-8 byte length with Unicode character count.

Activities

1. Run the worked example unchanged; trace each binding, expression, ownership transfer, and output line.
2. Change the canonical Avery profile, calculator bill, or grade thresholds and predict the result before running it.
3. Add one boundary case and one invalid or missing-data case; state expected behavior before implementing the change.
4. Refactor one part using the Rust feature named in this lesson (chained conditionals and grade classification); run cargo check and explain the diagnostic output.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect punctuation in ownership and borrowing expressions (&, &mut, ::, ->, ?). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in

document order and verify that output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain chained conditionals and grade classification using Rust terminology.
- ☐ I can compile and run the example with cargo run.
- ☐ I can explain the ownership, borrowing, and type decisions used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 3.3: Arrays, vectors, and indexing

Reading References

- The Rust Programming Language
- Rust standard library
- Cargo Book
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain Arrays, vectors, and indexing using Rust's types, ownership model, and standard-library conventions.
- Select an idiomatic Rust construct rather than transliterating Python syntax.
- Predict, compile, and verify the example's behavior.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

Conditions are expressions and branches must have compatible types. Arrays have fixed length; vectors grow dynamically. Prefer iteration or checked get access over unchecked indexing. String owns UTF-8 bytes and &str borrows a slice; use chars when counting Unicode scalar values. This lesson retains the canonical course data so learners practice Rust rather than a new domain problem. Use cargo fmt, cargo check, and cargo test as routine development tools.

Worked Example

```
fn main() { let scores=[82,91,76,88]; println!("first={}, last={}, count={}",scores[0],score
```

Expected behavior: Runs independently with the standard library and prints the result of this lesson's focused operation. Predict the output, then compare it with the terminal output.

Common Mistakes

1. Using an out-of-range index instead of checked access with get.
2. Returning incompatible types from if branches.
3. Confusing UTF-8 byte length with Unicode character count.

Activities

1. Run the worked example unchanged; trace each binding, expression, ownership transfer, and output line.
2. Change the canonical Avery profile, calculator bill, or grade thresholds and predict the result before running it.
3. Add one boundary case and one invalid or missing-data case; state expected behavior before implementing the change.
4. Refactor one part using the Rust feature named in this lesson (arrays, vectors, and indexing); run cargo check and explain the diagnostic output.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect punctuation in ownership and borrowing expressions (&, &mut, ::, ->, ?). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order and verify that output is announced without unexpected focus movement.

Skills Checklist

☐ I can explain arrays, vectors, and indexing using Rust terminology.
☐ I can compile and run the example with cargo run.
☐ I can explain the ownership, borrowing, and type decisions used here.
☐ I can test a boundary case and an invalid-input or error case.
☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 3.4: For loops and iteration

Reading References

- The Rust Programming Language: Repetition with Loops
- The Rust Programming Language: Processing a Series of Items with Iterators
- Rust standard library documentation
- Matching Python lesson: Lesson 3.4: Iterating with for Loops

Learning Objectives

- Use `for`, ranges, and `enumerate`.

- Compute the total and average of the shared scores.
- Use `while` and `loop` with `break`.

Concept Explanation

`for item in collection` visits every element. A range `1..=3` includes both ends and `1..3` excludes the end; `(1..=10).rev().step_by(3)` counts down by 3. `iter().enumerate()` gives each item with its position. Looping over `&scores` borrows the vector, so it can still be used afterward; looping over `scores` itself would *move* it.

Iterators can summarize without an explicit loop: `scores.iter().sum::<i32>()` adds a vector's items. A loop is still the way to learn what such a call does. `while condition { ... }` repeats while a condition is true, and `loop { ... }` repeats until a `break`, which can give back a value.

The shared scores 82, 91, 76, 88 total 337 and average 84.25. Convert the integers with `as f64` before dividing.

Worked Example

```
fn main() {
    let scores = vec![82, 91, 76, 88];
    let mut total = 0;
    for (index, score) in scores.iter().enumerate() {
        println!("Adding score {}: {}", index + 1, score);
        total += score;
    }
    println!("Total: {total}");
    println!("Average: {}", total as f64 / scores.len() as f64);
    println!("Same total with an iterator: {}", scores.iter().sum::<i32>());

    for i in 1..=3 {
        println!("Count: {i}");
    }
    for i in (1..=10).rev().step_by(3) {
        println!("Step: {i}");
    }

    let mut count = 3;
    while count > 0 {
        println!("Countdown: {count}");
        count -= 1;
    }

    let mut tries = 0;
    let found = loop {
```

```
        tries += 1;
        if tries == 4 {
            break tries * 10;
        }
    };
    println!("Found {found}");
}
```

Expected Behavior or Output

The program prints:

```
Adding score 1: 82
Adding score 2: 91
Adding score 3: 76
Adding score 4: 88
Total: 337
Average: 84.25
Same total with an iterator: 337
Count: 1
Count: 2
Count: 3
Step: 10
Step: 7
Step: 4
Step: 1
Countdown: 3
Countdown: 2
Countdown: 1
Found 40
```

Common Mistakes

1. Looping over `scores` (not `&scores`) and then using `scores` again, which is a compile error because the vector was moved.
2. Writing `1..3` and expecting it to include 3.
3. Dividing the integer total by the length and getting a truncated result.

Activities

1. Predict the last nine output lines before running.
2. Change the loop to print only the scores above 80.
3. Write a loop that counts from 20 down to 0 by 5.
4. Explain why the average prints `84.25` and not `84`.

Accessibility Practice

Each output line starts with a label and a number, so the running total is read step by step. Count the lines to confirm that the loop ran four times.

Skills Checklist

- ☐ I can write `for` loops with ranges.
- ☐ I can use `enumerate`.
- ☐ I can total and average a vector.
- ☐ I know `&scores` borrows and `scores` moves.
- ☐ I can use `while` and `loop`.

Lesson 3.5: Strings and string slices

Reading References

- The Rust Programming Language: Storing UTF-8 Encoded Text with Strings
- Rust standard library documentation
- Matching Python lesson: Lesson 3.5: Basic String Operations

Learning Objectives

- Use `len`, `to_lowercase`, `contains`, `find`, and `trim`.
- Explain `String` versus `&str` and byte length versus characters.
- Normalize a name and search the shared text sample.

Concept Explanation

Rust has two main string types. `&str` is a borrowed *string slice* (a string literal is one). `String` is an owned, growable string. Both hold **UTF-8** text, so `len()` counts **bytes**, not characters: `"héllo".len()` is 6 but `"héllo".chars().count()` is 5. For ASCII text, the two agree.

Useful methods: `trim()`, `to_lowercase()`, `to_uppercase()`, `contains(text)`, `find(text)` (returns `Option<usize>`, a byte position), `matches(text).count()` (non-overlapping matches), `replace(from, to)`, and slicing `&text[..9]`. Slicing at a position that is not a character boundary panics.

Strings are not changed by these methods; each returns a new `String`. The shared text sample is "A student studies programming. Programming develops problem-solving skills." It has 75 characters and contains `programming` twice (once in each case), first found at index 18 after lowercasing.

Worked Example

```rust
fn main() {
    let raw_name = "  avery  ";
    let stripped = raw_name.trim();
    let mut chars = stripped.chars();
    let clean_name = match chars.next() {
        Some(first) => first.to_uppercase().collect::<String>() + chars.as_str(),
        None => String::new(),
    };
    println!("Name: '{clean_name}'");

    let text = "A student studies programming. Programming develops problem-solving skills.";
    let lowered = text.to_lowercase();
    println!("Characters: {}", text.len());
    println!("Contains 'programming': {}", lowered.contains("programming"));
    println!("First index of 'programming': {:?}", lowered.find("programming"));
    println!("Times 'programming' appears: {}", lowered.matches("programming").count());
    println!("Missing word index: {:?}", lowered.find("rust"));

    let upper = text.to_uppercase();
    println!("{}", &upper[..9]);
    println!("{} bytes, {} characters", "héllo".len(), "héllo".chars().count());
}
```

Expected Behavior or Output

The program prints:

```
Name: 'Avery'
Characters: 75
Contains 'programming': true
First index of 'programming': Some(18)
Times 'programming' appears: 2
Missing word index: None
A STUDENT
6 bytes, 5 characters
```

Common Mistakes

1. Treating `len()` as the number of characters for text that is not ASCII.
2. Slicing with `&text[..n]` at a position inside a multi-byte character, which panics.
3. Expecting `find` to return `-1` when the text is missing. It returns `None`.

Activities

1. Predict the printed values, then run and compare.

2. Find the index of the word `student` in the sample text.
3. Replace every `programming` with `coding` and print the result.
4. Write code that trims and capitalizes `"  riley  "`.

Accessibility Practice

Strings with many quotation marks are hard to read by ear. Say 'single quote' and 'double quote' aloud when checking a line, and print results inside labelled sentences so values are not run together.

Skills Checklist

- ☐ I can use common string methods.
- ☐ I know `String` versus `&str`.
- ☐ I know `len()` counts bytes.
- ☐ I can search text and find positions.
- ☐ I can trim and capitalize a name.

14.6 Unit 4: Data Structures

Overview

This unit develops data structures through Rust's type system and standard library. Prerequisites: prior units, a working Cargo project, and the ability to read compiler diagnostics. Preserve shared course datasets and test boundary conditions explicitly.

Lesson 4.1: Vectors and collection mutation

Reading References

- The Rust Programming Language: Vectors
- Rust standard library documentation
- Matching Python lesson: Lesson 4.1: More on Lists

Learning Objectives

- Push, insert, replace, and remove vector items.
- Remove by value using `position` and by condition using `retain`.
- Handle an empty `pop` with `Option`.

Concept Explanation

A `Vec` changes with `push(value)` (append), `insert(index, value)` (insert and shift), `scores[i] = value` (replace), `remove(index)` (remove by **position** and

return the item), `pop()` (remove and return the last item as an `Option`, because the vector might be empty), and `retain(|item| condition)` (keep only matching items). Rust has no "remove by value"; find the position first with `iter().position(|&s| s == value)`, which returns an `Option<usize>`.

`contains(&value)` tests membership. All of these methods need the vector to be declared `mut`, and Rust checks at compile time that nothing else holds a borrow of the vector while it changes.

The example applies the same operations as the Python course to the shared scores, from [82, 91, 76, 88] to [82, 85, 93, 88].

Worked Example

```
fn main() {
    let mut scores = vec![82, 91, 76, 88];

    scores.push(95);
    scores.insert(1, 85);
    scores[2] = 93;
    if let Some(position) = scores.iter().position(|&s| s == 76) {
        scores.remove(position);
    }
    let last = scores.pop();

    println!("{:?}", scores);
    println!("{} items; removed {:?}", scores.len(), last);
    println!("{}", scores.contains(&88));
    println!("{:?}", scores.iter().position(|&s| s == 88));
    println!("{:?}", scores.iter().position(|&s| s == 100));

    scores.retain(|&s| s >= 90);
    println!("{:?}", scores);
}
```

Expected Behavior or Output

The program prints:

```
[82, 85, 93, 88]
4 items; removed Some(95)
true
Some(3)
None
[93]
```

Common Mistakes

1. Looking for a `remove_value` method. Find the position first, then remove by index.
2. Calling `remove(index)` with an index that is too large, which panics.
3. Holding a reference into the vector (`let first = &scores[0];`) and then calling `push`, which the borrow checker rejects.

Activities

1. Trace the vector after every operation and write each version as a sentence.
2. Remove the first score with `remove(0)` and describe what happens to the others.
3. Remove every score below 80 with `retain`.
4. Print the position of the first score above 90.

Accessibility Practice

After every change, print the whole vector on one line so the screen reader reads a complete state in one pass. Say the method names aloud and notice how `remove` and `retain` differ in what they take.

Skills Checklist

☐ I can push, insert, replace, and remove items.
☐ I can find a position with `position`.
☐ I can filter with `retain`.
☐ I know `pop` returns an `Option`.
☐ I can show a vector in one line.

Lesson 4.2: Sorting and organizing data

Reading References

- The Rust Programming Language: Vectors
- Vec sorting methods
- Rust standard library documentation
- Matching Python lesson: Lesson 4.2: Organizing Lists

Learning Objectives

- Sort a vector ascending and descending.
- Sort a clone so the original is unchanged.
- Sort by a computed key and sort decimals.

Concept Explanation

`sort()` sorts a `mut` vector **in place**, smallest first. `sort_by(|a, b| b.cmp(a))` takes a comparison closure, here reversing the order to get descending. `sort_by_key(|w| w.len())` sorts by a computed key and keeps equal keys in their original order (the sort is *stable*). `reverse()` flips the current order. To keep the original, sort a `clone()`.

Decimals (`f64`) cannot use `sort()` because `NaN` has no order. Use `sort_by(|a, b| a.partial_cmp(b).unwrap())`, which is safe when the data contains no `NaN`.

The shared scores sort to 76, 82, 88, 91 in ascending order and 91, 88, 82, 76 in descending order.

Worked Example

```
fn main() {
    let scores = vec![82, 91, 76, 88];

    let mut ascending = scores.clone();
    ascending.sort();
    let mut descending = scores.clone();
    descending.sort_by(|a, b| b.cmp(a));
    println!("Original: {:?}", scores);
    println!("Ascending: {:?}", ascending);
    println!("Descending: {:?}", descending);

    let mut in_place = scores.clone();
    in_place.sort();
    in_place.reverse();
    println!("Sorted then reversed in place: {:?}", in_place);

    let mut words = vec!["pencil", "folder", "notebook"];
    words.sort();
    println!("{:?}", words);
    words.sort_by_key(|w| w.len());
    println!("{:?}", words);

    let mut prices = vec![2.5, 0.5, 1.25];
    prices.sort_by(|a, b| a.partial_cmp(b).unwrap());
    println!("{:?}", prices);
}
```

Expected Behavior or Output

The program prints:

```
Original: [82, 91, 76, 88]
Ascending: [76, 82, 88, 91]
Descending: [91, 88, 82, 76]
Sorted then reversed in place: [91, 88, 82, 76]
["folder", "notebook", "pencil"]
["folder", "pencil", "notebook"]
[0.5, 1.25, 2.5]
```

Common Mistakes

1. Calling `sort()` on a vector of `f64`, which does not compile because decimals are only partially ordered.
2. Forgetting `mut` on the vector, so `sort` is rejected.
3. Expecting `sort` to return the sorted vector. It sorts in place and returns nothing.

Activities

1. Predict each printed line before running.
2. Print the scores in descending order without changing the original.
3. Sort the words by their last letter with `sort_by_key`.
4. Explain why decimals need `partial_cmp`.

Accessibility Practice

Print each result on its own labelled line (`Ascending:`, `Descending:`). The first and last numbers are the quickest to check by ear.

Skills Checklist

- ☐ I can sort ascending and descending.
- ☐ I can sort a clone.
- ☐ I can sort by a key.
- ☐ I can sort decimals.
- ☐ I can reverse a vector.

Lesson 4.3: Tuples and immutable values

Reading References

- The Rust Programming Language: The Tuple Type
- The Rust Programming Language: Using Tuple Structs
- Rust standard library documentation
- Matching Python lesson: Lesson 4.3: Tuples - Immutable Lists

Learning Objectives

- Create tuples and unpack them by destructuring.
- Declare a tuple struct for a named fixed group.
- Return several values from a function.

Concept Explanation

A *tuple* groups a fixed number of values that may have different types: `(3, 4)` or `("Avery", 16, 1.72)`. Read a part with `point.0`, or **destructure** the whole tuple: `let (x, y) = point;`. A tuple is immutable unless its binding is `mut`, and its length and types cannot change.

A *tuple struct* gives a tuple a name: `struct Point(i32, i32);` so that two values of different meaning cannot be mixed up. For anything with more than two or three parts, a struct with named fields (Unit 7) reads better.

Functions return several values as a tuple: `fn min_max(values: &[i32]) -> (i32, i32)`. The shared coordinates are x = 3 and y = 4.

Worked Example

```
struct Point(i32, i32);

fn min_max(values: &[i32]) -> (i32, i32) {
    let mut low = values[0];
    let mut high = values[0];
    for &value in values {
        if value < low {
            low = value;
        }
        if value > high {
            high = value;
        }
    }
    (low, high)
}

fn main() {
    let (x, y) = (3, 4);
    println!("x={x} y={y}");

    let point = (3, 4);
    println!("{:?} {} {}", point, point.0, point.1);

    let named = Point(3, 4);
    println!("{} {}", named.0, named.1);
```

```
    let moved = (point.0, 10);
    println!("{:?} -> {:?}", point, moved);
    println!("{}", point == (3, 4));

    let (low, high) = min_max(&[82, 91, 76, 88]);
    println!("low={low} high={high}");

    let profile = ("Avery", 16, 1.72);
    println!("{} {} {}", profile.0, profile.1, profile.2);
}
```

Expected Behavior or Output

The program prints:

```
x=3 y=4
(3, 4) 3 4
3 4
(3, 4) -> (3, 10)
true
low=76 high=91
Avery 16 1.72
```

Common Mistakes

1. Trying to assign `point.0 = 5` on a binding that is not `mut`, which is a compile error.
2. Forgetting that `min_max` panics on an empty slice because it reads `values[0]`.
3. Using a tuple where named fields would make the code readable.

Activities

1. Predict the output, then run.
2. Create a tuple for the shared profile and destructure it into three bindings.
3. Write a function that returns the lowest and highest scores as a tuple and test it with one score.
4. Explain when a tuple is a better choice than a struct.

Accessibility Practice

Printing a tuple with `{:?}` shows the parentheses and commas, so the screen reader announces `(3, 4)` as a complete phrase. Named fields read better aloud than `.0` and `.1`.

Skills Checklist

- ☐ I can create and destructure tuples.

- ☐ I can use a tuple struct.
- ☐ I can return several values.
- ☐ I know tuples cannot change length.
- ☐ I can choose between a tuple and a struct.

Lesson 4.4: Hash maps and key-value data

Reading References

- The Rust Programming Language: Storing Keys with Associated Values in Hash Maps
- Rust standard library documentation
- Matching Python lesson: Lesson 4.4: Introducing Dictionaries

Learning Objectives

- Create a `HashMap` and insert, read, and remove entries.
- Handle a missing key with `Option`.
- Model the shared inventory item with a struct.

Concept Explanation

A `HashMap<K, V>` from `std::collections` pairs keys with values. `insert(key, value)` adds or replaces an entry, `get(&key)` returns an `Option<&V>` (`None` when the key is missing, never a crash), `remove(&key)` deletes one, `contains_key(&key)` tests for a key, and `len()` counts the entries. `entry(key).or_insert(0)` gives a mutable reference to a value, inserting a starting value if the key is new: `*counts.entry("pencil").or_insert(0) += 1;`.

Every value in a map has the **same type**. For a record whose fields have different types, use a struct, as with the shared Notebook item. A `HashMap` has no useful order, so the example prints only counts and single lookups.

Worked Example

```rust
use std::collections::HashMap;

struct Item {
    name: String,
    quantity: u32,
    unit_price: f64,
}

fn main() {
    let mut profile: HashMap<String, String> = HashMap::new();
```

```
    profile.insert("name".to_string(), "Avery".to_string());
    profile.insert("age".to_string(), "16".to_string());
    profile.insert("height_m".to_string(), "1.72".to_string());
    profile.insert("favorite_subject".to_string(), "Computer Science".to_string());

    println!("{:?}", profile.get("name"));
    profile.insert("age".to_string(), "17".to_string());
    profile.insert("grade_level".to_string(), "11".to_string());
    profile.remove("height_m");
    println!("{}", profile.len());
    println!("{:?}", profile.get("height_m"));
    println!("{}", profile.get("height_m").map(String::as_str).unwrap_or("unknown"));
    println!("{}", profile.contains_key("grade_level"));

    let mut counts: HashMap<&str, u32> = HashMap::new();
    *counts.entry("pencil").or_insert(0) += 1;
    *counts.entry("pencil").or_insert(0) += 1;
    println!("{:?}", counts.get("pencil"));

    let mut item = Item { name: "Notebook".to_string(), quantity: 3, unit_price: 2.50 };
    item.quantity += 2;
    println!("{}: {} at {} = {}", item.name, item.quantity, item.unit_price, item.quantity a
}
```

Expected Behavior or Output

The program prints:

```
Some("Avery")
4
None
unknown
true
Some(2)
Notebook: 5 at 2.5 = 12.5
```

Common Mistakes

1. Using `profile["height_m"]` on a missing key, which panics. Use `get`.
2. Printing a whole `HashMap` and expecting the same order every run.
3. Trying to store values of different types in one `HashMap`.

Activities

1. Predict the printed lines before running.
2. Add a `discount` field to `Item` and print the discounted value.
3. Remove `favorite_subject` and show what `get` returns.

4. Explain why a struct is safer than a map of text for a known set of fields.

Accessibility Practice

Print one lookup per line, because a one-line `{key: value, ...}` form is hard to navigate and has no stable order. Choose key names that read well aloud, such as `favorite_subject`.

Skills Checklist

- ☐ I can create and change a `HashMap`.
- ☐ I know `get` returns an `Option`.
- ☐ I can use `entry` to count.
- ☐ I know map values share one type.
- ☐ I can model an item with a struct.

Lesson 4.5: Collection traversal and iterators

Reading References

- The Rust Programming Language: Iterators
- The Rust Programming Language: Hash Maps
- Rust standard library documentation
- Matching Python lesson: Lesson 4.5: Looping Through Dictionaries

Learning Objectives

- Visit pairs in a stable order.
- Explain why `HashMap` order is unspecified.
- Sort keys with `BTreeMap` and total the shared inventory.

Concept Explanation

Iterating a `HashMap` visits entries in an **unspecified order that can change between runs**; Rust randomizes it on purpose. For readable, testable output choose a collection with a defined order: a `Vec` of pairs keeps insertion order, and a `BTreeMap` keeps its keys sorted.

`for (name, stock) in &inventory` visits a `Vec` of pairs and borrows it. Iterator methods can summarize: `.iter().map(|(_, s)| ...).sum::<f64>()` totals a computed value, and `.collect()` can build a `BTreeMap` from pairs.

The shared inventory is Notebook (3 at 2.50), Pencil (10 at 0.50), and Folder (4 at 1.25), worth 7.50 + 5.00 + 5.00 = 17.50.

Worked Example

```
use std::collections::BTreeMap;

struct Stock {
    quantity: u32,
    unit_price: f64,
}

fn main() {
    let inventory = vec![
        ("Notebook", Stock { quantity: 3, unit_price: 2.50 }),
        ("Pencil", Stock { quantity: 10, unit_price: 0.50 }),
        ("Folder", Stock { quantity: 4, unit_price: 1.25 }),
    ];

    println!("Keys:");
    for (name, _) in &inventory {
        println!("{name}");
    }

    println!("Quantities:");
    for (_, stock) in &inventory {
        println!("{}", stock.quantity);
    }

    let mut total_value = 0.0;
    for (name, stock) in &inventory {
        let line_value = stock.quantity as f64 * stock.unit_price;
        println!("{name}: {} in stock, value {line_value:.2}", stock.quantity);
        total_value += line_value;
    }
    println!("Total inventory value: {total_value:.2}");

    let sorted: BTreeMap<&str, u32> = inventory.iter().map(|(name, stock)| (*name, stock.qua
    println!("{:?}", sorted);
    println!("{}", inventory.iter().map(|(_, s)| s.quantity as f64 * s.unit_price).sum::<f64
}
```

Expected Behavior or Output

The program prints:

```
Keys:
Notebook
Pencil
Folder
```

```
Quantities:
3
10
4
Notebook: 3 in stock, value 7.50
Pencil: 10 in stock, value 5.00
Folder: 4 in stock, value 5.00
Total inventory value: 17.50
{"Folder": 4, "Notebook": 3, "Pencil": 10}
17.5
```

Common Mistakes

1. Using a `HashMap` and expecting the same order every run.
2. Looping over `inventory` instead of `&inventory`, which moves the vector so that it cannot be used again.
3. Forgetting `*name` when building a map from borrowed pairs, which gives a type error.

Activities

1. Run the example and write the three key-value lines in order.
2. Print only the items with a quantity below 5.
3. Print the items from the highest line value to the lowest.
4. Explain how `BTreeMap` differs from `HashMap`.

Accessibility Practice

Printing each inventory line as one sentence lets the screen reader announce the name, quantity, and value together. Because the order is stable, you can learn it and compare runs by ear.

Skills Checklist

- ☐ I can visit pairs in a stable order.
- ☐ I know `HashMap` order is unspecified.
- ☐ I can sort keys with `BTreeMap`.
- ☐ I can total with an iterator.
- ☐ I can total the shared inventory.

14.7 Unit 5: Input and Iteration

Overview

This unit develops input and iteration through Rust's type system and standard library. Prerequisites: prior units, a working Cargo project, and the ability to read compiler diagnostics. Preserve shared course datasets and test boundary conditions explicitly.

Lesson 5.1: Input and validation

Reading References

- The Rust Programming Language: Programming a Guessing Game
- Rust standard library documentation
- Matching Python lesson: Lesson 5.1: How the input() Function Works

Learning Objectives

- Read a line and recognize it as text.
- Convert it safely and handle failure.
- Handle the end of input.

Concept Explanation

`read_line` appends the next line to a `String` and returns a `Result`; `Ok(0)` means the input has ended. The text keeps its line ending, so `trim()` it. Typed digits are text until you convert them: `text.parse::<i32>()` returns `Ok(number)` or `Err(error)`.

`.ok()` turns a `Result` into an `Option` and `and_then` chains steps that may each fail: `text.as_deref().and_then(|t| t.parse::<i32>().ok())` means "if there is text, and it parses, give the number". A program that reads input must plan for three cases: a valid value, an invalid value, and no value at all. With the typed value 16 the age five years later is 21; the second value, `abc`, is rejected.

Worked Example

```rust
use std::io;

fn next_line() -> Option<String> {
    let mut line = String::new();
    match io::stdin().read_line(&mut line) {
        Ok(0) | Err(_) => None,
        Ok(_) => Some(line.trim().to_string()),
    }
}
```

```rust
fn main() {
    println!("How old are you?");
    let text = next_line();
    match &text {
        Some(t) => println!("Read: {} characters of text", t.len()),
        None => println!("Read: nothing"),
    }

    let age = text.as_deref().and_then(|t| t.parse::<i32>().ok());
    match age {
        Some(age) => println!("In 5 years: {}", age + 5),
        None => println!("That was not a whole number."),
    }

    println!("Enter another value:");
    let second = next_line().and_then(|t| t.parse::<i32>().ok());
    match second {
        Some(number) => println!("Number: {number}"),
        None => println!("Not a number; the parse returned None."),
    }
}
```

Expected Behavior or Output

With the following lines typed as standard input (piped input is not echoed, so it does not appear in the output):

```
16
abc
```

the program prints:

```
How old are you?
Read: 2 characters of text
In 5 years: 21
Enter another value:
Not a number; the parse returned None.
```

With no standard input at all, the program ends cleanly and prints:

```
How old are you?
Read: nothing
That was not a whole number.
Enter another value:
Not a number; the parse returned None.
```

Common Mistakes

1. Adding to the text without converting it, which does not compile.
2. Forgetting `trim()`, so the line ending makes `parse` fail.
3. Using `unwrap()` on user input and crashing on `abc`.

Activities

1. Run with 16 and then `abc` and compare with the shown output.
2. Run with no input and explain every line printed.
3. Accept an age only from 0 to 120.
4. Rewrite the second input with a `match` on the `Result` and print the parse error.

Accessibility Practice

Each prompt is a complete sentence on its own line. Run the program twice, once typing and once with piped input, and compare what your screen reader announces.

Skills Checklist

☐ I can read a line of input.
☐ I can convert and validate it.
☐ I handle the end of input.
☐ I know typed input is text.
☐ I can chain `Option` steps.

Lesson 5.2: While loops and sentinel repetition

Reading References

- The Rust Programming Language: Repetition with Loops
- Rust standard library documentation
- Matching Python lesson: Lesson 5.2: The while Loop in Action

Learning Objectives

- Write a counting `while` loop.
- Write a sentinel loop that ends on a command.
- Stop at the end of input.

Concept Explanation

`while condition { ... }` repeats while the condition is true, testing **before** each pass. A loop needs a way to make its condition false, normally by chang-

ing a variable inside the body: `count += 1` adds one. `while let Some(x) = value { ... }` repeats while a pattern matches.

A *sentinel* is a special input that tells the loop to stop, here the word `quit`. The standard pattern reads once before the loop and again at the end of each pass, and ends when there is no more input (`None`) as well as on `quit`, so the loop stops when input runs out even without `quit`.

The example counts from 1 to 5, then repeats a message prompt until the user types `quit`.

Worked Example

```
use std::io;

fn next_line() -> Option<String> {
    let mut line = String::new();
    match io::stdin().read_line(&mut line) {
        Ok(0) | Err(_) => None,
        Ok(_) => Some(line.trim().to_string()),
    }
}

fn main() {
    let mut count = 1;
    while count <= 5 {
        println!("{count}");
        count += 1;
    }
    println!("Counting finished!");

    println!("Enter a message, or type quit to exit:");
    let mut message = next_line();
    while let Some(text) = &message {
        if text == "quit" {
            break;
        }
        println!("You said: {text}");
        println!("Enter a message, or type quit to exit:");
        message = next_line();
    }
    println!("Goodbye!");
}
```

Expected Behavior or Output

With the following lines typed as standard input (piped input is not echoed, so it does not appear in the output):

```
hello
scores
quit
```

the program prints:

```
1
2
3
4
5
Counting finished!
Enter a message, or type quit to exit:
You said: hello
Enter a message, or type quit to exit:
You said: scores
Enter a message, or type quit to exit:
Goodbye!
```

With no standard input at all, the program ends cleanly and prints:

```
1
2
3
4
5
Counting finished!
Enter a message, or type quit to exit:
Goodbye!
```

Common Mistakes

1. Forgetting to change the loop variable, which creates an endless loop.
2. Testing for `quit` but not for `None`, so the loop reads forever after input ends.
3. Comparing a `String` with `"quit"` after forgetting that the text still contains its line ending.

Activities

1. Predict the output for the typed lines hello, scores, and quit.
2. Change the sentinel to `exit` and update the prompt.
3. Count the messages entered and print the count at the end.
4. Rewrite the counting loop as `for count in 1..=5` and compare.

Accessibility Practice

Each prompt is on its own line and each response begins with `You said:`, so the conversation can be followed in order by ear. If a loop floods the screen reader, stop it with Ctrl+C.

Skills Checklist

- ☐ I can write a counting `while` loop.
- ☐ I can write a sentinel loop.
- ☐ I test for `None` as well as the sentinel.
- ☐ I update the loop variable.
- ☐ I can use `while let`.

Lesson 5.3: Flags and loop exits

Reading References

- The Rust Programming Language: Loop Labels to Disambiguate Between Multiple Loops
- Rust standard library documentation
- Matching Python lesson: Lesson 5.3: Using Flags and break to Exit Loops

Learning Objectives

- Control a loop with a Boolean flag.
- Leave a loop with `break` and an outer loop with a label.
- Search a collection and remember the position.

Concept Explanation

A *flag* is a `bool` variable that controls a loop: the loop runs while `running` is true, and the body sets it to `false` to stop. `break` leaves the nearest enclosing loop at once.

A search keeps its answer in an `Option<usize>` (`None` means "not found"), sets it when the item is found, and uses `break`. After the loop, the `Option` says whether a match was found. Positions start at 0. The iterator method `position` does the same search in one line.

A *label* lets `break` leave an outer loop from inside an inner one: write `'outer:` before the outer loop and `break 'outer;` inside.

The shared scores 82, 91, 76, 88 have their first score of 90 or higher at index 1, with the value 91.

Worked Example

```
use std::io;

fn next_line() -> Option<String> {
    let mut line = String::new();
    match io::stdin().read_line(&mut line) {
        Ok(0) | Err(_) => None,
        Ok(_) => Some(line.trim().to_string()),
    }
}

fn main() {
    let mut running = true;
    let mut handled = 0;
    while running {
        println!("Say something, or type quit to stop:");
        match next_line() {
            Some(message) if message != "quit" => {
                handled += 1;
                println!("You said: {message}");
            }
            _ => running = false,
        }
    }
    println!("Messages handled: {handled}");

    let scores = [82, 91, 76, 88];
    let mut found_index = None;
    for (index, &score) in scores.iter().enumerate() {
        if score >= 90 {
            found_index = Some(index);
            break;
        }
    }
    match found_index {
        Some(index) => println!("First score of 90 or higher: {} at index {}", scores[index]
        None => println!("No score of 90 or higher."),
    }
    println!("{:?}", scores.iter().position(|&s| s >= 90));

    'outer: for row in 1..=3 {
        for col in 1..=3 {
            if row * col == 4 {
                println!("Stopped at row {row}, column {col}");
                break 'outer;
```

```
            }
        }
    }
}
```

Expected Behavior or Output

With the following lines typed as standard input (piped input is not echoed, so it does not appear in the output):

```
hello
quit
ignored
```

the program prints:

```
Say something, or type quit to stop:
You said: hello
Say something, or type quit to stop:
Messages handled: 1
First score of 90 or higher: 91 at index 1
Some(1)
Stopped at row 2, column 2
```

With no standard input at all, the program ends cleanly and prints:

```
Say something, or type quit to stop:
Messages handled: 0
First score of 90 or higher: 91 at index 1
Some(1)
Stopped at row 2, column 2
```

Common Mistakes

1. Setting the flag but running more code in the same pass that should be skipped.
2. Forgetting to give the result variable a starting value (`None`), then misreading it after the loop.
3. Using `break` in an inner loop and expecting it to leave the outer loop.

Activities

1. Run with hello and quit and confirm the count is 1.
2. Change the search to the first score below 80 and predict the index.
3. Rewrite the first loop without a flag, using `break`.
4. Explain in two sentences when a flag is clearer than `break`.

Accessibility Practice

Report search results as complete sentences that include both the value and its position. Remember that Rust indexes start at 0, and say 'index 1' aloud to avoid confusing it with 'the first'.

Skills Checklist

- ☐ I can control a loop with a flag.
- ☐ I can use `break`.
- ☐ I can use a label to leave an outer loop.
- ☐ I can search with an `Option`.
- ☐ I can explain flag versus `break`.

Lesson 5.4: Continue and filtering

Reading References

- The Rust Programming Language
- Rust standard library
- Cargo Book
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain Continue and filtering using Rust's types, ownership model, and standard-library conventions.
- Select an idiomatic Rust construct rather than transliterating Python syntax.
- Predict, compile, and verify the example's behavior.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

Read input into String, trim it, then parse. loop, while, and iterator loops express different repetition patterns. Every loop needs a clear termination condition; break and continue should make control flow easier to reason about. This lesson retains the canonical course data so learners practice Rust rather than a new domain problem. Use cargo fmt, cargo check, and cargo test as routine development tools.

Worked Example

```
fn main() { let values=[-2,3,0,7,-1,9]; let mut positives=Vec::new(); for v in values {if v<
```

Expected behavior: Runs independently with the standard library and prints the result of this lesson's focused operation. Predict the output, then compare it with the terminal output.

Common Mistakes

1. Failing to trim the newline before parsing input.
2. Choosing a sentinel that is also valid data without documenting it.
3. Creating a loop whose state is not updated on every path.

Activities

1. Run the worked example unchanged; trace each binding, expression, ownership transfer, and output line.
2. Change the canonical scores [82, 91, 76, 88] and predict the result before running it.
3. Add one boundary case and one invalid or missing-data case; state expected behavior before implementing the change.
4. Refactor one part using the Rust feature named in this lesson (continue and filtering); run cargo check and explain the diagnostic output.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect punctuation in ownership and borrowing expressions (&, &mut, ::, ->, ?). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order and verify that output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain continue and filtering using Rust terminology.
- ☐ I can compile and run the example with cargo run.
- ☐ I can explain the ownership, borrowing, and type decisions used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 5.5: Preventing infinite loops

Reading References

- The Rust Programming Language: Repetition with Loops
- Rust standard library documentation
- Matching Python lesson: Lesson 5.5: Avoiding Infinite Loops

Learning Objectives

- Identify what makes a loop stop.
- Add an attempt limit and handle the end of input.
- Write a loop that runs at least once.

Concept Explanation

An *infinite loop* never ends because its condition never becomes false. Common causes are forgetting to change the loop variable, testing the wrong variable, and waiting for input that never arrives. In a terminal, stop a running program with Ctrl+C. Rust's `loop { ... }` is an *intentional* endless loop and must contain a `break` or `return`.

Protect every loop with a *progress rule* and a *limit*: count attempts, and stop when the limit is reached or the input ends. Rust has no `do ... while`; to run a body at least once, use `loop` and put the exit test at the end.

The example allows three attempts to enter a score from 0 to 100. With the typed lines abc, 105, and 88, the first two are rejected and the third is accepted.

Worked Example

```rust
use std::io;

fn next_line() -> Option<String> {
    let mut line = String::new();
    match io::stdin().read_line(&mut line) {
        Ok(0) | Err(_) => None,
        Ok(_) => Some(line.trim().to_string()),
    }
}

fn main() {
    let max_attempts = 3;
    let mut attempts = 0;
    let mut score: Option<i32> = None;

    while attempts < max_attempts && score.is_none() {
        attempts += 1;
        println!("Enter a score from 0 to 100 (attempt {attempts} of {max_attempts}):");
        let Some(text) = next_line() else {
            println!("No more input.");
            break;
        };
        match text.parse::<i32>() {
            Ok(number) if (0..=100).contains(&number) => score = Some(number),
            _ => println!("Invalid score: {text}"),
```

```
        }
    }

    match score {
        Some(score) => println!("Accepted score: {score}"),
        None => println!("No valid score was entered."),
    }

    let mut count = 1;
    loop {
        println!("{count}");
        count += 1;
        if count > 3 {
            break;
        }
    }
}
```

Expected Behavior or Output

With the following lines typed as standard input (piped input is not echoed, so it does not appear in the output):

```
abc
105
88
```

the program prints:

```
Enter a score from 0 to 100 (attempt 1 of 3):
Invalid score: abc
Enter a score from 0 to 100 (attempt 2 of 3):
Invalid score: 105
Enter a score from 0 to 100 (attempt 3 of 3):
Accepted score: 88
1
2
3
```

With no standard input at all, the program ends cleanly and prints:

```
Enter a score from 0 to 100 (attempt 1 of 3):
No more input.
No valid score was entered.
1
2
3
```

Common Mistakes

1. Forgetting to increase `attempts`, so the limit is never reached.
2. Writing `loop { ... }` with no `break` or `return`, which never ends.
3. Looking for a `do ... while` loop. Use `loop` with the test at the end.

Activities

1. Run with abc, 105, and 88 and confirm the accepted score.
2. Run with three invalid lines and read the final message.
3. Change the limit to 5 and test it.
4. For each loop in the example, write down what makes it stop.

Accessibility Practice

A loop that cannot be stopped is an accessibility problem, because it can flood a screen reader with output. Learn Ctrl+C before running loops, give every input loop a limit, and write error messages that say what is valid.

Skills Checklist

☐ I can name what ends each loop.
☐ I can add an attempt limit.
☐ I handle the end of input.
☐ I can write a loop that runs at least once.
☐ I know how to interrupt a program.

14.8 Unit 6: Functions

Overview

This unit develops functions through Rust's type system and standard library. Prerequisites: prior units, a working Cargo project, and the ability to read compiler diagnostics. Preserve shared course datasets and test boundary conditions explicitly.

Lesson 6.1: Defining and calling functions

Reading References

- The Rust Programming Language
- Rust standard library
- Cargo Book
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain Defining and calling functions using Rust's types, ownership model, and standard-library conventions.
- Select an idiomatic Rust construct rather than transliterating Python syntax.
- Predict, compile, and verify the example's behavior.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

Functions declare parameter and return types. Borrow &T for read-only access and &mut T for exclusive mutation; accept owned values when retaining or transforming ownership. Generics express behavior over multiple types while preserving static checking. This lesson retains the canonical course data so learners practice Rust rather than a new domain problem. Use cargo fmt, cargo check, and cargo test as routine development tools.

Worked Example

```rust
fn greet(name:&str) {println!("Hello, {name}!");} fn main() {greet("Avery");}
```

Expected behavior: Runs independently with the standard library and prints the result of this lesson's focused operation. Predict the output, then compare it with the terminal output.

Common Mistakes

1. Returning a reference that outlives its owner.
2. Taking ownership when a shared borrow is sufficient.
3. Adding a semicolon to the final expression that should be returned.

Activities

1. Run the worked example unchanged; trace each binding, expression, ownership transfer, and output line.
2. Change the canonical scores [82, 91, 76, 88] and predict the result before running it.
3. Add one boundary case and one invalid or missing-data case; state expected behavior before implementing the change.
4. Refactor one part using the Rust feature named in this lesson (defining and calling functions); run cargo check and explain the diagnostic output.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect punctuation in ownership and borrowing expressions (&, &mut, ::, ->, ?). Do not rely on syntax high-

lighting or color to distinguish errors. Read prompts and validation messages in document order and verify that output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain defining and calling functions using Rust terminology.
- ☐ I can compile and run the example with cargo run.
- ☐ I can explain the ownership, borrowing, and type decisions used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 6.2: Parameters, ownership, and borrowing

Reading References

- The Rust Programming Language: What Is Ownership?
- The Rust Programming Language: References and Borrowing
- Rust standard library documentation
- Matching Python lesson: Lesson 6.2: Passing Arguments

Learning Objectives

- Explain moving versus borrowing a value.
- Use a struct with `Default` in place of default and named arguments.
- Pass a reference to read data without taking it.

Concept Explanation

Rust has no default arguments and no named arguments. A struct with `Default` gives both: `TipOptions { bill: 24.5, ..Default::default() }` names each field and fills the rest from the default, here a tip rate of 0.15.

Passing a value to a function normally **moves** it: the function owns it, and the caller can no longer use it. Passing a **reference**, `&name`, lets the function *borrow* the value to read it, and the caller keeps ownership. `&mut` borrows it for changing, and Rust allows either many readers or one writer at a time.

With the shared bill of 24.50, the total at the default rate 0.15 is 28.18, at 0.20 it is 29.40, and at 0.10 it is 26.95.

Worked Example

```
struct TipOptions {
    bill: f64,
    tip_rate: f64,
}
```

```
impl Default for TipOptions {
    fn default() -> Self {
        TipOptions { bill: 0.0, tip_rate: 0.15 }
    }
}

fn calculate_total(options: &TipOptions) -> f64 {
    options.bill + options.bill * options.tip_rate
}

fn borrow_length(text: &String) -> usize {
    text.len()
}

fn take_ownership(text: String) {
    println!("The function now owns {text}");
}

fn main() {
    let default_rate = TipOptions { bill: 24.5, ..Default::default() };
    let high = TipOptions { bill: 24.5, tip_rate: 0.20 };
    let low = TipOptions { tip_rate: 0.10, bill: 24.5 };
    println!("{:.2}", calculate_total(&default_rate));
    println!("{:.2}", calculate_total(&high));
    println!("{:.2}", calculate_total(&low));

    let name = String::from("Avery");
    let length = borrow_length(&name);
    println!("{name} has {length} letters");
    take_ownership(name);
    // println!("{name}");   // would not compile: name was moved
}
```

Expected Behavior or Output

The program prints:

```
28.18
29.40
26.95
Avery has 5 letters
The function now owns Avery
```

Common Mistakes

1. Using a value after passing it by value to a function, which is a compile error ("borrow of moved value").
2. Taking `&String` when `&str` would accept more kinds of text.
3. Expecting default or named arguments. Use a struct with `Default`.

Activities

1. Predict the output before running.
2. Uncomment the last line, read the compiler message, and restore the comment.
3. Change `borrow_length` to take `&str` and call it with a literal and with a `String`.
4. Explain in two sentences the difference between moving and borrowing.

Accessibility Practice

The compiler message for a moved value names the line where the move happened and the line where the value was used again, so read both. Say 'ampersand' aloud for a borrow and check that your screen reader announces it.

Skills Checklist

- ☐ I can explain move versus borrow.
- ☐ I can pass a reference.
- ☐ I can use a struct with `Default`.
- ☐ I can read a borrow-checker message.
- ☐ I know Rust has no default arguments.

Lesson 6.3: Return values and expressions

Reading References

- The Rust Programming Language: Functions with Return Values
- Rust standard library documentation
- Matching Python lesson: Lesson 6.3: Return Values

Learning Objectives

- Return a computed value as the final expression.
- Return an `Option` when a value may be missing.
- Return several values with a tuple or a struct.

Concept Explanation

A function's body ends with an *expression* **without a semicolon**, and that is its return value: `fn classify(score: f64) -> &'static str { if score >= 90.0 { "A" } else { "F" } }`. `return value;` leaves early. Adding a semicolon to the last line turns it into a statement and the function returns `()`, which the compiler rejects when a value is expected.

When a result may not exist, such as the average of an empty slice, return an `Option<f64>`: `Some(x)` or `None`. Rust has no null, so the caller must handle both cases. To return several values, return a tuple or a small struct.

The example returns the average of the shared scores, classifies the average, returns a minimum and maximum together, and shows the empty case.

Worked Example

```
struct ScoreRange {
    low: i32,
    high: i32,
}

fn average(scores: &[i32]) -> Option<f64> {
    if scores.is_empty() {
        return None;
    }
    let total: i32 = scores.iter().sum();
    Some(total as f64 / scores.len() as f64)
}

fn classify(score: f64) -> &'static str {
    if score >= 90.0 {
        "A"
    } else if score >= 80.0 {
        "B"
    } else if score >= 70.0 {
        "C"
    } else if score >= 60.0 {
        "D"
    } else {
        "F"
    }
}

fn min_max(scores: &[i32]) -> ScoreRange {
    let low = *scores.iter().min().unwrap_or(&0);
    let high = *scores.iter().max().unwrap_or(&0);
```

```
    ScoreRange { low, high }
}

fn main() {
    let scores = [82, 91, 76, 88];
    let avg = average(&scores);
    println!("Average: {:?}", avg);
    if let Some(value) = avg {
        println!("Grade for the average: {}", classify(value));
    }
    let range = min_max(&scores);
    println!("Lowest and highest: {} and {}", range.low, range.high);
    println!("{:?}", average(&[]));
}
```

Expected Behavior or Output

The program prints:

```
Average: Some(84.25)
Grade for the average: B
Lowest and highest: 76 and 91
None
```

Common Mistakes

1. Adding a semicolon to the last expression, so the function returns `()` instead of the value.
2. Calling `unwrap()` on an empty `Option` and crashing.
3. Printing inside the function and then trying to use the result in a calculation.

Activities

1. Predict the four printed lines before running.
2. Write `total(scores: &[i32]) -> i32` that returns the sum.
3. Write `is_passing(score: i32) -> bool`.
4. Explain how a caller should use a function that returns `Option<f64>`.

Accessibility Practice

Return values instead of printing inside logic functions, so the caller prints once and the screen reader announces each result a single time.

Skills Checklist

- ☐ I can return the final expression.
- ☐ I can return an `Option`.

- ☐ I can return several values.
- ☐ I know a trailing semicolon changes the meaning.
- ☐ I can classify an average with the shared thresholds.

Lesson 6.4: Functions over collections

Reading References

- The Rust Programming Language: References and Borrowing
- The Rust Programming Language: The Slice Type
- Rust standard library documentation
- Matching Python lesson: Lesson 6.4: Functions with Lists and Dictionaries

Learning Objectives

- Pass a slice to read and `&mut Vec` to change.
- Return a new vector and leave the original unchanged.
- Process the shared inventory with a function.

Concept Explanation

A function that only **reads** a list takes a *slice*, `&[i32]`, which accepts a vector, an array, or part of either without copying. A function that **changes** a vector takes `&mut Vec<i32>`, and the caller passes `&mut scores`; the call site shows that a change will happen. A function can also take ownership and return a new vector.

Rust enforces the rules at compile time: while a function holds a `&mut`, nothing else may read the vector. That removes a whole class of bugs that other languages find at run time.

The shared inventory is Notebook (3 at 2.50), Pencil (10 at 0.50), and Folder (4 at 1.25), worth 17.50 in total.

Worked Example

```rust
struct Item {
    name: String,
    quantity: u32,
    unit_price: f64,
}

fn add_score(score_list: &mut Vec<i32>, new_score: i32) {
    score_list.push(new_score);
}
```

```
fn with_score(score_list: &[i32], new_score: i32) -> Vec<i32> {
    let mut copy = score_list.to_vec();
    copy.push(new_score);
    copy
}

fn inventory_value(inventory: &[Item]) -> f64 {
    inventory.iter().map(|item| item.quantity as f64 * item.unit_price).sum()
}

fn main() {
    let mut scores = vec![82, 91, 76, 88];
    add_score(&mut scores, 95);
    println!("{:?}", scores);

    let longer = with_score(&scores, 100);
    println!("{} {}", scores.len(), longer.len());

    let inventory = vec![
        Item { name: "Notebook".to_string(), quantity: 3, unit_price: 2.50 },
        Item { name: "Pencil".to_string(), quantity: 10, unit_price: 0.50 },
        Item { name: "Folder".to_string(), quantity: 4, unit_price: 1.25 },
    ];
    println!("Total inventory value: {:.2}", inventory_value(&inventory));
    println!("{}", inventory.iter().map(|item| item.name.as_str()).collect::<Vec<_>>().join(
}
```

Expected Behavior or Output

The program prints:

```
[82, 91, 76, 88, 95]
5 6
Total inventory value: 17.50
Notebook, Pencil, Folder
```

Common Mistakes

1. Passing `&scores` to a function that needs `&mut Vec<i32>`, which is a compile error.
2. Taking `&Vec<i32>` in a function that only reads, instead of the more flexible `&[i32]`.
3. Holding a reference to an element while calling a function that pushes to the vector.

Activities

1. Predict the four printed lines before running.
2. Write `scale_prices(inventory: &mut [Item], factor: f64)` that changes prices in place.
3. Write a version that returns a new vector and leaves the original unchanged.
4. Explain why `&mut` appears at the call site.

Accessibility Practice

State aloud before each call whether the function changes its argument: `&mut` at the call site and a name such as `add_score` both say so. Print the list before and after so the effect is heard.

Skills Checklist

☐ I can pass a slice to read.
☐ I can pass `&mut Vec` to change.
☐ I can return a new vector.
☐ I can explain why `&mut` is written at the call.
☐ I can total the shared inventory.

Lesson 6.5: Scope, generics, and variadic alternatives

Reading References

- The Rust Programming Language: Generic Data Types
- Rust by Example: macro_rules!
- Rust standard library documentation
- Matching Python lesson: Lesson 6.5: Arbitrary Number of Arguments

Learning Objectives

- Take any number of values with a slice.
- Write a generic function over a type parameter.
- Explain scope and a variadic macro.

Concept Explanation

Rust functions have a fixed number of parameters; there are no variadic functions. The usual replacement is a **slice**: `average_of(&[82, 91, 76, 88])` takes any number of values. A *macro* defined with `macro_rules!` can accept a repeated pattern, which is how `println!` and `vec!` take any number of arguments.

A *generic* function works for many types: `fn largest<T: PartialOrd + Copy>(items: &[T]) -> T` accepts any type that can be compared and copied. *Scope* is the region where a name is visible: a `{ ... }` block creates a scope, and a name declared inside ends at the closing brace. An inner binding with the same name *shadows* the outer one.

The shared scores average 84.25, and the average of 90 and 80 is 85.

Worked Example

```
macro_rules! average {
    ($($x:expr),+ $(,)?) => {{
        let values = [$($x as f64),+];
        values.iter().sum::<f64>() / values.len() as f64
    }};
}

fn average_of(scores: &[i32]) -> Option<f64> {
    if scores.is_empty() {
        None
    } else {
        Some(scores.iter().sum::<i32>() as f64 / scores.len() as f64)
    }
}

fn largest<T: PartialOrd + Copy>(items: &[T]) -> T {
    let mut best = items[0];
    for &item in items {
        if item > best {
            best = item;
        }
    }
    best
}

fn main() {
    println!("{}", average!(82, 91, 76, 88));
    println!("{}", average!(90, 80));
    println!("{:?}", average_of(&[82, 91, 76, 88]));
    println!("{:?}", average_of(&[]));

    println!("{} {} {}", largest(&[82, 91, 76]), largest(&[2.5, 0.5]), largest(&['a', 'z',

    let value = 1;
    {
        let value = value + 41;
```

```
        println!("inner {value}");
    }
    println!("outer {value}");
}
```

Expected Behavior or Output

The program prints:

```
84.25
85
Some(84.25)
None
91 2.5 z
inner 42
outer 1
```

Common Mistakes

1. Looking for a variadic function. Use a slice or a macro.
2. Calling `largest(&[])` on an empty slice, which panics because it reads `items[0]`.
3. Forgetting the trait bounds on `T`, so the comparison does not compile.

Activities

1. Predict each printed line.
2. Write `maximum_of(scores: &[i32]) -> Option<i32>` that handles an empty slice.
3. Call `largest` with a slice of words (`&str`) and explain why it works.
4. Explain scope and shadowing using the last lines of the example.

Accessibility Practice

Read macro and generic syntax aloud one piece at a time: 'fn largest, angle bracket T colon PartialOrd plus Copy, close angle bracket'. Check that your screen reader announces the angle brackets and the plus sign.

Skills Checklist

- ☐ I can take many values with a slice.
- ☐ I can write a generic function.
- ☐ I understand scope and shadowing.
- ☐ I can read a macro call.
- ☐ I handle the empty case.

14.9 Unit 7: Data Modeling and Traits

Overview

This unit develops data modeling and traits through Rust's type system and standard library. Prerequisites: prior units, a working Cargo project, and the ability to read compiler diagnostics. Preserve shared course datasets and test boundary conditions explicitly.

Lesson 7.1: Structs, enums, and data modeling

Reading References

- The Rust Programming Language: Defining and Instantiating Structs
- The Rust Programming Language: Defining an Enum
- The Rust Programming Language: Modules and Privacy
- Rust standard library documentation
- Matching Python lesson: Lesson 7.1: Why Learn About Classes?

Learning Objectives

- Model data with a struct and with an enum.
- Hide a field with module privacy and change it only through methods.
- Explain that Rust has no classes and no inheritance.

Concept Explanation

Rust has no classes. Data is modeled with a **struct** (named fields) or an **enum** (a value that is exactly one of several variants), and behavior is added in `impl` blocks (Lesson 7.3) and *traits* (Lesson 7.4). Bundling data with the functions that work on it, and hiding the details, is still called **encapsulation**.

Privacy in Rust is controlled by *modules*. Fields and functions are private to the module they are written in unless marked `pub`. In the example, `mod inventory` holds an `Item` whose `quantity` field is private: code outside the module can read it only through the `quantity()` method and change it only through `restock`.

`match` on an enum must cover every variant, so adding a new variant later makes the compiler point at every place that needs a decision. The shared Notebook has quantity 3; restocking 2 gives 5.

Worked Example

```
mod inventory {
    pub struct Item {
        pub name: String,
        quantity: u32,
        pub unit_price: f64,
    }
```

```
    impl Item {
        pub fn new(name: &str, quantity: u32, unit_price: f64) -> Item {
            Item { name: name.to_string(), quantity, unit_price }
        }

        pub fn quantity(&self) -> u32 {
            self.quantity
        }

        pub fn restock(&mut self, amount: u32) {
            self.quantity += amount;
        }
    }
}

#[derive(Debug)]
enum Stock {
    Out,
    Low,
    Enough,
}

fn status(quantity: u32) -> Stock {
    match quantity {
        0 => Stock::Out,
        1..=4 => Stock::Low,
        _ => Stock::Enough,
    }
}

fn main() {
    let mut notebook = inventory::Item::new("Notebook", 3, 2.50);
    println!("{}: {} at {} is {:?}", notebook.name, notebook.quantity(), notebook.unit_price
    notebook.restock(2);
    println!("{}: {} is now {:?}", notebook.name, notebook.quantity(), status(notebook.quant
    println!("{:?}", status(0));
    // notebook.quantity = 99;   // would not compile: the field is private
}
```

Expected Behavior or Output

The program prints:

```
Notebook: 3 at 2.5 is Low
Notebook: 5 is now Enough
```

```
Out
```

Common Mistakes

1. Looking for a `class` keyword. Rust uses structs, enums, impl blocks, and traits.
2. Making every field `pub`, so outside code can put the value into an invalid state.
3. Forgetting a variant in a `match` on an enum, which the compiler rejects.

Activities

1. Predict the three printed lines before running.
2. Add a variant `Overstocked` for quantities above 100 and fix every `match` the compiler points to.
3. Uncomment the assignment, read the compiler message, and restore the comment.
4. Explain what encapsulation means and how modules provide it.

Accessibility Practice

Describe a data type as a list read in order: the name, then each field, then each variant. Reading a struct aloud works best by saying 'pub struct Item, open brace, pub name colon String comma...' and listening for the braces that show structure.

Skills Checklist

- ☐ I can model data with a struct and an enum.
- ☐ I can hide a field with a module.
- ☐ I know Rust has no classes.
- ☐ I can read a struct definition aloud.
- ☐ I can match every enum variant.

Lesson 7.2: Creating instances and methods

Reading References

- The Rust Programming Language: Method Syntax
- The Rust Programming Language: Recoverable Errors with Result
- Rust standard library documentation
- Matching Python lesson: Lesson 7.2: Creating and Using a Class

Learning Objectives

- Write a constructor function that validates and returns a `Result`.

- Create several instances with the same fields.
- Handle the error when a value is invalid.

Concept Explanation

Rust has no constructors in the language; by convention an *associated function* named `new` builds a value: `Item::new("Notebook", 3, 2.50)`. Because Rust has no exceptions, a constructor that can fail returns a `Result<Item, String>`: `Ok(item)` for success or `Err(message)` for a clear failure. The caller must handle both cases, usually with `match` or `?`.

An `impl Item { ... }` block holds the functions. A method takes `&self` to read the value or `&mut self` to change it. The example creates the three shared inventory items and then tries to create an invalid one.

Worked Example

```
struct Item {
    name: String,
    quantity: i32,
    unit_price: f64,
}

impl Item {
    fn new(name: &str, quantity: i32, unit_price: f64) -> Result<Item, String> {
        if quantity < 0 {
            return Err("quantity must not be negative".to_string());
        }
        if unit_price < 0.0 {
            return Err("unit price must not be negative".to_string());
        }
        Ok(Item { name: name.to_string(), quantity, unit_price })
    }

    fn total_value(&self) -> f64 {
        self.quantity as f64 * self.unit_price
    }
}

fn main() {
    let notebook = Item::new("Notebook", 3, 2.50).unwrap();
    let pencil = Item::new("Pencil", 10, 0.50).unwrap();
    let folder = Item::new("Folder", 4, 1.25).unwrap();

    for item in [&notebook, &pencil, &folder] {
        println!("{}: {} at {:.2}, value {:.2}", item.name, item.quantity, item.unit_price,
    }
```

```
    match Item::new("Broken", -1, 1.0) {
        Ok(item) => println!("Created {}", item.name),
        Err(message) => println!("Could not create the item: {message}"),
    }
}
```

Expected Behavior or Output

The program prints:

```
Notebook: 3 at 2.50, value 7.50
Pencil: 10 at 0.50, value 5.00
Folder: 4 at 1.25, value 5.00
Could not create the item: quantity must not be negative
```

Common Mistakes

1. Calling `unwrap()` on a `Result` from untrusted data, which panics on `Err`.
2. Forgetting `&self`, so the function is associated and cannot be called with the dot syntax.
3. Ignoring the `Err` case. The compiler warns about an unused `Result`.

Activities

1. Create a fourth item with your own values and print it.
2. Add a rule that the name must not be blank.
3. Create two items with the same name and different quantities and show that they are separate values.
4. Explain the difference between `Item::new` and `item.total_value()`.

Accessibility Practice

Say the signature aloud in order: 'fn new, open parenthesis, name colon ampersand str...' and check that your screen reader announces the arrow and the `Result` type. Print each item on its own line so output can be matched to values by ear.

Skills Checklist

- ☐ I can write a `new` function.
- ☐ I can return a `Result` from a constructor.
- ☐ I can create several instances.
- ☐ I can handle the `Err` case.
- ☐ I can tell `&self` from `&mut self`.

Lesson 7.3: Associated functions and impl blocks

Reading References

- The Rust Programming Language: Method Syntax
- The Rust Programming Language: Implementing a Trait on a Type
- Rust standard library documentation
- Matching Python lesson: Lesson 7.3: Working with Attributes and Methods

Learning Objectives

- Write methods that read, change, and consume a value.
- Define an associated constant.
- Implement `Display` so a value prints as a sentence.

Concept Explanation

An `impl` block can hold three kinds of function. A method with **`&self`** reads the value, a method with **`&mut self`** changes it, and a method with **`self`** *consumes* it, so the value cannot be used afterward. A function without a `self` parameter, such as `new`, is an *associated function* called with `Item::new`. An *associated constant* such as `const LOW_STOCK: u32 = 5;` belongs to the type and is read as `Item::LOW_STOCK`.

`impl fmt::Display for Item` teaches `println!("{}", item)` how to print the value, so it prints as a useful sentence. The example adds a computed value, a low-stock check, a restock method with validation, a consuming summary, and `Display` to the shared Notebook.

Worked Example

```rust
use std::fmt;

struct Item {
    name: String,
    quantity: u32,
    unit_price: f64,
}

impl Item {
    const LOW_STOCK: u32 = 5;

    fn new(name: &str, quantity: u32, unit_price: f64) -> Item {
        Item { name: name.to_string(), quantity, unit_price }
    }

    fn total_value(&self) -> f64 {
```

```
        self.quantity as f64 * self.unit_price
    }

    fn is_low_stock(&self) -> bool {
        self.quantity < Item::LOW_STOCK
    }

    fn restock(&mut self, amount: i32) {
        if amount < 0 {
            println!("Cannot restock a negative amount.");
            return;
        }
        self.quantity += amount as u32;
    }

    fn into_summary(self) -> String {
        format!("{} x{}", self.name, self.quantity)
    }
}

impl fmt::Display for Item {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "{}: {} at {:.2} each, value {:.2}", self.name, self.quantity, self.unit_p
    }
}

fn main() {
    let mut notebook = Item::new("Notebook", 3, 2.50);
    println!("{notebook}");
    println!("Low stock: {}", notebook.is_low_stock());
    notebook.restock(2);
    println!("{notebook}");
    println!("Low stock: {}", notebook.is_low_stock());
    notebook.restock(-1);
    println!("{}", notebook.into_summary());
}
```

Expected Behavior or Output

The program prints:

```
Notebook: 3 at 2.50 each, value 7.50
Low stock: true
Notebook: 5 at 2.50 each, value 12.50
Low stock: false
Cannot restock a negative amount.
```

```
Notebook x5
```

Common Mistakes

1. Using a value after calling a method that takes `self`, which is a compile error because the value was consumed.
2. Calling a `&mut self` method on a binding that is not `mut`.
3. Forgetting `use std::fmt;` or the `fmt::Result` return type when implementing `Display`.

Activities

1. Predict the six printed lines before running.
2. Add a method `discounted(&self, rate: f64) -> f64` that returns the value after a discount without changing the item.
3. Try to print `notebook` after `into_summary` and read the compiler message.
4. Explain the difference between `&self`, `&mut self`, and `self`.

Accessibility Practice

Name methods after actions (`restock`) so a call reads as a sentence. A good `Display` gives the screen reader one complete line per value; say 'ampersand self' and 'self' aloud to hear the difference between a borrow and a move.

Skills Checklist

- ☐ I can write read, change, and consume methods.
- ☐ I can define an associated constant.
- ☐ I can implement `Display`.
- ☐ I know a `self` method consumes the value.
- ☐ I can validate in a method.

Lesson 7.4: Traits and polymorphic behavior

Reading References

- The Rust Programming Language: Traits: Defining Shared Behavior
- The Rust Programming Language: Using Trait Objects
- Rust standard library documentation
- Matching Python lesson: Lesson 7.4: Inheritance

Learning Objectives

- Define a trait with a default method.
- Implement the trait for two types.
- Explain polymorphism with generics and trait objects.

Concept Explanation

Rust has no inheritance. Shared behavior is described by a **trait**: a set of methods that a type promises to provide. A trait may supply a *default method*, here `describe`, which calls the trait's other methods. Each type writes `impl Priced for Item { ... }` to implement the trait, and may override a default.

Polymorphism comes in two forms. A *generic* function, `fn print_total<T: Priced>(item: &T)`, is compiled for each type it is used with. A *trait object*, `Box<dyn Priced>`, lets one vector hold values of different types and picks the method at run time. A struct can hold another struct as a field (composition) instead of inheriting from it.

The example discounts the shared Folder (4 at 1.25) by 20 percent, so its value is 4.00 instead of 5.00.

Worked Example

```
trait Priced {
    fn name(&self) -> &str;
    fn total_value(&self) -> f64;

    fn describe(&self) -> String {
        format!("{}: value {:.2}", self.name(), self.total_value())
    }
}

struct Item {
    name: String,
    quantity: u32,
    unit_price: f64,
}

struct DiscountedItem {
    item: Item,
    discount_rate: f64,
}

impl Priced for Item {
    fn name(&self) -> &str {
        &self.name
    }

    fn total_value(&self) -> f64 {
        self.quantity as f64 * self.unit_price
    }
}
```

```rust
impl Priced for DiscountedItem {
    fn name(&self) -> &str {
        self.item.name()
    }

    fn total_value(&self) -> f64 {
        self.item.total_value() * (1.0 - self.discount_rate)
    }
}

fn print_total<T: Priced>(item: &T) {
    println!("Generic: {}", item.describe());
}

fn main() {
    let notebook = Item { name: "Notebook".to_string(), quantity: 3, unit_price: 2.50 };
    let folder = DiscountedItem {
        item: Item { name: "Folder".to_string(), quantity: 4, unit_price: 1.25 },
        discount_rate: 0.20,
    };

    print_total(&folder);

    let items: Vec<Box<dyn Priced>> = vec![Box::new(notebook), Box::new(folder)];
    for item in &items {
        println!("{}", item.describe());
    }
}
```

Expected Behavior or Output

The program prints:

```
Generic: Folder: value 4.00
Notebook: value 7.50
Folder: value 4.00
```

Common Mistakes

1. Looking for `extends`. Rust has no inheritance; use a trait or a field.
2. Forgetting to implement a required method, which the compiler reports by name.
3. Putting values of different types in a plain `Vec<T>`, which allows only one type; use `Vec<Box<dyn Trait>>`.

Activities

1. Predict the output before running.
2. Add a third type, `PerishableItem`, that implements `Priced` and override `describe`.
3. Explain why `describe` prints the discounted value without being rewritten.
4. Explain the difference between a generic function and a trait object.

Accessibility Practice

Describe a trait as a list of promised methods. Speaking `impl Priced for DiscountedItem` aloud ('implement Priced for DiscountedItem') is clearer than a diagram. Print one line per value so the screen reader announces each result in order.

Skills Checklist

- ☐ I can define a trait with a default method.
- ☐ I can implement a trait for two types.
- ☐ I can explain polymorphism.
- ☐ I know Rust has no inheritance.
- ☐ I can store trait objects in a vector.

Lesson 7.5: Composition and ownership

Reading References

- The Rust Programming Language: Structs
- The Rust Programming Language: Ownership
- Rust standard library documentation
- Matching Python lesson: Lesson 7.5: Instances as Attributes (Composition)

Learning Objectives

- Explain composition ("has a") and ownership of parts.
- Hold a vector of values inside a struct.
- Build an inventory from the shared items.

Concept Explanation

Composition means one value contains other values. An inventory *has* items; it is not a kind of item. In Rust a struct **owns** its fields: `Inventory { items: Vec<Item> }` owns the vector and every item in it. `add(&mut self, item:`

`Item)` takes the item by value, so ownership **moves** into the inventory and the caller can no longer use it.

Methods that look at the parts borrow: `find(&self, name)` returns `Option<&Item>`, a borrowed reference that lives no longer than the inventory. `remove` returns the owned item back as `Option<Item>`.

The example builds an `Inventory` with the three shared items and reports each quantity and the total value, 17.50.

Worked Example

```
struct Item {
    name: String,
    quantity: u32,
    unit_price: f64,
}

impl Item {
    fn total_value(&self) -> f64 {
        self.quantity as f64 * self.unit_price
    }
}

struct Inventory {
    items: Vec<Item>,
}

impl Inventory {
    fn new() -> Inventory {
        Inventory { items: Vec::new() }
    }

    fn add(&mut self, item: Item) {
        self.items.push(item);
    }

    fn find(&self, name: &str) -> Option<&Item> {
        self.items.iter().find(|item| item.name == name)
    }

    fn remove(&mut self, name: &str) -> Option<Item> {
        let position = self.items.iter().position(|item| item.name == name)?;
        Some(self.items.remove(position))
    }

    fn total_value(&self) -> f64 {
```

```
        self.items.iter().map(|item| item.total_value()).sum()
    }

    fn report(&self) {
        for item in &self.items {
            println!("{}: {}", item.name, item.quantity);
        }
        println!("Total value: {:.2}", self.total_value());
    }
}

fn main() {
    let mut inventory = Inventory::new();
    inventory.add(Item { name: "Notebook".to_string(), quantity: 3, unit_price: 2.50 });
    inventory.add(Item { name: "Pencil".to_string(), quantity: 10, unit_price: 0.50 });
    inventory.add(Item { name: "Folder".to_string(), quantity: 4, unit_price: 1.25 });
    inventory.report();

    println!("{:?}", inventory.find("Pencil").map(|item| item.quantity));
    println!("{}", inventory.find("Eraser").is_none());
    let removed = inventory.remove("Pencil");
    println!("{}", removed.map(|item| item.name).unwrap_or_default());
    println!("{}", inventory.items.len());
}
```

Expected Behavior or Output

The program prints:

```
Notebook: 3
Pencil: 10
Folder: 4
Total value: 17.50
Some(10)
true
Pencil
2
```

Common Mistakes

1. Using an item after passing it to `add`, which is a compile error because it was moved.
2. Returning a reference from `find` and then changing the inventory while still holding it.
3. Using inheritance thinking: an inventory is not a kind of item.

Activities

1. Predict the output.
2. Add a method that returns the names of items with a quantity below 5.
3. Try to use `notebook` after `inventory.add(notebook)` and read the compiler message.
4. Explain ”is a” versus ”has a” with two examples.

Accessibility Practice

Report the inventory as one labelled line per item and a final total line, so the screen reader reads a complete summary in order. Say 'ampersand' and 'ampersand mut' aloud so that borrows are heard.

Skills Checklist

- ☐ I can explain composition.
- ☐ I can hold a vector in a struct.
- ☐ I understand ownership moving into a container.
- ☐ I can return a borrowed `Option<&Item>`.
- ☐ I can build the shared inventory.

14.10 Unit 8: Files and Error Handling

Overview

This unit develops files and error handling through Rust's type system and standard library. Prerequisites: prior units, a working Cargo project, and the ability to read compiler diagnostics. Preserve shared course datasets and test boundary conditions explicitly.

Lesson 8.1: Reading text files

Reading References

- The Rust Programming Language
- Rust standard library
- Cargo Book
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain Reading text files using Rust's types, ownership model, and standard-library conventions.
- Select an idiomatic Rust construct rather than transliterating Python syntax.

- Predict, compile, and verify the example's behavior.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

File APIs return Result. The ? operator propagates compatible failures. RAII releases resources when owners leave scope. Use a maintained parser/serializer such as serde_json for production JSON; the standard-library-only example illustrates the serialized record shape. This lesson retains the canonical course data so learners practice Rust rather than a new domain problem. Use cargo fmt, cargo check, and cargo test as routine development tools.

Worked Example

```
use std::fs; fn main(){let p=std::env::temp_dir().join("rust_read_demo.txt");fs::write(&p,"A
```

Expected behavior: Demonstrates a scoped file operation in the system temporary directory and removes its demonstration file.

Common Mistakes

1. Using unwrap on production I/O paths instead of handling or propagating errors.
2. Assuming file handles require manual close; RAII drops them at scope exit.
3. Hand-building JSON without correct escaping; use serde_json in real projects.

Activities

1. Run the worked example unchanged; trace each binding, expression, ownership transfer, and output line.
2. Change the canonical inventory and CSV records and predict the result before running it.
3. Add one boundary case and one invalid or missing-data case; state expected behavior before implementing the change.
4. Refactor one part using the Rust feature named in this lesson (reading text files); run cargo check and explain the diagnostic output.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect punctuation in ownership and borrowing expressions (&, &mut, ::, ->, ?). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order and verify that output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain reading text files using Rust terminology.
- ☐ I can compile and run the example with cargo run.
- ☐ I can explain the ownership, borrowing, and type decisions used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 8.2: Writing and appending files

Reading References

- The Rust Programming Language
- Rust standard library
- Cargo Book
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain Writing and appending files using Rust's types, ownership model, and standard-library conventions.
- Select an idiomatic Rust construct rather than transliterating Python syntax.
- Predict, compile, and verify the example's behavior.
- Test a relevant boundary condition and explain a common failure.

Concept Explanation

File APIs return Result. The ? operator propagates compatible failures. RAII releases resources when owners leave scope. Use a maintained parser/serializer such as serde_json for production JSON; the standard-library-only example illustrates the serialized record shape. This lesson retains the canonical course data so learners practice Rust rather than a new domain problem. Use cargo fmt, cargo check, and cargo test as routine development tools.

Worked Example

```
use std::fs::OpenOptions;use std::io::Write;fn main(){let p=std::env::temp_dir().join("rust_
```

Expected behavior: Demonstrates a scoped file operation in the system temporary directory and removes its demonstration file.

Common Mistakes

1. Using unwrap on production I/O paths instead of handling or propagating errors.

2. Assuming file handles require manual close; RAII drops them at scope exit.
3. Hand-building JSON without correct escaping; use serde_json in real projects.

Activities

1. Run the worked example unchanged; trace each binding, expression, ownership transfer, and output line.
2. Change the canonical inventory and CSV records and predict the result before running it.
3. Add one boundary case and one invalid or missing-data case; state expected behavior before implementing the change.
4. Refactor one part using the Rust feature named in this lesson (writing and appending files); run cargo check and explain the diagnostic output.

Accessibility Practice

Use keyboard navigation to move between the lesson heading, code fence, compiler output, and terminal. With a screen reader, inspect punctuation in ownership and borrowing expressions (&, &mut, ::, ->, ?). Do not rely on syntax highlighting or color to distinguish errors. Read prompts and validation messages in document order and verify that output is announced without unexpected focus movement.

Skills Checklist

- ☐ I can explain writing and appending files using Rust terminology.
- ☐ I can compile and run the example with cargo run.
- ☐ I can explain the ownership, borrowing, and type decisions used here.
- ☐ I can test a boundary case and an invalid-input or error case.
- ☐ I can navigate source, diagnostics, and output with my keyboard and assistive technology.

Lesson 8.3: Result and error propagation

Reading References

- The Rust Programming Language: Recoverable Errors with Result
- The Rust Programming Language: To panic! or Not to panic!
- Rust standard library documentation
- Matching Python lesson: Lesson 8.3: Exceptions - Handling Errors

Learning Objectives

- Handle a `Result` with `match` and propagate it with `?`.
- Tell recoverable errors (`Result`) from panics.

- Use `Box<dyn Error>` to combine different error types.

Concept Explanation

Rust has no exceptions. A function that can fail returns `Result<T, E>`, and the caller must handle it: `match` on `Ok` and `Err`, `unwrap_or` to supply a default, or `?` to return the error to *its* caller. The `?` operator also converts the error: with a return type of `Result<T, Box<dyn Error>>`, `?` accepts both an `io::Error` and a `ParseIntError`.

A **panic** (`unwrap()` on an `Err`, an index out of range, integer division by zero) stops the program and is for bugs, not for expected problems like a missing file. `checked_div` returns an `Option` instead of panicking. The error kind of an I/O failure can be matched, for example `ErrorKind::NotFound`. Library error messages can depend on the operating system, so the example prints only messages that are the same everywhere.

Worked Example

```
use std::error::Error;
use std::fs;
use std::io::ErrorKind;
use std::num::ParseIntError;

fn parse_score(text: &str) -> Result<i32, ParseIntError> {
    let number = text.trim().parse::<i32>()?;
    Ok(number)
}

fn read_first_number(path: &str) -> Result<i32, Box<dyn Error>> {
    let text = fs::read_to_string(path)?;
    let first = text.lines().next().unwrap_or("");
    Ok(first.trim().parse::<i32>()?)
}

fn main() {
    match fs::read_to_string("missing_notes.txt") {
        Ok(_) => println!("The file was read."),
        Err(error) if error.kind() == ErrorKind::NotFound => println!("Error: missing_notes.
        Err(_) => println!("Error: the file could not be read."),
    }

    match parse_score("abc") {
        Ok(number) => println!("Score {number}"),
        Err(error) => println!("Error: 'abc' is not a valid score ({error})"),
    }
    println!("{:?}", parse_score(" 88 "));
```

```
    println!("{:?}", 10_i32.checked_div(0));
    println!("{}", 10_i32.checked_div(0).unwrap_or(0));

    match read_first_number("missing_notes.txt") {
        Ok(number) => println!("First number: {number}"),
        Err(_) => println!("Could not read a number from the file."),
    }
}
```

Expected Behavior or Output

The program prints:

```
Error: missing_notes.txt was not found.
Error: 'abc' is not a valid score (invalid digit found in string)
Ok(88)
None
0
Could not read a number from the file.
```

Common Mistakes

1. Calling `unwrap()` on a `Result` for expected failures, which turns them into panics.
2. Using `?` in a function whose return type is not a `Result` or `Option`, which is a compile error.
3. Printing an operating-system error message in an expected output, which differs between systems.

Activities

1. Predict each printed line before running.
2. Write a function that reads a file and returns the number of lines as a `Result`.
3. Use `?` in a function that parses two numbers and adds them.
4. Explain when to return a `Result` and when a panic is acceptable.

Accessibility Practice

Write error messages as complete sentences that name the problem and the next step, such as 'Enter a whole number from 0 to 100.' When a panic message appears, read the line that names your own file first.

Skills Checklist

- ☐ I can handle a `Result` with `match`.
- ☐ I can use `?` to pass errors up.

- ☐ I can tell a `Result` from a panic.
- ☐ I can use `Box<dyn Error>`.
- ☐ I can write clear error messages.

Lesson 8.4: Resource cleanup and RAII

Reading References

- The Rust Programming Language: Running Code on Cleanup with the Drop Trait
- Rust standard library documentation
- Matching Python lesson: Lesson 8.4: The else and finally Blocks

Learning Objectives

- Explain that Rust releases resources when a value goes out of scope.
- Implement `Drop` to see when cleanup runs.
- Close a file by letting it go out of scope.

Concept Explanation

Rust has no `finally`. Instead, every value is *dropped* (cleaned up) automatically when its owner goes out of scope, and values are dropped in **reverse order of creation**. This idea is called **RAII** (resource acquisition is initialization). A file handle closes itself when it is dropped, so a file cannot be left open by a forgotten call, even if the code returns early or a panic unwinds.

A type can run its own cleanup by implementing the `Drop` trait, whose `drop` method is called automatically. `drop(value)` ends a value's life early. A binding named `_name` stays alive until the end of its scope, but the bare pattern `_` drops the value immediately, which surprises many learners.

The example shows the order of cleanup and writes a file inside a block so that it is closed before it is read back.

Worked Example

```
use std::fs::{self, File};
use std::io::Write;

struct Guard(&'static str);

impl Drop for Guard {
    fn drop(&mut self) {
        println!("Releasing {}", self.0);
    }
}
```

```
fn main() {
    let _outer = Guard("outer resource");
    {
        let _inner = Guard("inner resource");
        println!("Using the inner resource");
    }
    println!("After the inner block");

    let early = Guard("early resource");
    drop(early);
    println!("After the explicit drop");

    {
        let mut file = File::create("cleanup_demo.txt").expect("could not create the file");
        file.write_all(b"Some data.\n").expect("could not write the file");
    }
    let text = fs::read_to_string("cleanup_demo.txt").expect("could not read the file");
    println!("{}", text.trim());
    println!("End of main");
}
```

Expected Behavior or Output

The program prints:

```
Using the inner resource
Releasing inner resource
After the inner block
Releasing early resource
After the explicit drop
Some data.
End of main
Releasing outer resource
```

Common Mistakes

1. Writing `let _ = Guard(...)` and expecting it to live until the end of the scope. It is dropped at once.
2. Expecting values to be dropped in the order they were created. The order is reversed.
3. Reading a file before the handle that wrote it has gone out of scope, which can miss buffered data.

Activities

1. Predict the eight printed lines, especially the order of the `Releasing` lines.

2. Change `_outer` to `_` and describe what changes.
3. Add a third guard and predict where its message appears.
4. Explain why Rust does not need a `finally` block.

Accessibility Practice

Print a short labelled line when each resource is released, so the screen reader announces the order of cleanup. Labelled lines make a rule that is normally invisible audible.

Skills Checklist

- ☐ I can explain scope-based cleanup.
- ☐ I can implement `Drop`.
- ☐ I know values drop in reverse order.
- ☐ I know a file closes when it is dropped.
- ☐ I know `_` and `_name` differ.

Lesson 8.5: JSON serialization and persistence

Reading References

- The Rust Programming Language: Packages and Crates
- serde_json documentation
- JSON specification: https://www.json.org
- Matching Python lesson: Lesson 8.5: Storing Data with JSON

Learning Objectives

- Explain why this lesson includes a JSON helper.
- Encode and decode the shared task records.
- Handle a missing or malformed file.

Concept Explanation

The Rust standard library has no JSON support. Real projects add the `serde` and `serde_json` crates through Cargo. To keep this course free of dependencies, the example includes a *teaching implementation*: an `enum Json` with an `encode` function and a `decode` function built from a small parser. It handles strings, numbers, booleans, `null`, lists, and objects, writes object keys in sorted order (a `BTreeMap`), and reports malformed text with a message that includes the position. It is not a replacement for a production library, and a decimal such as `2.0` is written as `2`, so it reads back as an integer.

JSON text stores data in a standard format: `{ ... }` holds key-value pairs, `[ ... ]` holds ordered items, and text is in double quotation marks. Decoded data is a `Json` enum, so `match` forces you to handle every kind of value.

The shared to-do record has the fields `id`, `title`, `description`, `done`, and `created_at`. The example saves two tasks, loads them back, and shows the message for a malformed file.

The teaching helper decodes JSON `null` as `Json::Null` and whole numbers as `Json::Int`. It does not handle every number edge case that a full parser must.

Worked Example

```
use std::collections::BTreeMap;

#[derive(Debug, Clone, PartialEq)]
enum Json {
    Null,
    Bool(bool),
    Int(i64),
    Float(f64),
    Str(String),
    List(Vec<Json>),
    Object(BTreeMap<String, Json>),
}

fn quote(text: &str) -> String {
    let mut out = String::from("\"");
    for c in text.chars() {
        match c {
            '"' => out.push_str("\\\""),
            '\\' => out.push_str("\\\\"),
            '\n' => out.push_str("\\n"),
            '\r' => out.push_str("\\r"),
            '\t' => out.push_str("\\t"),
            c if (c as u32) < 0x20 => out.push_str(&format!("\\u{:04x}", c as u32)),
            c => out.push(c),
        }
    }
    out.push('"');
    out
}

fn encode(value: &Json) -> String {
    match value {
        Json::Null => "null".to_string(),
        Json::Bool(flag) => flag.to_string(),
        Json::Int(number) => number.to_string(),
        Json::Float(number) => number.to_string(),
        Json::Str(text) => quote(text),
```

```
        Json::List(items) => {
            let parts: Vec<String> = items.iter().map(encode).collect();
            format!("[{}]", parts.join(","))
        }
        Json::Object(map) => {
            let parts: Vec<String> = map.iter().map(|(key, item)| format!("{}:{}", quote(key
            format!("{{{}}}", parts.join(","))
        }
    }
}

struct Parser {
    chars: Vec<char>,
    pos: usize,
}

impl Parser {
    fn fail<T>(&self, message: &str) -> Result<T, String> {
        Err(format!("{message} at position {}", self.pos))
    }

    fn skip(&mut self) {
        while self.pos < self.chars.len() && self.chars[self.pos].is_whitespace() {
            self.pos += 1;
        }
    }

    fn peek(&self) -> Option<char> {
        self.chars.get(self.pos).copied()
    }

    fn starts_with(&self, word: &str) -> bool {
        word.chars().enumerate().all(|(i, c)| self.chars.get(self.pos + i) == Some(&c))
    }

    fn value(&mut self) -> Result<Json, String> {
        self.skip();
        match self.peek() {
            None => self.fail("unexpected end of text"),
            Some('{') => self.object(),
            Some('[') => self.list(),
            Some('"') => Ok(Json::Str(self.string()?)),
            Some(c) if c == '-' || c.is_ascii_digit() => self.number(),
            Some(_) if self.starts_with("true") => {
                self.pos += 4;
                Ok(Json::Bool(true))
```

```
        }
        Some(_) if self.starts_with("false") => {
            self.pos += 5;
            Ok(Json::Bool(false))
        }
        Some(_) if self.starts_with("null") => {
            self.pos += 4;
            Ok(Json::Null)
        }
        Some(_) => self.fail("unexpected character"),
    }
}

fn number(&mut self) -> Result<Json, String> {
    let start = self.pos;
    if self.peek() == Some('-') {
        self.pos += 1;
    }
    while let Some(c) = self.peek() {
        if c.is_ascii_digit() || ".eE+-".contains(c) {
            self.pos += 1;
        } else {
            break;
        }
    }
    let token: String = self.chars[start..self.pos].iter().collect();
    if let Ok(number) = token.parse::<i64>() {
        return Ok(Json::Int(number));
    }
    match token.parse::<f64>() {
        Ok(number) => Ok(Json::Float(number)),
        Err(_) => self.fail("bad number"),
    }
}

fn string(&mut self) -> Result<String, String> {
    self.pos += 1;
    let mut out = String::new();
    loop {
        let Some(c) = self.peek() else {
            return self.fail("unterminated string");
        };
        self.pos += 1;
        match c {
            '"' => return Ok(out),
            '\\' => {
```

```
                    let Some(escape) = self.peek() else {
                        return self.fail("bad escape");
                    };
                    self.pos += 1;
                    match escape {
                        'n' => out.push('\n'),
                        't' => out.push('\t'),
                        'r' => out.push('\r'),
                        '"' => out.push('"'),
                        '\\' => out.push('\\'),
                        '/' => out.push('/'),
                        'u' => {
                            if self.pos + 4 > self.chars.len() {
                                return self.fail("bad escape");
                            }
                            let hex: String = self.chars[self.pos..self.pos + 4].iter().coll
                            let code = u32::from_str_radix(&hex, 16).map_err(|_| "bad escape
                            out.push(char::from_u32(code).unwrap_or('?'));
                            self.pos += 4;
                        }
                        _ => return self.fail("bad escape"),
                    }
                }
                other => out.push(other),
            }
        }
    }

    fn object(&mut self) -> Result<Json, String> {
        self.pos += 1;
        let mut map = BTreeMap::new();
        self.skip();
        if self.peek() == Some('}') {
            self.pos += 1;
            return Ok(Json::Object(map));
        }
        loop {
            self.skip();
            if self.peek() != Some('"') {
                return self.fail("expected a key");
            }
            let key = self.string()?;
            self.skip();
            if self.peek() != Some(':') {
                return self.fail("expected ':'");
            }
```

```
            self.pos += 1;
            let item = self.value()?;
            map.insert(key, item);
            self.skip();
            match self.peek() {
                None => return self.fail("unterminated object"),
                Some('}') => {
                    self.pos += 1;
                    return Ok(Json::Object(map));
                }
                Some(',') => self.pos += 1,
                Some(_) => return self.fail("expected ',' or '}'"),
            }
        }
    }

    fn list(&mut self) -> Result<Json, String> {
        self.pos += 1;
        let mut items = Vec::new();
        self.skip();
        if self.peek() == Some(']') {
            self.pos += 1;
            return Ok(Json::List(items));
        }
        loop {
            items.push(self.value()?);
            self.skip();
            match self.peek() {
                None => return self.fail("unterminated list"),
                Some(']') => {
                    self.pos += 1;
                    return Ok(Json::List(items));
                }
                Some(',') => self.pos += 1,
                Some(_) => return self.fail("expected ',' or ']'"),
            }
        }
    }
}

fn decode(text: &str) -> Result<Json, String> {
    let mut parser = Parser { chars: text.chars().collect(), pos: 0 };
    let value = parser.value()?;
    parser.skip();
    if parser.pos < parser.chars.len() {
        return parser.fail("unexpected extra text");
```

```
    }
    Ok(value)
}

use std::fs;
use std::io;

fn task(id: i64, title: &str, description: &str, done: bool, created_at: &str) -> Json {
    let mut map = BTreeMap::new();
    map.insert("id".to_string(), Json::Int(id));
    map.insert("title".to_string(), Json::Str(title.to_string()));
    map.insert("description".to_string(), Json::Str(description.to_string()));
    map.insert("done".to_string(), Json::Bool(done));
    map.insert("created_at".to_string(), Json::Str(created_at.to_string()));
    Json::Object(map)
}

fn main() -> io::Result<()> {
    let tasks = Json::List(vec![
        task(1, "Study programming", "Finish Unit 8", false, "2026-01-15T09:00:00"),
        task(2, "Review scores", "Check the four scores", true, "2026-01-15T09:30:00"),
    ]);
    fs::write("tasks.json", encode(&tasks))?;

    let text = match fs::read_to_string("tasks.json") {
        Ok(text) => text,
        Err(_) => {
            println!("Tasks file not found.");
            return Ok(());
        }
    };

    match decode(&text) {
        Ok(Json::List(items)) => {
            for item in &items {
                if let Json::Object(map) = item {
                    let id = match map.get("id") {
                        Some(Json::Int(number)) => *number,
                        _ => 0,
                    };
                    let title = match map.get("title") {
                        Some(Json::Str(text)) => text.as_str(),
                        _ => "?",
                    };
                    let status = if map.get("done") == Some(&Json::Bool(true)) { "done" } el
                    println!("{id}: {title} ({status})");
```

```
                }
            }
        }
        Ok(_) => println!("Error: the tasks file must contain a list."),
        Err(message) => println!("Error: could not decode tasks file: {message}"),
    }

    println!("{:?}", decode("[{\"id\": 1,"));
    Ok(())
}
```

Expected Behavior or Output

The program prints:

```
1: Study programming (not done)
2: Review scores (done)
Err("expected a key at position 10")
```

Common Mistakes

1. Assuming Rust has built-in JSON support.
2. Trusting decoded data without matching on the kind of each value.
3. Overwriting a good data file with an empty list after a failed load.

Activities

1. Run the program and read the saved JSON text from `tasks.json`.
2. Edit `tasks.json` by hand to remove a closing bracket and run the loading part again. Read the message.
3. Add a third task and confirm that it is saved and loaded.
4. Explain why a production project should use `serde_json`.

Accessibility Practice

Open `tasks.json` in the terminal and read it as text. JSON in a single line is difficult to navigate, so practise stepping through it by word or character.

Skills Checklist

- ☐ I know Rust has no built-in JSON.
- ☐ I can encode and decode records.
- ☐ I handle missing and malformed files.
- ☐ I match on decoded values.
- ☐ I know when to use a crate.

14.11 Unit 9: Testing and Data Analysis

Overview

This unit develops testing and data analysis through Rust's type system and standard library. Prerequisites: prior units, a working Cargo project, and the ability to read compiler diagnostics. Preserve shared course datasets and test boundary conditions explicitly.

Lesson 9.1: Unit tests

Reading References

- The Rust Programming Language: How to Write Tests
- The Rust Programming Language: Test Organization
- Rust standard library documentation
- Matching Python lesson: Lesson 9.1: Testing Your Code

Learning Objectives

- Write checks with chosen inputs and expected results.
- Write a `#[cfg(test)]` module with `#[test]` functions.
- Compare decimals with a tolerance and test boundary values.

Concept Explanation

Rust has testing built in. A function marked `#[test]` inside a module marked `#[cfg(test)]` is compiled and run only by the test harness (`cargo test`, or `rustc --test`), and `assert_eq!(actual, expected)` stops that test with a clear message when the values differ. A normal build ignores the module, so tests cost nothing at run time.

The example shows both styles: a small hand-written `Tally` that prints PASS and FAIL lines when the program runs, and the idiomatic `#[test]` module, which is verified separately in the Expected section. Decimal results are compared with a *tolerance*, not `==`, because the tip for 24.50 at 0.15 is stored as 3.6749999999999998.

The same tests appear in every language track: `add(5, 3)` is 8, `add(-5, -3)` is -8, `add(10, -4)` is 6, the tip is 3.675, and the grade boundaries 90, 89, 60, and 59 give A, B, D, and F. `std::process::exit(1)` ends the program with a failure status.

Worked Example

```
fn add(a: i32, b: i32) -> i32 {
    a + b
}
```

```rust
fn calculate_tip(bill: f64, rate: f64) -> f64 {
    bill * rate
}

fn classify(score: i32) -> &'static str {
    if score >= 90 {
        "A"
    } else if score >= 80 {
        "B"
    } else if score >= 70 {
        "C"
    } else if score >= 60 {
        "D"
    } else {
        "F"
    }
}

struct Tally {
    passed: u32,
    failed: u32,
}

impl Tally {
    fn check<T: PartialEq + std::fmt::Debug>(&mut self, name: &str, actual: T, expected: T) {
        if actual == expected {
            self.passed += 1;
            println!("PASS {name}");
        } else {
            self.failed += 1;
            println!("FAIL {name}: expected {:?} but got {:?}", expected, actual);
        }
    }

    fn check_close(&mut self, name: &str, actual: f64, expected: f64, tolerance: f64) {
        self.check(name, (actual - expected).abs() <= tolerance, true);
    }
}

fn main() {
    let mut tally = Tally { passed: 0, failed: 0 };
    tally.check("add 5 and 3", add(5, 3), 8);
    tally.check("add -5 and -3", add(-5, -3), -8);
    tally.check("add 10 and -4", add(10, -4), 6);
    tally.check_close("tip on 24.50 at 0.15", calculate_tip(24.5, 0.15), 3.675, 1e-9);
    tally.check("tip on a zero bill", calculate_tip(0.0, 0.15), 0.0);
```

```
    tally.check("classify 90", classify(90), "A");
    tally.check("classify 89", classify(89), "B");
    tally.check("classify 60", classify(60), "D");
    tally.check("classify 59", classify(59), "F");

    println!("{} passed, {} failed", tally.passed, tally.failed);
    if tally.failed > 0 {
        std::process::exit(1);
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn adds_numbers() {
        assert_eq!(add(5, 3), 8);
        assert_eq!(add(-5, -3), -8);
        assert_eq!(add(10, -4), 6);
    }

    #[test]
    fn tip_is_close_to_expected() {
        assert!((calculate_tip(24.5, 0.15) - 3.675).abs() < 1e-9);
    }

    #[test]
    fn grade_boundaries() {
        assert_eq!(classify(90), "A");
        assert_eq!(classify(89), "B");
        assert_eq!(classify(60), "D");
        assert_eq!(classify(59), "F");
    }
}
```

Expected Behavior or Output

The program prints:

```
PASS add 5 and 3
PASS add -5 and -3
PASS add 10 and -4
PASS tip on 24.50 at 0.15
PASS tip on a zero bill
PASS classify 90
```

```
PASS classify 89
PASS classify 60
PASS classify 59
9 passed, 0 failed
```

The `#[cfg(test)]` module is not compiled in a normal build. Running it with the test harness (`rustc --test main.rs -o tests` then `./tests --test-threads=1`, or `cargo test`) prints:

```
running 3 tests
test tests::adds_numbers ... ok
test tests::grade_boundaries ... ok
test tests::tip_is_close_to_expected ... ok

test result: ok. 3 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.0
```

Common Mistakes

1. Comparing decimal results with `assert_eq!` and `==`.
2. Testing only typical values and skipping the boundaries 90, 89, 60, and 59.
3. Putting `#[test]` functions outside a `#[cfg(test)]` module, which compiles them into the normal build.

Activities

1. Change one expected value so a check fails, read the message, and restore it.
2. Add checks for the grades at 80, 79, 70, and 69.
3. Write the tests for `is_passing(score)` before writing the function.
4. Explain why the tip test uses a tolerance.

Accessibility Practice

Each check prints one complete line that starts with PASS or FAIL, and the last line gives the counts, so the result can be understood from the first word of each line and the final summary. The test harness prints one line per test and a final result line in the same style.

Skills Checklist

- ☐ I can write a check with an expected value.
- ☐ I can write a `#[test]` module.
- ☐ I can compare decimals with a tolerance.
- ☐ I can choose boundary tests.
- ☐ I know how to signal failure with an exit status.

Lesson 9.2: Testing stateful types

Reading References

- The Rust Programming Language: How to Write Tests
- The Rust Programming Language: Checking for Panics with should_panic
- Rust standard library documentation
- Matching Python lesson: Lesson 9.2: Testing a Class

Learning Objectives

- Create a fresh value for every test.
- Test a return value, a change of state, and an error result.
- Write the same tests in a `#[cfg(test)]` module.

Concept Explanation

Each test should start from a known state, so build a new value for every test with a *setup* function. Tests that share a value can affect each other and fail in confusing ways.

Test three kinds of behavior: that a method returns the right value (`total_value` for 3 at 2.50 is 7.5), that a method changes the value correctly (restocking 2 makes the quantity 5), and that invalid use is reported (restocking a negative amount returns an `Err`). Because Rust reports expected failures with `Result`, a test can simply check `is_err()`. A function that is expected to panic can be tested with `#[should_panic]`.

The example runs each test as a function returning `Result<(), String>`, so a failed check carries a message, and the same tests also appear in a `#[cfg(test)]` module.

Worked Example

```
struct Item {
    name: String,
    quantity: i32,
    unit_price: f64,
}

impl Item {
    fn total_value(&self) -> f64 {
        self.quantity as f64 * self.unit_price
    }

    fn restock(&mut self, amount: i32) -> Result<(), String> {
        if amount < 0 {
```

```
            return Err("amount must not be negative".to_string());
        }
        self.quantity += amount;
        Ok(())
    }
}

fn setup() -> Item {
    Item { name: "Notebook".to_string(), quantity: 3, unit_price: 2.50 }
}

fn expect(condition: bool, message: &str) -> Result<(), String> {
    if condition {
        Ok(())
    } else {
        Err(message.to_string())
    }
}

fn total_value_is_7_5() -> Result<(), String> {
    expect((setup().total_value() - 7.5).abs() < 1e-9, "wrong total")
}

fn restock_adds_to_the_quantity() -> Result<(), String> {
    let mut item = setup();
    item.restock(2)?;
    expect(item.quantity == 5, "quantity should be 5")
}

fn restock_rejects_a_negative_amount() -> Result<(), String> {
    let mut item = setup();
    expect(item.restock(-1).is_err(), "an error should have been returned")?;
    expect(item.quantity == 3, "the quantity should be unchanged")?;
    expect(!item.name.is_empty(), "the name should be kept")
}

fn main() {
    let tests: [(&str, fn() -> Result<(), String>); 3] = [
        ("total value is 7.5", total_value_is_7_5),
        ("restock adds to the quantity", restock_adds_to_the_quantity),
        ("restock rejects a negative amount", restock_rejects_a_negative_amount),
    ];

    let mut failures = 0;
    for (name, test) in tests {
        match test() {
```

```
            Ok(()) => println!("PASS {name}"),
            Err(message) => {
                failures += 1;
                println!("FAIL {name}: {message}");
            }
        }
    }
    println!("{}", if failures == 0 { "All tests passed.".to_string() } else { format!("{fai
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn total_value() {
        assert!(total_value_is_7_5().is_ok());
    }

    #[test]
    fn restock_adds() {
        assert_eq!(restock_adds_to_the_quantity(), Ok(()));
    }

    #[test]
    fn restock_rejects_negative() {
        assert_eq!(restock_rejects_a_negative_amount(), Ok(()));
    }
}
```

Expected Behavior or Output

The program prints:

```
PASS total value is 7.5
PASS restock adds to the quantity
PASS restock rejects a negative amount
All tests passed.
```

The `#[cfg(test)]` module is not compiled in a normal build. Running it with the test harness (`rustc --test main.rs -o tests` then `./tests --test-threads=1`, or `cargo test`) prints:

```
running 3 tests
test tests::restock_adds ... ok
test tests::restock_rejects_negative ... ok
test tests::total_value ... ok
```

```
test result: ok. 3 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00
```

Common Mistakes

1. Sharing one value between tests, so an earlier test changes the later ones.
2. Testing only the successful path and never the error path.
3. Using `unwrap()` inside a test helper so that a failure panics with no message about which check failed.

Activities

1. Add a test that `restock(0)` leaves the quantity unchanged.
2. Add a test for an `is_valid` method.
3. Break `restock` on purpose and read the failure line.
4. Explain why `setup` creates a new item each time.

Accessibility Practice

Name each test as a short sentence that says what it checks, so the PASS and FAIL lines read as clear statements. Read the final summary line first.

Skills Checklist

☐ I can create a fresh value per test.
☐ I can test a return value.
☐ I can test a change of state.
☐ I can test an error result.
☐ I can write a clear test name.

Lesson 9.3: Random values

Reading References

- The rand crate
- The Rust Programming Language: Method Syntax
- Rust standard library documentation
- Matching Python lesson: Lesson 9.3: Random Numbers and Simulation

Learning Objectives

- Explain why the standard library has no random numbers.
- Use a small seeded generator and test it by its properties.
- Pick a random list item.

Concept Explanation

The Rust standard library has **no random-number generator**. Real programs add the `rand` crate through Cargo. To keep this course free of dependencies, the example uses a tiny *xorshift* generator written in a few lines. It is predictable and **not secure**, so it must never be used for passwords or tokens, but it is enough to learn the ideas.

A generator started from the same *seed* produces the same sequence, so a test can check *repeatability*. A seed must not be zero for this algorithm. Because the exact numbers depend on the algorithm, tests should check *properties* (every roll is between 1 and 6, a decimal is in 0 up to but not including 1) and not particular values.

Worked Example

```
struct Rng {
    state: u64,
}

impl Rng {
    fn new(seed: u64) -> Rng {
        Rng { state: if seed == 0 { 1 } else { seed } }
    }

    fn next_u64(&mut self) -> u64 {
        let mut x = self.state;
        x ^= x << 13;
        x ^= x >> 7;
        x ^= x << 17;
        self.state = x;
        x
    }

    fn range(&mut self, low: u64, high: u64) -> u64 {
        low + self.next_u64() % (high - low)
    }

    fn next_f64(&mut self) -> f64 {
        (self.next_u64() >> 11) as f64 / (1u64 << 53) as f64
    }
}

fn main() {
    let mut first = Rng::new(42);
    let mut second = Rng::new(42);
    let first_rolls: Vec<u64> = (0..5).map(|_| first.range(1, 7)).collect();
```

```
    let second_rolls: Vec<u64> = (0..5).map(|_| second.range(1, 7)).collect();
    println!("Same seed gives the same rolls: {}", first_rolls == second_rolls);

    let mut random = Rng::new(7);
    let in_range = (0..1000).all(|_| (1..=6).contains(&random.range(1, 7)));
    println!("All rolls between 1 and 6: {in_range}");

    let decimal = random.next_f64();
    println!("Decimal in [0, 1): {}", (0.0..1.0).contains(&decimal));

    let scores = [82, 91, 76, 88];
    let chosen = scores[random.range(0, scores.len() as u64) as usize];
    println!("Chosen score is from the list: {}", scores.contains(&chosen));
}
```

Expected Behavior or Output

The program prints:

```
Same seed gives the same rolls: true
All rolls between 1 and 6: true
Decimal in [0, 1): true
Chosen score is from the list: true
```

Common Mistakes

1. Looking for `std::random`. It does not exist; use the `rand` crate in real programs.
2. Using a simple generator for passwords or tokens, which it is not designed for.
3. Seeding with zero, which makes this algorithm produce only zeros.

Activities

1. Run the program three times and compare.
2. Change the seeds and check that the property lines still print `true`.
3. Simulate 600 rolls and count how many of each face appeared (the counts depend on the seed).
4. Explain why a test should check a property rather than a specific random number.

Accessibility Practice

Print yes or no style results for random behavior, so the announced output is the same on every run and the screen reader does not read unpredictable values.

Skills Checklist

- ☐ I know the standard library has no random numbers.
- ☐ I can use a seeded generator.
- ☐ I can test properties of random code.
- ☐ I know this generator is not secure.
- ☐ I can pick a random list item.

Lesson 9.4: Text analysis

Reading References

- The Rust Programming Language: Hash Maps
- The Rust Programming Language: Strings
- Rust standard library documentation
- Matching Python lesson: Lesson 9.4: Data Analysis - Text Analysis

Learning Objectives

- Split text into words and count them.
- Count word and letter frequencies in a stable order.
- Sort with a tie-break.

Concept Explanation

`text.split_whitespace()` iterates over the words of a string. Counting uses a map; because a `HashMap` has no stable order, the example keeps first-seen order in a `Vec<(String, u32)>` for words and uses a `BTreeMap` for letters, then sorts the letter pairs with `sort_by`: count descending first, then letter ascending, so ties print in a fixed order.

`to_lowercase()` and `replace(".", "")` normalize the text first. The shared text sample is "A student studies programming. Programming develops problem-solving skills." After lower-casing and removing the periods it has 8 words, and `programming` appears twice.

Worked Example

```rust
use std::collections::BTreeMap;

fn main() {
    let text = "A student studies programming. Programming develops problem-solving skills.
    let cleaned = text.to_lowercase().replace('.', "");
    let words: Vec<&str> = cleaned.split_whitespace().collect();

    let mut frequencies: Vec<(String, u32)> = Vec::new();
    for word in &words {
```

```
        match frequencies.iter_mut().find(|(known, _)| known == word) {
            Some((_, count)) => *count += 1,
            None => frequencies.push((word.to_string(), 1)),
        }
    }

    println!("Word count: {}", words.len());
    println!("Word frequencies:");
    for (word, count) in &frequencies {
        println!("'{word}': {count}");
    }

    let mut letters: BTreeMap<char, u32> = BTreeMap::new();
    for c in text.to_lowercase().chars().filter(|c| c.is_alphabetic()) {
        *letters.entry(c).or_insert(0) += 1;
    }
    let mut pairs: Vec<(char, u32)> = letters.into_iter().collect();
    pairs.sort_by(|a, b| b.1.cmp(&a.1).then(a.0.cmp(&b.0)));
    println!("Most common letters:");
    for (letter, count) in pairs.iter().take(3) {
        println!("{letter}: {count}");
    }
}
```

Expected Behavior or Output

The program prints:

```
Word count: 8
Word frequencies:
'a': 1
'student': 1
'studies': 1
'programming': 2
'develops': 1
'problem-solving': 1
'skills': 1
Most common letters:
s: 7
e: 5
g: 5
```

Common Mistakes

1. Counting punctuation as part of a word, so `programming.` and `programming` are different words.

2. Forgetting to lower-case first, so `Programming` and `programming` are counted separately.
3. Printing a `HashMap` and expecting a stable order.

Activities

1. Predict the word count and the count for `programming` before running.
2. Count the words longer than 8 letters.
3. Print the longest word.
4. Change the sort so ties are listed in reverse alphabetical order.

Accessibility Practice

Read the frequency list in its fixed order. Because the order is stable, you can learn it and compare runs by ear. Print each pair as a short sentence with the same wording every time.

Skills Checklist

☐ I can split text into words.
☐ I can count frequencies in a stable order.
☐ I can sort with a tie-break.
☐ I can remove punctuation first.
☐ I can read a stable report.

Lesson 9.5: CSV records

Reading References

- The Rust Programming Language: Reading a File
- The csv crate
- Rust standard library documentation
- Matching Python lesson: Lesson 9.5: Data Analysis - CSV Files

Learning Objectives

- Create and read a CSV file.
- Split lines into fields and convert numbers.
- Compute an average and a count.

Concept Explanation

A CSV (comma-separated values) file stores a table as text. The first line names the columns and each later line is a record. The shared student file has the columns `student_id`, `first_name`, `last_name`, `score`, and `active`, and three records.

The standard library has no CSV reader. `line.split(',')` works only for **simple** files: a value that contains a comma inside quotation marks, such as `"Smith, Jr."`, would be split wrongly. For such files use the `csv` crate or write a real parser.

Every field is text. Convert the score with `parse::<i32>()` and compare `active` with the text `"true"`. With scores 91, 82, and 76 the average is 83.00, and two students are active.

Worked Example

```
use std::fs;
use std::io;

fn main() -> io::Result<()> {
    let content = [
        "student_id,first_name,last_name,score,active",
        "1,Avery,Student,91,true",
        "2,Jordan,Lee,82,true",
        "3,Casey,Smith,76,false",
    ]
    .join("\n")
        + "\n";
    fs::write("students.csv", content)?;

    let text = match fs::read_to_string("students.csv") {
        Ok(text) => text,
        Err(_) => {
            println!("Error: students.csv was not found.");
            return Ok(());
        }
    };
    let mut lines = text.lines();
    let header = lines.next().unwrap_or("");
    println!("Columns: {}", header.split(',').collect::<Vec<_>>().join(", "));

    let mut total = 0;
    let mut active = 0;
    let mut count = 0;
    for line in lines {
        let row: Vec<&str> = line.split(',').collect();
        let score: i32 = row[3].parse().unwrap_or(0);
        total += score;
        count += 1;
        if row[4] == "true" {
            active += 1;
```

```
        }
        println!("{}: {} {} scored {}", row[0], row[1], row[2], score);
    }

    println!("Students: {count}");
    println!("Average score: {:.2}", total as f64 / count as f64);
    println!("Active students: {active}");
    Ok(())
}
```

Expected Behavior or Output

The program prints:

```
Columns: student_id, first_name, last_name, score, active
1: Avery Student scored 91
2: Jordan Lee scored 82
3: Casey Smith scored 76
Students: 3
Average score: 83.00
Active students: 2
```

Common Mistakes

1. Splitting a CSV line at every comma when a value contains a quoted comma.
2. Forgetting to convert the text in the score column to a number.
3. Not skipping the header line, so `parse` fails on the text `score`.

Activities

1. Predict the printed lines before running.
2. Add a fourth student and check the new average.
3. Print only the active students.
4. Explain, with an example, why `"Smith, Jr."` breaks `split(',')`.

Accessibility Practice

Print one student per line with the same wording each time, so the screen reader announces a repeating pattern that is easy to compare. Read the column names aloud once at the start.

Skills Checklist

- ☐ I can write and read a CSV file.
- ☐ I can split a line into fields.
- ☐ I convert text fields to numbers.
- ☐ I can compute an average and a count.

☐ I know when `split` is not enough.

14.12 Unit 10: Project-Based Learning

Overview

This unit develops project-based learning through Rust's type system and standard library. Prerequisites: prior units, a working Cargo project, and the ability to read compiler diagnostics. Preserve shared course datasets and test boundary conditions explicitly.

Lesson 10.1: Requirements and top-down design

Reading References

- The Rust Programming Language: Hash Maps
- The Rust Programming Language: Functions
- Rust standard library documentation
- Matching Python lesson: Lesson 10.1: Project Planning and Design

Learning Objectives

- Write a short requirements list.
- Design a command loop with a map of function pointers.
- Plan in small steps and test each one.

Concept Explanation

A project is easier to finish when it is planned in small steps. Write down what the program must do (*requirements*), what information it keeps (*data*), and the commands a person can give (*interface*). Build one small piece, test it, then add the next.

Text-based projects share a *command loop*: show a prompt, read a line, find the matching action, and repeat until the user quits. In Rust, a `HashMap<&str, fn()>` holds actions: the keys are command names and the values are *function pointers*. A new command is one new entry, and an unknown command is a failed lookup (`None`) that prints a helpful message.

The loop stops on `quit` and also when input ends. The example is a skeleton with two commands; typing `help`, `look`, an unknown word, and `quit` shows each path.

Worked Example

```
use std::collections::HashMap;
use std::io;
```

```rust
fn next_line() -> Option<String> {
    let mut line = String::new();
    match io::stdin().read_line(&mut line) {
        Ok(0) | Err(_) => None,
        Ok(_) => Some(line.trim().to_string()),
    }
}

fn help() {
    println!("Commands: help, look, quit");
}

fn look() {
    println!("You are in a dark cave.");
}

fn main() {
    let mut commands: HashMap<&str, fn()> = HashMap::new();
    commands.insert("help", help);
    commands.insert("look", look);

    println!("Type a command:");
    let mut line = next_line();
    while let Some(text) = &line {
        if text == "quit" {
            break;
        }
        match commands.get(text.as_str()) {
            Some(handler) => handler(),
            None => println!("I don't understand that command. Type help for a list."),
        }
        println!("Type a command:");
        line = next_line();
    }
    println!("Goodbye!");
}
```

Expected Behavior or Output

With the following lines typed as standard input (piped input is not echoed, so it does not appear in the output):

```
help
look
dance
```

```
quit
```

the program prints:

```
Type a command:
Commands: help, look, quit
Type a command:
You are in a dark cave.
Type a command:
I don't understand that command. Type help for a list.
Type a command:
Goodbye!
```

With no standard input at all, the program ends cleanly and prints:

```
Type a command:
Goodbye!
```

Common Mistakes

1. Starting to code before listing the requirements.
2. Using a long chain of `if` statements instead of a map of handlers.
3. Forgetting that an unknown command should say what to do next.

Activities

1. Write five requirements for the text adventure and the to-do list.
2. Add a `score` command that prints the shared scores.
3. Add the aliases `h` and `l` without copying the functions.
4. Describe the data for the capstone in three sentences.

Accessibility Practice

Prompts and responses are on separate lines and every response is a complete sentence. Test the prompts by listening to them before you add features.

Skills Checklist

- ☐ I can list requirements.
- ☐ I can design a command loop.
- ☐ I can use a map of function pointers.
- ☐ I handle unknown commands.
- ☐ I handle the end of input.

Lesson 10.2: Text Adventure, Part 1: Five Rooms and Movement

Reading References

- The Rust Programming Language: Hash Maps
- The Rust Programming Language: Structs
- Rust standard library documentation
- Canonical shared project specification: Python Unit 10, Lesson 10.3

Learning Objectives

- Represent the five canonical rooms and their exits with Rust structs and hash maps.
- Move through valid exits without changing location after an invalid command.
- Explain why the north exit from the locked-door room is not available yet.

Concept Explanation

The shared adventure has five rooms: start_room, tunnel, clearing, locked_door_room, and treasure_room. The unconditional connections are start north to tunnel; tunnel south to start and east to clearing; clearing west to tunnel and north to the locked-door room; and locked-door room south to clearing. The north exit to the treasure room is conditional and is added in Part 2 only after the shiny-key puzzle is solved.

A Room struct stores a description and a HashMap of exits. HashMap::get returns an Option; a missing direction must not change the current room.

Worked Example

```
use std::collections::HashMap;
use std::io;

struct Room {
    description: &'static str,
    exits: HashMap<&'static str, &'static str>,
}

fn next_line() -> Option<String> {
    let mut line = String::new();
    match io::stdin().read_line(&mut line) {
        Ok(0) | Err(_) => None,
        Ok(_) => Some(line.trim().to_lowercase()),
    }
}
```

```
fn main() {
    let rooms = HashMap::from([
        ("start_room", Room {
            description: "A dimly lit cave. A narrow passage leads north.",
            exits: HashMap::from([("north", "tunnel")]),
        }),
        ("tunnel", Room {
            description: "A damp tunnel. Passages lead south and east.",
            exits: HashMap::from([("south", "start_room"), ("east", "clearing")]),
        }),
        ("clearing", Room {
            description: "A quiet clearing. An antechamber lies to the north.",
            exits: HashMap::from([("west", "tunnel"), ("north", "locked_door_room")]),
        }),
        ("locked_door_room", Room {
            description: "A chamber with a heavy door to the north.",
            exits: HashMap::from([("south", "clearing")]),
        }),
        ("treasure_room", Room {
            description: "You found the treasure. You win!",
            exits: HashMap::new(),
        }),
    ]);

    let mut current_room = "start_room";
    loop {
        let room = rooms.get(current_room).expect("current room must exist");
        println!("---");
        println!("{}", room.description);
        println!("What do you want to do?");
        let command = next_line().unwrap_or_else(|| "quit".to_string());
        if command == "quit" {
            println!("Exiting game. Goodbye!");
            break;
        }
        match room.exits.get(command.as_str()) {
            Some(destination) => current_room = destination,
            None => println!("You cannot go that way. Enter a listed direction or quit."),
        }
    }
}
```

Expected Behavior or Output

With north, east, north, north, quit entered as separate input lines, the player visits tunnel, clearing, and locked_door_room. The second north from

locked_door_room is rejected and the current room does not change. The treasure room exists in the model but cannot be reached in Part 1.

Common Mistakes

1. Adding the treasure-room north exit before implementing the key puzzle.
2. Updating current_room before confirming that the requested exit exists.
3. Assuming a direction is present instead of handling None from HashMap::get.
4. Defining an exit whose destination is not one of the five room identifiers.

Activities

1. Trace north, east, north, north and record the room after each command.
2. Try west from start_room; confirm that the location does not change.
3. Check that every destination in every unconditional exit exists in rooms.
4. Explain why treasure_room has no incoming exit in Part 1.

Accessibility Practice

Read the separator, room description, and prompt in order. The invalid-movement response explains what happened and how to continue. Test the sequence by listening to terminal output; do not rely on a visual map.

Skills Checklist

☐ I can model all five canonical rooms.
☐ I can represent exits with a hash map.
☐ I can move only when an exit exists.
☐ I can reject an invalid direction without changing state.
☐ I can explain why the treasure room is not yet reachable.

Lesson 10.3: Text Adventure, Part 2: Shiny Key and Locked Door

Reading References

- The Rust Programming Language: Vectors
- The Rust Programming Language: References and Borrowing
- Rust standard library documentation
- Canonical shared project specification: Python Unit 10, Lesson 10.3

Learning Objectives

- Place the shiny key in start_room and transfer it to inventory exactly once.

- Require the player to use the key while in locked_door_room.
- Unlock the north exit to treasure_room only after the puzzle succeeds.
- End the game when the player reaches the treasure room.

Concept Explanation

Part 2 keeps the same five-room graph. The shiny key begins in start_room. The take shiny key command removes it from that room and adds it to inventory. The use shiny key command works only in locked_door_room and only when the key is in inventory. Attempting north from the locked-door room before that action prints a locked-door message and leaves the player in place. After the key is used, north leads to treasure_room.

Room descriptions do not claim that an item is still present. Room items and inventory are announced on separate lines.

Worked Example

```
use std::collections::HashMap;
use std::io;

struct Room {
    description: &'static str,
    exits: HashMap<&'static str, &'static str>,
    items: Vec<&'static str>,
}

fn next_line() -> Option<String> {
    let mut line = String::new();
    match io::stdin().read_line(&mut line) {
        Ok(0) | Err(_) => None,
        Ok(_) => Some(line.trim().to_lowercase()),
    }
}

fn main() {
    let mut rooms = HashMap::from([
        ("start_room", Room {
            description: "A dimly lit cave.",
            exits: HashMap::from([("north", "tunnel")]),
            items: vec!["shiny key"],
        }),
        ("tunnel", Room {
            description: "A damp tunnel. Passages lead south and east.",
            exits: HashMap::from([("south", "start_room"), ("east", "clearing")]),
            items: Vec::new(),
        }),
```

```
        ("clearing", Room {
            description: "A quiet clearing. An antechamber lies to the north.",
            exits: HashMap::from([("west", "tunnel"), ("north", "locked_door_room")]),
            items: Vec::new(),
        }),
        ("locked_door_room", Room {
            description: "A chamber with a heavy door to the north.",
            exits: HashMap::from([("south", "clearing"), ("north", "treasure_room")]),
            items: Vec::new(),
        }),
        ("treasure_room", Room {
            description: "You found the treasure. You win!",
            exits: HashMap::new(),
            items: Vec::new(),
        }),
    ]);

    let mut current_room = "start_room";
    let mut inventory: Vec<&'static str> = Vec::new();
    let mut door_locked = true;

    loop {
        let room = rooms.get(current_room).expect("current room must exist");
        println!();
        println!("{}", room.description);
        if current_room == "treasure_room" {
            break;
        }
        if !room.items.is_empty() {
            println!("You see: {}", room.items.join(", "));
        }
        println!(
            "Your inventory: {}",
            if inventory.is_empty() { "empty" } else { "shiny key" }
        );
        println!("What do you want to do?");

        let command = next_line().unwrap_or_else(|| "quit".to_string());
        match command.as_str() {
            "quit" => {
                println!("You quit the adventure.");
                break;
            }
            "take shiny key" => {
                let taken = {
                    let room = rooms.get_mut(current_room).expect("current room must exist")
```

```
                    if let Some(index) = room.items.iter().position(|item| *item == "shiny k
                        room.items.remove(index);
                        true
                    } else {
                        false
                    }
                };
                if taken {
                    inventory.push("shiny key");
                    println!("You picked up the shiny key.");
                } else {
                    println!("There is no shiny key here to take.");
                }
            }
            "use shiny key" => {
                if current_room != "locked_door_room" {
                    println!("You must be at the locked door to use the key.");
                } else if !inventory.contains(&"shiny key") {
                    println!("You do not have the shiny key.");
                } else {
                    door_locked = false;
                    println!("You unlocked the door!");
                }
            }
            "north" if current_room == "locked_door_room" && door_locked => {
                println!("The door is locked! You need to use the shiny key here.");
            }
            direction => {
                let destination = rooms
                    .get(current_room)
                    .and_then(|room| room.exits.get(direction))
                    .copied();
                match destination {
                    Some(next_room) => current_room = next_room,
                    None => println!(
                        "Invalid command. Enter north, south, east, west, take shiny key, us
                    ),
                }
            }
        }
    }
}
```

Expected Behavior or Output

Use this sequence to verify the puzzle: north, east, north, north, south, west, south, take shiny key, north, east, north, use shiny key, north.

The first north attempt from locked_door_room must be rejected. Taking the key removes it from start_room and adds it to inventory. Using it outside locked_door_room must be rejected. Using it there prints You unlocked the door!; the next north reaches treasure_room and prints You found the treasure. You win!. End-of-file is treated as quit.

Common Mistakes

1. Leaving the shiny key in the room after adding it to inventory.
2. Unlocking the door outside locked_door_room.
3. Unlocking the door without checking inventory.
4. Allowing north movement before door_locked becomes false.
5. Describing the key as still on the floor after it has been collected.

Activities

1. Run the full sequence and record room, room items, inventory, and door state after every command.
2. Try take shiny key twice; the second attempt must not duplicate the item.
3. Try use shiny key in start_room and clearing; both attempts must be rejected.
4. Try north from locked_door_room before and after unlocking.
5. Add a look command that announces room, visible items, inventory, and exits in a consistent order.

Accessibility Practice

Announce room description, visible items, inventory, and prompt as separate text lines. Every failed action explains why it failed. Listen to the full puzzle sequence and verify that key location and door state are unambiguous.

Skills Checklist

- ☐ I can model the five canonical rooms.
- ☐ I can transfer the shiny key from a room to inventory.
- ☐ I can prevent duplicate pickup.
- ☐ I can require key use in the locked-door room.
- ☐ I can block and unlock the north exit.
- ☐ I can end the game when the treasure room is reached.

Lesson 10.4: Command-line to-do manager

Reading References

- The Rust Programming Language: Recoverable Errors with Result
- The Rust Programming Language: Reading a File
- Rust standard library documentation
- Matching Python lesson: Lesson 10.4: Building a Command-Line To-Do List

Learning Objectives

- Keep records with the shared fields.
- Save and load them as JSON.
- Offer add, view, mark-done, save, load, and quit commands.

Concept Explanation

A to-do list ties together records, loops, functions, and files. Each task is a struct with the shared fields `id`, `title`, `description`, `done`, and `created_at`. The program keeps a `Vec<Task>` in memory, shows a numbered menu, and saves the list to `todos.json` when asked and on exit.

Saving uses the teaching JSON helper from Lesson 8.5, included here so the example runs on its own. Each task is converted to a `Json::Object` for saving and rebuilt from one when loading, and the conversion refuses a record with a missing field. At start-up the program loads the file if it exists. A missing file is normal on first use, and a damaged file must not crash the program. The timestamp is computed from the system clock in UTC with a short date algorithm, because the standard library has no date formatting.

In the sample session the user adds two tasks, views the list, marks task 2 done, views it again, tries an invalid menu choice, views the list, and quits.

Worked Example

```
#![allow(dead_code)]
use std::collections::BTreeMap;

#[derive(Debug, Clone, PartialEq)]
enum Json {
    Null,
    Bool(bool),
    Int(i64),
    Float(f64),
    Str(String),
    List(Vec<Json>),
    Object(BTreeMap<String, Json>),
```

```
}

fn quote(text: &str) -> String {
    let mut out = String::from("\"");
    for c in text.chars() {
        match c {
            '"' => out.push_str("\\\""),
            '\\' => out.push_str("\\\\"),
            '\n' => out.push_str("\\n"),
            '\r' => out.push_str("\\r"),
            '\t' => out.push_str("\\t"),
            c if (c as u32) < 0x20 => out.push_str(&format!("\\u{:04x}", c as u32)),
            c => out.push(c),
        }
    }
    out.push('"');
    out
}

fn encode(value: &Json) -> String {
    match value {
        Json::Null => "null".to_string(),
        Json::Bool(flag) => flag.to_string(),
        Json::Int(number) => number.to_string(),
        Json::Float(number) => number.to_string(),
        Json::Str(text) => quote(text),
        Json::List(items) => {
            let parts: Vec<String> = items.iter().map(encode).collect();
            format!("[{}]", parts.join(","))
        }
        Json::Object(map) => {
            let parts: Vec<String> = map.iter().map(|(key, item)| format!("{}:{}", quote(key
            format!("{{{}}}", parts.join(","))
        }
    }
}

struct Parser {
    chars: Vec<char>,
    pos: usize,
}

impl Parser {
    fn fail<T>(&self, message: &str) -> Result<T, String> {
        Err(format!("{message} at position {}", self.pos))
    }
```

```
fn skip(&mut self) {
    while self.pos < self.chars.len() && self.chars[self.pos].is_whitespace() {
        self.pos += 1;
    }
}

fn peek(&self) -> Option<char> {
    self.chars.get(self.pos).copied()
}

fn starts_with(&self, word: &str) -> bool {
    word.chars().enumerate().all(|(i, c)| self.chars.get(self.pos + i) == Some(&c))
}

fn value(&mut self) -> Result<Json, String> {
    self.skip();
    match self.peek() {
        None => self.fail("unexpected end of text"),
        Some('{') => self.object(),
        Some('[') => self.list(),
        Some('"') => Ok(Json::Str(self.string()?)),
        Some(c) if c == '-' || c.is_ascii_digit() => self.number(),
        Some(_) if self.starts_with("true") => {
            self.pos += 4;
            Ok(Json::Bool(true))
        }
        Some(_) if self.starts_with("false") => {
            self.pos += 5;
            Ok(Json::Bool(false))
        }
        Some(_) if self.starts_with("null") => {
            self.pos += 4;
            Ok(Json::Null)
        }
        Some(_) => self.fail("unexpected character"),
    }
}

fn number(&mut self) -> Result<Json, String> {
    let start = self.pos;
    if self.peek() == Some('-') {
        self.pos += 1;
    }
    while let Some(c) = self.peek() {
        if c.is_ascii_digit() || ".eE+-".contains(c) {
```

```
            self.pos += 1;
        } else {
            break;
        }
    }
    let token: String = self.chars[start..self.pos].iter().collect();
    if let Ok(number) = token.parse::<i64>() {
        return Ok(Json::Int(number));
    }
    match token.parse::<f64>() {
        Ok(number) => Ok(Json::Float(number)),
        Err(_) => self.fail("bad number"),
    }
}

fn string(&mut self) -> Result<String, String> {
    self.pos += 1;
    let mut out = String::new();
    loop {
        let Some(c) = self.peek() else {
            return self.fail("unterminated string");
        };
        self.pos += 1;
        match c {
            '"' => return Ok(out),
            '\\' => {
                let Some(escape) = self.peek() else {
                    return self.fail("bad escape");
                };
                self.pos += 1;
                match escape {
                    'n' => out.push('\n'),
                    't' => out.push('\t'),
                    'r' => out.push('\r'),
                    '"' => out.push('"'),
                    '\\' => out.push('\\'),
                    '/' => out.push('/'),
                    'u' => {
                        if self.pos + 4 > self.chars.len() {
                            return self.fail("bad escape");
                        }
                        let hex: String = self.chars[self.pos..self.pos + 4].iter().coll
                        let code = u32::from_str_radix(&hex, 16).map_err(|_| "bad escape
                        out.push(char::from_u32(code).unwrap_or('?'));
                        self.pos += 4;
                    }
```

```
                    _ => return self.fail("bad escape"),
                }
            }
            other => out.push(other),
        }
    }
}

fn object(&mut self) -> Result<Json, String> {
    self.pos += 1;
    let mut map = BTreeMap::new();
    self.skip();
    if self.peek() == Some('}') {
        self.pos += 1;
        return Ok(Json::Object(map));
    }
    loop {
        self.skip();
        if self.peek() != Some('"') {
            return self.fail("expected a key");
        }
        let key = self.string()?;
        self.skip();
        if self.peek() != Some(':') {
            return self.fail("expected ':'");
        }
        self.pos += 1;
        let item = self.value()?;
        map.insert(key, item);
        self.skip();
        match self.peek() {
            None => return self.fail("unterminated object"),
            Some('}') => {
                self.pos += 1;
                return Ok(Json::Object(map));
            }
            Some(',') => self.pos += 1,
            Some(_) => return self.fail("expected ',' or '}'"),
        }
    }
}

fn list(&mut self) -> Result<Json, String> {
    self.pos += 1;
    let mut items = Vec::new();
    self.skip();
```

```
        if self.peek() == Some(']') {
            self.pos += 1;
            return Ok(Json::List(items));
        }
        loop {
            items.push(self.value()?);
            self.skip();
            match self.peek() {
                None => return self.fail("unterminated list"),
                Some(']') => {
                    self.pos += 1;
                    return Ok(Json::List(items));
                }
                Some(',') => self.pos += 1,
                Some(_) => return self.fail("expected ',' or ']'"),
            }
        }
    }
}

fn decode(text: &str) -> Result<Json, String> {
    let mut parser = Parser { chars: text.chars().collect(), pos: 0 };
    let value = parser.value()?;
    parser.skip();
    if parser.pos < parser.chars.len() {
        return parser.fail("unexpected extra text");
    }
    Ok(value)
}

use std::fs;
use std::io;
use std::time::{SystemTime, UNIX_EPOCH};

struct Task {
    id: i64,
    title: String,
    description: String,
    done: bool,
    created_at: String,
}

fn civil_from_days(days: i64) -> (i64, i64, i64) {
    let z = days + 719468;
    let era = z.div_euclid(146097);
    let doe = z.rem_euclid(146097);
```

```
    let yoe = (doe - doe / 1460 + doe / 36524 - doe / 146096) / 365;
    let year = yoe + era * 400;
    let doy = doe - (365 * yoe + yoe / 4 - yoe / 100);
    let mp = (5 * doy + 2) / 153;
    let day = doy - (153 * mp + 2) / 5 + 1;
    let month = if mp < 10 { mp + 3 } else { mp - 9 };
    (if month <= 2 { year + 1 } else { year }, month, day)
}

fn now_iso() -> String {
    let seconds = SystemTime::now().duration_since(UNIX_EPOCH).map(|d| d.as_secs() as i64).u
    let (year, month, day) = civil_from_days(seconds.div_euclid(86400));
    let rest = seconds.rem_euclid(86400);
    format!("{year:04}-{month:02}-{day:02}T{:02}:{:02}:{:02}", rest / 3600, rest % 3600 / 60
}

fn to_json(task: &Task) -> Json {
    let mut map = BTreeMap::new();
    map.insert("id".to_string(), Json::Int(task.id));
    map.insert("title".to_string(), Json::Str(task.title.clone()));
    map.insert("description".to_string(), Json::Str(task.description.clone()));
    map.insert("done".to_string(), Json::Bool(task.done));
    map.insert("created_at".to_string(), Json::Str(task.created_at.clone()));
    Json::Object(map)
}

fn text_field(map: &BTreeMap<String, Json>, key: &str) -> Result<String, String> {
    match map.get(key) {
        Some(Json::Str(text)) => Ok(text.clone()),
        _ => Err(format!("missing field {key}")),
    }
}

fn from_json(value: &Json) -> Result<Task, String> {
    let Json::Object(map) = value else {
        return Err("a task must be an object".to_string());
    };
    let id = match map.get("id") {
        Some(Json::Int(number)) => *number,
        _ => return Err("missing field id".to_string()),
    };
    Ok(Task {
        id,
        title: text_field(map, "title")?,
        description: text_field(map, "description")?,
        done: map.get("done") == Some(&Json::Bool(true)),
```

```
        created_at: text_field(map, "created_at")?,
    })
}

fn load_todos() -> Vec<Task> {
    let text = match fs::read_to_string("todos.json") {
        Ok(text) => text,
        Err(_) => {
            println!("No existing tasks found.");
            return Vec::new();
        }
    };
    let loaded = match decode(&text) {
        Ok(Json::List(items)) => items.iter().map(from_json).collect::<Result<Vec<Task>, Str
        Ok(_) => Err("expected a list".to_string()),
        Err(message) => Err(message),
    };
    match loaded {
        Ok(tasks) => {
            println!("Tasks loaded.");
            tasks
        }
        Err(_) => {
            println!("Error: todos.json is not valid JSON. Starting with an empty list.");
            Vec::new()
        }
    }
}

fn save_todos(todos: &[Task]) -> io::Result<()> {
    let list = Json::List(todos.iter().map(to_json).collect());
    fs::write("todos.json", encode(&list))?;
    println!("Tasks saved.");
    Ok(())
}

fn add_task(todos: &mut Vec<Task>, title: &str, description: &str) {
    if title.trim().is_empty() {
        println!("A title is required.");
        return;
    }
    let id = todos.iter().map(|task| task.id).max().unwrap_or(0) + 1;
    todos.push(Task {
        id,
        title: title.trim().to_string(),
        description: description.trim().to_string(),
```

```
        done: false,
        created_at: now_iso(),
    });
    println!("Task '{}' added.", title.trim());
}

fn view_tasks(todos: &[Task]) {
    if todos.is_empty() {
        println!("No tasks in your list.");
        return;
    }
    println!("--- Your To-Do List ---");
    for task in todos {
        let status = if task.done { "done" } else { "not done" };
        println!("{}. {} ({})", task.id, task.title, status);
    }
    println!("-----------------------");
}

fn mark_done(todos: &mut [Task], text: &str) {
    let Ok(id) = text.trim().parse::<i64>() else {
        println!("Please enter a whole number.");
        return;
    };
    match todos.iter_mut().find(|task| task.id == id) {
        Some(task) => {
            task.done = true;
            println!("Task '{}' marked as done.", task.title);
        }
        None => println!("No task has that number."),
    }
}

fn show_menu() {
    println!();
    println!("--- To-Do List Menu ---");
    println!("1. Add Task");
    println!("2. View Tasks");
    println!("3. Mark Task Done");
    println!("4. Save Tasks");
    println!("5. Load Tasks");
    println!("6. Quit");
    println!("-----------------------");
}

fn next_line() -> Option<String> {
```

```
    let mut line = String::new();
    match io::stdin().read_line(&mut line) {
        Ok(0) | Err(_) => None,
        Ok(_) => Some(line.trim().to_string()),
    }
}

fn main() -> io::Result<()> {
    let mut todos = load_todos();
    loop {
        show_menu();
        println!("Enter your choice (1-6):");
        let choice = next_line().unwrap_or_else(|| "6".to_string());
        match choice.as_str() {
            "1" => {
                println!("Enter task title:");
                let title = next_line().unwrap_or_default();
                println!("Enter task description:");
                let description = next_line().unwrap_or_default();
                add_task(&mut todos, &title, &description);
            }
            "2" => view_tasks(&todos),
            "3" => {
                view_tasks(&todos);
                println!("Enter task number to mark done:");
                let text = next_line().unwrap_or_default();
                mark_done(&mut todos, &text);
            }
            "4" => save_todos(&todos)?,
            "5" => todos = load_todos(),
            "6" => {
                save_todos(&todos)?;
                println!("Goodbye!");
                return Ok(());
            }
            _ => println!("Invalid choice. Please try again."),
        }
    }
}
```

Expected Behavior or Output

With the following lines typed as standard input (piped input is not echoed, so it does not appear in the output):

```
1
Study programming
Finish Unit 10
1
Review scores
Check the four scores
2
3
2
3
7
2
6
```

the program prints:

```
No existing tasks found.

--- To-Do List Menu ---
1. Add Task
2. View Tasks
3. Mark Task Done
4. Save Tasks
5. Load Tasks
6. Quit
----------------------
Enter your choice (1-6):
Enter task title:
Enter task description:
Task 'Study programming' added.

--- To-Do List Menu ---
1. Add Task
2. View Tasks
3. Mark Task Done
4. Save Tasks
5. Load Tasks
6. Quit
----------------------
Enter your choice (1-6):
Enter task title:
Enter task description:
Task 'Review scores' added.

--- To-Do List Menu ---
1. Add Task
2. View Tasks
```

```
3. Mark Task Done
4. Save Tasks
5. Load Tasks
6. Quit
-----------------------
Enter your choice (1-6):
--- Your To-Do List ---
1. Study programming (not done)
2. Review scores (not done)
-----------------------

--- To-Do List Menu ---
1. Add Task
2. View Tasks
3. Mark Task Done
4. Save Tasks
5. Load Tasks
6. Quit
-----------------------
Enter your choice (1-6):
--- Your To-Do List ---
1. Study programming (not done)
2. Review scores (not done)
-----------------------
Enter task number to mark done:
Task 'Review scores' marked as done.

--- To-Do List Menu ---
1. Add Task
2. View Tasks
3. Mark Task Done
4. Save Tasks
5. Load Tasks
6. Quit
-----------------------
Enter your choice (1-6):
--- Your To-Do List ---
1. Study programming (not done)
2. Review scores (done)
-----------------------
Enter task number to mark done:
No task has that number.

--- To-Do List Menu ---
1. Add Task
2. View Tasks
```

```
3. Mark Task Done
4. Save Tasks
5. Load Tasks
6. Quit
-----------------------
Enter your choice (1-6):
--- Your To-Do List ---
1. Study programming (not done)
2. Review scores (done)
-----------------------

--- To-Do List Menu ---
1. Add Task
2. View Tasks
3. Mark Task Done
4. Save Tasks
5. Load Tasks
6. Quit
-----------------------
Enter your choice (1-6):
Tasks saved.
Goodbye!
```

With no standard input at all, the program ends cleanly and prints:

```
No existing tasks found.

--- To-Do List Menu ---
1. Add Task
2. View Tasks
3. Mark Task Done
4. Save Tasks
5. Load Tasks
6. Quit
-----------------------
Enter your choice (1-6):
Tasks saved.
Goodbye!
```

Common Mistakes

1. Saving the file but never loading it, so tasks disappear between runs.
2. Reusing a deleted task's id, which can confuse saved references.
3. Letting a damaged `todos.json` crash the program instead of printing a message.

Activities

1. Run the sample session, then run the program again typing `2` and `6` to see the saved tasks.
2. Add a command to delete a task by number.
3. Open `todos.json` and read the saved text. Edit it to remove a bracket and run the program again.
4. Show each task's description in the view command.

Accessibility Practice

Every menu is printed as numbered lines and every prompt is on its own line. The status words are `done` and `not done`, not symbols, so the list reads clearly in speech and braille.

Skills Checklist

- ☐ I can keep records with the shared fields.
- ☐ I can save and load JSON.
- ☐ I handle a missing or damaged file.
- ☐ I keep ids unique.
- ☐ I show status in words.

Lesson 10.5: Accessible interaction and validation

Reading References

- The Rust Programming Language: Recoverable Errors with Result
- The Rust Programming Language: Option
- Rust standard library documentation
- Matching Python lesson: Lesson 10.5: Refining User Experience and Validation

Learning Objectives

- Write reusable validation functions.
- Limit retries and handle the end of input.
- Write clear, accessible messages.

Concept Explanation

Reusable validation functions make every menu and form consistent. `ask_number(prompt, low, high, attempts)` prints a prompt, reads a line, converts it, checks the range, and either returns `Some(number)` or explains the problem and asks again. After the allowed attempts, or at the end of input, it returns `None`. `ask_yes_no(prompt)` returns `Option<bool>` and accepts `y`,

`yes`, `n`, and `no` in any letter case. The `Option` return type makes the caller deal with the failure case.

Good messages say *what was wrong* and *what to do*: "Enter a whole number from 0 to 100," not "Invalid." They never depend on color, position, or symbols alone, and they use the same wording every time.

The example reads a score with the typed lines abc, 105, and 88, then asks a yes or no question with the lines maybe and y.

Worked Example

```
use std::io;

fn next_line() -> Option<String> {
    let mut line = String::new();
    match io::stdin().read_line(&mut line) {
        Ok(0) | Err(_) => None,
        Ok(_) => Some(line.trim().to_string()),
    }
}

fn ask_number(prompt: &str, low: i32, high: i32, attempts: u32) -> Option<i32> {
    for attempt in 1..=attempts {
        println!("{prompt}");
        let text = next_line()?;
        if let Ok(number) = text.parse::<i32>() {
            if (low..=high).contains(&number) {
                return Some(number);
            }
        }
        println!("Enter a whole number from {low} to {high}. You have {} attempt(s) left.",
    }
    None
}

fn ask_yes_no(prompt: &str) -> Option<bool> {
    loop {
        println!("{prompt} (yes or no)");
        let text = next_line()?.to_lowercase();
        match text.as_str() {
            "y" | "yes" => return Some(true),
            "n" | "no" => return Some(false),
            _ => println!("Please answer yes or no."),
        }
    }
}
```

```rust
fn main() {
    match ask_number("Enter a score from 0 to 100:", 0, 100, 3) {
        Some(score) => println!("Accepted score: {score}"),
        None => println!("No valid score was entered."),
    }

    let keep = ask_yes_no("Save this score?");
    println!("Save answer: {:?}", keep);
}
```

Expected Behavior or Output

With the following lines typed as standard input (piped input is not echoed, so it does not appear in the output):

```
abc
105
88
maybe
y
```

the program prints:

```
Enter a score from 0 to 100:
Enter a whole number from 0 to 100. You have 2 attempt(s) left.
Enter a score from 0 to 100:
Enter a whole number from 0 to 100. You have 1 attempt(s) left.
Enter a score from 0 to 100:
Accepted score: 88
Save this score? (yes or no)
Please answer yes or no.
Save this score? (yes or no)
Save answer: Some(true)
```

With no standard input at all, the program ends cleanly and prints:

```
Enter a score from 0 to 100:
No valid score was entered.
Save this score? (yes or no)
Save answer: None
```

Common Mistakes

1. Writing the message ”Invalid” without saying what is valid.
2. Accepting a decimal such as 88.5 for a field that requires a whole number.
3. Looping forever on bad input instead of limiting attempts.

Activities

1. Run the sample input and read each message aloud.
2. Run with three invalid numbers and read the final message.
3. Add `ask_text(prompt) -> Option<String>` that rejects empty answers.
4. Rewrite two messages from earlier lessons so they say what to do next.

Accessibility Practice

Check that every error message ends with an action the user can take, and that the same situation always produces the same words.

Skills Checklist

- ☐ I can write reusable validation functions.
- ☐ I limit retries.
- ☐ I handle the end of input.
- ☐ I write messages that say what to do.
- ☐ I keep wording consistent.

14.13 Unit 11: Capstone

Overview

This unit develops capstone through Rust's type system and standard library. Prerequisites: prior units, a working Cargo project, and the ability to read compiler diagnostics. Preserve shared course datasets and test boundary conditions explicitly.

Lesson 11.1: Requirements and design

Reading References

- The Rust Programming Language: Structs
- The Rust Programming Language: Result
- Rust standard library documentation
- Matching Python lesson: Lesson 11.1: Introduction to NiceGUI

Learning Objectives

- Write requirements for a personal data manager.
- Define the shared record schema as a struct.
- Check records against the validation rules before building features.

Concept Explanation

The capstone is a *personal data manager*: a program that keeps a list of records, lets the user add, view, search, change, complete, and delete them, and saves them to a file. In the Rust track it is a text program, so it runs from the terminal and is fully usable with a screen reader or braille display.

Shared record schema (all general-purpose language tracks). Every record has `id` (unique whole number), `title`, `description`, `category`, `created_at`, and `updated_at` (ISO 8601 text). A project may add fields, such as an optional `done` flag, but must not remove these. Rust field names use `snake_case`, the same as the stored JSON.

Validation rules. A record must have a non-empty title and a non-empty category. Timestamps are ISO 8601 text such as `2026-01-15T09:00:00`. A `Result<(), String>` return type states in the signature that validation can fail with a message.

Design before coding: list the operations, then the data, then the messages. The example checks the three shared sample records against the rules and prints them as a plain-text list.

Worked Example

```
struct Entry {
    id: i32,
    title: String,
    description: String,
    category: String,
    created_at: String,
    updated_at: String,
}

fn validate(title: &str, category: &str) -> Result<(), String> {
    if title.trim().is_empty() {
        return Err("A title is required.".to_string());
    }
    if category.trim().is_empty() {
        return Err("A category is required.".to_string());
    }
    Ok(())
}

fn entry(id: i32, title: &str, description: &str, category: &str, created_at: &str) -> Entry
    Entry {
        id,
        title: title.to_string(),
        description: description.to_string(),
```

```
        category: category.to_string(),
        created_at: created_at.to_string(),
        updated_at: created_at.to_string(),
    }
}

fn main() {
    let sample = vec![
        entry(1, "Notebook", "Course notes", "supplies", "2026-01-15T09:00:00"),
        entry(2, "Pencil", "Box of 10", "supplies", "2026-01-15T09:05:00"),
        entry(3, "Study plan", "Weekly schedule", "planning", "2026-01-15T09:10:00"),
    ];

    for record in &sample {
        let outcome = match validate(&record.title, &record.category) {
            Ok(()) => "valid".to_string(),
            Err(message) => message,
        };
        println!("{}. {} [{}] - {}", record.id, record.title, record.category, outcome);
    }
    println!("{} | {} | {}", sample[0].description, sample[0].created_at, sample[0].updated_

    println!("{:?}", validate("  ", "supplies"));
    println!("{:?}", validate("Ruler", ""));
}
```

Expected Behavior or Output

The program prints:

```
1. Notebook [supplies] - valid
2. Pencil [supplies] - valid
3. Study plan [planning] - valid
Course notes | 2026-01-15T09:00:00 | 2026-01-15T09:00:00
Err("A title is required.")
Err("A category is required.")
```

Common Mistakes

1. Starting to build features before agreeing on the record fields.
2. Allowing a record with an empty title because the check was left for later.
3. Storing timestamps in different formats in different places.

Activities

1. Write the seven operations as a numbered list of requirements.
2. Add one project-specific field and say how it is validated.

3. Write three sample records for your own project.
4. Describe in plain sentences what each shared field means.

Accessibility Practice

Design the screen output as lines of plain text: one record per line, with the same field order each time. Decide the wording of every message now so that it can be tested.

Skills Checklist

- ☐ I can write requirements.
- ☐ I know the shared record fields.
- ☐ I can write validation rules.
- ☐ I can describe sample data.
- ☐ I can plan messages before coding.

Lesson 11.2: Data model and storage

Reading References

- The Rust Programming Language: Result
- The Rust Programming Language: Reading a File
- Rust standard library documentation
- Matching Python lesson: Lesson 11.2: Data Model and Storage

Learning Objectives

- Store records as structs.
- Save and load them as JSON.
- Confirm that every field survives a round trip.

Concept Explanation

Records are stored as a `Vec<Entry>` in memory and as a JSON array on disk. The storage layer has two functions: `save_entries(path, entries)` and `load_entries(path)`, which returns `Result<Vec<Entry>, String>` so the caller decides what to do when the file is missing or damaged.

An `Entry` is converted to a `Json::Object` for saving (`to_json`) and rebuilt from one when loading (`from_json`), which checks that each field is present and has the right type. The optional `done` field defaults to `false`.

The example uses the teaching JSON helper from Lesson 8.5 (included so the program runs on its own). After saving and reloading it compares the lists; deriving `PartialEq` lets Rust compare structs field by field, so this *round-trip test* checks every field of every entry. It is the first acceptance test for storage.

Worked Example

```
#![allow(dead_code)]
use std::collections::BTreeMap;

#[derive(Debug, Clone, PartialEq)]
enum Json {
    Null,
    Bool(bool),
    Int(i64),
    Float(f64),
    Str(String),
    List(Vec<Json>),
    Object(BTreeMap<String, Json>),
}

fn quote(text: &str) -> String {
    let mut out = String::from("\"");
    for c in text.chars() {
        match c {
            '"' => out.push_str("\\\""),
            '\\' => out.push_str("\\\\"),
            '\n' => out.push_str("\\n"),
            '\r' => out.push_str("\\r"),
            '\t' => out.push_str("\\t"),
            c if (c as u32) < 0x20 => out.push_str(&format!("\\u{:04x}", c as u32)),
            c => out.push(c),
        }
    }
    out.push('"');
    out
}

fn encode(value: &Json) -> String {
    match value {
        Json::Null => "null".to_string(),
        Json::Bool(flag) => flag.to_string(),
        Json::Int(number) => number.to_string(),
        Json::Float(number) => number.to_string(),
        Json::Str(text) => quote(text),
        Json::List(items) => {
            let parts: Vec<String> = items.iter().map(encode).collect();
            format!("[{}]", parts.join(","))
        }
        Json::Object(map) => {
            let parts: Vec<String> = map.iter().map(|(key, item)| format!("{}:{}", quote(key
```

```
            format!("{{{}}}", parts.join(","))
        }
    }
}

struct Parser {
    chars: Vec<char>,
    pos: usize,
}

impl Parser {
    fn fail<T>(&self, message: &str) -> Result<T, String> {
        Err(format!("{message} at position {}", self.pos))
    }

    fn skip(&mut self) {
        while self.pos < self.chars.len() && self.chars[self.pos].is_whitespace() {
            self.pos += 1;
        }
    }

    fn peek(&self) -> Option<char> {
        self.chars.get(self.pos).copied()
    }

    fn starts_with(&self, word: &str) -> bool {
        word.chars().enumerate().all(|(i, c)| self.chars.get(self.pos + i) == Some(&c))
    }

    fn value(&mut self) -> Result<Json, String> {
        self.skip();
        match self.peek() {
            None => self.fail("unexpected end of text"),
            Some('{') => self.object(),
            Some('[') => self.list(),
            Some('"') => Ok(Json::Str(self.string()?)),
            Some(c) if c == '-' || c.is_ascii_digit() => self.number(),
            Some(_) if self.starts_with("true") => {
                self.pos += 4;
                Ok(Json::Bool(true))
            }
            Some(_) if self.starts_with("false") => {
                self.pos += 5;
                Ok(Json::Bool(false))
            }
            Some(_) if self.starts_with("null") => {
```

```
                self.pos += 4;
                Ok(Json::Null)
            }
            Some(_) => self.fail("unexpected character"),
        }
    }

    fn number(&mut self) -> Result<Json, String> {
        let start = self.pos;
        if self.peek() == Some('-') {
            self.pos += 1;
        }
        while let Some(c) = self.peek() {
            if c.is_ascii_digit() || ".eE+-".contains(c) {
                self.pos += 1;
            } else {
                break;
            }
        }
        let token: String = self.chars[start..self.pos].iter().collect();
        if let Ok(number) = token.parse::<i64>() {
            return Ok(Json::Int(number));
        }
        match token.parse::<f64>() {
            Ok(number) => Ok(Json::Float(number)),
            Err(_) => self.fail("bad number"),
        }
    }

    fn string(&mut self) -> Result<String, String> {
        self.pos += 1;
        let mut out = String::new();
        loop {
            let Some(c) = self.peek() else {
                return self.fail("unterminated string");
            };
            self.pos += 1;
            match c {
                '"' => return Ok(out),
                '\\' => {
                    let Some(escape) = self.peek() else {
                        return self.fail("bad escape");
                    };
                    self.pos += 1;
                    match escape {
                        'n' => out.push('\n'),
```

```
                        't' => out.push('\t'),
                        'r' => out.push('\r'),
                        '"' => out.push('"'),
                        '\\' => out.push('\\'),
                        '/' => out.push('/'),
                        'u' => {
                            if self.pos + 4 > self.chars.len() {
                                return self.fail("bad escape");
                            }
                            let hex: String = self.chars[self.pos..self.pos + 4].iter().coll
                            let code = u32::from_str_radix(&hex, 16).map_err(|_| "bad escape
                            out.push(char::from_u32(code).unwrap_or('?'));
                            self.pos += 4;
                        }
                        _ => return self.fail("bad escape"),
                    }
                }
                other => out.push(other),
            }
        }
    }

    fn object(&mut self) -> Result<Json, String> {
        self.pos += 1;
        let mut map = BTreeMap::new();
        self.skip();
        if self.peek() == Some('}') {
            self.pos += 1;
            return Ok(Json::Object(map));
        }
        loop {
            self.skip();
            if self.peek() != Some('"') {
                return self.fail("expected a key");
            }
            let key = self.string()?;
            self.skip();
            if self.peek() != Some(':') {
                return self.fail("expected ':'");
            }
            self.pos += 1;
            let item = self.value()?;
            map.insert(key, item);
            self.skip();
            match self.peek() {
                None => return self.fail("unterminated object"),
```

```
                Some('}') => {
                    self.pos += 1;
                    return Ok(Json::Object(map));
                }
                Some(',') => self.pos += 1,
                Some(_) => return self.fail("expected ',' or '}'"),
            }
        }
    }

    fn list(&mut self) -> Result<Json, String> {
        self.pos += 1;
        let mut items = Vec::new();
        self.skip();
        if self.peek() == Some(']') {
            self.pos += 1;
            return Ok(Json::List(items));
        }
        loop {
            items.push(self.value()?);
            self.skip();
            match self.peek() {
                None => return self.fail("unterminated list"),
                Some(']') => {
                    self.pos += 1;
                    return Ok(Json::List(items));
                }
                Some(',') => self.pos += 1,
                Some(_) => return self.fail("expected ',' or ']'"),
            }
        }
    }
}

fn decode(text: &str) -> Result<Json, String> {
    let mut parser = Parser { chars: text.chars().collect(), pos: 0 };
    let value = parser.value()?;
    parser.skip();
    if parser.pos < parser.chars.len() {
        return parser.fail("unexpected extra text");
    }
    Ok(value)
}

use std::fs;
```

```
#[derive(Debug, Clone, PartialEq)]
struct Entry {
    id: i64,
    title: String,
    description: String,
    category: String,
    created_at: String,
    updated_at: String,
    done: bool,
}

fn to_json(entry: &Entry) -> Json {
    let mut map = BTreeMap::new();
    map.insert("id".to_string(), Json::Int(entry.id));
    map.insert("title".to_string(), Json::Str(entry.title.clone()));
    map.insert("description".to_string(), Json::Str(entry.description.clone()));
    map.insert("category".to_string(), Json::Str(entry.category.clone()));
    map.insert("created_at".to_string(), Json::Str(entry.created_at.clone()));
    map.insert("updated_at".to_string(), Json::Str(entry.updated_at.clone()));
    map.insert("done".to_string(), Json::Bool(entry.done));
    Json::Object(map)
}

fn text_field(map: &BTreeMap<String, Json>, key: &str) -> Result<String, String> {
    match map.get(key) {
        Some(Json::Str(text)) => Ok(text.clone()),
        _ => Err(format!("missing field {key}")),
    }
}

fn from_json(value: &Json) -> Result<Entry, String> {
    let Json::Object(map) = value else {
        return Err("a record must be an object".to_string());
    };
    let id = match map.get("id") {
        Some(Json::Int(number)) => *number,
        _ => return Err("missing field id".to_string()),
    };
    Ok(Entry {
        id,
        title: text_field(map, "title")?,
        description: text_field(map, "description")?,
        category: text_field(map, "category")?,
        created_at: text_field(map, "created_at")?,
        updated_at: text_field(map, "updated_at")?,
        done: map.get("done") == Some(&Json::Bool(true)),
```

```
    })
}

fn save_entries(path: &str, entries: &[Entry]) -> Result<(), String> {
    let list = Json::List(entries.iter().map(to_json).collect());
    fs::write(path, encode(&list)).map_err(|_| format!("Could not write {path}."))
}

fn load_entries(path: &str) -> Result<Vec<Entry>, String> {
    let text = fs::read_to_string(path).map_err(|_| format!("Could not open {path}: file not
    match decode(&text) {
        Ok(Json::List(items)) => items.iter().map(from_json).collect::<Result<Vec<Entry>, St
        Ok(_) => Err("The data file is not valid: expected a list".to_string()),
        Err(message) => Err(format!("The data file is not valid: {message}")),
    }
}

fn entry(id: i64, title: &str, description: &str, category: &str, stamp: &str) -> Entry {
    Entry {
        id,
        title: title.to_string(),
        description: description.to_string(),
        category: category.to_string(),
        created_at: stamp.to_string(),
        updated_at: stamp.to_string(),
        done: false,
    }
}

fn main() {
    let sample = vec![
        entry(1, "Notebook", "Course notes", "supplies", "2026-01-15T09:00:00"),
        entry(2, "Pencil", "Box of 10", "supplies", "2026-01-15T09:05:00"),
        entry(3, "Study plan", "Weekly schedule", "planning", "2026-01-15T09:10:00"),
    ];

    println!("{:?}", save_entries("records.json", &sample));
    match load_entries("records.json") {
        Ok(loaded) => {
            println!("Round trip preserved all fields: {}", loaded == sample);
            println!("Records loaded: {}", loaded.len());
        }
        Err(message) => println!("{message}"),
    }

    if let Err(message) = load_entries("missing.json") {
```

```
        println!("{message}");
    }

    fs::write("broken.json", "[{\"id\": 1,").expect("could not write the broken file");
    if let Err(message) = load_entries("broken.json") {
        println!("{message}");
    }
}
```

Expected Behavior or Output

The program prints:

```
Ok(())
Round trip preserved all fields: true
Records loaded: 3
Could not open missing.json: file not found.
The data file is not valid: expected a key at position 10
```

Common Mistakes

1. Overwriting the data file with an empty list after a failed load.
2. Saving only some of the fields.
3. Assuming a loaded record has every field without checking.

Activities

1. Run the program and open `records.json` to read the saved text.
2. Add a field to `Entry`, save it, and confirm that it survives the round trip.
3. Make `from_json` reject a record whose title is blank.
4. Explain why `load_entries` returns a `Result` and does not panic.

Accessibility Practice

Messages that name the file and the problem read clearly in speech. After saving, read the file in the terminal to confirm it contains what you expect.

Skills Checklist

- ☐ I can store records in structs.
- ☐ I can save and load JSON.
- ☐ I can run a round-trip test.
- ☐ I handle missing and damaged files.
- ☐ I do not overwrite good data after a failed load.

Lesson 11.3: Create, read, update, delete

Reading References

- The Rust Programming Language: Method Syntax
- The Rust Programming Language: Result
- Rust standard library documentation
- Matching Python lesson: Lesson 11.3: Displaying and Managing Items

Learning Objectives

- Add, list, search, change, complete, and delete records.
- Assign new ids and update timestamps correctly.
- Return a `Result` with a message when an operation cannot be done.

Concept Explanation

A `Manager` struct owns the entries and offers one method per operation. Each method that can fail returns a `Result<_, String>` whose `Err` holds the message to show. New ids are one more than the highest id, so ids are never reused while higher ids exist. Updating an entry sets `updated_at` but never changes `created_at`. A `Changes` struct with `Option` fields says which fields to change, so `Changes { done: Some(true), ..Default::default() }` changes only `done`. Completing an entry counts as an update.

For repeatable output the example passes in a *fake clock*: a boxed closure that returns a new fixed time on each call. A real program passes a closure that reads the real time.

The example adds the three shared sample records, lists them, searches for "note", changes one description, completes one entry, deletes one, and tries to delete an entry that does not exist.

Worked Example

```
#![allow(dead_code)]
struct Entry {
    id: i32,
    title: String,
    description: String,
    category: String,
    created_at: String,
    updated_at: String,
    done: bool,
}

impl Entry {
    fn describe(&self) -> String {
```

```
        format!("{}. {} [{}] ({})", self.id, self.title, self.category, if self.done { "done
    }
}

#[derive(Default)]
struct Changes {
    title: Option<String>,
    description: Option<String>,
    category: Option<String>,
    done: Option<bool>,
}

fn validate(title: &str, category: &str) -> Result<(), String> {
    if title.trim().is_empty() {
        return Err("A title is required.".to_string());
    }
    if category.trim().is_empty() {
        return Err("A category is required.".to_string());
    }
    Ok(())
}

struct Manager {
    entries: Vec<Entry>,
    clock: Box<dyn FnMut() -> String>,
}

impl Manager {
    fn new(clock: Box<dyn FnMut() -> String>) -> Manager {
        Manager { entries: Vec::new(), clock }
    }

    fn next_id(&self) -> i32 {
        self.entries.iter().map(|entry| entry.id).max().unwrap_or(0) + 1
    }

    fn find(&self, id: i32) -> Option<&Entry> {
        self.entries.iter().find(|entry| entry.id == id)
    }

    fn add(&mut self, title: &str, description: &str, category: &str) -> Result<i32, String>
        validate(title, category)?;
        let stamp = (self.clock)();
        let id = self.next_id();
        self.entries.push(Entry {
            id,
```

```
        title: title.trim().to_string(),
        description: description.trim().to_string(),
        category: category.trim().to_string(),
        created_at: stamp.clone(),
        updated_at: stamp,
        done: false,
    });
    Ok(id)
}

fn update(&mut self, id: i32, changes: Changes) -> Result<(), String> {
    let (title, category) = match self.find(id) {
        Some(entry) => (
            changes.title.clone().unwrap_or_else(|| entry.title.clone()),
            changes.category.clone().unwrap_or_else(|| entry.category.clone()),
        ),
        None => return Err("No record has that id.".to_string()),
    };
    validate(&title, &category)?;
    let stamp = (self.clock)();
    let entry = self.entries.iter_mut().find(|entry| entry.id == id).ok_or("No record ha
    if let Some(value) = changes.title {
        entry.title = value.trim().to_string();
    }
    if let Some(value) = changes.description {
        entry.description = value.trim().to_string();
    }
    if let Some(value) = changes.category {
        entry.category = value.trim().to_string();
    }
    if let Some(value) = changes.done {
        entry.done = value;
    }
    entry.updated_at = stamp;
    Ok(())
}

fn complete(&mut self, id: i32) -> Result<(), String> {
    self.update(id, Changes { done: Some(true), ..Default::default() })
}

fn delete(&mut self, id: i32) -> Result<(), String> {
    let position = self.entries.iter().position(|entry| entry.id == id).ok_or("No record
    self.entries.remove(position);
    Ok(())
}
```

```
    fn search(&self, text: &str) -> Vec<&Entry> {
        let needle = text.trim().to_lowercase();
        self.entries
            .iter()
            .filter(|entry| entry.title.to_lowercase().contains(&needle) || entry.descriptio
            .collect()
    }
}

fn fake_clock() -> Box<dyn FnMut() -> String> {
    let mut ticks = 0;
    Box::new(move || {
        ticks += 1;
        format!("2026-01-15T10:00:{ticks:02}")
    })
}

fn show(manager: &Manager) {
    for entry in &manager.entries {
        println!("{}", entry.describe());
    }
}

fn main() {
    let mut manager = Manager::new(fake_clock());
    manager.add("Notebook", "Course notes", "supplies").unwrap();
    manager.add("Pencil", "Box of 10", "supplies").unwrap();
    manager.add("Study plan", "Weekly schedule", "planning").unwrap();
    show(&manager);

    println!("Search for 'note':");
    for entry in manager.search("note") {
        println!("{}", entry.title);
    }

    manager.update(2, Changes { description: Some("Box of 12".to_string()), ..Default::defau
    manager.complete(3).unwrap();
    if let Some(entry) = manager.find(2) {
        println!("{} {} {}", entry.description, entry.created_at, entry.updated_at);
    }

    println!("{:?}", manager.delete(1));
    println!("{:?}", manager.delete(99));
    show(&manager);
}
```

Expected Behavior or Output

The program prints:

```
1. Notebook [supplies] (not done)
2. Pencil [supplies] (not done)
3. Study plan [planning] (not done)
Search for 'note':
Notebook
Box of 12 2026-01-15T10:00:02 2026-01-15T10:00:04
Ok(())
Err("No record has that id.")
2. Pencil [supplies] (not done)
3. Study plan [planning] (done)
```

Common Mistakes

1. Reusing a deleted entry's id by counting the vector length instead of using the highest id.
2. Changing `created_at` when updating a record.
3. Calling `unwrap()` on every `Result` in a real program instead of showing the message to the user.

Activities

1. Predict the output before running.
2. Add `list_by_category(category)` to `Manager`.
3. Add a rule that blocks completing an entry that is already complete, with a message.
4. Explain how the fake clock makes the output repeatable.

Accessibility Practice

List output uses the same fixed format for every record and the words `done` and `not done`. Search results print only titles, so each line is short.

Skills Checklist

- ☐ I can implement add, list, search, change, complete, and delete.
- ☐ I assign unique ids.
- ☐ I update timestamps correctly.
- ☐ I return clear messages for failures.
- ☐ I can make output repeatable with a fake clock.

Lesson 11.4: Validation and tests

Reading References

- The Rust Programming Language: How to Write Tests
- The Rust Programming Language: Test Organization
- Rust standard library documentation
- Matching Python lesson: Lesson 11.4: Testing and Refinement

Learning Objectives

- Run the shared acceptance tests.
- Verify that validation rejects bad data.
- Read and interpret test results.

Concept Explanation

The shared acceptance tests describe behavior that every language track's capstone must show:

1. Adding a record with an empty title is rejected with a clear message.
2. A new record receives an unused `id`.
3. Updating a record changes `updated_at` but not `created_at`.
4. Deleting a record removes only that record.
5. Saving and reloading preserves all fields and values (shown in Lesson 11.2).
6. Every message and control can be used without color or a mouse (checked by hand in Lesson 11.5).

This lesson automates tests 1 to 4 and also tests completion and the rejection of a blank title in an update. Each test builds a new `Manager`, so tests cannot affect each other. The fake clock keeps timestamps predictable so a test can check that `updated_at` changed while `created_at` did not. Each test is a function returning `Result<(), String>`, run by a small loop that prints PASS or FAIL.

Worked Example

```
#![allow(dead_code)]
struct Entry {
    id: i32,
    title: String,
    description: String,
    category: String,
    created_at: String,
    updated_at: String,
    done: bool,
}

impl Entry {
```

```
    fn describe(&self) -> String {
        format!("{}. {} [{}] ({})", self.id, self.title, self.category, if self.done { "done
    }
}

#[derive(Default)]
struct Changes {
    title: Option<String>,
    description: Option<String>,
    category: Option<String>,
    done: Option<bool>,
}

fn validate(title: &str, category: &str) -> Result<(), String> {
    if title.trim().is_empty() {
        return Err("A title is required.".to_string());
    }
    if category.trim().is_empty() {
        return Err("A category is required.".to_string());
    }
    Ok(())
}

struct Manager {
    entries: Vec<Entry>,
    clock: Box<dyn FnMut() -> String>,
}

impl Manager {
    fn new(clock: Box<dyn FnMut() -> String>) -> Manager {
        Manager { entries: Vec::new(), clock }
    }

    fn next_id(&self) -> i32 {
        self.entries.iter().map(|entry| entry.id).max().unwrap_or(0) + 1
    }

    fn find(&self, id: i32) -> Option<&Entry> {
        self.entries.iter().find(|entry| entry.id == id)
    }

    fn add(&mut self, title: &str, description: &str, category: &str) -> Result<i32, String>
        validate(title, category)?;
        let stamp = (self.clock)();
        let id = self.next_id();
        self.entries.push(Entry {
```

```
            id,
            title: title.trim().to_string(),
            description: description.trim().to_string(),
            category: category.trim().to_string(),
            created_at: stamp.clone(),
            updated_at: stamp,
            done: false,
        });
        Ok(id)
    }

    fn update(&mut self, id: i32, changes: Changes) -> Result<(), String> {
        let (title, category) = match self.find(id) {
            Some(entry) => (
                changes.title.clone().unwrap_or_else(|| entry.title.clone()),
                changes.category.clone().unwrap_or_else(|| entry.category.clone()),
            ),
            None => return Err("No record has that id.".to_string()),
        };
        validate(&title, &category)?;
        let stamp = (self.clock)();
        let entry = self.entries.iter_mut().find(|entry| entry.id == id).ok_or("No record ha
        if let Some(value) = changes.title {
            entry.title = value.trim().to_string();
        }
        if let Some(value) = changes.description {
            entry.description = value.trim().to_string();
        }
        if let Some(value) = changes.category {
            entry.category = value.trim().to_string();
        }
        if let Some(value) = changes.done {
            entry.done = value;
        }
        entry.updated_at = stamp;
        Ok(())
    }

    fn complete(&mut self, id: i32) -> Result<(), String> {
        self.update(id, Changes { done: Some(true), ..Default::default() })
    }

    fn delete(&mut self, id: i32) -> Result<(), String> {
        let position = self.entries.iter().position(|entry| entry.id == id).ok_or("No record
        self.entries.remove(position);
        Ok(())
```

```
    }

    fn search(&self, text: &str) -> Vec<&Entry> {
        let needle = text.trim().to_lowercase();
        self.entries
            .iter()
            .filter(|entry| entry.title.to_lowercase().contains(&needle) || entry.descriptio
            .collect()
    }
}

fn fake_clock() -> Box<dyn FnMut() -> String> {
    let mut ticks = 0;
    Box::new(move || {
        ticks += 1;
        format!("2026-01-15T10:00:{ticks:02}")
    })
}

fn expect(condition: bool, message: &str) -> Result<(), String> {
    if condition {
        Ok(())
    } else {
        Err(message.to_string())
    }
}

fn fresh() -> Manager {
    let mut manager = Manager::new(fake_clock());
    manager.add("Notebook", "Course notes", "supplies").unwrap();
    manager.add("Pencil", "Box of 10", "supplies").unwrap();
    manager.add("Study plan", "Weekly schedule", "planning").unwrap();
    manager
}

fn empty_title_is_rejected() -> Result<(), String> {
    let mut manager = fresh();
    expect(manager.add("   ", "x", "supplies") == Err("A title is required.".to_string()), "
    expect(manager.entries.len() == 3, "the list should be unchanged")
}

fn new_record_gets_an_unused_id() -> Result<(), String> {
    let mut manager = fresh();
    manager.delete(2)?;
    let id = manager.add("Ruler", "30 cm", "supplies")?;
    expect(id == 4, "expected id 4")
```

```
}

fn update_changes_updated_at_but_not_created_at() -> Result<(), String> {
    let mut manager = fresh();
    let (created, updated) = {
        let entry = manager.find(1).ok_or("record 1 missing")?;
        (entry.created_at.clone(), entry.updated_at.clone())
    };
    manager.update(1, Changes { description: Some("Lecture notes".to_string()), ..Default::d
    let entry = manager.find(1).ok_or("record 1 missing")?;
    expect(entry.created_at == created, "created_at changed")?;
    expect(entry.updated_at != updated, "updated_at did not change")
}

fn delete_removes_only_that_record() -> Result<(), String> {
    let mut manager = fresh();
    manager.delete(2)?;
    let ids: Vec<i32> = manager.entries.iter().map(|entry| entry.id).collect();
    expect(ids == vec![1, 3], "wrong records remain")
}

fn complete_marks_done_and_updates_the_record() -> Result<(), String> {
    let mut manager = fresh();
    manager.complete(3)?;
    let entry = manager.find(3).ok_or("record 3 missing")?;
    expect(entry.done && entry.updated_at != entry.created_at, "completion not recorded")
}

fn blank_title_cannot_be_saved_by_update() -> Result<(), String> {
    let mut manager = fresh();
    let result = manager.update(1, Changes { title: Some(String::new()), ..Default::default(
    expect(result == Err("A title is required.".to_string()), "update accepted a blank title
}

fn main() {
    let tests: [(&str, fn() -> Result<(), String>); 6] = [
        ("1. an empty title is rejected", empty_title_is_rejected),
        ("2. a new record gets an unused id", new_record_gets_an_unused_id),
        ("3. update changes updated_at but not created_at", update_changes_updated_at_but_no
        ("4. delete removes only that record", delete_removes_only_that_record),
        ("5. complete marks done and updates the record", complete_marks_done_and_updates_th
        ("6. a blank title cannot be saved by update", blank_title_cannot_be_saved_by_update
    ];

    let mut failures = 0;
    for (name, test) in tests {
```

```
        match test() {
            Ok(()) => println!("PASS {name}"),
            Err(message) => {
                failures += 1;
                println!("FAIL {name}: {message}");
            }
        }
    }
    if failures == 0 {
        println!("All acceptance tests passed.");
    } else {
        println!("{failures} test(s) failed.");
    }
}
```

Expected Behavior or Output

The program prints:

```
PASS 1. an empty title is rejected
PASS 2. a new record gets an unused id
PASS 3. update changes updated_at but not created_at
PASS 4. delete removes only that record
PASS 5. complete marks done and updates the record
PASS 6. a blank title cannot be saved by update
All acceptance tests passed.
```

Common Mistakes

1. Sharing one manager between tests, so one test's deletions change the next test.
2. Checking only that no error occurred, and not checking the result.
3. Writing the expected message differently from the program, so a correct program fails the test.

Activities

1. Break `Manager::update` so it also changes `created_at` and watch test 3 fail. Restore it.
2. Add a test that searching ignores case.
3. Add a test that deleting an unknown id returns a message.
4. Explain why test 2 deletes record 2 before adding the new one.

Accessibility Practice

The test names begin with the acceptance-test number, so PASS and FAIL lines can be matched to the specification. Read only the first word of each line to

hear the overall result quickly.

Skills Checklist

- ☐ I can run the shared acceptance tests.
- ☐ I can test validation failures.
- ☐ I can test timestamps with a fake clock.
- ☐ I can read PASS and FAIL lines.
- ☐ I use a fresh manager for every test.

Lesson 11.5: Documentation and presentation

Reading References

- Rust standard library documentation
- Unit P, Lesson P.6: files, data formats, and the shared accessible workflow
- Matching Python lesson: Lesson 11.5: Final Submission

Learning Objectives

- Write a plain-text project summary.
- Report a count of records by category.
- Complete a manual accessibility check.

Concept Explanation

Documentation tells a future reader what the program does and how to run it. Write it as plain text or Markdown with short headings, numbered steps, and complete sentences. Include: the purpose of the program, the `cargo` commands to build and run it, the shared record fields, the commands a user can give, the accessibility checks you carried out, and what you would improve next.

The example generates a summary from the shared sample data. It counts records by category in a `BTreeMap`, which keeps its keys sorted, so the output is stable. It then prints the manual accessibility checklist. The unchecked boxes `[ ]` are for *you* to complete by hand with your own access technology; the program cannot verify them.

Acceptance test 6 is a manual check: every message and control can be used without color or a mouse. Test it with the keyboard only, and with the screen reader or braille display you actually use. Record what you tested and its version; do not claim compatibility with software you did not test.

Worked Example

```
use std::collections::BTreeMap;
```

```rust
struct Entry {
    id: i32,
    title: String,
    category: String,
}

fn main() {
    let sample = vec![
        Entry { id: 1, title: "Notebook".to_string(), category: "supplies".to_string() },
        Entry { id: 2, title: "Pencil".to_string(), category: "supplies".to_string() },
        Entry { id: 3, title: "Study plan".to_string(), category: "planning".to_string() },
    ];

    let mut counts: BTreeMap<&str, u32> = BTreeMap::new();
    for entry in &sample {
        *counts.entry(entry.category.as_str()).or_insert(0) += 1;
    }

    println!("Personal Data Manager - Summary");
    println!("Records: {}", sample.len());
    println!("Categories:");
    for (category, count) in &counts {
        println!("- {category}: {count}");
    }
    let titles: Vec<String> = sample.iter().map(|entry| format!("{} ({})", entry.title, entr
    println!("Titles: {}", titles.join(", "));

    println!();
    println!("Manual accessibility checklist (complete this by hand):");
    let checks = [
        "[ ] Every command can be used with the keyboard only.",
        "[ ] Every message is a complete sentence that says what to do next.",
        "[ ] No meaning depends on color, position, or symbols alone.",
        "[ ] The output was read with my own screen reader or braille display.",
        "[ ] I recorded the operating system, program, and version I tested.",
    ];
    for check in checks {
        println!("{check}");
    }
}
```

Expected Behavior or Output

The program prints:

```
Personal Data Manager - Summary
Records: 3
Categories:
- planning: 1
- supplies: 2
Titles: Notebook (1), Pencil (2), Study plan (3)

Manual accessibility checklist (complete this by hand):
[ ] Every command can be used with the keyboard only.
[ ] Every message is a complete sentence that says what to do next.
[ ] No meaning depends on color, position, or symbols alone.
[ ] The output was read with my own screen reader or braille display.
[ ] I recorded the operating system, program, and version I tested.
```

Common Mistakes

1. Writing documentation that only the author can follow.
2. Claiming that a program is accessible without recording what was tested and how.
3. Letting the documentation drift from the program after later changes.

Activities

1. Write the project summary for your own capstone using the headings above.
2. Run the keyboard-only check and record each problem you find.
3. Ask a classmate to follow your run instructions from a fresh folder and note where they get stuck.
4. Write three improvements for the next version, ordered by user benefit.

Accessibility Practice

This is the final accessibility lesson. Read your own documentation aloud from start to finish. Anything you have to look at the screen to understand needs rewriting.

Skills Checklist

- ☐ I can write a plain-text project summary.
- ☐ I can report counts by category.
- ☐ I can run a manual accessibility check.
- ☐ I record what I tested.
- ☐ I do not claim untested compatibility.

Course-Wide Assessment and Portfolio

Submit a Cargo project that compiles with the Rust 2024 edition, passes cargo fmt --check, cargo check, and cargo test, and documents its data model, input validation, error handling, persistence format, and keyboard/screen-reader workflow. Include a README with exact build, test, and run commands.

AI Policy

AI tools may explain compiler diagnostics, compare ownership strategies, and suggest tests. Learners must explain every ownership transfer, borrow, trait bound, error path, and dependency. Compile and test generated code, review dependency provenance, and do not expose private data or credentials. AI output is a draft, not evidence of correctness.

Rust-Specific Verification Notes

The JSON example demonstrates the serialized record shape only; production code should use serde_json. The deterministic pseudo-random example is not cryptographically secure and is not a substitute for a maintained random-number generator.

Part XV

SWIFT

Chapter 15

Swift Curriculum

Before you start: complete Unit P: Computing Foundations Primer. It explains the language-neutral vocabulary and concepts used in every language track, using non-visual examples.

Swift is used for iOS, macOS, watchOS, tvOS, server applications, and systems components. Its static type system, optionals, value semantics, protocols, structured error handling, Codable, and concurrency support robust application development.

Course structure

A complete course aligned to the Python curriculum's Units 0–11, including four environment/toolchain lessons and 53 numbered lessons. The worked examples reuse shared course contexts: Avery's profile, calculator and tip values, grades, scores, inventory, coordinates, text samples, CSV fields, task records, text-adventure requirements, and personal-record management.

Environment

- Baseline: Swift 6 language mode and a current Swift 6 toolchain.
- Verify with `swift --version` and `swiftc --version`.
- Run a single source file with `swift main.swift`.
- Initialize a package with `swift package init --type executable`; build, run, and test with `swift build`, `swift run`, and `swift test`.
- Xcode and Apple platform SDKs are required for iOS simulator and Apple-platform application development.
- Use Foundation for file operations and Codable JSON encoding. Use a standards-aware CSV library for quoted fields and embedded newlines.

Complete the shared IDE setup lesson and terminal prerequisite. The course supports keyboard-first work with screen readers, braille displays, magnification, and other access technology.

- Complete lessons
- Topic index
- Example validation instructions

GUI, visualization, and workbench accessibility

See the shared GUI, visualization, and workbench accessibility reference for screen-reader support by toolkit, platform-specific activation settings, recommended alternatives, and interfaces that must not be required without an equivalent accessible route.

AI Policy

AI may be used only for personal conceptual understanding and reinforcement. It must not write, complete, autocomplete, repair, or generate course code or assessment solutions.

Recommended introductory books

- Mathias, Matthew, John Gallagher, and Mikey Ward. *Swift Programming: The Big Nerd Ranch Guide*, 3rd edition. Big Nerd Ranch Guides, 2020 (updated e-text edition, 2022). A structured introduction to Swift fundamentals, collections, functions, protocols, error handling, memory management, and application development. Publisher page.
- Apple. *Develop in Swift* learning materials. Apple's free curriculum materials introduce Swift and app development through guided projects. Apple Education.

Suggested use: use the Big Nerd Ranch guide for language concepts and Apple's materials for platform-specific context. The book predates Swift 6 language mode; follow the curriculum's Swift 6 baseline and current Swift documentation where concurrency or compiler diagnostics differ.

15.1 SWIFT Lessons

The curriculum is organized as one Markdown file per unit.

- accessible environment and toolchain
- foundations of computing
- numeric computations
- control flow

- data structures
- input and iteration
- functions
- object oriented programming
- files and exceptions
- testing and data analysis
- project based learning
- capstone

15.2 Unit 0: Accessible Environment and Toolchain

Course overview

Before you start: complete Unit P: Computing Foundations Primer. It explains the language-neutral vocabulary and concepts used in every language track, using non-visual examples.

Course Overview

Swift is used for iOS, macOS, watchOS, tvOS, server applications, and systems components. Its static type system, optionals, value semantics, protocols, structured error handling, Codable, and concurrency support robust application development. This course follows the Python curriculum's unit sequence and shared problem contexts while teaching Swift-native design.

Version baseline: Swift 6 language mode and a current Swift 6 toolchain. Verify with `swift --version`. Run a single file with `swift main.swift`; use Swift Package Manager for multi-file projects and dependencies.

Unit 0: Accessible Environment and Toolchain

Overview

Install and verify the Swift toolchain, create a predictable workspace, configure an accessible editor, and understand Swift Package Manager. On macOS, Xcode provides the integrated toolchain and simulator; supported Linux and Windows distributions can use the Swift toolchain for command-line development. iOS application development requires macOS and Xcode.

Lesson 0.1: Install and Verify Swift

Reading References

- Swift.org downloads
- The Swift Programming Language

Learning Objectives

- Install Swift 6 or Xcode with the Swift 6 toolchain.
- Verify compiler and interpreter availability.
- Distinguish the Swift REPL, script execution, and compilation.

Concept Explanation On macOS, install Xcode or the official Swift toolchain. On supported Linux or Windows systems, follow the official Swift installation instructions for the platform. Open a new terminal and run `swift --version` and `swiftc --version`. Run a source file with `swift main.swift`; compile an executable with `swiftc main.swift -o main` and run it with `./main`. Xcode is required for Apple-platform SDKs and iOS simulators.

Worked Example

```
print("Swift toolchain is available.")
print("Run swift --version to inspect the installed version.")
```

Expected Behavior or Output The program prints two lines confirming that the source file executes. The terminal version command reports the installed compiler version.

Common Mistakes

- Using a terminal session whose PATH has not refreshed after installation.
- Assuming the Swift REPL, script interpreter, compiler, and Xcode are interchangeable tools.
- Expecting iOS simulator tooling on a non-macOS host.

Activities

1. Install Swift or Xcode using the official platform instructions.
2. Record the output of `swift --version` and `swiftc --version`.
3. Run the worked example as a script and compile it as an executable.
4. Deliberately use an incorrect filename and read the compiler or runtime diagnostic.

Accessibility Practice Navigate the installer and terminal by keyboard. Confirm that prompts, compiler messages, punctuation, and build output are announced by the learner's screen reader or braille display.

Skills Checklist

☐ I can install and identify a supported Swift toolchain.
☐ I can report the installed Swift version.
☐ I can run a Swift script from the terminal.
☐ I can compile and execute a standalone Swift program.

☐ I can read terminal output and diagnostics using my access technology.

Lesson 0.2: Workspace and First Run

Reading References

- Swift command-line usage
- Swift Package Manager

Learning Objectives

- Create and identify a project root and Swift source file.
- Distinguish source files from generated build artifacts.
- Explain how the process working directory affects relative file paths.

Concept Explanation Create a course directory and save `main.swift` in its root. Use `pwd` in the terminal to confirm the current directory. Swift resolves relative file paths from the process working directory, not automatically from the source file's directory. Keep source, test, data, and generated files in predictable locations. Package builds create a `.build` directory.

Worked Example

```
import Foundation
print("Project: Swift course workspace")
print("Working directory: \(FileManager.default.currentDirectoryPath)")
```

Expected Behavior or Output The program prints a project label and the absolute path of the directory from which it is running.

Common Mistakes

- Opening a parent directory instead of the intended project root.
- Assuming relative file paths are based on the source file's location.
- Editing or deleting generated files under `.build` instead of source files.

Activities

1. Create the project directory and verify it with `pwd`.
2. Save and run `main.swift` from the project root.
3. Change to a different working directory and observe how a relative path resolves.
4. Create a Swift package and identify its source and build directories.

Accessibility Practice Use terminal history and directory commands without drag-and-drop. Verify that editor file navigation, indentation, build output, and diagnostics are exposed in a linear reading order.

Skills Checklist

- ☐ I can identify the project root and current working directory.
- ☐ I can create and run `main.swift`.
- ☐ I can explain how relative paths are resolved.
- ☐ I can distinguish source files from generated build artifacts.
- ☐ I can navigate the workspace without mouse-only actions.

Lesson 0.3: Editor and Accessibility Workflow

Reading References

- Swift tools and editors
- Shared IDE setup lesson

Learning Objectives

- Configure VS Code, VSCodium, Theia, or Xcode for Swift development.
- Navigate source, diagnostics, terminal, and project files by keyboard.
- Verify that editor language tooling uses the intended Swift toolchain.

Concept Explanation Open the project root in the chosen editor and install compatible Swift language support. On macOS, Xcode provides integrated Swift editing and debugging. In VS Code or VSCodium, configure the Swift extension and sourcekit-lsp using the selected toolchain; extension availability differs across marketplaces. Theia may require a compatible extension distribution. Verify diagnostics and symbol navigation rather than relying on syntax coloring alone.

Worked Example

```swift
func greet(_ name: String) -> String {
    return "Hello, \(name)!"
}
print(greet("Avery"))
```

Expected Behavior or Output The program prints `Hello, Avery!`. The editor should identify the function and its parameter type.

Common Mistakes

- Selecting a different Swift toolchain in the editor than in the terminal.
- Treating syntax coloring as proof that SourceKit-LSP diagnostics work.
- Depending on mouse-only navigation or inaccessible inline diagnostic panels.

Activities

1. Open the course project root in the selected editor.
2. Navigate the worked example by line, word, and character.
3. Introduce a type error, locate the diagnostic, and correct it.
4. Run the file from the integrated terminal and system terminal.

Accessibility Practice Test punctuation verbosity, indentation announcements, completion popups, diagnostic navigation, terminal focus, and editor panels with the learner's access technology. Keep a terminal-first workflow available.

Skills Checklist

- ☐ I can open the project root and select the intended Swift toolchain.
- ☐ I can navigate and edit Swift source by keyboard.
- ☐ I can locate and correct a compiler or language-server diagnostic.
- ☐ I can run source code without mouse-dependent actions.
- ☐ I can describe editor accessibility limitations and use a workable alternative.

Lesson 0.4: Project Environments, Dependencies, and Terminal Usage

Reading References

- Swift Package Manager
- PackageDescription API

Learning Objectives

- Create and inspect a Swift package.
- Explain the roles of `Package.swift`, source targets, and test targets.
- Add and resolve dependencies with Swift Package Manager.
- Build, test, and run a package from the terminal.

Concept Explanation Use Swift Package Manager for multi-file applications and dependencies. Run `swift package init --type executable` from the project root. The package manifest `Package.swift` declares products, targets, and dependencies. Run `swift package resolve`, `swift build`, `swift test`, and `swift run`. Package artifacts are generated under `.build`; do not edit them directly. For Apple platform applications, Xcode projects and platform SDK configuration may also be required.

Worked Example

```
let manifestResponsibilities = ["package name", "products", "targets", "dependencies"]
print("Package.swift declares: \(manifestResponsibilities.joined(separator: ", "))")
```

Expected Behavior or Output This manifest declares an executable package target named `CourseDemo`. Swift Package Manager uses it to determine how to build the package.

Common Mistakes

- Running package commands from a directory that does not contain `Package.swift`.
- Editing generated files in `.build` instead of the package sources or manifest.
- Adding a dependency without declaring the product in the appropriate target.

Activities

1. Initialize an executable package and inspect its directory structure.
2. Run `swift build` and `swift run`.
3. Add a test target and run `swift test`.
4. Add a small external dependency in a practice package and inspect the manifest and resolved package graph.

Accessibility Practice Read `Package.swift` as source text and inspect build/test output sequentially. Document commands and error messages; do not rely on transient editor notifications or mouse-only build controls.

Skills Checklist

☐ I can initialize a Swift package.
☐ I can explain the purpose of `Package.swift` and its targets.
☐ I can resolve declared package dependencies.
☐ I can build, run, and test from the terminal.
☐ I can inspect package output using keyboard-accessible tools.

15.3 Unit 1: Foundations of Computing

Overview

Computer science expresses problems as precise inputs, transformations, and outputs. A program executes statements in order unless control flow changes that order. Swift is statically typed with type inference; `let` creates an immutable binding and `var` a mutable binding.

Lesson 1.1: Computer science and problem solving

Reading References

- The Swift Programming Language

- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain computer science and problem solving using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

Computer science expresses problems as precise inputs, transformations, and outputs. A program executes statements in order unless control flow changes that order. Swift is statically typed with type inference; `let` creates an immutable binding and `var` a mutable binding.

Lesson focus: Computer science and problem solving. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```
let problem = "Calculate a class average"
let inputs = ["82", "91", "76", "88"]
let steps = ["collect scores", "validate each score", "calculate the mean", "report the resu
for (index, step) in steps.enumerated() {
    print("\(index + 1). \(step)")
}
print("Problem: \(problem); inputs: \(inputs.count)\n")
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- Prefer `let` for bindings that never change; use `var` only when mutation is required.
- Swift uses static typing with inference; a binding's type is fixed after initialization.
- Do not assume Python indexing, collection, or string semantics carry over.

Activities

1. Trace the algorithm and state the input, process, and output.
2. Change the canonical learner data and predict the printed result.

3. Write a short sequence of steps for calculating a class average.
4. Run the example and explain its output without syntax highlighting.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

☐ I can explain computer science and problem solving in Swift terminology.
☐ I can trace the example and predict its behavior.
☐ I can adapt the example to the shared canonical data.
☐ I can identify and correct a Swift-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 1.2: Your first program and sequential execution

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain your first program and sequential execution using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

Computer science expresses problems as precise inputs, transformations, and outputs. A program executes statements in order unless control flow changes that order. Swift is statically typed with type inference; `let` creates an immutable binding and `var` a mutable binding.

Lesson focus: Your first program and sequential execution. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```
print("Hello, Swift!")
print("Swift version is reported by the swift --version command.")
```

```swift
let greeting = "Welcome to the course"
print(greeting)
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- Prefer `let` for bindings that never change; use `var` only when mutation is required.
- Swift uses static typing with inference; a binding's type is fixed after initialization.
- Do not assume Python indexing, collection, or string semantics carry over.

Activities

1. Trace the algorithm and state the input, process, and output.
2. Change the canonical learner data and predict the printed result.
3. Write a short sequence of steps for calculating a class average.
4. Run the example and explain its output without syntax highlighting.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

- ☐ I can explain your first program and sequential execution in Swift terminology.
- ☐ I can trace the example and predict its behavior.
- ☐ I can adapt the example to the shared canonical data.
- ☐ I can identify and correct a Swift-specific error or edge case.
- ☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 1.3: Variables, constants, type inference, and primitive types

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain variables, constants, type inference, and primitive types using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

Computer science expresses problems as precise inputs, transformations, and outputs. A program executes statements in order unless control flow changes that order. Swift is statically typed with type inference; `let` creates an immutable binding and `var` a mutable binding.

Lesson focus: Variables, constants, type inference, and primitive types. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```
let studentName: String = "Avery"
var completedLessons = 3
let averageScore = 84.25
let isEnrolled = true
completedLessons += 1
print("\(studentName): \(completedLessons), average \(averageScore), enrolled \(isEnrolled)"
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- Prefer `let` for bindings that never change; use `var` only when mutation is required.
- Swift uses static typing with inference; a binding's type is fixed after initialization.
- Do not assume Python indexing, collection, or string semantics carry over.

Activities

1. Trace the algorithm and state the input, process, and output.
2. Change the canonical learner data and predict the printed result.
3. Write a short sequence of steps for calculating a class average.
4. Run the example and explain its output without syntax highlighting.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

- ☐ I can explain variables, constants, type inference, and primitive types in Swift terminology.
- ☐ I can trace the example and predict its behavior.
- ☐ I can adapt the example to the shared canonical data.
- ☐ I can identify and correct a Swift-specific error or edge case.
- ☐ I can run and inspect the program using keyboard-accessible tools.

15.4 Unit 2: Numeric Computations

Overview

Design calculations by naming inputs, deriving intermediate values, and verifying results. Swift's numeric operators do not perform implicit conversions between integer and floating-point types. `readLine()` returns `String?`; unwrap or validate it before use. Foundation supplies formatting and common mathematical functions.

Lesson 2.1: Software design and tracing

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain software design and tracing using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

Design calculations by naming inputs, deriving intermediate values, and verifying results. Swift's numeric operators do not perform implicit conversions between integer and floating-point types. `readLine()` returns `String?`; unwrap or validate it before use. Foundation supplies formatting and common mathematical functions.

Lesson focus: Software design and tracing. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```swift
let scores = [82.0, 91.0, 76.0, 88.0]
let total = scores.reduce(0, +)
let average = total / Double(scores.count)
print("total=\(total), count=\(scores.count), average=\(average)")
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- Swift does not implicitly convert between numeric types; convert explicitly.
- `readLine()` returns an optional string, not a guaranteed value.
- Use `Double` for fractional values and account for floating-point precision.

Activities

1. Trace the calculator example and calculate each intermediate value.
2. Calculate a $24.50 bill with a 15% tip and compare formatted output.
3. Test zero, negative, and fractional inputs where meaningful.
4. Refactor a calculation into a named function and verify it.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

- ☐ I can explain software design and tracing in Swift terminology.
- ☐ I can trace the example and predict its behavior.
- ☐ I can adapt the example to the shared canonical data.
- ☐ I can identify and correct a Swift-specific error or edge case.
- ☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 2.2: Expressions, operators, and assignment

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain expressions, operators, and assignment using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

Design calculations by naming inputs, deriving intermediate values, and verifying results. Swift's numeric operators do not perform implicit conversions between integer and floating-point types. `readLine()` returns `String?`; unwrap or validate it before use. Foundation supplies formatting and common mathematical functions.

Lesson focus: Expressions, operators, and assignment. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```swift
import Foundation
let subtotal = 24.50
let tipRate = 0.15
let tip = subtotal * tipRate
let total = subtotal + tip
print(String(format: "subtotal=$%.2f tip=$%.2f total=$%.2f", subtotal, tip, total))
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- Swift does not implicitly convert between numeric types; convert explicitly.
- `readLine()` returns an optional string, not a guaranteed value.
- Use `Double` for fractional values and account for floating-point precision.

Activities

1. Trace the calculator example and calculate each intermediate value.
2. Calculate a $24.50 bill with a 15% tip and compare formatted output.
3. Test zero, negative, and fractional inputs where meaningful.
4. Refactor a calculation into a named function and verify it.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

☐ I can explain expressions, operators, and assignment in Swift terminology.
☐ I can trace the example and predict its behavior.
☐ I can adapt the example to the shared canonical data.
☐ I can identify and correct a Swift-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 2.3: Input and output

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain input and output using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

Design calculations by naming inputs, deriving intermediate values, and verifying results. Swift's numeric operators do not perform implicit conversions between integer and floating-point types. `readLine()` returns `String?`; unwrap or validate it before use. Foundation supplies formatting and common mathematical functions.

Lesson focus: Input and output. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```
print("Enter a learner name:")
let name = readLine() ?? ""
print("Hello, \(name.isEmpty ? "learner" : name)")
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- Swift does not implicitly convert between numeric types; convert explicitly.
- `readLine()` returns an optional string, not a guaranteed value.
- Use `Double` for fractional values and account for floating-point precision.

Activities

1. Trace the calculator example and calculate each intermediate value.
2. Calculate a $24.50 bill with a 15% tip and compare formatted output.
3. Test zero, negative, and fractional inputs where meaningful.
4. Refactor a calculation into a named function and verify it.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

☐ I can explain input and output in Swift terminology.
☐ I can trace the example and predict its behavior.
☐ I can adapt the example to the shared canonical data.
☐ I can identify and correct a Swift-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 2.4: Numeric types, conversions, and math

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain numeric types, conversions, and math using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

Design calculations by naming inputs, deriving intermediate values, and verifying results. Swift's numeric operators do not perform implicit conversions between integer and floating-point types. `readLine()` returns `String?`; unwrap or validate it before use. Foundation supplies formatting and common mathematical functions.

Lesson focus: Numeric types, conversions, and math. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```swift
let wholeNumber = 7
let preciseNumber = Double(wholeNumber) / 2.0
let rounded = Int(preciseNumber.rounded())
print("precise=\(preciseNumber), rounded=\(rounded)")
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- Swift does not implicitly convert between numeric types; convert explicitly.
- `readLine()` returns an optional string, not a guaranteed value.
- Use `Double` for fractional values and account for floating-point precision.

Activities

1. Trace the calculator example and calculate each intermediate value.
2. Calculate a $24.50 bill with a 15% tip and compare formatted output.
3. Test zero, negative, and fractional inputs where meaningful.
4. Refactor a calculation into a named function and verify it.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and

read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

- ☐ I can explain numeric types, conversions, and math in Swift terminology.
- ☐ I can trace the example and predict its behavior.
- ☐ I can adapt the example to the shared canonical data.
- ☐ I can identify and correct a Swift-specific error or edge case.
- ☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 2.5: Using the Math Library

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain using the math library using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

Design calculations by naming inputs, deriving intermediate values, and verifying results. Swift's numeric operators do not perform implicit conversions between integer and floating-point types. `readLine()` returns `String?`; unwrap or validate it before use. Foundation supplies formatting and common mathematical functions.

Lesson focus: Using the Math Library. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```
import Foundation
let radius = 3.0
let area = Double.pi * pow(radius, 2)
let circumference = 2 * Double.pi * radius
let distance = hypot(3.0, 4.0)
print(String(format: "area=%.3f circumference=%.3f distance=%.1f", area, circumference, dist
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- Swift does not implicitly convert between numeric types; convert explicitly.
- `readLine()` returns an optional string, not a guaranteed value.
- Use `Double` for fractional values and account for floating-point precision.

Activities

1. Trace the calculator example and calculate each intermediate value.
2. Calculate a $24.50 bill with a 15% tip and compare formatted output.
3. Test zero, negative, and fractional inputs where meaningful.
4. Refactor a calculation into a named function and verify it.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

☐ I can explain using the math library in Swift terminology.
☐ I can trace the example and predict its behavior.
☐ I can adapt the example to the shared canonical data.
☐ I can identify and correct a Swift-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

15.5 Unit 3: Control Flow

Overview

Swift uses `Bool` conditions, `if/else if/else`, `switch`, zero-based arrays, and `for-in` loops. Array subscripting requires a valid index. Strings are collections of extended grapheme clusters; use String APIs rather than assuming integer offsets are byte offsets.

Lesson 3.1: Boolean expressions and if statements

Reading References

- The Swift Programming Language

- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain boolean expressions and if statements using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

Swift uses `Bool` conditions, `if`/`else if`/`else`, `switch`, zero-based arrays, and `for-in` loops. Array subscripting requires a valid index. Strings are collections of extended grapheme clusters; use String APIs rather than assuming integer offsets are byte offsets.

Lesson focus: Boolean expressions and if statements. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```swift
let score = 82
let isPassing = score >= 60 && score <= 100
let hasPerfectScore = score == 100
if isPassing {
    print("Passing: \(score)")
} else {
    print("Needs review")
}
print("Perfect: \(hasPerfectScore)")
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- Swift Boolean conditions must be `Bool`; integers are not truthy or falsy.
- Array indices are zero-based and valid indices stop before `count`.
- Handle optional lookups and user input instead of force-unwrapping.

Activities

1. Test scores 82, 91, 76, and 88 at grade boundaries.
2. Test empty and single-element collections.

3. Trace each loop iteration and its index.
4. Normalize the canonical text sample and search it case-insensitively.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

☐ I can explain boolean expressions and if statements in Swift terminology.
☐ I can trace the example and predict its behavior.
☐ I can adapt the example to the shared canonical data.
☐ I can identify and correct a Swift-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 3.2: Chained conditionals and grade classification

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain chained conditionals and grade classification using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

Swift uses `Bool` conditions, `if`/`else if`/`else`, `switch`, zero-based arrays, and `for-in` loops. Array subscripting requires a valid index. Strings are collections of extended grapheme clusters; use String APIs rather than assuming integer offsets are byte offsets.

Lesson focus: Chained conditionals and grade classification. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```
let score = 91
let grade: String
```

```swift
if score >= 90 { grade = "A" }
else if score >= 80 { grade = "B" }
else if score >= 70 { grade = "C" }
else if score >= 60 { grade = "D" }
else { grade = "F" }
print("Score \(score) earns \(grade)")
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- Swift Boolean conditions must be `Bool`; integers are not truthy or falsy.
- Array indices are zero-based and valid indices stop before `count`.
- Handle optional lookups and user input instead of force-unwrapping.

Activities

1. Test scores 82, 91, 76, and 88 at grade boundaries.
2. Test empty and single-element collections.
3. Trace each loop iteration and its index.
4. Normalize the canonical text sample and search it case-insensitively.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

- ☐ I can explain chained conditionals and grade classification in Swift terminology.
- ☐ I can trace the example and predict its behavior.
- ☐ I can adapt the example to the shared canonical data.
- ☐ I can identify and correct a Swift-specific error or edge case.
- ☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 3.3: Arrays and indexing

Reading References

- The Swift Programming Language
- Swift Standard Library

- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain arrays and indexing using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

Swift uses `Bool` conditions, `if`/`else if`/`else`, `switch`, zero-based arrays, and `for-in` loops. Array subscripting requires a valid index. Strings are collections of extended grapheme clusters; use String APIs rather than assuming integer offsets are byte offsets.

Lesson focus: Arrays and indexing. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```
let scores = [82, 91, 76, 88]
print("first=\(scores[0]), last=\(scores[scores.count - 1])")
print("count=\(scores.count), contains 91=\(scores.contains(91))")
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- Swift Boolean conditions must be `Bool`; integers are not truthy or falsy.
- Array indices are zero-based and valid indices stop before `count`.
- Handle optional lookups and user input instead of force-unwrapping.

Activities

1. Test scores 82, 91, 76, and 88 at grade boundaries.
2. Test empty and single-element collections.
3. Trace each loop iteration and its index.
4. Normalize the canonical text sample and search it case-insensitively.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

- ☐ I can explain arrays and indexing in Swift terminology.
- ☐ I can trace the example and predict its behavior.
- ☐ I can adapt the example to the shared canonical data.
- ☐ I can identify and correct a Swift-specific error or edge case.
- ☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 3.4: For loops and iteration

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain for loops and iteration using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

Swift uses `Bool` conditions, `if`/`else if`/`else`, `switch`, zero-based arrays, and `for-in` loops. Array subscripting requires a valid index. Strings are collections of extended grapheme clusters; use String APIs rather than assuming integer offsets are byte offsets.

Lesson focus: For loops and iteration. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```swift
let scores = [82, 91, 76, 88]
for (index, score) in scores.enumerated() {
    print("Score \(index + 1): \(score)")
}
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- Swift Boolean conditions must be `Bool`; integers are not truthy or falsy.

- Array indices are zero-based and valid indices stop before `count`.
- Handle optional lookups and user input instead of force-unwrapping.

Activities

1. Test scores 82, 91, 76, and 88 at grade boundaries.
2. Test empty and single-element collections.
3. Trace each loop iteration and its index.
4. Normalize the canonical text sample and search it case-insensitively.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

☐ I can explain for loops and iteration in Swift terminology.
☐ I can trace the example and predict its behavior.
☐ I can adapt the example to the shared canonical data.
☐ I can identify and correct a Swift-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 3.5: Strings and common operations

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain strings and common operations using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

Swift uses `Bool` conditions, `if`/`else if`/`else`, `switch`, zero-based arrays, and `for-in` loops. Array subscripting requires a valid index. Strings are collections of extended grapheme clusters; use String APIs rather than assuming integer offsets are byte offsets.

Lesson focus: Strings and common operations. The example uses Swift-native semantics and a canonical course context rather than mechanically translating

Python.

Worked Example

```
import Foundation
let rawText = "  Avery studies Swift Programming.  "
let cleaned = rawText.trimmingCharacters(in: .whitespacesAndNewlines)
print(cleaned.lowercased())
print("contains Swift: \(cleaned.contains("Swift")); characters: \(cleaned.count)")
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- Swift Boolean conditions must be `Bool`; integers are not truthy or falsy.
- Array indices are zero-based and valid indices stop before `count`.
- Handle optional lookups and user input instead of force-unwrapping.

Activities

1. Test scores 82, 91, 76, and 88 at grade boundaries.
2. Test empty and single-element collections.
3. Trace each loop iteration and its index.
4. Normalize the canonical text sample and search it case-insensitively.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

- ☐ I can explain strings and common operations in Swift terminology.
- ☐ I can trace the example and predict its behavior.
- ☐ I can adapt the example to the shared canonical data.
- ☐ I can identify and correct a Swift-specific error or edge case.
- ☐ I can run and inspect the program using keyboard-accessible tools.

15.6 Unit 4: Data Structures

Overview

Swift's Array, Dictionary, and Set are value types. Tuples group fixed related values; dictionaries map unique keys to values and return optionals on lookup. Collection methods such as `map`, `filter`, `sorted`, and `reduce` express transformations.

Lesson 4.1: Array mutation and collection methods

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain array mutation and collection methods using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

Swift's Array, Dictionary, and Set are value types. Tuples group fixed related values; dictionaries map unique keys to values and return optionals on lookup. Collection methods such as `map`, `filter`, `sorted`, and `reduce` express transformations.

Lesson focus: Array mutation and collection methods. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```swift
var scores = [82, 91, 76, 88]
scores.append(95)
scores.insert(85, at: 1)
scores.remove(at: 0)
print(scores)
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- Arrays and dictionaries are value types with copy-on-write behavior.
- Dictionary iteration order is not a stable presentation order; sort keys for deterministic output.
- A dictionary subscript returns an optional because the key may be absent.

Activities

1. Add, update, and remove canonical inventory entries.
2. Sort inventory keys and explain why output order matters.
3. Represent a coordinate and learner record with tuples.
4. Test missing dictionary keys and empty collections.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

☐ I can explain array mutation and collection methods in Swift terminology.
☐ I can trace the example and predict its behavior.
☐ I can adapt the example to the shared canonical data.
☐ I can identify and correct a Swift-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 4.2: Sorting and organizing data

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain sorting and organizing data using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

Swift's Array, Dictionary, and Set are value types. Tuples group fixed related values; dictionaries map unique keys to values and return optionals on lookup.

Collection methods such as `map`, `filter`, `sorted`, and `reduce` express transformations.

Lesson focus: Sorting and organizing data. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```swift
let inventory = ["Notebook": 3, "Pencil": 10, "Folder": 4]
for item in inventory.keys.sorted() {
    print("\(item): \(inventory[item]!)")
}
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- Arrays and dictionaries are value types with copy-on-write behavior.
- Dictionary iteration order is not a stable presentation order; sort keys for deterministic output.
- A dictionary subscript returns an optional because the key may be absent.

Activities

1. Add, update, and remove canonical inventory entries.
2. Sort inventory keys and explain why output order matters.
3. Represent a coordinate and learner record with tuples.
4. Test missing dictionary keys and empty collections.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

- ☐ I can explain sorting and organizing data in Swift terminology.
- ☐ I can trace the example and predict its behavior.
- ☐ I can adapt the example to the shared canonical data.
- ☐ I can identify and correct a Swift-specific error or edge case.
- ☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 4.3: Tuples and immutable values

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain tuples and immutable values using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

Swift's Array, Dictionary, and Set are value types. Tuples group fixed related values; dictionaries map unique keys to values and return optionals on lookup. Collection methods such as `map`, `filter`, `sorted`, and `reduce` express transformations.

Lesson focus: Tuples and immutable values. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```swift
let coordinate = (x: 3.0, y: 4.0)
let studentRecord = ("Avery", 16, "Computer Science")
print("point=(\(coordinate.x), \(coordinate.y))")
print("student=\(studentRecord.0), age=\(studentRecord.1)")
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- Arrays and dictionaries are value types with copy-on-write behavior.
- Dictionary iteration order is not a stable presentation order; sort keys for deterministic output.
- A dictionary subscript returns an optional because the key may be absent.

Activities

1. Add, update, and remove canonical inventory entries.
2. Sort inventory keys and explain why output order matters.

3. Represent a coordinate and learner record with tuples.
4. Test missing dictionary keys and empty collections.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

☐ I can explain tuples and immutable values in Swift terminology.
☐ I can trace the example and predict its behavior.
☐ I can adapt the example to the shared canonical data.
☐ I can identify and correct a Swift-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 4.4: Dictionaries and key-value data

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain dictionaries and key-value data using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

Swift's Array, Dictionary, and Set are value types. Tuples group fixed related values; dictionaries map unique keys to values and return optionals on lookup. Collection methods such as `map`, `filter`, `sorted`, and `reduce` express transformations.

Lesson focus: Dictionaries and key-value data. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```swift
var inventory: [String: Int] = ["Notebook": 3, "Pencil": 10]
inventory["Folder"] = 4
inventory["Pencil"] = nil
```

```
print("notebooks=\(inventory["Notebook"] ?? 0)")
print("keys=\(inventory.keys.sorted())")
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- Arrays and dictionaries are value types with copy-on-write behavior.
- Dictionary iteration order is not a stable presentation order; sort keys for deterministic output.
- A dictionary subscript returns an optional because the key may be absent.

Activities

1. Add, update, and remove canonical inventory entries.
2. Sort inventory keys and explain why output order matters.
3. Represent a coordinate and learner record with tuples.
4. Test missing dictionary keys and empty collections.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

☐ I can explain dictionaries and key-value data in Swift terminology.
☐ I can trace the example and predict its behavior.
☐ I can adapt the example to the shared canonical data.
☐ I can identify and correct a Swift-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 4.5: Collection traversal

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain collection traversal using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

Swift's Array, Dictionary, and Set are value types. Tuples group fixed related values; dictionaries map unique keys to values and return optionals on lookup. Collection methods such as `map`, `filter`, `sorted`, and `reduce` express transformations.

Lesson focus: Collection traversal. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```swift
let scores = [82, 91, 76, 88]
let passing = scores.filter { $0 >= 80 }
let doubled = scores.map { $0 * 2 }
let total = scores.reduce(0, +)
print("passing=\(passing), doubled=\(doubled), total=\(total)")
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- Arrays and dictionaries are value types with copy-on-write behavior.
- Dictionary iteration order is not a stable presentation order; sort keys for deterministic output.
- A dictionary subscript returns an optional because the key may be absent.

Activities

1. Add, update, and remove canonical inventory entries.
2. Sort inventory keys and explain why output order matters.
3. Represent a coordinate and learner record with tuples.
4. Test missing dictionary keys and empty collections.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and

read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

- ☐ I can explain collection traversal in Swift terminology.
- ☐ I can trace the example and predict its behavior.
- ☐ I can adapt the example to the shared canonical data.
- ☐ I can identify and correct a Swift-specific error or edge case.
- ☐ I can run and inspect the program using keyboard-accessible tools.

15.7 Unit 5: Input and Iteration

Overview

Console input arrives as an optional string. Convert and validate it before use. `while`, `break`, and `continue` control repetition; a loop must make progress or reach an exit condition.

Lesson 5.1: Input and validation

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain input and validation using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

Console input arrives as an optional string. Convert and validate it before use. `while`, `break`, and `continue` control repetition; a loop must make progress or reach an exit condition.

Lesson focus: Input and validation. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```
print("Enter a score from 0 to 100:")
let input = readLine() ?? ""
if let score = Int(input), (0...100).contains(score) {
    print("Accepted score: \(score)")
```

```
} else {
    print("Invalid score")
}
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- A `while` loop must change state or terminate explicitly.
- Use `break` to exit the intended loop and `continue` to skip only the current iteration.
- Validate text before converting it to a numeric value.

Activities

1. Test valid, invalid, and blank user input.
2. Trace a sentinel loop with both quit and non-quit commands.
3. Find the first matching score and stop immediately.
4. Prove a loop terminates for empty and no-match inputs.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

☐ I can explain input and validation in Swift terminology.
☐ I can trace the example and predict its behavior.
☐ I can adapt the example to the shared canonical data.
☐ I can identify and correct a Swift-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 5.2: While loops and sentinel repetition

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain while loops and sentinel repetition using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

Console input arrives as an optional string. Convert and validate it before use. `while`, `break`, and `continue` control repetition; a loop must make progress or reach an exit condition.

Lesson focus: While loops and sentinel repetition. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```swift
var commands = ["list", "help", "quit"]
var position = 0
while position < commands.count {
    let command = commands[position]
    print("Command: \(command)")
    position += 1
    if command == "quit" { break }
}
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- A `while` loop must change state or terminate explicitly.
- Use `break` to exit the intended loop and `continue` to skip only the current iteration.
- Validate text before converting it to a numeric value.

Activities

1. Test valid, invalid, and blank user input.
2. Trace a sentinel loop with both quit and non-quit commands.
3. Find the first matching score and stop immediately.
4. Prove a loop terminates for empty and no-match inputs.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

- ☐ I can explain while loops and sentinel repetition in Swift terminology.
- ☐ I can trace the example and predict its behavior.
- ☐ I can adapt the example to the shared canonical data.
- ☐ I can identify and correct a Swift-specific error or edge case.
- ☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 5.3: Flags and early exits

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain flags and early exits using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

Console input arrives as an optional string. Convert and validate it before use. `while`, `break`, and `continue` control repetition; a loop must make progress or reach an exit condition.

Lesson focus: Flags and early exits. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```swift
let scores = [82, 91, 76, 88]
var foundIndex: Int? = nil
for (index, score) in scores.enumerated() {
    if score == 91 {
        foundIndex = index
        break
    }
}
print("91 found at index: \(foundIndex.map(String.init) ?? "not found")")
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- A `while` loop must change state or terminate explicitly.
- Use `break` to exit the intended loop and `continue` to skip only the current iteration.
- Validate text before converting it to a numeric value.

Activities

1. Test valid, invalid, and blank user input.
2. Trace a sentinel loop with both quit and non-quit commands.
3. Find the first matching score and stop immediately.
4. Prove a loop terminates for empty and no-match inputs.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

☐ I can explain flags and early exits in Swift terminology.
☐ I can trace the example and predict its behavior.
☐ I can adapt the example to the shared canonical data.
☐ I can identify and correct a Swift-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 5.4: Continue and filtering

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain continue and filtering using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

Console input arrives as an optional string. Convert and validate it before use. `while`, `break`, and `continue` control repetition; a loop must make progress or reach an exit condition.

Lesson focus: Continue and filtering. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```swift
let values = [82, -1, 91, 0, 76]
var positiveValues: [Int] = []
for value in values {
    if value <= 0 { continue }
    positiveValues.append(value)
}
print(positiveValues)
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- A `while` loop must change state or terminate explicitly.
- Use `break` to exit the intended loop and `continue` to skip only the current iteration.
- Validate text before converting it to a numeric value.

Activities

1. Test valid, invalid, and blank user input.
2. Trace a sentinel loop with both quit and non-quit commands.
3. Find the first matching score and stop immediately.
4. Prove a loop terminates for empty and no-match inputs.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

☐ I can explain continue and filtering in Swift terminology.

☐ I can trace the example and predict its behavior.
☐ I can adapt the example to the shared canonical data.
☐ I can identify and correct a Swift-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 5.5: Preventing infinite loops

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain preventing infinite loops using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

Console input arrives as an optional string. Convert and validate it before use. `while`, `break`, and `continue` control repetition; a loop must make progress or reach an exit condition.

Lesson focus: Preventing infinite loops. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```swift
func countDown(from start: Int) {
    var current = start
    while current > 0 {
        print(current)
        current -= 1
    }
    print("Done")
}
countDown(from: 3)
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- A `while` loop must change state or terminate explicitly.
- Use `break` to exit the intended loop and `continue` to skip only the current iteration.
- Validate text before converting it to a numeric value.

Activities

1. Test valid, invalid, and blank user input.
2. Trace a sentinel loop with both quit and non-quit commands.
3. Find the first matching score and stop immediately.
4. Prove a loop terminates for empty and no-match inputs.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

☐ I can explain preventing infinite loops in Swift terminology.
☐ I can trace the example and predict its behavior.
☐ I can adapt the example to the shared canonical data.
☐ I can identify and correct a Swift-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

15.8 Unit 6: Functions

Overview

Functions declare parameter types, return types, and external argument labels. Swift supports default arguments, closures, and variadic parameters. Return optional values when a result may be absent; use `guard` for early validation exits.

Lesson 6.1: Defining and calling functions

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain defining and calling functions using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

Functions declare parameter types, return types, and external argument labels. Swift supports default arguments, closures, and variadic parameters. Return optional values when a result may be absent; use `guard` for early validation exits.

Lesson focus: Defining and calling functions. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```swift
func greet(_ name: String) {
    print("Hello, \(name)!")
}
greet("Avery")
greet("Jordan")
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- Swift argument labels are part of a function's call-site API.
- A function returning an optional must handle the empty case explicitly.
- Closures capture values; understand capture semantics and avoid unnecessary shared mutable state.

Activities

1. Write a function with a clear input and output contract.
2. Add argument labels and a default parameter value.
3. Test return values for normal, boundary, and invalid cases.
4. Compose a closure with a variadic reporting function.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and

read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

- ☐ I can explain defining and calling functions in Swift terminology.
- ☐ I can trace the example and predict its behavior.
- ☐ I can adapt the example to the shared canonical data.
- ☐ I can identify and correct a Swift-specific error or edge case.
- ☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 6.2: Parameters and argument labels

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain parameters and argument labels using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

Functions declare parameter types, return types, and external argument labels. Swift supports default arguments, closures, and variadic parameters. Return optional values when a result may be absent; use `guard` for early validation exits.

Lesson focus: Parameters and argument labels. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```swift
import Foundation
func invoiceTotal(subtotal: Double, tipRate: Double = 0.15) -> Double {
    return subtotal + subtotal * tipRate
}
print(String(format: "%.2f", invoiceTotal(subtotal: 24.50)))
print(String(format: "%.2f", invoiceTotal(subtotal: 24.50, tipRate: 0.20)))
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- Swift argument labels are part of a function's call-site API.
- A function returning an optional must handle the empty case explicitly.
- Closures capture values; understand capture semantics and avoid unnecessary shared mutable state.

Activities

1. Write a function with a clear input and output contract.
2. Add argument labels and a default parameter value.
3. Test return values for normal, boundary, and invalid cases.
4. Compose a closure with a variadic reporting function.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

☐ I can explain parameters and argument labels in Swift terminology.
☐ I can trace the example and predict its behavior.
☐ I can adapt the example to the shared canonical data.
☐ I can identify and correct a Swift-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 6.3: Return values

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain return values using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

Functions declare parameter types, return types, and external argument labels. Swift supports default arguments, closures, and variadic parameters. Return optional values when a result may be absent; use `guard` for early validation exits.

Lesson focus: Return values. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```swift
func letterGrade(for score: Int) -> String {
    guard (0...100).contains(score) else { return "Invalid" }
    switch score {
    case 90...100: return "A"
    case 80..<90: return "B"
    case 70..<80: return "C"
    case 60..<70: return "D"
    default: return "F"
    }
}
print(letterGrade(for: 82))
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- Swift argument labels are part of a function's call-site API.
- A function returning an optional must handle the empty case explicitly.
- Closures capture values; understand capture semantics and avoid unnecessary shared mutable state.

Activities

1. Write a function with a clear input and output contract.
2. Add argument labels and a default parameter value.
3. Test return values for normal, boundary, and invalid cases.
4. Compose a closure with a variadic reporting function.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and

read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

- ☐ I can explain return values in Swift terminology.
- ☐ I can trace the example and predict its behavior.
- ☐ I can adapt the example to the shared canonical data.
- ☐ I can identify and correct a Swift-specific error or edge case.
- ☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 6.4: Functions over collections

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain functions over collections using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

Functions declare parameter types, return types, and external argument labels. Swift supports default arguments, closures, and variadic parameters. Return optional values when a result may be absent; use `guard` for early validation exits.

Lesson focus: Functions over collections. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```swift
func mean(of values: [Double]) -> Double? {
    guard !values.isEmpty else { return nil }
    return values.reduce(0, +) / Double(values.count)
}
let scores = [82.0, 91.0, 76.0, 88.0]
print(mean(of: scores) ?? 0)
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- Swift argument labels are part of a function's call-site API.
- A function returning an optional must handle the empty case explicitly.
- Closures capture values; understand capture semantics and avoid unnecessary shared mutable state.

Activities

1. Write a function with a clear input and output contract.
2. Add argument labels and a default parameter value.
3. Test return values for normal, boundary, and invalid cases.
4. Compose a closure with a variadic reporting function.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

☐ I can explain functions over collections in Swift terminology.
☐ I can trace the example and predict its behavior.
☐ I can adapt the example to the shared canonical data.
☐ I can identify and correct a Swift-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 6.5: Scope, closures, and variadic parameters

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain scope, closures, and variadic parameters using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

Functions declare parameter types, return types, and external argument labels. Swift supports default arguments, closures, and variadic parameters. Return optional values when a result may be absent; use `guard` for early validation exits.

Lesson focus: Scope, closures, and variadic parameters. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```swift
func makeMultiplier(by factor: Int) -> (Int) -> Int {
    return { value in value * factor }
}
let triple = makeMultiplier(by: 3)
func report(_ title: String, scores: Int...) {
    print("\(title): \(scores.map(String.init).joined(separator: ", "))")
}
report("Tripled score", scores: triple(82), triple(91))
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- Swift argument labels are part of a function's call-site API.
- A function returning an optional must handle the empty case explicitly.
- Closures capture values; understand capture semantics and avoid unnecessary shared mutable state.

Activities

1. Write a function with a clear input and output contract.
2. Add argument labels and a default parameter value.
3. Test return values for normal, boundary, and invalid cases.
4. Compose a closure with a variadic reporting function.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

- ☐ I can explain scope, closures, and variadic parameters in Swift terminology.
- ☐ I can trace the example and predict its behavior.
- ☐ I can adapt the example to the shared canonical data.
- ☐ I can identify and correct a Swift-specific error or edge case.
- ☐ I can run and inspect the program using keyboard-accessible tools.

15.9 Unit 7: Object-Oriented Programming

Overview

Swift models data with structs, classes, enums, and protocols. Structs and enums have value semantics; classes have reference semantics. Protocols specify behavior, and composition models relationships without relying on inheritance.

Lesson 7.1: Types and object models

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain types and object models using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

Swift models data with structs, classes, enums, and protocols. Structs and enums have value semantics; classes have reference semantics. Protocols specify behavior, and composition models relationships without relying on inheritance.

Lesson focus: Types and object models. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```swift
struct Student {
    let name: String
    var age: Int
}
let learner = Student(name: "Avery", age: 16)
print("\(learner.name) is \(learner.age)")
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- A `struct` instance is a value; mutating methods require a `var` receiver.
- Use `class` only when reference identity or shared mutation is required.
- Protocols define requirements; they are not concrete class inheritance.

Activities

1. Model Avery or an inventory item with a Swift struct.
2. Create multiple instances and inspect stored and computed properties.
3. Define a protocol and conform a custom type to it.
4. Compare value semantics and reference semantics with a copied value.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

☐ I can explain types and object models in Swift terminology.
☐ I can trace the example and predict its behavior.
☐ I can adapt the example to the shared canonical data.
☐ I can identify and correct a Swift-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 7.2: Creating instances

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain creating instances using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

Swift models data with structs, classes, enums, and protocols. Structs and enums have value semantics; classes have reference semantics. Protocols specify behavior, and composition models relationships without relying on inheritance.

Lesson focus: Creating instances. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```swift
struct InventoryItem {
    let name: String
    var quantity: Int
}
let notebook = InventoryItem(name: "Notebook", quantity: 3)
let pencil = InventoryItem(name: "Pencil", quantity: 10)
print("\(notebook.name): \(notebook.quantity); \(pencil.name): \(pencil.quantity)")
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- A `struct` instance is a value; mutating methods require a `var` receiver.
- Use `class` only when reference identity or shared mutation is required.
- Protocols define requirements; they are not concrete class inheritance.

Activities

1. Model Avery or an inventory item with a Swift struct.
2. Create multiple instances and inspect stored and computed properties.
3. Define a protocol and conform a custom type to it.
4. Compare value semantics and reference semantics with a copied value.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

☐ I can explain creating instances in Swift terminology.

☐ I can trace the example and predict its behavior.
☐ I can adapt the example to the shared canonical data.
☐ I can identify and correct a Swift-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 7.3: Properties and methods

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain properties and methods using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

Swift models data with structs, classes, enums, and protocols. Structs and enums have value semantics; classes have reference semantics. Protocols specify behavior, and composition models relationships without relying on inheritance.

Lesson focus: Properties and methods. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```swift
struct Circle {
    var radius: Double
    var area: Double { Double.pi * radius * radius }
    mutating func scale(by factor: Double) { radius *= factor }
}
var circle = Circle(radius: 2)
print(circle.area)
circle.scale(by: 2)
print(circle.area)
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- A `struct` instance is a value; mutating methods require a `var` receiver.
- Use `class` only when reference identity or shared mutation is required.
- Protocols define requirements; they are not concrete class inheritance.

Activities

1. Model Avery or an inventory item with a Swift struct.
2. Create multiple instances and inspect stored and computed properties.
3. Define a protocol and conform a custom type to it.
4. Compare value semantics and reference semantics with a copied value.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

☐ I can explain properties and methods in Swift terminology.
☐ I can trace the example and predict its behavior.
☐ I can adapt the example to the shared canonical data.
☐ I can identify and correct a Swift-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 7.4: Protocols and inheritance

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain protocols and inheritance using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

Swift models data with structs, classes, enums, and protocols. Structs and enums have value semantics; classes have reference semantics. Protocols specify behavior, and composition models relationships without relying on inheritance.

Lesson focus: Protocols and inheritance. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```swift
protocol Describable {
    var summary: String { get }
}
struct Task: Describable {
    let title: String
    var isComplete: Bool
    var summary: String { "\(title): \(isComplete ? "complete" : "open")" }
}
let task = Task(title: "Read Swift lesson", isComplete: false)
print(task.summary)
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- A `struct` instance is a value; mutating methods require a `var` receiver.
- Use `class` only when reference identity or shared mutation is required.
- Protocols define requirements; they are not concrete class inheritance.

Activities

1. Model Avery or an inventory item with a Swift struct.
2. Create multiple instances and inspect stored and computed properties.
3. Define a protocol and conform a custom type to it.
4. Compare value semantics and reference semantics with a copied value.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

☐ I can explain protocols and inheritance in Swift terminology.
☐ I can trace the example and predict its behavior.

☐ I can adapt the example to the shared canonical data.
☐ I can identify and correct a Swift-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 7.5: Composition and value semantics

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain composition and value semantics using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

Swift models data with structs, classes, enums, and protocols. Structs and enums have value semantics; classes have reference semantics. Protocols specify behavior, and composition models relationships without relying on inheritance.

Lesson focus: Composition and value semantics. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```swift
struct Address {
    var city: String
}
struct Person {
    let name: String
    var address: Address
}
var person = Person(name: "Avery", address: Address(city: "Farmington"))
var updatedPerson = person
updatedPerson.address.city = "Ogden"
print(person.address.city)
print(updatedPerson.address.city)
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- A `struct` instance is a value; mutating methods require a `var` receiver.
- Use `class` only when reference identity or shared mutation is required.
- Protocols define requirements; they are not concrete class inheritance.

Activities

1. Model Avery or an inventory item with a Swift struct.
2. Create multiple instances and inspect stored and computed properties.
3. Define a protocol and conform a custom type to it.
4. Compare value semantics and reference semantics with a copied value.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

☐ I can explain composition and value semantics in Swift terminology.
☐ I can trace the example and predict its behavior.
☐ I can adapt the example to the shared canonical data.
☐ I can identify and correct a Swift-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

15.10 Unit 8: Files and Exceptions

Overview

Foundation provides file URLs, FileManager, String file I/O, FileHandle, and Codable. File APIs can throw; handle errors with `do`/`catch`, use `defer` for scope cleanup, and encode structured records with JSONEncoder and JSONDecoder.

Lesson 8.1: Reading text files

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain reading text files using Swift terminology.
- Apply the relevant language and standard-library features correctly.

- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

Foundation provides file URLs, FileManager, String file I/O, FileHandle, and Codable. File APIs can throw; handle errors with `do`/`catch`, use `defer` for scope cleanup, and encode structured records with JSONEncoder and JSONDecoder.

Lesson focus: Reading text files. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```
import Foundation
let folder = FileManager.default.temporaryDirectory
let file = folder.appendingPathComponent(UUID().uuidString + ".txt")
try! "Avery studies Swift.\nScores are stored safely.\n".write(to: file, atomically: true, e
let contents = try! String(contentsOf: file, encoding: .utf8)
print(contents.trimmingCharacters(in: .whitespacesAndNewlines))
try? FileManager.default.removeItem(at: file)
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- File operations can throw; handle errors instead of assuming success.
- Use explicit encodings and atomic writes for text-file safety.
- A `defer` block runs when the current scope exits, not at an arbitrary later time.

Activities

1. Create a temporary file fixture and read its contents.
2. Test missing, empty, and malformed files.
3. Append to a file and verify cleanup after the operation.
4. Encode and decode a Codable record; inspect the JSON fields.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

- ☐ I can explain reading text files in Swift terminology.
- ☐ I can trace the example and predict its behavior.
- ☐ I can adapt the example to the shared canonical data.
- ☐ I can identify and correct a Swift-specific error or edge case.
- ☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 8.2: Writing and appending files

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain writing and appending files using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

Foundation provides file URLs, FileManager, String file I/O, FileHandle, and Codable. File APIs can throw; handle errors with `do`/`catch`, use `defer` for scope cleanup, and encode structured records with JSONEncoder and JSONDecoder.

Lesson focus: Writing and appending files. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```
import Foundation
let file = FileManager.default.temporaryDirectory.appendingPathComponent(UUID().uuidString +
try! "First line\n".write(to: file, atomically: true, encoding: .utf8)
let handle = try! FileHandle(forWritingTo: file)
try! handle.seekToEnd()
try! handle.write(contentsOf: Data("Appended line\n".utf8))
try! handle.close()
print(try! String(contentsOf: file, encoding: .utf8))
try? FileManager.default.removeItem(at: file)
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- File operations can throw; handle errors instead of assuming success.
- Use explicit encodings and atomic writes for text-file safety.
- A `defer` block runs when the current scope exits, not at an arbitrary later time.

Activities

1. Create a temporary file fixture and read its contents.
2. Test missing, empty, and malformed files.
3. Append to a file and verify cleanup after the operation.
4. Encode and decode a Codable record; inspect the JSON fields.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

☐ I can explain writing and appending files in Swift terminology.
☐ I can trace the example and predict its behavior.
☐ I can adapt the example to the shared canonical data.
☐ I can identify and correct a Swift-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 8.3: Errors and throwing functions

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain errors and throwing functions using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

Foundation provides file URLs, FileManager, String file I/O, FileHandle, and Codable. File APIs can throw; handle errors with `do`/`catch`, use `defer` for scope cleanup, and encode structured records with JSONEncoder and JSONDecoder.

Lesson focus: Errors and throwing functions. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```swift
enum ScoreError: Error {
    case outOfRange
}
func validate(score: Int) throws -> Int {
    guard (0...100).contains(score) else { throw ScoreError.outOfRange }
    return score
}
do {
    print(try validate(score: 82))
    print(try validate(score: 120))
} catch {
    print("Rejected score: \(error)")
}
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- File operations can throw; handle errors instead of assuming success.
- Use explicit encodings and atomic writes for text-file safety.
- A `defer` block runs when the current scope exits, not at an arbitrary later time.

Activities

1. Create a temporary file fixture and read its contents.
2. Test missing, empty, and malformed files.
3. Append to a file and verify cleanup after the operation.
4. Encode and decode a Codable record; inspect the JSON fields.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

- ☐ I can explain errors and throwing functions in Swift terminology.
- ☐ I can trace the example and predict its behavior.
- ☐ I can adapt the example to the shared canonical data.
- ☐ I can identify and correct a Swift-specific error or edge case.
- ☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 8.4: Resource lifetime and cleanup

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain resource lifetime and cleanup using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

Foundation provides file URLs, FileManager, String file I/O, FileHandle, and Codable. File APIs can throw; handle errors with `do`/`catch`, use `defer` for scope cleanup, and encode structured records with JSONEncoder and JSONDecoder.

Lesson focus: Resource lifetime and cleanup. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```
import Foundation
func writeTemporaryReport() throws -> URL {
    let url = FileManager.default.temporaryDirectory.appendingPathComponent(UUID().uuidStrin
    defer { try? FileManager.default.removeItem(at: url) }
    try "Report complete".write(to: url, atomically: true, encoding: .utf8)
    print(try String(contentsOf: url, encoding: .utf8))
    return url
}
_ = try writeTemporaryReport()
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- File operations can throw; handle errors instead of assuming success.
- Use explicit encodings and atomic writes for text-file safety.
- A `defer` block runs when the current scope exits, not at an arbitrary later time.

Activities

1. Create a temporary file fixture and read its contents.
2. Test missing, empty, and malformed files.
3. Append to a file and verify cleanup after the operation.
4. Encode and decode a Codable record; inspect the JSON fields.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

☐ I can explain resource lifetime and cleanup in Swift terminology.
☐ I can trace the example and predict its behavior.
☐ I can adapt the example to the shared canonical data.
☐ I can identify and correct a Swift-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 8.5: JSON with Codable

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain json with codable using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

Foundation provides file URLs, FileManager, String file I/O, FileHandle, and Codable. File APIs can throw; handle errors with `do`/`catch`, use `defer` for scope cleanup, and encode structured records with JSONEncoder and JSONDecoder.

Lesson focus: JSON with Codable. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```swift
import Foundation
struct LearnerRecord: Codable {
    let id: Int
    let name: String
    let scores: [Int]
}
let learner = LearnerRecord(id: 1, name: "Avery", scores: [82, 91, 76])
let data = try! JSONEncoder().encode(learner)
let decoded = try! JSONDecoder().decode(LearnerRecord.self, from: data)
print(String(data: data, encoding: .utf8)!)
print(decoded.name, decoded.scores)
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- File operations can throw; handle errors instead of assuming success.
- Use explicit encodings and atomic writes for text-file safety.
- A `defer` block runs when the current scope exits, not at an arbitrary later time.

Activities

1. Create a temporary file fixture and read its contents.
2. Test missing, empty, and malformed files.
3. Append to a file and verify cleanup after the operation.
4. Encode and decode a Codable record; inspect the JSON fields.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

- ☐ I can explain json with codable in Swift terminology.

☐ I can trace the example and predict its behavior.
☐ I can adapt the example to the shared canonical data.
☐ I can identify and correct a Swift-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

15.11 Unit 9: Testing and Data Analysis

Overview

Test pure functions and stateful types with assertions or XCTest in a package. Floating-point comparisons often need tolerances. Foundation and Swift standard-library collections support text and data processing; robust CSV requires a parser that understands quoting and escaping.

Lesson 9.1: Testing pure functions

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain testing pure functions using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

Test pure functions and stateful types with assertions or XCTest in a package. Floating-point comparisons often need tolerances. Foundation and Swift standard-library collections support text and data processing; robust CSV requires a parser that understands quoting and escaping.

Lesson focus: Testing pure functions. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```swift
func tip(for bill: Double, rate: Double) -> Double {
    bill * rate
}
precondition(tip(for: 20, rate: 0.2) == 4)
precondition(abs(tip(for: 24.50, rate: 0.15) - 3.675) < 0.000001)
print("Tip calculation tests passed")
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- Use tolerance-based assertions for floating-point calculations.
- Random results are not reproducible unless the generator and seed are controlled.
- A simple comma split is not a complete CSV parser for quoted fields.

Activities

1. Test pure functions with ordinary and boundary inputs.
2. Test a stateful type before and after mutation.
3. Generate random values and discuss repeatability.
4. Analyze the canonical text sample and process simple CSV rows.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

☐ I can explain testing pure functions in Swift terminology.
☐ I can trace the example and predict its behavior.
☐ I can adapt the example to the shared canonical data.
☐ I can identify and correct a Swift-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 9.2: Testing stateful types

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain testing stateful types using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

Test pure functions and stateful types with assertions or XCTest in a package. Floating-point comparisons often need tolerances. Foundation and Swift standard-library collections support text and data processing; robust CSV requires a parser that understands quoting and escaping.

Lesson focus: Testing stateful types. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```swift
struct Counter {
    private(set) var value = 0
    mutating func increment() { value += 1 }
}
var counter = Counter()
counter.increment()
counter.increment()
precondition(counter.value == 2)
print("Counter state test passed: \(counter.value)")
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- Use tolerance-based assertions for floating-point calculations.
- Random results are not reproducible unless the generator and seed are controlled.
- A simple comma split is not a complete CSV parser for quoted fields.

Activities

1. Test pure functions with ordinary and boundary inputs.
2. Test a stateful type before and after mutation.
3. Generate random values and discuss repeatability.
4. Analyze the canonical text sample and process simple CSV rows.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

- ☐ I can explain testing stateful types in Swift terminology.
- ☐ I can trace the example and predict its behavior.
- ☐ I can adapt the example to the shared canonical data.
- ☐ I can identify and correct a Swift-specific error or edge case.
- ☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 9.3: Random values

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain random values using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

Test pure functions and stateful types with assertions or XCTest in a package. Floating-point comparisons often need tolerances. Foundation and Swift standard-library collections support text and data processing; robust CSV requires a parser that understands quoting and escaping.

Lesson focus: Random values. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```
import Foundation
var generator = SystemRandomNumberGenerator()
let diceRolls = (1...5).map { _ in Int.random(in: 1...6, using: &generator) }
print("Dice rolls: \(diceRolls)")
let seededLikeValues = (1...5).map { _ in Int.random(in: 1...10) }
print("Additional sample: \(seededLikeValues)")
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- Use tolerance-based assertions for floating-point calculations.
- Random results are not reproducible unless the generator and seed are controlled.
- A simple comma split is not a complete CSV parser for quoted fields.

Activities

1. Test pure functions with ordinary and boundary inputs.
2. Test a stateful type before and after mutation.
3. Generate random values and discuss repeatability.
4. Analyze the canonical text sample and process simple CSV rows.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

☐ I can explain random values in Swift terminology.
☐ I can trace the example and predict its behavior.
☐ I can adapt the example to the shared canonical data.
☐ I can identify and correct a Swift-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 9.4: Text analysis

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain text analysis using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

Test pure functions and stateful types with assertions or XCTest in a package. Floating-point comparisons often need tolerances. Foundation and Swift

standard-library collections support text and data processing; robust CSV requires a parser that understands quoting and escaping.

Lesson focus: Text analysis. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```swift
let sample = "A student studies Swift. Swift helps students build useful programs."
let words = sample.lowercased().split { !$0.isLetter }
var counts: [String: Int] = [:]
for word in words { counts[String(word), default: 0] += 1 }
for word in counts.keys.sorted() { print("\(word): \(counts[word]!)") }
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- Use tolerance-based assertions for floating-point calculations.
- Random results are not reproducible unless the generator and seed are controlled.
- A simple comma split is not a complete CSV parser for quoted fields.

Activities

1. Test pure functions with ordinary and boundary inputs.
2. Test a stateful type before and after mutation.
3. Generate random values and discuss repeatability.
4. Analyze the canonical text sample and process simple CSV rows.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

- ☐ I can explain text analysis in Swift terminology.
- ☐ I can trace the example and predict its behavior.
- ☐ I can adapt the example to the shared canonical data.
- ☐ I can identify and correct a Swift-specific error or edge case.
- ☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 9.5: CSV records

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain csv records using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

Test pure functions and stateful types with assertions or XCTest in a package. Floating-point comparisons often need tolerances. Foundation and Swift standard-library collections support text and data processing; robust CSV requires a parser that understands quoting and escaping.

Lesson focus: CSV records. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```swift
let csv = "id,first_name,score\n1,Avery,91\n2,Jordan,84"
let lines = csv.split(separator: "\n")
for line in lines.dropFirst() {
    let fields = line.split(separator: ",")
    if fields.count == 3, let score = Int(fields[2]) {
        print("\(fields[1]): \(score)")
    }
}
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- Use tolerance-based assertions for floating-point calculations.
- Random results are not reproducible unless the generator and seed are controlled.
- A simple comma split is not a complete CSV parser for quoted fields.

Activities

1. Test pure functions with ordinary and boundary inputs.
2. Test a stateful type before and after mutation.
3. Generate random values and discuss repeatability.
4. Analyze the canonical text sample and process simple CSV rows.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

☐ I can explain csv records in Swift terminology.
☐ I can trace the example and predict its behavior.
☐ I can adapt the example to the shared canonical data.
☐ I can identify and correct a Swift-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

15.12 Unit 10: Project-Based Learning

Overview

Projects begin with requirements and acceptance criteria. Implement a text-based adventure and todo manager using explicit state, validated commands, and predictable output. Accessible interaction uses keyboard-operable text prompts and readable error messages.

Lesson 10.1: Requirements and top-down design

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain requirements and top-down design using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

Projects begin with requirements and acceptance criteria. Implement a text-based adventure and todo manager using explicit state, validated commands, and predictable output. Accessible interaction uses keyboard-operable text prompts and readable error messages.

Lesson focus: Requirements and top-down design. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```
let requirements = [
    "List current records",
    "Add a record with a unique identifier",
    "Search by title",
    "Update completion state",
    "Reject blank titles"
]
for (number, requirement) in requirements.enumerated() {
    print("\(number + 1). \(requirement)")
}
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- Write acceptance criteria before implementing project features.
- Reject invalid directions and unknown commands explicitly.
- Do not make interaction depend on color, animation, or mouse-only controls.

Activities

1. Translate project requirements into acceptance tests.
2. Test valid room movement and unavailable exits.
3. Test inventory puzzles with and without required items.
4. Exercise todo list, add, completion, and invalid-title workflows.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and

read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

- ☐ I can explain requirements and top-down design in Swift terminology.
- ☐ I can trace the example and predict its behavior.
- ☐ I can adapt the example to the shared canonical data.
- ☐ I can identify and correct a Swift-specific error or edge case.
- ☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 10.2: Text adventure: five rooms and movement

Reading References

- The Swift Programming Language
- Swift Dictionary
- Canonical shared project specification: Python Unit 10, Lesson 10.3.

Learning Objectives

- Represent all five canonical rooms and exits with Swift dictionaries and structs.
- Validate movement before changing the current room.
- Explain why treasure_room is not reachable until the key puzzle is implemented.

Concept Explanation

The shared graph has five rooms: start_room, tunnel, clearing, locked_door_room, and treasure_room. A Room value stores a description and its exits. The player begins in start_room. Part 1 leaves the north exit from locked_door_room unavailable. Optional dictionary lookup makes a missing direction explicit, so invalid movement cannot change state.

Worked Example

```
import Foundation

struct Room {
    let description: String
    let exits: [String: String]
}

let rooms: [String: Room] = [
    "start_room": Room(description: "A dimly lit cave. A narrow passage leads north.", exits
    "tunnel": Room(description: "A damp tunnel. Passages lead south and east.", exits: ["sou
```

```
    "clearing": Room(description: "A quiet clearing. An antechamber lies to the north.", exi
    "locked_door_room": Room(description: "A chamber with a heavy door to the north.", exits
    "treasure_room": Room(description: "You found the treasure. You win!", exits: [:])
]

var currentRoom = "start_room"
while true {
    let room = rooms[currentRoom]!
    print("---")
    print(room.description)
    print("What do you want to do?")
    let command = readLine()?.trimmingCharacters(in: .whitespacesAndNewlines).lowercased() ?
    if command == "quit" {
        print("Exiting game. Goodbye!")
        break
    }
    if let destination = room.exits[command] {
        currentRoom = destination
    } else {
        print("You cannot go that way. Enter a listed direction or quit.")
    }
}
```

Expected Behavior or Output

Enter north, east, north, north, quit. The player reaches tunnel, clearing, and locked_door_room. The second north is rejected and the current room does not change. The treasure room exists but has no incoming exit in Part 1.

Common Mistakes

- Force-unwrapping a direction lookup that may be nil.
- Updating currentRoom before validating the exit.
- Defining a destination that is not one of the five room identifiers.
- Adding the treasure exit before implementing the key puzzle.

Activities

1. Trace north, east, north, north and record the current room.
2. Try west from start_room and verify that the room does not change.
3. Check every exit destination against the room dictionary.
4. Explain why treasure_room cannot yet be reached.

Accessibility Practice

Announce the room description and prompt in a consistent order. The invalid-movement response explains what happened and how to continue. Run from the

terminal and inspect output sequentially with the learner's access technology.

Skills Checklist

- ☐ I can model all five canonical rooms.
- ☐ I can represent exits with dictionaries.
- ☐ I can reject invalid movement without changing state.
- ☐ I can explain why the treasure room is not yet reachable.

Lesson 10.3: Text adventure: shiny key and locked door

Reading References

- The Swift Programming Language: Collections
- Swift Array
- Canonical shared project specification: Python Unit 10, Lesson 10.3.

Learning Objectives

- Place the shiny key in start_room and transfer it to inventory exactly once.
- Require key use in locked_door_room.
- Unlock the north exit only after the puzzle succeeds.
- End the game when the player reaches treasure_room.

Concept Explanation

Keep the same five-room graph. Each Room reference stores a mutable item array and exit dictionary. Picking up the shiny key removes it from start_room and adds it to inventory. Using the key succeeds only in locked_door_room when inventory contains it. The north exit remains blocked until the key is used.

Worked Example

```
import Foundation

final class Room {
    let description: String
    var exits: [String: String]
    var items: [String]

    init(_ description: String, exits: [String: String], items: [String] = []) {
        self.description = description
        self.exits = exits
        self.items = items
```

```
    }
}

let rooms: [String: Room] = [
    "start_room": Room("A dimly lit cave.", exits: ["north": "tunnel"], items: ["shiny key"]
    "tunnel": Room("A damp tunnel. Passages lead south and east.", exits: ["south": "start_r
    "clearing": Room("A quiet clearing. An antechamber lies to the north.", exits: ["west":
    "locked_door_room": Room("A chamber with a heavy door to the north.", exits: ["south": "
    "treasure_room": Room("You found the treasure. You win!", exits: [:])
]

var currentRoom = "start_room"
var inventory: [String] = []
var doorLocked = true

while true {
    let room = rooms[currentRoom]!
    print()
    print(room.description)
    if currentRoom == "treasure_room" { break }
    if !room.items.isEmpty { print("You see: " + room.items.joined(separator: ", ")) }
    print("Your inventory: " + (inventory.isEmpty ? "empty" : inventory.joined(separator: ",
    print("What do you want to do?")
    let command = readLine()?.trimmingCharacters(in: .whitespacesAndNewlines).lowercased() ?

    if command == "quit" {
        print("You quit the adventure.")
        break
    } else if command == "take shiny key" {
        if let index = room.items.firstIndex(of: "shiny key") {
            room.items.remove(at: index)
            inventory.append("shiny key")
            print("You picked up the shiny key.")
        } else {
            print("There is no shiny key here to take.")
        }
    } else if command == "use shiny key" {
        if currentRoom != "locked_door_room" {
            print("You must be at the locked door to use the key.")
        } else if !inventory.contains("shiny key") {
            print("You do not have the shiny key.")
        } else {
            doorLocked = false
            print("You unlocked the door!")
        }
    } else if currentRoom == "locked_door_room" && command == "north" && doorLocked {
```

```
        print("The door is locked! You need to use the shiny key here.")
    } else if let destination = room.exits[command] {
        currentRoom = destination
    } else {
        print("Invalid command. Enter north, south, east, west, take shiny key, use shiny ke
    }
}
```

Expected Behavior or Output

Enter north, east, north, north, south, west, south, take shiny key, north, east, north, use shiny key, north. The first north from locked_door_room is rejected. Taking the key removes it from the room and adds it to inventory. Using it outside locked_door_room is rejected. Using it there unlocks the door; the final north reaches treasure_room and prints the win description.

Common Mistakes

- Removing the key from the room without adding it to inventory.
- Unlocking the door outside locked_door_room.
- Unlocking the door without checking inventory.
- Allowing north movement before doorLocked becomes false.
- Treating an invalid optional lookup as a destination.

Activities

1. Run the complete puzzle sequence and record room, items, inventory, and door state.
2. Attempt to take the key twice; the second attempt must be rejected.
3. Try to use the key outside locked_door_room.
4. Try north before and after unlocking.
5. Add a help command listing every supported command.

Accessibility Practice

Announce the room, visible items, inventory, and prompt as separate text lines. Error messages explain the failed action. Test the complete interaction with keyboard-only input and sequential screen-reader output.

Skills Checklist

- ☐ I can model the five canonical rooms.
- ☐ I can transfer the key from a room to inventory.
- ☐ I can prevent duplicate pickup.
- ☐ I can require key use in locked_door_room.
- ☐ I can block and unlock the north exit.
- ☐ I can end the game in treasure_room.

Lesson 10.4: Command-line to-do manager

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain command-line to-do manager using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

Projects begin with requirements and acceptance criteria. Implement a text-based adventure and todo manager using explicit state, validated commands, and predictable output. Accessible interaction uses keyboard-operable text prompts and readable error messages.

Lesson focus: Command-line to-do manager. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```swift
struct Todo: Identifiable {
    let id: Int
    var title: String
    var isComplete: Bool
}
var tasks = [Todo(id: 1, title: "Read lesson", isComplete: false)]
tasks.append(Todo(id: 2, title: "Run examples", isComplete: true))
for task in tasks {
    print("[\(task.isComplete ? "x" : " ")] \(task.id): \(task.title)")
}
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- Write acceptance criteria before implementing project features.
- Reject invalid directions and unknown commands explicitly.
- Do not make interaction depend on color, animation, or mouse-only controls.

Activities

1. Translate project requirements into acceptance tests.
2. Test valid room movement and unavailable exits.
3. Test inventory puzzles with and without required items.
4. Exercise todo list, add, completion, and invalid-title workflows.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

☐ I can explain command-line to-do manager in Swift terminology.
☐ I can trace the example and predict its behavior.
☐ I can adapt the example to the shared canonical data.
☐ I can identify and correct a Swift-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 10.5: Accessible interaction and validation

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain accessible interaction and validation using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

Projects begin with requirements and acceptance criteria. Implement a text-based adventure and todo manager using explicit state, validated commands,

and predictable output. Accessible interaction uses keyboard-operable text prompts and readable error messages.

Lesson focus: Accessible interaction and validation. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```
enum TitleError: Error { case blank }
import Foundation
func cleanTitle(_ raw: String) -> Result<String, TitleError> {
    let title = raw.trimmingCharacters(in: .whitespacesAndNewlines)
    return title.isEmpty ? .failure(.blank) : .success(title)
}
for raw in ["  Study Swift  ", "   "] {
    switch cleanTitle(raw) {
    case .success(let title): print("Accepted: \(title)")
    case .failure: print("Error: title cannot be blank")
    }
}
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- Write acceptance criteria before implementing project features.
- Reject invalid directions and unknown commands explicitly.
- Do not make interaction depend on color, animation, or mouse-only controls.

Activities

1. Translate project requirements into acceptance tests.
2. Test valid room movement and unavailable exits.
3. Test inventory puzzles with and without required items.
4. Exercise todo list, add, completion, and invalid-title workflows.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

- ☐ I can explain accessible interaction and validation in Swift terminology.
- ☐ I can trace the example and predict its behavior.
- ☐ I can adapt the example to the shared canonical data.
- ☐ I can identify and correct a Swift-specific error or edge case.
- ☐ I can run and inspect the program using keyboard-accessible tools.

15.13 Unit 11: Capstone

Overview

The capstone is a personal-record manager with stable identifiers, validated create/read/update/delete operations, and Codable persistence. Separate the data model, validation, storage, and command-line interface; verify save/reload and document limitations.

Lesson 11.1: Requirements and design

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain requirements and design using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

The capstone is a personal-record manager with stable identifiers, validated create/read/update/delete operations, and Codable persistence. Separate the data model, validation, storage, and command-line interface; verify save/reload and document limitations.

Lesson focus: Requirements and design. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```
let capstoneRequirements = [
    "Accessible text-first personal record manager",
    "Stable record identifiers",
    "Create, list, search, update, and delete",
```

```
    "Validate user input",
    "Save and reload records"
]
print("Capstone design:")
capstoneRequirements.forEach { print("- \($0)") }
```

Expected Behavior or Output

The encoder emits the six shared record fields using the canonical snake_case JSON names. Sorted keys make the captured output deterministic.

```
{"category":"learning","created_at":"2026-11-01","description":"Review Codable","id":1,"titl
Decoded record: 1, Study Swift, learning
```

Common Mistakes

- Keep stable identifiers separate from editable record fields.
- Validate before mutating stored records.
- Test save/reload, duplicate identifiers, missing records, and accessibility behavior.

Activities

1. Specify the record schema and validation rules before coding.
2. Test create, list, search, update, and delete operations.
3. Test duplicate, missing, blank, and invalid values.
4. Test save/reload and document keyboard-first operation and limitations.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

- ☐ I can explain requirements and design in Swift terminology.
- ☐ I can trace the example and predict its behavior.
- ☐ I can adapt the example to the shared canonical data.
- ☐ I can identify and correct a Swift-specific error or edge case.
- ☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 11.2: Data model and storage

Reading References

- The Swift Programming Language

- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain data model and storage using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

The capstone is a personal-record manager with stable identifiers, validated create/read/update/delete operations, and Codable persistence. Separate the data model, validation, storage, and command-line interface; verify save/reload and document limitations.

Lesson focus: Data model and storage. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```
import Foundation
struct PersonalRecord: Codable, Identifiable {
    let id: Int
    var title: String
    var description: String
    var category: String
    let createdAt: String
    var updatedAt: String
}
let record = PersonalRecord(id: 1, title: "Study Swift", description: "Review Codable",
                            category: "learning", createdAt: "2026-11-01", updatedAt: "2026-
do {
    let encoder = JSONEncoder()
    encoder.keyEncodingStrategy = .convertToSnakeCase
    encoder.outputFormatting = [.sortedKeys]
    let json = try encoder.encode(record)
    print(String(decoding: json, as: UTF8.self))
    let decoder = JSONDecoder()
    decoder.keyDecodingStrategy = .convertFromSnakeCase
    let restored = try decoder.decode(PersonalRecord.self, from: json)
    print("Decoded record: \(restored.id), \(restored.title), \(restored.category)")
} catch {
    print("JSON round trip failed: \(error)")
}
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- Keep stable identifiers separate from editable record fields.
- Validate before mutating stored records.
- Test save/reload, duplicate identifiers, missing records, and accessibility behavior.

Activities

1. Specify the record schema and validation rules before coding.
2. Test create, list, search, update, and delete operations.
3. Test duplicate, missing, blank, and invalid values.
4. Test save/reload and document keyboard-first operation and limitations.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

☐ I can explain data model and storage in Swift terminology.
☐ I can trace the example and predict its behavior.
☐ I can adapt the example to the shared canonical data.
☐ I can identify and correct a Swift-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 11.3: Create, read, update, delete

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain create, read, update, delete using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

The capstone is a personal-record manager with stable identifiers, validated create/read/update/delete operations, and Codable persistence. Separate the data model, validation, storage, and command-line interface; verify save/reload and document limitations.

Lesson focus: Create, read, update, delete. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```
struct Record {
    let id: Int
    var title: String
}
var records = [Record(id: 1, title: "Study Swift")]
records.append(Record(id: 2, title: "Practice optionals"))
let found = records.first { $0.id == 1 }
if let found {
    records[records.firstIndex { $0.id == found.id }!].title = "Study Swift deeply"
}
records.removeAll { $0.id == 2 }
print(records.map(\.title))
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- Keep stable identifiers separate from editable record fields.
- Validate before mutating stored records.
- Test save/reload, duplicate identifiers, missing records, and accessibility behavior.

Activities

1. Specify the record schema and validation rules before coding.
2. Test create, list, search, update, and delete operations.
3. Test duplicate, missing, blank, and invalid values.
4. Test save/reload and document keyboard-first operation and limitations.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

- ☐ I can explain create, read, update, delete in Swift terminology.
- ☐ I can trace the example and predict its behavior.
- ☐ I can adapt the example to the shared canonical data.
- ☐ I can identify and correct a Swift-specific error or edge case.
- ☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 11.4: Validation and tests

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain validation and tests using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

The capstone is a personal-record manager with stable identifiers, validated create/read/update/delete operations, and Codable persistence. Separate the data model, validation, storage, and command-line interface; verify save/reload and document limitations.

Lesson focus: Validation and tests. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```swift
import Foundation
func validateTitle(_ value: String) -> Bool {
    !value.trimmingCharacters(in: .whitespacesAndNewlines).isEmpty
}
precondition(validateTitle("Study Swift"))
precondition(!validateTitle("   "))
let duplicateIDs = [1, 2, 1]
```

```
precondition(Set(duplicateIDs).count != duplicateIDs.count)
print("Validation checks passed")
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- Keep stable identifiers separate from editable record fields.
- Validate before mutating stored records.
- Test save/reload, duplicate identifiers, missing records, and accessibility behavior.

Activities

1. Specify the record schema and validation rules before coding.
2. Test create, list, search, update, and delete operations.
3. Test duplicate, missing, blank, and invalid values.
4. Test save/reload and document keyboard-first operation and limitations.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Skills Checklist

☐ I can explain validation and tests in Swift terminology.
☐ I can trace the example and predict its behavior.
☐ I can adapt the example to the shared canonical data.
☐ I can identify and correct a Swift-specific error or edge case.
☐ I can run and inspect the program using keyboard-accessible tools.

Lesson 11.5: Documentation and presentation

Reading References

- The Swift Programming Language
- Swift Standard Library
- Matching Python curriculum lesson for the shared problem context.

Learning Objectives

- Explain documentation and presentation using Swift terminology.
- Apply the relevant language and standard-library features correctly.
- Test normal, boundary, and invalid cases using the shared course data.

Concept Explanation

The capstone is a personal-record manager with stable identifiers, validated create/read/update/delete operations, and Codable persistence. Separate the data model, validation, storage, and command-line interface; verify save/reload and document limitations.

Lesson focus: Documentation and presentation. The example uses Swift-native semantics and a canonical course context rather than mechanically translating Python.

Worked Example

```swift
let releaseNotes = [
    "Purpose: manage personal learning records",
    "Input: text commands and record fields",
    "Storage: Codable JSON document",
    "Accessibility: keyboard-first, plain-text prompts",
    "Testing: CRUD, validation, and save/reload"
]
print(releaseNotes.joined(separator: "\n"))
```

Expected Behavior or Output

The example is a self-contained Swift program. Predict the output from the stated values before running it, then compare the actual result. Examples that read input accept a line from standard input; file examples use temporary paths.

Common Mistakes

- Keep stable identifiers separate from editable record fields.
- Validate before mutating stored records.
- Test save/reload, duplicate identifiers, missing records, and accessibility behavior.

Activities

1. Specify the record schema and validation rules before coding.
2. Test create, list, search, update, and delete operations.
3. Test duplicate, missing, blank, and invalid values.
4. Test save/reload and document keyboard-first operation and limitations.

Accessibility Practice

Navigate source by line, word, and character. Inspect punctuation, braces, optionals, argument labels, and compiler diagnostics. Run from the terminal and read prompts and output sequentially. Do not rely on syntax coloring, color-only status, or mouse-only actions.

Runnable Reference Implementation

The repository includes a tested Swift domain and JSON-storage reference core in capstone/README.md. Compile and run the acceptance suite from the repository root using the commands in that README. The reference core does not replace the requirement to build and evaluate an accessible user interface.

Skills Checklist

- ☐ I can explain documentation and presentation in Swift terminology.
- ☐ I can trace the example and predict its behavior.
- ☐ I can adapt the example to the shared canonical data.
- ☐ I can identify and correct a Swift-specific error or edge case.
- ☐ I can run and inspect the program using keyboard-accessible tools.

Swift-specific platform and package notes

- Use Swift Package Manager for cross-platform command-line packages and reusable modules.
- iOS, watchOS, tvOS, and many macOS application workflows require Xcode and Apple platform SDKs.
- Foundation provides file handling, JSONEncoder/JSONDecoder, date and URL facilities, and string formatting.
- Use XCTest in a Swift package for a full test suite; the standalone examples use `precondition` for dependency-free smoke checks.
- The introductory CSV example handles simple unquoted rows only. Use a maintained CSV library for quoted commas, escaped quotes, and embedded newlines.
- Swift concurrency uses structured tasks and actors; add concurrency only when the problem requires it and test isolation and cancellation explicitly.

AI Policy

AI may be used only for personal conceptual understanding and reinforcement. It must not write, complete, autocomplete, repair, or generate course code or assessment solutions.

Part XVI

TYPESCRIPT

Chapter 16

Introduction to Computer Science with TypeScript

Before you start: complete Unit P: Computing Foundations Primer. It explains the language-neutral vocabulary and concepts used in every language track, using non-visual examples.

Accessible beginner curriculum using Node.js

This curriculum teaches introductory programming and computer science through TypeScript, using Node.js for program execution and text-based input/output. It is designed for blind and visually impaired learners who use screen readers, braille displays, magnification, keyboard navigation, or combinations of access technologies. It is also suitable for sighted beginners and mixed-vision classrooms.

TypeScript is widely used in browser applications, server-side systems, and developer tools. This course deliberately starts with Node.js and terminal programs so learners can master programming concepts without needing to interpret a visual web page or graphical interface. Browser-based development can be a later extension.

Course outcomes

By the end of this curriculum, learners should be able to:

- Explain introductory computer science concepts and the software-development cycle.
- Install and verify Node.js and use an editor, terminal, and Node.js REPL.

- Write TypeScript programs using variables, primitive values, expressions, and output.
- Explain TypeScript's compile-time type checking and JavaScript's runtime coercion, truthy/falsy values, `undefined`, `null`, and `NaN`.
- Accept and validate terminal input using Node.js input mechanisms.
- Use decisions, loops, arrays, readonly tuples, typed objects, interfaces, Maps, and common collection operations.
- Use union types, generic types, discriminated unions, and type guards to model and validate data.
- Design functions and use parameters, return values, scope, and closures.
- Explain classes and object-oriented programming at an introductory level.
- Work with files, modules, errors, and asynchronous operations introduced by the curriculum.
- Use Maps, Sets, data processing, and automated tests.
- Design, implement, test, and explain text-based projects and a capstone.
- Evaluate prompts, output order, error messages, and interaction for screen-reader usability.

The goal is a strong introductory foundation, not mastery of the entire TypeScript language or ecosystem.

Prerequisites and intended learners

No previous programming experience is required. Learners need a computer on which a supported Node.js release can be installed, a keyboard-accessible editor, and a terminal. Basic keyboard and file-management skills are useful. Screen-reader proficiency can develop alongside programming.

The course is intended primarily for beginning learners, including students using NVDA, JAWS, VoiceOver, Narrator, braille displays, or magnification. The TypeScript teacher's guide includes strategies for pacing, asynchronous concepts, accessible debugging, and differentiated instruction.

Why TypeScript and Node.js?

TypeScript connects foundational programming concepts to a large software ecosystem. The compiler checks TypeScript and emits JavaScript; Node.js executes that output, supporting a text-first course.

- Programs can run in a terminal and provide text output.
- The Node.js REPL supports short JavaScript runtime experiments; use the TypeScript compiler, editor, or playground to check TypeScript annotations.
- The same language can later be used for accessible web applications and server-side programs.

- Learners can encounter real-world concepts such as asynchronous operations and modules in a controlled sequence.

TypeScript adds static checking, but its emitted JavaScript still has JavaScript's runtime behavior. Coercion can produce surprising results when values enter through untrusted boundaries or types are bypassed; `undefined`, `null`, and `NaN` are distinct; arrays and objects have different semantics; and asynchronous code introduces ordering and error-handling questions.

Learning sequence

Unit	Main topics	Evidence of progress
0. Environment setup	Node.js, editor setup, terminal, first file, REPL	Verify Node.js and run a script
1. Foundations	Computer science, TypeScript role, variables, output, values and types	Explain a simple script and its values
2. Numbers and input	Math operations, Node.js input, conversion, and validation	Build an interactive terminal calculation
3. Control flow	Boolean logic, grade classification, arrays, iteration, and string operations	Predict and control program paths
4. Arrays and objects	Collections, indexed data, properties, and iteration	Organize and process related data
5. User input and loops	Typed terminal input, validation, while loops, flags, break, and continue	Build a robust interactive loop
6. Functions	Function forms, parameters, return values, scope, and closures	Decompose a problem into reusable functions
7. Classes and OOP	Classes, instances, fields, methods, and object-oriented design	Model a simple problem domain
8. Files and errors	File operations, modules where introduced, and error handling	Persist data and handle expected failures

Unit	Main topics	Evidence of progress
9. Testing and data analysis	node:test, class tests, random values, text frequency, and CSV parsing	Verify behavior and summarize structured data
10. Project-based learning	Text-based applications and incremental design	Implement a multi-feature project
11. Capstone	Integration, testing, documentation, and explanation	Demonstrate an independent solution

The course contains **57 mapped lesson sections across Units 0–11**. Each substantive lesson includes references, objectives, a concept explanation, a TypeScript worked example, expected behavior, common mistakes, activities, accessibility practice, and a skills checklist.

Materials and navigation

- Units 0–1: Setup and Foundations
- Units 2–4: Numbers, Input, Control Flow, and Data
- Units 5–8: Strings, Functions, OOP, and Files
- Units 9–11: Modules, Collections, Testing, and Projects
- Teacher's guide: instructor planning, accessible instruction, assessment, and AI guidance
- Topic index
- Standardization audit and validation status

Return to the curriculum overview to compare Python, JavaScript, TypeScript, and Java.

Runtime version and modern TypeScript conventions

Use **Node.js 26.13.0 LTS** (the current LTS release as of October 30, 2026) for the course. Node.js 24 is also in LTS maintenance, but new installations should use the current 26.x LTS line. Keep examples compatible with the Node.js 26 LTS runtime and use the latest security/maintenance patch in that line. Check the official Node.js release schedule before installing or upgrading.

The course follows modern TypeScript practices:

- Use const by default and let only when reassignment is required; avoid var in new code.

- Use === and !== for comparisons; convert user input explicitly with Number, String, or other deliberate parsing.
- Prefer descriptive names, small functions, template literals, array methods where they improve clarity, and async/await for asynchronous work.
- Use node: specifiers for built-in modules (for example, node:fs and node:readline/promises).
- Declare module format explicitly in package.json. Use type: commonjs for CommonJS examples and type: module for ES module examples; do not mix require/module.exports with import/export in the same module.
- Handle rejected promises and file-system errors explicitly. Do not assume parsed JSON or external input has the expected shape.
- Use the built-in node:test and node:assert/strict modules for dependency-free introductory tests; Jest is an optional extension, not a course prerequisite.
- Keep dependencies local to a project, review package provenance, and commit a lockfile for reproducible installations.

Why TypeScript is emerging

TypeScript is an increasingly common choice for application-scale JavaScript projects. It keeps JavaScript's ecosystem and runtime model while adding a static type system, so many mistakes can be found by the editor or compiler before a program runs. This is particularly useful as projects grow: function contracts, object shapes, nullability, and module boundaries become more explicit, and refactoring tools can reason about those declarations.

TypeScript is safer in important ways, but it is not a proof of correctness. Types are erased when TypeScript is compiled to JavaScript; they do not validate JSON, user input, network responses, database records, or other runtime data. A type assertion can also silence the checker without making a value safe. Teach strict checking, runtime validation at trust boundaries, and tests together.

Recommended environment

Use **Node.js 26.13.0 LTS** (current LTS as of October 30, 2026) and the current stable TypeScript compiler, **7.0.2**. TypeScript is a compile-time language layer: Node.js runs the JavaScript output, not TypeScript source in the ordinary compiled workflow.

Follow Unit 0 for detailed installation and verification. Use the latest security/maintenance patch in the Node.js 26 LTS line. Check the official Node.js download page before installing or upgrading.

Recommended tools include Node.js and npm, Visual Studio Code or Theia IDE, a terminal, and the learner's preferred access technology. Theia is an alternative with similar keyboard- and screen-reader-oriented workflows; verify language

extensions through Open VSX and test the learner's screen reader. Use project-local npm dependencies and `npx`/npm scripts; avoid global installs. Explain each third-party dependency and review its provenance and maintenance before adding it.

Typical verification commands:

```
node --version
npx tsc --version
npm --version
node
```

Windows Powershell

```
node --version
npx tsc --version
npm --version
node
```

At the Node.js prompt, enter a short JavaScript expression and read the returned value. This does not type-check TypeScript annotations. Use a TypeScript-aware editor, the TypeScript playground, or `npx tsc` to check TypeScript code. Use `.exit` or the documented keyboard command to leave the REPL.

TypeScript conventions

- Enable strict type checking in tsconfig.json; do not use any as a routine escape hatch.
- Prefer inference for obvious local values and explicit annotations for function parameters, public APIs, object shapes, and values whose intent is not obvious.
- Use interfaces or type aliases to describe object contracts, discriminated unions for state variants, and unknown rather than any for untrusted values.
- Keep strictNullChecks enabled. Check nullable values explicitly and narrow union types before using them.
- Treat type assertions as a last resort; they do not perform runtime conversion or validation.
- Compile with tsc before running the emitted JavaScript. Keep Node.js module mode explicit and use matching TypeScript module settings.

Accessibility and instructional design

TypeScript uses braces to delimit blocks and semicolons may be optional in many contexts. Teach one consistent style and explain punctuation explicitly. Screen-

reader users benefit from short functions, descriptive identifiers, predictable formatting, and output that is meaningful when read linearly.

The curriculum emphasizes keyboard-first editor, terminal, and REPL workflows; explicit reading of braces, parentheses, brackets, commas, operators, and template literals; terminal-based core activities; clear prompts and errors; and step-by-step debugging of coercion and asynchronous behavior.

Accessibility is not guaranteed merely because an application prints text. Consider output order, repeated announcements, asynchronous messages, and whether a learner can review the result after it appears. Adapt to the student's actual technology and preferences.

How to study each lesson

1. Read the lesson overview and identify the new language features.
2. Read each example structurally. Identify blocks, expressions, function calls, and data values.
3. Predict output and note assumptions about types or coercion.
4. Run the example in Node.js and compare actual and expected results.
5. Change one variable or expression and observe behavior.
6. Complete the exercises and record questions.
7. When a result is unexpected, inspect the value and its type, then trace the relevant control flow.
8. For asynchronous examples, identify when an operation begins, what is awaited, and when its result becomes available.
9. Explain the program in your own words and save the work in a portfolio.

Use the REPL for short experiments and files for complete programs. Keep experiments small enough that output can be reviewed without losing context.

Assessment and projects

Assessment can include predicting output, explaining type behavior, tracing a loop, writing a function, refactoring a program, identifying a bug, creating tests, and building a text-based application. Evaluate reasoning, correctness, testing, clarity, and the learner's ability to explain and change their code.

The capstone should include a written problem statement, requirements, input/output design, data representation, implementation milestones, test cases, and a short demonstration or explanation. The learner should be able to discuss known limitations and how to improve the program.

Teaching with AI

Teaching with AI

For student coursework and assessments, AI may be used only to support personal conceptual understanding and reinforcement. It may explain a language concept, clarify terminology, or help a learner understand a compiler diagnostic at a conceptual level. It must not write, autocomplete, repair, complete, refactor, or generate course code, project code, test solutions, or assessment answers. It must not provide a ready-to-submit algorithm or implementation.

The learner must write and debug all course code independently. Accessibility tools that provide keyboard navigation, screen-reader output, braille access, magnification, or speech input are not prohibited answer-generation tools. Instructors remain responsible for following institutional policy and protecting student information. See the teacher's guide for implementation guidance.

Repeatable curriculum validation

The repository includes a strict TypeScript checker and GitHub Actions workflow. From the repository root, install the documented checker dependencies and run:

```
node scripts/check_typescript_curriculum.mjs
```

The checker verifies all 57 lesson structures and type-checks isolated TypeScript examples in strict mode. It does not execute examples or assert their expected output; runtime and end-to-end testing remain separate verification tasks.

Recommended introductory books

- Goldberg, Josh. *Learning TypeScript*. O'Reilly Media, 2022. A practical guide to TypeScript's type system, inference, unions, narrowing, generics, and integration with JavaScript. Publisher page.

Prerequisite note: this book is most useful after learners understand basic JavaScript syntax, values, functions, arrays, objects, and modules. Complete the JavaScript foundations first or use the JavaScript curriculum as a companion. TypeScript's types are primarily checked at compile time; pair the book with this curriculum's explanations of runtime validation and JavaScript execution.

Further learning

Use MDN's TypeScript Guide for language concepts and the Node.js documentation for runtime APIs. Eloquent TypeScript is a further introductory text. Check version-specific documentation when examples depend on newer features.

The course uses TypeScript to teach transferable problem-solving skills. Learners who can reason about values, control flow, data, functions, errors, and tests are prepared to explore browser programming and other parts of the ecosystem.

GUI, visualization, and workbench accessibility

See the shared GUI, visualization, and workbench accessibility reference for screen-reader support by toolkit, platform-specific activation settings, recommended alternatives, and interfaces that must not be required without an equivalent accessible route.

16.1 TYPESCRIPT Lessons

The curriculum is organized as one Markdown file per unit.

- accessible development environment
- foundations of computing
- numeric computations
- control flow decisions and loops
- data structures arrays tuples and objects
- user input and while loops
- functions
- classes and object oriented programming
- files and exceptions
- testing and data analysis
- project based learning text based applications
- capstone accessible personal data manager

16.2 Unit 0: Accessible Development Environment

Before you start: complete Unit P: Computing Foundations Primer. It explains the language-neutral vocabulary and concepts used in every language track, using non-visual examples.

Overview

Set up Node.js, TypeScript, an editor, workspace, and terminal workflow. The TypeScript adaptation preserves the shared Python curriculum's learning objectives, datasets, expected results, and assessment criteria while teaching TypeScript's compile-time type system and JavaScript runtime behavior.

Lesson 0.1: Installing Node.js, npm, and TypeScript

Reading References

- Python source counterpart: Unit 0 lesson 0.1 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Install and verify the Node.js runtime, npm, and a project-local TypeScript compiler.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

Node.js executes JavaScript. TypeScript is checked and compiled before execution in this course. npm installs project-local tools so learners use a reproducible compiler version.

Worked Example

```
const runtime = "Node.js 26 LTS";
const compiler = "TypeScript 7.x";
console.log(`Runtime: ${runtime}`);
console.log(`Compiler: ${compiler}`);
```

Expected Behavior or Output

The two lines identify the runtime and compiler separately. `node --version`, `npm --version`, and `npx tsc --version` verify the actual installations.

Common Mistakes

- Confusing the TypeScript compiler with the Node.js runtime.
- Installing tools globally and losing project reproducibility.
- Assuming a Node.js REPL type-checks TypeScript annotations.

Activities

1. Verify Node.js and npm versions in a terminal.
2. Create a project folder and initialize npm.
3. Install TypeScript as a development dependency and verify it with npx.

Accessibility Practice

Read each command and its purpose before running it. Keep terminal output in a sequential log. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

- ☐ I can distinguish Node.js from TypeScript.
- ☐ I can verify the runtime, npm, and compiler.
- ☐ I can install a project-local compiler.

Lesson 0.2: Configuring VS Code or Theia for accessible TypeScript work

Reading References

- Python source counterpart: Unit 0 lesson 0.2 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Configure an editor, TypeScript language service, terminal, and keyboard-first navigation.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

The editor provides navigation, diagnostics, and refactoring support; the compiler remains the authoritative type checker. VS Code and Theia may differ in shortcuts and extension availability.

Worked Example

```
const editor = "VS Code or Theia";
const languageService = "TypeScript";
console.log(`${editor}: ${languageService} support enabled`);
```

Expected Behavior or Output

The message confirms the intended editor and language support; verify actual navigation and diagnostic announcements with the learner's access technology.

Common Mistakes

- Assuming the editor's visual layout is required for completing tasks.
- Using mouse-only instructions or undocumented shortcuts.
- Treating a language-service suggestion as proof that a program is correct.

Activities

1. Open a workspace using the keyboard.
2. Navigate to a diagnostic by keyboard and read its full message.
3. Run a TypeScript build from the integrated terminal.

Accessibility Practice

Test line, word, character, symbol, and diagnostic navigation with the learner's screen reader or braille display. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

☐ I can navigate the editor by keyboard.
☐ I can locate and understand a TypeScript diagnostic.
☐ I can run a build without relying on mouse-only controls.

Lesson 0.3: Creating the course workspace and editor extensions

Reading References

- Python source counterpart: Unit 0 lesson 0.3 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Create a consistent workspace and configure TypeScript formatting and project navigation.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.

- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

A workspace groups source files and configuration. TypeScript projects use `package.json` for scripts and dependencies and `tsconfig.json` for compiler behavior.

Worked Example

```
const project = { name: "typescript-course", source: "src", output: "dist" };
console.log(`${project.name}: ${project.source} -> ${project.output}`);
```

Expected Behavior or Output

The project name and source/output folders are announced as text. A standard project keeps editable TypeScript in src and emitted JavaScript in dist.

Common Mistakes

- Editing generated JavaScript instead of TypeScript source.
- Placing course files in an unknown working directory.
- Installing editor extensions without checking publisher and availability.

Activities

1. Create src and dist folders.
2. Open the project folder rather than an individual file.
3. Enable TypeScript language support and verify its version.

Accessibility Practice

Use descriptive folder and file names. Announce the current working directory before running project commands. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

☐ I can create and open a project workspace.
☐ I can identify source and generated-output folders.
☐ I can verify TypeScript editor support.

Project configuration and module mode

Use a project-local TypeScript compiler and strict checking. A representative `tsconfig.json` is:

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "NodeNext",
    "moduleResolution": "NodeNext",
    "strict": true,
    "noEmitOnError": true,
    "outDir": "dist",
    "rootDir": "src"
  },
  "include": ["src/**/*.ts"]
}
```

For the compiled Node.js workflow, set `"type": "module"` in package.json and use ES module imports consistently. If a project uses CommonJS instead, set `"type": "commonjs"` and use matching compiler settings. Do not mix `require()` and `import` in one module.

A typical script is `"build": "tsc"` and `"start": "node dist/hello.js"`. Run `npm run build` before `npm start`.

Lesson 0.4: Your first TypeScript file and terminal workflow

Reading References

- Python source counterpart: Unit 0 lesson 0.4 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Write, compile, and execute a small TypeScript program from the terminal.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

The compiler checks `.ts` source and emits `.js`. Node.js runs the emitted JavaScript. A package script makes the workflow repeatable.

Worked Example

```
const greeting: string = "Hello, World!";
console.log(greeting);
console.log("Welcome to TypeScript.");
```

Expected Behavior or Output

Compilation succeeds and Node.js prints `Hello, World!` and `Welcome to TypeScript.` on separate lines.

Common Mistakes

- Trying to run a TypeScript file as though it were ordinary JavaScript.
- Running Node.js from the wrong working directory.
- Ignoring compiler errors and executing stale output.

Activities

1. Save the example as src/hello.ts.
2. Compile with the project tsconfig.
3. Run the emitted JavaScript with Node.js.

Accessibility Practice

Read compiler diagnostics before running the program. Keep source and output paths explicit. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

☐ I can write a .ts file.
☐ I can compile it with tsc.
☐ I can run emitted JavaScript with Node.js.

16.3 Unit 1: Foundations of Computing

Overview

Introduce computer science, first programs, variables, and primitive types. The TypeScript adaptation preserves the shared Python curriculum's learning objectives, datasets, expected results, and assessment criteria while teaching TypeScript's compile-time type system and JavaScript runtime behavior.

Lesson 1.1: Welcome to computer science

Concept Explanation

Computer science studies computation and information. An algorithm is a finite, ordered set of steps for solving a problem; TypeScript is one language used to express algorithms.

Worked Example

```
const steps = ["understand the problem", "design steps", "implement", "test"];
for (const [index, step] of steps.entries()) {
  console.log(`${index + 1}. ${step}`);
}
```

Expected Behavior or Output

Four ordered steps are printed, from understanding the problem through testing.

Common Mistakes

- Equating computer science with learning syntax only.
- Beginning implementation before defining the problem.
- Treating one programming language as the only way to express an algorithm.

Activities

1. Describe an everyday algorithm in plain language.
2. Identify input, processing, and output for a tip calculator.
3. Explain how testing provides evidence about a solution.

Accessibility Practice

Present algorithms as numbered text steps. Do not rely on flowchart colors or spatial layout alone. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

☐ I can define computer science and an algorithm.
☐ I can identify input, processing, and output.
☐ I can describe a testable solution plan.

Lesson 1.2: Your first TypeScript program: Hello World

Reading References

- Python source counterpart: Unit 1 lesson 1.2 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Create a source file, compile it, and explain sequential execution and output.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

Statements execute in sequence unless control flow changes that order. `console.log` writes a line of text to standard output.

Worked Example

```
console.log("Hello, World!");
console.log("Welcome to TypeScript.");
```

Expected Behavior or Output

The terminal prints the two messages in source order, one per line.

Common Mistakes

- Forgetting to save the source file before compiling.
- Expecting TypeScript annotations to appear in runtime output.
- Changing capitalization or punctuation in an exact-output exercise.

Activities

1. Change the second message and compile again.
2. Add a third line and predict its position.
3. Explain the difference between source code and terminal output.

Accessibility Practice

Read one statement at a time and compare source order with output order. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

- ☐ I can create and run a first program.
- ☐ I can predict sequential output.
- ☐ I can distinguish source text from program output.

Lesson 1.3: Variables and simple data types

Reading References

- Python source counterpart: Unit 1 lesson 1.3 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Declare typed variables and represent the canonical student profile.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

TypeScript can infer many local types, but annotations make contracts explicit. `const` prevents reassignment of a binding; it does not deeply freeze an object.

Worked Example

```
const student: { name: string; age: number; heightM: number; favoriteSubject: string } = {
  name: "Avery", age: 16, heightM: 1.72, favoriteSubject: "Computer Science"
};
console.log(`${student.name}, age ${student.age}`);
console.log(`Height: ${student.heightM} m`);
console.log(student.favoriteSubject);
```

Expected Behavior or Output

The output identifies Avery, age 16, height 1.72 m, and Computer Science.

Common Mistakes

- Using a string where a number is required.
- Assuming const makes object properties immutable.
- Using any to bypass a type error rather than correcting the type.

Activities

1. Declare string, number, and boolean values.
2. Add a field to the student profile and update its type.
3. Explain type inference versus an explicit annotation.

Accessibility Practice

Read property names and types aloud. Use one property per line for structured output. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

☐ I can declare typed variables.
☐ I can explain const and type inference.
☐ I can model the canonical student profile.

16.4 Unit 2: Numeric Computations

Overview

Translate requirements into programs, expressions, input, conversions, and mathematical calculations. The TypeScript adaptation preserves the shared Python curriculum's learning objectives, datasets, expected results, and assessment criteria while teaching TypeScript's compile-time type system and JavaScript runtime behavior.

Overview

Translate requirements into programs, expressions, input, conversions, and mathematical calculations. The TypeScript adaptation preserves the shared Python curriculum's learning objectives, datasets, expected results, and assessment criteria while teaching TypeScript's compile-time type system and JavaScript runtime behavior.

Lesson 2.1: The software development process

Reading References

- Python source counterpart: Unit 2 lesson 2.1 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Apply the design, implementation, and testing cycle to the canonical tip calculator.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

A problem statement becomes requirements, an algorithm, an implementation, and tests. The shared calculator uses a $24.50 bill and a 15% tip rate.

Worked Example

```
const bill = 24.50;
const tipRate = 0.15;
const tip = bill * tipRate;
const total = bill + tip;
console.log(`Tip: $${tip.toFixed(2)}`);
console.log(`Total: $${total.toFixed(2)}`);
```

Expected Behavior or Output

The tip is $3.67 and the total is $28.18 when rounded with `toFixed(2)`. The exact product 3.675 is stored as 3.67499999999999982, so `toFixed(2)` rounds the tip down.

Common Mistakes

- Coding before identifying inputs and expected results.
- Confusing the tip amount with the final total.
- Assuming binary floating-point represents every decimal exactly.

Activities

1. Write requirements for the tip calculator.
2. Predict the result before running the example.
3. Test a zero tip rate and a different bill amount.

Accessibility Practice

State the input values and output labels explicitly. Explain rounding rather than relying on visual currency formatting. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

- ☐ I can describe the development cycle.
- ☐ I can translate requirements into an algorithm.
- ☐ I can compare expected and actual results.

Lesson 2.2: Expressions and assignment

Reading References

- Python source counterpart: Unit 2 lesson 2.2 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Evaluate arithmetic expressions and update a score using typed variables.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

An expression produces a value. Assignment stores a value in a variable. Parentheses make grouping explicit and TypeScript checks the declared value types.

Worked Example

```
let score: number = 10;
score = score + 5;
const subtotal = 2 + 3 * 4;
const grouped = (2 + 3) * 4;
console.log(score, subtotal, grouped);
```

Expected Behavior or Output

The output values are 15, 14, and 20.

Common Mistakes

- Assuming multiplication and addition have equal precedence.
- Using const for a variable that must be reassigned.
- Confusing equality comparison with assignment.

Activities

1. Predict the value of expressions with and without parentheses.
2. Update a score by adding 5.
3. Use a comparison expression and identify its boolean result.

Accessibility Practice

Keep each expression on a separate line and narrate operator order. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

- ☐ I can evaluate arithmetic expressions.
- ☐ I can explain assignment and reassignment.
- ☐ I can use parentheses to make precedence clear.

Lesson 2.3: Input and output in Node.js

Reading References

- Python source counterpart: Unit 2 lesson 2.3 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Prompt for a name and two numbers using asynchronous terminal input.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

Node.js readline/promises returns Promises. `await` pauses the async function until the user responds. Input arrives as text and must be converted and validated.

Worked Example

```
import { createInterface } from "node:readline/promises";
import { stdin, stdout } from "node:process";
```

```ts
async function main(): Promise<void> {
  const rl = createInterface({ input: stdin, output: stdout });
  try {
    const name = await rl.question("Enter your name: ");
    const first = Number(await rl.question("First number: "));
    const second = Number(await rl.question("Second number: "));
    if (!Number.isFinite(first) || !Number.isFinite(second)) {
      console.log("Enter valid numbers.");
      return;
    }
    console.log(`Hello, ${name}!`);
    console.log(`Sum: ${first + second}`);
  } finally {
    rl.close();
  }
}
void main();
```

Expected Behavior or Output

For name Avery and numbers 12 and 8, output is `Hello, Avery!` and `Sum: 20`. The interface closes on both success and early return.

Common Mistakes

- Forgetting that terminal input is text.
- Using a number without checking whether conversion succeeded.
- Leaving the readline interface open so the process does not exit.

Activities

1. Prompt for a name and two numbers.
2. Try blank and nonnumeric input.
3. Explain why await and finally are used.

Accessibility Practice

Use clear prompts with trailing spaces and one question at a time. Announce validation errors in plain text. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

- ☐ I can collect terminal input asynchronously.
- ☐ I can convert and validate numeric text.
- ☐ I can close the input interface reliably.

Lesson 2.4: Computing with numbers and type conversions

Reading References

- Python source counterpart: Unit 2 lesson 2.4 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Convert numeric text explicitly and calculate age, measurement, and validation results.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

TypeScript's static types do not change runtime parsing. `Number` converts text; `Number.isFinite` rejects NaN and infinities. The source type must reflect values after validation.

Worked Example

```
const ageText: string = "16";
const age = Number(ageText);
const heightM = 1.72;
const heightCm = heightM * 100;
console.log(`Age next year: ${age + 1}`);
console.log(`Height: ${heightCm} cm`);
console.log(Number.isFinite(age));
console.log(Number.isFinite(Number("sixteen")));
```

Expected Behavior or Output

Output: age next year 17, height 172 cm, true, false.

Common Mistakes

- Assuming a type annotation validates text received at runtime.
- Treating Number("") as invalid even though it becomes zero.
- Failing to check NaN after conversion.

Activities

1. Convert the string 16 to a number.
2. Compare Number("16"), Number(""), and Number("sixteen").
3. Validate a user-entered age within a reasonable range.

Accessibility Practice

Explain the difference between compile-time type checking and runtime input validation. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

☐ I can convert numeric strings.
☐ I can detect invalid numeric conversion.
☐ I can explain why external values need runtime checks.

Lesson 2.5: Using the Math object

Reading References

- Python source counterpart: Unit 2 lesson 2.5 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Use Math constants and functions for circle measurements and distance.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

JavaScript and TypeScript expose mathematical functions through the built-in Math object. `Math.PI`, `Math.hypot`, `Math.ceil`, and `Math.floor` avoid unnecessary imports.

Worked Example

```
const radius = 5;
const area = Math.PI * radius ** 2;
const circumference = 2 * Math.PI * radius;
```

```
const distance = Math.hypot(3, 4);
console.log(area.toFixed(2));
console.log(circumference.toFixed(2));
console.log(distance.toFixed(2));
```

Expected Behavior or Output

Area 78.54, circumference 31.42, and distance 5.00.

Common Mistakes

- Using degrees where a function expects radians.
- Using the wrong exponent or formula for area.
- Rounding intermediate values instead of formatting final output.

Activities

1. Calculate a circle's area and circumference for radius 5.
2. Calculate distance from coordinates (3, 4).
3. Compare Math.ceil and Math.floor for 4.2.

Accessibility Practice

Print each result with a descriptive label and units. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

☐ I can use Math constants and functions.
☐ I can calculate circle measurements.
☐ I can calculate distance from the origin.

16.5 Unit 3: Control Flow — Decisions and Loops

Overview

Use boolean logic, grade decisions, arrays, iteration, and text operations. The TypeScript adaptation preserves the shared Python curriculum's learning objectives, datasets, expected results, and assessment criteria while teaching TypeScript's compile-time type system and JavaScript runtime behavior.

Lesson 3.1: Boolean expressions and if statements

Reading References

- Python source counterpart: Unit 3 lesson 3.1 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Combine comparisons and boolean operators to validate a score.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

A boolean expression evaluates to true or false. `&&`, `||`, and `!` combine or reverse conditions. TypeScript requires conditions to be boolean in strict code.

Worked Example

```
const score = 82;
const valid = score >= 0 && score <= 100;
if (valid && score >= 60) {
  console.log("Valid passing score.");
} else {
  console.log("Review score.");
}
```

Expected Behavior or Output

For score 82, the program prints `Valid passing score.`

Common Mistakes

- Using & instead of && for logical conjunction.
- Forgetting range validation before classification.
- Assuming a numeric value is itself a boolean.

Activities

1. Test scores -1, 0, 59, 60, 82, and 100.
2. Combine two conditions with &&.
3. Explain the difference between a comparison and a boolean variable.

Accessibility Practice

State each condition in words before reading its punctuation. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

- ☐ I can write comparisons.
- ☐ I can combine boolean conditions.
- ☐ I can validate a score range.

Lesson 3.2: Chained conditionals and grade classification

Reading References

- Python source counterpart: Unit 3 lesson 3.2 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Classify scores using the shared A–F thresholds and ordered branches.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

An if/else-if/else chain evaluates conditions in order and executes the first matching branch. Order matters when conditions overlap.

Worked Example

```
function getGrade(score: number): string {
  if (score >= 90) return "A";
  else if (score >= 80) return "B";
  else if (score >= 70) return "C";
  else if (score >= 60) return "D";
  else return "F";
}
for (const score of [82, 91, 76, 88]) {
  console.log(`${score}: ${getGrade(score)}`);
}
```

Expected Behavior or Output

The classifications are 82:B, 91:A, 76:C, and 88:B.

Common Mistakes

- Checking a lower threshold before a higher threshold.
- Omitting the boundary score 60 or 90 from tests.
- Accepting scores outside 0–100 without validation.

Activities

1. Classify the canonical scores.
2. Test every threshold and the value immediately below it.
3. Add a guard for scores outside 0–100.

Accessibility Practice

Use one result per line and include the original score in the output. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

☐ I can write a chained conditional.
☐ I can classify the canonical scores.
☐ I can test boundary conditions.

Lesson 3.3: Introduction to arrays

Reading References

- Python source counterpart: Unit 3 lesson 3.3 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Create, index, and modify the canonical score array.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

An array stores an ordered sequence. TypeScript uses `number[]` or `Array<number>` to describe a numeric array. Indexes begin at zero.

Worked Example

```
const scores: number[] = [82, 91, 76, 88];
console.log(scores.length);
console.log(scores[0]);
scores[2] = 80;
console.log(scores[2]);
```

Expected Behavior or Output

The array length is 4; the first score is 82; after modification, the third score is 80.

Common Mistakes

- Using one-based indexes.
- Reading an index that may be out of range without checking.
- Assuming a const array cannot have its elements changed.

Activities

1. Create the four canonical scores.
2. Read the first and last values safely.
3. Change the third value to 80 and print the array.

Accessibility Practice

Announce indexes as zero-based and read values with their positions. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

☐ I can declare a typed array.
☐ I can access and update elements.
☐ I can explain zero-based indexing.

Lesson 3.4: Iterating with for loops

Reading References

- Python source counterpart: Unit 3 lesson 3.4 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Iterate through scores and calculate their total and average.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

A for...of loop visits each array value. A traditional for loop exposes the index. Choose the form that makes the task easiest to understand.

Worked Example

```ts
const scores: number[] = [82, 91, 76, 88];
let total = 0;
for (const score of scores) {
  total += score;
}
console.log(`Total: ${total}`);
console.log(`Average: ${total / scores.length}`);
```

Expected Behavior or Output

Total 337 and average 84.25.

Common Mistakes

- Adding array indexes instead of values.
- Dividing by zero for an empty collection.
- Changing the array while iterating without planning the effect.

Activities

1. Print every score on a separate line.
2. Calculate the canonical total and average.
3. Repeat using an indexed for loop.

Accessibility Practice

Keep loop output sequential and label totals and averages. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

- ☐ I can use for...of to visit values.
- ☐ I can accumulate a total.
- ☐ I can calculate an average safely.

Lesson 3.5: Basic string operations

Reading References

- Python source counterpart: Unit 3 lesson 3.5 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Normalize names and search text using immutable string methods.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

Strings are immutable. Methods such as trim, toUpperCase, and includes return values rather than modifying the original string. Template literals interpolate expressions.

Worked Example

```
const rawName = "  Avery Student  ";
const name = rawName.trim();
const sentence = "A student studies programming. Programming develops problem-solving skills
console.log(name);
console.log(name.toUpperCase());
console.log(sentence.toLowerCase().includes("programming"));
console.log(name.length);
```

Expected Behavior or Output

Output includes `Avery Student`, uppercase form, true, and character count 13.

Common Mistakes

- Expecting trim or toUpperCase to mutate the original string.
- Searching without considering case normalization.
- Counting spaces that should have been trimmed.

Activities

1. Trim the canonical name.
2. Search the canonical text case-insensitively.
3. Count characters before and after trimming.

Accessibility Practice

Print original and normalized values with clear labels. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

☐ I can use common string methods.
☐ I can explain string immutability.
☐ I can normalize and search text.

16.6 Unit 4: Data Structures — Arrays, Tuples, and Objects

Overview

Manipulate arrays and model fixed sequences, records, Maps, and object iteration. The TypeScript adaptation preserves the shared Python curriculum's learning objectives, datasets, expected results, and assessment criteria while teaching TypeScript's compile-time type system and JavaScript runtime behavior.

Lesson 4.1: More on arrays

Reading References

- Python source counterpart: Unit 4 lesson 4.1 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html

- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Add, insert, update, remove, and inspect elements in a typed array.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

Array methods such as push, splice, and pop mutate an array. `slice` returns a shallow copy. Use types to constrain the values stored.

Worked Example

```
const scores: number[] = [82, 91, 76, 88];
scores.push(95);
scores.splice(1, 1);
console.log(scores.join(", "));
console.log(scores.length);
```

Expected Behavior or Output

The resulting values are 82, 76, 88, 95 and the length is 4.

Common Mistakes

- Confusing splice with slice.
- Using an invalid index without checking it.
- Assuming push returns the updated array rather than the new length.

Activities

1. Append a score.
2. Remove a selected score with splice.
3. Compare the original array with a slice copy.

Accessibility Practice

Read the mutation step before reporting the new array contents. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

☐ I can add and remove array elements.
☐ I can distinguish splice from slice.

☐ I can inspect array length and contents.

Lesson 4.2: Organizing arrays: sorting and reversing

Reading References

- Python source counterpart: Unit 4 lesson 4.2 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Sort and reverse scores without unexpectedly changing the original collection.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

The default sort compares values as strings. Numeric sorting needs a comparator. `toSorted` returns a sorted copy on supported modern runtimes; spread plus sort is a compatible alternative.

Worked Example

```
const scores = [82, 91, 76, 88];
const ascending = [...scores].sort((a, b) => a - b);
const descending = [...ascending].reverse();
console.log(scores.join(", "));
console.log(ascending.join(", "));
console.log(descending.join(", "));
```

Expected Behavior or Output

Original: 82, 91, 76, 88. Ascending: 76, 82, 88, 91. Descending: 91, 88, 82, 76.

Common Mistakes

- Using default lexicographic sort for numbers.
- Sorting the original when it must be preserved.
- Reversing a copy before it has been sorted.

Activities

1. Sort the canonical scores ascending.
2. Create a descending copy.
3. Explain why the original array remains unchanged.

Accessibility Practice

Print each sequence with a label so screen-reader users can distinguish them. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

☐ I can sort numeric values with a comparator.
☐ I can reverse a sequence.
☐ I can preserve the original array.

Lesson 4.3: Tuples and readonly data

Reading References

- Python source counterpart: Unit 4 lesson 4.3 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Represent fixed coordinates with a TypeScript readonly tuple.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

TypeScript supports tuple types for fixed-length sequences with known positions. `readonly` prevents element reassignment during type checking; it does not create a runtime security boundary.

Worked Example

```
const coordinates: readonly [number, number] = [3, 4];
const [x, y] = coordinates;
console.log(`x: ${x}`);
```

```
console.log(`y: ${y}`);
console.log(`Distance: ${Math.hypot(x, y)}`);
```

Expected Behavior or Output

The output is x 3, y 4, and distance 5.

Common Mistakes

- Using a general number array when exactly two values are required.
- Assuming readonly adds deep runtime immutability.
- Reversing the semantic order of x and y.

Activities

1. Declare the canonical coordinate tuple.
2. Destructure x and y into named variables.
3. Compare a tuple type with a general number array.

Accessibility Practice

Use named variables and labels so tuple positions have clear meaning. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

☐ I can declare a tuple type.
☐ I can use readonly for compile-time protection.
☐ I can explain tuple positions.

Lesson 4.4: Introducing dictionaries: typed objects and records

Reading References

- Python source counterpart: Unit 4 lesson 4.4 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Represent inventory items with interfaces and look up quantities with Map.

- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

An interface describes the shape of an object. A Map stores dynamic key-value pairs and supports keys of different types. These structures serve different purposes.

Worked Example

```
interface InventoryItem { name: string; quantity: number; unitPrice: number }
const inventory: InventoryItem[] = [
  { name: "Notebook", quantity: 3, unitPrice: 2.5 },
  { name: "Pencil", quantity: 10, unitPrice: 0.5 },
  { name: "Folder", quantity: 4, unitPrice: 1.25 }
];
const quantities = new Map<string, number>(inventory.map(item => [item.name, item.quantity]))
console.log(quantities.get("Notebook"));
console.log(quantities.has("Pencil"));
```

Expected Behavior or Output

The Notebook quantity is 3 and the Map contains Pencil.

Common Mistakes

- Using a plain object when Map key semantics are needed.
- Using Map bracket notation instead of get and set.
- Forgetting to define a type for structured records.

Activities

1. Create the three canonical inventory items.
2. Create a Map from item names to quantities.
3. Calculate the total inventory value of $17.50.

Accessibility Practice

Print each inventory item as a labeled record rather than a visual table only. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

☐ I can describe an object with an interface.
☐ I can create and query a Map.

☐ I can distinguish record types from dynamic key-value collections.

TypeScript type-system extension

Interfaces describe object shapes; generic types let a function or collection preserve the type of the values it handles. A discriminated union represents a value that can be one of several known states.

```
type Result<T> =
  | { ok: true; value: T }
  | { ok: false; error: string };

function first<T>(values: readonly T[]): T | undefined {
  return values[0];
}

function describeResult<T>(result: Result<T>): string {
  if (result.ok) return `Success: ${String(result.value)}`;
  return `Failure: ${result.error}`;
}

console.log(first<number>([82, 91, 76, 88]));
console.log(describeResult<number>({ ok: false, error: "No score" }));
```

The `ok` property narrows the union to its success or failure shape. Generic `T` preserves the element type rather than discarding it as `any`.

Lesson 4.5: Looping through objects and Maps

Reading References

- Python source counterpart: Unit 4 lesson 4.5 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Traverse object properties and Map entries to create readable reports.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

`Object.entries` returns key-value pairs from an object. Map is directly iterable and yields typed key-value pairs. Use explicit types to preserve useful information during traversal.

Worked Example

```
const student = { name: "Avery", age: 16, favoriteSubject: "Computer Science" };
for (const [key, value] of Object.entries(student)) {
  console.log(`${key}: ${value}`);
}
const counts = new Map<string, number>([["Notebook", 3], ["Pencil", 10]]);
for (const [item, quantity] of counts) {
  console.log(`${item}: ${quantity}`);
}
```

Expected Behavior or Output

The program prints each student property and then the two inventory quantities.

Common Mistakes

- Confusing object iteration with Map iteration.
- Assuming Object.entries preserves a domain-specific order contract.
- Using values without labeling their keys.

Activities

1. Print the Avery profile one property per line.
2. Iterate over the inventory Map.
3. Compare Object.keys, Object.values, and Object.entries.

Accessibility Practice

Keep one key-value pair per output line. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

☐ I can traverse object entries.
☐ I can traverse Map entries.
☐ I can produce a sequential text report.

16.7 Unit 5: User Input and while Loops

Overview

Build input validation and sentinel-controlled loops. The TypeScript adaptation preserves the shared Python curriculum's learning objectives, datasets, expected results, and assessment criteria while teaching TypeScript's compile-time type system and JavaScript runtime behavior.

Overview

Build input validation and sentinel-controlled loops. The TypeScript adaptation preserves the shared Python curriculum's learning objectives, datasets, expected results, and assessment criteria while teaching TypeScript's compile-time type system and JavaScript runtime behavior.

Lesson 5.1: How terminal input works

Reading References

- Python source counterpart: Unit 5 lesson 5.1 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Collect text from the user and validate it before using it.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

The readline/promises interface returns a string for each answer. TypeScript's annotation does not convert or validate that string.

Worked Example

```
import { createInterface } from "node:readline/promises";
import { stdin, stdout } from "node:process";
async function main(): Promise<void> {
  const rl = createInterface({ input: stdin, output: stdout });
  try {
    const subject = await rl.question("Favorite subject: ");
    if (subject.trim() === "") console.log("A subject is required.");
```

```
    else console.log(`Favorite subject: ${subject.trim()}`);
  } finally { rl.close(); }
}
void main();
```

Expected Behavior or Output

A valid answer prints `Favorite subject: Computer Science`; blank input prints the required-field message.

Common Mistakes

- Declaring the input as number before parsing it.
- Forgetting to trim whitespace before checking blank input.
- Not closing the readline interface.

Activities

1. Prompt for a favorite subject.
2. Test blank input and surrounding spaces.
3. Explain why the input variable has type string.

Accessibility Practice

Use one prompt and one response at a time. State validation failures explicitly. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

☐ I can collect a string from terminal input.
☐ I can reject blank input.
☐ I can close the readline interface.

Overview

Build input validation and sentinel-controlled loops. The TypeScript adaptation preserves the shared Python curriculum's learning objectives, datasets, expected results, and assessment criteria while teaching TypeScript's compile-time type system and JavaScript runtime behavior.

Lesson 5.1: How terminal input works

Reading References

- Python source counterpart: Unit 5 lesson 5.1 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html

- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Collect text from the user and validate it before using it.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

A while loop repeats while its condition is true. The loop body must eventually change the state used by the condition or exit explicitly.

Worked Example

```
const commands = ["view", "help", "quit"];
let index = 0;
while (commands[index] !== "quit") {
  console.log(`Command: ${commands[index]}`);
  index += 1;
}
console.log("Session ended.");
```

Expected Behavior or Output

The program prints view, help, and Session ended. It does not print quit as an ordinary command.

Common Mistakes

- Forgetting to update the loop index.
- Checking the wrong sentinel value.
- Accessing an array element without accounting for the end of the array.

Activities

1. Trace each loop iteration on paper.
2. Change the sentinel command and predict termination.
3. Write a menu loop that ends on quit.

Accessibility Practice

Show the loop condition and state update as separate lines. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

- ☐ I can identify a while-loop condition.
- ☐ I can update state to ensure progress.
- ☐ I can use a sentinel to end a loop.

Lesson 5.3: Using flags and break to exit loops

Reading References

- Python source counterpart: Unit 5 lesson 5.3 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Use a boolean flag and break to stop a search at the first match.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

A flag records whether a condition has been met. `break` exits the nearest loop immediately. TypeScript can infer a boolean flag or use an explicit annotation.

Worked Example

```
const scores = [82, 91, 76, 88];
let found = false;
for (const score of scores) {
  if (score > 90) {
    console.log(`First score above 90: ${score}`);
    found = true;
    break;
  }
}
if (!found) console.log("No score above 90.");
```

Expected Behavior or Output

The first score above 90 is 91.

Common Mistakes

- Setting the flag after a break that can never be reached.
- Using break outside a loop or switch.
- Failing to report the no-match case.

Activities

1. Search for a score above 90.
2. Search for a score above 95 and test the fallback.
3. Compare flag-controlled termination with a return from a function.

Accessibility Practice

Report the match or no-match result in text, not by color or sound alone. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

☐ I can use a boolean flag.
☐ I can stop a loop with break.
☐ I can handle a search with no result.

Lesson 5.4: Using continue in loops

Reading References

- Python source counterpart: Unit 5 lesson 5.4 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Skip invalid values while continuing to process later entries.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

`continue` skips the rest of the current iteration and begins the next one. It does not terminate the loop. A typed array can still contain invalid values when values come from runtime sources.

Worked Example

```
const entries: string[] = ["82", "", "91", "not a score", "76", "88"];
let total = 0;
for (const entry of entries) {
  const score = Number(entry);
  if (!Number.isFinite(score)) continue;
  total += score;
}
console.log(total);
```

Expected Behavior or Output

The valid entries total 337; the blank entry converts to zero with Number, so a stronger version should reject blank strings explicitly before conversion.

Common Mistakes

- Assuming Number on blank text is an error.
- Using break when later values still need processing.
- Placing the update after continue so it is skipped.

Activities

1. Add an explicit blank-string check.
2. Skip nonnumeric entries and count accepted values.
3. Compare continue and break with a traced example.

Accessibility Practice

Print the number of accepted and skipped entries so outcomes are clear. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

☐ I can use continue to skip an iteration.
☐ I can validate before conversion.
☐ I can distinguish continue from break.

Lesson 5.5: Avoiding infinite loops

Reading References

- Python source counterpart: Unit 5 lesson 5.5 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Diagnose loop termination by tracing the control variable and condition.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

An infinite loop occurs when its continuation condition never becomes false. TypeScript cannot prove that a loop will terminate; test the control variable and every branch that updates it.

Worked Example

```
let count = 1;
while (count <= 3) {
  console.log(`Count ${count}`);
  count += 1;
}
console.log("Loop finished.");
```

Expected Behavior or Output

The output is Count 1, Count 2, Count 3, then Loop finished.

Common Mistakes

- Forgetting to increment or otherwise update the control variable.
- Using continue before the update statement.
- Using a boundary condition that never becomes false.

Activities

1. Trace the value of count on each iteration.
2. Identify a continue path that skips an update.
3. Describe a safe way to stop a runaway terminal process.

Accessibility Practice

Provide a keyboard-only recovery procedure and explicit termination message. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

- ☐ I can trace loop state.
- ☐ I can identify non-terminating paths.
- ☐ I can design a loop with a clear exit.

16.8 Unit 6: Functions

Overview

Define typed functions, arguments, return values, and collection-processing functions. The TypeScript adaptation preserves the shared Python curriculum's learning objectives, datasets, expected results, and assessment criteria while teaching TypeScript's compile-time type system and JavaScript runtime behavior. The readline/promises interface returns a string for each answer. TypeScript's annotation does not convert or validate that string.

Worked Example

```typescript
import { createInterface } from "node:readline/promises";
import { stdin, stdout } from "node:process";
async function main(): Promise<void> {
  const rl = createInterface({ input: stdin, output: stdout });
  try {
    const subject = await rl.question("Favorite subject: ");
    if (subject.trim() === "") console.log("A subject is required.");
    else console.log(`Favorite subject: ${subject.trim()}`);
  } finally { rl.close(); }
}
void main();
```

Expected Behavior or Output

A valid answer prints `Favorite subject: Computer Science`; blank input prints the required-field message.

Common Mistakes

- Declaring the input as number before parsing it.

- Forgetting to trim whitespace before checking blank input.
- Not closing the readline interface.

Activities

1. Prompt for a favorite subject.
2. Test blank input and surrounding spaces.
3. Explain why the input variable has type string.

Accessibility Practice

Use one prompt and one response at a time. State validation failures explicitly. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

☐ I can collect a string from terminal input.
☐ I can reject blank input.
☐ I can close the readline interface.

Lesson 5.2: The while loop in action

Reading References

- Python source counterpart: Unit 5 lesson 5.2 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Repeat a prompt until a sentinel command is entered.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

A while loop repeats while its condition is true. The loop body must eventually change the state used by the condition or exit explicitly.

Worked Example

```
const commands = ["view", "help", "quit"];
let index = 0;
while (commands[index] !== "quit") {
  console.log(`Command: ${commands[index]}`);
  index += 1;
}
console.log("Session ended.");
```

Expected Behavior or Output

The program prints view, help, and Session ended. It does not print quit as an ordinary command.

Common Mistakes

- Forgetting to update the loop index.
- Checking the wrong sentinel value.
- Accessing an array element without accounting for the end of the array.

Activities

1. Trace each loop iteration on paper.
2. Change the sentinel command and predict termination.
3. Write a menu loop that ends on quit.

Accessibility Practice

Show the loop condition and state update as separate lines. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

☐ I can identify a while-loop condition.
☐ I can update state to ensure progress.
☐ I can use a sentinel to end a loop.

Lesson 5.3: Using flags and break to exit loops

Reading References

- Python source counterpart: Unit 5 lesson 5.3 in the canonical Python curriculum.

- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Use a boolean flag and break to stop a search at the first match.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

A flag records whether a condition has been met. `break` exits the nearest loop immediately. TypeScript can infer a boolean flag or use an explicit annotation.

Worked Example

```
const scores = [82, 91, 76, 88];
let found = false;
for (const score of scores) {
  if (score > 90) {
    console.log(`First score above 90: ${score}`);
    found = true;
    break;
  }
}
if (!found) console.log("No score above 90.");
```

Expected Behavior or Output

The first score above 90 is 91.

Common Mistakes

- Setting the flag after a break that can never be reached.
- Using break outside a loop or switch.
- Failing to report the no-match case.

Activities

1. Search for a score above 90.
2. Search for a score above 95 and test the fallback.
3. Compare flag-controlled termination with a return from a function.

Accessibility Practice

Report the match or no-match result in text, not by color or sound alone. All prompts, errors, and status messages must have meaningful text equivalents;

keyboard operation is required.

Skills Checklist

- ☐ I can use a boolean flag.
- ☐ I can stop a loop with break.
- ☐ I can handle a search with no result.

Lesson 5.4: Using continue in loops

Reading References

- Python source counterpart: Unit 5 lesson 5.4 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Skip invalid values while continuing to process later entries.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

`continue` skips the rest of the current iteration and begins the next one. It does not terminate the loop. A typed array can still contain invalid values when values come from runtime sources.

Worked Example

```
const entries: string[] = ["82", "", "91", "not a score", "76", "88"];
let total = 0;
for (const entry of entries) {
  const score = Number(entry);
  if (!Number.isFinite(score)) continue;
  total += score;
}
console.log(total);
```

Expected Behavior or Output

The valid entries total 337; the blank entry converts to zero with Number, so a stronger version should reject blank strings explicitly before conversion.

Common Mistakes

- Assuming Number on blank text is an error.
- Using break when later values still need processing.
- Placing the update after continue so it is skipped.

Activities

1. Add an explicit blank-string check.
2. Skip nonnumeric entries and count accepted values.
3. Compare continue and break with a traced example.

Accessibility Practice

Print the number of accepted and skipped entries so outcomes are clear. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

☐ I can use continue to skip an iteration.
☐ I can validate before conversion.
☐ I can distinguish continue from break.

Lesson 5.5: Avoiding infinite loops

Reading References

- Python source counterpart: Unit 5 lesson 5.5 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Diagnose loop termination by tracing the control variable and condition.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

An infinite loop occurs when its continuation condition never becomes false. TypeScript cannot prove that a loop will terminate; test the control variable and every branch that updates it.

Worked Example

```
let count = 1;
while (count <= 3) {
  console.log(`Count ${count}`);
  count += 1;
}
console.log("Loop finished.");
```

Expected Behavior or Output

The output is Count 1, Count 2, Count 3, then Loop finished.

Common Mistakes

- Forgetting to increment or otherwise update the control variable.
- Using continue before the update statement.
- Using a boundary condition that never becomes false.

Activities

1. Trace the value of count on each iteration.
2. Identify a continue path that skips an update.
3. Describe a safe way to stop a runaway terminal process.

Accessibility Practice

Provide a keyboard-only recovery procedure and explicit termination message. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

☐ I can trace loop state.
☐ I can identify non-terminating paths.
☐ I can design a loop with a clear exit.

Overview

Define typed functions, arguments, return values, and collection-processing functions. The TypeScript adaptation preserves the shared Python curriculum's learning objectives, datasets, expected results, and assessment criteria while teaching TypeScript's compile-time type system and JavaScript runtime behavior.

Lesson 6.1: Defining functions

Reading References

- Python source counterpart: Unit 6 lesson 6.1 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Define a typed function that calculates a tip and total.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

A function packages reusable behavior. Parameter types describe accepted inputs and a return type describes the result. A function can return a value without printing it.

Worked Example

```
function calculateTip(bill: number, rate: number): number {
  return bill * rate;
}
console.log(calculateTip(24.5, 0.15).toFixed(2));
```

Expected Behavior or Output

The tip amount is 3.68 when formatted to two decimal places.

Common Mistakes

- Omitting a return on one branch of a typed function.
- Printing instead of returning a result needed by the caller.
- Using overly broad any types.

Activities

1. Write a function that returns a tip amount.
2. Add a function that returns bill plus tip.
3. Test the canonical bill and rate.

Accessibility Practice

Keep the signature on one line and read parameter and return types explicitly. All prompts, errors, and status messages must have meaningful text equivalents;

keyboard operation is required.

Skills Checklist

- ☐ I can declare a function.
- ☐ I can type parameters and return values.
- ☐ I can distinguish returning from printing.

Lesson 6.2: Passing arguments

Reading References

- Python source counterpart: Unit 6 lesson 6.2 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Use positional, optional, and default parameters in typed function calls.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

TypeScript supports positional arguments and default values. It does not have Python-style named keyword arguments; use an options object when callers need named configuration.

Worked Example

```
interface TipOptions { rate?: number; includeTotal?: boolean }
function describeTip(bill: number, options: TipOptions = {}): string {
  const rate = options.rate ?? 0.15;
  const tip = bill * rate;
  return options.includeTotal ? `Tip ${tip.toFixed(2)}, total ${(bill + tip).toFixed(2)}` :
}
console.log(describeTip(24.5, { rate: 0.2, includeTotal: true }));
```

Expected Behavior or Output

The output is `Tip 4.90, total 29.40`.

Common Mistakes

- Assuming TypeScript supports named keyword arguments.
- Using || when zero is a meaningful option value.
- Making an options field required when a default is intended.

Activities

1. Call a function with positional arguments.
2. Add a default tip rate.
3. Use an options object to request the total.

Accessibility Practice

Read option names and values explicitly. Avoid deeply nested option objects in beginner examples. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

☐ I can pass positional arguments.
☐ I can use default and optional parameters.
☐ I can model named options with an interface.

Lesson 6.3: Return values

Reading References

- Python source counterpart: Unit 6 lesson 6.3 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Return computed values and handle empty collections explicitly.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

A return value is data supplied to the caller. A return type can include null to represent an absent result; strictNullChecks requires the caller to handle that possibility.

Worked Example

```
function average(values: number[]): number | null {
  if (values.length === 0) return null;
  return values.reduce((sum, value) => sum + value, 0) / values.length;
}
console.log(average([82, 91, 76, 88]));
console.log(average([]));
```

Expected Behavior or Output

The averages are 84.25 and null.

Common Mistakes

- Returning undefined accidentally on an empty path.
- Ignoring a nullable return value.
- Confusing a side effect with a computed result.

Activities

1. Calculate the canonical score average.
2. Call the function with an empty array.
3. Use a null check before formatting the result.

Accessibility Practice

Explain every possible return value before showing the caller. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

☐ I can return a computed value.
☐ I can express an absent result with null.
☐ I can handle a union return type.

Lesson 6.4: Functions with arrays and objects

Reading References

- Python source counterpart: Unit 6 lesson 6.4 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Pass typed arrays and inventory records to reusable processing functions.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

Arrays and objects are reference values. A function can mutate caller-visible data. Use readonly parameters or copy inputs when mutation is not part of the contract.

Worked Example

```
interface Item { name: string; quantity: number; unitPrice: number }
function inventoryValue(items: readonly Item[]): number {
  return items.reduce((sum, item) => sum + item.quantity * item.unitPrice, 0);
}
const items: Item[] = [
  { name: "Notebook", quantity: 3, unitPrice: 2.5 },
  { name: "Pencil", quantity: 10, unitPrice: 0.5 },
  { name: "Folder", quantity: 4, unitPrice: 1.25 }
];
console.log(inventoryValue(items).toFixed(2));
```

Expected Behavior or Output

The inventory value is 17.50.

Common Mistakes

- Mutating an input collection unexpectedly.
- Using an object shape inconsistent with the interface.
- Forgetting that readonly is a compile-time constraint, not deep freezing.

Activities

1. Write a function that totals inventory value.
2. Write a function that returns scores above 80.
3. Explain whether each function mutates its input.

Accessibility Practice

Use named fields and one output line per record. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

- ☐ I can type an object interface.
- ☐ I can accept readonly arrays.
- ☐ I can write a collection-processing function.

Lesson 6.5: Arbitrary numbers of arguments

Reading References

- Python source counterpart: Unit 6 lesson 6.5 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Use typed rest parameters and options objects to handle variable inputs.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

A rest parameter gathers remaining positional arguments into an array. TypeScript requires the rest parameter's element type. An options object is a clear alternative to Python-style keyword arguments.

Worked Example

```
function sum(...values: number[]): number {
  return values.reduce((total, value) => total + value, 0);
}
console.log(sum(10, 20, 30));
console.log(sum(82, 91, 76, 88));
```

Expected Behavior or Output

The results are 60 and 337.

Common Mistakes

- Treating the rest parameter as a single number.
- Placing parameters after a rest parameter.
- Using any[] instead of a meaningful element type.

Activities

1. Call sum with three values.
2. Call sum with the canonical scores.
3. Compare a rest parameter with an options object.

Accessibility Practice

Describe the collected values as an ordered list. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

☐ I can declare a typed rest parameter.
☐ I can pass a variable number of values.
☐ I can explain the difference between rest parameters and options objects.

16.9 Unit 7: Classes and Object-Oriented Programming

Overview

Model state and behavior using classes, inheritance, and composition. The TypeScript adaptation preserves the shared Python curriculum's learning objectives, datasets, expected results, and assessment criteria while teaching TypeScript's compile-time type system and JavaScript runtime behavior.

Lesson 7.1: Why learn about classes?

Reading References

- Python source counterpart: Unit 7 lesson 7.1 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Explain classes, instances, encapsulation, and object state with an inventory example.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

A class describes the state and behavior of objects. TypeScript classes add static type checking, access modifiers, and optional private fields; types are erased from emitted JavaScript.

Worked Example

```
class InventoryItem {
  constructor(public name: string, public quantity: number, public unitPrice: number) {}
  value(): number { return this.quantity * this.unitPrice; }
}
const notebook = new InventoryItem("Notebook", 3, 2.5);
console.log(`${notebook.name}: ${notebook.value().toFixed(2)}`);
```

Expected Behavior or Output

The output is `Notebook: 7.50`.

Common Mistakes

- Treating a class as only a data container without behavior.
- Confusing a class with an instance.
- Assuming TypeScript access modifiers create runtime security.

Activities

1. Identify the fields and behavior of an inventory item.
2. Create two instances with different values.
3. Explain what the class and each instance represent.

Accessibility Practice

Use meaningful instance names and print a labeled summary. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

☐ I can explain class and instance.
☐ I can identify state and behavior.
☐ I can create a simple class.

Lesson 7.2: Creating and using a class

Reading References

- Python source counterpart: Unit 7 lesson 7.2 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Create a Student class and instantiate the canonical learner profile.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

A constructor initializes an instance. Parameter properties can declare and initialize fields in one place. Explicit types make the class contract readable.

Worked Example

```
class Student {
  constructor(public name: string, public age: number, public favoriteSubject: string) {}
  describe(): string { return `${this.name}, age ${this.age}, studies ${this.favoriteSubject
}
const learner = new Student("Avery", 16, "Computer Science");
console.log(learner.describe());
```

Expected Behavior or Output

The output is `Avery, age 16, studies Computer Science`.

Common Mistakes

- Forgetting new when constructing an instance.
- Using a parameter type that conflicts with the property type.
- Writing a method that refers to a variable instead of this.

Activities

1. Create Avery's Student instance.
2. Create a second student instance.
3. Call describe on both instances.

Accessibility Practice

Read constructor parameters in order and label each property. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

- ☐ I can define a constructor.
- ☐ I can instantiate a class.
- ☐ I can call an instance method.

Lesson 7.3: Working with attributes and methods

Reading References

- Python source counterpart: Unit 7 lesson 7.3 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Add validation and update behavior to a class.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

Methods define operations over an instance's state. Encapsulation keeps validation close to the state it protects. Private fields and access modifiers guide compile-time access.

Worked Example

```
class StockItem {
  #quantity: number;
  constructor(public name: string, quantity: number) { this.#quantity = quantity; }
  get quantity(): number { return this.#quantity; }
  setQuantity(value: number): void {
    if (!Number.isInteger(value) || value < 0) throw new RangeError("Quantity must be a nonn
    this.#quantity = value;
  }
}
```

```
const pencil = new StockItem("Pencil", 10);
pencil.setQuantity(12);
console.log(pencil.quantity);
```

Expected Behavior or Output

The output is 12. A negative or fractional quantity throws RangeError.

Common Mistakes

- Changing state without validation.
- Assuming a private field is accessible outside its class.
- Allowing negative quantities.

Activities

1. Update Pencil quantity to 12.
2. Try a negative quantity and inspect the error.
3. Explain the difference between a property and a method.

Accessibility Practice

Use explicit validation messages and one state change per operation. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

☐ I can define methods and fields.
☐ I can protect state with private fields.
☐ I can validate state updates.

Lesson 7.4: Inheritance

Reading References

- Python source counterpart: Unit 7 lesson 7.4 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Create a specialized item class that extends a base class.

- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

Inheritance models an is-a relationship. A subclass uses extends and calls super before accessing this. Use inheritance only when the subtype genuinely satisfies the base contract.

Worked Example

```
class Item {
  constructor(public name: string, public quantity: number) {}
}
class PerishableItem extends Item {
  constructor(name: string, quantity: number, public expiresOn: string) {
    super(name, quantity);
  }
  describe(): string { return `${this.name}: ${this.quantity} in stock, expires ${this.expi
}
console.log(new PerishableItem("Milk", 2, "2026-11-10").describe());
```

Expected Behavior or Output

The output is `Milk: 2 in stock, expires 2026-11-10`.

Common Mistakes

- Forgetting to call super before using this.
- Extending a class only to reuse unrelated code.
- Changing a subclass method so it violates the base contract.

Activities

1. Create a PerishableItem.
2. Override or add a method with a clear contract.
3. Compare inheritance with composition.

Accessibility Practice

Use a descriptive sentence for the specialized instance. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

☐ I can extend a base class.

☐ I can call super in a constructor.
☐ I can explain when inheritance is appropriate.

Lesson 7.5: Composition: instances as attributes

Reading References

- Python source counterpart: Unit 7 lesson 7.5 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Build a record from related objects using composition.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

Composition models a has-a relationship. A class stores another object as a property rather than inheriting its behavior. This often keeps responsibilities smaller.

Worked Example

```
class Student {
  constructor(public name: string, public age: number, public favoriteSubject: string) {}
}
class Course {
  constructor(public title: string) {}
}
class StudentRecord {
  constructor(public student: Student, public course: Course) {}
  describe(): string { return `${this.student.name} studies ${this.course.title}.`; }
}
const record = new StudentRecord(new Student("Avery", 16, "Computer Science"), new Course("
console.log(record.describe());
```

Expected Behavior or Output

The output is `Avery studies Computer Science.`

Common Mistakes

- Using inheritance for a has-a relationship.
- Creating tightly coupled classes with unclear responsibilities.
- Duplicating data already stored in a related object.

Activities

1. Create a StudentRecord containing a Student and Course.
2. Add a method that describes the relationship.
3. Draw a text outline of the object relationships.

Accessibility Practice

Explain object relationships in words before reading nested property access. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

☐ I can model a has-a relationship.
☐ I can compose objects from instances.
☐ I can distinguish composition from inheritance.

16.10 Unit 8: Files and Exceptions

Overview

Read and write files, handle errors, and persist validated JSON. The TypeScript adaptation preserves the shared Python curriculum's learning objectives, datasets, expected results, and assessment criteria while teaching TypeScript's compile-time type system and JavaScript runtime behavior.

Overview

Read and write files, handle errors, and persist validated JSON. The TypeScript adaptation preserves the shared Python curriculum's learning objectives, datasets, expected results, and assessment criteria while teaching TypeScript's compile-time type system and JavaScript runtime behavior.

Lesson 8.1: Reading from a file

Reading References

- Python source counterpart: Unit 8 lesson 8.1 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Read a line-oriented notes file with the promise-based file system API.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

The node:fs/promises API returns Promises. `readFile` with UTF-8 encoding returns a string. Await file operations and handle missing-file errors explicitly.

Worked Example

```
import { readFile } from "node:fs/promises";
async function main(): Promise<void> {
  const content = await readFile("notes.txt", "utf8");
  console.log("File contents:");
  console.log(content);
}
void main();
```

Expected Behavior or Output

With the sample file containing two notes, both lines appear after `File contents:`.

Common Mistakes

- Reading from the wrong working directory.
- Forgetting await and logging a Promise.
- Assuming a file exists without handling ENOENT.

Activities

1. Create notes.txt with two lines.
2. Read and print the file contents.
3. Run from a different working directory and diagnose the path.

Accessibility Practice

Print a label before file contents and explain the working directory. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

- ☐ I can read a UTF-8 text file.
- ☐ I can await an asynchronous operation.
- ☐ I can identify the current working directory.

Lesson 8.2: Writing to a file

Reading References

- Python source counterpart: Unit 8 lesson 8.2 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Create and append to the same notes file using promise-based file operations.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

`writeFile` creates or replaces content; `appendFile` adds content. Await each operation so writes happen in the intended order.

Worked Example

```
import { writeFile, appendFile, readFile } from "node:fs/promises";
async function main(): Promise<void> {
  await writeFile("notes.txt", "Review arrays\n", "utf8");
  await appendFile("notes.txt", "Practice typed functions\n", "utf8");
  console.log(await readFile("notes.txt", "utf8"));
}
void main();
```

Expected Behavior or Output

The file contains `Review arrays` followed by `Practice typed functions`, each on its own line.

Common Mistakes

- Using writeFile when existing content must be preserved.
- Not awaiting a write before reading the file.
- Writing to an unexpected path.

Activities

1. Create a new notes file.
2. Append a second line.
3. Read the file and verify both lines and their order.

Accessibility Practice

Announce whether an operation creates, replaces, appends, or reads data. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

- ☐ I can write a UTF-8 file.
- ☐ I can append without replacing existing text.
- ☐ I can sequence asynchronous file operations.

Lesson 8.3: Exceptions and error handling

Reading References

- Python source counterpart: Unit 8 lesson 8.3 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Handle expected file and validation errors with try/catch.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

An exception interrupts normal control flow. Catch expected failures and inspect unknown error values safely; do not assume every thrown value is an Error instance.

Worked Example

```
function validateAge(age: number): void {
  if (!Number.isInteger(age) || age < 0 || age > 120) {
    throw new RangeError("Age must be a whole number from 0 to 120.");
  }
}
try {
  validateAge(16);
  console.log("Age accepted.");
  validateAge(-1);
} catch (error: unknown) {
  if (error instanceof Error) console.log(error.message);
}
```

Expected Behavior or Output

The program prints `Age accepted.` followed by the validation message.

Common Mistakes

- Catching every error and silently ignoring it.
- Typing catch values as any.
- Using exceptions for ordinary expected branching.

Activities

1. Test a valid age.
2. Test -1 and 121.
3. Handle a missing file separately from malformed file contents.

Accessibility Practice

Use explicit error messages that identify the invalid field and recovery action. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

☐ I can throw a validation error.
☐ I can catch unknown errors safely.
☐ I can distinguish expected failures.

Lesson 8.4: The else and finally patterns

Reading References

- Python source counterpart: Unit 8 lesson 8.4 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Separate successful operations from cleanup that must always occur.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

JavaScript has no try-else clause. Put success-only work after a successful try operation and use finally for cleanup. A finally block runs whether the try succeeds or fails.

Worked Example

```
async function loadMessage(): Promise<void> {
  let loaded = false;
  try {
    const message = "Notes loaded.";
    console.log(message);
    loaded = true;
  } catch (error: unknown) {
    console.log("Unable to load notes.");
  } finally {
    console.log("Load attempt finished.");
  }
  if (loaded) console.log("Ready to continue.");
}
void loadMessage();
```

Expected Behavior or Output

On success, the program prints Notes loaded, Load attempt finished, and Ready to continue. On failure, the final status still prints.

Common Mistakes

- Putting success-only code in finally.
- Assuming TypeScript has Python's try-else syntax.
- Returning from finally and accidentally overriding an earlier result.

Activities

1. Trace success and failure paths.
2. Move the success message outside the try block.
3. Explain which cleanup must always run.

Accessibility Practice

Label success, failure, and finalization messages distinctly. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

☐ I can model success and failure paths.
☐ I can use finally for cleanup.
☐ I can explain the absence of a try-else clause.

Lesson 8.5: Storing structured data with JSON

Reading References

- Python source counterpart: Unit 8 lesson 8.5 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Serialize typed records and validate parsed JSON before use.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

JSON.parse returns untrusted runtime data. A TypeScript assertion does not validate its shape. Parse into unknown and use a type guard before treating it as a record.

Worked Example

```
interface Task { id: number; title: string; done: boolean }
function isTask(value: unknown): value is Task {
  return typeof value === "object" && value !== null &&
    "id" in value && typeof value.id === "number" &&
    "title" in value && typeof value.title === "string" &&
    "done" in value && typeof value.done === "boolean";
}
const encoded = JSON.stringify({ id: 1, title: "Review arrays", done: false });
const parsed: unknown = JSON.parse(encoded);
if (isTask(parsed)) console.log(`${parsed.id}: ${parsed.title}`);
```

Expected Behavior or Output

The program prints `1: Review arrays` only after validating the parsed object.

Common Mistakes

- Using `as Task` as if it checked runtime data.
- Assuming JSON.parse cannot fail.
- Failing to validate each required field.

Activities

1. Serialize a task object.
2. Parse it as unknown and validate it.
3. Test missing fields and wrong field types.

Accessibility Practice

Print field names and validation errors as text. Never rely on color-only success indicators. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

- ☐ I can serialize an object to JSON.
- ☐ I can parse JSON as unknown.
- ☐ I can validate external data with a type guard.

16.11 Unit 9: Testing and Data Analysis

Overview

Test functions and classes, generate random values, analyze text, and parse CSV. The TypeScript adaptation preserves the shared Python curriculum's learning objectives, datasets, expected results, and assessment criteria while teaching TypeScript's compile-time type system and JavaScript runtime behavior.

Lesson 9.1: Testing a function

Reading References

- Python source counterpart: Unit 9 lesson 9.1 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Write built-in Node.js tests for arithmetic and grade classification functions.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

Unit tests verify a function's contract with known inputs and expected outputs. Node's built-in node:test and node:assert/strict avoid a third-party test framework for introductory work.

Worked Example

```
import test from "node:test";
import assert from "node:assert/strict";
function add(a: number, b: number): number { return a + b; }
function getGrade(score: number): string { return score >= 90 ? "A" : score >= 80 ? "B" : sc
test("add returns the sum", () => assert.equal(add(5, 3), 8));
test("82 receives grade B", () => assert.equal(getGrade(82), "B"));
```

Expected Behavior or Output

The test runner reports two passing tests and no failures.

Common Mistakes

- Testing only one ordinary value.
- Using an incorrect expected value copied from the implementation.
- Failing to run tests after changing a function.

Activities

1. Test add with positive, negative, and zero values.
2. Test every grade threshold and its lower neighbor.
3. Run tests with node --test after compiling the TypeScript test file.

Accessibility Practice

Use descriptive test names that read as complete sentences. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

☐ I can write a unit test.
☐ I can assert an expected result.
☐ I can test normal and boundary inputs.

Lesson 9.2: Testing a class

Reading References

- Python source counterpart: Unit 9 lesson 9.2 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Test class construction, state changes, and invalid operations.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

A class test checks observable behavior: constructor values, method results, state updates, and rejected invalid states. Keep tests independent and deterministic.

Worked Example

```
import test from "node:test";
import assert from "node:assert/strict";
class StockItem {
  constructor(public name: string, public quantity: number) {}
  setQuantity(value: number): void { if (value < 0) throw new RangeError("Invalid quantity")
}
test("quantity can be updated", () => {
  const item = new StockItem("Pencil", 10);
  item.setQuantity(12);
  assert.equal(item.quantity, 12);
});
```

Expected Behavior or Output

The test passes when the quantity changes from 10 to 12.

Common Mistakes

- Sharing mutable state between tests.
- Testing private implementation details rather than behavior.
- Failing to test invalid state changes.

Activities

1. Test a valid quantity update.
2. Assert that a negative quantity throws RangeError.
3. Create a fresh instance in each test.

Accessibility Practice

Keep test names and failure output specific to the state being checked. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

☐ I can test class behavior.
☐ I can test state changes.
☐ I can assert that invalid operations throw.

Lesson 9.3: Introduction to random numbers

Reading References

- Python source counterpart: Unit 9 lesson 9.3 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Generate bounded integers, decimals, and selections with Math.random.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

Math.random returns a pseudorandom number in the range zero inclusive to one exclusive. Scale and floor it to produce a bounded integer; do not use it for security tokens.

Worked Example

```
const integer = Math.floor(Math.random() * 10) + 1;
const decimal = Math.random();
const choices = ["red", "blue", "green"];
const choice = choices[Math.floor(Math.random() * choices.length)];
console.log(integer >= 1 && integer <= 10);
console.log(decimal >= 0 && decimal < 1);
console.log(choices.includes(choice));
```

Expected Behavior or Output

All three checks print true; sampled values can differ on each run.

Common Mistakes

- Using Math.round and creating uneven endpoint probabilities.
- Forgetting that the upper bound is exclusive.
- Using Math.random for passwords or cryptographic secrets.

Activities

1. Generate an integer from 1 to 10.
2. Generate a decimal in [0,1).
3. Choose one item from a typed array.

Accessibility Practice

Report range checks as text; explain that the actual sample changes between runs. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

- ☐ I can describe Math.random's range.
- ☐ I can generate a bounded integer.
- ☐ I can select a random array element.

Lesson 9.4: Analyzing text data

Reading References

- Python source counterpart: Unit 9 lesson 9.4 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Count words in the canonical text using normalization and a typed Map.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

Text analysis requires explicit normalization and tokenization rules. A Map stores each token and its frequency. The regular expression below splits on non-letter characters.

Worked Example

```
const text = "A student studies programming. Programming develops problem-solving skills.";
const words = text.toLowerCase().match(/[a-z]+/g) ?? [];
const counts = new Map<string, number>();
for (const word of words) counts.set(word, (counts.get(word) ?? 0) + 1);
console.log(`Tokens: ${words.length}`);
console.log(`Programming: ${counts.get("programming")}`);
```

Expected Behavior or Output

The sample produces 10 alphabetic tokens under this tokenization rule; programming appears twice.

Common Mistakes

- Assuming punctuation and hyphens are words.
- Using a plain object without considering inherited property names.
- Failing to define how case and punctuation are normalized.

Activities

1. Count all tokens in the canonical text.
2. Report the frequency of programming.
3. Compare token counts with a rule that treats problem-solving as one token.

Accessibility Practice

State the tokenization rule before reporting counts. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

☐ I can normalize text.
☐ I can count tokens with a Map.
☐ I can explain how tokenization affects results.

Lesson 9.5: Working with CSV data

Reading References

- Python source counterpart: Unit 9 lesson 9.5 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Read the shared CSV columns using a standards-aware parser.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

CSV supports quoted fields, commas inside quotes, and escaped quotation marks. Use a maintained parser rather than splitting lines on commas. Parsed records still need validation.

Worked Example

```
import { parse } from "csv-parse/sync";
const csv = "student_id,first_name,last_name,score,active\n1,Avery,Stone,91,true\n2,Jordan,
const records = parse(csv, { columns: true, skip_empty_lines: true }) as Array<Record<string
const average = records.reduce((sum, row) => sum + Number(row.score), 0) / records.length;
console.log(`Records: ${records.length}`);
console.log(`Average: ${average.toFixed(2)}`);
```

Expected Behavior or Output

The two sample records have scores 91 and 82; the average is 86.50.

Common Mistakes

- Splitting CSV lines and fields manually.
- Assuming every parsed field is already a number or boolean.
- Ignoring malformed rows and missing columns.

Activities

1. Parse the shared CSV columns.
2. Calculate the score average for the sample records.
3. Test a quoted name containing a comma.

Accessibility Practice

Print record count and summary labels rather than a visual-only chart. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

☐ I can parse CSV with a dedicated parser.
☐ I can validate parsed field values.
☐ I can calculate a summary from CSV records.

16.12 Unit 10: Project-Based Learning — Text-Based Applications

Overview

Design, build, and refine equivalent command-line projects. The TypeScript adaptation preserves the shared Python curriculum's learning objectives, datasets, expected results, and assessment criteria while teaching TypeScript's compile-time type system and JavaScript runtime behavior.

Lesson 10.1: Designing a text-based project

Reading References

- Python source counterpart: Unit 10 lesson 10.1 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Plan requirements, data structures, commands, and acceptance tests before implementation.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

Top-down design decomposes a project into smaller responsibilities. For an accessible terminal project, define prompts, commands, textual feedback, invalid-input behavior, and a quit path before coding.

Worked Example

```
const requirements = ["list rooms", "move by direction", "take item", "view inventory", "qui
for (const requirement of requirements) console.log(`Requirement: ${requirement}`);
```

Expected Behavior or Output

Five requirements are printed in a stable order.

Common Mistakes

- Starting implementation before writing acceptance criteria.
- Designing a visual-only interaction for a text-based project.
- Omitting error and quit behavior.

Activities

1. Write the project requirements in plain language.
2. Identify input, state, and output for each command.
3. Write at least five acceptance tests before implementation.

Accessibility Practice

Use a numbered text specification and test keyboard-only interaction. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

☐ I can decompose a project.
☐ I can define commands and data.
☐ I can write acceptance tests before coding.

Lesson 10.2: Text adventure part 1: rooms and movement

Reading References

- Python source counterpart: Unit 10 lesson 10.2 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Model the five canonical rooms with typed exits and movement.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

A room has an identifier, description, and exits. A movement operation changes the current room only when the requested exit exists. A Record type can model a fixed set of room identifiers.

Worked Example

```
type RoomId = "start_room" | "tunnel" | "clearing" | "locked_door_room" | "treasure_room";
const exits: Record<RoomId, Partial<Record<string, RoomId>>> = {
  start_room: { north: "tunnel" },
```

```
  tunnel: { south: "start_room", east: "clearing" },
  clearing: { west: "tunnel", north: "locked_door_room" },
  locked_door_room: { south: "clearing" },
  treasure_room: {}
};
function move(room: RoomId, direction: string): RoomId | null {
  return exits[room][direction.toLowerCase()] ?? null;
}
console.log(move("start_room", "north"));
console.log(move("start_room", "west"));
```

Expected Behavior or Output

Moving north from start_room returns tunnel; moving west returns null. The north exit from locked_door_room is conditional on using shiny key.

Common Mistakes

- Allowing movement through an exit that is not defined.
- Changing the room before validating the direction.
- Using arbitrary strings where a RoomId union is safer.

Activities

1. Create all five canonical rooms and unconditional exits.
2. Move north from start_room and east from tunnel.
3. Attempt an invalid direction and report the result.
4. Add the conditional locked_door_room north exit only after the key is used.

Accessibility Practice

Announce current room and available exits as labeled text. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

- ☐ I can model rooms and exits.
- ☐ I can use a union type for room identifiers.
- ☐ I can validate movement.

Lesson 10.3: Text adventure part 2: items, inventory, and puzzle

Reading References

- Python source counterpart: Unit 10 lesson 10.3 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Implement pickup of shiny key from start_room and the conditional locked-door puzzle.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

An item moves from one room to inventory as one state transition. The shiny key moves from start_room to inventory as one state transition. The player must use it in locked_door_room to unlock the north exit to treasure_room.

Worked Example

```
type RoomId = "start_room" | "tunnel" | "clearing" | "locked_door_room" | "treasure_room";
type Direction = "north" | "south" | "east" | "west";
const exits: Record<RoomId, Partial<Record<Direction, RoomId>>> = {
  start_room: { north: "tunnel" },
  tunnel: { south: "start_room", east: "clearing" },
  clearing: { west: "tunnel", north: "locked_door_room" },
  locked_door_room: { south: "clearing" },
  treasure_room: {}
};
const roomItems: Record<RoomId, string[]> = {
  start_room: ["shiny key"], tunnel: [], clearing: [], locked_door_room: [], treasure_room:
};
const inventory: string[] = [];
let doorLocked = true;
function takeItem(room: RoomId, item: string): boolean {
  const items = roomItems[room];
  const index = items.findIndex(value => value.toLowerCase() === item.toLowerCase());
  if (index < 0) return false;
  inventory.push(items.splice(index, 1)[0]);
  return true;
}
```

```javascript
function useKey(room: RoomId): string {
  if (room !== "locked_door_room") return "You must be at the locked door to use the key.";
  if (!inventory.includes("shiny key")) return "The door is locked.";
  doorLocked = false;
  exits.locked_door_room.north = "treasure_room";
  return "The door unlocks; north now leads to treasure_room.";
}
function runCommand(room: RoomId, command: string): boolean | string {
  if (command.startsWith("take ")) return takeItem(room, command.slice(5));
  if (command === "use shiny key") return useKey(room);
  return "I don't understand that command.";
}
console.log(runCommand("start_room", "take shiny key"));
console.log(runCommand("start_room", "take shiny key"));
console.log(inventory.join(", "));
console.log(runCommand("clearing", "use shiny key"));
console.log(runCommand("locked_door_room", "use shiny key"));
console.log(doorLocked);
console.log(exits.locked_door_room.north);
```

Expected Behavior or Output

The command dispatcher accepts the literal `take shiny key` and `use shiny key` commands. The second pickup is rejected without duplicating the inventory item. Key use is rejected outside `locked_door_room`; using it there unlocks the north exit.

```
true
false
shiny key
You must be at the locked door to use the key.
The door unlocks; north now leads to treasure_room.
false
treasure_room
```

Common Mistakes

- Allowing the same item to be collected repeatedly.
- Updating inventory without removing the room item.
- Checking for the wrong key name or case.

Activities

1. Take shiny key from start_room.
2. Attempt to take it twice and verify duplicate pickup is rejected.
3. Attempt to use the key outside locked_door_room.
4. Use the key in locked_door_room and verify the north exit is unlocked.

Accessibility Practice

Confirm each pickup by item name and state whether the puzzle is locked or open. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

- ☐ I can transfer an item into inventory.
- ☐ I can prevent duplicate pickup.
- ☐ I can implement the shiny-key puzzle: move the key from `start_room` to inventory, require its use in `locked_door_room`, and unlock the north exit to `treasure_room`.

Lesson 10.4: Developing a command-line to-do tool

Reading References

- Python source counterpart: Unit 10 lesson 10.4 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Implement add, view, complete, delete, and quit operations with typed tasks.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

A task has an id, title, description, done state, and creation timestamp. User-facing task numbers are one-based; array indexes are zero-based. Persist after every state-changing operation.

Worked Example

```
interface Todo { id: number; title: string; description: string; done: boolean; created_at:
const tasks: Todo[] = [];
function addTask(title: string, description: string): Todo {
  const task: Todo = { id: tasks.length + 1, title, description, done: false, created_at: ne
  tasks.push(task);
  return task;
```

```
}
console.log(addTask("Review arrays", "Practice typed collection methods").done);
```

Expected Behavior or Output

A new task is added with done false and an ISO timestamp.

Common Mistakes

- Using an array index as a permanent identifier.
- Failing to save after a state change.
- Accepting a blank task title.

Activities

1. Add a task and view it.
2. Mark a task complete and delete another.
3. Persist and reload the task array as JSON.

Accessibility Practice

Use one task per output block with field labels and a clear confirmation. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

☐ I can model a typed task.
☐ I can implement task creation.
☐ I can identify persistence points.

Lesson 10.5: Refinement and accessible text user experience

Reading References

- Python source counterpart: Unit 10 lesson 10.5 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Improve validation, prompts, error messages, and recovery paths.

- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

A usable terminal interface has explicit prompts, predictable output, help text, and recoverable errors. A robust program validates untrusted values before changing state.

Worked Example

```
function normalizeCommand(input: string): string {
  return input.trim().toLowerCase();
}
function validateTitle(input: string): string | null {
  const title = input.trim();
  return title.length === 0 ? "Title cannot be blank." : null;
}
console.log(normalizeCommand("  VIEW "));
console.log(validateTitle("   "));
```

Expected Behavior or Output

The output is view and Title cannot be blank.

Common Mistakes

- Providing vague messages such as invalid input without a recovery hint.
- Using color as the only status indicator.
- Failing to test blank input, unknown commands, and empty collections.

Activities

1. Add a help command.
2. Test blank, malformed, and unknown commands.
3. Review every message in sequential screen-reader output.

Accessibility Practice

Provide text equivalents for every status and avoid cursor-position-dependent instructions. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

☐ I can normalize command input.
☐ I can validate required fields.

☐ I can review a text interface for accessibility.

16.13 Unit 11: Capstone — Accessible Personal Data Manager

Overview

Integrate typed records, terminal interaction, persistence, CRUD, testing, and accessibility. The TypeScript adaptation preserves the shared Python curriculum's learning objectives, datasets, expected results, and assessment criteria while teaching TypeScript's compile-time type system and JavaScript runtime behavior.

Lesson 11.1: Capstone interface and requirements

Reading References

- Python source counterpart: Unit 11 lesson 11.1 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Plan an accessible terminal interface for the personal data manager.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

The Python source uses NiceGUI, but the TypeScript track uses a terminal interface to preserve the same user-facing requirements without requiring browser UI. Commands include list, add, search, update, delete, help, and quit.

Worked Example

```
const commands = ["list", "add", "search", "update", "delete", "help", "quit"];
console.log(commands.join(", "));
```

Expected Behavior or Output

The command list contains all seven required operations.

Common Mistakes

- Copying a GUI-specific implementation when the language track requires an accessible alternative.
- Omitting a help or quit command.
- Designing commands without defining expected output.

Activities

1. Choose a terminal command syntax.
2. Write a help response for every command.
3. Define keyboard-only acceptance tests.

Accessibility Practice

Specify prompt order and output as linear text. Do not require mouse use. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

☐ I can explain the language-specific interface adaptation.
☐ I can define the required commands.
☐ I can create accessibility acceptance tests.

Lesson 11.2: Capstone data model and JSON storage

Reading References

- Python source counterpart: Unit 11 lesson 11.2 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Define the shared record schema and save/load records safely.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

Each record has id, title, description, category, created_at, and updated_at. TypeScript interfaces describe the expected shape, but JSON input must be

validated at runtime.

Worked Example

```
interface DataRecord { id: number; title: string; description: string; category: string; cre
const record: DataRecord = { id: 1, title: "Course notes", description: "Review arrays", cat
const serialized = JSON.stringify([record], null, 2);
console.log(serialized.includes("Course notes"));
```

Expected Behavior or Output

The check prints true and the JSON contains all six fields.

Common Mistakes

- Using a type assertion instead of validating loaded JSON.
- Reusing record identifiers.
- Overwriting valid data after a parse failure without reporting it.

Activities

1. Define the six-field interface.
2. Serialize one record and inspect the JSON.
3. Load and validate records before using them.

Accessibility Practice

Read the schema field by field and keep timestamps in a consistent ISO format. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

☐ I can define the capstone record interface.
☐ I can serialize structured data.
☐ I can explain runtime validation of stored JSON.

Lesson 11.3: Capstone list and display operations

Reading References

- Python source counterpart: Unit 11 lesson 11.3 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Display records in a stable, labeled, sequential format.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

Listing is a read operation and should not mutate data. Stable identifiers are distinct from temporary one-based display positions.

Worked Example

```
const records = [{ id: 7, title: "Course notes", category: "Study", done: false }];
for (const [index, record] of records.entries()) {
  console.log(`Item ${index + 1}`);
  console.log(`ID: ${record.id}`);
  console.log(`Title: ${record.title}`);
  console.log(`Category: ${record.category}`);
  console.log(`Done: ${record.done}`);
}
```

Expected Behavior or Output

The record is displayed as Item 1 with ID 7 and its labeled fields.

Common Mistakes

- Using the displayed list number as the permanent ID.
- Omitting labels and making output ambiguous.
- Failing to handle an empty collection.

Activities

1. Display one record.
2. Display multiple records.
3. Display an empty list with a clear message.

Accessibility Practice

Use stable labels and one field per line for sequential reading. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

☐ I can display records with labels.

☐ I can distinguish display position from record ID.
☐ I can handle an empty collection.

Lesson 11.4: Capstone add, search, and update operations

Reading References

- Python source counterpart: Unit 11 lesson 11.4 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Implement creation, case-insensitive search, and safe record updates.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

Create, search, and update are separate operations. Update only permitted fields and refresh updated_at. Search should not change stored records.

Worked Example

```
interface Entry { id: number; title: string; description: string; updated_at: string }
function search(entries: readonly Entry[], query: string): Entry[] {
  const term = query.trim().toLowerCase();
  return entries.filter(entry => `${entry.title} ${entry.description}`.toLowerCase().include
}
const entries: Entry[] = [{ id: 1, title: "Course notes", description: "Review arrays", upda
console.log(search(entries, "ARRAYS").length);
```

Expected Behavior or Output

The search returns one matching record.

Common Mistakes

- Searching case-sensitively without documenting that choice.
- Allowing edits to immutable identifiers.
- Forgetting to refresh updated_at after an edit.

Activities

1. Add a record with all required fields.
2. Search by title and description.
3. Update a title and verify the timestamp changes.

Accessibility Practice

Confirm each successful action by record ID and field name. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Skills Checklist

☐ I can add a valid record.
☐ I can search without mutating records.
☐ I can update permitted fields and timestamps.

Lesson 11.5: Capstone delete, completion, and final verification

Reading References

- Python source counterpart: Unit 11 lesson 11.5 in the canonical Python curriculum.
- TypeScript Handbook: https://www.typescriptlang.org/docs/handbook/intro.html
- Node.js documentation where terminal, file, or runtime APIs are used.

Learning Objectives

- Implement delete and completion behavior and verify persistence and accessibility.
- Explain the TypeScript-specific language or runtime behavior used in the lesson.
- Predict the example's behavior and test a boundary or invalid case.

Concept Explanation

Deletion removes a selected record by stable ID. Completion updates the record state. The capstone should have tests for CRUD behavior, JSON persistence, invalid input, and empty data.

Worked Example

```
interface TaskRecord { id: number; title: string; done: boolean }
function markDone(records: TaskRecord[], id: number): boolean {
  const record = records.find(item => item.id === id);
  if (!record) return false;
  record.done = true;
  return true;
}
const records: TaskRecord[] = [{ id: 4, title: "Test persistence", done: false }];
console.log(markDone(records, 4));
console.log(records[0].done);
```

Expected Behavior or Output

The function returns true and the record's done field becomes true.

Common Mistakes

- Deleting by a stale array index.
- Failing to save after a mutation.
- Testing only the successful path.

Activities

1. Delete a record by ID and verify it remains absent after reload.
2. Mark a record complete and test missing IDs.
3. Run at least five tests covering CRUD, persistence, and validation.

Accessibility Practice

Verify keyboard-only use, help text, confirmations, errors, and sequential output with the intended access technology. All prompts, errors, and status messages must have meaningful text equivalents; keyboard operation is required.

Runnable Reference Implementation

The repository includes a tested TypeScript domain and JSON-storage reference core in capstone/README.md. Run `tsc -p TYPESCRIPT/capstone/tsconfig.json`, then `node --test TYPESCRIPT/capstone/records.test.js` from the repository root. The reference core does not replace the requirement to build and evaluate an accessible user interface.

Skills Checklist

- ☐ I can delete by stable identifier.
- ☐ I can update completion state.
- ☐ I can plan end-to-end and accessibility tests.

Part XVII

Standalone Topics/Languages

Chapter 17

Standalone Topics and Focused Language Modules

This section contains focused introductions to important areas of computing that complement the full programming-language curricula. These topics are useful in their own right and are frequently used alongside one or more general-purpose programming languages.

The standalone modules are intentionally narrower than the full language courses. Each introduces a specific technology or discipline, explains its core concepts and common workflows, and provides practice activities and further reading. They are not arranged as a sequence of increasingly difficult languages, and they do not replace the shared computing foundations or terminal prerequisite.

Modules in this section

Git: Version Control

Git is the distributed version control system used by nearly all modern software development. It records the history of a project as a series of snapshots and lets multiple people work concurrently.

This module introduces repositories, commits and commit messages, the staging area, reading `git status`, `git log`, and `git diff` output, branching and merging, conflict resolution, remotes, and collaboration through pull requests, including review from the command line. It builds the commit-graph mental model through textual history rather than diagrams.

HTML and CSS: Structure and Presentation for the Web

HTML and CSS are the foundational technologies for structuring and presenting web content.

- **HTML** describes the meaning and structure of a document: headings, paragraphs, links, lists, forms, tables, and other content.
- **CSS** controls presentation and layout, including typography, spacing, responsive design, and visual states.
- Together, they form the foundation for browser-based user interfaces and are used with JavaScript or TypeScript in interactive web applications.

This module emphasizes semantic HTML, separation of structure from presentation, and accessibility as part of document design. It is a useful starting point for web development regardless of which scripting language or framework is introduced later.

Markdown: Technical Documentation

Markdown is a lightweight plain-text markup language used for documentation, README files, technical notes, and web content. It is the default documentation format of GitHub, package registries, static site generators, and developer communication tools.

This module introduces headings, lists, links, images, code blocks, tables, and blockquotes; reference-style links; dialect differences between CommonMark and GitHub Flavored Markdown; and a screen-reader authoring workflow that verifies structure by heading navigation and linting rather than visual preview.

R: Statistical Computing

R is a language and environment for statistical computing, data analysis, and graphics. It is widely used in research, public health, economics, social science, government, and business analytics.

This module introduces R's core data types and structures, vectorized operations, functions, data frames, data import, descriptive statistics, and reproducible analysis. It is focused on understanding and working with data rather than covering every part of the R ecosystem.

Regular Expressions: Pattern-Based Text Processing

A regular expression is a compact pattern language for finding, extracting, validating, and replacing text that shares a structure. Regex is supported in every mainstream language, in editors, and in command-line tools such as `grep` and `sed`.

This module introduces literals, character classes, quantifiers and greediness, anchors, groups and backreferences, flags, and dialect differences (PCRE, POSIX,

RE2, JavaScript). It emphasizes verbose-mode patterns with comments, interactive testing, and the classic pitfalls, including catastrophic backtracking and applying regex to nested structured formats.

SQL: Relational Databases

SQL (Structured Query Language) is used to define, query, and manipulate data in relational database systems. It is declarative: a query describes the information to retrieve or the change to make, while the database engine determines how to execute it.

This module introduces tables, rows, columns, keys, filtering, sorting, aggregation, joins, data definition and manipulation, transactions, and safe query practices. SQL is commonly used from applications written in other languages, as well as in database administration and analytics tools.

Structured Data Formats: JSON, YAML, and TOML

Modern software exchanges configuration and data through a small set of text formats: JSON for APIs and data interchange, YAML for infrastructure and pipeline configuration, and TOML for application and tool configuration.

This module introduces the syntax and data model of each format, multi-line strings and block scalars in YAML, tables and arrays of tables in TOML, validation tooling such as `jq`, `yq`, and schema validators, and guidance on choosing the appropriate format for a task. It pays particular attention to YAML's indentation sensitivity in non-visual workflows.

Web Accessibility: ARIA, WCAG, and Accessible Design

Web accessibility is the practice of building websites and applications usable by people with disabilities, including screen-reader users, keyboard-only users, and users with cognitive or motor disabilities.

This module extends the HTML and CSS module. It introduces the accessibility tree, the WCAG principles and levels, semantic HTML that requires no ARIA, correct use of ARIA roles, states, and properties, keyboard interaction and focus management, accessible forms and error reporting, and a testing workflow covering keyboard, screen reader, and automated scans. It also outlines accessibility as a professional specialization.

How to use these modules

1. Complete the shared Computing Foundations Primer.
2. Complete the Terminal Skills prerequisite, using Bash on macOS/Linux or PowerShell on Windows.

3. Review the development environment guide and environment isolation guide when you need an editor, runtime, database, or project environment.
4. Choose the module that matches the task or domain you want to understand. The modules can be studied independently; there is no required order among them. Two light recommendations apply: complete the Git module before modules that assume repository workflows, and complete the Markdown module before modules that ask for written documentation.
5. Work through each module's learning objectives, examples, practice sequence, and recommended reading. Type and modify the examples rather than only reading them.

These modules can also be used alongside the full curricula:

- Git complements every curriculum in this textbook: all project work, collaboration, and capstone submission assumes version control.
- HTML and CSS pair naturally with the JavaScript and TypeScript curricula, but the markup and style-sheet concepts are language-independent.
- Markdown is the writing medium of every repository and pairs with any curriculum; its documentation workflows are language-independent.
- R can complement Python and other languages used for data analysis. It also provides a separate statistical-computing workflow.
- Regular expressions are used across all the language curricula for searching, validation, and text transformation, and are a daily tool in shell and editor workflows.
- SQL complements nearly every language that reads from or writes to relational databases. The SQL module focuses on the query language and relational concepts, not on a specific database vendor or application framework.
- Structured data formats appear throughout the full curricula: JSON in APIs and data files, YAML in CI and deployment configuration, and TOML in Python and Rust project files.
- Web accessibility extends the HTML and CSS module and pairs with the JavaScript and TypeScript curricula; it is also a professional specialization in its own right.

Accessibility and learning approach

The modules follow the same accessibility-first approach as the rest of the textbook:

- Examples are presented as text and are intended to be usable with a keyboard and screen reader.
- Concepts are explained in prose as well as code; learners should not have to infer essential information from color, layout, or an image.
- Web examples emphasize semantic HTML, meaningful labels, logical head-

ings, keyboard access, and visible focus.
- Data and visualization examples should provide text summaries or structured tables as alternatives to graphics.
- Database examples explain query results through column names, row counts, and values rather than relying on a visual result grid alone.

No technology or editor is automatically accessible in every configuration. See the screen-reader-accessible GUI libraries and implementation guide for framework selection and testing guidance.

Scope and prerequisites

These are focused modules, not complete reference manuals. They introduce foundational ideas and direct learners to maintained documentation and additional reading. They do not attempt to teach every framework, database product, statistical method, or deployment workflow.

The modules assume basic computer use, file and folder navigation, and familiarity with the shared foundations material. They do not require prior experience with Git, HTML, CSS, Markdown, R, regular expressions, SQL, structured data formats, or web accessibility.

Section index

Use the Standalone Topics Index for a compact topic-by-topic reference.

17.1 Git and Version Control

Purpose

Git is the distributed version control system used by nearly all modern software development. It records the history of a project as a series of snapshots, lets multiple people work concurrently, and provides the mechanism for reviewing and reverting changes. This module introduces Git as a command-line workflow: repositories, commits, branches, remotes, and collaboration through pull requests, with attention to how each step reads in a non-visual environment.

Git is almost entirely text-based, which makes it unusually accessible — but its output can be verbose, and its mental model (a directed graph of commits) is usually taught visually. This module builds the model through linear history and textual output rather than diagrams.

Learning objectives

By the end of this module, learners should be able to explain what version control is and why it matters; initialize a repository and make commits with

meaningful messages; read `git status`, `git log`, and `git diff` output; create, switch, and merge branches; resolve a simple merge conflict; connect a local repository to a remote and push and pull; open and review a pull request; and describe what `HEAD`, the staging area, and the working tree are.

The mental model

Git manages three areas:

- **Working tree:** the files as they currently exist on disk.
- **Staging area (index):** the set of changes selected to go into the next commit.
- **Repository history:** the chain of committed snapshots.

A commit is a named snapshot of the staging area, linked to its parent commit. The history is a graph, but in ordinary work it behaves like a line: each new commit points back to the one before it. `HEAD` is a pointer to the commit currently checked out — usually the tip of the current branch.

Basic workflow

```
git init                      # create a repository in the current directory
git status                    # what is modified, staged, or untracked
git add README.md             # stage one file
git add -p                    # stage interactively, hunk by hunk
git commit -m "Add README"    # record a snapshot
git log --oneline             # compact history, one line per commit
```

Write commit messages as complete, imperative sentences: "Add input validation to the parser", not "stuff". The first line should stand alone; larger explanations go after a blank line. Good messages are how future readers — including screen reader users skimming history — understand the project.

Reading changes

`git diff` is the most important accessibility surface in Git. It prints changes as unified diff hunks: lines beginning with `-` were removed, lines beginning with `+` were added, and context lines are unchanged. Learn to read diffs linearly before relying on any visual diff tool.

```
git diff              # unstaged changes
git diff --staged     # what the next commit will contain
git diff main..task   # differences between two branches
```

Useful reading options:

- `git diff --word-diff` shows changes within a line, which is often easier to follow by ear than whole-line replacement.
- `git log --stat` summarizes which files each commit touched.

- `git show` prints the full diff of a single commit.

Branching and merging

A branch is a movable pointer to a commit. Creating one is instant and cheap.

```
git switch -c fix-typo      # create and switch to a new branch
# ... edit and commit ...
git switch main             # return to main
git merge fix-typo          # bring the work in
git branch -d fix-typo      # delete the merged branch
```

A fast-forward merge simply moves the branch pointer. When both branches have diverged, Git creates a merge commit; when the same lines changed in both, it reports a conflict. Conflicts are marked in the file itself with `<<<<<<<`, `=======`, and `>>>>>>>` markers. Resolve by editing the file to the intended final text, removing the markers, staging the file, and committing. `git status` lists conflicted files; `git diff` shows the markers in context.

Remotes and collaboration

```
git clone https://github.com/example/project.git
git remote -v               # list configured remotes
git push origin main        # upload commits
git pull                    # fetch and merge others' commits
git fetch                   # download without merging
```

The standard hosted workflow (GitHub, GitLab, Bitbucket) is:

1. Fork or clone the repository.
2. Create a topic branch.
3. Commit changes with clear messages.
4. Push the branch and open a pull request (PR).
5. Respond to review comments, push follow-up commits, and merge after approval.

PR review is text-based: the diff, the comments, and the checks are all readable in a browser with a screen reader, or via the `gh` command-line tool (`gh pr view`, `gh pr diff`, `gh pr checks`), which many screen reader users prefer because everything stays in the terminal.

Undoing things

- `git restore file` — discard unstaged changes to a file.
- `git restore --staged file` — unstage without discarding.
- `git commit --amend` — fix the most recent commit (before it has been pushed).
- `git revert <commit>` — create a new commit that undoes an old one. Safe on shared branches.

- `git reset` — move a branch pointer; prefer `revert` on pushed history.

Accessibility notes

- Terminal-based Git is fully screen-reader compatible; prefer `--oneline` and `--stat` flags to keep output compact.
- `git add -p` and interactive rebase are prompt-driven and work well with a linear reading flow.
- Web-based review interfaces change frequently; the `gh` CLI provides a stable, textual alternative for PR workflows.
- Never rely on color alone to interpret diffs — read the `-`/`+` prefixes.

Practice sequence

1. Initialize a repository, create three files, and make three separate commits, each touching one file.
2. Read `git log --oneline` and `git show HEAD` aloud-in-your-head: confirm you can tell what each commit did.
3. Edit two files, stage only one, and verify with `git diff --staged` versus `git diff`.
4. Create a branch, make a commit, switch back, make a conflicting change on `main`, and resolve the merge conflict.
5. Push a branch to a hosted remote and open a pull request; review it with `gh pr diff`.
6. Use `git revert` to undo a commit and inspect the resulting history.

Recommended introductory books

- Chacon, Scott, and Ben Straub. *Pro Git*, 2nd edition. Apress, 2014. The canonical, freely licensed reference; chapters 1–3 cover everything in this module. Free online book.
- Withee, Sarah. *Git For Dummies*. Wiley, 2023. A gentler first pass if *Pro Git* feels dense on a first read.

Suggested use: work through *Pro Git* chapters 1–3 sequentially, then keep the rest as reference. Skim chapter 7 (Git tools) only after the practice sequence.

Further learning

- Official Git documentation
- GitHub CLI (`gh`) manual
- Conventional Commits — a commit-message convention widely used in open source
- Oh Shit, Git!?! — plain-language recipes for recovering from common mistakes

17.2 HTML and CSS: Structure and Presentation for the Web

Purpose

This introductory module explains how web documents are structured with HTML and presented with CSS. It is designed for learners who already understand basic files, folders, and text editing. It does not assume prior web-development experience.

Learning objectives

By the end of this module, learners should be able to explain the roles of HTML and CSS; create a semantic HTML document with a meaningful title, headings, paragraphs, links, lists, and form controls; distinguish semantic structure from visual presentation; associate labels with form controls; apply basic CSS selectors and box-model properties; and explain why accessibility belongs in document structure.

HTML: semantic document structure

HTML (HyperText Markup Language) describes the meaning and structure of web content. Elements are represented by tags and may contain text, attributes, and nested elements. Browsers construct a document object model from the markup.

Semantic elements communicate purpose. Use headings in a logical hierarchy, paragraphs for prose, lists for grouped items, links for navigation, buttons for actions, and labels for form controls. Prefer native HTML semantics over generic containers with improvised behavior.

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Course Notes</title>
</head>
<body>
  <header>
    <h1>Course Notes</h1>
  </header>
  <main>
    <h2>Current topic</h2>
    <p>Semantic HTML describes the meaning of content.</p>
    <ul>
      <li>Structure content with meaningful elements.</li>
```

```
      <li>Use links for navigation.</li>
    </ul>
  </main>
</body>
</html>
```

The heading hierarchy should reflect the document outline. A heading is not simply a larger font. A screen-reader user may navigate directly by headings, links, landmarks, or form controls, so each must have a clear purpose.

CSS: presentation rules

CSS (Cascading Style Sheets) describes how elements are presented. A rule consists of a selector and declarations. A declaration contains a property and value.

```
body {
  font-family: system-ui, sans-serif;
  line-height: 1.5;
  margin: 2rem;
}

main {
  max-width: 70ch;
}

a {
  text-decoration: underline;
}
```

Selectors identify elements. Properties describe presentation. The cascade resolves competing declarations based on origin, importance, specificity, and source order. The box model describes content, padding, border, and margin. Layout systems such as Flexbox and Grid support responsive page structure.

Accessibility principles

- Use semantic HTML before adding ARIA attributes.
- Preserve a logical heading hierarchy.
- Provide descriptive link text and labels for all form controls.
- Ensure keyboard access and visible focus.
- Do not use color alone to communicate state or meaning.
- Maintain sufficient text and interface contrast.
- Avoid removing browser zoom or text-resizing capabilities.
- Use alternative text for informative images; use empty alternative text for decorative images.

- Ensure dynamic content changes are perceivable and do not unexpectedly move focus.
- Test with keyboard navigation and screen readers, not only visual inspection.

Accessible HTML does not guarantee an accessible page. The complete interaction, browser behavior, CSS, and any JavaScript must be tested.

Practice sequence

1. Identify the semantic elements in a supplied page and explain their roles.
2. Create a page with one main heading, two subheadings, paragraphs, a list, and descriptive links.
3. Add CSS for readable line length, spacing, and link distinction without relying on color alone.
4. Inspect the page using keyboard navigation and a screen reader.
5. Explain one difference between a semantic HTML element and a generic div.

Recommended introductory books

- Robbins, Jennifer. *Learning Web Design: A Beginner's Guide to HTML, CSS, JavaScript, and Web Graphics*, 5th edition. O'Reilly Media, 2018. A broad introduction to web fundamentals, semantic HTML, CSS, responsive layouts, and web design tools. Publisher page.

Accessibility note: this is a visual-design-oriented book and may use diagrams, screenshots, and visual examples. Use it selectively for terminology and reference; this module's semantic, keyboard-first, and screen-reader-oriented explanations remain the required learning path. Validate examples against current HTML and CSS standards.

Further learning

- MDN HTML documentation
- MDN CSS documentation
- W3C Web Accessibility Initiative

17.3 Web Accessibility: ARIA, WCAG, and Accessible Design

Purpose

Web accessibility is the practice of building websites and applications that people can use regardless of disability — including blind and low-vision users who rely on screen readers, keyboard-only users, and users with cognitive or motor

disabilities. This module introduces accessibility as an engineering discipline: the standards (WCAG), the technologies (semantic HTML, ARIA, CSS, and the accessibility tree), the testing workflow (keyboard first, then screen reader), and the legal and professional context.

This module extends the HTML and CSS module. It assumes learners can write semantic HTML and basic stylesheets, and builds the accessibility layer on top. For a curriculum aimed at visually impaired learners, accessibility is not merely a compliance topic: it is a domain in which blind developers are, with proper training, among the most credible experts in the industry.

Learning objectives

By the end of this module, learners should be able to explain how the accessibility tree connects the DOM to assistive technology; state the four WCAG principles (POUR) and give an example conformance technique for each; write semantic HTML that needs no ARIA; apply ARIA roles, states, and properties correctly and explain the first rule of ARIA; implement accessible forms, tables, and dialogs; design and test keyboard interaction, focus order, and visible focus; write text alternatives for images, charts, and controls; and conduct a basic screen-reader and keyboard audit of a web page.

How assistive technology sees the web

A screen reader does not read the rendered pixels; it reads the **accessibility tree**, a parallel structure derived from the DOM. Each element exposes a **name** (its accessible label), a **role** (what it is), a **value** (its current state, where applicable), and its **state and properties** (expanded, checked, disabled, required). Browsers compute this tree from native HTML semantics first, then from ARIA attributes where authors supply them.

This is why semantic HTML is the foundation: a native `<button>` automatically has role `button`, is focusable, responds to Enter and Space, and announces itself correctly. A `<div onclick=...>` has none of that, and no amount of styling will add it.

WCAG: the standard

The Web Content Accessibility Guidelines (WCAG), published by the W3C, organize success criteria under four principles — **POUR**:

- **Perceivable:** information must be available to sight, hearing, or touch. Examples: text alternatives for images, captions for video, sufficient contrast.
- **Operable:** every function must be usable without requiring specific physical abilities. Examples: full keyboard operability, no time limits without a way to extend, no seizure-inducing content.

- **Understandable:** content and controls must be predictable. Examples: consistent navigation, labeled inputs, clear error messages.
- **Robust:** content must work with current and future assistive technology. Examples: valid markup, correct name/role/value.

WCAG 2.1 and 2.2 define levels A, AA, and AAA. Level AA is the common legal and contractual target (including under the ADA, Section 508, and the European Accessibility Act). Learn the AA criteria relevant to your work; do not try to memorize all of them at once.

Semantic HTML first

The most effective accessibility technique is choosing the right element:

- `<button>` for actions, `<a href>` for navigation — never a clickable `<div>`.
- `<nav>`, `<main>`, `<header>`, `<footer>`, `<section>`, `<article>` for landmarks, which screen readers expose for direct navigation.
- One `<h1>` per page and a heading hierarchy without gaps.
- `<label for="id">` bound to every form control.
- `<table>` with `<th scope="col|row">` for genuine data tables — never for layout.
- Native `<details>`/`<summary>` for disclosure widgets when possible.

ARIA: when HTML is not enough

ARIA (Accessible Rich Internet Applications) supplies semantics the platform does not. Its first rule: **do not use ARIA if a native element would work** — incorrect ARIA is worse than none.

```
<!-- Bad: a div with a click handler has no role, no focus, no keyboard support -->
<div class="btn" onclick="save()">Save</div>

<!-- Good: native semantics -->
<button type="button" onclick="save()">Save</button>

<!-- Necessary ARIA: a tabbed interface has no single native element -->
<div role="tablist">
  <button role="tab" aria-selected="true" aria-controls="panel-1" id="tab-1">Details</buttor
  ...
</div>
<div role="tabpanel" id="panel-1" aria-labelledby="tab-1">...</div>
```

Key attribute families:

- **Roles:** `tab`, `dialog`, `alert`, `switch`, `progressbar`, `grid`.
- **States:** `aria-expanded`, `aria-checked`, `aria-disabled`, `aria-current`.
- **Properties:** `aria-label` and `aria-labelledby` (naming), `aria-describedby` (description), `aria-live` (announcing dynamic updates).

`aria-live` regions are how a page reports changes without a page load: form validation results, status messages, new chat lines. Use `role="status"` (polite) or `role="alert"` (assertive) rather than raw `aria-live` values where possible.

Keyboard and focus

- Everything clickable must be reachable with Tab and operable with Enter, Space, and arrow keys as appropriate.
- Manage focus in scripts: when a dialog opens, move focus into it; when it closes, restore focus to the trigger; trap focus inside while it is open (use the native `<dialog>` element, which does much of this for you).
- Never remove the focus outline without providing an equally visible replacement style.
- Respect `prefers-reduced-motion` in animations.
- Provide a "skip to main content" link as the first focusable element on pages with long navigation.

Forms and errors

```
<label for="email">Email address</label>
<input id="email" type="email" autocomplete="email"
       aria-describedby="email-hint email-error">
<p id="email-hint">We send one confirmation message.</p>
<p id="email-error" role="alert">Enter an address in the name@example.com format.</p>
```

Errors must be text, associated with the field, and announced. Do not report errors by color or icon alone, and do not rely on placeholder text as the only label.

Testing workflow

1. **Unplug the mouse (figuratively):** navigate the page with Tab, Shift+Tab, Enter, Space, and arrow keys only.
2. **Screen-reader pass:** test with at least NVDA (free, Windows) and VoiceOver (built into macOS and iOS); JAWS on Windows is also common. Learn the headings list, links list, and form-controls list commands — those are how real users audit a page.
3. **Automated scan:** axe DevTools, Lighthouse, or WAVE catch roughly a third of issues (missing alt text, contrast, unlabeled controls). Automation finds *violations*, not *usability*.
4. **Zoom and reflow:** at 200% zoom and 320px width, no content may be lost or require two-dimensional scrolling (WCAG 1.4.10).
5. **Manual judgment:** the remaining two-thirds of accessibility problems — focus order, announcement timing, meaningful alt text — require a human, and this is where trained screen-reader users have a professional edge.

Accessibility as a career path

Digital accessibility is a growing, undersupplied specialty: auditors, remediation engineers, accessibility QA, and inclusive-design consultants. Professional credentials include the IAAP Certified Professional in Accessibility Core Competencies (CPACC) and Web Accessibility Specialist (WAS). Blind professionals are well represented — and highly sought after — in this field.

Practice sequence

1. Take a page built with `<div>` buttons and rebuild it with semantic elements; verify the tab order and announcements improve with no JavaScript changes.
2. Add a complete ARIA tabbed interface with correct roving tabindex and arrow-key navigation, testing entirely by keyboard.
3. Build a form with labeled fields, hint text, and an `aria-live` error region; trigger each error and confirm it is announced.
4. Run an axe or Lighthouse scan on a real page, fix every reported issue, and list two issues the scan missed that you found by keyboard testing.
5. Write alt text for five images of increasing complexity, including one chart, using a short description plus a data table for the chart.
6. Audit one public website with NVDA's headings and links lists and write a one-page findings memo.

Recommended introductory books

- W3C Web Accessibility Initiative. *Web Accessibility Perspectives* and the WCAG quick reference. Free, and the authoritative mapping of requirements to techniques. WCAG 2 quick reference.
- Gilbert, Regine. *Inclusive Design for a Digital World*. Apress, 2019. A practitioner-oriented introduction covering design and code.
- Kalbag, Laura. *Accessibility for Everyone*. A Book Apart, 2017. A short, plain-language orientation to the whole discipline, useful before diving into standards.

Suggested use: start with *Accessibility for Everyone* for the framing, then work directly from the WCAG quick reference while coding — it is the reference you will actually use professionally. Use the *Perspectives* videos to understand user impact.

Further learning

- W3C WAI: Introduction to Web Accessibility
- ARIA Authoring Practices Guide — tested interaction patterns for widgets
- MDN Accessibility documentation
- Deque axe DevTools and WebAIM — testing tools and articles

- NVDA screen reader — free, open source, and scriptable

17.4 Markdown: Technical Documentation

Purpose

Markdown is a lightweight plain-text markup language used to write documentation, README files, technical notes, and web content. It is the default documentation format of GitHub, most package registries, static site generators, and developer communication tools. This module introduces Markdown as a practical documentation skill, with emphasis on writing documents that are both easy to maintain and easy to navigate with a screen reader.

Markdown is particularly well suited to non-visual workflows. Its source is plain text, its syntax is a small set of punctuation conventions, and its structure is expressed linearly rather than spatially. A screen reader user can both author and verify Markdown without any visual rendering.

Learning objectives

By the end of this module, learners should be able to explain where Markdown is used and why it dominates technical documentation; write headings, paragraphs, lists, links, images, code blocks, tables, and blockquotes; distinguish inline from reference-style links; write documents with a single top-level heading and a logical heading hierarchy; use semantic alternatives (descriptive link text, alt text, tables with headers); explain the differences between CommonMark, GitHub Flavored Markdown, and MultiMarkdown; and choose and use a linting or preview tool appropriate to a screen-reader workflow.

Why Markdown for documentation

- **Plain text:** version-controllable with Git, diffable line by line, and readable in any editor or terminal.
- **Portable:** renders on GitHub, GitLab, npm, PyPI, Read the Docs, Discourse, Slack, and hundreds of other tools.
- **Convertable:** Pandoc and static site generators transform Markdown into HTML, PDF, EPUB, and Word formats.
- **Low syntax cost:** the punctuation marks carry meaning, so documents remain readable even as raw text.

Core syntax

Headings

Headings are written with leading hash marks. The number of hash marks is the heading level.

```
# Document title (h1)
## Section (h2)
### Subsection (h3)
```

Use exactly one `h1` per document and never skip levels (for example, `h2` directly to `h4`). Screen readers navigate by heading level, so a clean hierarchy is an accessibility feature of the document itself.

Paragraphs and emphasis

Paragraphs are separated by blank lines. Within a line, wrap text in single asterisks for emphasis and double asterisks for strong importance.

```
This is a paragraph with *emphasis* and **strong importance**.
```

Lists

Unordered lists use hyphens; ordered lists use numbers followed by periods. Nested items are indented by two to four spaces.

```
- First item
- Second item
  - Nested item

1. Install the runtime
2. Configure the environment
3. Run the test suite
```

Links and images

Inline links place the text in square brackets and the target in parentheses. Reference-style links move targets to the bottom of the document, which keeps prose readable when read aloud.

```
See the [installation guide](https://example.com/install) for details.

See the [installation guide][install] for details.

[install]: https://example.com/install
```

Images use the same syntax with a leading exclamation mark. The alt text is what a screen reader announces, so describe the image's content or purpose.

```
![Bar chart of test coverage rising from 40 to 90 percent](coverage.png)
```

Write link text that makes sense out of context. Avoid bare URLs and phrases like ”click here” or ”this link”; screen reader users often pull up a list of links and read it in isolation.

Code

Inline code uses backticks; fenced code blocks use triple backticks, with a language identifier that enables syntax highlighting on renderers that support it.

````
Run `npm install` before building.

```python
def greet(name):
    return f"Hello, {name}"
```
````

Blockquotes and horizontal rules

```
> Quoted material goes here.

---
```

Tables (GitHub Flavored Markdown)

Tables use pipes and a separator row. Include a header row; screen readers announce column headers as the reader moves between cells.

```
| Command | Purpose |
| --- | --- |
| `git status` | Show working tree state |
| `git log` | Show commit history |
```

Tables are the least screen-reader-friendly Markdown structure. For anything beyond a few rows, consider a definition list or a sequence of short headings and paragraphs instead.

Escaping

Prefix a literal punctuation character with a backslash to prevent it from being interpreted as syntax: `\*not emphasis\*`.

Dialects and specifications

- **CommonMark** is the baseline specification most renderers implement.
- **GitHub Flavored Markdown (GFM)** extends CommonMark with tables, task lists, strikethrough, and automatic linking. It is the practical standard for repository documentation.
- **MultiMarkdown** and **Pandoc Markdown** add footnotes, citations, and definition lists, and are common in academic and publishing workflows.

When writing for an unknown renderer, stay within CommonMark plus tables.

Screen-reader workflow

- Read documents by heading navigation to check structure before checking content.
- Use reference-style links in long documents so prose reads cleanly.
- Verify that every image has alt text and every link has descriptive text.
- Prefer lists and headings over ASCII layout, aligned columns, and horizontal spacing, which do not survive linear reading.
- A linter such as `markdownlint` catches broken nesting, missing alt text, and inconsistent heading levels without requiring visual preview.

Practice sequence

1. Write a README for a small script with one `h1`, three `h2` sections, and a bulleted requirements list.
2. Add a fenced code block with a language identifier, then a table of three commands.
3. Convert three inline links to reference-style links and re-read the document by headings and links list.
4. Add an image with descriptive alt text, then temporarily rename the image file to confirm the alt text is what carries the meaning.
5. Run `markdownlint` (or an equivalent) on the document and fix every warning.
6. Use Pandoc to convert the document to HTML and inspect the heading structure.

Recommended introductory books

- MacFarlane, John. *Pandoc User's Guide*. Freely available and the definitive reference for Markdown's extensions and conversion workflows. Free online manual.
- LeBlanc, David. *The Markdown Guide*. 2020. A free, syntax-focused reference organized by element, useful as a lookup companion. Free online book.

Suggested use: keep the Markdown Guide open as a syntax reference while writing; consult the Pandoc manual when converting documents to other formats.

Further learning

- CommonMark specification
- GitHub Flavored Markdown spec
- markdownlint
- Semantic line breaks — a convention that makes Markdown diffs and screen-reader line navigation more meaningful

17.5 R: Statistical Computing

Purpose

R is a programming language and environment designed for statistical computing, data analysis, and graphics. It is widely used in research, public health, economics, social science, government, and industry analytics. This module introduces R as a programming language for reproducible data work.

Learning objectives

By the end of this module, learners should be able to describe the role of R in statistical computing; assign values and recognize common data types; create and inspect vectors, lists, matrices, and data frames; use indexing and basic functions; summarize a small dataset; read a delimited data file; and explain reproducible scripts and documented analysis.

R as a language and environment

R provides an interactive console, a language runtime, a large package ecosystem, and tools for statistical analysis and visualization. The R Project maintains the language and core environment. Posit develops widely used products such as RStudio, but R and RStudio are distinct: R is the language and runtime; RStudio is an IDE that can support R workflows.

Core values and objects

R has numeric, integer, character, logical, complex, and other specialized data types. Assignment is commonly written with the left-arrow operator, though equals is also accepted in many contexts.

```
student_name <- "Avery"
scores <- c(82, 91, 76, 88)

mean_score <- mean(scores)
score_count <- length(scores)

print(mean_score)
print(score_count)
```

The c() function combines values into a vector. Vectors are central to R's model of data. Many operations are vectorized, meaning an operation can apply across a vector without writing an explicit loop.

Data structures

- **Vector:** one-dimensional collection whose elements share a common type.
- **List:** collection that can contain elements of different types and structures.

- **Matrix:** two-dimensional homogeneous data structure.
- **Array:** multidimensional homogeneous data structure.
- **Data frame:** tabular structure whose columns may have different types.
- **Factor:** categorical data with defined levels.

Indexing in R commonly begins at 1. Square brackets select elements or subsets. The dollar-sign operator is often used to access a named data-frame column or list element.

Functions and analysis

Functions accept arguments and return values. R includes built-in functions for summary statistics, sorting, aggregation, and modeling. Packages extend the language for specialized methods. Package versions and data sources should be recorded so analyses can be reproduced.

```
scores <- c(82, 91, 76, 88)

summary(scores)
median(scores)
sd(scores)
```

Statistical summaries should be interpreted in context. A numerical result is not automatically evidence of a meaningful effect or causal relationship.

Data import and reproducibility

Delimited files are common sources of tabular data. Learners should inspect column names, data types, missing values, and row counts before analysis. Base R includes read.csv() for common CSV files. Larger workflows may use maintained packages such as readr or data.table.

Keep analysis steps in scripts or documented notebooks rather than relying only on a sequence of commands entered interactively. Record package versions, data provenance, assumptions, and transformations. Protect confidential or personally identifying data.

Accessibility and visualization

R's text console and printed summaries support text-first analysis. Graphics and IDE data viewers may require additional accessibility evaluation. A chart should not be the only representation of a result: provide a textual description, table, or statistical summary where appropriate. Do not require learners to infer exact values from color or spatial position alone.

Practice sequence

1. Create a numeric vector and calculate its mean, median, minimum, and maximum.
2. Build a data frame with names and scores.
3. Select one column and identify the number of observations.
4. Read a CSV file and inspect its dimensions and column names.
5. Write a short paragraph interpreting the summary without overstating what the data establish.

Recommended introductory books

- Grolemund, Garrett. *Hands-On Programming with R.* O'Reilly Media, 2014. Written for readers without programming experience; introduces R through small projects, data objects, functions, scripts, and practical analysis. The full text is freely available online. Free book and publisher page.
- Wickham, Hadley, Mine Çetinkaya-Rundel, and Garrett Grolemund. *R for Data Science*, 2nd edition. O'Reilly Media, 2023. A free guide to importing, transforming, visualizing, and communicating data with the tidyverse. Free online book.

Suggested use: start with *Hands-On Programming with R* if programming itself is new. Use *R for Data Science* as a follow-on for data analysis workflows. The second book focuses on the tidyverse ecosystem and is not a substitute for learning base R fundamentals.

Further learning

- The R Project
- An Introduction to R
- RStudio / Posit documentation

17.6 Regular Expressions

Purpose

A regular expression (regex) is a compact pattern language for describing sets of text: finding, extracting, validating, and replacing strings that share a structure. Regex is supported in every mainstream language (Python `re`, JavaScript, Java, .NET, Go, Rust), in editors, in command-line tools such as `grep` and `sed`, and in many screen-reader and text-processing workflows. This module introduces the core pattern syntax, the main dialect differences, and the discipline of writing patterns that are readable and testable.

Regex is a high-leverage skill for non-visual computing: a single well-written pattern can accomplish text restructuring that would otherwise require visually

scanning a document.

Learning objectives

By the end of this module, learners should be able to explain what a regex matches and where regex is and is not appropriate; use literals, character classes, quantifiers, anchors, groups, and alternation; use capture groups and backreferences to extract and restructure text; read and apply common flags such as case-insensitive and multiline matching; distinguish the major dialects (PCRE, POSIX, and language-specific variants); use an online tester and command-line tools to develop patterns; and recognize the classic pitfalls, including catastrophic backtracking and using regex on nested structures.

Literals, metacharacters, and escapes

Most characters match themselves. A small set are metacharacters: `. ^ $ * + ? ( ) [ ] { } | \`. Prefix a metacharacter with a backslash to match it literally — for example, `\.` matches a period.

Character classes

```
[abc]        one of the characters a, b, or c
[a-z0-9]     any lowercase letter or digit
[^abc]       any character except a, b, or c
\d           a digit (same as [0-9] in most dialects)
\w           a word character (letter, digit, underscore)
\s           whitespace
.            any character except newline
```

Negated classes and `[^...]` are the workhorses of "match until" patterns. `\b` is a word boundary: `\bcat\b` matches the word "cat" but not "category".

Quantifiers and greediness

```
*        zero or more
+        one or more
?        zero or one
{3}      exactly three
{2,5}    between two and five
```

Quantifiers are **greedy** by default: they consume as much as possible. Adding `?` makes them **lazy**: `.*?` stops at the first possible match. Greediness is the single most common source of surprising matches — `<.*>` against `<a> <b>` matches the whole string, while `<.*?>` matches `<a>`.

Anchors, alternation, and grouping

```
^cat$        the entire line is exactly "cat" (with multiline flag)
cat|dog      "cat" or "dog"
(colou?r)    group: matches "color" or "colour"
(?:cat)      non-capturing group
```

Parentheses create capture groups, which number from 1 in order of opening parenthesis. Group 0 is the whole match. Backreferences (`\1`) and, in some dialects, named groups (`(?P<year>\d{4})`) make patterns self-documenting and replacements precise.

Extraction and replacement

The classic restructuring pattern: capture the pieces, then rebuild them.

```
import re

text = "Published 2026-03-14; revised 2026-09-02."
pattern = r"(\d{4})-(\d{2})-(\d{2})"
matches = re.findall(pattern, text)          # [('2026','03','14'), ...]
swapped = re.sub(pattern, r"\2/\3/\1", text) # 03/14/2026
```

Flags

- `i` — case-insensitive matching.
- `m` — `^` and `$` match at line boundaries, not just string boundaries.
- `s` — `.` also matches newlines.
- `x` — verbose mode: whitespace and `#` comments in the pattern are ignored. Verbose mode is the most effective accessibility feature regex has; use it for any nontrivial pattern.

```
pattern = re.compile(r"""
    (\d{4})   # year
    -         # separator
    (\d{2})   # month
    -         # separator
    (\d{2})   # day
""", re.VERBOSE)
```

Dialects and tooling

- **PCRE** (Perl-compatible) is the most feature-rich dialect and the de facto reference; Python, PHP, and most editors are close to it.
- **POSIX** regex (as in basic `grep`) is more limited; use `grep -E` for extended syntax.
- **RE2** (Go, and optional elsewhere) forbids backreferences in exchange for guaranteed linear-time matching.

- **JavaScript** regex lacks lookbehind in older engines and uses `/pattern/flags` literals.

Develop patterns interactively in an online tester such as regex101 (which explains each token in plain language — a genuinely screen-reader-friendly explanation mode), then confirm behavior in the target language, because dialects differ. On the command line, `grep -E`, `sed -E`, and `rg` (ripgrep) are the daily tools.

Pitfalls and discipline

- **Regex is the wrong tool for nested or structured formats** such as HTML, JSON, or XML. Use a parser.
- **Catastrophic backtracking:** nested quantifiers like `(a+)+` can make matching time explode. Prefer explicit, anchored patterns.
- **Validation:** a regex can confirm shape, not correctness. `\d{5}` matches a five-digit string; it does not confirm it is a real postal code.
- **Readability:** prefer verbose mode, named groups, and comments. A pattern no one can read is a bug waiting to happen.

Practice sequence

1. Use `grep -E` to find every line in a text file containing a four-digit year.
2. Write a pattern that matches US phone numbers in both `555-1234` and `(555) 555-1234` formats; test it in regex101.
3. Extract all email-shaped strings from a document and print only the domain parts, using a capture group.
4. Use `sed -E` to convert `YYYY-MM-DD` dates in a file to `MM/DD/YYYY`.
5. Rewrite one dense one-line pattern in verbose mode with comments, and confirm it still matches the same strings.
6. Deliberately construct a nested-quantifier pattern, observe the slowdown on a long input, then fix it.

Recommended introductory books

- Goyvaerts, Jan, and Steven Levithan. *Regular Expressions Cookbook*, 2nd edition. O'Reilly Media, 2012. Task-oriented recipes (dates, URLs, identifiers) with dialect-specific variants; ideal as a lookup reference after the fundamentals.
- Friedl, Jeffrey E. F. *Mastering Regular Expressions*, 3rd edition. O'Reilly Media, 2006. The deep treatment of how regex engines work, including backtracking and efficiency.

Suggested use: learn the syntax from any tutorial or the regex101 explanation panel, then use the *Cookbook* as your day-to-day reference. Read *Mastering Regular Expressions* only if you need to debug performance or engine behavior.

Further learning

- regex101 — interactive tester with plain-language token explanations
- Regular-Expressions.info — comprehensive syntax reference
- Python `re` module documentation
- MDN: Regular Expressions (JavaScript)

17.7 SQL: Relational Databases

Purpose

SQL (Structured Query Language) is the standard language used to define, query, and manipulate data in relational database systems. This module introduces relational concepts and the core query patterns used to retrieve and maintain structured data.

Learning objectives

By the end of this module, learners should be able to explain tables, rows, columns, primary keys, and foreign keys; write a basic SELECT query with filtering and sorting; use aggregate functions and GROUP BY; combine related tables with joins; explain data definition and manipulation; describe transactions and integrity constraints; and recognize parameterized queries and least-privilege access.

Relational database concepts

A relational database organizes information into tables. Each table has columns that define attributes and rows that represent records. A primary key identifies a row. A foreign key expresses a relationship to a key in another table.

A database schema defines tables, columns, types, constraints, and relationships. Normalization is a design discipline that reduces unnecessary duplication and update anomalies. The appropriate degree of normalization depends on the application and its performance requirements.

Querying data

The SELECT statement retrieves data. WHERE filters rows, ORDER BY sorts results, and LIMIT restricts the number of rows returned in systems that support that syntax.

```
SELECT student_name, score
FROM student_scores
WHERE score >= 80
ORDER BY score DESC;
```

SQL is declarative: a query describes the result to retrieve, while the database engine chooses an execution plan. Result order is not guaranteed unless an explicit ORDER BY is included.

Aggregation

Aggregate functions summarize groups of rows. Common functions include COUNT, SUM, AVG, MIN, and MAX.

```
SELECT course_id, AVG(score) AS average_score
FROM student_scores
GROUP BY course_id
ORDER BY course_id;
```

GROUP BY forms groups for aggregation. HAVING filters groups after aggregation, while WHERE filters individual rows before grouping. NULL represents missing or unknown information and requires careful handling; ordinary equality comparisons do not test for null values.

Joining tables

Joins combine rows from related tables. An inner join returns matching rows. A left join preserves all rows from the left table and supplies nulls for unmatched right-side columns. Join conditions should use the intended keys; incorrect joins can multiply rows or silently omit records.

```
SELECT students.student_name, enrollments.course_id
FROM students
INNER JOIN enrollments
  ON students.student_id = enrollments.student_id;
```

Data definition and manipulation

- CREATE TABLE defines a table.
- ALTER TABLE changes a table definition.
- INSERT adds rows.
- UPDATE modifies rows.
- DELETE removes rows.
- DROP removes database objects.

Data-changing statements should be tested carefully, especially when they lack a restrictive WHERE clause. Use transactions when a group of changes must succeed or fail together. Understand the database's transaction and isolation behavior before relying on assumptions.

Security and responsible use

Use parameterized queries rather than constructing SQL by concatenating untrusted input. Grant applications only the database permissions they require. Avoid storing plaintext passwords or exposing confidential records in logs and examples. Do not run destructive queries against production data during practice. Use an isolated training database with synthetic data.

Accessibility

A database client should expose query editors, result grids, errors, and schema navigation through keyboard and assistive technology. A visual result grid should not be the only way to inspect results; learners should be able to retrieve rows as text or another accessible representation. Explain query output in terms of column names, row counts, and values.

Practice sequence

1. Create or inspect a table with a primary key.
2. Select chosen columns and filter rows.
3. Sort results explicitly.
4. Calculate a group average with GROUP BY.
5. Join two tables using a foreign-key relationship.
6. Explain how a parameterized query protects user-supplied values.

Recommended introductory books

- Beaulieu, Alan. *Learning SQL*, 3rd edition. O'Reilly Media, 2020. A beginner-oriented introduction to relational database concepts, schema creation, data manipulation, joins, and querying. Publisher page.
- Viescas, John L. *SQL Queries for Mere Mortals: A Hands-On Guide to Data Manipulation in SQL*, 4th edition. Addison-Wesley Professional, 2018. A step-by-step treatment of relational thinking and query construction. Publisher page.

Suggested use: *Learning SQL* is a suitable primary companion for the module. *SQL Queries for Mere Mortals* provides additional practice in reasoning about queries and relational structures. Both books discuss particular database implementations; SQL dialects differ, so check syntax against the database engine used in a project.

Further learning

- PostgreSQL documentation
- SQLite documentation
- SQL language reference in Microsoft Learn

17.8 Structured Data Formats: JSON, YAML, and TOML

Purpose

Modern software exchanges configuration and data through a small set of text formats: JSON for APIs and data interchange, YAML for infrastructure and pipeline configuration, and TOML for application and tool configuration. None of these is a programming language, but fluency in them is a daily professional requirement. This module introduces the syntax, data model, and appropriate use of each format, and the errors that most commonly occur when writing them.

All three formats are plain text and therefore fully workable with a screen reader. The module pays particular attention to indentation-sensitive syntax (YAML), which is the main accessibility pitfall, and to validation tooling that catches mistakes without visual inspection.

Learning objectives

By the end of this module, learners should be able to read and write valid JSON, including nested objects and arrays; read and write YAML, including lists, mappings, and multi-line strings; read and write TOML, including tables and arrays of tables; choose the appropriate format for a given task and justify the choice; validate files with the appropriate schema or linting tool; and recognize the characteristic failure modes of each format (trailing commas in JSON, indentation errors in YAML, mis-nested tables in TOML).

JSON

JSON (JavaScript Object Notation) has exactly six value types: string, number, boolean (`true`/`false`), null, object, and array. Strings use double quotes; object keys must be strings; no comments, no trailing commas.

```
{
  "name": "avery",
  "roles": ["student", "tutor"],
  "active": true,
  "score": 91.5,
  "address": {
    "city": "Denver",
    "state": "CO"
  },
  "courses": [
    { "title": "Algorithms", "grade": "A" },
    { "title": "Databases", "grade": "B+" }
```

```
  ]
}
```

Reading rules of thumb:

- Objects `{}` are unordered key→value mappings; arrays `[]` are ordered lists.
- Nesting is unlimited, but deeply nested JSON is hard to navigate by ear — flatten structures where you control the schema.
- JSON has no date type; dates are strings, conventionally ISO 8601 (`2026-11-11T09:30:00Z`).

Tooling: `jq` for querying and transforming JSON in the terminal (`jq '.courses[].title' file.json`), and JSON Schema for validating structure. Most editors and linters detect trailing commas and unquoted keys.

YAML

YAML is indentation-based: nesting is expressed with spaces (never tabs). It is a superset of JSON in most implementations, and is the standard format for GitHub Actions, Kubernetes, Docker Compose, Ansible, and countless CI systems.

```
name: curriculum-build
on:
  push:
    branches: [main]
schedule:
  - day: monday
    topic: git
  - day: wednesday
    topic: testing
notes: >
  Folded scalar: newlines become spaces.
raw: |
  Literal block: newlines are preserved.
```

Key syntax:

- **Mappings:** `key: value`, one per line; nested keys are indented (commonly two spaces).
- **Sequences:** `- item` lines, or inline `[a, b]`.
- **Scalars:** usually unquoted; quote when the value could be misread (for example, `"on"`, `"no"`, `"3.10"` — YAML 1.1 treats some unquoted words and version-like strings as booleans or numbers).
- **Block scalars:** `>` folds newlines into spaces; `|` preserves them.
- **Anchors and aliases** (`&base`, `*base`) reuse blocks; **merge keys** (`<<: *base`) combine mappings.

YAML's indentation sensitivity is its main hazard in a non-visual workflow: a one-space error silently changes the structure. Mitigate it by (1) using an editor that shows indentation levels audibly or via status information, (2) running a YAML parser (`yq`, or `python -c "import yaml, sys; yaml.safe_load(sys.stdin)"`) after every edit, and (3) validating against the tool's schema (for example, `actionlint` for GitHub Actions). Where a choice exists, JSON or TOML is often the safer authoring format.

TOML

TOML (Tom's Obvious, Minimal Language) is designed for application configuration: explicit, section-based, and insensitive to indentation. It is the format of `Cargo.toml` (Rust), `pyproject.toml` (Python), and Go modules.

```
title = "Curriculum settings"

[author]
name = "Avery"
roles = ["student", "tutor"]

[build]
target = "web"
flags = ["--verbose", "--watch"]

[[courses]]
title = "Algorithms"
grade = "A"

[[courses]]
title = "Databases"
grade = "B+"
```

Key syntax:

- **Key/value pairs** at the top level or inside `[table]` sections.
- **Sub-tables** use dotted names: `[build.options]` nests under `[build]`.
- **Arrays of tables** use `[[name]]`; each occurrence appends one element.
- **Types:** strings (double quotes, or single quotes for literal strings), integers, floats, booleans, arrays, inline tables (`{ x = 1 }`), and a native date/time type — a genuine advantage over JSON.
- Multiline strings use triple quotes; comments start with `#`.

TOML's error messages from standard parsers are precise and line-numbered, which makes it the most screen-reader-friendly of the three formats to debug.

Choosing a format

- **JSON:** machine-to-machine interchange, API payloads, data files consumed by code. No comments makes it poor for hand-edited configuration.
- **YAML:** configuration written and read by humans in DevOps/CI contexts, especially where a tool mandates it. Powerful but indentation-fragile.
- **TOML:** application configuration with a schema the author controls; explicit and robust, but less common in interchange.

Practice sequence

1. Write a JSON file describing a course catalog (three courses, each with topics and a reading list), then query it with `jq` to print only course titles.
2. Introduce a trailing comma and an unquoted key; run a JSON validator and read the error messages.
3. Rewrite the same data in YAML, then deliberately break the indentation of one nested list and observe what the parser reports.
4. Rewrite the data a third time in TOML using arrays of tables, and add comments explaining each section.
5. Validate a GitHub Actions workflow with `actionlint` (or any schema validator) and fix every reported problem.
6. Write a short paragraph arguing which of the three formats you would choose for a new tool's configuration file, and why.

Recommended introductory books

- Tom Preston-Werner, Pradyun Gedam, et al. *TOML v1.0.0 specification*. Short, free, and readable end to end. Specification.
- YAML and JSON are best learned from their official specifications and tutorials rather than books: the YAML specification and Learn YAML in Y Minutes, plus MDN on JSON.
- Hiltgen, Sam. *jq in Action*. Manning (forthcoming/current editions). For learners who want terminal-side JSON querying as a daily skill; the jq manual is free and comprehensive.

Suggested use: the specifications are short enough to be the primary texts. Use a cookbook-style resource (Learn X in Y Minutes) for a first pass, then keep the TOML and jq references at hand.

Further learning

- JSON specification (ECMA-404)
- JSON Schema
- yq — command-line YAML/JSON/TOML processor
- actionlint — GitHub Actions workflow validator

Part XVIII

Backmatter

Chapter 18

Teacher's Guides for Each Course

Use these guides to plan instruction, adapt the course for learners who use assistive technology, teach language-specific concepts, assess learning, and prepare the development environment. Each guide supplements the corresponding curriculum overview and lessons.

Full language curricula

- C Teacher's Guide
- C# Teacher's Guide
- C++ Teacher's Guide
- Go Teacher's Guide
- Java Teacher's Guide
- JavaScript Teacher's Guide
- Julia Teacher's Guide
- Kotlin Teacher's Guide
- Lua Teacher's Guide
- Python Teacher's Guide
- Rust Teacher's Guide
- Swift Teacher's Guide
- TypeScript Teacher's Guide

The guides are ordered alphabetically by language name. The standalone HTML/CSS, R, and SQL modules are focused technology resources rather than full language curricula, so they are not included in this language-guide set.

18.1 C Teacher's Guide

Purpose and course context

This guide supports instructors teaching the C curriculum to beginning programmers, including blind and visually impaired learners using screen readers, braille displays, magnification, keyboard navigation, or combinations of access technology. It supplements the C curriculum overview, complete lessons, and topic index.

The course targets **C17** and a GCC or Clang workflow. C is a compiled procedural language in which learners work directly with explicit types, arrays, structs, pointers, memory lifetimes, and return-code-based error handling. Do not treat C as merely Python with different punctuation: its model of memory and program execution is a central learning objective.

Outcomes and sequence

Follow Units 0–11 in order, after the shared Computing Foundations Primer and Terminal Skills prerequisite. Learners should be able to create and compile a project, interpret compiler diagnostics, use scalar types and expressions, implement decisions and loops, work with arrays and strings, define functions and structs, reason about pointers and object lifetimes, process files, check return values, test behavior, and complete the capstone.

Use the same lesson rhythm throughout: state the objective; define vocabulary; explain the code's structure and punctuation; ask for a prediction; compile and run; inspect output and diagnostics; modify one feature; and ask the learner to explain the result.

Toolchain and compilation

Use a C17-capable GCC or Clang compiler. The curriculum's representative command is:

```
cc -std=c17 -Wall -Wextra -Wpedantic -g main.c -o main
./main
```

Explain each compiler option: language standard, warnings, debug information, source input, and output executable. Compile after small changes rather than accumulating many errors. Teach the learner to read the first diagnostic, identify its file and line, navigate directly to that location, and distinguish a warning from an error. Warnings are instructional feedback; do not normalize ignoring them.

Teaching C's distinctive concepts

Types, expressions, and input

C does not provide Python-style automatic conversion for every operation. Teach integer division, signed and unsigned values, floating-point representation, integer promotions, and explicit conversions with small examples. Use format specifiers carefully and explain that a mismatch between a format string and the corresponding argument can cause undefined behavior. Validate input results; never assume a conversion succeeded merely because a prompt was displayed.

Arrays, strings, and bounds

An array is a fixed-size contiguous sequence; it does not carry a runtime length that can safely be recovered from an arbitrary pointer. A C string is a character sequence terminated by a null character. Teach the distinction between array capacity, current string length, and the terminating null byte. Make bounds checks explicit and use size-aware library operations. Do not teach unsafe input functions or imply that a buffer automatically expands.

Pointers, structs, and lifetime

Introduce addresses and indirection with a stable verbal model: a pointer stores an address, and dereferencing accesses the object at that address. Distinguish a pointer from the object it points to. Teach null pointers, initialization, pointer arithmetic limits, and the lifetime of automatic, static, and dynamically allocated objects. Every allocation needs a clear ownership and release plan; pair allocation and deallocation in the same documented responsibility. Do not present a pointer as an integer or suggest that every pointer is safe to dereference.

Use structs to model named records. Prefer named fields over parallel arrays when they improve the learner's ability to inspect and explain a record. Explain that passing a struct by value copies it, while passing a pointer permits access to the original object.

Errors and files

C has no built-in exception mechanism. Teach functions' return values, status codes, output parameters, and relevant library error indicators. Check the result of file opening, reading, writing, and closing operations. Make failure paths explicit and ensure resources are released on every path. JSON persistence in the course uses a documented external library; distinguish library functionality from the C standard library and explain its build/link requirements.

Accessibility and classroom delivery

- Keep demonstrations and assessments text-first; make terminal output and compiler diagnostics available as selectable text.
- Describe braces, parentheses, brackets, pointer operators, ampersands, semicolons, and nesting explicitly when first introduced.
- Teach editor navigation by line, symbol, diagnostic, and file; provide keyboard equivalents for every essential action.
- Let learners configure punctuation verbosity and braille output to match their needs.
- Avoid visual memory diagrams as the only explanation. Describe memory regions, addresses, array positions, and ownership verbally or with structured text.
- Do not require a visual debugger, graphical IDE feature, or GUI-only workflow without an equivalent accessible route.
- Allow time for navigating compiler output and reviewing punctuation; assess conceptual understanding separately from navigation speed.

A text interface is not automatically accessible. Test the actual compiler, terminal, editor, and assistive-technology combination used in class.

Assessment and common misconceptions

Assess compilation, runtime behavior, reasoning, and safety separately. Ask learners to explain array bounds, string termination, pointer validity, object lifetime, and the consequences of unchecked input. Include normal, boundary, and invalid-input tests. Useful debugging tasks include a format-specifier mismatch, an off-by-one loop, a failed file open, and a pointer that is not initialized.

Do not equate a successful compile with correctness or memory safety. The repository's example checks, where available, establish only their documented scope; they do not replace behavioral tests or hands-on accessibility validation.

Differentiation and pacing

Some learners will need additional practice with punctuation, nested blocks, compiler invocation, or terminal navigation. Others may be ready for deeper work on const-correctness, allocation strategies, or API contracts. Keep the learning objective constant while varying the amount of scaffolding, number of examples, and time allowed. Offer a short trace table in text for loops and pointer changes; never require a visual memory diagram.

AI-use policy

Follow the repository's course policy: AI may be used only for personal conceptual understanding and reinforcement. It must not write, complete, autocomplete, repair, refactor, or generate course code or assessment solutions. Apply the same rule to instructor demonstrations and student submissions.

Instructor preparation and maintenance

Before teaching, verify the installed compiler and its C17 support, run the repository's current example-validation procedure if provided, and confirm the terminal/editor workflow with the learner's assistive technology. Record compiler version, platform, command, and validation scope. When changing examples, update related expected output and validation materials together. Consult the shared accessibility guidance before selecting any graphical tool.

18.2 C# Teacher's Guide

Purpose and course context

This guide supports the C# curriculum, designed for beginning programmers including blind and visually impaired learners who use screen readers, braille displays, magnification, or keyboard navigation. It supplements the complete lessons and topic index.

The course uses the **.NET 10 SDK and C# 14 toolchain**. C# is a statically typed, managed language with a broad .NET library ecosystem. Teach the language and the .NET runtime as related but distinct ideas: C# is the source language; the SDK builds projects; the runtime executes applications; and libraries provide APIs.

Outcomes and course sequence

Complete the shared Computing Foundations Primer and Terminal Skills prerequisite, then follow Units 0–11. Learners should be able to create and run a console project, read compiler diagnostics, use types and expressions, implement control flow, work with collections, write methods and classes, handle files and exceptions, test programs, and complete the capstone. They should understand nullable reference types and the role of the .NET project system.

Use a repeatable lesson structure: objective and vocabulary; explicit code walkthrough; prediction; build/run; output and diagnostic review; independent modification; explanation. Require learners to explain what the program does, not merely repeat the compiler's response.

Toolchain and build workflow

Use the .NET 10 SDK and create a console project with:

```
dotnet new console
dotnet run
dotnet build
dotnet test
```

Explain the project file, target framework, SDK, compiler, runtime, NuGet dependency, and build artifacts. Run commands from the project directory and teach learners to identify the current working directory. Use repository-defined commands and dependencies; avoid global, undocumented tooling. The curriculum's automated checker builds and smoke-executes each fenced C# example with nullable analysis and warnings treated as errors. This is a useful baseline check, not proof of expected-output correctness or complete capstone behavior.

Teaching C# concepts

Types, conversions, and nullability

C# is statically typed. Explain type inference with var without implying dynamic typing. Teach numeric conversions, integer division, overflow behavior, and the difference between value types and reference types at an introductory level. Nullable reference types provide compiler analysis; they do not make runtime null values impossible. Use nullable annotations and flow analysis to reason about possible absence. Avoid suppressing warnings with the null-forgiving operator as a routine fix.

Collections and data modeling

Compare arrays, List, Dictionary<TKey,TValue>, and other collections by ordering, key lookup, mutability, and intended use. Teach generic type parameters as constraints on the values a collection accepts. Use named record or class models when they make domain data easier to inspect and explain. Avoid opaque tuples for long-lived records.

Methods, classes, and object-oriented design

Teach method signatures as contracts: name, parameters, return type, and observable behavior. Explain constructors, properties, access modifiers, static members, interfaces, and composition. Inheritance should solve a clear substitutability problem rather than serve as the default reuse technique. Distinguish reference identity from value equality and teach equality deliberately when learners use records or custom types.

Exceptions, files, and tests

Teach exceptions for exceptional failures and validation for expected invalid input. Use explicit paths, predictable fixtures, and deterministic output. Explain working directories and dispose resources reliably, using using declarations or statements where appropriate. Tests should cover normal, boundary, and invalid cases. Learners should read assertion failures and compiler diagnostics as structured text.

Accessibility and delivery

- Keep essential instruction, code, diagnostics, and output in selectable text.
- State braces, angle brackets, nullable markers, question marks, lambda arrows, and generic type syntax explicitly when introduced.
- Teach keyboard-first editor, terminal, build, and test workflows.
- Ensure diagnostics can be navigated by file, line, and severity; explain the difference between build warnings and errors.
- Do not make a visual debugger, GUI designer, or IDE-only feature the sole route to completing an objective.
- Use text descriptions for object relationships and collection contents; do not rely on diagrams or syntax color.
- Allow time for screen-reader verbosity and braille review. Assess reasoning independently of navigation speed.

Test the actual .NET SDK, editor, and assistive technology used by the learner. A successful build does not establish that the workflow is accessible.

Assessment and differentiation

Ask learners to explain type choices, nullable-flow assumptions, collection selection, method contracts, object responsibilities, and exception paths. Require tests for empty collections, missing keys, null values, malformed input, and unavailable files. Evaluate correctness, reasoning, test quality, and communication separately.

Provide scaffolded starter projects and short diagnostics for learners who need additional practice. Extend advanced learners with LINQ, asynchronous methods, interfaces, or dependency design only when these topics reinforce the stated outcomes; do not overload the beginner sequence.

AI-use policy

AI may be used only for personal conceptual understanding and reinforcement. It must not write, complete, autocomplete, repair, refactor, or generate course code or assessment solutions. Keep this rule consistent across demonstrations, homework, and assessment.

Instructor preparation and maintenance

Before the course, verify the .NET 10 SDK and C# 14 compiler baseline, run the repository's C# example checker, and separately test expected behavior and accessibility. The published smoke check is not a substitute for complete behavioral testing. Record exact SDK versions and validation scope. Consult the shared GUI and workbench accessibility guidance before requiring graphical tools.

18.3 C++ Teacher's Guide

Purpose and course context

This guide supports instruction in the C++ curriculum for beginning programmers, including blind and visually impaired learners using screen readers, braille displays, magnification, or keyboard-first workflows. It supplements the complete lessons and topic index.

The course targets **C++20** and GCC, Clang, or MSVC. C++ combines procedural, object-oriented, and generic programming with a large standard library. Instruction should make abstraction useful without hiding object lifetime, value semantics, resource ownership, or the behavior of the compiled program.

Outcomes and instructional sequence

Complete the shared Computing Foundations Primer and Terminal Skills prerequisite, then teach Units 0–11 in order. Learners should be able to build and run a project, interpret compiler diagnostics, use types and expressions, control flow, standard containers, functions, classes, files, exceptions, tests, and an integrated capstone. They should also explain the difference between copying a value, borrowing through a reference, and transferring or sharing ownership.

Use a consistent cycle: objective and vocabulary; code-structure preview; learner prediction; compilation and execution; diagnostic/output review; one deliberate modification; learner explanation. Keep examples small enough that a screen-reader user can navigate the complete construct without losing context.

Toolchain and project workflow

Use a C++20-capable GCC, Clang, or MSVC toolchain. The representative GCC/Clang command is:

```
c++ -std=c++20 -Wall -Wextra -Wpedantic -Werror main.cpp -o main
./main
```

Explain compiler, linker, source file, executable, standard-library headers, and build configuration. Teach project builds consistently and explain when a lesson uses a fragment that is wrapped by a validation harness. Treat warnings as feedback. The course's published example check compiles and links fenced examples; compilation is not equivalent to full behavioral testing or accessibility certification.

Teaching C++ concepts deliberately

Values, references, and object lifetime

Start with values and ordinary function calls, then introduce references and pointers as different forms of access to objects. Teach const references for read-

only access where appropriate. Explain scope, lifetime, temporary objects, and the consequences of returning a reference to an object that no longer exists. Learners should be able to state which function or object owns a resource and how long it remains valid.

Standard containers and algorithms

Prefer standard containers such as std::vector, std::array, std::string, and std::map over raw arrays for ordinary application-level tasks. Explain container size, indexing, iteration, and invalidation rules at the level needed by each lesson. Teach iterators as positions in a sequence and algorithms as operations over ranges. Do not imply that every container has constant-time indexing or that modifying a container can never invalidate an iterator or reference.

Classes, RAII, and resource management

Teach classes as contracts around state and behavior, not as a requirement to model every noun. Introduce constructors, access control, invariants, and member functions. RAII is a core C++ design principle: resource acquisition and release should be tied to object lifetime. Use standard library resource-managing types before introducing manual new/delete. Explain copying and moving as distinct operations, and introduce smart pointers only when ownership relationships justify them. Avoid teaching raw owning pointers as the default.

Exceptions, files, and generic code

Explain exceptions as a control-flow mechanism for exceptional failure, not as a substitute for validating ordinary user input. Teach file streams and check their state. Introduce templates and generic algorithms through concrete use cases, emphasizing type requirements and compiler diagnostics. When diagnostics are long, teach learners to locate the first meaningful error rather than reading every cascading message as an independent failure.

Accessibility and delivery

- Use selectable text for source, build output, test results, and diagnostics.
- Explicitly name punctuation and delimiters, including angle brackets, scope resolution, ampersands, asterisks, braces, and template syntax.
- Teach keyboard navigation by symbol, line, file, and diagnostic; provide non-mouse build and run commands.
- Avoid relying on syntax colors, indentation guides, object inspectors, or graphical debugger panes to communicate program structure.
- Describe container contents, iterator positions, object lifetimes, and ownership in ordered text.
- Provide a text alternative for diagrams and ensure any GUI or game-engine work has an accessible nonvisual route.

- Allow additional time for verbose compiler diagnostics and template errors without reducing the conceptual objective.

Validate the actual compiler/editor/screen-reader combination in the classroom. Accessibility support varies by platform and version.

Assessment

Assess both code behavior and explanation. Ask learners to justify a container choice, trace a function call, identify an object's lifetime, explain a resource's owner, and distinguish an exception from an ordinary validation result. Include tests for empty and non-empty collections, boundary indexes, failed file operations, and invalid input. A compile-only example check is useful but does not establish correct output, resource safety, or complete project behavior.

Differentiation

For learners who need more structure, use short programs, explicit build commands, and one new abstraction at a time. For learners ready to extend the material, compare vector and map tradeoffs, exception guarantees, move semantics, interfaces, and algorithmic complexity. Keep visual memory models optional; use text traces and named states as the shared representation.

AI-use policy

AI may be used only for personal conceptual understanding and reinforcement. It must not write, complete, autocomplete, repair, refactor, or generate course code or assessment solutions. Apply this consistently to instructor and learner work.

Instructor preparation

Verify a C++20 compiler, run the current repository validation script, and separately execute representative examples with expected input and output. Check that build commands and diagnostics are accessible through the learner's tools. Record compiler versions and distinguish compilation/linking evidence from behavioral and assistive-technology testing. See the shared GUI and workbench accessibility guidance before requiring a graphical environment.

18.4 Go Teacher's Guide

Purpose and course context

This guide supports instructors using the Go curriculum with beginning programmers, including blind and visually impaired learners using screen readers,

braille displays, magnification, or keyboard navigation. It supplements the complete lessons and topic index.

Go is a compiled, statically typed language used for command-line tools, network services, infrastructure, and concurrent systems. Its package model, slices, maps, structs, interfaces, explicit error returns, and goroutines shape the course. The curriculum uses a current stable Go toolchain; check the repository's stated baseline before each term.

Outcomes and sequence

Complete the shared Computing Foundations Primer and Terminal Skills prerequisite, then follow Units 0–11. Learners should create a module, build and run programs, interpret compiler diagnostics, use static types and control flow, work with slices and maps, define functions and structs, use interfaces, handle errors, process files and data, test code, and complete the capstone. They should understand Go's concurrency model conceptually even when concurrency is not required for every task.

Use a consistent cycle: objective and vocabulary; code walkthrough; prediction; gofmt/build/run; output and diagnostic review; modification; learner explanation. Keep examples small and avoid introducing concurrency, interfaces, and error handling all at once.

Toolchain and modules

Verify the installed version with `go version`. Create a project with:

```
mkdir courseapp
cd courseapp
go mod init example.com/courseapp
go run .
go test ./...
```

Explain package declarations, imports, module paths, the go.mod file, and the difference between go run and go build. Use gofmt consistently; formatting is part of the standard Go workflow, not a cosmetic optional step. Teach learners to run commands from the module root and interpret package-qualified diagnostics.

Teaching Go's language model

Types, zero values, and conversions

Go is statically typed and uses explicit conversions between many numeric types. Explain zero values and initialization, and contrast a declared variable with an inferred short declaration. Teach that short declarations have scope rules and are not interchangeable with assignment. Do not imply that Go performs arbitrary implicit numeric conversion.

Slices, maps, structs, and interfaces

A slice is a descriptor over an underlying array; its length and capacity are different. Appending may allocate a new backing array, so explain when two slices may share storage. Maps provide key/value lookup but do not promise iteration order. Use structs for named records and interfaces to describe behavior at the point where abstraction is useful. Explain that Go interfaces are satisfied implicitly; avoid teaching interfaces as mandatory ceremony for every type.

Errors and concurrency

Errors are values returned by functions. Teach the pattern of checking an error before using the accompanying result. Do not normalize ignored errors or use panic/recover for ordinary input validation. Introduce goroutines and channels with a clear model of independent execution and communication; discuss cancellation, blocking, and synchronization before assigning multi-goroutine projects. Race freedom cannot be inferred from a successful build or test run without an appropriate race check.

Files and tests

Use explicit paths and deterministic fixtures. Explain the process working directory, os package operations, and error checks. Teach table-driven tests after learners understand functions and assertions. Include normal, boundary, and invalid cases; use temporary directories for file exercises where appropriate.

Accessibility and instructional delivery

- Present source, gofmt output, compiler messages, test output, and command results as text.
- Explicitly describe braces, short declarations, pointers, channel operators, and interface method sets.
- Teach keyboard-first editor navigation and terminal commands; no mouse-only run controls.
- Explain goroutine and channel traces as ordered text events rather than relying on animation or diagrams.
- Ensure learners can navigate test failures by package, file, line, and test name.
- Avoid color-only status indicators and graphical workbenches without an equivalent text workflow.
- Allow additional time for punctuation verbosity and diagnostic navigation.

Verify accessibility with the actual OS, editor, terminal, screen reader, and braille display used in class.

Assessment and differentiation

Assess whether the learner can explain types, slice length versus capacity, map ordering, interface satisfaction, explicit error handling, and concurrency risks. Require tests for empty collections, missing keys, file errors, and malformed input. For concurrency, ask learners to identify possible blocking and shared-state access before execution. Distinguish compilation, smoke execution, expected-output validation, race testing, and accessibility testing; they are different forms of evidence.

Scaffold with single-package programs and sequential code first. Extend learners with additional packages, interfaces, context cancellation, or concurrent workers only after the core data and error models are stable.

AI-use policy

AI may be used only for personal conceptual understanding and reinforcement. It must not write, complete, autocomplete, repair, refactor, or generate course code or assessment solutions.

Instructor preparation

Verify the Go toolchain, run the repository's gofmt/build/smoke-check workflow, and separately inspect expected behavior and accessibility. The current example check does not establish exact output, capstone correctness, race freedom, or assistive-technology certification. Record toolchain versions and validation scope. See the shared accessibility guidance.

18.5 Teacher's Guide: Introduction to Computer Science with Java

For Students with Visual Impairments --- Complete Instructor Edition

How to use this teacher's guide

This guide is a companion for instructors, not a script that must be followed word for word. Read the philosophy, learner-context, and technology sections before the first lesson; use the unit notes while planning; and return to the assessment, differentiation, AI, and quick-reference sections throughout the course. Adapt procedures to the learner's actual assistive technology, operating system, course goals, and prior experience. Confirm tool-specific instructions in current official documentation because editor, screen-reader, and runtime behavior changes over time.

Plan for both programming learning and access to the programming environment. Establish what the student can already do independently, what access

skills need instruction, what supports are already in place, and how the learner prefers to receive feedback. Never assume a diagnosis predicts a student's preferred tools or level of independence.

JDK 25 LTS baseline and current Java conventions

Use **JDK 25 LTS** for the learner materials. JDK 25 is the current LTS release as of October 30, 2026; install its latest security update. JDK 27 is newer but is not the LTS baseline. Teach and assess against standard Java 25, not preview features or vendor-specific extensions. Keep classic class-based program structure in the early lessons even though newer Java releases offer more concise launch forms; this keeps the progression explicit and transferable.

Model current Java conventions consistently: PascalCase types; camelCase fields, methods, parameters, and local variables; UPPER_SNAKE_CASE constants; explicit imports; generics instead of raw types; interfaces such as List/Map in declarations; braces for all control-flow blocks; and meaningful method boundaries. Use String.equals for content equality, never ==. Prefer try-with-resources for files and scanners that own resources. Use var only for local variables when the initializer makes the type obvious; keep explicit types in beginner code. Use descriptive exception handling and avoid swallowing exceptions. Avoid preview features unless a lesson explicitly identifies them.

Table of Contents

1. Philosophy and Why Java
2. Java-Specific Accessibility Challenges
3. Classroom Setup and Configuration
4. Spoken Java Conventions
5. Unit 0--1: Setup and Foundations
6. Unit 2: I/O and the Math Class
7. Unit 3: Control Flow
8. Unit 4: Arrays and ArrayLists
9. Unit 5: Strings and String Processing
10. Unit 6: Methods
11. Unit 7: Classes and OOP
12. Unit 8: Files and Exception Handling
13. Unit 9: Collections, Analysis, and Testing
14. Units 10--11: Projects and Capstone
15. Bridging from Python to Java
16. Assessment Strategies
17. Supporting Diverse Needs
18. Alternative and Supplementary Resources
19. Quick Reference Sheets
20. Teaching AI Tools Responsibly

---

1. Philosophy and Why Java

The Case for Teaching Java to Blind Students

Java is not the easiest language for beginners, and its verbosity creates overhead that requires specific pedagogical attention. Despite this, Java has genuine advantages for visually impaired learners that are often overlooked:

Compile-time error detection. Java's compiler catches type errors, name errors, and structural mistakes before any code runs. A blind student does not need to search through runtime output for subtle type mismatches --- the compiler announces them precisely and immediately with file name, line number, and error description.

Strong, static typing is auditorily informative. When IntelliSense in VS Code suggests methods, it includes the return type: "`double getBalance()` --- returns a double." This type information helps a screen reader user understand what a method does without having to navigate to its definition.

Enforced code structure. Every Java program requires a class; every executable program requires a `main` method with a specific signature. This structure is more overhead to type but less ambiguity about how programs are organized. For a student who cannot visually scan a file for structure, predictable structural conventions reduce cognitive load.

Exception specification. Java's checked exceptions force programmers to declare which exceptions a method can throw. This means a screen reader user reading a method signature gets information about failure modes that Python provides only through documentation.

Enterprise relevance. Java remains a dominant language in enterprise software, Android development, and many backend systems. A student who learns Java learns patterns that transfer directly to professional employment.

---

2. Java-Specific Accessibility Challenges

The Verbosity Problem

Java requires significantly more text than Python to accomplish the same task:

```
// Java: Hello, World
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
```

```
    }
}
```

```
# Python: Hello, World
print("Hello, World!")
```

For a sighted student, extra text is a visual annoyance. For a blind student, it is additional audio before they reach the meaningful code. **Mitigation strategies:**

1. **Code templates.** Provide a starter file the student copies for every new program. They modify only the method body for early lessons. This reduces the typing burden and prevents structural errors while the student is learning concepts.
2. **JShell (Java REPL).** Java includes an interactive shell since Java 9: run `jshell` in the terminal. JShell strips away class and method boilerplate. For exploration and experimentation, JShell is far more accessible than creating files:

   ```
   jshell> int x = 5;
   x ==> 5
   jshell> System.out.println(x * 2);
   10
   ```

3. **Progressive disclosure.** In early lessons, treat `public static void main(String[] args) {` as a formula, not something to understand. Students learn to recognize it by ear. Explain each word only when the concept it relates to (access modifiers, static methods, arrays) is being taught.

Integer Division Surprises

Java's `/` operator performs integer division when both operands are integers:

- `15 / 4` = `3` (not `3.75`)
- `15.0 / 4` = `3.75`
- `(double) 15 / 4` = `3.75`

This is silent --- there is no error or warning. A blind student may not catch a calculation that silently discards its decimal portion. Teach the rule explicitly: **if you want a decimal result, at least one operand must be a `double`.**

The String Equality Trap

`==` on Strings compares memory addresses, not content. Two strings with identical characters in different memory locations are `==` false but `.equals()` true. This never appears in Python; blind students coming from Python are not conditioned to suspect `==` as a source of String bugs.

Teaching approach: Show the broken behavior explicitly. Use `new String("hello") == new String("hello")` which is guaranteed to be `false`

even though the strings are identical. Then show `.equals()` which is `true`. Make `.equals()` for Strings a non-negotiable rule from day one.

Null Pointer Exceptions

`NullPointerException` is the most common Java runtime error. It occurs when you call a method or access a field on a variable that holds `null` (no object reference). Unlike Python's `AttributeError` which gives useful context, Java's NPE message is often terse. Teaching students to recognize and debug NPEs is essential.

---

3. Classroom Setup and Configuration

IDE Configuration

Visual Studio Code with Extension Pack for Java: This is the primary documented setup. The Extension Pack includes Language Support for Java (Red Hat), Debugger for Java, Java Test Runner, Maven for Java, and Project Manager for Java.

Theia IDE: Theia IDE is an alternative with the keyboard- and screen-reader-oriented accessibility workflows expected in this course. Check the Open VSX registry for compatible Java language support, debugging, and test extensions. Not every VS Code extension is available or behaves identically, so verify the student's actual workflow.

Other tools: BlueJ, DrJava, and Eclipse may be considered only when their accessibility and feature set fit the learner and the lesson. JetBrains IDEs (including IntelliJ IDEA) are intentionally not recommended for this introductory course. They are capable products, but their extensive feature set and complexity add unnecessary overhead for beginning programmers.

VS Code Java Specific Settings:

```
{
  "java.completion.enabled": true,
  "java.format.enabled": true,
  "editor.formatOnSave": true,
  "java.saveActions.organizeImports": true,
  "java.completion.importOrder": ["java", "javax", "com", ""],
  "editor.suggestOnTriggerCharacters": true
}
```

Key accessibility setting: Enable "Trigger suggestions" on `.` so that typing `myObject.` causes IntelliSense to announce available methods. This allows audio exploration of an object's API.

JDK Installation Verification

```
java --version      # Runtime version
javac --version     # Compiler version
jshell              # Start REPL (Ctrl+D or /exit to quit)
```

Windows Powershell

```
java --version      # Runtime version
javac --version     # Compiler version
jshell              # Start REPL (Ctrl+D or /exit to quit)
```

All three must work before proceeding. If `jshell` is not found, the student may have installed only the JRE (runtime), not the full JDK. The JDK must be installed.

---

4. Spoken Java Conventions

Extend the Python symbol conventions with Java-specific additions:

Symbol/Pattern	Spoken Form
`public`	”public”
`private`	”private”
`static`	”static”
`void`	”void” (returns nothing)
`new`	”new”
`this.`	”this dot”
`super.`	”super dot”
`@Override`	”at-override annotation”
`// comment`	”double-slash comment”
`/* ... */`	”block comment”
`/** ... */`	”JavaDoc comment”
`String[]`	”String array”
`ArrayList<String>`	”ArrayList of String”
`HashMap<String, Integer>`	”HashMap from String to Integer”
`throws IOException`	”throws IOException”
`System.out.println()`	”sysout” or ”system-out-println”
`System.out.printf()`	”system-out-printf”
`Integer.parseInt()`	”Integer-parse-int”
`(int) x`	”cast to int”
`null`	”null”
`NullPointerException`	”null pointer exception”
`ArrayIndexOutOfBoundsException`	”array index out of bounds”

Reading a method signature:

```
public double calculateArea(double radius) throws IllegalArgumentException
```

"public double method calculateArea, takes a double called radius, may throw IllegalArgumentException"

---

5. Unit 0--1: Setup and Foundations

Lesson 0.1: Installing the JDK

Instructional Cues

- **Use Adoptium (Eclipse Temurin) rather than Oracle JDK.** The Oracle JDK requires account creation and license agreement navigation. Adoptium is fully open-source and simpler to download.
- **Verify the download before running.** Adoptium provides SHA checksums. Teaching students to verify downloads is a security habit worth establishing.
- **Windows installer tip:** The Adoptium `.msi` installer includes a checkbox for "Set JAVA_HOME variable." Ensure this is checked. Many Java tools (Maven, Gradle, some IDEs) require `JAVA_HOME` to be set.

JShell as the First Environment Before any file-based Java, spend time in JShell:

```
$ jshell
|  Welcome to JShell -- Version 17.0.x
|  For an introduction type: /help intro

jshell> 5 + 3
$1 ==> 8

jshell> "Hello " + "World"
$2 ==> "Hello World"

jshell> int x = 42;
x ==> 42

jshell> System.out.println("Value: " + x);
Value: 42
```

JShell gives immediate auditory feedback --- type an expression, hear the result. This is much more accessible for early exploration than the compile-run-read cycle.

JShell commands to teach:

- `/list` --- show all code entered so far

- `/edit` --- open in editor
- `/vars` --- show all variables
- `/methods` --- show all methods defined
- `/exit` --- quit JShell

Thought Experiments

1. *"Python is interpreted --- each line runs immediately. Java compiles to bytecode first, then runs. What does this mean for the kind of errors you encounter, and when? If you have a SyntaxError on line 50, in Python you might not discover it until the program reaches line 50. In Java, when do you discover it?"*
2. *"The Java Virtual Machine (JVM) allows the same Java bytecode to run on Windows, macOS, and Linux. The JVM is different on each platform, but the bytecode is the same. What does this tell you about how Java achieves portability? What alternative approaches could a language designer take?"*
3. *"JShell is a Read-Eval-Print Loop (REPL). Python has one; JavaScript has one; many languages have one. But Java didn't have one until 2016. Why might a language's designers add a REPL 20 years after the language was created? What did programmers do without it?"*
4. *"Java was originally designed for embedded systems (like televisions), then became dominant for web servers, then for Android apps. Each time it succeeded in a new domain, it required extending the language. What does this history suggest about the relationship between a language's design and the problems it eventually solves?"*

Lesson 1.2: Your First Java Program

Instructional Cues **Anatomy reading exercise --- before typing any code:** Read the Hello World program aloud, symbol by symbol, using the pronunciation table:

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

"public class HelloWorld open-brace newline four-spaces public static void main open-paren String array args close-paren open-brace newline eight-spaces System dot out dot println open-paren quote Hello comma World! close-quote close-paren semicolon newline four-spaces close-brace newline close-brace"

This exercise establishes pattern recognition for the boilerplate that the student will type hundreds of times. Making it auditorily familiar early reduces cognitive

load later.

Three `System.out` methods:

```
System.out.println("With newline at end");      // Most common
System.out.print("No newline at end");           // For prompts
System.out.printf("Pi is %.2f%n", Math.PI);     // Formatted
```

For accessible programs, `printf` is extremely valuable --- it produces consistent, predictable output formatting that reads cleanly with a screen reader.

Thought Experiments

1. *"Every Java program requires a `public class` whose name matches the file name. Why does Java enforce this one-to-one correspondence? What would break if you could put multiple public classes in one file?"*
2. *"The `main` method signature is exactly `public static void main(String[] args)`. It cannot be `public void main()` or `static void main(String[] args)`. Why is this signature fixed? What does each word contribute?"*
3. *"Java uses curly braces `{}` to delimit code blocks; Python uses indentation. From an accessibility perspective --- for a blind programmer --- which approach provides better information about structure when read aloud? Consider that braces can be read as 'open block' and 'close block,' while indentation changes require screen reader configuration."*
4. *"`System.out.println()` is long to type but descriptively named. Python's `print()` is short. What naming philosophy does each represent? When does conciseness become ambiguity?"*

Lesson 1.3: Variables and Primitive Data Types

Instructional Cues **Type declaration as specification:** Frame Java's type declarations positively: *"When you write `int score = 85`, you're telling Java: this variable will always hold a whole number. Java then protects you --- if you accidentally try to put a name or a decimal into it, the compiler tells you immediately."*

The eight primitive types --- prioritize four: In this curriculum, students need: `int`, `double`, `boolean`, `char`. The others (`byte`, `short`, `long`, `float`) exist but are rarely needed in beginner programs. Introduce the four primary types fully before acknowledging the others exist.

Type casting demonstration:

```
// Widening (automatic)
int i = 100;
double d = i;   // 100 -> 100.0, automatic
```

```java
// Narrowing (explicit, may lose data)
double pi = 3.14159;
int truncated = (int) pi;  // 3.14159 -> 3, data LOST

System.out.println("Truncated: " + truncated);  // 3, not 4!
```

The truncation behavior (not rounding) is consistently surprising. Demonstrate it with `(int) 3.9` producing `3`, and ask: "Is this rounding or truncating? What is the difference? When would truncation cause a bug?"

Non-Obvious Accessibility Strategies

- **`System.out.printf()` for consistent numeric output:** `printf("Balance: $%.2f%n", balance)` always produces exactly 2 decimal places. Without formatting, `double` values may produce varying numbers of decimal places (`3.0`, `3.14`, `3.1415926535...`) that read differently each time. Consistent formatting makes output more predictable.
- **`Scanner.hasNext___()` methods:** Before calling `scanner.nextInt()`, check `scanner.hasNextInt()`. If the user typed a non-integer, this returns `false` and prevents the exception, allowing graceful re-prompting. For a blind student who may not immediately catch an exception in terminal output, this is defensive programming with real accessibility value.

Thought Experiments

1. *"Java's `int` holds values from about -2.1 billion to +2.1 billion. What happens if you exceed this limit (integer overflow)? The value silently wraps around to the opposite extreme. Name a real-world system where integer overflow has caused a documented failure."*
2. *"Java has `int` (primitive) and `Integer` (object). They hold the same values but behave differently. `int` cannot be null; `Integer` can. `int` cannot be used in a `List<int>` --- you need `List<Integer>`. Why would a language designer make this distinction? What does it cost? What does it protect against?"*
3. *"Java's `char` holds a single Unicode character. `char c = 'A'` stores the letter A. But `char` is also a numeric type --- `(int) 'A'` is 65. What does this dual nature (character and number) tell you about how computers store text internally?"*

---

6. Unit 2: I/O and the Math Class

Lesson 2.2: Scanner for User Input

Instructional Cues The Scanner as a wrapper --- explain the layers:

```
Keyboard
  V produces bytes
System.in (InputStream)
  V raw byte stream
new Scanner(System.in)
  V wraps with text-reading methods
scanner.nextLine(), scanner.nextInt(), etc.
```

This layered model explains why `Scanner` is needed rather than reading from `System.in` directly. It also foreshadows file reading where `new Scanner(new File(...))` replaces `new Scanner(System.in)`.

`nextInt()` / `nextLine()` pitfall --- mandatory demonstration:

```java
Scanner scanner = new Scanner(System.in);
System.out.print("Enter your age: ");
int age = scanner.nextInt();          // Reads "25", leaves "\n" in buffer
System.out.print("Enter your name: ");
String name = scanner.nextLine();     // Reads the leftover "\n" --- gets EMPTY string!
System.out.println("Hello " + name + ", age " + age);
// Output: "Hello , age 25" --- name is empty!
```

The fix:

```java
int age = scanner.nextInt();
scanner.nextLine();          // Consume the leftover newline
String name = scanner.nextLine();   // Now reads properly
```

Run the broken version first. The student should hear the incorrect output and understand why before seeing the fix.

Thought Experiments

1. *"The **`Scanner`** reads from a 'stream' of input. A stream is a sequence of data that flows from a source to a destination. **`System.in`** is the keyboard stream. What other streams exist in a typical program? (Think about file streams, network streams, the output stream.)"*

2. *"After **`scanner.nextInt()`**, there's a leftover newline in the buffer. What IS a buffer? Why does the operating system buffer keyboard input rather than delivering each character the moment a key is pressed?"*

3. *"In GUI (graphical) programs, users fill out forms with text fields and click buttons. There's no **`scanner.nextLine()`**. What mechanism do GUI*

programs use to collect user input? How does event-driven GUI input differ from sequential console input?"

---

7. Unit 3: Control Flow

Lessons 3.1--3.3

Instructional Cues String comparison --- immediate practice:

```
String day = "Wednesday";

// Wrong --- compares memory addresses
if (day == "Wednesday") {
    System.out.println("Middle of the week");  // May or may not print
}

// Correct --- compares content
if (day.equals("Wednesday")) {
    System.out.println("Middle of the week");  // Always correct
}

// Case-insensitive
if (day.equalsIgnoreCase("wednesday")) {
    System.out.println("This works too");
}
```

Create a practice file with 10 comparisons, half using `==` and half using `.equals()`. Ask the student to predict which ones will work correctly before running.

Switch expressions (Java 14+):

Modern Java supports switch expressions that return a value:

```
String dayType = switch (day) {
    case "Monday", "Tuesday", "Wednesday", "Thursday", "Friday" -> "Weekday";
    case "Saturday", "Sunday" -> "Weekend";
    default -> "Unknown";
};
System.out.println(day + " is a " + dayType);
```

This eliminates the fallthrough problem and is more concise. Teach both traditional `switch` (for reading legacy code) and switch expressions (for writing new code).

Thought Experiments

1. *"Java uses `&&` for AND, `||` for OR, `!` for NOT. Python uses the words `and`, `or`, `not`. Which approach is more readable when heard aloud by a screen reader? Is there a meaningful accessibility argument for one over the other?"*

2. *"The enhanced for loop `for (String item : list)` is simpler to write than the traditional `for (int i = 0; i < list.size(); i++)`. But sometimes you need the index. Name three operations you can do with the traditional loop that you cannot do (directly) with the enhanced loop."*

3. *"Java's `switch` statement traditionally falls through from one case to the next unless you write `break`. This 'feature' has caused countless bugs. Modern Java (14+) has switch expressions that don't fall through. What does this change tell you about how programming language design evolves in response to real-world usage patterns?"*

4. *"Short-circuit evaluation: `if (obj != null && obj.getValue() > 0)` --- if `obj` is null, the second condition is never evaluated. This prevents a NullPointerException. How does this behavior make certain coding patterns possible that would otherwise crash?"*

---

8. Unit 4: Arrays and ArrayLists

Lesson 4.1: Arrays

Instructional Cues The fixed-size limitation --- demonstrate, don't just describe:

```
String[] fruits = new String[3];
fruits[0] = "apple";
fruits[1] = "banana";
fruits[2] = "cherry";
fruits[3] = "date";   // ArrayIndexOutOfBoundsException!
```

Let the student hear the exception and analyze the message: `Index 3 out of bounds for length 3`. Ask: "The message gives you two numbers. What are they? What does each tell you?"

`Arrays.toString()` as mandatory print method:

The first time `print(fruits)` produces `[Ljava.lang.String;@7a5d012b`, stop and explain: "This is the memory address of the array, not its contents. Java arrays don't have a built-in 'print nicely' behavior. Always use `Arrays.toString(fruits)` or loop through manually."

The `for` vs `for-each` tradeoff --- make it concrete:

```
// for-each: simple, clear, but no index
for (String fruit : fruits) {
```

```
    System.out.println(fruit);
}

// traditional: more complex, but gives index
for (int i = 0; i < fruits.length; i++) {
    System.out.println(i + ": " + fruits[i]);
}

// The numbered version is more accessible!
```

For blind students, numbered output (showing both index and value) is generally more useful than unnumbered output. Teach the traditional `for` loop for array printing specifically because it enables indexed output.

Thought Experiments

1. *"Arrays are stored in contiguous memory --- all elements in a row, one after another. This is why accessing by index is fast: the address of element `i` is just the start address plus `i × element size`. Why does this contiguous storage make inserting into the middle of an array slow? What must happen to all the elements after the insertion point?"*
2. *"If you create `String[] arr = new String[5]`, what are the initial values of `arr[0]` through `arr[4]`? Now create `int[] nums = new int[5]` --- what are the initial values? Why are the defaults different between String arrays and int arrays?"*
3. *"Two-dimensional arrays in Java are 'arrays of arrays.' This means each row can have a different length --- a 'jagged array.' When would a jagged array be more appropriate than a rectangular 2D array? Give a real-world data example."*

Lesson 4.2: ArrayLists

Instructional Cues Autoboxing --- explain the invisibility:

```
ArrayList<Integer> numbers = new ArrayList<>();
numbers.add(42);        // 42 (int) is auto-boxed to Integer(42)
int first = numbers.get(0);  // Integer(42) is auto-unboxed to 42
```

This happens invisibly but has a performance cost. For the course, tell students: "Use `ArrayList<Integer>` when you need a flexible list of integers. The boxing/unboxing is automatic but slightly less efficient than an array of `int`."

Removal by index vs. by value:

```
ArrayList<String> list = new ArrayList<>(Arrays.asList("a", "b", "c", "b", "d"));
list.remove(1);            // Removes by INDEX --- removes "b" at position 1
```

```java
list.remove("b");          // Removes by VALUE --- removes first "b" found
list.removeAll(Arrays.asList("b", "c"));  // Removes all matching values
```

Ask the student: "If I call `list.remove(0)` on a list of integers `[10, 20, 30]`, does this remove the element at index 0 (which is `10`) or remove the value `0` (which doesn't exist)? How does Java decide?" (Answer: removes index 0, because `0` is an `int`, not an `Integer`.)

Thought Experiments

1. *"An `ArrayList` doubles its internal capacity when it runs out of space. If you start with capacity 16 and keep adding elements, when does reallocation occur? Trace the capacity: 16 -> 32 -> 64 -> 128... What is the total cost (in element moves) of adding 1000 elements to an initially empty ArrayList this way? Is this efficient?"*
2. *"Java generics `ArrayList<String>` enforce type safety at compile time. Python lists can hold any types mixed together. When is a typed list (Java style) better? When is a mixed list (Python style) more flexible? Which approach produces more reliable programs?"*

---

9. Unit 5: Strings and String Processing

Instructional Cues String immutability --- the Java version:

In Java, Strings are immutable for the same reason as Python. But Java's `String` object is also **interned** --- the JVM may reuse the same `String` object for two variables holding the same literal value. This is why `==` sometimes works for String comparison (when both refer to the same interned object) but is not reliable.

`StringBuilder` motivation:

```java
// Inefficient --- creates 100,000 String objects
String result = "";
for (int i = 0; i < 100000; i++) {
    result += "x";
}

// Efficient --- modifies a single buffer
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 100000; i++) {
    sb.append("x");
}
String result = sb.toString();
```

The `+` in a loop creates a new String object on every iteration, then discards the old one. For small loops this is fine; for large loops it's a significant performance problem. `StringBuilder` maintains a mutable buffer.

Thought Experiments

1. *"String concatenation with `+` is convenient but expensive in loops. In Python, the equivalent is also discouraged --- use `''.join(list)` instead. What does this common advice across two languages tell you about how immutable strings interact with loops?"*
2. *"Java's `String.format()` and `printf()` use format specifiers like `%s`, `%d`, `%f`. Python uses f-strings with `{}`. What are the accessibility tradeoffs of each approach? Which reads more naturally when heard aloud? Which is easier to type?"*
3. *"Regular expressions are a tiny language inside a programming language. `\\d+` matches one or more digits. `[a-zA-Z]+` matches one or more letters. Why do programmers use regular expressions despite their difficult-to-read syntax? What would the alternative be for complex pattern matching?"*

---

10. Unit 6: Methods

Instructional Cues Method overloading --- auditory navigation:

```
public static int add(int a, int b) { return a + b; }
public static double add(double a, double b) { return a + b; }
public static int add(int a, int b, int c) { return a + b + c; }
```

When IntelliSense shows overloads in VS Code, pressing `Shift+Alt+F2` (Accessible View) lists them all. Teach students to use this to discover available overloads.

`void` vs. return type --- make the distinction tangible:

```
// void: for side effects only
public static void printGreeting(String name) {
    System.out.println("Hello, " + name + "!");
}

// Return type: for computation
public static double calculateTip(double billAmount, double rate) {
    return billAmount * rate;
}

// Using them:
printGreeting("Alice");                    // Standalone call
```

```java
double tip = calculateTip(50.00, 0.18);       // Must capture return value
System.out.printf("Tip: $%.2f%n", tip);
```

"`void` methods exist to DO something. Return-type methods exist to CALCULATE something and give you the result. Ask: 'Do I need to use the output later in my code?' If yes, it needs a return type."

Thought Experiments

1. *"Java methods can throw exceptions: `public double sqrt(double n) throws IllegalArgumentException`. This signature tells callers that the method might fail in a specific way. Python has no equivalent syntax. What is the programmer's obligation when they see `throws` in a method signature?"*
2. *"Static methods belong to the class, not to any instance: `Math.sqrt(25)` --- you don't need a `Math` object. Instance methods belong to an object: `myString.length()`. What is the conceptual difference? When should a method be static vs. instance?"*
3. *"Method parameters in Java are 'pass by value' for primitives and 'pass by reference' for objects. For primitives: the method gets a copy, and modifying the parameter doesn't affect the original. For objects: the method gets the reference, and modifications to the object's fields DO affect the original. Design a test to confirm this behavior for both cases."*
4. _"Recursion: a method that calls itself. `int factorial(int n) { return n == 0 ? 1 : n _ factorial(n-1); }`. Every recursive solution has an iterative equivalent. When is the recursive version clearer? What is the risk of deep recursion (many nested calls)?"*

---

11. Unit 7: Classes and OOP

Extended Instructional Cues

The mandatory `__str__` equivalent in Java --- `toString()`:

Every student-defined class must override `toString()`. Without it, `System.out.println(myObject)` produces an unreadable string. The `@Override` annotation makes this explicit:

```java
@Override
public String toString() {
    return String.format("Dog[name=%s, age=%d]", name, age);
}
```

Require this in every class from Unit 7 onward. Explain: "When your screen reader reads `System.out.println(myDog)`, it reads whatever `toString()` re-

turns. If you don't define `toString()`, you'll hear a memory address. That's useless."

JavaDoc comments as accessible documentation:

Teach this format for every method:

```
/**
 * Calculates the area of this circle.
 *
 * @return the area as a double, in square units
 * @throws IllegalArgumentException if radius is negative
 */
public double getArea() {
    if (radius < 0) throw new IllegalArgumentException("Radius cannot be negative");
    return Math.PI * radius * radius;
}
```

When IntelliSense shows this method in VS Code, pressing `Shift+Alt+F2` reads the JavaDoc aloud. This makes the student's own code navigable via audio --- a profound accessibility benefit.

The VS Code symbol navigator: `Ctrl+Shift+O` opens a dropdown listing all methods in the current file. Type a few letters to filter. This allows jumping to any method by name --- much more efficient than arrow-key navigation through a long file.

Access modifiers --- enforce from day one:

Modifier	Rule	Reason
`private` on fields	Always	Encapsulation; prevents direct manipulation
`public` on getters/setters	When needed	Controlled access
`private` on helper methods	Yes	Internal implementation detail
`public` on methods the user calls	Yes	Interface to the class

Establish this as a non-negotiable habit, not a preference. *"Private fields, public interface. Every time."*

Thought Experiments

1. *"Encapsulation means the internal state of an object is private. Only its public methods can access or modify it. Why would you want to prevent direct access to `account.balance`? What bugs could occur if balance were public?"*

2. *"Inheritance creates a hierarchy. A `SavingsAccount` extends `BankAccount`. Every `SavingsAccount` is-a `BankAccount`, but not every `BankAccount` is-a `SavingsAccount`. What are the type-safety implications of this? If you have a `BankAccount` variable, can it hold a `SavingsAccount` object?"*

3. *"Java interfaces define contracts --- a list of methods that any implementing class must provide. A class can implement multiple interfaces but can only extend one class. Why this asymmetry? What problem does single inheritance solve? What does it prevent?"*

4. *"Design Pattern: the Factory Method creates objects without specifying the exact class. Instead of `new Dog()`, you'd call `AnimalFactory.create("dog")`. What is the advantage of this indirection? When would it be useful?"*

5. *"Object-oriented programming was supposed to model the real world. A `Dog` is-a `Animal`. But real-world relationships are complex --- a `FlyingFish` is both a `Fish` and can fly like a `Bird`. How does Java's single-inheritance model handle this situation? What alternatives exist?"*

6. *"Polymorphism means one interface, many implementations. `Animal[] animals = {new Dog(), new Cat(), new Bird()}; for (Animal a : animals) { a.speak(); }` --- each animal speaks differently. Why is this more flexible than checking `instanceof` and calling different methods explicitly?"*

Alternative Resources

- **"Head First Design Patterns" by Freeman, Robson et al.:** Visual book, but audio-describable concepts. Covers fundamental OOP design patterns. For advanced students.
- **"Effective Java" by Joshua Bloch:** The definitive book on Java best practices. Dense but invaluable for understanding WHY Java is designed the way it is.
- **Oracle Java Tutorials (docs.oracle.com/javase/tutorial/):** Comprehensive, plain HTML, fully accessible. Covers OOP concepts with extensive examples.

---

12. Unit 8: Files and Exception Handling

Instructional Cues Checked vs. unchecked exceptions --- the core distinction:

Type	Examples	Must Handle?	Why?
Checked	`IOException`, `FileNotFoundException`, `SQLException`	Yes --- compiler enforces	These are recoverable; programmer should handle them
Unchecked (RuntimeException)	`NullPointerException`, `ArrayIndexOutOfBoundsException`, `IllegalArgumentException`	No --- optional	Usually indicate programming errors; may not be recoverable

"Checked exceptions are like warning labels: 'This operation can fail. You must decide what to do if it does.' Unchecked exceptions are usually bugs --- the program did something wrong."

Try-with-resources --- teach first, manual close second:

```
// Modern: try-with-resources (always teach this first)
try (BufferedReader reader = new BufferedReader(new FileReader("data.txt"))) {
    String line;
    while ((line = reader.readLine()) != null) {
        System.out.println(line);
    }
} catch (IOException e) {
    System.out.println("Error reading file: " + e.getMessage());
}
// reader is automatically closed here, even if an exception occurred

// Legacy: manual close (show this so students understand what try-with-resources does)
BufferedReader reader = null;
try {
    reader = new BufferedReader(new FileReader("data.txt"));
    // ... use reader
} catch (IOException e) {
    System.out.println("Error: " + e.getMessage());
} finally {
    if (reader != null) {
        try { reader.close(); } catch (IOException e) { /* ignore */ }
    }
}
```

Show the legacy version to explain why try-with-resources exists and what it simplifies.

Thought Experiments

1. *"Checked exceptions force you to handle failures at compile time. Critics argue this creates 'exception chaining hell' --- every method signature becomes littered with `throws` clauses. Defenders argue it makes failure modes explicit and programs more reliable. Which side do you find more persuasive? Why?"*

2. *"Java's `Path` and `Files` classes (java.nio.file) are a modern replacement for the older `File` class. The older `File` class's `delete()` method returns a `boolean` (true/false) instead of throwing an exception. What is the danger of reporting failure via a return value? What might a programmer accidentally do wrong?"*

3. *"A `NullPointerException` is called 'The Billion Dollar Mistake' by its inventor Tony Hoare. Explain why null references cause so many bugs. Newer languages (Kotlin, Swift) have built-in null safety. What would Java code look like if null didn't exist?"*

---

13. Unit 9: Collections, Analysis, and Testing

JUnit Testing Deep Dive

Instructional Cues **Test output is inherently accessible:** JUnit produces plain text output: `.` for each passing test, `F` for failures, with descriptive failure messages. This is one of the most screen-reader-friendly outputs in the entire course. Let students experience testing by running a test suite and listening to the results before writing their own tests.

Test naming convention --- sentence style:

```
@Test
public void calculateArea_withPositiveRadius_returnsCorrectValue() { }

@Test
public void calculateArea_withZeroRadius_returnsZero() { }

@Test
public void calculateArea_withNegativeRadius_throwsException() { }
```

The naming convention `methodName_condition_expectedBehavior` makes test names read as complete sentences when the test runner announces them. This is both good practice and an accessibility benefit --- each test announcement tells you exactly what was tested.

`assertThrows` --- test that exceptions work correctly:

```
@Test
public void withdraw_withInsufficientFunds_throwsException() {
    BankAccount account = new BankAccount(100.0);
```

```
    assertThrows(InsufficientFundsException.class, () -> account.withdraw(200.0));
}
```

Testing for exceptions is as important as testing for correct outputs. Many beginners only test the "happy path."

Thought Experiments

1. *"Test-driven development (TDD) says: write the test first, then write the implementation. How does writing a test first change the way you think about a method? What is forced to be clear before you write a line of implementation?"*
2. *"Mocking means replacing real dependencies (like a database or file system) with fake versions for testing. Why would you want to test a method that reads from a database without using a real database? What complexity does mocking introduce?"*
3. *"A project has 100% test coverage --- tests execute every line. Is the software correct? Name a bug that 100% coverage would not catch. (Hint: think about a bug that only appears with certain combinations of inputs.)"*

---

14. Units 10--11: Projects and Capstone

Java Project Architecture

For the capstone, encourage a clean package structure:

```
src/
|--- com/
|   +--- student/
|       +--- datamanager/
|           |--- model/
|           |   +--- Contact.java
|           |--- storage/
|           |   |--- DataManager.java
|           |   +--- JsonStorage.java
|           |--- ui/
|           |   +--- ConsoleUI.java
|           +--- Main.java
test/
+--- com/
    +--- student/
        +--- datamanager/
            |--- model/
            |   +--- ContactTest.java
```

```
+--- storage/
    +--- DataManagerTest.java
```

Navigating packages in VS Code: `Ctrl+Shift+E` for Explorer (shows package tree), `Ctrl+P` to open any file by name, `Ctrl+Shift+O` for symbols within the current file.

IDE-Generated Code Accessibility

VS Code and compatible Java IDE extensions can generate getters, setters, constructors, and toString. Teach these as time-savers, but require students to read and understand all generated code:

- `Ctrl+Shift+P` -> "Java: Generate Getters and Setters"
- `Ctrl+Shift+P` -> "Java: Override/Implement Methods"
- `Ctrl+Shift+P` -> "Java: Generate toString"

Generated code is correct but often not styled for accessibility (missing JavaDoc, generic names). Require students to add JavaDoc and verify `toString()` output is meaningful when heard.

Thought Experiments

1. *"Java in enterprise software is often criticized for having too much 'boilerplate' --- code that exists for structural reasons rather than to express logic. How does this boilerplate affect the experience of maintaining a large Java codebase? What tools and patterns does the Java ecosystem provide to reduce it?"*
2. *"Your capstone application stores data in a file. If your team adds a new field to the data class, existing saved files no longer match the new format. This is a 'schema migration' problem. How would you design your storage layer to handle this gracefully?"*

---

15. Bridging from Python to Java

For courses that teach Python before Java, use these comparison thought experiments to bridge the languages:

Syntax bridge table:

Python	Java	Notes
`print("Hello")`	`System.out.println("Hello");`	
`x = 5`	`int x = 5;`	Java requires type declaration and semicolon
`if condition:`	`if (condition) {`	Parens around condition; brace instead of colon

Python	Java	Notes
`elif`	`else if`	Two words in Java
`for item in list:`	`for (Type item : list) {`	Java requires type
`def func(x):`	`public static void func(int x) {`	Return type and access modifier required
`return x`	`return x;`	Semicolon
`# comment`	`// comment`	
`None`	`null`	
`True/False`	`true/false`	Lowercase in Java
`len(list)`	`list.size()`	Method call, not function
`list.append(x)`	`list.add(x)`	
`dict["key"]`	`map.get("key")`	
`try/except`	`try/catch`	
`import math`	`import java.lang.Math;` (or use `Math.` directly)	

Thought experiments for bridging:

1. *"In Python you write `x = 5` and Python figures out the type. In Java you write `int x = 5` and declare the type explicitly. Which approach do you prefer after using both? Which one would you choose for a large team project? For a quick experiment?"*
2. *"Python lists can hold any mixture of types: `[1, 'hello', True, 3.14]`. Java's `ArrayList<String>` can only hold Strings. What bugs does Python's flexibility enable? What bugs does it prevent compared to Java?"*
3. *"Python functions are first-class objects --- you can store them in variables, pass them as arguments. In Java (before lambdas), this was much more complex. Java added lambdas in 2014. What does it mean for a language to add a major feature 19 years after it was created?"*

---

16. Assessment Strategies

Alternative Assessment Forms

Code review presentations: The student explains their code out loud, as if presenting to a team. The teacher asks questions about design decisions. This tests understanding more authentically than multiple-choice questions.

Bug-hunting assessments: Provide a working program with 5--8 intentional bugs of different types. The student must find and fix them using only their screen reader and development tools. This directly tests real-world debugging skill.

Design walkthroughs: Give the student a problem and ask them to design the solution --- class structure, methods, data structures --- without writing any code. Assess the quality of the design, not the typing speed.

Pair programming sessions: Two students work on one computer. One dictates code; the other types. Roles swap every 10--15 minutes. This surfaces communication and reasoning skills that solo assessments miss.

---

17. Supporting Diverse Needs

For Braille Display Users

Java's verbose syntax can be particularly burdensome on a braille display with limited width. Strategies:

- Keep lines under 79 characters (PEP 8 applies to style; the Java equivalent is generally under 100).
- Use shorter variable names for very long class hierarchies where the full name is unwieldy.
- Test the learner's chosen editor with their braille display and screen reader. Prefer the documented VS Code or Theia workflows and avoid adding IDE complexity before it is needed.

For Students New to Screen Readers

If a student is still developing screen reader proficiency:

- Begin with JShell instead of file-based programming. Single-line interactions provide immediate feedback.
- Use pair work more extensively until independent navigation is fluent.
- Reduce the number of new concepts per session until screen reader fluency stabilizes.

---

18. Alternative and Supplementary Resources

Textbooks and Reading

Resource	Level	Format	Accessibility
Introduction to Programming Using Java (Eck, free online)	Beginner-Intermediate	Free HTML/PDF	HTML version excellent; comprehensive
Big Java: Early Objects (Horstmann)	Beginner	Print/ebook	Standard university text; clear explanations

Resource	Level	Format	Accessibility
Effective Java, 3rd Ed. (Bloch)	Intermediate - Advanced	Print/ebook	Essential for understanding Java's design philosophy
Head First Java (Sierra, Bates)	Beginner	Print	Very visual; but text portions accessible; engaging writing style
Java: The Complete Reference (Schildt)	Reference	Print/ebook	Encyclopedic; good for looking up specific features
Oracle Java Tutorials (docs.oracle.com)	Beginner - Advanced	Free HTML	Excellent HTML accessibility; definitive
Java Documentation (javadoc)	Reference	Free HTML	Standard library docs; fully screen-reader-accessible

Online Courses and Platforms

Platform	Course	Accessibility	Notes
edX / MITx 6.092	Introduction to Programming in Java	High	MIT course; video transcripts available
Coursera	Java Programming (Duke)	Medium	Free audit available
Codecademy	Learn Java	Partial	Some UI inaccessible; editor readable
MOOC.fi (University of Helsinki)	Java Programming MOOC	High	Text-based exercises; excellent accessibility
CodingBat Java	Java practice problems	High	Plain HTML; no complex UI
Exercism Java Track	Structured exercises with mentoring	High	Terminal-based; excellent
LeetCode	Algorithm problems	Medium	Code editor accessible; UI moderate

MOOC.fi is especially recommended for blind students --- the University of Helsinki's Java course has no-frills HTML exercises and very clear progression. It is one of the most accessible Java learning platforms available.

Video and Audio Resources

Resource	Description	Accessibility
CS50 AP (Harvard)	AP Computer Science A course using Java	Captions and transcripts
Derek Banas Java Tutorials (YouTube)	Concise concept-by-concept tutorials	Captions available
Cave of Programming Java (YouTube/website)	Beginner Java series	Captions; text summaries
thenewboston Java (YouTube)	Large beginner series	Captions available
Coding with John (YouTube)	Clear explanations of Java concepts	Good pacing for beginners

---

19. Quick Reference Sheets

Java Compilation and Running

```
# Compile a single file
javac HelloWorld.java    # Creates HelloWorld.class

# Run compiled program
java HelloWorld          # No .class extension!

# Compile with output directory
javac -d bin src/HelloWorld.java

# Run JShell
jshell                   # Interactive REPL
/exit                    # Exit JShell

# Run tests (if the project includes Maven Wrapper)
./mvnw test

# Run tests (if using Gradle)
./gradlew test
```

Windows Powershell

```
# Compile a single file
javac HelloWorld.java    # Creates HelloWorld.class
# Run compiled program
java HelloWorld          # No .class extension!
# Compile with output directory
javac -d bin src/HelloWorld.java
```

```
# Run JShell
jshell                    # Interactive REPL
/exit                     # Exit JShell
# Run tests (if the project includes Maven Wrapper)
.\mvnw.cmd test
# Run tests (if using Gradle)
.\gradlew.bat test
```

Error Type Quick Reference

Error Type	Common Cause	First Check
`cannot find symbol`	Misspelled name or missing import	Spelling; check imports
`incompatible types`	Assigning wrong type or missing cast	Type of both sides of assignment
`';' expected`	Missing semicolon	End of the line indicated
`reached end of file while parsing`	Missing `}`	Count open braces vs close braces
`non-static method cannot be referenced from a static context`	Calling instance method from `main` without object	Need to create an object first
`NullPointerException`	Calling method on null reference	Which variable is null
`ArrayIndexOutOfBoundsException`	Index beyond array length	Print length and the index used
`ClassCastException`	Invalid cast	Check `instanceof` before casting
`StackOverflowError`	Infinite recursion	Verify base case
`NumberFormatException`	`parseInt()` on non-numeric string	Check input string before parsing

Scanner Method Reference

```
Scanner scanner = new Scanner(System.in);

String line   = scanner.nextLine();     // Entire line as String
String word   = scanner.next();         // Single token (whitespace-delimited)
int n         = scanner.nextInt();      // Integer
double d      = scanner.nextDouble();   // Double
boolean b     = scanner.nextBoolean();  // Boolean (true/false)
```

```java
// Safe versions with type checking
if (scanner.hasNextInt()) {
    int n = scanner.nextInt();
}

// After nextInt(), consume leftover newline:
scanner.nextLine();  // ALWAYS do this before reading a String

// From file
Scanner fileScanner = new Scanner(new File("data.txt"));
// Same methods work; wrap in try-catch for FileNotFoundException

scanner.close();  // Always close when done
```

Accessible Output Template

```java
// Numbered list output
System.out.println("--- Contacts ---");
for (int i = 0; i < contacts.size(); i++) {
    System.out.printf("%d. %s%n", i + 1, contacts.get(i));
}
System.out.printf("Total: %d contacts%n", contacts.size());

// Menu
System.out.println("\n--- Main Menu ---");
System.out.println("1. Add contact");
System.out.println("2. View all contacts");
System.out.println("3. Search contacts");
System.out.println("4. Delete contact");
System.out.println("5. Save and quit");
System.out.println("-----------------");
System.out.print("Enter option (1-5): ");

// Confirmation messages
System.out.printf("Contact '%s' added successfully.%n", name);
System.out.printf("Found %d matching records.%n", count);
System.out.println("Data saved to contacts.json.");
System.out.println("Error: Name field cannot be empty. Please try again.");
```

20. Teaching AI Tools Responsibly

AI coding assistants can explain compiler messages, offer hints, compare designs, or suggest test cases. They can also invent Java APIs, use language features unsupported by the configured JDK, omit imports, produce incompatible types, mishandle exceptions, or create unnecessary class hierarchies. Treat generated

answers as unverified drafts. The course must remain teachable without paid AI access or a personal account.

Instructor use of AI during preparation

Instructors may use approved AI tools to brainstorm examples, draft alternate explanations, suggest test cases, or identify likely misconceptions. The instructor remains accountable for correctness and accessibility. Verify APIs and language features against the target JDK and official Java documentation; compile and test examples; inspect dependencies and build instructions; and test the workflow with the intended access technology. Never upload identifiable student work, accommodation plans, credentials, assessment answers, or confidential school information. Do not use AI-generated feedback as the sole basis for grading. Disclose substantive AI assistance when institutional policy or the publication process requires it.

Learning goals and appropriate uses

Students should explain that generated responses can be plausible and incorrect, ask focused questions, verify claims against official Java documentation and compiler feedback, identify unfamiliar APIs and dependencies, review code for accessibility and security, and disclose assistance when required. When permitted, AI may explain a diagnostic, ask guiding questions, rephrase a concept, suggest tests, or critique a proposed class design.

Java-specific risks

Review generated Java for:

- Language features or APIs requiring a newer JDK than the course environment.
- Incorrect imports, package declarations, class/file names, or `main` signatures.
- Incompatible types, unsafe casts, incorrect generics, and confusion between primitive and reference types.
- Incorrect use of `==` for String content comparison instead of `.equals()`.
- Null handling that can cause `NullPointerException`.
- Incorrect resource handling, file paths, exception declarations, or broad catch blocks.
- Confusion among `Scanner.next()`, `nextLine()`, and numeric input, including the pending-newline issue.
- Unnecessary frameworks, Maven/Gradle dependencies, or architecture beyond the lesson.
- Long declarations, deeply nested code, and IDE-generated scaffolding that is difficult to navigate.

Compile with the course's JDK, read the complete diagnostic, and run tests. Check API and language claims in official Java documentation and the Java

Language Specification.

Screen-reader and IDE review

Review whether the learner can navigate package and class declarations, fields, constructors, methods, and `main`; identify braces and nested blocks; understand method names, parameters, return types, and access modifiers; and access diagnostics as text with reliable file and line references. Prefer a small number of clear classes and methods over elaborate architecture. Explain generated boilerplate rather than treating it as magic.

Privacy, security, copyright, and equity

Do not enter student records, credentials, API keys, private repositories, or confidential school information into public AI systems. Follow school policy, approved-tool lists, age requirements, and data-protection obligations. Treat generated shell commands, build files, and dependency recommendations as untrusted. Verify library coordinates and maintainers before adding dependencies. Do not assume generated code is original or correctly licensed. Provide a non-AI alternative and discuss assumptions about visual IDE workflows and graphical output.

Verification protocol: Ask – Inspect – Verify – Adapt – Explain – Disclose

1. **Ask:** state the concept or error and what the student understands.
2. **Inspect:** identify unfamiliar types, APIs, dependencies, and assumptions.
3. **Verify:** consult official documentation, compile with the target JDK, and test normal, boundary, and invalid cases.
4. **Adapt:** simplify, correct, secure, and format for the learner's level and access workflow.
5. **Explain:** have the learner explain each class, method, type, and exception path and make a modification independently.
6. **Disclose:** record the tool and its contribution when required.

Activity-specific policy

- **Practice:** hints, explanations, and test ideas may be allowed; full solutions should be reserved for explicit code-review activities.
- **Projects:** define permitted AI assistance in advance; require disclosure, tests, and an explanation of design decisions.
- **Individual assessments:** AI should not perform the skill being measured unless the assessment explicitly evaluates AI-assisted development or an accommodation requires it.
- **Accessibility:** screen readers, braille displays, magnification, and approved access tools are not prohibited answer-generation tools.

Discussion prompts

- An AI example compiles on a newer JDK but not on class machines. How can the learner identify the version dependency?
- An assistant uses == to compare two strings. How can the student demonstrate the defect?
- A generated program catches every exception and prints "Something went wrong." Why is that inadequate?
- How can a learner distinguish understanding an object-oriented design from merely running it?

---

21. Comprehensive Instructional Planning and Course Implementation

This section consolidates practices that apply across the philosophy, learner-context, classroom setup, spoken-code, unit, assessment, differentiation, and resource sections above. Use it as a course-planning reference from intake through capstone evaluation.

21.1 Course planning and alignment

Map each unit to prerequisites, learning objectives, practice opportunities, and evidence of mastery. Do not assume every unit needs equal time.

- Identify outcomes: problem decomposition, code reading and writing, data representation, control flow, functions or methods, debugging, testing, and communication.
- Separate programming prerequisites from access-tool prerequisites.
- Record the operating system, editor/IDE, runtime, terminal, screen reader, braille display, magnification, and adaptive input technology used.
- Identify essential concepts, enrichment topics, and material that can be deferred without breaking later lessons.
- Plan checkpoints after each major concept group and adjust pacing using student evidence.
- Maintain a map linking each lesson to objectives, practice, assessment, accessibility considerations, and likely misconceptions.

A suggested progression is environment and navigation; computing foundations; expressions and data; input/output; decisions and loops; collections; functions or methods; object-oriented concepts; files and errors; testing; projects; capstone. Preserve prerequisites when reordering.

21.2 Readiness and learner partnership

Ask about preferred access methods, prior programming experience, keyboard navigation, file management, and what makes technical instruction usable. Ask about functional access requirements rather than requesting unnecessary diagnostic details.

Use a low-stakes baseline activity to observe how the learner opens and navigates a source file, moves by line/word/character/symbol, creates and saves a file, runs a command, locates output, and reads a diagnostic. Record strengths and supports, not just deficits. Revisit the baseline to recognize growth. Keep the learner central to decisions about tools and workflow.

21.3 Repeatable lesson architecture

1. **Purpose:** state the problem the lesson addresses.
2. **Prerequisite retrieval:** revisit the prior concept needed today.
3. **Vocabulary:** define technical terms and spoken symbol conventions.
4. **Explicit model:** demonstrate a small runnable example and narrate navigation and code changes.
5. **Trace:** read the code structurally and predict behavior.
6. **Guided practice:** solve a similar problem with prompts and fading support.
7. **Independent transfer:** apply the concept to a new requirement.
8. **Debug and reflect:** examine an intentional error or unexpected result.
9. **Exit check:** collect an explanation, prediction, code change, or test.

State one navigation action at a time, allow the learner to complete it, and confirm the result before continuing. Provide source files and commands in advance when possible. Narrate all actions in shared-screen demonstrations and provide an equivalent text-based route.

21.4 Explicit instruction for accessible code navigation

Teach file structure before reading every line. Identify current focus and how to return to a known location. Teach line, word, character, symbol, selection, search, and go-to-line navigation, plus navigation to functions, methods, classes, and diagnostics. Explain indentation and block boundaries. Use manageable line lengths and avoid unnecessarily nested expressions. Give students time to read diagnostics and output independently. Do not take over the keyboard without explaining the action and obtaining the learner's agreement.

21.5 Language-specific instructional emphasis: Java

Java's explicit types, compilation, and class structure require deliberate sequencing:

- Teach the JDK, compiler, runtime, source file, class, and `main` method before IDE-generated projects.

- State the target JDK version and explain when features require a newer release.
- Contrast primitive values and reference types, assignment and comparison, conversions, and integer versus floating-point division.
- Teach String content comparison with `.equals()`, not `==`, and explain null references.
- Model `Scanner` input carefully, especially the newline remaining after token or numeric reads.
- Teach array bounds, ArrayList operations, generic types, and collection selection with small examples.
- Separate method signature, parameter, argument, return type, and method call. Explain `static` versus instance behavior.
- Teach constructors, encapsulation, inheritance, and composition as design choices, not just syntax.
- Read compiler diagnostics before running; distinguish compile-time errors, runtime exceptions, and logic defects.
- Keep build tools and external dependencies out of early lessons unless required. Explain Maven or Gradle when introduced.

21.6 Misconceptions and diagnostic teaching

Ask learners to predict behavior, explain their prediction, run a small experiment, and revise their mental model.

Common issue	Instructional response
Confusing assignment with comparison	Contrast operators and trace values before and after execution.
Treating input as already numeric	Identify input type, convert explicitly, and test valid and invalid values.
Off-by-one loop or index errors	Trace first, typical, and final iterations; state the valid index range.
Choosing an unsuitable collection	Describe access patterns and required operations before choosing a structure.
Writing one large function or method	Decompose the task by responsibility and test each part.
Treating a running program as necessarily correct	Define expected behavior and test normal, boundary, and invalid inputs.
Copying code without tracing it	Ask the learner to predict a new case and make a modification.
Misreading an error message	Locate the useful diagnostic, identify file and line, and inspect surrounding code.

Language-specific pitfalls include: primitive versus reference values; integer di-

vision; String identity versus content equality; null references; array bounds; constructor and method signatures; static versus instance members; checked exceptions; and the Scanner newline behavior.

21.7 Differentiation without lowering the learning objective

Adjust the path, not the intellectual goal. Supports may include smaller task increments, additional practice, explicit worked examples, alternative information formats, extra navigation time, starter code with gradually faded scaffolding, oral explanation or accessible demonstration, pair programming with rotating roles, predictable routines, and enrichment tasks that deepen the same concept.

Do not assume all blind students need the same support or that all low-vision students prefer magnification. Ask, document, and review supports with the learner.

21.8 Formative assessment, feedback, and grading

Collect evidence frequently: explain a concept, predict output, trace a loop, justify a data structure, find and fix a bug, write a boundary test, refactor code, or demonstrate an accessible workflow.

A practical rubric can score each dimension from 1 (beginning) to 4 (independent and consistent):

Dimension	Evidence
Conceptual reasoning	Explains the problem, algorithm, and language concepts
Implementation	Produces a solution that meets requirements
Code comprehension	Reads, traces, explains, and modifies code
Testing and debugging	Uses test cases and interprets diagnostics
Accessibility and communication	Produces understandable interaction and operates the workflow with access tools
Independence and reflection	Describes decisions, limitations, and next steps

Share criteria before assessment. Feedback should name observed behavior, explain its impact, and identify an actionable next step. Do not grade screen-reader speed, typing speed, or visual formatting as proxies for programming competence.

21.9 Debugging and classroom support protocol

1. Ask the learner to describe expected behavior.
2. Ask what actually happened.
3. Read the complete error or diagnostic, including source and location.
4. Identify the smallest relevant code section.
5. Form one hypothesis.
6. Change one thing and rerun the program or test.
7. Record what the result taught the learner.

Distinguish environment failures, tool-access failures, syntax/compile errors, runtime errors, and logic errors. Maintain a troubleshooting record with environment, exact command, error text, attempted correction, and outcome. Remove credentials and private information before sharing logs.

21.10 Pair programming and collaboration

Use explicit roles: the **driver** controls the keyboard and implements the agreed step; the **navigator** reasons about the algorithm, checks requirements, proposes tests, and reviews the result. Rotate roles. Do not automatically assign a blind student the navigator role. Both learners should have opportunities to operate tools and write code. Provide shared source files and task instructions in accessible formats.

21.11 Project and capstone supervision

Require a proposal that includes the problem and intended user, functional requirements, out-of-scope features, input/output design, data representation, major functions/methods/classes, accessibility requirements, milestones, tests, and any libraries or AI assistance used.

Review milestones: requirements, design, first working slice, feature completion, tests, documentation, and final explanation. Encourage a modest, complete, well-tested project over an ambitious unfinished one.

21.12 Collaboration with families and support professionals

When appropriate and with learner consent, coordinate with special educators, teachers of students with visual impairments, assistive-technology specialists, interpreters, and families. Agree on who supports access technology, who teaches programming, and how the student obtains help. Keep the learner's agency central, share only necessary information, and follow school privacy policies. Do not make aides the default operators of the student's computer.

21.13 Instructor preparation checklist

Before each unit, complete the learner-facing activities; verify commands and versions on the classroom platform; test keyboard and assistive-technology work-

flows; prepare expected output and common error examples; identify vocabulary, symbols, misconceptions, and prerequisites; provide accessible instructions and assessment criteria; and decide how understanding will be observed.

After each unit, review student work and feedback, note barriers, adjust pacing and scaffolds, and record reproducible corrections. Document agreed accommodations rather than silently changing expectations.

21.14 Quality assurance and continuous improvement

Review links, software versions, commands, APIs, package recommendations, and assessment assumptions periodically. Test with actual screen readers and braille displays where possible; a successful build or visual inspection alone does not establish accessibility.

When reporting a barrier, include the page, reproduction steps, operating system, editor/runtime version, assistive technology and version, expected behavior, and actual behavior. Exclude personally identifying or sensitive student information. Submit corrections through the project contribution process.

18.6 Teacher's Guide: Introduction to Computer Science with JavaScript

For Students with Visual Impairments --- Complete Instructor Edition

How to use this teacher's guide

This guide is a companion for instructors, not a script that must be followed word for word. Read the philosophy, learner-context, and technology sections before the first lesson; use the unit notes while planning; and return to the assessment, differentiation, AI, and quick-reference sections throughout the course. Adapt procedures to the learner's actual assistive technology, operating system, course goals, and prior experience. Confirm tool-specific instructions in current official documentation because editor, screen-reader, and runtime behavior changes over time.

Plan for both programming learning and access to the programming environment. Establish what the student can already do independently, what access skills need instruction, what supports are already in place, and how the learner prefers to receive feedback. Never assume a diagnosis predicts a student's preferred tools or level of independence.

Table of Contents

1. Philosophy and Why JavaScript
2. JavaScript-Specific Accessibility Challenges
3. Classroom Setup and Configuration
4. Spoken JavaScript Conventions
5. Unit 0--1: Setup and Foundations
6. Unit 2: Numbers, Input, and Math
7. Unit 3: Control Flow
8. Unit 4: Arrays and Objects
9. Unit 5: Strings
10. Unit 6: Functions
11. Unit 7: Classes and OOP
12. Unit 8: Files, Modules, and Error Handling
13. Unit 9: Collections, Analysis, and Testing
14. Units 10--11: Projects and Capstone
15. Bridging from Python/Java to JavaScript
16. Assessment Strategies
17. Supporting Diverse Needs
18. Alternative and Supplementary Resources
19. Quick Reference Sheets
20. Teaching AI Tools Responsibly
21. Comprehensive Instructional Planning and Course Implementation

---

1. Philosophy and Why JavaScript

The Case for Teaching JavaScript to Blind Students

JavaScript has a unique profile of benefits and challenges for visually impaired learners:

Universal relevance. JavaScript is the only language that runs natively in every web browser. A student who learns JavaScript can eventually build tools that improve their own accessibility experience --- custom browser extensions, accessible widgets, automation scripts. This self-advocacy potential is meaningful.

Node.js enables accessible terminal programs. While JavaScript began in browsers, Node.js allows running JavaScript programs in the terminal --- the same accessible text-based environment used for Python. This curriculum focuses on Node.js to keep all I/O in the terminal where screen readers excel.

Asynchronous programming is a real-world skill. JavaScript's `async/await` model reflects how most modern software actually works --- programs that wait for I/O, network requests, and user events. Learning this early prepares students for professional development.

The npm ecosystem. Over 2 million packages on npm provide pre-built solutions for almost any problem. This is a double-edged concern for beginners (dependency complexity), but the ability to say `npm install` and gain powerful functionality is professionally relevant.

JSON is native. `JSON.parse()` and `JSON.stringify()` are built into JavaScript with no import required. For data persistence projects, this reduces friction.

An Honest Account of Challenges

JavaScript's permissiveness is a silent-failure risk. Unlike Python (which raises exceptions for type mismatches) and Java (which rejects them at compile time), JavaScript silently converts types. `"5" + 3` = `"53"`. `"5" - 3` = `2`. `0 == false` = `true`. Without visual scanning of intermediate results, these silent failures can propagate through a program undetected. This must be addressed through defensive programming habits taught from day one.

Asynchronous programming is conceptually hard. The `async/await` pattern required for user input is a genuine hurdle for beginners. There is no way to eliminate it from a Node.js input-based curriculum. The approach in this guide is to scaffold it carefully and defer full explanation of the event loop to later units.

Multiple valid syntaxes. Function declarations, function expressions, arrow functions, methods --- all do essentially the same thing. `var`, `let`, `const` --- three variable declarations. `==` and `===` --- two equality operators. This variety, which professionals call "JavaScript fatigue," creates unnecessary cognitive overhead for beginners. The approach in this curriculum is to teach one style consistently (`const/let`, arrow functions, `===`) and acknowledge alternatives exist without teaching them in parallel.

---

2. JavaScript-Specific Accessibility Challenges

Silent Type Coercion

This is the greatest accessibility risk in JavaScript for blind students. Unlike Python's `TypeError` and Java's compile-time type check, JavaScript silently converts types in ways that can produce wrong answers with no error:

Expression	Result	Expected by Programmer	Problem
`"5" + 3`	`"53"`	`8`	`+` concatenates when one operand is a string
`"5" - 3`	`2`	`2`	`-` converts string to number
`"5" * "3"`	`15`	`15`	Multiplication converts both

Expression	Result	Expected by Programmer	Problem
`"" == false`	`true`	Maybe?	Loose equality coercion
`0 == false`	`true`	Maybe?	Same
`null == undefined`	`true`	Probably not	
`NaN === NaN`	`false`	`true`	NaN is not equal to itself

Mitigation: Require `Number()` before any arithmetic on user input. Require `===` always. Teach `typeof` and `isNaN()` as defensive checks. Make logging intermediate values an explicit exercise before running any calculation.

`undefined` vs. `null` vs. `NaN` vs. `false` vs. `0` vs. `""`

All six of these are "falsy" in JavaScript. Aurally, they are indistinguishable unless you call `typeof` or check explicitly. A student who hears `console.log(result)` and the screen reader announces "false" cannot know whether `result` is `false`, `0`, `""`, `null`, `undefined`, or `NaN` without further inspection.

Teaching approach: Establish a diagnostic template:

```
console.log("Value:", result, "Type:", typeof result);
```

This outputs both the value and its type, providing audio context that `console.log(result)` alone does not.

Callback Hell and Asynchronous Confusion

Before `async`/`await`, Node.js input used nested callbacks:

```
// Confusing nesting --- avoid in teaching
rl.question("Name? ", (name) => {
  rl.question("Age? ", (age) => {
    rl.question("City? ", (city) => {
      console.log(`${name}, ${age}, ${city}`);
    });
  });
});
```

This nesting produces code that is confusing to read AND to hear --- the indentation deepens with each question. The `async`/`await` pattern linearizes this:

```
// Clear and linear --- use this exclusively
const name = await askQuestion("Name? ");
const age = await askQuestion("Age? ");
const city = await askQuestion("City? ");
console.log(`${name}, ${age}, ${city}`);
```

Teach `async/await` from the start and avoid callback patterns entirely in this course.

---

3. Classroom Setup and Configuration

VS Code or Theia IDE for JavaScript

Theia IDE is an alternative editor with keyboard- and screen-reader-oriented workflows suitable for this course. It supports many VS Code extensions through Open VSX; check availability of ESLint, Prettier, and JavaScript language support, and verify each workflow with the learner's screen reader. VS Code-specific settings and shortcuts may differ.

Required extensions:

- **ESLint** --- catches type issues, undefined variables, and style violations before running code
- **Prettier** --- auto-formats code consistently (run on save with `editor.formatOnSave: true`)
- **JavaScript (ES6) code snippets** --- tab-complete common patterns

Recommended ESLint flat configuration (`eslint.config.mjs` in project root; install the current ESLint, `@eslint/js`, and `globals` packages as local development dependencies):

```
import js from "@eslint/js";
import globals from "globals";

export default [
  {
    files: ["**/*.js"],
    languageOptions: {
      ecmaVersion: "latest",
      sourceType: "commonjs",
      globals: globals.node
    },
    rules: {
      ...js.configs.recommended.rules,
      eqeqeq: "error",
      "no-unused-vars": "warn",
      "no-console": "off"
    }
  }
];
```

The `eqeqeq: error` rule makes `==` a linting error --- it must be `===`. This is an automated enforcement of the rule that prevents the most common silent bugs.

Node.js REPL accessibility:

```
node                    # Start REPL
.help                   # List REPL commands
.exit                   # Quit (or Ctrl+D)
```

Windows Powershell

```
node                    # Start REPL
.help                   # List REPL commands
.exit                   # Quit (or Ctrl+D)
```

The Node.js REPL is excellent for early exploration. Results are printed immediately after each expression, providing audio feedback without the compile-run-read cycle.

Alternative JavaScript Environments

Environment	Platform	Accessibility	Notes
Node.js + VS Code	All	High	Recommended setup
Node.js + terminal only	All	High	For students who prefer minimal UI
Replit.com	Web	Medium	No-install; test with student's setup
CodeSandbox	Web	Medium	Cloud editor; variable accessibility
GitHub Codespaces	Web	High	Cloud VS Code; same accessibility as local
Glitch.com	Web	Medium	Project-based; variable accessibility
Observable	Web	Low	Notebook-style; primarily visual
Deno	Runtime	High	Alternative to Node.js; similar accessibility

---

4. Spoken JavaScript Conventions

Symbol/Pattern	Spoken Form
`const`	”const”
`let`	”let”
`var`	”var” (avoid; legacy)
`===`	”triple equals” or ”strict equal”
`!==`	”strict not equal”
`==`	”loose equal” (should hear this rarely)

Symbol/Pattern	Spoken Form
`=>`	”arrow” or ”fat arrow”
`...arr`	”spread arr”
`...args`	”rest args”
`async`	”async”
`await`	”await”
`undefined`	”undefined”
`null`	”null”
`NaN`	”NaN” (say each letter N-A-N)
`typeof`	”typeof”
`instanceof`	”instanceof”
`new`	”new”
`this`	”this”
`class`	”class”
`extends`	”extends”
`super`	”super”
`import`	”import”
`export`	”export”
`require()`	”require”
`module.exports`	”module dot exports”
`Promise`	”Promise”
`async function`	”async function”
`() => {}`	”arrow function”
`(x) => x * 2`	”arrow function taking x, returning x times 2”
`#field`	”hash field” or ”private field”
`` `template ${var}` ``	”backtick template literal with variable var”

Reading an arrow function:

```
const doubleIfPositive = (n) => (n > 0 ? n * 2 : n);
```

”const doubleIfPositive gets arrow function taking n, returning: if n greater than 0, n times 2, else n”

Modern JavaScript and Node.js conventions

Teach the curriculum against the current Node.js 26 LTS line. Node.js 26.13.0 entered LTS on October 28, 2026; use the latest security patch in the 26.x line rather than pinning students to an old patch. Node.js 24 remains supported as LTS, but is not the current LTS line.

For new course examples, consistently use const/let, strict equality, explicit numeric conversion, async/await, and node: built-in module specifiers. Prefer the built-in node:test runner for basic unit tests; Jest is a valid optional framework,

not a requirement. Keep the two module systems explicit: use a package.json type field and matching file extensions/configuration, and never mix CommonJS and ES module syntax in one file. Use current ESLint flat configuration (eslint.config.mjs) rather than the deprecated .eslintrc format. Check input and parsed JSON at runtime: JavaScript's runtime values remain dynamic even when editor tooling or static analysis is used.

5. Unit 0--1: Setup and Foundations

Lesson 0.1: Installing Node.js

Instructional Cues Verify installation at every step:

```
node --version    # Should print v26.x.x (26.13.0 or a newer 26.x maintenance release)
npm --version     # Should print a version number
npx --version     # Should print a version number (npx is bundled with npm)
```

Windows Powershell

```
node --version    # Should print v26.x.x (26.13.0 or a newer 26.x maintenance release)
npm --version     # Should print a version number
npx --version     # Should print a version number (npx is bundled with npm)
```

All three must work. If `npm` is missing, the student may have installed only the Node.js runtime without npm. Reinstall from nodejs.org (npm is included in the standard installer).

REPL first --- files second: Begin with `node` in the terminal. Let the student type expressions and hear results immediately. This builds confidence before introducing the file -> run -> read cycle:

```
$ node
> 5 + 3
8
> "Hello " + "World"
'Hello World'
> typeof 42
'number'
> typeof "hello"
'string'
> .exit
```

The npm init conversation: When initializing a project, `npm init -y` generates `package.json` automatically. Walk the student through reading the generated file:

```
npm init -y
cat package.json   # or type package.json on Windows
```

Windows Powershell

```
npm init -y
Get-Content package.json
```

`package.json` is plain JSON text --- fully readable with a screen reader. Understanding its structure is important for later lessons about dependencies and scripts.

Non-Obvious Accessibility Strategies

- **Node.js version managers:** `nvm` (Node Version Manager) allows multiple Node.js versions. For a student who may work on multiple machines or projects, run `nvm install 26` and then `nvm use 26` to select the Node.js 26 LTS line. `nvm` is command-line-only and fully accessible.
- **Project-local dependencies:** install course tools with `npm install` or `npm install --save-dev` from the project folder. Run local executables with `npx` or npm scripts. Avoid `npm install -g` for course tools; global installations can conflict across projects.

Thought Experiments

1. *"JavaScript was created in 10 days in 1995. What marks of that hasty design are still visible in modern JavaScript? (Think about `typeof null === 'object'`, `NaN !== NaN`, `0 == false`.) Why haven't these been fixed?"*
2. *"Node.js uses a single-threaded event loop to handle many concurrent operations. A traditional multi-threaded approach uses multiple threads running in parallel. For a web server handling thousands of requests, what are the tradeoffs of each approach? Which tends to be more memory efficient?"*
3. *"npm has millions of packages. The average Node.js project depends on hundreds of packages, many maintained by single developers with no organizational backing. The 'left-pad incident' in 2016 broke thousands of projects when one 11-line package was unpublished. What does this incident reveal about the risks and benefits of the npm ecosystem?"*
4. *"JavaScript runs in browsers, in Node.js servers, in mobile apps via React Native, in desktop apps via Electron, in IoT devices, and even in serverless cloud functions. What does this ubiquity tell you about the relationship between a language's design and its adoption?"*

Lesson 1.3: Variables and Data Types

Instructional Cues `const` vs. `let` --- establish the rule on day one:

"Use `const` for everything. If you get an error saying you're trying to reassign a `const`, THEN change it to `let`. This approach produces cleaner code and prevents accidental reassignment bugs."

Demonstrate the error:

```
const x = 5;
x = 10; // TypeError: Assignment to constant variable.
```

"This error is helpful. `const` is protecting you from changing something you said was fixed."

Dynamic typing with auditory ambiguity:

Run this sequence in the REPL and ask the student to predict each output before pressing Enter:

```
typeof 42; // "number"
typeof "42"; // "string"
typeof true; // "boolean"
typeof null; // "object"  <- the famous bug
typeof undefined; // "undefined"
typeof {}; // "object"
typeof []; // "object"  <- arrays are objects!
typeof function () {}; // "function"
```

"Notice that both `{}` (object) and `[]` (array) produce 'object'. How do you tell them apart? `Array.isArray([])` returns `true`. `Array.isArray({})` returns `false`."

Template literals vs. concatenation:

```
const name = "Alice";
const age = 15;

// Concatenation --- confusing order of operations when read aloud
console.log("Hello, " + name + "! You are " + age + " years old.");

// Template literal --- linear, clean, easy to hear
console.log(`Hello, ${name}! You are ${age} years old.`);
```

Read both versions aloud with the student. Ask which is easier to follow aurally. Template literals are almost universally clearer when heard, because the variable values appear where they belong in the sentence.

Thought Experiments

1. *"JavaScript has only one number type for integers and decimals. Python has separate `int` and `float`. Java has `int`, `long`, `double`, `float`. What precision problems arise from having a single number type? When does `0.1 + 0.2 = 0.30000000000000004` matter, and when doesn't it?"*

2. *"JavaScript has six falsy values: `false`, `0`, `""`, `null`, `undefined`, `NaN`. Everything else is truthy --- including `"false"` (a non-empty string), `[]` (an empty array), and `{}` (an empty object). Design a quiz: ask the student*

whether ten different expressions are truthy or falsy. Which answers are surprising?"

3. *"`var` has function scope; `let` and `const` have block scope. What does 'scope' mean? How does a variable declared inside an `if` block with `var` vs. `let` behave differently? Which behavior is more predictable and why?"*

---

6. Unit 2: Numbers, Input, and Math

Lesson 2.2: User Input in Node.js

Instructional Cues The input helper module pattern:

Provide the `input.js` module in every project skeleton. Students should not need to re-create this boilerplate:

```
// input.js
const readline = require("node:readline");
const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout,
});

function askQuestion(prompt) {
  return new Promise((resolve) => rl.question(prompt, resolve));
}

function close() {
  rl.close();
}

module.exports = { askQuestion, close };
```

Walk through what each line does at least once so students understand the structure, but do not require students to write it from scratch until they're comfortable with async programming.

The `async/await` mental model --- teach this story:

"When your code hits `await`, imagine it puts up a 'please wait' sign and steps aside. Other things can happen while you wait. When the answer arrives, your code picks up exactly where it paused. The `async` keyword on the function declaration is your promise that 'this function may pause at some point.'"

```
async function main() {
  console.log("Starting..."); // 1: runs immediately
  const name = await askQuestion("Name? "); // 2: pauses here until user types
  console.log("Got name:", name); // 3: runs after user answers
```

```
  const age = await askQuestion("Age? "); // 4: pauses again
  console.log(`${name} is ${age}`); // 5: runs after user answers again
  close();
}

main(); // Kick off the async function
```

Ask: "What line is running while the program waits for user input?" (None --- the function is paused. The event loop can handle other things.)

Why `main()` must be called at the bottom: `async function main()` defines the function but doesn't run it. The `main()` call at the end actually starts execution. Without the call, nothing happens. This trips up beginners --- demonstrate it.

Non-Obvious Accessibility Strategies

- **`rl.close()` is mandatory:** Without `rl.close()`, the Node.js process continues running after `main()` finishes. The terminal appears to hang --- cursor blinks but no prompt. For a screen reader user, this is confusing. Teach `close()` as the final step of every program that uses input.
- **Prompt trailing space:** Always include a trailing space in prompt strings: `"Enter your name: "`. Without the space, the cursor appears immediately after the colon with no separation, which some screen readers render as "Enter your name:cursor" --- an awkward audio experience.
- **Input validation loop:** Without validation, a blank Enter or a non-numeric string for a number causes silent failure. Provide this reusable pattern:

```
async function getNumber(prompt) {
  while (true) {
    const input = await askQuestion(prompt);
    const num = Number(input.trim());
    if (!isNaN(num)) return num;
    console.log("Invalid input: please enter a number.");
  }
}
```

Teach this pattern and require students to use it whenever they need numeric input.

Thought Experiments

1. *"The `readline` module uses an 'event listener' pattern --- `rl.question(prompt, callback)` means 'when the user answers, call this function.' The `async/await` pattern wraps this in a Promise. What problem does the*

Promise solve that the raw callback doesn't? (Think about reading the code linearly.)"

2. *"Node.js uses a single-threaded event loop. While your program is waiting for user input (`await askQuestion`), the thread is free. What else could the thread be doing? In a real application, this might mean handling a different request or updating state. What does this tell you about why `async/await` is important for scalable programs?"*

3. *"Synchronous input (blocking until the user responds) and asynchronous input (doing other things while waiting) represent fundamentally different programming models. For a single-user command-line program, which matters? For a server handling 10,000 simultaneous users, which matters more? Why?"*

---

7. Unit 3: Control Flow

Lessons 3.1--3.3

Instructional Cues `===` enforcement --- make it automatic:

With ESLint configured to flag `==`, the student will hear a warning whenever they use loose equality. Walk through the first warning together:

```
if (userInput == 5) {   // ESLint: Expected '===' and instead saw '=='
```

"ESLint is protecting you from a common silent bug. Change this to `===`. The only exception is checking for null-or-undefined: `x == null` is sometimes used as a shorthand for `x === null || x === undefined` --- but avoid this until you understand it."

Truthy/falsy in input validation:

```
const name = await askQuestion("Enter your name: ");
if (!name.trim()) {
  console.log("Name cannot be empty.");
  // re-prompt...
}
```

`!name.trim()` is true when the trimmed string is empty (`""` is falsy). This is idiomatic JavaScript. Read it aloud: "if not name dot trim" --- ask: "What does 'not an empty string' mean? Why is an empty string falsy?"

Ternary operator --- limit its use:

The ternary `condition ? valueIfTrue : valueIfFalse` is concise but can be hard to follow when read aloud:

```
const status = score >= 60 ? "pass" : "fail";
```

Read: *"const status gets: if score greater-than-or-equal 60, quote pass, else quote fail"*

Teach it but establish this rule: "If the ternary is easy to read aloud in one breath, use it. If it requires re-reading, use if-else."

Thought Experiments

1. *"JavaScript's `==` performs type coercion before comparing. TypeScript (a typed superset of JavaScript) was partly created to address this. What does TypeScript's existence tell you about the real-world cost of JavaScript's permissiveness? What did developers gain from TypeScript that JavaScript didn't provide?"*
2. *"Consider: `if (userName)` vs. `if (userName !== '' && userName !== null && userName !== undefined)`. The first uses JavaScript's truthiness; the second is explicit. In a code review, which would you prefer seeing? What are the readability and correctness tradeoffs?"*
3. *"JavaScript's optional chaining (`?.`) and nullish coalescing (`??`) were added to the language to handle null/undefined more gracefully. `user?.address?.city` returns `undefined` instead of throwing if `user` or `address` is null. Why did this feature take decades to add to a language created in 1995?"*
4. *"The `switch` statement falls through by default in JavaScript. Modern JavaScript has been moving away from `switch` toward mapping objects for dispatch tables. `const actions = { 'add': addFunc, 'delete': deleteFunc }; actions[command]?.();`. What are the accessibility implications of this pattern vs. a `switch` statement when read with a screen reader?"*

---

8. Unit 4: Arrays and Objects

Lesson 4.1: Arrays and Functional Methods

Instructional Cues Introduce `map`, `filter`, `reduce` as a trio:

Don't teach them one per lesson --- teach the relationship between them first:

"These three methods each take a function and apply it to every element in an array. `map` transforms elements. `filter` keeps or removes elements. `reduce` combines elements into one value. Together they cover most array processing."

Then give one shared example that uses all three:

```
const students = [
  { name: "Alice", score: 88 },
  { name: "Bob", score: 62 },
```

```
  { name: "Charlie", score: 95 },
  { name: "Diana", score: 71 },
];

// 1. Filter passing students (score >= 70)
const passing = students.filter((s) => s.score >= 70);

// 2. Map to get just names
const passingNames = passing.map((s) => s.name);

// 3. Reduce to build a report string
const report = passingNames.reduce(
  (acc, name) => acc + name + ", ",
  "Passing: ",
);

console.log(report); // "Passing: Alice, Charlie, Diana, "
console.log("Count:", passing.length);
```

Accessible printing for arrays of objects:

```
// Accessible: numbered, labeled fields
students.forEach((student, index) => {
  console.log(`${index + 1}. ${student.name}: ${student.score}`);
});

// Less accessible: raw object output
console.log(students); // Produces cluttered bracket notation when read aloud
```

Chaining --- build up incrementally:

Show chaining as a composition of steps, each of which you can trace separately:

```
const result = numbers
  .filter((n) => n % 2 === 0) // Step 1: keep evens
  .map((n) => n ** 2) // Step 2: square each
  .reduce((sum, n) => sum + n, 0); // Step 3: sum

// DEBUG: break the chain to trace each step
const step1 = numbers.filter((n) => n % 2 === 0);
console.log("After filter:", step1);
const step2 = step1.map((n) => n ** 2);
console.log("After map:", step2);
const step3 = step2.reduce((sum, n) => sum + n, 0);
console.log("Final sum:", step3);
```

The debug version (step-by-step) is more accessible during development. Consolidate to the chain after confirming correctness.

Thought Experiments

1. *"`map`, `filter`, and `reduce` come from functional programming --- a paradigm based on mathematical functions that have no side effects. JavaScript arrays also have `forEach` which is for side effects only. What is the philosophical distinction between `map` and `forEach`? Why does this distinction matter for reasoning about code correctness?"*

2. *"The `sort` method in JavaScript sorts in-place and mutates the original array. `filter`, `map`, and `reduce` return new arrays, leaving the original unchanged. Why the asymmetry? What bugs could arise from `sort`'s mutation? How would you sort without mutating: `[...arr].sort(...)`?"*

3. _"Functional composition: `const double = n => n _ 2; const addOne = n => n + 1; const doubleAndAdd = n => addOne(double(n))`. How does function composition scale? What is the advantage of composing small, focused functions over writing one large function that does everything?"*

4. *"The spread operator `[...arr]` creates a shallow copy of an array. For an array of primitives `[1, 2, 3]`, the spread copy is fully independent. For an array of objects `[{name: 'A'}, {name: 'B'}]`, the objects inside are still shared. What does 'shallow copy' mean? When does this distinction cause bugs?"*

Lesson 4.2: Objects

Instructional Cues Destructuring --- introduce before objects are complex:

```
const person = { name: "Alice", age: 15, city: "Portland" };

// Without destructuring:
const name = person.name;
const age = person.age;

// With destructuring:
const { name, age } = person;

// In function parameter:
function introduce({ name, age }) {
  console.log(`Hi, I'm ${name}, ${age} years old.`);
}
introduce(person);
```

Destructuring is ubiquitous in modern JavaScript. Teaching it early reduces the verbosity of object access code, which in turn makes programs cleaner to read

and hear.

`Object.keys()`, `Object.values()`, `Object.entries()` as navigation tools:

For a student who cannot visually inspect an object in a debugger panel, these methods provide audio-accessible object introspection:

```
const config = { theme: "dark", fontSize: 14, notifications: true };

// What properties does this object have?
console.log(Object.keys(config).join(", "));

// What are all the values?
Object.values(config).forEach((v) => console.log(v));

// Both together:
Object.entries(config).forEach(([key, value]) => {
  console.log(`${key}: ${value}`);
});
```

Optional chaining for safe nested access:

```
const user = { profile: { address: null } };

// Without optional chaining --- crashes if address is null
const city = user.profile.address.city; // TypeError!

// With optional chaining --- returns undefined instead
const city = user?.profile?.address?.city; // undefined, no crash

// With nullish coalescing for a default
const city = user?.profile?.address?.city ?? "Unknown city";
```

This pattern is essential for working with real-world data where fields may be missing.

Thought Experiments

1. *"JavaScript objects can have properties added or removed at any time. `user.newField = 'value'` just works. Java objects have a fixed set of fields defined at compile time. Which approach is better for representing data from an API that might add new fields in future versions? Which is better for ensuring all required fields are present?"*

2. *"`Object.freeze(obj)` makes an object immutable --- you can't add, remove, or modify properties. This mimics the behavior of a Java `final` field or Python's `frozenset`. When would immutable objects be useful? What patterns become possible when you can guarantee an object won't change?"*

3. *"Prototype-based inheritance means JavaScript objects can inherit from other objects directly, without classes. Classes (introduced in ES6) are 'syntactic sugar' over this prototype system. What does this mean? What can you do with prototypes that you cannot easily express with class syntax?"*

4. *"JSON (JavaScript Object Notation) is the universal data exchange format. It looks almost like JavaScript object literals but has stricter rules: all keys must be double-quoted strings, no functions, no comments. Why must JSON be stricter than JavaScript object literals? What security problems would arise if JSON allowed functions?"*

---

9. Unit 5: Strings

Instructional Cues Template literals for multiline strings:

```
const menu = `
--- Main Menu ---
1. Add item
2. View all
3. Search
4. Delete
5. Quit
-----------------`;

console.log(menu);
```

Template literals with newlines are much cleaner for building multiline output than repeated `console.log()` calls. Teach this pattern for menus and reports.

Regular expression basics --- connect to real need:

Before introducing regex syntax, establish the problem it solves:

"You have user input that might have extra spaces, uppercase, or punctuation. You want just the words. Without regex, you'd call `.trim().toLowerCase().replace(',', '').replace('.', '')...` for every punctuation mark. Regex lets you express 'any non-letter character' once."

```
const messy = "  Hello, World!   How are you?  ";
const words = messy
  .trim()
  .toLowerCase()
  .split(/\W+/)
  .filter((w) => w.length > 0);
console.log(words); // ['hello', 'world', 'how', 'are', 'you']
```

Teach `\w+` (word characters), `\d+` (digits), `\s+` (whitespace), and the `g` and `i` flags. Don't attempt to teach full regex --- a basic vocabulary is enough for data cleaning tasks.

Thought Experiments

1. *"Template literals use backticks `` ` `` while regular strings use `"` or `'`. Backticks on a keyboard are typically in the upper-left corner. For touch typists, backticks are less natural than quotes. What keyboard shortcuts or text expansions might a blind programmer use to reduce the friction of frequently-needed but awkward characters?"*
2. *"String immutability in JavaScript (same as Python) means every `.toUpperCase()`, `.replace()`, and `.slice()` creates a new string. For a loop that builds a long string incrementally, what is the efficient alternative? (Hint: consider `Array.push()` + `.join()` as a form of StringBuilder.)"*

---

10. Unit 6: Functions

Extended Instructional Cues

The four function forms --- teach one style, acknowledge the rest:

```
// 1. Function declaration (hoisted --- can be called before declaration)
function add(a, b) {
  return a + b;
}

// 2. Function expression (not hoisted)
const multiply = function (a, b) {
  return a * b;
};

// 3. Arrow function --- SHORT BODY (implicit return)
const square = (n) => n * n;

// 4. Arrow function --- LONG BODY (explicit return required)
const absolute = (n) => {
  if (n < 0) return -n;
  return n;
};
```

Teach style 3 (short arrow) and style 4 (long arrow) as the primary forms. Acknowledge 1 and 2 exist and how to read them. Do not attempt to teach all four simultaneously.

The implicit return trap --- a critical accessibility issue:

```
// These DO return a value:
const double = (n) => n * 2; // [x] implicit return
const greet = (name) => `Hi ${name}`; // [x] implicit return

// This does NOT return a value (the curly brace requires explicit return):
const broken = (n) => {
  n * 2; // No return statement! Returns undefined.
};

// Fixed:
const fixed = (n) => {
  return n * 2;
};
```

When heard aloud, `n => { n * 2; }` and `n => { return n * 2; }` sound different --- the `return` keyword is present in one and absent in the other. Teach students to listen for `return` inside block-bodied arrow functions.

Higher-order functions --- connect to previous array methods:

```
// map() takes a function as argument
const doubled = [1, 2, 3].map((n) => n * 2);

// We can also pass a named function
const double = (n) => n * 2;
const doubled2 = [1, 2, 3].map(double);

// This works because map expects a function --- it doesn't care if it's
// named or anonymous, arrow or traditional
```

"Every time you used `array.map(...)`, you were passing a function as an argument. Higher-order functions --- functions that take other functions --- are normal JavaScript."

Closures --- establish the mental model before the code:

"A closure is a function that remembers the environment where it was created, even after that environment is gone. Imagine a letter in an envelope. The letter is the function. The envelope is the environment --- it contains variables the function needs. Even if the person who wrote the letter is gone, the letter still has access to everything in its envelope."

```
function makeCounter(startValue = 0) {
  let count = startValue; // This is the "envelope"

  return {
    increment: () => ++count,
    decrement: () => --count,
```

```
    reset: () => {
      count = startValue;
    },
    getCount: () => count,
  };
}

const counter = makeCounter(10);
counter.increment();
counter.increment();
counter.increment();
console.log(counter.getCount()); // 13 --- count persists!
```

Thought Experiments

1. *"Arrow functions don't have their own* `this` *--- they inherit* `this` *from the surrounding scope. Regular functions create their own* `this`*. This was a major source of bugs in pre-ES6 JavaScript. How many of the following do you think involved confusing* `this`*? (a) Typos, (b) Variable scope errors, (c) Async timing bugs, (d)* `this` *binding bugs. In JavaScript specifically, (d) is disproportionately common --- why?"*

2. *"Pure functions always produce the same output for the same input and have no side effects.* `Math.sin(x)` *is pure.* `console.log(x)` *is not (it has a side effect). Why are pure functions easier to test? Why are pure functions easier to reason about in a screen-reader debugging session?"*

3. *"Memoization caches the results of function calls so repeated calls with the same arguments return the cached result. Write a description of a memoized* `fibonacci(n)` *function. What is saved? What is the tradeoff? When would memoization be harmful rather than helpful?"*

4. *"JavaScript is a multi-paradigm language: you can write imperative code (explicit step-by-step), object-oriented code (classes and methods), or functional code (pure functions and immutable data). For the accessible data manager project, which paradigm produces the most maintainable code? The most testable? The most readable when heard aloud?"*

---

11. Unit 7: Classes and OOP

Instructional Cues `toString()` **is mandatory:**

```
class BankAccount {
  #balance;

  constructor(owner, initialBalance = 0) {
    this.owner = owner;
```

```
    this.#balance = initialBalance;
  }

  toString() {
    return `BankAccount[owner=${this.owner}, balance=$${this.#balance.toFixed(2)}]`;
  }
}

const account = new BankAccount("Alice", 1000);
console.log(account.toString()); // BankAccount[owner=Alice, balance=$1000.00]
console.log(`${account}`); // Also calls toString() implicitly
```

Require `toString()` in every class. Explain: "When your screen reader reads `console.log(myAccount)`, it reads `toString()`. Without it, you hear `[object Object]` --- completely useless. Define `toString()` to hear meaningful information."

Private fields # --- establish as the default:

```
class Dog {
  #name; // Private --- cannot be accessed outside the class
  #age;

  constructor(name, age) {
    this.#name = name;
    this.#age = age;
  }

  get name() {
    return this.#name;
  } // Getter --- controlled access
  get age() {
    return this.#age;
  }

  set age(value) {
    if (value < 0) throw new Error("Age cannot be negative");
    this.#age = value;
  }

  bark() {
    console.log(`${this.#name} says: Woof!`);
  }

  toString() {
    return `Dog[name=${this.#name}, age=${this.#age}]`;
  }
```

```
}
```

"Private fields starting with `#` are truly private --- not just conventional privacy. `myDog.#name` outside the class throws a SyntaxError. This is real encapsulation."

Thought Experiments

1. *"JavaScript classes are 'syntactic sugar' over prototype-based inheritance. TypeScript adds static typing on top of JavaScript classes. With three layers (prototypes -> ES6 classes -> TypeScript classes), is JavaScript's OOP model more or less trustworthy than Java's, which has only one layer? What does 'syntactic sugar' imply about the behavior of the underlying mechanism?"*
2. *"The `class` keyword was added to JavaScript in 2015, 20 years after the language was created. In those 20 years, JavaScript developers built massive applications without classes using only prototypes and closures. What does this tell you about the role of syntax vs. capability in programming languages?"*
3. *"Mixins are a way to compose behavior from multiple sources without inheritance. Instead of `class A extends B`, you copy methods from B onto A's prototype. JavaScript supports this; Java does not (without interfaces). When would mixins be more flexible than single inheritance? What problems could they cause?"*

---

12. Unit 8: Files, Modules, and Error Handling

Lesson 8.1: Files

Instructional Cues `__dirname` and path construction:

```
const path = require("node:path");
const fs = require("node:fs");

// __dirname is the directory of the current file
const dataFile = path.join(__dirname, "data", "contacts.json");

// Path joining prevents slash vs. backslash issues on Windows/macOS/Linux
console.log("Data file location:", dataFile);
```

For a student who cannot visually navigate a file tree, `console.log(__dirname)` and `console.log(dataFile)` are essential diagnostic tools. Teach them to print file paths as the first debugging step when a file isn't found.

`fs.existsSync()` before file operations:

```
if (fs.existsSync(dataFile)) {
  const data = fs.readFileSync(dataFile, "utf8");
  return JSON.parse(data);
} else {
  console.log("No data file found. Starting with empty data.");
  return [];
}
```

Always check existence before reading. The alternative --- catching `ENOENT` in a try-catch --- works but puts normal ”first run” behavior in the error path.

Lesson 8.3: CommonJS Modules

Module design for accessibility:

A well-designed module has a clear exports interface. Teach students to think of their module's exports as its ”public API” --- what other files can use:

```
// contacts.js --- exports only what other files need
const path = require("node:path");
const fs = require("node:fs");

const DATA_FILE = path.join(__dirname, "..", "data", "contacts.json");

function loadContacts() {
  /* ... */
}
function saveContacts(contacts) {
  /* ... */
}
function findContact(contacts, name) {
  /* ... */
}

// Export only the public interface
module.exports = { loadContacts, saveContacts, findContact };
```

”Everything not in `module.exports` is private to this file. Other files can use `loadContacts` and `saveContacts`, but they cannot directly access `DATA_FILE`. This is encapsulation at the module level.”

Thought Experiments

1. *”Node.js's CommonJS (`require`) and the ES Module standard (`import`) are two different module systems in JavaScript. This creates confusion --- some packages use one, some use the other. TypeScript can use both. Why did two module systems emerge? What does this history say about how standards develop in open-source ecosystems?”*

2. *"Synchronous file I/O (`readFileSync`) blocks execution until the file is read. Asynchronous file I/O (`readFile` or `fs.promises.readFile`) allows other code to run while waiting. For a startup script that reads configuration before doing anything else, which makes more sense? For a web server that handles thousands of requests, which matters more?"*

3. *"Custom error classes allow code to throw and catch specific, named errors: `throw new ValidationError('email', 'Invalid format')`. What is the advantage of this over throwing `new Error('Validation failed: email field invalid format')`? Think about code that catches and handles the error."*

---

13. Unit 9: Collections, Analysis, and Testing

Jest Testing

Instructional Cues Jest output is accessible:

Run `npx jest --verbose` for maximum accessibility. The `--verbose` flag produces:

```
PASS src/math.test.js
  add function
    [x] adds two positive numbers (3ms)
    [x] adds negative numbers (1ms)
    [x] handles zero (0ms)
  getGrade function
    [x] returns A for score >= 90 (0ms)
    [x] returns F for score < 60 (0ms)

Test Suites: 1 passed, 1 total
Tests:       5 passed, 5 total
```

Each test name is announced. Failures include file path, test name, expected vs. received values, and a diff. All of this is plain text --- fully screen-reader-accessible.

Add to `package.json` scripts:

```
"scripts": {
    "test": "jest --verbose",
    "test:watch": "jest --verbose --watch"
}
```

`npm test` runs all tests. `npm run test:watch` reruns tests whenever a file changes --- useful during active development.

Test naming convention:

```
describe("BankAccount", () => {
  describe("deposit", () => {
    test("increases balance by deposited amount", () => {
      /* ... */
    });
    test("rejects negative amounts", () => {
      /* ... */
    });
    test("rejects zero amount", () => {
      /* ... */
    });
  });
  describe("withdraw", () => {
    test("decreases balance by withdrawn amount", () => {
      /* ... */
    });
    test("throws InsufficientFundsError when balance is too low", () => {
      /* ... */
    });
  });
});
```

The nested `describe` / `test` structure, when announced by Jest with `--verbose`, reads as complete sentences: "BankAccount > deposit > increases balance by deposited amount."

Thought Experiments

1. *"Jest's `expect(value).toEqual(expected)` performs deep comparison --- it checks that two objects have the same structure and values, not that they are the same object in memory. Why is deep comparison usually what you want when testing? When would you specifically want reference equality (`toBe`)?"*

2. *"Snapshot testing in Jest captures the output of a component or function the first time, then checks future runs against that snapshot. What are the advantages? What are the risks? When might a snapshot become 'stale' and mislead you into thinking code is correct when it isn't?"*

3. *"Code coverage measures what percentage of your code is executed by tests. 100% coverage is a goal but not a guarantee of correctness. Design a function with 100% test coverage that still has a bug. (Hint: think about boundary conditions --- what if the test covers the case where x=5 but not x=0?)"*

---

14. Units 10--11: Projects and Capstone

JavaScript Project Structure

```
my-data-manager/
|--- src/
|   |--- models/
|   |   +--- contact.js
|   |--- storage/
|   |   +--- dataManager.js
|   |--- ui/
|   |   +--- consoleUI.js
|   +--- utils/
|       |--- input.js         (the readline helper)
|       +--- validation.js    (reusable validation functions)
|--- data/
|   +--- contacts.json
|--- tests/
|   |--- models/
|   |   +--- contact.test.js
|   +--- storage/
|       +--- dataManager.test.js
|--- eslint.config.mjs
|--- package.json
+--- index.js
```

`package.json` scripts --- teach as accessibility tools:

```
{
  "scripts": {
    "start": "node index.js",
    "test": "jest --verbose",
    "test:watch": "jest --verbose --watch",
    "lint": "eslint src/",
    "lint:fix": "eslint src/ --fix"
  }
}
```

`npm start` --- consistent command regardless of entry point filename. `npm test` --- runs all tests. `npm run lint` --- checks for errors without running the program.

`npm run lint` as a pre-run check: Teach students to run `npm run lint` before running the program. ESLint catches undefined variables, unused variables, and `==` before the program runs --- much faster than finding runtime errors.

Accessible Output Design for Capstone

Require students to design their program's output to be navigable and self-describing:

```
// Navigation landmark prints
function printSeparator(title) {
  const line = "-".repeat(40);
  console.log(line);
  if (title) console.log(`  ${title.toUpperCase()}`);
  console.log(line);
}

// Numbered list with clear field labels
function printContactList(contacts) {
  if (contacts.length === 0) {
    console.log("No contacts found.");
    return;
  }
  printSeparator("contacts");
  contacts.forEach((c, i) => {
    console.log(`${i + 1}. ${c.name}`);
    console.log(`   Phone: ${c.phone || "not provided"}`);
    console.log(`   Email: ${c.email || "not provided"}`);
  });
  console.log(`Total: ${contacts.length} contact(s)`);
}

// Confirmation messages --- every action must produce one
function confirmAction(action, details) {
  console.log(`[x] ${action}: ${details}`);
}
```

Thought Experiments

1. *"Your capstone is a command-line program. Eventually, it could become a web application. What would need to change? What would transfer directly? Write a two-column list: 'Can reuse as-is' and 'Must rewrite.'"*
2. *"The accessible design principle 'no visual-only cues for critical information' means every status, error, or confirmation must be communicated through text, not color or icon alone. Where in your program have you relied on visual formatting? How would you replace it?"*
3. *"Open-source your capstone project. Write a README that explains: what the program does, how to install it, how to run it, and specifically how to use it with a screen reader. What information would a sighted user find obvious but a blind user needs explicitly stated?"*

15. Bridging from Python/Java to JavaScript

From Python to JavaScript

Python	JavaScript	Key Difference
`print("hello")`	`console.log("hello")`	Method vs. function
`input("prompt")`	`await askQuestion("prompt")`	Async in JavaScript
`f"Hello {name}"`	`` `Hello ${name}` ``	f-string vs. template literal
`def func(x):`	`const func = x => ...`	Arrow function style
`True/False`	`true/false`	Lowercase in JavaScript
`None`	`null` and `undefined`	JS has two "nothing" values
`my_list`	`myList`	JavaScript uses camelCase
`len(x)`	`x.length`	Property, not function
`x is None`	`x === null` or `x === undefined`	Different null checks
`import json`	`const fs = require('node:fs')`	Built-in `JSON` object, no import
`if __name__ == '__main__':`	No direct equivalent (just call `main()`)	

Thought experiments for Python->JS students:

1. *"In Python, you use spaces to define blocks (indentation). In JavaScript, you use `{}`. Python's blocks are invisible --- they're just the alignment of characters. JavaScript's blocks are explicitly marked. Which feels more natural? Which provides clearer information when read aloud?"*
2. *"Python's `None` and JavaScript's `null`/`undefined` both represent 'nothing,' but they behave differently. Python has one 'nothing'; JavaScript has two. When is `null` appropriate? When is `undefined` appropriate? Under what circumstances would you encounter one vs. the other?"*
3. *"Python's type system and JavaScript's both use dynamic typing, but JavaScript's is more permissive --- it coerces types rather than raising errors. After using both, which approach do you find more helpful for learning? Which is more appropriate for production software?"*

16. Assessment Strategies

Alternative Assessments for JavaScript

Console output analysis: Provide a JavaScript program and 3--4 sample inputs. Ask the student to predict what the program prints for each input before running it. This tests reading comprehension without requiring writing.

Linting-first review: Submit code that is free of all ESLint warnings. A student who achieves this has internalized JavaScript's best practices at the syntax level.

Refactoring exercises: Give a working but poorly structured program. Ask the student to refactor it to use functions, modules, or classes appropriately. Assess the quality of the architecture, not just whether it runs.

Test-first implementation: Give the student a set of failing tests. Ask them to write the implementation that makes the tests pass. This tests code comprehension and problem-solving without a blank-page start.

---

17. Supporting Diverse Needs

For Students Who Are New to Async Programming

Async/await is conceptually hard. For students who struggle:

1. Provide the complete `input.js` helper and instruct them to use it without modifying it.
2. For the first several lessons, treat `async function main() { ... } main();` as boilerplate, not content.
3. Introduce the conceptual explanation of async/await only after students are comfortable with the rest of the language.
4. Consider deferring async until after basic functions are mastered.

For Students with Additional Cognitive Considerations

- ESLint provides immediate text feedback for common errors --- use it as a learning tool. Hearing "no-unused-vars" and then explaining what that means is a teaching moment triggered by the student's own code.
- JavaScript's many syntactic variations can be overwhelming. Enforce a single consistent style (arrow functions, `const`/`let`, `===`) and defer alternatives.
- Use the REPL liberally for short experiments so the student gets immediate feedback without file management overhead.

---

18. Alternative and Supplementary Resources

Textbooks and Reading

Resource	Level	Format	Accessibility
Eloquent JavaScript (Haverbeke, free online)	Beginner--Intermediate	Free HTML	Excellent screen-reader-friendly HTML
You Don't Know JS (Simpson, free on GitHub)	Intermediate	Free HTML/PDF	In-depth on closures, scope, types
JavaScript: The Good Parts (Crockford)	Intermediate	Print	Short; focuses on the reliable subset of JS
JavaScript: The Definitive Guide (Flanagan)	Reference	Print/ebook	Comprehensive; good for looking up specifics
MDN Web Docs (developer.mozilla.org)	Reference	Free HTML	Excellent HTML accessibility; definitive JS reference
Node.js Documentation (nodejs.org/docs)	Reference	Free HTML	Official Node.js API reference
JavaScript.info	Beginner--Advanced	Free HTML	Very well-organized; plain HTML; comprehensive

javascript.info is especially recommended --- it's a comprehensive free tutorial organized from basics to advanced topics, all in clean, accessible HTML. It's often more readable than MDN for learning (vs. reference), and screen readers navigate it very well.

Online Courses and Platforms

Platform	Course	Accessibility	Notes
freeCodeCamp	JavaScript Algorithms & Data Structures	Medium	Certification path; some UI inaccessible
Codecademy	Learn JavaScript	Partial	Editor accessible; some UI elements not
Exercism.io	JavaScript Track	High	Terminal-based; mentor feedback
JavaScript30 (Wesbos)	30 vanilla JS projects	Medium	Video-based; transcripts available

Platform	Course	Accessibility	Notes
edX	Various JavaScript courses	High	Video transcripts; accessible exercises
Scrimba	Learn JavaScript	Low	Interactive video; poor screen reader support
Khan Academy	JavaScript course	Medium	Browser-based editor; variable accessibility
NodeSchool.io	Node.js workshops	High	Command-line workshopper format; fully accessible

NodeSchool.io is especially recommended --- workshops are run entirely in the terminal via a command-line tool. Students work through exercises, get feedback from the tool, and progress at their own pace. The terminal-first design makes it highly accessible.

Video and Audio Resources

Resource	Description	Accessibility
Traversy Media (YouTube)	JavaScript crash courses and projects	Captions available
The Net Ninja (YouTube)	In-depth JavaScript series	Captions available
Academind (YouTube)	Deep dives into JS/Node	Captions available
Kevin Powell (YouTube)	JavaScript and web fundamentals	Captions; clear explanations
CS50W (Harvard)	Web programming with Python and JavaScript	Full transcripts; high quality
JavaScript: Understanding the Weird Parts (Udemy)	Deep JS internals	Captions

JavaScript-Specific Accessibility Tools

Tool	Purpose	Notes
ESLint	Code quality and error prevention	Command-line; fully accessible
Prettier	Consistent code formatting	Run on save; no screen interaction needed
Jest	Testing framework	Accessible output; `--verbose` flag
jshint.com	Online JS linter	Plain HTML; basic linting

Tool	Purpose	Notes
node-inspect	Node.js debugger	Terminal-based debugging
0x	Node.js profiler	Terminal output
nvm	Node version manager	Command-line; accessible

Communities

Community	Relevance
Node.js Community (nodejs.org/community)	Official Node.js community resources
JavaScript Weekly (newsletter)	Weekly digest of JS news; plain HTML email
Stack Overflow JavaScript	Q&A community; plain HTML; excellent accessibility
Dev.to	Developer blog platform; reasonable accessibility
GitHub Discussions on popular JS projects	Community support; text-based
NFB Jernigan Institute	Resources for blind STEM students
SIGACCESS	Research on accessible computing including programming

---

19. Quick Reference Sheets

JavaScript vs Python vs Java at a Glance

Concept	Python	Java	JavaScript
Print	`print("x")`	`System.out.println("x")`	`console.log("x")`
Variable	`x = 5`	`int x = 5;`	`const x = 5;`
String format	`f"Hi {name}"`	`String.format("Hi %s", name)`	`` `Hi ${name}` ``
User input	`input("? ")`	`scanner.nextLine()`	`await askQuestion("? ")`
Array/List	`[1, 2, 3]`	`int[] arr = {1,2,3}`	`[1, 2, 3]`
Dictionary/Map	`{"a": 1}`	`new HashMap<>()`	`{ a: 1 }`
Function	`def f(x):`	`public int f(int x) {`	`const f = x => ...`
Class	`class Dog:`	`public class Dog {`	`class Dog {`
Exception	`try/except`	`try/catch`	`try/catch`

Concept	Python	Java	JavaScript
Null	`None`	`null`	`null/undefined`
Boolean	`True/False`	`true/false`	`true/false`
Length	`len(x)`	`x.length`	`x.length`
For each	`for x in list:`	`for (T x : list) {`	`for (const x of list) {`
Import	`import math`	`import java.util.*;`	`const x = require('./x')`

Accessible Output Patterns

```
// Standard separator
console.log("-".repeat(40));

// Section header
console.log(`\n=== ${title.toUpperCase()} ===`);

// Numbered list with details
items.forEach((item, i) => {
  console.log(`${i + 1}. ${item.name}`);
  console.log(`   ${item.detail}`);
});
console.log(`Total: ${items.length}`);

// Menu
const MENU = `
--- Main Menu ---
1. Add item
2. View all items
3. Search
4. Edit item
5. Delete item
6. Save
7. Quit
-----------------`;

// Confirmation
console.log(`[x] Added: "${name}"`);
console.log(`[x] Deleted: "${name}"`);
console.log(`[x] Saved to ${filename}`);

// Error messages
console.log(`Error: Name cannot be empty. Please try again.`);
console.log(`Error: "${filename}" not found. Starting with empty data.`);
console.log(`Error: Task number must be between 1 and ${count}.`);
```

Common JavaScript Error Quick Reference

Error	Common Cause	First Check
`ReferenceError: x is not defined`	Typo or not declared	Spelling; scope
`TypeError: Cannot read property of undefined`	Accessing property on `undefined`	Which variable is undefined; optional chaining
`TypeError: x is not a function`	Calling non-function	`typeof x` before calling
`SyntaxError: Unexpected token`	Syntax mistake	Line number; check brackets, commas
`SyntaxError: Cannot use import statement`	ES module syntax in CommonJS	Use `require()` or add `"type": "module"`
`Error: ENOENT: no such file or directory`	File not found	`console.log(__dirname)` and check path
`Error: EACCES: permission denied`	No write permission	Check file permissions
`RangeError: Maximum call stack size exceeded`	Infinite recursion	Check recursive function's base case
`UnhandledPromiseRejection`	Missing `await` or `.catch()`	Add `try/catch` around `await` calls
`NaN` (not an error, but wrong value)	Arithmetic on non-number	`console.log(typeof variable)` before math

Input Helper Module (Copy into every project)

```
// utils/input.js
const readline = require("node:readline");

const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout,
});

/**
 * Prompts the user and returns their response as a string.
 * @param {string} prompt - The question to display
 * @returns {Promise<string>} The user's trimmed response
 */
function askQuestion(prompt) {
  return new Promise((resolve) => {
    rl.question(prompt, (answer) => resolve(answer));
  });
```

```
}

/**
 * Prompts until a valid number is entered.
 * @param {string} prompt - The question to display
 * @returns {Promise<number>} A valid number
 */
async function askNumber(prompt) {
  while (true) {
    const input = await askQuestion(prompt);
    const num = Number(input.trim());
    if (!isNaN(num)) return num;
    console.log("Invalid input: please enter a number.");
  }
}

/**
 * Closes the readline interface. Call at the end of every program.
 */
function close() {
  rl.close();
}

module.exports = { askQuestion, askNumber, close };
```

20. Teaching AI Tools Responsibly

AI coding assistants can provide explanations, hints, test ideas, and alternative examples. They can also produce incorrect JavaScript, misunderstand Node.js module configuration, recommend untrusted npm packages, or conceal asynchronous behavior. Treat every response as an unverified proposal. The course must remain accessible without paid AI access or a personal account.

Instructor use of AI during preparation

Instructors may use approved AI tools to brainstorm examples, draft alternative explanations, propose test inputs, or identify likely misconceptions. The instructor remains accountable for accuracy and pedagogy. Verify every API and language claim against the target Node.js release and MDN or Node.js documentation; run examples and tests; review package recommendations; and test the learning workflow with the intended access technology. Never upload identifiable student work, accommodation plans, credentials, assessment answers, or confidential school information. Do not use AI-generated feedback as the sole basis for grading. Disclose substantive AI assistance when institutional policy or the publication process requires it.

Learning goals and appropriate uses

Students should describe the limitations of code generation, ask focused questions, verify JavaScript behavior using the runtime and official documentation, identify generated code beyond their current level, and disclose assistance when required. When permitted, AI may explain a diagnostic, rephrase a concept, suggest edge cases, critique an algorithm, or generate test inputs. Prefer hint-based prompts and have the learner describe their own understanding first.

JavaScript and Node.js risks

Review AI output for:

- Confusion between CommonJS `require()` and ES module `import`, including package `type`, file extensions, and runtime version.
- Incorrect claims about semicolon insertion, equality, coercion, truthy/falsy values, `undefined`, `null`, or `NaN`.
- Missing `await`, unhandled promise rejections, callbacks that do not complete, or incorrect execution-order assumptions.
- Browser-only APIs used in a Node.js lesson or incorrect Node.js APIs.
- Unsafe file paths, command execution, network requests, or untrusted input handling.
- Unnecessary npm dependencies, typo-squatted or abandoned packages, or packages with excessive permissions.
- Dense chaining, nested callbacks, clever one-liners, and output difficult to review sequentially.

Run examples in the course's declared Node.js environment, inspect values and types, and test success, boundary, and failure cases. Verify API claims in official Node.js documentation or MDN.

Screen-reader and output review

Review whether braces, nested callbacks, and asynchronous control flow are navigable and explainable; identifiers are descriptive; prompts and output provide context; results arrive in a predictable order; module style is consistent; and the AI interface itself is accessible. Prefer named functions and intermediate variables when they improve understanding. Make asynchronous behavior explicit.

Privacy, security, and equity

Do not submit student records, credentials, private code, API keys, or confidential school information to public AI systems. Follow institutional policies, age requirements, and approved-tool lists. Treat terminal commands and package recommendations as untrusted. Verify package names and provenance before installation. Provide a non-AI path and never make a paid account a prereq-

uisite. Discuss how generated examples may assume mouse use, visual browser interaction, graphical output, or a particular operating system.

Verification protocol: Ask – Inspect – Verify – Adapt – Explain – Disclose

1. **Ask:** state the problem and what has already been tried.
2. **Inspect:** read the response and flag unfamiliar APIs, packages, syntax, and assumptions.
3. **Verify:** check MDN or Node.js documentation; run a minimal example; test normal, boundary, and invalid input.
4. **Adapt:** simplify, secure, format, and make code accessible to sequential reading.
5. **Explain:** have the student explain control flow and make a new change without AI.
6. **Disclose:** record the tool and assistance when required.

Activity-specific policy

- **Practice:** hints and explanations may be allowed; request complete solutions only for code-review activities.
- **Homework/projects:** define permitted assistance and require disclosure and verification.
- **Individual assessments:** do not permit AI to perform the skill being measured unless explicitly included in the assessment or an accommodation.
- **Accessibility tools:** screen-reader, braille, magnification, and keyboard access tools are not prohibited answer-generation tools.

Discussion prompts

- An AI answer mixes CommonJS and ES module syntax. How would you diagnose the mismatch?
- A suggested npm package has a plausible name but few trustworthy details. What should you check?
- A promise-based program prints results in an unexpected order. How can you reason about `await`?
- What must a student understand if AI generated most of a function?

---

21. Comprehensive Instructional Planning and Course Implementation

This section consolidates practices that apply across the philosophy, learner-context, classroom setup, spoken-code, unit, assessment, differentiation, and re-

source sections above. Use it as a course-planning reference from intake through capstone evaluation.

21.1 Course planning and alignment

Map each unit to prerequisites, learning objectives, practice opportunities, and evidence of mastery. Do not assume every unit needs equal time.

- Identify outcomes: problem decomposition, code reading and writing, data representation, control flow, functions or methods, debugging, testing, and communication.
- Separate programming prerequisites from access-tool prerequisites.
- Record the operating system, editor/IDE, runtime, terminal, screen reader, braille display, magnification, and adaptive input technology used.
- Identify essential concepts, enrichment topics, and material that can be deferred without breaking later lessons.
- Plan checkpoints after each major concept group and adjust pacing using student evidence.
- Maintain a map linking each lesson to objectives, practice, assessment, accessibility considerations, and likely misconceptions.

A suggested progression is environment and navigation; computing foundations; expressions and data; input/output; decisions and loops; collections; functions or methods; object-oriented concepts; files and errors; testing; projects; capstone. Preserve prerequisites when reordering.

21.2 Readiness and learner partnership

Ask about preferred access methods, prior programming experience, keyboard navigation, file management, and what makes technical instruction usable. Ask about functional access requirements rather than requesting unnecessary diagnostic details.

Use a low-stakes baseline activity to observe how the learner opens and navigates a source file, moves by line/word/character/symbol, creates and saves a file, runs a command, locates output, and reads a diagnostic. Record strengths and supports, not just deficits. Revisit the baseline to recognize growth. Keep the learner central to decisions about tools and workflow.

21.3 Repeatable lesson architecture

1. **Purpose:** state the problem the lesson addresses.
2. **Prerequisite retrieval:** revisit the prior concept needed today.
3. **Vocabulary:** define technical terms and spoken symbol conventions.
4. **Explicit model:** demonstrate a small runnable example and narrate navigation and code changes.
5. **Trace:** read the code structurally and predict behavior.

6. **Guided practice:** solve a similar problem with prompts and fading support.
7. **Independent transfer:** apply the concept to a new requirement.
8. **Debug and reflect:** examine an intentional error or unexpected result.
9. **Exit check:** collect an explanation, prediction, code change, or test.

State one navigation action at a time, allow the learner to complete it, and confirm the result before continuing. Provide source files and commands in advance when possible. Narrate all actions in shared-screen demonstrations and provide an equivalent text-based route.

21.4 Explicit instruction for accessible code navigation

Teach file structure before reading every line. Identify current focus and how to return to a known location. Teach line, word, character, symbol, selection, search, and go-to-line navigation, plus navigation to functions, methods, classes, and diagnostics. Explain indentation and block boundaries. Use manageable line lengths and avoid unnecessarily nested expressions. Give students time to read diagnostics and output independently. Do not take over the keyboard without explaining the action and obtaining the learner's agreement.

21.5 Language-specific instructional emphasis: JavaScript

JavaScript's dynamic type system and runtime should be made explicit:

- Teach `let`, `const`, and scope before elaborate function patterns.
- Contrast `===` and `==` with concrete values; make coercion visible in REPL experiments.
- Treat `undefined`, `null`, empty strings, zero, false, and `NaN` as distinct values.
- Distinguish arrays from objects and explain property access, index ranges, mutation, and iteration.
- Introduce asynchronous input deliberately. Explain promises, `async`, `await`, and output order.
- Teach CommonJS and ES modules as separate systems. State module mode and Node.js version before troubleshooting imports.
- Make error handling and file paths explicit; distinguish syntax errors, runtime exceptions, rejected promises, and logic errors.
- Use ESLint and formatting tools as feedback aids, but explain each diagnostic.

21.6 Misconceptions and diagnostic teaching

Ask learners to predict behavior, explain their prediction, run a small experiment, and revise their mental model.

Common issue	Instructional response
Confusing assignment with comparison	Contrast operators and trace values before and after execution.
Treating input as already numeric	Identify input type, convert explicitly, and test valid and invalid values.
Off-by-one loop or index errors	Trace first, typical, and final iterations; state the valid index range.
Choosing an unsuitable collection	Describe access patterns and required operations before choosing a structure.
Writing one large function or method	Decompose the task by responsibility and test each part.
Treating a running program as necessarily correct	Define expected behavior and test normal, boundary, and invalid inputs.
Copying code without tracing it	Ask the learner to predict a new case and make a modification.
Misreading an error message	Locate the useful diagnostic, identify file and line, and inspect surrounding code.

Language-specific pitfalls include: coercion and equality; `undefined` versus `null`; truthy and falsy values; array mutation; callback and promise ordering; CommonJS versus ES modules; and browser APIs versus Node.js APIs.

21.7 Differentiation without lowering the learning objective

Adjust the path, not the intellectual goal. Supports may include smaller task increments, additional practice, explicit worked examples, alternative information formats, extra navigation time, starter code with gradually faded scaffolding, oral explanation or accessible demonstration, pair programming with rotating roles, predictable routines, and enrichment tasks that deepen the same concept.

Do not assume all blind students need the same support or that all low-vision students prefer magnification. Ask, document, and review supports with the learner.

21.8 Formative assessment, feedback, and grading

Collect evidence frequently: explain a concept, predict output, trace a loop, justify a data structure, find and fix a bug, write a boundary test, refactor code, or demonstrate an accessible workflow.

A practical rubric can score each dimension from 1 (beginning) to 4 (independent and consistent):

Dimension	Evidence
Conceptual reasoning	Explains the problem, algorithm, and language concepts
Implementation	Produces a solution that meets requirements
Code comprehension	Reads, traces, explains, and modifies code
Testing and debugging	Uses test cases and interprets diagnostics
Accessibility and communication	Produces understandable interaction and operates the workflow with access tools
Independence and reflection	Describes decisions, limitations, and next steps

Share criteria before assessment. Feedback should name observed behavior, explain its impact, and identify an actionable next step. Do not grade screen-reader speed, typing speed, or visual formatting as proxies for programming competence.

21.9 Debugging and classroom support protocol

1. Ask the learner to describe expected behavior.
2. Ask what actually happened.
3. Read the complete error or diagnostic, including source and location.
4. Identify the smallest relevant code section.
5. Form one hypothesis.
6. Change one thing and rerun the program or test.
7. Record what the result taught the learner.

Distinguish environment failures, tool-access failures, syntax/compile errors, runtime errors, and logic errors. Maintain a troubleshooting record with environment, exact command, error text, attempted correction, and outcome. Remove credentials and private information before sharing logs.

21.10 Pair programming and collaboration

Use explicit roles: the **driver** controls the keyboard and implements the agreed step; the **navigator** reasons about the algorithm, checks requirements, proposes tests, and reviews the result. Rotate roles. Do not automatically assign a blind student the navigator role. Both learners should have opportunities to operate tools and write code. Provide shared source files and task instructions in accessible formats.

21.11 Project and capstone supervision

Require a proposal that includes the problem and intended user, functional requirements, out-of-scope features, input/output design, data representation, major functions/methods/classes, accessibility requirements, milestones, tests, and any libraries or AI assistance used.

Review milestones: requirements, design, first working slice, feature completion, tests, documentation, and final explanation. Encourage a modest, complete, well-tested project over an ambitious unfinished one.

21.12 Collaboration with families and support professionals

When appropriate and with learner consent, coordinate with special educators, teachers of students with visual impairments, assistive-technology specialists, interpreters, and families. Agree on who supports access technology, who teaches programming, and how the student obtains help. Keep the learner's agency central, share only necessary information, and follow school privacy policies. Do not make aides the default operators of the student's computer.

21.13 Instructor preparation checklist

Before each unit, complete the learner-facing activities; verify commands and versions on the classroom platform; test keyboard and assistive-technology workflows; prepare expected output and common error examples; identify vocabulary, symbols, misconceptions, and prerequisites; provide accessible instructions and assessment criteria; and decide how understanding will be observed.

After each unit, review student work and feedback, note barriers, adjust pacing and scaffolds, and record reproducible corrections. Document agreed accommodations rather than silently changing expectations.

21.14 Quality assurance and continuous improvement

Review links, software versions, commands, APIs, package recommendations, and assessment assumptions periodically. Test with actual screen readers and braille displays where possible; a successful build or visual inspection alone does not establish accessibility.

When reporting a barrier, include the page, reproduction steps, operating system, editor/runtime version, assistive technology and version, expected behavior, and actual behavior. Exclude personally identifying or sensitive student information. Submit corrections through the project contribution process.

18.7 Julia Teacher's Guide

Purpose and course context

This guide supports instruction in the Julia curriculum for beginning programmers, including blind and visually impaired learners using screen readers, braille displays, magnification, and keyboard-first workflows. It supplements the complete lessons and topic index.

Julia is a high-level, dynamically typed, JIT-compiled language used for numerical computing, simulation, optimization, statistics, and research. The course baseline is **Julia 1.12**. Teach interactive experimentation while preserving reproducibility through project environments and explicit dependencies.

Outcomes and sequence

Complete the shared Computing Foundations Primer and Terminal Skills prerequisite, then follow Units 0–11. Learners should run scripts in a project, work with values and types, use control flow and collections, write functions, reason about multiple dispatch, model data, handle files and errors, test code, and complete the capstone. They should be able to distinguish interactive convenience from a reproducible project.

Use a predictable lesson cycle: objective and vocabulary; explain the code and expected data shape; predict the result; execute; inspect textual output; change one operation; explain the result. Encourage learners to state expected types and dimensions before running array operations.

Environment and package management

Verify with `julia --version`. Run a script with:

```
julia --project=. main.jl
```

Activate and manage the project with Julia's Pkg environment. Keep Project.toml and Manifest.toml under version control when the course's reproducibility requirements call for them. Teach that JSON3.jl and CSV.jl are external packages, not standard-library modules. Use the documented project environment rather than installing packages globally or relying on a learner's personal default environment.

Teaching Julia concepts

Dynamic typing, multiple dispatch, and functions

Julia variables are not statically declared with a fixed type in the same way as C or Go. Values have types, and functions can dispatch on argument types. Teach multiple dispatch with a small set of methods whose behavior is easy to compare. Avoid introducing many overlapping methods before learners understand

function calls and argument types. Explain that type annotations in method signatures participate in dispatch; they are not simply variable declarations.

Collections, arrays, and indexing

Arrays are central to Julia. Teach dimensions, axes, shape, element type, and indexing explicitly. Julia's conventional arrays are one-based; use eachindex when iterating over arbitrary arrays. Explain broadcasting with the dot syntax as a distinct operation and teach learners to predict shape and element-wise behavior. Never rely on a visual plot or matrix layout as the only way to understand an array; provide a textual description of dimensions and representative values.

Performance and compilation

Julia compiles methods just in time. Beginners do not need premature optimization, but instructors should explain that first-call compilation can affect timing. Do not compare a first invocation with a warmed-up run as if the difference were algorithmic. Teach benchmarking only after correctness and reproducibility, and use established benchmarking tools in project scope where required.

Files, tabular data, and errors

Use explicit paths and deterministic data fixtures. CSV.jl and JSON3.jl are dependencies, not built-ins. Explain package APIs and version constraints. Teach exceptions and validation as different concepts; handle expected missing or malformed data intentionally. Keep output in text tables or labelled records where that is more accessible than a plot.

Accessibility and instructional delivery

- Use the REPL and terminal as text-first learning environments; make prompts and output understandable when read linearly.
- Explain parentheses, square brackets, colons, semicolons, transpose marks, and dotted operators when introduced.
- Describe array dimensions, axes, and data shape in text. Do not make color maps or plots essential to an assessment.
- Teach keyboard navigation and command history without relying on mouse-driven notebook controls.
- Keep package installation, environment activation, and execution steps explicit.
- Offer a text summary or tabular representation for any visualization.
- Allow time for speech/braille review of dense expressions and nested array operations.

Test the actual REPL, terminal, editor, and assistive technology in use. A notebook or plot that works visually is not necessarily accessible.

Assessment and differentiation

Assess function design, type reasoning, dispatch selection, array shape/indexing, reproducible package use, and correct handling of files and errors. Ask learners to predict result types and dimensions, not only numeric values. Include empty arrays, boundary indexes, missing data, and package/environment failures. Evaluate correctness separately from performance; do not reward optimization that obscures a clear solution.

For additional scaffolding, use vectors and scalar functions before multidimensional arrays and multiple dispatch. For advanced learners, extend to parametric methods, performance measurement, and numerical stability while preserving the introductory learning outcomes.

AI-use policy

AI may be used only for personal conceptual understanding and reinforcement. It must not write, complete, autocomplete, repair, refactor, or generate course code or assessment solutions.

Instructor preparation

Verify Julia 1.12, activate the repository's project environment, and confirm required packages install from the documented manifest or project file. Run the current example checks if provided and separately validate expected results and assistive-technology access. Record package versions and whether performance measurements include compilation warm-up. Consult the shared accessibility guidance before requiring plotting or notebook tools.

18.8 Kotlin Teacher's Guide

Purpose and audience

This guide supports instructors teaching the Kotlin pathway to beginning programmers, including blind and visually impaired learners who use screen readers, braille displays, keyboard navigation, magnification, or other access technology. It supplements the Kotlin curriculum overview, unit lessons, and complete lesson reference.

Kotlin is a statically typed language that runs primarily on the Java Virtual Machine in this course. It interoperates with Java libraries but has distinct syntax, null-safety rules, collection interfaces, and language conventions. Do not assume Java knowledge is a prerequisite.

Course outcomes

Learners should be able to explain the role of source code, compiler, runtime, project, and dependency; write, compile, and run small Kotlin programs; declare values and variables; use numeric types and explicit conversions; read input and produce output; construct decisions and loops; select and manipulate collections; design functions; explain classes and objects; work with files and errors; test behavior; and complete and communicate an integrated project.

Recommended instructional sequence

Follow the unit order in the curriculum. Do not skip computing foundations or the terminal prerequisite simply because a learner can use an IDE. The terminal provides a transferable route to running tools, interpreting diagnostics, and understanding project structure.

Use a predictable lesson rhythm:

1. State the learning objective and prerequisite concepts.
2. Define new vocabulary before using it in a code example.
3. Explain the structure of the example before asking learners to execute it.
4. Ask learners to predict the result.
5. Run the example and compare actual behavior with the prediction.
6. Discuss diagnostics and common mistakes as evidence about program structure.
7. Assign a modification that requires reasoning rather than copying.
8. Ask the learner to explain the result in their own words.

Teaching Kotlin's type system

Explicitly distinguish a variable's declared type from its current value. Kotlin's type inference reduces annotation where the type is clear, but inference does not make Kotlin dynamically typed. Use examples that contrast val and var, and explain that val prevents reassignment of the reference rather than guaranteeing deep immutability of every referenced object.

Teach numeric conversions as explicit operations. Do not imply that an Int can be used wherever a Double is expected without conversion. Explain integer division, floating-point representation, and overflow at a level appropriate to the learner's stage.

Teaching null safety

Null safety is a central Kotlin concept. Introduce nullable types deliberately and contrast a non-nullable declaration with a nullable one. Teach the following as distinct tools:

- Safe call ?. performs an access only when the receiver is non-null.

- Elvis operator ?: provides an alternate expression when the left side is null.
- Not-null assertion !! asserts that a value is not null and accepts the possibility of a runtime exception.

Do not teach !! as the routine solution to a compiler error. Ask learners to explain why a value may be absent and choose a design that handles that possibility.

Collections and data modeling

Distinguish arrays, lists, sets, and maps by their behavior and intended use. Kotlin's read-only collection interfaces do not necessarily guarantee that the underlying collection cannot be changed through another reference. Avoid teaching read-only as synonymous with deeply immutable.

Use named data classes when records have domain meaning. Pair and Triple are useful for small temporary groupings, but opaque positional fields can make a program harder to read and explain through a screen reader.

Object-oriented instruction

Kotlin classes are final by default. Explain that inheritance requires an explicit design decision and the appropriate open declarations. Teach composition and interfaces alongside inheritance rather than presenting subclassing as the default reuse strategy. Distinguish properties from Java-style field access and explain constructors, visibility, and member functions.

Files, resources, and errors

Teach file operations with explicit paths and predictable test data. Discuss working directories because relative paths are interpreted relative to the process's current directory, not necessarily the source file's location. Use resource-safe constructs such as use when appropriate. Learners should inspect returned values and exceptions rather than suppressing failures.

Accessibility and instructional delivery

- Provide all essential content as selectable text.
- Use semantic headings and descriptive links.
- State code punctuation and nesting explicitly when introducing unfamiliar syntax.
- Avoid instructions such as “click the green triangle” without a keyboard equivalent.
- Ensure the learner can inspect compiler messages and program output as text.

- Provide keyboard-first navigation paths for editing, saving, running, and debugging.
- Allow additional time for screen-reader verbosity and braille review; do not interpret slower navigation as weaker conceptual understanding.
- Test the actual toolchain with the learner's assistive technology. Do not infer accessibility from a vendor statement or another learner's configuration.
- Do not make a visual debugger, diagram, or GUI-only tool the sole assessment route.

See Development editors and IDEs for the role of VS Code, Theia, and other professional environments. That reference discusses product purposes and accessibility evaluation but does not provide third-party IDE setup procedures.

Assessment

Assess both the program and the learner's reasoning. Suitable evidence includes a prediction of output before execution; a clear explanation of variables, types, and control flow; a test that distinguishes normal from boundary or invalid input; a diagnosis of a compiler or runtime error; a justified choice of collection or data model; a function or class explanation in terms of responsibility and inputs/outputs; a demonstration of file handling and error reporting; and a reflection describing what was tested and what remains uncertain.

Do not equate a program that compiles with a program that is correct. Require expected-output checks or explicit behavioral tests where the lesson calls for them.

AI-use policy

Follow the repository's course policy. AI may be used only for personal conceptual understanding and reinforcement; it must not write, complete, autocomplete, repair, refactor, or generate course code or assessment solutions for students or instructors. Instructors should provide the same policy consistently across language tracks.

Validation and maintenance

The course includes expected-output fixtures and an example-checking workflow. Before a teaching term, instructors should review the current compiler and JDK baseline, run the validation script from a clean checkout, verify relevant mdBook links, and record exact versions and results. Runtime validation and assistive-technology validation are separate activities.

Report corrections through the repository's contribution process. When changing examples, update expected output and validation evidence together. Do not claim that the course has been fully accessibility-tested unless the exact supported workflows have been exercised and documented.

18.9 Lua Teacher's Guide

Purpose and course context

This guide supports the Lua curriculum for beginning programmers, including blind and visually impaired learners using screen readers, braille displays, magnification, and keyboard navigation. It supplements the complete lessons, topic index, and Lua course overview.

The course targets **Lua 5.4** and the standard library. Lua is a lightweight, dynamically typed, embeddable scripting language. Its table-centric data model and use in game scripting, application extension, configuration, and automation should be taught without making a visual game editor or graphical environment a prerequisite.

Outcomes and sequence

Complete the shared Computing Foundations Primer and Terminal Skills prerequisite, then follow Units 0–11. Learners should run scripts with the interpreter, work with values and expressions, use decisions and loops, model data with tables, write functions and modules, apply object patterns where useful, process files and errors, test behavior, and complete the capstone.

Use a consistent lesson cycle: objective and vocabulary; explicit walkthrough; prediction; interpreter execution; output review; one modification; learner explanation. Ask learners to predict both the returned value and any side effects.

Toolchain and packages

Verify with `lua -v` and run a script with `lua main.lua`. LuaRocks is optional for the core course; use it only when a lesson documents an external dependency. JSON and standards-aware CSV support are provided through packages rather than the Lua core. Explain package installation, module lookup, and runtime version requirements before assigning dependency-based work.

Teaching Lua's distinctive semantics

Dynamic values and truthiness

Variables hold values whose types are determined at runtime. Only `false` and `nil` are falsey; values such as zero and the empty string are truthy. This differs from several other languages and should be tested explicitly. Teach nil as absence and distinguish it from false.

Tables and iteration

Tables can model sequences, maps, records, and object-like structures. Sequence indexing conventionally begins at 1. Teach the distinction between the sequence

portion and arbitrary key/value entries. `ipairs` iterates sequence entries; `pairs` iterates key/value pairs without a guaranteed order. Do not write assessments that expect a stable `pairs` order. Explain that the length operator has specific sequence semantics and should not be treated as a universal count of every key.

Functions, modules, and metatables

Functions are first-class values and can return multiple results. Teach parameter lists, return values, closures, and module boundaries with small examples. Introduce metatables only after learners can explain ordinary table lookup; metamethod behavior can otherwise obscure the data model. Object-oriented patterns in Lua are conventions built on tables and metatables, not a built-in class syntax.

Errors and file operations

File functions commonly return a value plus an error message rather than raising an exception for every failure. Teach learners to check returned values before use. Use explicit paths and close files reliably; explain working-directory behavior. Keep external JSON/CSV dependencies documented and avoid implying that these formats are built into the core language.

Accessibility and delivery

- Use the interpreter and terminal as the baseline, with text-only examples and output.
- Describe punctuation including braces, brackets, dots, colons, concatenation, and the length operator.
- Explain table contents as labelled key/value pairs or ordered sequence entries.
- Avoid visual game editors, scene trees, and color-only debugging as required learning tools.
- Provide keyboard equivalents for editor and execution actions.
- Make iteration order explicit in examples; do not ask learners to infer order from a visual display.
- Allow time to navigate nested tables and punctuation through speech or braille.

Verify the learner's actual interpreter, editor, terminal, and assistive technology. If a game or host application is used for enrichment, provide a standalone text-based path for every assessed objective.

Assessment and differentiation

Assess truthiness, nil handling, table modeling, indexing, iteration, multiple return values, module design, and file-error handling. Include tests for zero, empty

strings, missing keys, sparse tables, and unordered map traversal. Require learners to explain why `pairs` order is not a contract. A successful script exit does not prove correct data handling or accessible interaction.

Begin with simple sequences and records before nested tables and metatables. Extend advanced learners with closures, coroutines, or host-application scripting only after core language semantics are secure.

AI-use policy

AI may be used only for personal conceptual understanding and reinforcement. It must not write, complete, autocomplete, repair, refactor, or generate course code or assessment solutions.

Instructor preparation

Verify Lua 5.4 and run the repository's current example-validation procedure. Confirm all external package steps and test file operations from a known working directory. Validate screen-reader output in the actual classroom environment. See the shared accessibility guidance.

18.10 Teacher's Guide: Introduction to Computer Science with Python

For Students with Visual Impairments --- Complete Instructor Edition

How to use this teacher's guide

This guide is a companion for instructors, not a script that must be followed word for word. Read the philosophy, learner-context, and technology sections before the first lesson; use the unit notes while planning; and return to the assessment, differentiation, AI, and quick-reference sections throughout the course. Adapt procedures to the learner's actual assistive technology, operating system, course goals, and prior experience. Confirm tool-specific instructions in current official documentation because editor, screen-reader, and runtime behavior changes over time.

Plan for both programming learning and access to the programming environment. Establish what the student can already do independently, what access skills need instruction, what supports are already in place, and how the learner prefers to receive feedback. Never assume a diagnosis predicts a student's preferred tools or level of independence.

Python 3.12+ baseline and current conventions

The learner materials target **Python 3.12 and newer**. Python 3.15.2 is the current stable release as of October 30, 2026, while 3.12 is the minimum compatibility floor. Teach and assess against 3.12-compatible syntax and standard-library APIs unless a lesson explicitly identifies a newer feature. Do not require prerelease builds or features introduced after 3.12.

Model current conventions consistently:

- Follow PEP 8 naming and layout: four-space indentation, snake_case functions/variables, PascalCase classes, UPPER_SNAKE_CASE constants, readable imports, and short logical lines. A strict 79-character limit is a guideline, not a reason to create awkward wrapping; preserve clarity and accessible reading order.
- Prefer built-in collection annotations (list[str], dict[str, int]) over legacy typing.List/Dict in new Python 3.12+ examples. Annotate function inputs and return values when it clarifies the contract; explain that annotations do not enforce runtime types by themselves.
- Use f-strings for interpolation, pathlib.Path for paths, context managers (with) for files, and explicit main guards for runnable modules.
- Use specific exception types, avoid bare except, avoid mutable default arguments, and compare absence with is None.
- Use uv to create project environments and manage dependencies: uv init --python 3.12, uv add package, and uv run command. Commit uv.lock for shared projects. Do not install course dependencies with global pip.
- Introduce Ruff for linting/formatting and pytest for tests only after learners understand the underlying Python and standard-library workflow. Do not make either tool a prerequisite for basic lessons.

Table of Contents

1. Philosophy and Foundational Principles
2. Understanding Your Students
3. Classroom Setup and Technology Configuration
4. Universal Spoken Code Conventions
5. Unit 0 --- Setup and Environment
6. Unit 1 --- Foundations and Getting Started
7. Unit 2 --- Numeric Computations and I/O
8. Unit 3 --- Control Flow, Decisions, and Loops
9. Unit 4 --- Data Structures: Lists and Dictionaries
10. Unit 5 --- User Input and While Loops
11. Unit 6 --- Functions
12. Unit 7 --- Classes and Object-Oriented Programming
13. Unit 8 --- Files and Exceptions
14. Unit 9 --- Testing and Data Analysis
15. Unit 10 --- Project-Based Learning

16. Unit 11 --- Capstone Project
17. Assessment Strategies
18. Supporting Diverse Needs Within the Visually Impaired Population
19. Teaching AI Tools Responsibly
20. Alternative and Supplementary Resources
21. Quick Reference Sheets
22. Comprehensive Instructional Planning and Course Implementation

---

1. Philosophy and Foundational Principles

Why Python for Blind Students?

Python is widely recognized as one of the most accessible first programming languages for visually impaired learners for several concrete reasons:

- **Text-based I/O is the default.** Unlike languages that push beginners toward GUI frameworks or graphical plotting, Python's most natural outputs are plain text to the terminal --- fully accessible to any screen reader.
- **Indentation enforces structure.** While indentation introduces a learning challenge (addressed in detail below), it also means that code read aloud by a screen reader carries structural meaning in the form of announced indent levels --- something curly-brace languages do not provide.
- **Error messages are descriptive.** Python's tracebacks include the file name, line number, error type, and a message. A screen reader user can navigate directly to the problem without visual scanning.
- **The standard library is enormous.** Python ships with modules for file I/O, JSON, math, random numbers, CSV parsing, unit testing, and more --- reducing the complexity of dependency management in a learning context.
- **The REPL is an accessible sandbox.** Python's interactive interpreter provides immediate spoken feedback for any expression, enabling rapid experimentation without the overhead of creating and running files.

Core Pedagogical Commitments

Commit to ears-first instruction. Every demonstration, every example, every debugging session should be conducted as if the teacher is also using a screen reader. Ask yourself before every lesson: "If I could not see the screen, how would I know what was happening right now?" If the answer is unclear, redesign the demonstration.

Commit to explicit navigation. Sighted students can glance at code and immediately locate structures. Blind students must navigate deliberately using keyboard shortcuts and screen reader commands. Teach navigation skills as primary content, not auxiliary support.

Commit to verbal precision. Vague instructions like "look at the next line" or "you can see here that..." create invisible barriers. Replace all visual language with navigation language: "Move down two lines," "Jump to line 14 with Ctrl+G," "Navigate to the end of the file with Ctrl+End."

Commit to error-positive culture. For a sighted student, an error is a red underline they can ignore or address at leisure. For a screen reader user, an unaddressed error is noise that clutters the output of every subsequent navigation. Treat errors as immediate priorities and build the habit of reading and resolving them before moving on.

Commit to redundancy. Provide concepts in multiple formats --- spoken explanation, written pseudocode, physical analogy, and code. Blind students cannot fall back on visual re-reading; redundant representation builds retention.

---

2. Understanding Your Students

The Heterogeneity of Visual Impairment

"Visually impaired" encompasses a wide spectrum. Your students may include:

Profile	Implications for Teaching
Congenitally blind (blind from birth)	May have no visual spatial metaphors. Analogies must be tactile, auditory, or conceptual. May be faster with screen readers from years of experience.
Late-onset blindness (lost vision after sighted years)	May retain strong visual mental models. Can use these as bridges but should not be required to. May still be learning screen reader skills.
Low vision (functional but impaired vision)	May use a combination of magnification and screen reading. May be able to see some screen content but cannot rely on fine detail. Do not assume screen-only or vision-only.
Color blindness	Primarily relevant for syntax highlighting. Ensure themes are not relied upon for meaning.
Progressive conditions	Vision may change during the course. Build habits that remain useful at any level of vision.

Never assume a student's profile without asking. Some students are private about their diagnoses; frame questions around what tools they use and what works best, not what their condition is.

Screen Reader Literacy Varies

Some students arrive with years of fluent screen reader experience. Others are relatively new to their screen reader and may still be developing efficiency. Assess screen reader fluency separately from programming aptitude --- a student who struggles to navigate a file is not struggling with Python; they need more time with their tool.

Suggest that students spend time with their screen reader in non-programming contexts (browsing documentation, reading formatted text) before or alongside the first programming units.

Braille Display Users

Some students use a refreshable braille display in addition to or instead of audio output. Key considerations:

- Braille displays have limited width (typically 40 or 80 cells). Long lines of code may need to be panned across. Encourage keeping line length under 79 characters (PEP 8 standard) for this reason.
- Code rendered in braille uses Grade 1 (uncontracted) braille because contractions would misrepresent code characters. Confirm this setting is active.
- Navigation with a braille display typically involves routing cursor buttons --- physical buttons above each braille cell that move the cursor to that position. Students using these may navigate differently than audio-only users.
- Braille display users often prefer lower verbosity audio output since they read braille silently. Do not require audio announcements as the only feedback mechanism.

Students Who Are Also Deaf-Blind

Students who are deaf-blind rely exclusively on braille for all information. Audio feedback from the screen reader is not available. Considerations:

- All instructional communication must be via tactile means, text-based communication, or tactile sign language interpreters.
- Braille output of code is the primary interface. Pay special attention to line length and symbol representation.
- Terminal output must be navigable via braille display --- avoid programs that produce output faster than a braille display can be read.
- Refer to specialists in deaf-blind education for specific accommodation strategies beyond the scope of this guide.

---

3. Classroom Setup and Technology Configuration

Physical Environment

- Ensure the student's workstation does not have excessive background noise that interferes with screen reader audio. Headphones are often preferable in a shared classroom.
- Position the workstation so the teacher can sit beside (not in front of) the student, allowing them to listen to the screen reader output together.
- If a classroom assistant or orientation and mobility specialist is present, coordinate roles in advance. The programming teacher leads on programming content; the VI specialist leads on accessibility tool configuration.

VS Code / Theia IDE Configuration Checklist

Work through the equivalent accessibility and language-support settings with the student on day one. VS Code-specific setting names and shortcuts may differ in Theia; confirm each workflow with the learner's screen reader.

Screen Reader

- ☐ Screen Reader Optimized Mode ON (Ctrl+Shift+P -> Toggle Screen Reader Optimized)
- ☐ Accessibility help panel reviewed (Shift+Alt+F1)

Editor Settings (Ctrl+,)

- ☐ editor.lineNumbers = ”on”
- ☐ editor.renderWhitespace = ”all” or ”boundary”
- ☐ editor.wordWrap = ”on” (for long lines)
- ☐ workbench.reduceMotion = ”on” (reduces distracting animations)
- ☐ editor.tabSize = 4
- ☐ editor.insertSpaces = true (spaces not tabs for Python)

Screen Reader Indentation (in screen reader settings, not VS Code)

- ☐ NVDA: Document Formatting -> Indentation -> ”Say all” or ”Beeps”
- ☐ JAWS: Configure speech for VS Code environment
- ☐ VoiceOver: Verbosity -> Text -> announce indentation
- ☐ Narrator: No built-in indent announcement; use Beep plugin or extension

Extensions Installed

- ☐ Python (Microsoft)
- ☐ Pylance
- ☐ Python Indent (for consistent auto-indentation)
- ☐ GitLens or Git Graph (for version control visibility)
- ☐ Error Lens (inline error annotations --- screen-reader-readable)

Keyboard Shortcuts Practiced

- ☐ Ctrl+G --- Go to line
- ☐ F8 --- Next error
- ☐ Shift+F8 --- Previous error
- ☐ Ctrl+Shift+M --- Problems panel
- ☐ Shift+Alt+F2 --- Accessible View
- ☐ Ctrl+Shift+P --- Command Palette
- ☐ Ctrl+‘ --- Toggle terminal
- ☐ Ctrl+Shift+E --- Explorer
- ☐ Ctrl+Shift+O --- Go to symbol (function/class navigation)
- ☐ Alt+F12 --- Peek Definition
- ☐ F12 --- Go to Definition
- ☐ Shift+F12 --- Find all References

Terminal Configuration

- **Windows Terminal** (preferred over legacy cmd.exe on Windows) has better accessibility with NVDA and JAWS.
- **NVDA + Windows Terminal:** Enable ”Use UI Automation” in NVDA settings for better terminal reading.
- **VoiceOver + macOS Terminal:** Works well by default. Use `VO+Shift+Down` to interact with terminal content.
- **Orca + Linux terminal:** Orca reads terminal output well. Use `Orca+T` (flat review) to re-read recent output.
- Configure the terminal font to a monospace font at a size comfortable for any residual vision.

Python Project Environment Setup with uv

Use `uv` as the standard environment and dependency manager. It creates a project-local `.venv`, resolves dependencies, and runs commands in that environment. Manual activation is not required; use `uv run` so the correct interpreter and dependencies are selected.

macOS and Linux:

```
uv init --python 3.12
uv add requests
uv run python --version
uv tree
```

Windows Powershell

```
uv init --python 3.12
uv add requests
uv run python --version
uv tree
```

For an existing project, use `uv sync` to install the dependencies recorded in `pyproject.toml` and `uv.lock`. Do not install course packages with global pip.

Consider creating a startup script (`start.bat` on Windows, `start.sh` on macOS/Linux) that opens VS Code or Theia IDE in the project folder. Use uv run for Python commands. Students can run one command to get their environment ready, reducing setup cognitive load.

---

4. Universal Spoken Code Conventions

Establish this vocabulary on the first day and reinforce it consistently. When a student is reading code aloud (for debugging, peer review, or just narrating their work), they should use these terms without hesitation.

Symbol Pronunciation Table

Written Symbol	Spoken Form	Notes
`=`	”gets”	Assignment only
`==`	”equals equals” or ”is equal to”	Comparison
`!=`	”not equal to” or ”bang equals”	
`>=`	”greater than or equal to”	
`<=`	”less than or equal to”	
`>`	”greater than”	
`<`	”less than”	
`+=`	”plus equals” or ”add and assign”	
`-=`	”minus equals”	
`*=`	”times equals”	
`**`	”to the power of” or ”double star”	
`//`	”floor divide” or ”integer divide”	
`%`	”modulo” or ”remainder”	
`()`	”open paren ... close paren”	Always say both
`[]`	”open bracket ... close bracket”	
`{}`	”open brace ... close brace”	
`:`	”colon”	End of if/for/def/class line
`"text"`	”quote text quote”	

Written Symbol	Spoken Form	Notes
`'text'`	”single-quote text single-quote”	
`f"..."`	”f-string”	Mention the f prefix
`{var}` inside f-string	”open brace var close brace”	
`#`	”hash”	Followed by comment text
`\n`	”backslash n” or ”newline”	
`\t`	”backslash t” or ”tab”	
`_`	”underscore”	In names: ”my underscore name”
`__init__`	”dunder init”	Double-underscore methods
`__str__`	”dunder str”	
`.`	”dot”	In `math.sqrt`, say ”math dot sqrt”
`->`	”arrow”	Type hints
`*args`	”star args”	
`**kwargs`	”double-star kwargs”	
`None`	”None” (capital N)	Distinct from the word ”none”
`True` / `False`	”True” / ”False” (capital)	
`import`	”import”	
`from x import y`	”from x import y”	Read as English sentence

Reading a Line of Code Aloud

Teach students to read full lines this way, left to right, then explain:

```
total_price = unit_price * quantity + tax
```

”total underscore price gets unit underscore price times quantity plus tax”

Then explain: *”This line calculates the total price by multiplying the unit price by the quantity, then adding tax, and stores the result in total_price.”*

The two-pass approach --- first read the symbols, then explain the meaning --- builds both syntax recognition and semantic understanding.

---

5. Unit 0 --- Setup and Environment

Lesson 0.1: Installing Python and VS Code

Before This Lesson Survey the student's current technology setup:

- What screen reader do they use, and what version?
- Do they use a braille display?

- What operating system?
- Have they used a terminal before?
- Do they have administrator access to their computer?

Confirm administrator access before scheduling this lesson. Installation without admin rights will fail.

Instructional Cues Walk through every installer step verbally as the student performs it. Do not say "click the next button" --- say "Tab until you hear 'Next button,' then press Enter." Treat this as teaching screen reader navigation, not just software installation.

Critical checkpoint --- PATH configuration: After installation, before anything else: `python3 --version`. If this fails, stop and address the PATH issue before proceeding. A student who continues without Python on PATH will encounter mysterious failures throughout the course.

Non-Obvious Accessibility Strategies

- **Windows Defender SmartScreen:** On Windows, running a downloaded `.exe` from Python.org may trigger a SmartScreen dialog. This dialog can be navigated with Tab and Enter but is not always announced immediately by NVDA. Tell the student what to expect before they run the installer.
- **macOS Gatekeeper:** Downloaded packages may require right-click -> Open to bypass security. On a screen reader, right-click is VO+Shift+M (VoiceOver) or App Key (NVDA with keyboard access).
- **Linux package managers as an alternative:** For Linux students, `sudo apt install python3 curl` is often more reliable than the Python.org download and is fully scriptable.
- **Portable Python for school computers:** Some schools restrict software installation. Python.org offers portable/embeddable distributions. Replit.com is a web-based Python IDE that requires no installation and is reasonably accessible.

Alternative Installation Methods

Method	Accessibility	Best For
Python.org installer	High --- standard dialog flow	Most installations
Chocolatey (Windows)	High --- command-line, screen-reader friendly	Windows power users
Homebrew (macOS)	High --- command-line	macOS users comfortable with terminal
apt/dnf/pacman (Linux)	High --- command-line	Linux users

Method	Accessibility	Best For
Replit.com	Medium --- web-based, some accessibility gaps	No local install possible
Theia IDE	Verify with learner's screen reader	VS Code alternative with similar keyboard workflows
Thonny IDE	Medium --- simpler UI, bundled Python	Beginners who struggle with VS Code

On Thonny: Thonny is a Python IDE specifically designed for beginners. It has a simpler interface than VS Code and is sometimes more accessible for students who are new to both screen readers and programming. The trade-off is fewer advanced accessibility features long-term. Consider Thonny for the first 2--3 lessons to reduce setup burden, then migrate to VS Code or Theia IDE.

Thought Experiments

1. *"Why do you think the Python installer has a checkbox called 'Add Python to PATH'? What is a PATH, and what would happen if Python weren't on it? Think about it like a phonebook --- your operating system looks up programs by name. If Python isn't in the phonebook, what happens when you type* `python3`*?"*
2. *"We're installing two things: Python and VS Code. What role does each one play? If you had only Python and no VS Code, could you still write programs? How? What would you lose?"*
3. *"Virtual environments keep each project's dependencies separate. Imagine two school projects --- one requiring a specific edition of a textbook that contradicts the edition another class uses. What would happen if you tried to use both editions at the same time? How does a virtual environment solve the programming equivalent of this problem?"*
4. *"Why do we verify every installation immediately after completing it? What could go wrong between downloading something and confirming it works? Name three things that could silently fail."*

Lesson 0.2: Configuring Your Screen Reader for VS Code

Instructional Cues Before showing any configuration screens, do an unguided exploration: "Open VS Code or Theia IDE, navigate around with only your keyboard and screen reader, and come back in ten minutes. Tell me what you found, what worked, what was confusing." This builds genuine familiarity and identifies that student's specific challenges.

Map each configuration step to a problem the student already identified. This makes the configuration feel purposeful rather than arbitrary.

Screen Reader Specific Guidance NVDA users:

- Enable "Use UI Automation to access VS Code" in NVDA preferences (Tools -> Manage Plugins, or via NVDA+N -> Preferences -> Settings -> Advanced -> Use UI Automation where available). This dramatically improves reading accuracy in VS Code.
- Set "Report dynamic content changes" to "All" to hear error highlights and IntelliSense popups.
- Configure Browse Mode to be OFF by default in VS Code --- programming requires forms mode (interactive mode) at all times.
- Speech rate: many experienced NVDA users run at 70--90% of maximum speed. For new students, 40--60% allows better comprehension of code read aloud.

JAWS users:

- JAWS has VS Code-specific scripts maintained by the JAWS community. Check `FreedomScientific.com` and GitHub for the latest VS Code JAWS scripts.
- PlaceMarkers in JAWS can mark frequently visited locations in a file --- teach `Ctrl+Win+K` to set and `Ctrl+Win+J` to jump.
- JAWS Verbosity settings: set "Announce line numbers" to ON for code files.

VoiceOver (macOS) users:

- VoiceOver interacts with VS Code via the macOS Accessibility API. Ensure VS Code is not run with elevated permissions (which can break accessibility API access).
- Use `VO+Shift+Down` to enter interactive elements (the editor, terminal), then navigate normally.
- Enable "Speak text under mouse" in VoiceOver Utility -> Verbosity if the student uses residual vision with a mouse.
- VoiceOver's "item chooser" (VO+I) can provide a quick list of interactive elements in the current window.

Windows Narrator users:

- Narrator is less feature-rich than NVDA or JAWS for programming. It works but may require more keystrokes.
- Narrator scan mode must be OFF in VS Code for normal keyboard input to work. Toggle with Caps Lock+Space.
- Consider recommending the free NVDA screen reader as an upgrade for students using Narrator, as NVDA's VS Code support is significantly better.

Orca (Linux) users:

- Orca works well with VS Code on Linux via the AT-SPI2 accessibility interface.
- Use `Orca+H` for "say all" from cursor position.
- Ensure the `at-spi2-core` package is installed: `sudo apt install at-spi2-core`.
- Orca's "flat review" mode (`Orca+KP_Enter` or `Orca+Insert+KP_Enter`) is useful for reading lines that are not focus-accessible.

Indentation Configuration --- Step by Step This is the most important configuration in the entire course. Walk through it during the first session and verify it is working before writing any Python.

Test procedure:

1. Create a temporary file `test_indent.py`.
2. Write:
   ```
   if True:
       print("indented")
   print("not indented")
   ```
3. Navigate line by line with Down arrow.
4. The screen reader should announce the indent level on line 2 before reading its content.
5. On line 3, the indent should return to zero.

If announcement is not happening, troubleshoot screen reader settings before proceeding.

Thought Experiments

1. *"A sighted programmer can look at a Python file and immediately understand its structure from indentation --- like an outline. How does your screen reader translate that same 2D visual structure into a 1D audio stream? What information might be lost in that translation?"*
2. *"VS Code has hundreds of keyboard shortcuts. No sighted programmer uses all of them --- they use the 15 or 20 that matter for their workflow. If you could only keep 10 shortcuts, which 10 would make the biggest difference for writing Python? Why?"*
3. *"The Accessible View (Shift+Alt+F2) converts complex visual elements into flat text. What kind of information in VS Code might require this conversion? Can you think of a scenario where the 'visual' presentation of something would be unusable without translation?"*

Lesson 0.3: Setting Up Your VS Code Workspace

Alternative Workspace Tools If VS Code proves too complex initially, consider these alternatives as temporary or permanent solutions:

Tool	Platform	Accessibility Notes
IDLE (bundled with Python)	All	Very simple UI. Limited features but highly keyboard accessible. Good for first programs.
Thonny	All	Beginner-focused. Has a "simple mode." Some screen reader users find it more approachable.
Mu Editor	All	Designed for education. Simpler than VS Code. Python 3 only.
Jupyter Notebook (terminal)	All	Text-based notebook. Accessible for structured exploration.
Braille Note Apex/Touch	Device	For students using these notetakers, Python can be run via their built-in terminals.
Replit	Web	No install required. Accessibility varies; test before recommending.
NVDA Remote + VS Code	Windows	Student uses their own screen reader on their own machine while teacher controls a shared session.

Thought Experiments

1. *"Programmers organize their code into folders by project. What happens if every programmer at a company kept all their files in one shared folder with no organization? How does this analogy apply to organizing your own Python projects?"*
2. *"VS Code Extensions add features to the editor the same way apps add features to a phone. If you could design one VS Code extension specifically for blind Python programmers that doesn't exist yet, what would it do?"*

Lesson 0.4: Python Environments and Terminal Usage

Terminal Proficiency as a Prerequisite Before writing any Python, ensure the student can fluently:

- Navigate to a directory with `cd`
- List files with `ls` (macOS/Linux) or `dir` (Windows)
- Create a directory with `mkdir`
- Print the current directory with `pwd` (macOS/Linux) or `cd` with no arguments (Windows)
- Use Tab completion for filenames

- Use Up/Down arrow for command history
- Use `Ctrl+C` to stop a running program

If these are not automatic, spend additional time on terminal basics. Terminal fluency is foundational to everything else.

Alternative Terminals for Accessibility

Terminal	Platform	Screen Reader Compatibility
Windows Terminal	Windows	Excellent with NVDA/JAWS (use UI Automation mode)
cmd.exe	Windows	Good; legacy but very reliable
PowerShell	Windows	Good with NVDA; some JAWS quirks
macOS Terminal	macOS	Excellent with VoiceOver
iTerm2	macOS	Good; more features, slightly more complex
GNOME Terminal	Linux	Good with Orca
Konsole	Linux	Good with Orca
Terminator	Linux	Good; split-pane support

Thought Experiments

1. *"Virtual environments isolate each project's dependencies. Why would this matter for a professional programmer with 20 active projects? What could go wrong without them?"*
2. *"When you type `python3 hello.py` and press Enter, what is the operating system doing? What is Python doing? Walk through it step by step as if explaining to someone who has never used a computer."*
3. *"`uv add requests` downloads and adds a dependency to the current Python project. What are the risks of running code written by strangers? What does PyPI (the Python Package Index) do to reduce these risks?"*

---

6. Unit 1 --- Foundations and Getting Started

Lesson 1.1: Welcome to Computer Science

Instructional Cues Open with **the "Human Computer" activity** --- this is the most valuable exercise in the entire unit for blind students. The student gives verbal, step-by-step instructions for a simple physical task. The teacher follows them literally, making no inferences. This demonstrates the core challenge of programming: computers are perfectly literal and do not fill in gaps.

Suggest tasks that connect to the student's daily experience:

- Instructions for making tea or coffee
- Instructions for navigating from the classroom door to their desk
- Instructions for finding a contact in their phone using a screen reader

After the activity, debrief: *"Where did your instructions assume I could see? Where did they assume I already knew something? A computer makes neither assumption."*

Connecting Computing to Accessibility Technology Begin with what the student already knows:

- Their screen reader is software. It runs on hardware. It was written by programmers. It converts visual information (text rendering) into audio output.
- Ask: "What are the inputs to your screen reader? What are the outputs? What would happen if a website used images of text instead of real text --- how would that change what your screen reader can do?"
- Connect to ARIA attributes, alt text, and semantic HTML as programming concepts that directly affect the student's daily experience.

Thought Experiments

1. *"Computer science is described as the study of problem-solving, not just programming. What non-programming problems have you solved using computational thinking --- breaking a problem down into steps, looking for patterns, and abstracting the details? Describe one example from daily life."*

2. *"Your screen reader converts text to audio in real time. What 'algorithm' do you think it uses? What decisions does it make --- when to pause, how to handle punctuation, what to do with symbols? What would a flowchart of those decisions look like?"*

3. *"Early computers could only run one program at a time. Modern computers run hundreds simultaneously. Without changing the physical hardware, what concept makes this possible? (Hint: the answer is software --- specifically, an operating system.)"*

4. *"Software has often been described as 'making the hardware do something useful.' But hardware without software is just metal and silicon. In what sense is software more important than hardware? In what sense is hardware more important?"*

5. *"Accessibility technology like screen readers, braille displays, and audio description didn't exist when computers were first invented. Who decided these should be built? What does this history tell you about who gets to shape what technology does?"*

Additional Resources for This Lesson

- **"The Computer Boys Take Over" by Nathan Ensmenger** --- accessible history of how programming became a profession.
- **"Technically Wrong" by Sara Wachter-Boettcher** --- explores how biases in tech affect users, including disabled users. Good for older students.
- **American Foundation for the Blind (AFB) CareerConnect** --- profiles of blind professionals in STEM fields, including software engineers. Good for motivation and role models.
- **"Haben: The Deafblind Woman Who Conquered Harvard Law" by Haben Girma** --- autobiography of a deafblind lawyer and disability rights advocate. Haben's work directly involves technology access.
- **Code.org's "How Computers Work" video series** --- audio-described versions available. Covers hardware and software basics accessibly.

Lesson 1.2: Your First Python Program

Instructional Cues Before any code: Ask the student to predict. "If I write `print("Hello, World!")` and run it, what do you think will appear in the terminal? Where exactly will it appear?" Let them answer, then verify together.

Make the first error deliberate. Remove a closing parenthesis, run the program, and navigate to the error message together. Establish the error-reading protocol:

1. Terminal output appears after the code.
2. Read from the bottom of the error upward --- the bottom is the most specific message.
3. Line number tells you where to go. Use `Ctrl+G`.
4. Error type (`SyntaxError`, `NameError`) narrows the category of problem.

The REPL as the first environment. Before creating any files, start with `python3` in the terminal and type `print("Hello, World!")` directly. The immediate audio feedback of the REPL establishes cause and effect more directly than the file -> run -> find output workflow.

Common First Errors and How to Explain Them

Error	Likely Cause	Explanation for Student
`SyntaxError: unexpected EOF while parsing`	Missing closing `"`, `)`, or `]`	"Python reached the end of the file before it expected to. It was waiting for something --- probably a closing character."

Error	Likely Cause	Explanation for Student
`NameError: name 'print' is not defined`	Capital P in `Print`	"Python is case-sensitive. `Print` and `print` are completely different names. Only `print` (lowercase) is the built-in function."
`IndentationError`	Extra space before first line	"Python's first rule: top-level code cannot be indented. Every line that is not inside a function or class must start at column 0."
`SyntaxError: invalid syntax` at `:`	Using = instead of == in a comparison	"The assignment operator = doesn't belong inside an if condition. For comparison, you need the double equals ==."

Thought Experiments

1. *"`print()` produces output in the terminal. What would a program be like with NO output at all? Could such a program still do something useful? (Hint: think about programs that read files, send network requests, or modify a database.)"*
2. *"Python is an interpreted language --- each line is read and executed one at a time. Other languages are compiled --- the entire program is translated before any of it runs. How does this affect the kind of errors you encounter, and when you encounter them?"*
3. *"The `#` symbol starts a comment --- text that Python ignores entirely. Why do comments exist? If the computer ignores them, who are they for? Could you write good programs with no comments? Should you?"*
4. *"'Hello, World!' is the traditional first program in almost every programming language. Why do you think this tradition exists? What does this simple program prove about a working environment?"*

Alternative Resources for This Lesson

- **"Invent Your Own Computer Games with Python" by Al Sweigart (free online):** Text-based games that produce only terminal output --- perfectly accessible. Chapter 1 is a good companion to this lesson.
- **CS Unplugged --- "Parity Magic" activity:** Teaches binary representation and algorithms without a computer. Can precede or follow this lesson.
- **Python REPL via iOS/Android with screen reader:** Several apps provide a Python REPL on mobile. For students whose phone screen reader is more familiar than their computer setup, this may be a comfortable way to experiment.

Lesson 1.3: Variables and Simple Data Types

Instructional Cues **The labeled box metaphor** works well across modalities --- introduce it verbally and reinforce with tactile analogy if helpful. Provide a physical box (a small container) with a label stuck to it and a slip of paper inside representing the value. The box remains; the paper changes. The label stays the same; the contents change with reassignment.

`type()` as a diagnostic reflex: Begin every data type lesson with `type()`. `print(type(42))` -> `int`. `print(type("hello"))` -> `str`. Make this a reflex the student uses whenever they're unsure what kind of data they have. For a student who cannot visually scan code for context clues, `type()` is often the only reliable way to confirm what a variable contains.

The string/number audio confusion: When a screen reader reads `"42"` and `42`, they sound identical. This is a critical accessibility issue. Address it directly:

- *"Your screen reader will read these the same way. The only reliable way to tell them apart is `type()`. Use it. Constantly."*
- Demonstrate the consequence: `"42" + 1` raises `TypeError`. `42 + 1` gives `43`.

Multiple Explanations for Variable Naming Rules Provide all of these; let the student choose what sticks:

- *Rule-based:* Must start with letter or underscore. May contain letters, digits, underscores. May not be a Python keyword.
- *Analogy:* "Variable names are like legal names. They can't start with a number (no one's legal name is '2Fast'), can't contain hyphens (the name Mc-Donald would confuse paperwork), and can't be reserved words like 'True' or 'if' that already mean something official."
- *Mnemonic for invalid starts:* "Numbers first? Never. Hyphens? Horrible. Spaces? Surely not."

Thought Experiments

1. *"If variables are labeled boxes, what happens when you do `x = 5` and then `y = x` and then `x = 10`? What is `y` now? Does reassigning `x` affect `y`? Why or why not?"*
2. *"Python is 'dynamically typed,' meaning a variable can change its type at any time: `x = 5` then `x = 'hello'`. Many languages forbid this. What bugs could dynamic typing accidentally create? What flexibility does it offer?"*
3. *"The integer `42` and the string `'42'` look identical when spoken aloud. How many other pairs of values can you think of that might be ambiguous when*

a screen reader reads them? What coding habits help prevent confusion?"

4. *"Constants in programming represent values that never change --- like `SPEED_OF_LIGHT` or `MAX_USERS`. Python has no true constants (you can always reassign them). Why do programmers use `CAPS_SNAKE_CASE` as a convention for constants even though Python can't enforce them?"*

Alternative Resources

- **CS50 Python (Harvard, edX):** David Malan's accessible video lectures. Closed-captions and transcripts available for all lessons. Covers variables and types in Week 0.
- **Python.org official tutorial:** Plain HTML, navigable by screen reader. Section 3 covers numbers, strings, and basic types.
- **"Python for Everybody" by Dr. Chuck Severance (free online):** Audio-described video lectures available. Chapter 2 covers variables and expressions.
- **Codecademy "Learn Python 3":** Web-based coding environment. Accessibility is partial --- code editor is accessible but some UI elements are not. Worth testing with the student's specific setup.

---

7. Unit 2 --- Numeric Computations and I/O

Lesson 2.1: The Software Development Process

Instructional Cues This is a conceptual lesson. Resist the urge to rush to code. Spend the full session on design and pseudocode.

The "Five Whys" technique: Before writing pseudocode, ask the student to state the problem in one sentence. Then ask "why?" five times to reach the root of what the program really needs to do. This technique from Lean manufacturing translates well to programming.

Example: "My program calculates a restaurant tip."

- Why? "So the user knows how much to pay."
- Why do they need to calculate it? "The tip amount isn't printed on the bill."
- Why? "The restaurant leaves the amount to the customer's discretion."
- So the real problem is: "Given a bill amount, calculate 15%, 18%, and 20% tip options, plus the total."

This more complete problem statement produces a better program design.

Pseudocode format for screen reader users: Plain `.txt` files work best for pseudocode. Avoid word processors or special formatting tools that add accessibility overhead. The student should be able to read their pseudocode fluently while writing their Python.

Thought Experiments

1. *"In engineering, a bridge is designed many times before a single beam is placed. The cost of redesigning a blueprint is near zero; the cost of redesigning a half-built bridge is enormous. How does this ratio (cheap to redesign early, expensive to redesign later) apply to software?"*
2. *"The design phase involves thinking about edge cases --- unusual inputs. For a program that calculates how many buses are needed to transport N students, with each bus holding 40 students, what edge cases exist? (Consider: What if N=0? What if N=40? What if N=41? What if N is negative?)"*
3. *"Some agile software development methodologies say 'working software over comprehensive documentation.' Does this conflict with the value of design? How do you reconcile the two?"*
4. *"Testing is described as part of the development process. But some teams test after development is complete. What is lost by testing at the end rather than continuously throughout? What is gained?"*

Lessons 2.2--2.5: Expressions, Input, Types, Math

Instructional Cues `input()` **always returns a string --- make it tangible:**

```
user_input = input("Type the number 5: ")
print(type(user_input))   # <class 'str'>
print(user_input + 1)      # TypeError
print(int(user_input) + 1) # 6
```

Run this sequence and let the student hear both the error and the fix. The `TypeError` message should be read aloud and analyzed before the fix is shown.

Order of operations --- the hearing test: Read `10 + 5 * 2` aloud and ask for the answer before Python is consulted. Most students say 30; Python says 20. This is a memorable demonstration of why precise symbol reading matters. "Your ear parsed it as 'add 10 and 5, then multiply by 2.' Python parses it as '10 plus the result of 5 times 2.'"

`math` module exploration via REPL:

```
import math
dir(math)  # Lists all functions in the module
help(math.sqrt)  # Reads the docstring aloud
```

`dir()` and `help()` are screen-reader-friendly tools for exploring any module. Teach them as first-resort documentation tools.

Thought Experiments

1. *"Type conversion functions like `int()` and `float()` can fail: `int('hello')` raises a `ValueError`. Who is responsible --- the programmer who called `int()` on untrusted input, or the user who typed 'hello'? How should a well-designed program handle this?"*
2. *"`math.sqrt(25)` returns `5.0`, not `5`. Why does a function that logically produces a whole number return a float? What would break if it returned an integer instead?"*
3. *"Floating-point numbers in computers are approximations --- `0.1 + 0.2` gives `0.30000000000000004`. Describe a real-world application where this imprecision could cause serious problems. What strategies exist to avoid it?"*
4. *"The `//` operator performs floor division: `7 // 2 = 3`. The `%` operator gives the remainder: `7 % 2 = 1`. Together they completely describe integer division. What real-world problem does each solve? Describe a program that would need both."*

Alternative Resources

- **"Automate the Boring Stuff with Python" Chapter 1--3 (free online):** Covers expressions, data types, and input with practical exercises. Plain HTML, accessible.
- **Khan Academy "Introduction to Programming":** Video series with transcripts. Good for visual-thinker students who benefit from hearing concepts explained differently.
- **SoloLearn Python course:** Mobile app with accessible exercises. Works well with iOS VoiceOver and Android TalkBack for students who prefer mobile.
- **"Python Crash Course" by Eric Matthes:** The primary textbook for this curriculum. Chapter 2 aligns directly with this unit.

---

8. Unit 3 --- Control Flow, Decisions, and Loops

Lessons 3.1--3.2: Conditionals

Instructional Cues **Verbal simulation protocol:** Before running any conditional code, have the student perform the simulation verbally:

1. "What is the value of `score`?"
2. "Evaluate the condition: is `85 >= 90`? That's False."
3. "Because the condition is False, the if block is skipped."
4. "Evaluate the next condition: is `85 >= 80`? That's True."
5. "The elif block runs. We print 'Grade: B'."

This 5-step protocol should become automatic before the student runs any code. It builds the mental simulation ability that is the core of debugging.

The assignment-vs-comparison trap:

```
# This is a bug that is SILENT in some languages
if score = 90:   # SyntaxError in Python (good!)
    print("Perfect!")
```

Python raises a `SyntaxError` here, which is actually a safety feature. Other languages (C, JavaScript) allow this and it becomes a silent logic bug. Appreciate Python's strictness here.

Thought Experiments

1. *"If Python didn't have `elif`, only `if` and `else`, could you still write a grading program? Show how. What is the cost of doing it without `elif`?"*
2. *"The `else` clause at the end of an if-elif chain is a catch-all --- it runs if nothing else matched. What is the programmer asserting when they write `else: print('Invalid input')`? What assumptions about the universe of possible inputs does this make? When might those assumptions be wrong?"*
3. *"Boolean logic (`and`, `or`, `not`) comes from mathematics (George Boole, 1847). Boolean logic also describes how circuits in a computer work. In what sense is every program you write built on top of millions of tiny `and`, `or`, and `not` operations happening in hardware?"*
4. *"Short-circuit evaluation: in `if A and B`, Python doesn't evaluate `B` if `A` is False (because the result is already False). In `if A or B`, Python doesn't evaluate `B` if `A` is True. When is this behavior a useful feature rather than just an optimization?"*
5. *"Consider a program that decides whether to grant access based on two conditions: correct password AND recognized device. Map this to Python `and` logic. Now consider: unlock if either condition passes (maybe for emergency access). Map this to `or`. What are the security implications of each design?"*

Alternative Resources

- **CS Unplugged "Binary Numbers" activity:** Physical, non-computer activity that illustrates Boolean thinking. Can be adapted for tactile use.
- **"Think Python" by Allen B. Downey (free online):** Chapter 5 covers conditionals with clear explanation and exercises. HTML version is screen-reader-friendly.
- **Parsons Problems (online):** Code ordering exercises --- the student arranges given lines into correct order. Does not require typing code from

scratch. Available at parsons.problemsolving.org (accessibility varies; test first).

Lessons 3.3--3.4: Lists and For Loops

Instructional Cues Mental models for lists --- choose one per student:

- *The train:* Each car has a number starting at 0. Car 0 is at the front. You can add cars to the end, remove from anywhere, and swap contents.
- *The playlist:* Songs in order, starting at position 0. You can shuffle, sort, add, or remove tracks.
- *The physical stack of cards:* A sequence of items that can be addressed by position.

The for loop as a bookmark reader: *"Imagine Python has a finger that moves along a list. It starts at position 0. It reads what's there, runs the loop body, moves the finger to position 1, reads, runs the body, and continues until the finger falls off the end."*

Tracing loops audibly --- required exercise: For the first five loops students write, require them to verbally trace every iteration before running the code:

- "Iteration 1: `fruit` gets 'apple'. We print 'I like apple.'"
- "Iteration 2: `fruit` gets 'banana'. We print 'I like banana.'"
- etc.

Non-Obvious Accessibility Strategies

- **Printing list contents accessibly:** A bare `print(my_list)` produces `['apple', 'banana', 'cherry']` with brackets and quotes. This is cluttered when read aloud. Teach the clean print pattern early:

  ```
  for i, item in enumerate(my_list):
      print(f"  {i}: {item}")
  ```

  This gives numbered output, one item per line, that is easy to navigate and understand when heard.

- **`len()` as a safety check before index access:** Teach students to always know the length before indexing into a list. This prevents `IndexError`. Make `len(my_list)` the first thing printed when debugging a list problem.

- **Index arithmetic verbalized:** When accessing `my_list[-1]`, say "the last element, which is at index `len(my_list) minus 1`, which Python shortens to `-1`." Connecting the shorthand to the underlying meaning prevents confusion.

Thought Experiments

1. *"Zero-based indexing means the first element is at position 0. This feels unnatural but is standard across almost all programming languages. Why? (Hint: think about memory addresses --- the address of the first element, and the address of any element as an offset from there.)"*
2. *"A list allows duplicate values. A set (which you'll learn later) does not. When would duplicates in a list be meaningful data? When would they indicate an error in your program?"*
3. *"A `for` loop iterates over every element in a list. Is there any computation a `for` loop can do that a `while` loop cannot? Is there any computation a `while` loop can do that a `for` loop cannot?"*
4. *"What happens when you iterate over an empty list: `for item in []:`? Does Python crash, print an error, or do nothing? Why is 'do nothing' the correct behavior rather than an error?"*

Lesson 3.5: String Operations

Instructional Cues **Immutability demonstration:**

```
text = "hello"
text.upper()         # Creates a new string --- doesn't modify text
print(text)          # Still "hello"
text = text.upper() # Now reassigned
print(text)          # "HELLO"
```

This demonstration catches a persistent beginner misunderstanding. Ask the student to predict the output of `print(text)` after `text.upper()` before running it.

Method chaining verbalized: `"  hello world  ".strip().title()` --- read this as: "Start with the string. Strip whitespace from both ends. Then convert the result to title case."

Thought Experiments

1. *"Strings are immutable --- they cannot be changed, only replaced. Lists are mutable --- they can be changed in-place. What architectural difference between strings and lists makes one mutable and the other not? (Think about how they're stored in memory.)"*
2. *"`.split()` splits a string into a list of words. `.join()` combines a list of strings into one string. They are inverses. What is the inverse of `.replace()`? Does it have a direct inverse? If not, how would you reverse a replacement?"*

3. *"String formatting with f-strings:* `f'Value is {x:.2f}'` *formats* `x` *as a float with 2 decimal places. Why does a programming language need a mini-language for formatting numbers? What would programs look like if we had to format every number manually with division and modulo?"*

Alternative Resources

- **Python String Methods documentation** (docs.python.org/3/library/stdtypes.html#string-methods): Plain HTML table of all string methods. Screen-reader navigable. Essential reference.
- **"Python for Data Analysis" by Wes McKinney:** Chapter on string manipulation provides real-world examples beyond what this curriculum covers. For motivated students.

---

9. Unit 4 --- Data Structures: Lists and Dictionaries

(See all lessons above for list material; this section adds dictionary-specific guidance.)

Dictionary Lessons

Instructional Cues Multiple mental models for dictionaries:

- *Real dictionary:* You look up a word (key) to find its definition (value). Words are unique; definitions can be anything.
- *Contact list:* Each name (key) maps to a phone number (value). No two contacts have the same name in a well-maintained list.
- *JSON configuration file:* Settings names map to their values. This foreshadows Unit 8.

The `.get()` method vs. direct access:

```
# This crashes on missing key
print(my_dict["nonexistent_key"])   # KeyError

# This returns a default --- never crashes
print(my_dict.get("nonexistent_key", "Not found"))  # "Not found"
```

For a screen reader user who may not immediately catch a crash in terminal output, teaching `.get()` as the default access pattern reduces debugging overhead.

Thought Experiments

1. *"Dictionaries use keys to retrieve values in O(1) time --- essentially instant regardless of dictionary size. Lists use indices, also O(1). But searching*

a list for a specific value (not by index) takes O(n) time --- proportional to list length. When does this difference matter in practice?"

2. *"Your dictionary cannot have duplicate keys --- adding a second value for an existing key replaces the first. What real-world data problems does this property solve? What problems does it create?"*

3. *"JSON (JavaScript Object Notation) is essentially Python dictionaries written in a slightly different format. What does it mean that a data format invented for JavaScript has become the universal language of data exchange? What does this tell you about the value of common standards?"*

4. *"A 'nested dictionary' contains other dictionaries as values. Model a university course catalog as a nested dictionary. What are the outer keys? What are the inner keys? What would* `catalog['CS']['101']['instructor']` *represent?"*

---

10. Unit 5 --- User Input and While Loops

Thought Experiments

1. *"A while loop with a flag variable and a while loop with* `break` *can often accomplish the same thing. Compare these two equivalent programs and explain when you would prefer each:*

```
# Version A: flag
active = True
while active:
    cmd = input("> ")
    if cmd == "quit": active = False
    else: print("You said:", cmd)

# Version B: break
while True:
    cmd = input("> ")
    if cmd == "quit": break
    print("You said:", cmd)
```

Which is clearer? Which is easier to extend if you add more exit conditions?"

2. *"An infinite loop that waits for user input is the core pattern of every interactive program --- email clients, video games, operating systems. Describe the 'event loop' of your screen reader in terms of while loops and if statements. What is it waiting for? What does it do when it gets an event?"*

3. *"The `continue` statement skips the rest of the current iteration. An alternative is to wrap the loop body in an `if` statement. When is `continue` cleaner? When does the `if` approach produce clearer code?"*

---

11. Unit 6 --- Functions

Thought Experiments (in addition to existing guide)

1. *"A function is described as a 'black box' --- you give it inputs and get outputs without knowing the internal details. What are the benefits of this abstraction for the programmer who writes the function? For the programmer who uses it? What are the limits of this metaphor?"*
2. *"Default parameter values: `def greet(name='World')`. This allows the function to be called with or without an argument. How does this enable backward compatibility --- adding a new feature without breaking existing code that calls the function?"*
3. *"Functions should 'do one thing.' What does 'one thing' mean? Is `calculate_and_print_tax()` one thing or two? If you separate it into `calculate_tax()` and `print_tax()`, what becomes possible that wasn't before?"*
4. _"Recursive functions call themselves. `factorial(n) = n _ factorial(n-1)`. What is the base case and why is it required? What goes wrong if a recursive function has no base case?"*
5. _"A function is a way to name a piece of code. Why does naming matter in programming? What is the cognitive difference between `x = a _ b + c -` dand`x = calculate_net_profit(revenue, cost, overhead, taxes)`?"*

Alternative Resources

- **"Structure and Interpretation of Computer Programs" (SICP) --- MIT OpenCourseWare:** Conceptually deep treatment of functions as first-class objects. Lectures have transcripts. For advanced students.
- **Exercism.io Python Track:** Mentor-guided exercises with community feedback. Web platform with reasonable accessibility. Students can complete exercises at their own pace.
- **Python Tutor (pythontutor.com):** Step-by-step execution with variable state display. The text-based "state panel" below the code is screen-reader accessible and shows variable values at each step.

---

12. Unit 7 --- Classes and Object-Oriented Programming

Extended Instructional Cues

Staged introduction to OOP --- do not rush to syntax:

Stage 1 (no code): Discuss the concept. What is an object? What does "bundling data and behavior" mean? Use tangible examples the student knows.

Stage 2 (pseudocode): Write a class in plain English before Python syntax:

```
Class: BankAccount
  Has: owner_name, balance, account_number
  Can do: deposit(amount), withdraw(amount), get_balance()
```

Stage 3 (Python skeleton): Write the class structure with empty methods and docstrings.

Stage 4 (full implementation): Fill in method bodies.

This staging prevents syntax confusion from obscuring the conceptual learning.

`__str__` is mandatory for accessibility: Every class a student creates must define `__str__`. Without it, `print(my_object)` produces `<__main__.BankAccount object at 0x7f...>` --- meaningless when heard. The requirement to define `__str__` also reinforces thinking about what the object's most important information is.

```python
def __str__(self):
    return f"BankAccount(owner={self.owner_name}, balance=${self.balance:.2f})"
```

Thought Experiments

1. *"Encapsulation means hiding internal details. A car's engine is encapsulated under the hood --- you don't need to know how fuel injection works to drive. What are the analogous 'hidden internals' in a BankAccount class? What should be private? What should be public? Why?"*
2. *"Inheritance creates 'is-a' relationships. Composition creates 'has-a' relationships. A `SavingsAccount` is-a `BankAccount`. A `Bank` has-a list of `BankAccount` objects. Design a simple banking system using both relationships. What would go wrong if you tried to use only inheritance?"*
3. *"Method overriding allows a subclass to replace a parent's method. When is this the right design? When should you call `super().method()` inside the override vs. replace the method entirely?"*
4. *"Python allows multiple inheritance (inheriting from two parent classes). Most languages do not permit this. What problems does multiple inheritance create? What problems does it solve? Would you use it?"*

5. *"Object-oriented programming was supposed to make code more reusable. Has it delivered on this promise? What kinds of code reuse are well-served by OOP? What kinds aren't?"*

Alternative Resources

- **"Object-Oriented Programming in Python" by Michael Goldwasser and David Letscher:** Textbook specifically pairing Python and OOP concepts. More formal treatment for students who want depth.
- **"Fluent Python" by Luciano Ramalho:** Advanced but deeply accessible to screen readers. Covers Python's data model including `__str__`, `__repr__`, and special methods. For motivated students.
- **Real Python "Object-Oriented Programming in Python" tutorial (realpython.com/python3-object-oriented-programming/):** Long but thorough. Plain HTML.

---

13. Unit 8 --- Files and Exceptions

Extended Instructional Cues

Files as persistence --- make it tangible: Run a program that stores a number in a variable, quits, and starts again --- the variable is gone. Then run the same program storing the number in a file --- it persists. Ask: "What changed? What is different about a file vs. a variable?" Let the student reason through it before explaining.

The `with` statement --- demonstrate the alternative:

```
# Without 'with' --- dangerous
f = open("data.txt", "r")
contents = f.read()
# If an error occurs here, f.close() is never called
f.close()

# With 'with' --- safe
with open("data.txt", "r") as f:
    contents = f.read()
# f is closed automatically even if an error occurs
```

Exception types --- teach by demonstration, not memorization: Create a "crash collection" --- a series of small programs that each produce a specific exception. Let the student run each one, read the error, and discuss why it occurred before seeing the solution. The goal is pattern recognition, not memorization.

Thought Experiments

1. *"Files are persistent storage --- they survive when your program exits. Variables are temporary --- they're gone when the program ends. What determines which data should be stored in files vs. kept only in memory? What are the tradeoffs?"*

2. *"The* `with` *statement uses Python's 'context manager' protocol. This protocol says: 'when this block starts, do X; when it ends (even if it ends with an error), do Y.' What other situations in programming require guaranteed cleanup? What happens in real systems when cleanup is skipped?"*

3. *"Exceptions propagate up the call stack --- if function A calls B which calls C, and C raises an exception, it moves up to B, then A, until something catches it. If nothing catches it, the program crashes. Design the exception handling strategy for a program that reads a config file, parses it, and then connects to a database. Where should exceptions be caught?"*

4. *"JSON stores data as text. Binary formats (like* `pickle` *in Python) store data in a compact binary form. JSON is human-readable; binary is not. JSON is language-independent; pickle is Python-only. For each of these use cases, which format would you choose: (a) a config file the user might edit by hand, (b) a large dataset processed only by your own Python code, (c) data shared between a Python backend and a JavaScript frontend?"*

---

14. Unit 9 --- Testing and Data Analysis

Extended Instructional Cues

Tests as documentation --- make this concrete: Show two identical functions --- one with tests that describe the expected behavior, one without. Ask: "Which function is easier to understand without reading the implementation?" The tests are a specification in executable form.

Test-driven development (TDD) as a preview: For motivated students, introduce TDD: write the test before the implementation. This inverts the usual workflow:

1. Write a test that fails (because the function doesn't exist yet).
2. Write the minimum implementation to make the test pass.
3. Refactor.

This discipline is genuinely useful for beginners because it forces them to think about what a function should do before worrying about how.

Thought Experiments

1. *"Unit testing tests one function in isolation. Integration testing tests multiple components working together. System testing tests the entire program.*

Why have three levels rather than just one? What does each level catch that the others might miss?"

2. *"A test can only prove that a program behaves correctly for the inputs tested. It cannot prove correctness for all possible inputs. What does this mean for the value of testing? How do you decide which inputs to test?"*
3. *"Random numbers make testing difficult --- if the output changes every run, how do you write an assertion? Describe a strategy for testing a function that uses `random.randint()`. (Hint: think about seeding.)"*
4. *"CSV data from the real world is often 'dirty' --- missing values, extra commas, inconsistent formatting, encoding issues. What does a robust CSV parser need to handle? What assumptions would a naive parser make that would cause it to fail on real-world data?"*

---

15. Unit 10 --- Project-Based Learning

Extended Instructional Cues

Design document requirements: Require a written design document before any code is written. For a screen reader user, a plain text file works best. The document should include:

```
Project Name: [title]
One-sentence purpose: [what problem does this solve?]
Target user: [who will use this?]

Features (ranked by priority):
1. [most important feature]
2. ...

Data I will store: [what information needs to persist?]
Data structures: [which Python structures will hold this?]

Functions I will write:
- [function name](inputs) -> output: [brief description]
- ...

Main loop pseudocode:
[plain English description of program flow]

Edge cases I have considered:
- [what happens if the user enters nothing?]
- [what happens if the file doesn't exist?]
- ...
```

Iterative development enforcement: Require demonstrating a working version at the end of every session, no matter how minimal. "Working" means runnable and producing correct output for at least one input. A student who cannot demonstrate working code at session end needs to reduce scope, not work harder overnight.

Thought Experiments

1. *"A program designed for sighted users might use color to indicate status (red for error, green for success), icons to represent actions, and layout to group related information. How would you replicate each of these design patterns in a screen-reader-accessible terminal program?"*
2. *"Your contact manager has a search function. How many different search behaviors could be useful? Search by exact name match, partial match, phone prefix, email domain, notes keywords? What should the program do when there are 0 results? 1 result? Multiple results?"*
3. *"Error messages should be actionable --- they should tell the user not just what went wrong, but what to do next. Rewrite these unhelpful messages to be actionable: 'Error: invalid input', 'File not found', 'Operation failed.'"*

---

16. Unit 11 --- Capstone Project

Assessment Guidance

Process Checkpoints

Checkpoint	Timing	What to Evaluate
Written design document	Before any code	Is the scope realistic? Are features prioritized? Are data structures appropriate?
Data model working	End of week 1	Can items be created, stored in memory, and printed?
Core CRUD functional	End of week 2	Add, view, edit, delete all work?
Persistence working	End of week 3	Data survives program restart?
Error handling complete	End of week 4	Program handles all invalid inputs without crashing?
Accessibility review	End of week 4	Is all output meaningful when read aloud?
Final demo	End	Full demonstration with screen reader active

The Screen Reader Demo Rubric The final demonstration should be performed entirely with the screen reader active. Evaluate each:

Criterion	Excellent	Satisfactory	Needs Work
Menu navigation	Student navigates menu by listening; confirms choice before acting	Student navigates but occasionally needs visual cues	Student needs visual confirmation of menu state
Input prompts	All prompts clearly describe expected format and type	Most prompts are descriptive	Some prompts are vague or missing
Confirmation messages	Every action produces a clear spoken confirmation	Most actions confirmed	Some actions succeed silently
Error messages	All error messages are specific and actionable	Most errors give useful messages	Some errors produce generic or missing messages
Data persistence	Student demonstrates save and load by quitting and restarting	Save/load works but student doesn't demonstrate live	Save or load not fully functional
Code explanation	Student can explain any line of their code when asked	Student can explain most of their code	Student cannot explain sections of their code

---

17. Assessment Strategies

Accessible Assessment Design

Traditional programming assessments (written tests, timed coding) create unnecessary barriers for blind students. Use alternatives that assess the same competencies:

Think-aloud problems: Give the student a short piece of code and ask them to read it aloud and explain what it does, step by step. This assesses reading comprehension and debugging ability without requiring written answers.

Debugging exercises: Provide a working program and a broken version. Ask the student to identify the error using only their screen reader and debugging tools. This directly tests real-world skill.

Feature extension: Give a working program and ask the student to add a specific feature. This is a more realistic assessment than writing from scratch and allows scaffolding.

Verbal design review: Present a programming problem and ask the student to describe their approach --- data structures, functions, algorithm --- without writing any code. Assess reasoning quality.

Portfolio review: Maintain a collection of all programs written during the course. Assess growth, consistency, commenting quality, and the student's ability to explain their own work.

Grading Considerations

- Do not penalize for missing visual formatting (e.g., a missing newline that only matters visually).
- Do reward for accessibility features the student adds above requirements: clear prompts, confirmation messages, help text.
- Recognize that a blind student may spend significantly more time debugging than a sighted peer on identical problems. Factor this into time expectations.
- Consider allowing oral explanations as alternatives to written code comments for students who find comment-writing slow.

---

18. Supporting Diverse Needs Within the Visually Impaired Population

For Students with Low Vision

- Use high-contrast VS Code themes: Dark High Contrast, Light High Contrast (available under Color Themes in settings).
- Increase editor font size: `Ctrl+Shift+P` -> "Preferences: Open Settings" -> `editor.fontSize`.
- Enable zoom magnification: `Ctrl++` to zoom the VS Code window.
- Test whether syntax highlighting is visible with their specific visual condition. Some students benefit from; others are confused by color information they cannot fully perceive.

For Students Who Are Deaf-Blind

- All communication with the teacher must be through the braille display or tactile signing --- no spoken instruction.
- Screen reader audio verbosity settings must be managed by the student (since they cannot hear audio feedback during class).
- Consider pairing with a trained interpreter who knows both tactile communication and computing concepts.
- Review the Perkins School for the Blind's resources on deaf-blind education technology.

For Students with Physical Disabilities in Addition to Visual Impairment

- Switch access: VS Code is fully keyboard-navigable and can be operated with adapted keyboards, switch interfaces, or eye-gaze systems paired with virtual keyboards.
- Voice control: Dragon NaturallySpeaking and macOS Voice Control can both operate VS Code. Voice coding requires additional voice commands for symbols.
- Sip-and-puff devices: Can control keyboard input. Students using these will have different typing rates --- extend all time-based tasks.

For Students with Cognitive Differences in Addition to Visual Impairment

- Reduce the amount of new vocabulary per session.
- Use consistent metaphors across the entire course --- do not introduce a new metaphor for the same concept.
- Provide written summaries of every lesson that the student can review with their screen reader later.
- Break complex concepts into smaller steps; complete each step before introducing the next.
- Build in regular review sessions rather than only forward-moving instruction.

---

19. Teaching AI Tools Responsibly

AI-assisted coding is an optional source of explanation and critique, not an authority, substitute teacher, or replacement for learning to program. The course should remain fully teachable without paid AI access or a student account.

Course AI policy (shared by all language tracks). Students may use AI only for personal conceptual understanding and reinforcement. AI may not write, autocomplete, repair, complete, or generate course code or assessment solutions. The rest of this section should be read within that limit.

Instructor use of AI during preparation

An instructor may use an approved AI tool to brainstorm alternative explanations, generate draft practice inputs, propose misconceptions, or review a lesson outline. The instructor remains responsible for the content. Verify every technical statement against the target Python version and official documentation; execute code examples and tests; inspect accessibility with the intended tools; and check references, licensing, bias, and reading level. Never upload identifiable student work, accommodation plans, assessment answers, or confidential school material. Do not use AI-generated feedback as the sole basis for grading

or placement. Keep a record of substantive AI assistance when institutional policy or the publication process requires it.

Learning outcomes

Students should be able to explain what an AI coding assistant can and cannot reliably do; write a bounded prompt without exposing private information; verify generated explanations and code against documentation, tests, and course concepts; identify code they cannot understand or navigate; and disclose assistance when required.

Appropriate uses

Within the course policy, students may ask an AI tool to explain a Python concept in another way, for example what a dictionary is or why indexes start at zero, or to restate in plain words what an error category means. Students must not paste course code or assignment prompts into an AI tool and must not accept generated, repaired, or completed code. Model the permitted uses, and ask students to state their own understanding before and after consulting any explanation, and to check it against the official documentation and their own tests.

Uses that undermine learning or assessment

Do not permit AI to complete an individual assessment when the purpose is to measure independent design, implementation, code reading, or debugging. Do not accept generated code as evidence of mastery merely because it runs or passes one test. Because the course policy limits AI to conceptual explanation, no assignment or assessment permits AI-written code.

A student must be able to explain every submitted statement, identify dependencies, predict behavior for new inputs, modify the code, and debug a failure. Inability to do so is a learning need to address instructionally.

Python-specific failure modes

AI-generated Python may invent or misname APIs, recommend unnecessary packages, produce advanced syntax beyond the course level, use mutable default arguments, broad exception handling, unsafe deserialization, or fragile file paths. It may omit validation, resource cleanup, tests, and meaningful errors. It may also generate long lines, deeply nested indentation, unclear identifiers, or output difficult to review with speech or braille.

Teach students to consult official Python documentation, inspect package provenance, use a virtual environment, and test small examples. Simplify generated code into constructs the learner can explain. Prefer explicit loops and intermediate variables over compressed expressions when they improve comprehension and navigation.

Accessibility review

Before accepting AI-generated material, review it with the learner:

1. Can the learner navigate by line and indentation level?
2. Are nested blocks easy to identify through speech or braille?
3. Are identifiers descriptive and consistent?
4. Are lines and function bodies manageable?
5. Are prompts, labels, errors, and results understandable when read sequentially?
6. Does the program avoid conveying essential information only through color, layout, graphics, or timing?
7. Can the learner inspect output and errors without visual-only panels?

AI tools may themselves have inaccessible suggestion popups, chat panes, or completion workflows. Test the interface and offer an equivalent documentation- or teacher-supported route.

Privacy, security, copyright, and equity

- Never enter student records, individualized education program details, disability or medical information, passwords, API keys, private repository contents, or confidential school data into a public AI service.
- Follow district, school, institutional, and applicable data-protection policies; verify age requirements and account terms.
- Treat generated shell commands, installation steps, file operations, and network code as untrusted until reviewed.
- Verify a suggested package's name, maintainer, source, permissions, and necessity before installing it.
- Do not assume generated content is original or correctly licensed. Attribute sources and avoid submitting copied material.
- Provide a non-AI path. Do not grade students on access to a paid model, faster hardware, or a personal account.
- Discuss bias: generated examples may assume visual interaction, mouse use, graphical output, or a narrow range of users.

Verification protocol: Ask – Inspect – Verify – Adapt – Explain – Disclose

1. **Ask:** formulate a narrow question and state the student's current understanding.
2. **Inspect:** read the entire response; identify assumptions, unfamiliar syntax, dependencies, and claims.
3. **Verify:** consult official documentation; run the smallest safe experiment; test normal, boundary, and invalid inputs.
4. **Adapt:** simplify and revise for course level, security, maintainability, and assistive-technology access.

5. **Explain:** have the student explain the revised result and make a small change independently.
6. **Disclose:** record the tool and assistance when required by the assignment.

Activity-specific policy

- **Practice:** hints, explanations, and test suggestions may be allowed; complete solutions should be reserved for explicit code-review activities.
- **Homework/projects:** define allowed assistance in advance and require disclosure, verification, and explanation of the learner's contribution.
- **Quizzes/individual assessments:** do not permit AI to perform the skill being measured unless the assessment explicitly includes AI use or an accommodation requires it.
- **Accessibility support:** screen readers, braille, magnification, and approved access tools are not prohibited answer-generation assistance.

Discussion prompts

1. A generated program works on sample input but fails on an empty file. What should have been tested?
2. An assistant proposes a one-line comprehension that is difficult to navigate. Is shorter always clearer?
3. An assistant invents a package with a plausible name. How can the student verify it?
4. If AI can generate code, why do problem decomposition, testing, debugging, and explanation remain essential?

---

20. Alternative and Supplementary Resources

Textbooks and Reading

Resource	Level	Format	Accessibility
Python Crash Course, 3rd Ed. (Matthes)	Beginner	Print/ebook	Good; primary textbook for this curriculum
Python Programming: An Introduction to CS (Zelle)	Beginner	Print/ebook	Good; companion textbook
Automate the Boring Stuff with Python (Sweigart)	Beginner--Intermediate	Free HTML / print	Excellent; HTML version fully accessible
Think Python (Downey)	Beginner	Free HTML / print	Excellent HTML version

Resource	Level	Format	Accessibility
Learn Python 3 the Hard Way (Shaw)	Beginner	Print/ebook	Plain text exercises, highly accessible
Python for Everybody (Severance)	Beginner	Free HTML + video	Video lectures have transcripts; good accessibility
Fluent Python (Ramalho)	Advanced	Print/ebook	Dense but thorough; for motivated advanced students
Python Docs (python.org/3)	Reference	Free HTML	Excellent screen-reader accessibility; definitive

Online Courses and Platforms

Platform	Course	Screen Reader Compatibility	Notes
edX / CS50P (Harvard)	Introduction to Programming with Python	High --- video transcripts, accessible exercises	Free audit. Very high quality.
Coursera	Python for Everybody (Michigan)	Medium --- some interactive elements inaccessible	Free audit available
Codecademy	Learn Python 3	Partial --- editor accessible, some UI not	Requires testing with specific screen reader
SoloLearn	Python Core	High --- mobile app works well with VoiceOver/TalkBack	Good for mobile-first learners
freeCodeCamp	Python curriculum	Medium --- varies by module	Free; large community
Exercism	Python Track	High --- terminal-based, mentor feedback	Excellent for structured practice after basics
CodingBat	Python problems	High --- plain HTML, browser-based	Good for logic practice; no complex UI
replit.com	Any Python project	Medium --- test with student's setup	No-install option; accessibility variable

Video Resources

Resource	Description	Accessibility
CS50P (Harvard)	Full course lectures, free on YouTube and edX	Closed captions, transcripts; very high production quality
Corey Schafer Python tutorials (YouTube)	In-depth concept tutorials	Closed captions available
Tech With Tim Python (YouTube)	Project-based tutorials	Closed captions available
Sentdex Python tutorials (YouTube)	Large series from beginner to advanced	Captions available
Python.org Getting Started	Short official tutorials	Plain text, no video

Tools Specifically for Accessibility in Programming Education

Tool	Purpose	Notes
Python Tutor (pythontutor.com)	Step-by-step code execution with state display	Text-based state panel is screen-reader accessible
Mu Editor	Beginner-focused Python IDE	Simpler than VS Code; reasonable screen reader support
Thonny	Beginner IDE with step debugger	Variable viewer is navigable via keyboard
NVDA + Remote	Remote screen sharing for teacher support	Student's screen reader on their machine; teacher can see output
GitHub Codespaces	Cloud VS Code environment	Same accessibility as local VS Code; useful when local setup fails
Braille Institute Resources	General adaptive technology guidance	brailleinstitute.org
APH (American Printing House)	Educational materials for blind students	aph.org

Communities and Professional Organizations

Organization	Relevance
National Federation of the Blind (NFB) --- Science & Engineering Division	Blind scientists and engineers; mentorship opportunities
American Council of the Blind (ACB) --- Technology Interest Group	Accessible technology advocacy and resources
CSUN Assistive Technology Conference	Annual conference; many papers on computing education for blind students
SIGACCESS (ACM SIG on Computers and Accessibility)	Academic research on accessible computing
Ability Central	Technology resources for people with disabilities
Hadley Institute for the Blind	Professional development for educators of blind students
Perkins School for the Blind	Resources including STEM education for blind students
Carroll Center for the Blind	Technology training for blind adults; may have curriculum resources
Stack Overflow Blind/VI Community	Informal community; blind developers helping each other

---

21. Quick Reference Sheets

Error Message Reading Protocol

When an error occurs, guide the student through this sequence:

```
1. Navigate to terminal output (Ctrl+` or click panel)
2. Read from the BOTTOM of the error output upward
3. Note the ERROR TYPE (last line): SyntaxError, NameError, TypeError, etc.
4. Note the ERROR MESSAGE: specific description of what went wrong
5. Note the LINE NUMBER in the traceback
6. Jump to that line with Ctrl+G -> [line number] -> Enter
7. Read the line and the lines immediately before and after it
8. Categorize the error (see table below)
9. Fix and re-run
```

Error Type Quick Reference

Error Type	Common Cause	First Thing to Check
`SyntaxError`	Missing `:`, `)`, `"`, `]`, or `=` instead of `==`	The exact column number in the error message
`IndentationError`	Inconsistent spaces/tabs	Line indentation; use `cat -A` or `show whitespace` in VS Code
`NameError`	Variable not defined, or misspelled	Check spelling; check if variable was defined before use
`TypeError`	Wrong data type	Use `type()` on each variable involved in the operation
`ValueError`	Right type, wrong value	Check what you're converting: `int('hello')` fails; `int('5')` works
`IndexError`	List index out of range	Print `len(list)` and check what index you're trying to use
`KeyError`	Dictionary key doesn't exist	Use `.get()` or check with `if key in dict` first
`FileNotFoundError`	File doesn't exist at that path	Print `os.getcwd()` and verify the file path
`AttributeError`	Called a method that doesn't exist	Check spelling and whether the object is the type you think it is
`ZeroDivisionError`	Dividing by zero	Add a check before the division: `if denominator != 0`
`RecursionError`	Infinite recursion	Verify base case exists and is reachable

Accessible Output Patterns

```
# Instead of: print(my_list)
# Use:
print("Items in collection:")
for i, item in enumerate(my_list, 1):
    print(f"  {i}. {item}")
print(f"Total: {len(my_list)} items")

# Instead of: print(my_dict)
# Use:
print("Current settings:")
for key, value in my_dict.items():
    print(f"  {key}: {value}")

# Menu display:
print("\n--- Main Menu ---")
print("1. Add item")
print("2. View all items")
```

```python
print("3. Search")
print("4. Delete item")
print("5. Save and quit")
print("------------------")
choice = input("Enter option number (1-5): ")

# Confirmation messages:
print(f"Contact '{name}' added successfully.")
print(f"Found {len(results)} matching records.")
print("Data saved to contacts.json.")
print("No matching records found.")
print("Error: Name field cannot be empty. Please try again.")
```

VS Code Keyboard Shortcut Reference Card

```
NAVIGATION
  Ctrl+G          Go to specific line number
  Ctrl+Home       Jump to start of file
  Ctrl+End        Jump to end of file
  Ctrl+Shift+O    Go to function/class (symbol picker)
  F12             Go to Definition
  Alt+F12         Peek Definition
  Shift+F12       Find all References
  Ctrl+P          Quick open file by name

ERRORS AND DIAGNOSTICS
  F8              Next error/warning
  Shift+F8        Previous error/warning
  Ctrl+Shift+M    Open Problems panel
  Shift+Alt+F2    Accessible View (for hover/tooltip content)

EDITING
  Shift+Alt+F     Format document (auto-indent)
  Ctrl+/          Toggle comment on selected lines
  Tab             Indent selected lines
  Shift+Tab       Outdent selected lines
  Ctrl+D          Select next occurrence of word
  Ctrl+H          Find and replace

TERMINAL
  Ctrl+`          Toggle integrated terminal
  Ctrl+Shift+`    New terminal
  Up/Down arrows  Navigate command history
  Tab             Auto-complete filenames
  Ctrl+C          Stop running program
```

```
SCREEN READER
  Ctrl+Shift+P    Command Palette
  Ctrl+Shift+P -> "Toggle Screen Reader Optimized"
  Shift+Alt+F1    Accessibility help
```

22. Comprehensive Instructional Planning and Course Implementation

This section supplements the unit-by-unit notes with a full course implementation framework. Use it alongside Sections 1–21.

22.1 Course alignment and pacing

Map each unit to prerequisites, learning objectives, practice opportunities, and evidence of mastery. The sequence builds from environment setup and computing foundations through expressions and input/output, control flow, collections, functions, object-oriented programming, files, exceptions, testing, data analysis, and projects. Do not rush the transition from collections to functions or from functions to classes; learners need to trace and explain smaller programs first.

Plan time to learn the tools. Editor navigation, terminal use, indentation awareness, virtual environments, and reading tracebacks are course skills, not unrelated overhead.

22.2 Lesson planning template

For every lesson, prepare measurable objectives, a plain-language explanation, new vocabulary and spoken symbol conventions, a small example with expected output, a prediction question, guided and independent practice, a likely misconception or intentional bug, a formative check, and an optional extension.

Use a consistent rhythm: orient, retrieve prior knowledge, model, trace, predict, run, practice, debug, explain, and reflect.

22.3 Python-specific instructional checkpoints

- **Indentation:** teach indentation as syntax and structure. Identify the suite belonging to each `if`, loop, function, and class. Distinguish tabs from spaces and inspect whitespace.
- **Names and types:** ask learners to state the type and current value of important variables. Use `type()` for targeted experiments, not as a substitute for explanation.
- **Input conversion:** reinforce that `input()` returns a string. Require explicit conversion and validation before arithmetic.
- **Loops and collections:** trace index ranges, iteration variables, mutation, and empty collections. Distinguish iterating over values from indexing by position.

- **Functions:** identify parameters, arguments, local variables, return values, and side effects. Test functions with multiple inputs.
- **Classes:** connect each instance to its own state and distinguish class definitions from object creation and method calls.
- **Files and exceptions:** teach working directories, relative paths, context managers, expected exceptions, and meaningful recovery messages.
- **Testing:** begin with expected results and simple assertions; introduce a framework after learners understand what a test checks.

22.4 Accessibility and independence checkpoints

Ask the learner to demonstrate how to locate a function, identify the active indentation level, read a traceback, move to a reported line, run a program, and review output. If a step is inaccessible, troubleshoot the tool configuration and provide an alternate route before concluding that the programming concept is not understood.

Do not prescribe one screen-reader verbosity setting. Work with the learner to balance punctuation, indentation announcements, and speech rate. Ensure braille users can inspect symbols and line structure and low-vision students can adjust font, contrast, and magnification without losing keyboard access.

22.5 Assessment and feedback

Use low-stakes checks: predict output, explain a branch, trace a loop, select a collection, write a function with a clear contract, find and fix an exception or logic error, add a boundary test, or explain a design decision. Assess reasoning, correctness, code comprehension, testing, accessible interaction, and reflection. Do not grade speech, typing, or navigation speed as proxies for programming competence.

22.6 Differentiation and enrichment

Use starter code, worked examples, vocabulary previews, smaller practice sets, additional time, or alternative explanation modes where appropriate. Fade scaffolds to build independence. Enrichment can compare algorithms, improve validation, add tests, refactor repeated logic, or document trade-offs.

22.7 Project checkpoints

Require a problem statement, user needs, feature list, input/output plan, data model, and test plan before implementation. Review one working feature at a time. For a text adventure, test movement, inventory, invalid commands, and save/load behavior separately. For command-line tools and capstones, review persistence, validation, error handling, and the clarity of prompts and confirmations.

22.8 Collaboration and safeguarding

Coordinate with assistive-technology specialists and other professionals with the learner's participation. Keep confidential student information out of shared examples, public issue trackers, AI prompts, and source-control history. Use synthetic data and remove credentials from logs before sharing them.

22.9 Continuous improvement

At the end of each unit, record outdated setup steps, unclear directions, inaccessible output, and concepts needing another example. Verify software instructions against official documentation. Report reproducible issues with operating system, Python version, editor, screen reader, and steps—without sharing personal student information.

18.11 Rust Teacher's Guide

Purpose and course context

This guide supports the Rust curriculum for beginning programmers, including blind and visually impaired learners using screen readers, braille displays, magnification, or keyboard-first workflows. It supplements the complete lessons, topic index, and validation instructions.

The course uses stable Rust with the **Rust 2024 edition** and Cargo. Rust's ownership and borrowing rules are not incidental syntax; they are a central part of how learners reason about memory safety, data access, and program design. The official Rust book recommends that learners with no programming background first study general programming concepts. Complete shared foundations and ensure learners understand variables, functions, collections, and control flow before expecting them to work independently with ownership.

Outcomes and sequence

Complete the shared Computing Foundations Primer and Terminal Skills prerequisite, plus introductory programming preparation where needed. Then follow Units 0–11. Learners should use Cargo, compile and run programs, interpret compiler diagnostics, work with ownership and borrowing, model data with structs and enums, use pattern matching and collections, handle errors with Result and Option, define traits and functions, test behavior, and complete the capstone.

Use a predictable cycle: objective and vocabulary; trace a small code example; predict ownership and output; compile; inspect diagnostics; make one change; explain why the borrow checker accepts or rejects the program. Treat compiler feedback as a source of information, not an adversary.

Toolchain and Cargo

Install Rust with rustup. Verify `rustc --version` and `cargo --version`. Create a project with:

```
cargo new course_app
cd course_app
cargo run
cargo test
```

Explain Cargo.toml, Cargo.lock, crates, modules, build profiles, and the distinction between rustc and Cargo. Use the Rust 2024 edition consistently. Keep dependencies explicit and project-scoped. Teach learners to read the first relevant compiler diagnostic and follow the borrow checker explanation back to the expression or binding that created the constraint.

Teaching Rust concepts

Ownership, borrowing, and lifetimes

Teach ownership through concrete values and moves before introducing references. Distinguish Copy types from moved values. Explain shared references and exclusive mutable references, and connect borrowing rules to aliasing and mutation. Do not reduce lifetimes to punctuation or tell learners to add annotations until the code compiles; first ask what data must remain valid and for how long. Avoid presenting clone as a universal repair for ownership errors.

Structs, enums, and pattern matching

Use structs for records and enums for alternatives that must be represented explicitly. Teach exhaustive match expressions as a way to account for every variant. Explain Option for possible absence and Result<T, E> for operations that may fail. Encourage explicit handling rather than unwrap or expect in ordinary application paths; explain panic as a deliberate failure mechanism, not normal validation.

Collections, traits, and errors

Teach Vec, String, HashMap, and iterators with attention to ownership and borrowing of elements. Introduce traits as shared behavior contracts and generics only after learners understand concrete functions. Explain iterator adapters as transformations and distinguish lazy adapters from terminal operations. Use the standard error propagation operator only after learners understand Result and the caller's responsibility.

Files and tests

Use explicit paths and deterministic fixtures. Explain that relative paths resolve from the process working directory. Check I/O errors and avoid panics for

expected file failures. Teach unit tests, integration tests, and test assertions through Cargo after learners can explain function inputs and outputs.

Accessibility and delivery

- Keep code, rustc/cargo output, and test failures available as selectable text.
- Explicitly name angle brackets, ampersands, asterisks, colons, double colons, lifetimes, and pattern syntax when introduced.
- Teach keyboard navigation by diagnostic, line, symbol, and file; avoid mouse-only IDE features.
- Describe ownership changes as ordered events: binding, move, borrow, mutation, and scope end.
- Do not rely on graphical ownership diagrams; provide text traces with named variables and lifetimes.
- Ensure compiler diagnostics and test names are navigable with the learner's assistive technology.
- Allow time for dense compiler messages and repeated code navigation.

Test the actual rustup, editor, terminal, and screen-reader combination. Accessibility validation is separate from successful compilation.

Assessment and differentiation

Ask learners to predict whether a value is moved, copied, borrowed, or dropped; justify data-model choices; handle every enum variant; and explain how errors propagate. Include empty collections, missing map keys, invalid input, and file failures. Evaluate the quality of reasoning, not simply whether the learner found a way to silence the compiler.

Start with owned values and short scopes. Add nested borrowing, traits, iterators, and more advanced lifetimes only when prerequisite concepts are stable. Advanced extensions should deepen understanding without turning the course into a contest to satisfy the borrow checker.

AI-use policy

AI may be used only for personal conceptual understanding and reinforcement. It must not write, complete, autocomplete, repair, refactor, or generate course code or assessment solutions.

Instructor preparation

Verify stable Rust and the Rust 2024 edition, run the repository's rustc example checker, and separately test expected behavior and accessibility. The documented smoke checks establish compilation and process exit only; they

do not prove capstone correctness or assistive-technology certification. Record toolchain versions and validation scope. See the shared accessibility guidance.

18.12 Swift Teacher's Guide

Purpose and course context

This guide supports the Swift curriculum for beginning programmers, including blind and visually impaired learners using VoiceOver, other screen readers, braille displays, magnification, and keyboard navigation. It supplements the complete lessons, topic index, and validation instructions.

The course baseline is **Swift 6 language mode** with a current Swift 6 toolchain. Swift is used for Apple-platform applications, server software, and systems components. Teach Swift's value semantics, optionals, protocols, structured error handling, Codable, and concurrency as language concepts. Do not conflate learning Swift with building a graphical iOS interface; the core course should remain accessible through command-line projects.

Outcomes and sequence

Complete the shared Computing Foundations Primer and Terminal Skills prerequisite, then follow Units 0–11. Learners should compile and run Swift programs, use types and control flow, work with arrays and dictionaries, write functions, define structs and classes, use protocols, handle optional and throwing operations, process files and Codable data, test behavior, and complete the capstone. They should understand the distinction between a command-line Swift package and an Apple-platform application requiring Xcode and platform SDKs.

Use a consistent lesson cycle: objective and vocabulary; code walkthrough; prediction; swift execution or package build; output/diagnostic review; modification; learner explanation. Encourage learners to predict both values and types, especially when optionals and inferred types are involved.

Toolchain and packages

Verify with `swift --version` and `swiftc --version`. Run a source file with `swift main.swift`; create an executable package with `swift package init --type executable`, then use `swift build`, `swift run`, and `swift test`. Xcode and Apple SDKs are needed for simulator and native application work, but not for the core command-line sequence. Use Foundation for file operations and Codable JSON encoding. The curriculum uses a standards-aware external CSV library where quoted fields and embedded newlines require full CSV parsing.

Teaching Swift concepts

Types, value semantics, and collections

Swift is statically typed with type inference. Explain that inference does not make the language dynamically typed. Distinguish let from var and explain value semantics for structs and enums. Arrays and dictionaries are value types with copy-on-write implementation behavior; teach observable value behavior without overpromising a particular copy timing. Explain dictionary lookup as optional because a key may be absent.

Optionals and error handling

An optional explicitly represents a value that may be absent. Teach optional binding, if let/guard let, nil coalescing, and optional chaining as separate tools. Do not normalize force unwraps as the default response to compiler errors. Distinguish optional absence from thrown errors: they represent different forms of control flow. Teach do/try/catch and throwing function contracts with explicit failure cases.

Protocols, generics, and concurrency

Introduce protocols as behavior contracts and generics as reusable abstractions with type constraints. Use concrete examples before protocol-heavy designs. Swift 6 concurrency checking can surface data-isolation and Sendable diagnostics. Teach async/await and task structure only after sequential control flow and function contracts are secure. Explain actor isolation and shared mutable state conceptually; do not dismiss concurrency diagnostics as compiler noise.

Files and Codable

Use explicit paths and deterministic fixtures. Explain process working directories and Foundation file APIs. Teach Codable as a structured serialization mechanism, not as a guarantee that arbitrary external data is valid. Handle decoding failures explicitly and test missing, malformed, and unexpected fields. Use a CSV parser that supports quoted fields and embedded newlines rather than splitting every row on commas.

Accessibility and delivery

- Keep the introductory workflow command-line and text-first; do not require Xcode's visual interface for core outcomes.
- Teach VoiceOver and editor navigation appropriate to the learner's platform; provide keyboard paths for build, run, and test.
- Explicitly describe optionals, question marks, exclamation marks, arrows, generics, and closure syntax.
- Make compiler diagnostics, test output, and program output navigable as text.

- Describe collection contents, protocol conformance, and asynchronous event order in text.
- Treat simulator and GUI testing as an extension with a documented accessible alternative, not the only way to demonstrate programming knowledge.
- Allow time for punctuation review and verbose Swift 6 concurrency diagnostics.

Validate the actual macOS or Linux toolchain, editor, terminal, and assistive technology. VoiceOver and other screen readers differ by platform; do not assume a workflow transfers unchanged.

Assessment and differentiation

Assess type reasoning, value semantics, optional handling, collection selection, protocol use, error propagation, Codable validation, and tests. Include absent dictionary keys, empty collections, malformed JSON, file errors, and throwing operations. Require learners to explain why a force unwrap is or is not justified. Separate compilation, expected-output behavior, package tests, platform UI behavior, and accessibility evidence.

Scaffold with structs, simple functions, and sequential code before protocols and concurrency. Extend advanced learners with generic constraints, actors, or platform APIs only after the core language outcomes are secure.

AI-use policy

AI may be used only for personal conceptual understanding and reinforcement. It must not write, complete, autocomplete, repair, refactor, or generate course code or assessment solutions.

Instructor preparation

Verify Swift 6 language mode, run the repository's example validation instructions, and separately exercise expected behavior and accessibility. Record OS, compiler version, package commands, and any platform-specific prerequisites. See the shared GUI and workbench accessibility guidance before requiring Xcode or graphical application tooling.

18.13 Teacher's Guide: Introduction to Computer Science with TypeScript

For Students with Visual Impairments --- Complete Instructor Edition

How to use this teacher's guide

This guide is a companion for instructors, not a script that must be followed word for word. Read the philosophy, learner-context, and technology sections before the first lesson; use the unit notes while planning; and return to the assessment, differentiation, AI, and quick-reference sections throughout the course. Adapt procedures to the learner's actual assistive technology, operating system, course goals, and prior experience. Confirm tool-specific instructions in current official documentation because editor, screen-reader, and runtime behavior changes over time.

Plan for both programming learning and access to the programming environment. Establish what the student can already do independently, what access skills need instruction, what supports are already in place, and how the learner prefers to receive feedback. Never assume a diagnosis predicts a student's preferred tools or level of independence.

TypeScript and Node.js version baseline

Use Node.js 26 LTS (26.13.0 or a newer 26.x maintenance release) and TypeScript 7.0.2, the current stable compiler as of October 30, 2026. TypeScript is checked at build time and emitted as JavaScript; Node.js executes the emitted JavaScript. Keep the project's module format explicit and use a local compiler version recorded in the package lockfile.

Teach strict type checking progressively. Prefer explicit function parameter and return types where they clarify contracts; use inference for obvious local values; use interfaces/type aliases for object shapes; prefer unknown over any for untrusted values; and keep strictNullChecks enabled. Explain that type assertions do not validate data and that types are erased from emitted JavaScript. JSON, user input, file contents, and network responses still require runtime validation. TypeScript's compiler catches many classes of mistakes but does not prove program correctness.

Table of Contents

1. Philosophy and Why TypeScript
2. TypeScript-Specific Accessibility Challenges
3. Classroom Setup and Configuration

4. Spoken TypeScript Conventions
5. Unit 0--1: Setup and Foundations
6. Unit 2: Numbers, Input, and Math
7. Unit 3: Control Flow
8. Unit 4: Arrays and Objects
9. Unit 5: Strings
10. Unit 6: Functions
11. Unit 7: Classes and OOP
12. Unit 8: Files, Modules, and Error Handling
13. Unit 9: Collections, Analysis, and Testing
14. Units 10--11: Projects and Capstone
15. Bridging from Python/Java to TypeScript
16. Assessment Strategies
17. Supporting Diverse Needs
18. Alternative and Supplementary Resources
19. Quick Reference Sheets
20. Teaching AI Tools Responsibly
21. Comprehensive Instructional Planning and Course Implementation

---

1. Philosophy and Why TypeScript

The Case for Teaching TypeScript to Blind Students

TypeScript has a unique profile of benefits and challenges for visually impaired learners:

Universal relevance. JavaScript is the language that runs natively in every major web browser; TypeScript is compiled to JavaScript before browser execution. A student who learns TypeScript can eventually build tools that improve their own accessibility experience --- custom browser extensions, accessible widgets, automation scripts. This self-advocacy potential is meaningful.

Node.js enables accessible terminal programs. JavaScript runs on Node.js outside a browser. TypeScript source is first type-checked and compiled to JavaScript, which Node.js then executes. This curriculum uses Node.js to keep I/O in the terminal where screen readers excel.

Asynchronous programming is a real-world skill. TypeScript's `async`/`await` model reflects how most modern software actually works --- programs that wait for I/O, network requests, and user events. Learning this early prepares students for professional development.

The npm ecosystem. Over 2 million packages on npm provide pre-built solutions for almost any problem. This is a double-edged concern for beginners (dependency complexity), but the ability to say `npm install` and gain powerful functionality is professionally relevant.

JSON is part of the JavaScript runtime. `JSON.parse()` and `JSON.stringify()` are available without an import; TypeScript can type the values around these calls, but `JSON.parse()` returns untrusted data that should be validated. For data persistence projects, this reduces friction.

An Honest Account of Challenges

JavaScript runtime permissiveness remains a risk even in TypeScript projects. TypeScript's compiler catches many statically visible type mismatches before execution, but its types are erased when JavaScript is emitted. Runtime coercion still exists (`"5" + 3` produces `"53"`; `"5" - 3` produces `2`; `0 == false` is `true`). Coercion can reappear at `any`/assertion boundaries and when data arrives from JSON, files, users, or APIs. Teach strict checking, explicit conversion, type guards, and runtime validation together.

Asynchronous programming is conceptually hard. The `async`/`await` pattern required for user input is a genuine hurdle for beginners. There is no way to eliminate it from a Node.js input-based curriculum. The approach in this guide is to scaffold it carefully and defer full explanation of the event loop to later units.

Multiple valid syntaxes. Function declarations, function expressions, arrow functions, methods --- all do essentially the same thing. `var`, `let`, `const` --- three variable declarations. `==` and `===` --- two equality operators. This variety can create unnecessary cognitive overhead for beginners. The approach in this curriculum is to teach one style consistently (`const`/`let`, arrow functions, `===`) and acknowledge alternatives exist without teaching them in parallel.

---

2. TypeScript-Specific Accessibility Challenges

JavaScript runtime coercion and static type checking

This is the greatest accessibility risk in TypeScript for blind students. Unlike Python's `TypeError` and Java's compile-time type check, TypeScript silently converts types in ways that can produce wrong answers with no error:

Expression	Result	Expected by Programmer	Problem
`"5" + 3`	`"53"`	`8`	`+` concatenates when one operand is a string
`"5" - 3`	`2`	`2`	`-` converts string to number
`"5" * "3"`	`15`	`15`	Multiplication converts both
`"" == false`	`true`	Maybe?	Loose equality coercion
`0 == false`	`true`	Maybe?	Same
`null == undefined`	`true`	Probably not	

Expression	Result	Expected by Programmer	Problem
`NaN === NaN`	`false`	`true`	NaN is not equal to itself

Mitigation: Require `Number()` before any arithmetic on user input. Require `===` always. Teach `typeof` and `isNaN()` as defensive checks. Make logging intermediate values an explicit exercise before running any calculation.

`undefined` vs. `null` vs. `NaN` vs. `false` vs. `0` vs. `""`

All six of these are "falsy" in TypeScript. Aurally, they are indistinguishable unless you call `typeof` or check explicitly. A student who hears `console.log(result)` and the screen reader announces "false" cannot know whether `result` is `false`, `0`, `""`, `null`, `undefined`, or `NaN` without further inspection.

Teaching approach: Establish a diagnostic template:

```
console.log("Value:", result, "Type:", typeof result);
```

This outputs both the value and its type, providing audio context that `console.log(result)` alone does not.

Callback Hell and Asynchronous Confusion

Before `async`/`await`, Node.js input used nested callbacks:

```
// Confusing nesting --- avoid in teaching
rl.question("Name? ", (name) => {
  rl.question("Age? ", (age) => {
    rl.question("City? ", (city) => {
      console.log(`${name}, ${age}, ${city}`);
    });
  });
});
```

This nesting produces code that is confusing to read AND to hear --- the indentation deepens with each question. The `async`/`await` pattern linearizes this:

```
// Clear and linear --- use this exclusively
const name = await askQuestion("Name? ");
const age = await askQuestion("Age? ");
const city = await askQuestion("City? ");
console.log(`${name}, ${age}, ${city}`);
```

Teach `async`/`await` from the start and avoid callback patterns entirely in this course.

3. Classroom Setup and Configuration

VS Code or Theia IDE for TypeScript

Theia IDE is an alternative editor with keyboard- and screen-reader-oriented workflows suitable for this course. It supports many VS Code extensions through Open VSX; check availability of ESLint, Prettier, and TypeScript language support, and verify each workflow with the learner's screen reader. VS Code-specific settings and shortcuts may differ.

Required extensions:

- **ESLint** --- catches type issues, undefined variables, and style violations before running code
- **Prettier** --- auto-formats code consistently (run on save with `editor.formatOnSave: true`)
- **TypeScript (ES6) code snippets** --- tab-complete common patterns

Recommended TypeScript ESLint flat configuration (`eslint.config.mjs`; install eslint, @eslint/js, typescript-eslint, and globals as local development dependencies):

```
import js from "@eslint/js";
import tseslint from "typescript-eslint";
import globals from "globals";

export default tseslint.config(
  js.configs.recommended,
  ...tseslint.configs.recommended,
  {
    files: ["**/*.ts"],
    languageOptions: {
      sourceType: "commonjs",
      globals: globals.node
    },
    rules: {
      eqeqeq: "error",
      "no-console": "off",
      "@typescript-eslint/no-explicit-any": "error"
    }
  }
);
```

The `eqeqeq: error` rule makes `==` a linting error --- it must be `===`. This is an automated enforcement of the rule that prevents the most common silent bugs.

Node.js REPL accessibility:

```
node                    # Start REPL
.help                   # List REPL commands
```

```
.exit                    # Quit (or Ctrl+D)
```

Windows Powershell

```
node                     # Start REPL
.help                    # List REPL commands
.exit                    # Quit (or Ctrl+D)
```

The Node.js REPL is excellent for early exploration. Results are printed immediately after each expression, providing audio feedback without the compile-run-read cycle.

Alternative TypeScript Environments

Environment	Platform	Accessibility	Notes
Node.js + VS Code	All	High	Recommended setup
Node.js + terminal only	All	High	For students who prefer minimal UI
Replit.com	Web	Medium	No-install; test with student's setup
CodeSandbox	Web	Medium	Cloud editor; variable accessibility
GitHub Codespaces	Web	High	Cloud VS Code; same accessibility as local
Glitch.com	Web	Medium	Project-based; variable accessibility
Observable	Web	Low	Notebook-style; primarily visual
Deno	Runtime	High	Alternative to Node.js; similar accessibility

---

4. Spoken TypeScript Conventions

Symbol/Pattern	Spoken Form
`const`	”const”
`let`	”let”
`var`	”var” (avoid; legacy)
`===`	”triple equals” or ”strict equal”
`!==`	”strict not equal”
`==`	”loose equal” (should hear this rarely)
`=>`	”arrow” or ”fat arrow”
`...arr`	”spread arr”
`...args`	”rest args”
`async`	”async”

Symbol/Pattern	Spoken Form
`await`	”await”
`undefined`	”undefined”
`null`	”null”
`NaN`	”NaN” (say each letter N-A-N)
`typeof`	”typeof”
`instanceof`	”instanceof”
`new`	”new”
`this`	”this”
`class`	”class”
`extends`	”extends”
`super`	”super”
`import`	”import”
`export`	”export”
`require()`	”require”
`module.exports`	”module dot exports”
`Promise`	”Promise”
`async function`	”async function”
`() => {}`	”arrow function”
`(x) => x * 2`	”arrow function taking x, returning x times 2”
`#field`	”hash field” or ”private field”
`` `template ${var}` ``	”backtick template literal with variable var”

Reading an arrow function:

```
const doubleIfPositive = (n) => (n > 0 ? n * 2 : n);
```

”const doubleIfPositive gets arrow function taking n, returning: if n greater than 0, n times 2, else n”

Modern TypeScript and Node.js conventions

Teach the curriculum against the current Node.js 26 LTS line. Node.js 26.13.0 entered LTS on October 28, 2026; use the latest security patch in the 26.x line rather than pinning students to an old patch. Node.js 24 remains supported as LTS, but is not the current LTS line.

For new course examples, consistently use const/let, strict equality, explicit numeric conversion, async/await, and node: built-in module specifiers. Prefer the built-in node:test runner for basic unit tests; Jest is a valid optional framework, not a requirement. Keep the two module systems explicit: use a package.json type field and matching file extensions/configuration, and never mix CommonJS and ES module syntax in one file. Use current ESLint flat configuration (eslint.config.mjs) rather than the deprecated .eslintrc format. Check

input and parsed JSON at runtime: JavaScript runtime values remain dynamic, and static types do not validate external data at runtime.

5. Unit 0--1: Setup and Foundations

Lesson 0.1: Installing Node.js

Instructional Cues **Verify installation at every step:**

```
node --version    # Should print v26.x.x (26.13.0 or a newer 26.x maintenance release)
npm --version     # Should print a version number
npx --version     # Should print a version number (npx is bundled with npm)
```

Windows Powershell

```
node --version    # Should print v26.x.x (26.13.0 or a newer 26.x maintenance release)
npm --version     # Should print a version number
npx --version     # Should print a version number (npx is bundled with npm)
```

All three must work. If `npm` is missing, the student may have installed only the Node.js runtime without npm. Reinstall from nodejs.org (npm is included in the standard installer).

REPL first --- files second: Begin with `node` in the terminal. Let the student type expressions and hear results immediately. This builds confidence before introducing the file -> run -> read cycle:

```
$ node
> 5 + 3
8
> "Hello " + "World"
'Hello World'
> typeof 42
'number'
> typeof "hello"
'string'
> .exit
```

The npm init conversation: When initializing a project, `npm init -y` generates `package.json` automatically. Walk the student through reading the generated file:

```
npm init -y
cat package.json   # or type package.json on Windows
```

Windows Powershell

```
npm init -y
Get-Content package.json
```

`package.json` is plain JSON text --- fully readable with a screen reader. Understanding its structure is important for later lessons about dependencies and scripts.

Non-Obvious Accessibility Strategies

- **Node.js version managers:** `nvm` (Node Version Manager) allows multiple Node.js versions. For a student who may work on multiple machines or projects, `nvm install 26`, then `nvm use 26` is a consistent way to set the Node.js version. `nvm` is command-line-only and fully accessible.
- **Project-local dependencies:** install course tools with `npm install` or `npm install --save-dev` from the project folder. Run local executables with `npx` or npm scripts. Avoid `npm install -g` for course tools; global installations can conflict across projects.

Thought Experiments

1. *"TypeScript was created in 10 days in 1995. What marks of that hasty design are still visible in modern TypeScript? (Think about `typeof null === 'object'`, `NaN !== NaN`, `0 == false`.) Why haven't these been fixed?"*
2. *"Node.js uses a single-threaded event loop to handle many concurrent operations. A traditional multi-threaded approach uses multiple threads running in parallel. For a web server handling thousands of requests, what are the tradeoffs of each approach? Which tends to be more memory efficient?"*
3. *"npm has millions of packages. The average Node.js project depends on hundreds of packages, many maintained by single developers with no organizational backing. The 'left-pad incident' in 2016 broke thousands of projects when one 11-line package was unpublished. What does this incident reveal about the risks and benefits of the npm ecosystem?"*
4. *"TypeScript compiles to JavaScript for browser applications, Node.js servers, mobile apps via React Native, desktop apps via Electron, IoT devices, and serverless cloud functions. What does this ubiquity tell you about the relationship between a language's design and its adoption?"*

Lesson 1.3: Variables and Data Types

Instructional Cues `const` vs. `let` --- establish the rule on day one:

"Use `const` for everything. If you get an error saying you're trying to reassign a `const`, THEN change it to `let`. This approach produces cleaner code and prevents accidental reassignment bugs."

Demonstrate the error:

```
const x = 5;
x = 10; // TypeScript compile-time error: cannot assign to a const
```

"This error is helpful. `const` is protecting you from changing something you said was fixed."

JavaScript runtime types and auditory ambiguity:

Run this sequence in the REPL and ask the student to predict each output before pressing Enter:

```
typeof 42; // "number"
typeof "42"; // "string"
typeof true; // "boolean"
typeof null; // "object"  <- the famous bug
typeof undefined; // "undefined"
typeof {}; // "object"
typeof []; // "object"  <- arrays are objects!
typeof function () {}; // "function"
```

"Notice that both `{}` (object) and `[]` (array) produce 'object'. How do you tell them apart? `Array.isArray([])` returns `true`. `Array.isArray({})` returns `false`."

Template literals vs. concatenation:

```
const name = "Alice";
const age = 15;

// Concatenation --- confusing order of operations when read aloud
console.log("Hello, " + name + "! You are " + age + " years old.");

// Template literal --- linear, clean, easy to hear
console.log(`Hello, ${name}! You are ${age} years old.`);
```

Read both versions aloud with the student. Ask which is easier to follow aurally. Template literals are almost universally clearer when heard, because the variable values appear where they belong in the sentence.

Thought Experiments

1. *"TypeScript has only one number type for integers and decimals. Python has separate `int` and `float`. Java has `int`, `long`, `double`, `float`. What precision problems arise from having a single number type? When does `0.1 + 0.2 = 0.30000000000000004` matter, and when doesn't it?"*

2. *"TypeScript has six falsy values: `false`, `0`, `""`, `null`, `undefined`, `NaN`. Everything else is truthy --- including `"false"` (a non-empty string), `[]` (an empty array), and `{}` (an empty object). Design a quiz: ask the student*

whether ten different expressions are truthy or falsy. Which answers are surprising?"

3. *"`var` has function scope; `let` and `const` have block scope. What does 'scope' mean? How does a variable declared inside an `if` block with `var` vs. `let` behave differently? Which behavior is more predictable and why?"*

---

6. Unit 2: Numbers, Input, and Math

Lesson 2.2: User Input in Node.js

Instructional Cues The input helper module pattern:

Provide the `input.ts` module in every project skeleton. Students should not need to re-create this boilerplate:

```
// input.ts
const readline = require("node:readline");
const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout,
});

function askQuestion(prompt) {
  return new Promise((resolve) => rl.question(prompt, resolve));
}

function close() {
  rl.close();
}

module.exports = { askQuestion, close };
```

Walk through what each line does at least once so students understand the structure, but do not require students to write it from scratch until they're comfortable with async programming.

The `async/await` mental model --- teach this story:

"When your code hits `await`, imagine it puts up a 'please wait' sign and steps aside. Other things can happen while you wait. When the answer arrives, your code picks up exactly where it paused. The `async` keyword on the function declaration is your promise that 'this function may pause at some point.'"

```
async function main() {
  console.log("Starting..."); // 1: runs immediately
  const name = await askQuestion("Name? "); // 2: pauses here until user types
  console.log("Got name:", name); // 3: runs after user answers
```

```
  const age = await askQuestion("Age? "); // 4: pauses again
  console.log(`${name} is ${age}`); // 5: runs after user answers again
  close();
}

main(); // Kick off the async function
```

Ask: "What line is running while the program waits for user input?" (None --- the function is paused. The event loop can handle other things.)

Why `main()` must be called at the bottom: `async function main()` defines the function but doesn't run it. The `main()` call at the end actually starts execution. Without the call, nothing happens. This trips up beginners --- demonstrate it.

Non-Obvious Accessibility Strategies

- **`rl.close()` is mandatory:** Without `rl.close()`, the Node.js process continues running after `main()` finishes. The terminal appears to hang --- cursor blinks but no prompt. For a screen reader user, this is confusing. Teach `close()` as the final step of every program that uses input.
- **Prompt trailing space:** Always include a trailing space in prompt strings: `"Enter your name: "`. Without the space, the cursor appears immediately after the colon with no separation, which some screen readers render as "Enter your name:cursor" --- an awkward audio experience.
- **Input validation loop:** Without validation, a blank Enter or a non-numeric string for a number causes silent failure. Provide this reusable pattern:

```
async function getNumber(prompt) {
  while (true) {
    const input = await askQuestion(prompt);
    const num = Number(input.trim());
    if (!isNaN(num)) return num;
    console.log("Invalid input: please enter a number.");
  }
}
```

Teach this pattern and require students to use it whenever they need numeric input.

Thought Experiments

1. *"The `readline` module uses an 'event listener' pattern --- `rl.question(prompt, callback)` means 'when the user answers, call this function.' The `async/await` pattern wraps this in a Promise. What problem does the*

Promise solve that the raw callback doesn't? (Think about reading the code linearly.)"

2. *"Node.js uses a single-threaded event loop. While your program is waiting for user input (`await askQuestion`), the thread is free. What else could the thread be doing? In a real application, this might mean handling a different request or updating state. What does this tell you about why `async/await` is important for scalable programs?"*

3. *"Synchronous input (blocking until the user responds) and asynchronous input (doing other things while waiting) represent fundamentally different programming models. For a single-user command-line program, which matters? For a server handling 10,000 simultaneous users, which matters more? Why?"*

---

7. Unit 3: Control Flow

Lessons 3.1--3.3

Instructional Cues `===` enforcement --- make it automatic:

With ESLint configured to flag `==`, the student will hear a warning whenever they use loose equality. Walk through the first warning together:

```
if (userInput == 5) {   // ESLint: Expected '===' and instead saw '=='
```

"ESLint is protecting you from a common silent bug. Change this to `===`. The only exception is checking for null-or-undefined: `x == null` is sometimes used as a shorthand for `x === null || x === undefined` --- but avoid this until you understand it."

Truthy/falsy in input validation:

```
const name = await askQuestion("Enter your name: ");
if (!name.trim()) {
  console.log("Name cannot be empty.");
  // re-prompt...
}
```

`!name.trim()` is true when the trimmed string is empty (`""` is falsy). This is idiomatic TypeScript. Read it aloud: "if not name dot trim" --- ask: "What does 'not an empty string' mean? Why is an empty string falsy?"

Ternary operator --- limit its use:

The ternary `condition ? valueIfTrue : valueIfFalse` is concise but can be hard to follow when read aloud:

```
const status = score >= 60 ? "pass" : "fail";
```

Read: *"const status gets: if score greater-than-or-equal 60, quote pass, else quote fail"*

Teach it but establish this rule: "If the ternary is easy to read aloud in one breath, use it. If it requires re-reading, use if-else."

Thought Experiments

1. *"JavaScript's `==` performs type coercion before comparing. TypeScript adds compile-time checks but preserves JavaScript runtime semantics. What does TypeScript's adoption tell you about the value of static analysis in large JavaScript codebases?"*
2. *"Consider: `if (userName)` vs. `if (userName !== '' && userName !== null && userName !== undefined)`. The first uses JavaScript's runtime truthiness; the second is explicit. In a code review, which would you prefer seeing? What are the readability and correctness tradeoffs?"*
3. *"JavaScript's optional chaining (`?.`) and nullish coalescing (`??`) were added to the language to handle null/undefined more gracefully. `user?.address?.city` returns `undefined` instead of throwing if `user` or `address` is null. Why did this feature take decades to add to a language created in 1995?"*
4. *"The JavaScript `switch` statement falls through by default, and TypeScript preserves that runtime behavior. A mapping object can be an alternative to `switch` for dispatch tables. `const actions = { 'add': addFunc, 'delete': deleteFunc }; actions[command]?.();`. What are the accessibility implications of this pattern vs. a `switch` statement when read with a screen reader?"*

---

8. Unit 4: Arrays and Objects

Lesson 4.1: Arrays and Functional Methods

Instructional Cues Introduce `map`, `filter`, `reduce` as a trio:

Don't teach them one per lesson --- teach the relationship between them first:

"These three methods each take a function and apply it to every element in an array. `map` transforms elements. `filter` keeps or removes elements. `reduce` combines elements into one value. Together they cover most array processing."

Then give one shared example that uses all three:

```
const students = [
  { name: "Alice", score: 88 },
  { name: "Bob", score: 62 },
  { name: "Charlie", score: 95 },
```

```
  { name: "Diana", score: 71 },
];

// 1. Filter passing students (score >= 70)
const passing = students.filter((s) => s.score >= 70);

// 2. Map to get just names
const passingNames = passing.map((s) => s.name);

// 3. Reduce to build a report string
const report = passingNames.reduce(
  (acc, name) => acc + name + ", ",
  "Passing: ",
);

console.log(report); // "Passing: Alice, Charlie, Diana, "
console.log("Count:", passing.length);
```

Accessible printing for arrays of objects:

```
// Accessible: numbered, labeled fields
students.forEach((student, index) => {
  console.log(`${index + 1}. ${student.name}: ${student.score}`);
});

// Less accessible: raw object output
console.log(students); // Produces cluttered bracket notation when read aloud
```

Chaining --- build up incrementally:

Show chaining as a composition of steps, each of which you can trace separately:

```
const result = numbers
  .filter((n) => n % 2 === 0) // Step 1: keep evens
  .map((n) => n ** 2) // Step 2: square each
  .reduce((sum, n) => sum + n, 0); // Step 3: sum

// DEBUG: break the chain to trace each step
const step1 = numbers.filter((n) => n % 2 === 0);
console.log("After filter:", step1);
const step2 = step1.map((n) => n ** 2);
console.log("After map:", step2);
const step3 = step2.reduce((sum, n) => sum + n, 0);
console.log("Final sum:", step3);
```

The debug version (step-by-step) is more accessible during development. Consolidate to the chain after confirming correctness.

Thought Experiments

1. *"`map`, `filter`, and `reduce` come from functional programming --- a paradigm based on mathematical functions that have no side effects. TypeScript arrays also have `forEach` which is for side effects only. What is the philosophical distinction between `map` and `forEach`? Why does this distinction matter for reasoning about code correctness?"*

2. *"The `sort` method in TypeScript sorts in-place and mutates the original array. `filter`, `map`, and `reduce` return new arrays, leaving the original unchanged. Why the asymmetry? What bugs could arise from `sort`'s mutation? How would you sort without mutating: `[...arr].sort(...)`?"*

3. _"Functional composition: `const double = n => n _ 2; const addOne = n => n + 1; const doubleAndAdd = n => addOne(double(n))`. How does function composition scale? What is the advantage of composing small, focused functions over writing one large function that does everything?"*

4. *"The spread operator `[...arr]` creates a shallow copy of an array. For an array of primitives `[1, 2, 3]`, the spread copy is fully independent. For an array of objects `[{name: 'A'}, {name: 'B'}]`, the objects inside are still shared. What does 'shallow copy' mean? When does this distinction cause bugs?"*

Lesson 4.2: Objects

Instructional Cues Destructuring --- introduce before objects are complex:

```
const person = { name: "Alice", age: 15, city: "Portland" };

// Without destructuring:
const name = person.name;
const age = person.age;

// With destructuring:
const { name, age } = person;

// In function parameter:
function introduce({ name, age }) {
  console.log(`Hi, I'm ${name}, ${age} years old.`);
}
introduce(person);
```

Destructuring is ubiquitous in modern TypeScript. Teaching it early reduces the verbosity of object access code, which in turn makes programs cleaner to

read and hear.

`Object.keys()`, `Object.values()`, `Object.entries()` as navigation tools:

For a student who cannot visually inspect an object in a debugger panel, these methods provide audio-accessible object introspection:

```
const config = { theme: "dark", fontSize: 14, notifications: true };

// What properties does this object have?
console.log(Object.keys(config).join(", "));

// What are all the values?
Object.values(config).forEach((v) => console.log(v));

// Both together:
Object.entries(config).forEach(([key, value]) => {
  console.log(`${key}: ${value}`);
});
```

Optional chaining for safe nested access:

```
const user = { profile: { address: null } };

// Without optional chaining --- crashes if address is null
const city = user.profile.address.city; // TypeError!

// With optional chaining --- returns undefined instead
const city = user?.profile?.address?.city; // undefined, no crash

// With nullish coalescing for a default
const city = user?.profile?.address?.city ?? "Unknown city";
```

This pattern is essential for working with real-world data where fields may be missing.

Thought Experiments

1. *"TypeScript objects can have properties added or removed at any time. `user.newField = 'value'` just works. Java objects have a fixed set of fields defined at compile time. Which approach is better for representing data from an API that might add new fields in future versions? Which is better for ensuring all required fields are present?"*

2. *"`Object.freeze(obj)` makes an object immutable --- you can't add, remove, or modify properties. This mimics the behavior of a Java `final` field or Python's `frozenset`. When would immutable objects be useful? What patterns become possible when you can guarantee an object won't change?"*

3. *"Prototype-based inheritance means TypeScript objects can inherit from other objects directly, without classes. Classes (introduced in ES6) are 'syntactic sugar' over this prototype system. What does this mean? What can you do with prototypes that you cannot easily express with class syntax?"*

4. *"JSON (TypeScript Object Notation) is the universal data exchange format. It looks almost like TypeScript object literals but has stricter rules: all keys must be double-quoted strings, no functions, no comments. Why must JSON be stricter than TypeScript object literals? What security problems would arise if JSON allowed functions?"*

---

9. Unit 5: Strings

Instructional Cues Template literals for multiline strings:

```
const menu = `
--- Main Menu ---
1. Add item
2. View all
3. Search
4. Delete
5. Quit
-----------------`;

console.log(menu);
```

Template literals with newlines are much cleaner for building multiline output than repeated `console.log()` calls. Teach this pattern for menus and reports.

Regular expression basics --- connect to real need:

Before introducing regex syntax, establish the problem it solves:

"You have user input that might have extra spaces, uppercase, or punctuation. You want just the words. Without regex, you'd call `.trim().toLowerCase().replace(',', '').replace('.', '')...` for every punctuation mark. Regex lets you express 'any non-letter character' once."

```
const messy = "  Hello, World!   How are you?  ";
const words = messy
  .trim()
  .toLowerCase()
  .split(/\W+/)
  .filter((w) => w.length > 0);
console.log(words); // ['hello', 'world', 'how', 'are', 'you']
```

Teach `\w+` (word characters), `\d+` (digits), `\s+` (whitespace), and the `g` and `i` flags. Don't attempt to teach full regex --- a basic vocabulary is enough for data cleaning tasks.

Thought Experiments

1. *"Template literals use backticks `` ` `` while regular strings use `"` or `'`. Backticks on a keyboard are typically in the upper-left corner. For touch typists, backticks are less natural than quotes. What keyboard shortcuts or text expansions might a blind programmer use to reduce the friction of frequently-needed but awkward characters?"*
2. *"String immutability in TypeScript (same as Python) means every `.toUpperCase()`, `.replace()`, and `.slice()` creates a new string. For a loop that builds a long string incrementally, what is the efficient alternative? (Hint: consider `Array.push()` + `.join()` as a form of StringBuilder.)"*

---

10. Unit 6: Functions

Extended Instructional Cues

The four function forms --- teach one style, acknowledge the rest:

```
// 1. Function declaration (hoisted --- can be called before declaration)
function add(a, b) {
  return a + b;
}

// 2. Function expression (not hoisted)
const multiply = function (a, b) {
  return a * b;
};

// 3. Arrow function --- SHORT BODY (implicit return)
const square = (n) => n * n;

// 4. Arrow function --- LONG BODY (explicit return required)
const absolute = (n) => {
  if (n < 0) return -n;
  return n;
};
```

Teach style 3 (short arrow) and style 4 (long arrow) as the primary forms. Acknowledge 1 and 2 exist and how to read them. Do not attempt to teach all four simultaneously.

The implicit return trap --- a critical accessibility issue:

```
// These DO return a value:
const double = (n) => n * 2; // [x] implicit return
const greet = (name) => `Hi ${name}`; // [x] implicit return

// This does NOT return a value (the curly brace requires explicit return):
const broken = (n) => {
  n * 2; // No return statement! Returns undefined.
};

// Fixed:
const fixed = (n) => {
  return n * 2;
};
```

When heard aloud, `n => { n * 2; }` and `n => { return n * 2; }` sound different --- the `return` keyword is present in one and absent in the other. Teach students to listen for `return` inside block-bodied arrow functions.

Higher-order functions --- connect to previous array methods:

```
// map() takes a function as argument
const doubled = [1, 2, 3].map((n) => n * 2);

// We can also pass a named function
const double = (n) => n * 2;
const doubled2 = [1, 2, 3].map(double);

// This works because map expects a function --- it doesn't care if it's
// named or anonymous, arrow or traditional
```

"Every time you used `array.map(...)`, you were passing a function as an argument. Higher-order functions --- functions that take other functions --- are normal TypeScript."

Closures --- establish the mental model before the code:

"A closure is a function that remembers the environment where it was created, even after that environment is gone. Imagine a letter in an envelope. The letter is the function. The envelope is the environment --- it contains variables the function needs. Even if the person who wrote the letter is gone, the letter still has access to everything in its envelope."

```
function makeCounter(startValue = 0) {
  let count = startValue; // This is the "envelope"

  return {
    increment: () => ++count,
    decrement: () => --count,
```

```
    reset: () => {
      count = startValue;
    },
    getCount: () => count,
  };
}

const counter = makeCounter(10);
counter.increment();
counter.increment();
counter.increment();
console.log(counter.getCount()); // 13 --- count persists!
```

Thought Experiments

1. *"Arrow functions don't have their own `this` --- they inherit `this` from the surrounding scope. Regular functions create their own `this`. This was a major source of bugs in pre-ES6 TypeScript. How many of the following do you think involved confusing `this`? (a) Typos, (b) Variable scope errors, (c) Async timing bugs, (d) `this` binding bugs. In TypeScript specifically, (d) is disproportionately common --- why?"*

2. *"Pure functions always produce the same output for the same input and have no side effects. `Math.sin(x)` is pure. `console.log(x)` is not (it has a side effect). Why are pure functions easier to test? Why are pure functions easier to reason about in a screen-reader debugging session?"*

3. *"Memoization caches the results of function calls so repeated calls with the same arguments return the cached result. Write a description of a memoized `fibonacci(n)` function. What is saved? What is the tradeoff? When would memoization be harmful rather than helpful?"*

4. *"TypeScript is a multi-paradigm language: you can write imperative code (explicit step-by-step), object-oriented code (classes and methods), or functional code (pure functions and immutable data). For the accessible data manager project, which paradigm produces the most maintainable code? The most testable? The most readable when heard aloud?"*

---

11. Unit 7: Classes and OOP

Instructional Cues `toString()` is mandatory:

```
class BankAccount {
  #balance;

  constructor(owner, initialBalance = 0) {
    this.owner = owner;
```

```
    this.#balance = initialBalance;
  }

  toString() {
    return `BankAccount[owner=${this.owner}, balance=$${this.#balance.toFixed(2)}]`;
  }
}

const account = new BankAccount("Alice", 1000);
console.log(account.toString()); // BankAccount[owner=Alice, balance=$1000.00]
console.log(`${account}`); // Also calls toString() implicitly
```

Require `toString()` in every class. Explain: "When your screen reader reads `console.log(myAccount)`, it reads `toString()`. Without it, you hear `[object Object]` --- completely useless. Define `toString()` to hear meaningful information."

Private fields # --- establish as the default:

```
class Dog {
  #name; // Private --- cannot be accessed outside the class
  #age;

  constructor(name, age) {
    this.#name = name;
    this.#age = age;
  }

  get name() {
    return this.#name;
  } // Getter --- controlled access
  get age() {
    return this.#age;
  }

  set age(value) {
    if (value < 0) throw new Error("Age cannot be negative");
    this.#age = value;
  }

  bark() {
    console.log(`${this.#name} says: Woof!`);
  }

  toString() {
    return `Dog[name=${this.#name}, age=${this.#age}]`;
  }
```

```
}
```

"Private fields starting with `#` are truly private --- not just conventional privacy. `myDog.#name` outside the class throws a SyntaxError. This is real encapsulation."

Thought Experiments

1. *"TypeScript classes are 'syntactic sugar' over prototype-based inheritance. TypeScript adds static typing on top of TypeScript classes. With three layers (prototypes -> ES6 classes -> TypeScript classes), is TypeScript's OOP model more or less trustworthy than Java's, which has only one layer? What does 'syntactic sugar' imply about the behavior of the underlying mechanism?"*
2. *"The `class` keyword was added to TypeScript in 2015, 20 years after the language was created. In those 20 years, TypeScript developers built massive applications without classes using only prototypes and closures. What does this tell you about the role of syntax vs. capability in programming languages?"*
3. *"Mixins are a way to compose behavior from multiple sources without inheritance. Instead of `class A extends B`, you copy methods from B onto A's prototype. TypeScript supports this; Java does not (without interfaces). When would mixins be more flexible than single inheritance? What problems could they cause?"*

---

12. Unit 8: Files, Modules, and Error Handling

Lesson 8.1: Files

Instructional Cues `__dirname` and path construction:

```
const path = require("node:path");
const fs = require("node:fs");

// __dirname is the directory of the current file
const dataFile = path.join(__dirname, "data", "contacts.json");

// Path joining prevents slash vs. backslash issues on Windows/macOS/Linux
console.log("Data file location:", dataFile);
```

For a student who cannot visually navigate a file tree, `console.log(__dirname)` and `console.log(dataFile)` are essential diagnostic tools. Teach them to print file paths as the first debugging step when a file isn't found.

`fs.existsSync()` before file operations:

```
if (fs.existsSync(dataFile)) {
  const data = fs.readFileSync(dataFile, "utf8");
  return JSON.parse(data);
} else {
  console.log("No data file found. Starting with empty data.");
  return [];
}
```

Always check existence before reading. The alternative --- catching `ENOENT` in a try-catch --- works but puts normal ”first run” behavior in the error path.

Lesson 8.3: CommonJS Modules

Module design for accessibility:

A well-designed module has a clear exports interface. Teach students to think of their module's exports as its ”public API” --- what other files can use:

```
// contacts.ts --- exports only what other files need
const path = require("node:path");
const fs = require("node:fs");

const DATA_FILE = path.join(__dirname, "..", "data", "contacts.json");

function loadContacts() {
  /* ... */
}
function saveContacts(contacts) {
  /* ... */
}
function findContact(contacts, name) {
  /* ... */
}

// Export only the public interface
module.exports = { loadContacts, saveContacts, findContact };
```

”Everything not in `module.exports` is private to this file. Other files can use `loadContacts` and `saveContacts`, but they cannot directly access `DATA_FILE`. This is encapsulation at the module level.”

Thought Experiments

1. *”Node.js's CommonJS (`require`) and the ES Module standard (`import`) are two different module systems in TypeScript. This creates confusion --- some packages use one, some use the other. TypeScript can use both. Why did two module systems emerge? What does this history say about how standards develop in open-source ecosystems?”*

2. *"Synchronous file I/O (`readFileSync`) blocks execution until the file is read. Asynchronous file I/O (`readFile` or `fs.promises.readFile`) allows other code to run while waiting. For a startup script that reads configuration before doing anything else, which makes more sense? For a web server that handles thousands of requests, which matters more?"*

3. *"Custom error classes allow code to throw and catch specific, named errors: `throw new ValidationError('email', 'Invalid format')`. What is the advantage of this over throwing `new Error('Validation failed: email field invalid format')`? Think about code that catches and handles the error."*

---

13. Unit 9: Collections, Analysis, and Testing

Jest Testing

Instructional Cues Jest output is accessible:

Run `npx jest --verbose` for maximum accessibility. The `--verbose` flag produces:

```
PASS src/math.test.ts
  add function
    [x] adds two positive numbers (3ms)
    [x] adds negative numbers (1ms)
    [x] handles zero (0ms)
  getGrade function
    [x] returns A for score >= 90 (0ms)
    [x] returns F for score < 60 (0ms)

Test Suites: 1 passed, 1 total
Tests:       5 passed, 5 total
```

Each test name is announced. Failures include file path, test name, expected vs. received values, and a diff. All of this is plain text --- fully screen-reader-accessible.

Add to `package.json` scripts:

```
"scripts": {
    "test": "jest --verbose",
    "test:watch": "jest --verbose --watch"
}
```

`npm test` runs all tests. `npm run test:watch` reruns tests whenever a file changes --- useful during active development.

Test naming convention:

```
describe("BankAccount", () => {
  describe("deposit", () => {
    test("increases balance by deposited amount", () => {
      /* ... */
    });
    test("rejects negative amounts", () => {
      /* ... */
    });
    test("rejects zero amount", () => {
      /* ... */
    });
  });
  describe("withdraw", () => {
    test("decreases balance by withdrawn amount", () => {
      /* ... */
    });
    test("throws InsufficientFundsError when balance is too low", () => {
      /* ... */
    });
  });
});
```

The nested `describe` / `test` structure, when announced by Jest with `--verbose`, reads as complete sentences: "BankAccount > deposit > increases balance by deposited amount."

Thought Experiments

1. *"Jest's `expect(value).toEqual(expected)` performs deep comparison --- it checks that two objects have the same structure and values, not that they are the same object in memory. Why is deep comparison usually what you want when testing? When would you specifically want reference equality (`toBe`)?"*

2. *"Snapshot testing in Jest captures the output of a component or function the first time, then checks future runs against that snapshot. What are the advantages? What are the risks? When might a snapshot become 'stale' and mislead you into thinking code is correct when it isn't?"*

3. *"Code coverage measures what percentage of your code is executed by tests. 100% coverage is a goal but not a guarantee of correctness. Design a function with 100% test coverage that still has a bug. (Hint: think about boundary conditions --- what if the test covers the case where x=5 but not x=0?)"*

---

14. Units 10--11: Projects and Capstone

TypeScript Project Structure

```
my-data-manager/
|--- src/
|   |--- models/
|   |   +--- contact.ts
|   |--- storage/
|   |   +--- dataManager.ts
|   |--- ui/
|   |   +--- consoleUI.ts
|   +--- utils/
|       |--- input.ts         (the readline helper)
|       +--- validation.ts    (reusable validation functions)
|--- data/
|   +--- contacts.json
|--- tests/
|   |--- models/
|   |   +--- contact.test.ts
|   +--- storage/
|       +--- dataManager.test.ts
|--- eslint.config.mjs
|--- package.json
+--- index.ts
```

`package.json` scripts --- teach as accessibility tools:

```
{
  "scripts": {
    "start": "npx tsc && node dist/index.js",
    "test": "jest --verbose",
    "test:watch": "jest --verbose --watch",
    "lint": "eslint src/",
    "lint:fix": "eslint src/ --fix"
  }
}
```

`npm start` --- consistent command regardless of entry point filename. `npm test` --- runs all tests. `npm run lint` --- checks for errors without running the program.

`npm run lint` as a pre-run check: Teach students to run `npm run lint` before running the program. ESLint catches undefined variables, unused variables, and `==` before the program runs --- much faster than finding runtime errors.

Accessible Output Design for Capstone

Require students to design their program's output to be navigable and self-describing:

```
// Navigation landmark prints
function printSeparator(title) {
  const line = "-".repeat(40);
  console.log(line);
  if (title) console.log(`  ${title.toUpperCase()}`);
  console.log(line);
}

// Numbered list with clear field labels
function printContactList(contacts) {
  if (contacts.length === 0) {
    console.log("No contacts found.");
    return;
  }
  printSeparator("contacts");
  contacts.forEach((c, i) => {
    console.log(`${i + 1}. ${c.name}`);
    console.log(`   Phone: ${c.phone || "not provided"}`);
    console.log(`   Email: ${c.email || "not provided"}`);
  });
  console.log(`Total: ${contacts.length} contact(s)`);
}

// Confirmation messages --- every action must produce one
function confirmAction(action, details) {
  console.log(`[x] ${action}: ${details}`);
}
```

Thought Experiments

1. *"Your capstone is a command-line program. Eventually, it could become a web application. What would need to change? What would transfer directly? Write a two-column list: 'Can reuse as-is' and 'Must rewrite.'"*

2. *"The accessible design principle 'no visual-only cues for critical information' means every status, error, or confirmation must be communicated through text, not color or icon alone. Where in your program have you relied on visual formatting? How would you replace it?"*

3. *"Open-source your capstone project. Write a README that explains: what the program does, how to install it, how to run it, and specifically how to use it with a screen reader. What information would a sighted user find obvious but a blind user needs explicitly stated?"*

15. Bridging from Python/Java to TypeScript

From Python to TypeScript

Python	TypeScript	Key Difference
`print("hello")`	`console.log("hello")`	Method vs. function
`input("prompt")`	`await askQuestion("prompt")`	Async in TypeScript
`f"Hello {name}"`	`` `Hello ${name}` ``	f-string vs. template literal
`def func(x):`	`const func = x => ...`	Arrow function style
`True/False`	`true/false`	Lowercase in TypeScript
`None`	`null` and `undefined`	JS has two "nothing" values
`my_list`	`myList`	TypeScript uses camelCase
`len(x)`	`x.length`	Property, not function
`x is None`	`x === null` or `x === undefined`	Different null checks
`import json`	`const fs = require('node:fs')`	Built-in `JSON` object, no import
`if __name__ == '__main__':`	No direct equivalent (just call `main()`)	

Thought experiments for Python->JS students:

1. *"In Python, you use spaces to define blocks (indentation). In TypeScript, you use `{}`. Python's blocks are invisible --- they're just the alignment of characters. TypeScript's blocks are explicitly marked. Which feels more natural? Which provides clearer information when read aloud?"*
2. *"Python's `None` and TypeScript's `null`/`undefined` both represent 'nothing,' but they behave differently. Python has one 'nothing'; TypeScript has two. When is `null` appropriate? When is `undefined` appropriate? Under what circumstances would you encounter one vs. the other?"*
3. *"Python is dynamically typed, while TypeScript checks declared types before emitting JavaScript. The emitted JavaScript still has dynamic runtime behavior. How do these differences affect debugging, flexibility, and runtime validation?"*

16. Assessment Strategies

Alternative Assessments for TypeScript

Console output analysis: Provide a TypeScript program and 3--4 sample inputs. Ask the student to predict what the program prints for each input before running it. This tests reading comprehension without requiring writing.

Linting-first review: Submit code that is free of all ESLint warnings. A student who achieves this has internalized TypeScript's best practices at the syntax level.

Refactoring exercises: Give a working but poorly structured program. Ask the student to refactor it to use functions, modules, or classes appropriately. Assess the quality of the architecture, not just whether it runs.

Test-first implementation: Give the student a set of failing tests. Ask them to write the implementation that makes the tests pass. This tests code comprehension and problem-solving without a blank-page start.

---

17. Supporting Diverse Needs

For Students Who Are New to Async Programming

Async/await is conceptually hard. For students who struggle:

1. Provide the complete `input.ts` helper and instruct them to use it without modifying it.
2. For the first several lessons, treat `async function main() { ... } main();` as boilerplate, not content.
3. Introduce the conceptual explanation of async/await only after students are comfortable with the rest of the language.
4. Consider deferring async until after basic functions are mastered.

For Students with Additional Cognitive Considerations

- ESLint provides immediate text feedback for common errors --- use it as a learning tool. Hearing "no-unused-vars" and then explaining what that means is a teaching moment triggered by the student's own code.
- TypeScript's many syntactic variations can be overwhelming. Enforce a single consistent style (arrow functions, `const`/`let`, `===`) and defer alternatives.
- Use the REPL liberally for short experiments so the student gets immediate feedback without file management overhead.

---

18. Alternative and Supplementary Resources

Textbooks and Reading

Resource	Level	Format	Accessibility
Eloquent TypeScript (Haverbeke, free online)	Beginner--Intermediate	Free HTML	Excellent screen-reader-friendly HTML
You Don't Know JS (Simpson, free on GitHub)	Intermediate	Free HTML/PDF	In-depth on closures, scope, types
TypeScript: The Good Parts (Crockford)	Intermediate	Print	Short; focuses on the reliable subset of JS
TypeScript: The Definitive Guide (Flanagan)	Reference	Print/ebook	Comprehensive; good for looking up specifics
MDN Web Docs (developer.mozilla.org)	Reference	Free HTML	Excellent HTML accessibility; definitive JS reference
Node.js Documentation (nodejs.org/docs)	Reference	Free HTML	Official Node.js API reference
TypeScript.info	Beginner--Advanced	Free HTML	Very well-organized; plain HTML; comprehensive

typescript.info is especially recommended --- it's a comprehensive free tutorial organized from basics to advanced topics, all in clean, accessible HTML. It's often more readable than MDN for learning (vs. reference), and screen readers navigate it very well.

Online Courses and Platforms

Platform	Course	Accessibility	Notes
freeCodeCamp	TypeScript Algorithms & Data Structures	Medium	Certification path; some UI inaccessible
Codecademy	Learn TypeScript	Partial	Editor accessible; some UI elements not
Exercism.io	TypeScript Track	High	Terminal-based; mentor feedback
TypeScript30 (Wesbos)	30 vanilla JS projects	Medium	Video-based; transcripts available

Platform	Course	Accessibility	Notes
edX	Various TypeScript courses	High	Video transcripts; accessible exercises
Scrimba	Learn TypeScript	Low	Interactive video; poor screen reader support
Khan Academy	TypeScript course	Medium	Browser-based editor; variable accessibility
NodeSchool.io	Node.js workshops	High	Command-line workshopper format; fully accessible

NodeSchool.io is especially recommended --- workshops are run entirely in the terminal via a command-line tool. Students work through exercises, get feedback from the tool, and progress at their own pace. The terminal-first design makes it highly accessible.

Video and Audio Resources

Resource	Description	Accessibility
Traversy Media (YouTube)	TypeScript crash courses and projects	Captions available
The Net Ninja (YouTube)	In-depth TypeScript series	Captions available
Academind (YouTube)	Deep dives into JS/Node	Captions available
Kevin Powell (YouTube)	TypeScript and web fundamentals	Captions; clear explanations
CS50W (Harvard)	Web programming with Python and TypeScript	Full transcripts; high quality
TypeScript: Understanding the Weird Parts (Udemy)	Deep JS internals	Captions

TypeScript-Specific Accessibility Tools

Tool	Purpose	Notes
ESLint	Code quality and error prevention	Command-line; fully accessible
Prettier	Consistent code formatting	Run on save; no screen interaction needed
Jest	Testing framework	Accessible output; `--verbose` flag

Tool	Purpose	Notes
jshint.com	Online JS linter	Plain HTML; basic linting
node-inspect	Node.js debugger	Terminal-based debugging
0x	Node.js profiler	Terminal output
nvm	Node version manager	Command-line; accessible

Communities

Community	Relevance
Node.js Community (nodejs.org/community)	Official Node.js community resources
TypeScript Weekly (newsletter)	Weekly digest of JS news; plain HTML email
Stack Overflow TypeScript	Q&A community; plain HTML; excellent accessibility
Dev.to	Developer blog platform; reasonable accessibility
GitHub Discussions on popular JS projects	Community support; text-based
NFB Jernigan Institute	Resources for blind STEM students
SIGACCESS	Research on accessible computing including programming

---

19. Quick Reference Sheets

TypeScript vs Python vs Java at a Glance

Concept	Python	Java	TypeScript
Print	`print("x")`	`System.out.println("x")`	`console.log("x")`
Variable	`x = 5`	`int x = 5;`	`const x = 5;`
String format	`f"Hi {name}"`	`String.format("Hi %s", name)`	`` `Hi ${name}` ``
User input	`input("? ")`	`scanner.nextLine()`	`await askQuestion("? ")`
Array/List	`[1, 2, 3]`	`int[] arr = {1,2,3}`	`[1, 2, 3]`
Dictionary/Map	`{"a": 1}`	`new HashMap<>()`	`{ a: 1 }`
Function	`def f(x):`	`public int f(int x) {`	`const f = x => ...`
Class	`class Dog:`	`public class Dog {`	`class Dog {`

Concept	Python	Java	TypeScript
Exception	`try/except`	`try/catch`	`try/catch`
Null	`None`	`null`	`null/undefined`
Boolean	`True/False`	`true/false`	`true/false`
Length	`len(x)`	`x.length`	`x.length`
For each	`for x in list:`	`for (T x : list) {`	`for (const x of list) {`
Import	`import math`	`import java.util.*;`	`const x = require('./x')`

Accessible Output Patterns

```
// Standard separator
console.log("-".repeat(40));

// Section header
console.log(`\n=== ${title.toUpperCase()} ===`);

// Numbered list with details
items.forEach((item, i) => {
  console.log(`${i + 1}. ${item.name}`);
  console.log(`   ${item.detail}`);
});
console.log(`Total: ${items.length}`);

// Menu
const MENU = `
--- Main Menu ---
1. Add item
2. View all items
3. Search
4. Edit item
5. Delete item
6. Save
7. Quit
----------------`;

// Confirmation
console.log(`[x] Added: "${name}"`);
console.log(`[x] Deleted: "${name}"`);
console.log(`[x] Saved to ${filename}`);

// Error messages
console.log(`Error: Name cannot be empty. Please try again.`);
console.log(`Error: "${filename}" not found. Starting with empty data.`);
```

```
console.log(`Error: Task number must be between 1 and ${count}.`);
```

Common TypeScript Error Quick Reference

Error	Common Cause	First Check
`ReferenceError: x is not defined`	Typo or not declared	Spelling; scope
`TypeError: Cannot read property of undefined`	Accessing property on `undefined`	Which variable is undefined; optional chaining
`TypeError: x is not a function`	Calling non-function	`typeof x` before calling
`SyntaxError: Unexpected token`	Syntax mistake	Line number; check brackets, commas
`SyntaxError: Cannot use import statement`	ES module syntax in CommonJS	Use `require()` or add `"type": "module"`
`Error: ENOENT: no such file or directory`	File not found	`console.log(__dirname)` and check path
`Error: EACCES: permission denied`	No write permission	Check file permissions
`RangeError: Maximum call stack size exceeded`	Infinite recursion	Check recursive function's base case
`UnhandledPromiseRejection`	Missing `await` or `.catch()`	Add `try/catch` around `await` calls
`NaN` (not an error, but wrong value)	Arithmetic on non-number	`console.log(typeof variable)` before math

Input Helper Module (Copy into every project)

```
// utils/input.ts
const readline = require("node:readline");

const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout,
});

/**
 * Prompts the user and returns their response as a string.
 * @param {string} prompt - The question to display
 * @returns {Promise<string>} The user's trimmed response
 */
function askQuestion(prompt) {
  return new Promise((resolve) => {
```

```
    rl.question(prompt, (answer) => resolve(answer));
  });
}

/**
 * Prompts until a valid number is entered.
 * @param {string} prompt - The question to display
 * @returns {Promise<number>} A valid number
 */
async function askNumber(prompt) {
  while (true) {
    const input = await askQuestion(prompt);
    const num = Number(input.trim());
    if (!isNaN(num)) return num;
    console.log("Invalid input: please enter a number.");
  }
}

/**
 * Closes the readline interface. Call at the end of every program.
 */
function close() {
  rl.close();
}

module.exports = { askQuestion, askNumber, close };
```

20. Teaching AI Tools Responsibly

AI coding assistants can provide explanations, hints, test ideas, and alternative examples. They can also produce incorrect TypeScript, misunderstand Node.js module configuration, recommend untrusted npm packages, or conceal asynchronous behavior. Treat every response as an unverified proposal. The course must remain accessible without paid AI access or a personal account.

Instructor use of AI during preparation

Instructors may use approved AI tools to brainstorm examples, draft alternative explanations, propose test inputs, or identify likely misconceptions. The instructor remains accountable for accuracy and pedagogy. Verify every API and language claim against the target Node.js release and MDN or Node.js documentation; run examples and tests; review package recommendations; and test the learning workflow with the intended access technology. Never upload identifiable student work, accommodation plans, credentials, assessment answers, or confidential school information. Do not use AI-generated feedback as the sole basis for grading. Disclose substantive AI assistance when institutional policy

or the publication process requires it.

Learning goals and appropriate uses

Students should describe the limitations of code generation, ask focused questions, verify TypeScript behavior using the runtime and official documentation, identify generated code beyond their current level, and disclose assistance when required. When permitted, AI may explain a diagnostic, rephrase a concept, suggest edge cases, critique an algorithm, or generate test inputs. Prefer hint-based prompts and have the learner describe their own understanding first.

TypeScript and Node.js risks

Review AI output for:

- Confusion between CommonJS `require()` and ES module `import`, including package `type`, file extensions, and runtime version.
- Incorrect claims about semicolon insertion, equality, coercion, truthy/falsy values, `undefined`, `null`, or `NaN`.
- Missing `await`, unhandled promise rejections, callbacks that do not complete, or incorrect execution-order assumptions.
- Browser-only APIs used in a Node.js lesson or incorrect Node.js APIs.
- Unsafe file paths, command execution, network requests, or untrusted input handling.
- Unnecessary npm dependencies, typo-squatted or abandoned packages, or packages with excessive permissions.
- Dense chaining, nested callbacks, clever one-liners, and output difficult to review sequentially.

Run examples in the course's declared Node.js environment, inspect values and types, and test success, boundary, and failure cases. Verify API claims in official Node.js documentation or MDN.

Screen-reader and output review

Review whether braces, nested callbacks, and asynchronous control flow are navigable and explainable; identifiers are descriptive; prompts and output provide context; results arrive in a predictable order; module style is consistent; and the AI interface itself is accessible. Prefer named functions and intermediate variables when they improve understanding. Make asynchronous behavior explicit.

Privacy, security, and equity

Do not submit student records, credentials, private code, API keys, or confidential school information to public AI systems. Follow institutional policies, age requirements, and approved-tool lists. Treat terminal commands and package recommendations as untrusted. Verify package names and provenance before

installation. Provide a non-AI path and never make a paid account a prerequisite. Discuss how generated examples may assume mouse use, visual browser interaction, graphical output, or a particular operating system.

Verification protocol: Ask – Inspect – Verify – Adapt – Explain – Disclose

1. **Ask:** state the problem and what has already been tried.
2. **Inspect:** read the response and flag unfamiliar APIs, packages, syntax, and assumptions.
3. **Verify:** check MDN or Node.js documentation; run a minimal example; test normal, boundary, and invalid input.
4. **Adapt:** simplify, secure, format, and make code accessible to sequential reading.
5. **Explain:** have the student explain control flow and make a new change without AI.
6. **Disclose:** record the tool and assistance when required.

Activity-specific policy

- **Practice:** hints and explanations may be allowed; request complete solutions only for code-review activities.
- **Homework/projects:** define permitted assistance and require disclosure and verification.
- **Individual assessments:** do not permit AI to perform the skill being measured unless explicitly included in the assessment or an accommodation.
- **Accessibility tools:** screen-reader, braille, magnification, and keyboard access tools are not prohibited answer-generation tools.

Discussion prompts

- An AI answer mixes CommonJS and ES module syntax. How would you diagnose the mismatch?
- A suggested npm package has a plausible name but few trustworthy details. What should you check?
- A promise-based program prints results in an unexpected order. How can you reason about `await`?
- What must a student understand if AI generated most of a function?

---

21. Comprehensive Instructional Planning and Course Implementation

This section consolidates practices that apply across the philosophy, learner-context, classroom setup, spoken-code, unit, assessment, differentiation, and re-

source sections above. Use it as a course-planning reference from intake through capstone evaluation.

21.1 Course planning and alignment

Map each unit to prerequisites, learning objectives, practice opportunities, and evidence of mastery. Do not assume every unit needs equal time.

- Identify outcomes: problem decomposition, code reading and writing, data representation, control flow, functions or methods, debugging, testing, and communication.
- Separate programming prerequisites from access-tool prerequisites.
- Record the operating system, editor/IDE, runtime, terminal, screen reader, braille display, magnification, and adaptive input technology used.
- Identify essential concepts, enrichment topics, and material that can be deferred without breaking later lessons.
- Plan checkpoints after each major concept group and adjust pacing using student evidence.
- Maintain a map linking each lesson to objectives, practice, assessment, accessibility considerations, and likely misconceptions.

A suggested progression is environment and navigation; computing foundations; expressions and data; input/output; decisions and loops; collections; functions or methods; object-oriented concepts; files and errors; testing; projects; capstone. Preserve prerequisites when reordering.

21.2 Readiness and learner partnership

Ask about preferred access methods, prior programming experience, keyboard navigation, file management, and what makes technical instruction usable. Ask about functional access requirements rather than requesting unnecessary diagnostic details.

Use a low-stakes baseline activity to observe how the learner opens and navigates a source file, moves by line/word/character/symbol, creates and saves a file, runs a command, locates output, and reads a diagnostic. Record strengths and supports, not just deficits. Revisit the baseline to recognize growth. Keep the learner central to decisions about tools and workflow.

21.3 Repeatable lesson architecture

1. **Purpose:** state the problem the lesson addresses.
2. **Prerequisite retrieval:** revisit the prior concept needed today.
3. **Vocabulary:** define technical terms and spoken symbol conventions.
4. **Explicit model:** demonstrate a small runnable example and narrate navigation and code changes.
5. **Trace:** read the code structurally and predict behavior.

6. **Guided practice:** solve a similar problem with prompts and fading support.
7. **Independent transfer:** apply the concept to a new requirement.
8. **Debug and reflect:** examine an intentional error or unexpected result.
9. **Exit check:** collect an explanation, prediction, code change, or test.

State one navigation action at a time, allow the learner to complete it, and confirm the result before continuing. Provide source files and commands in advance when possible. Narrate all actions in shared-screen demonstrations and provide an equivalent text-based route.

21.4 Explicit instruction for accessible code navigation

Teach file structure before reading every line. Identify current focus and how to return to a known location. Teach line, word, character, symbol, selection, search, and go-to-line navigation, plus navigation to functions, methods, classes, and diagnostics. Explain indentation and block boundaries. Use manageable line lengths and avoid unnecessarily nested expressions. Give students time to read diagnostics and output independently. Do not take over the keyboard without explaining the action and obtaining the learner's agreement.

21.5 Language-specific instructional emphasis: TypeScript

TypeScript's static type system and JavaScript runtime should be distinguished explicitly:

- Teach `let`, `const`, and scope before elaborate function patterns.
- Contrast `===` and `==` with concrete values; make coercion visible in REPL experiments.
- Treat `undefined`, `null`, empty strings, zero, false, and `NaN` as distinct values.
- Distinguish arrays from objects and explain property access, index ranges, mutation, and iteration.
- Introduce asynchronous input deliberately. Explain promises, `async`, `await`, and output order.
- Teach CommonJS and ES modules as separate systems. State module mode and Node.js version before troubleshooting imports.
- Make error handling and file paths explicit; distinguish syntax errors, runtime exceptions, rejected promises, and logic errors.
- Use ESLint and formatting tools as feedback aids, but explain each diagnostic.

21.6 Misconceptions and diagnostic teaching

Ask learners to predict behavior, explain their prediction, run a small experiment, and revise their mental model.

Common issue	Instructional response
Confusing assignment with comparison	Contrast operators and trace values before and after execution.
Treating input as already numeric	Identify input type, convert explicitly, and test valid and invalid values.
Off-by-one loop or index errors	Trace first, typical, and final iterations; state the valid index range.
Choosing an unsuitable collection	Describe access patterns and required operations before choosing a structure.
Writing one large function or method	Decompose the task by responsibility and test each part.
Treating a running program as necessarily correct	Define expected behavior and test normal, boundary, and invalid inputs.
Copying code without tracing it	Ask the learner to predict a new case and make a modification.
Misreading an error message	Locate the useful diagnostic, identify file and line, and inspect surrounding code.

Language-specific pitfalls include: coercion and equality; `undefined` versus `null`; truthy and falsy values; array mutation; callback and promise ordering; CommonJS versus ES modules; and browser APIs versus Node.js APIs.

21.7 Differentiation without lowering the learning objective

Adjust the path, not the intellectual goal. Supports may include smaller task increments, additional practice, explicit worked examples, alternative information formats, extra navigation time, starter code with gradually faded scaffolding, oral explanation or accessible demonstration, pair programming with rotating roles, predictable routines, and enrichment tasks that deepen the same concept.

Do not assume all blind students need the same support or that all low-vision students prefer magnification. Ask, document, and review supports with the learner.

21.8 Formative assessment, feedback, and grading

Collect evidence frequently: explain a concept, predict output, trace a loop, justify a data structure, find and fix a bug, write a boundary test, refactor code, or demonstrate an accessible workflow.

A practical rubric can score each dimension from 1 (beginning) to 4 (independent and consistent):

Dimension	Evidence
Conceptual reasoning	Explains the problem, algorithm, and language concepts
Implementation	Produces a solution that meets requirements
Code comprehension	Reads, traces, explains, and modifies code
Testing and debugging	Uses test cases and interprets diagnostics
Accessibility and communication	Produces understandable interaction and operates the workflow with access tools
Independence and reflection	Describes decisions, limitations, and next steps

Share criteria before assessment. Feedback should name observed behavior, explain its impact, and identify an actionable next step. Do not grade screen-reader speed, typing speed, or visual formatting as proxies for programming competence.

21.9 Debugging and classroom support protocol

1. Ask the learner to describe expected behavior.
2. Ask what actually happened.
3. Read the complete error or diagnostic, including source and location.
4. Identify the smallest relevant code section.
5. Form one hypothesis.
6. Change one thing and rerun the program or test.
7. Record what the result taught the learner.

Distinguish environment failures, tool-access failures, syntax/compile errors, runtime errors, and logic errors. Maintain a troubleshooting record with environment, exact command, error text, attempted correction, and outcome. Remove credentials and private information before sharing logs.

21.10 Pair programming and collaboration

Use explicit roles: the **driver** controls the keyboard and implements the agreed step; the **navigator** reasons about the algorithm, checks requirements, proposes tests, and reviews the result. Rotate roles. Do not automatically assign a blind student the navigator role. Both learners should have opportunities to operate tools and write code. Provide shared source files and task instructions in accessible formats.

21.11 Project and capstone supervision

Require a proposal that includes the problem and intended user, functional requirements, out-of-scope features, input/output design, data representation, major functions/methods/classes, accessibility requirements, milestones, tests, and any libraries or AI assistance used.

Review milestones: requirements, design, first working slice, feature completion, tests, documentation, and final explanation. Encourage a modest, complete, well-tested project over an ambitious unfinished one.

21.12 Collaboration with families and support professionals

When appropriate and with learner consent, coordinate with special educators, teachers of students with visual impairments, assistive-technology specialists, interpreters, and families. Agree on who supports access technology, who teaches programming, and how the student obtains help. Keep the learner's agency central, share only necessary information, and follow school privacy policies. Do not make aides the default operators of the student's computer.

21.13 Instructor preparation checklist

Before each unit, complete the learner-facing activities; verify commands and versions on the classroom platform; test keyboard and assistive-technology workflows; prepare expected output and common error examples; identify vocabulary, symbols, misconceptions, and prerequisites; provide accessible instructions and assessment criteria; and decide how understanding will be observed.

After each unit, review student work and feedback, note barriers, adjust pacing and scaffolds, and record reproducible corrections. Document agreed accommodations rather than silently changing expectations.

21.14 Quality assurance and continuous improvement

Review links, software versions, commands, APIs, package recommendations, and assessment assumptions periodically. Test with actual screen readers and braille displays where possible; a successful build or visual inspection alone does not establish accessibility.

When reporting a barrier, include the page, reproduction steps, operating system, editor/runtime version, assistive technology and version, expected behavior, and actual behavior. Exclude personally identifying or sensitive student information. Submit corrections through the project contribution process.

Chapter 19

Programming Code Glossary

A detailed student reference for Python, JavaScript, TypeScript, Java, and the optional post-capstone Kotlin supplement. Find the term in the language section, then use that language's INDEX.md to locate the teaching lesson. Definitions distinguish concepts that are easy to confuse.

Shared computer science vocabulary

Algorithm

A finite, ordered method for solving a problem. It defines steps, decisions, inputs, and expected outputs; a program implements an algorithm.

Argument

The actual value supplied to a function call. The parameter is the receiving name in the function definition.

Assignment

Storing a value in a variable or assignable location. Assignment is not equality comparison.

Boolean

A logical value that is true or false. Python spells literals True and False; the other languages use lowercase.

Bug

A defect causing a crash, incorrect result, or unmet requirement.

Class

A definition describing data and behavior for a kind of object.

Compiler

A program translating source code and reporting compile-time problems. Java compiles to JVM bytecode; TypeScript emits JavaScript.

Condition

An expression deciding which branch runs or whether a loop continues. Java requires Boolean conditions; Python and JavaScript also allow truth-value testing.

Constant

A binding that cannot be reassigned. This does not necessarily make the referenced object immutable.

Data type

A classification defining possible values and valid operations, such as integer, string, Boolean, array, or object.

Debugging

Systematically reproducing a problem, narrowing its cause, inspecting state, testing a hypothesis, correcting the defect, and retesting.

Declaration

An instruction introducing a name such as a variable, function, class, or type. Initialization gives a variable its first value.

Expression

A combination of values, names, operators, and calls evaluated to produce a value.

Function

A callable unit of code performing a task. It may accept parameters and return a result.

Input

Data received from a person, file, command line, network, or another system. Validate external input.

Instance

One particular object created from a class.

Iteration

One repetition of a loop body, or traversal of a sequence or collection.

Library

Reusable code packaged for use by programs. A standard library ships with a language or runtime.

Literal

A value written directly in source code, such as 42, 3.5, a quoted string, or true.

Loop

A control-flow structure repeating instructions. for commonly traverses items; while repeats while a condition is true.

Method

A function associated with an object or class.

Module

A file or unit grouping code and exposing selected names for reuse.

Object

A value grouping data and often behavior. Java objects are class instances; JavaScript and TypeScript also have flexible property objects.

Operator

A symbol or keyword performing an operation, such as arithmetic, comparison, or logic.

Parameter

A named variable in a function definition that receives an argument from the caller.

Return value

The value a function or method gives back to its caller.

Scope

The region of code where a name can be accessed. Local, block, function, and module scope rules differ by language.

Statement

A complete instruction such as a declaration, assignment, conditional, loop, or return.

String

A sequence of text characters. Strings are immutable in all languages covered.

Syntax

Formal rules defining valid source-code structure. Valid syntax does not guarantee correct logic.

Testing

Checking program behavior against expected outcomes. Unit tests focus on a small function or method.

Variable

A named binding used to refer to a value. Declaration, assignment, mutability, and type rules vary by language.

Python glossary

init

Initialization method called when an instance is initialized. Commonly assigns self attributes; allocation itself involves **new**.

name and main guard

Module attribute equal to **main** when run as entry point. The if **name** == **main** guard protects entry-point code from running on import.

append()

List method adding one item to the same list. It mutates the list and returns None.

break and continue

break exits the nearest loop. continue skips the rest of the current iteration and proceeds to the next.

dict

Mutable mapping of keys to values. Keys must be hashable; values can be any objects.

elif

Additional conditional branch evaluated only if earlier conditions are false. Python uses elif, not else if.

except and finally

except handles a matching exception. finally is used for cleanup and runs whether or not an exception occurred, subject to process termination.

float and int

float is binary floating point; many decimal fractions are approximate. int is arbitrary precision in ordinary Python implementations.

for and while

for iterates over an iterable. while repeats while its condition is true.

F-string

String prefixed with f that evaluates expressions in braces, for example f-string interpolation of a name.

import

Loads a module and binds names for use, such as importing math.

input()

Reads one line from standard input and returns a string. Convert it before numeric operations.

is and is not

Identity comparisons testing whether references are the same object. Use == for value equality; is None is conventional.

Iterable

Object supplying members one at a time, such as list, tuple, string, dictionary, set, or range.

len()

Returns number of items in a collection or code points in a string, not necessarily user-perceived Unicode characters.

list and tuple

list is mutable and ordered. tuple is ordered and immutable; a one-item tuple needs a trailing comma.

None

Singleton meaning absence of a value. Compare using is None.

open() and file modes

Opens a file. r reads, w writes and truncates, a appends. Prefer a with statement and specify UTF-8 encoding.

range()

Arithmetic sequence used in loops. Stop is excluded: range(3) yields 0, 1, 2.

return

Ends a function and optionally sends a value to caller. Falling off the end returns None.

set

Mutable collection of unique hashable elements; no positional indexing.

Slicing

Selects a sequence portion with start, stop, and optional step; start is included and stop excluded.

with and context manager

Managed context with reliable cleanup. File contexts close files automatically through the context-manager protocol.

Comprehension

Concise collection construction from an iterable, optionally filtered. Use a loop when clearer.

Docstring

String literal first in a module, class, or function, stored in **doc** and used to document behavior.

Exception

Object representing an error. Common types include ValueError, TypeError, KeyError, IndexError, and FileNotFoundError.

Generator

Iterator-producing function or expression yielding values lazily. Generator functions use yield.

Lambda

Small anonymous function expression containing one expression. Use def for multi-statement or documented functions.

MRO

Method Resolution Order: order Python searches base classes for attributes, especially with multiple inheritance.

Mutable and immutable

Lists and dictionaries can change in place; strings, integers, and tuples cannot. A variable can be rebound either way.

PEP 8

Python style guide recommending four-space indentation and snake_case names for functions and variables.

Property

Managed attribute commonly created with property decorator, allowing attribute-style access with getter/setter logic.

Type hint

Optional syntax describing expected types. Not generally enforced by Python at runtime; static tools can analyze hints.

Virtual environment

Isolated Python interpreter/package context. The course uses uv to create and manage the project's `.venv`, install dependencies, and run commands without global pip installation.

uv

A Python version, project, and package manager. `uv init` creates project configuration, `uv add package` installs and records a project dependency, `uv run command` executes inside the managed environment, `uv sync` installs the dependencies recorded in the project files, and `uv tree` displays the dependency tree. uv manages the project-local `.venv` and can install Python interpreters.

JavaScript glossary

Array

Indexed ordered collection that can change length. Methods include push, pop, map, filter, find, reduce, and forEach.

async and await

An async function always returns a Promise. await waits for a Promise inside an async function without blocking the whole event loop.

BigInt

Primitive for integers beyond Number's safe range. BigInt and Number arithmetic cannot be mixed directly.

const, let, and var

const is block-scoped and cannot be reassigned; let is block-scoped and reassignable; var is function-scoped legacy syntax.

console.log()

Writes values to the console; common terminal output in Node.js, not a file-writing API.

Destructuring

Extracts array elements or object properties into variables.

Event loop

Coordinates synchronous call-stack execution, queued callbacks, and asynchronous completions. Long synchronous work delays other tasks.

export and import

ES module declarations exposing and bringing in bindings. Dynamic import returns a Promise.

filter, map, and forEach

filter creates an array of passing elements; map creates a transformed array; forEach calls a callback and returns undefined.

for...of and for...in

for...of iterates values from an iterable; for...in iterates enumerable property keys.

Map and Set

Map stores key-value pairs with arbitrary key types. Set stores unique values.

NaN

Special Number value for invalid numeric results. It is not equal to itself; use Number.isNaN().

null and undefined

null represents intentional absence of an object value. undefined commonly means missing, uninitialized, or no returned value.

Number

IEEE 754 binary64 numeric primitive. Use Number.isFinite() and Number.isSafeInteger() where appropriate.

Object

Property collection. Ordinary keys are strings or symbols; use Map for arbitrary key types.

package.json

Node project metadata. Its type field selects ES module or CommonJS interpretation for .js files in that package scope.

Promise

Represents an asynchronous result that is pending, fulfilled, or rejected. Handle with then/catch or async/await.

readline

Node built-in module for line-by-line input from a stream such as standard input.

require() and CommonJS

require loads modules using Node's CommonJS system; module.exports exposes values. ES modules use import.

Spread syntax

Expands iterables or object properties. Array/object spreads make shallow copies, not deep clones.

Strict equality ===

Compares without loose equality coercion. Preferred for most comparisons; objects compare by identity.

Template literal

Backtick-delimited string supporting multiline text and expression interpolation.

Arrow function

Function expression using arrow syntax. It has no own this or arguments; not interchangeable with every traditional function.

Callback and closure

Callback is a function passed to be called later. Closure is a function plus access to surrounding lexical variables.

ECMAScript

Standard defining JavaScript syntax and semantics. Node and browsers add different host APIs.

Hoisting

Processing declarations before scope execution. Function declarations and var differ from let/const temporal-dead-zone behavior.

JSON

Text format for objects, arrays, strings, numbers, booleans, and null. Use JSON.parse() and JSON.stringify().

Node.js and npm

Node.js runs JavaScript outside browsers. npm manages dependencies, scripts, and lockfiles. Use `npm install` or `npm install --save-dev` in the project directory to keep dependencies local.

npx

npm's package-execution command. It runs a package binary from the current project's dependencies when available, or can fetch a package into npm's cache for one-off execution. Prefer project-local development dependencies and `npx` or npm scripts over global npm installs.

Prototype

Object used in property lookup when a property is not found directly.

Type coercion and truthiness

Coercion converts types automatically or explicitly. Falsy values include false, 0, empty string, null, undefined, and NaN; empty arrays/objects are truthy.

TypeScript glossary

any and unknown

any disables most type checking. unknown accepts any value but requires narrowing before use; prefer unknown for untrusted data.

as assertion

Tells compiler to treat an expression as a type. It does not convert or validate runtime data.

enum

Defines named constants. Numeric enums can create runtime objects; string unions or const objects may be simpler.

interface and type alias

interface describes object shape; type names a type expression including unions, intersections, tuples, and function types.

never

Type representing values that should not occur; used for non-returning functions and exhaustive checks.

Null and undefined in strict mode

With strictNullChecks, they are distinct types and must be handled or narrowed before non-null use.

Union and intersection

Union uses vertical-bar syntax for alternatives; intersection combines requirements.

Type annotation and inference

Annotation explicitly states a type; inference derives a type from values and context. Neither adds runtime validation.

Generic

Type or function parameterized by a type variable, preserving type information across reusable operations.

keyof and readonly

keyof produces a union of keys of a type. readonly prevents assignment through a type but does not freeze runtime objects.

strict and tsconfig.json

strict enables stronger checks. tsconfig.json controls source inclusion, target, module resolution, strictness, and output.

tsc

TypeScript compiler. Checks types and can emit JavaScript; prefer the project-local compiler.

Discriminated union and narrowing

A shared distinguishing property enables compiler narrowing based on control flow.

Structural typing

Compatibility based on members a value provides, not its declared type name.

Type guard

Condition or function giving evidence about a value's type so compiler can narrow it.

Type erasure

TypeScript-only annotations, interfaces, and aliases are removed during compilation. Validate external data at runtime.

Utility type

Built-in transformations such as Partial, Required, Readonly, Pick, Omit, and Record.

Java glossary

abstract class

Cannot be instantiated directly; can contain implemented and abstract methods.

Array and ArrayList

Array has fixed length and one component type; use its length field. ArrayList is a resizable List; generic arguments use reference types such as Integer.

boolean, char, int, and long

boolean is true/false. char is a UTF-16 code unit. int is 32-bit signed; long is 64-bit signed.

break and enhanced for

break exits the nearest loop or switch. Enhanced for traverses array or Iterable elements.

catch and finally

catch handles an exception. finally performs cleanup; prefer try-with-resources for AutoCloseable resources.

class, constructor, and this

class declares a type. Constructor initializes an instance and has no return type. this refers to the current instance.

final and static

final prevents reassignment, overriding, or subclassing depending on use. static associates a member with the class.

HashMap, HashSet, and TreeMap

HashMap stores pairs without guaranteed order. HashSet stores unique values. TreeMap orders keys.

implements and interface

A class uses implements to satisfy an interface contract. Interfaces define member contracts.

JDK, JRE, and JVM

JDK provides development tools including javac. JRE describes runtime components. JVM executes bytecode.

List and Map

List is an ordered collection allowing duplicates. Map stores key-value associations.

main method

Traditional entry point: public static void main(String[] args).

null

Absence of an object reference. Calling an instance method through null causes NullPointerException; primitives cannot hold null.

package and import

Package organizes types and access. Import allows simple type names; importing a package does not import subpackages.

private and public

private restricts member access; public exposes a member wherever its declaring type is accessible.

return and void

return ends a method and may send a value. void means no return value.

Scanner

java.util input parser. Mixing nextInt() and nextLine() can leave a newline for the next read.

String and equals()

String is immutable. Compare contents with equals(), not ==, which compares reference identity.

switch

Selects a branch; Java supports switch statements and newer switch expressions.

throw and throws

throw raises an exception object. throws declares checked exceptions a method may propagate.

Try-with-resources

Automatically closes resources implementing AutoCloseable.

Wrapper and autoboxing

Wrapper classes such as Integer correspond to primitives. Unboxing null throws NullPointerException.

Checked exception

Compiler requires code to catch or declare it with throws, unless it is unchecked.

Encapsulation, inheritance, and polymorphism

Encapsulation hides internal state; inheritance extends a superclass; polymorphism lets a general reference invoke concrete overridden behavior.

Garbage collection

Reclaims unreachable heap objects. It does not replace closing files or sockets.

Generics

Type parameters provide compile-time type safety; Java implements generics with type erasure.

Maven and Gradle wrappers

Project-local scripts that run a declared Maven or Gradle version for a project. Maven commonly uses `./mvnw` on macOS/Linux and `mvnw.cmd` on Windows; Gradle uses `./gradlew` and `gradlew.bat`. The wrapper avoids requiring each learner to install a global build tool.

JUnit, overloading, and overriding

JUnit tests behavior with assertions. Overloading changes parameter lists. Overriding replaces inherited instance behavior; use @Override.

Kotlin supplement glossary

data class

Compiler generates equals, hashCode, toString, and copy based on primary-constructor properties.

fun, val, and var

fun declares a function. val is a read-only binding. var is reassignable.

Null safety and nullable type

Kotlin distinguishes non-null String from nullable String?. Java interoperability and external data may still require checks.

Safe call and Elvis operator

Safe call ?. accesses only when receiver is non-null. Elvis ?: supplies a fallback when its left side is null.

when and smart cast

when is a multi-branch expression or statement. Smart cast narrows a stable value after a type or null check.

Extension function

Function with a receiver type, callable with member-like syntax; it does not add a real class member.

object and companion object

object declares a singleton. Companion object provides class-associated members but is not identical to Java static.

Coroutine

Concurrency abstraction for suspending/resuming work. kotlinx.coroutines supplies the framework; suspend alone is not the framework.

Cross-language distinctions

Concept	Python	JavaScript	TypeScript	Java	Kotlin
Variable	assignment; optional hints	let / const	let / const plus types	declared type; final	var / val
Boolean	True / False	true / false	true / false	true / false	true / false
Function	def	function / arrow	typed function / arrow	method in class	fun

Concept	Python	JavaScript	TypeScript	Java	Kotlin
Null-like	None	null / undefined	both; strict null checks	null reference	nullable types such as String?
Sequence	list / tuple	Array	Array / T[]	array / List	List / MutableList
Equality	== value; is identity	=== strict equality	JavaScript runtime ===	equals() for objects; == for primitives	== structural; === referential

Runtime and type-system distinctions

- Python type hints are optional metadata; ordinary execution does not enforce them.
- JavaScript is dynamically typed and its runtime does not enforce TypeScript annotations.
- TypeScript checks types before execution, then emits JavaScript. Type annotations are erased; validate external data at runtime.
- Java checks declared types at compile time. References can still be null, and generics use type erasure.
- Kotlin distinguishes nullable and non-nullable types, but Java interoperability and external data can still require runtime checks.

Chapter 20

Global Index

Indices for each course in this book. Rather than page numers specific to each item, only guidance to which course lesson contains the material is given.

20.1 C Topic Index

Use this index to locate topics in the C curriculum and complete C lessons. The course targets modern C and emphasizes explicit memory, types, and program behavior.

Learning sequence

1. Computing foundations, source files, compiler, and first C program.
2. Variables, constants, fundamental types, expressions, and conversions.
3. Standard input and output, format specifiers, and input validation.
4. Boolean expressions, decisions, and control flow.
5. Loops, counters, accumulators, and nested repetition.
6. Arrays, strings, and indexed data.
7. Functions, parameters, return values, and scope.
8. Pointers, addresses, and the relationship between arrays and pointers.
9. Structures, enums, and user-defined data.
10. Dynamic memory, allocation, deallocation, and ownership discipline.
11. Files, error handling, testing, and integrated projects.

C-specific concepts

- Compilation and linking; compiler diagnostics and warning flags.
- Integer and floating-point types, conversions, and overflow considerations.
- Format strings and safe input handling.
- Array bounds and null-terminated character strings.

- Function declarations, definitions, and header files.
- Pointers, dereferencing, pointer arithmetic, and NULL.
- struct, enum, typedef, and modular interfaces.
- Dynamic allocation with malloc, calloc, realloc, and free.
- File streams and return-value-based error handling.
- Undefined behavior and the importance of warnings and testing.

Course references

- C overview
- Complete lessons
- Development editors and IDEs
- Terminal prerequisite

20.2 C# Topic Index

Use this index with the C# curriculum overview and complete C# lessons. The course targets modern C# and .NET application development.

Learning sequence

1. .NET toolchain, projects, and first console application.
2. Variables, types, literals, and expressions.
3. Console input/output and explicit conversions.
4. Boolean logic, decisions, and pattern matching.
5. Loops, arrays, strings, and collections.
6. Methods, parameters, return values, and scope.
7. Classes, objects, properties, and constructors.
8. Inheritance, interfaces, and polymorphism.
9. Files, exceptions, and structured data.
10. Testing, debugging, and data processing.
11. Integrated projects and capstone.

C# and .NET concepts

- .NET SDK, runtime, project files, and dotnet CLI.
- Value types, reference types, nullable reference types, and conversions.
- string, arrays, List, dictionaries, and generic collections.
- Methods, overloads, optional parameters, and expression-bodied members.
- Classes, records, properties, constructors, and access modifiers.
- Interfaces, inheritance, and polymorphism.
- Exceptions, file APIs, and resource disposal.
- LINQ and collection processing.
- Unit testing, compiler diagnostics, and debugging.

Course references

- C# overview
- Complete lessons
- Development editors and IDEs
- Terminal prerequisite

20.3 C++ Topic Index

Use this index with the C++ curriculum overview and complete C++ lessons. The course uses modern C++ and emphasizes resource safety, standard-library abstractions, and clear program design.

Learning sequence

1. Toolchain, compilation, source files, and first program.
2. Variables, fundamental types, constants, and expressions.
3. Input, output streams, and conversions.
4. Boolean logic, decisions, and loops.
5. Strings, arrays, and standard containers.
6. Functions, parameters, return values, and scope.
7. References, pointers, and object lifetime.
8. Classes, objects, constructors, and encapsulation.
9. Inheritance, polymorphism, and interfaces.
10. Files, exceptions, and resource management.
11. Algorithms, testing, and integrated projects.

C++-specific concepts

- C++20 compilation and standard-library headers.
- std::cin, std::cout, and stream state.
- std::string, std::vector, std::array, and associative containers.
- References, pointers, const, and lifetime.
- Classes, access control, constructors, and member functions.
- RAII and deterministic resource management.
- Exceptions and error-handling design.
- Templates and standard algorithms at an introductory level.
- Smart pointers and ownership-aware design.
- Build, compile, link, and test workflows.

Course references

- C++ overview
- Complete lessons
- Development editors and IDEs
- Terminal prerequisite

20.4 Go Topic Index

Use this index with the Go curriculum overview and complete Go lessons. The course introduces Go for services, command-line programs, and concurrent systems.

Learning sequence

1. Go toolchain, modules, source files, and first program.
2. Variables, constants, types, and expressions.
3. Input/output and explicit conversion.
4. Boolean logic, decisions, and loops.
5. Arrays, slices, maps, and iteration.
6. Functions, multiple return values, and scope.
7. Structs, methods, and interfaces.
8. Packages, modules, and error handling.
9. Files and structured data.
10. Testing, formatting, and standard library use.
11. Integrated projects and capstone.

Go-specific concepts

- go.mod, modules, packages, and imports.
- Static typing and short variable declarations.
- Slices, maps, arrays, and their semantics.
- Functions and multiple return values.
- Structs, methods, and interface satisfaction.
- Explicit error values and wrapping.
- Pointers and value semantics.
- defer and resource cleanup.
- Goroutines and channels as introductory concurrency concepts.
- gofmt, go test, and toolchain workflows.

Course references

- Go overview
- Complete lessons
- Validation record
- Development editors and IDEs

20.5 Java Index

Use this index to locate Java lessons by language feature, API, development tool, or learner task. Java and Kotlin are kept distinct: Kotlin is an optional post-capstone supplement, not a fifth core curriculum. For definitions, see the Java glossary and Kotlin supplement glossary.

JDK, editor, project setup, and accessibility

Topic and learner search terms	Lesson
JDK; Java installation; `java -version`; `javac -version`; PATH	Unit 0: JDK installation
JRE; JVM; bytecode; compiler; runtime; Java execution model	Unit 0: Java platform
VS Code; Theia IDE; Java language tools; editor; terminal	Unit 0: Editor and language tools
Screen reader mode; keyboard shortcuts; accessible Java workflow	Unit 0: Accessible development environment
Java project; source file; class; `main()`; entry point; Hello World	Unit 0: First Java project
Primitive types; `int`; `long`; `double`; `boolean`; `char`; `String`	Unit 1: Variables and data types
Variable declaration; assignment; `final`; constants; type conversion	Unit 1: Variables and operators

Problem solving, input, decisions, and loops

Topic and learner search terms	Lesson
Computer science; algorithms; problem solving; software development process	Unit 1: Foundations and Unit 2: Development process
`Scanner`; standard input; `nextInt()`; `nextLine()`; user prompts	Unit 2: Input with Scanner
Scanner newline issue; mixing token input and line input	Unit 2: Scanner pitfall
`Math`; numeric expressions; arithmetic; conversions	Unit 2: Math operations
Boolean expressions; comparisons; `&&`; ‘	
`if`; `else if`; `else`; `switch`; branching	Unit 3: Decisions
String comparison; `equals()`; reference equality; `==`	Unit 3: String comparison
`for`; enhanced for; `while`; `do-while`; iteration	Unit 3: Loops
`break`; `continue`; loop control; nested loops	Unit 3: Loop control

Arrays, strings, methods, and object-oriented programming

Topic and learner search terms	Lesson
Array; fixed length; indexes; `length`; initialization	Unit 4: Arrays
Multidimensional array; rows; columns; nested arrays	Unit 4: Multidimensional arrays
`ArrayList`; `List`; resizable collection; generics	Unit 4: ArrayLists
String; immutability; string methods; `equals()`; `StringBuilder`	Unit 5: Strings
Method; method declaration; return type; parameter; argument; `void`	Unit 6: Methods
Method overloading; varargs; arrays as arguments	Unit 6: Overloading and arguments
Class; object; instance; constructor; `this`; fields; methods	Unit 7: Classes and objects
`private`; `public`; access modifiers; encapsulation	Unit 7: Encapsulation
Inheritance; `extends`; `super`; overriding; polymorphism	Unit 7: Inheritance
Abstract class; interface; `implements`; abstraction	Unit 7: Abstract classes and interfaces

Files, exceptions, collections, and testing

Topic and learner search terms	Lesson
Read file; write file; `Path`; `Files`; `java.nio.file`	Unit 8: File operations
Exception; `try`; `catch`; `finally`; `throw`; `throws`	Unit 8: Exception handling
Checked exception; unchecked exception; custom exception	Unit 8: Java exceptions
Collections Framework; `List`; `Set`; `Map`; interfaces	Unit 9: Collections
`HashMap`; `HashSet`; `TreeMap`; sorting; key-value storage	Unit 9: Collection implementations
Word frequency; CSV; data analysis; text processing	Unit 9: Data analysis
JUnit; test method; assertion; test fixture; unit test	Unit 9: Testing with JUnit
Maven; Gradle; build wrapper; project dependencies	Unit 9: JUnit project setup

Projects and Kotlin supplement

Topic and learner search terms	Lesson
Requirements; architecture; text adventure; rooms; player; command-line to-do	Units 10–11: Projects
Personal data manager; capstone; accessibility; documentation; testing	Unit 11: Capstone
Kotlin/JVM; Java interoperability; Kotlin setup; Java libraries from Kotlin	Kotlin supplement: Relationship and setup
`val`; `var`; type inference; nullable types; null safety; safe call `?.`; Elvis `?:`	Kotlin supplement: Variables and null safety
`fun`; `when`; data classes; properties; extension functions	Kotlin supplement: Functions and classes
Kotlin collections; Java collections; testing; debugging; coroutines; `suspend`	Kotlin supplement: Collections and further topics

Java curriculum · Book overview · Programming glossary

20.6 JavaScript Index

Use this topic index to locate the lesson for a JavaScript concept, API, or programming task. Search phrases include standard terminology and common learner wording. For definitions, see the JavaScript glossary.

Runtime, project setup, and accessibility

Topic and learner search terms	Lesson
Install Node.js LTS; runtime; `node --version`; npm; version verification	Unit 0: Node.js installation
Node.js versus browser JavaScript; ECMAScript; host environment; runtime APIs	Unit 0: JavaScript runtime
VS Code; Theia IDE; editor; workspace; terminal; keyboard shortcuts	Unit 0: Editor setup
Screen reader; keyboard navigation; accessible editor workflow	Unit 0: Development environment
`.js` file; `node app.js`; first program; Hello World; REPL	Unit 0: First JavaScript file

Topic and learner search terms	Lesson
`let`; `const`; variables; reassignment; block scope	Unit 1: Variables
`console.log()`; output; template literal; backticks; interpolation	Unit 1: Output

Values, numbers, input, and control flow

Topic and learner search terms	Lesson
Primitive types; string; number; boolean; null; undefined; `typeof`	Unit 1: Data types
Arithmetic operators; `Math`; `NaN`; numeric calculations	Unit 2: Numbers and Math
`readline`; standard input; terminal prompts; asynchronous input	Unit 2: Node.js input
Number conversion; `Number()`; `String()`; coercion; validation	Unit 2: Input conversion
Comparison operators; Boolean expressions; `&&`; ‘	
`if`; `else if`; `else`; nested conditions; branching	Unit 3: Conditional statements
Strict equality `===`; loose equality `==`; truthy; falsy	Unit 3: Comparisons and truthiness
Ternary operator; `switch`; multi-branch decisions	Unit 3: Alternative branching
`for`; `while`; `break`; `continue`; `forEach()`	Unit 3: Loops

Arrays, objects, strings, and functions

Topic and learner search terms	Lesson
Array; indexes; `length`; `push()`; `pop()`; array mutation	Unit 4: Arrays
`map()`; `filter()`; `forEach()`; functional array methods	Unit 4: Array methods
Object literal; properties; keys; values; nested objects and arrays	Unit 4: Objects
Object iteration; destructuring; spread syntax; rest syntax; shallow copy	Unit 4: Object operations

Topic and learner search terms	Lesson
String methods; template strings; string processing; regular expressions; RegExp	Unit 5: Strings
Function declaration; function expression; arrow function; `return`	Unit 6: Defining functions
Parameters; arguments; default parameters; rest parameters; callbacks	Unit 6: Function parameters
Scope; lexical scope; closure; function passed to another function	Unit 6: Closures

Classes, files, asynchronous operations, and errors

Topic and learner search terms	Lesson
Class; instance; object; `constructor`; `this`; methods	Unit 7: Classes
Private class fields; `#field`; static methods; encapsulation	Unit 7: Class features
`extends`; `super`; inheritance; subclass; overriding	Unit 7: Inheritance
Node `fs`; file system; read file; write file; file paths	Unit 8: File I/O
Asynchronous file operations; Promise; `async`; `await`	Unit 8: Asynchronous file I/O
`try`; `catch`; `finally`; `throw`; custom Error class	Unit 8: Error handling
JSON; `JSON.parse()`; `JSON.stringify()`; serialization	Unit 8: JSON

Modules, collections, testing, and projects

Topic and learner search terms	Lesson
ES modules; `import`; `export`; `type: module`; module resolution	Unit 9: ES modules
CommonJS; `require()`; `module.exports`; `type: commonjs`	Unit 9: CommonJS
`package.json`; package type; dependencies; npm scripts; project-local tools	Unit 9: Node project configuration

Topic and learner search terms	Lesson
`Map`; key-value collection; arbitrary key types; `Set`; unique values	Unit 9: Maps and sets
Word frequency; text processing; CSV; data parsing	Unit 9: Data projects
Jest; test suite; test case; matcher; assertion; test setup	Unit 9: Testing with Jest
Text adventure; command-line to-do manager; requirements; capstone; accessibility	Units 10–11: Projects

JavaScript curriculum · Book overview · Programming glossary

20.7 Julia Topic Index

Use this index with the Julia curriculum overview and complete Julia lessons. The course introduces Julia for numerical, scientific, and data-oriented computing.

Learning sequence

1. Julia runtime, REPL, project, and first script.
2. Values, variables, types, and numeric expressions.
3. Input/output and conversion.
4. Boolean logic, conditionals, and loops.
5. Arrays, tuples, dictionaries, and iteration.
6. Functions, multiple dispatch, and scope.
7. Composite types and data modeling.
8. Files, exceptions, and structured data.
9. Packages, project environments, and reproducibility.
10. Testing, numerical analysis, and data processing.
11. Integrated projects and capstone.

Julia-specific concepts

- Dynamic typing with a rich type system.
- REPL-driven exploration and script execution.
- Multiple dispatch and method definitions.
- One-based indexing and array-oriented operations.
- Tuples, named tuples, dictionaries, and arrays.
- Broadcasting and dotted operations.
- Composite types and constructors.
- Package environments using Project.toml and Manifest.toml.
- Performance considerations, compilation, and type stability.

- CSV and JSON processing through external packages.

Course references

- Julia overview
- Complete lessons
- Development editors and IDEs
- Terminal prerequisite

20.8 Kotlin Topic Index

Use this index with the Kotlin curriculum overview and complete standalone lessons.

Course units

- Unit 0–1: Setup and foundations: toolchain concepts, first program, variables, values, types, and introductory problem solving.
- Unit 2–4: Input, expressions, control flow, and collections: input, conversions, Boolean logic, decisions, loops, arrays, lists, maps, and data operations.
- Unit 5–8: Strings, functions, object-oriented programming, and files: string processing, function design, classes, objects, properties, constructors, files, and errors.
- Unit 9–11: Collections, testing, and projects: collection operations, modular design, testing, data analysis, and capstone work.
- Complete Kotlin reference lessons: consolidated explanations and examples.

Kotlin language concepts

- **Values and variables:** val, var, type inference, explicit types, mutability.
- **Type system:** numeric types, strings, characters, Boolean values, conversions, and nullable types.
- **Null safety:** nullable declarations, safe calls, Elvis expressions, and not-null assertions.
- **Control flow:** Boolean expressions, if, when, loops, ranges, and iteration.
- **Collections:** arrays, read-only and mutable lists, sets, maps, and collection operations.
- **Functions:** parameters, return types, default arguments, named arguments, and scope.
- **Object-oriented programming:** classes, properties, constructors, methods, inheritance, interfaces, and data classes.
- **JVM interoperability:** Kotlin and Java libraries, JVM execution, and build tooling.

- **Files and resources:** reading and writing text, resource management, and error handling.
- **Testing and projects:** testable functions, assertions, project organization, and incremental development.

Accessibility and workflow

The curriculum emphasizes keyboard-first editor use, explicit discussion of punctuation and nesting, text-based input and output, and independent interpretation of compiler diagnostics. For editor selection and accessibility evaluation, see Development editors and IDEs.

Related references

- Kotlin standardization audit
- Kotlin teacher guide
- Expected example outputs
- Terminal prerequisite

20.9 Lua Topic Index

Use this index with the Lua curriculum overview, the existing Lua course overview, and the complete Lua lessons.

Course sequence

- **Unit 0: Environment and workflow** — interpreter, workspace, accessible editor workflow, terminal, and project dependencies.
- **Unit 1: Computing foundations** — program structure, values, variables, and first scripts.
- **Units 2–4: Expressions and control flow** — numeric computation, input/output, conditions, loops, and tables.
- **Units 5–8: Program organization** — iteration, functions, modules, object patterns, files, and errors.
- **Units 9–11: Applied programming** — testing, data processing, projects, and capstone record management.

Lua language concepts

- Dynamic typing and the nil, Boolean, number, string, function, table, and userdata value categories.
- Truthiness: only nil and false are falsey.
- Tables as sequences, maps, records, and objects.
- One-based sequence conventions.
- ipairs for sequence traversal and pairs for key/value traversal.
- Functions as values, closures, and multiple return values.

- Lexical scope and local variables.
- Metatables, metamethods, and prototype-style lookup.
- Modules and require.
- File handles, return values, and explicit error checks.
- External packages for JSON and CSV workflows.

Important distinctions

A Lua table is a general-purpose data structure, not simply an array. Sequence operations conventionally use integer keys beginning at 1, while map-like tables may use strings or other valid key types. The iteration order of pairs is not guaranteed. The length operator is not a general count of every key in an arbitrary table; teach its sequence assumptions explicitly.

Lua's falsey rules differ from languages where zero or an empty string are false. Functions can return multiple values, and callers may receive fewer values than a function returns. These behaviors should be explained through concrete examples.

Course references

- Lua curriculum overview
- Existing toolchain and course notes
- Complete lessons
- Automated validation record
- Development editors and IDEs
- Terminal prerequisite

20.10 Python Index

Use this index to find the lesson that teaches a Python concept or workflow. Search terms include canonical names, common aliases, and beginner descriptions. For definitions, see the Python glossary.

Environment, editor, and accessibility

Topic and learner search terms	Lesson
Install Python 3; interpreter; PATH; `py -3`; `python3`; version check	Unit 0: Python installation
Install and use uv; `uv init`; `uv add`; `uv run`; `uv sync`; `uv tree`; project dependencies	Unit 0: Managed Python environment
Virtual environment; `.venv`; isolated packages; dependency management; avoid global pip	Unit 0: Environments

Topic and learner search terms	Lesson
Visual Studio Code; Theia IDE; workspace; extensions; editor setup	Unit 0: Editor setup
Terminal; integrated terminal; create a project folder; run a script; REPL	Unit 0: Workspace and terminal
NVDA; JAWS; VoiceOver; screen reader; braille; indentation; keyboard navigation	Unit 0: Screen-reader configuration
Accessible editor; spoken punctuation; line and indentation feedback; accessibility tools	Unit 0: Accessibility strategies

Computer science, syntax, and numeric programs

Topic and learner search terms	Lesson
Computer science; hardware versus software; algorithms; problem solving	Unit 1: Computer science foundations
Software development process; requirements; design; implementation; testing; iteration	Unit 2: Development process
First program; Hello World; `print()`; output; comments	Unit 1: First Python program
Variables; assignment; identifiers; naming; statements; expressions	Unit 1: Variables
Built-in types; `int`; `float`; `str`; `bool`; `None`; type inspection	Unit 1: Data types
Arithmetic; operators; precedence; modulo; exponentiation; floor division	Unit 2: Expressions and numeric operations
User input; `input()`; standard input; prompts; convert input text	Unit 2: Input and output
F-strings; formatted output; interpolation; formatting numeric values	Unit 2: Formatted output
Type conversion; `int()`; `float()`; `str()`; conversion errors	Unit 2: Type conversions
`math` module; `sqrt()`; `ceil()`; `floor()`; constants; importing modules	Unit 2: Math library

Decisions, iteration, strings, and collections

Topic and learner search terms	Lesson
Comparisons; Boolean expressions; `and`; `or`; `not`; truth values	Unit 3: Boolean expressions
`if`; `elif`; `else`; nested conditions; branching	Unit 3: Conditional statements
Lists; list literals; indexes; zero-based indexing; slicing; mutation	Unit 3: List introduction
`for`; `range()`; iteration; iterable; loop variable	Unit 3: For loops
String methods; `strip()`; `lower()`; `upper()`; `split()`; `join()`	Unit 3: String operations
List methods; `append()`; `extend()`; `insert()`; `remove()`; `pop()`	Unit 4: List operations
Sorting; `sort()`; `sorted()`; `reverse()`; organizing a list	Unit 4: Organizing lists
Tuples; immutability; unpacking; one-item tuple	Unit 4: Tuples
Dictionaries; key-value pairs; lookup; keys; values; `items()`; iteration	Unit 4: Dictionaries
Repeated prompts; input validation; `while`; sentinel-controlled loop	Unit 5: Input and while loops
Infinite loop; loop condition; flag variable; `break`; `continue`	Unit 5: Loop control

Functions and object-oriented programming

Topic and learner search terms	Lesson
`def`; function definition; function call; reusable code; decomposition	Unit 6: Defining functions
Parameters; arguments; positional arguments; keyword arguments	Unit 6: Passing arguments
Default parameter; optional argument; `*args`; `**kwargs`	Unit 6: Flexible arguments
`return`; return value; local variable; scope; function result	Unit 6: Return values
Functions with lists and dictionaries; mutable arguments; reusable operations	Unit 6: Functions and collections
Class; object; instance; attribute; method; `self`	Unit 7: Classes and objects
`__init__()`; initialization; instance attributes; constructors	Unit 7: Creating a class

Topic and learner search terms	Lesson
Inheritance; child class; parent class; `super()`; method reuse	Unit 7: Inheritance
Composition; has-a relationship; object as an attribute	Unit 7: Composition

Files, errors, testing, data, and projects

Topic and learner search terms	Lesson
File input; `open()`; read mode; `read()`; `readline()`; `readlines()`	Unit 8: Reading files
File output; write mode; append mode; `write()`; encoding; UTF-8	Unit 8: Writing files
`with`; context manager; close a file; resource cleanup	Unit 8: Safe file handling
Exceptions; `try`; `except`; `else`; `finally`; `raise`	Unit 8: Exception handling
`ValueError`; `TypeError`; `KeyError`; `IndexError`; `FileNotFoundError`	Unit 8: Common exceptions
JSON; `json`; `json.load()`; `json.dump()`; serialization; persistent data	Unit 8: JSON storage
Unit testing; `unittest`; test case; `setUp()`; assertions; test a class	Unit 9: Testing
Random numbers; `random`; random integer; random choice	Unit 9: Random data
Text analysis; word frequency; CSV; structured data; parsing	Unit 9: Data analysis
Requirements; top-down design; decomposition; accessible input/output	Unit 10: Project design
Text adventure; rooms; movement; inventory; puzzles; command-line tool; to-do list	Unit 10: Projects
Personal data manager; NiceGUI; data model; persistence; capstone; Git; version control	Unit 11: Capstone

Python curriculum · Book overview · Programming glossary

20.11 Rust Topic Index

Use this index with the Rust curriculum overview and complete Rust lessons. The course introduces Rust's ownership model and systems-oriented programming practices.

Learning sequence

1. Rust toolchain, Cargo projects, and first program.
2. Variables, mutability, types, and expressions.
3. Input/output, parsing, and error reporting.
4. Conditions, loops, and control flow.
5. Strings, vectors, slices, and collections.
6. Functions, ownership, borrowing, and references.
7. Structs, enums, and pattern matching.
8. Methods, traits, and generic concepts.
9. Error handling with Option and Result.
10. Files, modules, testing, and package management.
11. Integrated projects and capstone.

Rust-specific concepts

- Cargo manifest and lockfile.
- Immutable bindings by default and explicit mut.
- Ownership, moves, borrowing, and lifetimes at an introductory level.
- Stack and heap concepts.
- String versus string slices.
- Vec, slices, and iterators.
- Structs, enums, and exhaustive match.
- Traits, implementations, and generics.
- Option and Result<T, E>.
- ? error propagation and recoverable errors.
- Unit tests, modules, and documentation.

Course references

- Rust overview
- Complete lessons
- Validation record
- Development editors and IDEs

20.12 Introductory Modules Index

These modules introduce focused areas of computing that complement the full programming-language curricula. They are not substitutes for the terminal prerequisite or the shared computing foundations.

Modules

- Git: Version control: repositories, commits, branches, merges, remotes, pull requests, and reading diffs as text.
- HTML and CSS: Web structure and presentation: semantic HTML, CSS rules, document structure, and accessible web content.
- Markdown: Technical documentation: headings, lists, links, code blocks, tables, dialects, and screen-reader-friendly authoring.
- R: Statistical computing: R values and data structures, descriptive statistics, data import, and reproducible analysis.
- Regular expressions: Pattern-based text processing: literals, character classes, quantifiers, groups, flags, dialects, and command-line tools.
- SQL: Relational databases: relational models, queries, filtering, aggregation, joins, transactions, and safe data access.
- Structured data formats: JSON, YAML, and TOML: syntax, data models, validation tooling, and format selection for each format.
- Web accessibility: ARIA, WCAG, and accessible design: the accessibility tree, WCAG principles, semantic HTML and ARIA, keyboard and focus, forms, and testing workflow.

Suggested prerequisites

Complete the Computing Foundations Primer and Terminal Skills prerequisite before working through these modules. Basic file and folder navigation is assumed.

Learners new to version control should complete the Git module before modules that assume repository workflows, such as Web accessibility practice on hosted projects. The Markdown module is recommended before any module that asks for written documentation; the Regular expressions module and Structured data formats module are independent of the others and can be studied in any order.

Related development references

- Development editors and IDEs
- Project environments and dependency management
- Lesson structure and audit standard

20.13 Swift Topic Index

Use this index with the Swift curriculum overview and complete Swift lessons. The course introduces Swift's type system and application-development foundations.

Learning sequence

1. Swift toolchain, source files, and first program.

2. Constants, variables, types, and expressions.
3. Input/output and conversion.
4. Conditions, switch, and loops.
5. Strings, arrays, dictionaries, and sets.
6. Functions, parameters, and return values.
7. Structures, classes, properties, and methods.
8. Protocols, extensions, and value/reference semantics.
9. Optionals and error handling.
10. Files, Codable, testing, and packages.
11. Integrated projects and capstone.

Swift-specific concepts

- let and var, type inference, and static typing.
- Optionals, optional binding, and safe unwrapping.
- Value semantics for structures and enums.
- Classes, reference semantics, and inheritance.
- Protocols, extensions, and generics.
- Arrays, dictionaries, sets, and collection operations.
- Codable for structured serialization.
- throws, do, try, and error propagation.
- Swift Package Manager and package structure.
- Structured concurrency concepts where appropriate.

Course references

- Swift overview
- Complete lessons
- Example validation
- Development editors and IDEs

20.14 TypeScript Index

Use this index to find the lesson that teaches a TypeScript feature, JavaScript runtime behavior, or typed development workflow. TypeScript checks source types before execution; its annotations do not validate external data at runtime. For definitions, see the TypeScript glossary.

Runtime, compiler, project setup, and accessibility

Topic and learner search terms	Lesson
Node.js LTS; npm; install TypeScript; project-local dependency	Unit 0: Runtime setup

Topic and learner search terms	Lesson
`tsc`; TypeScript compiler; type checking; emit JavaScript	Unit 0: Compiler setup
`tsconfig.json`; target; module; strict mode; compiler options	Unit 0: Project configuration
`.ts` source file; compile; emitted `.js`; type erasure	Unit 0: First TypeScript file
VS Code; Theia IDE; editor; workspace; terminal; keyboard shortcuts	Unit 0: Editor setup
Screen reader; keyboard navigation; accessible development workflow	Unit 0: Development environment
`let`; `const`; type annotation; type inference; primitive types	Unit 1: Variables and types
`string`; `number`; `boolean`; `console.log()`; template literals	Unit 1: Foundations

Values, input, decisions, and collections

Topic and learner search terms	Lesson
Arithmetic; `Math`; numeric expressions; number conversion	Unit 2: Numbers
`readline`; Node.js standard input; terminal prompts; async input	Unit 2: User input
Untrusted input; parsing; validation; runtime type checks	Unit 2: Input validation
Boolean expressions; comparisons; logical operators; `if`; `else`; `switch`	Unit 3: Control flow
Loops; `for`; `while`; `break`; `continue`; `forEach()`	Unit 3: Iteration
Arrays; `T[]`; `Array<T>`; indexes; array methods	Unit 4: Arrays
Object shape; property types; nested objects; optional properties	Unit 4: Objects
Destructuring; spread; rest; shallow copies; object properties	Unit 4: Object operations

Strings, typed functions, and object-oriented programming

Topic and learner search terms	Lesson
Strings; string methods; template literals; regular expressions	Unit 5: Strings
Type annotations; function contracts; parameter types; return types	Unit 5: Type annotations and contracts
Function declaration; function expression; arrow function; callback	Unit 6: Functions
Optional parameters; default parameters; rest parameters; arguments	Unit 6: Function parameters
Return annotation; function result; scope; closure; reusable logic	Unit 6: Function behavior
Type alias; interface; object shape; structural typing	Unit 5: Type contracts
Class; constructor; instance; access modifiers; private fields; static methods	Unit 7: Classes
Inheritance; `extends`; `super`; abstract class; method override	Unit 7: Inheritance

Files, asynchronous code, errors, and JSON

Topic and learner search terms	Lesson
Node file system; read file; write file; file paths; `fs`	Unit 8: File I/O
Promise; asynchronous file I/O; `async`; `await`	Unit 8: Asynchronous operations
Exceptions; `try`; `catch`; `finally`; custom Error	Unit 8: Error handling
JSON; `JSON.parse()`; `JSON.stringify()`; external data	Unit 8: JSON and data boundaries

Advanced types, modules, collections, testing, and projects

Topic and learner search terms	Lesson
Union type; intersection type; generic type; type parameters	Unit 9: Typed data
Narrowing; type guard; discriminated union; exhaustive checking	Unit 9: Narrowing and guards
`unknown`; `any`; type assertion; runtime validation	Unit 9: Safe type boundaries

Topic and learner search terms	Lesson
ES modules; `import`; `export`; module resolution	Unit 9: TypeScript modules
CommonJS; `require()`; `module.exports`; `package.json`	Unit 9: CommonJS and packages
`Map<K, V>`; `Set<T>`; typed collections; unique values	Unit 9: Maps and sets
CSV; word frequency; typed data processing; project structure	Unit 9: Data processing
Jest; test fixtures; assertions; matchers; test suite	Unit 9: Testing with Jest
Text adventure; command-line to-do manager; capstone; accessibility requirements	Units 10–11: Projects

TypeScript curriculum · Book overview · Programming glossary

Chapter 21

Contributors to this Curriculum

Michael Ryan Hunsaker, M.Ed., Ph.D. (he/him)

About Dr. Hunsaker

Michael Ryan Hunsaker holds a Ph.D. in neuroscience (behavioral/translational neuroscience focus) from UC Davis and a Master's degree in Special Education from the University of Utah with a credential to teach blind and visually impaired students. His career uniquely bridges scientific research, clinical experience, and classroom practice-positioning him to design accessible educational materials grounded in both neuroscience and practical pedagogy.

Research & Clinical Background

Following his doctoral work at UC Davis (2012), Dr. Hunsaker completed postdoctoral research in primate neural development at the UC Davis MIND Institute, where he developed sophisticated research methods to evaluate developmental disorders. During this period, he gained extensive clinical experience working directly with children and adults across a broad spectrum of diagnoses, including autism spectrum disorder, cerebral palsy, genetic conditions (fragile X syndrome, Williams syndrome, 22q11.2DS, Down syndrome), ADHD, and Tourette's syndrome. This diverse exposure to different neurological profiles taught him a crucial lesson: each learner's disability uniquely shapes their cognitive strengths, learning preferences, and educational needs.

Transition to Education

In 2013, recognizing the gap between academic research and classroom reality, Dr. Hunsaker made a deliberate career shift to direct education. Rather than pursue traditional academic pathways, he committed to applying his scientific training to help students in real classroom environments. As a special education teacher, he focused on understanding the individual relationship between disability and learning-developing instructional approaches that enable students to access their full potential rather than working around perceived limitations.

Expertise in Accessibility & Inclusive Design

His specialization in teaching blind and visually impaired students, particularly those with additional disabilities, has given Dr. Hunsaker deep expertise in accessible pedagogy and non-visual problem-solving. He grounds all curriculum design in evidence-based practice informed by neuroscience, accessibility research, and years of direct classroom experience with diverse learners. He understands that accessibility is not an afterthought or accommodation; it is the foundation upon which effective education for all students is built.

Creation of Accessible Educational Resources

Motivated by observing persistent gaps in affordable, practical, and culturally responsive accessible educational materials, Dr. Hunsaker created TVI Resources, a collection of free, openly licensed instructional materials designed to meet the real needs of educators and students. This commitment to accessible education informed his development of this curriculum: a screenreader accessible computer science curriculum designed explicitly for blind and visually impaired learners, emphasizing independence, technical mastery, and real-world application.

Acknowledgements

This curriculum stands on the shoulders of innovators, educators, and open-source maintainers whose commitment to accessibility and open education made this work possible.

Open Education & Accessibility Movement

Finally, this curriculum is part of a broader movement toward open educational resources that prioritize accessibility from the ground up. It reflects the conviction that education should be barrier-free, that students with disabilities bring unique perspectives and strengths, and that accessible design benefits all learners. The work of accessibility advocates, neurodivergent educators, and disability justice activists informed every decision in building this material.